Concise axiomatizations of intuitionistic primitive recursive arithmetic

November 2026

Abstract

Intuitionistic first-order primitive recursive arithmetic is bi-interpretable with a calculus that has variables, zero, successor and one iterator as its terms, equations and two binders as its formulas, and eight inference rules. Both numerical domains are whole, equality is actual equality, and both composite maps have graph $x = y$. This article states that calculus and the bridge once, and then collects every refinement of it that the research packages of the `ConciseIPRA` directory of the `ProveIt` repository contain: changes of the rule basis, guarded formula languages, rules that merge computation with equality, normal forms, clause calculi, rules that discharge a witness, introductions that take no witness, other recursors, obstruction results, and proof-certificate formats with their checkers. Each definition, rule inventory, theorem and proof is given once, in one notation. Statements that the packages leave unproved are collected at the end of each chapter.

Contents

Introduction **12**

1 The base theories and their bridge **14**
1.1 Terms and substitution . . . 14
1.2 Native formulas and their ordinary reading . . . 15
1.3 The native scalar core . . . 16
1.4 Structural properties and derived rules . . . 17
1.4.1 Weakening, substitution and cut . . . 17
1.4.2 Closure by cases and numerical cases . . . 18
1.4.3 Equality . . . 19
1.4.4 Congruence of the binders and the Leibniz rule . . . 20
1.4.5 Gates and selectors of formulas . . . 20
1.4.6 The connectives as native formulas . . . 22
1.4.7 A universal quantifier with one binder . . . 23
1.4.8 Finite blocks under one binder . . . 25
1.5 The ordinary theory T_{eq} . . . 31
1.6 Intuitionistic primitive recursive arithmetic . . . 32
1.7 Arithmetic inside T_{eq} . . . 32
1.7.1 The equational fragment . . . 32
1.7.2 Addition . . . 33
1.7.3 Zero test, interpolation and parity . . . 34
1.7.4 A three-cycle and a ramp . . . 34
1.7.5 The factor map and the comparator . . . 36
1.7.6 The saturating counter and the predecessor . . . 37
1.7.7 A smaller predecessor . . . 38
1.7.8 Coverage from a zero-test split . . . 47
1.7.9 Successor laws, subtraction and the reflection of equality . . . 49
1.7.10 Equality from the absorbing orbit . . . 50
1.7.11 Induction for quantifier-free formulas . . . 53
1.7.12 Order, powers of two and pairing . . . 53
1.7.13 Primitive recursion . . . 55
1.8 The bridge . . . 56
1.8.1 The native core and T_{eq} . . . 56
1.8.2 Term interpretations . . . 58
1.8.3 From T_{eq} to iPRA . . . 58
1.8.4 From iPRA to T_{eq} . . . 60
1.8.5 The two round trips on terms . . . 60
1.8.6 The bi-interpretation theorem . . . 61
1.9 Clause contexts . . . 63
1.10 The measurement protocol . . . 64

1.11 Further questions . . . 65

2 Refinements of the eight rules 68
2.1 Conventions and the comparison method . . . 68
2.2 The introduction rules . . . 70
2.2.1 Progress through a witness and progress at the closure . . . 70
2.2.2 One closure introduction with a mode read off the matrix . . . 71
2.2.3 What cannot be removed from the introductions . . . 71
2.2.4 One introduction with a mode read off the matrix and the witness . . . 73
2.3 The transport rule . . . 74
2.3.1 One template, two templates, a formula template . . . 74
2.3.2 Sources other than the literal zero . . . 75
2.3.3 Reversal of the root equation . . . 77
2.3.4 The same restriction for the uncrossed rule . . . 77
2.3.5 Replacement with no template . . . 80
2.3.6 Backward replacement . . . 81
2.3.7 Replacement of one occurrence . . . 85
2.3.8 A restricted inventory of computation instances . . . 86
2.4 Elimination of the closure by abstraction of the goal . . . 86
2.4.1 Replacement in formulas and abstraction . . . 87
2.4.2 The rule and the calculus . . . 88
2.4.3 Introductions before elimination . . . 88
2.4.4 Recovery of the ordinary elimination . . . 89
2.5 Induction and uniqueness of iteration . . . 90
2.5.1 The rule and the two theorems . . . 91
2.5.2 First consequences of uniqueness . . . 92
2.5.3 Retractions and induction for zero equations . . . 92
2.5.4 A predecessor from uniqueness . . . 95
2.5.5 Truncated subtraction and diagonal descent . . . 97
2.5.6 An accumulator and the commutativity of the maximum . . . 98
2.5.7 Reflection of equality and the proofs of the theorems . . . 98
2.5.8 Other forms of the uniqueness rule . . . 99
2.6 The progressive reading of the closure . . . 102
2.6.1 The calculus with orbit elimination . . . 103
2.6.2 Derived rules of the orbit calculus . . . 104
2.6.3 Soundness for the progressive reading . . . 105
2.6.4 The embedding of the core . . . 105
2.6.5 The reading inside the core and the two round trips . . . 106
2.6.6 Transport in a formula template . . . 108
2.6.7 Elimination at a target . . . 109
2.6.8 The calculi of the chapter . . . 110
2.7 Two results on other recursors . . . 111
2.7.1 Counter congruence at a zero seed . . . 112
2.7.2 Recursors seeded at zero . . . 113
2.8 Binder depth and finite axiomatizations . . . 115
2.8.1 The relations defined at bounded depth . . . 115
2.8.2 Finite ordinary axiomatizations . . . 117
2.9 Checker sizes . . . 118
2.10 Further questions . . . 122

3 Guards and public languages **127**
3.1 Guards as literal shapes . . . 128
3.1.1 The root-determined introduction . . . 128
3.1.2 Structural operations, the kernel, and equality at admissible sources . . . 129
3.1.3 Deleting the choice is not conservative . . . 130
3.1.4 The shapes . . . 131
3.1.5 Public languages . . . 133
3.2 The guarded bootstrap and the translation into a public language . . . 133
3.2.1 Guard schemes . . . 134
3.2.2 The translation and its two composites . . . 136
3.2.3 The guarded core . . . 138
3.3 Which transport sources are needed . . . 139
3.4 The failure of composition and its repairs . . . 141
3.4.1 The obstruction . . . 141
3.4.2 First repair: auxiliary formulas . . . 143
3.4.3 Second repair: the administrative orbit . . . 144
3.4.4 Third repair: the mirror guard . . . 145
3.4.5 Fourth repair: protected atoms . . . 146
3.5 Guards on the full native language . . . 147
3.5.1 Selectors and the criterion . . . 148
3.5.2 The diagonal guard . . . 150
3.5.3 The mirror selector and six rules . . . 150
3.6 Comparison of the guards . . . 152
3.7 Checkers and their sizes . . . 153
3.8 Further questions . . . 155

4 Computation, equality and normal forms **158**
4.1 Conventions and shared lemmas . . . 158
4.1.1 Notation . . . 159
4.1.2 Equality from reflexivity and crossed transport . . . 160
4.1.3 The single introduction . . . 160
4.1.4 Congruence below binders and provability of conversion . . . 161
4.2 The rule Eval on raw terms . . . 162
4.2.1 The operation Φ and the calculi . . . 162
4.2.2 The identity frame . . . 163
4.2.3 Computation, equality and transport recovered . . . 163
4.2.4 Equivalence with the core . . . 164
4.2.5 Substitution . . . 165
4.2.6 Irredundancy . . . 165
4.3 Normalizers and the transfer of derivations . . . 166
4.3.1 Admissible normalizers and normalizing substitution . . . 166
4.3.2 The kernel on normal forms . . . 167
4.4 Zero-normal terms . . . 169
4.4.1 Zero normalization . . . 169
4.4.2 The kernel without the zero computation rule . . . 170
4.4.3 Six rules with the single introduction . . . 171
4.5 The introduction selected by a reflexive base . . . 172
4.5.1 The calculus . . . 172
4.5.2 The structural rules, without circularity . . . 172
4.5.3 Progress recovered by two decoders . . . 173
4.5.4 Equivalence . . . 174

4.5.5 The same guard on raw terms, and two variants . . . 175
4.6 Equality as replacement of an initializer . . . 176
4.6.1 The rule Seed and the calculus . . . 176
4.6.2 Equality recovered . . . 177
4.6.3 Equivalence . . . 178
4.7 One successor step . . . 178
4.7.1 The rule Step . . . 178
4.7.2 Five patterns with clause contexts . . . 180
4.8 Seed evaluation and suspended application . . . 181
4.8.1 The rule SeedEval . . . 181
4.8.2 Suspended application . . . 182
4.9 Atomic-seed zero contraction with raw rule metadata . . . 186
4.9.1 Normalization after raw instantiation . . . 186
4.9.2 The atomic-seed normalizer . . . 188
4.9.3 Six rules . . . 190
4.10 Weak-head conversion with raw hints . . . 192
4.10.1 The head normalizer . . . 192
4.10.2 Six rules . . . 194
4.10.3 Conversion bounded to three counter levels . . . 197
4.10.4 Raw computation equations do not replace conversion . . . 201
4.11 Full normalization . . . 204
4.11.1 Termination and unique normal forms . . . 204
4.11.2 The kernel without computation rules . . . 206
4.11.3 Six rules on full normal forms . . . 206
4.11.4 What normalization costs . . . 208
4.12 A smaller checker for the unchanged core . . . 209
4.13 Checker sizes . . . 209
4.14 Further questions . . . 214
4.14.1 What is not achieved . . . 214
4.14.2 Claims stated without proof . . . 215
4.14.3 Open questions . . . 216

5 Clause calculi **219**
5.1 From Q formulas to clauses . . . 219
5.1.1 The Q-free form of a formula . . . 219
5.1.2 Clause occurrences and their origins . . . 220
5.2 The five-rule calculus . . . 220
5.2.1 Rules . . . 221
5.2.2 Derived rules . . . 221
5.2.3 Equivalence with the native scalar core . . . 223
5.3 Variants of the arithmetic interface . . . 224
5.3.1 Two evaluation cases . . . 224
5.3.2 Zero-normal terms and one evaluation scheme . . . 225
5.3.3 Reflexivity as one fixed clause . . . 228
5.3.4 Four schemes on zero-normal terms . . . 230
5.4 Variants of the introduction . . . 230
5.4.1 The diagonal guard . . . 230
5.4.2 Guards with a derivable base . . . 232
5.4.3 The fixed zero base on zero-normal terms . . . 233
5.5 Variants of the context . . . 234
5.5.1 The prefix representation . . . 234

5.5.2 Rank from variable support . . . 236
5.5.3 One body per context entry . . . 237
5.5.4 Homogeneous contexts with an asymmetric gate . . . 239
5.6 Fusion: evaluation at the discharge . . . 240
5.6.1 The raw calculus . . . 241
5.6.2 The zero-normal companion . . . 242
5.6.3 Partial evaluation and the diagonal introduction . . . 243
5.7 The ordinary realization . . . 246
5.8 Successor rigidity . . . 247
5.9 Checker sizes . . . 248
5.10 Further questions . . . 250

6 Discharge of a witness and restricted equality rules 253
6.1 Normal forms, clause contexts, and the reference calculus . . . 253
6.1.1 Conventions on full normal forms . . . 253
6.1.2 Formulas, clauses, and the two conventions . . . 255
6.1.3 The shared rules and the reference calculus . . . 255
6.1.4 Structural operations . . . 256
6.1.5 Closures, numerical cases, and the two introductions . . . 257
6.1.6 Equality from one term transport . . . 258
6.1.7 Two-row packets . . . 258
6.1.8 Interchangeability of the two conventions . . . 259
6.2 Naming packets: equality at the discharge of a witness . . . 259
6.2.1 The rule NAME . . . 259
6.2.2 Transport from one discharge . . . 260
6.2.3 Equivalence . . . 260
6.3 The inferred witness . . . 261
6.3.1 The selector and the two forms of the rule . . . 261
6.3.2 The export form . . . 263
6.3.3 The open form . . . 264
6.3.4 Summary and comparison of the four packages . . . 265
6.4 Substitutive discharge on the canonical recursor syntax . . . 266
6.4.1 Syntax . . . 267
6.4.2 Four rule families . . . 267
6.4.3 The bootstrap of equality . . . 267
6.4.4 The canonical basis and the bridge . . . 268
6.5 Restricted substitution . . . 270
6.5.1 Substitution for a variable . . . 270
6.5.2 The newest variable . . . 271
6.5.3 Substitution for a parameter on raw terms . . . 273
6.5.4 Unary congruence does not suffice . . . 276
6.6 Specialization without a term . . . 278
6.6.1 The calculus . . . 278
6.6.2 Naming a term . . . 279
6.6.3 Persistent proofs . . . 280
6.6.4 Equivalence . . . 281
6.7 Fresh aliases and backward substitution . . . 282
6.7.1 The two rules . . . 283
6.7.2 Equality from three rules . . . 284
6.7.3 The kernel with aliases . . . 285
6.7.4 Seven rules on clause syntax and the newest parameter . . . 286

6.7.5 Equation-only aliases and the rank of formulas . . . 289
6.8 A constancy rule in place of induction . . . 292
6.8.1 The theory . . . 292
6.8.2 Recovery of quantifier-free induction . . . 293
6.8.3 Both interpretations . . . 294
6.8.4 Constancy for Boolean tests . . . 295
6.8.5 A scalar iterator with one comparison equation . . . 296
6.8.6 Omitting the comparison equation . . . 300
6.9 Checkers and program equivalences . . . 302
6.10 Checker sizes . . . 305
6.11 Further questions . . . 307

7 Introduction without a supplied witness 311
7.1 Restricted witnesses . . . 311
7.1.1 Scopes and the restricted rules . . . 311
7.1.2 What a restricted witness loses . . . 313
7.2 Obstructions . . . 313
7.2.1 Each shape alone is incomplete . . . 314
7.2.2 The witness and the target cannot both be restricted . . . 314
7.3 Names . . . 316
7.3.1 Naming matrices . . . 316
7.3.2 Derived rules with variable witnesses . . . 317
7.3.3 The introduction at a name . . . 317
7.4 The coverage packet . . . 318
7.4.1 The predecessor . . . 318
7.4.2 The packet and the names . . . 319
7.4.3 A mode selected by the witness . . . 320
7.5 The newest parameter . . . 321
7.5.1 Stable derivations and hereditary families . . . 321
7.5.2 Closure by cases, the Leibniz rule and equation cases . . . 322
7.5.3 The packet in every scope . . . 323
7.5.4 The equivalence theorem . . . 324
7.5.5 The seven-rule diagonal variant . . . 325
7.5.6 One bit . . . 326
7.5.7 The five-family clause calculus . . . 327
7.5.8 The newest parameter in the Alias/Back calculus . . . 329
7.6 The raw guarded calculus . . . 333
7.6.1 The calculus and its three introductions . . . 333
7.6.2 The naming exception . . . 334
7.6.3 Uniform introduction: the stable fragment . . . 336
7.6.4 Uniform introduction: names and completeness . . . 337
7.7 A recursor whose predecessor computes . . . 339
7.7.1 Terms in normal form . . . 339
7.7.2 Three rule families . . . 340
7.7.3 Constructions that precede equality . . . 341
7.7.4 Completeness . . . 341
7.7.5 The ordinary theory and the bridge . . . 343
7.7.6 Checking a template through its two instances . . . 343
7.8 Conditional copy . . . 344
7.8.1 Use, Close and Export . . . 344
7.8.2 Preparations without equality . . . 345

7.8.3 The conditional-copy theorem . . . 346
7.8.4 Removal of the witness . . . 346
7.8.5 The liberal calculus presents the ordinary theory . . . 347
7.8.6 Quantifier-free induction and the bridge . . . 349
7.8.7 Discharge by a supplied name . . . 349
7.9 Specialization at the newest parameter . . . 350
7.9.1 Newest specialization with the inferred discharge . . . 350
7.9.2 The vacuous selector . . . 353
7.9.3 Conditional copy and the removal of both terms . . . 354
7.10 Checkers and sizes . . . 357
7.11 Further questions . . . 360

8 Recursors with a built-in seed **364**
8.1 The successor-seeded recursor . . . 365
8.1.1 Terms, equations and the two theories . . . 365
8.1.2 Equality, cases and induction before the predecessor . . . 366
8.1.3 Boolean terms and contextual case analysis . . . 367
8.1.4 The predecessor . . . 368
8.1.5 Recovery of the iterator . . . 372
8.1.6 Comparison of the two term languages . . . 372
8.2 Tied-seed recursors . . . 374
8.2.1 The recursor and its elementary terms . . . 374
8.2.2 The one-coordinate bound . . . 375
8.2.3 The theory with a Boolean equality test . . . 376
8.2.4 A compiler with caps . . . 379
8.2.5 The functions denoted by terms . . . 382
8.2.6 Truncated subtraction in place of the equality test . . . 382
8.2.7 The native calculus with truncated subtraction . . . 384
8.3 The self-seeded recursor . . . 386
8.3.1 The theories . . . 386
8.3.2 Without injectivity of the successor . . . 387
8.3.3 Injectivity from paired stopping orbits . . . 389
8.3.4 With injectivity: predecessor and order . . . 392
8.3.5 Bounded iteration of finite-case functions . . . 393
8.3.6 Arithmetic by finite-case graphs . . . 394
8.4 Three observations on the native core . . . 397
8.4.1 One native binder over a literal equation . . . 397
8.4.2 The case rule at a literal equation cannot be deleted . . . 398
8.4.3 Transport from the literal zero alone . . . 399
8.5 Checker sizes . . . 399
8.6 Further questions . . . 402

9 Canonical and counter-aware recursors **404**
9.1 Term languages and normal forms . . . 404
9.1.1 Three recursors without a successor constructor . . . 404
9.1.2 Zero-body canonical terms . . . 405
9.1.3 Full normal forms . . . 406
9.1.4 The status of the successor and of each computation . . . 408
9.2 What the counter gives . . . 409
9.2.1 A decoder that computes . . . 409
9.2.2 Primitive recursion without pairing . . . 409

9.2.3 The register iterator . . . 410
9.2.4 A register with one bound variable . . . 412
9.2.5 Without a counter . . . 413
9.3 Translations to iterator terms and the transfer of the bridge . . . 413
9.3.1 The ordinary theories . . . 413
9.3.2 The translations . . . 414
9.4 A register language without constants . . . 417
9.4.1 The language and the theory . . . 417
9.4.2 Zero and the successor . . . 419
9.4.3 Translations and the bi-interpretation . . . 420
9.4.4 Sizes of the constant terms . . . 422
9.4.5 The reference checker . . . 423
9.5 Guarded scalar registers . . . 423
9.5.1 Terms and the guard . . . 423
9.5.2 Arithmetic from the two equations . . . 425
9.5.3 Initialized iteration . . . 427
9.5.4 Equivalence with the native core . . . 428
9.5.5 The strict checker . . . 430
9.6 The seeded recursor: unrestricted calculi and six rules . . . 431
9.6.1 The raw and the canonical unrestricted calculus . . . 431
9.6.2 The six rules . . . 433
9.6.3 The bridge for the seeded calculi . . . 435
9.7 Five rules with clause contexts . . . 435
9.7.1 The calculus . . . 436
9.7.2 Closure as an abbreviation . . . 437
9.7.3 Comparison with the six rules . . . 437
9.8 Counter evaluation . . . 438
9.8.1 The clause syntax and a liberal calculus . . . 438
9.8.2 The five patterns of counter evaluation . . . 440
9.8.3 Ordinary logic with the decoder . . . 441
9.8.4 Adequacy and the main theorem . . . 443
9.9 Three families on normal forms . . . 444
9.9.1 Use, Close and Name . . . 444
9.9.2 Use, Introduce and Open with an inferred witness . . . 446
9.9.3 The scalar normal forms embed exactly . . . 448
9.9.4 The main theorem for the two calculi . . . 449
9.10 Root conversion on the counter-aware recursor with successor . . . 449
9.10.1 Terms and root steps . . . 449
9.10.2 The calculus . . . 450
9.10.3 Equality from root steps . . . 451
9.10.4 Equivalence with the native kernel . . . 453
9.10.5 The bridge . . . 454
9.10.6 The checkers . . . 457
9.11 Binary trees with a paramorphism . . . 458
9.12 Uniform binding and one traversal . . . 461
9.12.1 The concrete syntax . . . 461
9.12.2 One traversal . . . 462
9.12.3 The checkers . . . 463
9.12.4 Three refinements of checkers for the kernel on iterator terms . . . 464
9.13 Boundary results . . . 465
9.13.1 A recursor with neither counter nor initializer . . . 465

9.13.2 The two computation equations of the iterator are independent 466
9.13.3 Induction on zero equations does not give induction on equations 467
9.13.4 The last computation rule 469
9.14 Checker sizes 469
9.15 Further questions 472

10 Obstructions **478**
10.1 Conventions and the presentations under test 480
10.1.1 Three kinds of exclusion 480
10.1.2 The paired-introduction form of the native core 480
10.1.3 Reduced term languages 481
10.1.4 The progressive calculus 482
10.2 Coordinatewise powers and truth transfer 483
10.2.1 The powers 483
10.2.2 Equation induction does not replace numerical cases 485
10.2.3 Boolean cases are not admissible in the progressive calculus 486
10.3 Symmetry 487
10.4 Majorants: almost unary term languages 489
10.4.1 Zero-seeded recursion is monotone 489
10.4.2 Bounding coordinates 489
10.4.3 Adaptive query programs with several registers 490
10.4.4 Single-body constructors 492
10.4.5 Adaptive query circuits 493
10.5 Finite-dimensional term interpretations 495
10.6 Finite profiles and relational bi-interpretations 497
10.6.1 Finite control 497
10.6.2 An existential-free presentation 498
10.6.3 No compression of scalars 499
10.6.4 No finite-dimensional bi-interpretation 500
10.6.5 The selector boundary 501
10.7 Zero profiles of positive iteration 502
10.8 Rule-level obstructions 503
10.8.1 Atomic case introduction cannot be made automatic 503
10.8.2 Elimination majors must be proved, not only retained 503
10.8.3 Root replacement does not give congruence 504
10.8.4 The witness order of the paired binder 504
10.8.5 Two side conditions that cannot be dropped 505
10.9 Independence of the progress rule 505
10.9.1 Coherent iterator structures 506
10.9.2 Native terms in the tower 508
10.9.3 The theorem 508
10.9.4 Fixed points and seed shift do not give induction 511
10.9.5 Numeral induction, numbering and fusion 516
10.9.6 No successor cycle without progress 521
10.10 The progressive binder presentation 522
10.11 The counter-congruence form of the calculus 525
10.12 Constructed sources and certificate interfaces 528
10.12.1 Successor sources constructed from predecessors 528
10.12.2 Statements about the checkers of the packages 530
10.13 Partial routes 531
10.13.1 From uniqueness of iteration toward induction 531

10.13.2 What fixed countdown uniqueness determines . . . 533
10.13.3 Stationarity over an explicit primitive recursive interface . . . 533
10.14 Checker sizes . . . 534
10.15 Further questions . . . 536

11 Certificates and checkers **542**
11.1 The calculi that are checked . . . 544
11.2 The sorted arity encoding . . . 546
11.3 The instruction format and the certified zero . . . 549
11.3.1 The base protocol . . . 549
11.3.2 Restricted computation and the certified zero . . . 550
11.3.3 Correspondence with the calculus . . . 551
11.4 Uniform zero and constructed majors . . . 553
11.4.1 The recoding . . . 553
11.4.2 Constructed majors and local floors . . . 554
11.4.3 Sizes of certificates . . . 556
11.5 Neutral major premises . . . 556
11.5.1 The protocol and its hazard . . . 557
11.5.2 The invariant of successful runs . . . 557
11.5.3 Correspondence . . . 558
11.5.4 Composition with local floors . . . 559
11.6 Fixing an elimination before its major is proved . . . 559
11.6.1 The global-floor version . . . 560
11.6.2 The local-floor version . . . 561
11.7 Scalar neutral premises and loops . . . 562
11.7.1 The guarded-source protocol . . . 562
11.7.2 Loops over immutable contexts . . . 563
11.7.3 The root test is a logical boundary . . . 564
11.7.4 Local rewritings of the program text . . . 565
11.8 Validated presentations . . . 566
11.8.1 Input contracts and the public grammar . . . 566
11.8.2 Scope is checked by the first opening . . . 567
11.8.3 Typed checkers . . . 567
11.8.4 Goal-validated checkers . . . 569
11.8.5 A rejected saving . . . 570
11.9 Inferred majors, eager selection and the phase of validation . . . 572
11.9.1 The controls . . . 572
11.9.2 The inferred major . . . 573
11.9.3 Retained majors . . . 574
11.9.4 Eager selection of the goal . . . 579
11.9.5 The remainder selector . . . 580
11.9.6 Truth-valued results and the tail of a packet . . . 580
11.9.7 The phase of validation . . . 581
11.9.8 A typed native checker and typed guarded checkers . . . 582
11.9.9 Three-field certificates . . . 583
11.9.10 Allocation before the elimination test . . . 586
11.9.11 Refinements of the allocation-first programs . . . 588
11.9.12 The floor recomputed from the context . . . 592
11.9.13 Refinements of the native goal-validated and compact programs . . . 594
11.9.14 Two fields, and computation equations recognized from the goal . . . 607
11.9.15 Computation leaves, uniform terminals and their compositions . . . 610

11.9.16 An even offset of the depths 623
11.9.17 Uniform terminals in the validated programs with uniform zero 628
11.9.18 The audit of sources 635
11.10 Results on designs that were examined 637
11.10.1 Projection boxes lose reflexivity 637
11.10.2 Eager initialization has a two-element model 638
11.10.3 Positive iteration in the standard model 639
11.10.4 Conditional and progressive closure of an equation 640
11.10.5 Further rejected designs 641
11.10.6 The entry scan is necessary 642
11.10.7 Reserved coordinates are not spellings of zero 643
11.10.8 Variable-only substitution without reflexivity 645
11.10.9 Finite compositional signatures 645
11.10.10 Majors taken only from the context 646
11.10.11 Newest-parameter use together with a restricted witness 647
11.10.12 Standard-numeral rigidity without induction 647
11.10.13 Relations compatible with unary term functions 648
11.11 Checker sizes 650
11.11.1 What the measurement does not control 663
11.11.2 The coordinatewise envelope 665
11.12 Further questions 677

A The packages **690**

Introduction

The question

The question behind all of this work is how small a formal system can be while it still is, in the strongest reasonable sense, intuitionistic primitive recursive arithmetic iPRA. "Is" means a bi-interpretation in which the numerical domain of each interpretation is the whole domain, equality is interpreted by equality, and each composite of the two numerical maps is the identity, provably. "Small" has two readings. The mathematical one counts term constructors, formula constructors and inference rules, and asks which of them can be removed, restricted or fused. The empirical one fixes a measurement protocol for complete proof checkers and compares their sizes (Section 1.10). The two readings do not always agree, and the text keeps them apart: a smaller rule basis can need a larger checker, and several of the smallest checkers implement an unchanged rule basis.

How the article is organized

Section 1.1 to Section 1.10 of the first chapter state the common core: the terms, the native formulas and their ordinary reading, the eight rules of the native scalar core N, its structural properties and derived connectives, the ordinary theory T_{eq}, the arithmetic inside T_{eq}, and the bridge to iPRA. No other chapter restates this material.

The next six chapters stay on iterator terms. The second treats refinements of the eight rules on the unchanged language: the introductions, the transport rule, uniqueness of iteration in place of induction, and the progressive reading of the closure binder. The third treats guards: formula languages in which the shape of a matrix determines the mode of its introduction. The fourth treats rules that merge computation with equality and calculi on normal forms. The fifth treats clause calculi, in which contexts hold reusable clauses and the closure binder is not a formula. The sixth treats rules that fuse equality reasoning with the discharge of an opened witness, and restricted substitution rules. The seventh treats introductions that take no witness.

Two chapters change the term language: recursors with a built-in seed, and canonical and counter-aware recursors in which the successor is defined. The tenth chapter collects the obstruction and independence results, with the models and invariants that prove them. The last chapter treats proof-certificate formats and checkers, and records the measured sizes.

Each chapter ends with a table of the checker sizes that its packages report and with a section of further questions: claims stated without proof, conjectures, and what was tried and not achieved.

Conventions

The notation of the first chapter is used throughout. Names of calculi other than N, T_{eq} and iPRA are local to the chapter that introduces them; the same letter can denote different calculi

in different chapters. Every chapter compares its calculi with the eight-rule core of the first chapter. A research package is cited by its directory name, as in `witness_rule`; Chapter A lists every package with the chapters that draw on it. Where a result is formalized in Lean in the repository, the name of the Lean declaration is given in typewriter type; the status of the formalization is recorded in the README of the directory and is not repeated here in full. Editorial repairs of the source reports are stated where they occur.

Chapter 1

The base theories and their bridge

Three formal systems recur in every chapter of the article: the native scalar core N, a calculus with eight rules and no connectives; the ordinary theory T_{eq}, intuitionistic logic with equality over the same terms; and iPRA, intuitionistic first-order primitive recursive arithmetic with quantifier-free induction. This chapter states the three systems, develops the derived rules of N and the arithmetic of T_{eq}, and proves that the three systems are bi-interpretable with whole domains, actual equality and identity composites (Theorem 1.119). It also fixes the notion of a clause context and the protocol by which proof checkers are measured. The later chapters use these statements without repeating them.

Notation that is fixed here and used throughout: $1 := S0$, $2 := SS0$, $3 := SSS0$; a term or formula written $h(v)$ has a distinguished variable v, and $h(s)$ abbreviates $h[s/v]$; $A \dashv\vdash B$ means that each of A, B is derivable from the other in the system under discussion.

1.1 Terms and substitution

This section draws on the reference statement of the inherited basis filed with `certified_zero` and on the syntax section of `raw_clausal`.

Definition 1.1 (Terms)**.** There is one numerical sort. The terms are

$$t ::= x \mid 0 \mid St \mid I(n, a;\, u.\, h),$$

where x ranges over a countable set of variables. In the *iterator* $I(n, a;\, u.\, h)$ the term n is the *counter*, a is the *seed* and h is the *body*. The iterator binds the variable u in h and nowhere else; u is not bound in n or in a. The body may contain other free variables, which act as fixed numerical parameters. Terms that differ only in the names of bound variables are identified. $\mathrm{FV}(t)$ is the set of free variables of t.

A *substitution* θ is a map from variables to terms that is the identity outside a finite set. The term $t[\theta]$ is defined by

$$x[\theta] = \theta(x), \qquad 0[\theta] = 0, \qquad (St)[\theta] = S(t[\theta]), \qquad I(n, a;\, u.\, h)[\theta] = I(n[\theta], a[\theta];\, u.\, h[\theta]),$$

where in the last clause the bound variable u is first renamed so that $\theta(u) = u$ and $u \notin \mathrm{FV}(\theta(x))$ for every $x \neq u$. The substitution that sends x to s and fixes all other variables is written $[s/x]$. Substitution is capture-free by construction, and the following standard facts hold by induction on t.

Lemma 1.2 (Substitution)**.** (i) *If* $x \notin \mathrm{FV}(t)$ *then* $t[s/x] = t$.

(ii) $t[\theta][\theta'] = t[\theta'']$, *where* $\theta''(x) = \theta(x)[\theta']$.

(iii) *If* $x \neq z$ *and* $x \notin \mathrm{FV}(r)$*, then* $t[s/x][r/z] = t[r/z][s[r/z]/x]$.

Definition 1.3 (Selector, zero test, parity)**.** For terms c, a, b put

$$\mathrm{sel}(c,a,b) := I(c,a;\, u.\, b) \quad (u \notin \mathrm{FV}(b)), \qquad Z(c) := \mathrm{sel}(c,1,0), \qquad N(n) := I(n,0;\, u.\, Z(u)).$$

The italic letter N names the parity term; the calculus is written with the sans-serif letter N.

The two *computation equations* of the iterator are

$$I(0,a;\, u.\, h) = a, \tag{1.1}$$
$$I(Sn,a;\, u.\, h) = h[I(n,a;\, u.\, h)/u]. \tag{1.2}$$

They are equations between distinct terms, to be derived or assumed in a calculus; no term is identified with its value. For the selector they read $\mathrm{sel}(0,a,b) = a$ and $\mathrm{sel}(Sc,a,b) = b$, because the bound variable does not occur in the body.

Remark 1.4 (What the language does not contain)*.* Every term is finite syntax. No numerical variable ranges over bodies or over descriptions of functions, and there is no term that evaluates descriptions uniformly. The body of an iterator receives the previous value only; it does not receive the counter. The reference statements present the same language with function symbols instead of a binder: at stage zero the symbols are 0 and S, and for every earlier-stage term $h(u,\vec{p})$ a symbol $i_h(n,a,\vec{p})$ is added with the equations $i_h(0,a,\vec{p}) = a$ and $i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p})$. The formal parameters of a symbol are distinct and its actual arguments are arbitrary. The two presentations have the same terms up to this renaming, and the article uses the binder.

1.2 Native formulas and their ordinary reading

This section draws on the reference statements filed with `certified_zero` (the inherited basis and the inherited ordinary bridge).

Definition 1.5 (Native formulas)**.** The native formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

Q_y binds y; $D_{x,y}$ binds the two distinct variables x and y. Formulas that differ only in the names of bound variables are identified, and $A[\theta]$ is capture-free substitution. A *literal equation* is a formula of the form $s = t$; an equation below a binder is not a literal equation. The designated contradiction is the equation $0 = S0$.

There are no connectives, no falsity constant and no ordinary quantifier in the native language.

Definition 1.6 (Ordinary formulas and the ordinary reading)**.** The *ordinary formulas* are the first-order formulas over the terms of Theorem 1.1, built from equations by $\bot$, $\wedge$, $\vee$, $\to$, $\forall$ and $\exists$. The *ordinary reading* A° of a native formula A is the ordinary formula defined by

$$\begin{aligned} (s = t)^\circ &:= (s = t), \\ (Q_y B)^\circ &:= B^\circ(0) \to \forall y\, B^\circ(y), \\ (D_{x,y} B)^\circ &:= \exists x \left(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\right). \end{aligned}$$

For a list Γ of native formulas, Γ° is the list of the readings of its members.

Thus Q_yB says that B holds everywhere provided it holds at zero, and $D_{x,y}B$ says that some x has this property. The reading uses the whole numerical domain and reads the equality sign as equality. By induction on A,

$$(A[\theta])^\circ = A^\circ[\theta]. \tag{1.3}$$

In the Lean formalization the native formulas are `NFm` and the reading is `NFm.toFm`.

1.3 The native scalar core

This section draws on the reference statement of the inherited basis filed with `certified_zero`; the two rules TR and QI are taken in the forms of `one_context` and `direct_closure`, as in the Lean kernel.

A *sequent* is $\Gamma \vdash A$, where the *context* Γ is a finite list of native formulas and A is a native formula.

Definition 1.7 (The native scalar core N)**.** The derivable sequents of N are generated by the following eight rules. In TR, $h(v)$ is a term with a distinguished variable v.

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \text{HYP} \qquad \frac{}{\Gamma \vdash I(0,a;\,u.\,h) = a}\ \text{I0}$$

$$\frac{}{\Gamma \vdash I(Sn,a;\,u.\,h) = h[I(n,a;\,u.\,h)/u]}\ \text{IS}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}\ \text{TR} \qquad \frac{\Gamma \vdash Q_yB(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE}$$

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma,\ Q_yB(b,y) \vdash A}{\Gamma \vdash A}\ \text{DE}$$

$$\frac{\Gamma,\ B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \text{DI} \qquad \frac{\Gamma,\ E(z) \vdash E(Sz)}{\Gamma \vdash Q_yE(y)}\ \text{QI}$$

Side conditions. In DE the variable b is not free in Γ, in A or in $D_{x,y}B$. In DI the variable z is not free in Γ, in t or in $D_{x,y}B$; the term t is the *witness*. In QI the variable z is not free in Γ or in Q_yE, and E is a literal equation. The displayed assumptions $Q_yB(b,y)$, $B(t,0)$ and $E(z)$ are discharged.

The rules are named as follows: hypothesis; the two computation equations; transport in one term; elimination of Q; elimination of D; introduction of D *by cases*; introduction of Q *by progress*. The matrix of DI is an arbitrary native formula, and its premise has one assumption, the instance at zero. The matrix of QI is a literal equation, and its premise has one assumption, the instance at the eigenvariable.

Remark 1.8 (The complete inventory). The eight rules are all there is. There is no reflexivity axiom and no other equality rule; no axiom states that a number is zero or a successor, that the successor is injective or that $Sx \neq 0$; there is no induction rule other than QI, whose matrix is one equation; there is no logical rule for a connective, since there is no connective; and there are no structural rules. Weakening, substitution and cut are admissible (Theorem 1.10). Every one of the omitted principles is derived in Section 1.4 or, through the bridge, in Section 1.7. In Lean the calculus is the inductive predicate `NPrf`, with de Bruijn indices: a fresh variable is the new index 0 over a context whose formulas are shifted.

Remark 1.9 (The calculus as the reports state it). The research packages state the calculus with two rules in a stronger form: transport in an arbitrary native formula template,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)},$$

and a witnessed introduction by progress, which infers $D_{x,y}E(x,y)$ from $\Gamma, E(t,z) \vdash E(t,Sz)$ for a literal equation E. Both are derived rules of N (Theorems 1.16 and 1.23). Conversely, in the calculus of the reports reflexivity follows by transporting $I(0,t;\, u.\, 0) = t$ along itself in the template $v = t$, symmetry and transitivity follow by transport in the templates $v = s$ and $s = v$, the rule TR follows from the transport of $r = h(s)$ in the template $r = h(v)$, and QI follows by introducing $D_{x,y}E(y)$ with x not free in E and opening it by DE, which yields the hypothesis $Q_y E(y)$ itself. The two calculi therefore derive the same sequents (Lean: `iprf_iff_nprf`). The reports also write the calculus with clause contexts instead of the binder Q; that form is described in Section 1.9. The comparison of rule bases is the subject of later chapters and is not pursued here.

1.4 Structural properties and derived rules

This section draws on the reference statement of the inherited basis filed with `certified_zero` for the logical interface, and follows the Lean modules `NativeStruct`, `NativeEq`, `NativeGate` and `NativeLogic` in the organization of the proofs. Throughout the section $\vdash$ is derivability in N.

1.4.1 Weakening, substitution and cut

Lemma 1.10 (Weakening and substitution)**.** *If $\Gamma \vdash A$, then $\Delta \vdash A[\theta]$ for every substitution θ and every context Δ that contains $B[\theta]$ for each $B \in \Gamma$. In particular $\Gamma \vdash A$ implies $\Gamma[\theta] \vdash A[\theta]$, and $\Gamma \vdash A$ with $\Gamma \subseteq \Delta$ implies $\Delta \vdash A$.*

Proof. By induction on the derivation of $\Gamma \vdash A$, for all θ and Δ at once.

HYP: $A \in \Gamma$ gives $A[\theta] \in \Delta$.

I0, IS: an instance of a computation equation under θ is again an instance, for (1.2) by Theorem 1.2: $(h[I(n,a;\, u.\, h)/u])[\theta] = h[\theta][I(n[\theta], a[\theta];\, u.\, h[\theta])/u]$.

TR: choose the template variable v fixed by θ and not free in its values. Then $h(s)[\theta] = h[\theta](s[\theta])$ and $h(t)[\theta] = h[\theta](t[\theta])$, so the two premises given by the induction hypothesis are premises of TR with the template $h[\theta]$.

QE: $B(t)[\theta] = B[\theta](t[\theta])$ and $B(0)[\theta] = B[\theta](0)$, and the induction hypothesis gives $\Delta \vdash Q_y B[\theta]$ and $\Delta \vdash B[\theta](0)$.

DE: choose b' not free in Δ, $A[\theta]$, $D_{x,y}B[\theta]$ and the values of θ, and let θ' be θ changed to send b to b'. Because b is not free in Γ, A and $D_{x,y}B$, the substitutions θ and θ' agree on them, and $(Q_y B(b,y))[\theta'] = Q_y B[\theta](b',y)$. The induction hypothesis for the second premise, with θ' and the context $\Delta, Q_y B[\theta](b',y)$, gives $\Delta, Q_y B[\theta](b',y) \vdash A[\theta]$; for the first premise it gives $\Delta \vdash D_{x,y}B[\theta]$. DE applies.

DI: choose z' fresh and let θ' send z to z' and agree with θ elsewhere. The induction hypothesis with θ' and the context $\Delta, B[\theta](t[\theta], 0)$ gives $\Delta, B[\theta](t[\theta], 0) \vdash B[\theta](t[\theta], Sz')$, and DI with the witness $t[\theta]$ applies.

QI: as for DI, with the premise $\Delta, E[\theta](z') \vdash E[\theta](Sz')$; the formula $E[\theta]$ is a literal equation because substitution preserves the root of a formula. □

Lemma 1.11 (Cut)**.** *If $\Gamma \vdash A$ and $\Delta \vdash B$ for every $B \in \Gamma$, then $\Delta \vdash A$.*

Proof. The following more general statement is proved by induction on the derivation of $\Gamma \vdash A$: if $\Delta \vdash B[\theta]$ for every $B \in \Gamma$, then $\Delta \vdash A[\theta]$. HYP is the assumption. I0 and IS are instances. TR and QE follow from the induction hypotheses for their premises as in Theorem 1.10. For DE choose b' and θ' as in that proof. Every $C \in \Gamma$ satisfies $C[\theta'] = C[\theta]$, hence $\Delta, Q_y B[\theta](b', y) \vdash C[\theta']$ by weakening, and the new hypothesis is derivable from itself. The induction hypothesis for the second premise gives $\Delta, Q_y B[\theta](b', y) \vdash A[\theta]$, and DE applies. DI and QI are treated in the same way. □

In Lean the three properties are `NPrf.weaken`, `NPrf.subst` and `NPrf.cut`. They are used below without mention. For a context Γ, formulas A and B are *interderivable over* Γ if $\Gamma, A \vdash B$ and $\Gamma, B \vdash A$; for the empty Γ this is $A \dashv\vdash B$. By cut, interderivable formulas may replace each other as hypotheses and as conclusions.

1.4.2 Closure by cases and numerical cases

Lemma 1.12 (A witness that is not used)**.** *If $x \notin \mathrm{FV}(B)$, then $D_{x,y}B \dashv\vdash Q_y B$.*

Proof. From $D_{x,y}B$, DE with a fresh b adds the hypothesis $Q_y B[b/x]$, which is $Q_y B$, and HYP derives the conclusion $Q_y B$. From $Q_y B$, DI with the witness 0 asks for $Q_y B,\ B(0) \vdash B(Sz)$, which is QE at Sz. □

Lemma 1.13 (Introduction of Q by cases)**.** *If $\Gamma, B(0) \vdash B(Sz)$ with z not free in Γ and $Q_y B$, then $\Gamma \vdash Q_y B(y)$. Here B is an arbitrary native formula.*

Proof. Choose x not free in B. DI with the witness 0 and the matrix B derives $\Gamma \vdash D_{x,y}B$ from the given premise, and Theorem 1.12 gives $\Gamma \vdash Q_y B$. □

Corollary 1.14 (Numerical cases)**.** *If $\Gamma \vdash B(0)$ and $\Gamma, B(0) \vdash B(Sz)$ with z fresh, then $\Gamma \vdash B(t)$ for every term t. In particular, if $\Gamma \vdash B(0)$ and $\Gamma \vdash B(Sz)$ with z fresh, then $\Gamma \vdash B(t)$.*

Proof. Theorem 1.13 and QE. □

Lemma 1.15 (Introduction of D from a closure)**.** *If $\Gamma \vdash Q_y B(t, y)$, then $\Gamma \vdash D_{x,y}B(x, y)$.*

Proof. DI with the witness t asks for $\Gamma, B(t, 0) \vdash B(t, Sz)$, which is QE at Sz applied to the given closure and the assumption. □

Corollary 1.16 (The witnessed and the combined introduction)**.** *Let $E(x, y)$ be a literal equation and z a fresh variable.*

(i) *If $\Gamma, E(t, z) \vdash E(t, Sz)$, then $\Gamma \vdash D_{x,y}E(x, y)$.*

(ii) *If $\Gamma, E(t, 0), E(t, z) \vdash E(t, Sz)$, then $\Gamma \vdash D_{x,y}E(x, y)$.*

Proof. (i) QI gives $\Gamma \vdash Q_y E(t, y)$, and Theorem 1.15 applies. (ii) Over the context $\Gamma, E(t, 0)$, QI gives $Q_y E(t, y)$, and QE at Sz with the hypothesis $E(t, 0)$ gives $E(t, Sz)$. This is the premise of DI with the witness t. □

In Lean the rules of this subsection are `NPrf.qI`, `NPrf.cases` and `NPrf.dIProg`.

Remark 1.17 (The older paired introduction). Part (ii) is the introduction rule of the oldest reports, which retains the instance at zero and, for an equation, the instance at the eigenvariable. The inherited basis replaces it by a choice between the two assumptions, one per inference, and proves that the consequence relation is unchanged. In one direction each choice is the old rule after a weakening. In the other direction only an old introduction at an equation

needs a construction, and it is the one in the proof of (ii). The witness t is kept as a parameter of the matrix $E(t, y)$, and the closure is introduced without a witness; an introduction and elimination of $D_{x,y}E(x, y)$ itself would return a fresh witness, not t. The reference counts inference nodes for its own presentation, in which the inner step is a witnessed introduction with an unused coordinate followed by DE: an old derivation with n nodes, r of them introductions at an equation, becomes a derivation with exactly $n + 5r \leq 6n$ nodes, the six nodes being the outer introduction by cases, the inner introduction by progress, DE, QE and two uses of HYP. In N the inner introduction and its elimination are the single inference QI, so four nodes replace one. The bound counts nodes of the inference tree; it is not a bound on the size of formulas or of serialized certificates.

1.4.3 Equality

Lemma 1.18 (Equality rules)**.** *The following rules are derivable, each by the stated number of applications of* TR*.*

(i) *Reflexivity:* $\Gamma \vdash t = t$ *(one application, on two instances of* I0*).*

(ii) *Symmetry:* $\Gamma \vdash s = t$ *implies* $\Gamma \vdash t = s$ *(one more).*

(iii) *Transitivity:* $\Gamma \vdash s = t$ *and* $\Gamma \vdash t = r$ *imply* $\Gamma \vdash s = r$ *(one more).*

(iv) *Congruence:* $\Gamma \vdash s = t$ *implies* $\Gamma \vdash h(s) = h(t)$ *for every term template* $h(v)$ *(one more).*

(v) *Transport in an equation template:* $\Gamma \vdash s = t$ *and* $\Gamma \vdash a(s) = b(s)$ *imply* $\Gamma \vdash a(t) = b(t)$ *(two).*

Proof. Write TR_h for the rule with the template $h(v)$. (i) Apply TR_v to the premises $I(0, t;\, u.\, u) = t$ and $I(0, t;\, u.\, u) = t$, both instances of I0; the conclusion is $t = t$. (ii) Apply TR_v to $s = s$ and $s = t$; the conclusion is $t = s$. (iii) Apply TR_v to $t = r$ and $t = s$, the latter by (ii); the conclusion is $s = r$. (iv) Apply TR_h to $s = t$ and $h(s) = h(s)$; the conclusion is $h(s) = h(t)$. (v) TR_a applied to $s = t$ and $a(s) = b(s)$ gives $b(s) = a(t)$, and TR_b applied to $s = t$ and $b(s) = a(t)$ gives $a(t) = b(t)$. □

By (ii)–(iv), provable equality over a fixed context is a congruence for every term template, and chains of equations are used below in the usual way. The selector equations

$$\Gamma \vdash \mathrm{sel}(0, a, b) = a, \qquad \Gamma \vdash \mathrm{sel}(Sc, a, b) = b \tag{1.4}$$

are instances of I0 and IS.

Lemma 1.19 (Simultaneous congruence)**.** *If* $\Gamma \vdash s_i = t_i$ *for* $i = 1, \ldots, k$*, then* $\Gamma \vdash a[\vec{s}/\vec{x}] = a[\vec{t}/\vec{x}]$ *for every term* a*.*

Proof. Let a_j be a with $t_1, \ldots, t_j, s_{j+1}, \ldots, s_k$ substituted for $\vec{x}$. With a fresh variable v and the template $h_j(v)$ obtained by substituting $t_1, \ldots, t_{j-1}, v, s_{j+1}, \ldots, s_k$, one has $a_{j-1} = h_j(s_j)$ and $a_j = h_j(t_j)$ as terms, so Theorem 1.18(iv) gives $\Gamma \vdash a_{j-1} = a_j$. Transitivity joins $a_0 = a[\vec{s}/\vec{x}]$ to $a_k = a[\vec{t}/\vec{x}]$. □

Lemma 1.20 (Equation induction)**.** *Let* $E(y)$ *be a literal equation and* z *a variable not free in* Γ *and* $Q_y E$*. If* $\Gamma \vdash E(0)$ *and* $\Gamma, E(z) \vdash E(Sz)$*, then* $\Gamma \vdash E(t)$ *for every term* t*.*

Proof. QI and QE. □

In Lean this rule is `NPrf.eqInd`.

1.4.4 Congruence of the binders and the Leibniz rule

Lemma 1.21 (Congruence of Q and D)**.** *Let B and B' be interderivable over Γ.*

(i) *If y is not free in Γ, then Q_yB and Q_yB' are interderivable over Γ.*

(ii) *If x and y are not free in Γ, then $D_{x,y}B$ and $D_{x,y}B'$ are interderivable over Γ.*

Proof. Both parts rest on one derivation. Let $C(y)$ and $C'(y)$ be interderivable over Γ, with y not free in Γ, and let z be fresh. Then

$$\Gamma,\ Q_yC(y),\ C'(0) \vdash C'(Sz). \tag{1.5}$$

Indeed, substituting 0 for y in $\Gamma, C' \vdash C$ gives $\Gamma, C'(0) \vdash C(0)$, because y is not free in Γ; QE at Sz gives $C(Sz)$; and substituting Sz for y in $\Gamma, C \vdash C'$ gives $\Gamma, C(Sz) \vdash C'(Sz)$.

(i) Theorem 1.13 applied to (1.5) for $C = B$ and $C' = B'$ gives $\Gamma, Q_yB \vdash Q_yB'$. The converse is symmetric.

(ii) Over $\Gamma, D_{x,y}B$ apply DE with a fresh b; the new hypothesis is $Q_yB(b,y)$. Substituting b for x shows that $B(b,y)$ and $B'(b,y)$ are interderivable over Γ. DI with the witness b and the matrix B' asks for $\Gamma, Q_yB(b,y), B'(b,0) \vdash B'(b,Sz)$, which is (1.5) for $C = B(b,y)$ and $C' = B'(b,y)$. Hence $\Gamma, D_{x,y}B \vdash D_{x,y}B'$, and the converse is symmetric. □

The binder Q is contravariant in its instance at zero, which is why both directions of the hypothesis are used in each direction of the conclusion.

Theorem 1.22 (Leibniz rule)**.** *Let A be a native formula and let $\Gamma \vdash s_i = t_i$ for $i = 1, \dots, k$. Then $A[\vec{s}/\vec{x}]$ and $A[\vec{t}/\vec{x}]$ are interderivable over Γ. In particular $\Gamma \vdash A[\vec{s}/\vec{x}]$ implies $\Gamma \vdash A[\vec{t}/\vec{x}]$.*

Proof. By induction on A, for all Γ, $\vec{s}$ and $\vec{t}$. If A is $a = b$, then Theorem 1.19 gives $\Gamma \vdash a[\vec{s}/\vec{x}] = a[\vec{t}/\vec{x}]$ and the same for b, and from the hypothesis $a[\vec{s}/\vec{x}] = b[\vec{s}/\vec{x}]$ symmetry and transitivity give $a[\vec{t}/\vec{x}] = b[\vec{t}/\vec{x}]$; the converse is the same argument with $\vec{s}$ and $\vec{t}$ exchanged. If A is Q_yB, rename y so that it is not free in Γ, $\vec{s}$, $\vec{t}$ and differs from $\vec{x}$. The induction hypothesis makes $B[\vec{s}/\vec{x}]$ and $B[\vec{t}/\vec{x}]$ interderivable over Γ, and Theorem 1.21(i) applies. The case $D_{x,y}B$ is the same with Theorem 1.21(ii). □

Corollary 1.23 (Transport in a formula template)**.** *If $\Gamma \vdash s = t$ and $\Gamma \vdash F(s)$, then $\Gamma \vdash F(t)$, for every native formula template $F(v)$.*

In Lean these are `NPrf.leibniz` and `NPrf.tr`; the two-step rule of Theorem 1.18(v) is `NPrf.trEq`. Transport never has to be primitive below a binder: the template is taken apart by QE, DE and DI.

1.4.5 Gates and selectors of formulas

A case distinction on a term cannot be expressed by a connective. It is pushed into a formula, down to its equations.

Definition 1.24 (Selector and gate of formulas)**.** For a term v and native formulas A, B the native formula $\mathrm{sel}(v, A, B)$ is defined by recursion, first on A and, for an equation A, on B. Bound variables are chosen fresh for v and for the other formula.

$$\begin{aligned}
\mathrm{sel}(v,\ s = t,\ r = w) &:= \big(\mathrm{sel}(v,s,r) = \mathrm{sel}(v,t,w)\big),\\
\mathrm{sel}(v,\ s = t,\ Q_yB) &:= Q_y\ \mathrm{sel}\big(v,\ \mathrm{sel}(y,0,s) = \mathrm{sel}(y,0,t),\ B\big),\\
\mathrm{sel}(v,\ s = t,\ D_{x,y}B) &:= D_{x,y}\ \mathrm{sel}\big(v,\ \mathrm{sel}(y,0,s) = \mathrm{sel}(y,0,t),\ B\big),\\
\mathrm{gate}(v,B) &:= \mathrm{sel}(v,\ 0 = 0,\ B),\\
\mathrm{sel}(v,\ Q_yA,\ B) &:= Q_y\ \mathrm{sel}\big(v,\ A,\ \mathrm{gate}(y,B)\big),\\
\mathrm{sel}(v,\ D_{x,y}A,\ B) &:= D_{x,y}\ \mathrm{sel}\big(v,\ A,\ \mathrm{gate}(y,B)\big).
\end{aligned}$$

The intention is that $\mathrm{sel}(v, A, B)$ is A when v is zero and B when v is a successor, and that $\mathrm{gate}(v, B)$ is trivially derivable when v is zero and is B when v is a successor. When a binder of one formula has no counterpart in the other, the other formula is guarded by the bound variable, so that the instance of the binder at zero is trivial. By induction on the definition, substitution commutes with both operations:

$$\mathrm{sel}(v, A, B)[\theta] = \mathrm{sel}(v[\theta], A[\theta], B[\theta]), \qquad \mathrm{gate}(v, B)[\theta] = \mathrm{gate}(v[\theta], B[\theta]). \tag{1.6}$$

Lemma 1.25 (A guarded equation under a binder)**.** *If x and y are not free in s and t, then*

$$Q_y\big(\mathrm{sel}(y, 0, s) = \mathrm{sel}(y, 0, t)\big) \dashv\vdash (s = t) \dashv\vdash D_{x,y}\big(\mathrm{sel}(y, 0, s) = \mathrm{sel}(y, 0, t)\big).$$

Proof. From the closure: its instance at 0 is $\mathrm{sel}(0, 0, s) = \mathrm{sel}(0, 0, t)$, which is derivable because both sides equal 0 by (1.4). QE at 1 gives $\mathrm{sel}(1, 0, s) = \mathrm{sel}(1, 0, t)$, and (1.4) turns it into $s = t$. From $s = t$: (1.4) gives $\mathrm{sel}(Sz, 0, s) = s = t = \mathrm{sel}(Sz, 0, t)$, which is the premise of Theorem 1.13. The second equivalence follows from the first by Theorem 1.12. □

Lemma 1.26 (Laws of the gate and the selector)**.** *For all terms v and native formulas A, B:*

(i) $\vdash \mathrm{gate}(0, B)$*;*

(ii) $\mathrm{gate}(Sv, B) \dashv\vdash B$*;*

(iii) $Q_y\,\mathrm{gate}(y, B) \dashv\vdash B \dashv\vdash D_{x,y}\,\mathrm{gate}(y, B)$ *for $x, y \notin \mathrm{FV}(B)$;*

(iv) $\mathrm{sel}(0, A, B) \dashv\vdash A$*;*

(v) $\mathrm{sel}(Sv, A, B) \dashv\vdash B$*.*

Proof. First let A be an equation $s = t$. The two statements

$$\mathrm{sel}(0,\ s = t,\ B) \dashv\vdash (s = t), \qquad \mathrm{sel}(Sv,\ s = t,\ B) \dashv\vdash B$$

are proved by induction on B, for all s, t and v. If B is $r = w$, they hold by (1.4) and the equality rules. If B is $Q_y B'$, the induction hypothesis for B' and the equation $\mathrm{sel}(y, 0, s) = \mathrm{sel}(y, 0, t)$ gives

$$\mathrm{sel}\big(0,\ \mathrm{sel}(y, 0, s) = \mathrm{sel}(y, 0, t),\ B'\big) \dashv\vdash \big(\mathrm{sel}(y, 0, s) = \mathrm{sel}(y, 0, t)\big),$$
$$\mathrm{sel}\big(Sv,\ \mathrm{sel}(y, 0, s) = \mathrm{sel}(y, 0, t),\ B'\big) \dashv\vdash B'.$$

By Theorem 1.21(i) the first formula of the statement is interderivable with $Q_y(\mathrm{sel}(y, 0, s) = \mathrm{sel}(y, 0, t))$, hence with $s = t$ by Theorem 1.25, and the second with $Q_y B'$. The case $D_{x,y}B'$ is the same with Theorem 1.21(ii) and the second equivalence of Theorem 1.25.

(i) and (ii) are the two statements for the equation $0 = 0$, which is derivable by reflexivity.

(iii) From $Q_y\,\mathrm{gate}(y, B)$: the instance at 0 is derivable by (i), QE at 1 gives $\mathrm{gate}(1, B)$, and (ii) gives B. From B: (ii) gives $\mathrm{gate}(Sz, B)$, the premise of Theorem 1.13. The second equivalence follows by Theorem 1.12.

(iv) and (v) are proved by induction on A, for all v and B. For an equation they are the two statements above. If A is $Q_y A'$, then $\mathrm{sel}(0, A, B) = Q_y\,\mathrm{sel}(0, A', \mathrm{gate}(y, B))$ is interderivable with $Q_y A'$ by the induction hypothesis and Theorem 1.21(i); and $\mathrm{sel}(Sv, A, B) = Q_y\,\mathrm{sel}(Sv, A', \mathrm{gate}(y, B))$ is interderivable with $Q_y\,\mathrm{gate}(y, B)$ by the induction hypothesis and Theorem 1.21(i), hence with B by (iii). The case $D_{x,y}A'$ is the same with Theorem 1.21(ii). □

Lemma 1.27 (Selector at an arbitrary index)**.** (i) *If $\Gamma \vdash A$ and $\Gamma \vdash B$, then $\Gamma \vdash \mathrm{sel}(w, A, B)$ for every term w.*

(ii) *If $\Gamma \vdash v = w$ and $\Gamma \vdash \mathrm{sel}(v, A, B)$, then $\Gamma \vdash \mathrm{sel}(w, A, B)$.*

Proof. (i) Apply Theorem 1.14 to the formula $\mathrm{sel}(y, A, B)$ with y fresh: its instance at 0 follows from A by Theorem 1.26(iv), and its instance at Sz follows from B by Theorem 1.26(v). The instances are computed by (1.6). (ii) Theorem 1.23 with the template $\mathrm{sel}(y, A, B)$. □

1.4.6 The connectives as native formulas

Definition 1.28 (Native connectives)**.** With fresh bound variables and the parity term N of Theorem 1.3,

$$\begin{aligned} \bot_* &:= (0 = S0),\\ A \Rightarrow_* B &:= Q_y\ \mathrm{sel}(y, A, B),\\ A \wedge_* B &:= Q_y\ \mathrm{gate}\big(y,\ \mathrm{sel}(N(y), A, B)\big),\\ \exists_* x\, A &:= D_{x,y}\ \mathrm{gate}(y, A),\\ A \vee_* B &:= \exists_* x\ \mathrm{sel}(x, A, B),\\ \forall_* x\, A &:= A(0) \wedge_* Q_x A. \end{aligned}$$

By (1.6) each of these operations commutes with substitution. The parity term satisfies

$$\vdash N(0) = 0, \qquad \vdash N(1) = 1, \qquad \vdash N(2) = 0, \tag{1.7}$$

by I0, by $N(S0) = Z(N(0)) = Z(0) = \mathrm{sel}(0, 1, 0) = 1$, and by $N(S1) = Z(N(1)) = Z(1) = \mathrm{sel}(S0, 1, 0) = 0$. The conjunction reads its two operands off the instances at 1 and at 2.

Theorem 1.29 (Natural deduction for the native connectives)**.** *The following rules are derivable in* N*, for all native formulas.*

(i) $\bot_* E$*:* $\Gamma \vdash \bot_*$ *implies* $\Gamma \vdash A$*.*

(ii) $\Rightarrow_* I$*:* $\Gamma, A \vdash B$ *implies* $\Gamma \vdash A \Rightarrow_* B$*.* $\Rightarrow_* E$*:* $\Gamma \vdash A \Rightarrow_* B$ *and* $\Gamma \vdash A$ *imply* $\Gamma \vdash B$*.*

(iii) $\wedge_* I$*:* $\Gamma \vdash A$ *and* $\Gamma \vdash B$ *imply* $\Gamma \vdash A \wedge_* B$*.* $\wedge_* E$*:* $\Gamma \vdash A \wedge_* B$ *implies* $\Gamma \vdash A$ *and* $\Gamma \vdash B$*.*

(iv) $\exists_* I$*:* $\Gamma \vdash A(t)$ *implies* $\Gamma \vdash \exists_* x\, A(x)$*.* $\exists_* E$*:* $\Gamma \vdash \exists_* x\, A(x)$ *and* $\Gamma, A(b) \vdash C$*, with* b *not free in* Γ*,* C *and* $\exists_* x\, A$*, imply* $\Gamma \vdash C$*.*

(v) $\vee_* I$*: each of* $\Gamma \vdash A$*,* $\Gamma \vdash B$ *implies* $\Gamma \vdash A \vee_* B$*.* $\vee_* E$*:* $\Gamma \vdash A \vee_* B$*,* $\Gamma, A \vdash C$ *and* $\Gamma, B \vdash C$ *imply* $\Gamma \vdash C$*.*

(vi) $\forall_* I$*:* $\Gamma \vdash A(x)$ *with* x *not free in* Γ *implies* $\Gamma \vdash \forall_* x\, A(x)$*.* $\forall_* E$*:* $\Gamma \vdash \forall_* x\, A(x)$ *implies* $\Gamma \vdash A(t)$*.*

Proof. (i) From $0 = S0$ every equation $s = t$ follows: $s = \mathrm{sel}(0, s, t) = \mathrm{sel}(S0, s, t) = t$, by (1.4) and congruence in the first argument of the selector. Every formula follows by induction on the number of its binders, a measure that substitution does not change. For $Q_y B$ use Theorem 1.13, whose premise $B(Sz)$ holds by the induction hypothesis; for $D_{x,y} B$ use DI with the witness 0, whose premise $B(0, Sz)$ holds by the induction hypothesis.

(ii) Introduction. By Theorem 1.13 it suffices to derive $\Gamma, \mathrm{sel}(0, A, B) \vdash \mathrm{sel}(Sz, A, B)$. Theorem 1.26(iv) gives A, the premise gives B, and Theorem 1.26(v) gives $\mathrm{sel}(Sz, A, B)$. Elimination. Theorem 1.26(iv) turns A into $\mathrm{sel}(0, A, B)$, QE at 1 gives $\mathrm{sel}(1, A, B)$, and Theorem 1.26(v) gives B.

(iii) Introduction. By Theorem 1.13 it suffices to derive $\mathrm{gate}(Sz, \mathrm{sel}(N(Sz), A, B))$ from Γ, and by Theorem 1.26(ii) it suffices to derive $\mathrm{sel}(N(Sz), A, B)$, which is Theorem 1.27(i). Elimination. The instance of the matrix at 0 is derivable by Theorem 1.26(i). QE at Sc and Theorem 1.26(ii) give $\Gamma \vdash \mathrm{sel}(N(Sc), A, B)$ for every term c. For $c = 1$, (1.7) and Theorem 1.27(ii) give $\mathrm{sel}(0, A, B)$, hence A. For $c = 0$ they give $\mathrm{sel}(1, A, B)$, hence B.

(iv) Introduction. DI with the witness t asks for $\mathrm{gate}(Sz, A(t))$, which follows from $A(t)$ by Theorem 1.26(ii). Elimination. DE with the fresh variable b adds the hypothesis $Q_y\, \mathrm{gate}(y, A(b))$, which gives $A(b)$ by Theorem 1.26(iii); cut with the second premise gives C.

(v) Introduction is (iv) with the witnesses 0 and 1 and Theorem 1.26(iv), (v). Elimination. By (iv) it suffices to derive C from $\Gamma, \mathrm{sel}(b, A, B)$ with b fresh. Apply Theorem 1.14 to the formula $\mathrm{sel}(y, A, B) \Rightarrow_* C$ with y fresh. Its instance at 0 follows by (ii) from $\Gamma, \mathrm{sel}(0, A, B) \vdash C$, which holds by Theorem 1.26(iv) and the premise $\Gamma, A \vdash C$; its instance at Sz follows in the same way from Theorem 1.26(v) and $\Gamma, B \vdash C$. The instance at b and (ii) give C.

(vi) Introduction. Substituting 0 for x in the premise gives $\Gamma \vdash A(0)$, and substituting Sz gives $\Gamma \vdash A(Sz)$, hence $\Gamma \vdash Q_x A$ by Theorem 1.13; (iii) combines the two. Elimination. By (iii) the premise gives $A(0)$ and $Q_x A$, and QE at t gives $A(t)$. □

Remark 1.30 (Which rules the logical interface uses). No derivation in this section applies QI, except the derivations of Theorem 1.20 and Theorem 1.16 and, through Theorem 1.65, the elimination rule of the single-binder universal (Theorem 1.32(ii)). The equality rules, the Leibniz rule, numerical cases and the whole of Theorem 1.29 are derived from the seven other rules. Cases supply the logic and progress supplies induction for equations; the two do not mix. If QI were allowed at a compound matrix, the proof of Theorem 1.20 would give induction for every native formula. The universal quantifier needs no numerical case distinction: its elimination is QE. The reference statement of the inherited basis, which has no binder Q, defines implication and the universal quantifier through D with an unused coordinate and eliminates the universal quantifier through the derived coverage $x = 0 \vee_* \exists_* v\,(x = Sv)$. Its abbreviations differ in detail from the ones above (the selector aligns the binders of its two formulas, and the matrix of the universal quantifier is gated), but they satisfy the same introduction and elimination rules and are therefore interderivable with them. In Lean the connectives and their rules are the module `NativeLogic`.

1.4.7 A universal quantifier with one binder

This subsection draws on `single_binder`, which writes the universal quantifier with one paired binder over one gated copy of its matrix, at the price of the predecessor term P of Theorem 1.65.

Definition 1.31 (The single-binder universal)**.** Let P be the predecessor term of Theorem 1.65; its only free variable is its argument. For a native formula $A(x)$ and distinct variables u, z not free in A, put

$$\mathcal{U}_x A(x) := D_{u,z}\,\mathrm{gate}(z,\ A(P(z))),$$

where $A(P(z))$ is $A[P(z)/x]$. The variable u does not occur in the matrix, and $\mathrm{FV}(\mathcal{U}_x A) = \mathrm{FV}(A) \setminus \{x\}$.

The predecessor is derived in the equational fragment $\mathcal{E}$ of Section 1.7, whose rules are rules or derived rules of N obtained without the connectives (Theorem 1.54(iv), Theorem 1.20); no result of this subsection is used there, so the forward reference is not circular. The consequence used below is

$$\Gamma \vdash P(St) = t \qquad \text{for every context } \Gamma \text{ and term } t, \tag{1.8}$$

by substitution of t for the variable of Theorem 1.65 and weakening. Because $P(z)$ has no free variable other than z, (1.6) gives $(\mathcal{U}_x A)[\theta] = \mathcal{U}_x(A[\theta])$ for every substitution θ that fixes x, u and z and whose values do not contain them. Since u does not occur in the matrix, Theorem 1.12 gives

$$\mathcal{U}_x A(x) \dashv\vdash Q_z\,\mathrm{gate}(z,\ A(P(z))), \tag{1.9}$$

a formula with the same single binder.

Proposition 1.32 (Rules of the single-binder universal)**.** *For every native formula $A(x)$, context Γ and term t:*

(i) $\mathcal{U}I$: *if* $\Gamma \vdash A(x)$ *and* x *is not free in* Γ, *then* $\Gamma \vdash \mathcal{U}_x A(x)$;

(ii) $\mathcal{U}E$: *if* $\Gamma \vdash \mathcal{U}_x A(x)$, *then* $\Gamma \vdash A(t)$;

(iii) $\mathcal{U}_x A \dashv\vdash \forall_* x\, A$. *The derivation of* $\forall_* x\, A \vdash \mathcal{U}_x A$ *does not apply* QI; *the derivation of* $\mathcal{U}_x A \vdash \forall_* x\, A$ *uses* (1.8) *once.*

Proof. Choose u and z as in Theorem 1.31 and write $B(z) := \mathrm{gate}(z, A(P(z)))$. For every term r,

$$B(r) = \mathrm{gate}\big(r,\ A(P(r))\big),$$

by (1.6) and Theorem 1.2, because z is not free in A and $P(z)[r/z] = P(r)$.

(i) Let z' be a variable not free in Γ and A. Since x is not free in Γ, substituting $P(Sz')$ for x in the premise (Theorem 1.10) gives $\Gamma \vdash A(P(Sz'))$, and Theorem 1.26(ii) at the term z' gives $\Gamma \vdash \mathrm{gate}(Sz', A(P(Sz')))$, that is, $\Gamma \vdash B(Sz')$. Weakening by $B(0)$ gives $\Gamma, B(0) \vdash B(Sz')$. This is the premise of DI with the matrix B, in which u does not occur, the witness 0 and the eigenvariable z'; the side conditions hold because z' is not free in Γ, in 0 or in $\mathcal{U}_x A$. DI gives $\Gamma \vdash D_{u,z} B = \mathcal{U}_x A$. The assumption $B(0)$ is not used.

(ii) Let b be a variable not free in Γ, A and t. By DE it suffices to derive $A(t)$ over the context Γ' that extends Γ by the hypothesis $Q_z B(z)$, which is the opened formula $Q_z B[b/u]$ because u does not occur in B; the side conditions of DE hold because b is not free in Γ, in $A(t)$ or in $\mathcal{U}_x A$. Over Γ': $B(0) = \mathrm{gate}(0, A(P(0)))$ is derivable by Theorem 1.26(i); QE at St gives $B(St) = \mathrm{gate}(St, A(P(St)))$; Theorem 1.26(ii) at the term t gives $A(P(St))$; and (1.8) with Theorem 1.23 for the template $A(x)$ gives $A(t)$.

(iii) By $\forall_*$E of Theorem 1.29(vi) at the variable x, $\forall_* x\, A \vdash A(x)$, and x is not free in $\forall_* x\, A$, so (i) gives $\forall_* x\, A \vdash \mathcal{U}_x A$. The rules used are $\forall_*$E, substitution, Theorem 1.26(ii) and DI; none of them applies QI (Theorem 1.30). Conversely, (ii) at the term x gives $\mathcal{U}_x A \vdash A(x)$, and x is not free in $\mathcal{U}_x A$, so $\forall_*$I gives $\mathcal{U}_x A \vdash \forall_* x\, A$. This direction uses (1.8), and the derivation of Theorem 1.65 applies equation induction, that is, QI. □

Thus $\mathcal{U}$ satisfies the same introduction and elimination rules as $\forall_*$ and may replace it wherever only these rules are used; the replacement in the bridge is Theorem 1.122. The interderivability holds in N with all eight rules. Whether it holds without QI is open (Theorem 1.131).

Remark 1.33 (What the single binder saves). Let the *binder rank* $r(A)$ be the number of occurrences of Q and D in A: $r(s = t) = 0$ and $r(Q_y B) = r(D_{x,y} B) = 1 + r(B)$. Substitution preserves rank. By induction on Theorem 1.24, first for an equation as first argument by induction on B and then by induction on A,

$$r\big(\mathrm{gate}(v, B)\big) = r(B), \qquad r\big(\mathrm{sel}(v, A, B)\big) = r(A) + r(B)\,:$$

the selector of Theorem 1.24 stacks the binders of its two formulas and does not align them. Hence, by Theorem 1.28, $r(A \wedge_* B) = 1 + r(A) + r(B)$ and

$$r(\forall_* x\, A) = 1 + r(A(0)) + r(Q_x A) = 2\, r(A) + 2, \qquad r(\mathcal{U}_x A) = r(A) + 1.$$

The formula $\forall_* x\, A$ contains two copies of A, the instance $A(0)$ and the matrix of $Q_x A$; the formula $\mathcal{U}_x A$ contains one, $A(P(z))$. For n universal quantifiers over one equation E the recursion $r_n = 2r_{n-1} + 2$, $r_0 = 0$, gives the rank $2^{n+1} - 2$ and 2^n copies of E for $\forall_*$, against the rank n and one copy of E for $\mathcal{U}$. The cost is moved into the terms: every free occurrence of x in A is replaced by the fixed term $P(z)$, and the gate replaces each side s of the terminal equation of A by $\mathrm{sel}(z, a, s)$, where a is a nest of $r(A)$ selectors over 0, one for each binder of A. No formula is duplicated. In the measure of Theorem 1.67, $|P(z)| = 75$, so each occurrence

of x adds 74 nodes. Any term with the two properties named in Theorem 1.132 may take the place of P; the predecessor P_{45} of Theorem 1.74 has them and $|P_{45}(z)| = 45$, so with it each occurrence adds 44 nodes (Theorem 1.78).

Remark 1.34 (The rules as `single_binder` states them). `single_binder` states the definition for the formulas of the inherited basis, built from equations and D alone, with the gate $\mathrm{gate}(v, s = t) := (\mathrm{sel}(v, 0, s) = \mathrm{sel}(v, 0, t))$ and $\mathrm{gate}(v, D_{a,b}B) := D_{a,b}\,\mathrm{gate}(v, B)$, and with a selector that aligns the binders of its two formulas; there the old universal adds two to the rank instead of $r(A) + 2$, and the package proves that its translation of an ordinary formula has binder rank exactly its logical depth. Its introduction transports $A(v)$ along $P(Sv) = v$; the substitution of $P(Sz')$ for the eigenvariable in the proof of Theorem 1.32(i) makes that transport unnecessary, so that only the elimination uses a law of the predecessor, and only $P(St) = t$; the equation $P(0) = 0$ is never used. The package adds that an introduction by progress serves as well, because the assumption of the premise is not used. In N this applies only when the matrix $\mathrm{gate}(z, A(P(z)))$ is a literal equation, that is, when A is an equation, since QI is restricted to literal equations; for a compound A the introduction is by cases. The package obtains the preservation of the formula round trips by an induction on ordinary formulas whose congruence steps it does not write out; they are given in the proof of Theorem 1.122.

1.4.8 Finite blocks under one binder

This subsection draws on the mathematical part of `fused_blocks_and_checkers`, which proves the compiler of finite conjunctions and disjunctions that `479_predecessor57` states in outline (Theorem 1.135) and shortens it by letting the absorbing factor map H_- of Section 1.7.7 both select the branch and supply the next control. The construction works with the formulas of the inherited basis and with the selector that aligns binders (Theorem 1.34), not with the stacking selector of Theorem 1.24. It uses arithmetic of Section 1.7.7, which is derived in the fragment $\mathcal{E}$ without any result of this subsection, so the forward reference is not circular. Throughout, $\vdash$ is derivability in N, and for a natural number k the *numeral* $\overline{k}$ is the term $S^k 0$.

Definition 1.35 (D-formulas, aligned gate and aligned selector)**.** A *D-formula* is a native formula without Q. It has the form $D_{x_1,y_1} \cdots D_{x_r,y_r}(s = t)$, with the *prefix* $D_{x_1,y_1} \cdots D_{x_r,y_r}$ and the *terminal equation* $s = t$; its rank $r(A) = r$ is that of Theorem 1.33. Put $T(A) := |s| + |t|$ in the measure of Theorem 1.67, and count one node for each binder and one for the equation:

$$|A| := r(A) + 1 + T(A).$$

Every repeated subterm is counted at each occurrence and every abbreviation is expanded. For a term c whose free variables are not bound in the prefix of A, the *aligned gate* $\gamma_c(A)$ is A with its terminal equation $s = t$ replaced by $\mathrm{sel}(c, 0, s) = \mathrm{sel}(c, 0, t)$. The *aligned selector* $\mu_c(A, B)$ of two D-formulas is defined by recursion on $\max\{r(A), r(B)\}$. At two equations,

$$\mu_c(s = t,\ p = q) := \big(\mathrm{sel}(c, s, p) = \mathrm{sel}(c, t, q)\big).$$

If at least one operand begins with a binder, choose distinct variables x, y not free in c, A, B; let A' be the matrix of A with its bound pair renamed to x, y if A begins with a binder, and $A' := \gamma_y(A)$ if A is an equation; define B' from B in the same way; and put

$$\mu_c(A, B) := D_{x,y}\,\mu_c(A', B').$$

The gate γ is the gate of Theorem 1.34, and μ is the selector named there. At a level where only one operand has a binder, the other operand, an equation, is gated by the new

bound variable; it remains an equation and is gated again at every further such level. By induction on the definitions, with bound variables renamed away from θ,

$$\gamma_c(A)[\theta] = \gamma_{c[\theta]}(A[\theta]), \qquad \mu_c(A,B)[\theta] = \mu_{c[\theta]}(A[\theta], B[\theta]).$$

Remark 1.36 (Arbitrary native operands). Let A^D be A with every Q_y replaced by $D_{x,y}$ for a fresh x. Then $A \dashv\vdash A^D$, by induction on A: for $A = Q_y B$, Theorem 1.21(i) and the induction hypothesis give $Q_y B \dashv\vdash Q_y B^D$, and Theorem 1.12 gives $Q_y B^D \dashv\vdash D_{x,y} B^D$; the case $D_{x,y}B$ is Theorem 1.21(ii). The formulas A and A^D have the same rank and the same terminal terms. The blocks below therefore apply to arbitrary native operands through A^D.

Lemma 1.37 (Laws of the aligned gate). *For every D-formula A and term v:*

(i) $\vdash \gamma_0(A)$;

(ii) $\gamma_{Sv}(A) \dashv\vdash A$;

(iii) $D_{x,y}\gamma_y(A) \dashv\vdash A$ *for* $x, y \notin \mathrm{FV}(A)$;

(iv) $r(\gamma_c(A)) = r(A)$ *and* $T(\gamma_c(A)) = T(A) + 2|c| + 4$.

Proof. (i) By induction on $r(A)$. For the terminal equation, $\mathrm{sel}(0,0,s) = 0 = \mathrm{sel}(0,0,t)$ by (1.4) and the equality rules. For $A = D_{a,b}B$, $\gamma_0(A) = D_{a,b}\gamma_0(B)$; DI with the witness 0 asks for $\gamma_0(B)[0/a][Sz/b]$, which is $\gamma_0(B[0/a][Sz/b])$ and is derivable by the induction hypothesis; the assumption of the premise is not used.

(ii) At the terminal equation, $\mathrm{sel}(Sv,0,s) = s$ and $\mathrm{sel}(Sv,0,t) = t$ by (1.4), so the two equations are interderivable by the equality rules; the prefix is added by Theorem 1.21(ii), one binder at a time, over the empty context.

(iii) From $D_{x,y}\gamma_y(A)$, DE with a fresh b adds $Q_y\gamma_y(A)$, since x does not occur in the matrix. Its instance at 0 is derivable by (i), QE at 1 gives $\gamma_1(A)$, and (ii) gives A. Conversely, from A (ii) gives $\gamma_{Sz}(A)$ for a fresh z, and DI with the witness 0, whose assumption is not used, gives $D_{x,y}\gamma_y(A)$.

(iv) Each side s of the terminal equation becomes $\mathrm{sel}(c,0,s) = I(c,0;\, u.\, s)$, with $1 + |c| + 1$ new nodes. □

Lemma 1.38 (Laws and cost of the aligned selector). *For all D-formulas A, B:*

(i) $\mu_0(A,B) \dashv\vdash A$ *and* $\mu_{Sv}(A,B) \dashv\vdash B$ *for every term v;*

(ii) $r(\mu_c(A,B)) = \max\{r(A), r(B)\}$ *and*

$$T(\mu_c(A,B)) = T(A) + T(B) + 2|c| + 2 + 6\,|r(A) - r(B)|;$$

(iii) *if* $\Gamma \vdash A$ *and* $\Gamma \vdash B$, *then* $\Gamma \vdash \mu_c(A,B)$, *for every term c;*

(iv) *if* $\Gamma, A \vdash F$ *and* $\Gamma, B \vdash F$, *then* $\Gamma, \mu_c(A,B) \vdash F$, *for every term c and native formula F.*

Proof. (i) By induction on $\max\{r(A), r(B)\}$, for all pairs and all v. At rank 0 the formulas are $\mathrm{sel}(0,s,p) = \mathrm{sel}(0,t,q)$ and $\mathrm{sel}(Sv,s,p) = \mathrm{sel}(Sv,t,q)$, and (1.4) with the equality rules makes them interderivable with $s = t$, respectively $p = q$. At positive rank $\mu_c(A,B) = D_{x,y}\mu_c(A',B')$, and the pair A', B' has smaller maximal rank. By the induction hypothesis and Theorem 1.21(ii), $\mu_0(A,B) \dashv\vdash D_{x,y}A'$ and $\mu_{Sv}(A,B) \dashv\vdash D_{x,y}B'$. If A begins with a binder, $D_{x,y}A'$ is A up to the renaming of its bound pair. If A is an equation, $D_{x,y}A' = D_{x,y}\gamma_y(A)$, which is interderivable with A by Theorem 1.37(iii), because x and y are fresh for A. The same holds for B.

(ii) By the same induction. At rank 0 each side gains $\mathrm{sel}(c,\cdot,\cdot)$, that is $1+|c|$ nodes besides the operand terms, so T grows by $2|c|+2$. At a level where both operands have a binder, the level is matched, the recursive pair has the same terminal terms up to renaming and the same rank difference, and the rank grows by one. At a level where only A has a binder, $B' = \gamma_y(B)$ adds $2|y|+4 = 6$ to T by Theorem 1.37(iv), and $|r(A') - r(B')| = |r(A) - r(B)| - 1$; the case where only B has a binder is symmetric. There are exactly $|r(A) - r(B)|$ unmatched levels, and the rank of the result is the maximum.

(iii) Let y be fresh. The instance of $\mu_y(A,B)$ at 0 is $\mu_0(A,B)$ and follows from A by (i); the instance at Sz follows from B. Theorem 1.14 gives the instance at c.

(iv) Let y be fresh for Γ, A, B, F, and apply Theorem 1.14 to the formula $\mu_y(A,B) \Rightarrow_* F$ of Theorem 1.28. Its instance at 0 follows by $\Rightarrow_*$I (Theorem 1.29(ii)) from $\Gamma, \mu_0(A,B) \vdash F$, which holds by (i) and $\Gamma, A \vdash F$; its instance at Sz follows in the same way from $\Gamma, B \vdash F$. The instance at c and $\Rightarrow_*$E give $\Gamma, \mu_c(A,B) \vdash F$. □

The rules (iii) and (iv) are cases on a number, not an induction on a compound formula.

Definition 1.39 (A local substitution)**.** Let A be a D-formula with terminal equation $s = t$, let w be a variable not bound in the prefix of A, and let the free variables of a not be bound in that prefix. Then $\mathrm{Let}_{w=a}(A)$ is A with its terminal equation replaced by

$$I(1, a;\, w.\, s) = I(1, a;\, w.\, t).$$

Lemma 1.40 (Local substitution)**.** $\mathrm{Let}_{w=a}(A) \dashv\vdash A[a/w]$, $r(\mathrm{Let}_{w=a}(A)) = r(A)$ *and* $T(\mathrm{Let}_{w=a}(A)) = T(A) + 2|a| + 6$.

Proof. By IS, $I(S0, a;\, w.\, s) = s[I(0, a;\, w.\, s)/w]$; by I0 and congruence, $s[I(0, a;\, w.\, s)/w] = s[a/w]$. So $\vdash I(1, a;\, w.\, s) = s[a/w]$, and likewise for t, and the equality rules make the two terminal equations interderivable. Because w and the free variables of a are not bound in the prefix, $A[a/w]$ has the prefix of A and the terminal equation $s[a/w] = t[a/w]$; Theorem 1.21(ii) adds the prefix. Each terminal term gains an iterator node, the two nodes of 1 and a copy of a. □

The argument a occurs twice in the expanded formula; there is no sharing. The operation is used so that a large argument is not substituted at every occurrence of w in the operand.

Lemma 1.41 (Transitions on numerals)**.** *The fragment $\mathcal{E}$ derives, for every term u and all natural numbers m, j and odd k:*

(i) $H_-(2u) = 3u$ *and* $H_-(S(2u)) = 0$*;*

(ii) $\bar{a} + \bar{b} = \overline{a+b}$, $2\bar{m} = \overline{2m}$ *and* $3\bar{m} = \overline{3m}$*;*

(iii) $H_-(\bar{k}) = 0$ *and, for* $j \geq 1$, $H_-(\overline{k2^j}) = \overline{3k2^{j-1}}$*;*

(iv) $N(\bar{k}) = 1$ *and, for* $j \geq 1$, $N(\overline{k2^j}) = 0$*;* $G_-(\overline{2m}) = \overline{3m} = G'(\overline{2m})$*;*

(v) $SI(\bar{j}, 0;\, u.\, S(u+u)) = \overline{2^j}$.

Proof. (i) The first equation is Theorem 1.71(ii). By Theorem 1.71(i), Theorem 1.59 and (1.4), $H_-(S(2u)) = \mathrm{sel}(N(S(2u)), G'(S(2u)), 0) = \mathrm{sel}(1, G'(S(2u)), 0) = 0$. (ii) $\bar{a} + \bar{b} = \overline{a+b}$ follows from $a + 0 = a$ and $a + Sb = S(a+b)$ (Theorem 1.57) by b steps, and the other two equations are instances. (iii) Write $k = 2i+1$; then $\bar{k} = S\overline{2i} = S(2\bar{i})$ by (ii), and (i) gives $H_-(\bar{k}) = 0$. For $j \geq 1$, $\overline{k2^j} = 2\,\overline{k2^{j-1}}$ by (ii), and (i) and (ii) give $H_-(\overline{k2^j}) = 3\,\overline{k2^{j-1}} = \overline{3k2^{j-1}}$. (iv) $N(S(2u)) = 1$ and $N(2u) = 0$ (Theorem 1.59) at $u = \bar{i}$, respectively $u = \overline{k2^{j-1}}$, with (ii); and $G_-(2u) = 3u = G'(2u)$ (Theorem 1.68(iv), Theorem 1.69(iv)) at $u = \bar{m}$, with (ii). (v) By I0 and IS the iterator is 0 at $\bar{0}$ and is $S(e+e)$ at $\overline{j+1}$, where e is its value at $\bar{j}$; by (ii) and induction on j (outside the calculus) it equals $\overline{2^j - 1}$. □

Recall from Section 1.7.7 the floor pair and the fused factor map,

$$G'(n) = I(n, 0;\ u.\, S\,\mathrm{sel}(M'(u),\ u,\ Su)), \qquad H_-(n) = I(n, n;\ v.\ \mathrm{sel}(v, G'(n), 0)),$$

with $|G'(v)| = 20$ and $|H_-(v)| = 26$ for a variable v, and the ceiling ramp G_- with $|G_-(v)| = 21$; also $|N(v)| = 8$. On the natural numbers H_- sends $2m$ to $3m$ and every odd number to 0; only the equations of Theorem 1.41 are used.

Definition 1.42 (Finite selectors)**.** Let $A_0, \dots, A_{n-1}$ be D-formulas, $n \geq 1$, and let v be a variable not free in them. The *fused selector* is

$$\begin{aligned} \Sigma_1(v; A_0) &:= A_0, \\ \Sigma_{n+1}(v; A_0, \dots, A_n) &:= \mathrm{Let}_{w=H_-(v)}\ \mu_w(A_0,\ \Sigma_n(w; A_1, \dots, A_n)), \end{aligned}$$

with w a fresh variable. For G one of G_-, G', the *separate selector* is

$$\begin{aligned} \Sigma^G_1(v; A_0) &:= A_0, \\ \Sigma^G_{n+1}(v; A_0, \dots, A_n) &:= \mu_{N(v)}(\mathrm{Let}_{w=G(v)}\ \Sigma^G_n(w; A_1, \dots, A_n),\ A_0). \end{aligned}$$

For a term t, $\Sigma_n(t; \vec{A})$ and $\Sigma^G_n(t; \vec{A})$ are the results of substituting t for v.

The selector $\Sigma^{G_-}_n$ is the construction of `479_predecessor57` stated in Theorem 1.135(b). In Σ_n the computed value $H_-(v)$ serves twice: zero selects the first operand, and a positive value selects the tail and becomes its control; one local substitution serves both purposes. The variable v occurs only in the two copies of the argument of the outermost local substitution, so $\Sigma_{n+1}(t; \vec{A}) = \mathrm{Let}_{w=H_-(t)}\, \mu_w(A_0, \Sigma_n(w; A_1, \dots, A_n))$, and by Theorem 1.40 and the substitution property of μ,

$$\Sigma_{n+1}(t; \vec{A}) \dashv\vdash \mu_{H_-(t)}(A_0,\ \Sigma_n(H_-(t); A_1, \dots, A_n)), \tag{1.10}$$

while $\Sigma^G_{n+1}(t; \vec{A}) = \mu_{N(t)}(\mathrm{Let}_{w=G(t)}\, \Sigma^G_n(w; A_1, \dots, A_n), A_0)$ with

$$\mathrm{Let}_{w=G(t)}\, \Sigma^G_n(w; A_1, \dots, A_n) \dashv\vdash \Sigma^G_n(G(t); A_1, \dots, A_n). \tag{1.11}$$

Theorem 1.43 (Selection and uniform deduction)**.** *Let Σ be Σ_n or Σ^G_n.*

(i) *For $0 \leq j < n$ and every odd k, $\Sigma(\overline{k2^j}; A_0, \dots, A_{n-1}) \dashv\vdash A_j$; in particular $\Sigma(\overline{2^j}; \vec{A}) \dashv\vdash A_j$, and $\Sigma^{G_-}_n(\omega_j; \vec{A}) \dashv\vdash A_j$ for the term ω_j of Theorem 1.135(b).*

(ii) *If $\Gamma \vdash A_i$ for every $i < n$, then $\Gamma \vdash \Sigma(t; \vec{A})$, for every term t.*

(iii) *If $\Gamma, A_i \vdash F$ for every $i < n$, then $\Gamma, \Sigma(t; \vec{A}) \vdash F$, for every term t.*

Proof. All three parts are proved by induction on n outside the calculus; for $n = 1$ the selector is A_0.

(i) For Σ_{n+1} and the control $\overline{k2^j}$, by (1.10), Theorem 1.41(iii) and Theorem 1.23 with the template $\mu_y(A_0, \Sigma_n(y; A_1, \dots, A_n))$, the selector is interderivable with $\mu_0(A_0, \dots)$ if $j = 0$, hence with A_0 by Theorem 1.38(i); and, if $j \geq 1$, with $\mu_{\bar{p}}(A_0, \Sigma_n(\bar{p}; A_1, \dots, A_n))$ for $p = (3k)2^{j-1} > 0$, hence with $\Sigma_n(\bar{p}; A_1, \dots, A_n)$, and since $3k$ is odd the induction hypothesis gives A_j, the operand of index $j-1$ of the tail. For Σ^G_{n+1} the same argument uses Theorem 1.41(iv) and transport in the control of μ: at $j = 0$ the control $N(\bar{k})$ is 1 and Theorem 1.38(i) selects A_0; at $j \geq 1$ it is 0 and selects the local substitution, which is interderivable with $\Sigma^G_n(G(\overline{k2^j}); A_1, \dots, A_n)$ by (1.11), and $G(\overline{k2^j}) = \overline{3k2^{j-1}}$. The case $k = 1$ is the particular

statement, and $\omega_j = \overline{2^j}$ by Theorem 1.41(v) and Theorem 1.23. For leaf j of Σ_n the successive controls are $2^j,\ 3\cdot 2^{j-1},\ \dots,\ 3^j$, and then 0 unless $j = n-1$.

(ii) By (1.10) it suffices to derive $\mu_{H_-(t)}(A_0, \Sigma_n(H_-(t); A_1, \dots, A_n))$. The induction hypothesis at the term $H_-(t)$ gives the second operand, and Theorem 1.38(iii) applies. For Σ^G, the induction hypothesis at $G(t)$ and (1.11) give the first operand of $\mu_{N(t)}$, and Theorem 1.38(iii) applies.

(iii) The induction hypothesis at $H_-(t)$ gives $\Gamma, \Sigma_n(H_-(t); A_1, \dots, A_n) \vdash F$; with $\Gamma, A_0 \vdash F$, Theorem 1.38(iv) gives $\Gamma, \mu_{H_-(t)}(\dots) \vdash F$, and (1.10) with cut gives the claim. For Σ^G, the induction hypothesis at $G(t)$ and (1.11) with cut give $\Gamma, \mathrm{Let}_{w=G(t)}\, \Sigma_n^G(w; A_1, \dots, A_n) \vdash F$, and Theorem 1.38(iv) with $\Gamma, A_0 \vdash F$ gives the claim directly. □

Parts (ii) and (iii) assert nothing about the orbit of the arithmetic control at an arbitrary t: only the finite number of levels is used, and no induction is applied to a formula with a binder. In (i) the closed odd coefficients suffice, since the stated endpoints have $k = 1$.

Theorem 1.44 (Exact size and optimal padding)**.** *Let $r_i := r(A_i)$ with $r_0 \ge r_1 \ge \cdots \ge r_{n-1}$, and put $r_{\max} := r_0$, $r_{\min} := r_{n-1}$. For a variable control v, the selectors have rank $r_{\max}$ and*

$$T(\Sigma_n(v; \vec{A})) = \textstyle\sum_i T(A_i) + 62(n-1) + 6(r_{\max} - r_{\min}),$$
$$T(\Sigma_n^{G'}(v; \vec{A})) = \textstyle\sum_i T(A_i) + 64(n-1) + 6(r_{\max} - r_{\min}),$$
$$T(\Sigma_n^{G_-}(v; \vec{A})) = \textstyle\sum_i T(A_i) + 66(n-1) + 6(r_{\max} - r_{\min}).$$

Over all binary trees of aligned selectors, with arbitrary controls and local substitutions, whose leaves are $A_0, \dots, A_{n-1}$, the number of unmatched levels, summed over the nodes, is at least $r_{\max} - r_{\min}$.

Proof. By induction on n; the rank of the tail is r_1 in each case. At a level of Σ_{n+1}, the selector μ_w at the variable w adds $2|w| + 2 = 4$ nodes and $6(r_0 - r_1)$ for padding (Theorem 1.38(ii)), and the local substitution adds $2|H_-(v)| + 6 = 58$ (Theorem 1.40); together 62. At a level of Σ_{n+1}^G the selector $\mu_{N(v)}$ adds $2|N(v)| + 2 = 18$ and $6(r_0 - r_1)$, and the local substitution adds $2|G(v)| + 6$, which is 48 for G_- and 46 for G'; together 66 and 64. The recursive selector is built at the fresh variable w, so no expanded arithmetic term is substituted into it, and every level has the same charge. The padding charges $(r_0 - r_1) + \cdots + (r_{n-2} - r_{n-1})$ telescope to $r_{\max} - r_{\min}$, and the rank is $r_{\max}$ by Theorem 1.38(ii).

For the lower bound, local substitutions do not change ranks, and by Theorem 1.38(ii) a node with children of ranks ρ and ρ' has rank $\max\{\rho, \rho'\}$ and $|\rho - \rho'|$ unmatched levels. Follow the path from the root to a leaf of rank $r_{\min}$. At each node of the path the number of unmatched levels is at least the drop from the rank of the node to the rank of the chosen child; the drops sum to $r_{\max} - r_{\min}$, and the other nodes contribute nonnegative amounts. The sorted right comb attains the bound. □

The fusion saves 2 nodes per branch against the separate selector with the floor ramp and 4 against the documented one. Replacing G_- by G' in the separate selector is valid because the tail is selected only at an even control, where both ramps give $3m$ (Theorem 1.41(iv)).

Definition 1.45 (Finite blocks)**.** With u, x, z distinct variables not free in $\vec{A}$,

$$\mathsf{And}(\vec{A}) := D_{u,z}\, \gamma_z(\Sigma_n(z; \vec{A})), \qquad \mathsf{Or}(\vec{A}) := D_{x,z}\, \gamma_z(\Sigma_n(x; \vec{A})),$$

and And^G, Or^G in the same way with Σ_n^G. The witness coordinate u does not occur in the matrix of the conjunction.

Theorem 1.46 (Finite conjunction and disjunction)**.** *For every context Γ and native formula F, and with Σ, And, Or fused or separate:*

(i) *if* $\Gamma \vdash A_i$ *for every* $i < n$, *then* $\Gamma \vdash \mathsf{And}(\vec{A})$; *and* $\mathsf{And}(\vec{A}) \vdash A_j$ *for every* $j < n$;

(ii) $A_j \vdash \mathsf{Or}(\vec{A})$ *for every* $j < n$; *and if* $\Gamma, A_i \vdash F$ *for every* $i < n$, *then* $\Gamma, \mathsf{Or}(\vec{A}) \vdash F$;

(iii) *with the operands ordered by nonincreasing rank, both blocks have rank* $r_{\max} + 1$ *and*

$$|\mathsf{And}(\vec{A})| = |\mathsf{Or}(\vec{A})| = \textstyle\sum_i T(A_i) + 62(n-1) + 6(r_{\max} - r_{\min}) + r_{\max} + 8,$$

with 66 *for* G_- *and* 64 *for* G' *in place of* 62.

Proof. (i) Introduction: Theorem 1.43(ii) at Sz', for z' fresh, gives $\Sigma(Sz'; \vec{A})$, and Theorem 1.37(ii) gives $\gamma_{Sz'}(\Sigma(Sz'; \vec{A}))$, the instance of the matrix at $z := Sz'$. DI with the witness 0, whose assumption is not used, gives $\mathsf{And}(\vec{A})$. Projection: DE with a fresh b adds $Q_z\gamma_z(\Sigma(z; \vec{A}))$. Its instance at 0 is derivable by Theorem 1.37(i). QE at the positive numeral $\overline{2^j}$ and Theorem 1.37(ii) give $\Sigma(\overline{2^j}; \vec{A})$, and Theorem 1.43(i) gives A_j, in which b does not occur.

(ii) Injection: from A_j, Theorem 1.43(i) gives $\Sigma(\overline{2^j}; \vec{A})$ and Theorem 1.37(ii) gives $\gamma_{Sz'}(\Sigma(\overline{2^j}; \vec{A}))$; DI with the witness $\overline{2^j}$ gives $\mathsf{Or}(\vec{A})$. Elimination: DE with a b not free in Γ, F and $\vec{A}$ adds $Q_z\gamma_z(\Sigma(b; \vec{A}))$; its instance at 0 is derivable, QE at 1 and Theorem 1.37(ii) give $\Sigma(b; \vec{A})$, and Theorem 1.43(iii) at $t = b$ with cut gives F.

(iii) By Theorem 1.44 the selector has rank $r_{\max}$; the gate at the variable z adds 6 to T (Theorem 1.37(iv)) and the outer binder adds one to the rank. Hence the size is $(r_{\max} + 1) + 1 + T(\Sigma) + 6$. □

All derivations are uniform over the context. For the same operands the three compilers have the same surrounding and padding charges, so the fused blocks are smaller by exactly $4(n-1)$ nodes than those of Σ^{G_-} and by $2(n-1)$ than those of $\Sigma^{G'}$. When all operands are equations with $T(A_i) = 2$, the three block sizes are $68n - 58$, $66n - 56$ and $64n - 54$:

n	Σ^{G_-} (ceiling ramp)	$\Sigma^{G'}$ (floor ramp)	Σ (fused)
1	10	10	10
2	78	76	74
4	214	208	202
8	486	472	458
16	1030	1000	970
32	2118	2056	1994
64	4294	4168	4042

These blocks have rank 1; their operand equations need not be derivable.

Remark 1.47 (Scope and sources). The rules of N are unchanged, and so is Theorem 1.119; the result is a smaller expansion of finite logical blocks, not a new rule basis and not a bound on the size of a checker. No optimality of the constant 62 is claimed; the padding bound of Theorem 1.44 concerns trees of aligned selectors only and is not a lower bound for all interderivable native formulas. The counts concern the native D-formula syntax. For operands with binders, the terminal gate and the local substitution need not preserve the literal public guard of the compact checkers of the chapter on certificates, so such a block must first be translated into that guarded form before it serves as a checker goal; the size of that translation is not included. `fused_blocks_and_checkers` proves the endpoint law and the uniform rules for Σ_n only, and states the charges 66 and 64 of the separate selectors with the validity of the floor ramp; the proofs for Σ_n^G in Theorems 1.43 and 1.46 are the same induction. The package also supplies a program that expands the old and the new constructions, checks their raw-tree counts and scopes, checks the padding minimum exhaustively for small trees and

evaluates the routes of the controls; these finite checks supplement the proofs above and are not a formalization. None of the constructions of this subsection is formalized in Lean or Rocq.

1.5 The ordinary theory T_{eq}

This section draws on the reference statement of the inherited ordinary bridge filed with `certified_zero` and on the section on the ordinary realization of `raw_clausal`.

Definition 1.48 (The logic)**.** *Intuitionistic logic with equality* over a language of terms is the natural deduction calculus for sequents $\Gamma \vdash \varphi$ of ordinary formulas with the rules: hypothesis; $\bot$E; introduction and elimination for $\wedge$, $\vee$, $\to$, $\forall$ and $\exists$, with the usual eigenvariable conditions; reflexivity $\Gamma \vdash t = t$; and the Leibniz rule, from $\Gamma \vdash s_i = t_i$ for all i and $\Gamma \vdash \varphi[\vec{s}/\vec{x}]$ infer $\Gamma \vdash \varphi[\vec{t}/\vec{x}]$. A *theory* is a set of formulas, its axioms; every substitution instance of an axiom is derivable in every context. Derivability is closed under weakening, substitution and cut.

Definition 1.49 (T_{eq})**.** T_{eq} is the theory in intuitionistic logic with equality over the terms of Theorem 1.1 with the following axioms.

(T1) The computation equations (1.1) and (1.2), for all terms.

(T2) Coverage: $x = 0 \ \vee \ \exists y\,(x = Sy)$.

(T3) Separation: $0 = S0 \to \bot$.

(T4) Induction for literal equations: for every equation $E(z)$,

$$E(0) \to (\forall z\,(E(z) \to E(Sz)) \to \forall z\,E(z)).$$

Injectivity of the successor, $Sx \neq 0$ and induction for compound formulas are not axioms. In Lean the axioms are `TeqAx`.

Lemma 1.50 (Cases and contextual induction in T_{eq})**.** (i) *For every ordinary formula $\varphi(y)$: if $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi(0)$ and $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi(Sz)$ with z not free in Γ and φ, then $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \forall y\,\varphi(y)$.*

(ii) *For every equation $E(z)$: if $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} E(0)$ and $\Gamma, E(z) \vdash_{\mathsf{T}_{\mathrm{eq}}} E(Sz)$ with z not free in Γ, then $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} E(t)$ for every term t.*

Proof. (i) Let y be fresh and eliminate the disjunction (T2) at $x = y$. In the branch $y = 0$ the Leibniz rule turns $\varphi(0)$ into $\varphi(y)$. In the other branch eliminate the existential quantifier at the fresh variable z; from $y = Sz$ the Leibniz rule turns $\varphi(Sz)$ into $\varphi(y)$. Generalize over y. (ii) Discharge $E(z)$, generalize over z, apply (T4) and instantiate at t. The context Γ is arbitrary, so the rule holds with quantified side hypotheses. □

Remark 1.51 (Separation is an axiom). The computation equations, coverage and equation induction hold in the structure with one element, in which every equation is true. Separation fails there. Hence separation does not follow from the other axioms, and the native contradiction $0 = S0$ has to be linked to $\bot$ explicitly: the reading of the native calculus proves every formula from $0 = S0$, and the translation of $\bot$ is that equation, but neither fact removes (T3) from T_{eq}.

1.6 Intuitionistic primitive recursive arithmetic

This section fixes the usual presentation, as in the reference statement on equality reflection filed with `certified_zero` and in the Lean module `PR`.

Definition 1.52 (iPRA)**.** The *function symbols* are generated as follows, each with an arity: zero of arity 0; succ of arity 1; the projections π_i^k of arity k, for $1 \le i \le k$; the composition $\mathrm{comp}(f; g_1, \dots, g_m)$ of arity k, for f of arity m and $g_1, \dots, g_m$ of arity k; and the recursion $\mathrm{prec}(f, g)$ of arity $k+1$, for f of arity k and g of arity $k+2$. The terms are the variables and the applications $f(t_1, \dots, t_k)$; 0 is zero() and St is succ(t). iPRA is the theory in intuitionistic logic with equality over these terms with the axioms

$$\begin{aligned} \pi_i^k(\vec{x}) &= x_i, & \mathrm{comp}(f; \vec{g})(\vec{x}) &= f(g_1(\vec{x}), \dots, g_m(\vec{x})), \\ \mathrm{prec}(f, g)(0, \vec{x}) &= f(\vec{x}), & \mathrm{prec}(f, g)(Sy, \vec{x}) &= g\big(y, \mathrm{prec}(f, g)(y, \vec{x}), \vec{x}\big), \\ Sx = 0 &\to \bot, & Sx = Sy &\to x = y, \end{aligned}$$

and induction for quantifier-free formulas: for every quantifier-free $\varphi(z)$,

$$\varphi(0) \to \big(\forall z\, (\varphi(z) \to \varphi(Sz)) \to \forall z\, \varphi(z)\big).$$

The recursion argument of $\mathrm{prec}(f, g)$ comes first. Distinct descriptions are distinct symbols, even when they compute the same function on the natural numbers. The quantifiers of the logic are unrestricted; only the induction formula is quantifier free. In Lean the axioms are `PRAAx`.

1.7 Arithmetic inside T_{eq}

This section draws on the reference statements filed with `certified_zero` (scalar predecessor, equality reflection, primitive recursion) and, for the predecessor, on the equational derivations of `uniform_raw`, `newest_parameter`, `parameter_witness` and `no_witness`, with a smaller predecessor from `479_predecessor57`, `absorbing_orbit`, `absorbing_predecessor`, `pred50` and `fused_iteration`, the replacement of coverage from `zero_test_split`, and the diagonal counter and the orbit equality test from `orbit_equality`. The language has no predecessor and no pairing, and the body of an iterator does not see the counter. The arithmetic is developed in this order: a predecessor term; truncated subtraction and a term that reflects equality; induction for quantifier-free formulas; pairing; primitive recursion. No later item is used for an earlier one.

1.7.1 The equational fragment

Definition 1.53 (The fragment $\mathcal{E}$)**.** A sequent of $\mathcal{E}$ is $\Gamma \vdash s = t$ with Γ a finite list of equations. The rules of $\mathcal{E}$ are HYP, I0, IS, TR and *equation induction*:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}\ \mathrm{EI} \qquad (E \text{ an equation, } z \text{ not free in } \Gamma \text{ and in } E(0)).$$

Lemma 1.54 (Properties of $\mathcal{E}$)**.** (i) *$\mathcal{E}$ is closed under weakening, substitution and cut.*

(ii) *Reflexivity, symmetry, transitivity and congruence (Theorems 1.18 and 1.19) are derived rules of $\mathcal{E}$.*

(iii) Equational cases*: if $\Gamma \vdash E(0)$ and $\Gamma \vdash E(Sz)$ with z fresh, then $\Gamma \vdash E(t)$.*

(iv) *If $\Gamma \vdash_{\mathcal{E}} s = t$, then $\Gamma \vdash_{\mathsf{N}} s = t$ and $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} s = t$.*

Proof. (i) The proofs of Theorems 1.10 and 1.11 apply; EI is treated like QI, with the eigenvariable renamed. (ii) The derivations use I0 and TR only. (iii) Weaken the second premise by $E(z)$ and apply EI. (iv) EI is the derived rule of Theorem 1.20 in N and of Theorem 1.50(ii) in T_{eq}; TR follows in T_{eq} from the Leibniz rule; the other rules are shared. □

By (i), an equation derived in $\mathcal{E}$ with variables that are not free in the context may be instantiated at arbitrary terms. Every case distinction below proves an equation with an independent case variable and is an instance of (iii); no hypothesis is ever specialized to a case. The fragment has no coverage axiom, no disjunction, no separation and no rule for D. Until Theorem 1.65 all derivations are in $\mathcal{E}$, in the empty context unless a context is shown.

Lemma 1.55 (Uniqueness of iteration)**.** *Let $f(z)$ be a term template and a, $h(u)$ terms in which z is not free, and let z be not free in Γ. If $\Gamma \vdash f(0) = a$ and $\Gamma \vdash f(Sz) = h(f(z))$, then $\Gamma \vdash f(c) = I(c, a;\, u.\, h)$ for every term c.*

Proof. EI on the equation $f(z) = I(z, a;\, u.\, h)$. The base is $f(0) = a = I(0, a;\, u.\, h)$. In the step, $f(Sz) = h(f(z)) = h(I(z, a;\, u.\, h)) = I(Sz, a;\, u.\, h)$, by the hypothesis, congruence on the induction equation, and IS. □

In Lean the rule is `NPrf.uniq`.

Corollary 1.56 (Rotation and congruence of the body)**.** (i) $\vdash I(Sy, a;\, u.\, h) = I(y, h(a);\, u.\, h)$.

(ii) *If $\Gamma \vdash h(u) = h'(u)$ with u not free in Γ, then $\Gamma \vdash I(c, a;\, u.\, h) = I(c, a;\, u.\, h')$.*

Proof. (i) Take $f(z) := I(Sz, a;\, u.\, h)$. Then $f(0) = h(I(0, a;\, u.\, h)) = h(a)$ and $f(Sz) = h(f(z))$ by IS, I0 and congruence, and Theorem 1.55 gives the claim. (ii) Take $f(z) := I(z, a;\, u.\, h)$. Then $f(0) = a$ and $f(Sz) = h(f(z)) = h'(f(z))$, the last step by instantiating the hypothesis at $u = f(z)$. Theorem 1.55 with the body h' gives the claim. □

Thus no extensionality rule for bodies is assumed; it is an equation induction.

1.7.2 Addition

Put $a + b := I(b, a;\, u.\, Su)$, $2a := a + a$ and $3a := (a + a) + a$; the same terms are also written $2 \cdot a$ and $3 \cdot a$.

Lemma 1.57 (Addition)**.** *$\mathcal{E}$ derives*

$$a + 0 = a, \qquad a + Sb = S(a + b), \qquad 0 + b = b, \qquad Sa + b = S(a + b),$$
$$a + b = b + a, \qquad (a + b) + c = a + (b + c).$$

Consequently $Sa = a + 1$, $2(Sa) = SS(2a)$, $3(Sa) = SSS(3a)$, $3(a + b) = 3a + 3b$, $2 \cdot 0 = 0$ and $3 \cdot 0 = 0$.

Proof. The first two equations are I0 and IS. The others are equation inductions on the last variable shown. $0 + b = b$: the base is $0 + 0 = 0$, and $0 + Sb = S(0 + b) = Sb$. $Sa + b = S(a + b)$: the base is $Sa + 0 = Sa = S(a + 0)$, and $Sa + Sb = S(Sa + b) = SS(a + b) = S(a + Sb)$. Commutativity: the base is $a + 0 = a = 0 + a$, and $a + Sb = S(a + b) = S(b + a) = Sb + a$. Associativity: the base is $(a + b) + 0 = a + b = a + (b + 0)$, and $(a + b) + Sc = S((a + b) + c) = S(a + (b + c)) = a + S(b + c) = a + (b + Sc)$. The consequences follow by these equations. □

Sums are rearranged below by Theorem 1.57 without comment. No cancellation law is used, and there is no multiplication.

1.7.3 Zero test, interpolation and parity

Lemma 1.58 (Zero test and interpolation)**.** *$\mathcal{E}$ derives the following, for all terms and every term template $F(v)$.*

(i) $Z(0) = 1$ *and* $Z(Sc) = 0$.

(ii) $\mathrm{sel}(c, a, a) = a$ *and* $\mathrm{sel}(c, Sa, Sb) = S\,\mathrm{sel}(c, a, b)$.

(iii) $Z(Z(Z(c))) = Z(c)$.

(iv) Equational interpolation*:* $F(Z(c)) = \mathrm{sel}\big(c, F(1), F(0)\big)$.

(v) Two-point rule*: if* $\Gamma \vdash F(0) = F'(0)$, $\Gamma \vdash F(1) = F'(1)$ *and* $\Gamma \vdash Z(Z(b)) = b$, *then* $\Gamma \vdash F(b) = F'(b)$.

Proof. (i) is (1.4). (ii)–(iv) are equational cases on a fresh variable in the place of c. For (ii): at 0 both sides of each equation are a, respectively Sa; at Sz they are a, respectively Sb. For (iii): $Z(Z(Z(0))) = Z(Z(1)) = Z(0)$, and $Z(Z(Z(Sz))) = Z(Z(0)) = Z(1) = 0 = Z(Sz)$. For (iv): at 0, $F(Z(0)) = F(1) = \mathrm{sel}(0, F(1), F(0))$; at Sz, $F(Z(Sz)) = F(0) = \mathrm{sel}(Sz, F(1), F(0))$. (v) Instantiate (iv) for F and for F' at $c = Z(b)$:

$$F(Z(Z(b))) = \mathrm{sel}\big(Z(b), F(1), F(0)\big) = \mathrm{sel}\big(Z(b), F'(1), F'(0)\big) = F'(Z(Z(b))),$$

the middle step by congruence from the two hypotheses. Congruence along $Z(Z(b)) = b$ on both ends gives $F(b) = F'(b)$. □

The two-point rule is the replacement for a case distinction on a Boolean value. It is an equality rule, not an elimination of a disjunction: the term b and the context Γ are arbitrary, and Γ may contain equations in which the variables of b occur.

Lemma 1.59 (Parity)**.** *$\mathcal{E}$ derives*

$$N(0) = 0, \qquad N(Sn) = Z(N(n)), \qquad N(1) = 1, \qquad Z(Z(N(n))) = N(n),$$
$$N(n) + Z(N(n)) = 1, \qquad N(SSn) = N(n),$$
$$N(2n) = 0, \qquad N(S(2n)) = 1, \qquad N(3n) = N(n).$$

Proof. The first two equations are computations and give $N(1) = Z(0) = 1$. $Z(Z(N(n))) = N(n)$ is an equation induction on n: the base is $Z(Z(0)) = Z(1) = 0 = N(0)$, and $Z(Z(N(Sn))) = Z(Z(Z(N(n)))) = Z(N(n)) = N(Sn)$ by Theorem 1.58(iii); the induction equation is not used. The equation $N(n) + Z(N(n)) = 1$ is the two-point rule for $F(v) := v + Z(v)$, $F'(v) := 1$ and $b := N(n)$: $F(0) = 0 + 1 = 1$ and $F(1) = 1 + 0 = 1$. Two computations give $N(SSn) = Z(Z(N(n))) = N(n)$. $N(2n) = 0$ is an induction: $N(2 \cdot 0) = N(0) = 0$, and $N(2(Sn)) = N(SS(2n)) = N(2n) = 0$. Then $N(S(2n)) = Z(N(2n)) = Z(0) = 1$. $N(3n) = N(n)$ is an induction: $N(3 \cdot 0) = N(0)$, and $N(3(Sn)) = N(SSS(3n)) = Z(N(SS(3n))) = Z(N(3n)) = Z(N(n)) = N(Sn)$. □

No statement that a number is even or odd is involved.

1.7.4 A three-cycle and a ramp

Put

$$j(u) := Z(u) + 2N(u), \qquad M(n) := I(n, 0;\, u.\, j(u)), \qquad G(n) := I(n, 0;\, u.\, S(u + M(u))).$$

Lemma 1.60 (Three-cycle)**.** $\mathcal{E}$ *derives* $j(0) = 1$, $j(1) = 2$, $j(2) = 0$, $M(0) = 0$ *and* $M(Sn) = j(M(n))$.

Proof. $j(0) = Z(0) + 2N(0) = 1 + (0 + 0) = 1$; $j(1) = Z(1) + 2N(1) = 0 + (1 + 1) = 2$; $j(2) = Z(S1) + 2N(SS0) = 0 + 2N(0) = 0$. The last two equations are I0 and IS. □

Only these three values of j are used; no theorem about remainders modulo three is needed.

Lemma 1.61 (Ramp)**.** $\mathcal{E}$ *derives*

(i) $M(G(n)) = N(n)$;

(ii) $G(0) = 0$, $G(Sn) = S(G(n) + N(n))$ *and* $G(SSn) = SSSG(n)$;

(iii) $G(2n) = 3n$;

(iv) $G(3n) = 3G(n) + N(n)$.

Proof. (i) Two identities are derived first, for variables g and b, in the context that consists of the one equation $Z(Z(b)) = b$:

$$M(S(g + b)) = \mathrm{sel}\big(b,\ j(M(g)),\ j(j(M(g)))\big), \tag{1.12}$$

$$\mathrm{sel}\big(b,\ j(b),\ j(j(b))\big) = Z(b). \tag{1.13}$$

Both are instances of the two-point rule. For (1.12) the templates are $F(v) := M(S(g + v))$ and $F'(v) := \mathrm{sel}(v, j(M(g)), j(j(M(g))))$: $F(0) = M(Sg) = j(M(g)) = F'(0)$ and $F(1) = M(SSg) = j(j(M(g))) = F'(1)$. For (1.13) the templates are $\mathrm{sel}(v, j(v), j(j(v)))$ and $Z(v)$: at 0 both sides are $j(0) = 1 = Z(0)$, and at 1 they are $j(j(1)) = j(2) = 0 = Z(1)$, by Theorem 1.60. By Theorem 1.59 the hypothesis is derivable for $b = N(n)$, so by substitution and cut both identities hold at $g = G(n)$ and $b = N(n)$ in every context.

Now EI on n. The base is $M(G(0)) = M(0) = 0 = N(0)$. In the step the induction equation is $M(G(n)) = N(n)$, and

$$\begin{aligned} M(G(Sn)) &= M\big(S(G(n) + M(G(n)))\big) = M\big(S(G(n) + N(n))\big) \\ &= \mathrm{sel}\big(N(n),\ j(M(G(n))),\ j(j(M(G(n))))\big) \\ &= \mathrm{sel}\big(N(n),\ j(N(n)),\ j(j(N(n)))\big) = Z(N(n)) = N(Sn). \end{aligned}$$

The second and fourth steps use the induction equation by congruence, the third is (1.12) and the fifth is (1.13). The value $N(n)$ is not split under the induction hypothesis.

(ii) The first equation is I0; the second is IS and (i). Applying it twice, $G(SSn) = S(S(G(n)+N(n))+Z(N(n))) = SS\big(G(n)+(N(n)+Z(N(n)))\big) = SS(G(n)+1) = SSSG(n)$, by Theorems 1.57 and 1.59.

(iii) EI: $G(2 \cdot 0) = G(0) = 0 = 3 \cdot 0$, and $G(2(Sn)) = G(SS(2n)) = SSSG(2n) = SSS(3n) = 3(Sn)$.

(iv) EI. The base is $G(3 \cdot 0) = 0 = 3G(0) + N(0)$. In the step, by (ii), the induction equation and $N(3n) = N(n)$,

$$G(3(Sn)) = G(SSS(3n)) = SSS\,S\big(G(3n) + N(3n)\big) = SSSS\big(3G(n) + N(n) + N(n)\big),$$

and, by (ii) and $N(n) + Z(N(n)) = 1$,

$$\begin{aligned} 3G(Sn) + N(Sn) &= 3\,S\big(G(n) + N(n)\big) + Z(N(n)) = SSS\big(3G(n) + 3N(n) + Z(N(n))\big) \\ &= SSS\big(3G(n) + N(n) + N(n) + 1\big). \end{aligned}$$

The two right-hand sides are equal by Theorem 1.57. □

1.7.5 The factor map and the comparator

Put

$$H(u) := \mathrm{sel}\big(N(u), G(u), u\big), \qquad 2^x := I(x, 1;\, u.\, u + u), \qquad L(x, y) := N\big(I(y, 2^x;\, u.\, H(u))\big).$$

The term H replaces a factor two by a factor three and fixes the numbers of parity one; 2^x is a term, not an operation of the language.

Lemma 1.62 (Factor map)**.** *$\mathcal{E}$ derives $H(2u) = 3u$, $H(1) = 1$ and $H(3u) = 3H(u)$.*

Proof. $H(2u) = \mathrm{sel}(N(2u), G(2u), 2u) = \mathrm{sel}(0, 3u, 2u) = 3u$ by Theorems 1.59 and 1.61. $H(1) = \mathrm{sel}(N(1), G(1), 1) = \mathrm{sel}(1, G(1), 1) = 1$. For the third equation the two-point rule gives, for variables a, c and b in the context $Z(Z(b)) = b$,

$$\mathrm{sel}(b,\ 3a + b,\ 3c) = 3\ \mathrm{sel}(b, a, c)\ :$$

at $b = 0$ both sides are $3a$, and at $b = 1$ both sides are $3c$. Instantiating at $b = N(u)$, $a = G(u)$ and $c = u$,

$$H(3u) = \mathrm{sel}\big(N(3u), G(3u), 3u\big) = \mathrm{sel}\big(N(u),\ 3G(u){+}N(u),\ 3u\big) = 3\ \mathrm{sel}\big(N(u), G(u), u\big) = 3H(u).$$

The identity is proved with an independent variable b and then instantiated; the dependence of $N(u)$ on u causes no difficulty. □

Lemma 1.63 (Orbits of the factor map)**.** *$\mathcal{E}$ derives $I(y, 3a;\, u.\, H(u)) = 3\, I(y, a;\, u.\, H(u))$ and $I(y, 1;\, u.\, H(u)) = 1$.*

Proof. Both are equation inductions on y with a fixed. The bases are ı0. The steps are $I(Sy, 3a;\, u.\, H(u)) = H(I(y, 3a;\, u.\, H(u))) = H(3\, I(y, a;\, u.\, H(u))) = 3H(I(y, a;\, u.\, H(u))) = 3\, I(Sy, a;\, u.\, H(u))$ and $I(Sy, 1;\, u.\, H(u)) = H(I(y, 1;\, u.\, H(u))) = H(1) = 1$, by Theorem 1.62. □

Lemma 1.64 (Comparator)**.** *$\mathcal{E}$ derives*

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y), \qquad L(Sn, n) = 0.$$

Proof. $2^0 = 1$ and $2^{Sx} = 2 \cdot 2^x$ are computations. Hence $L(0, y) = N(I(y, 1;\, u.\, H(u))) = N(1) = 1$ by Theorem 1.63, and $L(Sx, 0) = N(2 \cdot 2^x) = 0$ by ı0 and Theorem 1.59. For the third equation,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2 \cdot 2^x;\, u.\, H(u))\big) = N\big(I(y, H(2 \cdot 2^x);\, u.\, H(u))\big) \\ &= N\big(I(y, 3 \cdot 2^x;\, u.\, H(u))\big) = N\big(3\, I(y, 2^x;\, u.\, H(u))\big) = N\big(I(y, 2^x;\, u.\, H(u))\big) = L(x, y), \end{aligned}$$

by rotation (Theorem 1.56), $H(2u) = 3u$, Theorem 1.63 and $N(3n) = N(n)$. The fourth is an equation induction: $L(S0, 0) = 0$ by the second equation, and $L(SSn, Sn) = L(Sn, n)$ by the third. □

On the natural numbers $L(x, y)$ is the indicator of $x \le y$. That reading is not used: only the four equations are.

1.7.6 The saturating counter and the predecessor

Put

$$\beta(k,u) := \mathrm{sel}\big(L(SSu,k),\, u,\, Su\big), \qquad W(t,k) := I(t,0;\, u.\, \beta(k,u)), \qquad P(m) := W(m,m).$$

The counter $W(t,k)$ counts up to t and stops below k. Written out,

$$P(m) = I(m,0;\, u.\, \mathrm{sel}\big(L(SSu,m),\, u,\, Su\big)),$$

a term in the one variable m, which occurs both as the counter and in the body.

Theorem 1.65 (Predecessor)**.** *The equational fragment derives, in the empty context,*

$$\vdash_{\mathcal{E}} P(0) = 0, \qquad \vdash_{\mathcal{E}} P(Sx) = x.$$

Consequently $Sx = Sy \vdash_{\mathcal{E}} x = y$*, and the same sequents are derivable in* N *and in* T_{eq}*.*

Proof. By I0 and IS,

$$W(0,k) = 0, \qquad W(St,k) = \beta(k, W(t,k)). \tag{1.14}$$

By Theorems 1.58 and 1.64(ii),

$$\beta(Sk,Su) = \mathrm{sel}\big(L(SSSu,Sk),Su,SSu\big) = \mathrm{sel}\big(L(SSu,k),Su,SSu\big) = S\beta(k,u), \tag{1.15}$$

$$\beta(SSk,0) = \mathrm{sel}\big(L(SS0,SSk),0,1\big) = \mathrm{sel}\big(L(0,k),0,1\big) = \mathrm{sel}(1,0,1) = 1. \tag{1.16}$$

Conjugacy. $W(St,SSk) = SW(t,Sk)$, by EI on t with k fixed. The base is $W(S0,SSk) = \beta(SSk, W(0,SSk)) = \beta(SSk,0) = 1 = SW(0,Sk)$, by (1.14) and (1.16). The step is

$$\begin{aligned} W(SSt,SSk) &= \beta\big(SSk,\ W(St,SSk)\big) = \beta\big(SSk,\ SW(t,Sk)\big) \\ &= S\beta\big(Sk,\ W(t,Sk)\big) = SW(St,Sk), \end{aligned}$$

where the second step uses the induction equation and the third is (1.15).

Below its cap the counter does not saturate. $W(n,Sn) = n$, by EI on n. The base is $W(0,S0) = 0$. In the step, $W(Sn,SSn) = SW(n,Sn) = Sn$, by the conjugacy instantiated at $t = k = n$ and the induction equation. The instantiation is made in an identity that is already derived with independent variables; no parameter of an active induction hypothesis is changed.

The two equations. $P(0) = W(0,0) = 0$. Substituting Sx for both variables in (1.14),

$$\begin{aligned} P(Sx) = W(Sx,Sx) &= \beta\big(Sx,\ W(x,Sx)\big) = \beta(Sx,x) \\ &= \mathrm{sel}\big(L(SSx,Sx),\, x,\, Sx\big) = \mathrm{sel}\big(L(Sx,x),\, x,\, Sx\big) = \mathrm{sel}(0,x,Sx) = x, \end{aligned}$$

by Theorem 1.64. Finally, from $Sx = Sy$ congruence gives $P(Sx) = P(Sy)$, hence $x = y$. The transfer to N and T_{eq} is Theorem 1.54(iv). □

Remark 1.66 (Sources and the use of coverage in the inherited proof)*.* The construction (parity, three-cycle, ramp, factor map, comparator, saturating counter) is the scalar predecessor of the reference statement filed with `certified_zero`. That proof works in the theory with numerical cases. It uses them to show that $Z(n)$ and $N(n)$ take the values 0 and 1 only, and then splits the value $N(n)$ under the induction hypothesis in the ramp invariant Theorem 1.61(i), again in Theorem 1.61(iv), and in $H(3u) = 3H(u)$; it also proves $\mathrm{sel}(c,Sa,Sb) = S\,\mathrm{sel}(c,a,b)$ by cases, which is an equational case. The splits are replaced here by the equation $Z(Z(N(n))) = N(n)$ and the two-point rule, applied to identities that are derived in advance with independent variables, which is the contribution of

`uniform_raw`, `newest_parameter`, `parameter_witness` and `no_witness`. The four notes differ in presentation. `uniform_raw` states the interpolation as $F(Z(c)) = \mathrm{sel}(c, F(1), F(0))$, as above; `no_witness` states it as $F(Z(Z(y))) = \mathrm{sel}(y, F(0), F(1))$; `parameter_witness` uses a finite-value transfer lemma for $Z(t)$ and $N(t)$ proved by equational cases; `newest_parameter` instantiates two identities in $Z(v)$ at $v = Z(N(n))$ and replaces the split in $H(3u)$ by $\mathrm{sel}(c, a+b, a'+b') = \mathrm{sel}(c, a, a') + \mathrm{sel}(c, b, b')$ and $\mathrm{sel}(c, c, 0) = 0$. The reference defines the counter with the body $\mathrm{sel}(L(Su, m), u, Su)$ and then identifies the body of $P(Sm)$ with it by a separate induction; the form with $L(SSu, k)$ and the conjugacy between the caps SSk and Sk, taken from `no_witness`, avoids that induction. The stronger statement matters where a calculus is closed under substitution only on its equational part, or restricts the witnesses of DI: the fragment $\mathcal{E}$ has no rule for D, so the predecessor is available before any naming of terms.

Both forms are formalized in Lean. The fragment $\mathcal{E}$ is the calculus `EqPrf` of the modules `EqKernel`, `EqArith`, `EqParity`, `EqCompare` and `EqPred`; it has exactly the five rules of Theorem 1.53: hypothesis, the two computations, transport in one term template, and induction for a literal equation with parameters. Theorem 1.65 is `EqPrf.pred_zero`, `EqPrf.pred_succ` (in the form `EqPrf Γ (Tm.pred (succ t)) t`, for every term and every context) and `EqPrf.succ_inj`. The calculus embeds in the kernel under any restriction of the witness of DI (`EqPrf.toDW`), because its derivations contain no introduction of D. The older modules `TeqParity`, `TeqCompare` and `TeqPred` prove the two equations in T_{eq} and split the Boolean value by coverage.

1.7.7 A smaller predecessor

This subsection draws on `479_predecessor57`, which shortens the three-cycle and the ramp of Theorem 1.65 and reverses the comparison in the saturating counter, on `absorbing_orbit`, which replaces the factor map by one that sends the odd numbers to 0 and tests the orbit itself, on `absorbing_predecessor`, which has the same absorbing factor map and replaces its parity control by a shorter one, on `pred50`, which tests the orbit with a different orientation, and on `fused_iteration`, which shortens the three-cycle and fuses the parity control with the selection of the factor map, and on `orbit_equality`, whose diagonal counter lemma shortens the derivation of the 45-node predecessor. The size of the resulting term is measured as follows.

Definition 1.67 (Size of a term)**.** $|x| := |0| := 1$, $|St| := 1 + |t|$ and $|I(n, a;\, u.\, h)| := 1 + |n| + |a| + |h|$. Thus one node is counted for every occurrence of a variable, free or bound, of 0, of S and of the iterator; the name of the bound variable is not counted. The size of an abbreviation is the size of its full expansion, with every repeated subterm counted at each occurrence.

Hence $|\mathrm{sel}(c, a, b)| = 1 + |c| + |a| + |b|$, $|Z(c)| = |c| + 4$, $|N(c)| = |c| + 7$, $|a + b| = |a| + |b| + 3$ and $|2^x| = |x| + 8$. Two three-cycles and ramps occur. The *ceiling* pair, of `479_predecessor57`, is

$$J(u) := I(u, 2;\, w.\, Z(w)), \qquad M_-(n) := I(n, 0;\, u.\, J(u)), \qquad G_-(n) := I(n, 0;\, u.\, S\,\mathrm{sel}(M_-(u),\, Su,\, u)),$$

and the *floor* pair, of `fused_iteration`, is

$$J'(v) := I(Sv, v;\, w.\, \mathrm{sel}(w, Sv, 0)), \qquad M'(n) := I(n, 0;\, v.\, J'(v)), \qquad G'(n) := I(n, 0;\, u.\, S\,\mathrm{sel}(M'(u),\, u,\, Su)).$$

Put further

$$\tilde{N}(k, a) := I(k, 0;\, v.\, \mathrm{sel}(v, a, 0)), \qquad \tilde{N}(n) := \tilde{N}(n, n), \qquad H_-(n) := I(n, n;\, v.\, \mathrm{sel}(v, G'(n), 0)),$$

$$H_{51}(u) := \mathrm{sel}(N(u), G_-(u), 0), \qquad H_{50}(u) := \mathrm{sel}(\tilde{N}(u), G_-(u), 0),$$

and, for H one of H_-, H_{51}, H_{50},

$$O_H(y,a) := I(y,a;\, u.\, H(u)), \qquad \Lambda_H(x,y) := Z\big(O_H(Sy, 2^x)\big),$$

$$\beta_H(k,u) := \mathrm{sel}\big(O_H(SSu, 2^k),\, u,\, Su\big), \qquad W_H(t,k) := I(t,0;\, u.\, \beta_H(k,u)), \qquad P_H(m) := W_H(m,m).$$

The main term is $P_- := P_{H_-}$, and $P_{51} := P_{H_{51}}$, $P_{50} := P_{H_{50}}$; we write O, Λ, β_-, W_- for O_{H_-}, Λ_{H_-}, β_{H_-}, W_{H_-}. Written out,

$$P_-(m) = I(m,0;\, u.\, \mathrm{sel}\big(I(SSu, 2^m;\, v.\, H_-(v)),\, u,\, Su\big)),$$

a term whose only free variable is m.

Against Theorem 1.65: the ceiling three-cycle M_- iterates the zero test from the seed 2, and the floor three-cycle M' iterates, from the seed v and Sv times, the update $w \mapsto \mathrm{sel}(w, Sv, 0)$, instead of evaluating $Z(u)+2N(u)$; the ramps add 1 or 2 according to whether the state of the three-cycle is zero, instead of adding one more than the state; the factor maps select 0 instead of their argument; H_{50} uses the *weighted parity* $\tilde{N}$, which is not normalized to 0 and 1, and H_- *fuses* the parity test with the selection: one iteration seeded by its argument alternates between 0 and $G'(n)$; and the counter tests the orbit $O_H(SSu, 2^k)$ itself, not its parity, with the two outputs of the selector in the order of Theorem 1.65. On the natural numbers J' and M map $0 \mapsto 1 \mapsto 2 \mapsto 0$, M_- runs through $0, 2, 1, 0, \ldots$, $G'(n) = G(n) = \lfloor 3n/2 \rfloor$ and $G_-(n) = \lceil 3n/2 \rceil$; $\tilde{N}(n)$ is 0 at even n and n at odd n; each of H_-, H_{51}, H_{50} sends $2u$ to $3u$ and every odd number to 0; so the orbit $O_H(y, 2^x)$ is $3^y 2^{x-y}$ for $y \le x$ and 0 for $y > x$, and $\Lambda_H(x,y)$ is the indicator of $x \le y$. These readings are not used.

Lemma 1.68 (Ceiling three-cycle and ramp)**.** $\mathcal{E}$ *derives*

(i) $J(0) = 2$, $J(1) = 0$, $J(2) = 1$, $M_-(0) = 0$ *and* $M_-(Sn) = J(M_-(n))$;

(ii) $M_-(G_-(n)) = N(n)$;

(iii) $G_-(0) = 0$, $G_-(Sn) = S\big(G_-(n) + Z(N(n))\big)$ *and* $G_-(SSn) = SSSG_-(n)$;

(iv) $G_-(2n) = 3n$;

(v) $G_-(3n) + N(n) = 3G_-(n)$.

Proof. (i) $J(0) = 2$ is I0. By IS and Theorem 1.58(i), $J(1) = Z(J(0)) = Z(S1) = 0$ and $J(2) = Z(J(1)) = Z(0) = 1$. The last two equations are I0 and IS.

(ii) As in the proof of Theorem 1.61(i), two identities are derived first, for variables g and b, in the context that consists of the one equation $Z(Z(b)) = b$:

$$M_-\big(S\,\mathrm{sel}(b, Sg, g)\big) = \mathrm{sel}\big(b,\ J(J(M_-(g))),\ J(M_-(g))\big), \tag{1.17}$$

$$\mathrm{sel}\big(b,\ J(J(b)),\ J(b)\big) = Z(b). \tag{1.18}$$

Both are instances of the two-point rule. For (1.17) the templates are $F(v) := M_-(S\,\mathrm{sel}(v, Sg, g))$ and $F'(v) := \mathrm{sel}(v, J(J(M_-(g))), J(M_-(g)))$: $F(0) = M_-(SSg) = J(M_-(Sg)) = J(J(M_-(g))) = F'(0)$ and $F(1) = M_-(Sg) = J(M_-(g)) = F'(1)$, by $\mathrm{sel}(0,a,b) = a$, $\mathrm{sel}(1,a,b) = b$ and (i). For (1.18) the templates are $\mathrm{sel}(v, J(J(v)), J(v))$ and $Z(v)$: at 0 both sides are $J(J(0)) = J(2) = 1 = Z(0)$, and at 1 they are $J(1) = 0 = Z(1)$. By Theorem 1.59 the hypothesis is derivable for $b = N(n)$, so both identities hold at $g = G_-(n)$ and $b = N(n)$ in every context. Now EI on n. The base is $M_-(G_-(0)) = M_-(0) = 0 = N(0)$. In the step the induction equation is $M_-(G_-(n)) = N(n)$, and

$$\begin{aligned} M_-(G_-(Sn)) &= M_-\big(S\,\mathrm{sel}(M_-(G_-(n)), SG_-(n), G_-(n))\big) = M_-\big(S\,\mathrm{sel}(N(n), SG_-(n), G_-(n))\big) \\ &= \mathrm{sel}\big(N(n),\ J(J(M_-(G_-(n)))),\ J(M_-(G_-(n)))\big) \\ &= \mathrm{sel}\big(N(n),\ J(J(N(n))),\ J(N(n))\big) = Z(N(n)) = N(Sn). \end{aligned}$$

The second and fourth steps use the induction equation by congruence, the third is (1.17) and the fifth is (1.18).

(iii) The first equation is I0. The two-point rule gives, for variables g and b in the context $Z(Z(b)) = b$, $\mathrm{sel}(b, Sg, g) = g + Z(b)$: at 0 both sides are $Sg = g + 1$, and at 1 both sides are $g = g + 0$, by Theorem 1.57. Instantiating at $g = G_-(n)$ and $b = N(n)$, IS and (ii) give

$$G_-(Sn) = S\,\mathrm{sel}\big(M_-(G_-(n)), SG_-(n), G_-(n)\big) = S\,\mathrm{sel}\big(N(n), SG_-(n), G_-(n)\big) = S\big(G_-(n)+Z(N(n))\big).$$

Applying it twice, with $N(Sn) = Z(N(n))$, $Z(Z(N(n))) = N(n)$ and $N(n) + Z(N(n)) = 1$ (Theorem 1.59), $G_-(SSn) = S\big(S(G_-(n) + Z(N(n))) + N(n)\big) = SS\big(G_-(n) + (N(n) + Z(N(n)))\big) = SS(G_-(n) + 1) = SSSG_-(n)$.

(iv) EI: $G_-(2 \cdot 0) = G_-(0) = 0 = 3 \cdot 0$, and $G_-(2(Sn)) = G_-(SS(2n)) = SSSG_-(2n) = SSS(3n) = 3(Sn)$.

(v) EI. The base is $G_-(3{\cdot}0)+N(0) = G_-(0)+0 = 0 = 3G_-(0)$. In the step write $b := N(n)$ and $z := Z(N(n))$. By (iii) at $SS(3n)$ and at $3n$, and by $N(SS(3n)) = N(3n) = N(n)$ and $N(Sn) = z$ (Theorem 1.59),

$$G_-(3(Sn))+N(Sn) = G_-\big(SSS(3n)\big)+z = S\big(SSSG_-(3n)+z\big)+z = SSSS\big(G_-(3n)+z+z\big).$$

By (iii), Theorem 1.57, the induction equation $3G_-(n) = G_-(3n) + b$ and $b + z = 1$,

$$3G_-(Sn) = 3\,S\big(G_-(n)+z\big) = SSS\big(3G_-(n)+3z\big) = SSS\big(G_-(3n)+(b+z)+z+z\big) = SSSS\big(G_-(3n)+z+z\big).$$

No cancellation is used. □

Lemma 1.69 (Floor three-cycle and ramp)**.** $\mathcal{E}$ *derives*

(i) $J'(0) = 1$, $J'(1) = 2$, $J'(2) = 0$, $M'(0) = 0$ *and* $M'(Sn) = J'(M'(n))$;

(ii) $M'(G'(n)) = N(n)$;

(iii) $G'(0) = 0$, $G'(Sn) = S\big(G'(n) + N(n)\big)$ *and* $G'(SSn) = SSSG'(n)$;

(iv) $G'(2n) = 3n$;

(v) $G'(3n) = 3G'(n) + N(n)$.

Proof. (i) Write $k_c(w) := \mathrm{sel}(w, c, 0)$. By IS, I0 and the selector equations, $J'(0) = k_1(0) = 1$; $J'(1) = k_2(k_2(1)) = k_2(0) = 2$; and $J'(2) = k_3(k_3(k_3(2))) = k_3(k_3(0)) = k_3(3) = 0$. The last two equations are I0 and IS.

(ii) As in the proof of Theorem 1.68(ii), the two-point rule gives, for variables g and b in the context $Z(Z(b)) = b$,

$$M'\big(S\,\mathrm{sel}(b, g, Sg)\big) = \mathrm{sel}\big(b,\ J'(M'(g)),\ J'(J'(M'(g)))\big), \qquad \mathrm{sel}\big(b,\ J'(b),\ J'(J'(b))\big) = Z(b) :$$

at $b = 0$ the first is $M'(Sg) = J'(M'(g))$ and at $b = 1$ it is $M'(SSg) = J'(J'(M'(g)))$; the second is $J'(0) = 1 = Z(0)$ at 0 and $J'(J'(1)) = J'(2) = 0 = Z(1)$ at 1. EI on n: the base is $M'(G'(0)) = M'(0) = 0 = N(0)$, and in the step, by the induction equation and the two identities at $g = G'(n)$, $b = N(n)$,

$$M'(G'(Sn)) = M'\big(S\,\mathrm{sel}(N(n), G'(n), SG'(n))\big) = \mathrm{sel}\big(N(n), J'(N(n)), J'(J'(N(n)))\big) = Z(N(n)) = N(Sn).$$

(iii) The first equation is I0. The two-point rule gives $\mathrm{sel}(b, g, Sg) = g + b$ in the context $Z(Z(b)) = b$: at 0 both sides are g, and at 1 both are $Sg = g + 1$. With (ii), $G'(Sn) = S\,\mathrm{sel}(N(n), G'(n), SG'(n)) = S(G'(n) + N(n))$. These are the first two equations of Theorem 1.61(ii) for G' in place of G. The proofs of the third equation of Theorem 1.61(ii) and of Theorem 1.61(iii) and (iv) use of G only those two equations, Theorem 1.59 and Theorem 1.57; with G' in place of G they give the rest of (iii), and (iv) and (v). □

Lemma 1.70 (Weighted parity)**.** $\mathcal{E}$ *derives, for all terms,*

(i) $\mathrm{sel}(c,c,0) = 0$ *and* $\mathrm{sel}(Z(c),a,b) = \mathrm{sel}(c,b,a)$*;*

(ii) $\tilde{N}(k,a) = \mathrm{sel}\big(N(k),0,a\big)$*;*

(iii) $\mathrm{sel}\big(\tilde{N}(n),a,b\big) = \mathrm{sel}\big(N(n),a,b\big)$*.*

Proof. (i) Equational cases on c: $\mathrm{sel}(0,0,0) = 0$ and $\mathrm{sel}(Sz,Sz,0) = 0$; $\mathrm{sel}(Z(0),a,b) = \mathrm{sel}(1,a,b) = b = \mathrm{sel}(0,b,a)$ and $\mathrm{sel}(Z(Sz),a,b) = \mathrm{sel}(0,a,b) = a = \mathrm{sel}(Sz,b,a)$.

(ii) EI on k with a fixed. The base is $\tilde{N}(0,a) = 0 = \mathrm{sel}(0,0,a) = \mathrm{sel}(N(0),0,a)$. The two-point rule gives, for variables a and b in the context $Z(Z(b)) = b$, $\mathrm{sel}(\mathrm{sel}(b,0,a),a,0) = \mathrm{sel}(Z(b),0,a)$: at $b = 0$ both sides are a, since $\mathrm{sel}(0,a,0) = a = \mathrm{sel}(1,0,a)$, and at $b = 1$ both sides are 0, since $\mathrm{sel}(\mathrm{sel}(1,0,a),a,0) = \mathrm{sel}(a,a,0) = 0$ by (i) and $\mathrm{sel}(Z(1),0,a) = \mathrm{sel}(0,0,a) = 0$. Instantiated at $b = N(k)$, with Theorem 1.59, it gives the step: $\tilde{N}(Sk,a) = \mathrm{sel}(\tilde{N}(k,a),a,0) = \mathrm{sel}(\mathrm{sel}(N(k),0,a),a,0) = \mathrm{sel}(Z(N(k)),0,a) = \mathrm{sel}(N(Sk),0,a)$, the second step by the induction equation.

(iii) Equational cases on n. At 0 both controls are 0: $\tilde{N}(0) = \tilde{N}(0,0) = 0 = N(0)$. At Sz, (ii) at $k = a = Sz$ gives $\tilde{N}(Sz) = \mathrm{sel}(N(Sz),0,Sz)$, and the two-point rule gives, for a variable q in the context $Z(Z(q)) = q$, $\mathrm{sel}(\mathrm{sel}(q,0,Sz),a,b) = \mathrm{sel}(q,a,b)$: at $q = 0$ both sides are $\mathrm{sel}(0,a,b) = a$, and at $q = 1$ the left side is $\mathrm{sel}(Sz,a,b) = b$ and the right side is $\mathrm{sel}(1,a,b) = b$. Instantiating at $q = N(Sz)$, $\mathrm{sel}(\tilde{N}(Sz),a,b) = \mathrm{sel}(N(Sz),a,b)$. □

The equation $\tilde{N}(n) = N(n)$ is not claimed and is false on the natural numbers ($\tilde{N}(3) = 3$, $N(3) = 1$); (iii) says that the two terms are interchangeable as the control of a selector. The size $|\tilde{N}(t)| = 2|t| + 5$ is smaller than $|N(t)| = |t| + 7$ only when $|t| = 1$, as in H_{50}.

Lemma 1.71 (Absorbing factor maps and comparators)**.** $\mathcal{E}$ *derives*

(i) $H_-(n) = \mathrm{sel}\big(N(n),G'(n),0\big)$ *and* $H_{50}(u) = H_{51}(u)$*;*

and, for each H *among* H_-*,* H_{51}*,* H_{50}*,*

(ii) $H(0) = 0$*,* $H(1) = 0$*,* $H(2u) = 3u$ *and* $H(3u) = 3H(u)$*;*

(iii) $O_H(Sy,a) = O_H(y,H(a))$*,* $O_H(y,0) = 0$*,* $O_H(y,3a) = 3\,O_H(y,a)$ *and* $Z(2^x) = 0$*;*

(iv) $\Lambda_H(0,y) = 1$*,* $\Lambda_H(Sx,0) = 0$*,* $\Lambda_H(Sx,Sy) = \Lambda_H(x,y)$ *and* $\Lambda_H(n,n) = 1$*.*

Proof. (i) *Fusion.* Equational cases on n. At 0, $H_-(0) = 0 = G'(0) = \mathrm{sel}(N(0),G'(0),0)$. At Sz, write $h(v) := \mathrm{sel}(v,G'(Sz),0)$, so that $h(Sz) = 0$. By rotation (Theorem 1.56(i)), Theorem 1.70(ii) at $k = z$, $a = G'(Sz)$, and Theorem 1.70(i),

$$\begin{aligned} H_-(Sz) &= I(Sz,Sz;\, v.\,h(v)) = I(z,h(Sz);\, v.\,h(v)) = I(z,0;\, v.\,h(v)) = \tilde{N}\big(z,G'(Sz)\big) \\ &= \mathrm{sel}\big(N(z),0,G'(Sz)\big) = \mathrm{sel}\big(Z(N(z)),G'(Sz),0\big) = \mathrm{sel}\big(N(Sz),G'(Sz),0\big). \end{aligned}$$

The second equation is Theorem 1.70(iii) at $a = G_-(u)$, $b = 0$.

(ii) By (i) it suffices to treat $\mathrm{sel}(N(u),G(u),0)$ for G either G' or G_-. Then $H(0) = \mathrm{sel}(0,G(0),0) = G(0) = 0$, $H(1) = \mathrm{sel}(1,G(1),0) = 0$ and $H(2u) = \mathrm{sel}(N(2u),G(2u),0) = \mathrm{sel}(0,3u,0) = 3u$, by Theorem 1.59 and Theorem 1.69(iv), respectively Theorem 1.68(iv). For H_-, the identity $\mathrm{sel}(b,3a+b,3c) = 3\,\mathrm{sel}(b,a,c)$ of the proof of Theorem 1.62, at $b = N(u)$, $a = G'(u)$, $c = 0$, with $3 \cdot 0 = 0$, Theorem 1.69(v) and $N(3u) = N(u)$, gives

$$H_-(3u) = \mathrm{sel}\big(N(3u),G'(3u),0\big) = \mathrm{sel}\big(N(u),3G'(u)+N(u),3{\cdot}0\big) = 3\,\mathrm{sel}\big(N(u),G'(u),0\big) = 3H_-(u).$$

For H_{51} the two-point rule gives, for variables a, c, d, e and b in the context $Z(Z(b)) = b$,

$$3\ \mathrm{sel}(b, a, c) = \mathrm{sel}(b, 3a, 3c), \qquad \mathrm{sel}(b, d + b, e) = \mathrm{sel}(b, d, e)\ :$$

at $b = 0$ the sides are $3a$, respectively $d + 0 = d$, and at $b = 1$ they are $3c$, respectively e. Instantiating at $b = N(u)$, $a = G_-(u)$, $c = 0$, $d = G_-(3u)$ and $e = 0$, and using $3 \cdot 0 = 0$, Theorem 1.68(v) and $N(3u) = N(u)$,

$$\begin{aligned} 3H_{51}(u) &= 3\ \mathrm{sel}\big(N(u), G_-(u), 0\big) = \mathrm{sel}\big(N(u), 3G_-(u), 0\big) = \mathrm{sel}\big(N(u), G_-(3u) + N(u), 0\big) \\ &= \mathrm{sel}\big(N(u), G_-(3u), 0\big) = \mathrm{sel}\big(N(3u), G_-(3u), 0\big) = H_{51}(3u). \end{aligned}$$

The equations for H_{50} follow from those for H_{51} by (i).

(iii) The first equation is rotation (Theorem 1.56(i)). The second and third are equation inductions on y with a fixed: the bases are I0, and the steps are $O_H(Sy, 0) = H(O_H(y, 0)) = H(0) = 0$ and $O_H(Sy, 3a) = H(O_H(y, 3a)) = H(3\,O_H(y, a)) = 3H(O_H(y, a)) = 3\,O_H(Sy, a)$. Equational cases on a give $Z(2a) = Z(a)$: $Z(2 \cdot 0) = Z(0)$, and $Z(2(Sz)) = Z(SS(2z)) = 0 = Z(Sz)$; likewise $Z(3a) = Z(a)$, with $3(Sz) = SSS(3z)$ (Theorem 1.57). Then $Z(2^x) = 0$ is an equation induction: $Z(2^0) = Z(1) = 0$, and $Z(2^{Sx}) = Z(2 \cdot 2^x) = Z(2^x) = 0$, since $2^{Sx} = 2 \cdot 2^x$ is IS.

(iv) Note $O_H(S0, a) = H(a)$ by IS and I0. Now

$$\begin{aligned} \Lambda_H(0, y) &= Z\big(O_H(Sy, 1)\big) = Z\big(O_H(y, H(1))\big) = Z\big(O_H(y, 0)\big) = Z(0) = 1, \\ \Lambda_H(Sx, 0) &= Z\big(H(2 \cdot 2^x)\big) = Z(3 \cdot 2^x) = Z(2^x) = 0, \\ \Lambda_H(Sx, Sy) &= Z\big(O_H(SSy, 2 \cdot 2^x)\big) = Z\big(O_H(Sy, H(2 \cdot 2^x))\big) = Z\big(O_H(Sy, 3 \cdot 2^x)\big) \\ &= Z\big(3\,O_H(Sy, 2^x)\big) = Z\big(O_H(Sy, 2^x)\big) = \Lambda_H(x, y). \end{aligned}$$

The last equation is an equation induction: $\Lambda_H(0, 0) = 1$ by the first, and $\Lambda_H(Sn, Sn) = \Lambda_H(n, n)$ by the third. □

Proposition 1.72 (A predecessor of 46 nodes)**.** (i) *$\vdash_{\mathcal{E}} P_-(0) = 0$ and $\vdash_{\mathcal{E}} P_-(Sx) = x$; hence the same sequents are derivable in N and in T_{eq}. The same holds for P_{51} and P_{50}.*

(ii) *For a variable m, $|P_-(m)| = 46$, $|P_{50}(m)| = 50$ and $|P_{51}(m)| = 51$, while the predecessor $P(m)$ of Theorem 1.65 has $|P(m)| = 75$.*

Proof. (i) Let H be one of H_-, H_{51}, H_{50}, and write O, Λ, β_-, W_-, P_- for O_H, Λ_H, β_H, W_H, P_H; only Theorem 1.71(iv) is used. By I0 and IS, $W_-(0, k) = 0$ and $W_-(St, k) = \beta_-(k, W_-(t, k))$. Since $\Lambda(k, Su) = Z(O(SSu, 2^k))$, Theorem 1.70(i) gives

$$\beta_-(k, u) = \mathrm{sel}\big(Z(O(SSu, 2^k)), Su, u\big) = \mathrm{sel}\big(\Lambda(k, Su), Su, u\big).$$

By Theorem 1.71 and Theorem 1.58(ii),

$$\begin{aligned} \beta_-(Sk, Su) &= \mathrm{sel}\big(\Lambda(Sk, SSu), SSu, Su\big) = \mathrm{sel}\big(\Lambda(k, Su), SSu, Su\big) = S\beta_-(k, u), \\ \beta_-(SSk, 0) &= \mathrm{sel}\big(\Lambda(SSk, S0), 1, 0\big) = \mathrm{sel}\big(\Lambda(Sk, 0), 1, 0\big) = \mathrm{sel}(0, 1, 0) = 1. \end{aligned}$$

Conjugacy. $W_-(St, SSk) = SW_-(t, Sk)$, by EI on t with k fixed. The base is $W_-(S0, SSk) = \beta_-(SSk, 0) = 1 = SW_-(0, Sk)$. The step is

$$W_-(SSt, SSk) = \beta_-\big(SSk,\ SW_-(t, Sk)\big) = S\beta_-\big(Sk,\ W_-(t, Sk)\big) = SW_-(St, Sk),$$

by the induction equation and the first displayed identity.

Below its cap the counter does not saturate. $W_-(n, Sn) = n$, by EI on n: $W_-(0, S0) = 0$, and $W_-(Sn, SSn) = SW_-(n, Sn) = Sn$ by the conjugacy, derived with independent variables and instantiated at $t = k = n$, and the induction equation.

The two equations. $P_-(0) = W_-(0,0) = 0$, and

$$P_-(Sx) = W_-(Sx,Sx) = \beta_-\big(Sx, W_-(x,Sx)\big) = \beta_-(Sx,x) = \mathrm{sel}\big(\Lambda(Sx,Sx), Sx, x\big)$$
$$= \mathrm{sel}\big(\Lambda(x,x), Sx, x\big) = \mathrm{sel}(1, Sx, x) = x,$$

by Theorem 1.71. The transfer is Theorem 1.54(iv).

(ii) By Theorem 1.67, with variables as arguments, $|J'(v)| = 1+2+1+(1+1+2+1) = 9$, $|M'(n)| = 12$, $|G'(n)| = 20$ (its body has 17 nodes), $|H_-(n)| = 1+1+1+(1+1+20+1) = 26$, $|O(y,a)| = |y| + |a| + 27$, $|\Lambda(x,y)| = 4 + |O(Sy, 2^x)| = |x| + |y| + 40 = 42$, $|\beta_-(k,u)| = 1 + |O(SSu, 2^k)| + 1 + 2 = 1 + 39 + 3 = 43$ and $|P_-(m)| = 1 + 1 + 1 + 43 = 46$. For the ceiling pair, $|J(u)| = 10$, $|M_-(n)| = 13$ and $|G_-(n)| = 21$; with $|\tilde{N}(u)| = 7$, $|H_{50}(u)| = 1+7+21+1 = 30$ and $|H_{51}(u)| = 1+8+21+1 = 31$, so that $|O_H(y,a)| = |y|+|a|+|H(u)|+1$ and $|P_H(m)| = 3 + 1 + (3 + 9 + |H(u)| + 1) + 3 = |H(u)| + 20$, which is 50 and 51. For the terms of Theorem 1.65, $|j(u)| = 5 + 19 + 3 = 27$, $|M(n)| = 30$, $|G(n)| = 38$, $|H(u)| = 48$, $|L(x,y)| = |x| + |y| + 64 = 66$, $|\beta(k,u)| = 1 + |L(SSu,k)| + 1 + 2 = 72$ and $|P(m)| = 75$. □

The other terms of the sources are

$$H_{57}(u) := \mathrm{sel}\big(N(u), G_-(u), u\big), \qquad L_{57}(x,y) := N\big(I(y, 2^x;\ u.\, H_{57}(u))\big),$$
$$P_{57}(m) := I(m, 0;\ u.\ \mathrm{sel}\big(L_{57}(m, Su),\ Su,\ u\big)).$$

The orientation of `pred50` runs the orbit the other way. For H one of H_-, H_{51} put

$$K_H(x,y) := Z\big(O_H(y, 2^x)\big), \qquad \beta'_H(k,u) := \mathrm{sel}\big(O_H(k, 2^{Su}),\ Su,\ u\big),$$
$$W'_H(t,k) := I(t, 0;\ u.\, \beta'_H(k,u)), \qquad P'_H(m) := W'_H(m,m),$$

and $P_{45} := P'_{H_-}$, $P'_{50} := P'_{H_{51}}$. Written out,

$$P_{45}(m) = I(m, 0;\ u.\ \mathrm{sel}\big(I(m, 2^{Su};\ v.\, H_-(v)),\ Su,\ u\big)).$$

On the natural numbers $K_H(x,y)$ is the indicator of $x < y$. The term P'_{50} is the one of `pred50`; the term P_{45} combines its orientation with the fused factor map of `fused_iteration`. It was first assembled in this article; `abstraction_and_refinements` and `computation_leaves` later deliver the same term independently, with the same reduction to the factor interface $F(0) = F(1) = 0$, $F(2a) = 3a$, $F(3a) = 3F(a)$ and the reversed comparator. The derivation below reuses the arguments of `pred50` and `fused_iteration` for the factor map and the comparator, and concludes with the following lemma of `orbit_equality` on counters of the form $I(t, 0;\ u.\, f(k,u))$.

Lemma 1.73 (Diagonal counter)**.** *Let $f(k,u)$ be a term template in the variables k and u, whose other free variables are distinct from t and x, and put*

$$W_f(t,k) := I(t, 0;\ u.\, f(k,u)), \qquad P_f(m) := W_f(m,m).$$

If $\mathcal{E}$ derives

$$f(1,0) = 0, \qquad f(SSk, 0) = 1, \qquad f(Sk, Su) = Sf(k,u),$$

then $\vdash_{\mathcal{E}} P_f(0) = 0$ and $\vdash_{\mathcal{E}} P_f(Sx) = x$.

Proof. By I0 and IS, $W_f(0,k) = 0$ and $W_f(St,k) = f(k, W_f(t,k))$. By Theorem 1.54(i) the three hypotheses may be instantiated at arbitrary terms.

Conjugacy. $W_f(St, SSk) = SW_f(t, Sk)$, by EI on t with k fixed. The base is $W_f(S0, SSk) = f(SSk, W_f(0, SSk)) = f(SSk, 0) = 1 = SW_f(0, Sk)$. In the step the induction equation is $W_f(St, SSk) = SW_f(t, Sk)$, and

$$W_f(SSt, SSk) = f\big(SSk, W_f(St, SSk)\big) = f\big(SSk, SW_f(t, Sk)\big) = Sf\big(Sk, W_f(t, Sk)\big) = SW_f(St, Sk),$$

by IS, the induction equation by congruence, the third hypothesis at $k := Sk$ and $u := W_f(t, Sk)$, and IS.

The diagonal. The conjugacy is derived in the empty context with the independent variables t and k; its instance at $t := Sx$ and $k := x$ is $W_f(SSx, SSx) = SW_f(Sx, Sx)$, that is

$$P_f(SSx) = SP_f(Sx).$$

Also $P_f(1) = W_f(S0, S0) = f(1, W_f(0, 1)) = f(1, 0) = 0$. Now $P_f(Sx) = x$ by EI on x: the base is $P_f(1) = 0$, and in the step $P_f(SSx) = SP_f(Sx) = Sx$, by the displayed instance and the induction equation by congruence. Finally $P_f(0) = W_f(0, 0) = 0$ is I0. The first induction is complete before it is instantiated, so no parameter of an active induction hypothesis is changed. □

Proposition 1.74 (A predecessor of 45 nodes)**.** (i) *For H one of H_-, H_{51}, $\vdash_{\mathcal{E}} P'_H(0) = 0$ and $\vdash_{\mathcal{E}} P'_H(Sx) = x$; hence the same sequents are derivable in* N *and in* T_{eq}.

(ii) *For a variable m, $|P_{45}(m)| = 45$ and $|P'_{50}(m)| = 50$.*

Proof. (i) Of the factor map H the argument uses exactly the following equations, all proved for H_- and for H_{51} in Theorem 1.71: $H(1) = 0$ and $H(2a) = 3a$ (part (ii)); rotation $O_H(Sy, a) = O_H(y, H(a))$, $O_H(y, 0) = 0$ and $O_H(y, 3a) = 3\,O_H(y, a)$ (part (iii), the last two proved there from $H(0) = 0$ and $H(3a) = 3H(a)$ of part (ii)). It uses, besides, $Z(2^x) = 0$ and $Z(3a) = Z(a)$ (proof of part (iii)), which do not involve H, the selector laws Theorem 1.58(ii) and Theorem 1.70(i), and the computation equations. It uses no property of the parity N and no other property of H; in particular the form $\mathrm{sel}(N(u), \cdot, 0)$ of H_{51}, and the fusion of Theorem 1.71(i), are not used. Write K, β', W', P' for K_H, β'_H, W'_H, P'_H, and O for O_H.

Comparator. By I0, $K(x, 0) = Z(2^x) = 0$. By $2^0 = 1$ (I0) and rotation, $K(0, Sy) = Z(O(Sy, 1)) = Z(O(y, H(1))) = Z(O(y, 0)) = Z(0) = 1$. By $2^{Sx} = 2 \cdot 2^x$ (IS), rotation, $H(2a) = 3a$, $O(y, 3a) = 3\,O(y, a)$ and $Z(3a) = Z(a)$,

$$K(Sx, Sy) = Z\big(O(Sy, 2{\cdot}2^x)\big) = Z\big(O(y, H(2{\cdot}2^x))\big) = Z\big(O(y, 3{\cdot}2^x)\big) = Z\big(3\,O(y, 2^x)\big) = Z\big(O(y, 2^x)\big) = K(x, y).$$

In particular $K(1, 1) = K(0, 0) = 0$, by the third and the first equation.

Counter. Since $K(Su, k) = Z(O(k, 2^{Su}))$, Theorem 1.70(i) gives $\beta'(k, u) = \mathrm{sel}\big(Z(O(k, 2^{Su})), u, Su\big) = \mathrm{sel}\big(K(Su, k), u, Su\big)$. Hence, by the comparator equations and Theorem 1.58(ii),

$$\begin{aligned}
\beta'(1, 0) &= \mathrm{sel}\big(K(1, 1), 0, 1\big) = \mathrm{sel}(0, 0, 1) = 0,\\
\beta'(SSk, 0) &= \mathrm{sel}\big(K(S0, SSk), 0, 1\big) = \mathrm{sel}\big(K(0, Sk), 0, 1\big) = \mathrm{sel}(1, 0, 1) = 1,\\
\beta'(Sk, Su) &= \mathrm{sel}\big(K(SSu, Sk), Su, SSu\big) = \mathrm{sel}\big(K(Su, k), Su, SSu\big) = S\beta'(k, u).
\end{aligned}$$

The two equations. These are the three hypotheses of Theorem 1.73 for $f(k, u) := \beta'(k, u)$, for which $W_f = W'$ and $P_f = P'$. The lemma gives $\vdash_{\mathcal{E}} P'(0) = 0$ and $\vdash_{\mathcal{E}} P'(Sx) = x$. The transfer is Theorem 1.54(iv).

(ii) By Theorem 1.67, $|2^{Su}| = 1 + 2 + 2 + 5 = 10$ and $|O_H(m, 2^{Su})| = 1 + 1 + 10 + |H(v)|$. For H_-, with $|H_-(v)| = 26$ (Theorem 1.72(ii)), this is 38; the body $\mathrm{sel}(O_{H_-}(m, 2^{Su}), Su, u)$ has $1+38+2+1 = 42$ nodes; and $|P_{45}(m)| = 1+1+1+42 = 45$. For H_{51}, with $|H_{51}(v)| = 31$, the same count gives 43, 47 and $|P'_{50}(m)| = 50$. The comparators have $|K_{H_-}(x, y)| = 4 + 1 + 1 + 9 + 26 = 41$ and $|K_{H_{51}}(x, y)| = 46$ nodes. □

Remark 1.75 (The diagonal counter; sources). The route through Theorem 1.73 is that of `orbit_equality`. The derivation that it replaces, which follows `pred50` and `fused_iteration`, proves the comparator diagonal $K(n, n) = 0$ by an equation induction, the conjugacy as in

Theorem 1.73, the invariant $W'(n, Sn) = n$ below the cap by a second induction, and then $P'(Sx) = \beta'(Sx, W'(x, Sx)) = \beta'(Sx, x) = \mathrm{sel}(K(x, x), x, Sx) = x$. The lemma needs only the instance $K(1, 1) = 0$, which is a computation, and replaces the invariant and the final calculation by one induction on $P'(Sx) = x$: two equation inductions instead of three. The term P_{45} and its size are unchanged. The lemma applies in the same way to the counters of Theorem 1.65 and Theorem 1.72: by Theorem 1.64, $\beta(1, 0) = \mathrm{sel}(L(2, 1), 0, 1) = \mathrm{sel}(L(1, 0), 0, 1) = \mathrm{sel}(0, 0, 1) = 0$, and by Theorem 1.71(iv), $\beta_-(1, 0) = \mathrm{sel}(\Lambda(1, 1), 1, 0) = \mathrm{sel}(\Lambda(0, 0), 1, 0) = \mathrm{sel}(1, 1, 0) = 0$; the other two hypotheses are (1.15), (1.16) and their counterparts in the proof of Theorem 1.72. The proofs given there are kept. The Lean modules `TeqPred` and `EqPred` prove the predecessor equation of Theorem 1.65 by this route: `Q.pred_succ` is one equation induction on $P(Sx) = x$ from the conjugacy `Q.bump_sat_succ` and $\beta(1, 0) = 0$, so neither the comparator diagonal nor the invariant below the cap is formalized there. `orbit_equality` checks Theorem 1.73 by a symbolic certificate of 37 equality-proof nodes with exactly two equation inductions, in the schematic equational theory whose only axioms are $W(0, n) = 0$, $W(St, n) = F(n, W(t, n))$ and the three hypotheses of the lemma for F in place of f. The lemma itself, for an arbitrary template f, is not formalized in Lean or Rocq.

The sizes of the pieces, with variables as arguments, are

term	P	P_{57}	P_{51}	P_{50}	P'_{50}	P_-	P_{45}
three-cycle	30	13	13	13	13	12	12
ramp	38	21	21	21	21	20	20
parity control of the factor map	8	8	8	7	8	fused	fused
factor map	48	31	31	30	31	26	26
comparator	66	49	47	46	46	42	41
predecessor	75	57	51	50	50	46	45

The comparators are L, L_{57}, $\Lambda_{H_{51}}$, $\Lambda_{H_{50}}$, $K_{H_{51}}$, Λ and K_{H_-}. The orientation of `pred50` saves one node against the orientation of Λ, with either factor map: the orbit counter m and the seed 2^{Su} have $1 + 10$ nodes against $3 + 9$ for SSu and 2^m. In P_{51}, P_{50}, P'_{50}, P_- and P_{45} the comparator is not a subterm: the counter tests the orbit without the outer zero test, by Theorem 1.70(i). Against P_{57} this removes the parity wrapper $N(\cdot)$ of L_{57} (7 nodes) and costs one successor in the orbit; the weighted parity of H_{50} saves one more node; and against P_{50}, the floor three-cycle saves one node in the clock and so in the ramp, and the fused factor map H_- saves four nodes against H_{50} (26 against $1 + 7 + 20 + 1 = 29$ with the floor ramp). The ceiling three-cycle and ramp alone, with the counter of Theorem 1.65 ($\mathrm{sel}(L_{57}(SSu, m), u, Su)$ as body), give 58 nodes; the counter of P_{57} alone, with L, gives 74. The saving of 30 nodes, from P to P_{45}, concerns one expanded term, not the size of a checker or of a derivation.

Proposition 1.76 (A lower bound for the parity control)**.** *For a term t whose only free variable is n write $t(k)$ for its value in the natural numbers at $n = k$.*

(i) *If t contains at most one iterator, then $t(k)$ is nondecreasing on the positive numbers k.*

(ii) *If $c(1) > 0$ and $c(2) = 0$, then $|c| \geq 7$; the term $\tilde{N}(n)$ attains the bound.*

(iii) *If c is such a term and $\mathrm{sel}(c, G_-(n), 0)(1) = 0$ and $\mathrm{sel}(c, G_-(n), 0)(2) = 3$, then $|\mathrm{sel}(c, G_-(n), 0)| \geq 30$; the term $H_{50}(n)$ attains the bound.*

Proof. (i) Above the only iterator of t there are only successors, so t is $S^\ell n$, $S^\ell 0$ or $S^\ell I(k, a;\, u.\, h)$ with k, a and h free of iterators. Then k and a are $S^i n$ or $S^i 0$, and h is $S^r u$, $S^r n$ or $S^r 0$. If k is 0, the value is that of a. Otherwise $k(j) \geq 1$ for every positive j, and the three bodies give the values $a(j) + r\, k(j)$, $j + r$ and r. Each is nondecreasing, and so is its ℓ-fold successor.

(ii) A term with p iterators and s successors has $1 + 2p$ leaves, since each iterator has three children and each successor one; so $|t| = 1 + 3p + s$. A term with $|t| \leq 6$ has $p \leq 1$, and by (i)

$t(1) \le t(2)$. Hence $c(1) > 0 = c(2)$ forces $|c| \ge 7$. For $\tilde{N}$: $|\tilde{N}(n)| = 7$, $\tilde{N}(1) = \mathrm{sel}(0,1,0) = 1$ and $\tilde{N}(2) = \mathrm{sel}(\mathrm{sel}(0,2,0),2,0) = \mathrm{sel}(2,2,0) = 0$.

(iii) $G_-(1) = 2$ and $G_-(2) = 3$. The value at 1 is 0 only if $c(1) > 0$, and the value at 2 is 3 only if $c(2) = 0$. By (ii) $|c| \ge 7$, so the size is $1 + |c| + 21 + 1 \ge 30$. $H_{50}(n)$ has 30 nodes, and $H_{50}(1) = 0$, $H_{50}(2) = 3$ by Theorem 1.71 and $2 = 2 \cdot 1$. □

The bound concerns the control alone and the factor map of the form $\mathrm{sel}(c, G_-(n), 0)$ with the ramp G_- fixed; it is not a lower bound for a factor map of another form or with another ramp, or for a predecessor. The fused factor map H_-, a single iteration seeded by its argument, is of another form, and its 26 nodes do not contradict (iii).

Proposition 1.77 (A lower bound for the three-cycle)**.** *With the notation of Theorem 1.76, let f be a term whose only free variable is n, and suppose that its values have a three-cycle through 0: $f(0) = p$, $f(p) = q$ and $f(q) = 0$ for distinct positive numbers p and q. Then $|f| \ge 9$. The term $J'(n)$ attains the bound, and so a three-cycle of the form $I(n, 0; v.\, f(v))$ has at least 12 nodes, attained by M'.*

Proof. By the count $|t| = 1 + 3j + s$, for j iterators and s successors, of the proof of Theorem 1.76(ii), a term with at most 8 nodes has $j \le 2$, and $s \le 1$ if $j = 2$.

At most one iterator. Then f is $S^\ell n$, $S^\ell 0$ or $S^\ell I(k, a;\, u.\, h)$ with $k, a \in \{S^i n, S^i 0\}$ and $h \in \{S^r u, S^r n, S^r 0\}$. A constant has no such cycle, since $f(0) = f(p)$ would give $p = q$. If $\ell > 0$ or $f = n$, then $f(q) > 0$. So $f = I(k, a;\, u.\, h)$. If k is the leaf 0, then $f = a$, treated already. Otherwise $k(i) \ge 1$ for every positive i, and at positive i the body $S^r u$ gives $a(i) + r\, k(i)$, which vanishes at q only if a is the leaf 0 and $r = 0$, and then f is identically 0; the body $S^r n$ gives $i + r > 0$; and the body $S^r 0$ gives the constant r, so that $q = f(p) = r = f(q) = 0$. None has the cycle.

Two iterators and no successor. By induction on terms, a term without S has the value 0 when all its free variables have the value 0; so $f(0) = 0 \ne p$.

Two iterators and one successor. Suppose that the successor is in the body of an iterator X, and let Y be the other iterator. If X lies in the body of Y, then Y is the whole term, its counter has neither a successor nor a bound variable, so it has the value 0 at $n = 0$, and Y returns its seed, which has no successor: $f(0) = 0$. Otherwise the counter of X has neither a successor nor a bound variable, so it has the value 0 at $n = 0$, X returns its seed, which has no successor, and every subterm has the value 0 at $n = 0$: again $f(0) = 0$. Hence the successor is in no body. If $f = St'$, then $f(q) > 0$. So $f = I(k, a;\, u.\, h)$ with the successor in k or in a. If it is in k, then a and h have no successor and $f(0) = 0$ as before. So the successor is in a, and k and h have none. The second iterator is in k, in a or in h. (1) In k: then $a \in \{1, Sn\}$ and $h \in \{0, n, u\}$, and k is an iterator over the leaves 0, n and its own state, whose value at a positive i is uniformly 0 or i; so at positive i the value of f is uniformly one of 0, i, 1, $i + 1$, and $f(q) = 0$ forces $f(i) = 0$ for all positive i, whence $q = f(p) = 0$. (2) In a: then $k \in \{0, n\}$ and $h \in \{0, n, u\}$. If $k = 0$ or $h = u$, then $f = a$, a term with one iterator, treated above. Otherwise $k = n$, $h \in \{0, n\}$, and $f(i)$ is uniformly 0 or i at positive i, so that $q = f(p)$ is 0 or p. (3) In h: then $a \in \{1, Sn\}$ and $k \in \{0, n\}$. If $k = 0$, then $f = a > 0$. If $k = n$, then $f(0) = a(0) = 1$, so $p = 1$; and h is an iterator over the leaves 0, n, u and its own state, without successor, whose value at a positive n and a positive u is uniformly 0, n or u. Then $f(1) = h(1, a(1)) \in \{0, 1, a(1)\}$, and $q = f(1) \notin \{0, 1\}$ forces $a = Sn$ and a body that returns u; but then at every positive i the state stays $i + 1$, and $f(q) = q + 1 \ne 0$.

So no term with at most 8 nodes has the cycle. The term $J'(n) = I(Sn, n;\, w.\, \mathrm{sel}(w, Sn, 0))$ has 9 nodes and the cycle $0 \mapsto 1 \mapsto 2 \mapsto 0$ by Theorem 1.69(i), which holds in the natural numbers. The last claim follows from $|I(n, 0; v.\, f(v))| = 3 + |f|$. □

The bound concerns the update of a three-cycle of this form; it is not a lower bound for a clock of another form or for a predecessor.

Remark 1.78 (Use of the smaller predecessor; sources). The term P_- has the two properties of P that the later sections use: its only free variable is its argument, and it satisfies the two equations of Theorem 1.65 in $\mathcal{E}$, hence in every context and at every term. The constructions of the rest of this section, and the single-binder universal of Section 1.4.7, use P only through these properties, and so do P_- and P_{45}, which may replace P in each of them; with P_{45} every occurrence of a quantified variable in Theorem 1.31 costs $|P_{45}(z)| - 1 = 44$ added nodes instead of 74 (45 with P_-). Moreover, if two terms p and p' both satisfy the two equations in $\mathcal{E}$, then equational cases give $p(x) = p'(x)$, so a replacement inside a term yields a provably equal term (inside an iterator body by Theorem 1.56(ii)). The article keeps P as the reference term because its derivation is formalized in Lean.

The terms of the table come from five packages: P_{57} from `479_predecessor57`, P_{51} from `absorbing_orbit`, P_{50} from `absorbing_predecessor`, P'_{50} from `pred50`, and P_- from `fused_iteration`; P_{45} is the combination of Theorem 1.74. P_{50} and P'_{50} are different terms of the same size: P'_{50} keeps the parity N in its factor map and saves its node in the comparator, whose orbit runs m steps from the seed 2^{Su} instead of SSu steps from 2^m. The derivations of P_{51} and P_{50} are part of Theorem 1.72, and that of P'_{50} is part of Theorem 1.74. For P_{57}, the two identities in the proof of Theorem 1.71(ii), instantiated at $c = u$ and $e = 3u$, give $3H_{57}(u) = H_{57}(3u)$; with $H_{57}(2u) = \mathrm{sel}(0, 3u, 2u) = 3u$ and $H_{57}(1) = \mathrm{sel}(1, G_-(1), 1) = 1$ these are the three equations of Theorem 1.62, so the proofs of Theorems 1.63 and 1.64 give the four comparator equations of Theorem 1.71(iv) for L_{57} in place of Λ_H, the fourth by the same equation induction; the proof of Theorem 1.72(i) then applies with $L_{57}(k, Su)$ in place of $\Lambda(k, Su)$, without the use of Theorem 1.70(i).

The packages prove the ramp laws by the same inductions; the main report of `479_predecessor57` splits the Boolean value $N(n)$ in the induction step, and the detailed notes of all five replace those splits by the equational two-point argument, as in the proofs above. `absorbing_predecessor` and `fused_iteration` prove $H(3u) = 3H(u)$ by distributing the selector over a sum and $\mathrm{sel}(c, c, 0) = 0$ instead of the identities used here. `absorbing_predecessor` obtains P_{50} from P_{51} by two applications of the congruence of iterator bodies, using Theorem 1.70(iii). `fused_iteration` states the fusion of Theorem 1.71(i) for every template $A(n)$ with $\vdash_{\mathcal{E}} A(0) = 0$ in place of $G'(n)$, with the same proof. All five prove the predecessor equations through a counter of the form $W(t, m) = I(t, 0;\, u.\, \mathrm{sel}(L'(m, u), Su, u))$ (for `pred50`, $\mathrm{sel}(K(u, m), u, Su)$), with L' the respective comparator, the shift $W(St, Sm) = SW(t, m)$, $L'(n, n) = 1$ and $W(n, n) = n$, followed by a further equation induction that identifies the body of the predecessor at Sm with that of $W(\cdot, m)$ (Theorem 1.56(ii)), and by a further body congruence for the step to the final orbit test; the counters $W_H(t, k)$ and the conjugacy between the caps SSk and Sk used above avoid those inductions, as in Theorem 1.65. `absorbing_predecessor` also proves Theorem 1.76, and `fused_iteration` proves Theorem 1.77. None of the new terms is formalized in Lean or Rocq.

1.7.8 Coverage from a zero-test split

This subsection draws on `zero_test_split`. With the equational predecessor available, the coverage axiom (T2) of Theorem 1.49 can be replaced by a disjunction of two equations without an existential quantifier. Let $\mathsf{T}^-_{\mathrm{eq}}$ be the theory with the axioms (T1) and (T4) of Theorem 1.49, and consider the following schemas, for every term n:

$$\mathsf{Cov}:\ n = 0 \vee \exists m\,(n = Sm), \qquad \mathsf{Split}:\ n = 0 \vee Z(n) = 0, \qquad \mathsf{BR}:\ Z(n) = 0 \vee Z(n) = 1.$$

Cov is (T2), and Split written out is $n = 0 \vee I(n, 1;\, u.\, 0) = 0$. The proof of Theorem 1.54(iv) derives EI from (T4) by Theorem 1.50(ii), which does not use (T2) or (T3), and TR from the Leibniz rule. Hence every sequent derivable in $\mathcal{E}$ is derivable in $\mathsf{T}^-_{\mathrm{eq}}$, in particular the equations of Theorem 1.65.

Lemma 1.79 (Decomposition)**.** *$\mathcal{E}$ derives $n = \mathrm{sel}(Z(n), SP(n), 0)$.*

Proof. Equational cases (Theorem 1.54(iii)) on n. At 0 the right side is $\mathrm{sel}(Z(0), SP(0), 0) = \mathrm{sel}(1, SP(0), 0) = 0$. At Sx it is $\mathrm{sel}(Z(Sx), SP(Sx), 0) = \mathrm{sel}(0, SP(Sx), 0) = SP(Sx) = Sx$, by Theorem 1.65. □

Theorem 1.80 (Three equivalent case axioms)**.** *Over $\mathsf{T}^-_{\mathrm{eq}}$ the schemas* Cov*,* Split *and* BR *are pairwise interderivable: each instance of one is derivable from the instances of another at the same term. Separation is not used.*

Proof. Cov ⇒ Split: eliminate the disjunction. The branch $n = 0$ is the first disjunct. In the other branch eliminate the existential quantifier at a fresh m; from $n = Sm$ the Leibniz rule gives $Z(n) = Z(Sm) = 0$.

Split ⇒ Cov: the branch $n = 0$ is the first disjunct. In the branch $Z(n) = 0$, Theorem 1.79 and the Leibniz rule give $n = \mathrm{sel}(0, SP(n), 0) = SP(n)$, and ∃I at the witness $P(n)$ gives the second disjunct.

Split ⇒ BR: in the branch $n = 0$, $Z(n) = Z(0) = 1$; the branch $Z(n) = 0$ is the first disjunct.

BR ⇒ Split: the branch $Z(n) = 0$ is the second disjunct. In the branch $Z(n) = 1$, Theorem 1.79 gives $n = \mathrm{sel}(1, SP(n), 0) = 0$.

Each argument is one elimination of a disjunction, with at most one elimination of an existential quantifier. □

Corollary 1.81 (Coverage replaced by the split)**.** *Let $\mathsf{T}^{\mathrm{Z}}_{\mathrm{eq}}$ be T_{eq} with (T2) replaced by* Split*. Then $\mathsf{T}^{\mathrm{Z}}_{\mathrm{eq}}$ and T_{eq} have the same theorems, and Theorem 1.119 holds with $\mathsf{T}^{\mathrm{Z}}_{\mathrm{eq}}$ in place of T_{eq}.*

Proof. The two theories share the logic, the language and the axioms (T1), (T3) and (T4), and so both contain $\mathsf{T}^-_{\mathrm{eq}}$; they differ only in (T2) and Split. By Theorem 1.80 each of these is a theorem of the other theory, so a derivation in one becomes a derivation in the other by replacing each instance of the differing axiom with its derivation. The identity translation is therefore a bi-interpretation of the two theories, and Theorem 1.119 transfers verbatim. □

Proposition 1.82 (The split is not derivable without a case axiom)**.** *The axioms (T1), (T3), (T4) do not derive* Split*,* Cov *or* BR *at a variable, even together with every instance of excluded middle. In particular $n = 0 \vee (n = 0 \to \bot)$ cannot replace* Split*.*

Proof. Interpret the language in the classical structure with domain $\mathbb{N} \times \mathbb{N}$: 0 is $(0, 0)$, the successor acts on both coordinates, and the value of a term under an assignment ρ is the pair of its values in $\mathbb{N}$ under the two coordinate assignments ρ_0 and ρ_1; this is well defined and commutes with substitution, because the values in $\mathbb{N}$ do. The computation equations (T1) hold in each coordinate, and (T3) holds because $(0, 0) \neq (1, 1)$. For (T4) fix the parameters of an equation $E(z)$ and suppose that $E(0)$ holds and that $E(z)$ implies $E(Sz)$ for every z. An induction in $\mathbb{N}$ gives $E((k, k))$ for every k. An equation holds at a pair if and only if it holds in each coordinate, and the coordinate i of $E((a_0, a_1))$ depends only on a_i and the coordinate i of the parameters. Hence the coordinate 0 of $E((a_0, a_1))$ is that of $E((a_0, a_0))$ and the coordinate 1 is that of $E((a_1, a_1))$, and $E((a_0, a_1))$ holds. The structure is classical, so it satisfies every instance of excluded middle, and intuitionistic derivations are sound in it. At $n = (0, 1)$, however, $n \neq (0, 0)$ and $Z(n) = (Z(0), Z(1)) = (1, 0)$ differs from $(0, 0)$ and from $(1, 1)$, so Split and BR fail, and Cov fails by Theorem 1.80. □

Remark 1.83 (Scope of the split; sources). Split is quantifier free and has no witness; coverage becomes a theorem with the fixed witness $P(n)$. Counting one node for each term node of Theorem 1.67, each equation, connective and quantifier, the universal closure of Cov has 10 nodes

and that of Split, expanded, has 12; the existential-free alternative $\forall n\,(n = 0 \vee n = SP_{45}(n))$ has $8 + 45 = 53$ (65 with the term P_{57} of Section 1.7.7, as `zero_test_split` counts it). The change removes an existential quantifier, not nodes. Separation stays an axiom of $\mathsf{T}_{\mathrm{eq}}^{\mathrm{Z}}$ for the reason of Theorem 1.51: the one-element structure also satisfies Split. `zero_test_split` adds that computation and the Leibniz rule derive $0 = 1 \vdash s = t$, by transport into $\mathrm{sel}(\cdot, s, t)$, and hence, by induction on formulas with the witness 0 at an existential quantifier, $0 = 1 \vdash \varphi$ for every formula φ built from equations with $\wedge$, $\vee$, $\to$, $\forall$ and $\exists$; this does not make (T3) derivable, since $\bot$ is a logical constant. The same product structure shows, in the chapter on the kernel rules, that coverage does not follow from the other scalar principles. The equivalence is not formalized in Lean.

1.7.9 Successor laws, subtraction and the reflection of equality

From here on the derivations are in T_{eq} and use coverage and separation. "By cases on r" means the elimination of the disjunction (T2) at r followed by the Leibniz rule; unlike equational cases it may be applied under hypotheses that mention r.

Lemma 1.84 (Successor laws)**.** T_{eq} *proves* $Sx = Sy \to x = y$, $Sx = 0 \to \bot$ *and* $x = 0 \vee x = SP(x)$.

Proof. The first is Theorem 1.65. From $Sx = 0$ congruence gives $Z(Sx) = Z(0)$, that is $0 = S0$, and (T3) gives $\bot$. For the third, eliminate (T2): if $x = 0$ the first disjunct holds, and if $x = Sy$ then $P(x) = y$ and $x = SP(x)$. □

Put $x \mathbin{\dot{-}} y := I(y, x;\, u.\, P(u))$ and $K(u) := \mathrm{sel}(u, 1, P(u))$, and define the *equality test*

$$\varepsilon(a, b) := K(Sa \mathbin{\dot{-}} b).$$

Lemma 1.85 (Truncated subtraction)**.** T_{eq} *proves*

(i) $x \mathbin{\dot{-}} 0 = x$, $x \mathbin{\dot{-}} Sy = P(x \mathbin{\dot{-}} y)$ *and* $Sx \mathbin{\dot{-}} Sy = x \mathbin{\dot{-}} y$;

(ii) $Sa \mathbin{\dot{-}} a = 1$;

(iii) $\mathrm{sel}(x \mathbin{\dot{-}} y,\ x,\ y + (x \mathbin{\dot{-}} y)) = x$;

(iv) $Sa \mathbin{\dot{-}} b = 1 \leftrightarrow a = b$.

Proof. (i) The first two are computations. The equation $P(Sx \mathbin{\dot{-}} y) = x \mathbin{\dot{-}} y$ holds by induction on y: the base is $P(Sx) = x$, and $P(Sx \mathbin{\dot{-}} Sy) = P(P(Sx \mathbin{\dot{-}} y)) = P(x \mathbin{\dot{-}} y) = x \mathbin{\dot{-}} Sy$. Its left side is $Sx \mathbin{\dot{-}} Sy$.

(ii) Induction on a: $S0 \mathbin{\dot{-}} 0 = 1$, and $SSa \mathbin{\dot{-}} Sa = Sa \mathbin{\dot{-}} a$ by (i).

(iii) Induction on y with x fixed. The base is $\mathrm{sel}(x, x, 0 + x) = \mathrm{sel}(x, x, x) = x$, by Theorem 1.58(ii). For the step write $r := x \mathbin{\dot{-}} y$; the induction equation is $\mathrm{sel}(r, x, y + r) = x$ and the goal is $\mathrm{sel}(P(r), x, Sy + P(r)) = x$. Argue by cases on r. If $r = 0$, then $P(r) = 0$ and the goal is $\mathrm{sel}(0, x, \dots) = x$. If $r = Sz$, the induction equation gives $y + Sz = x$, and $P(r) = z$, so $Sy + P(r) = Sy + z = y + Sz = x$ and the goal is $\mathrm{sel}(z, x, x) = x$.

(iv) If $Sa \mathbin{\dot{-}} b = 1$, then (iii) at $x = Sa$, $y = b$ gives $\mathrm{sel}(1, Sa, b + 1) = Sa$, that is $Sb = Sa$, and Theorem 1.84 gives $a = b$. The converse is (ii). □

The equation (iii) says, in one literal equation, that a nonzero difference is an actual difference.

Theorem 1.86 (Reflection of equality)**.** T_{eq} *proves, for all terms* a *and* b,

$$\varepsilon(a, b) = 0 \ \leftrightarrow\ a = b.$$

Consequently $\mathsf{T}_{\mathrm{eq}} \vdash a = b \vee (a = b \to \bot)$.

Proof. First, $K(u) = 0 \leftrightarrow u = 1$. If $u = S0$ then $K(u) = P(S0) = 0$. Conversely assume $K(u) = 0$ and argue by cases on u. If $u = 0$ then $K(u) = 1$, so $S0 = 0$, which is contradictory by Theorem 1.84. If $u = Sz$ then $K(u) = P(Sz) = z$, so $z = 0$ and $u = S0$. Hence $\varepsilon(a, b) = 0$ is equivalent to $Sa \dot{-} b = 1$, and to $a = b$ by Theorem 1.85(iv). For the consequence argue by cases on $\varepsilon(a, b)$: if it is 0 then $a = b$; if it is Sz, then $a = b$ would give $Sz = 0$. □

The theorem concerns the equality of the logic on the whole domain. It is not restricted to numerals, and no faithful comparison symbol is assumed.

Remark 1.87 (Variants). The reference statement uses the sign $\mathrm{sg}(u) := \mathrm{sel}(u, 0, 1)$, the invariant $\mathrm{sel}(\mathrm{sg}(x \dot{-} y), x, y + (x \dot{-} y)) = x$ and the test $\mathrm{sg}(K(Sa \dot{-} b))$, which takes the values 0 and 1 only, and it carries the disjunction "$b_\varphi = 0 \vee b_\varphi = 1$" through the construction below. The form above omits the sign: the selector tests its first argument for zero, so a two-valued characteristic is not needed. The Lean module `TeqReflect` uses the symmetric test $\varepsilon'(a, b) := \mathrm{sel}(a \dot{-} b,\ b \dot{-} a,\ 1)$ instead, whose reflection needs neither K nor Theorem 1.85(iv). If $\varepsilon'(a, b) = 0$, then $a \dot{-} b = 0$, since a successor would give $1 = 0$; hence $\varepsilon'(a, b) = b \dot{-} a = 0$, and Theorem 1.96(iii) gives $a = a + (b \dot{-} a) = b$. Conversely $\varepsilon'(a, a) = \mathrm{sel}(0, 0, 1) = 0$ by Theorem 1.96(i). The test ε contains each argument three times and ε' twice; Section 1.7.10 gives a test that contains each once.

1.7.10 Equality from the absorbing orbit

This subsection draws on `orbit_equality`. The absorbing factor map H_- of Section 1.7.7 gives a second equality test. With $O = O_{H_-}$, put

$$\delta(a, b) := N\big(O(b, 2^a)\big), \qquad \eta(a, b) := I(O(b, 2^a), 1;\ v.\, Z(v)).$$

Each of a and b occurs once in either term. On the natural numbers the orbit $O(b, 2^a)$ is $3^b 2^{a-b}$ for $b \le a$ and 0 for $b > a$ (Section 1.7.7), whose parity is 1 exactly when $b = a$; so δ is the indicator of $a = b$ and $\eta = 1 - \delta$. This reading is not used.

Lemma 1.88 (Recurrences of the orbit test)**.** *$\mathcal{E}$ derives, for all terms,*

(i) $\delta(0, 0) = 1$, $\delta(Sa, 0) = 0$, $\delta(0, Sb) = 0$ *and* $\delta(Sa, Sb) = \delta(a, b)$;

(ii) $\delta(a, a) = 1$, $\delta(0, b) = Z(b)$ *and* $\delta(a, 0) = Z(a)$;

(iii) $\eta(a, b) = Z\big(\delta(a, b)\big)$.

Proof. (i) The equations $2^0 = 1$, $2^{Sa} = 2 \cdot 2^a$ and $O(0, c) = c$ are I0 and IS. With Theorem 1.71(ii) and (iii) for H_- and Theorem 1.59,

$$\begin{aligned}
\delta(0, 0) &= N\big(O(0, 1)\big) = N(1) = 1, \qquad \delta(Sa, 0) = N\big(2 \cdot 2^a\big) = 0,\\
\delta(0, Sb) &= N\big(O(Sb, 1)\big) = N\big(O(b, H_-(1))\big) = N\big(O(b, 0)\big) = N(0) = 0,\\
\delta(Sa, Sb) &= N\big(O(Sb, 2 \cdot 2^a)\big) = N\big(O(b, H_-(2 \cdot 2^a))\big) = N\big(O(b, 3 \cdot 2^a)\big)\\
&= N\big(3\, O(b, 2^a)\big) = N\big(O(b, 2^a)\big) = \delta(a, b),
\end{aligned}$$

by rotation, $H_-(1) = 0$, $O(y, 0) = 0$, $H_-(2c) = 3c$, $O(y, 3c) = 3\, O(y, c)$, $N(2n) = 0$ and $N(3n) = N(n)$.

(ii) $\delta(a, a) = 1$ is an equation induction on a: the base is $\delta(0, 0) = 1$, and $\delta(Sa, Sa) = \delta(a, a) = 1$ by (i) and the induction equation. $\delta(0, b) = Z(b)$ is equational cases on b: $\delta(0, 0) = 1 = Z(0)$ and $\delta(0, Sz) = 0 = Z(Sz)$, by (i) and Theorem 1.58(i). Likewise $\delta(a, 0) = Z(a)$ by cases on a.

(iii) Put $B(q) := I(q, 1;\ v.\, Z(v))$ for a variable q. Then $B(q) = Z(N(q))$ by EI on q: the base is $B(0) = 1 = Z(0) = Z(N(0))$, and $B(Sq) = Z(B(q)) = Z(Z(N(q))) = Z(N(Sq))$,

by IS, the induction equation by congruence, and $N(Sq) = Z(N(q))$ (Theorem 1.59). The instance at $q := O(b, 2^a)$ is $\eta(a,b) = Z(\delta(a,b))$. □

The derivation uses no predecessor, no order and no equality test; it uses of H_- only the equations of Theorem 1.71(ii) and (iii), and so holds for H_{51} in place of H_- as well.

Theorem 1.89 (Reflection of equality by the orbit test)**.** T_{eq} *proves, for all terms* a *and* b*,*

(i) $\delta(a,b) = 1 \leftrightarrow a = b$ *and* $\eta(a,b) = 0 \leftrightarrow a = b$*;*

(ii) $\delta(a,b) = 0 \vee \delta(a,b) = 1$ *and* $\eta(a,b) = 0 \vee \eta(a,b) = 1$*.*

Proof. The proof uses the predecessor P, truncated subtraction and Theorem 1.85(i)–(iii); it does not use Theorem 1.86, Theorem 1.85(iv) or induction for compound formulas. Every induction is Theorem 1.50(ii) for a literal equation.

Zero fibre. $Z(r) = 1 \to r = 0$. By cases on r: if $r = 0$ there is nothing to prove; if $r = Sy$, then $1 = Z(r) = Z(Sy) = 0$, so $0 = S0$ and (T3) gives $\bot$, from which $r = 0$ follows. By Theorem 1.88(ii), $\delta(0,r) = 1 \to r = 0$.

One fibre. $\delta(1,r) = 1 \to r = 1$. By cases on r: if $r = 0$, then $\delta(1,0) = 0$ by Theorem 1.88(i), so $1 = 0$, a contradiction by (T3); if $r = Ss$, then $1 = \delta(S0, Ss) = \delta(0,s) = Z(s)$ by Theorem 1.88(i),(ii), so $s = 0$ by the zero fibre, and $r = 1$.

Residual invariant. For variables a, b and t,

$$\delta(a,b) = 1 \ \vdash_{\mathsf{T}_{\mathrm{eq}}} \ \delta(a \mathbin{\dot-} t,\ b \mathbin{\dot-} t) = 1,$$

by Theorem 1.50(ii) on t in the context $\Gamma := \{\delta(a,b) = 1\}$, in which t is not free. The base is the hypothesis, since $a \mathbin{\dot-} 0 = a$ and $b \mathbin{\dot-} 0 = b$. In the step write $A := a \mathbin{\dot-} t$ and $B := b \mathbin{\dot-} t$; the induction equation is $\delta(A,B) = 1$, and by Theorem 1.85(i) the goal is $\delta(P(A), P(B)) = 1$. Argue by cases on A and on B. If $A = B = 0$, then $P(A) = P(B) = 0$ and the goal is $\delta(0,0) = 1$. If $A = 0$ and $B = Sd$, the induction equation becomes $\delta(0, Sd) = 1$, while $\delta(0, Sd) = 0$, so $0 = S0$, and (T3) gives $\bot$ and hence the goal; the case $A = Sc$, $B = 0$ is the same with $\delta(Sc, 0) = 0$. If $A = Sc$ and $B = Sd$, then $\delta(P(A), P(B)) = \delta(c,d) = \delta(Sc, Sd) = \delta(A,B) = 1$, by Theorem 1.65 and Theorem 1.88(i). The induction formula is the one equation displayed; a, b and the hypothesis stay fixed.

(i) Assume $\delta(a,b) = 1$ and argue by cases on a. If $a = 0$, the zero fibre gives $b = 0 = a$. If $a = Sc$, the residual invariant at $t := c$ gives $\delta(Sc \mathbin{\dot-} c,\ b \mathbin{\dot-} c) = 1$, that is $\delta(1, b \mathbin{\dot-} c) = 1$ by Theorem 1.85(ii), and the one fibre gives $b \mathbin{\dot-} c = 1$. Theorem 1.85(iii) at $x := b$, $y := c$ then gives

$$b = \mathrm{sel}\big(b \mathbin{\dot-} c,\ b,\ c + (b \mathbin{\dot-} c)\big) = \mathrm{sel}(1, b, c+1) = c + 1 = Sc = a.$$

Conversely, $a = b$ gives $\delta(a,b) = \delta(a,a) = 1$ by the Leibniz rule and Theorem 1.88(ii).

(ii) Put $q := O(b, 2^a)$, so that $\delta(a,b) = N(q)$, and recall $N(q) + Z(N(q)) = 1$ (Theorem 1.59). By cases on $N(q)$: if $N(q) = 0$, the first disjunct holds; if $N(q) = Sy$, then $Z(N(q)) = 0$, and $N(q) = N(q) + 0 = 1$. By Theorem 1.88(iii), $\eta(a,b) = Z(\delta(a,b))$ is $Z(0) = 1$ in the first case and $Z(1) = 0$ in the second.

The second equivalence of (i) follows: if $a = b$, then $\delta(a,b) = 1$ and $\eta(a,b) = Z(1) = 0$; if $\eta(a,b) = 0$, then by (ii) either $\delta(a,b) = 0$, whence $\eta(a,b) = Z(0) = 1$ and $1 = 0$, a contradiction by (T3), or $\delta(a,b) = 1$, whence $a = b$. □

Like Theorem 1.86, the theorem concerns the equality of the logic on the whole domain, and it gives $\mathsf{T}_{\mathrm{eq}} \vdash a = b \vee (a = b \to \bot)$ by (ii). It is not a predecessor-free reconstruction: the residual invariant and the last step use P and Theorem 1.85.

Corollary 1.90 (A predecessor from the orbit test)**.** *Put* $P_\delta(m) := I(m, 0;\ u.\ \mathrm{sel}(\delta(Su, m), Su, u))$*. Then* $\vdash_{\mathcal{E}} P_\delta(0) = 0$ *and* $\vdash_{\mathcal{E}} P_\delta(Sx) = x$*, and* $|P_\delta(m)| = 52$*.*

Proof. Take $f(k,u) := \mathrm{sel}(\delta(Su,k),Su,u)$ in Theorem 1.73, so that $P_f = P_\delta$. By Theorem 1.88(i) and Theorem 1.58(ii),

$$\begin{aligned} f(1,0) &= \mathrm{sel}\big(\delta(1,1),1,0\big) = \mathrm{sel}\big(\delta(0,0),1,0\big) = \mathrm{sel}(1,1,0) = 0,\\ f(SSk,0) &= \mathrm{sel}\big(\delta(1,SSk),1,0\big) = \mathrm{sel}\big(\delta(0,Sk),1,0\big) = \mathrm{sel}(0,1,0) = 1,\\ f(Sk,Su) &= \mathrm{sel}\big(\delta(SSu,Sk),SSu,Su\big) = \mathrm{sel}\big(\delta(Su,k),SSu,Su\big) = Sf(k,u). \end{aligned}$$

The lemma gives the two equations. By Theorem 1.91(i), $|\delta(Su,m)| = 42+2+1 = 45$; the body has $1+45+2+1 = 49$ nodes and $|P_\delta(m)| = 1+1+1+49 = 52$. □

The derivation uses only Theorem 1.88(i), not the reflection of equality. The body of P_δ is that of P_{45} with the parity term N around the orbit $O(m,2^{Su})$, which accounts for the 7 further nodes; P_δ is larger than P_{45} and is not used below.

Proposition 1.91 (Sizes of the equality tests)**.** *In the measure of Theorem 1.67, for all terms s and t:*

(i) $|\delta(s,t)| = 42+|s|+|t|$ *and* $|\eta(s,t)| = 43+|s|+|t|$*; at variables* 44 *and* 45*;*

(ii) *with P_{45} in place of P,* $|\varepsilon(s,t)| = 187+3|s|+3|t|$ *and* $|\varepsilon'(s,t)| = 95+2|s|+2|t|$*; at variables* 193 *and* 99*;*

(iii) *with P itself,* $|\varepsilon(s,t)| = 307+3|s|+3|t|$ *and* $|\varepsilon'(s,t)| = 155+2|s|+2|t|$*; at variables* 313 *and* 159*.*

Proof. All bound variables of the abbreviations are fresh, so a size is additive in the sizes of the arguments. (i) $|2^s| = |s|+8$ and $|H_-(v)| = 26$ (Theorem 1.72(ii)), so $|O(t,2^s)| = 1+|t|+(|s|+8)+26 = 35+|s|+|t|$. Then $|\delta(s,t)| = 7+|O(t,2^s)|$, since $|N(c)| = |c|+7$, and $|\eta(s,t)| = 1+|O(t,2^s)|+|1|+|Z(v)| = 1+|O(t,2^s)|+2+5$. (ii) The term $P_{45}(m)$ has 45 nodes and two occurrences of m, so $|P_{45}(c)| = 43+2|c|$ and $|P_{45}(u)| = 45$. Hence $|r| = 1+|t|+(1+|s|)+45 = 47+|s|+|t|$ for $r := Ss\dot{-}t$, and $|\varepsilon(s,t)| = |\,\mathrm{sel}(r,1,P_{45}(r))| = 1+|r|+2+43+2|r| = 46+3|r|$. Further $|s\dot{-}t| = 1+|t|+|s|+45$, and $|\varepsilon'(s,t)| = 1+|s\dot{-}t|+|t\dot{-}s|+2$. (iii) The same count with $|P(m)| = 75$ (Theorem 1.72(ii)), whose argument also occurs twice, so that $|P(c)| = 73+2|c|$. □

Remark 1.92 (Use of the orbit test; sources). By Theorem 1.78 the predecessor P_{45} may replace P in the definition of truncated subtraction, and so in ε. `orbit_equality` compares η with ε expanded with P_{45}, the 193 nodes of Theorem 1.91(ii): η saves 148 nodes, and contains each argument once instead of three times. The figure 193 is not stated elsewhere in the article; it is the expansion of the test ε defined before Theorem 1.85. Against the symmetric test ε' of Theorem 1.87, also expanded with P_{45}, the saving is 54 nodes. Neither 45 nor any other of these sizes is claimed to be minimal, and the counts concern one expanded term, not a checker or a derivation.

In Theorem 1.93 the atomic clause may be $\chi_{s=t} := \eta(s,t)$ instead of $\varepsilon(s,t)$; the proof of Theorem 1.94 is unchanged, its atomic case being Theorem 1.89(i), and so is that of Theorem 1.95. No axiom, rule, domain or interpretation changes, so Theorem 1.119 holds verbatim with either atomic clause. The order of the construction is kept: the recurrences of Theorem 1.88 are available after Theorem 1.71, and the reflection after Theorem 1.85(iii). The checkers delivered with `orbit_equality` implement the inherited guarded calculus and do not use η; they are recorded with the other checkers in the chapter on certificates. None of the terms of this subsection is formalized in Lean or Rocq.

1.7.11 Induction for quantifier-free formulas

Definition 1.93 (Characteristic term)**.** For a quantifier-free ordinary formula φ the term χ_φ is

$$\chi_{s=t} := \varepsilon(s,t), \qquad \chi_\bot := 1, \qquad \chi_{\varphi\wedge\psi} := \mathrm{sel}(\chi_\varphi, \chi_\psi, 1),$$
$$\chi_{\varphi\vee\psi} := \mathrm{sel}(\chi_\varphi, 0, \chi_\psi), \qquad \chi_{\varphi\to\psi} := \mathrm{sel}(\chi_\varphi, \chi_\psi, 0).$$

The definition is a recursion on finite syntax, and $\chi_{\varphi[\theta]} = \chi_\varphi[\theta]$.

Proposition 1.94 (Reflection of quantifier-free formulas)**.** *For every quantifier-free φ, $\mathsf{T}_{\mathrm{eq}} \vdash \chi_\varphi = 0 \leftrightarrow \varphi$.*

Proof. By induction on φ. An equation is Theorem 1.86. For $\bot$: $S0 = 0$ implies $\bot$ by Theorem 1.84, and $\bot$ implies everything.

Conjunction. Assume $\mathrm{sel}(\chi_\varphi, \chi_\psi, 1) = 0$ and argue by cases on χ_φ. If $\chi_\varphi = 0$ the hypothesis is $\chi_\psi = 0$, and the induction hypotheses give φ and ψ. If χ_φ is a successor the hypothesis is $1 = 0$, a contradiction. Conversely, φ and ψ give $\chi_\varphi = 0$ and $\chi_\psi = 0$, hence $\mathrm{sel}(0, \chi_\psi, 1) = \chi_\psi = 0$.

Disjunction. Assume $\mathrm{sel}(\chi_\varphi, 0, \chi_\psi) = 0$ and argue by cases on χ_φ. If it is 0, then φ. If it is a successor, the hypothesis is $\chi_\psi = 0$, hence ψ. Conversely, φ gives $\mathrm{sel}(0, 0, \chi_\psi) = 0$, and ψ gives $\mathrm{sel}(\chi_\varphi, 0, 0) = 0$ by Theorem 1.58(ii).

Implication. Assume $\mathrm{sel}(\chi_\varphi, \chi_\psi, 0) = 0$ and φ. Then $\chi_\varphi = 0$, the hypothesis is $\chi_\psi = 0$, and ψ follows. Conversely assume $\varphi \to \psi$ and argue by cases on χ_φ. If it is 0, then φ, hence ψ, hence $\chi_\psi = 0$ and $\mathrm{sel}(0, \chi_\psi, 0) = 0$. If it is a successor, the selector is 0. □

No excluded middle for arbitrary formulas is used: the case distinctions are on terms.

Theorem 1.95 (Quantifier-free induction)**.** *For every quantifier-free $\varphi(z)$,*

$$\mathsf{T}_{\mathrm{eq}} \vdash \varphi(0) \to \big(\forall z\,(\varphi(z) \to \varphi(Sz)) \to \forall z\,\varphi(z)\big).$$

Equivalently, if $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi(0)$ and $\Gamma, \varphi(z) \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi(Sz)$ with z not free in Γ, then $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi(t)$ for every t. The context Γ may contain quantified formulas.

Proof. Assume the two antecedents. By Theorem 1.94 the first gives $\chi_\varphi(0) = 0$. Under the further assumption $\chi_\varphi(z) = 0$ the proposition gives $\varphi(z)$, the second antecedent gives $\varphi(Sz)$, and the proposition gives $\chi_\varphi(Sz) = 0$. Theorem 1.50(ii) for the equation $\chi_\varphi(z) = 0$ gives $\chi_\varphi(z) = 0$ for all z, and the proposition gives $\varphi(z)$. □

In Lean this is `Teq.qfInd`.

1.7.12 Order, powers of two and pairing

Write $x \le y$ for the equation $x \mathbin{\dot-} y = 0$.

Lemma 1.96 (Order)**.** T_{eq} *proves*

(i) $x \mathbin{\dot-} x = 0$, $(a+b) \mathbin{\dot-} a = b$, $a \mathbin{\dot-} (a+b) = 0$ *and* $S(x+d) \mathbin{\dot-} x = Sd$;

(ii) $\mathrm{sel}(x \mathbin{\dot-} y,\ x + (y \mathbin{\dot-} x),\ y) = y$;

(iii) $x \le y \to x + (y \mathbin{\dot-} x) = y$;

(iv) $x \le y \to x \le y + z$; $\quad Sx \le y \to x \le y$; $\quad x \le y \wedge y \le z \to x \le z$.

Proof. (i) By induction: $0 \dot{-} 0 = 0$ and $Sx \dot{-} Sx = x \dot{-} x$; $(0 + b) \dot{-} 0 = b$ and $(Sa + b) \dot{-} Sa = S(a + b) \dot{-} Sa = (a + b) \dot{-} a$, by induction on a; $a \dot{-} (a + 0) = a \dot{-} a = 0$ and $a \dot{-} (a + Sb) = P(a \dot{-} (a + b)) = P(0) = 0$, by induction on b. The fourth equation is the second for $x + Sd$.

(ii) Induction on y with x fixed. The base $\mathrm{sel}(x, x + (0 \dot{-} x), 0) = 0$ holds by cases on x: at 0 the left side is $0 + (0 \dot{-} 0) = 0$, and at a successor it is 0. For the step write $r := x \dot{-} y$; the induction equation is $\mathrm{sel}(r, x + (y \dot{-} x), y) = y$ and the goal is

$$\mathrm{sel}(P(r),\ x + (Sy \dot{-} x),\ Sy) = Sy.$$

Argue by cases on r. If $r = 0$, the induction equation gives $x + (y \dot{-} x) = y$. By (i), $Sy \dot{-} x = S(x + (y \dot{-} x)) \dot{-} x = S(y \dot{-} x)$, so $x + (Sy \dot{-} x) = S(x + (y \dot{-} x)) = Sy$, which is the goal since $P(r) = 0$. If $r = Sz$, then $P(r) = z$ and Theorem 1.85(iii) gives $y + Sz = x$. Argue by cases on z. If $z = 0$, then $x = Sy$ and the goal is $Sy + (Sy \dot{-} Sy) = Sy$, which holds by (i). If z is a successor, the selector returns Sy.

(iii) is (ii) with $x \dot{-} y = 0$.

(iv) By (iii), $y + z = x + ((y \dot{-} x) + z)$, and (i) applies. If $Sx \le y$, then $y = Sx + d = x + Sd$ with $d := y \dot{-} Sx$, and (i) applies. If $x \le y$ and $y \le z$, then $z = y + (z \dot{-} y)$ and the first part applies. □

Remark 1.97 (The route of the Lean module). The module `TeqOrder` proves (iii) through the maximum identity $x + (y \dot{-} x) = y + (x \dot{-} y)$ instead of (ii): equation induction on y, with base $x + 0 = 0 + x$ by $0 \dot{-} x = 0$, and a step by cases on $x \dot{-} y$ that uses only (i) and Theorem 1.85(iii); then $x \le y$ gives $x + (y \dot{-} x) = y$ at once. Of the transitivity laws in (iv) it needs only $x + y \le k \to x \le k$, and it does not use the monotonicity of 2^x in Theorem 1.98: the logarithm counter of Theorem 1.99 is proved from the invariant "$2^i \le k$ implies $\mathrm{lgc}(i, k) = i$", whose step needs only $2^i \le 2^i + 2^i \le k$.

Lemma 1.98 (Powers of two)**.** T_{eq} *proves* $2^0 = 1$, $2^{Sx} = 2^x + 2^x$, $2^x \le 2^{x+d}$, $x \le y \to 2^x \le 2^y$ *and* $Sx \le 2^x$.

Proof. The first two are computations. $2^x \le 2^{x+d}$ by induction on d: the base is Theorem 1.96(i), and $2^{x+Sd} = 2^{x+d} + 2^{x+d}$, so the step is Theorem 1.96(iv). Monotonicity follows by $y = x + (y \dot{-} x)$. $Sx \le 2^x$ by induction on x: the base is $1 \dot{-} 1 = 0$. In the step let $p := 2^x$ and $d := p \dot{-} Sx$, so that $p = Sx + d$ by the induction equation and Theorem 1.96(iii). Then $p + p = SSx + (x + d + d)$ by Theorem 1.57, and $SSx \le p + p$ by Theorem 1.96(i). □

Put

$$\ell(k, v) := \mathrm{sel}(2^{Sv} \dot{-} k,\ Sv,\ v), \qquad \Lambda(t, k) := I(t, 0;\, v.\, \ell(k, v)), \qquad \mathrm{lg}(w) := \Lambda(w, w),$$
$$\langle a, b\rangle := 2^{a+b} + a, \qquad \mathrm{fst}(w) := w \dot{-} 2^{\mathrm{lg}(w)}, \qquad \mathrm{snd}(w) := \mathrm{lg}(w) \dot{-} \mathrm{fst}(w).$$

The step $\ell(k, v)$ increments v while $2^{v+1} \le k$, so $\Lambda(t, k)$ is a saturating counter for the binary logarithm of k.

Theorem 1.99 (Pairing)**.** T_{eq} *proves* $\mathrm{lg}\langle a, b\rangle = a + b$, $\mathrm{fst}\langle a, b\rangle = a$ *and* $\mathrm{snd}\langle a, b\rangle = b$.

Proof. Fix a, b and put $c := a + b$ and $k := 2^c + a$.

Below the exponent the counter follows its index: $\mathrm{sel}(i \dot{-} c,\ \Lambda(i, k),\ i) = i$, by induction on i. The base is $\mathrm{sel}(0 \dot{-} c, 0, 0) = 0$. In the step argue by cases on $Si \dot{-} c$. If it is a successor, the goal $\mathrm{sel}(Si \dot{-} c, \Lambda(Si, k), Si) = Si$ is a computation. If it is 0, then $Si \le c$, hence $i \le c$ by Theorem 1.96(iv), and the induction equation gives $\Lambda(i, k) = i$. Moreover $2^{Si} \le 2^c \le k$ by

Theorems 1.96 and 1.98, so $\Lambda(Si,k) = \ell(k,i) = \mathrm{sel}(0,Si,i) = Si$, which is the goal. At $i = c$ this gives

$$\Lambda(c,k) = c. \tag{1.19}$$

At the exponent the counter stops: $\ell(k,c) = c$. By Theorem 1.98, $2^c = Sc + d$ with $d := 2^c \dot{-} Sc$, hence $2^{Sc} = 2^c + 2^c = (2^c + a) + S(b+d)$ and $2^{Sc} \dot{-} k = S(b+d)$ by Theorem 1.96(i). The selector in $\ell(k,c)$ therefore returns c.

Beyond the exponent: $\Lambda(c+j,k) = c$, by induction on j. The base is (1.19), and $\Lambda(S(c+j),k) = \ell(k,\Lambda(c+j,k)) = \ell(k,c) = c$.

Conclusion. $Sc \le 2^c$ gives $c \le 2^c \le k$, so $k = c + (k \dot{-} c)$ and $\lg(k) = \Lambda(k,k) = c$. Then $\mathrm{fst}(k) = (2^c + a) \dot{-} 2^c = a$ and $\mathrm{snd}(k) = (a+b) \dot{-} a = b$ by Theorem 1.96(i). □

Surjectivity of the pairing is not needed and not claimed. The numerical domain is never replaced by the range of the coding; a pair only stores the state of one recursion.

Remark 1.100 (The pairing of the reference statement). The reference statement filed with `certified_zero` uses the pairing $(a+b)^2 + a$, with multiplication $a \cdot b := I(b,0;\, u.\, u + a)$ and a bounded search for the integer square root, $q(n) := I(n,0;\, v.\, \mathrm{sel}((Sv)^2 \dot{-} n, Sv, v))$, whose invariants are proved by quantifier-free induction; it appeals to "the usual linear-order and semiring identities" without deriving them. The pairing above is the one of the Lean modules `TeqOrder` and `TeqPair`. It needs no multiplication, and all its inductions are on literal equations, so Theorem 1.95 is not used for it. Either pairing serves the next theorem.

1.7.13 Primitive recursion

Definition 1.101 (The derived recursor)**.** Let a be a term and $g(y,v)$ a term with two distinguished variables. With a fresh variable w put

$$s_g(w) := \langle S\,\mathrm{fst}(w),\ g(\mathrm{fst}(w),\mathrm{snd}(w))\rangle, \qquad R(c,a;\, y\,v.\, g) := \mathrm{snd}\big(I(c,\langle 0,a\rangle;\, w.\, s_g(w))\big).$$

The recursor binds y and v in g.

The iterator carries the pair of the counter and the value. For a function symbol f of iPRA of arity k the term $\ulcorner f \urcorner$, with free variables among the fixed variables $x_1,\dots,x_k$, is defined by recursion on f; $\ulcorner f \urcorner(t_1,\dots,t_k)$ abbreviates $\ulcorner f \urcorner[\vec{t}/\vec{x}]$.

$$\ulcorner \mathrm{zero} \urcorner := 0, \qquad \ulcorner \mathrm{succ} \urcorner := Sx_1, \qquad \ulcorner \pi_i^k \urcorner := x_i, \qquad \ulcorner \mathrm{comp}(f;\vec{g}) \urcorner := \ulcorner f \urcorner(\ulcorner g_1 \urcorner,\dots,\ulcorner g_m \urcorner),$$
$$\ulcorner \mathrm{prec}(f,g) \urcorner := R\big(x_1,\ \ulcorner f \urcorner(x_2,\dots,x_{k+1});\ y\,v.\, \ulcorner g \urcorner(y,v,x_2,\dots,x_{k+1})\big).$$

The translation σ of the terms of iPRA into the terms of T_{eq} is

$$\sigma(x) := x, \qquad \sigma\big(f(t_1,\dots,t_k)\big) := \ulcorner f \urcorner(\sigma(t_1),\dots,\sigma(t_k)).$$

It commutes with substitution literally, $\sigma(t[\theta]) = \sigma(t)[\sigma \circ \theta]$, and $\sigma(0) = 0$, $\sigma(St) = S\sigma(t)$.

Theorem 1.102 (Primitive recursion)**.** (i) T_{eq} *proves* $R(0,a;\, y\,v.\, g) = a$ *and* $R(Sc,a;\, y\,v.\, g) = g\big(c,\ R(c,a;\, y\,v.\, g)\big)$.

(ii) *For every defining equation* $s = t$ *of a function symbol of* iPRA *(Theorem 1.52),* $\mathsf{T}_{\mathrm{eq}} \vdash \sigma(s) = \sigma(t)$.

Proof. (i) Write $A(c) := I(c,\langle 0,a\rangle;\, w.\, s_g(w))$. By induction on c, $\mathrm{fst}(A(c)) = c$: the base is $\mathrm{fst}\langle 0,a\rangle = 0$, and $\mathrm{fst}(A(Sc)) = \mathrm{fst}(s_g(A(c))) = S\,\mathrm{fst}(A(c)) = Sc$, by Theorem 1.99. Then $R(0,a;\, y\,v.\, g) = \mathrm{snd}\langle 0,a\rangle = a$, and

$$R(Sc,a;\, y\,v.\, g) = \mathrm{snd}\big(s_g(A(c))\big) = g\big(\mathrm{fst}(A(c)),\mathrm{snd}(A(c))\big) = g\big(c,\ R(c,a;\, y\,v.\, g)\big).$$

(ii) For a projection, $\sigma(\pi_i^k(\vec{x}))$ is the term x_i. For a composition, $\sigma(\mathrm{comp}(f;\vec{g})(\vec{x}))$ and $\sigma(f(g_1(\vec{x}),\dots,g_m(\vec{x})))$ are the same term. For a recursion the two equations are (i) for the seed $\ulcorner f \urcorner(\vec{x})$ and the step $\ulcorner g \urcorner(y,v,\vec{x})$. □

The construction is an external recursion on each finite description. It is not an internal quantification over functions, and no universal evaluator is introduced. In Lean the recursor is the content of the module `TeqRec`.

1.8 The bridge

This section draws on the reference statements filed with `certified_zero` (the inherited basis, the inherited ordinary bridge and the section on primitive recursion) and follows the Lean modules `NativeTranslate`, `NativeComposite`, `NativeRoundTrip`, `ScalarToPRA`, `PRAToScalar`, `RoundTripScalar` and `RoundTripPRA` in the organization of the proofs. The bridge has two spans: between N and T_{eq} the terms are the same and the formulas are translated; between T_{eq} and iPRA the logic is the same and the terms are translated.

1.8.1 The native core and T_{eq}

Definition 1.103 (Native reading of an ordinary formula)**.** For an ordinary formula φ over the terms of Theorem 1.1, the native formula φ^* replaces every connective by its native abbreviation (Theorem 1.28):

$$(s=t)^* := (s=t), \qquad \bot^* := \bot_*, \qquad (\varphi\wedge\psi)^* := \varphi^* \wedge_* \psi^*, \qquad (\varphi\vee\psi)^* := \varphi^* \vee_* \psi^*,$$
$$(\varphi\to\psi)^* := \varphi^* \Rightarrow_* \psi^*, \qquad (\forall x\,\varphi)^* := \forall_* x\,\varphi^*, \qquad (\exists x\,\varphi)^* := \exists_* x\,\varphi^*.$$

By (1.6), $(\varphi[\theta])^* = \varphi^*[\theta]$.

Proposition 1.104 (Soundness of the ordinary reading)**.** *If* $\Gamma \vdash_{\mathsf{N}} A$*, then* $\Gamma^\circ \vdash_{\mathsf{T}_{\mathrm{eq}}} A^\circ$.

Proof. By induction on the derivation, using (1.3) for the instances. HYP is hypothesis; I0 and IS are (T1). TR: from $s=t$ and $h(s)=r$ the equality rules give $r=h(t)$. QE: the premises read $B^\circ(0)\to\forall y\,B^\circ$ and $B^\circ(0)$, which give $B^\circ(t)$. DE: the first premise reads $\exists x\,(B^\circ(x,0)\to\forall y\,B^\circ(x,y))$ and the hypothesis $Q_y B(b,y)$ of the second reads $B^\circ(b,0)\to\forall y\,B^\circ(b,y)$, so the rule is existential elimination. DI: the premise reads $\Gamma^\circ, B^\circ(t,0)\vdash B^\circ(t,Sz)$. Over $\Gamma^\circ, B^\circ(t,0)$, Theorem 1.50(i) for the formula $B^\circ(t,y)$ gives $\forall y\,B^\circ(t,y)$; discharge $B^\circ(t,0)$ and introduce the existential quantifier at t. QI: the premise reads $\Gamma^\circ, E(z)\vdash E(Sz)$; over $\Gamma^\circ, E(0)$, Theorem 1.50(ii) gives $E(y)$ for a fresh y; generalize and discharge $E(0)$. □

Coverage is used for DI only and equation induction for QI only.

Proposition 1.105 (Completeness of the native reading)**.** *If* $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi$*, then* $\Gamma^* \vdash_{\mathsf{N}} \varphi^*$.

Proof. By induction on the derivation. Every logical rule of Theorem 1.48 for a connective translates into the rule of the same name in Theorem 1.29; the translation commutes with the substitutions in the quantifier rules. Reflexivity is Theorem 1.18(i), and the Leibniz rule is Theorem 1.22. It remains to derive the native reading of each axiom in the empty context; by Theorem 1.10 all substitution instances then follow in every context.

(T1) The readings are the conclusions of I0 and IS.

(T2) Apply Theorem 1.14 to the formula $C(y) := (y=0)\vee_*\exists_* v\,(y=Sv)$. $C(0)$ follows from $0=0$ by $\vee_*$I. $C(Sz)$ follows from $Sz=Sz$ by $\exists_*$I with the witness z and $\vee_*$I. Hence $\vdash C(x)$.

(T3) The reading is $(0=S0)\Rightarrow_*(0=S0)$, which is $\Rightarrow_*$I applied to HYP.

(T4) Let $U := \forall_* z\,(E(z) \Rightarrow_* E(Sz))$. By $\Rightarrow_*$I twice it suffices to derive $E(0), U \vdash E(0) \wedge_* Q_z E(z)$, the conclusion being $\forall_* z\, E(z)$ by definition. The first component is a hypothesis. For the second apply QI: over $E(0), U, E(z)$ with z fresh, $\forall_*$E gives $E(z) \Rightarrow_* E(Sz)$ and $\Rightarrow_*$E gives $E(Sz)$. □

Proposition 1.106 (The two formula round trips)**.** (i) *For every native formula A: $(A^\circ)^* \dashv\vdash A$ in* N.

(ii) *For every ordinary formula φ: $\mathsf{T}_{\mathrm{eq}} \vdash \varphi \leftrightarrow (\varphi^*)^\circ$.*

Proof. (i) By induction on A. An equation is its own round trip.

Let A be $Q_y B$ and put $C := (B^\circ)^*$, so that $C \dashv\vdash B$ by the induction hypothesis and $(A^\circ)^* = C(0) \Rightarrow_* \forall_* y\, C$. From this formula derive $Q_y B$ by Theorem 1.13: under the assumption $B(0)$ one has $C(0)$, hence $\forall_* y\, C$, hence $C(Sz)$ by $\forall_*$E, hence $B(Sz)$. Conversely, from $Q_y B$ and the assumption $C(0)$ one has $B(0)$, hence $B(y)$ for a fresh y by QE, hence $C(y)$; $\forall_*$I and $\Rightarrow_*$I give $(A^\circ)^*$.

Let A be $D_{x,y} B$. Then $(A^\circ)^* = \exists_* x\, Y$, where $Y := ((Q_y B)^\circ)^*$ and $Y \dashv\vdash Q_y B$ by the previous case, with x as a parameter. From $\exists_* x\, Y$: $\exists_*$E at a fresh b gives $Y(b)$, hence $Q_y B(b, y)$, and Theorem 1.15 gives $D_{x,y} B$. From $D_{x,y} B$: DE at a fresh b gives $Q_y B(b, y)$, hence $Y(b)$, and $\exists_*$I gives $\exists_* x\, Y$.

(ii) A derivation in N can be used inside T_{eq}: if $\Delta \vdash_{\mathsf{N}} A$ and $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} B^\circ$ for every $B \in \Delta$, then $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} A^\circ$, by Theorem 1.104 and cut. This is called *transfer* below. The following equivalences are provable in T_{eq} for all native formulas X, Y.

(a) $(\bot_*)^\circ \leftrightarrow \bot$: by (T3) and $\bot$E.

(b) $(X \wedge_* Y)^\circ \leftrightarrow X^\circ \wedge Y^\circ$: by transfer of $\wedge_*$E and of $\wedge_*$I.

(c) $(\exists_* x\, X)^\circ \leftrightarrow \exists x\, X^\circ$: the left side is $\exists x\, (Q_y \,\mathrm{gate}(y, X))^\circ$, and $(Q_y \,\mathrm{gate}(y, X))^\circ \leftrightarrow X^\circ$ by transfer of Theorem 1.26(iii).

(d) $(X \Rightarrow_* Y)^\circ \leftrightarrow (X^\circ \to Y^\circ)$: from left to right by transfer of $\Rightarrow_*$E. From right to left, the goal $(X \Rightarrow_* Y)^\circ$ is $\mathrm{sel}(0, X, Y)^\circ \to \forall y\, \mathrm{sel}(y, X, Y)^\circ$. Assume $\mathrm{sel}(0, X, Y)^\circ$. Transfer of Theorem 1.26(iv) gives X°, hence Y°. Transfer of $Y \vdash X \Rightarrow_* Y$ gives $(X \Rightarrow_* Y)^\circ$, and its antecedent is the assumption.

(e) $(X \vee_* Y)^\circ \leftrightarrow X^\circ \vee Y^\circ$: by (c) the left side is equivalent to $\exists x\, \mathrm{sel}(x, X, Y)^\circ$. It suffices to derive $\forall x\, (\mathrm{sel}(x, X, Y)^\circ \to X^\circ \vee Y^\circ)$, and by Theorem 1.50(i) its two instances at 0 and at Sz suffice. They are transfers of $\mathrm{sel}(0, X, Y) \vdash X$ and $\mathrm{sel}(Sz, X, Y) \vdash Y$, which hold by Theorem 1.26(iv), (v), followed by $\vee$I. No coverage of the witness x and no transport along $x = 0$ or $x = Sz$ is needed; this is the route of the Lean module `NativeRoundTrip`. The converse is transfer of $\vee_*$I.

(f) $(\forall_* x\, X)^\circ \leftrightarrow \forall x\, X^\circ$: transfer of $\wedge_*$E gives $X^\circ(0)$ and $(Q_x X)^\circ = X^\circ(0) \to \forall x\, X^\circ$ from the left side, hence the right side. Conversely $\forall x\, X^\circ$ gives $X^\circ(0)$ and $(Q_x X)^\circ$, and transfer of $\wedge_*$I gives the left side.

The claim follows by induction on φ, because provable equivalence is a congruence for the connectives and quantifiers of the logic. □

Corollary 1.107 (N and T_{eq} prove the same)**.** $\Gamma \vdash_{\mathsf{N}} A$ *if and only if* $\Gamma^\circ \vdash_{\mathsf{T}_{\mathrm{eq}}} A^\circ$, *and* $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi$ *if and only if* $\Gamma^* \vdash_{\mathsf{N}} \varphi^*$.

Proof. If $\Gamma^\circ \vdash_{\mathsf{T}_{\mathrm{eq}}} A^\circ$, then $(\Gamma^\circ)^* \vdash_{\mathsf{N}} (A^\circ)^*$ by Theorem 1.105, and Theorem 1.106(i) with cut gives $\Gamma \vdash_{\mathsf{N}} A$. If $\Gamma^* \vdash_{\mathsf{N}} \varphi^*$, then $(\Gamma^*)^\circ \vdash_{\mathsf{T}_{\mathrm{eq}}} (\varphi^*)^\circ$ by Theorem 1.104, and Theorem 1.106(ii) gives $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi$. The other two implications are the two propositions. □

In Lean these results are `NPrf.sound`, `NPrf.ofPrf`, `NEquiv.ofFm_toFm`, `Teq.iff_toFm_ofFm`, `NPrf.iff_teq` and `Teq.iff_native`.

1.8.2 Term interpretations

Definition 1.108 (Term interpretation)**.** Let T and T' be theories in intuitionistic logic with equality over term languages $\mathcal{L}$ and $\mathcal{L}'$. A *term interpretation* of T in T' is a map F from the terms of $\mathcal{L}$ to the terms of $\mathcal{L}'$ such that

(i) $F(x) = x$ for every variable x, and $\mathrm{FV}(F(t)) \subseteq \mathrm{FV}(t)$;

(ii) $T' \vdash F(t[\theta]) = F(t)[F \circ \theta]$ for every term t and substitution θ;

(iii) $T' \vdash \varphi^F$ for every axiom φ of T,

where φ^F is φ with each equation $s = t$ replaced by $F(s) = F(t)$.

A term interpretation does not relativize quantifiers and reads equality as equality: its domain is the whole domain.

Lemma 1.109 (Term interpretations preserve derivability)**.** *Let F be a term interpretation of T in T'.*

(i) $T' \vdash (\varphi[\theta])^F \leftrightarrow \varphi^F[F \circ \theta]$.

(ii) *If* $\Gamma \vdash_T \varphi$, *then* $\Gamma^F \vdash_{T'} \varphi^F$.

Proof. (i) By induction on φ. For an equation the two sides are $F(s[\theta]) = F(t[\theta])$ and $F(s)[F \circ \theta] = F(t)[F \circ \theta]$, which are equivalent by condition (ii) of the definition. The connectives and quantifiers are handled by the congruence of provable equivalence, the bound variables being fixed by θ and by F. (ii) By induction on the derivation. The propositional rules, $\forall$I and $\exists$E translate literally. For $\forall$E the premise gives $\forall x\, \varphi^F$, hence $\varphi^F[F(t)/x]$, which is equivalent to $(\varphi[t/x])^F$ by (i); $\exists$I is similar. Reflexivity is reflexivity. For the Leibniz rule the premises give $F(s_i) = F(t_i)$ and, by (i), $\varphi^F[F(\vec{s})/\vec{x}]$; the Leibniz rule of T' and (i) give $(\varphi[\vec{t}/\vec{x}])^F$. An instance $\varphi[\theta]$ of an axiom translates to a formula equivalent to $\varphi^F[F \circ \theta]$, an instance of a theorem of T'. □

Lemma 1.110 (Round trips give faithfulness)**.** *Let F be a term interpretation of T in T' and G one of T' in T, with $T \vdash G(F(t)) = t$ and $T' \vdash F(G(t')) = t'$ for all terms. Then*

(i) $T \vdash \varphi \leftrightarrow (\varphi^F)^G$ *and* $T' \vdash \psi \leftrightarrow (\psi^G)^F$;

(ii) $\Gamma \vdash_T \varphi$ *if and only if* $\Gamma^F \vdash_{T'} \varphi^F$, *and* $\Gamma \vdash_{T'} \psi$ *if and only if* $\Gamma^G \vdash_T \psi^G$.

Proof. (i) By induction on the formula; for an equation $s = t$ the round trip is $G(F(s)) = G(F(t))$, equivalent to $s = t$ by the hypothesis. (ii) One direction is Theorem 1.109. For the other, if $\Gamma^F \vdash_{T'} \varphi^F$ then $(\Gamma^F)^G \vdash_T (\varphi^F)^G$, and (i) gives $\Gamma \vdash_T \varphi$. □

1.8.3 From T_{eq} to iPRA

A term of iPRA has no binder. An iterator term is therefore compiled into a function symbol of its free variables, and the iterator becomes a primitive recursion whose step ignores the counter. For a symbol A of arity k and a symbol H of arity $k+1$ put

$$\iota(A, H) := \mathrm{prec}\big(A,\ \mathrm{comp}(H; \pi_2^{k+2}, \pi_3^{k+2}, \ldots, \pi_{k+2}^{k+2})\big),$$

a symbol of arity $k+1$. By the axioms for recursion, composition and projection,

$$\mathsf{iPRA} \vdash \iota(A,H)(0,\vec{x}) = A(\vec{x}), \qquad \mathsf{iPRA} \vdash \iota(A,H)(Sy,\vec{x}) = H\big(\iota(A,H)(y,\vec{x}),\vec{x}\big). \tag{1.20}$$

Definition 1.111 (The translation τ)**.** Fix an enumeration of the variables. For a list $\vec{x} = x_1, \dots, x_k$ of distinct variables and a term t with $\mathrm{FV}(t) \subseteq \vec{x}$, the symbol $[t]_{\vec{x}}$ of arity k is

$$[x_i]_{\vec{x}} := \pi^k_i, \qquad [0]_{\vec{x}} := \mathrm{comp}(\mathrm{zero};\,), \qquad [St]_{\vec{x}} := \mathrm{comp}(\mathrm{succ}; [t]_{\vec{x}}),$$
$$[I(c,a;\,u.\,h)]_{\vec{x}} := \mathrm{comp}\big(\iota([a]_{\vec{x}}, [h]_{u\vec{x}});\ [c]_{\vec{x}}, \pi^k_1, \dots, \pi^k_k\big) \qquad (u \notin \vec{x}).$$

The translation of terms is

$$\tau(x) := x, \qquad \tau(0) := 0, \qquad \tau(St) := S\tau(t), \qquad \tau(I(c,a;\,u.\,h)) := \iota\big([a]_{\vec{y}}, [h]_{u\vec{y}}\big)\big(\tau(c), \vec{y}\big),$$

where $\vec{y}$ lists the variables of $\mathrm{FV}(a) \cup (\mathrm{FV}(h) \setminus \{u\})$ in the order of the enumeration.

The translation does not commute with substitution literally: a substitution into the body of an iterator changes the symbol. It commutes provably.

Lemma 1.112 (Extensionality of the simulated iterator)**.** *Let $\vec{z}$ be a list of distinct variables, $\vec{s}$ a list of terms, and u a variable that is not in $\vec{z}$ and not free in $\vec{s}$. If* $\mathsf{iPRA} \vdash A'(\vec{z}) = A(\vec{s})$ *and* $\mathsf{iPRA} \vdash H'(u, \vec{z}) = H(u, \vec{s})$*, then* $\mathsf{iPRA} \vdash \iota(A', H')(t, \vec{z}) = \iota(A, H)(t, \vec{s})$ *for every term t.*

Proof. By quantifier-free induction on a fresh variable n in the equation $\iota(A', H')(n, \vec{z}) = \iota(A, H)(n, \vec{s})$. The base is $A'(\vec{z}) = A(\vec{s})$ by (1.20). In the step,

$$\begin{aligned}\iota(A', H')(Sn, \vec{z}) &= H'\big(\iota(A', H')(n, \vec{z}), \vec{z}\big) = H\big(\iota(A', H')(n, \vec{z}), \vec{s}\big)\\ &= H\big(\iota(A, H)(n, \vec{s}), \vec{s}\big) = \iota(A, H)(Sn, \vec{s}),\end{aligned}$$

by (1.20), the second hypothesis instantiated at $u = \iota(A', H')(n, \vec{z})$, and the induction equation. Instantiate n at t. □

Lemma 1.113 (Evaluation and substitution)**.** (i) *If* $\mathrm{FV}(t) \subseteq \vec{x}$*, then* $\mathsf{iPRA} \vdash [t]_{\vec{x}}(\vec{r}) = \tau(t)[\vec{r}/\vec{x}]$ *for all terms $\vec{r}$. In particular* $\mathsf{iPRA} \vdash [t]_{\vec{x}}(\vec{x}) = \tau(t)$*.*

(ii) $\mathsf{iPRA} \vdash \tau(t[\theta]) = \tau(t)[\tau \circ \theta]$ *for every substitution θ.*

Proof. (i) By induction on t, for all lists $\vec{x}$. For a variable, 0 and a successor the claim follows from the axioms for projection and composition and the induction hypothesis. Let t be $I(c,a;\,u.\,h)$, and let $\vec{y}$ be the list of Theorem 1.111, a sublist of $\vec{x}$. By the induction hypothesis for a and h with variables as arguments, $[a]_{\vec{x}}(\vec{x}) = \tau(a) = [a]_{\vec{y}}(\vec{y})$ and $[h]_{u\vec{x}}(u, \vec{x}) = \tau(h) = [h]_{u\vec{y}}(u, \vec{y})$. By Theorem 1.112,

$$\iota\big([a]_{\vec{x}}, [h]_{u\vec{x}}\big)(n, \vec{x}) = \iota\big([a]_{\vec{y}}, [h]_{u\vec{y}}\big)(n, \vec{y})$$

for a fresh variable n. By the axioms for composition and projection and the induction hypothesis for c, $[t]_{\vec{x}}(\vec{r}) = \iota([a]_{\vec{x}}, [h]_{u\vec{x}})(\tau(c)[\vec{r}/\vec{x}], \vec{r})$. Substituting $\vec{r}$ for $\vec{x}$ and $\tau(c)[\vec{r}/\vec{x}]$ for n in the displayed equation identifies this term with $\tau(t)[\vec{r}/\vec{x}]$.

(ii) By induction on t. The cases other than the iterator are immediate. Let t be $I(c,a;\,u.\,h)$ with u fixed by θ and not free in its values, so that $t[\theta] = I(c[\theta], a[\theta];\,u.\,h[\theta])$. Let $\vec{y}$ be the list for a, h and $\vec{z}$ the list for $a[\theta]$, $h[\theta]$. By (i) and the induction hypothesis,

$$\begin{aligned}[a[\theta]]_{\vec{z}}(\vec{z}) &= \tau(a[\theta]) = \tau(a)[\tau \circ \theta] = [a]_{\vec{y}}\big(\tau(\theta(\vec{y}))\big),\\ [h[\theta]]_{u\vec{z}}(u, \vec{z}) &= \tau(h[\theta]) = \tau(h)[\tau \circ \theta] = [h]_{u\vec{y}}\big(u, \tau(\theta(\vec{y}))\big).\end{aligned}$$

Theorem 1.112 with $\vec{s} := \tau(\theta(\vec{y}))$ and the induction hypothesis for c give

$$\begin{aligned}\tau(t[\theta]) &= \iota\big([a[\theta]]_{\vec{z}}, [h[\theta]]_{u\vec{z}}\big)\big(\tau(c[\theta]), \vec{z}\big)\\ &= \iota\big([a]_{\vec{y}}, [h]_{u\vec{y}}\big)\big(\tau(c)[\tau \circ \theta], \tau(\theta(\vec{y}))\big) = \tau(t)[\tau \circ \theta].\end{aligned}$$

□

Proposition 1.114 (T_{eq} in iPRA)**.** *τ is a term interpretation of T_{eq} in* iPRA*.*

Proof. Conditions (i) and (ii) of Theorem 1.108 are the definition and Theorem 1.113(ii). The axioms:

(T1) With the notation of Theorem 1.111, $\tau(I(0,a;\,u.\,h)) = \iota(\dots)(0,\vec{y}) = [a]_{\vec{y}}(\vec{y}) = \tau(a)$ by (1.20) and Theorem 1.113(i). Writing $q := \tau(I(n,a;\,u.\,h))$,

$$\tau(I(Sn,a;\,u.\,h)) = \iota(\dots)(S\tau(n),\vec{y}) = [h]_{u\vec{y}}(q,\vec{y}) = \tau(h)[q/u] = \tau\big(h[I(n,a;\,u.\,h)/u]\big),$$

by (1.20) and both parts of Theorem 1.113.

(T2) Let $\mathrm{pd} := \mathrm{prec}(\mathrm{zero},\pi_1^2)$, so that $\mathrm{pd}(0)=0$ and $\mathrm{pd}(Sy)=y$. Quantifier-free induction on x proves $x = 0 \vee x = S\mathrm{pd}(x)$: the base is the left disjunct, and the step is the right disjunct, since $Sx = S\mathrm{pd}(Sx)$. Existential introduction gives the image of (T2).

(T3) is the axiom $Sx = 0 \to \bot$ at $x = 0$, with symmetry.

(T4) For an equation $E(z)$, written $s = t$, the image of the axiom is

$$\tau(s[0/z]) = \tau(t[0/z]) \to \Big(\forall z\,(\tau(s)=\tau(t) \to \tau(s[Sz/z]) = \tau(t[Sz/z])) \to \forall z\,\tau(s) = \tau(t)\Big),$$

which by Theorem 1.113(ii) is equivalent to the induction axiom of iPRA for the equation $\tau(s) = \tau(t)$. □

1.8.4 From iPRA to T_{eq}

Proposition 1.115 (iPRA in T_{eq})**.** *The translation σ of Section 1.7 is a term interpretation of* iPRA *in T_{eq}.*

Proof. σ fixes variables and commutes with substitution literally. The defining equations hold by Theorem 1.102(ii). The images of $Sx = 0 \to \bot$ and $Sx = Sy \to x = y$ are those formulas themselves, which hold by Theorem 1.84. For a quantifier-free φ the formula φ^σ is quantifier free and $(\varphi[t/z])^\sigma = \varphi^\sigma[\sigma(t)/z]$, so the image of the induction axiom for φ is the instance of Theorem 1.95 for φ^σ. □

1.8.5 The two round trips on terms

Lemma 1.116 (A recursion that ignores its counter is the iterator)**.** *If y is not free in $h(v)$, then $\mathsf{T}_{\mathrm{eq}} \vdash R(c,a;\,y\,v.\,h(v)) = I(c,a;\,u.\,h(u))$.*

Proof. Theorem 1.55 for $f(z) := R(z,a;\,y\,v.\,h(v))$: by Theorem 1.102(i), $f(0) = a$ and $f(Sz) = h(f(z))$. □

Proposition 1.117 (Round trips on terms)**.** (i) *$\mathsf{T}_{\mathrm{eq}} \vdash \sigma(\tau(t)) = t$ for every term t of T_{eq}.*

(ii) *iPRA $\vdash \tau(\sigma(t)) = t$ for every term t of* iPRA*.*

Proof. (i) By the definition of $\ulcorner\cdot\urcorner$ the term $\ulcorner\mathrm{comp}(H;\pi_2^{k+2},\dots,\pi_{k+2}^{k+2})\urcorner(y,v,\vec{x})$ is $\ulcorner H\urcorner(v,\vec{x})$, so by Theorem 1.116

$$\mathsf{T}_{\mathrm{eq}} \vdash \ulcorner\iota(A,H)\urcorner(c,\vec{x}) = I(c,\ulcorner A\urcorner(\vec{x});\,u.\ulcorner H\urcorner(u,\vec{x})). \tag{1.21}$$

First, $\mathsf{T}_{\mathrm{eq}} \vdash \ulcorner[t]_{\vec{x}}\urcorner(\vec{x}) = t$, by induction on t. For a variable, 0 and a successor both sides are the same term up to the induction hypothesis. For an iterator,

$$\begin{aligned}\ulcorner[I(c,a;\,u.\,h)]_{\vec{x}}\urcorner(\vec{x}) &= \ulcorner\iota([a]_{\vec{x}},[h]_{u\vec{x}})\urcorner(\ulcorner[c]_{\vec{x}}\urcorner(\vec{x}),\vec{x})\\ &= I(\ulcorner[c]_{\vec{x}}\urcorner(\vec{x}),\ulcorner[a]_{\vec{x}}\urcorner(\vec{x});\,u.\ulcorner[h]_{u\vec{x}}\urcorner(u,\vec{x})),\end{aligned}$$

by (1.21), and the last term equals $I(c,a;\,u.\,h)$ by the induction hypothesis, congruence in the counter and the seed, and Theorem 1.56(ii) for the body. Now the claim follows by induction on t; for an iterator, with the notation of Theorem 1.111,

$$\begin{aligned}\sigma(\tau(I(c,a;\,u.\,h))) &= \ulcorner\iota([a]_{\vec{y}},[h]_{u\vec{y}})\urcorner(\sigma(\tau(c)),\vec{y})\\ &= I(\sigma(\tau(c)),\ulcorner[a]_{\vec{y}}\urcorner(\vec{y});\,u.\ulcorner[h]_{u\vec{y}}\urcorner(u,\vec{y})) = I(c,a;\,u.\,h).\end{aligned}$$

(ii) First, $\mathsf{iPRA}\vdash\tau(\ulcorner f\urcorner)=f(x_1,\dots,x_k)$ for every symbol f of arity k, by induction on f. For zero and succ the two sides are the same term, and for a projection the claim is its axiom. For a composition, by Theorem 1.113(ii) and the induction hypotheses,

$$\tau\big(\ulcorner f\urcorner(\ulcorner g_1\urcorner,\dots,\ulcorner g_m\urcorner)\big)=\tau(\ulcorner f\urcorner)[\tau(\ulcorner\vec{g}\urcorner)/\vec{x}]=f(g_1(\vec{x}),\dots,g_m(\vec{x}))=\mathrm{comp}(f;\vec{g})(\vec{x}).$$

For a recursion $p:=\mathrm{prec}(f,g)$ write $q:=\tau(\ulcorner p\urcorner)$ and $\vec{x}':=x_2,\dots,x_{k+1}$. By Theorem 1.102(i), T_{eq} proves $\ulcorner p\urcorner[0/x_1]=\ulcorner f\urcorner(\vec{x}')$ and $\ulcorner p\urcorner[Sx_1/x_1]=\ulcorner g\urcorner(x_1,\ulcorner p\urcorner,\vec{x}')$. By Theorem 1.114, Theorem 1.109 and Theorem 1.113(ii), iPRA proves the images of these equations, and with the induction hypotheses for f and g they read

$$q[0/x_1]=f(\vec{x}'),\qquad q[Sx_1/x_1]=g(x_1,q,\vec{x}').$$

Quantifier-free induction on x_1 in the equation $q=p(x_1,\vec{x}')$ now succeeds: the base is $q[0/x_1]=f(\vec{x}')=p(0,\vec{x}')$, and under the induction equation $q[Sx_1/x_1]=g(x_1,q,\vec{x}')=g(x_1,p(x_1,\vec{x}'),\vec{x}')=p(Sx_1,\vec{x}')$.

The claim follows by induction on t: $\tau(\sigma(f(\vec{t})))=\tau(\ulcorner f\urcorner)[\tau(\sigma(\vec{t}))/\vec{x}]=f(\tau(\sigma(\vec{t})))=f(\vec{t})$, by Theorem 1.113(ii), the first step and the induction hypothesis. □

Remark 1.118 (The route of the Lean modules). The modules `ScalarToPRA` and `RoundTripScalar` compile terms of iPRA rather than iterator terms. For a term p of iPRA the symbol $\langle p\rangle_{\vec{x}}$ is π^k_i at the variable x_i and $\mathrm{comp}(f;\langle p_1\rangle_{\vec{x}},\dots,\langle p_m\rangle_{\vec{x}})$ at $f(p_1,\dots,p_m)$, and the iterator case of τ reads $\iota(\langle\tau(a)\rangle_{\vec{y}},\langle\tau(h)\rangle_{u\vec{y}})(\tau(c),\vec{y})$. Then $\langle p\rangle_{\vec{x}}(\vec{s})=p[\vec{s}/\vec{x}]$ is a fact about iPRA alone, proved by an induction with two cases, so Theorem 1.113(i) needs no induction on iterator terms; the hypotheses of Theorem 1.112 are equations between terms; and $\sigma(\langle p\rangle_{\vec{x}})$ coincides syntactically with $\sigma(p)$, so the first step of the proof of Theorem 1.117(i) disappears and $\sigma(\tau(t))=t$ is one induction on t. The recursion equations of the code of $\mathrm{prec}(f,g)$ are proved once, from those of the recursor of Theorem 1.102, and the translations of the two defining axioms of $\mathrm{prec}(f,g)$ are derived from them.

1.8.6 The bi-interpretation theorem

Theorem 1.119 (Bi-interpretation)**.** *The systems* N, T_{eq} *and* iPRA *are pairwise bi-interpretable. Precisely:*

(i) N and T_{eq}. $\Gamma\vdash_{\mathsf{N}}A$ *if and only if* $\Gamma^\circ\vdash_{\mathsf{T}_{\mathrm{eq}}}A^\circ$; $\Gamma\vdash_{\mathsf{T}_{\mathrm{eq}}}\varphi$ *if and only if* $\Gamma^*\vdash_{\mathsf{N}}\varphi^*$; $(A^\circ)^*\dashv\vdash A$ *in* N; *and* $\mathsf{T}_{\mathrm{eq}}\vdash\varphi\leftrightarrow(\varphi^*)^\circ$.

(ii) T_{eq} and iPRA. τ *and* σ *are term interpretations in opposite directions;* $\mathsf{T}_{\mathrm{eq}}\vdash\sigma(\tau(t))=t$ *and* $\mathsf{iPRA}\vdash\tau(\sigma(t))=t$; $\Gamma\vdash_{\mathsf{T}_{\mathrm{eq}}}\varphi$ *if and only if* $\Gamma^\tau\vdash_{\mathsf{iPRA}}\varphi^\tau$; $\Gamma\vdash_{\mathsf{iPRA}}\psi$ *if and only if* $\Gamma^\sigma\vdash_{\mathsf{T}_{\mathrm{eq}}}\psi^\sigma$; $\mathsf{T}_{\mathrm{eq}}\vdash\varphi\leftrightarrow(\varphi^\tau)^\sigma$; *and* $\mathsf{iPRA}\vdash\psi\leftrightarrow(\psi^\sigma)^\tau$.

(iii) N and iPRA. *With* $A^\flat:=(A^\circ)^\tau$ *and* $\psi^\sharp:=(\psi^\sigma)^*$: $\Gamma\vdash_{\mathsf{N}}A$ *if and only if* $\Gamma^\flat\vdash_{\mathsf{iPRA}}A^\flat$; $\Gamma\vdash_{\mathsf{iPRA}}\psi$ *if and only if* $\Gamma^\sharp\vdash_{\mathsf{N}}\psi^\sharp$; $(A^\flat)^\sharp\dashv\vdash A$ *in* N; *and* $\mathsf{iPRA}\vdash\psi\leftrightarrow(\psi^\sharp)^\flat$.

(iv) Shape of the interpretations. *Every translation fixes the variables, leaves the range of every quantifier and binder the whole numerical domain, and reads equality as equality. On terms the composites are provably the identity, so the definable map between the domain and its double interpretation is the identity, with graph $x = y$; it is total, bijective and commutes with every term former and function symbol.*

(v) Equiconsistency. $\vdash_{\mathsf{N}} 0 = S0$ *if and only if* $\vdash_{\mathsf{T}_{\mathrm{eq}}} \bot$ *if and only if* $\vdash_{\mathsf{iPRA}} \bot$.

Proof. (i) is Theorems 1.106 and 1.107. (ii) is Theorems 1.114, 1.115 and 1.117 and Theorem 1.110. (iii) The two equivalences are the compositions of those in (i) and (ii). For the first round trip, $(A^\flat)^\sharp = (((A^\circ)^\tau)^\sigma)^*$. By (ii), T_{eq} proves $A^\circ \leftrightarrow ((A^\circ)^\tau)^\sigma$, so by Theorem 1.105 the native readings of the two sides are interderivable in N, and $(A^\circ)^* \dashv\vdash A$ by (i). For the second, $(\psi^\sharp)^\flat = (((\psi^\sigma)^*)^\circ)^\tau$. By (i), T_{eq} proves $\psi^\sigma \leftrightarrow ((\psi^\sigma)^*)^\circ$; applying τ, iPRA proves $(\psi^\sigma)^\tau \leftrightarrow (\psi^\sharp)^\flat$, and $\mathsf{iPRA} \vdash \psi \leftrightarrow (\psi^\sigma)^\tau$ by (ii). (iv) The first sentence holds by the definitions of A°, φ^*, τ and σ; the second is Theorem 1.117, where for the span between N and T_{eq} the terms are not translated at all. (v) If $\vdash_{\mathsf{N}} 0 = S0$ then $\vdash_{\mathsf{T}_{\mathrm{eq}}} 0 = S0$ by Theorem 1.104, and (T3) gives $\bot$. If $\vdash_{\mathsf{T}_{\mathrm{eq}}} \bot$ then $\vdash_{\mathsf{N}} \bot^*$, which is $0 = S0$. Since $\bot^\tau = \bot = \bot^\sigma$, (ii) gives the second equivalence. □

The translations act on finite derivations effectively. No claim is made that any of the systems proves its own consistency. The interpretations use no restriction to standard numerals, no retract or quotient of the domain, and no assumed symbol for equality: the reflection of equality is Theorem 1.86, a theorem.

In Lean, part (ii) is `biInterp` with `teq_iff_pra`, `pra_iff_teq`, `teq_roundTrip`, `pra_roundTrip` and `teq_consistent_iff`; part (iii) is `native_iff_pra`, `pra_iff_native`, `native_roundTrip` and `pra_native_roundTrip`; and part (v) is `native_inconsistent_iff`. The final theorems depend on no axiom beyond `propext`, `Classical.choice` and `Quot.sound`, and part (ii) on `propext` and `Quot.sound` alone.

Remark 1.120 (Consistency). Read over the natural numbers, with the iterator as iteration, every rule of N preserves truth, and $0 = S0$ is false. Hence N does not derive $0 = S0$, and by Theorem 1.119(v) neither T_{eq} nor iPRA derives $\bot$. This argument is carried out in the metatheory and is formalized in Lean as `NPrf.sound_std`, `native_consistent`, `teq_consistent` and `pra_consistent`.

Remark 1.121 (How the packages state the bridge). Every package repeats this theorem for its own calculus, by composing an equivalence of its calculus with the inherited one and the inherited bridge. The reference statement composes two steps: the native calculus against an ordinary theory T_I, which is T_{eq} with the iterators presented as staged function symbols (Theorem 1.4), and T_I against iPRA. Its proof of the second step identifies the double translation of an operation with the operation by uniqueness of iteration and of primitive recursion, as in Theorem 1.117; the explicit treatment of substitution (Theorems 1.112 and 1.113) is spelled out in the Lean formalization and is given here in full.

The universal quantifier of the native reading may be replaced by the single-binder universal of Theorem 1.31, as `single_binder` proposes. Let $\varphi^\dagger$ be defined as φ^* in Theorem 1.103, except that $(\forall x\, \varphi)^\dagger := \mathcal{U}_x\, \varphi^\dagger$. By the commutation of $\mathcal{U}$ with substitution, $(\varphi[\theta])^\dagger = \varphi^\dagger[\theta]$.

Corollary 1.122 (The bridge with the single-binder universal). (i) $\varphi^\dagger \dashv\vdash \varphi^*$ *in* N *for every ordinary formula* φ.

(ii) $\mathsf{T}_{\mathrm{eq}} \vdash (\mathcal{U}_x X)^\circ \leftrightarrow \forall x\, X^\circ$ *for every native formula* X.

(iii) *Theorem 1.119 holds with* $\varphi^\dagger$ *in place of* φ^* *in part (i) and with* $\psi^\sharp := (\psi^\sigma)^\dagger$ *in part (iii); parts (ii), (iv) and (v) are unchanged, except that in (iv) the translation* † *fixes the*

free variables and replaces a bound universal variable by the term $P(z)$ of its new bound variable.

Proof. (i) By induction on φ. Equations and $\bot$ have the same two translations. In the other cases the induction hypothesis gives $\psi^\dagger \dashv\vdash \psi^*$ and $\chi^\dagger \dashv\vdash \chi^*$ for the immediate subformulas, and, by Theorem 1.10, for all their substitution instances. The rules cited are those of Theorem 1.29 and Theorem 1.32, and every step from one formula to an interderivable one is a cut. For $\psi \wedge \chi$: $\wedge_*$E gives $\psi^\dagger$ and $\chi^\dagger$, hence ψ^* and χ^*, and $\wedge_*$I gives $\psi^* \wedge_* \chi^*$. For $\psi \vee \chi$: $\vee_*$E with the branches $\psi^\dagger \vdash \psi^* \vdash \psi^* \vee_* \chi^*$ and $\chi^\dagger \vdash \chi^* \vdash \psi^* \vee_* \chi^*$, the last steps by $\vee_*$I. For $\psi \to \chi$: over the context $\psi^\dagger \Rightarrow_* \chi^\dagger$, ψ^* one has $\psi^\dagger$, hence $\chi^\dagger$ by $\Rightarrow_*$E, hence χ^*, and $\Rightarrow_*$I discharges ψ^*. For $\exists x\, \psi$: $\exists_*$E at a fresh b with the premise $\psi^\dagger(b) \vdash \psi^*(b) \vdash \exists_* x\, \psi^*$, the last step by $\exists_*$I. For $\forall x\, \psi$: Theorem 1.32(ii) at the variable x gives $\mathcal{U}_x \psi^\dagger \vdash \psi^\dagger(x)$, hence $\psi^*(x)$, and $\forall_*$I gives $\forall_* x\, \psi^*$, because x is not free in $\mathcal{U}_x \psi^\dagger$; conversely $\forall_*$E at x, the induction hypothesis and Theorem 1.32(i). In each case the converse is the same derivation with $\dagger$ and $*$ exchanged.

(ii) Transfer, in the sense of the proof of Theorem 1.106(ii), of the two sequents of Theorem 1.32(iii) gives $\mathsf{T}_{\mathrm{eq}} \vdash (\mathcal{U}_x X)^\circ \leftrightarrow (\forall_* x\, X)^\circ$, and Theorem 1.106(ii)(f) gives $(\forall_* x\, X)^\circ \leftrightarrow \forall x\, X^\circ$.

(iii) Part (i) of the theorem: $\Gamma \vdash_{\mathsf{T}_{\mathrm{eq}}} \varphi$ if and only if $\Gamma^* \vdash_{\mathsf{N}} \varphi^*$, by Theorem 1.107, if and only if $\Gamma^\dagger \vdash_{\mathsf{N}} \varphi^\dagger$, by (i) and Theorem 1.11 applied to each member of the context and to the conclusion. $(A^\circ)^\dagger \dashv\vdash (A^\circ)^* \dashv\vdash A$ by (i) and Theorem 1.106(i). By Theorem 1.104 applied to the two sequents of (i), $\mathsf{T}_{\mathrm{eq}} \vdash (\varphi^\dagger)^\circ \leftrightarrow (\varphi^*)^\circ$, and Theorem 1.106(ii) gives $\mathsf{T}_{\mathrm{eq}} \vdash \varphi \leftrightarrow (\varphi^\dagger)^\circ$. The proof of part (iii) of the theorem uses only parts (i) and (ii) and therefore applies verbatim. Part (iv): the binders of $\varphi^\dagger$ range over the whole domain under the ordinary reading, equality is read as equality, and the term composites are those of Theorem 1.117, which $\dagger$ does not touch. Part (v): $\bot^\dagger = \bot^*$ is $0 = S0$. □

The translation $\dagger$ trades the exponential growth of the binder rank and of the number of copies of a matrix under nested universals (Theorem 1.33) for one occurrence of $P(z)$ per occurrence of a quantified variable and the selectors of one gate per universal. The axioms and rules of N, T_{eq} and iPRA are unchanged.

1.9 Clause contexts

This section draws on the definition in the main report of `raw_clausal` and on the reference statement of the inherited basis filed with `certified_zero`. Only the notion and its reading are fixed here; the clause calculi are the subject of other chapters, beginning with Section 5.2.

Definition 1.123 (Clauses and clause contexts)**.** A *clause* is an expression

$$[A;\, y.\, B(y)], \qquad y \notin \mathrm{FV}(A),$$

where A and B are native formulas. The clause binds y in its consequent B only. It is a member of a context, not a formula: it cannot occur inside a formula or as the conclusion of a sequent. A *clause context* is a finite list of clause occurrences. Its *ordinary reading* is

$$[A;\, y.\, B(y)]^\circ := A^\circ \to \forall y\, B^\circ(y).$$

An assumption of a formula A is the *constant clause*

$$[\,0 = 0;\, v.\, A\,], \qquad v \notin \mathrm{FV}(A),$$

and Γ, A abbreviates $\Gamma, [\,0 = 0;\, v.\, A\,]$.

A clause is used by the inference that, for an occurrence $[A;\, y.\, B(y)]$ in Γ, passes from $\Gamma \vdash A$ to $\Gamma \vdash B(t)$ for a term t. Three observations connect clauses to the formulas of this chapter.

(i) The reading of the constant clause is $0 = 0 \to \forall v\, A^{\circ}$, which is equivalent to A° in logic with equality. The use of a constant clause is the use of the hypothesis A, preceded by a derivation of $0 = 0$.

(ii) The clause $[B(b, 0);\, y.\, B(b, y)]$ has the reading of $Q_y B(b, y)$. It is the clause that the elimination of $D_{x,y}B$ adds for a fresh b, and its use is QE. A calculus with clause contexts therefore needs no binder Q: its formulas are generated from equations by D alone.

(iii) A native formula has a form without Q: by Theorem 1.12, $Q_y B$ is interderivable with $D_{x,y}B$ for x not free in B.

The reference statement of the inherited basis uses the antecedent $I(0, 0;\, u.\, 0) = 0$, an instance of I0, in the constant clause; `raw_clausal` and the later packages use $0 = 0$. A context retains clause occurrences, not only their values: an inference that uses a clause names an occurrence.

1.10 The measurement protocol

This section states the convention common to all packages; the wording follows `certified_zero` and `evaluation_discharge`.

Definition 1.124 (Measured object and coordinates)**.** The measured object is a *complete standalone proof checker*: one Python module that reads a certificate in JSON and accepts exactly the derivable sequents of the calculus it presents. The whole module is counted: imports, input handling, syntax and scope checks, all rules, and the acceptance driver. The source is first *normalized*: docstrings and comments are removed by reconstruction from the syntax tree, the tree is printed by `ast.unparse` of CPython 3.12.14, and one final line feed is appended. The three coordinates of the normalized source are

(i) the number of significant *tokens*, excluding whitespace, comments, indentation markers and end markers;

(ii) the number of *AST nodes* of the whole module;

(iii) the number of *normalized bytes* in UTF-8.

The size of the normalized source under `gzip` at level 9, with the modification time set to zero and no file name, is reported for information and is not a coordinate. A presentation qualifies only if its ordinary reading is bi-interpretable with iPRA in the sense of Theorem 1.119: whole numerical domains, actual equality, identity composites with graph $x = y$, and both formula round trips. Each package compares its checker with a *matched control*, an implementation of the calculus it improves on, written with the same input contract, variable encoding and context representation, so that a difference in size is attributable to the change of rules. Where different programs attain the minima of different coordinates, they are reported as separate programs and no row combines measurements of different files.

Three limits apply to every size in the article. A measured size is an achieved upper bound on an unknown optimum; it is not a lower bound for all calculi or all implementations. A different Python version can change the normalization. The finite tests that accompany a checker supplement the written proofs and are not proofs.

No checker belongs to this chapter: the calculus N and its bridge are the common reference, and the sizes of the checkers that implement N or its variants are listed in the chapters on those variants.

1.11 Further questions

Question 1.125 (Independence of the progress rule)**.** Is QI derivable from the seven other rules of N? The repository records that each of the other seven rules is independent, and that a reading over the natural numbers cannot separate QI: there the soundness conditions of DI and DE imply the condition of QI. A countermodel needs nonstandard elements.

Question 1.126 (Progress at compound matrices)**.** If QI is allowed for an arbitrary matrix, the proof of Theorem 1.20 gives induction for every native formula, and so, by Theorem 1.29, for the native reading of every ordinary formula. The reference statement of the inherited basis asserts that this changes the theory. It gives no proof that the resulting theory is not bi-interpretable with iPRA in the sense of Theorem 1.119.

Question 1.127 (Other term languages)**.** The reference statement of the inherited basis asserts that its argument applies unchanged when the iterator terms are replaced by the terms of iPRA, with the defining equations of projection, composition and recursion in place of I0 and IS. The assertion is stated without proof. The derivations of Section 1.4 use the iterator only through the selector and the parity term.

Question 1.128 (A shorter bridge)**.** Three constructions account for most of the length of the proof of Theorem 1.119: the predecessor from iteration alone (Theorem 1.65), the conjunction of two formulas through the parity term (Theorem 1.28), and the selector of two formulas by a double recursion on their binders (Theorem 1.24). No shorter replacement for any of them is known; the predecessor P_- of Theorem 1.72 is a smaller term, but its derivation has the same structure and length as that of Theorem 1.65. The diagonal counter of Theorem 1.73 saves one equation induction and the final calculation in the last stage of every one of these derivations (Theorem 1.75); the factor map, the three-cycle, the ramp and the comparator remain. On the side of iPRA, the extensionality of the simulated iterator (Theorem 1.112) and the round trip of a recursion (Theorem 1.117(ii)) are two separate inductions; in T_{eq} their counterparts are both instances of Theorem 1.55. Is there one lemma of iPRA that covers both?

Question 1.129 (The orbit equality test)**.** `orbit_equality` asks the following. (a) Is there a zero-true equality test smaller than the 45 nodes of η (Theorem 1.91)? No lower bound is known; the package suggests fusing the parity term with the absorbing iterator, keeping each argument in a single occurrence. (b) Can the reflection of Theorem 1.89 be proved without the predecessor and truncated subtraction, for instance by an equational interpolation identity? The recurrences of Theorem 1.88 use neither; the residual invariant and the last step of the proof use both. (c) Does a characteristic term with the atomic clause $\eta(s, t)$, or a primitive basis built on it, reduce the size of a complete checker under a fixed public syntax and framing contract? A smaller derived term does not establish this, and the checkers delivered with the package do not use η; the package also records that no new axiomatization below every current checker minimum is obtained. (d) The package proposes formalizing Theorem 1.73 in the Lean development, so that the predecessor proofs need neither the comparator diagonal nor the invariant below the cap. Its instance for the predecessor of Theorem 1.65 is now the Lean proof (Theorem 1.75); the generic lemma, and a formalization of Theorems 1.88 and 1.89, are not done.

Question 1.130 (The square-root pairing)**.** The reference statement on primitive recursion obtains pairing from a bounded square-root search and refers to the linear-order and semiring identities of multiplication as usual without deriving them in T_{eq}. The article replaces that pairing by the one of Theorem 1.99. A derivation of the square-root pairing in T_{eq} from Theorem 1.95 is not written out in any package.

Question 1.131 (The single-binder universal without progress)**.** Is $\mathcal{U}_x A \vdash \forall_* x\, A$, equivalently the rule $\mathcal{U}$E of Theorem 1.32(ii), derivable for every native A from the seven rules of N other than QI? The converse sequent is derived without QI (Theorem 1.32(iii)); the proof of $\mathcal{U}$E uses $P(St) = t$, whose derivation applies equation induction. A negative answer would show that $\mathcal{U}$ cannot replace $\forall_*$ in the logical interface of Theorem 1.30, which is derived without QI.

Question 1.132 (A smaller left inverse of the successor)**.** The proofs of Theorems 1.32 and 1.122 use of P only that it is a term whose only free variable is its argument and that $\vdash_{\mathsf{N}} P(Sx) = x$. Every term with these two properties can replace P in Theorem 1.31. `single_binder` asks whether a term simpler than the predecessor serves, and reports that its expansion of the same predecessor construction, with all abbreviations unfolded, has 75 nodes; in the measure of Theorem 1.67 this is $|P(m)| = 75$ (Theorem 1.72(ii)). The terms P_{57}, P_{51}, P_{50}, P'_{50} and P_- of Section 1.7.7, from `479_predecessor57`, `absorbing_orbit`, `absorbing_predecessor`, `pred50` and `fused_iteration`, are such terms with 57, 51, 50, 50 and 46 nodes, and so is the combination P_{45} of Theorem 1.74, with 45 nodes. Is there a smaller term $p(m)$, in the grammar of 0, S, variables and the iterator, with $\mathrm{FV}(p(m)) = \{m\}$ and $\vdash_{\mathsf{N}} p(Sx) = x$, or one with $p(0) = 0$ as well? No lower bound for the size of such a term is known. Theorem 1.76 bounds only the parity control and a factor map of the form $\mathrm{sel}(c, G_-(n), 0)$, and Theorem 1.77 only the update of a three-cycle; neither bounds a factor map of another form, as the fused H_- shows, nor a predecessor built without a three-cycle. The value 45 is an achieved construction, not a minimum. `479_predecessor57`, `absorbing_predecessor` and `fused_iteration` also ask whether the Lean development of the bridge can be stated with the predecessor as a parameter subject only to its two equations, so that P_- inherits the formalization of P; this is not done. Whether the other constructions of Section 1.7.7 combine further, for instance the weighted parity $\tilde{N}$ with another ramp, is not examined.

Question 1.133 (Weighted parity as a selector control)**.** `absorbing_predecessor` states that Theorem 1.70(iii) extends, by induction on the selector of formulas, to $\mathrm{sel}(\tilde{N}(n), A, B) \dashv\vdash \mathrm{sel}(N(n), A, B)$ for native formulas, so that $\tilde{N}$ may replace N as the variable control of the conjunction of Theorem 1.28; the induction is not written out, and the statement is not checked here against Theorem 1.24, whose selector also introduces the gates of the bound variables. The package further asks whether interchangeability as a selector control, which is weaker than equality ($\tilde{N}(3) \neq N(3)$), could serve as a primitive notion for control states in a presentation that still recovers actual equality and the identity composites of Theorem 1.119.

Question 1.134 (Size of the single-binder translation)**.** `single_binder` states, for the formulas of the inherited basis built from equations and D with a selector that aligns binders (Theorem 1.34), and for raw formula trees with all abbreviations expanded and one node per variable, 0, S, iterator, equation and D, the following counts. For $F_n := \forall x_0 \cdots \forall x_{n-1}\, (x_0 + \cdots + x_{n-1} = S0)$, the old translation has $(4n{+}39)2^n{+}2n{-}35$ nodes and the single-binder translation $85n{+}4$ (at $n = 10$: 80881 against 854; the new tree is larger at $n = 1$ and $n = 2$); and for an arbitrary ordinary formula with N source nodes the single-binder translation has at most $6N^2 + 104N$ nodes. The counts are checked by a program for $n \leq 10$ and the general bound is argued only in outline. Neither is proved here, and the corresponding counts for the definitions of the article (Theorem 1.33) are not computed. In the accounting of `479_predecessor57`, `absorbing_orbit`, `absorbing_predecessor`, `pred50` and `fused_iteration`, the atomic formula has $4n + 4$ nodes and each universal adds 7 nodes for its gate and binder and $p - 1$ for the expansion of a predecessor of p nodes at the single occurrence of its variable, so that the single-binder translation of F_n has $(p + 10)n + 4$ nodes, with binder rank n: $85n + 4$ for P, and $67n + 4$, $61n + 4$, $60n + 4$, $56n + 4$ and $55n + 4$ for P_{57}, P_{51}, P_{50} or P'_{50}, P_-, and P_{45} (at

$n = 10$: 674, 614, 604, 564 and 554). Since each x_i occurs once in F_n, these follow from the count $85n + 4$ and the differences $|P(z)| - |p(z)|$, but $85n + 4$ itself is not proved here.

Question 1.135 (Finite blocks under one binder; answered in part)**.** `479_predecessor57` states, for the selector μ_c of D-formulas that aligns common binders and pads the shorter operand with gates (Theorem 1.35), the following, with proofs given only in outline. (a) The rank and terminal size of $\mu_c(A, B)$, and the padding bound $r_{\max} - r_{\min}$ over binary selector trees, attained by a comb ordered by rank. (b) The finite selector Σ_n^{G-} of Theorem 1.42 satisfies $\Sigma_n^{G-}(\omega_j; A_0, \ldots, A_{n-1}) \dashv\vdash A_j$ for $\omega_j := SI(\bar{j}, 0; u.\, S(u+u))$, $j < n$. (c) The formulas $\mathsf{And}^{G-}(\vec{A})$ and $\mathsf{Or}^{G-}(\vec{A})$ of Theorem 1.45 satisfy the rules of a finite conjunction and a finite disjunction, have rank $r_{\max} + 1$ and $\sum_j T(A_j) + 66(n-1) + 6(r_{\max} - r_{\min}) + r_{\max} + 8$ nodes. (d) After pairing (Theorem 1.99), with tuple codes e_n and decoders $d_1, \ldots, d_n$, the formula $D_{u,z}\gamma_z(A(d_1(P_-(z)), \ldots, d_n(P_-(z))))$ is a universal quantifier over a block of n variables, of rank $r(A) + 1$.

Settled (Section 1.4.8, from `fused_blocks_and_checkers`): (a) is Theorem 1.38(ii) and Theorem 1.44; (b) is Theorem 1.43(i), with the uniform introduction and elimination of Theorem 1.43(ii),(iii); (c) is Theorem 1.46, with complete proofs in N. The branch cost 66 is lowered to 64 by the floor ramp and to 62 by the fused selector Σ_n, in which H_- both selects and supplies the next control.

Open. (d) is not proved in any package, and its size is not computed. No lower bound for the branch cost is known, so whether 62 can be lowered (by another control orbit, or by a tree that is not a comb of aligned selectors) is open; the padding bound is proved only for trees of aligned selectors. The corresponding counts for the stacking selector of Theorem 1.24, to which (a) and the sizes in (c) do not apply (Theorem 1.33), are not computed. The size of the translation of a block with internal binders into the guarded public goal of the compact checkers (Theorem 1.47) is not computed. None of Section 1.4.8 is formalized in Lean or Rocq.

Chapter 2

Refinements of the eight rules

The native scalar core N has eight rules (Section 1.3). This chapter collects the changes of that rule basis which keep the language of equations, Q and $D_,$ as it is: there are no clause contexts, no restricted class of public formulas and, with the one exception of the calculus U_c in Section 2.5.8, no normal forms of terms. The changes fall into five groups. Section 2.2 treats the two introduction rules: progress may introduce a closure directly or through a witnessed $D_,$, and the two introductions may be merged into one rule whose mode is read off the matrix and the witness. Section 2.3 treats the transport rule: the number of templates, the crossing of the conclusion, the exclusion of the literal 0 as a source, the reversal of the root equation, replacement with no template in the forward and in the backward direction, and replacement of a single occurrence. Section 2.4 replaces the elimination QE of a closure by a rule that computes its matrix from the goal. Section 2.5 replaces induction for equations by uniqueness of iteration. Section 2.6 changes the reading of Q at a literal equation to progressivity and presents the seven-rule calculi that result. Section 2.7 records two results on recursors other than the iterator, which two of the packages prove on the side. Section 2.8 records two limits that concern the native language and its presentation rather than a rule: the classes of relations defined by native formulas of bounded binder depth, and an obstruction to finite ordinary axiomatizations of iPRA.

Every calculus of Sections 2.2 to 2.5 derives exactly the sequents of N, on the same formulas. The calculi of Section 2.6 do not; they are equivalent to N through a pair of translations with both formula round trips. Table 2.1 at the end of Section 2.6 lists all calculi of the chapter.

2.1 Conventions and the comparison method

This section fixes what all packages of the chapter restate before their own results; it draws on no package in particular.

The rules of N are named as in Section 1.3: HYP, I0, IS, TR, QE, DE, DI (introduction of $D_,$ by cases) and QI (introduction of Q by progress at a literal equation). A *term template* $h(v)$ is a term with a designated variable v, which may occur several times, once, or not at all; in the last case the template is *constant*. An *equation template* is $a(v) = b(v)$ for two term templates, and a *formula template* $F(v)$ is a native formula with a designated variable. The sign $\equiv$ is identity of syntax (up to the names of bound variables), so that $s \not\equiv 0$ says that s is not the term 0. This is a test on syntax and not on values: $I(0,0;\, u.\,0) \not\equiv 0$, although the term equals 0 by I0. A formula is *compound* if it is not a literal equation, that is, if its root is Q or $D_,$. Substitution preserves the root of a formula and may change whether a term is the literal 0 or a literal successor.

All calculi of the chapter have the terms of Section 1.1, the formulas of Section 1.2, finite

lists of formulas as contexts, and the rules HYP, I0, IS and DE in their forms of Section 1.3. A rule inventory below therefore names these four rules and displays the others. The one exception is the calculus $\mathsf{U_c}$ of Theorem 2.105, which is stated on normal terms and has reflexivity in place of the two computation equations. Derivability in a calculus X is written $\vdash_{\mathsf{X}}$, and $\vdash$ without a subscript is derivability in N.

Definition 2.1 (Equivalence on the same formulas)**.** Two calculi X and Y on the native language are *equivalent* if $\Gamma \vdash_{\mathsf{X}} A$ holds exactly when $\Gamma \vdash_{\mathsf{Y}} A$ holds, for every context Γ and every native formula A.

The equivalence is about sequents with arbitrary contexts and arbitrary formulas, not about closed equations. It is always proved by two translations of derivations that copy the common rules and replace each instance of a changed rule by a fixed finite derivation; the translations are effective and leave terms, formulas, contexts and conclusions unchanged.

Proposition 2.2 (Transfer of the bridge)**.** *Let* X *be equivalent to* N*. Then every statement of Theorem 1.119 holds for* X *with the same translations:* X*,* $\mathsf{T_{eq}}$ *and* iPRA *are bi-interpretable with whole domains, actual equality and identity numerical composites, both formula round trips hold, and* $\vdash_{\mathsf{X}} 0 = S0$ *holds exactly when* iPRA *is inconsistent. Weakening, substitution of terms, cut and the Leibniz rule are admissible in* X*.*

Proof. The statements of Theorem 1.119 and Section 1.4 about N are statements about its derivable sequents, and X has the same derivable sequents. □

Each package of the chapter restates this transfer for its own calculus; it is not repeated below. The last sentence of the proposition concerns the consequence relation of X. Whether an operation maps each rule instance to an instance of the same rule is a different question.

Lemma 2.3 (Structural operations by direct induction)**.** *In every calculus of this chapter, weakening of the context, injective renaming of free variables, and cut are admissible by a direct induction on derivations. Substitution of terms for variables maps rule instances to rule instances of the same rule in a calculus none of whose side conditions tests a term for being the literal* 0*, a literal successor or a syntactic match of another term.*

Proof. The side conditions of the rules below are of four kinds: freshness of an eigenvariable; a test on the root of the matrix; a test whether a term of the instance is the literal 0 or a literal successor; and, for the two rules of replacement with no template and for the rule AQE of Section 2.4, a syntactic match of subterms. An injective renaming preserves all four. For weakening, rename the eigenvariables of the derivation away from the added formulas and keep the added formulas in every context; each inference is still an instance of its rule with the same terms. For cut, let $\Gamma \vdash_{\mathsf{X}} A$ and $\Gamma, A \vdash_{\mathsf{X}} C$. Rename the eigenvariables of the second derivation away from the first, and replace every use of HYP for the assumption A by the first derivation, weakened by the local assumptions. No term of an inference changes, so every side condition is preserved. For substitution the first two kinds of side condition are preserved after a renaming of eigenvariables, as in the proof for N in Section 1.4; the last two kinds need not be. □

Several derived rules of Section 1.4 use only part of N, and they are used in that form.

Lemma 2.4 (Derived rules and the rules they use)**.** *Let* X *be a calculus of this chapter.*

(i) *If* X *contains* QE *and* DI*, then* X *derives:* $D_{x,y}B \dashv\vdash Q_yB$ *for* $x \notin \mathrm{FV}(B)$*; the introduction of* Q_yB *by cases, from* $\Gamma, B(0) \vdash B(Sz)$ *with* z *fresh, for an arbitrary matrix* B*; numerical cases, from* $\Gamma \vdash B(0)$ *and* $\Gamma \vdash B(Sz)$ *with* z *fresh to* $\Gamma \vdash B(t)$*; and* $D_{x,y}B(x,y)$ *from* $Q_yB(t,y)$*.*

(ii) *If* X *contains* QE *and* QI*, then* X *derives equation induction: from* $\Gamma \vdash E(0)$ *and* $\Gamma, E(z) \vdash E(Sz)$*, with* E *a literal equation and* z *fresh, to* $\Gamma \vdash E(t)$*.*

(iii) *If* X *contains* TR*, then* X *derives reflexivity, symmetry, transitivity, congruence in every term template, and transport in an equation template, from* $s = t$ *and* $a(s) = b(s)$ *to* $a(t) = b(t)$*.*

Proof. The derivations of these rules in Section 1.4 use HYP, DE, weakening and the rules named in each item; for (iii) they use I0 and TR alone. □

In (iii) each of the four equality rules is one application of TR, and transport in an equation template is two. Symmetry needs a constant template: TR with the template $h(v) := s$, any derivable equation as its first premise and $s = t$ as its second premise concludes $t = s$.

2.2 The introduction rules

This section draws on `direct_closure`, `direct_closure_qprogress` and `witness_rule`, and on the obstruction section of `reversal_core`.

2.2.1 Progress through a witness and progress at the closure

The reports state induction for equations as an introduction of D, with a witness:

$$\frac{\Gamma,\ E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)}\ \text{DP} \qquad (E \text{ a literal equation}),$$

where z is not free in Γ, in t or in $D_{x,y}E$.

Definition 2.5. N_D is the calculus with the rules HYP, I0, IS, TR, QE, DE, DI and DP. It is N with QI replaced by DP.

The rule DP has a matrix in two variables and a witness term; the rule QI has a matrix in one variable and no witness.

Theorem 2.6 (Direct closure)**.** N_D *and* N *are equivalent. The proof uses only the rules* HYP*,* QE*,* DE*,* DI *and weakening; it holds for any common set of further rules for which weakening and renaming are admissible.*

Proof. All rules but one are common. QI *in* N_D*.* Let $\Gamma, E(z) \vdash E(Sz)$ be the translated premise. Choose a variable x not free in E and Γ. The rule DP with the matrix $E(y)$, in which the coordinate x is not used, and the witness 0 has exactly this premise and concludes $\Gamma \vdash D_{x,y}E(y)$. Apply DE with a fresh b: the opened hypothesis is $Q_yE(y)$, because x does not occur in E, and the second premise $\Gamma, Q_yE(y) \vdash Q_yE(y)$ is HYP. The conclusion is $\Gamma \vdash Q_yE(y)$.

DP *in* N*.* Let $\Gamma, E(t,z) \vdash E(t,Sz)$ be the translated premise. The formula $E(t,y)$ is a literal equation in y with t as a parameter, and z is not free in t, so QI gives $\Gamma \vdash Q_yE(t,y)$. Let w be fresh. Over $\Gamma, E(t,0)$, weakening and QE at Sw give $E(t,Sw)$. This is the premise of DI with the witness t, which concludes $\Gamma \vdash D_{x,y}E(x,y)$. No equality rule, no computation equation and no substitution into the given premise is used. □

Proposition 2.7 (Node counts)**.** *Count every instance of a rule as one node. A derivation of* N *with* n *nodes,* r *of them instances of* QI*, translates to a derivation of* N_D *with* $n + 2r \le 3n$ *nodes. A derivation of* N_D *with* n *nodes,* r *of them instances of* DP*, translates to a derivation of* N *with* $n + 3r \le 4n$ *nodes.*

Proof. An instance of QI becomes DP, DE and HYP over the same premise. An instance of DP becomes QI, QE, HYP and DI over the same premise. Each premise derivation is used once, and weakening and renaming do not change the number of nodes. □

The bounds count nodes of the inference tree; they do not bound the size of formulas, of serialized certificates or of a checker.

Remark 2.8 (Sources). `direct_closure` and `direct_closure_qprogress` are two independent accounts of Theorems 2.6 and 2.7, with the same two constructions and the same counts; the second contains the proof twice, once as a proof note and once as an audit. Both state the calculi with transport in a formula template in place of TR, which the theorem permits. Both stress that the rule QI was a derived rule of the calculus before it was taken as primitive, so that the change is one of the choice of primitives. Their example is $F(n) := I(n, 0;\, u.\, Su)$ with the equation $F(y) = y$: with transport in a formula template the derivation of $F(n) = n$ by QI and QE has six nodes, its translation to N_D has eight, and the translation of that back has eleven. In Lean the rule QI is the constructor `qIInd` of the kernel, adopted from these packages, and DP is the derived rule `NPrf.dIProg`; the equivalence of the calculus of the reports, with DP and transport in a formula template, is `iprf_iff_nprf`.

2.2.2 One closure introduction with a mode read off the matrix

For a matrix $B(y)$ and a variable z put $\eta_B(z) := z$ if B is a literal equation and $\eta_B(z) := 0$ if B is compound.

Definition 2.9. N_det is N with QI replaced by

$$\frac{\Gamma,\ B(\eta_B(z)) \vdash B(Sz)}{\Gamma \vdash Q_y B(y)}\ \mathrm{QI}_\eta,$$

where B is an arbitrary native formula and z is not free in Γ and $Q_y B$. Its rules are HYP, I0, IS, TR, QE, DE, DI and QI_η.

A certificate for QI_η carries no choice between cases and progress: the root of the matrix decides.

Corollary 2.10. N_det, N *and* N_D *are equivalent, and the bounds of Theorem 2.7 hold between* N_det *and* N_D.

Proof. At a literal equation QI_η is QI. At a compound matrix it is the introduction of Q by cases, which is derivable in N by Theorem 2.4(i): DI with an unused coordinate and the witness 0, then DE and HYP. This is the same three-node construction as in Theorem 2.6, with DI in place of DP. Conversely N_det contains QI. □

The introduction of $Q_y E$ by cases for a literal equation E is not an instance of QI_η, and it remains derivable through DI and DE. For example $Q_y(y = 1)$ is derivable in all three calculi from the premise $0 = 1 \vdash Sz = 1$ of the introduction by cases, while the premise $z = 1 \vdash Sz = 1$ of progress is not derivable.

2.2.3 What cannot be removed from the introductions

Proposition 2.11 (The guard of QI)**.** *If* QI *is allowed at every matrix, the resulting calculus derives induction for every native formula: from* $\Gamma \vdash B(0)$ *and* $\Gamma, B(z) \vdash B(Sz)$ *with* z *fresh to* $\Gamma \vdash B(t)$.

Proof. The unrestricted rule gives $\Gamma \vdash Q_y B(y)$, and QE gives $B(t)$. □

Through the native connectives of Section 1.4 this is induction for every first-order formula, and the bridge of Theorem 1.119, whose target has induction for quantifier-free formulas only, no longer applies. The rule QI_η avoids this by using the instance at zero at a compound matrix.

Proposition 2.12 (DI cannot be dropped)**.** *Let* N^- *be* N *without* DI*, or* $\mathsf{N}_{\det}$ *without* DI*. If no formula of* Γ *has the root* $D_,$*, then* N^- *derives from* Γ *no formula with the root* $D_,$*. In particular* N^- *does not derive* $D_{x,y}(0 = 0)$*, which* N *derives.*

Proof. Suppose a derivation of $\Gamma \vdash_{\mathsf{N}^-} A$ with A of root $D_,$ and Γ as stated, and take one with the least number of nodes. Its last rule is not HYP, by the assumption on Γ; not a computation equation or TR, whose conclusions are equations; and not an introduction of Q, whose conclusion has the root Q. If it is QE with the conclusion $B(t)$, then its second premise $\Gamma \vdash B(0)$ has the same root $D_,$ and the same context, and a smaller derivation. If it is DE, then its first premise $\Gamma \vdash D_{x,y}B$ has the root $D_,$ and the same context, and a smaller derivation. Both contradict minimality. In N the formula $D_{x,y}(0 = 0)$ follows by DI with the witness 0 from the premise $0 = 0 \vdash 0 = 0$. □

The assumptions inside a subderivation of an introduction of Q do not affect the argument, because such an introduction is never the last rule of a derivation of a formula with the root $D_,$.

Proposition 2.13 (DI cannot be restricted to compound matrices)**.** *Restrict* DI *to compound matrices* B*.*

(i) *The calculus with* HYP*,* I0*,* IS*,* TR*,* QE*,* DE*,* QI *and the restricted* DI *does not derive* $D_{x,y}(0 = 0)$*.*

(ii) *The calculus with* HYP*,* I0*,* IS*,* TR*,* QE*,* DE*,* DP *and the restricted* DI *does not derive* $D_{x,y}(y = 1)$*.*

Both formulas are derivable in N*. The statements remain true with transport in a formula template in place of* TR*.*

Proof. Both parts interpret terms in the standard natural numbers, equations as equality and Q_yB by its ordinary reading, and give a formula $D_{x,y}B$ with a compound matrix its ordinary reading. The interpretation of a formula $D_{x,y}E$ with a literal equation E is changed. It commutes with substitution, because substitution preserves the root of a matrix, and truth of a sequent means that every assignment satisfying the context satisfies the conclusion.

(i) Let $D_{x,y}E$ be false for every literal equation E. The rules HYP, I0, IS, transport and QE preserve truth as they do for the ordinary reading. QI preserves truth by induction on the natural numbers. DE at an atomic matrix has a false first premise, and at a compound matrix it is the elimination of an existential quantifier. The restricted DI preserves truth because every natural number is zero or a successor. The formula $D_{x,y}(0 = 0)$ is false.

(ii) Let $D_{x,y}E$ mean $\exists x \forall y \, (E(x, y) \to E(x, Sy))$ for a literal equation E. DP preserves truth: its premise states the progress of $E(t, y)$, and t is a witness. DE at an atomic matrix preserves truth: if $\forall y \, (E(b, y) \to E(b, Sy))$ holds, then $E(b, 0) \to \forall y \, E(b, y)$ holds by induction on the natural numbers, and this is the reading of the opened hypothesis $Q_yE(b, y)$. The other rules preserve truth as in (i). The formula $D_{x,y}(y = 1)$ is false, because at $y = 1$ the equation $1 = 1$ holds and $2 = 1$ does not.

In N, $D_{x,y}(0 = 0)$ is derived in Theorem 2.12, and $D_{x,y}(y = 1)$ follows by DI with the witness 0 from $0 = 1 \vdash Sz = 1$, which holds because $0 = 1$ derives every equation (Section 1.4). □

Remark 2.14 (Sources). Theorems 2.11 and 2.12 are from `direct_closure_qprogress`; `direct_closure` states the first. Theorem 2.13(i) is the obstruction section of `reversal_core`. Part (ii) is stated in `direct_closure_qprogress` with the interpretation and a reference to an earlier package for the verification of the rules. `direct_closure_qprogress` also recalls why the ordinary theory T_{eq} needs its coverage axiom: the product of two copies of the standard model, with all operations coordinatewise, satisfies the computation equations, separation and induction for equations, because the solution set of an equation is a product $U_0 \times U_1$, the base puts 0 into both factors, and the step applied at $(r, 0)$ and at $(0, r)$ closes each factor under successor; and the element $(0, 1)$ is neither zero nor a successor.

2.2.4 One introduction with a mode read off the matrix and the witness

The two introductions DI and QI can be replaced by one introduction of $D_{,}$. For a matrix $B(x, y)$, a witness t and a variable z put

$$\eta_{B,t}(z) := \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases}$$

Definition 2.15 (The calculus W)**.** W has the seven rules HYP, I0, IS, TR, QE, DE and

$$\frac{\Gamma,\ B(t, \eta_{B,t}(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}\ \mathrm{DI_w},$$

where z is not free in Γ, in t or in $D_{x,y}B$.

The rule selects progress exactly when the matrix is a literal equation and the witness is the literal 0, and cases otherwise. A witness such as $I(0, 0;\ u.\, 0)$, which equals 0, selects cases. The calculus has no rule that introduces Q.

Lemma 2.16 (A second name for zero)**.** *Let $c := I(0, 0;\ u.\, 0)$. Every calculus containing I0 and TR derives $c = 0$ and $0 = c$, and $c \not\equiv 0$.*

Proof. $c = 0$ is an instance of I0, and $0 = c$ follows by symmetry (Theorem 2.4(iii)). □

Theorem 2.17 (Witness-selected introduction)**.** W *and* N *are equivalent.*

Proof. From W *to* N. An instance of $\mathrm{DI_w}$ in the mode of cases is an instance of DI. An instance in the mode of progress has a literal equation $E(x, y)$ as its matrix, the witness 0 and the premise $\Gamma, E(0, z) \vdash E(0, Sz)$; it is an instance of DP with the witness 0, which N derives by Theorem 2.6.

From N *to* W. The rule QI with the premise $\Gamma, E(z) \vdash E(Sz)$: choose x not free in E and Γ. The rule $\mathrm{DI_w}$ with the matrix $E(y)$ and the witness 0 is in the mode of progress, has this premise and concludes $D_{x,y}E(y)$; DE and HYP give $Q_y E(y)$ as in Theorem 2.6.

The rule DI with a compound matrix, or with a witness $t \not\equiv 0$, is an instance of $\mathrm{DI_w}$. There remains DI with a literal equation $E(x, y)$ as its matrix and the witness 0. Let

$$\pi\colon \quad \Gamma,\ E(0, 0) \vdash_{\mathsf{W}} E(0, Sz)$$

be the translated premise, with z fresh. Work over $\Gamma, E(c, 0)$ with c as in Theorem 2.16. Transport in the equation template $E(v, 0)$ along $c = 0$ gives $E(0, 0)$ (Theorem 2.4(iii)). Cut with π, which is admissible in W by Theorem 2.3, gives $E(0, Sz)$. Transport in the equation template $E(v, Sz)$ along $0 = c$ gives $E(c, Sz)$. Hence $\Gamma, E(c, 0) \vdash_{\mathsf{W}} E(c, Sz)$. Since $c \not\equiv 0$, the rule $\mathrm{DI_w}$ with the witness c is in the mode of cases and has exactly this premise. It concludes $D_{x,y}E(x, y)$; the conclusion does not record the witness. □

The construction changes an assumption by a derived transport and a cut. It does not substitute a term into the derivation π, so it cannot change the mode of an introduction inside π.

Remark 2.18 (Substitution in W). A substitution can turn a witness that is a variable into the literal 0, and then an instance of DI_w in the mode of cases is mapped to a sequent that is not the premise of an instance in the mode of progress. Substitution is admissible in W by Theorem 2.2, through N, and is not obtained by a direct induction on the derivations of W. For this reason the Lean kernel keeps the two introductions of N.

Remark 2.19 (Sources and variants). Theorem 2.17 is the theorem of `witness_rule`; in Lean it is `wprf_iff_nprf`, for the calculus `WPrf`. The package compares W with the calculus that has DI and DP, so its proof has four cases: DI repeated; DI at an equation with the witness 0, as above; DP with the witness 0 repeated; and DP with a witness $t \not\equiv 0$, which is the second construction of Theorem 2.6 with the inner closure introduced through an unused coordinate. The package works with a restricted inventory of computation instances and therefore uses $I(2, 1;\, u.\, 0)$ as its second name for zero; that inventory is treated in Theorem 2.62.

The argument does not use any property of the number zero in the witness coordinate. If p is a closed term and $q \not\equiv p$ is a closed term with $p = q$ and $q = p$ derivable from the common rules, then progress may be selected exactly at the witness p: an introduction by cases at p is repaired at the witness q, and every auxiliary introduction with an unused coordinate takes the witness p. The coordinate y of the induction still starts at 0. The literal 0 is chosen because it is the cheapest witness to recognize.

`one_context` states the same calculus with the two-template transport of Section 2.3.1 replaced by TR and calls it $\mathcal{O}_7$; since N already has TR, its calculus is W as defined here.

2.3 The transport rule

This section draws on `one_context`, `one_sided_transport`, `reversal_core`, `source_dispatch`, `template_free`, `backward_replacement` and the note on single occurrences in `kernel_refinement`.

Throughout the section $J(a) := I(0, a;\, u.\, 0)$ and $c := J(0)$. The equation $J(a) = a$ is an instance of I0, and $J(a) \not\equiv 0$ for every term a; in particular $c = 0$ is an instance of I0 and $c \not\equiv 0$.

2.3.1 One template, two templates, a formula template

The rule TR of N has one term template and a crossed conclusion: from $s = t$ and $h(s) = r$ it infers $r = h(t)$. The reports use two stronger forms,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash a(s) = b(s)}{\Gamma \vdash a(t) = b(t)}\ \text{TR}_2 \qquad\qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{TR}_F,$$

with two term templates $a(v)$, $b(v)$ and with a native formula template $F(v)$.

Definition 2.20. N_2 is N with TR replaced by TR_2, and N_F is N with TR replaced by TR_F. Each has the rules HYP, I0, IS, its transport rule, QE, DE, DI and QI.

Theorem 2.21 (One term template suffices)**.** *N, N_2 and N_F are equivalent. An instance of TR_2 is simulated in N by two instances of TR with the same first premise. An instance of TR is simulated in N_2 by one instance of TR_2 with a constant second template, followed by symmetry.*

Proof. *TR_2 in N.* TR with the template a, applied to $s = t$ and $a(s) = b(s)$, gives $b(s) = a(t)$. TR with the template b, applied to $s = t$ and $b(s) = a(t)$, gives $a(t) = b(t)$; the term $a(t)$ is the fixed side of the second step.

TR in N_2. First, N_2 derives reflexivity and symmetry: TR_2 with the templates v and a, applied to $J(a) = a$ and $J(a) = a$, gives $a = a$; and TR_2 with the templates v and s, applied to $s = t$ and $s = s$, gives $t = s$. Now let $s = t$ and $h(s) = r$ be given. TR_2 with the templates $h(v)$ and the constant r gives $h(t) = r$, and symmetry gives $r = h(t)$.

N_F. An instance of TR_2 is the instance of TR_F at the template $a(v) = b(v)$, so N_F derives every sequent of N_2. Conversely TR_F is a derived rule of N, by the Leibniz rule of Section 1.4.

In each direction the seven other rules are copied, so the induction on derivations applies to open sequents and to premises under both binders. □

The crossing of the conclusion is what makes two successive replacements in one template reproduce a simultaneous replacement in both sides of an equation. The rule TR has no side condition, so substitution maps its instances to instances (Theorem 2.3).

Remark 2.22 (Size of the simulation). The simulation of TR_2 uses the derivation of $s = t$ twice. A proof represented as a directed acyclic graph shares it; a proof represented as a tree copies it, and repeated translation can enlarge a tree substantially. No linear bound on the size of translated trees is claimed in `one_context` or `one_sided_transport`.

Remark 2.23 (Sources). The comparison of N with N_2 is the theorem of `one_context`, whose reference calculus is N_2. In Lean the rule TR is the constructor `NPrf.trOne` of the kernel, adopted from that package, and TR_2 is the derived rule `NPrf.trEq`. The comparison with N_F is the remark on the calculus of the reports in Section 1.3.

2.3.2 Sources other than the literal zero

Let TR^- be the rule TR with the side condition $s \not\equiv 0$ on the left side of its first premise, the *source*:

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}\ \text{TR}^- \qquad (s \not\equiv 0).$$

Every variable and every term with the root S or I is a permitted source, including terms whose value is zero.

Definition 2.24 (The calculi O and M)**.** O is N with TR replaced by TR^-; its rules are HYP, I0, IS, TR^-, QE, DE, DI and QI. M is W with TR replaced by TR^-; its seven rules are HYP, I0, IS, TR^-, QE, DE and DI_W.

Lemma 2.25 (Equality from I0 and TR^-)**.** *The following are derivable from I0 and* TR^- *alone, over an arbitrary context.*

(i) *Reflexivity:* $a = a$.

(ii) *Symmetry: from* $a = b$ *to* $b = a$*, also when* a *or* b *is the literal* 0.

(iii) *Restricted transport in an equation template: if* $s \not\equiv 0$*, from* $s = t$ *and* $a(s) = b(s)$ *to* $a(t) = b(t)$.

(iv) *Joining through zero: if* $p \not\equiv 0$ *and* $q \not\equiv 0$*, from* $p = 0$ *and* $q = 0$ *to* $p = q$.

(v) *Transitivity: from* $r = s$ *and* $s = t$ *to* $r = t$.

(vi) *Congruence: from* $s = t$ *to* $f(s) = f(t)$*, for every term template* f.

(vii) *The rule* TR *without the side condition.*

Proof. (i) TR^- with the template v, applied to $J(a) = a$ and $J(a) = a$, gives $a = a$; the source is $J(a) \not\equiv 0$.

(ii) TR^- with the constant template a, applied to $c = 0$ and $a = b$, gives $b = a$; the source is $c \not\equiv 0$.

(iii) As in Theorem 2.21: the two steps have the same first premise $s = t$.

(iv) TR^- with the template $J(v)$, applied to $p = 0$ and $J(p) = p$, gives $p = J(0)$, that is $p = c$. In the same way $q = c$. By (ii), $c = p$ and $c = q$. TR^- with the template v, applied to $c = q$ and $c = p$, gives $p = q$. The sources are p, q and c.

(v) If $s \not\equiv 0$, reverse $r = s$ by (ii) and apply TR^- with the template v to $s = t$ and $s = r$. Let $s \equiv 0$. If $r \equiv 0$, the second premise is the conclusion; if $t \equiv 0$, the first premise is. Otherwise $r \not\equiv 0$ and $t \not\equiv 0$, the premises are $r = 0$ and, after (ii), $t = 0$, and (iv) applies.

(vi) If $s \not\equiv 0$, apply TR^- with the template f to $s = t$ and $f(s) = f(s)$. If $s \equiv 0$ and $t \not\equiv 0$, reverse the premise, obtain $f(t) = f(0)$ and reverse again. If both sides are the literal 0, use (i).

(vii) From $s = t$ and $h(s) = r$: (vi) gives $h(s) = h(t)$, (ii) gives $r = h(s)$, and (v) gives $r = h(t)$. □

The case distinctions in this proof are on the syntax of the given terms and belong to the construction of the derivation. The proof uses no induction, no numerical case distinction, no successor computation and no substitution into a derivation. It uses the constant template in (ii) and the crossing of the conclusion in (ii) and (iv).

Proposition 2.26 (Retraction criterion)**.** *Let a term language have a literal* 0 *and a term context* $J(v)$ *such that* $J(a) \not\equiv 0$ *for every term* a*, and let* $J(a) = a$ *be available as an axiom scheme in every context. Then the crossed rule restricted to sources other than the literal* 0 *derives the crossed rule at all sources, and with it reflexivity, symmetry, transitivity, congruence and transport in an equation template.*

Proof. The proof of Theorem 2.25 uses nothing else. □

Theorem 2.27 (Transport at sources other than the literal zero)**.** O*,* M *and* N *are equivalent.*

Proof. Every instance of TR^- is an instance of TR, and TR is derivable in O and in M by Theorem 2.25(vii). This proves the equivalence of O and N. For M, the rule DI_W is derivable in N by Theorem 2.17. Conversely the proof of Theorem 2.17 derives QI and DI in M: it uses DI_W, DE, HYP, cut, which is admissible in M by Theorem 2.3, the equations $c = 0$ and $0 = c$, and transport in an equation template along them, which M derives from TR by Theorem 2.4(iii). □

Remark 2.28 (Substitution and the choice of the kernel)*.* Substituting 0 for a variable can turn a permitted source into the literal 0, so substitution does not map instances of TR^- to instances. Substitution is admissible in O and M as an operation on derivable sequents (Theorem 2.2); what fails is the preservation of each rule instance. `reversal_core` makes the same correction to an earlier wording about the Lean calculus `ZPrf`. The Lean kernel keeps unrestricted sources so that its substitution theorem is a direct induction.

Remark 2.29 (Sources)*.* Theorems 2.25 and 2.27 are from `one_sided_transport`, which calls M the calculus $\mathcal{C}$ and reserves the letter $\mathcal{M}$ for the same seven rules with TR_2 at sources other than the literal 0. The package proves the equivalence twice: by simulating that two-template rule with Theorem 2.25(iii) and quoting an earlier equivalence, and directly as above. In Lean the statement that I0 and TR^- recover TR is `OKernel.trOneF`, for the class `OKernel`; the equivalences are `oprf_iff_nprf` for O and `mprf_iff_nprf`, `mprf_iff_iprf` for M. Theorem 2.26 is stated in `reversal_core`.

2.3.3 Reversal of the root equation

Define an operation on the root of a native formula:

$$(s = t)^\dagger := (t = s), \qquad (Q_y B)^\dagger := Q_y B, \qquad (D_{x,y} B)^\dagger := D_{x,y} B.$$

It does not descend into a matrix, it satisfies $A^{\dagger\dagger} = A$, and it commutes with substitution.

Definition 2.30 (The calculus X)**.** X has the eight rules HYP, I0, IS, DE, DI, QI and

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)^\dagger}\ \text{TRX} \quad (s \not\equiv 0),$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)^\dagger}\ \text{QX},$$

where $F(v)$ is an arbitrary native formula template, constant templates included.

Lemma 2.31 (Reversal)**.** *If* $\Gamma \vdash_{\mathsf{X}} A$*, then* $\Gamma \vdash_{\mathsf{X}} A^\dagger$*.*

Proof. Apply TRX with the first premise $c = 0$ and the constant template $F(v) := A$. □

Theorem 2.32 (Root-reversing eliminations)**.** X *and* N *are equivalent. The derivation of the rules of* N *in* X *uses neither* IS *nor an introduction rule nor* DE*.*

Proof. From X *to* N. For TRX, transport in the formula template F (Section 1.4) gives $F(t)$; if $F(t)$ is an equation, symmetry gives $F(t)^\dagger$, and otherwise $F(t)^\dagger$ is $F(t)$. For QX, use QE and the same step.

From N *to* X. The rule TR^- is the instance of TRX at the template $F(v) := (h(v) = r)$, since $F(t)^\dagger$ is $r = h(t)$. Hence TR is derivable in X by Theorem 2.25(vii). The rule QE is QX followed by Theorem 2.31; the two reversals cancel. The six other rules are common. The constructions hold over arbitrary contexts, so they apply inside the premises of DE and of the two introductions. □

Only the equational instances of transport have to be recovered in X; transport in a compound template is used in the other direction, where it is available.

Remark 2.33 (Sources). Theorem 2.32 is the theorem of `reversal_core`, which proves the items of Theorem 2.25 directly for TRX, with the templates $v = a$, $J(v) = p$, $r = v$ and $f(s) = f(v)$, and derives TR_2 from them. `one_sided_transport` observes that TR^- is the restriction of TRX to equations with one constant side, needs no reversal of arbitrary formulas and leaves QE unchanged. In Lean the calculus X is the module `NativeReversal`, adopted from the package, with `rprf_iff_nprf`, `reversal_iff_pra` and `pra_iff_reversal`. The package reports one comparison of proof sizes: a certificate of the same closed goal, symmetry of an equation with the source 0, has 114 instruction packets and 5435 bytes when produced by the method of Section 2.3.4 and 37 packets and 1259 bytes when produced with reversal.

2.3.4 The same restriction for the uncrossed rule

For transport in a formula template with an uncrossed conclusion the restriction of the source is also harmless, but the proof needs an induction. Let

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{TR}^-_F \qquad (s \not\equiv 0).$$

Definition 2.34 (The calculus Z)**.** Z is N_F with TR_F replaced by TR^-_F; its rules are HYP, I0, IS, TR^-_F, QE, DE, DI and QI.

The sans-serif letter Z names the calculus; the italic letters Z and N name the zero test $Z(c) = \mathrm{sel}(c, 1, 0)$ and the parity term $N(n) = I(n, 0;\, u.\, Z(u))$ of Section 1.1.

In Z the two eliminations with a template can be merged into one rule whose first premise is selected by the source. For a template $F(v)$ put $M(s, t; F) := Q_v F(v)$ if $s \equiv 0$ and $M(s, t; F) := (s = t)$ otherwise; then

$$\frac{\Gamma \vdash M(s,t;F) \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \textsc{se}$$

is QE when s is the literal 0 and $\textsc{tr}^-_F$ otherwise. This is the form in which `source_dispatch` states the calculus, and the reason for its name.

Theorem 2.35 (Literal-zero sources of uncrossed transport)**.** Z *and* N *are equivalent.*

Every instance of $\textsc{tr}^-_F$ is derivable in N by the Leibniz rule. For the converse it suffices to derive TR in Z; the seven other rules are common. The proof does not assume that substitution is admissible in Z: every auxiliary equation is derived at an arbitrary term by a derived rule applied at that term. By Theorem 2.4(i) and (ii), numerical cases and equation induction are derivable in Z, with no use of transport.

Lemma 2.36 (Equality at permitted sources)**.** Z *derives: $r = r$; from $p = q$ with $p \not\equiv 0$, the equations $q = p$ and $f(p) = f(q)$ for every term template f; from $p = q$ with $p \not\equiv 0$ and $p = r$, the equation $q = r$; from $r = s$ and $s = t$ with $s \not\equiv 0$, the equation $r = t$.*

Proof. Transport $J(r) = r$ along itself in the template $v = r$; the source is $J(r)$. For the other statements transport, along $p = q$ or along $s = t$, the equations $p = p$ in the template $v = p$, $f(p) = f(p)$ in the template $f(p) = f(v)$, $p = r$ in the template $v = r$, and $r = s$ in the template $r = v$. □

Lemma 2.37 (Joining through zero)**.** *If $p \not\equiv 0$ and $q \not\equiv 0$, then* Z *derives $p = q$ from $p = 0$ and $q = 0$.*

Proof. Transport $J(p) = J(p)$ along $p = 0$ in the template $J(v) = J(p)$; the result is $c = J(p)$. Symmetry at the source c gives $J(p) = c$, and transport along $J(p) = p$ in the template $v = c$ gives $p = c$. In the same way $q = c$. Symmetry at the source q gives $c = q$, and transport of $p = c$ along $c = q$ in the template $p = v$ gives $p = q$. The sources are p, q, c, $J(p)$ and $J(q)$. □

Define, with the selector of Section 1.1,

$$K(v) := \mathrm{sel}(v, 0, 1), \qquad H(v) := \mathrm{sel}(v, 1, 2), \qquad R(n) := I(n, 0;\, u.\, H(u)), \qquad L(n) := N(R(n)).$$

In the standard model R takes the values $0, 1, 2, 2, \ldots$ and L is 1 at 1 and 0 elsewhere; the proof derives what it needs and does not use this description.

Lemma 2.38 (A test for the argument one)**.** *For every term v,* Z *derives*

$$H(v) = SK(v), \quad R(1) = 1, \quad R(SSv) = 2, \quad N(1) = 1, \quad N(2) = 0, \quad L(1) = 1, \quad L(SSv) = 0, \quad L(Sv) = Z$$

Proof. All chains of equations below pass through middle terms with the root I or S, so each step is one of the operations of Theorem 2.36.

$H(v) = SK(v)$: by numerical cases on the matrix $H(y) = SK(y)$ at the term v. At 0: $H(0) = 1$ and $K(0) = 0$ by I0, hence $SK(0) = 1$ by congruence at the source $K(0)$, and $H(0) = SK(0)$ by symmetry at the source $SK(0)$ and transitivity through 1. At Sz: $H(Sz) = 2$ and $K(Sz) = 1$ by IS, hence $SK(Sz) = 2$, and the two equations are joined through 2.

$N(1) = Z(N(0))$ by IS, $N(0) = 0$ by I0, hence $Z(N(0)) = Z(0)$ by congruence at the source $N(0)$, and $Z(0) = 1$ by I0; so $N(1) = 1$. Likewise $N(2) = Z(N(1)) = Z(1) = 0$, the last by IS. In the same way $R(1) = H(R(0)) = H(0) = 1$.

IS and the first equation at the term $R(v)$ give $R(Sv) = H(R(v)) = SK(R(v))$. Then $R(SSv) = H(R(Sv)) = H(SK(R(v))) = 2$, by IS, congruence at the source $R(Sv)$, and IS for the selector.

$L(1) = N(R(1)) = N(1) = 1$ and $L(SSv) = N(R(SSv)) = N(2) = 0$, by congruence at the sources $R(1)$ and $R(SSv)$.

$L(Sv) = Z(v)$: by numerical cases on the matrix $L(Sy) = Z(y)$ at the term v. At 0, $L(1) = 1$ and $Z(0) = 1$ are joined through 1. At Sz, $L(SSz) = 0$ and $Z(Sz) = 0$, the latter by IS, are joined by Theorem 2.37; both left sides have the root I. □

Lemma 2.39 (Filter)**.** *For every term t, Z derives $t = \mathrm{sel}(Z(t), t, 0)$ in every context.*

Proof. By numerical cases on the matrix $y = \mathrm{sel}(Z(y), y, 0)$ at the term t. At 0: $Z(0) = 1$, so $\mathrm{sel}(Z(0), 0, 0) = \mathrm{sel}(1, 0, 0)$ by congruence at the source $Z(0)$, and $\mathrm{sel}(1, 0, 0) = 0$ by IS; symmetry at the left side, which has the root I, gives $0 = \mathrm{sel}(Z(0), 0, 0)$. At Sz: $Z(Sz) = 0$, so $\mathrm{sel}(Z(Sz), Sz, 0) = \mathrm{sel}(0, Sz, 0) = Sz$ by congruence at the source $Z(Sz)$ and I0, and symmetry at the left side gives the instance. □

Lemma 2.40 (Symmetry at the source zero)**.** *For every term t and every context Γ containing $0 = t$, Z derives $\Gamma \vdash t = 0$.*

Proof. If $t \equiv 0$, use reflexivity. Otherwise t is a permitted source. Put $A(n) := I(n, t; u.\, Su)$ and prove $n = A(n)$ by equation induction on n, with t as a parameter and $0 = t$ in the context.

Base. $A(0) = t$ by I0, and $t = A(0)$ by symmetry at the source $A(0)$. Transport of $0 = t$ along $t = A(0)$ in the template $0 = v$ gives $0 = A(0)$; the source is t.

Step. Assume $z = A(z)$ with z fresh. The source z is a variable, so congruence gives $Sz = SA(z)$. IS gives $A(Sz) = SA(z)$, and symmetry at the source $A(Sz)$ gives $SA(z) = A(Sz)$. Transport of $Sz = SA(z)$ along this equation in the template $Sz = v$ gives $Sz = A(Sz)$.

At the term 1 the induction gives $1 = A(1)$. IS gives $A(1) = SA(0)$, and congruence at the source $A(0)$ gives $SA(0) = St$; so $A(1) = St$, and transport at the source $A(1)$ gives $1 = St$. Congruence at the source 1 gives $L(1) = L(St)$. With $L(1) = 1$ and $L(St) = Z(t)$ from Theorem 2.38, the third operation of Theorem 2.36 at the source $L(1)$ gives $1 = L(St)$, and transport at the source $L(St)$ gives $1 = Z(t)$. Symmetry at the source 1 gives $Z(t) = 1$.

Transport of the equation $t = \mathrm{sel}(Z(t), t, 0)$ of Theorem 2.39 along $Z(t) = 1$ in the template $t = \mathrm{sel}(v, t, 0)$ gives $t = \mathrm{sel}(1, t, 0)$, and transport along the instance $\mathrm{sel}(1, t, 0) = 0$ of IS in the template $t = v$ gives $t = 0$. □

The chain of the proof is $0 = t \;\Rightarrow\; 1 = St \;\Rightarrow\; Z(t) = 1 \;\Rightarrow\; t = 0$. Numerical cases are used only for equations that hold in every context, and the one induction that keeps the assumption $0 = t$ has the literal equation $n = A(n)$ as its matrix.

Proof of Theorem 2.35. Symmetry holds in Z without restriction: at a source other than the literal 0 by Theorem 2.36, and at the source 0 by Theorem 2.40 and cut. Congruence from $s = t$: if $s \not\equiv 0$ use Theorem 2.36; if $s \equiv t \equiv 0$ use reflexivity; if $s \equiv 0$ and $t \not\equiv 0$, derive $t = 0$, then $f(t) = f(0)$ at the source t, and reverse. Transitivity from $r = s$ and $s = t$: if $s \not\equiv 0$ use Theorem 2.36; if $s \equiv 0$ and one of r, t is the literal 0, a premise is the conclusion; otherwise reverse $0 = t$ and apply Theorem 2.37 to $r = 0$ and $t = 0$. These constructions are not circular, since symmetry at the source 0 was proved before unrestricted congruence and transitivity. Finally, from $s = t$ and $h(s) = r$, congruence gives $h(s) = h(t)$, symmetry gives $r = h(s)$ and transitivity gives $r = h(t)$, which is TR. □

Remark 2.41 (Sources and comparison). Theorem 2.35 and its lemmas are from `source_dispatch`, whose calculus has DP in place of QI (Theorem 2.6). The package also derives unrestricted transport in a formula template inside its calculus, by induction on the template; here that step is the Leibniz rule of N, since the equivalence needs TR only. In Lean the theorem is `zprf_iff_nprf`, and the recovery of unrestricted transport is `ZKernel.trF` for a class `ZKernel` that has equation induction and no logical rule. The contrast with Theorem 2.25 is the point of the two later packages: for the crossed rule, and for the reversing rule, a constant template reverses an equation in one step, and neither the test terms nor an induction is needed.

2.3.5 Replacement with no template

A transport rule can be given no template at all. For terms s, t, u let $\mathrm{Rep}_s^t(u)$ be the result of one traversal of u from the root: a subterm that is identical with s is replaced by t and is not traversed further; at any other subterm the constructor is kept and its immediate subterms are traversed; a variable or a 0 that is not replaced is unchanged. Under the binder of an iterator the bound variable is renamed away from s and t, so that only free occurrences of s are matched and no variable of t is captured. The test at a subterm precedes the descent, inserted copies of t are not traversed, and the operation is total even when t contains s. It is one pass and not rewriting to a normal form. In particular $\mathrm{Rep}_s^t(s) = t$.

Definition 2.42 (The calculi G and G_7)**.** The rule of global replacement is

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash u = r}{\Gamma \vdash r = \mathrm{Rep}_s^t(u)}\ \textsc{rep}.$$

G is N with TR replaced by REP; its rules are HYP, I0, IS, REP, QE, DE, DI and QI. G_7 is W with TR replaced by REP; its seven rules are HYP, I0, IS, REP, QE, DE and DI_{w}.

The rule cannot choose which occurrences of s to replace. A chosen template is recovered by planting a marker that cannot occur by accident.

Lemma 2.43 (Equality from I0 and REP)**.** *From I0 and REP alone the following are derivable: from $s = t$ and $s = u$ the equation $u = t$; reflexivity; symmetry; transitivity.*

Proof. REP applied to $s = t$ and $s = u$ concludes $u = \mathrm{Rep}_s^t(s)$, which is $u = t$. This joining, applied to $J(a) = a$ and $J(a) = a$, gives $a = a$; applied to $a = a$ and $a = b$ it gives $b = a$. For transitivity let $a = b$ and $b = e$ be given: symmetry gives $b = a$, and the joining applied to $b = e$ and $b = a$ gives $a = e$. □

Only matches at the root are used in this proof. Let $|T|$ be the number of nodes of a term T, the body of every iterator included; renaming of variables does not change it.

Lemma 2.44 (Exact marking)**.** *Let $h(v)$ be a term template, s and t terms, and*

$$n := 1 + |h| + |s| + |t|, \qquad K := I(0, s;\, w.\, S^n 0).$$

Then the occurrences of K in $h(K)$ are exactly the copies inserted for v, and

$$\mathrm{Rep}_K^s(h(K)) = h(s), \qquad \mathrm{Rep}_K^t(h(K)) = h(t).$$

Proof. K has more nodes than h. Two subterm occurrences of a term are disjoint or nested. An occurrence in $h(K)$ of a subterm identical with K that is disjoint from all inserted copies lies in a part of h and has at most $|h|$ nodes. One that lies properly inside an inserted copy has fewer nodes than K. One that properly contains an inserted copy has more nodes than K. None of these is identical with K, so the occurrences are the inserted copies. Below a binder of h the inserted copy and the term compared with it are subject to the same renaming, which preserves the number of nodes. The traversal replaces exactly these copies and stops at each, which gives the two identities. □

The identities are facts about syntax; no rule of the calculus is applied in them. The term K keeps the free variables of s, and $K = s$ is an instance of I0.

Lemma 2.45 (Chosen templates from global replacement)**.** *From I0 and REP alone: from $s = t$ the equation $h(s) = h(t)$, for every term template h; and the rule TR.*

Proof. Let K be as in Theorem 2.44. From $K = s$ and $s = t$, transitivity gives $K = t$. Reflexivity gives $h(K) = h(K)$. REP applied to $K = s$ and $h(K) = h(K)$ concludes $h(K) = h(s)$, and applied to $K = t$ and $h(K) = h(K)$ it concludes $h(K) = h(t)$, by Theorem 2.44. The joining of Theorem 2.43, applied to $h(K) = h(t)$ and $h(K) = h(s)$, gives $h(s) = h(t)$. For TR, from $s = t$ and $h(s) = r$: symmetry gives $r = h(s)$ and transitivity gives $r = h(t)$. □

For example let $s = t$ be given and let $h(v) := I(v, s;\, w.\, s)$, in which the two displayed copies of s are to stay. Replacement of s in $h(s)$ would change all three copies. In $h(K) = I(K, s;\, w.\, s)$ the marker is too large to occur in the two fixed copies, and the two replacements produce $I(s, s;\, w.\, s)$ and $I(t, s;\, w.\, s)$.

Lemma 2.46 (REP in N)**.** *Every instance of REP is an instance of TR.*

Proof. The occurrences of s that the traversal of u replaces are disjoint. Replace them by a fresh variable v to obtain a term template $h(v)$; then $u \equiv h(s)$ and $\mathrm{Rep}^t_s(u) \equiv h(t)$. If no occurrence is replaced, h is constant. TR with this template, applied to $s = t$ and $h(s) = r$, concludes $r = h(t)$. □

Thus REP is TR with the template computed from the two premises.

Theorem 2.47 (Replacement with no template)**.** G, G_7 *and* N *are equivalent.*

Proof. REP is derivable in N by Theorem 2.46, and TR is derivable in G and in G_7 by Theorem 2.45. This settles G. For G_7 the introductions are treated as in Theorem 2.17: an instance of DI_w is derivable in N, and QI and DI are derivable in G_7 by the proof given there, with the alias c, transport in an equation template derived from TR, and cut, which is admissible in G_7 by Theorem 2.3. □

Remark 2.48 (Substitution and certificates). A substitution can create new occurrences of the source in the second premise of REP, so it does not map instances to instances; injective renaming does. Substitution is admissible through Theorem 2.2. An instance of REP in a certificate consists of its two premise derivations; the price is paid in the derivations, which grow by the marker.

Remark 2.49 (Sources). The section is `template_free`, whose calculus is G_7. In Lean the calculus G is `TPrf`, with `tprf_iff_nprf`; the combination G_7 with the single introduction is not formalized. The compact implementation of the package works with the restricted inventory of Theorem 2.62 and uses the aliases $C_s := I(2, 1;\, u.\, s)$ and $K := I(0, C_s;\, u.\, S^n 0)$, so that $C_s = s$ is a permitted instance of IS and $K = C_s$ a permitted instance of I0, also when s is the literal 0.

2.3.6 Backward replacement

The rule REP computes its conclusion from its two premises. Its goal-directed counterpart computes its second premise from its conclusion and its first premise. It uses the operation Rep of Section 2.3.5 with the two sides of the first premise exchanged: $\mathrm{Rep}^s_t(u)$ is the result of one traversal of u from the root in which every subterm identical with t is replaced by s and is not traversed further, the bound variables of u being renamed away from s and t. Inserted copies of s are not traversed, so the operation is total also when s contains t, and $\mathrm{Rep}^s_t(t) \equiv s$.

Definition 2.50 (The calculi B and B_7)**.** The rule of backward replacement is

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash \mathrm{Rep}_t^s(u) = r}{\Gamma \vdash r = u}\ \textsc{brep}.$$

B is N with TR replaced by BREP; its rules are HYP, I0, IS, BREP, QE, DE, DI and QI. B_7 is W with TR replaced by BREP; its seven rules are HYP, I0, IS, BREP, QE, DE and DI_W.

For a conclusion $r = u$ and a first premise $s = t$ the second premise is determined. An instance carries no template, no selection of occurrences and no term besides those of its conclusion and of its first premise. The source s may be the literal 0, and t may occur in s. Matching is a test on syntax: a bound variable of u never matches a free variable of t.

Among the transport rules of the section, BREP is closest to REP: both are instances of TR whose template is computed by one traversal (Theorems 2.46 and 2.51), and neither can choose which occurrences to replace. They differ in what is matched and where the result goes. REP matches the source s in the left side u of its second premise and puts $\mathrm{Rep}_s^t(u)$ into the conclusion. BREP matches the target t in the right side u of its conclusion and puts $\mathrm{Rep}_t^s(u)$ into the second premise. A checker that works from the goal backwards therefore computes the second premise of BREP from the goal and the first premise, whereas for REP it has to be given the term u, which the conclusion $r = \mathrm{Rep}_s^t(u)$ does not determine. Neither rule is an instance of the other: REP applied to $s = t$ and $\mathrm{Rep}_t^s(u) = r$ concludes $r = \mathrm{Rep}_s^t(\mathrm{Rep}_t^s(u))$, which is t and not u when $u \equiv s$ and t does not occur in s. In the proofs below the marker of Theorem 2.44 is used unchanged, with the first premises $s = K$ and $t = K$ in place of the first premises $K = s$ and $K = t$ of Theorem 2.45.

Lemma 2.51 (BREP in N)**.** *Every instance of BREP is an instance of TR.*

Proof. The occurrences of t that the traversal of u replaces are disjoint, and after the renaming of the bound variables of u away from s and t they are free occurrences. Replace them by a fresh variable v to obtain a term template $h(v)$; then $u \equiv h(t)$ and $\mathrm{Rep}_t^s(u) \equiv h(s)$, and no variable of s is captured in $h(s)$. If no occurrence is replaced, h is constant. TR with this template, applied to $s = t$ and $h(s) = r$, concludes $r = h(t)$, which is $r = u$. □

Thus BREP is TR with the template computed from the conclusion, as REP is TR with the template computed from the second premise.

Lemma 2.52 (Equality from I0 and BREP)**.** *From I0 and BREP alone the following are derivable over an arbitrary context: from $s = t$ and $s = r$ the equation $r = t$; reflexivity; symmetry; transitivity.*

Proof. BREP with the first premise $s = t$ and the conclusion $r = t$ has the second premise $\mathrm{Rep}_t^s(t) = r$, which is $s = r$, since the traversal matches at the root, also when one of s, t contains the other. This is the joining of Theorem 2.43, from $s = t$ and $s = u$ to $u = t$, with r in place of u. The proof of Theorem 2.43 uses REP only through this joining and otherwise only I0, so it gives reflexivity, symmetry and transitivity here as well. □

Lemma 2.53 (Chosen templates from backward replacement)**.** *From I0 and BREP alone: from $s = t$ the equation $h(s) = h(t)$, for every term template h; and the rule TR.*

Proof. Let K be as in Theorem 2.44. $K = s$ is an instance of I0, and symmetry gives $s = K$. Transitivity of $K = s$ and $s = t$ gives $K = t$, and symmetry gives $t = K$ (Theorem 2.52). BREP with the first premise $s = K$ and the conclusion $h(s) = h(K)$ has the second premise $\mathrm{Rep}_K^s(h(K)) = h(s)$, which is $h(s) = h(s)$ by Theorem 2.44; reflexivity provides it, and the conclusion is $h(s) = h(K)$. In the same way the first premise $t = K$ and reflexivity of $h(t)$ give $h(t) = h(K)$, since $\mathrm{Rep}_K^t(h(K)) \equiv h(t)$. Symmetry and transitivity give $h(s) = h(t)$. For TR, from $s = t$ and $h(s) = r$: the joining of Theorem 2.52, applied to $h(s) = h(t)$ and $h(s) = r$, gives $r = h(t)$. □

The proof covers constant templates, templates with several occurrences of v and occurrences below any number of binders. For the template $h(v) := I(v, s; w.\, s)$ of the example after Theorem 2.45, the two steps with the first premises $s = K$ and $t = K$ act on $h(K) = I(K, s; w.\, s)$ and leave the two fixed copies of s alone. The derivation of TR uses no logical rule, no successor computation and no induction.

Theorem 2.54 (Backward replacement)**.** B, B_7 *and* N *are equivalent.*

Proof. BREP is derivable in N by Theorem 2.51, and TR is derivable in B and in B_7 by Theorem 2.53. The six other rules of B are those of N, which settles B. For B_7 the introductions are treated as in the proof of Theorem 2.47: an instance of $\mathrm{DI_w}$ is derivable in N by Theorem 2.17, and QI and DI are derivable in B_7 by the proof of Theorem 2.17, with the alias c, transport in an equation template derived from TR (Theorem 2.4(iii)), and cut, which is admissible in B_7 by Theorem 2.3. Each translation replaces one inference by a derivation of the same sequent from the same premises over the same context; the marker has no free variable outside s, so the conditions on the eigenvariables of enclosing rules are kept, and no admissibility of substitution in B is assumed. □

A substitution can create new occurrences of the target in the right side of the conclusion of BREP that are not matched in its second premise, so, as for REP (Theorem 2.48), substitution does not map instances to instances, and it is admissible through Theorem 2.2.

Sources other than the literal zero. Let BREP^- be BREP with the side condition $s \not\equiv 0$ on the source, the term that the rule inserts. B^- is B and B_7^- is B_7 with BREP replaced by BREP^-. For the crossed rule with a template the restriction is lifted by Theorem 2.26, with a constant template and the first premise $c = 0$. That argument is not available here: the first premise $c = 0$ has the target 0, and $\mathrm{Rep}_0^c(u)$ replaces every leaf 0 of u, so the template is constant only when u contains no 0; and the marker of Theorem 2.53 needs the first premise $t = K$, which is excluded when $t \equiv 0$. The recovery below uses the logical rules. The terms $J(a)$ and $c = J(0) = I(0, 0; u.\, 0)$ are those of the beginning of the section.

Lemma 2.55 (Equality in B^-)**.** *In* B^-*, and in every calculus in which* I0*,* IS*,* QE*,* QI *and* BREP^- *are derivable, the following are derivable over an arbitrary context.*

(i) *Reflexivity; and joining from* $s = t$ *and* $s = r$ *to* $r = t$ *when* $s \not\equiv 0$*.*

(ii) *Symmetry, also when a side is the literal* 0*.*

(iii) $I(p, p; v.\, p) = p$ *for every term* p *and every* v *not free in* p*.*

(iv) *From* $p = 0$ *the equation* $c = p$*.*

(v) *Joining and transitivity without restriction.*

(vi) *From* $s = t$ *with* $s \not\equiv 0$ *and* $t \not\equiv 0$*, the equation* $h(s) = h(t)$ *for every term template* h*.*

Proof. (i) BREP^- with the first premise $J(a) = a$, an instance of I0 with the source $J(a) \not\equiv 0$, and the conclusion $a = a$ has the second premise $\mathrm{Rep}_a^{J(a)}(a) = a$, which is $J(a) = a$ again. Joining at a source $s \not\equiv 0$ is the joining of Theorem 2.52.

(ii) Let $u = r$ be given, and let $M := S^k 0$ with $k := |u|$, so that M is closed and has more nodes than u. $J(M) = M$ is an instance of I0 with the source $J(M) \not\equiv 0$. No subterm of u, also below a binder, is identical with M, so $\mathrm{Rep}_M^{J(M)}(u) \equiv u$, and BREP^- with the first premise $J(M) = M$ and the conclusion $r = u$ has the given equation $u = r$ as its second premise.

(iii) Let y be fresh and $E(y) := \big(I(y,p;\, v.\,p) = p\big)$, a literal equation. $E(Sz)$ is an instance of IS, because $p[I(z,p;\, v.\,p)/v] \equiv p$; so $\Gamma, E(z) \vdash E(Sz)$, the assumption unused, and QI gives $Q_y E(y)$. $E(0)$ is an instance of I0, and QE at the term p gives $E(p)$, which is $I(p,p;\, v.\,p) = p$.

(iv) If $p \equiv 0$, then $c = p$ is an instance of I0. Otherwise BREP$^-$ with the first premise $p = 0$, whose source is p, and the conclusion $p = c$ has the second premise $\mathrm{Rep}_0^p(c) = p$. The traversal of $c = I(0,0;\, u.\,0)$ does not match at the root, since $c \not\equiv 0$, and replaces the counter, the seed and the body, each the literal 0, by p, the bound variable being renamed away from p. So the second premise is $I(p,p;\, u.\,p) = p$, which is (iii). The conclusion is $p = c$, and (ii) gives $c = p$.

(v) Joining from $s = t$ and $s = r$: if $s \not\equiv 0$, use (i). If $s \equiv 0$, (ii) gives $t = 0$ and $r = 0$, (iv) gives $c = t$ and $c = r$, and joining at the source $c \not\equiv 0$ gives $r = t$. Transitivity from $a = b$ and $b = e$: (ii) gives $b = a$, and joining applied to $b = e$ and $b = a$ gives $a = e$.

(vi) The proof of Theorem 2.53: its two applications of BREP have the sources s and t, and its other steps are (i), (ii) and (v). □

Lemma 2.56 (Congruence at a zero endpoint)**.** *In* B^- *and in* B_7^-*, from* $0 = t$ *the equation* $h(0) = h(t)$ *is derivable over an arbitrary context, for every term template* h*.*

Proof. If $t \equiv 0$, use reflexivity; let $t \not\equiv 0$. By the proof of Theorem 2.17, the rule QI is derivable in B_7^- with $\mathrm{DI_w}$, DE and HYP and without an equality rule, so Theorem 2.55 applies to both calculi.

A variable equal to zero. Let b be a variable not free in h, and let $b = 0$ be derivable over a context Δ. Put $L := \mathrm{Rep}_0^b(h(0))$. Every occurrence of the term 0 is a leaf. In $h(0)$ the leaves 0 are those of h and the inserted copies; in $h(b)$ they are those of h, since the variable b contains no 0. Hence $\mathrm{Rep}_0^b(h(b)) \equiv L$ as well: both are h with v and every leaf 0 replaced by b, under the same renaming of bound variables away from b. BREP$^-$ with the first premise $b = 0$, whose source is a variable, and the conclusion $L = h(0)$ has the second premise $L = L$; with the conclusion $L = h(b)$ it has the second premise $L = L$ as well. Reflexivity gives $L = h(0)$ and $L = h(b)$ over Δ, and joining and symmetry (Theorem 2.55) give $h(0) = h(b)$.

A name for zero. Let $C(y,a,d) := I(y,a;\, v.\,d)$ with v not free in d, and $E(x,y) := \big(C(y,0,x) = C(y,0,0)\big)$. The formula $E(0,Sz)$ is an instance of reflexivity. Hence DI with the witness 0 and the premise $\Gamma, E(0,0) \vdash E(0,Sz)$, or in B_7^- the rule $\mathrm{DI_w}$ with the witness 0, which is in the mode of progress and has the premise $\Gamma, E(0,z) \vdash E(0,Sz)$, gives $\Gamma \vdash D_{x,y}E(x,y)$; in both the assumption is unused. Let b be fresh for Γ, h and t, and let $\Delta := \Gamma, Q_y E(b,y)$. Over Δ: $C(0,0,b) = 0$ and $C(0,0,0) = 0$ are instances of I0, so symmetry and transitivity give $E(b,0)$; HYP and QE at the term $S0$ give $E(b,S0)$, that is $C(S0,0,b) = C(S0,0,0)$; the instances $C(S0,0,b) = b$ and $C(S0,0,0) = 0$ of IS, symmetry and transitivity give $b = 0$. Only equations at the roots are compared; no congruence is used.

Completion. Over Δ, the equation $0 = t$ is derivable by weakening (Theorem 2.3), and transitivity with $b = 0$ gives $b = t$. Since $b \not\equiv 0$ and $t \not\equiv 0$, Theorem 2.55(vi) gives $h(b) = h(t)$. The first step gives $h(0) = h(b)$, and transitivity gives $h(0) = h(t)$ over Δ. The variable b occurs neither in Γ nor in $h(0) = h(t)$, so DE with the first premise $D_{x,y}E(x,y)$ concludes $\Gamma \vdash h(0) = h(t)$. □

Theorem 2.57 (Backward replacement at sources other than the literal zero)**.** B^-, B_7^-, B *and* N *are equivalent.*

Proof. Every instance of BREP$^-$ is an instance of BREP, so B^- and B_7^- derive only sequents of N by Theorem 2.54. Conversely, congruence from $s = t$ is derivable in both: if $s \not\equiv 0$ and $t \not\equiv 0$ by Theorem 2.55(vi); if $s \equiv 0$ by Theorem 2.56; if $t \equiv 0 \not\equiv s$, symmetry gives $0 = s$, Theorem 2.56 gives $h(0) = h(s)$, and symmetry gives $h(s) = h(0)$. From $s = t$ and $h(s) = r$, congruence gives $h(s) = h(t)$ and joining (Theorem 2.55(v)) gives $r = h(t)$, which is TR. In B^-

the other rules are those of N. In B_7^-, QI was derived before any equality rule was needed, and DI with a literal equation and the witness 0 is derived after TR by the proof of Theorem 2.17, with the alias c, transport in an equation template and cut. The order of the derivations is not circular: Theorems 2.55 and 2.56 use DI_w only in the mode of progress with the witness 0, which needs no equality rule, and the instance of DI that needs transport is derived only after TR. □

Remark 2.58 (Sources). The subsection is `backward_replacement`, which presents itself as an executable demonstrator of correctness for B and states Theorems 2.54 and 2.57; it takes the marker and the bridge to iPRA from `template_free` and Theorem 1.119, and finds no earlier statement of BREP in the corpus. No calculus of the subsection is formalized in Lean. The checker of the package implements B with unrestricted sources; B_7, B^- and B_7^- are written proofs only. Its tests are finite: 18 accepted open or closed sequents, with instances of all eight rules, zero and nonzero sources and five generated certificates of congruence by the marker, whose templates have fixed copies of the source, a repeated variable, an unused variable and one or two nested binders; 16 rejected certificates, among them both kinds of capture, an escaping eigenvariable, wrong computation equations, a wrong instance of a closure and progress at a compound matrix; 7 direct tests of capture avoidance under $D_{,}$, Q and I and of the absence of repeated replacement inside an inserted source; and 18 runs of the driver. An independent review in the package adds four accepted sequents, among them a closed certificate that introduces, uses and discharges the name for zero of Theorem 2.56, two rejected certificates and four runs of the driver. These tests concern particular certificates; the equivalence rests on the proofs above.

2.3.7 Replacement of one occurrence

A *term context with one hole* $C[\,]$ is a term with exactly one occurrence of a hole; the hole may lie in the body of an iterator, and $C[s]$ plugs s into the hole after the bound variables of C have been renamed away from s. The context consisting of the hole alone is allowed.

Definition 2.59 (The calculi L and N_u)**.** L is N with TR replaced by

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash C[s] = r}{\Gamma \vdash C[t] = r}\ \text{LOR},$$

for a term context C with one hole. N_u is N with TR replaced by the same rule for an arbitrary term template $h(v)$ in place of C: from $s = t$ and $h(s) = r$ infer $h(t) = r$. Each has the rules HYP, I0, IS, its replacement rule, QE, DE, DI and QI.

Theorem 2.60 (One occurrence suffices)**.** L, N_u *and* N *are equivalent.*

Proof. An instance of LOR is an instance of the rule of N_u, with the template obtained by putting a fresh variable into the hole. The rule of N_u is derivable in N: TR gives $r = h(t)$ and symmetry gives $h(t) = r$.

It remains to derive TR in L. LOR with the context consisting of the hole, applied to $J(a) = a$ and $J(a) = a$, gives $a = a$; applied to $s = t$ and $s = s$ it gives $t = s$. Now let $s = t$ and $h(s) = r$ be given. Rename the bound variables of h away from s and t and number the occurrences of v in h from 1 to k. Let h_j be h with the first j occurrences replaced by t and the others by s, and let C_j be h with the first $j-1$ occurrences replaced by t, the j-th by the hole and the others by s. Then $h_{j-1} \equiv C_j[s]$ and $h_j \equiv C_j[t]$, and C_j has one hole. Hence k applications of LOR lead from $h(s) = r$, which is $h_0 = r$, to $h_k = r$, which is $h(t) = r$, and symmetry gives $r = h(t)$. The occurrences are those of the template variable; a subterm of h that happens to be identical with s is part of every C_j and is not replaced. □

Remark 2.61 (Sources). The theorem for L is the auxiliary note of `kernel_refinement`, which states it against N_2: the occurrences in the left template are replaced one at a time, the equation is reversed, the occurrences in the right template are replaced, and the equation is reversed again. It answers the question of `one_context` whether a restriction to one occurrence is adequate under the binder of the iterator. In Lean, L is `LPrf` with `lprf_iff_nprf`; contexts with one hole are `Tm.Hole`, and the chain $h_0, \dots, h_k$ is `Reach.subst`, where below a binder the hole and the bound variable are exchanged (`Reach.iterH`). The calculus N_u is `UPrf` with `uprf_iff_nprf`. The package measures no checker for L: a traversal by position has a cost of its own, and the compact checkers share one opening routine between transport and QE.

2.3.8 A restricted inventory of computation instances

Several compact checkers of the packages accept a computation equation only when its seed, and for IS also the predecessor of its counter, is not the literal 0. Call an instance of I0 *permitted* if $a \not\equiv 0$, and an instance of IS permitted if $n \not\equiv 0$ and $a \not\equiv 0$. Let $c_* := I(2, 1;\, u.\, 0)$.

Lemma 2.62 (Repair of computation instances)**.** *Let a calculus have the permitted instances of I0 and IS and a transport rule from which transport in an equation template at sources other than the literal* 0 *is derivable without a computation equation. Then it derives every instance of I0 and of IS.*

Proof. $c_* = 0$ is a permitted instance of IS: the predecessor of the counter and the seed are 1, and the body is the constant 0. Write an instance that is not permitted as $P(0)$, where the equation template $P(v)$ has v at the offending positions:

$$I(0, v;\, u.\, h) = v, \qquad I(Sn, v;\, u.\, h) = h[I(n, v;\, u.\, h)/u],$$
$$I(Sv, a;\, u.\, h) = h[I(v, a;\, u.\, h)/u], \qquad I(Sv, v;\, u.\, h) = h[I(v, v;\, u.\, h)/u].$$

The variable v is fresh, and other occurrences of 0 are left alone. In each case $P(c_*)$ is a permitted instance. Transport in the template P along $c_* = 0$, at the source $c_* \not\equiv 0$, gives $P(0)$. □

The hypothesis on the transport rule holds for TR, TR^- (Theorem 2.25(iii)), TRX, TR_2 and TR^-_F. Hence every result of this chapter that uses $c = I(0, 0;\, u.\, 0)$ or another computation instance with a literal 0 holds for the restricted inventory as well: the instance is first derived by the lemma. The repair is one transport on two computation instances, and its only source is c_*.

Remark 2.63 (Sources). The lemma is proved in `witness_rule`, where c_* serves also as the second name for zero in Theorem 2.17, and in `source_dispatch`, which checks that the repair does not use transport at the source 0 before Theorem 2.35 is available. `reversal_core` and `template_free` use it in their certificate producers. The restricted inventory itself belongs to an earlier package, `certified_zero`.

2.4 Elimination of the closure by abstraction of the goal

This section draws on `abstraction_and_refinements`.

The rule QE is given a matrix $B(y)$ and a term t and concludes $B(t)$. Its goal-directed form is given the conclusion and the term t only, and computes the matrix by abstracting every occurrence of t in the conclusion. Together with BREP of Theorem 2.50 and DI_W of Theorem 2.15 this gives a calculus of seven rules in which neither the transport rule nor the elimination of a closure is given a template: the premises of each are computed from its conclusion and one further term or equation. The difficulty is completeness: an abstraction

cannot leave chosen occurrences of t in place, and the elimination at an arbitrary matrix has to be recovered without assuming the transport in compound formulas that it is used to prove.

2.4.1 Replacement in formulas and abstraction

The operation Rep_t^s of Section 2.3.6 extends to native formulas by the same traversal. The bound variables of Q and D, are renamed away from $\mathrm{FV}(s) \cup \mathrm{FV}(t)$, as those of iterators; an equation $u_1 = u_2$ becomes $\mathrm{Rep}_t^s(u_1) = \mathrm{Rep}_t^s(u_2)$; and $Q_y B$ and $D_{x,y} B$ become $Q_y \mathrm{Rep}_t^s(B)$ and $D_{x,y} \mathrm{Rep}_t^s(B)$. Only term nodes are compared with t, a node identical with t is replaced by s and not traversed further, and inserted copies of s are not traversed. The operation is total, also when s contains t. A bound variable of the formula never matches a free variable of t.

Lemma 2.64 (Abstraction)**.** *Let C be a term or a native formula and s, t terms, and let the bound variables of C be renamed away from $\mathrm{FV}(s) \cup \mathrm{FV}(t)$.*

(i) *The occurrences of t that the traversal of C replaces are pairwise disjoint free occurrences, and they depend on C and t only, not on s. Replacing them by a fresh variable v gives a template $H(v)$ with $C \equiv H(t)$ and $\mathrm{Rep}_t^s(C) \equiv H(s)$.*

(ii) *If y is not free in C or in t, then*

$$\mathrm{Rep}_t^y(C)[t/y] \equiv C, \qquad \mathrm{Rep}_t^y(C)[0/y] \equiv \mathrm{Rep}_t^0(C).$$

Proof. (i) At each node the traversal first compares the node with t and descends into its children exactly when the comparison fails. Which nodes are compared, and with what result, is therefore determined by C and t; the term s enters only as the inserted copy, which is not traversed. The traversal does not descend into a replaced occurrence, so two replaced occurrences are disjoint. A replaced occurrence is free in C, because the bound variables of C are distinct from the variables of t. Put v at the replaced occurrences to obtain $H(v)$. Then $H(t)$ is C, and $H(s)$ is the result of the traversal, in which no variable of s is captured because the bound variables of H avoid $\mathrm{FV}(s)$.

(ii) Rename the bound variables of C away from $\mathrm{FV}(t) \cup \{y\}$ and let H be the template of (i); by (i) it serves for $s := y$ and for $s := 0$, since $\mathrm{FV}(0)$ is empty. Then $\mathrm{Rep}_t^y(C) \equiv H(y)$ and $\mathrm{Rep}_t^0(C) \equiv H(0)$. The variable y is not free in C, so its free occurrences in $H(y)$ are exactly the positions of v. Hence $H(y)[t/y] \equiv H(t) \equiv C$, with no capture because the bound variables of H avoid $\mathrm{FV}(t)$, and $H(y)[0/y] \equiv H(0)$. □

For terms, (i) is the argument of Theorem 2.51. The identities of (ii) are facts about syntax and not derivations of equations.

Lemma 2.65 (Exact marking in a formula)**.** *Let $C(v)$ be a term template or a formula template, p a term, and let the bound variables of C be renamed away from $\mathrm{FV}(p) \cup \{v\}$. Let $|C|$ be the number of nodes of C, the nodes of its terms and the bodies of its iterators included, and put*

$$n := 1 + |C|, \qquad K := I(0, p;\, w.\, S^n 0).$$

Then the occurrences of K in $C(K)$ are exactly the copies inserted for v, and $\mathrm{Rep}_K^a(C(K)) \equiv C(a)$ for every term a. Moreover $K = p$ is an instance of $\imath 0$ and $\mathrm{FV}(K) = \mathrm{FV}(p)$.

Proof. The body $S^n 0$ alone has $n+1$ nodes, so $|K| > n > |C|$. Only term nodes are compared with K. Two term occurrences in $C(K)$ are disjoint or nested. A term occurrence that neither contains nor lies inside an inserted copy is an occurrence of a subterm of C and has at most $|C|$ nodes; one that lies properly inside an inserted copy has fewer nodes than K; one that

properly contains an inserted copy has more. None of these is identical with K, also after the renaming of bound variables, which preserves the number of nodes. So the traversal reaches every inserted copy, since no node above it matches, replaces it by a and stops there, and changes nothing else. The result is C with a at the positions of v, which is $C(a)$. The last sentence is immediate from the form of K. □

Theorem 2.44 is the same argument for term templates, with the seed s and a larger n. The proof allows several occurrences of v, no occurrence, and occurrences below any number of iterator, Q and D, binders. The bound is on the template C, not on $C(K)$.

2.4.2 The rule and the calculus

Definition 2.66 (The calculus A_7)**.** The rule of elimination by abstraction is

$$\frac{\Gamma \vdash Q_y \operatorname{Rep}_t^y(A) \qquad \Gamma \vdash \operatorname{Rep}_t^0(A)}{\Gamma \vdash A}\ \text{AQE},$$

where y is a variable not free in A or in t. A_7 is B_7 with QE replaced by AQE; its seven rules are HYP, I0, IS, BREP, AQE, DE and DI_w, with BREP at unrestricted sources, the literal 0 included.

For a conclusion A and a term t both premises of AQE are determined, up to the name of the bound variable y; an instance carries no matrix. The variable y is bound, so the condition on it only ensures that abstraction does not identify y with a variable of A or t.

Lemma 2.67 (AQE in N)**.** *Every instance of AQE is an instance of QE.*

Proof. Let $B(y) := \operatorname{Rep}_t^y(A)$. By Theorem 2.64(ii), $B(0) \equiv \operatorname{Rep}_t^0(A)$ and $B(t) \equiv A$, so the instance is QE with the matrix B, the premises $Q_y B(y)$ and $B(0)$ and the term t. □

The converse fails. QE with the matrix $y = x$, the premises $Q_y(y = x)$ and $0 = x$ and the term x concludes $x = x$; AQE with the conclusion $x = x$ and the term x asks for $Q_y(y = y)$ and $0 = 0$, and no other term gives the premise $0 = x$, since both sides of $x = x$ are the same term and are replaced together. An abstraction cannot leave a chosen occurrence in place; Theorem 2.65 recovers a chosen matrix.

Remark 2.68 (Substitution in A_7). Weakening, injective renaming and cut are admissible in A_7 by Theorem 2.3: weakening and cut change no term or formula of an instance of AQE, and an injective renaming commutes with Rep and so maps an instance to an instance. Substitution does not map instances to instances. AQE with the conclusion $x = z$, $z \not\equiv x$, and the term x has the premises $Q_y(y = z)$ and $0 = z$. The substitution of x for z maps it to the inference from $Q_y(y = x)$ and $0 = x$ to $x = x$, which by the preceding paragraph is not an instance of AQE. A substitution can also turn a witness of DI_w into the literal 0 (Theorem 2.18). For this reason the proofs below use no substitution into a derivation, and substitution is admissible in A_7 only through Theorem 2.2, after Theorem 2.71.

2.4.3 Introductions before elimination

Lemma 2.69 (Rules of A_7 derived without AQE)**.** *The following rules are derivable in* A_7 *over an arbitrary context by derivations that contain no instance of AQE.*

(i) *Reflexivity, symmetry, transitivity, congruence in every term template, TR, and transport in an equation template: from* $s = t$ *and* $a(s) = b(s)$ *to* $a(t) = b(t)$.

(ii) *QI.*

(iii) *DI, for every matrix and every witness.*

(iv) *Closure by cases: from* $\Gamma, B(0) \vdash B(Sz)$*, with* z *not free in* Γ *or in* $Q_y B(y)$*, to* $\Gamma \vdash Q_y B(y)$*, for every native matrix* B*.*

Proof. (i) Theorems 2.52 and 2.53 derive the equality rules, congruence and TR from I0 and BREP alone, and transport in an equation template is two applications of TR and the equality rules (Theorem 2.4(iii)).

(ii) and (iii) The proof of Theorem 2.17, from N to W, derives QI from $\mathrm{DI_w}$ in the mode of progress, DE and HYP, and the remaining case of DI, a literal equation with the witness 0, from $\mathrm{DI_w}$ in the mode of cases at the witness $c = I(0, 0; u.\, 0)$, the equation $c = 0$ of I0 and its symmetric form, transport in an equation template and cut. These are available by (i) and Theorem 2.68, and the proof uses no elimination of a closure.

(iv) Choose x not free in B or in Γ, and regard $B(y)$ as a matrix in x and y in which x is not used. At the closed witness c, $\mathrm{DI_w}$ is in the mode of cases, and its premise $\Gamma, B(0) \vdash B(Sz)$ is the given one, since x does not occur; z is not free in c. It concludes $\Gamma \vdash D_{x,y} B(y)$. DE with a fresh b opens $Q_y B(y)$, and its second premise $\Gamma, Q_y B(y) \vdash Q_y B(y)$ is HYP. □

2.4.4 Recovery of the ordinary elimination

Let $r(A)$ be the binder rank of Theorem 1.33, the number of occurrences of Q and $D_,$ in A. Substitution of terms and replacement Rep^s_t leave it unchanged.

Lemma 2.70 (Transport and elimination by rank)**.** *For every* $r \geq 0$ *the following hold in* A_7*, for every context* Γ*.*

(L_r) *If* $F(v)$ *is a formula template with* $r(F) \leq r$*,* $\Gamma \vdash s = t$ *and* $\Gamma \vdash F(s)$*, then* $\Gamma \vdash F(t)$*.*

(E_r) *If* $B(y)$ *is a matrix with* $r(B) \leq r$*,* $\Gamma \vdash Q_y B(y)$ *and* $\Gamma \vdash B(0)$*, then* $\Gamma \vdash B(t)$ *for every term* t*.*

Proof. By induction on r, proving (L_r) from (L_{r-1}) and (E_{r-1}), and then (E_r) from (L_r). The bound variables of F and B and the variable v are chosen fresh for the other data.

(L_r)*,* F *a literal equation.* This is transport in an equation template, Theorem 2.69(i).

(L_r)*,* $F(v) \equiv Q_y B(v, y)$. Then $r \geq 1$ and $r(B) \leq r - 1$. Let z be a variable not free in Γ, s, t or B, and $\Gamma' := \Gamma, B(t, 0)$. Over Γ', by weakening: $s = t$ and $Q_y B(s, y)$; symmetry gives $t = s$; HYP gives $B(t, 0)$; (L_{r-1}) with the template $B(v, 0)$ and $t = s$ gives $B(s, 0)$; (E_{r-1}) with the matrix $B(s, y)$ and the term Sz gives $B(s, Sz)$; and (L_{r-1}) with the template $B(v, Sz)$ and $s = t$ gives $B(t, Sz)$. Hence $\Gamma, B(t, 0) \vdash B(t, Sz)$, and closure by cases (Theorem 2.69(iv)) gives $\Gamma \vdash Q_y B(t, y)$, which is $F(t)$.

(L_r)*,* $F(v) \equiv D_{x,y} B(v, x, y)$. Then $r(B) \leq r - 1$. Let b and z be distinct variables not free in Γ, s, t or B, and $\Delta := \Gamma, Q_y B(s, b, y)$. Over $\Delta, B(t, b, 0)$ the steps of the preceding case, with the templates $B(v, b, 0)$ and $B(v, b, Sz)$ and the matrix $B(s, b, y)$, give $B(t, b, Sz)$. $\mathrm{DI_w}$ with the matrix $B(t, x, y)$ and the witness b, which is not the literal 0, is in the mode of cases and concludes $\Delta \vdash D_{x,y} B(t, x, y)$; z is not free in Δ, in b or in the conclusion. DE with the first premise $\Gamma \vdash D_{x,y} B(s, x, y)$ and the eigenvariable b, which is not free in Γ, in $D_{x,y} B(s, x, y)$ or in $D_{x,y} B(t, x, y)$, concludes $\Gamma \vdash D_{x,y} B(t, x, y)$, which is $F(t)$.

These three cases exhaust the formulas of rank at most r. Every transport and elimination invoked in the last two acts on a matrix of rank at most $r - 1$.

(E_r). Let $r(B) \leq r$, $\Gamma \vdash Q_y B(y)$, $\Gamma \vdash B(0)$, and let t be a term. Let the bound variables of B avoid $\mathrm{FV}(t)$, let v be fresh and $K := I(0, t; w.\, S^n 0)$ the marker of Theorem 2.65 for the template $C(v) := B(v)$ and the term $p := t$. Let y' be a variable not free in B or in t. By Theorem 2.65,

$$\mathrm{Rep}^{y'}_K(B(K)) \equiv B(y'), \qquad \mathrm{Rep}^0_K(B(K)) \equiv B(0).$$

Hence AQE with the conclusion $B(K)$ and the term K has the premises $Q_{y'}B(y')$, which is $Q_yB(y)$ up to the name of the bound variable, and $B(0)$. It concludes $\Gamma \vdash B(K)$. I0 gives $\Gamma \vdash K = t$, and (L_r) with the template $B(v)$ gives $\Gamma \vdash B(t)$. For $r = 0$ this uses (L_0) only. □

The order $(\mathrm{L}_0), (\mathrm{E}_0), (\mathrm{L}_1), (\mathrm{E}_1), \ldots$ is not circular: Theorem 2.69 uses no elimination of a closure, transport at rank r uses elimination at rank $r - 1$ only, and elimination at rank r uses AQE once, at the marker, and transport at rank r. No transport of a compound formula enters the derivation of the introductions, and no substitution into a derivation is used.

Theorem 2.71 (Elimination by abstraction)**.** A_7, B_7 *and* N *are equivalent.*

Proof. B_7 and N are equivalent by Theorem 2.54. *From* A_7 *to* N. HYP, I0, IS and DE are common; an instance of BREP is an instance of TR (Theorem 2.51), an instance of AQE one of QE (Theorem 2.67), and an instance of $\mathrm{DI_w}$ is derivable in N (Theorem 2.17). *From* N *to* A_7. By induction on a derivation in N: HYP, I0, IS and DE are common; TR, QI and DI are derivable by Theorem 2.69; and an instance of QE with the matrix B is derivable by $(\mathrm{E}_{r(B)})$ of Theorem 2.70. Each derived rule acts over the context of the inference it replaces and introduces only fresh variables, so the conditions on the eigenvariables of enclosing inferences are kept. Both translations leave terms, formulas, contexts and conclusions unchanged. □

By Theorem 2.2, A_7, T_{eq} and iPRA are bi-interpretable with whole domains, actual equality and identity numerical composites, and $\vdash_{\mathsf{A}_7} 0 = S0$ holds exactly when iPRA is inconsistent. The translations between A_7 and N are the identity on terms and formulas, so the bridge of Theorem 1.119 is used unchanged.

The proof needs BREP at the source 0. For the restriction BREP^- of Theorem 2.57 transport in an equation template is derived with QE and QI (Theorem 2.55(iii), Theorem 2.56), and here QE is recovered only after that transport; the restricted calculus is left open (Theorem 2.165).

Remark 2.72 (Sources). The section is the abstraction part of `abstraction_and_refinements`, whose calculus $\mathcal{A}_7$ is A_7; the package takes BREP, $\mathrm{DI_w}$ and the bridge from this chapter and from Theorem 1.119, and presents its proof twice, as a section and as a bootstrap note. The package proves the equality bootstrap again from I0 and BREP; the article cites Theorems 2.52 and 2.53. Its marker has the seed t and $N := |C| + 1$, as in Theorem 2.65. It derives closure by cases from the general introduction by cases; the article introduces at the witness c directly. The Lean module `NativeAbstraction` formalizes A_7 and Theorem 2.71 for it (`APrf`, `aprf_iff_nprf`), along the proof above: replacement and abstraction are one operation that puts a term at the occurrences of t and renames the other variables, so the identities of Theorem 2.64(ii) are instances of its commutation with substitution; weakening and injective renaming of derivations are proved by induction on derivations; and Theorem 2.70 is one induction on the number of binders (`APrf.trEl`). The tests of the package are finite: 804 checks of abstraction and of open inferences, 521 checks of the recovery by the marker, 3 further identities of capture avoidance, 28 runs of the driver on closed certificates, and 1521 independent symbolic checks on 300 generated syntax trees, repeated with assertions disabled. The 45-node predecessor of the package is Theorem 1.74; its refinements of checkers for inherited presentations are reported in the chapter on certificates.

2.5 Induction and uniqueness of iteration

This section draws on `scalar_uniqueness`, `orbit_closure`, `crossed_uniqueness` and the rigidity section of `reductions`.

Induction for literal equations can be replaced by a rule that has no induction hypothesis in its premise: a term that satisfies the recurrence of an iterator is that iterator. The three packages state the rule in three forms. `scalar_uniqueness` concludes the equation at a

supplied target; this is the rule UR below, and Sections 2.5.1 to 2.5.7 prove that it replaces equation induction. `orbit_closure` concludes the closure of the equation, with no target, and isolates what the restriction of the rule to the body Su already gives. `crossed_uniqueness` adds a premise for the seed and crosses the conclusion; on fully normalized terms that rule also replaces transport and the two computation equations. The second and third forms are treated in Section 2.5.8, which reduces each to UR; the calculus of `crossed_uniqueness` is the one calculus of the chapter that is stated on normal forms.

2.5.1 The rule and the two theorems

Let $f(z)$ and $h(u)$ be term templates, and write $h(f(z))$ for $h[f(z)/u]$. The rule of *uniqueness of iteration* is

$$\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash f(t) = I(t, f(0);\, u.\, h(u))}\ \text{UR},$$

where z is not free in Γ, in t or in h. The body h may have other free variables, which are parameters. The premise is derived in Γ itself, with no added assumption; the seed $f(0)$ is read off the template, so there is no premise for the base; and the body receives one state and no counter. The rule is a scheme over terms: z is a numerical variable and nothing ranges over functions.

Definition 2.73 (The calculus U and the theory T_{UR})**.** U is N with QI replaced by UR; its eight rules are HYP, I0, IS, TR, QE, DE, DI and UR. The ordinary theory T_{UR} is T_{eq} (Section 1.5) with its rule of induction for literal equations replaced by UR, read for contexts of ordinary formulas: intuitionistic logic with equality, the two computation equations, coverage $x = 0 \vee \exists y\, x = Sy$, separation $0 = S0 \to \bot$, and UR.

In U the introduction DI treats atomic and compound matrices alike, and no rule looks at a witness; substitution maps instances of UR to instances after a renaming of z (Theorem 2.3).

Definition 2.74 (Successor uniqueness)**.** SU is the rule UR restricted to the body $h(u) := Su$: from $\Gamma \vdash f(Sz) = Sf(z)$ infer $\Gamma \vdash f(t) = I(t, f(0);\, u.\, Su)$. The theory T_{SU} is T_{UR} with UR replaced by SU.

An instance of SU supplies no body. Every derivation of T_{SU} is a derivation of T_{UR}, and the lemmas below say which of them use SU only.

Proposition 2.75 (Uniqueness from induction)**.** *UR is a derived rule of* N *and of* T_{eq}.

Proof. Let $g(y) := I(y, f(0);\, u.\, h(u))$ and apply equation induction to $f(y) = g(y)$. Base: $g(0) = f(0)$ by I0, and symmetry. Step: assume $f(z) = g(z)$. The premise gives $f(Sz) = h(f(z))$, congruence gives $h(f(z)) = h(g(z))$, and IS gives $g(Sz) = h(g(z))$; hence $f(Sz) = g(Sz)$. □

In Lean this derived rule is `NPrf.uniq`.

Theorem 2.76 (Uniqueness of iteration in place of equation induction)**.** T_{UR} *and* T_{eq} *derive the same sequents of ordinary formulas. Consequently* T_{UR} *is bi-interpretable with* iPRA *with whole domains, actual equality and identity numerical composites.*

Theorem 2.77 (The native form)**.** U *and* N *are equivalent.*

One half of each theorem is Theorem 2.75. The other half, the derivation of induction for literal equations from UR, occupies Sections 2.5.2 to 2.5.7. Its order avoids a circle: uniqueness gives induction for equations of the form $q(n) = 0$ and, separately, a predecessor; truncated subtraction and an explicit accumulator give a term that reflects equality; and only

then is induction for arbitrary equations recovered. Pairing and the compiler for primitive recursion (Theorem 1.102) are not used; they become available afterwards. All derivations in Sections 2.5.2 to 2.5.7 are in T_{UR}, over an arbitrary fixed context Γ of ordinary formulas, and variables that are generalized or used as eigenvariables are not free in Γ.

2.5.2 First consequences of uniqueness

Lemma 2.78 (Common update, constancy, identity)**.** (i) *If* $f(Sz) = h(f(z))$, $g(Sz) = h(g(z))$ *and* $f(0) = g(0)$, *then* $f(t) = g(t)$.

(ii) *If* $f(Sz) = f(z)$, *then* $f(t) = f(0)$.

(iii) $I(n, 0;\, u.\, Su) = n$.

(iv) *If* $h(c) = c$, *then* $I(n, c;\, u.\, h(u)) = c$.

Proof. (i) UR for f and for g gives two iterators that differ only in the equal seeds. (ii) is (i) for f and the constant template $f(0)$, with $h(u) := u$. (iii) is UR for $f(z) := z$ and $h(u) := Su$, whose premise is reflexivity. (iv) is UR for the constant template c. □

Part (iii), and part (i) for the body Su, are derivable in T_{SU}. Part (ii) is uniqueness for the body u, and it is not an instance of SU.

Put $a + b := I(b, a;\, u.\, Su)$, $2a := a + a$ and $3a := a + a + a$. The computation equations give $a + 0 = a$ and $a + Sb = S(a + b)$, so $a + 1 = Sa$.

Lemma 2.79 (Addition and iteration)**.** *Addition is associative and commutative with the neutral element* 0, *and* $Sa + n = S(a + n)$. *For every body* h,

$$I(m + n, a;\, u.\, h) = I(n, I(m, a;\, u.\, h);\, u.\, h), \qquad I(Sn, a;\, u.\, h) = I(n, h(a);\, u.\, h).$$

Proof. Each statement is Theorem 2.78(i) or UR for functions of n. $0 + n = n$ is Theorem 2.78(iii). $S(a + n)$ has the seed Sa and the update S, hence equals $Sa + n$. The term $n + a$ has the seed $0 + a = a$ and satisfies $Sn + a = S(n + a)$, so it has the update S and equals $a + n$. The term $a + (b + n)$ has the seed $a + b$ and the update S, hence equals $(a + b) + n$. The term $I(m + n, a;\, u.\, h)$ has the seed $I(m, a;\, u.\, h)$ and the update h. The term $I(Sn, a;\, u.\, h)$ has the seed $I(1, a;\, u.\, h) = h(a)$ and the update h. □

The laws of addition in this lemma compare terms with the update S only, so they are derivable in T_{SU}; the two equations for an arbitrary body h use UR for h.

2.5.3 Retractions and induction for zero equations

Proposition 2.80 (A right inverse from successor uniqueness)**.** *Let* $e(x)$ *be a term template such that*

$$\Gamma \vdash e(0) = 0, \qquad \Gamma \vdash e(Se(x)) = Se(x)$$

for a variable x *not free in* Γ, *and put*

$$r(n) := I(n, 0;\, u.\, Se(u)).$$

Then for every term t

$$\Gamma \vdash e(r(t)) = t, \qquad \Gamma \vdash r(St) = St, \qquad \Gamma \vdash e(St) = St.$$

The derivation uses the two computation equations, equality reasoning and two instances of SU*; it uses neither coverage, nor separation, nor idempotence of* e.

Proof. Let z be a variable not free in Γ, in t or in e, and put $f(n) := e(r(n))$. By I0, $r(0) = 0$, so $f(0) = e(0) = 0$. By IS, $r(Sz) = Se(r(z))$, and the second hypothesis at the argument $r(z)$, obtained by substituting $r(z)$ for x, gives

$$f(Sz) = e(r(Sz)) = e\big(Se(r(z))\big) = Se(r(z)) = Sf(z).$$

SU for the template f gives $f(t) = I(t, f(0); u.\, Su)$; congruence with $f(0) = 0$ and Theorem 2.78(iii), the second instance of SU, give $f(t) = t$, that is, $e(r(t)) = t$. Then IS gives $r(St) = Se(r(t)) = St$. Finally, the second hypothesis at the argument $r(t)$ is $e(Se(r(t))) = Se(r(t))$, and replacing $e(r(t))$ by t on both sides gives $e(St) = St$. □

The first conclusion says that the term r is a right inverse of e: it is a term witness for the surjectivity of e, which is more than closure of the image of e under successor. The trajectory r applies S after e, and SU is applied to the projected orbit $e \circ r$; this is what makes the computation of $f(Sz)$ immediate.

Lemma 2.81 (Rigidity of retractions)**.** *Let $e(x)$ be a term template such that*

$$e(0) = 0, \qquad e(e(x)) = e(x), \qquad e(Se(x)) = Se(x)$$

are derivable for a variable x not free in Γ. Then $e(t) = t$ for every term t. The derivation is in T_{SU} and uses neither coverage nor separation.

Proof. By Theorem 2.80, $e(r(t)) = t$. Idempotence at the argument $r(t)$ gives

$$e(t) = e\big(e(r(t))\big) = e(r(t)) = t.$$ □

Corollary 2.82 (Rigidity without idempotence)**.** *Let $e(x)$ be a term template such that $e(0) = 0$ and $e(Se(x)) = Se(x)$ are derivable for a variable x not free in Γ. Then coverage and SU derive $e(t) = t$ for every term t, with the computation equations and equality reasoning. Neither separation nor idempotence is used.*

Proof. Coverage gives $t = 0 \vee \exists y\, t = Sy$. If $t = 0$, then $e(t) = e(0) = 0 = t$. If $t = Sy$ for a variable y not free in Γ or t, then Theorem 2.80 gives $e(Sy) = Sy$, hence $e(t) = t$. Elimination of the disjunction and of the existential quantifier concludes $e(t) = t$. □

Of the three hypotheses of Theorem 2.81, therefore, coverage may replace idempotence, and idempotence makes coverage unnecessary; the two remaining hypotheses alone already give $e(St) = St$ for every t.

Lemma 2.83 (Induction from a numerical split)**.** *Let $A(n)$ be an ordinary formula with $\Gamma \vdash A(0)$ and $\Gamma, A(z) \vdash A(Sz)$. Suppose that $\Gamma \vdash A(n) \vee \neg A(n)$ and that a term template $e(n)$ satisfies $\Gamma, A(n) \vdash e(n) = n$ and $\Gamma, \neg A(n) \vdash e(n) = 0$, with n not free in Γ. Then $\Gamma \vdash A(t)$ for every term t.*

Proof. By cases, $A(e(n))$: if $A(n)$, then $e(n) = n$; otherwise $e(n) = 0$ and $A(0)$ holds. The first property of e at the argument $e(n)$ gives $e(e(n)) = e(n)$. The step at $e(n)$ gives $A(Se(n))$, hence $e(Se(n)) = Se(n)$. From $A(0)$, $e(0) = 0$. Theorem 2.81 gives $e(n) = n$, and $A(e(n))$ becomes $A(n)$. □

The lemma is a statement about a split that has been justified; it gives no induction for formulas that are not decided by a term.

Lemma 2.84 (Induction for zero equations)**.** *Let $q(n)$ be a term template. If $\Gamma \vdash q(0) = 0$ and $\Gamma, q(z) = 0 \vdash q(Sz) = 0$ with z fresh, then $\Gamma \vdash q(t) = 0$.*

Proof. Separation gives $Sv \neq 0$: from $Sv = 0$, the template $\mathrm{sel}(w, 0, 1)$ gives $1 = 0$. With coverage, $q(n) = 0$ is decided. The template $e(n) := \mathrm{sel}(q(n), n, 0)$ equals n if $q(n) = 0$ and equals 0 if $q(n)$ is a successor. Theorem 2.83 applies. □

Theorems 2.83 and 2.84 use uniqueness only through Theorem 2.81. Hence:

Theorem 2.85 (Zero equations from successor uniqueness)**.** *T_{SU} derives induction for zero equations, in the form of Theorem 2.84, over arbitrary contexts of ordinary formulas.*

Remark 2.86 (Sources)*.* Theorems 2.81, 2.83 and 2.84 are reproduced in `scalar_uniqueness` from an earlier package, `scope_kernel`, which leaves the predecessor and the reflection of equality as an open bootstrap. The next three subsections close it. The three packages prove Theorem 2.81 in three ways. In `scope_kernel` and `scalar_uniqueness` the trajectory is $r'(n) := I(n, 0; u.\, h(u))$ with the unprojected body $h(u) := e(Se(u))$, and the first step, $e(r'(n)) = r'(n)$, is an instance of UR for the body h: by idempotence $e(h(u)) = h(u)$ and $h(e(u)) = h(u)$, so the template $e(r'(n))$ has the seed 0 and satisfies $e(r'(Sn)) = h(r'(n)) = h(e(r'(n)))$. The third hypothesis then gives $r'(Sn) = Sr'(n)$, SU gives $r'(n) = n$, and $e(n) = e(r'(n)) = r'(n) = n$. The same proof is Theorem 10.126. `orbit_closure` keeps r' and replaces the instance of UR by coverage on n: $r'(0) = 0 = e(0)$, and $r'(Sm) = e(Se(r'(m)))$ is a value of e, so idempotence gives $e(r'(Sm)) = r'(Sm)$; this places the derivation in T_{SU}, and Theorem 2.85 is the theorem of that package. The proof given above, through Theorem 2.80 with the body $Se(u)$, is from `reductions`; it needs neither the instance of UR nor coverage, and Theorem 2.82 is the other trade-off that it permits. In Theorems 2.83 to 2.85 coverage and separation are now used only in Theorem 2.84, to decide $q(n) = 0$ and so to justify the split $e(n) = \mathrm{sel}(q(n), n, 0)$; the right inverse does not remove them from that step.

Remark 2.87 (Successor rigidity in the clause chapter)*.* Theorem 5.79 derives induction for zero equations from the rule SR_0 (from $f(0) = 0$ and $f(Sz) = Sf(z)$ infer $f(t) = t$), which SU derives, with the trajectory $I(n, 0; u.\, e(Su))$ and with coverage. Its hypotheses on a formula E and a term e, $\Gamma \vdash E(e(x))$ and $\Gamma, E(x) \vdash e(x) = x$ with x fresh, together with the base $E(0)$ and the step $E(z) \vdash E(Sz)$, imply the three hypotheses of Theorem 2.81: substituting 0 for x in the second hypothesis and using the base gives $e(0) = 0$; substituting $e(x)$ for x and using the first gives $e(e(x)) = e(x)$; and the step at $e(x)$ gives $E(Se(x))$, whence $e(Se(x)) = Se(x)$. In the proof of Theorem 2.80 the two instances of SU combine into one instance of SR_0 for the template $e(r(n))$, whose premises $e(r(0)) = 0$ and $e(r(Sz)) = Se(r(z))$ are derived there. Hence the general statement of Theorem 5.79, $\Gamma \vdash E(t)$, holds over the computation equations and SR_0 without coverage: Theorem 2.81 gives $e(t) = t$, and $E(e(t))$ transports to $E(t)$. Coverage remains in the derivation of ZI there, where it proves $E(e(x))$ for $e(x) = \mathrm{sel}(q(x), x, 0)$.

Remark 2.88 (Coverage is not derivable from the other principles)*.* Theorem 2.81 removes coverage from the step of rigidity only, and coverage cannot be removed from the whole scalar presentation on the strength of SU. The product $\mathbb{N} \times \mathbb{N}$ of two copies of the standard model, with $0 = (0, 0)$ and every operation coordinatewise, satisfies the computation equations, separation and induction for literal equations over arbitrary contexts, and refutes coverage at $(0, 1)$ (Theorem 2.14). The derivation of UR in Theorem 2.75 uses equation induction, the computation equations and congruence only, so every instance of UR, and in particular of SU, is valid in the product. Hence intuitionistic logic with equality, the computation equations, separation and SU do not derive coverage. `reductions` checks the validity of SU in the product directly: its premise gives the successor recurrence in each coordinate, and external induction in each coordinate gives its conclusion. The model has the standard domain and the fixed language; it says nothing about an interpretation on a changed domain (Theorem 2.169).

2.5.4 A predecessor from uniqueness

The construction follows the predecessor of Theorem 1.65 (parity, three-cycle, ramp, comparator, saturating counter), with a ramp modified so that every induction of the original becomes an instance of Theorem 2.78. It uses coverage and does not use separation.

Coverage and the selector equations give, by cases on e,

$$\mathrm{sel}(e, Sa, Sb) = S\ \mathrm{sel}(e, a, b). \tag{2.1}$$

Recall $Z(c) = \mathrm{sel}(c, 1, 0)$ and $N(n) = I(n, 0;\, u.\, Z(u))$, and define

$$j(u) := Z(u)+2N(u), \qquad M(n) := I(n, 0;\, u.\, j(u)), \qquad g(u) := u+N(M(u))+1, \qquad G(n) := I(n, 0;\, u.\, g(u)).$$

Lemma 2.89 (Two clocks)**.** (i) $N(n) = 0 \vee N(n) = 1$; $Z(Z(N(n))) = N(n)$; $N(n) + Z(N(n)) = 1$; $N(N(n)) = N(n)$; $N(n+2) = N(n)$.

(ii) $N(2n) = 0$, $N(2n+1) = 1$ *and* $N(3n) = N(n)$.

(iii) $M(n) = 0 \vee M(n) = 1 \vee M(n) = 2$; $j(0) = 1$, $j(1) = 2$, $j(2) = 0$; $M(n+3) = M(n)$.

(iv) $M(3n) = 0$ *and* $M(3n+1) = 1$.

Proof. (i) By coverage on n: $N(0) = 0$, and $N(Sm) = Z(N(m))$ is a value of Z, which is 1 at 0 and 0 at a successor. The next three equations hold in the two cases $N(n) = 0$ and $N(n) = 1$, by $Z(0) = 1$, $Z(1) = 0$, $N(0) = 0$ and $N(1) = Z(N(0)) = 1$. Two computations give $N(n+2) = Z(Z(N(n)))$.

(ii) The template $N(2n)$ satisfies $N(2(Sn)) = N(2n+2) = N(2n)$, so Theorem 2.78(ii) gives $N(2n) = N(0) = 0$; then $N(2n+1) = Z(N(2n)) = 1$. The template $N(3n)$ has the seed 0 and satisfies $N(3(Sn)) = N(3n+3) = Z(N(3n+2)) = Z(N(3n))$, so UR gives $N(3n) = I(n, 0;\, u.\, Z(u)) = N(n)$.

(iii) $j(0) = Z(0) + 2N(0) = 1$. At a successor u, $Z(u) = 0$ and $2N(u)$ is 0 or 2. So j takes values in $\{0, 1, 2\}$, and so does M, by coverage on n. $j(1) = 0 + 2N(1) = 2$ and $j(2) = 0 + 2N(2) = 0$, since $N(2) = Z(N(1)) = 0$. Three computations give $M(n+3) = j(j(j(M(n))))$, and j permutes $0, 1, 2$ cyclically.

(iv) $M(3(Sn)) = M(3n+3) = M(3n)$, so Theorem 2.78(ii) gives $M(3n) = M(0) = 0$, and $M(3n+1) = j(M(3n)) = 1$. □

Lemma 2.90 (The ramp)**.** (i) $M(g(u)) = Z(M(u))$.

(ii) $M(G(n)) = N(n)$.

(iii) $G(Sn) = G(n) + N(n) + 1$, $G(n+2) = G(n) + 3$ *and* $G(n+3) = G(n) + N(n) + 4$.

(iv) $G(2n) = 3n$ *and* $G(3n) = 3G(n) + N(n)$.

Proof. (i) By the three cases for $M(u)$. If $M(u) = 0$, then $g(u) = Su$ and $M(Su) = j(0) = 1 = Z(0)$. If $M(u) = 1$, then $g(u) = SSu$ and $M(SSu) = j(j(1)) = 0 = Z(1)$. If $M(u) = 2$, then $N(2) = 0$, $g(u) = Su$ and $M(Su) = j(2) = 0 = Z(2)$.

(ii) The template $M(G(n))$ has the seed $M(0) = 0$ and, by (i), satisfies $M(G(Sn)) = M(g(G(n))) = Z(M(G(n)))$. UR gives $M(G(n)) = I(n, 0;\, u.\, Z(u)) = N(n)$.

(iii) $G(Sn) = g(G(n)) = G(n) + N(M(G(n))) + 1 = G(n) + N(N(n)) + 1$, and $N(N(n)) = N(n)$. Two steps give $G(n+2) = G(n) + N(n) + 1 + Z(N(n)) + 1 = G(n) + 3$, by $N(n) + Z(N(n)) = 1$. A third step gives $G(n+3) = G(n) + 3 + N(n+2) + 1 = G(n) + N(n) + 4$.

(iv) $G(2n)$ and $3n$ have the seed 0 and the update $v \mapsto v + 3$, by (iii); Theorem 2.78(i) gives $G(2n) = 3n$. For the second equation put

$$f(n) := G(3n), \qquad k(n) := 3G(n) + N(n), \qquad \kappa(v) := v + M(v) + 4.$$

By (ii) and Theorem 2.89(ii), $M(f(n)) = N(3n) = N(n)$. By Theorem 2.89(iv) and the two cases for $N(n)$, $M(k(n)) = N(n)$. By (iii), $f(Sn) = G(3n+3) = G(3n)+N(3n)+4 = \kappa(f(n))$. By (iii) and Theorem 2.89(i),

$$k(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)) = 3G(n) + 2N(n) + 4 = \kappa(k(n)).$$

Both templates have the seed 0, and Theorem 2.78(i) gives $f(n) = k(n)$. □

The update g differs from the update $u + M(u) + 1$ of the ramp in Theorem 1.65 away from the orbit of 0; the difference is what makes (i) hold at every argument, so that (ii) is an instance of UR with no induction hypothesis.

Now define the comparison step, the comparator, the saturating step and the predecessor:

$$\chi(u) := \mathrm{sel}(N(u), G(u), u), \qquad \varepsilon(x) := I(x, 1;\, u.\, 2u), \qquad \lambda(x, y) := N\big(I(y, \varepsilon(x);\, u.\, \chi(u))\big),$$
$$\beta_m(u) := \mathrm{sel}(\lambda(Su, m), u, Su), \qquad \sigma(t, m) := I(t, 0;\, u.\, \beta_m(u)),$$
$$P(m) := I(m, 0;\, u.\, \mathrm{sel}(\lambda(SSu, m), u, Su)).$$

Lemma 2.91 (The comparator)**.** (i) $\chi(2u) = 3u$, $\chi(1) = 1$ *and* $\chi(3u) = 3\chi(u)$.

(ii) $I(y, 3a;\, u.\, \chi(u)) = 3\, I(y, a;\, u.\, \chi(u))$ *and* $I(y, 1;\, u.\, \chi(u)) = 1$.

(iii) $\lambda(0, y) = 1$, $\lambda(Sx, 0) = 0$ *and* $\lambda(Sx, Sy) = \lambda(x, y)$.

Proof. (i) $N(2u) = 0$ and $G(2u) = 3u$ give $\chi(2u) = 3u$; $N(1) = 1$ gives $\chi(1) = 1$. For the third equation, $N(3u) = N(u)$. If $N(u) = 0$, then $\chi(3u) = G(3u) = 3G(u)+N(u) = 3G(u) = 3\chi(u)$. If $N(u) = 1$, then $\chi(3u) = 3u = 3\chi(u)$.

(ii) The template $3\, I(y, a;\, u.\, \chi(u))$ has the seed $3a$ and, by (i), the update χ; UR gives the first equation. The second is Theorem 2.78(iv).

(iii) $\varepsilon(0) = 1$, so $\lambda(0, y) = N(1) = 1$ by (ii). $\lambda(Sx, 0) = N(\varepsilon(Sx)) = N(2\varepsilon(x)) = 0$. By the second equation of Theorem 2.79, by (i) and by (ii),

$$I(Sy, 2\varepsilon(x);\, u.\, \chi(u)) = I(y, \chi(2\varepsilon(x));\, u.\, \chi(u)) = I(y, 3\varepsilon(x);\, u.\, \chi(u)) = 3\, I(y, \varepsilon(x);\, u.\, \chi(u)),$$

and $N(3v) = N(v)$ gives $\lambda(Sx, Sy) = \lambda(x, y)$. □

Theorem 2.92 (Predecessor from uniqueness)**.** *The computation equations, coverage and* UR *derive* $P(0) = 0$ *and* $P(Sm) = m$*, and hence that* $Sx = Sy$ *implies* $x = y$.

Proof. By (2.1) and Theorem 2.91(iii),

$$\beta_{Sm}(Su) = \mathrm{sel}(\lambda(Su, m), Su, SSu) = S\beta_m(u), \qquad \beta_{Sm}(0) = \mathrm{sel}(\lambda(0, m), 0, 1) = 1.$$

As functions of t, the terms $\sigma(St, Sm)$ and $S\sigma(t, m)$ have the seed 1 and the update β_{Sm}: for the first by IS, with $\sigma(1, Sm) = \beta_{Sm}(0) = 1$, and for the second by $S\beta_m(\sigma(t, m)) = \beta_{Sm}(S\sigma(t, m))$. Theorem 2.78(i) gives

$$\sigma(St, Sm) = S\sigma(t, m). \tag{2.2}$$

The template $\lambda(Sn, n)$ satisfies $\lambda(SSn, Sn) = \lambda(Sn, n)$ and has the value $\lambda(1, 0) = 0$ at 0, so $\lambda(Sn, n) = 0$ by Theorem 2.78(ii). The template $\sigma(n, n)$ has the seed 0 and, by (2.2), the update S, so $\sigma(n, n) = n$ by UR and Theorem 2.78(iii). Hence

$$\sigma(Sn, n) = \beta_n(\sigma(n, n)) = \mathrm{sel}(\lambda(Sn, n), n, Sn) = n. \tag{2.3}$$

$P(0) = 0$ is an instance of I0. For a variable m put $\sigma'(t, m) := I(t, 0;\, u.\, \mathrm{sel}(\lambda(SSu, Sm), u, Su))$, so that $P(Sm)$ is $\sigma'(Sm, m)$. By Theorem 2.91(iii) the body of σ' equals $\beta_m(u)$ at every u, so $\sigma'(t, m)$ and $\sigma(t, m)$ have the same seed and the same update, and Theorem 2.78(i) gives $\sigma'(t, m) = \sigma(t, m)$. With (2.3), $P(Sm) = \sigma(Sm, m) = m$. Applying P to $Sx = Sy$ gives $x = y$. □

Every appeal to induction in the construction of Theorem 1.65 has been replaced by an instance of Theorem 2.78, and every case distinction uses coverage and a finite range.

2.5.5 Truncated subtraction and diagonal descent

Put $d(a,b) := I(b,a;\, u.\, P(u))$ and $\mathrm{sg}(a) := \mathrm{sel}(a,0,1)$.

Lemma 2.93 (Truncated subtraction)**.** *$d(a,0) = a$, $d(a,Sb) = P(d(a,b))$, $d(Sa,Sb) = d(a,b)$, $d(a,a) = 0$, $d(0,a) = 0$, $P(d(Sn,a)) = d(n,a)$ and $d(a+n,n) = a$. Addition is cancellative: $a+n = b+n$ implies $a = b$.*

Proof. The first two are the computation equations. As functions of b, the terms $d(Sa,Sb)$ and $d(a,b)$ have the seed a, by $d(Sa,1) = P(Sa) = a$, and the update P. The template $d(a,a)$ satisfies $d(Sa,Sa) = d(a,a)$ and is 0 at 0; Theorem 2.78(ii) applies. The template $d(0,a)$ and the constant 0 have the seed 0 and the update P. For the sixth equation use coverage on a: at 0 it is $P(Sn) = n$, and at Sa' it is $P(d(n,a')) = d(n,Sa')$. The template $d(a+n,n)$ satisfies $d(a+Sn,Sn) = d(a+n,n)$ and is a at 0. Cancellation follows by applying $d(\,\cdot\,,n)$. □

Lemma 2.94 (Diagonal descent)**.** *Let $q(x_1,\dots,x_r)$ be a term template, $r \geq 1$, such that*

$$q(Sx_1,\dots,Sx_r) = q(x_1,\dots,x_r)$$

and such that $q(x_1,\dots,x_r) = 0$ whenever some x_i is 0. Then $q(x_1,\dots,x_r) = 0$.

Proof. Coverage in each argument gives $q(P(x_1),\dots,P(x_r)) = q(x_1,\dots,x_r)$: if some x_i is 0, both sides are 0, and otherwise the first hypothesis applies. The template $f(k) := q(d(x_1,k),\dots,d(x_r,k))$ therefore satisfies $f(Sk) = f(k)$, and Theorem 2.78(ii) gives $f(k) = f(0)$. At $k = x_1$ the first argument is $d(x_1,x_1) = 0$, so $f(x_1) = 0$; and $f(0)$ is $q(x_1,\dots,x_r)$. □

The lemma uses uniqueness for one scalar state; it does not quantify over tuples and uses no rule for simultaneous recursion.

A term is *Boolean* if it is derivably 0 or 1. Put $\mathrm{or}(p,q) := \mathrm{sel}(p,q,1)$ and $\mathrm{xor}(p,q) := \mathrm{sel}(p,q,Z(q))$. For Boolean p and q, four cases show that $\mathrm{xor}(p,q) = 0$ implies $p = q$.

Lemma 2.95 (Three Boolean identities)**.** (i) $Z(d(Sn,a)) = \mathrm{sg}(d(a,n))$.

(ii) $Z\big(d(d(x,a),d(b,a))\big) = \mathrm{or}\big(Z(d(x,a)),Z(d(x,b))\big)$.

(iii) $\mathrm{or}(\mathrm{sg}(u),\mathrm{sg}(v)) = \mathrm{sg}(u+v)$.

Proof. (i) Let $q(n,a)$ be the xor of the two sides, which are Boolean. By Theorem 2.93, $q(Sn,Sa) = q(n,a)$. If $a = 0$, the sides are $Z(Sn) = 0$ and $\mathrm{sg}(0) = 0$. If $n = 0$ and $a = Sb$, the sides are $Z(d(1,Sb)) = Z(d(0,b)) = 1$ and $\mathrm{sg}(Sb) = 1$. Theorem 2.94 gives $q(n,a) = 0$.

(ii) Let $q(x,a,b)$ be the xor of the two sides. Simultaneous successors cancel in every subtraction, so $q(Sx,Sa,Sb) = q(x,a,b)$. If $x = 0$, both sides are 1. If $a = 0$, the sides are $Z(d(x,b))$ and $\mathrm{or}(Z(x),Z(d(x,b)))$; they agree by coverage on x, since at $x = 0$ both are 1 and at a successor $Z(x) = 0$. If $b = 0$, the sides are $Z(d(x,a))$ and $\mathrm{or}(Z(d(x,a)),Z(x))$; they agree by coverage on x and the two values of the Boolean term $Z(d(x,a))$. Theorem 2.94 gives $q(x,a,b) = 0$.

(iii) By coverage on u: at 0 both sides are $\mathrm{sg}(v)$, and at a successor both are 1, by $Su' + v = S(u'+v)$. □

2.5.6 An accumulator and the commutativity of the maximum

Define

$$\mu(n,a) := d(n,d(n,a)), \qquad \varphi(n,a,b) := \mu(n,a)+\mu\big(d(n,a),d(b,a)\big), \qquad m(a,b) := a+d(b,a).$$

These terms are compositions of d and addition; none is introduced by a recursion whose body sees the counter.

Lemma 2.96 (The accumulator)**.** (i) $\mu(Sn,a) = \mu(n,a) + Z(d(Sn,a))$.

(ii) $\varphi(0,a,b) = 0$ *and* $\varphi(Sn,a,b) = \varphi(n,a,b) + \mathrm{sg}\big(d(a,n) + d(b,n)\big)$.

(iii) $m(a,b) = m(P(a),P(b)) + \mathrm{sg}(a+b)$.

Proof. (i) Split $d(Sn,a)$ by coverage. If it is 0, then $d(n,a) = P(0) = 0$ by Theorem 2.93, so $\mu(Sn,a) = Sn$ and $\mu(n,a) = n$, and $Z(0) = 1$. If it is Sr, then $d(n,a) = r$ and $\mu(Sn,a) = d(Sn,Sr) = d(n,r) = \mu(n,a)$, and $Z(Sr) = 0$.

(ii) $\varphi(0,a,b) = \mu(0,a) + \mu(0,d(b,a)) = 0$, because $d(0,x) = 0$. Put $w := d(b,a)$ and split $d(Sn,a)$ again. If it is 0, then $d(n,a) = 0$, $\varphi(Sn,a,b) = Sn$, $\varphi(n,a,b) = n$ and $Z(d(0,w)) = 1$. If it is Sr, then $d(n,a) = r$, the first summand is unchanged by (i), and (i) for the second summand gives $\mu(Sr,w) = \mu(r,w) + Z(d(Sr,w))$. In both cases

$$\varphi(Sn,a,b) = \varphi(n,a,b) + Z\big(d(d(Sn,a),d(b,a))\big).$$

Theorem 2.95(ii) at $x = Sn$, then (i) of that lemma for each of the two Boolean terms, then (iii), turn the last summand into $\mathrm{sg}(d(a,n) + d(b,n))$.

(iii) By coverage on a and b. At $(0,0)$ both sides are 0. At $(0,Sb)$ and at $(Sa,0)$ both sides are the successor of $m(0,b) = b$ and of $m(a,0) = a$. At (Sa,Sb), $m(Sa,Sb) = Sa + d(b,a) = Sm(a,b)$ and $\mathrm{sg}(Sa+Sb) = 1$. □

Lemma 2.97 (Commutativity of the maximum)**.** $a + d(b,a) = b + d(a,b)$.

Proof. For fixed a, b put

$$F(n) := m(d(a,n),d(b,n)) + \varphi(n,a,b), \qquad F'(n) := m(d(b,n),d(a,n)) + \varphi(n,a,b).$$

By Theorem 2.96(iii) at the arguments $d(a,n)$, $d(b,n)$, by Theorem 2.96(ii) and by the laws of addition, $F(Sn) = F(n)$, and likewise $F'(Sn) = F'(n)$. Theorem 2.78(ii) gives $F(n) = F(0) = m(a,b)$ and $F'(n) = F'(0) = m(b,a)$. At $n = b$, $d(b,b) = 0$ and $m(x,0) = x = m(0,x)$, so $F(b)$ and $F'(b)$ both equal $d(a,b) + \varphi(b,a,b)$. Hence $m(a,b) = m(b,a)$. □

The familiar proof of this identity is an induction whose hypothesis is used at the predecessor. Here the two sides are joined by a term φ whose recurrence depends on the counter, although its definition is a composition; the recurrence is Theorem 2.96(ii).

2.5.7 Reflection of equality and the proofs of the theorems

Lemma 2.98 (Reflection of equality)**.** *Let* $\Delta(a,b) := \mathrm{sg}(d(a,b) + d(b,a))$. *Then* $\Delta(a,b) = 0 \vee \Delta(a,b) = 1$*, and* $\Delta(a,b) = 0$ *holds if and only if* $a = b$.

Proof. By coverage and separation, $\mathrm{sg}(u) = 0$ if and only if $u = 0$, and $u + v = 0$ if and only if $u = 0$ and $v = 0$, since a sum with a successor summand is a successor. If $\Delta(a,b) = 0$, then $d(a,b) = 0 = d(b,a)$, and Theorem 2.97 gives $a = a + d(b,a) = b + d(a,b) = b$. If $a = b$, both subtractions are $d(a,a) = 0$. □

The equality reflected is the equality of the logic on the whole numerical domain.

Proof of Theorem 2.76. By Theorem 2.75, every derivation of T_{UR} is a derivation of T_{eq} after each instance of UR is replaced by its derivation. Conversely let $\Gamma \vdash s(0) = r(0)$ and $\Gamma, s(z) = r(z) \vdash s(Sz) = r(Sz)$ in T_{UR}. Put $q(n) := \Delta(s(n), r(n))$. By Theorem 2.98 the two premises give $\Gamma \vdash q(0) = 0$ and $\Gamma, q(z) = 0 \vdash q(Sz) = 0$; Theorem 2.84 gives $q(t) = 0$; and Theorem 2.98 gives $s(t) = r(t)$. The context Γ is arbitrary, so this is the contextual induction rule of T_{eq}. The two theories have the same language and the same derivable sequents, and Theorem 1.119 applies to T_{UR}. In particular induction for quantifier-free formulas (Theorem 1.95) holds in T_{UR}. □

Proof of Theorem 2.77. UR is derivable in N by Theorem 2.75. Conversely it suffices to derive equation induction in U: given $\Gamma, E(z) \vdash_{\mathsf{U}} E(Sz)$, equation induction over the context $\Gamma, E(0)$ gives $E(Sz')$ for a fresh z', and the introduction of Q by cases (Theorem 2.4(i)) gives $\Gamma \vdash_{\mathsf{U}} Q_y E(y)$.

The native connectives of Section 1.4 obey their natural deduction rules in U, and the equality rules, the Leibniz rule and numerical cases hold in U: by Theorem 1.30, their derivations use the seven rules of N other than QI. With these, U derives the reading of every axiom and rule of T_{UR}: the computation equations, coverage by numerical cases, separation because the native falsity is the equation $0 = S0$, and UR itself. Hence the derivation of the induction rule in the proof of Theorem 2.76, written with the native connectives, is a derivation in U. □

Remark 2.99 (Sources and status). Sections 2.5.1 to 2.5.7 are `scalar_uniqueness`, except for Theorem 2.80, the proof of Theorem 2.81 and Theorem 2.82, which are from `reductions`, and Theorem 2.85, which is from `orbit_closure` (Theorem 2.86). `scalar_uniqueness` attributes the idea of replacing induction by uniqueness of recursion to the literature (Okada and Scott study uniqueness of iteration in a typed setting) and claims the completion for the scalar language only: one state, no counter in the body, no pairing. Theorem 2.93 also yields the Mal'cev term $d(x + z, y)$, which the package notes. In Lean, Theorem 2.75 is proved; the converse direction of Theorems 2.76 and 2.77 is not formalized.

2.5.8 Other forms of the uniqueness rule

Three variations of UR occur in `orbit_closure` and `crossed_uniqueness`. They are related to UR as follows. The rule ORB concludes the closure of the equation of UR and takes no target; it is interderivable with UR by the rules for the closure alone (Theorem 2.101). The rule SU of Theorem 2.74 is the instance of UR at one body; it gives induction for zero equations (Theorem 2.85) and the laws of addition, and whether it gives induction for all equations is open. The rule UX has a premise for the seed in place of the seed $f(0)$ and a crossed conclusion; UR is its instance at the seed $f(0)$, and with transport it follows from UR. Taken as the only arithmetic rule on fully normalized terms, UX derives transport as well, so that it replaces I0, IS, TR and QI together (Theorem 2.108).

The closure of the orbit equation

For term templates f and h as in UR, and a variable y not free in h, the ordinary and the native form of the rule are

$$\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash \forall y\,\big(f(y) = I(y, f(0);\, u.\, h(u))\big)} \qquad \frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash Q_y\big(f(y) = I(y, f(0);\, u.\, h(u))\big)}\ \text{ORB},$$

where z is not free in Γ or in h.

Definition 2.100 (The calculus U_o and the theory T_{Orb})**.** U_o is N with QI replaced by the native form of ORB; its eight rules are HYP, I0, IS, TR, QE, DE, DI and ORB. T_{Orb} is T_{UR} with UR replaced by the ordinary form of ORB.

Proposition 2.101 (Orbit uniqueness)**.** *T_{Orb} and T_{UR} derive the same sequents of ordinary formulas, and U_o and U are equivalent. Hence T_{Orb} derives the sequents of T_{eq}, and U_o is equivalent to N.*

Proof. Let $E(y)$ be the equation $f(y) = I(y, f(0); u.\, h(u))$. In the ordinary theories, an instance of UR is ORB followed by the elimination of $\forall$ at t; and UR at a fresh variable w as its target, followed by the introduction of $\forall$, is ORB.

In the native calculi, ORB is derived in U thus: UR at the target Sw, with w fresh, gives $E(Sw)$, and the introduction of Q by cases (Theorem 2.4(i)), whose assumption $E(0)$ is not needed, gives $Q_y E(y)$. Conversely UR is derived in U_o by QE at t: its second premise $E(0)$ is the instance $I(0, f(0); u.\, h(u)) = f(0)$ of I0, reversed. Both constructions hold over arbitrary contexts. The last sentence of the proposition is Theorems 2.76 and 2.77. □

The base $E(0)$ of the orbit equation is a theorem, so $Q_y E$ and the unconditional $\forall y\, E$ say the same; in a calculus with clause contexts (Section 1.9) ORB adds the clause of E with a trivial antecedent, and the term at which it is used is chosen when the clause is used. The rule removes a parameter of the certificate and not the principle.

Remark 2.102 (The body must not see the counter). In ORB the variable y of the conclusion is not free in h. A checker that reads f and h off a proposed closure has to test this, also below nested binders. Without the test, $f(y) := y$ and the body Sy have the premise $Sz = Sz$ and the conclusion $Q_y\big(y = I(y, 0;\, u.\, Sy)\big)$, whose instance at 1 is $1 = 2$.

The successor body

Proposition 2.103 (Addition from successor uniqueness)**.** *T_{SU} derives that addition is associative and commutative with the neutral element 0. If $\Gamma \vdash k(Sz) = k(z)$ for a term template k and a fresh z, then T_{SU} derives $\Gamma \vdash n + k(n) = n + k(0)$.*

Proof. The laws of addition are those of Theorem 2.79, whose proofs compare terms with the update S. For the second statement let $f(n) := n + k(n)$. Then $f(0) = 0 + k(0) = k(0)$ and

$$f(Sn) = Sn + k(Sn) = Sn + k(n) = S(n + k(n)) = Sf(n),$$

so SU gives $f(n) = I(n, k(0);\, u.\, Su) = k(0) + n = n + k(0)$. □

If addition were cancellative in T_{SU}, the proposition would give $k(n) = k(0)$, which is Theorem 2.78(ii), uniqueness for the body u. Cancellation is proved in Theorem 2.93 from the predecessor, whose construction uses other bodies.

Remark 2.104 (What successor uniqueness is not known to give). To pass from Theorem 2.85 to induction for all equations by the route of Section 2.5.7, a term Δ with $\Delta(a, b) = 0 \leftrightarrow a = b$ is needed. The construction of Sections 2.5.4 to 2.5.6 uses UR for the bodies u, $Z(u)$, $j(u)$, $g(u)$, $\chi(u)$, $\beta_m(u)$ and $P(u)$, among others, and cannot be repeated in T_{SU} as it stands. The right inverse of Theorem 2.80 does not supply such a term either: it concerns a term e whose values are already known at 0 and along successors of its values, and `reductions` records that it does not construct Δ from SU. `orbit_closure` does not claim that T_{SU} is equivalent to T_{eq}. It also records that a model used in another package, `canonical_recursors`, to separate induction for zero equations from induction for all equations does not decide the question: in that model, for a nonstandard counter p, the term $e(a) := I(p, a;\, u.\, u)$ satisfies $e(0) = 0$ and $e(Sa) = Se(a)$ and is not the identity, whereas SU with Theorem 2.78(iii) gives $e(a) = a$. The model therefore refutes SU, and it is not a model of T_{SU} in which equation induction fails. The model is not reproduced here.

A seed premise and a crossed conclusion, on normal forms

Let $\mathrm{nf}(t)$ be the full normal form of a term t for the two computation equations, read from left to right, and let $\mathrm{nf}(A)$ and $\mathrm{nf}(\Gamma)$ be obtained by normalizing every term. The following facts are taken from the packages `full_normalization` and `computation_normal`, which belong to another chapter: nf is total; $\mathrm{nf}(\mathrm{nf}(t)[\theta]) = \mathrm{nf}(t[\theta])$ for every substitution θ; and N derives $t = \mathrm{nf}(t)$. A term, a formula or a context is *normal* if nf fixes it. For normal terms the selector satisfies $\mathrm{nf}(\mathrm{sel}(0, a, b)) = a$ and $\mathrm{nf}(\mathrm{sel}(Sc, a, b)) = b$, and $\mathrm{nf}(I(1, a;\, u.\, h)) = \mathrm{nf}(h(a))$.

Definition 2.105 (The calculus $\mathsf{U_c}$)**.** The sequents of $\mathsf{U_c}$ consist of normal formulas, and every instance $B(t)$, $f(n)$ or $h(f(z))$ in a rule denotes the normal form of the substitution. Its six rules are HYP, QE, DE, DI in their forms of Section 1.3, reflexivity,

$$\frac{}{\Gamma \vdash t = t}\ \text{REF},$$

and *seeded crossed uniqueness*,

$$\frac{\Gamma \vdash f(0) = a \qquad \Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash I(n, a;\, u.\, h(u)) = f(n)}\ \text{UX},$$

where $f(z)$ and $h(u)$ are term templates and z is not free in Γ, a, n or h.

There is no computation rule, no transport rule and no rule for progress: computation is in the syntax, and normalization is part of the specification of the calculus and of any checker for it. The introduction DI is by cases at every matrix. The body h receives the state and no counter, and the second premise has no induction hypothesis. This is the one calculus of the chapter without the rules I0 and IS.

Lemma 2.106 (Structural rules on normal forms)**.** *Weakening and cut are admissible in* $\mathsf{U_c}$*, and if* $\Gamma \vdash_{\mathsf{U_c}} A$ *then* $\mathrm{nf}(\Gamma[\theta]) \vdash_{\mathsf{U_c}} \mathrm{nf}(A[\theta])$.

Proof. By induction on derivations, with eigenvariables renamed. No rule has a mode or a test on a term, and $\mathrm{nf}(\mathrm{nf}(t)[\theta]) = \mathrm{nf}(t[\theta])$ identifies the substituted premises and conclusions of each rule with an instance of the same rule. □

Lemma 2.107 (Equality from seeded crossed uniqueness)**.** $\mathsf{U_c}$ *derives, over an arbitrary context:*

(i) *numerical cases and the introduction of* Q *by cases;*

(ii) *symmetry: from* $s = t$ *to* $t = s$*;*

(iii) *congruence: from* $s = t$ *to* $H(s) = H(t)$*, for every term template* H*;*

(iv) *joining: from* $s = t$ *and* $s = r$ *to* $t = r$*;*

(v) *transitivity, and the rule* TR*: from* $s = t$ *and* $H(s) = r$ *to* $r = H(t)$*;*

(vi) *the rule* UR*.*

None of these derivations uses an induction, a predecessor or a test of equality.

Proof. (i) Theorem 2.4(i); its derivations use HYP, QE, DE and DI.

(ii) Apply UX with the constant template $f(z) := s$, the seed $a := t$, the body $h(u) := u$ and the counter $n := 0$. The first premise is $s = t$, the second is $s = s$, and the conclusion $I(0, t;\, u.\, u) = s$ is, in normal form, $t = s$.

(iii) Apply UX with $f(z) := I(z, s; u.\, H(u))$, $a := t$, $h := H$ and $n := 1$. The first premise is $s = t$, because $f(0)$ has the normal form s. The two sides of the second premise, $I(Sz, s; u.\, H(u))$ and $H(I(z, s; u.\, H(u)))$, have the same normal form, so it is an instance of REF. The conclusion is $H(t) = H(s)$ in normal form, and (ii) reverses it. The template may use its variable several times or not at all, also below binders.

(iv) Apply UX with $f(z) := \mathrm{sel}(z, s, r)$, $a := t$, $h(u) := u$ and $n := 1$. The first premise is $s = t$. The second premise is $r = \mathrm{sel}(z, s, r)$, since $f(Sz)$ has the normal form r; it follows by numerical cases on z, from $r = s$ at 0, which is (ii) applied to $s = r$, and from $r = r$ at a successor. The conclusion is $t = r$ in normal form.

(v) From $s = t$ and $t = r$: (ii) gives $t = s$, and (iv) gives $s = r$. From $s = t$ and $H(s) = r$: (iii) gives $H(s) = H(t)$, and (iv) gives $r = H(t)$.

(vi) UX with the seed $a := f(0)$ and the first premise by REF, followed by (ii). □

The counter 0 makes UX a replacement at the root, the counter 1 makes it an application of a template, and the selector joins two equations; each conclusion is read after normalization, which is why the calculus is stated on normal forms.

Theorem 2.108 (Seeded crossed uniqueness)**.** *For every context* Γ *and every native formula* A*,*

$$\Gamma \vdash A \text{ in } \mathsf{N} \quad \Longleftrightarrow \quad \mathrm{nf}(\Gamma) \vdash_{\mathsf{U_c}} \mathrm{nf}(A).$$

Hence the bridge of Theorem 1.119 holds for $\mathsf{U_c}$*, with the translations composed with* nf*.*

Proof. From right to left. Every rule of $\mathsf{U_c}$ is derivable in N on normal formulas. REF is reflexivity. For UX apply equation induction to $I(y, a; u.\, h(u)) = f(y)$: the base follows from I0 and the first premise, and in the step IS, congruence under h applied to the induction hypothesis, and the second premise give $I(Sz, a; u.\, h(u)) = h(I(z, a; u.\, h(u))) = h(f(z)) = f(Sz)$. The normalization of an instance is justified by $t = \mathrm{nf}(t)$ and the Leibniz rule. The same two facts make A and $\mathrm{nf}(A)$ interderivable in N, so $\mathrm{nf}(\Gamma) \vdash \mathrm{nf}(A)$ gives $\Gamma \vdash A$.

From left to right. First, if $\Gamma \vdash_{\mathsf{U}} A$ in the calculus U of Theorem 2.73, then $\mathrm{nf}(\Gamma) \vdash_{\mathsf{U_c}} \mathrm{nf}(A)$, by induction on the derivation. HYP is clear. The two sides of an instance of I0 or IS have the same normal form, so the instance becomes REF. TR becomes Theorem 2.107(v) with the template $\mathrm{nf}(h)$, because $\mathrm{nf}(h(s)) = \mathrm{nf}(\mathrm{nf}(h)(\mathrm{nf}(s)))$. QE, DE and DI become the same rules, by the same identity. UR becomes Theorem 2.107(vi). Second, $\Gamma \vdash A$ in N implies $\Gamma \vdash_{\mathsf{U}} A$ by Theorem 2.77. □

Remark 2.109 (Sources, overlaps and status). Theorems 2.101 to 2.104 are from `orbit_closure`, which states ORB for the ordinary theory and describes the native form with its checker. Its translator maps a derivation with UR to one with ORB by the second construction in the proof of Theorem 2.101. Theorems 2.105, 2.107 and 2.108 are from `crossed_uniqueness`. That package also derives transport in a formula template inside $\mathsf{U_c}$ by induction on the template, and restates the native connectives through $D_{,}$ with an unused coordinate; here both follow from Theorem 2.108 and Section 1.4. Both packages take the passage from UR to equation induction from `scalar_uniqueness`, as the proofs above do, and neither claims it as new. Neither package is formalized in Lean.

2.6 The progressive reading of the closure

This section draws on `uniform_progress`, `target_closure` and the note on mixed closure in `scope_and_closure`.

The single introduction of Section 2.2.4 tests the witness. A single introduction whose mode depends on the root of the matrix alone is possible if the meaning of $Q_y E$ at a literal

equation E is changed from $E(0) \to \forall y\, E(y)$ to progressivity, $\forall y\, (E(y) \to E(Sy))$. The calculi of this section have seven rules, keep all native formulas, and are stable under substitution or under renaming; they are equivalent to N through two translations and not on the same formulas.

2.6.1 The calculus with orbit elimination

Extend the selector of modes of Section 2.2.2 to terms: $\eta_B(a) := a$ if the matrix B is a literal equation and $\eta_B(a) := 0$ if B is compound. The *orbit term* is

$$\omega(a, n) := I(a, n;\, u.\, Su),$$

with the counter a and the seed n.

Definition 2.110 (The calculus R)**.** R has the seven rules HYP, I0, IS, TR_2, DE and

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(\eta_B(a))}{\Gamma \vdash B\big(\omega(\eta_B(a), n)\big)}\ \mathrm{QE}_\omega$$

$$\frac{\Gamma,\ B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}\ \mathrm{DI}_\eta,$$

where a and n are arbitrary terms, and z is not free in Γ, in t or in $D_{x,y}B$. The designated contradiction is $0 = S0$.

At a compound matrix QE_ω passes from $B(0)$ to $B(\omega(0, n))$, and $\omega(0, n) = n$ is an instance of I0. At a literal equation E it passes from $E(a)$ to $E(\omega(a, n))$, the instance n steps further. Transport in a compound formula is not a rule, and there is no rule of numerical cases.

Definition 2.111 (The progressive reading)**.** The ordinary formula F^{p} is defined by

$$\begin{aligned}
(s = t)^{\mathrm{p}} &:= (s = t), \\
(Q_y E)^{\mathrm{p}} &:= \forall y\, (E(y) \to E(Sy)) && (E \text{ a literal equation}), \\
(Q_y B)^{\mathrm{p}} &:= B^{\mathrm{p}}(0) \to \forall y\, B^{\mathrm{p}}(y) && (B \text{ compound}), \\
(D_{x,y} B)^{\mathrm{p}} &:= \exists x\, (Q_y B)^{\mathrm{p}}.
\end{aligned}$$

The reading commutes with substitution, because substitution preserves the root of a matrix. The elimination and the reading have to change together:

Proposition 2.112 (The two readings differ)**.** *Let $E(y)$ be the equation $N(y) = 1$ for the parity term N of Section 1.1.*

(i) *In the standard model $E(0) \to \forall y\, E(y)$ is true and $\forall y\, (E(y) \to E(Sy))$ is false.*

(ii) *The rule QE_ω does not preserve truth for the ordinary reading of Section 1.2.*

(iii) N *derives $Q_y E(y)$, and* R *does not.*

Proof. $N(0) = 0$, $N(1) = 1$ and $N(2) = 0$. (i) The premise $E(0)$ is false, and progress fails at $y = 1$. (ii) With $a := 1$ and $n := 1$ the two premises $Q_y E$ and $E(1)$ are true in the ordinary reading and the conclusion $E(\omega(1, 1))$, which is $E(2)$, is false. (iii) In N the closure follows by the introduction by cases from $N(0) = 1 \vdash N(Sz) = 1$, which holds because $N(0) = 0$ and $0 = 1$ derives every equation. In R it is not derivable by Theorem 2.117 below and (i). □

2.6.2 Derived rules of the orbit calculus

Lemma 2.113 (Structural rules)**.** *Weakening, cut and substitution of terms are admissible in* R*, and substitution maps every rule instance to an instance of the same rule.*

Proof. Theorem 2.3: no rule of R tests a term, and $\eta_B(a)[\theta] = \eta_{B[\theta]}(a[\theta])$. □

Lemma 2.114 (Equality, closure introduction, equation induction)**.** R *derives:*

(i) *reflexivity, symmetry, transitivity, congruence in a term template, and* TR*;*

(ii) *the introduction* QI_η *of Theorem 2.9: from* $\Gamma, B(\eta_B(z)) \vdash B(Sz)$ *with* z *fresh to* $\Gamma \vdash Q_y B(y)$*;*

(iii) *equation induction: from* $\Gamma \vdash E(0)$ *and* $\Gamma, E(z) \vdash E(Sz)$*, for a literal equation* E *and* z *fresh, to* $\Gamma \vdash E(t)$*.*

The derivations of (i) and (ii) use HYP*,* I0*,* TR_2*,* DE *and* DI_η *only.*

Proof. (i) Reflexivity and symmetry are derived from I0 and TR_2 in the proof of Theorem 2.21. Transitivity and congruence are TR_2 applied to $r = s$ in the template $r = v$ and to $f(s) = f(s)$ in the template $f(s) = f(v)$. TR follows as in that proof.

(ii) Apply DI_η with the matrix $B(y)$, a coordinate x not free in B and the witness 0; then DE, whose opened hypothesis is $Q_y B(y)$, and HYP.

(iii) By (ii) the step gives $Q_y E(y)$. QE_ω with $a := 0$ and $n := t$ gives $E(\omega(0,t))$ from the base, and $\omega(0,t) = t$ is an instance of I0; transport in the equation template E gives $E(t)$. □

The order of the arguments of ω is what makes (iii) work: $\omega(0,t) = t$ is a computation, whereas $\omega(t,0) = t$ is a theorem that needs the induction under construction.

Lemma 2.115 (Orbit identities)**.** *In every calculus or theory with the equality rules, the computation equations and equation induction, in particular in* R*,* N *and* T_{eq}*,*

$$\omega(0,n) = n, \qquad \omega(Sa,n) = S\,\omega(a,n), \qquad \omega(a,0) = a,$$
$$\omega(a,Sn) = S\,\omega(a,n), \qquad \omega(a,n) = \omega(n,a), \qquad \omega(a,1) = Sa.$$

Proof. The first two are I0 and IS. The third is by induction on a: $\omega(0,0) = 0$, and $\omega(Sa,0) = S\,\omega(a,0) = Sa$. The fourth is by induction on a with the parameter n: both sides are Sn at 0, and $\omega(Sa,Sn) = S\,\omega(a,Sn) = SS\,\omega(a,n) = S\,\omega(Sa,n)$. The fifth is by induction on a: the base is $n = \omega(n,0)$, and $\omega(Sa,n) = S\,\omega(a,n) = S\,\omega(n,a) = \omega(n,Sa)$. The last is $\omega(a,1) = S\,\omega(a,0) = Sa$. □

Lemma 2.116 (The two interfaces of a closure, and transport)**.** *In* R*, over an arbitrary context:*

(i) *one step: from* $Q_y E(y)$ *and* $E(a)$ *to* $E(Sa)$*, for a literal equation* E*;*

(ii) *transport: from* $s = t$ *and* $F(s)$ *to* $F(t)$*, for every native formula template* F*;*

(iii) *instance from the base: from* $Q_y B(y)$ *and* $B(0)$ *to* $B(t)$*, for every matrix* B*;*

(iv) *homogeneous congruence: if* B *and* C *are interderivable over* Γ *and are both literal equations or both compound, then* $Q_y B$ *and* $Q_y C$ *are interderivable over* Γ*, and so are* $D_{x,y}B$ *and* $D_{x,y}C$*, where the bound variables are not free in* Γ*.*

Proof. (i) QE_ω with $n := 1$ gives $E(\omega(a,1))$, and $\omega(a,1) = Sa$ by Theorem 2.115.

(ii) and (iii) are proved together by induction on the number of binders of the formula, a measure that substitution does not change; (iii) for B uses (ii) for B. For an equation, (ii) is TR_2. For (iii): at a literal equation, QE_ω with $a := 0$ and $n := t$ gives $B(\omega(0,t))$; at a compound matrix it gives the same from $B(0)$; and (ii) for B along $\omega(0,t) = t$ gives $B(t)$.

(ii) for $Q_y B(v,y)$. Derive the target $Q_y B(t,y)$ by QI_η: under the assumption $B(t,\eta_B(z))$, transport in B along $t = s$ gives $B(s,\eta_B(z))$; then (i), if B is a literal equation, or (iii) at Sz, if B is compound, gives $B(s,Sz)$ from the given closure $Q_y B(s,y)$; and transport in B along $s = t$ gives $B(t,Sz)$.

(ii) for $D_{x,y}B(v,x,y)$. Apply DE to the given formula with a fresh b, and derive the target by DI_η with the witness b: the premise $B(t,b,\eta_B(z)) \vdash B(t,b,Sz)$ follows from the opened closure $Q_y B(s,b,y)$ exactly as in the previous case. The conclusion does not contain b.

Every use of transport or of (iii) inside these two cases is for the matrix B, which has fewer binders.

(iv) Repeat the two cases of (ii) with the given derivations between B and C, instantiated by substitution (Theorem 2.113), in place of transport. Because B and C have the same kind of root, $\eta_B = \eta_C$, so the assumptions of the two introductions match. □

The restriction in (iv) is necessary. An equation E and a compound formula C can be interderivable while $Q_y E$, which is progressive, and $Q_y C$, which is conditional, are not; the padding of Theorem 2.118 gives such pairs. Transport is unaffected, because it preserves the root of the matrix.

2.6.3 Soundness for the progressive reading

Theorem 2.117 (Soundness)**.** *If* $\Gamma \vdash_{\mathsf{R}} F$*, then* $\Gamma^{\text{p}} \vdash_{\mathsf{T}_{\text{eq}}} F^{\text{p}}$.

Proof. By induction on the derivation. HYP, the computation equations, TR_2 and DE have their ordinary meanings; for DE the opened hypothesis reads $(Q_y B(b,y))^{\text{p}}$, the matrix of the existential quantifier.

DI_η at a literal equation: the premise gives $\forall z\,(E(t,z) \to E(t,Sz))$, and t is a witness. At a compound matrix: the premise gives $B^{\text{p}}(t,Sz)$ under $B^{\text{p}}(t,0)$, so coverage gives $B^{\text{p}}(t,0) \to \forall y\, B^{\text{p}}(t,y)$.

QE_ω at a compound matrix: the first premise reads $B^{\text{p}}(0) \to \forall y\, B^{\text{p}}(y)$, the second gives $B^{\text{p}}(0)$, and $\omega(0,n) = n$. At a literal equation: assume $\forall y\,(E(y) \to E(Sy))$ and $E(a)$. Induction on k for the literal equation $E(\omega(k,a))$, with these two assumptions in the context, gives $E(\omega(k,a))$ for every k: the base is $E(a)$ by $\omega(0,a) = a$, and the step is the assumed progress at $\omega(k,a)$ with $\omega(Sk,a) = S\,\omega(k,a)$. At $k := n$, the equation $\omega(n,a) = \omega(a,n)$ of Theorem 2.115 gives $E(\omega(a,n))$. □

The use of the commutativity of ω in this proof is harmless: it is a theorem of T_{eq}, whose induction is given. It is separate from the recovery of induction inside R, which uses $\omega(0,t) = t$ only.

2.6.4 The embedding of the core

For a literal equation E and fresh variables r, v let

$$\text{pad}(E) := D_{r,v}\,\text{gate}(v,E), \qquad \text{that is,} \qquad D_{r,v}(\text{sel}(v,0,s) = \text{sel}(v,0,t)) \quad \text{for } E \equiv (s = t),$$

and for a native formula F let $F^\sharp := \text{pad}(F)$ if F is a literal equation and $F^\sharp := F$ if F is compound. The formula $F^\sharp$ is always compound. The translation $A^\uparrow$ of a native formula is

$$(s = t)^\uparrow := (s = t), \qquad (Q_y B)^\uparrow := Q_y (B^\uparrow)^\sharp, \qquad (D_{x,y} B)^\uparrow := D_{x,y} (B^\uparrow)^\sharp.$$

It fixes terms and equations, preserves the kind of the root, commutes with substitution, and puts a compound matrix below every binder.

Lemma 2.118 (Padding)**.** *For a literal equation E, the formulas* $\mathrm{pad}(E)$ *and E are interderivable in* R *over every context. Hence $F^\sharp$ and F are interderivable for every F.*

Proof. Let E be $s = t$. The computation equations give $\mathrm{gate}(0, E)$, both sides being 0, and make $\mathrm{gate}(Sz, E)$ and E interderivable. From E: the matrix $\mathrm{gate}(v, E)$ is a literal equation, so DI_η asks for $\mathrm{gate}(Sz, E)$ under the assumption $\mathrm{gate}(z, E)$, which is not needed. From $\mathrm{pad}(E)$: DE opens $Q_v\,\mathrm{gate}(v, E)$, and QE_ω with $a := 0$, $n := 1$ and the second premise $\mathrm{gate}(0, E)$ gives $\mathrm{gate}(\omega(0, 1), E)$. I0 and TR_2 give $\mathrm{gate}(1, E)$, hence E. □

Theorem 2.119 (Embedding)**.** *If $\Gamma \vdash A$ in* N*, then $\Gamma^\uparrow \vdash_{\mathsf{R}} A^\uparrow$.*

Proof. By induction on the derivation. HYP, I0, IS translate to themselves, and TR is derivable in R (Theorem 2.114(i)).

QE. The premises translate to $Q_y(B^\uparrow)^\sharp$ and $B^\uparrow(0)$. Theorem 2.118 gives $(B^\uparrow)^\sharp(0)$, Theorem 2.116(iii) gives $(B^\uparrow)^\sharp(t)$, and Theorem 2.118 gives $B^\uparrow(t)$, which is $B(t)^\uparrow$.

DE. The first premise translates to $D_{x,y}(B^\uparrow)^\sharp$, and the hypothesis opened by DE in R is $Q_y(B^\uparrow)^\sharp(b, y)$, the translation of $Q_y B(b, y)$.

DI. The translated premise is $\Gamma^\uparrow, B^\uparrow(t, 0) \vdash B^\uparrow(t, Sz)$. The target has the compound matrix $(B^\uparrow)^\sharp$, so DI_η asks for $(B^\uparrow)^\sharp(t, Sz)$ under $(B^\uparrow)^\sharp(t, 0)$; remove the padding of the assumption, cut with the premise, and pad the conclusion.

QI. The translated premise is $\Gamma^\uparrow, E(z) \vdash E(Sz)$, and the target is $Q_y\,\mathrm{pad}(E(y))$, with a compound matrix. By Theorem 2.114(ii) it suffices to derive $\mathrm{pad}(E(Sz'))$ from $\mathrm{pad}(E(0))$. The assumption gives $E(0)$; equation induction in R (Theorem 2.114(iii)) with the translated premise as its step gives $E(Sz')$; and Theorem 2.118 pads it. □

2.6.5 The reading inside the core and the two round trips

The translation $F^\downarrow$ expresses the progressive reading with the native connectives $\Rightarrow_*$, $\forall_*$, $\exists_*$ of Section 1.4. It fixes equations and is defined by

$$(Q_y E)^\downarrow := \forall_* y\,(E(y) \Rightarrow_* E(Sy)), \qquad (Q_y B)^\downarrow := Q_y B^\downarrow,$$
$$(D_{x,y} E)^\downarrow := \exists_* x\,\forall_* y\,(E(x, y) \Rightarrow_* E(x, Sy)), \qquad (D_{x,y} B)^\downarrow := D_{x,y} B^\downarrow,$$

where E is a literal equation and B is compound. It maps compound formulas to compound formulas and commutes with substitution.

Theorem 2.120 (The reading is derivation-preserving)**.** *If $\Gamma \vdash_{\mathsf{R}} F$, then $\Gamma^\downarrow \vdash F^\downarrow$ in* N*.*

Proof. By Theorem 1.119 the ordinary reading of a native connective is equivalent in T_{eq} to the corresponding connective of the readings. By induction on F, therefore, $(F^\downarrow)^\circ$ is equivalent in T_{eq} to F^{p}. Theorem 2.117 gives $\Gamma^{\mathrm{p}} \vdash_{\mathsf{T}_{\mathrm{eq}}} F^{\mathrm{p}}$, hence $(\Gamma^\downarrow)^\circ \vdash_{\mathsf{T}_{\mathrm{eq}}} (F^\downarrow)^\circ$, and the faithfulness of the ordinary reading (Theorem 1.119) gives $\Gamma^\downarrow \vdash F^\downarrow$. □

A formula of the form $A^\uparrow$ is an *image*. The rules of the native connectives are available in R for images, over arbitrary contexts of R.

Lemma 2.121 (Logic on images)**.** *Let Δ be a context of* R*, let A, A_i, B, C be native formulas, and let F be a formula of* R*.*

(i) *If $A_1, \ldots, A_k \vdash C$ in* N *and $\Delta \vdash_{\mathsf{R}} A_i^\uparrow$ for every i, then $\Delta \vdash_{\mathsf{R}} C^\uparrow$.*

(ii) *If $\Delta, C(0)^\uparrow \vdash_{\mathsf{R}} C(Sz)^\uparrow$ with z fresh, then $\Delta \vdash_{\mathsf{R}} (Q_y C(y))^\uparrow$.*

(iii) *If* $\Delta, A^{\uparrow} \vdash_{\mathsf{R}} B^{\uparrow}$*, then* $\Delta \vdash_{\mathsf{R}} (A \Rightarrow_* B)^{\uparrow}$.

(iv) *If* $\Delta \vdash_{\mathsf{R}} C(z)^{\uparrow}$ *with* z *not free in* Δ*, then* $\Delta \vdash_{\mathsf{R}} (\forall_* y\, C(y))^{\uparrow}$.

(v) *If* $\Delta \vdash_{\mathsf{R}} (\exists_* x\, C(x))^{\uparrow}$ *and* $\Delta, C(b)^{\uparrow} \vdash_{\mathsf{R}} F$ *with* b *not free in* Δ*,* F *and* C*, then* $\Delta \vdash_{\mathsf{R}} F$.

Proof. (i) Theorem 2.119, weakening and cut. Every rule of the native connectives without a discharged assumption is of this form.

(ii) The target is $Q_y(C^{\uparrow})^{\sharp}(y)$, with a compound matrix. By Theorem 2.114(ii) it suffices to derive $(C^{\uparrow})^{\sharp}(Sz)$ from $(C^{\uparrow})^{\sharp}(0)$, which follows from the hypothesis and Theorem 2.118.

(iii) $A \Rightarrow_* B$ is $Q_y \operatorname{sel}(y, A, B)$. By (ii) it suffices to derive $\operatorname{sel}(Sz, A, B)^{\uparrow}$ from $\operatorname{sel}(0, A, B)^{\uparrow}$. By (i) and $\operatorname{sel}(0, A, B) \vdash A$, the assumption gives $A^{\uparrow}$; the hypothesis gives $B^{\uparrow}$; and (i) with $B \vdash \operatorname{sel}(Sz, A, B)$ gives the conclusion.

(iv) $\forall_* y\, C$ is $C(0) \wedge_* Q_y C$. Substitution in R gives $C(0)^{\uparrow}$ and $C(Sz)^{\uparrow}$, (ii) gives $(Q_y C)^{\uparrow}$, and (i) with the introduction of $\wedge_*$ gives the conclusion.

(v) $\exists_* x\, C$ is $D_{x,y} \operatorname{gate}(y, C)$, and its image is $D_{x,y}\big(\operatorname{gate}(y, C)^{\uparrow}\big)^{\sharp}$. DE in R with the fresh variable b opens the image of $Q_y \operatorname{gate}(y, C(b))$. By (i) and $Q_y \operatorname{gate}(y, C(b)) \vdash C(b)$, this gives $C(b)^{\uparrow}$, and cut with the second hypothesis gives F. The formula F is carried through DE and is not analyzed. □

The lemma does not say that the selector and the connectives of N can be applied to arbitrary formulas of R; that would ignore the dependence of Q on the root of its matrix.

Lemma 2.122 (Normalization of atomic closures)**.** *For a literal equation* E*, over every context of* R*,*

$$Q_y E(y) \dashv\vdash_{\mathsf{R}} (Q_y E(y))^{\downarrow\uparrow}, \qquad D_{x,y} E(x, y) \dashv\vdash_{\mathsf{R}} (D_{x,y} E(x, y))^{\downarrow\uparrow}.$$

Proof. Equations are their own images. From $Q_y E(y)$ and $E(z)$, one step (Theorem 2.116(i)) gives $E(Sz)$; Theorem 2.121(iii) and (iv) give the image of $\forall_* y\, (E(y) \Rightarrow_* E(Sy))$. Conversely, from that image and $E(z)$, Theorem 2.121(i) with the eliminations of $\forall_*$ and $\Rightarrow_*$ gives $E(Sz)$, and QI_η at the literal equation E gives $Q_y E(y)$.

For $D_,$: apply DE at a fresh b, convert the opened $Q_y E(b, y)$ by the first part, and apply Theorem 2.121(i) with the introduction of $\exists_*$. Conversely apply Theorem 2.121(v) with $F := D_{x,y} E(x, y)$: under the image of $\forall_* y\, (E(b, y) \Rightarrow_* E(b, Sy))$ and the assumption $E(b, z)$ the eliminations give $E(b, Sz)$, which is the premise of DI_η with the witness b. □

Theorem 2.123 (Round trips)**.** *For every native formula* A *and every formula* F *of* R*,*

$$A \dashv\vdash A^{\uparrow\downarrow} \text{ in } \mathsf{N}, \qquad F \dashv\vdash_{\mathsf{R}} F^{\downarrow\uparrow},$$

over arbitrary contexts.

Proof. In N. First, $\operatorname{pad}(E)^{\downarrow}$ is interderivable with E: it is $\exists_* r\, \forall_* v\, (\operatorname{gate}(v, E) \Rightarrow_* \operatorname{gate}(Sv, E))$ with r unused. From E every $\operatorname{gate}(Sv, E)$ follows. Conversely $\operatorname{gate}(0, E)$ is derivable, and progress at 0 gives $\operatorname{gate}(1, E)$, hence E. Now induct on A. Equations are fixed. The matrix $(B^{\uparrow})^{\sharp}$ is compound, so $(Q_y B)^{\uparrow\downarrow}$ is $Q_y((B^{\uparrow})^{\sharp})^{\downarrow}$. If B is an equation, the matrix is $\operatorname{pad}(B)^{\downarrow}$, interderivable with B. If B is compound, the matrix is $B^{\uparrow\downarrow}$, interderivable with B by the induction hypothesis. The congruence of Q in N (Section 1.4) gives the claim, and $D_,$ is the same.

In R. Induct on F. Equations are fixed. If the matrix of F is a literal equation, the claim is Theorem 2.122. Let the matrix B be compound. The induction hypothesis makes B and $B^{\downarrow\uparrow}$ interderivable, and both are compound. Theorem 2.116(iv) makes $Q_y B$ and $Q_y B^{\downarrow\uparrow}$ interderivable, and since $B^{\downarrow\uparrow}$ is compound, $Q_y B^{\downarrow\uparrow}$ is $(Q_y B)^{\downarrow\uparrow}$ with no padding inserted. $D_,$ is the same. No congruence across different kinds of root is used, and the normalization is proved for every formula before the connectives are extended to all of R. □

Theorem 2.124 (The orbit calculus and the core)**.** (i) $\Gamma \vdash A$ *in* N *if and only if* $\Gamma^{\uparrow} \vdash_{\mathsf{R}} A^{\uparrow}$.

(ii) $\Gamma \vdash_{\mathsf{R}} F$ *if and only if* $\Gamma^{\downarrow} \vdash F^{\downarrow}$ *in* N.

(iii) R *derives* $0 = S0$ *if and only if* N *does.*

(iv) *With the progressive reading,* R *is bi-interpretable with* T_{eq} *and with* iPRA*, with whole domains, actual equality, identity numerical composites and both formula round trips.*

Proof. (i) One direction is Theorem 2.119. For the other, apply Theorem 2.120 to $\Gamma^{\uparrow} \vdash_{\mathsf{R}} A^{\uparrow}$ and remove $\uparrow\downarrow$ by Theorem 2.123 and cut. (ii) is symmetric, with Theorem 2.120, Theorem 2.119 and the second round trip. (iii) is (i) or (ii) for the equation $0 = S0$, which both translations fix. (iv) The translations $\uparrow$ and $\downarrow$ fix every term, so composing them with the interpretations of Theorem 1.119 adds no change of domain; the formula round trips compose. □

Corollary 2.125 (Connectives for all formulas of the orbit calculus)**.** *For formulas* F*,* G *of* R *put* $F \Rightarrow_{\mathsf{R}} G := (F^{\downarrow} \Rightarrow_{*} G^{\downarrow})^{\uparrow}$*, and likewise for* $\wedge$*,* $\vee$*,* $\exists$ *and* $\forall$*; falsity is* $0 = S0$*. These operations obey the rules of intuitionistic natural deduction in* R*, and* R *derives coverage and equation induction.*

Proof. By Theorem 2.123, $(F \Rightarrow_{\mathsf{R}} G)^{\downarrow}$ is interderivable in N with $F^{\downarrow} \Rightarrow_{*} G^{\downarrow}$. By Theorem 2.124(ii) each rule for $\Rightarrow_{\mathsf{R}}$ is therefore the rule for $\Rightarrow_{*}$ in N, applied to the readings. The other connectives are the same. □

Remark 2.126 (Sources and Lean). The calculus and Theorem 2.124 are from `uniform_progress`. The package compares R with an earlier progressive presentation that reads every closure progressively, has a recursor whose body sees the counter, two eliminations and a rule of Boolean cases; here the iterator is unchanged, compound closures keep the conditional reading, and one elimination serves both. In Lean (modules `NativeProgressive` and following) the calculus is `RPrf`, the translation $\downarrow$ is `NFm.rd` and $\uparrow$ is `NFm.emb`. Theorem 2.120 is `RPrf.toNPrf`, with `rprf_sound_pra` and `rprf_consistent`; Theorem 2.113 is `RPrf.subst`; Theorem 2.114(iii) is `RPrf.eqInd`; Theorem 2.116(ii) is `RPrf.leibniz`; Theorem 2.118 is `RPrf.padI` and `RPrf.padE`; Theorem 2.119 is `NPrf.toR`; Theorem 2.124(i) and (ii) are `rprf_emb_iff` and `rprf_iff_rd`; and the two round trips are `NEquiv.rd_emb` and `REquivIn.emb_rd`. In the Lean proof of Theorem 2.120 the only rule that needs an argument is QE_{ω} at an equation, which is an equation induction in the kernel (`NPrf.progE`).

2.6.6 Transport in a formula template

Definition 2.127 (The calculus R_F)**.** R_F is R with TR_2 replaced by TR_F; its seven rules are HYP, I0, IS, TR_F, QE_{ω}, DE and DI_{η}.

Proposition 2.128 (Mixed closure)**.** R_F *and* R *derive the same sequents. Hence Theorems 2.117 and 2.124 hold for* R_F*.*

Proof. TR_2 is the instance of TR_F at an equation template, and TR_F is derivable in R by Theorem 2.116(ii). □

Remark 2.129 (The account of `scope_and_closure`). R_F is the "mixed closure" calculus of `scope_and_closure`, written there with the orbit term $I(n, t;\, u.\, Su)$ and the selector d_B for η_B. The package proves the bi-interpretation with T_{eq} independently of `uniform_progress`, and its proof differs in three places. The gate is extended through binders, $G_v(Q_y B) := Q_y G_v B$ and $G_v(D_{x,y} B) := D_{x,y} G_v B$, with $\vdash G_0 B$ and $G_{Sv} B \dashv\vdash B$ proved by induction on B from homogeneous congruence. Every matrix is padded, $P(B) := D_{r,v} G_v B$, compound or not.

And the second formula round trip is proved through T_{eq}: with ρ the progressive reading and $K(\varphi)$ the padded native translation of an ordinary formula φ, the package shows $\rho(K\varphi) \leftrightarrow \varphi$ in T_{eq} and $B \dashv\vdash K(B^{\rho})$ in its calculus, the latter by the cases of Theorems 2.122 and 2.123. For QE_{ω} it needs only $\omega(a,1) = Sa$ inside the calculus, as here. In Lean the calculus is `MCPrf`, with `mcprf_iff_rprf`, `mcprf_iff_rd`, `mcprf_emb_iff` and `mcprf_consistent`. The package measures no checker for it and states that the single introduction moves a cost into the elimination, which builds a term and takes a second numerical parameter.

2.6.7 Elimination at a target

The orbit elimination takes two terms, a source and a seed. It can be replaced by an elimination that takes the target alone. For a matrix B and a term t put

$$\delta_B(t) := \begin{cases} r, & B \text{ is a literal equation and } t \equiv Sr, \\ 0, & \text{otherwise.} \end{cases}$$

The test is on the syntax of t; terms are not normalized before it, and $J(Sr) = I(0, Sr;\, u.\,0)$ is not a literal successor.

Definition 2.130 (The calculus T)**.** T has the seven rules HYP, I0, IS, TR_2, DE, DI_{η} and

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(\delta_B(t))}{\Gamma \vdash B(t)}\ \mathrm{QE}_{\delta}.$$

A target that is a literal successor asks for the instance at its predecessor; every other target of an atomic closure, and every target of a compound closure, asks for the instance at zero.

Lemma 2.131 (Interfaces and induction in T)**.** T *derives the equality rules and* QI_{η}*, and for a literal equation* E*, a compound matrix* B *and every term* t*:*

(i) *one step: from* $Q_y E(y)$ *and* $E(t)$ *to* $E(St)$*;*

(ii) *instance from the base: from* $Q_y E(y)$ *and* $E(0)$ *to* $E(t)$*, and from* $Q_y B(y)$ *and* $B(0)$ *to* $B(t)$*;*

(iii) *equation induction.*

Proof. The equality rules and QI_{η} are derived as in Theorem 2.114. (i) QE_{δ} at the target St. (ii) For B compound the rule is QE_{δ} itself. For E, take the target $J(t)$, which is not a literal successor: QE_{δ} gives $E(J(t))$ from $E(0)$, and $J(t) = t$ with TR_2 gives $E(t)$. (iii) QI_{η} and (ii). □

This bootstrap uses no predecessor, no numerical case distinction and no identity for the orbit term.

Lemma 2.132 (Substitution in T)**.** *If* $\Gamma \vdash_{\mathsf{T}} A$*, then* $\Gamma[\theta] \vdash_{\mathsf{T}} A[\theta]$*. Injective renaming maps rule instances to rule instances; substitution in general does not.*

Proof. By induction on the derivation, with eigenvariables renamed. All rules but QE_{δ} commute with substitution. Consider QE_{δ} with the matrix B and the target t. If B is compound, or $t \equiv Sr$, or neither t nor $t[\theta]$ is a literal successor, the substituted inference is an instance. In the remaining case B is a literal equation, the second premise is the instance at 0, and $t[\theta]$ is a literal successor; then Theorem 2.131(ii) gives $B[\theta](t[\theta])$ from the substituted premises. That lemma does not depend on the present one. □

Lemma 2.133 (Transport in T)**.** *The statements (ii) and (iv) of Theorem 2.116 hold in* T*.*

Proof. The proof of Theorem 2.116 uses TR_2, QI_η, DE, DI_η, one step at a literal equation, the instance from the base at a compound matrix, and substitution. In T these are Theorems 2.131 and 2.132, and the instance from the base at a compound matrix is primitive, so no redex has to be removed. □

Theorem 2.134 (Target elimination and orbit elimination)**.** T *and* R *derive the same sequents. Hence Theorems 2.117 and 2.124 hold for* T*.*

Proof. The two calculi differ in the elimination of Q only.

QE_δ *in* R. At a literal equation and a target Sr it is one step (Theorem 2.116(i)). In the other cases its second premise is the instance at 0, and it is Theorem 2.116(iii); for a compound matrix this uses transport in a compound formula of R, which is why that lemma precedes the comparison.

QE_ω *in* T. At a compound matrix, QE_δ with the target $\omega(0,n)$ gives the conclusion from $B(0)$ at once. At a literal equation, let $Q_y E(y)$ and $E(a)$ be given. Equation induction on k in T gives $E(\omega(k,a))$: the base is $E(a)$ with $\omega(0,a)=a$, and the step is one step at $\omega(k,a)$ with $\omega(Sk,a)=S\,\omega(k,a)$. At $k:=n$, the equation $\omega(n,a)=\omega(a,n)$, which holds in T by Theorem 2.115, gives $E(\omega(a,n))$. □

The orbit identities are needed to simulate the orbit rule in T. They are not needed by the rule QE_δ, by its soundness or by the induction of Theorem 2.131. Directly, QE_δ is sound for the progressive reading as follows: at a literal successor it uses one instance of progress, at another target of an atomic closure it uses equation induction from the instance at zero, and at a compound matrix it is the elimination of the conditional reading.

Remark 2.135 (Sources and Lean). The subsection is the first part of `target_closure`. In Lean the calculus is `TQPrf`, with `tqprf_iff_rprf`, the orbit rule derived as `TQPrf.orbit`, and the consequences `tqprf_emb_iff`, `tqprf_iff_rd` and `tqprf_consistent`. The second part of the package concerns the typing of certificates for the calculus with the conditional reading and is summarized in Section 2.9.

2.6.8 The calculi of the chapter

Table 2.1 lists the calculi of Sections 2.2 to 2.6. Each has HYP, I0, IS and DE; the second column gives its other rules. The calculus $\mathsf{U_c}$ is the exception: it is stated on normal terms and has neither I0 nor IS. The column "subst." says whether substitution maps rule instances to rule instances (Theorem 2.3); for $\mathsf{U_c}$ substitution is followed by normalization (Theorem 2.106).

Table 2.1: The calculi of the chapter.

Calculus	Rules besides HYP, I0, IS, DE	Rules	Relation to N	Subst.
N	TR, QE, DI, QI	8		yes
$\mathsf{N_D}$	TR, QE, DI, DP	8	equivalent, Theorem 2.6	yes
$\mathsf{N_{det}}$	TR, QE, DI, QI_η	8	equivalent, Theorem 2.10	yes
W	TR, QE, DI_w	7	equivalent, Theorem 2.17	no
$\mathsf{N_2}$	TR_2, QE, DI, QI	8	equivalent, Theorem 2.21	yes

Calculus	Rules besides HYP, I0, IS, DE	Rules	Relation to N	Subst.
N_F	TR_F, QE, DI, QI	8	equivalent, Theorem 2.21	yes
O	TR^-, QE, DI, QI	8	equivalent, Theorem 2.27	no
M	TR^-, QE, DI_w	7	equivalent, Theorem 2.27	no
X	TRX, QX, DI, QI	8	equivalent, Theorem 2.32	no
Z	TR_F^-, QE, DI, QI	8	equivalent, Theorem 2.35	no
G	REP, QE, DI, QI	8	equivalent, Theorem 2.47	no
G_7	REP, QE, DI_w	7	equivalent, Theorem 2.47	no
B	BREP, QE, DI, QI	8	equivalent, Theorem 2.54	no
B_7	BREP, QE, DI_w	7	equivalent, Theorem 2.54	no
B^-	BREP^-, QE, DI, QI	8	equivalent, Theorem 2.57	no
B_7^-	BREP^-, QE, DI_w	7	equivalent, Theorem 2.57	no
L	LOR, QE, DI, QI	8	equivalent, Theorem 2.60	yes
N_u	uncrossed replacement in a term template, QE, DI, QI	8	equivalent, Theorem 2.60	yes
A_7	BREP, AQE, DI_w	7	equivalent, Theorem 2.71	no
U	TR, QE, DI, UR	8	equivalent, Theorem 2.77	yes
U_o	TR, QE, DI, ORB	8	equivalent, Theorem 2.101	yes
U_c	REF, QE, DI, UX, on normal terms; no I0, IS	6	through nf, Theorem 2.108	yes
R	TR_2, QE_ω, DI_η	7	through $\uparrow$ and $\downarrow$, Theorem 2.124	yes
R_F	TR_F, QE_ω, DI_η	7	same sequents as R, Theorem 2.128	yes
T	TR_2, QE_δ, DI_η	7	same sequents as R, Theorem 2.134	no

2.7 Two results on other recursors

This section draws on the appendix of `witness_rule` on counter congruence and on the obstruction note of `scope_and_closure`. Both concern term languages other than that of Section 1.1; the formulas and the logical rules are the native ones.

2.7.1 Counter congruence at a zero seed

The appendix of `witness_rule` refines an earlier package, `counter_congruence`, whose term language has a recursor that sees the counter. The terms are variables, 0, St and $\mathrm{Rec}(n, a;\, u, v.\, h)$, which binds u, the previous counter, and v, the previous state, in h. The computation equations are

$$\mathrm{Rec}(0, a;\, u, v.\, h) = Sa, \qquad \mathrm{Rec}(Sn, a;\, u, v.\, h) = h[n/u][\mathrm{Rec}(n, a;\, u, v.\, h)/v].$$

The formulas are equations and $D_{x,y}A$; a closure $Q_y B$ occurs in a context only, as a resource in the sense of Section 1.9.

Definition 2.136 (The calculus $\mathcal{C}_0$)**.** The rules of $\mathcal{C}_0$ are: HYP for a formula of the context; lookup of a closure of the context; the two computation equations; QE, with its first premise a lookup; DE; DI; DP; root replacement on the left,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}\ \textsc{rl};$$

and counter congruence at a zero seed,

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash \mathrm{Rec}(s, 0;\, u, v.\, h) = \mathrm{Rec}(t, 0;\, u, v.\, h)}\ \textsc{zcc} \qquad (v \text{ not free in } h).$$

The seed of ZCC is the literal 0, and its body does not depend on the state, also not below nested binders. There is no reflexivity, no general congruence and no transport. The reference calculus of `counter_congruence` has, in place of ZCC, the rule CC with an arbitrary seed a and an arbitrary body. Count one node for every rule instance, lookups included.

Lemma 2.137 (Root equality and cases)**.** *In $\mathcal{C}_0$, without ZCC:*

(i) *reflexivity is derivable with three nodes, and symmetry and transitivity are derivable;*

(ii) *from $l = r$, $l = f$ and $r = g$, two instances of RL give $f = g$;*

(iii) *numerical cases, from $A(0)$ and $A(Sz)$ to $A(t)$, are derivable with four nodes besides the two premises, with no equality rule and no DP.*

Proof. (i) $\mathrm{Rec}(Ss, 0;\, u, v.\, u) = s$ is a computation equation, and RL applied to two copies of it gives $s = s$. RL applied to $s = t$ and $s = s$ gives $t = s$, and applied to $t = s$ and $t = r$ it gives $s = r$. (ii) RL on $l = r$ and $l = f$ gives $r = f$, and on $r = f$ and $r = g$ it gives $f = g$. (iii) DI with an unused coordinate and the witness 0, DE, a lookup of the opened closure, and QE. □

Theorem 2.138 (Congruence from ZCC)**.** *For every term template $F(x)$, $\mathcal{C}_0$ derives $F(s) = F(t)$ from $s = t$, with exactly 20 nodes besides the derivation of $s = t$ and with no instance of DP. Consequently $\mathcal{C}_0$ derives transport in every formula template, and it derives the same sequents as the calculus with CC and as the calculus with unrestricted transport.*

Proof. Put $U(n) := \mathrm{Rec}(n, 0;\, u, v.\, SSu)$. Its computation equations are $U(0) = S0$ and $U(Sz) = SSz$, so numerical cases give $U(n) = Sn$ for every term n, with six nodes. ZCC applies to U and gives $U(s) = U(t)$; with $U(s) = Ss$ and $U(t) = St$, Theorem 2.137(ii) gives $Ss = St$. This stage has $1 + 6 + 6 + 2 = 15$ nodes.

Put $V_F(n) := \mathrm{Rec}(Sn, 0;\, u, v.\, F(u))$. One computation equation gives $V_F(n) = F(n)$. ZCC, applied to $Ss = St$ with the body $F(u)$, gives $V_F(s) = V_F(t)$, and Theorem 2.137(ii) with $V_F(s) = F(s)$ and $V_F(t) = F(t)$ gives $F(s) = F(t)$. This stage has $1 + 1 + 1 + 2 = 5$ nodes.

Transport in a formula template $A(w)$ follows by induction on the number of binders of A. For an equation $p(w) = q(w)$, join $p(s) = p(t)$ and $q(s) = q(t)$ with $p(s) = q(s)$ by Theorem 2.137. For $D_{x,y}B(w, x, y)$, apply DE at a fresh b and DI at the witness b: the assumption $B(t, b, 0)$ is transported back to $B(s, b, 0)$, QE with the opened closure gives $B(s, b, Sz)$, and transport forward gives $B(t, b, Sz)$.

Every instance of ZCC is an instance of CC, and CC is the congruence just proved for the template $\mathrm{Rec}(x, a;\, u, v.\, h)$. In the calculus with transport, RL is transport at the left root and ZCC is transport of one occurrence in an instance of reflexivity. □

The construction of `counter_congruence` has 56 nodes for the same congruence; replacing its final assembly by Theorem 2.137(ii) gives 44, and the representation through U and V_F gives 20. These are sizes of the constructions described, not lower bounds, and the terms in the nodes grow with F.

Proposition 2.139 (A wrapped rule)**.** *Let* ZCC$^+$ *infer* $\mathrm{Rec}(Ss, 0;\, u, v.\, h) = \mathrm{Rec}(St, 0;\, u, v.\, h)$ *from* $s = t$*, with* v *not free in* h*. Replacing* ZCC *by* ZCC$^+$ *gives the same derivable sequents, and congruence in a term template then has* 5 *nodes besides the derivation of* $s = t$*.*

Proof. ZCC$^+$ with the body $F(u)$ gives $V_F(s) = V_F(t)$ directly; two computation equations and two instances of RL finish. Conversely Theorem 2.138 derives ZCC$^+$, as congruence for the template $\mathrm{Rec}(Sx, 0;\, u, v.\, h)$. □

ZCC$^+$ wraps arbitrary premises in successors. It is not ZCC with a side condition on its premise, and the difference matters:

Proposition 2.140 (Successor premises do not suffice)**.** *Restrict* ZCC *to premises of the form* $Sp = Sq$*, and keep the other rules of* $\mathcal{C}_0$*. The resulting calculus does not derive* $0 = S0 \vdash S0 = SS0$*, which* $\mathcal{C}_0$ *derives.*

Proof. Interpret terms in the standard natural numbers, with the recursor read as primitive recursion with the seed $a + 1$. Interpret an equation $s = t$ as $e(s, t)$, where $e(m, n)$ holds if $m = n$ or both m and n are in $\{0, 1\}$; interpret closures and $D_,$ by their ordinary readings over this interpretation of equations. The relation e is an equivalence, so RL preserves truth. Computation equations are true because their two sides have equal values. For the restricted rule the two counters are positive, and on positive numbers e is equality, so a true premise gives equal counters and equal values of the recursor. Lookup, QE and DE preserve truth by the readings, DI because every number is zero or a successor, and DP by induction on the natural numbers for the interpreted predicate. The assumption $0 = S0$ is true, since $e(0, 1)$ holds, and $S0 = SS0$ is false, since $e(1, 2)$ fails. $\mathcal{C}_0$ derives the sequent by congruence for S. □

The interpretation is an interpretation of the rules; it does not read the equation sign as equality and is not a model of the ordinary theory. The package assigns no checker size to $\mathcal{C}_0$ or to the wrapped rule.

2.7.2 Recursors seeded at zero

The iterator has three arguments: a counter, a seed and a body. The note of `scope_and_closure` examines what is lost if the seed is fixed to zero. Consider two term languages over variables, 0 and S. The *scalar* language has $\mathrm{It}_0(n;\, u.\, h)$ with

$$\mathrm{It}_0(0;\, u.\, h) = 0, \qquad \mathrm{It}_0(Sn;\, u.\, h) = h[\mathrm{It}_0(n;\, u.\, h)/u].$$

The *counter-aware* language has $\mathrm{Rec}_0(n;\, k, u.\, h)$ with

$$\mathrm{Rec}_0(0;\, k, u.\, h) = 0, \qquad \mathrm{Rec}_0(Sn;\, k, u.\, h) = h[n/k][\mathrm{Rec}_0(n;\, k, u.\, h)/u].$$

All statements below are about the denotations of terms in the standard natural numbers.

Theorem 2.141 (Monotonicity)**.** *Every term of either language denotes a function that is nondecreasing in each of its free variables.*

Proof. By induction on the term. Variables, 0 and S are clear. For the recursor, let the counter n and the body h denote nondecreasing functions, fix an assignment ρ of the parameters, and let a^ρ_j be the state after j steps: $a^\rho_0 = 0$ and $a^\rho_{j+1} = h(j, a^\rho_j, \rho)$. The sequence is nondecreasing in j: $a^\rho_0 = 0 \le a^\rho_1$, and $a^\rho_j \le a^\rho_{j+1}$ gives $a^\rho_{j+1} = h(j, a^\rho_j, \rho) \le h(j+1, a^\rho_{j+1}, \rho) = a^\rho_{j+2}$. For $\rho \le \sigma$, induction on j gives $a^\rho_j \le a^\sigma_j$. Since $n(\rho) \le n(\sigma)$,

$$a^\rho_{n(\rho)} \le a^\sigma_{n(\rho)} \le a^\sigma_{n(\sigma)}.$$

The scalar case is the same without the argument j of h. The argument covers a variable that occurs both in the counter and in the body. □

Theorem 2.142 (Zero sets in the scalar language)**.** *For a term $t(x_1, \ldots, x_r)$ of the scalar language let $\mathcal{Z}(t) := \{\vec{x} : t(\vec{x}) = 0\}$. Then $\mathcal{Z}(t)$ is the whole of $\mathbb{N}^r$ or a union $\bigcup_{j\in J}\{\vec{x} : x_j = 0\}$ of coordinate hyperplanes, for some $J \subseteq \{1, \ldots, r\}$, possibly empty. Every set of this form is the zero set of a term.*

Proof. By induction on t. The zero set of 0 is everything, that of a variable is its hyperplane, and that of a successor is empty. For $\mathrm{It}_0(n;\, u.\, h)$ fix the parameters and let b be the value of h at $u = 0$. If $b = 0$, every iterate from 0 is 0. If $b > 0$, monotonicity of h gives $h(v) \ge b > 0$ for every v, so every iterate after the first step is positive. Hence

$$\mathcal{Z}\big(\mathrm{It}_0(n;\, u.\, h)\big) = \mathcal{Z}(n) \cup \mathcal{Z}\big(h[0/u]\big).$$

The zero set of $h[0/u]$ is everything if the zero set of h is everything or contains the hyperplane $u = 0$, and otherwise it is the union of the same hyperplanes of the parameters. A union of two sets of the stated form is of that form. Conversely 0 and $S0$ realize the whole set and the empty set, and $\mathrm{It}_0(c;\, u.\, a)$ with u not in a has the zero set $\mathcal{Z}(c) \cup \mathcal{Z}(a)$, so nested terms of this kind realize every finite union of hyperplanes. □

Corollary 2.143 (What a zero seed loses)**.** (i) *No term of the scalar language denotes the predecessor, whose zero set is $\{0, 1\}$, or addition, whose zero set is $\{(0, 0)\}$.*

(ii) *No term of either language denotes the function that is 1 at 0 and 0 elsewhere. Hence no term denotes the selector $\mathrm{sel}(c, a, b)$, and no translation of terms recovers the iterator of Section 1.1 from either language on the same domain with the standard 0 and S.*

(iii) *In the counter-aware language $\mathrm{Rec}_0(n;\, k, u.\, k)$ denotes the predecessor, so Theorem 2.142 does not extend to it.*

Proof. (i) Theorem 2.142. (ii) The function decreases from 0 to 1, against Theorem 2.141; it is $\mathrm{sel}(c, 1, 0)$. (iii) The computation equations give 0 at 0 and n at Sn. □

These are statements about definability by terms on the same domain in the standard model. They do not exclude graphs of functions defined by formulas, definable subdomains, domains of tuples or quotient equalities; an interpretation of that kind would need its own proofs of totality and functionality, of the transfer of induction and of both composites. Whether addition is denoted by a term of the counter-aware language is not decided in the package. In Lean, Theorem 2.141 and part (ii) of the corollary are `ZTm.mono` and `ZTm.no_zero_test` in the module `ZeroSeedObstruction`, for a zero-seeded recursor whose body sees the counter, with `Tm.val_zf` for the selector of the kernel's language.

2.8 Binder depth and finite axiomatizations

This section draws on the binder-depth section and the appendix on finite axiomatizations of `reductions`. Neither result changes a rule. The first determines what native formulas of bounded binder depth express under the standard reading; the second recalls why a presentation of iPRA with finitely many principles has to use schemata, as the eight rules of N do.

2.8.1 The relations defined at bounded depth

For a native formula A let $\delta(A)$ be its *binder depth*: $\delta(s = t) := 0$ and $\delta(Q_y B) := \delta(D_{x,y} B) := 1 + \delta(B)$. Each binder has one immediate subformula, so $\delta(A)$ is also the number of occurrences of Q and $D_,$ in A; it is the binder rank $r(A)$ of Theorem 1.33 and the depth used in Theorem 8.87. Substitution of terms preserves it.

Let $\mathbb{N}$ be the standard model, in which the iterator denotes iteration. A native formula A whose free variables are among $p_1, \ldots, p_k$ *defines* the relation $\{\vec{a} \in \mathbb{N}^k : \mathbb{N} \models A^\circ[\vec{a}]\}$, where A° is the ordinary reading of Theorem 1.6; this is the *standard reading* of A, and it is classical. Let $\mathcal{R}_d(k)$ be the class of relations on $\mathbb{N}^k$ defined by native formulas of depth at most d. Numerical parameters may be among the p_i or be fixed as numerals; there are no set or function parameters. Let $\mathrm{PR}(k)$ be the class of primitive recursive relations on $\mathbb{N}^k$, and for $m \geq 1$ let $\Sigma^0_m(k)$ and $\Pi^0_m(k)$ be the classes of relations

$$\exists x_1\, \forall x_2\, \exists x_3 \cdots x_m\; R(x_1, \ldots, x_m, \vec{p}), \qquad \forall x_1\, \exists x_2\, \forall x_3 \cdots x_m\; R(x_1, \ldots, x_m, \vec{p})$$

with m alternating single quantifiers and R primitive recursive. The following facts are the standard theory of the arithmetical hierarchy (S. C. Kleene, Recursive predicates and quantifiers, Trans. Amer. Math. Soc. 53 (1943), 41–73); they rest on primitive recursive pairing, which contracts adjacent quantifiers of the same kind.

(H1) $\mathrm{PR}(k) \subseteq \Sigma^0_1(k) \cap \Pi^0_1(k)$ and $\Sigma^0_m(k) \cup \Pi^0_m(k) \subseteq \Sigma^0_{m+1}(k) \cap \Pi^0_{m+1}(k)$.

(H2) The complement of a Σ^0_m relation is Π^0_m, and conversely.

(H3) Σ^0_m is closed under existential quantification of an argument, Π^0_m under universal quantification; both are closed under finite conjunction and disjunction and under substitution of primitive recursive functions, in particular of constants, for arguments.

(H4) For every $m \geq 1$ there is a relation $U_m \in \Sigma^0_m(2)$ such that every relation in $\Sigma^0_m(1)$ is $\{n : U_m(e, n)\}$ for some e.

Lemma 2.144 (Strictness of the hierarchy)**.** *Let $\langle e, n\rangle$ be a primitive recursive pairing with primitive recursive projections $\pi_0\langle e, n\rangle = e$ and $\pi_1\langle e, n\rangle = n$, and put $H_m(q) := U_m(\pi_0 q, \pi_1 q)$ for $m \geq 1$. Then $H_m \in \Sigma^0_m(1) \setminus \Pi^0_m(1)$.*

Proof. $H_m \in \Sigma^0_m(1)$ by (H3). Suppose $H_m \in \Pi^0_m(1)$. Then $U_m(e, n) \leftrightarrow H_m(\langle e, n\rangle)$ is Π^0_m by (H3), so the relation $\neg U_m(n, n)$ is Σ^0_m by (H2) and (H3). By (H4) it is $\{n : U_m(e, n)\}$ for some e, and at $n = e$ this gives $U_m(e, e) \leftrightarrow \neg U_m(e, e)$. □

This is the diagonal argument for the hierarchy theorem. It uses the universal relations of (H4) and their primitive recursive matrices; it does not use an evaluator for all primitive recursive functions.

Theorem 2.145 (The depth classes)**.** *For every arity k,*

$$\mathcal{R}_0(k) = \mathrm{PR}(k), \qquad \mathcal{R}_d(k) = \Sigma^0_{2d}(k) \quad (d \geq 1).$$

Every relation in $\Sigma^0_{2d}(k)$ is defined by a formula of depth d without Q.

The equalities are equalities of classes of relations on the standard natural numbers. They do not say that every native formula of depth d is intuitionistically equivalent, in T_{eq} or in iPRA, to the reading of a prenex formula; the reading of a nested native formula contains implications with quantified antecedents.

Proof. Depth zero. By induction on terms, every term denotes a primitive recursive function of its free variables: $I(n, a;\, u.\, h)$ denotes the function F with $F(0, a, \vec{p}) = a$ and $F(m + 1, a, \vec{p}) = h(F(m, a, \vec{p}), \vec{p})$, where $h(\cdot, \vec{p})$ is the function denoted by the body. Equality of two primitive recursive functions is a primitive recursive relation. Conversely let $R \in \mathrm{PR}(k)$ and let χ be a function symbol of iPRA (Theorem 1.52) whose function is 0 exactly on R. By Theorem 1.102(ii), T_{eq} proves the translations σ of the defining equations of every function symbol, and T_{eq} is valid in $\mathbb{N}$; by induction on the generation of χ, the term $\sigma(\chi(\vec{p}))$ denotes the function of χ. The equation $\sigma(\chi(\vec{p})) = 0$ defines R.

Lower inclusion at positive depth. For a variable z let $G_z A$ be A with every equation $s = t$ replaced by $\mathrm{sel}(z, 0, s) = \mathrm{sel}(z, 0, t)$, and let P be the predecessor term of Theorem 1.65; $\delta(G_z A) = \delta(A)$. In $\mathbb{N}$, $\mathrm{sel}(0, 0, s)$ and $\mathrm{sel}(0, 0, t)$ both denote 0, and $\mathrm{sel}(w + 1, 0, s)$ denotes the value of s. Hence $G_0 A$ is true under every valuation, by induction on A, since a formula all of whose equations are true under every valuation is true; and $G_{w+1} A$ has the same reading as A. Since T_{eq} is valid in $\mathbb{N}$, $P(y + 1) = y$. For a formula $A(x, y, \vec{q})$ and a fresh z put $\mathcal{B}_{x,y} A := D_{x,z}\, G_z\big(A(x, P(z), \vec{q})\big)$. Its standard reading is

$$\exists x\, \big(G_0 A(x, P(0), \vec{q}) \to \forall z\, G_z A(x, P(z), \vec{q})\big),$$

whose antecedent is true; the instance $z = 0$ of the consequent is true, and the instance $z = y + 1$ has the reading of $A(x, y, \vec{q})$. So $\mathcal{B}_{x,y} A$ defines $\exists x \forall y\, A(x, y, \vec{q})$, and $\delta(\mathcal{B}_{x,y} A) = 1 + \delta(A)$. Now let $X(\vec{p}) \leftrightarrow \exists x_1 \forall y_1 \cdots \exists x_d \forall y_d\, R(\vec{x}, \vec{y}, \vec{p})$ with R primitive recursive, and let $r(\vec{x}, \vec{y}, \vec{p}) = 0$ define R, by the case of depth zero. Apply $\mathcal{B}$ to this equation d times, from the innermost pair x_d, y_d outward. The result defines X, has depth d and contains no Q. This is the construction of Theorem 8.87, which proves the equivalence in iPRA; only its standard reading is used here.

Upper inclusion. The argument is classical and concerns $\mathbb{N}$. It is proved by induction on $d \geq 1$ that every native formula A with $\delta(A) \leq d$ defines a Σ^0_{2d} relation, and a Π^0_{2d-1} relation if its root is Q. An equation defines a primitive recursive relation, which is Σ^0_{2d} by (H1). Let A be $D_{x,y} B$ or $Q_y B$ with $\delta(B) \leq d - 1$. If $d = 1$, B is an equation and defines a primitive recursive relation; if $d \geq 2$, B defines a Σ^0_{2d-2} relation by the induction hypothesis. In both cases, by (H1)–(H3), $\neg B(x, 0)$ is Π^0_{2d-1} and $\forall y\, B(x, y)$ is Π^0_{2d-1}, and so is their disjunction. Classically

$$(Q_y B)^\circ \leftrightarrow \neg B^\circ(0) \vee \forall y\, B^\circ(y), \qquad (D_{x,y} B)^\circ \leftrightarrow \exists x\, \big(\neg B^\circ(x, 0) \vee \forall y\, B^\circ(x, y)\big),$$

so $Q_y B$ defines a Π^0_{2d-1} relation, which is Σ^0_{2d} by (H1), and $D_{x,y} B$ defines a Σ^0_{2d} relation by (H3). □

Corollary 2.146 (Every level is needed)**.** (i) *For $m \geq 1$, a relation in $\Sigma^0_m \setminus \Pi^0_m$ is not defined by a native formula of depth less than $\lceil m/2 \rceil$, and a relation in $\Pi^0_m \setminus \Sigma^0_m$ is not defined by a native formula of depth less than $\lfloor m/2 \rfloor + 1$.*

(ii) *For every $d \geq 1$ the unary relation H_{2d} of Theorem 2.144 is defined by a native formula of depth d and by none of smaller depth.*

(iii) $\mathcal{R}_0(1) \subsetneq \mathcal{R}_1(1) \subsetneq \mathcal{R}_2(1) \subsetneq \cdots$.

Proof. (i) Let $X \in \Sigma^0_m \setminus \Pi^0_m$ be defined at depth e. If $e = 0$, X is primitive recursive and hence Π^0_m by (H1). If $e \geq 1$ and $2e \leq m - 1$, then $X \in \Sigma^0_{2e} \subseteq \Sigma^0_{m-1} \subseteq \Pi^0_m$ by Theorem 2.145

and (H1). Both are excluded, so $2e \geq m$. Let $X \in \Pi^0_m \setminus \Sigma^0_m$ be defined at depth e. If $e = 0$, $X \in \Sigma^0_m$; if $e \geq 1$ and $2e \leq m$, then $X \in \Sigma^0_{2e} \subseteq \Sigma^0_m$. So $2e \geq m+1$. (ii) $H_{2d} \in \Sigma^0_{2d} = \mathcal{R}_d(1)$ by Theorem 2.145, and (i) with Theorem 2.144 excludes smaller depth. (iii) The classes increase with d, and H_{2d+2} belongs to $\mathcal{R}_{d+1}(1)$ and not to $\mathcal{R}_d(1)$, by (ii). □

Corollary 2.147 (No translation of bounded depth)**.** *For every d there is an ordinary formula $\varphi(q)$ with one free variable such that no native formula A of depth at most d, with free variables among q, satisfies $\mathbb{N} \models \forall q\,(\varphi \leftrightarrow A^\circ)$. Hence no map, computable or not, from ordinary formulas to native formulas of depth at most d has $\mathsf{T}_{\mathrm{eq}} \vdash \varphi \leftrightarrow (\varphi')^\circ$ for every φ with one free variable. In particular the translations φ^* of Theorem 1.119 and $\varphi^\dagger$ of Theorem 1.122 have unbounded depth on such formulas.*

Proof. H_{2d+2} is Σ^0_{2d+2}; replacing its primitive recursive matrix by a defining equation (the case of depth zero above) gives a prenex ordinary formula $\varphi(q)$ that defines it. By Theorem 2.146(ii) no native formula of depth at most d defines it. T_{eq} is valid in $\mathbb{N}$, so a provable equivalence would be true there; and by Theorem 1.119(i), $\mathsf{T}_{\mathrm{eq}} \vdash \varphi \leftrightarrow (\varphi^*)^\circ$, and likewise for $\varphi^\dagger$ by Theorem 1.122(i). □

Remark 2.148 (Depth and the size of derived quantifiers)*.* Theorem 1.33 computes the binder rank of the derived quantifiers: $r(\forall_* x\, A) = 2r(A) + 2$, while $r(\mathcal{U}_x A) = r(A) + 1$, and $r(\exists_* x\, A) = r(A) + 1$ because the gate preserves rank. For a prenex ordinary formula φ with m single quantifiers over one equation, $\varphi^\dagger$ therefore has depth m, against $2^{m+1} - 2$ for φ^* when all quantifiers are universal. Theorem 2.146(i) bounds every native formula with the same standard reading from below: if φ defines a relation in $\Sigma^0_m \setminus \Pi^0_m$, its depth is at least $\lceil m/2 \rceil$, and Theorem 2.145 attains $m/2$ for even m by pairing each existential quantifier with the following universal one in one binder. On such formulas the single-binder translation is thus within a factor two of the least depth. The depth bound concerns binders only; it says nothing about the size of terms, about the selectors that the gates add, about formula sharing or about the length of proofs, and the node counts of Theorem 1.134 are a different measure.

Remark 2.149 (Sources and scope)*.* `reductions` states Theorem 2.145 for formulas built from equations and $D_{,}$, and adds that Q, counted as one level, does not change the classes, because a closure of outer depth d defines a Π^0_{2d-1} relation; the proof above treats both binders at once. The lower inclusion is Theorem 8.87; the upper inclusion, the strictness and Theorem 2.147 are the contribution of the package. The classification is a statement about the standard reading and the numerical variables of a formula. It is not an intuitionistic classification by provable prenex equivalence, and it does not exclude a different primitive connective, a different interpreted domain, or a shorter proof checker.

2.8.2 Finite ordinary axiomatizations

Here a *finite ordinary axiomatization* is a theory in first-order logic with finitely many nonlogical axioms in a finite signature; finitely many rule schemata with binding syntax, such as the eight rules of N, are not of this kind.

Proposition 2.150 (No finite ordinary axiomatization)**.** *No finitely axiomatized theory F in intuitionistic first-order logic over a finite signature is mutually interpretable with* iPRA *by ordinary finite-dimensional relative interpretations. In particular no such theory is bi-interpretable with* iPRA*.*

Proof. The proof applies two published theorems. Suppose F is such a theory. Replace intuitionistic logic by classical logic in F and in iPRA, keeping the nonlogical axioms; the second theory becomes classical PRA. A relative interpretation commutes with the connectives and

relativizes the quantifiers, so it maps every instance of excluded middle to a relativized instance of excluded middle, which is a classical theorem; the conditions on the domain and on the interpreted functions are proved in the intuitionistic theories already. Hence the two given interpretations remain interpretations, and the finite classical theory F^{cl} is mutually interpretable with PRA. It is consistent, because it is interpretable in PRA, which has the standard model.

The function symbols of PRA form an infinite signature. M. Bílková, D. de Jongh and J. J. Joosten (Interpretability in PRA, preprint, 2008, Section 2.1), following Beklemishev's analysis of reflection, state that PRA is mutually interpretable with the theory $(\mathrm{EA})_2^\omega$ in the finite signature of arithmetic, obtained from elementary arithmetic by ω iterations of uniform Π_2^0 reflection. A. Visser (The small-is-very-small principle, Math. Logic Quart. 65 (2019), 453–478, Section 7.3) identifies PRA as strongly locally reflexive, and his Theorem 7.5 states that a strongly locally reflexive theory is not locally mutually interpretable with a finitely axiomatized theory. Global mutual interpretability implies local mutual interpretability, which gives the contradiction. □

The reduction behind the cited theorem is the following. An interpretation of the finitely many axioms of F^{cl} is verified in one finite fragment U_0 of $(\mathrm{EA})_2^\omega$, so composition makes U_0, enlarged by the finitely many facts that witness sequentiality, interpret the whole theory; strong local reflexivity then interprets in that fragment arithmetic with its own consistency statement, which the generalized second incompleteness theorem excludes. The interpretations in Theorem 2.150 may use definable domains, finite tuples and definable equality relations, so the obstruction is not limited to the whole domain or to actual equality. It does not exclude a finite grammar with schematic formation of recursors, finitely many inference schemata, or a compact checker for an infinite family of axioms. `reductions` notes that PRA is reflexive and is not essentially reflexive, so that the argument has to use local reflexivity.

2.9 Checker sizes

This section collects the measurements reported by `direct_closure`, `direct_closure_qprogress`, `witness_rule`, `source_dispatch`, `reversal_core`, `one_context`, `one_sided_transport`, `template_free`, `backward_replacement`, `scalar_uniqueness`, `orbit_closure`, `crossed_uniqueness`, `uniform_progress` and `target_closure`. All numbers are for complete standalone Python programs under the protocol of Section 1.10, as the packages state them. Table 2.2 gives, for each program, its significant tokens, AST nodes and normalized bytes, and the same three numbers for the control it is matched against. Programs in different blocks have different input contracts and are not comparable across blocks. The compact programs of `kernel_refinement` and of `scope_and_closure` implement calculi of other chapters and are reported there; neither package measures a checker for the calculi L and R_F of this chapter. The readable complete checker of `abstraction_and_refinements` for A_7 has 1672 tokens, 1818 AST nodes and 7132 normalized bytes, against 1590 / 1741 / 6830 for its matched control, the same program for B_7 with QE and a supplied matrix, so the removal of the matrix from the certificate does not make the checker smaller; the other programs of that package check calculi of other chapters and are reported there.

Table 2.2: Checker sizes reported by the packages of the chapter: tokens, AST nodes, normalized bytes; the control is given as tokens / AST nodes / bytes.

Package	Program	Tok.	AST	Bytes	Matched control
`direct_closure`	strict tagged checker with QI	1169	1337	5116	with DP: 1187 / 1358 / 5199
	compact checker with counter vectors (experiment)	271	314	533	byte profile of `neutral_scope`: 266 / 310 / 529
`direct_closure_qprogress`	compact checker, QI at literal equations only	285	331	568	`neutral_scope`: 266 / 310 / 529
	compact checker for $\mathsf{N}_{\mathrm{det}}$	278	322	552	`neutral_scope`: 266 / 310 / 529
`witness_rule`	compact checker, two introductions, Boolean dispatch	264	310	550	byte profile of `certified_zero`: 264 / 310 / 558
	compact checker for W	266	312	555	the same; joint profile of `certified_zero`: 262 / 296 / 598
	the same with an explicit type test	266	312	559	the same
	the same with an explicit list test	266	310	563	the same
`source_dispatch`	byte profile	266	305	526	266 / 310 / 529
	AST profile	263	283	607	263 / 289 / 613
	token profile	259	289	572	261 / 292 / 576
`reversal_core`	typed checker for X	456	520	1055	old typed control: 532 / 618 / 1365; same refactoring without reversal: 450 / 514 / 1049
	X with a validator for goals of equations and $D_{,}$	379	432	830	old: 450 / 521 / 1109; without reversal: 373 / 426 / 824
	X with a validator for all goals	405	462	879	old: 481 / 560 / 1189; without reversal: 399 / 456 / 873
	compact byte profile with a scalar neutral premise	265	304	525	266 / 305 / 526
	compact AST profile with a scalar neutral premise	262	282	606	263 / 283 / 607
	compact checker for X	271	310	531	none stated
`one_context`	tagged checker for N	1089	1150	3358	for N_2: 1101 / 1160 / 3382
	strict validator, equations and $D_{,}$	359	414	803	373 / 426 / 824
	strict validator, all formulas	391	444	863	399 / 456 / 873
	full typing of the certificate	442	502	1039	450 / 514 / 1049
	strict validator with reversal, equations and $D_{,}$	365	420	809	379 / 432 / 830
	strict validator with reversal, all formulas	397	450	869	405 / 462 / 879

Package	Program	Tok.	AST	Bytes	Matched control
	full typing with reversal	448	508	1045	456 / 520 / 1055
`one_sided_transport`	strict checker for M	1171	1339	3246	TR_2 at the same sources: 1196 / 1364 / 3302
`template_free`	strict tagged checker for G_7	1027	1072	3147	W with three traversals: 1029 / 1089 / 3181; 1059 / 1124 / 3215; 1044 / 1096 / 3206
	compact forward checker with REP	397	464	865	crossed rule, another packet layout: 384 / 448 / 830
`backward_replacement`	readable strict checker for B	1647	1807	7121	the same with TR and a supplied template: 1581 / 1738 / 6971
	compact checker of the guarded calculus with regrouped packets, not of B	250	281	604	the same with flat packets: 251 / 283 / 605
	strict companion of that checker	402	451	971	the same with flat packets: 403 / 453 / 972
`scalar_uniqueness`	tagged checker for U	1124	1175	3393	for N: 1084 / 1141 / 3322
	validated codec with scope reuse, calculus unchanged	381	439	866	383 / 442 / 870
`orbit_closure`	tagged checker for U_o, templates supplied	1132	1178	3400	for U: 1124 / 1175 / 3393
	the same, templates read off the closure	1145	1194	3455	the same
`crossed_uniqueness`	strict checker for U_c, normalizer included	1909	2087	5618	none stated
	compact checker of the guarded calculus, not of U_c	269	301	502	byte profile of `deterministic_introduction`: 267 / 303 / 506
`uniform_progress`	typed checker for R	1206	1366	4553	for N: 1235 / 1407 / 4692; for W: 1173 / 1334 / 4456
	compact token profile, guarded-source calculus	256	283	593	257 / 285 / 568
	compact AST profile, the same calculus	260	278	602	261 / 279 / 603
	compact byte profile, the same calculus	263	300	521	264 / 301 / 522
`target_closure`	typed checker for T	1195	1358	4532	for R: 1206 / 1366 / 4553
	typed checker of the conditional calculus with a lexical gate	399	453	895	recursive typing: 442 / 502 / 1039
	the same with a descriptive error	404	455	963	the same

The following paragraphs summarize what the programs do beyond the rules.

direct_closure. The two strict checkers are built from one body and two fragments for the progress rule; they differ in that branch only. The saving is the witness, its scope check and one substitution; the normalized source compressed by gzip is 19 bytes larger. The experiment with counter vectors passes the counter of a computation as an empty or nonempty list, so that $I(1, a;\ u.\, h)$ is certified without the repair of Theorem 2.62; it costs four bytes. A variant of 528 bytes writes the empty list where the protocol uses the scalar zero, a canonical certificate fails on it, and it is not counted as a complete checker.

direct_closure_qprogress. The compact programs keep the arity encoding of the control. A failed test on the matrix must reject and not return a false value, which in this encoding would enter the context as the scalar zero; the first program uses a sentinel that is not JSON for the failure, and the second gives a compound matrix the mode of cases.

witness_rule. Every selected first premise is truthy and every generated computation equation is a nonempty list, so one Boolean expression selects the branch; null markers become empty lists. This saves eight bytes with the calculus unchanged, and the test of the witness adds five. The package proves that every accepting run is simulated, after a fixed change of markers, by an accepting run of the control.

source_dispatch. In canonical certificates the source is falsy exactly when it is the literal 0, so the comparison of the first premise against two patterns becomes one comparison selected by the source. The byte and AST profiles also move the pure computation of the replaced goal before the check of the first premise, and the byte profile uses the checker function itself as the invalid image of a substitution.

reversal_core. The typed checker shares one validator between terms and formulas and writes the neutral premise as a scalar; these two changes give the reduction of 310 bytes against the old control, and reversal itself adds six bytes. The compact programs of 525 bytes and 282 AST nodes check certificates without reversal, for the equivalent calculus.

one_context. In the tagged checker only the branch of transport changes. Independently, the strict validator of the goal omits the upper bound on variable indices: for a tree of the right shape that bound holds exactly when the first traversal of the unchanged kernel succeeds, which happens before any instruction is read, so the sets of accepted certificates are equal. The sizes after gzip increase in all six pairs.

one_sided_transport. The fixed side r of TR^{-} is read off the conclusion, so the instruction carries three terms where the control carries four.

template_free. One traversal serves lifting, substitution and replacement; an instruction for REP consists of its two premise derivations. Against the control with the same traversal the saving is 59 bytes. The compact forward program synthesizes its conclusion and is 35 bytes larger than its control, which has another packet layout.

backward_replacement. The readable checker validates the syntax and scope of every term, formula and proof node, rejects extra fields and Boolean values in place of integers, and accepts closed theorems at its driver; a separate entry point checks open sequents. Its control differs only in the branch of the equality rule, which takes a supplied template, and in the absence of the replacement traversal; that traversal costs 66 tokens, 69 AST nodes and 150 bytes, so removing the template from the certificate does not make the checker smaller. The two compact programs check the guarded calculus of another chapter, unchanged, and regroup the last three fields of a packet, $[m, s, u, t]$ becoming $[m, [s, u, t]]$; the package proves by a simulation of runs that the two formats accept certificates for the same canonical public theorems. The package compares with the least counts it collects from the corpus, 251 tokens, 275 AST nodes and 502 bytes, attained by three different programs: the 250 tokens lower the first by one, the other two are not improved, and the 250-token program does not implement BREP.

scalar_uniqueness. The instruction for UR carries the two templates, the target and

the premise, checked with no added assumption; the direct replacement of QI by UR costs 71 bytes. The second pair keeps the calculus of N: a fully typed codec whose goal validator relaxes the test `2 < a < d` on a variable index to `a > 2` and leaves the upper bound to the first scope traversal of the kernel, the factorization described for `one_context`; the change is four bytes.

orbit_closure. The instruction for ORB carries a proposed closure and one premise. The selected checker reads f and h off the closure, compares the seed with the instance of f at zero, and rejects a body that mentions the counter (Theorem 2.102); it is 62 bytes larger than its control, the checker for U of `scalar_uniqueness`. The translator of the package is not trusted; its output is checked by the target checker. The package also audits every Python source of the corpus, 5,825 paths with 1,380 distinct contents, by parsing and measuring without executing them, and finds the least values of the three coordinates, 256 tokens, 276 AST nodes and 506 bytes, in three different programs of other packages; it records which smaller files are not complete standalone checkers.

crossed_uniqueness. The checker for U_{c} counts the normalizer, substitution and all scope checks, introduces the eigenvariable of UX itself, and checks scopes before normalizing, so that a subterm erased by a computation is still checked. Separately, the package composes three known refinements (delayed elimination, eager selection from a pair, a shorter selector of the mode) into a compact checker of the guarded calculus of another chapter with 502 bytes against 506, and a validating companion with 1172 bytes against 1183; neither implements U_{c}, and the least token and AST counts of the corpus are not improved.

uniform_progress. The typed checker for R is 139 bytes smaller than the matched checker for N and 97 bytes larger than the matched checker for W. Its three compact programs are not checkers for R; they turn the continuation into a loop over an immutable context and write the neutral premise as a scalar.

target_closure. The checker for T saves 21 bytes against that for R by dropping the source and the seed of the orbit, and remains 76 bytes larger than that for W. The lexical gate replaces the recursive test that every value of a certificate is an integer or a list by a test on the input text: a text that the JSON parser accepts decodes to such a value exactly when all its characters are among the brackets, the comma, the minus sign, the digits and the four whitespace characters of JSON.

2.10 Further questions

The packages of the chapter state the following questions, limits and unproved claims. None of them claims a smallest calculus or a lower bound on the size of a checker.

Question 2.151 (Checkers for the direct closure)**.** The compact checkers of `direct_closure_qprogress` with QI are larger than their control, because the two openings at the root must be distinguished. Is there an encoding in which the direct closure rule gives a smaller complete checker, with malformed, aliased and scope-violating inputs counted?

Question 2.152 (The variant with counter vectors)**.** The 528-byte variant in `direct_closure` fails on a canonical certificate because the base counter is written as an empty list. Is there a conversion between the two spellings of zero that makes it complete? No false theorem is known to be accepted by it.

Question 2.153 (Deleting an inference case)**.** The direct closure rule removes a witness and keeps eight rules. Is there a basis on the same formulas that deletes a case of the checker and preserves arbitrary contexts and both formula round trips? (`direct_closure_qprogress`.)

Question 2.154 (Focused derivations)**.** Can a normalization theorem that permutes introductions and eliminations justify a simpler protocol for first premises? Equation induction makes the theorem nontrivial, and it has to precede any removal of checking. (`direct_closure_qprogress`.)

Question 2.155 (One-template transport in a compact protocol)**.** Can a certificate format expose TR or TR^- and keep the opening routine shared with QE, so that the complete checker is smaller than the compact programs under the same contract for the goal? (`one_context`, `one_sided_transport`.)

Question 2.156 (Sharing in certificates)**.** What is the least overhead in the size of certificates when TR_2 is replaced by TR? A certificate with sharing avoids the copy of the first premise (Theorem 2.22); can a checker for such certificates be smaller? (`one_context`, `one_sided_transport`.)

Question 2.157 (Constant templates)**.** Symmetry is derived from TR with a constant template. Is there an equality rule simpler than TR^- that keeps this step? A restriction to templates in which the variable occurs must replace it. (`one_sided_transport`.)

Question 2.158 (Removing both syntactic tests)**.** The test $s \not\equiv 0$ of TR^- and the test of the witness in DI_w serve the dispatch of compact checkers; the unrestricted rules are admissible. Can both tests be removed while a compact complete checker is kept? (`one_sided_transport`.)

Question 2.159 (The forward computation equation)**.** If the successor computation is replaced by $I(Sn, a;\, u.\, h) = I(n, h(a);\, u.\, h)$, is the equation IS derivable? The induction that suggests itself needs its hypothesis at the changed seed $h(a)$, while equation induction keeps parameters fixed. `one_sided_transport` leaves this open and claims no independence.

Question 2.160 (A fixed input contract)**.** Which contract should later comparisons fix: goals of equations and $D_{,}$, goals of all native formulas, or exact typing of the whole certificate? The minima reported by the packages are attained by different programs under different contracts. (`one_context`, `scalar_uniqueness`.)

Question 2.161 (Replacement without a template in a compact checker)**.** Does the rule REP give a smaller complete checker under a fixed compact contract? The compact forward program of `template_free` is larger than its control.

Question 2.162 (A checker for backward replacement)**.** Can matching under binders, with its scope shifts, share one traversal with the opening routine of the compact checkers, so that BREP removes more trusted code than it adds? The readable checker for B is larger than its matched control in all three coordinates. Does B_7^- admit a smaller complete checker than either of its two restrictions alone? An answer has to report one complete checker under a fixed input contract with all three counts. (`backward_replacement`.)

Question 2.163 (Fusing backward replacement)**.** Can BREP be fused with the computation equations or with the discharge of a witness without a second traversal of syntax? Such a rule has to handle the zero and the successor computation, constant templates and replacement below binders. What overhead in the size of certificates is unavoidable when a chosen replacement is compiled through the marker of Theorem 2.44, whose size is not optimized? (`backward_replacement`.)

Question 2.164 (A checker for elimination by abstraction)**.** The readable checker for A_7 is larger than its matched control for B_7 in all three coordinates: the abstraction traverses compound formulas and shifts its patterns below binders. Can the abstraction of AQE share one traversal with BREP and with the opening routine, or be combined with another sound protocol of certificates, so that one complete checker under a fixed input contract is below

the compact records of 244 tokens, 265 AST nodes and 477 bytes together, with an explicit interpretation? The package makes no assumption that such a combination exists. (`abstraction_and_refinements`.)

Question 2.165 (Elimination by abstraction with a restricted source)**.** Is A_7 with BREP replaced by BREP$^-$ equivalent to N? The bootstrap of Theorem 2.71 needs BREP at the source 0 for transport in an equation template, and the bootstrap of Theorem 2.57 for BREP$^-$ uses QE, which in A_7 is recovered only after that transport. A positive answer needs a different bootstrap; the package asserts no such restriction. (`abstraction_and_refinements`.)

Question 2.166 (Replacement of one occurrence in a checker)**.** Does LOR help a readable or a formally verified checker, even though a traversal by position costs more than it saves in the compact programs? (`kernel_refinement`.)

Question 2.167 (Uniqueness with fewer bodies)**.** Can UR be restricted to a family of bodies and still give the predecessor of Theorem 2.92? The reflection of equality uses mostly the case of a constant recurrence (Theorem 2.78(ii)), while the predecessor uses other bodies. For the retractions the question is answered: Theorem 2.81 needs the body Su only, and by Theorem 2.80 it needs no coverage. (`scalar_uniqueness`, `orbit_closure`, `reductions`.)

Question 2.168 (The strength of successor uniqueness)**.** Does T_{SU} derive induction for all literal equations? A positive answer has to supply a predecessor and a term that reflects equality from the body Su alone; a negative answer has to give a model of the computation equations for every body and of every instance of SU over arbitrary contexts in which equation induction fails. The model that separates induction for zero equations from induction for all equations is not such a model (Theorem 2.104). The concrete missing step is a term Δ with $\Delta(a, b) = 0 \leftrightarrow a = b$, derived from SU and the weak scalar axioms; the right inverse of Theorem 2.80 does not give it. (`orbit_closure`, `reductions`.)

Question 2.169 (Coverage on a changed domain)**.** The product model of Theorem 2.88 shows that coverage cannot be deleted from the scalar presentation on the whole domain with the fixed language. Can a presentation without coverage be bi-interpretable with iPRA through a definable domain? Such an interpretation needs definable comparison isomorphisms in both theories; an encoding that is correct in the standard model alone does not give bi-interpretability. (`reductions`.)

Question 2.170 (From depth to size)**.** Theorems 2.145 and 2.146 show that some relations need every binder of a given depth. Is there a corresponding lower bound on the size of numerical terms, on the cost of the gates and of pairing, or on the length of proofs? Such a bound would measure a different coordinate of simplicity; compare Theorem 1.134. (`reductions`.)

Question 2.171 (Successor and identity bodies)**.** Does SU together with uniqueness for the body u (Theorem 2.78(ii)) give induction for all equations? Does T_{SU} prove that addition is cancellative, which by Theorem 2.103 would give uniqueness for the body u? (`orbit_closure`.)

Question 2.172 (Compact encodings for the other forms of uniqueness)**.** Can ORB be combined with the compact encodings of goals without adding more recognition of syntax than it removes? Can UX receive a compact encoding that keeps the freshness of its eigenvariable and the independence of its body from the counter, and can the normalizer share the traversal by arity while respecting sorted positions, protected slots and reindexing? The checkers for U_{o} and U_{c} are larger than their comparators. (`orbit_closure`, `crossed_uniqueness`.)

Question 2.173 (Crossed uniqueness with clauses)**.** Can a calculus with clause contexts fuse UX with the discharge of a witness without complicating the derivation of equality in Theorem 2.107? (`crossed_uniqueness`.)

Question 2.174 (Uses of the accumulator)**.** The term φ of Theorem 2.96 is a composition of subtraction and addition with a recurrence that depends on the counter. Does it give a more direct compiler for primitive recursion, or a shorter predecessor, without coding tuples? (`scalar_uniqueness`.)

Question 2.175 (A checker for uniqueness)**.** The checker for U is 71 bytes larger than its control. Is there an architecture that shares the check of a computation equation with the check of UR and reverses the comparison? (`scalar_uniqueness`.)

Question 2.176 (Cost of the root-sensitive closure)**.** Can R, R_F or T be checked with less total work than the test of the witness in W? The matched checkers for R and T are larger than that for W, and the comparison has to include the instances at the source and at the target, the construction of the orbit term and the test of the root. (`uniform_progress`, `target_closure`, `scope_and_closure`.)

Question 2.177 (A zero seed through graphs)**.** Theorem 2.143 excludes a translation of terms. Can an interpretation by graphs of functions recover an arbitrary seed from a language with the seed fixed to zero, with proofs of totality and functionality, the transfer of induction and both composites? Is addition denoted by a term of the counter-aware language of Section 2.7.2? (`scope_and_closure`.)

Question 2.178 (Lower bounds)**.** Is there a lower bound on the size of presentations in a fixed class, with a fixed encoding and cost model? A census of existing programs is not such a bound. (`one_context`, `scope_and_closure`.)

The following statements of the packages are recorded without proof or as results that were not achieved.

- `direct_closure_qprogress` states that replacing the two computation equations by a rule of conversion in a context would still need both rewriting patterns and an account of reflexivity, and that a conversion engine left out of the count would not be a smaller checker. No calculus of this kind is given.

- `witness_rule`: no program of the package is below all three earlier coordinate minima; the restriction of the introduction is not credited with a saving of bytes.

- `one_context` states that removing a primitive equality test from the self-seeded proposals of other packages is a missing theorem and does not follow from coverage of a protected orbit.

- `scalar_uniqueness`: the converse of Theorem 2.75 is a written proof; it is not formalized in Lean, where only Theorem 2.75 is proved.

- `orbit_closure`: the equivalence of T_{SU} with T_{eq} is not established, and the package does not claim it. Its statement about the model of `canonical_recursors` (Theorem 2.104) rests on the construction of that package. The rule ORB removes a field of the certificate and makes the matched checker larger, by 7 bytes with supplied templates and by 62 bytes with templates read off the closure.

- `crossed_uniqueness`: six rule names do not by themselves make a smaller presentation than the three-family clause calculi of other packages; the checker for U_{c} is the largest of Table 2.2, and its normalizer is counted. The 502-byte program lowers one coordinate for a calculus of another chapter. The package asks whether a relational bi-interpretation on a changed domain could reduce a complete checker; the obstructions to definability by terms do not exclude it, and both composites would need proofs.

- `orbit_closure`, `crossed_uniqueness`: neither is formalized in Lean, and both rest on the written proof of Theorem 2.76.

- `template_free`: the calculus G_7, with REP and the single introduction together, is not formalized in Lean.

- `backward_replacement`: none of B, B_7, B^-, B_7^- is formalized in Lean; the package suggests extending the Lean calculus `TPrf` of G and reusing its marker lemma `Tm.rep_marker`, with the bridge unchanged. Its checker is not formally verified, and the restricted calculi are not implemented. No program of the package is below the three least counts of the corpus together, and the readable checker is not claimed to bound from below the size of an optimized checker for BREP. The package records three routes that gave no result: removing the seed of the iterator meets the obstructions for a single body, reusing the compact opening routine as a normalizer needs an account of dummy slots and of shifts of bound variables, and replacing a stored scope bound by the length of the context has to keep apart the check of a packet at the old scope and the opening of a branch at the new one.

- `abstraction_and_refinements`: Theorem 2.71 is written in the package and formalized in Lean in this repository (`NativeAbstraction`, `aprf_iff_nprf`), not in Rocq; the bridge to iPRA is the cited Theorem 1.119. The package does not claim that seven rules, the signature, the protocol of certificates or any measured source representation is minimal, and states that the coordinatewise envelope of the corpus is not a lower bound on bi-interpretable presentations. AQE is a reduction of the data of an inference and not of the size of the checker.

- `uniform_progress`, `target_closure`: neither calculus gives a new minimum in any coordinate of the matched comparison; the compact minima of other packages are not improved by a checker for R or T.

- `target_closure` reports two routes that gave no result: making the guard of an atomic matrix implicit simplifies the datatype of goals and adds a cost of elaboration that loses against a validator factored in the same way; and a stopped-clock construction in a self-seeded arithmetic gives a diagonal equation and does not prove that the successor is injective; two paired stopping orbits do (Theorem 8.69). No independence result is inferred from either.

- `scope_and_closure`: the obstruction of Section 2.7.2 is about terms in the standard model and is not an impossibility theorem for simpler axiomatizations.

- `reductions` states that it does not establish a new axiomatization below the complete-checker records, deletes no arithmetic or logical axiom, claims no absolute minimality and gives no new Lean or Rocq verification. Theorem 2.80 does not bootstrap equality reflection or induction for all equations from SU. Theorem 2.145 is a theorem about the classical standard reading; it is not an intuitionistic prenex normal form theorem and not a lower bound on the size of terms, of proofs or of a checker. Theorem 2.150 rests on the cited theorems of Visser and of Bílková, de Jongh and Joosten, which are not proved here, and it does not apply to finite rule schemata.

- The proofs of this chapter are written proofs; the Lean names in the remarks mark the statements that are formalized. The Python checkers are not formally verified, and the tests reported by the packages concern particular certificates.

Chapter 3

Guards and public languages

The native scalar core has two introduction rules: DI by cases, at every matrix, and QI by progress, at literal equations (Section 1.3). This chapter treats calculi with a single introduction of $D_{,}$. Its premise retains one local assumption, the instance of the matrix at zero (*cases*) or the instance at the eigenvariable (*progress*), and the choice between the two is made by the shape of the matrix alone: no certificate bit, no test of the witness. At a literal equation the two modes prove different formulas. They prove the same formula when the instance at zero is provable, and a *guard* is a literal shape of an equation whose instance at zero is provable by computation and into which every equation can be lifted. The guard is used in two ways. Either the formula language is restricted to a *public language* in which every atomic matrix below $D_{,}$ has the guarded shape, and an arbitrary native formula is translated into it; or the full language is kept and the progress mode is used exactly at the guarded shape.

The chapter is organized by idea. Section 3.1 defines the guarded calculi, the shapes, and what substitution does to them. Section 3.2 recovers numerical cases, implication between equations and equation induction from a guard, once for all guards, and translates the native language into a public language with both composites. Section 3.3 determines which transport sources are needed: the literal 0 is not. Section 3.4 shows that the two restrictions together are incomplete in a strictly guarded language, and gives four repairs: auxiliary formulas, the administrative orbit, the mirror guard, and protected atoms. Section 3.5 treats the guards that work on the full native language, the diagonal guard and the mirror guard with six rules. Section 3.6 compares the guards, and Section 3.7 records the checkers and their sizes.

Throughout the chapter 1 abbreviates $S0$ and 2 abbreviates $SS0$; $s \equiv t$ is literal identity of terms; a term is *iterator-headed* if it has the form $I(n, a;\, u.\, h)$ and *successor-headed* if it has the form St; and

$$W(r) := I(0, r;\, u.\, 0)$$

is the *wrapper* of r, an iterator-headed term with $W(r) = r$ by the first computation equation. For a calculus K and formulas X, Y of its language, $X \dashv\vdash_{\mathsf{K}} Y$ means that $\Gamma \vdash_{\mathsf{K}} X$ if and only if $\Gamma \vdash_{\mathsf{K}} Y$, for every context Γ of K; the subscript is omitted when the calculus is clear.

In Sections 3.1 to 3.4 transport is the uncrossed rule in a formula template,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)},$$

as the packages of those sections state it. For N it is a derived rule (Theorem 1.23). It is the rule written TR_F in Section 2.3.4, and its restriction to the sources $s \not\equiv 0$, written TR^{-} below, is the rule TR^{-}_F of that section, not the crossed rule of Theorem 2.24. The orientation of the conclusion matters as soon as the sources s are restricted (Theorem 3.34). In Section 3.5 the equality rule is the rule TR of the kernel.

3.1 Guards as literal shapes

This section collects the definitions common to `guarded_core`, `guarded_source`, `guarded_orbit`, `deterministic_auxiliary`, `mirror_guard`, `mirror_core`, `protected_core` and `diagonal_guard`.

3.1.1 The root-determined introduction

Definition 3.1 (Root-determined introduction)**.** For a matrix $B(x,y)$ put

$$\eta_B(z) := \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound (its root is } Q \text{ or } D_{,}). \end{cases}$$

The *root-determined introduction* is the rule

$$\frac{\Gamma,\ B(t,\eta_B(z)) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \mathrm{DI}_{\mathrm{root}},$$

where z is not free in Γ, in the witness t or in $D_{x,y}B$, and the displayed assumption is discharged. The rule is in *progress mode* at a literal equation and in *case mode* at a compound matrix.

The mode is read off the matrix before the witness is substituted, and it is the same for every substitution instance of the matrix, because substitution of terms for variables preserves and reflects the root of a formula.

Definition 3.2 (Guarded calculus)**.** Let $\mathcal{F}$ be a set of native formulas, the *proof language*, and $\mathcal{T} \subseteq \mathcal{F}$ a set of *templates*. Both contain every equation and are closed under substitution of terms for free variables; $\mathcal{F}$ contains $B(t,r)$ and $Q_yB(t,y)$ whenever it contains $D_{x,y}B(x,y)$, and $B(t)$ whenever it contains $Q_yB(y)$. Let $\mathcal{S}$ be a set of terms closed under injective renaming of variables, the *admissible sources*. The *guarded calculus* $\mathsf{K}(\mathcal{F},\mathcal{S})$ derives sequents $\Gamma \vdash A$, with Γ a finite list of formulas of $\mathcal{F}$ and $A \in \mathcal{F}$, by the following seven rules.

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \textsc{hyp} \qquad \frac{}{\Gamma \vdash I(0,a;\,u.\,h) = a}\ \textsc{i0} \qquad \frac{}{\Gamma \vdash I(Sn,a;\,u.\,h) = h[I(n,a;\,u.\,h)/u]}\ \textsc{is}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathrm{TR}_{\mathcal{S}} \quad (s \in \mathcal{S},\ F \in \mathcal{T}) \qquad \frac{\Gamma \vdash Q_yB(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \textsc{qe}$$

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma,\ Q_yB(b,y) \vdash A}{\Gamma \vdash A}\ \textsc{de} \qquad \frac{\Gamma,\ B(t,\eta_B(z)) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \mathrm{DI}_{\mathrm{root}}$$

In DE the variable b is not free in Γ, A or $D_{x,y}B$; in $\mathrm{DI}_{\mathrm{root}}$ the conclusion belongs to $\mathcal{F}$ and z is not free in Γ, t or $D_{x,y}B$. There is no reflexivity axiom, no successor axiom, no rule of induction, no rule that introduces Q, and no structural rule.

Three classes of sources occur: all terms; the terms $s \not\equiv 0$; and the iterator-headed terms. The condition $s \not\equiv 0$ is a test of syntax: a variable, or an iterator whose value is zero, is an admissible source. The rule with the sources $s \not\equiv 0$ is written TR^{-} and the rule with iterator-headed sources TR_I. Two kinds of proof language occur: the full native language $\mathcal{L}$, with $\mathcal{T} = \mathcal{L}$; and the proof language $\mathcal{P}^Q$ of a public language $\mathcal{P}$ (Theorem 3.16), in which Q occurs only at the root of a hypothesis and the templates are the equations and the formulas of $\mathcal{P}$.

Remark 3.3 (Source-selected elimination). With the sources $s \not\equiv 0$ the two rules TR^{-} and QE are the two branches of the rule SE of Section 2.3.4, whose first premise is $Q_vF(v)$ at the source 0 and $s = t$ at every other source. In this form, which is the one of `guarded_source`,

`guarded_orbit` and `deterministic_auxiliary`, a guarded calculus has six rule patterns: the source selects the elimination and the root of the matrix selects the introduction. The calculus Z of Theorem 2.34 has SE with the two introductions of N; the calculi of this chapter have it with DI_{root} alone.

Remark 3.4 (Clause contexts). `guarded_core` and `mirror_guard` state their calculi with the clause contexts of Section 1.9: DE adds the clause $[B(b,0);\, y.\, B(b,y)]$, an assumption is a constant clause, and one rule USE infers $B(t)$ from a derivation of A and a clause $[A;\, y.\, B(y)]$ of the context. A hypothesis $Q_y B(y)$ used by HYP and QE is the clause $[B(0);\, y.\, B(y)]$ used by USE, so the calculi below are the clause calculi of those packages for contexts of assumptions and opened clauses. The statements and proofs of this chapter hold verbatim for contexts with arbitrary clauses $[A;\, y.\, B(y)]$ of public formulas, with a translation of clauses componentwise.

3.1.2 Structural operations, the kernel, and equality at admissible sources

Lemma 3.5 (Structural operations)**.** *In every guarded calculus $\mathsf{K}(\mathcal{F},\mathcal{S})$ weakening, injective renaming of free variables and cut are admissible: if $\Gamma \vdash A$ and $\Delta \vdash C$ for every $C \in \Gamma$, then $\Delta \vdash A$. If $\mathcal{S}$ is closed under substitution, as are the class of all terms and the class of iterator-headed terms, then substitution is admissible: $\Gamma \vdash A$ implies $\Gamma[\theta] \vdash A[\theta]$.*

Proof. Each statement is proved by induction on the derivation, with the eigenvariables of DE and DI_{root} renamed away from the data that are inserted, as for N (Section 1.4). Weakening and cut do not change the matrix of an introduction or the source of a transport, so every inference keeps its rule and its mode. A renaming, and a substitution θ, preserve the root of every formula, so an introduction in progress mode remains one in progress mode and likewise for case mode; the languages $\mathcal{F}$ and $\mathcal{T}$ are closed under substitution; and the source $s[\theta]$ of a transported equation is admissible when $\mathcal{S}$ is closed under θ, which holds for every injective renaming and, in the two named cases, for every substitution. □

The class of terms $s \not\equiv 0$ is not closed under substitution: a variable is an admissible source and its instance 0 is not. A derivation with TR^- therefore cannot be substituted into rule by rule. The arguments of Sections 3.3 and 3.4 are arranged so that no such substitution is used before the unrestricted rule has been recovered.

Lemma 3.6 (Inclusion in the kernel)**.** *Every derivation of a guarded calculus $\mathsf{K}(\mathcal{F},\mathcal{S})$ is transformed, rule by rule, into a derivation of the same sequent in N.*

Proof. HYP, I0, IS, QE and DE are rules of N. $\text{TR}_{\mathcal{S}}$ is an instance of transport in a formula template (Theorem 1.23). DI_{root} at a compound matrix is DI, and at a literal equation it is the witnessed introduction by progress, which N derives (Theorem 1.16(i)). □

Consequently every guarded calculus is sound for the ordinary reading (Section 1.2): a derivable sequent of $\mathsf{K}(\mathcal{F},\mathcal{S})$ is derivable in N, hence its reading is provable in T_{eq} and in iPRA (Theorem 1.119).

Lemma 3.7 (Equality at admissible sources)**.** *Let $\mathsf{K} = \mathsf{K}(\mathcal{F},\mathcal{S})$ and let $\mathcal{S}$ contain every iterator-headed term. The following hold over every context Γ, by derivations that use only I0, IS and $\text{TR}_{\mathcal{S}}$ in equation templates.*

(i) *$\Gamma \vdash r = r$ for every term r.*

(ii) *If $\Gamma \vdash p = q$ and $p \in \mathcal{S}$, then $\Gamma \vdash q = p$; $\Gamma \vdash f(p) = f(q)$ for every term template $f(v)$; $\Gamma \vdash r = p$ implies $\Gamma \vdash r = q$; and $\Gamma \vdash E(p)$ implies $\Gamma \vdash E(q)$ for every equation template $E(v)$.*

(iii) *Every instance of* I0 *and* IS *can be reversed.*

(iv) Join at zero. *If* $p, q \in \mathcal{S}$, $\Gamma \vdash p = 0$ *and* $\Gamma \vdash q = 0$, *then* $\Gamma \vdash p = q$.

(v) Lifting. *Let* $p, a, b \in \mathcal{S}$, *let* $q \in \mathcal{S}$ *or* $q \equiv 0$, *and let* $\Gamma \vdash a = p$ *and* $\Gamma \vdash b = q$. *Then* $\Gamma \vdash p = q$ *if and only if* $\Gamma \vdash a = b$. *The implication from* $a = b$ *to* $p = q$ *needs no condition on* q.

(vi) Expansion on the right. *Let* $\mathcal{S}$ *contain every term other than the literal* 0, *let* $q \not\equiv 0$ *and* $\Gamma \vdash q = r$. *Then, for every term* s, $\Gamma \vdash s = q$ *if and only if* $\Gamma \vdash s = r$.

Proof. (i)–(iv) For the sources $s \not\equiv 0$ these are Theorems 2.36 and 2.37; the wrapper $W(r)$ is the term $J(r)$ used there. The derivations given there transport only at the sources p and q named in each item and at $W(0)$, $W(p)$, $W(q)$ and $W(r)$, and the left side of a computation equation is iterator-headed. Hence they are derivations of K for every class $\mathcal{S}$ that contains the iterator-headed terms, under the stated conditions $p, q \in \mathcal{S}$. No equation with the source 0 is used, and in particular transitivity through the literal 0 is avoided.

(v) Let $\Gamma \vdash p = q$. Transitivity through p gives $a = q$. If $q \in \mathcal{S}$, reverse $b = q$ at the source b and pass through q: $a = b$. If $q \equiv 0$, then $a = 0$ and $b = 0$, and (iv) gives $a = b$. Conversely let $\Gamma \vdash a = b$. Transport along $a = p$ in the template $v = b$ gives $p = b$, and transport along $b = q$ in the template $p = v$ gives $p = q$; the sources are a and b.

(vi) From $s = q$, transport along $q = r$ in the template $s = v$ gives $s = r$. Conversely let $\Gamma \vdash s = r$. If $r \not\equiv 0$, reverse $q = r$ at the source q and transport at the source r in the template $s = v$. If $r \equiv 0$ and $s \not\equiv 0$, then $s = 0$ and $q = 0$, and (iv) gives $s = q$. If $r \equiv 0$ and $s \equiv 0$, the required equation $0 = q$ is the reverse of $q = 0$ at the source q. The case distinction is on the syntax of the terms r and s; it is not a case distinction inside the calculus. □

With all terms as sources, (i) and (ii) are the ordinary equality rules, and the calculus has reflexivity, symmetry, transitivity, congruence and transport in every equation template.

3.1.3 Deleting the choice is not conservative

The administrative calculus is $\mathsf{U} := \mathsf{K}(\mathcal{L}, \text{all terms})$: the full native language with the root-determined introduction. It is the calculus that the compact checkers of `guarded_core` implement, and by Theorem 3.6 it is a subcalculus of N. It is a proper one.

Definition 3.8 (Mixed reading)**.** The *mixed reading* $A^\dagger$ of a native formula is

$$
\begin{aligned}
(s = t)^\dagger &:= (s = t),\\
(Q_y B)^\dagger &:= B^\dagger(0) \to \forall y\, B^\dagger(y),\\
(D_{x,y} B)^\dagger &:= \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ a literal equation},\\ \exists x\, (B^\dagger(x,0) \to \forall y\, B^\dagger(x,y)), & B \text{ compound}. \end{cases}
\end{aligned}
$$

The mixed reading depends on the syntax of the matrix, not on its meaning: it is not invariant under the replacement of a matrix by an interderivable one. It commutes with substitution, because substitution preserves the root of a matrix.

Lemma 3.9 (Soundness for the mixed reading)**.** *If* $\Gamma \vdash_{\mathsf{U}} A$, *then* T_{eq} *proves* $A^\dagger$ *from* $\Gamma^\dagger$.

Proof. By induction on the derivation. HYP, the computation equations and QE are immediate, and transport is sound because the reading commutes with substitution. At an atomic $\mathrm{DI}_{\mathrm{root}}$ the premise proves $B(t,z) \to B(t,Sz)$ for a fresh z, and generalization on z followed by existential introduction gives the progressive reading. At a compound $\mathrm{DI}_{\mathrm{root}}$ the premise proves

$B^\dagger(t,0) \to B^\dagger(t,Sz)$, and coverage $y = 0 \vee \exists z\, y = Sz$ extends it to $B^\dagger(t,0) \to \forall y\, B^\dagger(t,y)$. For DE at a literal equation E, fix x with $\forall y\,(E(x,y) \to E(x,Sy))$; induction for the literal equation $E(x,y)$ in T_{eq} gives $E(x,0) \to \forall y\, E(x,y)$, which is the reading of the hypothesis $Q_y E(b,y)$ that the rule supplies. DE at a compound matrix is existential elimination. □

Proposition 3.10 (The root-determined introduction alone is weaker)**.** *With x not free in the matrix,* N *derives* $D_{x,y}(y = 1)$ *and* U *does not.*

Proof. The rule $\mathrm{DI}_{\mathrm{root}}$ is the introduction by cases restricted to compound matrices together with the witnessed introduction by progress at literal equations. Hence U is the calculus of Theorem 2.13(ii) with transport in a formula template, and the statement is that proposition. Its interpretation is the truth of the mixed reading in the natural numbers, which Theorem 3.9 strengthens to provability in T_{eq}: the mixed reading $\exists x\, \forall y\,(y = 1 \to Sy = 1)$ of the formula fails at $y = 1$. □

Remark 3.11. `guarded_core` gives the example with the matrix $p(y) = 1$, where $p(y) := I(y,0;\, u.\, \mathrm{sel}(u,1,0))$ has the values $0,1,0$ at $0,1,2$: the introduction by cases has the base $0 = 1$, and the progressive reading fails between 1 and 2. The formula $D_{x,y}(y = 1)$ is the one of its implementation section. The formula of the proposition shows two readings of one raw binder that disagree in the natural numbers, so no formula composite can identify the atomic binder of U with its ordinary reading; no impossibility for other interpretations of U is claimed.

Hence the single introduction is complete only together with a restriction: of the language, so that the two readings agree at every atomic matrix; or of the progress mode, to matrices at which they agree.

3.1.4 The shapes

Definition 3.12 (Guarded shapes)**.** Let $B(x,y)$ be a literal equation with the distinguished variables x and y. It is

(a) *zero-seeded in y* if it is literally $I(y,0;\, u.\, h) = I(y,0;\, u'.\, k)$;

(b) *mirrored in y* if it is literally $a = I(y,a;\, u.\, h)$, the left side and the seed being the same raw term a;

(c) *diagonal in y* if it is literally $I(y,y;\, u.\, h) = I(y,y;\, u'.\, k)$;

(d) an *orbit matrix in y* if it is literally $y = I(y,a;\, u.\, Su)$ with y not free in a.

The bodies h and k are arbitrary: they may contain x, y, their own state variable and further variables; in (b) the term a may contain x and y. The two state binders of (a) and (c) are independent. An equation $p = q$ is *protected* if p and q are iterator-headed. For an equation $E \equiv (s = t)$ and a term v, the *gate* and the *mirror gate* of E at v are

$$G_v E := \big(\mathrm{sel}(v,0,s) = \mathrm{sel}(v,0,t)\big), \qquad H_v E := \big(s = \mathrm{sel}(v,s,t)\big),$$

where $G_v E$ is $\mathrm{gate}(v,E)$ of Theorem 1.24 at an equation; the *wrapped* equation is $W(s) = W(t)$; and for an equation $E(y) \equiv (s(y) = t(y))$ the *diagonal lift* is

$$\Delta_y E := \big(I(y,y;\, u.\, s(y)) = I(y,y;\, u.\, t(y))\big), \qquad u \notin \mathrm{FV}(s,t).$$

For a variable y, the gate $G_y E$ is zero-seeded in y, the mirror gate $H_y E$ is mirrored in y and $\Delta_y E$ is diagonal in y, whether or not y occurs in E; in each the update does not use its state. Each shape is a condition on the syntax. None asks for an arithmetical equality or for the provability of a base case.

Lemma 3.13 (Instances at zero)**.** *Let $\mathsf{K}(\mathcal{F}, \mathcal{S})$ be a guarded calculus whose sources include the iterator-headed terms, and let $B(x, y)$ be zero-seeded, mirrored or diagonal in y. Then $\Gamma \vdash B(t, 0)$ for every context Γ and every term t, by I0 and transport in equation templates at iterator-headed sources.*

Proof. If B is zero-seeded or diagonal, both sides of $B(t, 0)$ have the form $I(0, 0;\, u.\, h')$; each is equal to 0 by I0, and Theorem 3.7(iv) joins them. If B is mirrored, $B(t, 0)$ is $a' = I(0, a';\, u.\, h')$, the reverse of an instance of I0 (Theorem 3.7(iii)). □

An orbit matrix has no such property: its instance at zero is $0 = I(0, a;\, u.\, Su)$, which is interderivable with $0 = a$.

Lemma 3.14 (Cases at a matrix with a provable base)**.** *Let $B(x, y)$ be a literal equation such that $D_{x,y}B$ belongs to the proof language of a guarded calculus and $\Gamma \vdash B(t, 0)$ for every Γ and t. If $\Gamma, B(t, 0) \vdash B(t, Sz)$ with z fresh, then $\Gamma \vdash D_{x,y}B$.*

Proof. Cut the derivation of $B(t, 0)$ into the premise: $\Gamma \vdash B(t, Sz)$. Weaken by $B(t, z)$ and apply $\mathrm{DI}_{\mathrm{root}}$ in progress mode. The variable z stays fresh because the base that was removed does not contain it. □

Proposition 3.15 (Substitution and the shapes)**.** *Let σ be a capture-free substitution of terms for variables other than the distinguished x and y, with x and y not free in its values.*

(i) *The root of a formula is preserved and reflected by σ: B is a literal equation if and only if $B\sigma$ is.*

(ii) *The zero-seeded and the mirrored shape are preserved by σ. Neither is reflected: for variables p and q the matrix $I(y, p;\, u.\, 0) = I(y, 0;\, u.\, 0)$ is not zero-seeded and becomes zero-seeded under $p \mapsto 0$, and the matrix $p = I(y, q;\, u.\, 0)$ is not mirrored and becomes mirrored under $q \mapsto p$.*

(iii) *The diagonal shape is preserved and reflected: $B\sigma$ is diagonal in y if and only if B is. The same holds for the substitution of a witness for x, if y is not free in the witness.*

(iv) *The classes of protected equations and of iterator-headed terms are closed under σ; the class of terms other than the literal 0 is not.*

Proof. (i), (ii) and (iv) are read off the definition of substitution. In (ii) the shapes are preserved because the literal 0, the variable y and the two equal occurrences of a are mapped to 0, to y and to two equal occurrences of $a\sigma$.

(iii) Rename the binders away from the values of σ. A constructor-headed term remains headed by the same constructor. A term $r\sigma$ is the distinguished variable y if and only if r is y: another variable that is kept differs from y; the value of a substituted variable does not contain y; and a constructor-headed term does not become a variable. Hence an iterator $I(n, a;\, u.\, h)\sigma$ has the pair (y, y) as counter and seed if and only if $I(n, a;\, u.\, h)$ has. A side of the equation that is a substituted variable may become iterator-headed, but its value has no free y, so it does not acquire the shape. A side that is an iterator has the shape after σ exactly when it has it before. The bodies are unrestricted and impose no condition. The substitution of a witness is the same argument with x among the substituted variables. □

The proposition is the reason for the two treatments of a guard. In a public language the progress mode is selected by the root, which is stable, and the shape is a requirement on the language, for which preservation suffices. On the full language the progress mode is selected by the shape itself, and a shape that substitution can create changes the mode of an inference under substitution (Section 3.5).

3.1.5 Public languages

Definition 3.16 (Public language and its proof language)**.** Let $\mathfrak{A}$ be a class of equations and $\mathfrak{M}$ a class of atomic matrices, both closed under substitution, with every instance $B(t, r)$ of a matrix $B \in \mathfrak{M}$ in $\mathfrak{A}$. The *public language* $\mathcal{P}(\mathfrak{A}, \mathfrak{M})$ is the least set of formulas such that

(1) every equation of $\mathfrak{A}$ is public;

(2) $D_{x,y}B$ is public if B is a compound public formula;

(3) $D_{x,y}B$ is public if $B(x, y)$ is a matrix of $\mathfrak{M}$ in y.

A *matrix of $\mathcal{P}$ in y* is a compound public formula or a matrix of $\mathfrak{M}$ in y. The *proof language* $\mathcal{P}^Q$ consists of the public formulas, all equations, and the *resources* $Q_y B(y)$ for the matrices B of $\mathcal{P}$ in y.

A public language has no Q. Clause (3) is the restriction: an atomic matrix immediately below $D_,$ has a prescribed literal shape, and no decision procedure and no proof is involved in recognizing it. Compound matrices are unrestricted within the grammar. By Theorem 3.15 the languages below are closed under substitution. The public languages of the chapter are

$$\begin{aligned}
\mathcal{P}_0 &:= \mathcal{P}(\text{all equations, zero-seeded}),\\
\mathcal{P}_0^+ &:= \mathcal{P}(\text{all equations, zero-seeded or orbit}),\\
\mathcal{P}_m &:= \mathcal{P}(\text{all equations, mirrored}),\\
\mathcal{P}_W &:= \mathcal{P}(\text{protected equations, zero-seeded}),
\end{aligned}$$

and the calculi are listed in Table 3.1. The designated contradiction of $\mathcal{P}_0$, $\mathcal{P}_0^+$ and $\mathcal{P}_m$ is $0 = 1$; that of $\mathcal{P}_W$ is $W(0) = W(1)$.

Calculus	Proof language	Sources	Package
G	$\mathcal{P}_0^Q$	all terms	`guarded_core`
U	$\mathcal{L}$	all terms	`guarded_core`
U^-	$\mathcal{L}$	$s \not\equiv 0$	`guarded_source`, `deterministic_auxiliary`
G^-	$\mathcal{P}_0^Q$	$s \not\equiv 0$	incomplete (Theorem 3.37)
$\mathsf{G}^-_{\mathrm{orb}}$	$(\mathcal{P}_0^+)^Q$	$s \not\equiv 0$	`guarded_orbit`
M	$\mathcal{P}_m^Q$	$s \not\equiv 0$	`mirror_guard`
H	$\mathcal{P}_W^Q$	iterator-headed	`protected_core`

Table 3.1: The guarded calculi $\mathsf{K}(\mathcal{F}, \mathcal{S})$ of Sections 3.1 to 3.4. Each has the seven rules of Theorem 3.2.

3.2 The guarded bootstrap and the translation into a public language

This section draws on `guarded_core` for the gate, the padding, the implication between equations and the translation; on `mirror_guard` for the form of the induction argument that uses no substitution; on `guarded_orbit` and `protected_core` for the translation of Q; and on `deterministic_auxiliary` for the same translation. The argument is given once, for an arbitrary guard, and is instantiated in Sections 3.2.3 and 3.4.

3.2.1 Guard schemes

Definition 3.17 (Guard scheme)**.** Let $\mathsf{K} = \mathsf{K}(\mathcal{F}, \mathcal{S})$ be a guarded calculus and $\mathcal{P} = \mathcal{P}(\mathfrak{A}, \mathfrak{M})$ a public language with $\mathcal{P}^Q \subseteq \mathcal{F}$. A *guard scheme* for K and $\mathcal{P}$ on a class $\mathfrak{E} \subseteq \mathfrak{A}$ of equations, closed under substitution, consists of

- an equation $\gamma_v U \in \mathfrak{E}$ for every $U \in \mathfrak{E}$ and every term v, and
- a relation of *compatible pairs* on $\mathfrak{E}$, closed under substitution, with an equation $\mu_w(U, V) \in \mathfrak{E}$ for every compatible pair (U, V) and every term w,

both commuting with substitution, such that for all U, V, all terms w and all variables y:

(G1) $\gamma_y U$ is a matrix of $\mathfrak{M}$ in y, whether or not y occurs in U;

(G2) $\Gamma \vdash \gamma_0 U$ for every context Γ, and $\gamma_{Sw} U \dashv\vdash U$;

(G3) $\mu_0(U, V) \dashv\vdash U$ and $\mu_{Sw}(U, V) \dashv\vdash V$;

(G4) $(\gamma_y U,\, U)$ is a compatible pair.

Two schemes are used. The *zero-seeded scheme* has

$$\gamma_v(s = t) := G_v(s = t) = \big(\mathrm{sel}(v, 0, s) = \mathrm{sel}(v, 0, t)\big), \qquad \mu_w(s = t,\ r = q) := \big(\mathrm{sel}(w, s, r) = \mathrm{sel}(w, t, q)\big),$$

with every pair compatible. The *mirror scheme* has

$$\gamma_v(s = t) := H_v(s = t) = \big(s = \mathrm{sel}(v, s, t)\big), \qquad \mu_w(s = a,\ s = b) := \big(s = \mathrm{sel}(w, a, b)\big),$$

with (U, V) compatible when U and V have the same left side. In both, (G1) and (G4) hold by inspection: $G_y U$ is zero-seeded and $H_y U$ is mirrored in y, and $H_y(s = t)$ has the left side s. Conditions (G2) and (G3) are statements about a calculus and are verified where the scheme is used.

Fix a calculus, a public language and a guard scheme on $\mathfrak{E}$. For $U \in \mathfrak{E}$ and a compatible pair (U, V) put

$$L(U) := D_{a,v} \gamma_v U, \qquad (U \Rightarrow V) := D_{a,w} L\big(\mu_w(U, V)\big),$$

with fresh bound variables, a not occurring in the matrix. Both are compound public formulas, by (G1). For a formula A that is an equation of $\mathfrak{E}$ or a compound public formula, the *padding* is

$$P(A) := \begin{cases} L(A), & A \in \mathfrak{E}, \\ A, & A \text{ compound.} \end{cases}$$

$P(A)$ is always compound.

Lemma 3.18 (Padding)**.** *$L(U) \dashv\vdash U$ for every $U \in \mathfrak{E}$, and hence $P(A) \dashv\vdash A$.*

Proof. Let $\Gamma \vdash U$. The matrix $\gamma_v U$ is a literal equation, so the introduction of $L(U)$ is in progress mode; with the witness 0 its premise is $\Gamma, \gamma_z U \vdash \gamma_{Sz} U$, which follows from U by (G2) without the local assumption. Conversely let $\Gamma \vdash L(U)$. DE adds the resource $Q_v \gamma_v U$; its base $\gamma_0 U$ is derivable by (G2), QE at 1 gives $\gamma_1 U$, and (G2) gives U. The eigenvariable of DE does not occur in U. □

Lemma 3.19 (Numerical cases)**.** *Let $A(y)$ be a compound public formula or an equation of $\mathfrak{E}$, and let z be not free in Γ and A. If $\Gamma \vdash A(0)$ and $\Gamma \vdash A(Sz)$, then $\Gamma \vdash A(t)$ for every term t.*

Proof. Let A be compound. Introduce $D_{a,y}A(y)$, with a not free in A, in case mode with the witness 0: the premise $\Gamma, A(0) \vdash A(Sz)$ is the second hypothesis, weakened. DE adds $Q_yA(y)$, and QE with the first hypothesis gives $A(t)$. If A is an equation of $\mathfrak{E}$, pad the two hypotheses by Theorem 3.18, apply the compound case to $L(A(y))$, and remove the padding of $L(A(t))$. □

The context of the two hypotheses is the same Γ and does not contain the case variable. The lemma is not a rule for a case distinction on a variable that occurs in an assumption.

Lemma 3.20 (Implication between equations)**.** *For a compatible pair (U, V) the following rules are admissible:*

$$\frac{\Gamma,\ U \vdash V}{\Gamma \vdash U \Rightarrow V}, \qquad \frac{\Gamma \vdash U \Rightarrow V \qquad \Gamma \vdash U}{\Gamma \vdash V}.$$

Proof. Write μ_w for $\mu_w(U, V)$. The matrix $L(\mu_w)$ of $U \Rightarrow V$ is compound, so its introduction is in case mode, with the premise $\Gamma, L(\mu_0) \vdash L(\mu_{Sz})$. Over $\Gamma, L(\mu_0)$: Theorem 3.18 gives μ_0, (G3) gives U, the given derivation and cut give V, (G3) gives μ_{Sz}, and Theorem 3.18 gives $L(\mu_{Sz})$. For the elimination, DE adds $Q_wL(\mu_w)$; from U, (G3) and padding give $L(\mu_0)$; QE at 1 gives $L(\mu_1)$; and padding and (G3) give V. The conversions use transport outside the fresh binders only; no congruence of a binder is assumed. □

Theorem 3.21 (Equation induction from a guard)**.** *Let $E(y)$ be an equation of $\mathfrak{E}$ and z a variable not free in Γ and E. If $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$, then $\Gamma \vdash E(t)$ for every term t. The derivation uses the step premise at its own eigenvariable only: no substitution into the given derivations is made.*

Proof. Put

$$J(y) := \gamma_y E(y), \qquad \Phi(y) := \big(J(y) \Rightarrow E(y)\big),$$

which is defined by (G4) and is a compound public formula.

The implication at every term. $\Gamma \vdash \Phi(0)$ by the introduction rule of Theorem 3.20: the base $E(0)$ is derivable over Γ, and the assumption $J(0)$ is not used. For a fresh w, $\Gamma \vdash \Phi(Sw)$ by the same rule: the assumption $J(Sw)$ is $\gamma_{Sw}E(Sw)$, which gives $E(Sw)$ by (G2). Both derivations are over the context Γ, which does not contain the case variable, so Theorem 3.19 applies to the compound formula Φ and gives $\Gamma \vdash \Phi(v)$ for every term v.

Progress of the guard. Over $\Gamma, J(z)$ the elimination rule and $\Phi(z)$ give $E(z)$; the step premise and cut give $E(Sz)$; and (G2) gives $\gamma_{Sz}E(Sz)$, which is $J(Sz)$. Thus $\Gamma, J(z) \vdash J(Sz)$.

Conclusion. By (G1) the matrix $J(y)$ is atomic and admissible, so $\mathrm{DI}_{\mathrm{root}}$ in progress mode, with an unused witness coordinate a and the witness 0, gives $\Gamma \vdash D_{a,y}J(y)$. DE adds $Q_yJ(y)$. The base $J(0) = \gamma_0E(0)$ is derivable by (G2), and QE at t gives $J(t)$. With $\Phi(t)$ the elimination rule gives $E(t)$. □

Remark 3.22 (Two forms of the argument). `guarded_core`, `guarded_orbit` and `protected_core` use the implication $J(y) \Rightarrow E(Sy)$ in place of Φ. Its two branches then need the step premise at 0 and at Sw, that is, a substitution into the given derivation, and the conclusion needs a second application of numerical cases. That form is correct wherever substitution is admissible: in G and H by Theorem 3.5, and in $\mathsf{G}^{-}_{\mathrm{orb}}$ after the unrestricted transport has been recovered, where `guarded_orbit` performs the substitution in the calculus with the derived rule as an inference and then expands that rule. The form of the proof above is the one of `mirror_guard`, where it is needed because substitution is not available under the source restriction; it applies to every scheme. In each form the case distinction is made on a compound implication over a context that does not contain the case variable; a case distinction made while $J(z)$ is retained would violate the eigenvariable condition.

3.2.2 The translation and its two composites

Definition 3.23 (Translation into a public language)**.** Let a guard scheme on $\mathfrak{E}$ be given, and let ε map every equation to an equation of $\mathfrak{E}$, commuting with substitution. The translation T of native formulas is

$$\begin{aligned} T(s=t) &:= \varepsilon(s=t), \\ T(Q_y B) &:= D_{a,y} P(TB), \qquad a \text{ fresh}, \\ T(D_{x,y} B) &:= D_{x,y} P(TB). \end{aligned}$$

T leaves every term unchanged outside ε, commutes with substitution up to the names of bound variables, and maps every native formula to a public formula: every translated matrix is padded, hence compound, and the atomic matrices inside a padding are admissible by (G1). The embedding ε is the identity except for protected atoms, where it is the wrapping $\varepsilon(s=t) = (W(s) = W(t))$.

Theorem 3.24 (The public-language theorem)**.** *Let $\mathsf{K} = \mathsf{K}(\mathcal{F}, \mathcal{S})$, $\mathcal{P} = \mathcal{P}(\mathfrak{A}, \mathfrak{M})$ with $\mathcal{P}^Q \subseteq \mathcal{F}$, a guard scheme on $\mathfrak{E}$ and an embedding ε be given, and assume:*

(T1) *$\varepsilon(E) \dashv\vdash_{\mathsf{N}} E$ for every equation E;*

(T2) *K derives $\varepsilon(E)$ for every instance E of I0 and IS, and $\Gamma \vdash \varepsilon(s=t)$ and $\Gamma \vdash \varepsilon(h(s)=r)$ imply $\Gamma \vdash \varepsilon(r = h(t))$;*

(T3) *$U \dashv\vdash_{\mathsf{K}} \varepsilon(U)$ for every equation $U \in \mathfrak{A}$;*

(T4) *$\Gamma \vdash_{\mathsf{K}} B(t,0)$ for every matrix $B \in \mathfrak{M}$, every Γ and every t.*

Then:

(a) *$\Gamma \vdash_{\mathsf{N}} A$ implies $T\Gamma \vdash_{\mathsf{K}} TA$, for all native Γ and A;*

(b) *$A \dashv\vdash_{\mathsf{N}} TA$ for every native formula A;*

(c) *$B \dashv\vdash_{\mathsf{K}} TB$ for every public formula B;*

(d) *for public Γ and B: $\Gamma \vdash_{\mathsf{K}} B$ if and only if $\Gamma \vdash_{\mathsf{N}} B$.*

The transformations of derivations are effective.

Proof. For a native matrix B, the translated closure is $\mathrm{Cl}_y B := D_{a,y} P(TB)$, so $T(Q_y B) = \mathrm{Cl}_y B$.

(a). By induction on the derivation in N.

HYP: $TA \in T\Gamma$.

I0, IS, TR: these are (T2); the rule TR of N infers $r = h(t)$ from $s = t$ and $h(s) = r$.

QE: the induction hypotheses are $T\Gamma \vdash \mathrm{Cl}_y B$ and $T\Gamma \vdash TB(0)$. DE on the first adds the resource $Q_y P(TB)(y)$. Padding gives $P(TB)(0)$, QE at t gives $P(TB)(t)$, and removal of the padding gives $TB(t) = T(B(t))$.

DE: the induction hypotheses are $T\Gamma \vdash D_{x,y} P(TB)$ and $T\Gamma, \mathrm{Cl}_y B(b,y) \vdash TA$. DE in K on the first adds $Q_y P(TB)(b,y)$. Over this context derive $\mathrm{Cl}_y B(b,y)$: its matrix is compound, the introduction is in case mode with the witness 0, and its premise $P(TB)(b,0) \vdash P(TB)(b,Sz)$ is QE on the resource and the local base. Cut $\mathrm{Cl}_y B(b,y)$ into the second hypothesis, which gives TA, and close DE; b is not free in $T\Gamma$ and TA. This treats every use of the hypothesis $Q_y B(b,y)$ of the native derivation, not only its uses as the major premise of QE.

DI: the induction hypothesis is $T\Gamma, TB(t,0) \vdash TB(t,Sz)$. The matrix $P(TB)$ is compound, so the introduction of $D_{x,y} P(TB)$ is in case mode, with the premise $T\Gamma, P(TB)(t,0) \vdash$

$P(TB)(t, Sz)$: remove the padding of the base, cut it into the induction hypothesis, and pad the result.

QI: the induction hypothesis is $T\Gamma, \varepsilon E(z) \vdash \varepsilon E(Sz)$, and the goal is $D_{a,y}L(\varepsilon E(y))$. Its introduction is in case mode, with the premise $T\Gamma, L(\varepsilon E(0)) \vdash L(\varepsilon E(Sz'))$ for a fresh z'. Over $T\Gamma, L(\varepsilon E(0))$ the padding gives the base $\varepsilon E(0)$; Theorem 3.21 for the equation $\varepsilon E(y) \in \mathfrak{E}$, with the step premise weakened, gives $\varepsilon E(Sz')$; and padding gives the goal.

(b). By induction on A, using three facts about N: $Q_y B \dashv\vdash D_{a,y}B$ for a not free in B (Theorem 1.12); interderivable matrices give interderivable formulas under Q and under D, (Theorem 1.21); and $L(U) \dashv\vdash_{\mathsf{N}} U$. The third holds because (G2) holds in N by Theorem 3.6 and cut, and the proof of Theorem 3.18 then applies in N, where the introduction by cases is available at every matrix. The case of an equation is (T1). For $Q_y B$: $Q_y B \dashv\vdash D_{a,y}B \dashv\vdash D_{a,y}TB \dashv\vdash D_{a,y}P(TB)$. The case of $D_{x,y}B$ is the same without the first step.

(c). By induction on the number of binders of B, for all instances of B at once. For an equation of $\mathfrak{A}$ this is (T3).

Let $B = D_{x,y}C$ with C compound. Then TC is compound and $TB = D_{x,y}TC$. From $\Gamma \vdash D_{x,y}C$, DE adds $Q_y C(b, y)$; the introduction of $D_{x,y}TC$ at the witness b is in case mode, and its premise $TC(b, 0) \vdash TC(b, Sz)$ is obtained by converting the base to $C(b, 0)$ with the induction hypothesis, applying QE at Sz, and converting $C(b, Sz)$ to $TC(b, Sz)$. The converse is symmetric.

Let $B = D_{x,y}C$ with $C \in \mathfrak{M}$. Then $TB = D_{x,y}L(\varepsilon C)$, and every instance of C is an equation of $\mathfrak{A}$. From $\Gamma \vdash D_{x,y}C$, DE adds $Q_y C(b, y)$. The introduction of $D_{x,y}L(\varepsilon C)$ at the witness b is in case mode; its local base is not used: $C(b, 0)$ is derivable by (T4), QE gives $C(b, Sz)$, (T3) gives $\varepsilon C(b, Sz)$, and padding gives $L(\varepsilon C(b, Sz))$. Conversely, from $\Gamma \vdash D_{x,y}L(\varepsilon C)$, DE adds $Q_y L(\varepsilon C(b, y))$. The introduction of $D_{x,y}C$ at the witness b is in progress mode; its local assumption $C(b, z)$ is not used: $C(b, 0)$ is derivable by (T4), (T3) and padding give $L(\varepsilon C(b, 0))$, QE gives $L(\varepsilon C(b, Sz))$, and removal of the padding and (T3) give $C(b, Sz)$. This direction uses the provable base; it fails for an arbitrary atomic matrix, by Theorem 3.10.

(d). The implication from left to right is Theorem 3.6. Conversely let $\Gamma \vdash_{\mathsf{N}} B$. By (a), $T\Gamma \vdash_{\mathsf{K}} TB$. By (c), $\Gamma \vdash_{\mathsf{K}} TC$ for every $C \in \Gamma$, so cut gives $\Gamma \vdash_{\mathsf{K}} TB$, and (c) gives $\Gamma \vdash_{\mathsf{K}} B$. □

Corollary 3.25 (Bi-interpretation)**.** *Under the hypotheses of Theorem 3.24, the theory of the public sequents of* K*, with the ordinary reading* $(D_{x,y}B)^\circ = \exists x\,(B^\circ(x, 0) \to \forall y\, B^\circ(x, y))$*, is bi-interpretable with* iPRA*. Both numerical domains are whole, equality is equality, both numerical composites have the graph* $x = y$*, and both formula composites are provably equivalent to the identity. In particular* K *derives its contradiction* $\varepsilon(0 = 1)$ *if and only if* iPRA *proves* $\bot$*.*

Proof. The inclusion of the public language into the native language and the translation T are the identity on the numerical sort and on equality; if ε is the wrapping, the identity graph is expressed by $W(x) = W(y)$, which is $x = y$ in N by (T1). They preserve derivability by Theorem 3.6 and by (a), and their two composites are (b) and (c). Composition with the bi-interpretation of N, T_{eq} and iPRA (Theorem 1.119) gives the statement. For the contradiction: $\varepsilon(0 = 1)$ is public, so by (d) it is derivable in K if and only if it is derivable in N, that is, by (T1), if and only if N derives $0 = 1$. □

The arithmetic of the bridge, that is, the predecessor, the reflection of equality, quantifier-free induction, pairing and the compiler for primitive recursion (Theorems 1.65, 1.86 and 1.102), is not redone in any public language: it is transferred by (a) and (d).

3.2.3 The guarded core

This subsection draws on `guarded_core`. The guarded core is $\mathsf{G} = \mathsf{K}(\mathcal{P}_0^Q, \text{all terms})$: public formulas are equations and paired formulas whose atomic matrices are zero-seeded, transport is unrestricted, and the one introduction is determined by the root. Its seven rules are those of Theorem 3.2.

Theorem 3.26 (The guarded core)**.** (i) G *admits weakening, substitution and cut.*

(ii) *The zero-seeded scheme on all equations is a guard scheme for* G *and* $\mathcal{P}_0$*. Hence* G *admits numerical cases for every public formula, implication between equations, and contextual induction for every literal equation.*

(iii) *With* ε *the identity,* T *translates derivations of* N *into derivations of* G*;* $A \dashv\vdash_{\mathsf{N}} TA$ *for every native* A*;* $B \dashv\vdash_{\mathsf{G}} TB$ *for every* $B \in \mathcal{P}_0$*; and for* Γ, B *in* $\mathcal{P}_0$*,* $\Gamma \vdash_{\mathsf{G}} B$ *if and only if* $\Gamma \vdash_{\mathsf{N}} B$*.*

(iv) G *on* $\mathcal{P}_0$ *is bi-interpretable and equiconsistent with* iPRA*, with whole domains, equality as equality and identity numerical composites.*

Proof. (i) is Theorem 3.5. For (ii), all terms are sources, so Theorem 3.7(i),(ii) give the ordinary equality rules. The two sides of G_0U are equal to 0 and the two sides of $G_{Sw}(s = t)$ are equal to s and to t, by the selector equations; this is (G2). The two sides of $\mu_0(s = t, r = q)$ are equal to s and t, and those of μ_{Sw} to r and q; this is (G3). The consequences are Theorems 3.19 to 3.21. For (iii) and (iv) the hypotheses of Theorem 3.24 hold: (T1) and (T3) are trivial; in (T2) the computation equations are axioms and $r = h(t)$ follows from $s = t$ and $h(s) = r$ by transport in the template $h(v) = r$ and symmetry; (T4) is Theorem 3.13. Then apply Theorems 3.24 and 3.25. □

Corollary 3.27 (Administrative proofs are conservative)**.** *For* Γ *and* B *in* $\mathcal{P}_0$*,*

$$\Gamma \vdash_{\mathsf{U}} B \iff \Gamma \vdash_{\mathsf{G}} B \iff \Gamma \vdash_{\mathsf{N}} B.$$

On $\mathcal{P}_0$ *the mixed and the ordinary reading agree:* T_{eq} *proves* $A^\dagger \leftrightarrow A^\circ$ *for every* $A \in \mathcal{P}_0$*.*

Proof. A derivation of G is a derivation of U, a derivation of U is transformed into one of N by Theorem 3.6, and Theorem 3.26(iii) closes the circle. For the readings, at a zero-seeded matrix $E(x, y)$ the instance $E(x, 0)$ is provable, so with x fixed $\forall y\,(E(x, y) \to E(x, Sy))$ and $E(x, 0) \to \forall y\, E(x, y)$ are equivalent: from the first, by induction for the literal equation $E(x, y)$; from the second, because the base gives every instance. The equivalence passes under the existential quantifier, and at a compound matrix the two readings have the same form, so induction on A applies. □

A derivation may therefore use unguarded atomic matrices and Q formulas as intermediate formulas without changing the public theory. The converse statement fails for U as a whole (Theorem 3.10): the full language of U is not claimed to be bi-interpretable with iPRA under either reading.

Remark 3.28 (Versions). `guarded_core` states the comparison with the calculus of the reports on the language of equations and $D_,$ with clause contexts (Theorem 3.4); the clause $T(Q_yB) = D_{a,y}P(TB)$ is added in `guarded_orbit`, `deterministic_auxiliary` and `protected_core`, and `guarded_source` first replaces every Q_yB by $D_{a,y}B$. `guarded_core` derives conservativity of U from Theorem 3.9 and the agreement of the readings, through iPRA; the direct inclusion of Theorem 3.6 is the argument of `guarded_orbit` and `protected_core`. In Lean the aim of the package, one root-determined introduction, is reached on the full language by the witness-selected introduction (Theorem 2.17, `wprf_iff_nprf`); its own route is formalized in the common form described in Theorem 3.47.

3.3 Which transport sources are needed

This section draws on `guarded_source` for the dependency theorem; on `deterministic_auxiliary`, `guarded_orbit` and `mirror_guard`, each of which reproduces the same numerical argument inside its own calculus; and on `guarded_orbit` for the observation that the argument needs induction at one equation only. The singleton test, the filter and the join at zero are taken by all four from `source_dispatch`; they are proved in Section 2.3.4 for the calculus Z, which has both introductions of N, and are cited from there. This section proves what the guarded calculi need beyond that: the argument goes through with two closure conditions in place of the introductions.

Transport with the sources $s \not\equiv 0$ is as strong as unrestricted transport, on equations, as soon as a small amount of induction is available. The amount is made explicit, because in Section 3.4 it is exactly what a strictly guarded language fails to supply.

Definition 3.29 (Restricted equational kernel)**.** A *restricted equational kernel* is a relation $\Gamma \vdash A$ between finite contexts and formulas of a language that contains all equations, closed under weakening and under the rules HYP for equations, I0, IS and TR^- in equation templates: from $\Gamma \vdash s = t$ with $s \not\equiv 0$ and $\Gamma \vdash E(s)$ infer $\Gamma \vdash E(t)$, for a literal equation template $E(v)$.

The derivability relation of every guarded calculus whose sources include the terms $s \not\equiv 0$ is a restricted equational kernel, and Theorem 3.7 holds for every restricted equational kernel, since its proof uses these rules only. Consider the following two closure conditions on a restricted equational kernel.

(C) *Cases at stable left sides.* If $E(v)$ is an equation template whose left side is iterator-headed, z is a variable not in E, and $\vdash E(0)$ and $\vdash E(Sz)$ in the empty context, then $\vdash E(r)$ for every term r.

(O) *One step of an orbit.* For every term $t \not\equiv 0$ and every context Γ, if $\Gamma \vdash 0 = I(0, t;\, u.\, Su)$, then $\Gamma \vdash 1 = I(1, t;\, u.\, Su)$.

Define the terms

$$\begin{aligned} &\zeta(v) := \mathrm{sel}(v, 1, 0), && \kappa(v) := \mathrm{sel}(v, 0, 1), && \chi(v) := \mathrm{sel}(v, 1, 2), \\ &N(n) := I(n, 0;\, u.\, \zeta(u)), && R(n) := I(n, 0;\, u.\, \chi(u)), && \lambda(n) := N(R(n)). \end{aligned}$$

These are the terms Z, K, H, N, R and L of Section 2.3.4; three are renamed because H_v and $L(\cdot)$ denote the mirror gate and the padding in this chapter. The term ζ is the zero test, and $\lambda(Sv)$ tests whether v is 0.

Lemma 3.30 (Tests and filter)**.** *In a restricted equational kernel with* (C)*, the following equations are derivable over every context, for every term v:*

$$\begin{aligned} &\chi(v) = S\kappa(v), && R(1) = 1, && R(SSv) = 2, && N(1) = 1, && N(2) = 0, \\ &\lambda(1) = 1, && \lambda(SSv) = 0, && \lambda(Sv) = \zeta(v), && \mathrm{sel}(\zeta(v), v, 0) = v. \end{aligned}$$

Proof. The first eight equations are Theorem 2.38. The derivations given there are made over the empty context and use I0, IS, the operations of Theorem 3.7(i)–(iv) at sources that are iterator-headed, successor-headed or the numerals 1 and 2, and numerical cases at the two matrices $\chi(y) = S\kappa(y)$ and $\lambda(Sy) = \zeta(y)$. Both matrices have an iterator-headed left side, so the two case distinctions are instances of (C), and the derivations are valid in every restricted equational kernel with (C).

The filter of Theorem 2.39 is the equation $v = \mathrm{sel}(\zeta(v), v, 0)$, proved by cases at a matrix whose left side is a variable; (C) does not cover it. In the orientation stated here the left side

is iterator-headed and (C) applies. At 0, replace $\zeta(0)$ by 1; then $\mathrm{sel}(1,0,0) = 0$ by IS. At Sz, replace $\zeta(Sz)$ by 0; then $\mathrm{sel}(0,Sz,0) = Sz$ by I0. Both replacements are congruences at an iterator-headed source. □

Theorem 3.31 (Equational source elimination)**.** *Let $\vdash$ be a restricted equational kernel with* (C) *and* (O)*. Then $\vdash$ has unrestricted equality: over every context, symmetry, transitivity and congruence hold for all equations, and $\Gamma \vdash s = t$ and $\Gamma \vdash E(s)$ imply $\Gamma \vdash E(t)$ for every equation template $E(v)$ and every source s. Both* (C) *and* (O) *hold if $\vdash$ is closed under contextual induction for literal equations: from $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$, with z fresh, infer $\Gamma \vdash E(r)$.*

Proof. From induction to (C) *and* (O)*.* For (C), weaken the derivation of $E(Sz)$ by the assumption $E(z)$ and apply induction; the induction hypothesis is not used. For (O), write $A(n) := I(n,t;\, u.\, Su)$ and apply induction to the equation $n = A(n)$ at the target 1: the base is the hypothesis of (O), and the step $\Gamma, z = A(z) \vdash Sz = A(Sz)$ is the step of the proof of Theorem 2.40, whose transports have the sources z, $A(Sz)$ and $SA(z)$.

Symmetry at the source 0. Let $\Gamma \vdash 0 = t$ with $t \not\equiv 0$; the claim is $\Gamma \vdash t = 0$. Read the proof of Theorem 2.40 with the given derivation of $0 = t$ in place of the hypothesis. It derives $0 = A(0)$ from $0 = t$, uses its induction for the single conclusion $1 = A(1)$, which is (O), and derives $\zeta(t) = 1$ from $1 = A(1)$ by IS and Theorem 3.30. With the filter in the orientation of Theorem 3.30 the last step reads: put $q := \mathrm{sel}(\zeta(t), t, 0)$; the filter gives $q = t$; the replacement of $\zeta(t)$ by 1 and IS give $q = \mathrm{sel}(1,t,0) = 0$; reverse $q = t$ at the source q and pass through q to obtain $t = 0$.

Unrestricted equality. Symmetry, congruence and transitivity at every source follow from symmetry at the source 0 and Theorem 3.7(ii),(iv) as in the proof of Theorem 2.35, which uses nothing else; the cut made there is not needed, because symmetry at the source 0 has been obtained as a derived rule. For an equation template $a(v) = b(v)$, congruence gives $a(s) = a(t)$ and $b(s) = b(t)$, and symmetry and transitivity turn $a(s) = b(s)$ into $a(t) = b(t)$. □

The proof uses induction in two places only: through (C), where the induction hypothesis is ignored, and through (O), at the single equation $n = I(n,t;\, u.\, Su)$ with t and the context fixed. The variable source z of the step of (O) is never replaced by 0: the target 1 is supplied by the induction rule, not by a substitution into the step. No logical rule occurs: no binder, no introduction by cases. This removes the apparent circularity between the two reductions, in which the recovery of transport at the source 0 would need logical cases and the recovery of logical cases at an equation would need unrestricted transport.

Corollary 3.32 (The equational fragment)**.** *Let $\mathcal{E}^-$ be the calculus whose formulas are the equations and whose rules are* HYP, I0, IS, TR$^-$ *in equation templates and contextual induction for literal equations. Unrestricted transport in equation templates is admissible in $\mathcal{E}^-$. The restriction $s \not\equiv 0$ changes neither its theorems nor its consequences from hypotheses.*

Theorem 2.35 is the case of Theorem 3.31 for the calculus Z, which has QI and hence induction; what Theorem 3.31 adds is that the two closure conditions suffice, which is what the calculi of Section 3.4 need. The Lean statements for the fragment, `ZKernel.trF` and `zprf_iff_nprf`, are recorded in Theorem 2.41.

Proposition 3.33 (Transport in public templates)**.** *Let K be a guarded calculus whose proof language contains $\mathcal{P}^Q$ for a public language $\mathcal{P} = \mathcal{P}(\mathfrak{A}, \mathfrak{M})$. Assume that K has unrestricted equality in the sense of Theorem 3.31, and that for every $B \in \mathfrak{M}$ either $\Gamma \vdash B(t,0)$ for all Γ and t, or $\Gamma \vdash D_{x,y}B'$ for all Γ and all substitution instances B' of B. Then for every template $F(v) \in \mathcal{P}$,*

$$\Gamma \vdash s = t \quad \textit{and} \quad \Gamma \vdash F(s) \quad \textit{imply} \quad \Gamma \vdash F(t).$$

Proof. By induction on the number of binders of F, for all contexts and all s, t. For an equation this is the hypothesis. Let $F(v) = D_{x,y}B(v, x, y)$. If every instance of $D_{x,y}B$ is derivable, nothing is to be done. Otherwise apply DE to $F(s)$, which adds $Q_yB(s, b, y)$, and introduce $F(t)$ at the witness b.

If B is compound the introduction is in case mode. From its base $B(t, b, 0)$ the induction hypothesis, with the reversed equation $t = s$, gives $B(s, b, 0)$; QE gives $B(s, b, Sz)$; and the induction hypothesis gives $B(t, b, Sz)$.

If B is atomic the introduction is in progress mode and its local assumption is not used: $B(s, b, 0)$ is derivable, QE gives $B(s, b, Sz)$, and transport in the equation template gives $B(t, b, Sz)$. □

Hence, under the hypotheses of the proposition, a calculus $\mathsf{K}(\mathcal{P}^Q,\ s \not\equiv 0)$ derives the same sequents as $\mathsf{K}(\mathcal{P}^Q$, all terms): every unrestricted transport is replaced by its derivation and the other inferences are kept. The proposition is about public templates. No transport at the source 0 is asserted in a template that contains an unguarded atomic matrix below a binder.

Remark 3.34 (The crossed rule). The induction of Theorem 3.31 is the price of the uncrossed conclusion. For the rule TR of the kernel the restriction $s \not\equiv 0$ costs nothing: Theorem 2.25(ii) reverses an equation with the source 0 in one inference, by a constant template, and needs neither the tests nor an induction; the same holds for the reversing rule of Theorem 2.32. The calculi of Sections 3.1 to 3.4 have the uncrossed rule, and for it the obstruction of Theorem 3.36 is real.

Remark 3.35 (Substitution under a source restriction). As for the calculi of Theorem 2.28, substitution is admissible in a source-restricted guarded calculus once its equivalence with the unrestricted calculus is known, and what fails is the rule-by-rule substitution of Theorem 3.5; `guarded_source`, `guarded_orbit` and `protected_core` make the same point. The equivalence must be proved first, and none of the proofs of this chapter substitutes into a derivation with TR^- before that point.

3.4 The failure of composition and its repairs

This section draws on `mirror_guard`, `guarded_orbit` and `deterministic_auxiliary` for the obstruction; on `guarded_source` and `deterministic_auxiliary` for the repair by auxiliary formulas; on `guarded_orbit` for the repair by the orbit; on `mirror_guard` for the mirror guard; and on `protected_core` for protected atoms.

The guarded core removes the choice of the introduction mode (Theorem 3.26), and the source restriction removes the transports at the literal 0 (Theorem 3.31). The first result is stated for unrestricted transport and the second for a calculus with induction at arbitrary equations. In the strictly guarded calculus $\mathsf{G}^- = \mathsf{K}(\mathcal{P}_0^Q,\ s \not\equiv 0)$ neither hypothesis is available, and the two results do not compose.

3.4.1 The obstruction

An equation is *zero-left* if its left side is the literal 0. A formula of a proof language $\mathcal{P}^Q$ is *safe* if it is not a zero-left equation: it is a public compound formula, a resource, or an equation whose left side is not the literal 0.

Proposition 3.36 (An inert zero-left hypothesis)**.** *Let $\mathcal{P} = \mathcal{P}(\text{all equations}, \mathfrak{M})$, where the left side of every matrix of $\mathfrak{M}$ is iterator-headed or successor-headed, and let $\mathsf{K} = \mathsf{K}(\mathcal{P}^Q,\ s \not\equiv 0)$. If Γ consists of safe formulas, F is safe and $\Gamma,\ 0 = t \vdash_{\mathsf{K}} F$, then $\Gamma \vdash_{\mathsf{K}} F$.*

Proof. By induction on the derivation, for all safe Γ and F. Every instance of a matrix of $\mathcal{P}$ is safe: a compound formula stays compound, and the left side of an instance of a matrix of $\mathfrak{M}$ keeps its head.

HYP: the conclusion is safe, so it is not the hypothesis $0 = t$ and belongs to Γ. I0, IS: no hypothesis is used.

TR$^-$, with the premises $p = q$ and $F(p)$ and the safe conclusion $F(q)$: the major premise is safe because $p \not\equiv 0$. The minor premise is safe: if F is compound this is clear; if $F(v)$ is $a(v) = b(v)$ and $a(p) \equiv 0$, then $a(v)$ is the constant 0, because a constructor-headed term and the variable v do not become the literal 0 when v is replaced by $p \not\equiv 0$; then $a(q) \equiv 0$, against the safety of $F(q)$. The induction hypothesis applies to both premises.

QE, with the premises $Q_y B(y)$ and $B(0)$: both are safe, the first as a resource and the second as an instance of a matrix.

DE: the major premise is compound; the minor premise has the same conclusion and one more hypothesis, the resource $Q_y B(b, y)$, which is safe.

DI$_{\mathrm{root}}$: the local assumption $B(t', \eta_B(z))$ and the conclusion $B(t', Sz)$ of the premise are instances of a matrix, hence safe. □

Corollary 3.37 (The strictly guarded composition is incomplete)**.** *In* G^- *the sequents* $0 = 1 \vdash 1 = 0$ *and* $0 = x \vdash x = 0$*, for a variable* x*, are not derivable. Both are derivable in* G*. The same holds for every calculus of Theorem 3.36, for instance with the atomic matrices* $\mathrm{sel}(y, t, s) = t$*.*

Proof. The conclusions $1 = 0$ and $x = 0$ are safe. By Theorem 3.36 a derivation would give $\vdash 1 = 0$, respectively $\vdash x = 0$, in the empty context, hence in N by Theorem 3.6; both fail in the natural numbers. In G symmetry is Theorem 3.7(ii). □

The proposition says where the composition breaks. Every equation that a strictly guarded derivation can obtain from a binder has a stable left side: the instances of a zero-seeded matrix, the local assumptions and the premises of the introductions. A transport with a source $s \not\equiv 0$ cannot turn a zero-left equation into a safe one. A zero-left hypothesis therefore has no path to a safe conclusion. The proposition does not apply to a context with a clause whose antecedent is zero-left and whose consequent is safe, and each of the repairs below supplies such a clause or removes the zero-left hypotheses from the public language.

Remark 3.38 (Versions of the obstruction). Theorem 3.36 is stated in `mirror_guard`, for every guard with a constructor-headed left side, and in `guarded_orbit` for the hypothesis $0 = x$. `deterministic_auxiliary` proves the first sequent of Theorem 3.37 underivable by a counterinterpretation over the natural numbers: every zero-left equation is read as true, every other equation as equality, and the compound formulas by their ordinary reading. Every equation below a binder of a formula of $\mathcal{P}_0^Q$ has an iterator-headed left side, so it is read as equality and the reading commutes with substitution; the computation equations, both modes of the introduction and QE and DE are sound; and TR$^-$ is sound because its major premise $p = q$ is not zero-left and an equation template whose instance at p is zero-left has the constant left side 0, so that its instance at q is true by convention. The hypothesis $0 = 1$ is true and the conclusion $1 = 0$ is false. In Lean the calculus `PGPrf` with the root-determined introduction and transport at iterator-headed sources does not derive $x = z$ from $x = y$ and $y = z$ between variables (`pgprf_not_trans`), by a reading in which a value is a number with a colour and equality is not transitive; hence it is strictly weaker than the kernel on arbitrary sequents (`pgprf_strictly_weaker`).

3.4.2 First repair: auxiliary formulas

This subsection draws on `guarded_source` and `deterministic_auxiliary`. The public language remains $\mathcal{P}_0$, and derivations are made in $\mathsf{U}^- = \mathsf{K}(\mathcal{L},\, s \not\equiv 0)$: the proof language is the full native language, in which an atomic matrix below $D_{,}$ need not be guarded. A public theorem is a public formula with a derivation in U^-; its intermediate formulas need not be public. This convention is part of the calculus. By Theorem 3.3 its rules are HYP, the two computation equations, SE, DE and $\mathrm{DI}_{\mathrm{root}}$.

Theorem 3.39 (Source selection and root-determined introduction, with auxiliary formulas). (i) U^- *admits contextual induction for every literal equation, by a derivation without transport, and numerical cases for every native formula.*

(ii) U^- *has unrestricted equality and admits transport, at every source, in every template of* $\mathcal{P}_0$.

(iii) *With the zero-seeded scheme and* ε *the identity,* T *translates derivations of* N *into derivations of* U^-, *and* $B \dashv\vdash_{\mathsf{U}^-} TB$ *for every* $B \in \mathcal{P}_0$.

(iv) *For* Γ *and* B *in* $\mathcal{P}_0$,

$$\Gamma \vdash_{\mathsf{U}^-} B \iff \Gamma \vdash_{\mathsf{G}} B \iff \Gamma \vdash_{\mathsf{N}} B,$$

and U^- *on* $\mathcal{P}_0$ *is bi-interpretable and equiconsistent with* iPRA, *with whole domains, equality as equality and identity numerical composites.*

Proof. (i) Let $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$. The formula $D_{a,y}E(y)$, with a not free in E, belongs to $\mathcal{L}$, and its introduction is in progress mode: with the witness 0 its premise is the given step. DE adds $Q_y E(y)$, in which the eigenvariable does not occur, and QE with the base gives $E(t)$. For numerical cases at an equation, use this induction and ignore the induction hypothesis; at a compound formula $A(y)$, introduce $D_{a,y}A(y)$ in case mode and proceed in the same way.

(ii) By (i) and Theorem 3.31, U^- has unrestricted equality; additional hypotheses of the context are passive in that proof. Theorem 3.33 with Theorem 3.13 gives the transport in public templates.

(iii) and (iv). With unrestricted equality the zero-seeded scheme satisfies (G2) and (G3) in U^- exactly as in Theorem 3.26, and (T1) to (T4) hold for the same reasons. Apply Theorems 3.24 and 3.25; the middle member of the chain is Theorem 3.26(iii). □

The induction of (i) is made at an arbitrary literal equation, and the matrix E is in general not guarded. The proof of Theorem 3.31 uses it at the equation $n = I(n, t;\, u.\, Su)$, whose left side is a variable. By Theorem 3.37 this use of an unguarded matrix cannot be replaced by guarded ones: the auxiliary formulas are a necessary part of the calculus, and they are why the proof language is stated explicitly. The theorem asserts the equivalence and the composites for the public language. It asserts neither a formula composite nor transport at the source 0 for all auxiliary formulas; by Theorem 3.10 the former fails.

Remark 3.40 (Versions). `guarded_source` proves Theorem 3.31 for the fragment $\mathcal{E}^-$ and instantiates it in U^-, with the filter in the orientation $v = \mathrm{sel}(\zeta(v), v, 0)$; `deterministic_auxiliary` runs the numerical argument directly in U^-, with the filter as in Theorem 3.30. Both prove the implication from U^- to G in (iv) through iPRA, by Theorem 3.9, the agreement of the readings on $\mathcal{P}_0$ (Theorem 3.27) and the faithfulness of the bridge; Theorem 3.6 gives it directly. In Lean the combination of source selection with a single introduction is proved on the full native language, with the witness-selected introduction and the crossed rule (the seven-rule calculus of Theorem 2.24, `mprf_iff_nprf`); the auxiliary formulas of the two packages are not formalized.

3.4.3 Second repair: the administrative orbit

This subsection draws on `guarded_orbit`. The auxiliary formulas of the first repair are reduced to one family. The proof language is $(\mathcal{P}_0^+)^Q$: the guarded language together with the paired formulas over orbit matrices $y = I(y, a;\, u.\, Su)$, y not free in a. The calculus is $\mathsf{G}^-_{\mathrm{orb}} = \mathsf{K}((\mathcal{P}_0^+)^Q,\ s \not\equiv 0)$, with the rules of Theorem 3.2; no rule is added for the orbit matrices. The public language remains $\mathcal{P}_0$.

An equation is *stable-left* if its left side is iterator-headed or successor-headed. The class of stable-left equations is closed under substitution, and none of its members is zero-left.

Theorem 3.41 (The orbit bootstrap). (i) *In G^-, hence in $\mathsf{G}^-_{\mathrm{orb}}$, the zero-seeded scheme on the stable-left equations is a guard scheme for $\mathcal{P}_0$. Hence numerical cases hold for every compound public formula and every stable-left equation, and condition* (C) *of Section 3.3 holds.*

(ii) *For every term t with y not free in it and every context, $\mathsf{G}^-_{\mathrm{orb}}$ derives the* orbit closure

$$D_{a,y}(y = I(y, t;\, u.\, Su)),$$

and condition (O) *holds.*

(iii) *$\mathsf{G}^-_{\mathrm{orb}}$ has unrestricted equality, admits transport at every source in every template of $\mathcal{P}_0^+$, and derives the same sequents as $\mathsf{K}((\mathcal{P}_0^+)^Q$, all terms).*

(iv) *With the zero-seeded scheme on all equations and ε the identity, T translates derivations of N into derivations of $\mathsf{G}^-_{\mathrm{orb}}$; $B \dashv\vdash TB$ for every $B \in \mathcal{P}_0$; and for Γ and B in $\mathcal{P}_0$,*

$$\Gamma \vdash_{\mathsf{G}^-_{\mathrm{orb}}} B \iff \Gamma \vdash_{\mathsf{G}} B \iff \Gamma \vdash_{\mathsf{N}} B.$$

The public theory is bi-interpretable and equiconsistent with iPRA*, with whole domains, equality as equality and identity numerical composites.*

Proof. (i) (G1) and (G4) hold by inspection, and G_vU and $\mu_w(U, V)$ are stable-left because their left sides are iterator-headed. For (G2): the two sides of G_0U are iterator-headed and equal to 0 by I0, so Theorem 3.7(iv) proves G_0U. For $U \equiv (s = t)$ put $a := \mathrm{sel}(Sw, 0, s)$ and $b := \mathrm{sel}(Sw, 0, t)$; then $a = s$ and $b = t$ by IS, the term s is not the literal 0 because U is stable-left, and Theorem 3.7(v) gives $U \dashv\vdash G_{Sw}U$. (G3) is the same lemma: for $\mu_0(s = t, r = q)$ with $a := \mathrm{sel}(0, s, r)$, $b := \mathrm{sel}(0, t, q)$, and for μ_{Sw} with $a := \mathrm{sel}(Sw, s, r)$, $b := \mathrm{sel}(Sw, t, q)$, where $r \not\equiv 0$ because the second equation is stable-left. The consequences are Theorem 3.19, and (C) is its case of an equation with an iterator-headed left side over the empty context.

(ii) Write $A(n) := I(n, t;\, u.\, Su)$. The matrix $y = A(y)$ is an orbit matrix, so the formula belongs to $\mathcal{P}_0^+$ and its introduction is in progress mode. With the witness 0 the premise is $\Gamma, z = A(z) \vdash Sz = A(Sz)$, which is the step derived in the proof of Theorem 3.31: the source z is a variable. No base is needed. For (O), let $\Gamma \vdash 0 = A(0)$. DE on the orbit closure adds $Q_y(y = A(y))$, and QE with the base at the target 1 gives $1 = A(1)$.

(iii) By (i), (ii) and Theorem 3.31 the calculus has unrestricted equality. Theorem 3.33 applies: a zero-seeded matrix has provable instances at zero (Theorem 3.13), and every paired formula over an orbit matrix is derivable by (ii).

(iv) With unrestricted equality the zero-seeded scheme on all equations satisfies (G2) and (G3) as in Theorem 3.26, and (T1) to (T4) hold. Apply Theorems 3.24 and 3.25 to the calculus $\mathsf{G}^-_{\mathrm{orb}}$ and the public language $\mathcal{P}_0$. □

In the derivation of $0 = t \vdash t = 0$ the orbit closure is eliminated with the zero-left base $0 = A(0)$ and yields the stable-left equation $1 = A(1)$. This is the clause that Theorem 3.36

excludes from a strictly guarded context, and it is the only use of an unguarded matrix: by (i) the cases needed by Theorem 3.30 are available in G^- itself. The filter must be taken in the orientation of Theorem 3.30, with the iterator on the left; the equation $v = \mathrm{sel}(\zeta(v), v, 0)$ is not stable-left.

Remark 3.42 (Scope and versions). The theorem also holds with $\mathcal{P}_0^+$ as the public language: for an orbit matrix O both $D_{x,y}O$ and its translation $D_{x,y}L(O)$ are derivable, the second in case mode from its base by the elimination of the orbit closure, so the two are interderivable. The mixed reading of Theorem 3.8 agrees with the ordinary reading on $\mathcal{P}_0^+$ as well: at an orbit matrix both readings are provable, the progressive one by the step of (ii) and the conditional one from it by induction, so their agreement does not depend on a provable base. `guarded_orbit` states the calculus with the full native language as proof language, that is U^-, and observes that the derivations constructed lie in $\mathsf{G}^-_{\mathrm{orb}}$; it proves the induction of Theorem 3.21 in the form discussed in Theorem 3.22. The named derivation of $0 = t \vdash t = 0$ recorded by the package has 326 distinct nodes, of which 171 are transports, 122 computation equations and 10 introductions, exactly one of them at an orbit matrix.

3.4.4 Third repair: the mirror guard

This subsection draws on `mirror_guard`. The guard is turned around: an atomic matrix below $D_,$ has the mirrored shape $a = I(y, a;\, u.\, h)$, with one iterator, the seed repeated on the left. The calculus is $\mathsf{M} = \mathsf{K}(\mathcal{P}_m^Q,\, s \not\equiv 0)$ and is strictly guarded: every formula of a derivation is public or a resource. The left side of a mirrored matrix is arbitrary, so the hypothesis of Theorem 3.36 fails, and a zero-left equation $0 = t$ has the mirror gate $0 = \mathrm{sel}(v, 0, t)$, which is again zero-left: it can be padded without a transport at the source 0.

Theorem 3.43 (The mirror guard). (i) *The mirror scheme on all equations is a guard scheme for* M *and* $\mathcal{P}_m$. *Hence* M *admits numerical cases for every public formula, implication between equations with a common left side, and contextual induction for every literal equation, by derivations that substitute into no given derivation.*

(ii) M *has unrestricted equality, admits transport at every source in every template of* $\mathcal{P}_m$, *and derives the same sequents as* $\mathsf{K}(\mathcal{P}_m^Q, \textit{all terms})$.

(iii) *With the mirror scheme and* ε *the identity,* T *translates derivations of* N *into derivations of* M; $A \dashv\vdash_{\mathsf{N}} TA$ *for every native* A; $B \dashv\vdash_{\mathsf{M}} TB$ *for every* $B \in \mathcal{P}_m$; *and for* Γ, B *in* $\mathcal{P}_m$, $\Gamma \vdash_{\mathsf{M}} B$ *if and only if* $\Gamma \vdash_{\mathsf{N}} B$.

(iv) M *on* $\mathcal{P}_m$ *is bi-interpretable and equiconsistent with* iPRA, *with whole domains, equality as equality and identity numerical composites.*

Proof. (i) (G1) and (G4) hold by inspection. (G2): $H_0(s = t)$ is $s = \mathrm{sel}(0, s, t)$, the reverse of an instance of I0 (Theorem 3.7(iii)). For $H_{Sw}(s = t)$ put $q := \mathrm{sel}(Sw, s, t)$; then $q \not\equiv 0$ and $q = t$ by IS, and Theorem 3.7(vi) gives $(s = q) \dashv\vdash (s = t)$. (G3): for $\mu_0(s = a, s = b)$ take $q := \mathrm{sel}(0, a, b)$ with $q = a$, and for μ_{Sw} take $q := \mathrm{sel}(Sw, a, b)$ with $q = b$; Theorem 3.7(vi) gives $(s = q) \dashv\vdash (s = a)$ and $(s = q) \dashv\vdash (s = b)$. Only the right side of an equation is expanded or contracted; the left side s is never touched, and it may be the literal 0 or a variable. The consequences are Theorems 3.19 to 3.21.

(ii) By (i), M is closed under induction for literal equations, so Theorem 3.31 gives unrestricted equality. A mirrored matrix has provable instances at zero (Theorem 3.13), so Theorem 3.33 gives the transport in public templates and the equality of the two consequence relations.

(iii) and (iv). (G2) and (G3) hold by (i); (T1) and (T3) are trivial; (T2) holds by unrestricted equality; (T4) is Theorem 3.13. Apply Theorems 3.24 and 3.25. □

In the induction of Theorem 3.31 for the equation $n = I(n,t;\, u.\, Su)$, the guard $J(n)$ of Theorem 3.21 is $n = \mathrm{sel}\big(n, n, I(n,t;\, u.\, Su)\big)$: a mirrored matrix whose left side is the induction variable. The zero-seeded guard has no such matrix, which is the content of Theorem 3.36; the orbit matrix of the second repair supplies one by hand. The order of the recovery is: equality at admissible sources; expansion on the right; padding and cases; implication with a common left side; induction; equality at the source 0; transport in public templates; the translation. The induction is recovered before the source restriction is lifted, and it uses the implication $J(y) \Rightarrow E(y)$, not $J(y) \Rightarrow E(Sy)$ (Theorem 3.22).

Remark 3.44 (Scope). The mirrored shape shows one iterator where the zero-seeded shape shows two, and it repeats the seed; no statement about the length of expanded terms is made. `mirror_guard` states the calculus with clause contexts and proves its theorem against the calculus of the reports on the language of equations and $D_{,}$ (Theorems 3.4 and 3.28).

3.4.5 Fourth repair: protected atoms

This subsection draws on `protected_core`. The zero-left equations are removed from the public language. A public atom is a protected equation $p = q$, both sides iterator-headed, and ordinary equality is expressed by the wrapped equation:

$$s \simeq t \;:=\; \big(W(s) = W(t)\big), \qquad \bot_W \;:=\; \big(W(0) = W(1)\big).$$

The public language is $\mathcal{P}_W$, with zero-seeded atomic matrices, and the calculus is $\mathsf{H} = \mathsf{K}(\mathcal{P}_W^{Q}, \text{iterator-headed terms})$: its seven rules are HYP, I0, IS, TR_I, QE, DE and $\mathrm{DI}_{\mathrm{root}}$. Its proof language contains all equations as auxiliary formulas, which are needed for the computation equation $W(t) = t$ when t is not iterator-headed; they are not public atoms. The source condition is stronger than $s \not\equiv 0$: a variable is not a source. In return the class of sources is closed under substitution.

Lemma 3.45 (Lifting a wrapped equation)**.** *For all terms s, t and every term template $f(v)$, in* H*,*

$$\Gamma \vdash W(s) = W(t) \quad \textit{implies} \quad \Gamma \vdash W(f(s)) = W(f(t)).$$

Proof. Congruence along $W(s) = W(t)$ in the template $W(f(v))$ gives $W(f(W(s))) = W(f(W(t)))$. Congruence along the computation equations $W(s) = s$ and $W(t) = t$ in the same template gives $W(f(W(s))) = W(f(s))$ and $W(f(W(t))) = W(f(t))$. All sources and all six sides are iterator-headed, so Theorem 3.7(ii) reverses the first of the last two equations and composes the three. The argument does not depend on whether s is the literal 0. □

Theorem 3.46 (The protected core)**.** (i) H *admits weakening, substitution and cut.*

(ii) *The zero-seeded scheme on the protected equations is a guard scheme for* H *and* $\mathcal{P}_W$*. Hence* H *admits numerical cases for every public formula, implication between protected equations, and contextual induction for every protected equation.*

(iii) *With $\varepsilon(s = t) := (W(s) = W(t))$, T translates derivations of* N *into derivations of* H*, all of whose transports have iterator-headed sources; $A \dashv\vdash_{\mathsf{N}} TA$ for every native A; $B \dashv\vdash_{\mathsf{H}} TB$ for every $B \in \mathcal{P}_W$; and for Γ, B in $\mathcal{P}_W$, $\Gamma \vdash_{\mathsf{H}} B$ if and only if $\Gamma \vdash_{\mathsf{N}} B$.*

(iv) H *on $\mathcal{P}_W$ is bi-interpretable and equiconsistent with* iPRA*. Both numerical domains are whole; equality is equality, expressed in the public language by $\simeq$; both numerical composites are the identity, with the graph $x \simeq y$; and*

$$\vdash_{\mathsf{H}} \bot_W \iff \vdash_{\mathsf{N}} 0 = 1 \iff \mathsf{iPRA} \vdash \bot.$$

Proof. (i) is Theorem 3.5: an iterator-headed source keeps its head under substitution, and the zero-seeded shape is preserved. The argument would fail for the sources $s \not\equiv 0$.

(ii) (G1) and (G4) hold by inspection, and the gates and selectors of protected equations are protected. (G2): the two sides of G_0U are iterator-headed and equal to 0, so Theorem 3.7(iv) proves G_0U; and for $U \equiv (p = q)$, Theorem 3.7(v) with $a := \mathrm{sel}(Sw, 0, p)$ and $b := \mathrm{sel}(Sw, 0, q)$ gives $U \dashv\vdash G_{Sw}U$, since p and q are sources. (G3) is the same lemma applied to $\mathrm{sel}(0, p, r)$, $\mathrm{sel}(0, q, s)$ and to $\mathrm{sel}(Sw, p, r)$, $\mathrm{sel}(Sw, q, s)$.

(iii) and (iv). (T1): $W(s) = s$ and $W(t) = t$ in N. (T2): for an instance $c = d$ of I0 or IS the source c is iterator-headed and congruence in the template $W(v)$ gives $W(c) = W(d)$; and from $W(s) = W(t)$ and $W(h(s)) = W(r)$, Theorem 3.45 gives $W(h(s)) = W(h(t))$, and symmetry and transitivity at the source $W(h(s))$ give $W(r) = W(h(t))$. (T3): for a protected $p = q$, Theorem 3.7(v) with $a := W(p)$ and $b := W(q)$ gives $(p = q) \dashv\vdash (W(p) = W(q))$. (T4) is Theorem 3.13. Apply Theorems 3.24 and 3.25; the contradiction $\varepsilon(0 = 1)$ is $\bot_W$. □

The compatibility of the two reductions is obtained here without recovering symmetry at the source 0: every equation that must later act as a major premise is represented at stable iterator roots, the zero computations are joined through $W(0)$, and the equalities of the native derivation are moved by Theorem 3.45. Theorem 3.36 does not apply because no public equation is zero-left. No statement is made about an auxiliary hypothesis $0 = t$ in H.

Remark 3.47 (The executable extension and the form in Lean). The compact checkers of `protected_core`, and those of `guarded_source`, `guarded_orbit`, `deterministic_auxiliary` and `mirror_guard`, accept derivations of U^-. Since $\mathsf{H} \subseteq \mathsf{U}^- \subseteq \mathsf{N}$ as sets of derivable sequents, by inclusion and Theorem 3.6, the public consequences of U^- on $\mathcal{P}_W$ are those of H (Theorem 3.46(iii)), and likewise on $\mathcal{P}_m$ by Theorem 3.43(iii). `protected_core` and `mirror_guard` note that for this inclusion the ordinary reading suffices and the mixed reading is not needed.

In Lean the public languages of this section are formalized in a common form (`NativePublic`). The calculus `PGPrf` is the kernel with the root-determined introduction and with transport at iterator-headed sources only. A gating translation `pub` replaces every equation $s = t$ by the closure of its gate, $Q_w G_w(s = t)$, so that every atomic matrix is guarded (`NFm.pub_guarded`) and the two sides of an opened gate are iterators, and `pgprf_pub_iff`: `PGPrf` $(\Gamma.\mathrm{map}\ pub)\ (pub\ A) \leftrightarrow$ `NPrf` $\Gamma\ A$. The theorem is proved for any gating of equations (`Gating`); the mirror gate $s = I(w, s; u.\, t)$ is a second instance (`Gating.mirror`, `NFm.pub_mirrored`), with iterator-headed sources, which is stronger than the exclusion of the literal 0 in Theorem 3.43. The variant `NFm.pubD` uses the paired binder with an unused witness for every closure and reaches the grammar of the packages (`pgprf_pubD_iff`, `NFm.pubD_noQ`, `NFm.pubD_guarded`). The proofs use neither `Classical.choice` nor the bi-interpretation. The differences from the packages are: an equation at the root is gated, not left bare or wrapped as $W(s) = W(t)$; clause contexts and the auxiliary formulas of Section 3.4.2 and Section 3.4.3 are not formalized; and the certificate formats are not modelled.

3.5 Guards on the full native language

This section draws on `diagonal_guard` for the criterion, the diagonal guard and the lemmas on cases and implication, and on `mirror_core` for the mirror selector, its equational decoder, the repair of substitution and the six-rule form. `mirror_core` takes the repair of substitution from `reflexive_base` and the mirrored shape from `mirror_guard`.

In this section the language is the full native language, the contexts are lists of native formulas, and the readings of Q and D, are the ordinary ones. The progress mode of the single introduction is used at the guarded shape and nowhere else, so the atomic matrices that are not guarded keep their introduction by cases, and Theorem 3.10 does not arise. The equality rule is the rule TR of the kernel.

3.5.1 Selectors and the criterion

Definition 3.48 (Selector)**.** A *selector* is a property P of matrices $B(x,y)$ that holds only of literal equations, is decided from the syntax of B, and is invariant under renaming of bound variables and under injective renaming of the variables other than x and y. Put $\eta^P_B(z) := z$ if $P(B)$ and $\eta^P_B(z) := 0$ otherwise. The calculus N_P has the rules HYP, I0, IS, TR, QE and DE of N (Section 1.3) and the one introduction

$$\frac{\Gamma,\ B(t,\eta^P_B(z)) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \mathrm{DI}_P,$$

with z not free in Γ, t and $D_{x,y}B$. It has seven rules. The test P is applied to the matrix before the witness is substituted.

Lemma 3.49 (Structure and equality in N_P)**.** *N_P admits weakening, injective renaming of free variables and cut. Reflexivity, symmetry, transitivity, congruence and transport in an equation template are derivable.*

Proof. The structural statements are proved by induction on derivations as in Theorem 3.5: an injective renaming preserves P and its negation, and weakening and cut do not change a matrix. The equality rules are Theorem 2.4(iii): their derivations use I0 and TR only. □

Consider two conditions on a selector.

(S1) If $P(B)$, then $\Gamma \vdash B(t,0)$ for every Γ and t, by I0, IS and TR.

(S2) Every equation template $E(y)$ has a *lift* $\Lambda_E(y)$: an equation with $P(\Lambda_E)$, as a matrix in y with an unused witness coordinate, such that $\Lambda_E(Sv) \dashv\vdash E(Sv)$ for every term v, by I0, IS and TR.

Lemma 3.50 (Cases from (S1))**.** *Let P satisfy* (S1)*. In N_P, for every native matrix B and every native formula $F(y)$, with z fresh:*

(i) *if $\Gamma, B(t,0) \vdash B(t,Sz)$, then $\Gamma \vdash D_{x,y}B$;*

(ii) *if $\Gamma, F(0) \vdash F(Sz)$, then $\Gamma \vdash Q_y F(y)$;*

(iii) *if $\Gamma \vdash F(0)$ and $\Gamma, F(0) \vdash F(Sz)$, then $\Gamma \vdash F(t)$ for every term t.*

Proof. (i) If $P(B)$ fails, this is DI_P. If $P(B)$ holds, cut the derivation of $B(t,0)$ given by (S1) into the premise, weaken by $B(t,z)$, and apply DI_P in progress mode. (ii) and (iii) By (i) the rule DI is admissible in N_P, and the introduction of Q by cases and numerical cases follow from DI, DE, HYP and QE as in N (Theorem 2.4(i)). □

No induction for an equation is used in the lemma. For equations $U \equiv (a = b)$ and $V \equiv (c = d)$ and a fresh w put

$$(U \Rightarrow_= V) := Q_w\big(\mathrm{sel}(w,a,c) = \mathrm{sel}(w,b,d)\big).$$

This is the native implication $U \Rightarrow_* V$ of Theorem 1.28 at two equations. Its rules are proved for N in Section 1.4; the next lemma derives them in N_P, where QI is not yet available.

Lemma 3.51 (Implication between equations, and cases on a term of the context)**.** *Let P satisfy* (S1)*. In N_P:*

(i) *if $\Gamma, U \vdash V$, then $\Gamma \vdash U \Rightarrow_= V$; and $\Gamma \vdash U \Rightarrow_= V$ and $\Gamma \vdash U$ imply $\Gamma \vdash V$;*

(ii) *for an equation E and a term r, with v not free in Γ, r and E: if $\Gamma, r = 0 \vdash E$ and $\Gamma, r = Sv \vdash E$, then $\Gamma \vdash E$. The context Γ may contain r and its variables.*

Proof. (i) Let $J(w)$ be the matrix of $U \Rightarrow_= V$. By the selector equations $J(0) \dashv\vdash U$ and $J(Sz) \dashv\vdash V$. For the introduction, over $\Gamma, J(0)$ convert $J(0)$ to U, cut into the given derivation, convert V to $J(Sz)$, and apply Theorem 3.50(ii). For the elimination, convert U to $J(0)$, apply QE at 1 and convert $J(1)$ to V.

(ii) For a new variable y put $F(y) := ((r = y) \Rightarrow_= E)$. By (i) the two hypotheses give $\Gamma \vdash F(0)$ and $\Gamma \vdash F(Sv)$. Theorem 3.50(iii) at the target r gives $(r = r) \Rightarrow_= E$, and reflexivity and (i) give E. The variable of the case distinction is the new y, not a variable of Γ. □

Theorem 3.52 (The selector criterion)**.** *Let P be a selector with* (S1) *and* (S2)*. Then the rule* QI *of* N *is admissible in* N_P*, and for every context Γ and every native formula A,*

$$\Gamma \vdash_{\mathsf{N}_P} A \iff \Gamma \vdash_{\mathsf{N}} A.$$

Both transformations are effective and change no term, formula or assumption. Hence N_P *is bi-interpretable and equiconsistent with* iPRA *by the interpretations of* N *(Theorem 1.119).*

Proof. From N_P to N: six rules are shared. An inference DI$_P$ at a matrix without P is DI. At a matrix with P the matrix is a literal equation, and the inference is derived as in Theorem 3.6: QI, QE and DI.

From N to N_P: DI is Theorem 3.50(i), and it remains to derive QI. Let $\Gamma, E(z) \vdash E(Sz)$ with z fresh, let Λ be the lift of E, and put $\Delta := \Gamma, E(0)$.

Decoding the lift. $\Delta, \Lambda(z) \vdash E(z)$, by Theorem 3.51(ii) for the term z. In the branch $z = 0$, transport of the hypothesis $E(0)$ along $0 = z$ gives $E(z)$. In the branch $z = Sw$, transport of $\Lambda(z)$ gives $\Lambda(Sw)$, (S2) gives $E(Sw)$, and transport along $Sw = z$ gives $E(z)$.

Progress of the lift. Cut $E(z)$ into the step premise: $\Delta, \Lambda(z) \vdash E(Sz)$, and by (S2) $\Delta, \Lambda(z) \vdash \Lambda(Sz)$. The variable z is not free in Δ.

Conclusion. Since $P(\Lambda)$, the rule DI$_P$ in progress mode with the witness 0 gives $\Delta \vdash D_{x,y}\Lambda(y)$, and DE and HYP give $\Delta \vdash Q_y\Lambda(y)$. With $\Lambda(0)$ from (S1), QE at Sz' gives $\Lambda(Sz')$, and (S2) gives $E(Sz')$. Thus $\Gamma, E(0) \vdash E(Sz')$, and Theorem 3.50(ii) gives $\Gamma \vdash Q_y E(y)$.

The base $E(0)$ is retained and discharged at the end: the conditional closure is never identified with unconditional progress, and the argument does not need $E(0)$ to be derivable. □

Proposition 3.53 (Substitution in N_P)**.** (i) *If P is preserved and reflected by substitution, then substitution is admissible in N_P by an induction that keeps the rule and the mode of every inference.*

(ii) *If P is preserved by substitution and satisfies* (S1)*, then substitution is admissible in N_P.*

Proof. Induct on the derivation, renaming each eigenvariable away from the values of the substitution. All rules other than DI$_P$ commute with substitution as in N. For DI$_P$ in progress mode, preservation gives the same mode after substitution. For DI$_P$ in case mode at B: in (i) the substituted matrix does not have P, by reflection, and the same inference applies; in (ii) the induction hypothesis gives the substituted premise by cases, and Theorem 3.50(i) derives the conclusion whether or not the substituted matrix has P. □

Remark 3.54 (Versions of the proof)*.* The criterion is the *guard criterion* of `diagonal_guard`. That package derives QI with the implication $\Lambda(y) \Rightarrow_= E(Sy)$, proved by cases on a new variable, and needs the step premise at 0 and at Sv; it substitutes into the derivation of the premise, which is justified for the diagonal selector by Theorem 3.53(i) and, for a selector

without the stability property, by translating native derivations by induction on their height and substituting into the native premise first. `mirror_core` decodes the lift by two unconditional equations (Theorem 3.57) and uses the step premise at its own eigenvariable. The proof above takes the order of `mirror_core` and decodes with Theorem 3.51(ii), which is a lemma of `diagonal_guard`; it needs neither a substitution nor a special property of the lift.

3.5.2 The diagonal guard

Theorem 3.55 (The diagonal guard)**.** *Let P hold of the matrices that are diagonal in y (Theorem 3.12), and write $\mathsf{N}_{\mathrm{diag}}$ for N_P.*

(i) *P is a selector that satisfies* (S1) *and* (S2)*, the latter with the diagonal lift $\Delta_y E$.*

(ii) *$\Gamma \vdash_{\mathsf{N}_{\mathrm{diag}}} A$ if and only if $\Gamma \vdash_{\mathsf{N}} A$, for all native Γ and A.*

(iii) *$\mathsf{N}_{\mathrm{diag}}$ admits weakening, substitution and cut, and the substitution keeps the rule and the mode of every inference.*

Proof. (i) (S1): at $y = 0$ both sides of a diagonal matrix have the form $I(0,0;\, u.\, h')$ and are equal to 0 by I0. (S2): the lift $\Delta_y E$ is diagonal in y, and at $y = Sv$ its two sides are $I(Sv, Sv;\, u.\, s(Sv))$ and $I(Sv, Sv;\, u.\, t(Sv))$, which are equal to $s(Sv)$ and $t(Sv)$ by IS, the updates being constant. No identity between arbitrary iterator bodies is asserted. (ii) is Theorem 3.52. (iii) By Theorem 3.15(iii) the diagonal shape is preserved and reflected, so Theorem 3.53(i) and Theorem 3.49 apply. □

In Lean the calculus is `GPrf` (`NativeDiagonal`), with `gprf_iff_nprf`, `gprf_iff_iprf` and substitution by direct induction (`GPrf.subst`); the equivalence uses neither `Classical.choice` nor the bi-interpretation. The formalization recovers progress by an equational decoder in place of the implication of the package: with $E(y) \equiv (s(y) = t(y))$, numerical cases prove the unconditional equation

$$\mathrm{sel}\big(y,\ s(0),\ I(y, y;\, u.\, s(y))\big) = s(y)$$

and the same for t, and from $E(0)$ and $\Delta_v E$ they give $s(v) = \mathrm{sel}(v, s(0), I(v, v;\, u.\, s(v))) = \mathrm{sel}(v, t(0), I(v, v;\, u.\, t(v))) = t(v)$.

The diagonal guard keeps seven rules, the full language and the readings of both binders. The witness-selected calculus W (Theorems 2.15 and 2.17) also has seven rules on the full language, and its mode can change when a substitution turns a witness into the literal 0 (Theorem 2.18); the calculus with orbit elimination (Section 2.6.1) changes the reading of Q at equations and its elimination; and the guarded core restricts the public language. The diagonal guard does none of these.

3.5.3 The mirror selector and six rules

Theorem 3.56 (The mirror selector)**.** *Let* Mir *hold of the matrices that are mirrored in y, and write $\mathsf{N}_{\mathrm{mir}}$ for $\mathsf{N}_{\mathrm{Mir}}$.*

(i) Mir *is a selector that satisfies* (S1) *and* (S2)*, the latter with the lift $H_y E(y) = \big(s(y) = \mathrm{sel}(y, s(y), r(y))\big)$ of $E(y) \equiv (s(y) = r(y))$.*

(ii) *$\Gamma \vdash_{\mathsf{N}_{\mathrm{mir}}} A$ if and only if $\Gamma \vdash_{\mathsf{N}} A$.*

(iii) *Substitution is admissible in $\mathsf{N}_{\mathrm{mir}}$. It does not keep the mode of every inference.*

Proof. (i) (S1): the instance at zero is $a' = I(0, a'; u.\, h')$, the reverse of an instance of I0. (S2): $\mathrm{sel}(Sv, s, r) = r$ by IS. (ii) is Theorem 3.52. (iii) The shape is preserved (Theorem 3.15(ii)), so Theorem 3.53(ii) applies. It is not reflected: the matrix $p = I(y, q;\, u.\, u)$ is not mirrored for distinct variables p, q and is mirrored after both are replaced by one variable, so an introduction by cases at this matrix becomes, under that substitution, an instance of Theorem 3.50(i), which cuts the provable base and weakens to progress. □

The selector is tested on the matrix before the witness is substituted: the matrix $x = I(y, 0;\, u.\, 0)$ is not mirrored, although its instance at the witness 0 is $0 = I(y, 0;\, u.\, 0)$.

Lemma 3.57 (The equational decoder of the mirror lift)**.** *Let* $E(y) \equiv (s(y) = r(y))$ *and* $\Lambda(y) \equiv \big(s(y) = \mathrm{sel}(y, s(y), r(y))\big)$. *In* $\mathsf{N}_{\mathrm{mir}}$, *for every term* v, $\Gamma \vdash E(0)$ *and* $\Gamma \vdash \Lambda(v)$ *imply* $\Gamma \vdash E(v)$, *by numerical cases for equations and equality alone.*

Proof. The equations

$$\mathrm{sel}(v,\ s(0),\ s(v)) = s(v),$$
$$\mathrm{sel}(v,\ r(0),\ \mathrm{sel}(v, s(v), r(v))) = r(v)$$

are derivable over the empty context by Theorem 3.50(iii): at 0 both left sides are equal to their seed by I0, and at a successor the first uses one application of IS and the second two. They are proved before any assumption about v is retained, and then weakened. From $\Lambda(v)$, congruence in the template $\mathrm{sel}(v, s(0), -)$ gives $\mathrm{sel}(v, s(0), s(v)) = \mathrm{sel}(v, s(0), \mathrm{sel}(v, s(v), r(v)))$, and the base $s(0) = r(0)$ changes the seed on the right. Hence

$$s(v) = \mathrm{sel}\big(v, s(0), s(v)\big) = \mathrm{sel}\big(v, s(0), \mathrm{sel}(v, s(v), r(v))\big) = \mathrm{sel}\big(v, r(0), \mathrm{sel}(v, s(v), r(v))\big) = r(v).$$

□

The lemma replaces the first step of the proof of Theorem 3.52 for the mirror selector, without an implication between equations.

Definition 3.58 (The six-rule mirror calculus)**.** The calculus M_6 is the calculus E_2 of Theorem 4.9 with its introduction WI replaced by the mirror-selected one: its rules are HYP, REFL, the two cases EVAL_0 and EVAL_S of EVAL, QE, DE and

$$\frac{\Gamma,\ B(t, \eta_B^{\mathrm{Mir}}(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}\ \mathrm{DI}_{\mathrm{Mir}}.$$

It has six rule families and seven inference constructors, counted as in Theorem 4.10. There is no computation axiom, no transport rule, no structural rule and no rule of induction.

Theorem 3.59 (Six rules with the mirror selector)**.** *For every context* Γ *and every native formula* A,

$$\Gamma \vdash_{\mathsf{M}_6} A \iff \Gamma \vdash_{\mathsf{N}_{\mathrm{mir}}} A \iff \Gamma \vdash_{\mathsf{N}} A.$$

M_6 *admits weakening, injective renaming, cut and substitution, and is bi-interpretable and equiconsistent with* iPRA *by the interpretations of* N.

Proof. The derivations of symmetry and transitivity, of I0 and IS, and of TR in E_2 (Theorems 4.13, 4.14 and 4.16) use REFL and the two cases of EVAL only, so they are derivations of M_6; the other rules of $\mathsf{N}_{\mathrm{mir}}$ are rules of M_6. Conversely REFL and both cases of EVAL are derived from I0, IS and TR as in the proof of Theorem 4.18, hence in $\mathsf{N}_{\mathrm{mir}}$. The second equivalence is Theorem 3.56. Weakening, injective renaming and cut are proved for M_6 as for E_3 in the proof of Theorem 4.18, with the observation that an injective renaming preserves and reflects Mir. Substitution is proved as in Theorem 3.53(ii): EVAL commutes with substitution (Theorem 4.21), and the derived introduction by cases of Theorem 3.50(i) is available in M_6 by the first equivalence. □

Remark 3.60 (The derivations of `mirror_core`). `mirror_core` derives the equality rules of M_6 directly. It joins $s = t$ and $s = r$ to $r = t$ through $I(0, s;\ u.\, u)$ by two instances of $\textsc{eval}_0$, which is the identity frame of Theorem 4.11, and it derives $\textsc{tr}$ from the two iterators $I(0, s;\ v.\, h(v))$ and $I(1, s;\ v.\, h(v))$ by the composition described in the remark after Theorem 4.16.

In Lean the seven-rule calculus $\mathsf{N}_{\mathrm{mir}}$ is `MiPrf` (`miprf_iff_nprf`) and the six-rule calculus is `M6Prf` (`m6prf_iff_nprf`), in the module `NativeMirror`. The structural rules proved there are weakening, injective renaming (`MiPrf.ren`) and cut, which suffice for weakening below a binder and for the equivalence; the progress of an arbitrary equation uses the lift $s = \mathrm{sel}(y, s, t)$. Relative to the six-rule calculus with the diagonal selector (`SPrf`), the mirror selector shows one iterator where the diagonal shows two, and it loses the reflection of the shape.

3.6 Comparison of the guards

This section compares the guards of `guarded_core`, `protected_core`, `mirror_guard`, `mirror_core` and `diagonal_guard`; Table 3.2 restates results of the preceding sections.

Guard	Atomic matrix	Lift of $s = t$	Under substitution	Used in
zero-seeded	$I(y, 0;\ u.\, h) = I(y, 0;\ u'.\, k)$	$\mathrm{sel}(y, 0, s) = \mathrm{sel}(y, 0, t)$	preserved, not reflected	$\mathcal{P}_0$: G, U^-, $\mathsf{G}^-_{\mathrm{orb}}$
wrapped	zero-seeded; root atoms protected	$W(s) = W(t)$ at the root	preserved; sources closed	$\mathcal{P}_W$: H
mirror	$a = I(y, a;\ u.\, h)$	$s = \mathrm{sel}(y, s, t)$	preserved, not reflected	$\mathcal{P}_m$: M; full language: $\mathsf{N}_{\mathrm{mir}}$, M_6
diagonal	$I(y, y;\ u.\, h) = I(y, y;\ u'.\, k)$	$I(y, y;\ u.\, s) = I(y, y;\ u.\, t)$	preserved and reflected	full language: $\mathsf{N}_{\mathrm{diag}}$

Table 3.2: The four guards.

The base. Every guard has an instance at zero that is derivable by $\textsc{i}0$ and equality at iterator-headed sources (Theorem 3.13). For the mirror guard it is the reverse of one computation equation. For the zero-seeded and the diagonal guard two computations are joined; under a source restriction the join passes through $W(0)$ (Theorem 3.7(iv)).

The left side. The zero-seeded, wrapped and diagonal shapes have an iterator on the left. With the sources $s \not\equiv 0$ and the uncrossed transport this makes a zero-left hypothesis inert (Theorem 3.36): the zero-seeded guard then needs an unguarded matrix in the proof language (Theorems 3.39 and 3.41), or public atoms that are never zero-left (Theorem 3.46). The mirror guard leaves the left side free and is complete in a strictly guarded language (Theorem 3.43).

Sources. The guarded core uses all sources. The mirror guard, and the zero-seeded guard with auxiliary or orbit formulas, use the sources $s \not\equiv 0$, a class that is not closed under substitution; their proofs avoid substitution until equality is unrestricted. The protected core uses iterator-headed sources, a class closed under substitution, and pays with a wrapper on every public atom and with a larger validator of the public grammar (Section 3.7).

Substitution of the shape. In a public language the mode is selected by the root and the shape is a condition on the language, so preservation is enough. As a selector on the full language the shape must not be created by substitution if the mode of an inference is to be stable: the diagonal shape is the only one of the four with this property (Theorem 3.15), and for it substitution is a rule-preserving induction (Theorem 3.55). The mirror selector is closed under substitution with a repair (Theorem 3.56). The zero-seeded shape also satisfies (S1) and (S2), with the gate $G_y E$ as the lift, so by Theorems 3.52 and 3.53 it is a selector on the full language with the same repair; `diagonal_guard` records the matrix $I(y, p;\ u.\, 0) = I(y, 0;\ u.\, 0)$ as the witness that its mode is not stable.

Cost of recognition. The zero-seeded and the diagonal test inspect fixed positions of the two sides. The mirror test compares two term trees, which is linear in their size; `mirror_core` notes that its smaller checker source does not imply faster checking or shorter proofs.

3.7 Checkers and their sizes

This section records the programs of `guarded_core`, `guarded_source`, `guarded_orbit`, `deterministic_auxil` `mirror_guard`, `protected_core`, `mirror_core` and `diagonal_guard`. Sizes are measured by the protocol of Section 1.10.

The compact checkers. `guarded_core`, `guarded_source`, `guarded_orbit`, `deterministic_auxiliary`, `mirror_guard` and `protected_core` modify the arity-coded checkers of the earlier packages; the codec is inherited and is not changed. A formula array of length one is $D_,$, of length two an equation, of length three an administrative Q; a term array of length one is a successor and of length three an iterator; the other lengths denote zero or truth. The root-determined introduction replaces the comparison of the parity of the matrix with a field of the certificate, `len(a) % 2 == u`, by the parity alone, `not len(a) % 2`: among the meaningful roots only the equation has even length, so an equation retains its instance at the eigenvariable and a compound matrix its instance at zero. `guarded_source` writes the test as `len(a) == 2`, and `deterministic_auxiliary` has an optional spelling `~len(a) % 2` that saves three bytes. The byte profile of `guarded_core` also moves the check of the major premise into the expression that supplies the hypothesis, which preserves every successful run. Source selection is the test `m == ([s, u] if s else [t] * 3)`: the canonical literal 0 is the integer 0, which is false, and selects the Q major, and every other source selects the equation. Computation packets that cannot have a literal 0 as initializer or as predecessor are obtained from the *certified zero* $c_0 := I(2,1;\, u.\,0)$, with $c_0 = 0$, by a transport at the iterator-headed source c_0, so that no transport at the literal 0 is reintroduced. The byte and AST profiles use lexical scopes and the token profiles a global supply of fresh names; their certificates are not interchangeable.

The contract of the compact checkers. A compact checker accepts derivations of U (`guarded_core`) or of U^{-} (the other five) in the administrative syntax. Each package states that, for a closed public formula A in canonical encoding and in the ideal unbounded execution, an accepting certificate exists if and only if A is derivable in its calculus. The soundness half rests on the correctness of the inherited codec (opening, freshness and the readings of the surplus arities), which these packages cite and do not reprove, together with Theorems 3.27, 3.39, 3.41, 3.43 and 3.46 and Theorem 3.47; the completeness half encodes a derivation with fresh integer names. A compact checker does not test that the goal is public, and it accepts some goals that are not. Each package therefore supplies a *strict* checker, which validates the closed public goal and then runs the same kernel, and counts it in full. The strict checker of `mirror_guard` compares the two occurrences of the seed after raising the left side across the binder of the iterator; a comparison of the raw arrays would reject matrices with bound variables. One inherited negative control that changes only the retired mode field is accepted by the new checkers; an old certificate that introduces an unguarded equation by cases is rejected by them, which is Theorem 3.10 and the reason why the raw certificate languages of the old and the new checkers are not claimed to coincide.

The typed checkers. `mirror_core` and `diagonal_guard` measure checkers with tagged terms, formulas and proof trees in de Bruijn form, which validate every term, formula, scope and proof node of the full native language. The diagonal checker tests, after opening the witness, whether both sides begin with an iterator whose counter and seed are the bound

index 0; by Theorem 3.15(iii) the test after opening is equivalent to the test on the matrix. The mirror checker tests the matrix before the witness is substituted. The two packages differ from their controls in the guard of the introduction only. The translator of `diagonal_guard` implements both directions of Theorem 3.55(ii); three translated certificates grow from 485, 885 and 1,965 bytes to 11,506, 14,218 and 20,283 bytes. `mirror_core` supplies the translation from M_6 to the kernel, the derived introduction by cases and the repaired substitution as programs; the translation of QI into M_6 is given by the written proof only.

Recorded runs. The packages record the following executions of their programs on positive and negative certificates, in ordinary and optimized Python, all with the expected outcome: 800 (`guarded_core`), 150 (`guarded_source`), 224 (`guarded_orbit`), 242 (`deterministic_auxiliary`), 480 (`mirror_guard`), 608 (`protected_core`); for `mirror_core`, 92 command-line checks of 23 fixtures, 36 replays in the native checker and 72 replays of numerical substitutions; for `diagonal_guard`, 192 processes on 32 fixtures, 93 validations of translated certificates and 240 finite checks of the preservation and reflection of the guard. The runs test the programs and the certificates; the theorems of this chapter are the written proofs.

Table 3.3: Programs and sizes as the packages state them. The last column is the matched control with its tokens, AST nodes and normalized bytes. The smallest values of one package are attained by different programs.

Package	Program	Tokens	AST	Bytes	Matched control
`guarded_core`	direct	265	308	528	byte profile, 266/310/529
	byte profile	266	307	527	byte profile, 266/310/529
	token profile	260	290	575	token profile, 261/292/576
	AST profile	263	286	607	AST profile, 263/289/613
	readable	292	336	1275	none
	strict	491	568	1208	old core with goal parser, 4
`guarded_source`	token profile	257	285	568	`source_dispatch`, 259/289
	AST profile	261	279	603	`source_dispatch`, 263/283
	byte profile	264	301	522	`source_dispatch`, 266/305
	strict	490	561	1202	`guarded_core` strict, 491/5
`guarded_orbit`	token profile	258	287	571	`source_dispatch`, 259/289
	AST profile	262	281	606	`source_dispatch`, 263/283
	byte profile	265	303	525	`source_dispatch`, 266/305
	strict	491	563	1205	guarded 491/568/1208; scal
`deterministic_auxiliary`	direct	265	303	525	`source_dispatch`, 266/305
	byte profile	265	303	522	`source_dispatch`, 266/305
	AST profile	262	281	606	`source_dispatch`, 263/283
	token profile	258	287	571	`source_dispatch`, 259/289
	strict	491	563	1205	guarded 491/568/1208; nat
`mirror_guard`	token profile	258	287	571	`source_dispatch`, 259/289
	AST profile	262	281	606	`source_dispatch`, 263/283
	byte profile	265	303	525	`source_dispatch`, 266/305
	readable	279	319	1272	none
	strict	578	661	1503	strict derivative of the sour
`protected_core`	token profile	258	287	571	`source_dispatch`, 259/289

Package	Program	Tokens	AST	Bytes	Matched control
	AST profile	262	281	606	`source_dispatch`, 263/283
	byte profile	265	303	525	`source_dispatch`, 266/305
	readable	284	326	1301	none
	strict	505	582	1251	`guarded_core` strict, 491/5
`mirror_core`	mirror selector, six rules	1012	1082	3109	diagonal selector, six rules,
`diagonal_guard`	diagonal, seven rules	1069	1124	3244	native, eight rules, 1101/11 witness-selected, seven rule

The gains recorded in Table 3.3 are small. The compact profiles of `guarded_orbit`, `deterministic_auxiliary`, `mirror_guard` and `protected_core` have the same three sizes, since they share the two dispatch tests and differ in the calculus that justifies them and in the strict validator. The strict checkers do not improve on the unrestricted control of 1109 bytes; those of `mirror_guard` and `protected_core` are larger than their matched controls. The gzip size of the byte profile of `guarded_orbit`, `deterministic_auxiliary` (direct), `mirror_guard` and `protected_core` is 331 bytes against 330 for the control. The diagonal checker is 39 bytes larger than the witness-selected control; its advantage is the stability of the mode under substitution. The typed checkers are not smaller than the compact ones, which have a different input contract.

3.8 Further questions

This section collects the open questions and the unproved or negative statements of `guarded_core`, `guarded_source`, `guarded_orbit`, `deterministic_auxiliary`, `mirror_guard`, `protected_core`, `mirror_core` and `diagonal_guard`.

Question 3.61 (Minimal proof language)**.** For the zero-seeded public language $\mathcal{P}_0$ with the sources $s \not\equiv 0$, Theorem 3.37 shows that some unguarded matrix is needed and Theorem 3.41 shows that the orbit matrices suffice. Which substitution-stable families of auxiliary equation invariants suffice for the repair of equality and for the arithmetic of the bridge? Is there a comparably short checker that enforces the smaller intermediate grammar, with a simpler soundness proof than that of the full arity codec? (`guarded_source`, `guarded_orbit`, `deterministic_auxiliary`.)

Question 3.62 (The smallest stable guard)**.** Is there a syntactic class of equations that is cheaper to recognize than the diagonal shape and has provable instances at zero, a lift of every equation, and preservation and reflection under substitution? What is the smallest family of base recognizers that supports an equational decoder for every equation? The mirror and the diagonal shape are two instances; neither gives a lower bound. (`diagonal_guard`, `mirror_core`.)

Question 3.63 (A public grammar with a small validator)**.** Is there a substitution-stable public grammar that makes the root-determined introduction complete, keeps both composites, and has a strict checker no larger than the unrestricted control of 1109 bytes under the same complete-input contract? (`guarded_orbit`, `deterministic_auxiliary`; `guarded_source` asks whether the validator can share more of the kernel.)

Question 3.64 (Zero-source symmetry)**.** Can the derivation of $t = 0$ from $0 = t$ with the uncrossed rule (Theorem 3.31) be made shorter than the one through the singleton test and the filter? A shorter uniform derivation would shorten certificates without changing a checker. (`guarded_source`, `deterministic_auxiliary`.)

Question 3.65 (Sizes of translated derivations)**.** Give explicit bounds for the expansion caused by the join at zero, the padding, the implication between equations and the lift of an equation, and compare proof trees with a checked format with sharing. No polynomial bound on the size of a translated certificate is claimed. (`guarded_orbit`, `diagonal_guard`.)

The following items are stated by the packages without proof, or record what was not achieved.

- *No minimality.* No package claims a least number of rules, a least checker, or a lower bound for a class of checkers; the observed sizes are upper bounds, and the three smallest coordinates of a package belong to different programs. Whether one checker improves all three coordinates of its controls at once is open (`guarded_source`), as is a lower bound for a precisely specified class of complete checkers (`mirror_core`).

- *The codec.* The theorems on the compact checkers are stated relative to the inherited arity codec, whose soundness is cited and not reproved in these packages, and neither the codec nor a certificate format is formalized in a proof assistant.

- *Administrative formulas.* Completeness is not claimed for goals outside the public language, and the full language of U or U^{-} is not claimed to be bi-interpretable with iPRA; by Theorem 3.10 the two sound readings of an unguarded atomic binder differ. No impossibility is claimed for other interpretations of the administrative language (`guarded_core`). Whether the independent choice between cases and progress can be removed on every administrative formula with its ordinary reading and without a validator is asked by `guarded_orbit`; the selectors of Section 3.5 remove it on the full native language at the cost of a test of the shape.

- *Formalization.* In Lean the dependency theorem (`ZKernel.trF`), the diagonal guard (`GPrf`), the mirror selector (`MiPrf`, `M6Prf`) and the public languages in the gated form (`PGPrf`) are proved. Not formalized: the calculi with clause contexts; the auxiliary formulas of Section 3.4.2 and the orbit bootstrap of Section 3.4.3, including numerical cases at stable-left equations; the mirror calculus M with the sources $s \not\equiv 0$ in the form of Theorem 3.43; and the wrapped root atoms of H. `deterministic_auxiliary` asks whether the auxiliary progress of an equation can be a separate proof judgment with a smaller derivation datatype, the target being the equality of the public consequence relations and not a round trip for auxiliary formulas; `mirror_core` proposes its two unconditional selector equations (Theorem 3.57) as the route with the fewest dependencies. `mirror_guard` observes that a formalization of M cannot start by restricting the transport rule of the kernel and using its substitution theorem (Theorem 3.35).

- *Programs that are not complete translators.* The proof generators of `mirror_guard` and `protected_core` are collections of worked constructions; the normalizer of `protected_core` rejects an unsupported equation leaf with the source 0 instead of translating it; `mirror_core` has no program for the translation of QI.

- *Selectors in the compact interface.* Whether the mirror or the diagonal selector reduces the size of a compact checker with a complete validator of the goal is not answered (`mirror_core`, `diagonal_guard`); nor is the combination of the diagonal introduction with the crossed one-term transport in a matched typed checker (`diagonal_guard`), or a constant-time mirror test by sharing of syntax without a larger trusted reader (`mirror_core`).

- *Other bases.* `guarded_orbit` asks whether a different numerical basis can remove a constructor or a binding operation without a richer arithmetic axiom or a more complex

parser, and notes that the earlier register, tied-seed and zero-start results constrain the answer.

Chapter 4

Computation, equality and normal forms

Three of the eight rules of the native scalar core N (Section 1.3) concern numerical terms: the zero computation equation, the successor computation equation and the transport rule. This chapter collects the calculi in which these three rules are merged, and the calculi in which one or both computation equations leave the rule inventory and become part of the syntax of terms. Two ideas organize the material.

The first idea keeps the terms, the formulas and ordinary substitution unchanged. Reflexivity and one rule EVAL, which composes an equation with one computation step, replace the three rules (Section 4.2).

The second idea restricts the terms to the normal forms of a computation and lets every substitution of a rule be followed by normalization. A computation that is normalized needs no rule. Section 4.3 proves once, for an arbitrary normalizer that satisfies five conditions, that derivations transfer between N and the kernel on normal forms. The zero computation alone is normalized in Section 4.4; on the resulting zero-normal terms the remaining computation and the equality reasoning are presented by the rules of Sections 4.5 to 4.8. A weaker zero normalization, which contracts only zero iterators with a variable or 0 as initializer and is not coherent with substitution, is treated in Section 4.9 with rules that keep their raw instances, and the normalization of the heads of the endpoints of equations, with substitution for a variable as the only equality rule, in Section 4.10, where also a single conversion step of bounded depth replaces the normalizer (Section 4.10.3); the same section shows that this equality rule does not suffice on raw terms with the computation equations as axioms. Both computations are normalized in Section 4.11.

Every calculus K of the chapter is given with its complete rule inventory and with a theorem of one of two forms: K derives exactly the sequents of N, or K derives the normal form of a sequent exactly when N derives the sequent. In both cases the bi-interpretation of Theorem 1.119 transfers (Theorem 4.31). Table 4.1 lists the calculi; the rule names are explained in the sections cited.

4.1 Conventions and shared lemmas

This section fixes the notation common to the thirteen packages of the chapter (`computational_equality`, `equality_from_iteration`, `uniform_step`, `seed_evaluation`, `zero_normal`, `reflexive_base`, `initializer_equality`, `full_normalization`, `computation_normal`, `atomic_seed_refinement`, `head_conversion`, `bounded_head`, `bounded_conversion`) and proves once the arguments that several of them repeat.

Calculus	Terms	Rules	Equivalence
E_3	raw	HYP, REFL, EVAL (three cases), QE, DE, WI	Theorem 4.18
E_2	raw	the same, EVAL with two cases	Theorem 4.12
$\mathsf{R}^{\mathrm{raw}}$	raw	HYP, I0, IS, TR, QE, DE, RI	Theorem 4.53
N^0	zero-normal	HYP, IS, TR, QE, DE, DI, QI	Theorem 4.38
Z_6	zero-normal	HYP, IS, TR, QE, DE, WI	Theorem 4.42
R	zero-normal	HYP, IS, TR, QE, DE, RI	Theorem 4.51
S	zero-normal	HYP, IS, SEED, QE, DE, WI	Theorem 4.64
U	zero-normal	HYP, REFL, STEP, QE, DE, WI	Theorem 4.70
$\mathsf{C}_{\mathrm{Step}}$	zero-normal, clauses	USE, REFL, STEP, DE, WI	Theorem 4.73
$\mathsf{C}_{\mathrm{Seed}}$	zero-normal, clauses	USE, REFL, SEEDEVAL, DE, WI	Theorem 4.77
$\mathsf{C}_{\mathrm{App}}$	unit-normal, clauses	USE, REFL, APP, DE, WI	Theorem 4.85
A_6	atomic-seed normal, raw metadata	HYP, IS, TR, QE, DE, WI	Theorem 4.102
VH	head-normal formulas, raw hints	HYP, REFL, VSUB, QE, DE, WI	Theorem 4.114
BH_3	raw	HYP, REFL, VSUB, QE, DE, WI, HC_3	Theorem 4.120
N^{f}	full normal	HYP, REFL, TR, QE, DE, DI, QI	Theorem 4.137
F	full normal	HYP, REFL, TR, QE, DE, WI	Theorem 4.141

Table 4.1: The calculi of the chapter.

4.1.1 Notation

Raw terms are the terms of Section 1.1, with ordinary capture-free substitution; no computation is performed by substitution, so $I(0, a;\, u.\, h)$ is a raw term distinct from a. The relation $s \equiv t$ is literal identity of syntax, up to the renaming of bound variables; in a de Bruijn representation it is equality of finite trees. Every test in a rule of this chapter is a test of literal identity and never a test of provable equality. A formula is a *literal equation* if its outermost constructor is equality; an equation below Q_y or $D_{x,y}$ is not a literal equation. The numeral 1 is $S0$.

A *template* is a term h with a distinguished variable v; it may contain v several times, not at all, or below iterator binders, and it may contain other variables. Its instance at s is written $h(s)$. A *matrix* is a formula B with one or two distinguished variables; $B(r)$ and $B(t, r)$ are its instances. Instances are formed with the substitution of the calculus at hand: ordinary substitution on raw terms and the normalizing substitution of Theorem 4.25 on normal forms.

The rules of N are those of Section 1.3, with their names: HYP, I0, IS, TR (from $s = t$ and $h(s) = r$ infer $r = h(t)$), QE, DE, DI (by cases) and QI (by progress, for literal equations). The first premise of TR is its *major* premise and the second its *minor* premise. Weakening and substitution of terms in derivations (Theorem 1.10), cut (Theorem 1.11), the equality rules (Theorem 1.18) and the Leibniz rule (Theorem 1.22) are admissible in N.

Definition 4.1 (Witness-selected introduction)**.** WI is the rule DI_{w} of Theorem 2.15, with the function $\eta_{B,t}$ defined there, read with the terms and the substitution of the calculus at hand; the test $t \equiv 0$ is applied to the witness as it is supplied in that syntax. An instance with $\eta_{B,t}(z) = z$ is in *progress mode*; every other instance is in *case mode* and is an instance of DI.

On raw terms WI replaces DI and QI by Theorem 2.17, whose proof repairs the one missing instance of DI at the term $I(0, 0;\, u.\, 0)$ of Theorem 2.16. That term is not available on normal

forms. Theorem 4.3 states the argument for an arbitrary syntax and an arbitrary term that is provably 0 and not literally 0; the sections below supply the term.

4.1.2 Equality from reflexivity and crossed transport

Lemma 4.2. *Let* K *be a calculus that has* TR *for every template, with a substitution for which the instance of the identity template at* s *is* s*.*

(a) If $\Gamma \vdash_{\mathsf{K}} \ell = r$ *for some term* ℓ*, then* $\Gamma \vdash_{\mathsf{K}} r = r$*.*

(b) If K *derives* $r = r$ *for every term* r*, then the following rules are admissible in* K*, by derivations that use only reflexivity and* TR*: symmetry; transitivity; congruence, from* $s = t$ *infer* $h(s) = h(t)$*; and transport in a literal equation, from* $s = t$ *and* $a(s) = b(s)$ *infer* $a(t) = b(t)$*.*

Proof. (a) is the derivation of Theorem 1.18(i) with the equation $\ell = r$ in place of the instance of I0: TR with the identity template and $\ell = r$ as both premises concludes $r = r$. (b) The derivations of Theorem 1.18(ii)–(v) use reflexivity and TR only, one application for each of symmetry, transitivity and congruence and two for transport in an equation; they apply to K with the instances of the templates formed by the substitution of K. □

For a calculus on raw terms that contains I0 the lemma is Theorem 2.4(iii). It is stated here for the calculi in which I0 is not a rule and reflexivity has another source.

4.1.3 The single introduction

Lemma 4.3 (The single introduction)**.** *Let* K *be a calculus on native formulas over a term syntax with a substitution for which instances compose (ordinary substitution on raw terms, or a normalizing substitution, by Theorem 4.26). Assume that* K *has* HYP*,* QE *and* DE*; that weakening, injective renaming of free variables and cut on an assumption are admissible in* K*; and that symmetry of equality and transport in literal equations are admissible in* K*.*

(i) If K *has* WI*, then* QI *is admissible in* K*, and so is the introduction of a closure by cases, for every matrix* B*:*

$$\frac{\Gamma,\, B(0) \vdash B(Sz)}{\Gamma \vdash Q_y B(y)} \qquad (z \text{ fresh}).$$

Every instance of DI *is an instance of* WI*, except when* B *is a literal equation and* $t \equiv 0$*.*

(ii) If K *has* WI*,* $E(x, y)$ *is a literal equation and* Z *is a term with* $Z \not\equiv 0$ *and* $\Gamma \vdash_{\mathsf{K}} Z = 0$*, then*

$$\Gamma,\, E(0,0) \vdash_{\mathsf{K}} E(0, Sz) \quad (z \text{ fresh}) \qquad \text{implies} \qquad \Gamma \vdash_{\mathsf{K}} D_{x,y} E(x, y).$$

(iii) If K *has* DI *and* QI*, then* WI *is admissible in* K*.*

Proof. The three parts are the three constructions in the proof of Theorem 2.17, which use of the calculus only the rules and the admissible operations assumed here.

(i) For QI, the construction given there applies: WI for the matrix $E(y)$ with an unused witness coordinate, at the witness 0, is in progress mode and has the given premise, and DE with HYP returns $Q_y E(y)$. For the closure by cases use the same construction with $B(y)$ in place of $E(y)$ and the witness 1: since $1 \not\equiv 0$, WI is in case mode for every B, and its premise is $\Gamma, B(0) \vdash B(Sz)$. In N this rule is Theorem 1.13, with the witness 0; with WI the witness must not be the literal 0. The last assertion is the definition of the two modes.

(ii) This is the repair in the proof of Theorem 2.17, with Z in place of the term c of Theorem 2.16, after z is renamed so that it is fresh for Z: in the context $\Gamma, E(Z, 0)$, transport

along $Z = 0$, cut with the given derivation, and transport along $0 = Z$ give $E(Z, Sz)$, and WI at the witness Z is in case mode. Only transport in literal equations is used, and no term is substituted into the given derivation.

(iii) An instance of WI in case mode is an instance of DI. An instance in progress mode, with the premise $\Gamma, E(0, z) \vdash E(0, Sz)$, is derived as in Theorem 1.16(i), by QI, QE under the assumption $E(0, 0)$, and DI at the witness 0. □

Remark 4.4. The term Z of part (ii) is $I(0, 0; u.\, 0)$ on raw terms (Theorem 2.16), the terms $I(1, 0; u.\, 0)$ or $I(1, 0; u.\, u)$ on zero-normal terms (Theorem 4.40), and a variable b opened by DE on full normal forms (Theorem 4.140). As on raw terms (Theorem 2.18), no calculus with WI is shown closed under the substitution of terms by a direct induction on derivations; on normal forms a substitution can in addition create a redex whose contraction turns a witness into the literal 0. Substitution is transferred from an equivalent calculus with DI and QI. Weakening, injective renaming and cut leave the witness and the root of the matrix unchanged and are proved directly, as in Theorem 2.3.

4.1.4 Congruence below binders and provability of conversion

Transport in N replaces a term in a template by capture-free substitution. It does not replace the body of an iterator by a pointwise equal body, and it does not replace a matrix below Q_y or $D_{x,y}$. Both replacements are derived rules of N: the congruence of the body of an iterator is Theorem 1.56(ii), which rests on an equation induction, and the congruence of the binders is Theorem 1.21. Every comparison of N with a calculus on normal forms depends on them, because a normalizer rewrites below binders.

Definition 4.5 (Computation reduction)**.** The relation $\to$ on raw terms is the closure, under every term context including the bodies of iterators, of the two contractions

$$I(0, a; u.\, h) \to a, \qquad I(Sn, a; u.\, h) \to h[I(n, a; u.\, h)/u].$$

The first is the *zero contraction* and the second the *successor contraction*; $\to^*$ is the reflexive and transitive closure.

Lemma 4.6 (Conversion is provable)**.** *If $t \to^* t'$, then $\vdash_{\mathsf{N}} t = t'$, with all free variables of t as parameters. If the formula A' is obtained from the formula A by replacing each term by a reduct of it, then $\Gamma, A \vdash_{\mathsf{N}} A'$ and $\Gamma, A' \vdash_{\mathsf{N}} A$ for every Γ.*

Proof. For one step $t \to t'$ argue by induction on the position of the contracted redex. At the root the equation is an instance of I0 or IS. Below a successor, in the counter or in the initializer of an iterator, it follows from the induction hypothesis by congruence (Theorem 1.18(iv)) in the template with one hole at that position. In the body of $I(n, a; u.\, h)$, with $h \to h'$, the induction hypothesis is $\vdash_{\mathsf{N}} h = h'$ with u free, in the empty context, and Theorem 1.56(ii) gives $I(n, a; u.\, h) = I(n, a; u.\, h')$. Transitivity extends the statement to $\to^*$.

For formulas argue by induction on A. At a literal equation use the two term equations and the equality rules. At $Q_y B$ and $D_{x,y} B$ the induction hypothesis holds for the matrix with the bound variables free, in both directions, and Theorem 1.21 applies. □

Remark 4.7. The congruence of the body is restated as a lemma in `zero_normal`, `initializer_equality`, `full_normalization` and `computation_normal`, each time by a direct equation induction on $I(z, a; u.\, h) = I(z, a; u.\, h')$ in place of the uniqueness lemma used for Theorem 1.56; its version in T_{eq} is formalized as `Q.iter_body_congr` and `Q.iter_congr`. The pointwise equation of the bodies is derived in a context without u; it is not a hypothesis that may vary with the state of the iteration. The equality reasoning of the calculi below is recovered without

induction, whereas the comparison of each calculus on normal forms with N uses the equation induction inside Theorem 1.56(ii). The packages `uniform_step` and `seed_evaluation` state this dependency explicitly.

4.2 The rule Eval on raw terms

This section draws on `computational_equality`, which introduces the rule EVAL with three cases, and on `equality_from_iteration`, which shows that the first case is redundant and that neither of the other two is. The terms, the formulas and substitution are those of N; nothing is normalized.

4.2.1 The operation Φ and the calculi

Definition 4.8. For raw terms s, t, b the partial operation $\Phi(s, t, b)$ is given by the rows

shape of b	required source s	$\Phi(s, t, b)$
$b \equiv s$	s	t
$b \equiv I(0, a;\, u.\, h)$	$s \equiv a$	t
$b \equiv I(Sc, a;\, u.\, h)$	$s \equiv I(c, a;\, u.\, h)$	$h[t/u]$

and is undefined if no row applies.

The rows are disjoint. In the second row s is a proper subterm of b, and in the third row b has exactly one node more than s, so neither overlaps the first; the second and third rows have different constructors at the root of the counter. Hence Φ is a deterministic partial operation on syntax. The third row requires an equation for the whole preceding iterate $I(c, a;\, u.\, h)$; it does not compare initializers, and it applies unchanged when $c \equiv 0$.

Definition 4.9 (The calculi E_3 and E_2)**.** The calculus E_3 has raw terms, native formulas, ordinary substitution, finite lists of reusable assumptions as contexts, and exactly the six rules HYP, QE, DE, WI (Theorem 4.1) and

$$\frac{}{\Gamma \vdash s = s}\ (\textsc{refl}), \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash b = r}{\Gamma \vdash r = \Phi(s, t, b)}\ (\textsc{eval}),$$

the latter whenever $\Phi(s, t, b)$ is defined. The three rows of Φ give the three cases of EVAL:

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash r = t}\ (\textsc{root})$$

$$\frac{\Gamma \vdash a = t \qquad \Gamma \vdash I(0, a;\, u.\, h) = r}{\Gamma \vdash r = t}\ (\textsc{eval}_0)$$

$$\frac{\Gamma \vdash I(c, a;\, u.\, h) = t \qquad \Gamma \vdash I(Sc, a;\, u.\, h) = r}{\Gamma \vdash r = h[t/u]}\ (\textsc{eval}_S)$$

The calculus E_2 is E_3 without the case ROOT. In DE the eigenvariable is fresh for the context, the conclusion and the parameters of the matrix. No other logical, equality, successor, computation, induction or structural rule is primitive.

The conclusion of EVAL is crossed: its left side is the right side of the second premise. The rule takes no template, no occurrence path, no counter and no body as a parameter; these are read off the two premises, which determine the conclusion or the failure of the rule.

Remark 4.10 (Counting). E_3 has six rules if EVAL counts once and eight if its cases count separately; E_2 has six and seven. The Lean formalization `EPrf` has the two cases of E_2 as constructors (`evalZero`, `evalSucc`) and ROOT as the derived rule `EPrf.evalId`. The calculus U of Section 4.7 has one case and six rules under either convention. The number of arithmetic and equality schemas, including reflexivity, is four for E_3, three for E_2 and two for U.

4.2.2 The identity frame

Lemma 4.11 (Identity frame)**.** *Suppose that a calculus has reflexivity and, for every term s, a term $K(s)$ such that the inference*

$$\frac{s = t \qquad K(s) = r}{r = t}$$

is a rule. Then ROOT *is derivable, by two instances of that rule and one reflexivity. The equation $K(s) = s$ is not needed as an axiom.*

Proof. Let p derive $s = t$ and q derive $s = r$. The rule applied to q and to the reflexivity $K(s) = K(s)$ gives $K(s) = r$. The rule applied to p and to $K(s) = r$ gives $r = t$. Each of p and q is used once. □

Theorem 4.12 (ROOT is redundant)**.** *For every context Γ and native formula A,*

$$\Gamma \vdash_{\mathsf{E}_2} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{E}_3} A.$$

The translations leave every term and formula of the sequent unchanged. A derivation of E_3 with N inference nodes, k of which are instances of ROOT*, is translated to a derivation of E_2 with exactly $N + 2k$ nodes.*

Proof. From right to left, apply Theorem 4.11 with $K(s) := I(0, s;\, u.\, 0)$: its rule is EVAL_0 with the body 0. Explicitly, an instance of ROOT with premises $p\colon s = t$ and $q\colon s = r$ is replaced by

$$\mathrm{RC}(p, q) := \mathrm{Ev}_0(p,\ \mathrm{Ev}_0(q,\ \mathrm{R}_{K(s)})),$$

where R_w is the reflexivity leaf $w = w$ and Ev_0 is EVAL_0 with its premises in the displayed order. The replacement is carried out recursively; every other inference is copied. Each replaced node becomes two instances of EVAL_0 and one reflexivity, and its premises occur once, which gives the count. From left to right the translation is the inclusion. □

The count is of inference nodes. The added reflexivity leaf contains the term $K(s)$, whose size depends on s, so the theorem bounds neither the size of a serialized derivation nor the time of checking it.

4.2.3 Computation, equality and transport recovered

The derivations of this subsection are valid in E_2 over an arbitrary context and use only REFL, EVAL_0 and EVAL_S, apart from the given premises. No binder rule, no numerical case distinction and no induction is used. Write $\mathrm{Ev}(p, q)$ for EVAL with first premise p and second premise q.

Lemma 4.13 (Equality rules)**.** *Symmetry and transitivity are derivable in E_2: for $p\colon s = t$ put $\mathrm{Sym}(p) := \mathrm{RC}(\mathrm{R}_s, p)$, which derives $t = s$; for $p\colon s = t$ and $q\colon t = v$ put $\mathrm{Trans}(p, q) := \mathrm{RC}(q, \mathrm{Sym}(p))$, which derives $s = v$.*

Proof. RC derives $r = t$ from $s = t$ and $s = r$ (Theorem 4.12). With $s = s$ and $s = t$ it derives $t = s$. With $t = v$ and $t = s$ it derives $s = v$. □

Lemma 4.14 (Both computation equations)**.** *For all raw c, a, h the equations*

$$I(0, a;\, u.\, h) = a, \qquad I(Sc, a;\, u.\, h) = h[I(c, a;\, u.\, h)/u]$$

have derivations in E_2 with three nodes.

Proof. Put $A := I(0, a;\, u.\, h)$, $C := I(c, a;\, u.\, h)$ and $B := I(Sc, a;\, u.\, h)$. Then $\mathrm{Ev}(\mathrm{R}_a, \mathrm{R}_A)$ is an instance of EVAL$_0$ and concludes $A = a$, and $\mathrm{Ev}(\mathrm{R}_C, \mathrm{R}_B)$ is an instance of EVAL$_S$ and concludes $B = h[C/u]$. The second derivation is the same when $c \equiv 0$. □

Theorem 4.15 (Congruence in every template)**.** *For every template $h(u)$ the rule*

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash h[s/u] = h[t/u]}$$

is admissible in E_2*. The derivation adds eight inference nodes to the derivation of $s = t$, which occurs once.*

Proof. Let p derive $s = t$ and put $A_s := I(0, s;\, u.\, h)$ and $B_s := I(1, s;\, u.\, h)$. Four instances of EVAL give

$$\begin{aligned} z_s &:= \mathrm{Ev}(\mathrm{R}_s, \mathrm{R}_{A_s}) && : A_s = s, \\ c_s &:= \mathrm{Ev}(z_s, \mathrm{R}_{B_s}) && : B_s = h[s/u], \\ z_{s,t} &:= \mathrm{Ev}(p, \mathrm{R}_{A_s}) && : A_s = t, \\ \mathrm{Cong}_h(p) &:= \mathrm{Ev}(z_{s,t}, c_s) && : h[s/u] = h[t/u]. \end{aligned}$$

The first and third lines are instances of EVAL$_0$. In the second and fourth lines the second premise has the left side B_s, whose preceding iterate is literally A_s, the left side of the first premise; these are instances of EVAL$_S$. The substitutions into h are the ordinary substitutions performed by EVAL itself, so no congruence is assumed inside the derivation. The tree has four reflexivity leaves, of which R_{A_s} occurs twice, and four instances of EVAL. □

Corollary 4.16 (Transport)**.** *The rule* TR*, from $s = t$ and $h[s/u] = r$ infer $r = h[t/u]$, is admissible in* E_2 *for all templates and all terms s, including the literal* 0*. In* E_3 *the derivation adds nine inference nodes to the derivations of the premises; in* E_2 *it adds eleven. Each premise occurs once.*

Proof. Apply ROOT to $\mathrm{Cong}_h(p)$ and the second premise. In E_2 the instance of ROOT is the three nodes of RC. □

Remark 4.17. `equality_from_iteration` derives transport in E_2 by a different composition: from $C_0 := I(0, s;\, u.\, h)$ and $C_1 := I(1, s;\, u.\, h)$ it forms $C_0 = t$ and $C_1 = r$ by Theorems 4.13 and 4.14 and concludes with one instance of EVAL$_S$. The template h occurs in these derivations as the body of an iterator; it is not a parameter of a primitive rule. The node counts say nothing about the size of the term labels, which contain copies of h, s and t.

4.2.4 Equivalence with the core

Theorem 4.18 (Equivalence)**.** *For every finite context Γ of native formulas and every native formula A,*

$$\Gamma \vdash_{\mathsf{E}_3} A \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathsf{E}_2} A \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathsf{N}} A.$$

The translations are effective on finite derivations and leave every term, formula and assumption of the sequent unchanged.

Proof. The first equivalence is Theorem 4.12. For the second it suffices, by Theorem 2.17, to compare E_2 and E_3 with the calculus W of Theorem 2.15, which has the rules HYP, QE, DE and WI in common with them.

From W to E_2: I0 and IS are derived by Theorem 4.14 and TR by Theorem 4.16; every other inference is copied, and each instance of WI keeps its witness and its mode.

From E_3 to W: reflexivity and the equality rules are derivable in W (Theorem 2.4(iii)). For an instance of EVAL with premises $s = t$ and $b = r$ distinguish the rows of Φ. If $b \equiv s$, TR with the identity template gives $r = t$. If $b \equiv I(0, s;\, u.\, h)$, then I0 gives $b = s$, and the equality rules give $r = t$ from $b = s$, $s = t$ and $b = r$. If $b \equiv I(Sc, a;\, u.\, h)$ and $s \equiv I(c, a;\, u.\, h)$, then IS gives $b = h[s/u]$; with $b = r$ this gives $h[s/u] = r$, and TR with the template h and the major premise $s = t$ gives $r = h[t/u]$. Neither translation uses a structural rule or a substitution of terms in derivations. □

Remark 4.19. This is the proof of `computational_equality`, which states the theorem for E_3 and cites `wprf_iff_nprf` for the step from W to N; `equality_from_iteration` adds E_2. The Lean statements are `eprf_iff_nprf` and `eprf_iff_iprf`. Weakening, injective renaming and cut are admissible in E_3 and E_2 also by the direct induction of Theorem 2.3: the operation Φ is a match of literal shapes, which an injective renaming preserves and a cut does not touch.

Corollary 4.20 (Linear bound on inference nodes)**.** *A derivation with N inference nodes in W is translated to a derivation of E_3 with at most $9N$ nodes.*

Proof. Each leaf I0 or IS becomes three nodes (Theorem 4.14). Each instance of TR becomes the translations of its premises, each once, and nine further nodes (Theorem 4.16). Every other node is copied. Induction on the derivation gives the bound. □

The bound holds for this translation. It is not a bound on the size of formulas or of serialized derivations, and it is not a statement of optimality.

4.2.5 Substitution

Proposition 4.21 (EVAL is stable under substitution)**.** *If $\Phi(s, t, b)$ is defined, then for every capture-free substitution σ the term $\Phi(s[\sigma], t[\sigma], b[\sigma])$ is defined by the same row and*

$$\Phi(s[\sigma], t[\sigma], b[\sigma]) \equiv \Phi(s, t, b)[\sigma].$$

Proof. The first row is immediate. In the second row substitution keeps the outer iterator with the literal counter 0, and its initializer is the substituted source. In the third row $b[\sigma] \equiv I(S(c[\sigma]), a[\sigma];\, u.\, h[\sigma])$ and $s[\sigma] \equiv I(c[\sigma], a[\sigma];\, u.\, h[\sigma])$, where σ acts on the parameters of h and leaves u bound; the composition law of ordinary substitution identifies $(h[t/u])[\sigma]$ with $h[\sigma][t[\sigma]/u]$. No computation takes place: if $c[\sigma]$ is the literal 0, the preceding iterate is still the raw term $I(0, a[\sigma];\, u.\, h[\sigma])$. The size comparison after Theorem 4.8 excludes a collapse into the first row. □

The proposition concerns REFL and EVAL. The calculus E_3 keeps WI, so Theorem 4.4 applies: substitution of terms in derivations of E_3, the Leibniz rule, numerical cases and equation induction are admissible by Theorem 4.18 and Theorem 2.2, and EVAL adds no repair of its own.

4.2.6 Irredundancy

Proposition 4.22 (Neither remaining case can be deleted)**.** *The calculus E_2 without EVAL_0 does not derive $I(0, 0;\, u.\, 0) = 0$. The calculus E_2 without EVAL_S does not derive $I(1, 0;\, u.\, Su) = S(I(0, 0;\, u.\, Su))$.*

Proof. Interpret the formulas in the standard natural numbers, with the standard 0 and S, equality as equality and the readings of Section 1.2 for Q_y and $D_{x,y}$. Under any interpretation of the iterator as a function of its counter, its initializer and its parameters, HYP, REFL, QE

and DE are sound; WI is sound in progress mode by induction on the standard numbers and in case mode by the distinction between 0 and successors.

Without EVAL_0: interpret $I(n, a;\, u.\, h)$ as the n-fold iterate of h applied to $a + 1$. Then $I(Sc, a;\, u.\, h)$ is h applied to $I(c, a;\, u.\, h)$, so EVAL_S is sound, while $I(0, 0;\, u.\, 0)$ has the value 1.

Without EVAL_S: interpret $I(n, a;\, u.\, h)$ as a. EVAL_0 is sound, and the displayed successor equation reads $0 = 1$. □

Proposition 4.23 (Reflexivity cannot be deleted)**.** *In* E_2 *without* REFL*, no literal equation is derivable from a context that contains no literal equation. In particular no equation is derivable from the empty context.*

Proof. By induction on a derivation whose context contains no literal equation and whose conclusion is one. HYP cannot be the last rule. Either case of EVAL has a literal equation as a premise in the same context. An instance of QE whose conclusion $B(v)$ is a literal equation has a matrix that is a literal equation, so its second premise $B(0)$ is a literal equation in the same context. DE adds an assumption with root Q_y, so the context of its second premise still contains no literal equation and its conclusion is the same equation. WI concludes a formula with root $D_{x,y}$. Each case contradicts the induction hypothesis. □

The two propositions concern the deletion of these rules from this inventory. They exclude neither another term language nor another rule basis.

4.3 Normalizers and the transfer of derivations

This section states once the comparison argument that `zero_normal`, `initializer_equality`, `full_normalization` and `computation_normal` each carry out for their own normalizer, and on which `equality_from_iteration`, `uniform_step`, `seed_evaluation` and `reflexive_base` rely by citation. The argument uses five properties of the normalizer and nothing else.

4.3.1 Admissible normalizers and normalizing substitution

Definition 4.24 (Admissible normalizer)**.** An *admissible normalizer* is a map ν from raw terms to raw terms such that for all raw terms t, all renamings ρ of variables and all capture-free simultaneous substitutions σ:

(ν1) $\nu(x) \equiv x$, $\nu(0) \equiv 0$ and $\nu(St) \equiv S\nu(t)$;

(ν2) $\nu(t[\rho]) \equiv \nu(t)[\rho]$ and $\mathrm{FV}(\nu(t)) \subseteq \mathrm{FV}(t)$;

(ν3) $\nu(\nu(t)) \equiv \nu(t)$;

(ν4) $\nu(t[\sigma]) \equiv \nu(\nu(t)[\nu \circ \sigma])$;

(ν5) $\vdash_{\mathsf{N}} t = \nu(t)$, with the free variables of t as parameters.

A term t is *ν-normal* if $\nu(t) \equiv t$. The map ν is extended to formulas by $\nu(s = t) := (\nu(s) = \nu(t))$, $\nu(Q_y B) := Q_y \nu(B)$ and $\nu(D_{x,y} B) := D_{x,y} \nu(B)$, and to contexts entry by entry. A formula is ν-normal if $\nu(A) \equiv A$.

Condition (ν4) is the *coherence of normalization with substitution*: the normal form of an instance depends only on the normal forms of the term and of the substituted terms. Condition (ν5) says that ν is the identity inside N; it is a statement about each fixed term and does not say that N or iPRA proves a uniform theorem about ν. By (ν2) the extension to formulas does not depend on the names of bound variables, and ν never changes the root of a formula; in particular $\nu(A)$ is a literal equation exactly when A is.

Definition 4.25 (Normalizing substitution)**.** For a raw term or formula T and a substitution σ,

$$T\langle\sigma\rangle := \nu(T[\sigma]).$$

For a template h and a matrix B of ν-normal syntax, the instances $h(s)$, $B(r)$ and $B(t,r)$ are formed with this substitution.

Lemma 4.26 (Laws of normalizing substitution)**.** *Let ν be an admissible normalizer, T a raw term or formula, and σ, τ substitutions.*

(a) $\nu(\nu(T)[\sigma]) \equiv \nu(T[\sigma])$.

(b) $(T\langle\sigma\rangle)\langle\tau\rangle \equiv T\langle x \mapsto \sigma(x)\langle\tau\rangle\rangle \equiv \nu(T[\sigma][\tau])$.

(c) If T is ν-normal, then $T\langle\rho\rangle \equiv T[\rho]$ for every renaming ρ; in particular the identity substitution leaves T unchanged.

In particular instances compose: for a matrix $B(x,y)$ of ν-normal syntax, the instance at t of the template $B(v,r)$ is $B(t,r)$.

Proof. For terms, (a) follows from (ν4) applied to $\nu(T)$, from (ν3) and from (ν4) applied to T: $\nu(\nu(T)[\sigma]) \equiv \nu(\nu(\nu(T))[\nu \circ \sigma]) \equiv \nu(\nu(T)[\nu \circ \sigma]) \equiv \nu(T[\sigma])$. For (b), by (a), $(T\langle\sigma\rangle)\langle\tau\rangle \equiv \nu(\nu(T[\sigma])[\tau]) \equiv \nu(T[\sigma][\tau])$. Let $\theta(x) := \sigma(x)[\tau]$, so that $T[\sigma][\tau] \equiv T[\theta]$ by the composition law of ordinary substitution. By (ν4), (ν3) and (ν4) again, $\nu(T[\nu \circ \theta]) \equiv \nu(\nu(T)[\nu \circ \nu \circ \theta]) \equiv \nu(\nu(T)[\nu \circ \theta]) \equiv \nu(T[\theta])$, and the left side is $T\langle x \mapsto \sigma(x)\langle\tau\rangle\rangle$. (c) is ($\nu$2) with ($\nu$3). For formulas use induction on the formula: below a binder the substitution is lifted, and the lifting of $\nu \circ \sigma$ is ν composed with the lifting of σ by (ν1) and (ν2). □

Lemma 4.27 (A normalizer is the identity in the core)**.** *Let ν be an admissible normalizer. For every raw formula A and every context Γ, both $\Gamma, A \vdash_{\mathsf{N}} \nu(A)$ and $\Gamma, \nu(A) \vdash_{\mathsf{N}} A$. For every raw term t and substitution σ, $\vdash_{\mathsf{N}} t[\sigma] = t\langle\sigma\rangle$.*

Proof. The term statement is (ν5) for $t[\sigma]$. For formulas the induction on A in the proof of Theorem 4.6 applies, with (ν5) at the two sides of a literal equation in place of the equation of a reduct. □

The inclusion of ν-normal syntax into raw syntax therefore commutes with substitution up to provable equality: the term $t\langle\sigma\rangle$ is by definition the normal form of $t[\sigma]$, the two terms need not be identical, and their equation is a theorem of N.

4.3.2 The kernel on normal forms

Definition 4.28 (The kernel N^ν)**.** Let ν be an admissible normalizer. The calculus N^ν has the ν-normal native formulas as formulas, finite lists of them as contexts, the substitution of Theorem 4.25, and the following rules, in which every term is ν-normal:

- HYP, TR, QE, DE, DI and QI, read with normalizing substitution;
- COMP$^\nu$: $\Gamma \vdash \nu(e)$ for every instance e of I0 or of IS on raw terms.

Lemma 4.29. *Weakening, cut on an assumption and normalizing substitution are admissible in N^ν: if $\Gamma \vdash_{\mathsf{N}^\nu} A$, then $\Gamma\langle\sigma\rangle \vdash_{\mathsf{N}^\nu} A\langle\sigma\rangle$ for every σ.*

Proof. By induction on derivations, with eigenvariables renamed away from σ. No rule of N^ν tests a term or a formula for a literal shape other than the root of the matrix in QI, which substitution preserves. By Theorem 4.26(b) the instance under σ of a premise or conclusion of TR, QE, DE, DI or QI is the corresponding premise or conclusion of the same rule at the substituted parameters. For COMP$^\nu$, $\nu(e)\langle\sigma\rangle \equiv \nu(e[\sigma])$ by Theorem 4.26(a), and $e[\sigma]$ is an instance of the same computation equation. Weakening and cut are standard. □

Theorem 4.30 (Transfer)**.** *Let ν be an admissible normalizer. For every finite context Γ of raw native formulas and every raw native formula A,*

$$\Gamma \vdash_{\mathsf{N}} A \quad \Longleftrightarrow \quad \nu(\Gamma) \vdash_{\mathsf{N}^\nu} \nu(A).$$

Both directions are effective on finite derivations.

Proof. From left to right, by induction on the derivation in N, normalizing every formula. An assumption of Γ becomes an assumption of $\nu(\Gamma)$. A leaf I0 or IS with conclusion e becomes COMP^ν with conclusion $\nu(e)$. For TR with premises $s = t$ and $h[s/v] = r$, the induction hypotheses are $\nu(s) = \nu(t)$ and $\nu(h[s/v]) = \nu(r)$. By $(\nu 4)$, $\nu(h[s/v]) \equiv \nu(h)\langle \nu(s)/v\rangle$, so TR of N^ν with the template $\nu(h)$ gives $\nu(r) = \nu(h)\langle \nu(t)/v\rangle$, which is $\nu(r) = \nu(h[t/v])$ by $(\nu 4)$. In the same way $(\nu 4)$, with $(\nu 1)$ for the terms 0 and Sz, turns the premises and the conclusion of QE, DI and QI into those of the same rule at the matrix $\nu(B)$ and the parameters $\nu(t)$; the matrix of QI remains a literal equation. For DE the local assumption $Q_y B[b/x]$ becomes $Q_y \nu(B)[b/x]$ by $(\nu 2)$. Freshness conditions are preserved, because ν introduces no free variable by $(\nu 2)$.

From right to left, read a derivation of $\Delta \vdash B$ in N^ν as raw syntax and show $\Delta \vdash_{\mathsf{N}} B$ by induction. A leaf COMP^ν with conclusion $\nu(e)$: the equation e is an axiom of N, and Theorem 4.27 gives $\nu(e)$. For TR with premises $s = t$ and $h\langle s/v\rangle = r$: Theorem 4.27 gives $h[s/v] = h\langle s/v\rangle$, hence $h[s/v] = r$; TR of N gives $r = h[t/v]$, and Theorem 4.27 gives $r = h\langle t/v\rangle$. For QE convert the premise $B\langle 0/y\rangle$ to $B[0/y]$ by Theorem 4.27, apply QE of N, and convert $B[t/y]$ to $B\langle t/y\rangle$. For DE the local assumption is unchanged, since $B\langle b/x\rangle \equiv B[b/x]$ by Theorem 4.26(c). For DI work in the context $\Delta, B[t/x, 0/y]$: convert the assumption to $B\langle t/x, 0/y\rangle$, cut with the translated premise, convert the result to $B[t/x, Sz/y]$, and apply DI of N. QI is treated in the same way. The conversions are uniform in the parameters, so no eigenvariable enters a context or a conclusion. This gives $\nu(\Gamma) \vdash_{\mathsf{N}} \nu(A)$ from $\nu(\Gamma) \vdash_{\mathsf{N}^\nu} \nu(A)$; Theorem 4.27 and cut, applied to each assumption and to the conclusion, give $\Gamma \vdash_{\mathsf{N}} A$. □

A derivation with the single introduction must not be normalized rule by rule: a witness that is not literally 0 can have the normal form 0, and the mode of WI then changes. The theorem is therefore stated for the kernel with DI and QI, and each calculus with WI on normal forms is compared with N^ν afterwards.

Corollary 4.31 (Calculi equivalent to the kernel on normal forms)**.** *Let ν be an admissible normalizer and let K be a calculus on ν-normal native formulas that derives exactly the sequents of N^ν. Let J and H be the translations of formulas from N to iPRA and from iPRA to N of Theorem 1.119, and put $H_\nu := \nu \circ H$.*

(a) $\Gamma \vdash_{\mathsf{N}} A$ if and only if $\nu(\Gamma) \vdash_{\mathsf{K}} \nu(A)$, for raw Γ and A. For ν-normal Δ and B, $\Delta \vdash_{\mathsf{K}} B$ if and only if $\Delta \vdash_{\mathsf{N}} B$.

(b) Weakening, cut, normalizing substitution and the Leibniz rule, from $s = t$ and $F(s)$ infer $F(t)$ for every matrix F, are admissible in K.

(c) For ν-normal Δ, B and formulas Θ, φ of iPRA,

$$\begin{aligned} \Delta \vdash_{\mathsf{K}} B \quad &\Longleftrightarrow \quad J\Delta \vdash_{\mathsf{iPRA}} JB, \\ \Theta \vdash_{\mathsf{iPRA}} \varphi \quad &\Longleftrightarrow \quad H_\nu\Theta \vdash_{\mathsf{K}} H_\nu\varphi, \end{aligned}$$

the formulas B and $H_\nu JB$ are interderivable in K, and $\mathsf{iPRA} \vdash \varphi \leftrightarrow JH_\nu\varphi$. The numerical domains are whole, equality is interpreted by equality, and the numerical composites have the graph $x = y$. In particular $\vdash_{\mathsf{K}} 0 = S0$ if and only if $\mathsf{iPRA} \vdash \bot$.

For a calculus on raw terms that derives exactly the sequents of N, the same statements, with ν the identity, are Theorem 2.2.

Proof. (a) The first statement is Theorem 4.30. For the second, apply it to Δ and B, which are their own normal forms. (b) Weakening, cut and substitution are Theorem 4.29. For the Leibniz rule, N derives $F[t/v]$ from $s = t$ and $F[s/v]$ (Theorem 1.22), and normalizing this sequent gives the rule on normal forms by (ν4). (c) The first equivalence is (a) composed with Theorem 1.119. For the second, $\Theta \vdash_{\mathsf{iPRA}} \varphi$ if and only if $H\Theta \vdash_{\mathsf{N}} H\varphi$, if and only if $\nu(H\Theta) \vdash_{\mathsf{K}} \nu(H\varphi)$ by (a). The formulas B and HJB are interderivable in N; normalizing the two sequents gives the interderivability of B and $H_\nu JB$ in K. By Theorem 4.27, $H\varphi$ and $\nu(H\varphi)$ are interderivable in N, hence $\mathsf{iPRA} \vdash JH\varphi \leftrightarrow JH_\nu\varphi$, and $\mathsf{iPRA} \vdash \varphi \leftrightarrow JH\varphi$. The map ν fixes the variables and changes neither a domain formula nor the interpretation of equality: normal forms are representatives of expressions, not a restriction or a quotient of the domain of numbers. The formula $0 = S0$ is ν-normal by (ν1). □

Every calculus on normal forms in the following sections is compared with N^ν on the same ν-normal sequents, and Theorem 4.31 is then not repeated. The normalizer and the normalizing substitution are part of the specification of such a calculus. A checker for it contains them, and the measurements of Section 4.13 count them: the deletion of a rule does not delete the cost of the computation that the rule expressed.

4.4 Zero-normal terms

This section draws on `zero_normal`, which deletes the zero computation rule by normalizing it in the syntax, and on the restatements of the same normalization layer in `initializer_equality`, `equality_from_iteration`, `uniform_step`, `seed_evaluation` and `reflexive_base`.

4.4.1 Zero normalization

Definition 4.32 (Zero normalization)**.** For terms c, a, h the *smart constructor* is

$$\overline{I}(c, a;\, u.\, h) := \begin{cases} a, & c \equiv 0, \\ I(c, a;\, u.\, h), & c \not\equiv 0. \end{cases}$$

The map nf_0 on raw terms is defined by structural recursion:

$$\mathrm{nf}_0(x) := x, \qquad \mathrm{nf}_0(0) := 0, \qquad \mathrm{nf}_0(St) := S\,\mathrm{nf}_0(t),$$
$$\mathrm{nf}_0(I(c, a;\, u.\, h)) := \overline{I}(\mathrm{nf}_0(c), \mathrm{nf}_0(a);\, u.\, \mathrm{nf}_0(h)).$$

A term is *zero-normal* if no iterator in it, at any depth, has the literal 0 as its counter.

The test in the last clause is on the *normalized* counter. For example $\mathrm{nf}_0(I(I(0, 0;\, v.\, 0), a;\, u.\, h)) \equiv \mathrm{nf}_0(a)$, although the outer counter is not literally 0; the equivalence of "$\mathrm{nf}_0(t) \equiv 0$" with "$t \equiv 0$" fails. Only the zero computation is normalized: $I(1, a;\, u.\, h)$ is zero-normal and is not identified with $h[a/u]$. In $I(c, a;\, u.\, h)$ the binder u covers h only, so the initializer returned by a contraction keeps its scope and no variable index is adjusted; for instance $\mathrm{nf}_0(I(n, a;\, u.\, I(0, u;\, v.\, 0))) \equiv I(n, a;\, u.\, u)$ for a variable n.

Lemma 4.33. *(a)* $\mathrm{nf}_0(t)$ *is zero-normal;* $\mathrm{nf}_0(t) \equiv t$ *if and only if* t *is zero-normal;* nf_0 *is idempotent.*

(b) nf_0 *commutes with renaming, and* $\mathrm{FV}(\mathrm{nf}_0(t)) \subseteq \mathrm{FV}(t)$.

(c) $\mathrm{nf}_0(t)$ *has at most as many nodes as* t*, and* nf_0 *is a primitive recursive operation on codes of finite terms.*

(d) $t \to^* \mathrm{nf}_0(t)$ *by zero contractions. Zero contraction, in any position and in any order, terminates, and* $\mathrm{nf}_0(t)$ *is its unique normal form.*

Proof. (a) By structural induction: the children of a rebuilt iterator are zero-normal, and the smart constructor returns either the zero-normal initializer or an iterator whose counter is not 0. On a zero-normal term every clause returns its argument. (b) A renaming sends no variable, successor or iterator to the literal 0, so it commutes with the test of the smart constructor; a contraction deletes subterms and creates no variable. (c) A contraction deletes an iterator node with its counter and body and keeps the normalized initializer; otherwise the normalized children are kept below the same constructor. A structural recursion with these size bounds is primitive recursive. It never performs a successor contraction: $I(Sn, a;\, u.\, h)$ is kept even for a closed numeral n, so no evaluator for arbitrary primitive recursive programs is involved. (d) The bottom-up computation of nf_0 is a sequence of zero contractions. A zero contraction decreases the number of nodes, hence terminates. If $t \to t'$ by a zero contraction, then $\mathrm{nf}_0(t) \equiv \mathrm{nf}_0(t')$, by induction on the position: at the root both sides are the normalized initializer; in a child, the normal form of the child is unchanged by the induction hypothesis, and the result is unchanged even if the child is erased. A term without a zero redex is zero-normal and fixed by nf_0, so every normal form of t is $\mathrm{nf}_0(t)$. □

Lemma 4.34 (Coherence with substitution)**.** *For every raw term* t *and every capture-free simultaneous substitution* σ*,*

$$\mathrm{nf}_0(t[\sigma]) \equiv \mathrm{nf}_0(\mathrm{nf}_0(t)[\mathrm{nf}_0 \circ \sigma]).$$

Proof. By induction on t. For a variable both sides are $\mathrm{nf}_0(\sigma(x))$, by idempotence. The cases 0 and St are immediate. Let $t \equiv I(c, a;\, u.\, h)$ and let σ' be the lifting of σ below u; by Theorem 4.33(b) the lifting of $\mathrm{nf}_0 \circ \sigma$ is $\mathrm{nf}_0 \circ \sigma'$. The left side is $\overline{I}(\mathrm{nf}_0(c[\sigma]), \mathrm{nf}_0(a[\sigma]); u.\, \mathrm{nf}_0(h[\sigma']))$. If $\mathrm{nf}_0(c) \equiv 0$, then $\mathrm{nf}_0(t) \equiv \mathrm{nf}_0(a)$ and the right side is $\mathrm{nf}_0(\mathrm{nf}_0(a)[\mathrm{nf}_0 \circ \sigma]) \equiv \mathrm{nf}_0(a[\sigma])$ by the induction hypothesis for a; the induction hypothesis for c gives $\mathrm{nf}_0(c[\sigma]) \equiv \mathrm{nf}_0(0[\mathrm{nf}_0 \circ \sigma]) \equiv 0$, so the left side is $\mathrm{nf}_0(a[\sigma])$ too. If $\mathrm{nf}_0(c) \not\equiv 0$, then $\mathrm{nf}_0(t) \equiv I(\mathrm{nf}_0(c), \mathrm{nf}_0(a);\, u.\, \mathrm{nf}_0(h))$ and the right side is the smart constructor applied to $\mathrm{nf}_0(\mathrm{nf}_0(c)[\mathrm{nf}_0 \circ \sigma])$, $\mathrm{nf}_0(\mathrm{nf}_0(a)[\mathrm{nf}_0 \circ \sigma])$ and $\mathrm{nf}_0(\mathrm{nf}_0(h)[\mathrm{nf}_0 \circ \sigma'])$, which by the induction hypotheses are the three arguments of the smart constructor on the left side. The substituted counter may or may not be 0; both sides apply the same test to the same term. □

Proposition 4.35. nf_0 *is an admissible normalizer, and the* nf_0*-normal terms are the zero-normal terms.*

Proof. (ν1) holds by definition, (ν2) and (ν3) by Theorem 4.33, (ν4) by Theorem 4.34, and (ν5) by Theorem 4.33(d) and Theorem 4.6. □

In the rest of this section and in Sections 4.5 to 4.8, terms and formulas are zero-normal and $t\langle\sigma\rangle$ is $\mathrm{nf}_0(t[\sigma])$. The substitution is computed by rebuilding the term bottom-up with the smart constructor: in an iterator the substitution acts on the counter and the initializer in the current scope and, lifted, on the body. A substitution into a zero-normal term can create a counter 0, which is then contracted. Neither the normalizer nor any rule tests whether a term *denotes* 0.

4.4.2 The kernel without the zero computation rule

Definition 4.36 (The kernel N^0)**.** The calculus N^0 has zero-normal native formulas, finite lists of them as contexts, normalizing substitution, and the seven rules HYP, TR, QE, DE, DI, QI and

$$\frac{}{\Gamma \vdash I(Sc, a;\, u.\, h) = h\langle \overline{I}(c, a;\, u.\, h)/u\rangle} \qquad \text{(IS)}$$

for all zero-normal c, a, h. The left side is zero-normal because Sc is not the literal 0; at $c \equiv 0$ the right side is $h\langle a/u\rangle$.

Lemma 4.37. N^0 *derives* $r = r$ *for every zero-normal* r*, and* N^0 *derives exactly the sequents of* $\mathsf{N}^{\mathrm{nf}_0}$*.*

Proof. Put $J_r := I(1, r;\, u.\, u)$. IS gives $J_r = r$, because the right side is $u\langle \overline{I}(0, r; u.u)/u\rangle \equiv r$, and Theorem 4.2(a) gives $r = r$. For the second statement compare IS with $\mathrm{COMP}^{\mathrm{nf}_0}$. If e is the instance $I(0, a;\, u.\, h) = a$ of I0, then $\mathrm{nf}_0(e)$ is $\mathrm{nf}_0(a) = \mathrm{nf}_0(a)$, which N^0 derives. If e is an instance of IS at raw c, a, h, then by Theorem 4.34 $\mathrm{nf}_0(e)$ is the instance of IS of N^0 at $\mathrm{nf}_0(c), \mathrm{nf}_0(a), \mathrm{nf}_0(h)$. Conversely an instance of IS of N^0 at zero-normal c, a, h is nf_0 of the raw instance at the same terms. The other rules are common. □

Theorem 4.38 (The zero computation rule is deleted)**.** *For every finite context* Γ *of raw native formulas and every raw native formula* A*,*

$$\Gamma \vdash_{\mathsf{N}} A \quad \Longleftrightarrow \quad \mathrm{nf}_0(\Gamma) \vdash_{\mathsf{N}^0} \mathrm{nf}_0(A).$$

N^0 *is closed under weakening, cut and normalizing substitution, and Theorem 4.31 applies to it.*

Proof. Theorem 4.35, Theorem 4.30 and Theorems 4.29 and 4.37. □

Remark 4.39. The theorem is the faithfulness corollary of `zero_normal` for its seven-rule calculus and the raw/canonical equivalence of `initializer_equality`, whose comparison calculus lists reflexivity as an eighth rule; `equality_from_iteration`, `uniform_step`, `seed_evaluation` and `reflexive_base` cite it. The normalization layer of `zero_normal` is the Lean module `ZeroNormalization`, with `nf_subst`, `hsub_comp`, `nf_correct` and `inclusion_subst_coherent`; the theorem is `nzprf_iff_nprf` for the calculus `NZPrf`, with `NZPrf.normal` and `nzprf_iff_pra`. The Lean calculus admits raw rule parameters and proves as a theorem that it derives only zero-normal formulas from zero-normal assumptions; here this holds by the definition of the syntax. Definitional zero computation is proposed earlier in the package `prevalidated_elimination`, where it does not reduce the checker because the arity coding places the initializer below a dummy binder and a contraction must lower indices, and the package `uniform_zero` asks whether another layout of binders makes it cheaper; `zero_normal` carries it out for the present layout, in which the initializer lies outside the binder.

4.4.3 Six rules with the single introduction

Lemma 4.40 (Zero-normal aliases of zero)**.** *The closed terms* $I(1, 0;\, u.\, 0)$ *and* $I(1, 0;\, u.\, u)$ *are zero-normal and are not literally* 0*, and* IS *of* N^0 *derives* $I(1, 0;\, u.\, 0) = 0$ *and* $I(1, 0;\, u.\, u) = 0$*.*

Proof. The counter $S0$ is not 0, so neither term is contracted. The right sides of the two instances of IS are $0\langle \overline{I}(0, 0; u.0)/u\rangle \equiv 0$ and $u\langle \overline{I}(0, 0; u.u)/u\rangle \equiv 0$. □

The raw alias $I(0, 0;\, u.\, 0)$ of Theorem 4.18 is not available: it normalizes to 0 and selects the progress mode. The packages use $I(1, 0;\, u.\, 0)$ (`zero_normal`, `initializer_equality`, `equality_from_iteration`) or $I(1, 0;\, u.\, u)$ (`uniform_step`, `seed_evaluation`); the only property used is that the term is provably 0 and not literally 0.

Definition 4.41 (The calculus Z_6)**.** Z_6 has the syntax and the substitution of N^0 and the six rules HYP, IS, TR, QE, DE and WI; the test $t \equiv 0$ of WI is applied to the zero-normal witness as supplied. There is no primitive reflexivity, symmetry, substitution, cut, induction, coverage or separation rule.

Theorem 4.42. Z_6 *and* N^0 *derive the same sequents. Hence* $\Gamma \vdash_{\mathsf{N}} A$ *if and only if* $\mathrm{nf}_0(\Gamma) \vdash_{\mathsf{Z}_6} \mathrm{nf}_0(A)$.

Proof. Weakening, injective renaming and cut are admissible in Z_6 by direct induction on derivations: they preserve the root of a matrix and the literal-zero status of a witness, and cut substitutes no term. Reflexivity is derived as in Theorem 4.37, and Theorem 4.2(b) gives symmetry and transport in literal equations; the instances match by Theorem 4.26. By Theorem 4.3(i), QI is admissible in Z_6; by Theorem 4.3(ii) with the alias $Z := I(1, 0; u.\, 0)$ of Theorem 4.40, so is the one instance of DI that is not an instance of WI. Conversely WI is admissible in N^0 by Theorem 4.3(iii). The last statement is Theorem 4.38. □

Corollary 4.43 (Equation induction)**.** *For a literal equation* E *the rule that infers* $\Gamma \vdash E(t)$ *from* $\Gamma \vdash E(0)$ *and* $\Gamma, E(z) \vdash E(Sz)$*, with* z *fresh, is admissible in* Z_6.

Proof. QI, admissible by Theorem 4.42, and QE. □

Remark 4.44. `zero_normal` states Theorem 4.42 with a seven-rule bridge, the calculus N^0, and warns that a derivation of the calculus W of Theorem 2.15 on raw terms must not be normalized with its modes kept. The six-rule calculus is not formalized in Lean in this form: the formalized six-rule calculus on zero-normal terms is `NRPrf` of Section 4.5, with a different mode test, and the calculus U of Section 4.7, with the present test.

4.5 The introduction selected by a reflexive base

This section draws on `reflexive_base`. The single introduction is kept, and its mode is selected by the formula that the case mode would assume, in place of the pair of tests on the root of the matrix and on the witness. No alias of zero is needed.

4.5.1 The calculus

Definition 4.45 (The calculus R)**.** For a zero-normal matrix $B(x, y)$ and a zero-normal term t, let $\mathsf{r}(B, t)$ hold if the instance $B(t, 0)$ is literally $r = r$ for some term r, and put

$$\eta^{\mathsf{r}}_{B,t}(z) := \begin{cases} z, & \mathsf{r}(B, t), \\ 0, & \text{otherwise.} \end{cases}$$

The rule RI is

$$\frac{\Gamma,\ B(t, \eta^{\mathsf{r}}_{B,t}(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)} \qquad (z \text{ fresh for } \Gamma,\ t \text{ and the parameters of } B).$$

The calculus R has the syntax and the substitution of N^0 and the six rules HYP, IS, TR, QE, DE and RI.

The predicate r compares two finite trees. Since the instance $B(t, 0)$ has the root of B, the condition $\mathsf{r}(B, t)$ implies that B is a literal equation. The instance $B(t, 0)$ is normalized, so the predicate depends on the zero normalization of the calculus.

4.5.2 The structural rules, without circularity

The lemmas are proved in the following order: laws of normalizing substitution; weakening and cut; equality; unrestricted introduction by cases; substitution. None of them presupposes the equivalence theorem.

Lemma 4.46. *Weakening, injective renaming of free variables and cut on an assumption are admissible in* R. *Reflexivity, symmetry, transitivity, congruence and transport in literal equations are derivable in* R *from* IS *and* TR.

Proof. The first three operations leave the matrix and the witness of each instance of RI unchanged or apply an injective renaming to them. A reflexive equation stays reflexive, and an equation that is not reflexive does not become reflexive under an injective renaming, so no mode changes. The equality rules are those of Theorems 4.2 and 4.37; their derivations use no introduction rule. □

In particular every base selected by r is derivable in every context.

Lemma 4.47 (Every introduction by cases is admissible)**.** *For every zero-normal matrix* B *and witness* t, *if* $\Gamma, B(t,0) \vdash_{\mathsf{R}} B(t,Sz)$ *with* z *fresh, then* $\Gamma \vdash_{\mathsf{R}} D_{x,y}B$.

Proof. If $\mathsf{r}(B,t)$ fails, this is RI. If it holds, the assumption $B(t,0)$ is literally $r = r$; derive it by reflexivity and remove it by cut. Weaken the resulting derivation of $B(t,Sz)$ by the assumption $B(t,z)$ and apply RI in progress mode. □

Lemma 4.48 (Substitution)**.** *Normalizing substitution is admissible in* R.

Proof. If $B(t,0)$ is literally $r = r$, then for every σ its instance is $r\langle\sigma\rangle = r\langle\sigma\rangle$, and by Theorem 4.26 this is the zero instance of the substituted matrix at the substituted witness. Hence a true guard remains true. Now argue by induction on a derivation. IS, TR, QE and DE commute with normalizing substitution by Theorem 4.26, with fresh eigenvariables. For an instance of RI in progress mode the guard remains true and the induction hypothesis is the premise of RI in progress mode at the substituted data. For an instance in case mode the induction hypothesis is the substituted case premise, and Theorem 4.47 applies whether or not the guard has become true. □

The guard is preserved by substitution and not reflected: if the zero instance of an equation is $p = q$ for distinct variables p, q, the guard fails, and it holds after the substitution of 0 for both. A second way in which a false guard becomes true is the contraction of a counter that the substitution turns into 0. The proof repairs exactly this change of mode.

4.5.3 Progress recovered by two decoders

By Theorem 4.47, DE and the construction of Theorem 4.3(i), the closure by cases is admissible in R:

$$\frac{\Lambda,\ F(0) \vdash F(Sz)}{\Lambda \vdash Q_y F(y)} \qquad (z \text{ fresh for } \Lambda \text{ and the parameters of } F). \tag{4.1}$$

Recall the selector $\mathrm{sel}(v,a,b) = I(v,a;\,u.\,b)$, with u not in b. In zero-normal syntax the instance $\mathrm{sel}(0,a,b)$ is a, and IS gives $\mathrm{sel}(Sv,a,b) = b$.

Lemma 4.49 (Case decoder)**.** *Let* $r(y)$ *be a zero-normal template and* a *a term without* y, *and let* $\mathrm{Dec}_{r,a}(y)$ *be the equation*

$$\mathrm{sel}(y,\ a,\ \mathrm{sel}(y,0,r(y))) = r(y).$$

Then $\vdash_{\mathsf{R}} Q_y\mathrm{Dec}_{r,a}(y)$. *Consequently* $\Lambda \vdash_{\mathsf{R}} a = r(0)$ *implies* $\Lambda \vdash_{\mathsf{R}} \mathrm{Dec}_{r,a}(v)$ *for every term* v.

Proof. The zero instance of $\mathrm{Dec}_{r,a}$ is literally $a = r(0)$. At $y = Sz$ two instances of IS and transitivity give

$$\mathrm{sel}(Sz,a,\mathrm{sel}(Sz,0,r(Sz))) = \mathrm{sel}(Sz,0,r(Sz)) = r(Sz),$$

a derivation that does not use the assumption of the base. Rule (4.1) gives $Q_y\mathrm{Dec}_{r,a}(y)$ in the empty context. Weaken into Λ and apply QE with the given base and the target v. □

The lemma is a numerical case distinction in a fixed context: Λ does not contain the generalized variable, and the conclusion can be instantiated at a variable that occurs in Λ, because QE imposes no freshness condition on its target.

Theorem 4.50 (Equation progress)**.** *Let $E(y)$ be the zero-normal literal equation $s(y) = t(y)$. If $\Gamma, E(z) \vdash_{\mathsf{R}} E(Sz)$ with z fresh for Γ and the parameters of E, then $\Gamma \vdash_{\mathsf{R}} Q_y E(y)$.*

Proof. Let π be the given derivation, write $s_0 := s(0)$ and $t_0 := t(0)$, and keep the base as an assumption: $\Delta := \Gamma, E(0)$, where $E(0)$ is $s_0 = t_0$. The *progressive lift* of E is

$$H(y) := \big(\mathrm{sel}(y, 0, s(y)) = \mathrm{sel}(y, 0, t(y))\big).$$

Its zero instance is literally $0 = 0$. Consider the two decoders

$$\mathrm{Dec}_{s,s_0}(y)\colon\ \ \mathrm{sel}\big(y, s_0, \mathrm{sel}(y, 0, s(y))\big) = s(y), \qquad \mathrm{Dec}_{t,s_0}(y)\colon\ \ \mathrm{sel}\big(y, s_0, \mathrm{sel}(y, 0, t(y))\big) = t(y).$$

The zero instance of the first is $s_0 = s_0$, derivable by reflexivity, and that of the second is $s_0 = t_0$, the assumption $E(0)$. By Theorem 4.49, $\Delta \vdash \mathrm{Dec}_{s,s_0}(z)$ and $\Delta \vdash \mathrm{Dec}_{t,s_0}(z)$ at the fresh variable z.

Add the assumption $H(z)$. Congruence in the template $w \mapsto \mathrm{sel}(z, s_0, w)$ turns it into

$$\mathrm{sel}\big(z, s_0, \mathrm{sel}(z, 0, s(z))\big) = \mathrm{sel}\big(z, s_0, \mathrm{sel}(z, 0, t(z))\big).$$

The symmetric form of $\mathrm{Dec}_{s,s_0}(z)$, this equation and $\mathrm{Dec}_{t,s_0}(z)$ give $s(z) = t(z)$, which is $E(z)$. Weaken π into the context $\Delta, H(z)$ and cut $E(z)$ into its assumption; the result is $\Delta, H(z) \vdash E(Sz)$. Nothing is substituted into π: its variable is still z. IS on each side of $H(Sz)$ converts $E(Sz)$ to $H(Sz)$, so

$$\Delta,\ H(z) \vdash H(Sz).$$

Regard $H(y)$ as a matrix with an unused witness coordinate. Its zero instance at any witness is $0 = 0$, so RI is in progress mode and gives $\Delta \vdash D_{x,y} H(y)$; DE returns $\Delta \vdash Q_y H(y)$, since the witness is unused. Reflexivity derives $H(0)$, QE at Sz gives $H(Sz)$, and IS gives $E(Sz)$. This is a derivation of $\Gamma, E(0) \vdash E(Sz)$, and rule (4.1) gives $\Gamma \vdash Q_y E(y)$. □

The proof uses equality, the closure by cases, weakening, renaming and cut. It uses no implication between formulas, no substitution of terms into the given derivation and not Theorem 4.48.

4.5.4 Equivalence

Theorem 4.51. *For every zero-normal context Γ and formula A,*

$$\Gamma \vdash_{\mathsf{R}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{N}^0} A.$$

Both translations are effective and are the identity on the sequent. Hence $\Gamma \vdash_{\mathsf{N}} A$ if and only if $\mathrm{nf}_0(\Gamma) \vdash_{\mathsf{R}} \mathrm{nf}_0(A)$ for raw Γ, A.

Proof. From R to N^0: five rules are common. An instance of RI in case mode is an instance of DI. In progress mode the matrix is a literal equation; put $E(y) := B(t, y)$. QI gives $Q_y E(y)$; in the context $\Gamma, E(0)$, QE gives $E(Sz)$; DI at the witness t concludes $D_{x,y} B$. From N^0 to R: DI is admissible by Theorem 4.47 and QI by Theorem 4.50. The last statement is Theorem 4.38. □

Remark 4.52. The Lean formalization is `NRPrf`, with `nrprf_iff_nprf`, `NRPrf.normal` and `nrprf_iff_pra`; its completeness proof translates the calculus of the package `diagonal_guard`, with the invariant that a literally reflexive assumption may be dropped. The method of a guarded introduction is that of the package `diagonal_guard`; the guard on a computed zero instance, the proof of Theorem 4.48 for a guard that can change from false to true, and Theorem 4.53 are specific to `reflexive_base`. The rule that keeps both $B(t, 0)$ and, at a literal equation, $B(t, z)$ as assumptions of one introduction is a different, older rule.

4.5.5 The same guard on raw terms, and two variants

Proposition 4.53 (Seven rules without normalization)**.** *Let* $\mathsf{R}^{\mathrm{raw}}$ *be the calculus on raw terms with ordinary substitution and the rules* HYP, I0, IS, TR, QE, DE *and* RI, *where* $\mathsf{r}(B,t)$ *tests the literal syntax of the raw instance* $B(t,0)$*. Then* $\mathsf{R}^{\mathrm{raw}}$ *derives exactly the sequents of* N*.*

Proof. Reflexivity and the equality rules are derived from I0 and TR as in Theorem 1.18. Weakening, injective renaming and cut are direct inductions; Theorem 4.47 and rule (4.1) are obtained as before, and a true guard is preserved by ordinary substitution, so Theorem 4.48 holds with the same proof. The translation from $\mathsf{R}^{\mathrm{raw}}$ to N is that of Theorem 4.51.

Only the lift in the proof of progress changes, because the raw zero instance of H is not reflexive. For $E(y) \equiv (s(y) = t(y))$ use the asymmetric equation

$$K(y) := \big(\mathrm{sel}(y, t(y), s(y)) = \mathrm{sel}(0, t(y), s(y))\big).$$

Its raw zero instance is literally $\mathrm{sel}(0, t(0), s(0)) = \mathrm{sel}(0, t(0), s(0))$, so it satisfies the guard before any computation. For every v, IS computes the left side of $K(Sv)$ to $s(Sv)$ and I0 computes its right side to $t(Sv)$, so $K(Sv)$ and $E(Sv)$ are interderivable. In the context $\Gamma, E(0)$ the two decoder equations are

$$\mathrm{sel}\big(y, s(0), \mathrm{sel}(y, t(y), s(y))\big) = s(y), \qquad \mathrm{sel}\big(y, s(0), \mathrm{sel}(0, t(y), s(y))\big) = t(y).$$

Their zero instances follow from I0 and, respectively, reflexivity and the assumption $s(0) = t(0)$; their successor instances follow from IS for the outer selector and IS, respectively I0, for the inner one. The closure by cases and QE give both decoders at every term. Congruence of $K(z)$ in the template $w \mapsto \mathrm{sel}(z, s(0), w)$ and the decoders give $E(z)$. Cut it into the given derivation of $E(Sz)$, convert $E(Sz)$ to $K(Sz)$, and conclude with RI in progress mode, DE, QE and the closure by cases as in Theorem 4.50. □

The asymmetric lift also serves on zero-normal terms, where its base contracts to $t(0) = t(0)$. The guard is thus independent of the normalization of the zero computation. The Lean formalization of $\mathsf{R}^{\mathrm{raw}}$ is `RRPrf`, with `rrprf_iff_nprf`; its structural rules are weakening, injective renaming and cut.

Proposition 4.54 (Two variants of the guard)**.** *Theorem 4.51 remains true if* $\mathsf{r}(B,t)$ *is replaced by either of the following predicates:*

(a) $B(t,0)$ *is literally* $0 = 0$*;*

(b) $B(x,0)$*, with* x *still a variable, is literally* $r = r$*.*

Proof. The proofs of this section use three properties of the guard: a selected base is derivable; a true guard is preserved by substitution; and the lift H of Theorem 4.50 satisfies the guard. For (a) the base is $0 = 0$, which is preserved by substitution, and the zero instance of H is $0 = 0$. For (b), if $B(x,0)$ is $r = r$ then every $B(t,0)$ is reflexive, the guard is preserved by the substitution of the parameters, and the zero instance of H with an unused coordinate x is $0 = 0$. □

Variant (a) compares with one fixed formula and recognizes fewer matrices. Variant (b) removes the witness from the test and costs a checker one more normalizing substitution, since the instance $B(t,0)$ is needed for the assumption of the case mode in any event. The proof of Theorem 4.53 does not establish variant (a) on raw terms, where the base of the asymmetric lift is reflexive and in general not $0 = 0$.

4.6 Equality as replacement of an initializer

This section draws on `initializer_equality`. Successor computation is kept as a rule, and transport in an arbitrary template is replaced by a rule that changes only the initializer of an iterator. Because an iterator with the counter 0 is its initializer, the same rule replaces at the root.

4.6.1 The rule Seed and the calculus

Definition 4.55 (Seed replacement)**.** For zero-normal terms s, t, b the partial operation $\rho_{s,t}(b)$ is

$$\rho_{s,t}(b) := \begin{cases} t, & b \equiv s, \\ I(n, t;\, u.\, h), & b \equiv I(n, s;\, u.\, h), \\ \text{undefined}, & \text{otherwise.} \end{cases}$$

The two branches are disjoint, since a finite term is not its own proper subterm. The second branch inspects one constructor and its initializer and leaves the counter and the body unchanged; its result is zero-normal because the counter is the same term, which is not 0. No bound variable is substituted or renamed.

Definition 4.56 (The calculus S)**.** S has the syntax and the substitution of N^0 and the six rules HYP, IS (as in Theorem 4.36), QE, DE, WI and

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash b = r}{\Gamma \vdash \rho_{s,t}(b) = r} \quad (\rho_{s,t}(b) \text{ defined}) \qquad (\textsc{seed}).$$

The conclusion of SEED is not crossed. The rule has no parameter: the two premises determine the conclusion or the failure of the rule. Substitution is not removed from the calculus, since IS and the binder rules use it.

Lemma 4.57 (Schematic form)**.** *SEED is exactly the schema*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash \overline{I}(n, s;\, u.\, h) = r}{\Gamma \vdash \overline{I}(n, t;\, u.\, h) = r}$$

for arbitrary zero-normal n and h. Its instance at $n \equiv 0$ is the rule of root replacement*: from $s = t$ and $s = r$ infer $t = r$.*

Proof. At $n \equiv 0$ the schema is the first branch of ρ; the erased body is irrelevant. At any other n the second premise displays the iterator from which the counter and the body are read, and the schema is the second branch. Conversely the first branch is the schema at $n \equiv 0$, $h \equiv 0$, and the second branch is the schema at the counter and body of b. □

Lemma 4.58 (SEED is stable under substitution)**.** *If $\rho_{s,t}(b)$ is defined, then for every substitution σ the term $\rho_{s\langle\sigma\rangle, t\langle\sigma\rangle}(b\langle\sigma\rangle)$ is defined and equals $\rho_{s,t}(b)\langle\sigma\rangle$.*

Proof. In the root branch this is immediate. Let $b \equiv I(n, s;\, u.\, h)$. Then $b\langle\sigma\rangle$ is the smart constructor applied to the substituted counter, initializer and body. If the substituted counter is 0, both $b\langle\sigma\rangle$ and $\rho_{s,t}(b)\langle\sigma\rangle$ lose their iterator node and become $s\langle\sigma\rangle$ and $t\langle\sigma\rangle$, and the rule applies in its root branch. Otherwise both keep the node, with the same counter and body, and the rule applies in its second branch. The rule is stable although normalization can change the branch. □

4.6.2 Equality recovered

The derivations of this subsection hold in S over an arbitrary context and use only IS and SEED, apart from the given premises.

Lemma 4.59 (Root equality)**.** *Reflexivity, symmetry and transitivity are derivable in* S*.*

Proof. IS gives $I(1, a;\, u.\, u) = a$. Root replacement with this equation as both premises gives $a = a$. For $s = t$, root replacement with the second premise $s = s$ gives $t = s$. For $s = t$ and $t = r$, symmetry gives $t = s$, and root replacement with the premises $t = s$ and $t = r$ gives $s = r$. □

Lemma 4.60 (One-step application)**.** *For every zero-normal template $f(u)$ and term a, IS derives $I(1, a;\, u.\, f) = f\langle a/u\rangle$.*

Proof. The right side of IS at the counter 0 is $f\langle \overline{I}(0, a; u.f)/u\rangle \equiv f\langle a/u\rangle$. This is the point at which the definitional zero computation is used. □

Theorem 4.61 (Congruence in every template)**.** *For every zero-normal template $f(u)$ the rule that infers $f\langle s/u\rangle = f\langle t/u\rangle$ from $s = t$ is admissible in S. If the derivation of $s = t$ has P inference nodes, the derivation of the conclusion has $P + 4$ nodes, independently of the depth and the number of occurrences of u in f.*

Proof. Write $f(a)$ for $f\langle a/u\rangle$ and put $A_a := I(1, a;\, u.\, f)$. The derivation is

$$\dfrac{\dfrac{s = t \qquad A_s = f(s)}{A_t = f(s)} \qquad A_t = f(t)}{f(s) = f(t)}$$

The equations $A_s = f(s)$ and $A_t = f(t)$ are instances of IS (Theorem 4.60). The upper inference is SEED in its second branch, which replaces the initializer s of A_s by t. The lower inference is root replacement. The derivation adds two instances of IS and two of SEED; it uses neither reflexivity nor symmetry nor transitivity. The occurrences of u in f, including those below iterator binders, are handled by the substitution of IS. □

Corollary 4.62 (Transport)**.** *The following rules are admissible in S, for all terms and contexts: from $s = t$ and $f(s) = r$ infer $f(t) = r$; the rule TR; and transport in a literal equation, from $s = t$ and $a(s) = b(s)$ infer $a(t) = b(t)$.*

Proof. Theorem 4.61 gives $f(s) = f(t)$, and root replacement with the second premise $f(s) = r$ gives $f(t) = r$. Symmetry gives TR. Two uses of TR, as in Theorem 4.2(b), give transport in an equation. □

Remark 4.63. The node count is for the displayed tree; its labels contain copies of f, s and t, so it bounds neither the size of a serialized derivation nor the time of checking. The package `witness_rule` has a congruence rule restricted to the counter position of a different recursor, with a separate root rule and a construction with $P+5$ nodes; here the lifting and the first root replacement are one inference, and the instance of the same rule at the counter 0 is the root replacement. The two term languages differ, so the comparison is not a uniform shortening of derivations.

4.6.3 Equivalence

Theorem 4.64. *For every zero-normal context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathsf{S}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{N}^0} A.$$

Hence $\Gamma \vdash_{\mathsf{N}} A$ *if and only if* $\mathrm{nf}_0(\Gamma) \vdash_{\mathsf{S}} \mathrm{nf}_0(A)$ *for raw* Γ, A*.*

Proof. Weakening, injective renaming and cut are admissible in S by direct induction: they leave the terms of each rule instance unchanged up to an injective renaming, so neither the mode of WI nor the branch of SEED changes.

From N^0 to S: TR is admissible by Theorem 4.62, which with Theorem 4.59 also supplies the symmetry and the transport in literal equations required by Theorem 4.3. By Theorem 4.3(i), QI is admissible, and by Theorem 4.3(ii) with the alias $Z := I(1, 0;\, u.\, 0)$, for which IS derives $Z = 0$, so is the instance of DI that WI lacks. The other rules are common.

From S to N^0: by Theorem 4.57 an instance of SEED has the premises $s = t$ and $\overline{I}(n, s; u.h) = r$. TR of N^0 in the template $\overline{I}(n, v; u.h)$, which is v if $n \equiv 0$ and $I(n, v;\, u.\, h)$ otherwise, gives $r = \overline{I}(n, t; u.h)$, and symmetry gives the conclusion. WI is admissible in N^0 by Theorem 4.3(iii). The last statement is Theorem 4.38. □

By Theorem 4.31(b), substitution and the Leibniz rule are admissible in S. `initializer_equality` also proves the Leibniz rule directly in S, by the induction on the matrix of Theorem 1.22 with the binder steps of Theorem 1.21: at $Q_y B$ the closure by cases of Theorem 4.3(i) replaces Theorem 1.13, and at $D_{x,y} B$ the target is introduced by WI at the variable opened by DE, which is in case mode. Only the literal-equation case uses Theorem 4.62.

Remark 4.65. The Lean formalization of the rule is `NIPrf`, with `niprf_iff_nprf`; it keeps DI and QI and therefore has seven rules. A successor-seeded canonical recursor, as in the package `canonical_recursors`, erases the body of a recursor at the counter 0 and keeps the node; it does not make the zero step the identity of the initializer, and the root replacement is then not an instance of replacement of an initializer.

4.7 One successor step

This section draws on `uniform_step`, which presents the rule STEP with clause contexts, and on the companion calculus of `equality_from_iteration`, where the same rule is called UE and the contexts are lists of native formulas. On zero-normal terms the case EVAL_0 of Theorem 4.9 is absorbed by the syntax, and reflexivity with the one remaining case gives all computation and all equality reasoning.

4.7.1 The rule Step

Definition 4.66 (The rule STEP and the calculus U)**.** On zero-normal terms the rule STEP is

$$\frac{\Gamma \vdash \overline{I}(c, a;\, u.\, h) = t \qquad \Gamma \vdash I(Sc, a;\, u.\, h) = r}{\Gamma \vdash r = h\langle t/u \rangle}.$$

The calculus U has the syntax and the substitution of N^0 and the six rules HYP, REFL, STEP, QE, DE and WI.

The second premise shows c, a and h: its left side is an iterator whose counter is a successor. The left side of the first premise is compared with the predecessor iterate built by the smart constructor; at $c \equiv 0$ it is a. The rule has one case, takes no template and compares no term with an arbitrary context. On raw terms the same rule is EVAL_S.

Lemma 4.67 (Computation and equality from REFL and STEP)**.** *The following are derivable from REFL and STEP over an arbitrary context.*

(a) Successor computation: $I(Sc, a;\, u.\, h) = h\langle \overline{I}(c, a; u.h)/u\rangle$*, by one STEP on two reflexivities.*

(b) The pivot: from $s = t$ *and* $s = r$ *infer* $r = t$*, by two instances of STEP and one reflexivity.*

(c) Symmetry and transitivity.

(d) Congruence: from $s = t$ *infer* $h\langle s/v\rangle = h\langle t/v\rangle$*, by two instances of STEP and two reflexivities.*

(e) TR: from $s = t$ *and* $h\langle s/v\rangle = r$ *infer* $r = h\langle t/v\rangle$*, by four instances of STEP and three reflexivities; and transport in a literal equation.*

The derivations (b)–(e) use STEP at the counter $c \equiv 0$ *only.*

Proof. (a) Apply STEP to the reflexivities of $\overline{I}(c, a; u.h)$ and of $I(Sc, a;\, u.\, h)$.

(b) Put $J_s := I(1, s;\, v.\, v)$; its predecessor iterate is $\overline{I}(0, s; v.v) \equiv s$. STEP with the premises $s = r$ and $J_s = J_s$ gives $J_s = r$. STEP with the premises $s = t$ and $J_s = r$ gives $r = t$:

$$\dfrac{s = t \qquad \dfrac{s = r \qquad J_s = J_s}{J_s = r}}{r = t}$$

This is Theorem 4.11 with the frame $K(s) := J_s$, which is not normalized away. The derivation is valid for every s, including 0.

(c) The pivot applied to $s = s$ and $s = t$ gives $t = s$. From $s = t$ and $t = r$, symmetry gives $t = s$, and the pivot applied to $t = r$ and $t = s$ gives $s = r$.

(d) Put $H_s := I(1, s;\, v.\, h)$. STEP with the premises $s = s$ and $H_s = H_s$ gives $H_s = h\langle s/v\rangle$. STEP with the premises $s = t$ and $H_s = h\langle s/v\rangle$ gives $h\langle s/v\rangle = h\langle t/v\rangle$:

$$\dfrac{s = t \qquad \dfrac{s = s \qquad H_s = H_s}{H_s = h\langle s/v\rangle}}{h\langle s/v\rangle = h\langle t/v\rangle}$$

The first result is reused as the second premise; it is not necessary to derive both $H_s = h\langle s/v\rangle$ and $H_s = h\langle t/v\rangle$ and to pivot them.

(e) The pivot applied to the conclusion of (d) and to $h\langle s/v\rangle = r$ gives $r = h\langle t/v\rangle$. Transport in a literal equation follows by Theorem 4.2(b). □

Remark 4.68. The short congruence (d) is that of `seed_evaluation` and of the package `clausal_single_evaluation`. `uniform_step` and `equality_from_iteration` derive both $H_s = h\langle s/v\rangle$ and $H_s = h\langle t/v\rangle$ and pivot them; their transport has six instances of STEP and five reflexivities. None of the derivations uses equation induction, a numerical case distinction, the injectivity of the successor, a binder rule or the substitution of terms in derivations.

Proposition 4.69 (STEP is stable under substitution)**.** *The instance under a normalizing substitution* σ *of the premises and the conclusion of an instance of STEP are the premises and the conclusion of the instance of STEP at the substituted counter, initializer and body.*

Proof. The counter of the second premise stays a successor. For zero-normal c, a, h the term $\overline{I}(c, a; u.h)$ is $\mathrm{nf}_0(I(c, a;\, u.\, h))$, so by Theorem 4.26(a) its instance under σ is the smart constructor applied to the substituted counter, initializer and body, including when the substituted counter is 0. By Theorem 4.26(b) the instance of $h\langle t/u\rangle$ is the instance of the substituted body at $t\langle\sigma\rangle$. □

Theorem 4.70. *For every zero-normal context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathsf{U}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{N}^0} A.$$

Hence $\Gamma \vdash_{\mathsf{N}} A$ *if and only if* $\mathrm{nf}_0(\Gamma) \vdash_{\mathsf{U}} \mathrm{nf}_0(A)$ *for raw* Γ, A*. The equivalence holds also for the calculus with the seven rules* HYP*,* REFL*,* STEP*,* QE*,* DE*,* DI *and* QI*.*

Proof. Weakening, injective renaming and cut are admissible in U by direct induction, as in Theorem 4.42. From N^0 to U: IS and TR are derived by Theorem 4.67. The term $Z := I(1,0;\, u.\,0)$ is not literally 0, and STEP on the reflexivities of 0 and of Z derives $Z = 0$; Theorem 4.3(i) and (ii) give QI and DI. From U to N^0: reflexivity is Theorem 4.37. For an instance of STEP, IS gives $I(Sc, a;\, u.\,h) = h\langle \overline{I}(c, a; u.h)/u\rangle$; congruence along the first premise gives $h\langle \overline{I}(c, a; u.h)/u\rangle = h\langle t/u\rangle$, and with the second premise the equality rules give $r = h\langle t/u\rangle$. WI is admissible by Theorem 4.3(iii). For the seven-rule calculus the arithmetic rules are translated in the same way and the introductions are common. □

Remark 4.71. U is the companion calculus of `equality_from_iteration`, formalized as `ZEPrf` with `zeprf_iff_nprf`; the missing introduction by cases is derived there with the alias $I(1,0;\, u.\,0)$, with no weakening and no cut. The seven-rule calculus is the formalization `NSPrf` of the arithmetic interface of `uniform_step`, with `nsprf_iff_nprf`, `NSPrf.normal` and `nsprf_iff_pra`. The theorem is an equivalence of consequence relations under arbitrary assumptions; it is more than the agreement of the two interfaces on true equations between numerals.

4.7.2 Five patterns with clause contexts

`uniform_step` combines STEP with the clause contexts of Section 1.9: the formulas are the equations and the formulas $D_{x,y}B$, a context is a finite list of clauses, and an assumption of a formula A is the constant clause $\mathrm{ord}(A)$ of Section 5.1.2.

Definition 4.72 (The calculus $\mathsf{C}_{\mathrm{Step}}$)**.** $\mathsf{C}_{\mathrm{Step}}$ is the calculus C^0 of Theorem 5.20: zero-normal terms, normalizing substitution, clause contexts, and the five rules USE, REFL, the rule STEP of Theorem 4.66 (called EVAL there), the elimination DE that retains the clause $[B(b,0);\, y.B(b,y)]$ (called ELIM there), and WI with its assumption stored as a constant clause (called INTRO there).

Theorem 4.73 (Faithfulness of the five patterns)**.** *For every finite context* Γ *of raw native formulas and every raw native formula* A*,*

$$\Gamma \vdash_{\mathsf{N}} A \quad \Longleftrightarrow \quad \mathrm{ord}\big(\mathrm{nf}_0(\tau\Gamma)\big) \vdash_{\mathsf{C}_{\mathrm{Step}}} \mathrm{nf}_0(\tau A),$$

where τ *is the translation of Theorem 5.1, which replaces each* $Q_y B$ *by* $D_{w,y}B$ *with an unused* w*. The ordinary realization of* $\mathsf{C}_{\mathrm{Step}}$ *is bi-interpretable and equiconsistent with* iPRA*, with whole numerical domains, equality interpreted by equality and numerical composites with the graph* $x = y$*.*

Proof. Theorem 5.22(iv), with Theorems 5.12 and 5.78 for the composites and the bridge. □

The derived rules of the calculus are those of Theorem 5.21; its items (ii)–(v) are the derivations of Theorem 4.67.

Remark 4.74. The calculus of `uniform_step` is C^0; the package differs from Section 5.3.2 in its proof of Theorem 4.73, in three respects. First, from N it translates the derivations of N^0 (Theorem 4.38) rule by rule, where Theorem 5.22(i) normalizes derivations of the raw clause calculus: HYP is USE applied to the reflexivity of 0, IS and TR are Theorem 4.67, QE

and QI are the closure interfaces of Theorem 5.9, and for DE the retained clause derives the translated closure (Theorem 5.9(iii)) and a cut supplies the branch. Second, its alias of zero is $q := I(1,0;\, u.\, u)$, and it recovers the introduction by cases of a literal equation $E(x,y)$ at the witness 0 without a cut: the given premise is closed at the unused witness q into $D_{w,y}E(0,y)$, the elimination retains the clause $[E(0,0);\, y.E(0,y)]$, and under the assumption $E(q,0)$ a transport along $q = 0$, a USE of that clause and a transport along $0 = q$ give $E(q,Sz)$, the premise of WI at q in case mode. Third, toward N it argues by the soundness of the five rules in T_{eq} under the ordinary reading, recorded after Theorem 5.78, and by Theorem 1.119, in place of the translation of annotated contexts in Theorem 5.22(ii). The five patterns keep the choice between cases and progress inside WI; the distinction between coverage and equation induction is not removed. The Lean formalization of the package is the arithmetic interface with native contexts (`NSPrf`, Theorem 4.70).

4.8 Seed evaluation and suspended application

This section draws on `seed_evaluation`: its rule SEEDEVAL, which fuses the replacement of an initializer (Section 4.6) with the successor step (Section 4.7), and its second construction, in which every iteration at a successor counter is factored into a suspended one-step application.

4.8.1 The rule SeedEval

Definition 4.75. On zero-normal terms the rule SEEDEVAL is

$$\frac{\Gamma \vdash a = b \qquad \Gamma \vdash I(Sc,a;\, u.\, h) = r}{\Gamma \vdash r = h\langle \overline{I}(c,b;\, u.\, h)/u\rangle}.$$

The calculus $\mathsf{C}_{\mathrm{Seed}}$ is $\mathsf{C}_{\mathrm{Step}}$ (Theorem 4.72) with SEEDEVAL in place of STEP.

The counter, the initializer and the body are read off the second premise, and the left side of the first premise is compared with the initializer a alone. In comparison with STEP, the first premise no longer mentions the preceding iterate, and the conclusion changes at the same time: it iterates from the equal seed b before the last application of the body. At $c \equiv 0$ the two rules are the same inference,

$$\frac{a = b \qquad I(1,a;\, u.\, h) = r}{r = h\langle b/u\rangle},$$

and by Theorem 4.67 the pivot, symmetry, transitivity, congruence and TR are derived from reflexivity and this shared instance. They are therefore available with either rule.

Lemma 4.76 (The two rules are interderivable)**.** *Over an arbitrary context and with reflexivity:*

(a) SEEDEVAL *derives successor computation, by one instance on two reflexivities;*

(b) every instance of SEEDEVAL *is derivable from* STEP*, by two instances at the counter* 0 *and one further instance;*

(c) every instance of STEP *is derivable from* SEEDEVAL*, by seven instances.*

At $c \equiv 0$ no expansion is needed in (b) and (c). None of the derivations uses a binder rule, a case distinction or induction.

Proof. Put $P := \overline{I}(c, a; u.h)$ and $B := I(Sc, a;\, u.\, h)$.

(a) SEEDEVAL applied to $a = a$ and $B = B$ concludes $B = h\langle P/u\rangle$. Its first premise does not contain P.

(b) Let $a = b$ and $B = r$ be given. Congruence (Theorem 4.67(d)) in the template $v \mapsto \overline{I}(c, v; u.h)$, with v fresh for c and h, gives $\overline{I}(c, a; u.h) = \overline{I}(c, b; u.h)$ by two instances of STEP at the counter 0. STEP with this equation and $B = r$ gives $r = h\langle\overline{I}(c, b; u.h)/u\rangle$. The body h is fixed throughout; no agreement of two different bodies is proved.

(c) Let $P = t$ and $B = r$ be given. By (a), $B = h\langle P/u\rangle$. The pivot applied to $B = r$ and $B = h\langle P/u\rangle$ gives $h\langle P/u\rangle = r$. TR along $P = t$ in the template h gives $r = h\langle t/u\rangle$. The derivation has one instance for the computation, two for the pivot and four for TR. □

Theorem 4.77. *For every finite context* Γ *of clauses and every formula* A *of the clause calculi,*

$$\Gamma \vdash_{\mathsf{C}_{\mathrm{Step}}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{C}_{\mathrm{Seed}}} A.$$

The sequent is literally unchanged. The same equivalence holds for the calculi with native contexts: the calculus with the rules HYP, REFL, SEEDEVAL, QE, DE, DI and QI, and the calculus with WI in place of the last two, derive the sequents of N^0*. Consequently Theorem 4.73 holds for* $\mathsf{C}_{\mathrm{Seed}}$*.*

Proof. Replace each arithmetic inference of a derivation by its expansion in Theorem 4.76 and keep every other inference. The two calculi have the same terms, formulas, clauses, contexts, substitution and freshness conditions, and an expansion has the same premises and conclusion as the inference that it replaces. No witness is changed, so the mode of each WI is unchanged. The statement for native contexts follows in the same way from Theorem 4.70. □

The seven-rule calculus with native contexts is formalized as `NVPrf`, with `nvprf_iff_nprf`, `NVPrf.normal` and `nvprf_iff_pra`. The fusion differs from the rule of `initializer_equality`, which keeps successor computation as a separate rule, and from the package `counter_evaluation`, which fuses evaluation with an equation between counters for a recursor whose body binds the counter and the state.

Proposition 4.78 (Size of the computation certificate)**.** *Let* $c \not\equiv 0$*. The derivation of* $B = h\langle P/u\rangle$ *by STEP is* $\mathrm{Step}(\mathrm{Refl}(P), \mathrm{Refl}(B))$*, and by SEEDEVAL it is* $\mathrm{SeedEval}(\mathrm{Refl}(a), \mathrm{Refl}(B))$*. The two have the same number of inference nodes, and the second contains* $|P| - |a| = 1 + |c| + |h|$ *fewer nodes of term syntax, where* $|\cdot|$ *counts one node for each variable, constant and constructor. In a serialization that writes* $I(c, a;\, u.\, h)$ *as the JSON list* `["i",c,a,h]` *without optional whitespace, the second derivation is shorter by* $8 + \ell(c) + \ell(h)$ *bytes, where* ℓ *is the length of the serialized term. At* $c \equiv 0$ *the two derivations coincide.*

Proof. $|P| = 1 + |c| + |a| + |h|$, and the only difference between the two derivations is the term of the first reflexivity. The brackets, the tag and the separators of the list contribute eight bytes, and the counter and the body are no longer repeated. The conclusion is the same in both derivations. □

The saving grows without bound with the body. It is established for these derivations of a single successor computation; the translation of Theorem 4.76(c) expands other derivations, and a format that shares subterms changes the comparison.

4.8.2 Suspended application

The second construction of `seed_evaluation` removes the counter from the arithmetic rule altogether. The price is a normalization between the zero normalization and the full normalization of Section 4.11: an iterator at a successor counter is factored into a one-step application whose body is never executed by the normalizer.

Definition 4.79 (Unit normalization)**.** Extend the raw terms by the constructor $U(a;\, u.\, h)$, which binds u in h and not in a. Its *expansion* is $U(a; u.h)^\circ := I(1, a^\circ;\, u.\, h^\circ)$, and $(\cdot)^\circ$ commutes with the other constructors. On terms n, a, h define

$$K(n, a;\, u.\, h) := \begin{cases} a, & n \equiv 0, \\ U(K(m, a;\, u.\, h);\, u.\, h), & n \equiv Sm, \\ I(n, a;\, u.\, h), & \text{otherwise}, \end{cases}$$

by recursion on n, and define nf_U by structural recursion: $\mathrm{nf}_U(x) := x$, $\mathrm{nf}_U(0) := 0$, $\mathrm{nf}_U(St) := S\,\mathrm{nf}_U(t)$,

$$\mathrm{nf}_U(U(a; u.h)) := U(\mathrm{nf}_U(a); u.\,\mathrm{nf}_U(h)), \qquad \mathrm{nf}_U(I(n, a;\, u.\, h)) := K(\mathrm{nf}_U(n), \mathrm{nf}_U(a);\, u.\, \mathrm{nf}_U(h)).$$

A term is *unit-normal* if it is fixed by nf_U: its subterms are unit-normal and the counter of each of its iterators has neither 0 nor S at its root.

For example, for a variable x,

$$\mathrm{nf}_U\big(I(SSx, a;\, u.\, h)\big) \equiv U\big(U(I(x, \mathrm{nf}_U(a);\, u.\, \mathrm{nf}_U(h)); u.\,\mathrm{nf}_U(h)); u.\,\mathrm{nf}_U(h)\big).$$

Each displayed u binds its own copy of the body. The result is obtained by inspecting the leading successors of the counter; neither $h[a/u]$ nor any other execution of h occurs. A suspended application is always kept, also when its body is constant or the identity. In T_{eq} the expansion of $U(a; u.h)$ equals $h^\circ[a^\circ/u]$, so the constructor adds no numerical freedom and no strength.

Lemma 4.80 (Termination and uniqueness)**.** *Consider the two contractions*

$$I(0, a;\, u.\, h) \to a, \qquad I(Sn, a;\, u.\, h) \to U(I(n, a;\, u.\, h);\, u.\, h),$$

closed under every context, with no contraction for U. Every reduction sequence is finite, every term reduces to $\mathrm{nf}_U(t)$, and $\mathrm{nf}_U(t)$ is the unique normal form of t. In particular the reduction is confluent, and nf_U is idempotent and primitive recursive on codes of terms.

Proof. Assign positive integer weights by $w(x) = w(0) = 1$, $w(St) = w(t) + 1$,

$$w(U(a; u.h)) = w(a) + w(h) + 1, \qquad w(I(n, a;\, u.\, h)) = (w(n) + 1)(w(a) + w(h) + 1).$$

Write $A = w(a)$, $H = w(h)$ and $q = w(n)$. The zero contraction decreases the weight from $2(A + H + 1)$ to A. The second contraction changes it from $(q + 2)(A + H + 1)$ to $(q + 1)(A + H + 1) + H + 1$, a decrease by exactly $A > 0$; the duplication of the body is accounted for. Every constructor is strictly increasing in the weight of each child, so the decrease persists in every context, including bodies. Hence every reduction decreases a positive integer.

nf_U is invariant under both contractions at the root: for the first by its definition, and for the second because

$$\mathrm{nf}_U(I(Sn, a;\, u.\, h)) \equiv U\big(K(\mathrm{nf}_U(n), \mathrm{nf}_U(a); u.\,\mathrm{nf}_U(h)); u.\,\mathrm{nf}_U(h)\big) \equiv \mathrm{nf}_U\big(U(I(n, a;\, u.\, h); u.h)\big).$$

A structural induction extends the invariance to contractions in context. The irreducible terms are exactly the unit-normal terms. By termination every term has an irreducible reduct, which by invariance is $\mathrm{nf}_U(t)$. Any two reducts of t reduce to this term.

With a primitive recursive coding of terms, the weight and a deterministic choice of one contraction are primitive recursive, and fewer than $w(t)$ contractions occur; the bounded iteration of the step computes nf_U. The normalizer is not an evaluator of primitive recursive programs: it never substitutes an argument into the body of U. □

Lemma 4.81 (Coherence with substitution)**.** $\mathrm{nf}_U(t[\sigma]) \equiv \mathrm{nf}_U(\mathrm{nf}_U(t)[\mathrm{nf}_U \circ \sigma])$ *for all terms* t *and substitutions* σ*. Hence the normalizing substitution* $t\langle\sigma\rangle_U := \mathrm{nf}_U(t[\sigma])$ *satisfies the laws of Theorem 4.26;* nf_U *commutes with renaming and introduces no free variable. On terms without* U*,* $\mathrm{nf}_U \circ \mathrm{nf}_0 = \mathrm{nf}_U$*.*

Proof. Both contractions are preserved by every capture-free substitution, and contracting inside the terms substituted into a fixed term is a reduction of the instance. Hence $t[\sigma]$ reduces to $\mathrm{nf}_U(t)[\mathrm{nf}_U \circ \sigma]$, and both have the normal form of the left side, by Theorem 4.80. Below U and the body of an iterator the substitution is lifted. The laws follow as in Theorem 4.26, whose proof uses only $(\nu 1)$–$(\nu 4)$. The last statement holds because $t \to^* \mathrm{nf}_0(t)$ by zero contractions, which are among the two contractions. □

Definition 4.82 (The rule APP and the calculus $\mathsf{C}_{\mathrm{App}}$)**.** On unit-normal terms the rule APP is

$$\frac{\Gamma \vdash a = b \qquad \Gamma \vdash U(a;\, u.\, h) = r}{\Gamma \vdash r = h\langle b/u\rangle_U}.$$

The calculus $\mathsf{C}_{\mathrm{App}}$ has unit-normal terms, the formulas and clauses of Section 4.7.2 over them, the substitution $\langle\cdot\rangle_U$, and the five rules USE, REFL, APP, DE and WI, the last three logical rules as in Theorem 4.72.

APP inspects no counter and builds no predecessor iterate; that work is done by the constructor K.

Lemma 4.83. *From* REFL *and* APP*, over an arbitrary context:*

(a) the pivot, symmetry, transitivity, congruence in every unit-normal template and TR *are derivable, without induction or case distinction;*

(b) for unit-normal c, a, h*,* $K(Sc, a; u.h) = h\langle K(c, a; u.h)/u\rangle_U$ *is derivable by one* APP *on two reflexivities;*

(c) the image under nf_U *of an instance of* STEP *on zero-normal terms is an instance of* APP*.*

Proof. (a) As in Theorem 4.67, with the frame $J_s := U(s; v.v)$ for the pivot and $H_s := U(s; v.h)$ for congruence: APP on $s = r$ and $J_s = J_s$ gives $J_s = r$, and APP on $s = t$ and $J_s = r$ gives $r = t$; APP on $s = s$ and $H_s = H_s$ gives $H_s = h\langle s/v\rangle_U$, and APP on $s = t$ and this equation gives $h\langle s/v\rangle_U = h\langle t/v\rangle_U$.

(b) By definition $K(Sc, a; u.h) \equiv U(K(c, a; u.h); u.h)$. Apply APP to the reflexivities of $K(c, a; u.h)$ and of this term.

(c) Let the instance have the premises $\overline{I}(c, a; u.h) = t$ and $I(Sc, a;\, u.\, h) = r$ and the conclusion $r = \mathrm{nf}_0(h[t/u])$. Write c', a', h' for the images of c, a, h under nf_U. Then $\mathrm{nf}_U(\overline{I}(c, a; u.h)) \equiv K(c', a'; u.h')$, in both cases of the smart constructor, and $\mathrm{nf}_U(I(Sc, a;\, u.\, h)) \equiv U(K(c', a'; u.h'); u.h')$. APP concludes $\mathrm{nf}_U(r) = h'\langle \mathrm{nf}_U(t)/u\rangle_U$, and by Theorem 4.81 this term is $\mathrm{nf}_U(h[t/u]) \equiv \mathrm{nf}_U(\mathrm{nf}_0(h[t/u]))$. □

Lemma 4.84. *For every term* t *without* U*,* $\mathrm{nf}_U(t) \equiv 0$ *if and only if* $\mathrm{nf}_0(t) \equiv 0$*. In particular, for a zero-normal* t*,* $\mathrm{nf}_U(t) \equiv 0$ *if and only if* $t \equiv 0$*.*

Proof. By structural induction. A variable and a successor do not normalize to 0 under either map. For an iterator, nf_U yields 0 exactly when the normalized counter and the normalized initializer are both 0: otherwise the result is the initializer, which is not 0, or has U or an iterator at its root. The same holds for nf_0, and the induction hypotheses for the counter and the initializer apply. □

Theorem 4.85. *Let* Γ *be a context of clauses and* A *a formula of* $\mathsf{C}_{\mathrm{Step}}$*.*

(a) If $\Gamma \vdash_{\mathsf{C}_{\mathrm{Step}}} A$*, then* $\mathrm{nf}_U(\Gamma) \vdash_{\mathsf{C}_{\mathrm{App}}} \mathrm{nf}_U(A)$*.*

(b) $\mathsf{C}_{\mathrm{App}}$ *is sound for* T_{eq} *under the expansion* $(\cdot)^\circ$ *and the readings of clauses and formulas, and* T_{eq} *proves* $t^\circ = \mathrm{nf}_U(t)^\circ$ *for every term* t*.*

(c) If Γ *consists of assumption clauses and* $\mathrm{nf}_U(\Gamma) \vdash_{\mathsf{C}_{\mathrm{App}}} \mathrm{nf}_U(A)$*, then* $\Gamma \vdash_{\mathsf{C}_{\mathrm{Step}}} A$*.*

(d) For a unit-normal t*,* $\mathrm{nf}_U(t^\circ) \equiv t$*.*

Consequently the ordinary realization of $\mathsf{C}_{\mathrm{App}}$ *is bi-interpretable and equiconsistent with* iPRA*, with whole domains, equality interpreted by equality and numerical composites with the graph* $x = y$*.*

Proof. (a) By induction on the derivation, translating terms by nf_U, formulas termwise and contexts clause by clause. REFL is immediate, and each STEP becomes one APP by Theorem 4.83(c). USE and DE are translated by Theorem 4.81. For WI the root of the matrix is preserved, and by Theorem 4.84 the witness is literally 0 after the translation exactly when it was before, so the mode is the same; freshness is preserved because nf_U introduces no variable.

(b) In T_{eq} the zero contraction is the zero computation equation. For the second contraction both sides equal $h[I(n, a;\, u.\, h)/u]$: the left by successor computation, the expansion of the right by the successor and the zero computation at the counter 1. Contractions in the counter or the initializer are lifted by congruence, and contractions in a body by body congruence (Theorem 1.56(ii), transferred to T_{eq}), which uses the induction for literal equations. Hence $\mathsf{T}_{\mathrm{eq}} \vdash t^\circ = \mathrm{nf}_U(t)^\circ$. APP is sound by the computation at the counter 1 and equality, and the logical rules are sound under the ordinary reading as recorded after Theorem 5.78: USE and DE are ordinary eliminations, WI in case mode uses coverage and in progress mode the induction for a literal equation under the assumed base, and a normalized instance and the ordinary instance are provably equal.

(c) By (b), T_{eq} proves the reading of A from the readings of Γ. If Γ is $\mathrm{ord}(\Delta)$, then Δ and A are zero-normal native formulas without Q_y, so $\Delta \vdash_{\mathsf{N}} A$ by Theorem 1.119, and Theorem 5.22(iii) gives $\Gamma \vdash_{\mathsf{C}_{\mathrm{Step}}} A$.

(d) By structural induction: the expansion $I(1, a^\circ;\, u.\, h^\circ)$ of an application normalizes to $U(a; u.h)$, and an iterator of a unit-normal term has a counter with neither 0 nor S at its root, whose normalized expansion is the same counter.

The last statement follows from (a)–(d) and Theorem 4.73: the constructor U is an abbreviation of a fixed term, and normalization changes representatives of expressions only. □

Proposition 4.86 (The variant without a new constructor)**.** *Let* $U(a; u.h)$ *abbreviate* $I(1, a;\, u.\, h)$ *in the unchanged term grammar, and define* K *and* nf_U *by the clauses of Theorem 4.79, without the clause for* U*. The fixed points of* nf_U *are the terms in which every iterator counter is the literal* 1 *or has a root other than* 0 *and* S*. The normal forms are those of the contractions*

$$I(0, a;\, u.\, h) \to a, \qquad I(Sn, a;\, u.\, h) \to I(1, I(n, a;\, u.\, h);\, u.\, h) \quad (n \not\equiv 0),$$

which terminate; Theorems 4.81 and 4.84 hold; the rule APP *is the instance of* STEP *at the counter* 0*, followed by normalization; and Theorem 4.85 holds for the resulting calculus.*

Proof. The side condition on the second contraction is necessary: without it $I(1, a;\, u.\, h)$ would be contracted to $I(1, I(0, a;\, u.\, h);\, u.\, h)$, which the zero contraction contracts back. For termination use the weights $w(x) = w(0) = 1$, $w(St) = w(t) + 1$ and, with $A = w(a)$ and $H = w(h)$,

$$w(I(n, a;\, u.\, h)) = \begin{cases} A + H + 1, & n \equiv 1, \\ (w(n) + 1)(A + H + 1), & \text{otherwise.} \end{cases}$$

The zero contraction decreases the weight from $2(A+H+1)$ to A. In the second contraction the left counter Sn is not 1, so the left weight is $(w(n)+2)(A+H+1)$. If $n \not\equiv 1$ the right weight is $(w(n)+1)(A+H+1)+H+1$, smaller by A. If $n \equiv 1$ the right weight is $A+2H+2$ and the left weight is $4(A+H+1)$. The weight is strictly increasing in the initializer and the body. In the counter position the one exceptional case is a contraction that makes the counter the literal 1, which replaces the product by the smaller sum; a counter that is the irreducible term 1 is not contracted. Hence every contraction in context decreases the weight.

Both contractions preserve nf_U; for the second this is the identity $K(Sn, a; u.h) \equiv U(K(n, a; u.h); u.h)$, which holds also at $n \equiv 0$. For coherence with substitution the conditional contraction is not closed under substitution, since a substitution can turn n into a term with the normal form 0. Argue in two steps. First, replacing each substituted term by its normal form does not change the normal form of the instance, by the invariance in context. Second, if $t \to t'$ by one contraction, then $\mathrm{nf}_U(t[\sigma]) \equiv \mathrm{nf}_U(t'[\sigma])$: for the zero contraction literally, and for the second by the displayed identity for K, which is valid at 0. Following a reduction of t to $\mathrm{nf}_U(t)$ gives the statement. The remaining proofs are those of Theorems 4.83 to 4.85; the unit iterator is never removed and its body is never exposed. □

4.9 Atomic-seed zero contraction with raw rule metadata

This section draws on the mathematical result of `atomic_seed_refinement`: a six-rule presentation of N whose normalizer contracts $I(0, a;\, u.\, h)$ only when the normalized initializer a is a variable or the literal 0, and whose rules keep their raw instances. That normalizer is weaker than the zero normalization of Section 4.4 and is not an admissible normalizer: it violates the coherence $(\nu 4)$, and Theorem 4.30 does not apply to it. The first subsection gives the form of the transfer argument that needs no coherence, at the price of raw metadata in every rule; it applies to every normalizer that is the identity inside N and satisfies the structural conditions $(\nu 2)$, $(\nu 3)$ and a weak form of $(\nu 1)$. The remaining subsections apply it to the atomic-seed normalizer.

4.9.1 Normalization after raw instantiation

Definition 4.87 (Conversion normalizer)**.** A *conversion normalizer* is a map ν from raw terms to raw terms that satisfies $(\nu 2)$, $(\nu 3)$ and $(\nu 5)$ of Theorem 4.24 and the following weakening of $(\nu 1)$:

$(\nu 1^{-})$ $\nu(x) \equiv x$, $\nu(0) \equiv 0$ and $\nu(S0) \equiv S0$.

It is extended to formulas and contexts as in Theorem 4.24. Every admissible normalizer is a conversion normalizer. The successor clause of $(\nu 1)$ is not required: the weak-head normalizer of Section 4.10 does not normalize below a successor.

The proof of Theorem 4.27 uses $(\nu 5)$ and the extension of ν to formulas only. The lemma therefore holds for every conversion normalizer: $\Gamma, A \vdash_{\mathsf{N}} \nu(A)$ and $\Gamma, \nu(A) \vdash_{\mathsf{N}} A$ for every raw formula A and every context Γ.

Definition 4.88 (Raw-metadata reading)**.** Let ν be a conversion normalizer. A *sequent over* ν is a sequent $\Sigma \vdash C$ in which Σ is a finite list of ν-normal formulas and C is ν-normal. For each rule R among the eight rules of N, WI (Theorem 4.1, on raw terms), reflexivity REFL, $\Gamma \vdash t = t$, and substitution for a variable VSUB, from $\Gamma \vdash x = t$ and $\Gamma \vdash u = v$ infer $\Gamma \vdash v = u[t/x]$ for a free variable x (Eq. (6.16), on raw terms), the rule $R^{[\nu]}$ on sequents over ν is the following. $\mathrm{HYP}^{[\nu]}$ derives $\Sigma \vdash C$ for $C \in \Sigma$. For every other R, take an instance of R on raw syntax in the context Σ. It is fixed by its *raw metadata*: the raw terms, templates,

matrices and the witness that the schema of R mentions, and for DE the conclusion A, which is required to be ν-normal. Let its premises be $\Sigma, L_i \vdash P_i$, where L_i is the local assumption of the premise if there is one, and let its conclusion be $\Sigma \vdash C_0$. Then $R^{[\nu]}$ is the inference

$$\frac{\Sigma, \nu(L_i) \vdash \nu(P_i) \quad (\text{all } i)}{\Sigma \vdash \nu(C_0)}.$$

The side conditions are tested on the raw instance. The eigenvariable (b of DE, z of DI, QI and WI) is not free in Σ, in the conclusion, or in any of the raw metadata; the test of QI and WI for a literal equation is applied to the raw matrix, which has the root of its normal form; the test $t \equiv 0$ of WI is applied to the raw witness; and the source x of VSUB is a variable, which is its own normal form by ($\nu 1^-$), so that its first premise is $x = \nu(t)$. A *raw-metadata calculus over* ν is a calculus on sequents over ν whose rules are rules $R^{[\nu]}$. The calculus $\mathsf{N}^{[\nu]}$ has the eight rules $R^{[\nu]}$ for the rules R of N.

Lemma 4.89 (Soundness of raw-metadata rules)**.** *Let ν be a conversion normalizer and let R be one of the rules of Theorem 4.88. If the premises of an instance of $R^{[\nu]}$ are derivable in N, read as raw sequents, then so is its conclusion. Hence $\Sigma \vdash_{\mathsf{K}} C$ implies $\Sigma \vdash_{\mathsf{N}} C$ for every raw-metadata calculus K over ν.*

Proof. For HYP there is nothing to prove. Otherwise let $\Sigma, \nu(L_i) \vdash_{\mathsf{N}} \nu(P_i)$ for every premise. By Theorem 4.27, $\Sigma, L_i \vdash_{\mathsf{N}} \nu(L_i)$ and $\Sigma, L_i, \nu(P_i) \vdash_{\mathsf{N}} P_i$, and two cuts (Theorem 1.11) give $\Sigma, L_i \vdash_{\mathsf{N}} P_i$. These are the premises of the raw instance of R. Its side conditions hold in N: the conditions of N and of Theorem 2.15 require that the eigenvariable is not free in the context, in the conclusion, in the witness or in the formula $D_{x,y}B$ or the parameters of the matrix, and all of these are among the data tested. For a rule of N the rule gives $\Sigma \vdash_{\mathsf{N}} C_0$. For WI the mode is that of the raw instance, and WI on raw terms is admissible in N by Theorem 4.3(iii). REFL is derivable in N (Theorem 1.18), and an instance of VSUB is the instance of TR with the template $K(w) := u[w/x]$, w fresh, since $K(x) \equiv u$ and $K(t) \equiv u[t/x]$, also when t contains x. For DE the conclusion C_0 is ν-normal; in the other cases Theorem 4.27 and cut give $\Sigma \vdash_{\mathsf{N}} \nu(C_0)$. The second statement follows by induction on derivations. □

Theorem 4.90 (Transfer with raw metadata)**.** *Let ν be a conversion normalizer.*

(a) For every finite context Γ of raw native formulas and every raw native formula A,

$$\Gamma \vdash_{\mathsf{N}} A \quad \Longleftrightarrow \quad \nu(\Gamma) \vdash_{\mathsf{N}^{[\nu]}} \nu(A).$$

(b) Let K be a raw-metadata calculus over ν in which every rule of $\mathsf{N}^{[\nu]}$ is admissible. Then K derives exactly the sequents over ν that N derives, and (a) holds with K in place of $\mathsf{N}^{[\nu]}$.

(c) If ν is an admissible normalizer, the rules of $\mathsf{N}^{[\nu]}$ and the rules of N^{ν} (Theorem 4.28) have the same instances.

Both directions of (a) and (b) are effective on finite derivations.

Proof. (a) From left to right, by induction on the derivation in N, replace each sequent $\Gamma' \vdash A'$ by $\nu(\Gamma') \vdash \nu(A')$ and each inference of a rule R by the inference of $R^{[\nu]}$ with the same raw metadata. An assumption $A' \in \Gamma'$ gives $\nu(A') \in \nu(\Gamma')$. The premises, the local assumptions and the conclusion of the inference of $R^{[\nu]}$ are by definition the normal forms of those of the raw inference; for DE the conclusion $\nu(A')$ is ν-normal by ($\nu 3$). An eigenvariable that is not free in Γ', in A' or in the raw metadata is not free in $\nu(\Gamma')$ or in $\nu(A')$ by ($\nu 2$). No property of ν under substitution is used. From right to left, Theorem 4.89 gives $\nu(\Gamma) \vdash_{\mathsf{N}} \nu(A)$, and Theorem 4.27 with cut gives $\Gamma \vdash_{\mathsf{N}} A$.

(b) A sequent over ν that K derives is derivable in N by Theorem 4.89. Conversely let $\Delta \vdash_{\mathsf{N}} C$ with Δ and C ν-normal. By (a) and (ν3), $\Delta \vdash_{\mathsf{N}^{[\nu]}} C$, and induction on this derivation, with the admissibility of each rule in K, gives $\Delta \vdash_{\mathsf{K}} C$. For (a) with K: $\Gamma \vdash_{\mathsf{N}} A$ holds if and only if $\nu(\Gamma) \vdash_{\mathsf{N}} \nu(A)$, by Theorem 4.27 and cut, and the latter holds if and only if $\nu(\Gamma) \vdash_{\mathsf{K}} \nu(A)$.

(c) Let m be the raw metadata of an instance of a rule of $\mathsf{N}^{[\nu]}$. By (ν4) and Theorem 4.26(a), the normal form of each premise, local assumption and conclusion is the corresponding formula of the instance at the metadata $\nu(m)$, formed with normalizing substitution; for example $\nu(h[s/v]) \equiv \nu(h)\langle\nu(s)/v\rangle$, and $\nu(Q_y B[b/x]) \equiv Q_y \nu(B)[b/x]$ by (ν2). By (ν2) an eigenvariable that is fresh for m is fresh for $\nu(m)$, and the root of the matrix of QI is unchanged. Hence the instance is an instance of the same rule of N^{ν}; the instances of $\text{I0}^{[\nu]}$ and $\text{IS}^{[\nu]}$ are exactly those of COMP^{ν}. Conversely an instance of a rule of N^{ν} is the instance of the corresponding rule of $\mathsf{N}^{[\nu]}$ whose metadata are ν-normal. □

For an admissible normalizer, (a) and (c) give a second proof of Theorem 4.30; in it, (ν4) is used only to show that normal metadata suffice. For a conversion normalizer without (ν4), normal metadata need not suffice, and the instances with raw metadata are the reason why the rule-by-rule translation in (a) works.

Remark 4.91. The rule $\text{WI}^{[\nu]}$ selects its mode from the raw witness and is sound for every conversion normalizer by Theorem 4.89, also when the raw witness has the normal form 0. The mode change of Theorem 4.144 arises only when the witness is normalized before the mode is selected. Read as an inference of $\text{WI}^{[\text{nf}]}$, the introduction of that example at the raw witness $I(0,0;\, u.\,0)$ stays in case mode, and its normalized premise is $0 = 1 \vdash Sz = 1$.

Corollary 4.92 (Interpretation of raw-metadata calculi)**.** *Let ν be a conversion normalizer and let K be a calculus as in Theorem 4.90(b). Then statements (a) and (c) of Theorem 4.31 hold for K. Weakening and cut are admissible in K, and so are the substitution of terms, from $\Delta \vdash C$ infer $\nu(\Delta[\sigma]) \vdash \nu(C[\sigma])$, and the Leibniz rule with raw metadata, from $\Sigma \vdash \nu(s = t)$ and $\Sigma \vdash \nu(F[s/v])$ infer $\Sigma \vdash \nu(F[t/v])$, for raw s, t, F.*

Proof. Statement (a) of Theorem 4.31 is Theorem 4.90(b). The proof of statement (c) given there uses of ν only statement (a), Theorem 4.27, (ν2) and the instances $\nu(x) \equiv x$ and $\nu(0 = S0) \equiv (0 = S0)$ of (ν1), which ($\nu1^{-}$) contains, and applies unchanged. For the admissible rules pass through N. A derivation in K gives one in N by Theorem 4.89. Weakening and substitution in N are Theorem 1.10, cut is Theorem 1.11, and the Leibniz rule is Theorem 1.22, with Theorem 4.27 to convert its premises from and its conclusion to normal form. The resulting sequent of N is returned to K by Theorem 4.90(b), applied to its normal form. □

Unlike Theorem 4.29, the corollary obtains substitution in derivations through N and not by a direct induction on derivations of K; no coherence of ν with substitution is needed.

4.9.2 The atomic-seed normalizer

Definition 4.93 (Atomic-seed zero contraction)**.** A raw term is *atomic* if it is a variable or the literal 0. The map nf_{a} on raw terms is defined by structural recursion: $\text{nf}_{\text{a}}(x) := x$, $\text{nf}_{\text{a}}(0) := 0$, $\text{nf}_{\text{a}}(St) := S\,\text{nf}_{\text{a}}(t)$ and, with $n' := \text{nf}_{\text{a}}(n)$, $a' := \text{nf}_{\text{a}}(a)$ and $h' := \text{nf}_{\text{a}}(h)$,

$$\text{nf}_{\text{a}}(I(n,a;\, u.\,h)) := \begin{cases} a', & n' \equiv 0 \text{ and } a' \text{ is atomic,} \\ I(n', a';\, u.\,h'), & \text{otherwise.} \end{cases}$$

It is extended to formulas and contexts as in Theorem 4.24. A raw term is *atomic-seed normal* if no iterator in it, at any depth, has the literal 0 as its counter and an atomic initializer.

The recursion is that of nf_0 (Theorem 4.32) with one more test, on the normalized initializer. For example $\mathrm{nf_a}(I(0,x;\,u.\,h)) \equiv x$ and $\mathrm{nf_a}(I(I(0,0;\,v.\,0),y;\,u.\,h)) \equiv y$, while $I(0,S0;\,u.\,0)$ is atomic-seed normal and is not zero-normal.

Lemma 4.94 (Properties of the atomic-seed normalizer)**.** *(a)* $\mathrm{nf_a}(t)$ *is atomic-seed normal;* $\mathrm{nf_a}(t) \equiv t$ *if and only if* t *is atomic-seed normal;* $\mathrm{nf_a}$ *is idempotent.*

(b) $\mathrm{nf_a}(t[\rho]) \equiv \mathrm{nf_a}(t)[\rho]$ *for every renaming* ρ *of variables, and* $\mathrm{FV}(\mathrm{nf_a}(t)) \subseteq \mathrm{FV}(t)$. $\mathrm{nf_a}(t)$ *has at most as many nodes as* t*, and* $\mathrm{nf_a}$ *is a primitive recursive operation on codes of finite terms.*

(c) $t \to^* \mathrm{nf_a}(t)$ *by zero contractions. Hence* $\mathrm{nf}_0(\mathrm{nf_a}(t)) \equiv \mathrm{nf}_0(t)$*; every zero-normal term is atomic-seed normal, and* $\mathrm{nf_a}(\mathrm{nf}_0(t)) \equiv \mathrm{nf}_0(t)$.

(d) Fixed-context congruence. *Let* C *be a raw term or formula,* x *a variable, and* s, s' *raw terms none of whose free variables is bound in* C *at a free occurrence of* x*. If* $\mathrm{nf_a}(s) \equiv \mathrm{nf_a}(s')$*, then* $\mathrm{nf_a}(C[s/x]) \equiv \mathrm{nf_a}(C[s'/x])$.

Proof. (a) By structural induction. The children of a rebuilt iterator are atomic-seed normal and the iterator is rebuilt only when the test fails; a returned initializer is atomic-seed normal. On an atomic-seed normal term every clause returns its argument: the children are fixed by the induction hypothesis, and the test fails at a normal iterator. (b) A renaming sends variables to variables and fixes 0, so it sends atomic terms to atomic terms, other terms to other terms, and a counter to the literal 0 exactly when the counter is 0; by induction it commutes with the test and with the recursion. A contraction deletes the counter and the body and keeps the normalized initializer, so it creates no variable and adds no node. The structural recursion with these size bounds is primitive recursive, as in Theorem 4.33(c); no successor contraction is performed. (c) By structural induction: the children reduce to their normal forms by zero contractions in their positions, since the reduction of Theorem 4.5 is closed under all term contexts including the bodies of iterators, and the step at the root, if it is taken, is a zero contraction. By Theorem 4.33(d) the zero normal form is unique, so $\mathrm{nf}_0(\mathrm{nf_a}(t)) \equiv \mathrm{nf}_0(t)$. A zero-normal term has no iterator with the counter 0 and is atomic-seed normal, and (a) gives the last identity. (d) By induction on C. For each constructor, $\mathrm{nf_a}$ of a node is a function of the values of $\mathrm{nf_a}$ at its children; for an iterator the defining clause depends on n', a' and h' only, and the formula constructors are kept. The substitution commutes with the constructors, because no free variable of s or s' is captured, and at an occurrence of x the two values are $\mathrm{nf_a}(s)$ and $\mathrm{nf_a}(s')$. □

Proposition 4.95. $\mathrm{nf_a}$ *is a conversion normalizer and is not an admissible normalizer: it violates* ($\nu 4$)*.*

Proof. ($\nu 1$) holds by definition, ($\nu 2$) and ($\nu 3$) by Theorem 4.94(a),(b), and ($\nu 5$) by Theorem 4.94(c) and Theorem 4.6. For ($\nu 4$) let $t \equiv I(0,x;\,u.\,0)$ and $\sigma := [S0/x]$. Then

$$\mathrm{nf_a}(t[\sigma]) \equiv I(0,S0;\,u.\,0), \qquad \mathrm{nf_a}\big(\mathrm{nf_a}(t)[\mathrm{nf_a} \circ \sigma]\big) \equiv \mathrm{nf_a}(x[S0/x]) \equiv S0,$$

since $S0$ is not atomic and $\mathrm{nf_a}(t) \equiv x$. The two terms are different; their equation is derivable in N by ($\nu 5$), and it is not an identity of syntax. □

The failure concerns substitution into a term that is not normal. If t and the terms $\sigma(x)$ are atomic-seed normal, both sides of ($\nu 4$) are $\mathrm{nf_a}(t[\sigma])$. It matters for the rules: the calculus $\mathsf{N}^{\mathrm{nf_a}}$ defined as in Theorem 4.28 with $\nu := \mathrm{nf_a}$, whose metadata are normal, has only part of the instances of $\mathsf{N}^{[\mathrm{nf_a}]}$. For the raw body $B(y) := (I(0,y;\,u.\,h) = y)$, with y not free in h, the normal body is $\mathrm{nf_a}(B) \equiv (y = y)$, whose instance at a target t is $\mathrm{nf_a}(t) = \mathrm{nf_a}(t)$, while

the normal form of the raw instance $B(t)$ is $I(0, \mathrm{nf_a}(t);\, u.\ \mathrm{nf_a}(h)) = \mathrm{nf_a}(t)$ when $\mathrm{nf_a}(t)$ is not atomic. The normal form of a premise therefore does not determine the normal form of the conclusion; the raw body does.

Remark 4.96 (Why the initializer is atomic). In the native syntax the binder u of $I(n, a;\, u.\, h)$ covers h only, and a zero contraction returns the initializer without changing an index (Section 4.4). In an arity coding that places the initializer below a slot, a contraction must rebase the initializer (see the remark after Theorem 4.38). `atomic_seed_refinement` gives an example for an arity coding with de Bruijn levels, which it calls the compact protected-level codec. A position has a depth. The children of an iterator array $[n, a, h]$ at depth d lie at depth $d+2$, and the slot of depth d is the state u for h and a dummy for n and a. The child of a successor array $[t]$ at depth d lies at depth $d + 4$, and the slots of depths d and $d + 2$ are dummies. The leaf 0 is 0, a leaf k with $3 \le k < d$ names the slot of depth k, and a dummy reads as 0. At depth 3 the array $[0, [[0], [0], [5]], 0]$ is $I(0, I(1, 1;\, v.\, Sv);\, u.\, 0)$, with value 2: the inner body $[5]$ lies at depth 7, and its leaf, at depth 11, names the state v of depth 5. Replacing the outer array by its initializer array gives $[[0], [0], [5]]$ at depth 3. Its body lies at depth 5, its leaf at depth 9 names the dummy of depth 5 introduced by the successor, and the array is $I(1, 1;\, v.\, S0)$, with value 1. The correct contraction is $[[0], [0], [3]]$, with value 2. Both arrays are well-formed integer and list data, so a successful traversal does not detect the change of value. With an atomic initializer no binder below the initializer moves: a leaf that names an outer slot keeps its name, and a leaf that names the dummy of the contracted iterator denotes 0. A correct sort test and the handling of dummy slots are still required; the article does not give a smaller implementation in such a coding.

4.9.3 Six rules

Definition 4.97 (The calculus A_6). A_6 is the raw-metadata calculus over $\mathrm{nf_a}$ with the six rules $\textsc{hyp}^{[\mathrm{nf_a}]}$, $\textsc{is}^{[\mathrm{nf_a}]}$, $\textsc{tr}^{[\mathrm{nf_a}]}$, $\textsc{qe}^{[\mathrm{nf_a}]}$, $\textsc{de}^{[\mathrm{nf_a}]}$ and $\textsc{wi}^{[\mathrm{nf_a}]}$. Explicitly, Σ is a list of atomic-seed normal formulas, C is atomic-seed normal, $\overline{F}$ denotes $\mathrm{nf_a}(F)$, and the letters n, a, h, s, t, r, H, B denote raw metadata:

$$\frac{}{\Sigma \vdash C}\ (C \in \Sigma), \qquad \frac{}{\Sigma \vdash \overline{I(Sn, a;\, u.\, h) = h[I(n, a;\, u.\, h)/u]}},$$

$$\frac{\Sigma \vdash \overline{s = t} \qquad \Sigma \vdash \overline{H(s) = r}}{\Sigma \vdash \overline{r = H(t)}}, \qquad \frac{\Sigma \vdash \overline{Q_y B(y)} \qquad \Sigma \vdash \overline{B(0)}}{\Sigma \vdash \overline{B(t)}},$$

$$\frac{\Sigma \vdash \overline{D_{x,y} B(x, y)} \qquad \Sigma, \overline{Q_y B(b, y)} \vdash C}{\Sigma \vdash C},$$

$$\frac{\Sigma, \overline{B(t, \eta_{B,t}(z))} \vdash \overline{B(t, Sz)}}{\Sigma \vdash \overline{D_{x,y} B(x, y)}}.$$

They are HYP, IS, TR, QE, DE and WI. Every bar is taken after the displayed raw substitution. In DE the variable b is not free in Σ, in C or in $D_{x,y}B$. In WI the variable z is not free in Σ, in t or in $D_{x,y}B$, and $\eta_{B,t}(z) = z$ exactly when B is a literal equation and the raw witness t is literally 0, otherwise $\eta_{B,t}(z) = 0$. There is no restriction on a in IS or on s in TR. There is no primitive zero computation, reflexivity, symmetry, transitivity, general transport, induction or cut. The normalizer and the raw metadata are part of the specification.

Lemma 4.98 (Reflexivity without zero computation). $\Sigma \vdash_{\mathsf{A}_6} \overline{t = t}$ *for every raw term* t *and every* Σ.

Proof. Let u be a variable not free in t and put $L := I(S0, S0;\, u.\, t)$. The raw instance of IS at $n \equiv 0$, $a \equiv S0$ and $h \equiv t$ is $L = t[I(0, S0;\, u.\, t)/u] \equiv (L = t)$, so IS gives $\Sigma \vdash \overline{L = t}$. TR

with $s \equiv L$, the target t, the identity template $H(v) \equiv v$ and $r \equiv t$ has the premises $\overline{L = t}$ and $\overline{H(L) = t} \equiv \overline{L = t}$, and the conclusion $\overline{t = H(t)} \equiv \overline{t = t}$. □

The derivation does not use a zero computation, and its instance of IS has a nonzero initializer. The body of L is constant. The term $I(1, t;\, u.\, u)$ of Theorem 4.37 does not serve here: the right side of its instance of IS is $I(0, t;\, u.\, u)$, whose normal form is $I(0, \mathrm{nf_a}(t);\, u.\, u)$ and not $\mathrm{nf_a}(t)$ when $\mathrm{nf_a}(t)$ is not atomic.

Lemma 4.99 (A reflexive closure)**.** $\Sigma \vdash_{\mathsf{A}_6} Q_y(y = y)$ *for every* Σ.

Proof. Apply WI to the matrix $B(x, y) := (y = y)$, with x unused, at the raw witness 0. The matrix is a literal equation, so the mode is progress, and the premise, with z not free in Σ, is $\Sigma, z = z \vdash Sz = Sz$, which holds by Theorem 4.98. This gives $\Sigma \vdash D_{x,y}(y = y)$. Apply DE with a fresh b: the local assumption is $\overline{Q_y B(b, y)} \equiv Q_y(y = y)$, and HYP derives $\Sigma, Q_y(y = y) \vdash Q_y(y = y)$. The conclusion $Q_y(y = y)$ is closed, so b does not occur in it. □

Theorem 4.100 (Zero computation from atomic contraction)**.** *For all raw terms* t, h *and every* Σ,

$$\Sigma \vdash_{\mathsf{A}_6} \overline{I(0, t;\, u.\, h) = t}.$$

Proof. Let y be a variable different from u and not free in t or h, and apply QE with the raw body

$$B(y) := \big(I(0, y;\, u.\, h) = y\big).$$

Since the counter is 0 and the normalized initializer y is atomic,

$$\overline{Q_y B(y)} \equiv Q_y(y = y), \qquad \overline{B(0)} \equiv (0 = 0).$$

The first premise is Theorem 4.99 and the second is Theorem 4.98. At the target t the conclusion is $\overline{B(t)}$; the initializer lies outside the binder u and y is not free in h, so $B(t) \equiv (I(0, t;\, u.\, h) = t)$. □

If the raw body were replaced by its normal form $y = y$ before the target is substituted, the same inference would conclude $\overline{t = t}$ and not the zero computation. The raw body of QE is therefore necessary for this derivation; that a calculus with normal metadata does not derive the zero computation in another way is not claimed.

Lemma 4.101 (The two introductions)**.** *The rules* $\mathrm{DI}^{[\mathrm{nf_a}]}$ *and* $\mathrm{QI}^{[\mathrm{nf_a}]}$ *are admissible in* A_6.

Proof. An instance of $\mathrm{DI}^{[\mathrm{nf_a}]}$ is an instance of WI unless its matrix B is a literal equation and its raw witness is literally 0. In that case let $\Sigma, \overline{B(0, 0)} \vdash \overline{B(0, Sz)}$ be derivable, and use WI at the raw witness $j := I(0, 0;\, u.\, 0)$. It is not literally 0, so the mode is case. Since $\mathrm{nf_a}(j) \equiv 0$ and j is closed, Theorem 4.94(d) gives $\overline{B(j, 0)} \equiv \overline{B(0, 0)}$ and $\overline{B(j, Sz)} \equiv \overline{B(0, Sz)}$, so the premise of WI is the given sequent, and the conclusion $\overline{D_{x,y} B}$ does not depend on the witness. The variable z is fresh for j, which is closed. No derivation is rewritten and no equation is transported.

For $\mathrm{QI}^{[\mathrm{nf_a}]}$ let $E(y)$ be a literal equation and $\Sigma, \overline{E(z)} \vdash \overline{E(Sz)}$ derivable, with z not free in Σ or in $Q_y E$. Let x be a variable not free in E and put $B(x, y) := E(y)$. WI at the raw witness 0 is in progress mode, and its premise $\Sigma, \overline{B(0, z)} \vdash \overline{B(0, Sz)}$ is the given sequent; it derives $\Sigma \vdash \overline{D_{x,y} E(y)}$. DE at a fresh b has the local assumption $\overline{Q_y B(b, y)} \equiv \overline{Q_y E(y)}$, and HYP derives the conclusion $\overline{Q_y E(y)}$ from it; b is not free in Σ, in that conclusion or in $D_{x,y} E$. □

The alias j is the raw alias of Theorem 2.16. On zero-normal terms with normal metadata it is unavailable, and Theorem 4.3(ii) uses the alias $I(1, 0;\, u.\, 0)$ together with transport along its equation with 0 (Theorem 4.42). With raw metadata the witness keeps its raw shape for the mode test, and its normal form makes the normalized branch literally the branch at 0.

Theorem 4.102 (Six rules with atomic-seed contraction)**.** A_6 *derives exactly the sequents over* $\mathrm{nf_a}$ *that* N *derives. For every finite context* Γ *of raw native formulas and every raw native formula* A*,*

$$\Gamma \vdash_{\mathsf{N}} A \quad \Longleftrightarrow \quad \mathrm{nf_a}(\Gamma) \vdash_{\mathsf{A}_6} \mathrm{nf_a}(A),$$

effectively on finite derivations, and Theorem 4.92 applies to A_6*. In particular* A_6 *is equiconsistent and bi-interpretable with* iPRA*, with whole numerical domains, equality interpreted by equality, and numerical composites with the graph* $x = y$*.*

Proof. A_6 is a raw-metadata calculus over the conversion normalizer $\mathrm{nf_a}$ (Theorem 4.95). By Theorem 4.90(b) it suffices that each of the eight rules of $\mathsf{N}^{[\mathrm{nf_a}]}$ is admissible in A_6. Five of them are rules of A_6. An instance of $\textsc{i0}^{[\mathrm{nf_a}]}$ is $\Sigma \vdash \overline{I(0, a; u.\, h) = a}$ for raw a, h, which is Theorem 4.100. $\textsc{di}^{[\mathrm{nf_a}]}$ and $\textsc{qi}^{[\mathrm{nf_a}]}$ are admissible by Theorem 4.101. The last statements are Theorem 4.92 and statement (c) of Theorem 4.31; the contradiction $0 = S0$ is atomic-seed normal. □

Remark 4.103. `atomic_seed_refinement` proves the equivalence directly against N, in place of Theorem 4.90: a derivation of N is translated rule by rule with its raw metadata kept, and a derivation of A_6 is simulated in N by converting each normalized premise to its raw form, which uses that every raw term is provably equal to its partial normal form, by the zero computation and the congruence of iterator bodies (Theorem 4.6). Both directions of the comparison use only the derived rules of N, so the argument is not circular. The package composes the equivalence with the bridge of Theorem 1.119 for the bi-interpretation with iPRA, as in Theorem 4.92. The package also observes that IS can be restricted to raw initializers that are not literally 0, and TR to raw sources that are not literally 0, without changing a normalized instance: by Theorem 4.94(d) the raw term $I(0, 0; u.\, 0)$ in place of an excluded 0 gives the same normalized premises and conclusion. This concerns tests on raw data; it does not justify a restriction on normalized initializers or sources, and the primary calculus has neither restriction. Compared with Z_6, A_6 has the same six rule names, a weaker normalizer and raw metadata in every rule; the zero computation is not normalized away but derived. The calculus is not formalized in Lean; the package records finite tests of its reference checker (Section 4.13).

4.10 Weak-head conversion with raw hints

This section draws on `head_conversion`: a six-rule calculus in which only the heads of the two endpoints of an equation are computed, whose only equality rule is substitution for a variable and whose rules keep raw terms as hints; and a reading which shows that on raw terms, with the computation equations as axioms, the same equality rule does not suffice. The normalizer is a conversion normalizer and not an admissible one, so the calculus is compared with N through Theorem 4.90.

4.10.1 The head normalizer

Definition 4.104 (Weak-head normalization)**.** A *head context* is generated by $H ::= [\,] \mid I(H, a; u.\, h)$, with arbitrary raw a, h. *Head reduction* $\to_{\mathrm{h}}$ contracts a zero or successor redex of Theorem 4.5 in a head context. The *neutral* terms are generated by $e ::= x \mid I(e, a; u.\, h)$ with arbitrary raw a, h, and a raw term is *head normal* if it is 0, a successor Sa with an

arbitrary raw a, or neutral. The map W on raw terms is

$$W(x) := x, \qquad W(0) := 0, \qquad W(Sa) := Sa,$$
$$W(I(n,a;\,u.\,h)) := \begin{cases} W(a), & W(n) \equiv 0, \\ W\big(h[I(k,a;\,u.\,h)/u]\big), & W(n) \equiv Sk, \\ I(W(n),a;\,u.\,h), & W(n) \text{ neutral.} \end{cases}$$

It is extended to formulas and contexts as in Theorem 4.24: on a formula it normalizes the two endpoints of every equation, below Q_y and $D_{x,y}$. A formula A is *head normal* if $W(A) \equiv A$.

The recursion is not structural; its termination is part of the next lemma. W never looks below a successor, nor into the initializer or the body of an iterator whose counter is neutral. For example $W\big(SI(0,0;\,u.\,u)\big) \equiv SI(0,0;\,u.\,u)$, whereas $SW(I(0,0;\,u.\,u)) \equiv S0$.

Lemma 4.105 (Head reduction)**.** *(a) A raw term has at most one head redex, every head reduction sequence is finite, and the head normal terms are the terms without a head redex. The recursion defining W terminates, and $W(t)$ is the last term of the maximal head reduction sequence from t. In particular $W(t)$ is head normal, W is idempotent, and $t \to^* W(t)$.*

(b) Left coherence. *$W(W(t)[\sigma]) \equiv W(t[\sigma])$ for every raw term t and every capture-free simultaneous substitution σ, and $W(W(A)[\sigma]) \equiv W(A[\sigma])$ for every raw formula A.*

(c) $W(t[\rho]) \equiv W(t)[\rho]$ for every renaming ρ of variables, and $\mathrm{FV}(W(t)) \subseteq \mathrm{FV}(t)$.

(d) W is a conversion normalizer. It violates the successor clause of (ν1) and (ν4): with $j_1 := I(0,0;\,u.\,u)$, $W(Sj_1) \equiv Sj_1 \not\equiv SW(j_1)$, and for $t \equiv Sx$ and $\sigma := [j_1/x]$,

$$W(t[\sigma]) \equiv Sj_1 \not\equiv S0 \equiv W\big(W(t)[W \circ \sigma]\big).$$

Proof. (a) Only an iterator $I(n,a;\,u.\,h)$ can have a head redex. It is itself a redex if $n \equiv 0$ or $n \equiv Sk$; otherwise its head redexes are those of n. By induction on the term there is at most one. Head reduction is a subrelation of $\to$, which terminates (Theorem 4.131). By induction on the term, a term has no head redex exactly when it is head normal: an iterator without one has a counter without one that is neither 0 nor a successor, hence neutral by the induction hypothesis. For the recursion argue by induction on the length of the maximal head reduction sequence from t, and then on the size of t. As long as the counter of $I(n,a;\,u.\,h)$ is not 0 or a successor, the head redex of the iterator is that of n; so the sequence first reduces n to $W(n)$, by the induction hypothesis for the smaller term n. If $W(n)$ is neutral, $I(W(n),a;\,u.\,h)$ is head normal. Otherwise the next step contracts the root, to a or to $h[I(k,a;\,u.\,h)/u]$, whose maximal sequences are shorter, and the induction hypothesis applies. The remaining assertions follow, since a head normal term is fixed by W.

(b) If $t \to_{\mathrm{h}} t'$ then $t[\sigma] \to_{\mathrm{h}} t'[\sigma]$: the image of a head context is a head context, the image of a zero redex is a zero redex with the image of its result, and, with u renamed away from σ and σ' the lifting of σ, $I(Sn,a;\,u.\,h)[\sigma] \equiv I(Sn[\sigma],a[\sigma];\,u.\,h[\sigma'])$ contracts to $h[\sigma'][I(n[\sigma],a[\sigma];\,u.\,h[\sigma'])/u] \equiv (h[I(n,a;\,u.\,h)/u])[\sigma]$ by the composition law of substitution. Hence $t[\sigma] \to_{\mathrm{h}}^* W(t)[\sigma]$. By (a) the head reduction from $t[\sigma]$ is deterministic, so its maximal sequence passes through $W(t)[\sigma]$, and the remainder of that sequence is the maximal sequence from $W(t)[\sigma]$; both end at $W(t[\sigma])$. For formulas apply this to each endpoint, with σ lifted below the binders.

(c) A renaming sends variables to variables, 0 to 0, successors to successors and neutral terms to neutral terms, so it sends head normal terms to head normal terms, and (b) gives $W(t[\rho]) \equiv W(W(t)[\rho]) \equiv W(t)[\rho]$. A contraction creates no free variable.

(d) Variables, 0 and $S0$ are head normal, which is ($\nu1^-$); ($\nu2$) is (c), ($\nu3$) is (a), and ($\nu5$) follows from (a) and Theorem 4.6. In the examples Sj_1 is head normal and $W(j_1) \equiv 0$. □

Relation to the transfer theorems and to the atomic-seed normalizer. Theorem 4.30 does not apply to W, which violates (ν1) and (ν4); Theorem 4.90 applies, since W is a conversion normalizer. Between the two lies the left coherence (b), which is Theorem 4.26(a) for W: normalizing a term *before* a substitution does not change the normal form of the instance, while normalizing the *substituted terms* does. The atomic-seed normalizer violates even left coherence: for its counterexample in Theorem 4.95, $\mathrm{nf_a}(\mathrm{nf_a}(t)[\sigma]) \equiv S0$ and $\mathrm{nf_a}(t[\sigma]) \equiv I(0, S0;\, u.\, 0)$. The two normalizers are incomparable: W performs both contractions, also with compound initializers, but only along the counter spine of the root; $\mathrm{nf_a}$ works at every depth, also below a successor, but performs only zero contractions with an atomic initializer. Both are reached by reductions of Theorem 4.5, so $\mathrm{nf}(W(t)) \equiv \mathrm{nf}(t)$ by Theorem 4.132.

Left coherence determines which raw metadata are needed. In an instance of a rule $R^{[W]}$ the normal forms of the premises and of the conclusion are $W(B[\sigma]) \equiv W(W(B)[\sigma])$ for the matrices and templates B of the instance and the substitutions σ that the schema applies. A raw matrix or template may therefore be replaced by its head normal form without changing the instance, while the terms that are substituted, namely the targets of QE, the witnesses of WI and the replacement and minor terms of VSUB, must remain raw. In $\mathrm{QE}^{[\mathrm{nf_a}]}$ the matrix itself must remain raw (Theorem 4.100).

4.10.2 Six rules

Definition 4.106 (The calculus VH)**.** VH is the raw-metadata calculus over W with the six rules $\mathrm{HYP}^{[W]}$, $\mathrm{REFL}^{[W]}$, $\mathrm{VSUB}^{[W]}$, $\mathrm{QE}^{[W]}$, $\mathrm{DE}^{[W]}$ and $\mathrm{WI}^{[W]}$. Explicitly, Σ is a list of head normal formulas, C is head normal, and x is a variable, while t, u, v, r, s and B are raw:

$$\frac{}{\Sigma \vdash C}\ (C \in \Sigma), \qquad \frac{}{\Sigma \vdash W(t) = W(t)}, \qquad \frac{\Sigma \vdash x = W(t) \qquad \Sigma \vdash W(u) = W(v)}{\Sigma \vdash W(v) = W\big(u[t/x]\big)},$$

$$\frac{\Sigma \vdash W(Q_y B) \qquad \Sigma \vdash W(B[0/y])}{\Sigma \vdash W(B[r/y])}, \qquad \frac{\Sigma \vdash W(D_{x,y}B) \qquad \Sigma,\ W\big(Q_y B[b/x]\big) \vdash C}{\Sigma \vdash C},$$

$$\frac{\Sigma,\ W\big(B[s/x][\eta_{B,s}(z)/y]\big) \vdash W\big(B[s/x][Sz/y]\big)}{\Sigma \vdash W(D_{x,y}B)}.$$

These are HYP, REFL, VSUB, QE, DE and WI. In DE the variable b is not free in Σ, in C or in $D_{x,y}B$; in WI the variable z is not free in Σ, in s or in $D_{x,y}B$, and $\eta_{B,s}(z) = z$ exactly when B is a literal equation and the raw witness s is literally 0, otherwise $\eta_{B,s}(z) = 0$. By left coherence the matrices may be stored in head normal form, as `head_conversion` does; the terms t, u, v, r, s are raw hints. There is no primitive computation equation, transport, case rule, induction, weakening, substitution or cut. The formula $0 = S0$ is head normal.

Lemma 4.107 (Structural operations)**.** *Let ν be a conversion normalizer and* K *a raw-metadata calculus over ν whose rules include $\mathrm{HYP}^{[\nu]}$. Weakening, injective renaming of free variables and cut on an assumption are admissible in* K *by direct induction on derivations; none of them substitutes a term in a derivation.*

Proof. For an injective renaming ρ, replace the raw metadata m of each inference by $m[\rho]$, after renaming its eigenvariable away from ρ. By (ν2) the normal forms of the premises and of the conclusion are the renamed ones; a renaming preserves the literal 0, the root of a matrix and the property of being a variable, so every test of Theorem 4.88 gives the same result, and an injective renaming preserves freshness. Weakening renames the eigenvariables away from the added formulas and keeps every inference. For cut, replace each use of the cut formula by HYP with its given derivation, weakened to the local context after the eigenvariables are renamed. □

Lemma 4.108 (Derived rules carry over)**.** *Let ν be a conversion normalizer, and let a rule R on raw sequents be derived from rules of Theorem 4.88 by a derivation, uniform in its raw metadata, whose open leaves are the premises of R. Then $R^{[\nu]}$, defined as in Theorem 4.88, is derivable in every raw-metadata calculus over ν in which the readings of the rules used are derivable.*

Proof. Apply to the given derivation the translation of Theorem 4.90(a): every sequent is replaced by its normal form and every inference by the reading of its rule with the same raw metadata. The open leaves become the premises of $R^{[\nu]}$ and the root its conclusion; only (ν2) and (ν3) are used. □

Lemma 4.109 (Progress and independent cases)**.** *Let E be a raw literal equation and z a variable not free in Σ or in Q_yE.*

(a) If $\Sigma, W(E[z/y]) \vdash_{\mathsf{VH}} W(E[Sz/y])$, then $\Sigma \vdash_{\mathsf{VH}} W(Q_yE)$.

(b) If $\Sigma \vdash_{\mathsf{VH}} W(E[0/y])$ and $\Sigma \vdash_{\mathsf{VH}} W(E[Sz/y])$, then $\Sigma \vdash_{\mathsf{VH}} W(E[r/y])$ for every raw term r.

Proof. (a) is the second part of the proof of Theorem 4.101, with W in place of nf_{a}: WI at the raw witness 0 for the matrix $E(y)$ with an unused x is in progress mode, and DE with HYP returns $W(Q_yE)$. (b) Weaken the second derivation by the unused assumption $W(E[z/y])$ (Theorem 4.107), apply (a), and apply QE with the raw body E at the target r. □

With the selector $\mathrm{sel}(v,a,b) := I(v,a;\, u.\, b)$, $u \notin \mathrm{FV}(b)$, of Section 1.1, put

$$Z(v) := \mathrm{sel}(v,1,0), \qquad p(v) := I(v,0;\, u.\, Z(u)).$$

By Theorem 4.104, for all raw n, a, b,

$$W(\mathrm{sel}(n,a,b)) \equiv \begin{cases} W(a), & W(n) \equiv 0, \\ W(b), & W(n) \text{ is a successor}, \end{cases} \qquad W(p(1)) \equiv 1, \qquad W(p(2)) \equiv 0.$$

The first is one root step after the counter, since the state variable is not free in b. For the third, $p(1)$ steps to $\mathrm{sel}(I(0,0;\, u.\, Z(u)),1,0)$, whose counter has the head normal form 0, so the result is $W(1) \equiv 1$. For the fourth, $p(2)$ steps to $\mathrm{sel}(I(1,0;\, u.\, Z(u)),1,0)$, whose counter has, by the third computation, the head normal form 1, so the result is $W(0) \equiv 0$. No statement about parity at arbitrary numbers is used.

Definition 4.110 (The packet)**.** For raw terms s, t, r and a raw template $h(w)$, and a variable x that is not free in s, t, r, h, the *packet* is the literal equation

$$\Pi(x,y) := \Big(\mathrm{sel}\big(y,0,\mathrm{sel}(p(y),x,h(x))\big) = \mathrm{sel}\big(y,0,\mathrm{sel}(p(y),t,r)\big)\Big),$$

with the state variables of the selectors chosen fresh.

Its counters are the variable y, so Π is head normal, and by the four computations its rows are

$$W(\Pi(x,0)) \equiv (0=0), \qquad W(\Pi(x,1)) \equiv W(h(x)=r), \qquad W(\Pi(x,2)) \equiv (x = W(t)).$$

The packet is the packet of Theorem 6.11 with the rows $h(x) = r$ and $x = t$, read on raw terms with head normalization.

Lemma 4.111 (Packet introduction)**.** *If $\Sigma \vdash_{\mathsf{VH}} W(s=t)$ and $\Sigma \vdash_{\mathsf{VH}} W(h(s)=r)$, then $\Sigma \vdash_{\mathsf{VH}} D_{x,y}\Pi(x,y)$.*

Proof. Apply WI at the raw witness s, in whichever mode it selects. Its premise has the conclusion $W(\Pi(s, Sz))$, which by the computation of $W(\mathrm{sel}(Sz, 0, b))$ is $W(F(p(Sz)))$ for the literal equation

$$F(v) := \big(\mathrm{sel}(v, s, h(s)) = \mathrm{sel}(v, t, r)\big), \qquad v \text{ fresh.}$$

Since $W(F(0)) \equiv W(s = t)$ and $W(F(Sz')) \equiv W(h(s) = r)$, the two given derivations and Theorem 4.109(b) at the raw target $p(Sz)$ derive it, after weakening by the local assumption, which is not used. The proof is the same in both modes, also when $s \equiv 0$. □

Lemma 4.112 (Transport)**.** $\mathrm{TR}^{[W]}$ *is derivable in* VH*: from* $\Sigma \vdash W(s = t)$ *and* $\Sigma \vdash W(h(s) = r)$ *infer* $\Sigma \vdash W(r = h(t))$*, for raw* s, t, r *and every raw template* h*.*

Proof. Theorem 4.111 gives $D_{x,y}\Pi$. Open it by DE at a fresh b, with the local assumption $Q_y\Pi(b, y)$, which is head normal by Theorem 4.105(c). Under it REFL gives $0 = 0$, the zero row, and QE at the targets 1 and 2 gives $W(h(b) = r)$ and $b = W(t)$. VSUB with the source b, the raw replacement t and the raw minor terms $u \equiv h(b)$ and $v \equiv r$ has exactly these two premises and the conclusion $W(r) = W(h(b)[t/b]) \equiv W(r = h(t))$, since b is not free in h. The conclusion contains no b, and DE discharges it. □

The replacement must be the raw t. The stored row is $b = W(t)$, and a substitution of its right side would give $W(r = h(W(t)))$, which differs from $W(r = h(t))$ when h places its variable below a successor, because W does not normalize there.

By Theorem 4.108 applied to the derivations of Theorem 1.18, which use reflexivity and TR only, the readings with raw metadata of symmetry, transitivity, congruence and transport in literal equations are derivable in VH.

Lemma 4.113 (The introductions)**.** $\mathrm{QI}^{[W]}$ *and* $\mathrm{DI}^{[W]}$ *are admissible in* VH*.*

Proof. $\mathrm{QI}^{[W]}$ is Theorem 4.109(a). An instance of $\mathrm{DI}^{[W]}$ is an instance of WI unless its matrix B is a literal equation and its raw witness is literally 0. Let $\Sigma, W(B(0,0)) \vdash W(B(0, Sz))$ be derivable and put $j := I(0, 0; u.\, 0)$, which is not literally 0, with $W(j) \equiv 0$. In the context $\Sigma_j := \Sigma, W(B(j, 0))$, REFL at 0 derives $0 = 0$, which is both $W(j = 0)$ and $W(0 = j)$. Transport in the literal equation $B(w, 0)$ along $W(j = 0)$ gives $W(B(0, 0))$; cut with the given derivation, weakened (Theorem 4.107), gives $W(B(0, Sz))$; transport in $B(w, Sz)$ along $W(0 = j)$ gives $W(B(j, Sz))$. WI at the raw witness j is in case mode, and z is fresh for the closed j; it derives $W(D_{x,y}B)$. □

Unlike Theorem 4.101, the normalized branch at j is in general not literally the branch at 0: for the matrix $Sx = Sx$ the assumption $W(B(j, 0))$ is $Sj = Sj$. The repair therefore uses derived transport, as in Theorem 4.3(ii).

Theorem 4.114 (Weak-head presentation)**.** VH *derives exactly the sequents over* W *that* N *derives. For every finite context* Γ *of raw native formulas and every raw native formula* A*,*

$$\Gamma \vdash_{\mathsf{N}} A \quad \Longleftrightarrow \quad W(\Gamma) \vdash_{\mathsf{VH}} W(A),$$

effectively on finite derivations, and Theorem 4.92 applies to VH*: it is equiconsistent and bi-interpretable with* iPRA*, with whole numerical domains, equality interpreted by equality and numerical composites with the graph* $x = y$*.*

Proof. W is a conversion normalizer (Theorem 4.105(d)), and VH is a raw-metadata calculus over W. By Theorem 4.90(b) it suffices that the eight rules of $\mathsf{N}^{[W]}$ are admissible in VH. HYP, QE and DE are rules of VH. An instance of $\mathrm{I0}^{[W]}$ is $W(I(0, a; u.\, h)) = W(a)$, which is $W(a) = W(a)$ by Theorem 4.104, an instance of REFL; an instance of $\mathrm{IS}^{[W]}$ is $W(I(Sn, a; u.\, h)) = W(h[I(n, a; u.\, h)/u])$, and its left side is its right side because $W(Sn) \equiv Sn$. TR is Theorem 4.112, and QI and DI are Theorem 4.113. □

Remark 4.115. On full normal forms the same six rules are the calculus of Theorem 6.51, where the rows of the packet are literal; the theorem shows that the head normalization of the endpoints suffices, provided the hints stay raw. `head_conversion` proves the equivalence directly against N: a derivation of N is translated rule by rule, with the computation equations as reflexivities and (b) of Theorem 4.105 for the instances of the matrices; a derivation of VH is simulated in N by the conversions $t = W(t)$. The equivalence is not formalized in Lean or Rocq. The checker of the package substitutes the raw witness of WI first and normalizes the formula before it opens the other coordinate, an order that left coherence permits; the mode test examines the raw witness.

4.10.3 Conversion bounded to three counter levels

This subsection draws on `bounded_head` and on `bounded_conversion`, which presents the same calculus independently (Theorem 4.124). It removes the normalizer of VH altogether: terms and formulas stay raw, and the conversion becomes one primitive rule that performs a single certified contraction, in either direction, at a bounded depth of the counter spine of the left endpoint.

Definition 4.116 (The calculus BH_3)**.** A *counter context of depth* d is $H_0 := [\,]$ or $H_{d+1} := I(H_d, a; u.\, h)$ with arbitrary raw a, h; these are the head contexts of Theorem 4.104 with d nested counters. For raw terms s, s' write $s \leftrightsquigarrow_3 s'$ if there are $d \leq 3$, a counter context H_d and a zero or successor redex p with contractum q (Theorem 4.5) such that $\{s, s'\} = \{H_d[p], H_d[q]\}$. The calculus BH_3 has raw native formulas, contexts of them, ordinary substitution, and the seven rules HYP, REFL, VSUB, QE, DE, WI of N and Theorem 4.88, read on raw terms without normalization, and

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash s' = t}\ \mathrm{HC}_3, \qquad s \leftrightsquigarrow_3 s'.$$

An application of HC_3 carries its depth, the direction and the redex; no normal form is computed.

The first six rules are the rules of VH with the identity in place of W, which is an admissible normalizer. If $s \leftrightsquigarrow_3 s'$, then one of s, s' is a head reduct of the other: in $H_d[p]$ every counter above p is an iterator, so p is the head redex. Hence $W(s) \equiv W(s')$, and replacing every formula of a derivation of BH_3 by its head normal form turns each HC_3 into a repetition and each other rule into the rule of VH with the same raw hints. Three things change. The conversion is an inference and not an operation on every formula, so the right endpoint and the formulas below binders are not converted by the calculus. One application performs one step, so the counter spine is never evaluated to the end; checking is primitive recursive (Theorem 4.121). And the depth is bounded, so only the conversions that the completeness proof uses directly have to be primitive; all others are derived after transport.

Lemma 4.117 (Structural operations and symmetry)**.** *Weakening, injective renaming and cut on an assumption are admissible in* BH_3*, and so is symmetry: if* $\Gamma \vdash s = t$ *then* $\Gamma \vdash t = s$*. Hence conversion of the right endpoint by* $\leftrightsquigarrow_3$ *is admissible.*

Proof. The structural operations are proved as in Theorem 4.107, with the identity as normalizer; an injective renaming sends an instance of HC_3 to an instance of the same depth. For symmetry, let x be any variable. REFL derives $\Gamma \vdash x = x$, and VSUB with the source x, the replacement x, the first premise $x = x$ and the minor premise $s = t$ concludes $\Gamma \vdash t = s[x/x] \equiv t = s$; this holds whether or not x occurs in s. The last statement follows: symmetry, a conversion of the left endpoint, and symmetry again. The proof of symmetry uses no conversion. □

Remark 4.118. The derivation of symmetry needs only REFL and VSUB and holds in every calculus of Theorem 4.88 that has these two rules, since a variable is its own normal form by $(\nu 1^{-})$. `bounded_head` derives symmetry instead through the formula $D_{x,y}(\mathrm{sel}(y, 0, x) = 0)$, introduced by WI at the witness 0 and opened by DE at a fresh b, which yields $b = 0$ by conversions of depth 0, and then applies VSUB with the replacement 0. `bounded_conversion` declares, for every sequent, a finite set of free variables in which all terms must be well scoped (Theorem 4.124); when that set is empty no variable is available as a source. It then introduces $D_{x,y}(0 = 0)$ by WI at the witness 0, in progress mode, from the premise $0 = 0 \vdash 0 = 0$ given by REFL; opens it by DE at a fresh b, which extends the set of variables by b; applies the derivation above with $x := b$ to the weakened premise; and lets DE discharge b, which is not free in $t = s$. Neither detour uses a conversion of depth greater than 0, and the second uses none.

In VH symmetry is a consequence of transport, because W acts on both endpoints; in BH_3 it has to precede transport, since the packet introduction converts right endpoints.

Lemma 4.119 (Packet and transport on raw terms)**.** *In BH_3 the statements of Theorems 4.109, 4.111 and 4.112 hold with W replaced by the identity. In particular TR is derivable, and with it symmetry, transitivity, congruence and transport in literal equations (Theorem 1.18).*

Proof. The proofs of the three lemmas apply word for word with W the identity, once each identity $W(e) \equiv W(e')$ between equations that they use is replaced by a derivation of e' from e: steps of HC_3 on the left endpoint and, by Theorem 4.117, on the right endpoint. The proof of Theorem 4.109 uses no such identity. The proof of Theorem 4.111 uses $F(0) \leftrightarrow (s = t)$, $F(Sz') \leftrightarrow (h(s) = r)$ and $F(p(Sz)) \leftrightarrow \Pi(s, Sz)$, one step at the root on each side, since the state variables are not free in the bodies of the selectors. The proof of Theorem 4.112 uses the three rows of the packet. The row at 0 is one step at the root on each side. For the rows at 1 and 2, each endpoint first contracts at the root to $\mathrm{sel}(p(k), a, c)$, with (a, c) equal to $(b, h(b))$ on the left and to (t, r) on the right; write $p_0 := I(0, 0;\, u.\, Z(u))$. At $k = 1$ the counter is computed by

$$p(1) \to Z(p_0) \to Z(0) \to 1$$

at the depths 1, 2 and 1: the first step contracts $p(1)$, the counter of the selector; the second contracts p_0, the counter of Z inside that counter; the third contracts $Z(0)$. The final step $\mathrm{sel}(1, a, c) \to c$ is at the root. At $k = 2$,

$$p(2) \to Z(p(1)) \to Z(Z(p_0)) \to Z(Z(0)) \to Z(1) \to 0$$

at the depths 1, 2, 3, 2 and 1, the deepest step contracting p_0 below the selector and two copies of Z; then $\mathrm{sel}(0, a, c) \to a$ at the root. Every step has depth at most 3, and the terms a and c are never touched. The resulting rows are $0 = 0$, $h(b) = r$ and $b = t$, literally, and VSUB with the raw replacement t concludes $r = h(t)$. The last statement is Theorem 4.108 with the identity normalizer. □

The depth bound is the depth of the evaluation of the parity term $p(2)$ inside a selector: one level for the selector and two for the copies of Z. Every other conversion of the completeness proof is a step at the root. The bound three is therefore sufficient for this packet; that it is necessary is not claimed, and another parity term changes the bound.

Theorem 4.120 (Bounded conversion)**.** *For every finite context Γ of raw native formulas and every raw native formula A,*

$$\Gamma \vdash_{\mathsf{BH}_3} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{N}} A,$$

effectively on finite derivations. Hence the statements of Theorem 2.2 hold for BH_3: it is equiconsistent and bi-interpretable with iPRA, with whole numerical domains, equality interpreted by equality and numerical composites with the graph $x = y$.

Proof. From N to BH_3, by induction on derivations. HYP, QE and DE are rules. I0 is REFL at a followed by one expansion at the root of the left endpoint to $I(0, a;\, u.\, h)$, and IS is REFL at $h[I(n, a;\, u.\, h)/u]$ followed by one expansion to $I(Sn, a;\, u.\, h)$. TR is Theorem 4.119. QI is Theorem 4.109(a) with W the identity. For DI only a literal equation B at the witness 0 is not an instance of WI; the proof of Theorem 4.113 applies with W the identity, where $j = 0$ follows from $0 = 0$ by one expansion at the root of the left endpoint, $0 = j$ by Theorem 4.117, and transport in literal equations and cut are those of Theorems 4.117 and 4.119.

From BH_3 to N, by induction on derivations: the six rules other than HC_3 are sound by Theorem 4.89 for the identity normalizer. For HC_3, N derives the equation between s and s' by Theorem 4.6, since one is a one-step reduct of the other, and symmetry and transitivity (Theorem 1.18) turn $s = t$ into $s' = t$. The last statement is Theorem 2.2. □

Proposition 4.121 (What normalization costs, and what the bound saves)**.** *The relation "the finite certificate c is a derivation of the sequent S in BH_3" is primitive recursive. No primitive recursive function decides, for every closed raw term t, whether the value of t is 0; in particular W, restricted to closed terms, is not primitive recursive.*

Proof. A certificate is a finite tree with one node per inference. Each node is checked by tests of literal identity, scope and freshness, by substitutions, and, for HC_3, by one contraction at a supplied address of length at most three, which builds its contractum and evaluates nothing further. Each test is primitive recursive, and the number of nodes bounds the number of checks. The sequents that these tests compare need not be part of the certificate: in the checkers of `bounded_head` and `bounded_conversion` a node records its raw metadata only, and the sequents of its premises are computed from the sequent of the node and these metadata by at most a constant number k of substitutions, shifts of variables and one contraction, the context growing by at most one formula. An instance of a term or formula of size at most N at terms of size at most N has size at most N^2, and so has a contractum, since the body and the recursive iterator are subterms of the redex. If m is the size of the input, every formula at a node of depth d therefore has size at most $m^{2^{kd}} \le m^{2^{km}}$, and every context has at most $2m$ formulas. The length of the whole check and the size of its configurations are thus bounded by a primitive recursive function of m, and a computation with such bounds is primitive recursive; no search for a normal form occurs.

For the second statement suppose that ζ is such a function. Enumerate the raw terms with at most the free variable x primitive recursively, replacing a code that is not well formed by the code of 0, and put $d(e) := \zeta(\ulcorner t_e[\bar{e}/x] \urcorner)$, which is primitive recursive and takes the values 0 and 1. By Theorem 1.102 some t_k has the value $d(n)$ at every n; then $d(k) = 1$ exactly when $d(k)$, the value of $t_k[\bar{k}/x]$, is 0, a contradiction. A closed head normal term is 0 or a successor, because a neutral term has a free variable at the end of its counter spine, and W preserves the value by (ν5); so $W(t) \equiv 0$ exactly when the value of t is 0, and a primitive recursive W would give ζ. □

The second statement refines Theorem 4.145, whose argument it repeats with the zero test in place of the value: head normalization does not expand a closed term to a numeral, and is still not primitive recursive. The checkers of VH, F and N^{f} contain such a normalizer; the checker of BH_3 does not, at the price of certificates that list every conversion step. As for Theorem 4.145, this is a statement about idealized programs without resource limits.

The contrast concerns the derivation relations themselves, and not only the normalizer: one inference of VH with a fixed matrix already decides whether a closed term has a positive value.

Proposition 4.122 (Checking VH is not primitive recursive)**.** *Put $B(y) := (\mathrm{sel}(y, 0, 0) = \mathrm{sel}(y, 0, 1))$. For a closed raw term t let c_t be the derivation of VH that consists of one*

instance of $\textsc{qe}^{[W]}$ *with the raw matrix* B *and the raw target* t*, whose first premise is closed by* HYP *and whose second by* REFL *at* 0*. Then* c_t *is a derivation of the sequent*

$$Q_y B(y) \vdash 0 = 1$$

exactly when the value of t *is positive. Consequently the relation "the finite certificate* c *is a derivation of the sequent* S *in* VH*" is not primitive recursive, already for this fixed sequent and certificates of this fixed shape.*

Proof. The endpoints of B are iterators with the counter y, hence neutral, so B, $Q_y B$ and $0 = 1$ are head normal and the sequent is a sequent over W. The first premise of the instance is $W(Q_y B) \equiv Q_y B$, which is in the context. By the computation of $W(\mathrm{sel}(n, a, b))$ that precedes Theorem 4.110, with b equal to 0 or 1 and so without the state variable,

$$W(\mathrm{sel}(n,0,0)) \equiv 0, \qquad W(\mathrm{sel}(n,0,1)) \equiv \begin{cases} 0, & W(n) \equiv 0, \\ 1, & W(n) \text{ is a successor.} \end{cases}$$

At $n \equiv 0$ the second premise is therefore $W(B(0)) \equiv (0 = 0)$, which is $W(0) = W(0)$, an instance of REFL. The conclusion of the instance is $W(B(t))$, which is determined by its raw metadata. As shown in the proof of Theorem 4.121, $W(t)$ is 0 or a successor for a closed t, and $W(t) \equiv 0$ exactly when the value of t is 0. Hence $W(B(t)) \equiv (0 = 0)$ if the value of t is 0, and $W(B(t)) \equiv (0 = 1)$ otherwise; c_t derives $Q_y B(y) \vdash 0 = 1$ in the second case only. The map from the code of t to the codes of c_t and of the sequent is primitive recursive. If the relation were primitive recursive, $\zeta(t) := 1 \dot{-} \chi(t)$, with $\chi(t)$ the characteristic value of "c_t derives $Q_y B(y) \vdash 0 = 1$", would be a primitive recursive function that decides for every closed raw term whether its value is 0, against the second statement of Theorem 4.121. □

The context $Q_y B(y)$ is false in the standard model, since its closure at a positive instance gives $0 = 1$; an open-sequent interface admits such contexts, and c_t is sound by Theorem 4.89. The proposition concerns the acceptance of these particular certificates, not the derivability of the sequent by some other derivation, and it is not a lower bound for the verification of derivations of iPRA in general: the derivation relation of BH_3 is primitive recursive (Theorem 4.121), and by the same argument so are those of N and of the conventional presentations of iPRA, whose side conditions perform no computation. The checker of VH in `head_conversion` compares $W(B(t))$ with the goal and so realizes this relation. `bounded_conversion` proves the proposition by repeating the diagonal argument with the positivity test in place of the zero test; the reduction to Theorem 4.121 is the article's.

Remark 4.123. `bounded_head` states the seven rules, the derived symmetry, the packet with its depth count and the equivalence with N, and the two statements of Theorem 4.121; the article reuses for them the packet and transport arguments of VH. Its symbolic transport certificate uses $\textsc{hc}_3$ at the depths 0, 1, 2 and 3 respectively 60, 8, 6 and 2 times. The rule $\textsc{hc}_3$ is not sound for the colour reading of Section 4.10.4: at the root it derives $x = y$ from $I(0, x;\, u.\, u) = y$. A conversion of an endpoint inside an equation combines a computation with an equation, which the computation equations together with VSUB cannot do (Theorem 4.127). The theorem is not formalized in Lean or Rocq.

Remark 4.124 (The presentation of `bounded_conversion`). `bounded_conversion` presents a calculus $\mathcal{L}_3$ with the seven rule families HYP, REFL, V, Q, E, I and Conv, and proves $\Gamma \vdash_{\mathcal{L}_3} A \Leftrightarrow \Gamma \vdash_{\mathsf{N}} A$ for all raw open sequents. It is BH_3. V is VSUB on raw terms, also when the replacement contains the source and with no condition on which variable is the source; Q, E and I are QE, DE and WI, with the selector $\eta_{B,s}$ of Theorem 4.106 tested on the raw matrix and the raw witness; and Conv, from $H[\ell] = r$ infer $H[\ell'] = r$ or conversely, for a counter context H of depth at most 3 and a zero or successor contraction $\ell \to \ell'$, is $\textsc{hc}_3$. The differences are of presentation.

(a) Every sequent carries a finite set of free variables, its numerical environment, in which all terms and formulas must be well scoped, including every supplied redex and the parts of it that a contraction erases; the eigenvariables of E and I extend the environment of their premises. This affects only the derivation of symmetry (Theorem 4.118).

(b) The proof of the equivalence follows the same steps as that of Theorem 4.120: the structural operations, symmetry, equation progress (Theorem 4.109), the packet of Theorem 4.110 with the rows $0 = 0$, $h(x) = r$ and $x = t$ reached at the counter depths 0; $0, 1, 2, 1, 0$; and $0, 1, 2, 3, 2, 1, 0$ (Theorem 4.119), transport, the repair at a literal equation and the witness 0 by the alias $I(0, 0;\, u.\, 0)$ (Theorem 4.113), and the soundness of Conv by the computation equations, congruence through H and transport. Like the article, it uses no normalization, confluence or strong normalization theorem, and it substitutes no compound term in a derivation. It then composes the equivalence with Theorem 1.119, which is the last statement of Theorem 4.120.

(c) In its checker a conversion record supplies the depth and one term, which is either the left endpoint of the goal or the left endpoint of the premise; the checker contracts that term at that depth and requires the left endpoint of the goal to be the term or its contractum, so that no direction bit is needed. All other records keep the formats of the checker of VH in `head_conversion`, with raw substitution and raw stored formulas.

The symbolic transport certificate of `bounded_conversion` derives $s = t,\ S(s) = r \vdash r = S(t)$ with three free parameters. It has 7 806 bytes and uses 34 conversions, two of them at depth 3, with 36 instances of DE, 36 of WI, 35 of VSUB and three of QE; the package attributes the repeated discharges to the symmetry detour of Theorem 4.118. The symbolic transport certificate of `bounded_head` uses 76 conversions (Theorem 4.123). The equivalence is a written proof and is not formalized in Lean or Rocq.

4.10.4 Raw computation equations do not replace conversion

On raw terms with the computation equations as axioms, substitution for a variable is not a complete equality rule. The proof is a reading in which the free variables carry colours.

Definition 4.125 (The colour reading)**.** Let $V := \mathbb{N} \times \{0, 1, 2\}$ and $V^+ := \mathbb{N} \times \{1, 2\}$, with $\pi(n, c) := n$ and $\iota(n) := (n, 0)$; the colour 0 is *neutral*. For a map ρ from variables to V let $[\![x]\!]_\rho := \rho(x)$ for a variable x, and $[\![t]\!]_\rho := \iota(t^{\mathbb{N}}[\pi \circ \rho])$ for every term t that is not a variable, where $t^{\mathbb{N}}[\pi \circ \rho]$ is the standard value of t under the numerical valuation $\pi \circ \rho$. Put

$$R\big((n, c), (m, d)\big) :\Leftrightarrow n = m \text{ and } (c = 0 \text{ or } d = 0 \text{ or } c = d),$$

and define truth $T(A, \rho)$ by

$$\begin{aligned} T(s = t, \rho) &:\Leftrightarrow R([\![s]\!]_\rho, [\![t]\!]_\rho), \\ T(Q_y B, \rho) &:\Leftrightarrow \big(T(B, \rho[y \mapsto \iota(0)]) \Rightarrow \forall n\, T(B, \rho[y \mapsto \iota(n)])\big), \\ T(D_{x,y} B, \rho) &:\Leftrightarrow \exists m\, T(Q_y B, \rho[x \mapsto \iota(m)]), \end{aligned}$$

and, for a clause, $T([A;\, y.\, B(y)], \rho) :\Leftrightarrow \big(T(A, \rho) \Rightarrow \forall n\, T(B, \rho[y \mapsto \iota(n)])\big)$. A sequent, with a context of formulas or of clauses, is *colour-valid* if every map ρ from its free variables to V^+ that makes every member of the context true makes its conclusion true.

The bound variables of Q, D, and clauses receive neutral values, the free variables coloured ones. R is reflexive and symmetric and is not transitive: $(n, 1)$ and $(n, 2)$ are both related to $\iota(n)$ and not to each other. The reading is a device for proving that sequents are not derivable; it is not a structure with equality interpreted by equality.

Lemma 4.126 (Properties of the reading)**.** *Let ρ be a map from variables to V.*

(a) Substitution. $[\![t[s/x]]\!]_\rho \equiv [\![t]\!]_{\rho[x\mapsto[\![s]\!]_\rho]}$ *and* $T(A[s/x],\rho) \Leftrightarrow T(A,\rho[x\mapsto[\![s]\!]_\rho])$ *for capture-free substitutions.*

(b) Compound formulas ignore colours. *If A has the root Q or $D_,$ and $\pi\circ\rho = \pi\circ\rho'$, then $T(A,\rho) \Leftrightarrow T(A,\rho')$.*

(c) Neutral instances suffice. *If $T(B,\rho[y\mapsto\iota(n)])$ for every n, then $T(B,\rho[y\mapsto w])$ for every $w\in V$.*

(d) Coloured lift. *If E is a literal equation and $T(E,\rho[y\mapsto\iota(n)])$, then $T(E,\rho[y\mapsto(n,c)])$ for some $c\in\{1,2\}$.*

(e) Erasure. *If $R((n,c),w)$ then $R(\iota(n),w)$.*

Proof. Note first that $\pi[\![s]\!]_\rho = s^{\mathbb{N}}[\pi\circ\rho]$ for every term, and that $R(\iota(n),w)$ holds exactly when $\pi(w)=n$.

(a) For the term statement, if $t\equiv x$ both sides are $[\![s]\!]_\rho$; if t is another variable both are its value; otherwise $t[s/x]$ is not a variable, and both sides are ι of the standard value of t under $\pi\circ\rho$ with x sent to $s^{\mathbb{N}}[\pi\circ\rho]$. The formula statement follows by induction, with bound variables renamed away from s and x.

(b) By induction on the matrix B simultaneously for Q_yB and $D_{x,y}B$; the case $D_{x,y}B$ reduces to Q_yB at the neutral values $\iota(m)$, which do not depend on ρ. For Q_yB with a compound B the induction hypothesis applies to each $\rho[y\mapsto\iota(n)]$. Let B be the equation $s=t$. If a side is not a variable, it is neutral, and by the note each $T(B,\rho[y\mapsto\iota(n)])$ depends on $\pi\circ\rho$ only. If both sides are variables other than y, the closure is $P\Rightarrow P$ for one truth value P and holds. If both are y, every instance is true. If one side is y and the other a variable $a\neq y$, the instance at $\iota(n)$ holds exactly when $n=\pi\rho(a)$, which depends on numbers only.

(c) For a compound B use (b) with the value $\iota(\pi w)$. For an equation B: if no side is the bare variable y, or a side is not a variable, the truth of B at $\rho[y\mapsto w]$ is its truth at $\rho[y\mapsto\iota(\pi w)]$, by the note; $y=y$ is true. In the remaining case B is $y=a$ or $a=y$ with a variable $a\neq y$, and the instances at $\iota(0)$ and $\iota(1)$ would give $\pi\rho(a)=0$ and $\pi\rho(a)=1$; the hypothesis is false.

(d) If E is $y=a$ or $a=y$ with a variable $a\neq y$, then $\pi\rho(a)=n$; take for c the colour of $\rho(a)$ if it is not neutral and $c=1$ otherwise. In every other case the truth of E does not depend on the colour of the value of y, as in (c).

(e) $n=\pi(w)$, and the neutral colour is related to every colour. □

Theorem 4.127 (Substitution for a variable on raw terms)**.** *The following rules, with the side conditions of* N *and of the clause calculi, preserve colour-validity.*

(i) On raw native sequents: HYP, REFL, I0, IS, VSUB, *symmetry (from $u=r$ infer $r=u$), substitution for a variable in a formula (from $x=t$ and A infer $A[t/x]$, for a variable x),* QE, DE, DI, QI *and* WI.

(ii) On raw clause sequents, whose formulas are equations and formulas with the root $D_,$: REFL, VSUB *and symmetry as in (i),* COMPUTE, USE, OPEN *and* CLOSE *of Theorem 6.56, and the instances of* PARAM *in the first and in the third case of ψ, that is, all instances except those that join at the root.*

Consequently no calculus whose rules are among these derives

$$I(0,x;\,u.\,u)=y \;\vdash\; x=y, \qquad \textit{nor} \qquad \mathrm{ord}(I(0,x;\,u.\,u)=y) \;\vdash\; x=y,$$

whereas N *derives the first sequent and the calculus $\mathcal{P}$ of Theorem 6.56 derives the second (Theorem 6.64).*

Proof. Fix a map ρ to V^+ of the free variables that makes the context of the conclusion true; for a rule with an eigenvariable, extend it as stated.

HYP is immediate, and REFL and symmetry hold because R is reflexive and symmetric. In an instance of I0, IS or COMPUTE the left side is an iterator, hence neutral, and the two sides have the same standard value.

VSUB: let $x = t$ and $u = v$ be true; the conclusion is $v = u[t/x]$, and $[\![u[t/x]]\!]_\rho \equiv [\![u]\!]_{\rho'}$ with $\rho' := \rho[x \mapsto [\![t]\!]_\rho]$ by Theorem 4.126(a). If t is a variable, its value and that of x lie in V^+ and are related, so they are equal, and $\rho' = \rho$. Otherwise $[\![t]\!]_\rho = \iota(\pi\rho(x))$, so ρ' erases the colour of x and keeps its number. Then $[\![u]\!]_{\rho'} = [\![u]\!]_\rho$ unless $u \equiv x$, since a term that is not a variable has a neutral value determined by numbers; for $u \equiv x$ the value $\iota(\pi\rho(x))$ is related to $[\![v]\!]_\rho$ by (e). In every case symmetry of R gives the conclusion. The first case of ψ in PARAM is VSUB, and its third case concludes $r = u$ from $u = r$, which is symmetry. For substitution in a formula the same ρ' arises; a compound A is unaffected by (b), and in an equation erasing the colour of a bare occurrence of x preserves R by (e).

QE and USE: the premises give $T(B, \rho[y \mapsto \iota(n)])$ for every n (for QE the base is the instance at $\iota(0)$, since 0 is neutral). By (c) and (a), $T(B[t/y], \rho)$.

DE and OPEN: the major premise gives m with $T(Q_yB, \rho[x \mapsto \iota(m)])$. Put $\rho' := \rho[b \mapsto (m, 1)]$, which makes the context true, since b is not free in it. The local assumption $Q_yB(b, y)$, or the opened clause $[B(b, 0);\, y.\, B(b, y)]$, whose reading is the same, is true under ρ': by (a) it is $T(Q_yB, \rho[x \mapsto (m, 1)])$, and by (b) this is $T(Q_yB, \rho[x \mapsto \iota(m)])$, since b is not free in $D_{x,y}B$. The minor premise gives the conclusion under ρ', and b is not free in it.

DI, and WI or CLOSE in case mode: let $w := [\![t]\!]_\rho$ for the witness t and assume $T(B, \rho[x \mapsto w, y \mapsto \iota(0)])$, which is $T(B(t, 0), \rho)$ by (a). For each n the map $\rho[z \mapsto (n, 1)]$ makes the context and $B(t, 0)$ true, since z is fresh, so the premise gives $B(t, Sz)$, which is $T(B, \rho[x \mapsto w, y \mapsto \iota(n + 1)])$. Hence $T(Q_yB, \rho[x \mapsto w])$, and by (b) $T(Q_yB, \rho[x \mapsto \iota(\pi w)])$: the number πw is a witness for $D_{x,y}B$.

QI, and WI or CLOSE in progress mode at the witness 0: let the literal equation $E(y)$, respectively $E(0, y)$, be true at $\iota(0)$. By induction on n in the metatheory, if it is true at $\iota(n)$, (d) gives a colour c with truth at (n, c); the map $\rho[z \mapsto (n, c)]$ makes the context and the local assumption true, by (a), and the premise gives the truth at $\iota(n+1)$. Hence the closure holds, and for WI the number 0 is a witness. No induction for a compound formula is used.

By induction on derivations every derivable sequent is colour-valid. Let $\rho(x) := (0, 1)$ and $\rho(y) := (0, 2)$. The iterator $I(0, x;\, u.\, u)$ is not a variable and has the standard value 0, so its value $\iota(0)$ is related to $(0, 2)$, and the assumption, and its assumption clause $[0 = 0;\, v.\, \cdot]$, are true; the conclusion $R((0, 1), (0, 2))$ is false. N derives the sequent by I0, symmetry and transitivity (Theorem 1.18), and Theorem 6.64 transfers it to $\mathcal{P}$, where τ is the identity on equations. □

Corollary 4.128 (Substitution for a variable alone is strictly weaker)**.** *On raw terms, each of the following derives strictly fewer sequents* $\mathrm{ord}(\tau\Gamma) \vdash \tau A$*, respectively* $\Gamma \vdash A$*, than* $\mathcal{P}$*, respectively* N*: the calculus* $\mathcal{P}$ *with* PARAM *restricted to sources that are variables, with or without the reversal of the third case of* ψ *and with or without* REFL*; the five rules of the calculus* $\mathcal{V}$ *of Theorem 6.48 read on raw terms, with* COMPUTE *added; and* N *with* TR *replaced by* VSUB *and* REFL*, also with formula-wide substitution for a variable added.*

Proof. Each calculus derives only sequents derived by the larger one: the rules of the first are instances of rules of $\mathcal{P}$, with REFL derived in $\mathcal{P}$ (Theorem 6.57); the rules of $\mathcal{V}$ are REFL, VSUB, USE, ELIM and INTRO, with ELIM as OPEN and INTRO as CLOSE, and VSUB is the first case of PARAM; and the rules of the last are derivable in N, VSUB as one TR (proof of Theorem 4.89) and formula-wide substitution by Theorem 1.22. All their rules preserve colour-validity by Theorem 4.127, which also gives the separating sequent. □

Remark 4.129 (Scope of the separation). The separating sequent has an assumption. Every sequent with the empty context that N derives is colour-valid, so the reading does not separate theorems. Under a map ρ to V^+, a formula with the root Q or $D_!$ has, by Theorem 4.126(b), its truth value under the neutral map $\iota \circ \pi \circ \rho$, at which R is numerical equality and T is the standard reading; and an equation is true exactly when it is true in the standard reading, unless it equates two distinct variables, which N does not derive. N is sound for the standard reading. The reading also does not exclude another translation into a restricted language, or another interpretation. If the zero computation were a conversion, the assumption of the separating sequent would become $x = y$: the reading does not respect conversion, since the iterator is neutral and its normal form x is coloured. This is the reason why Theorem 6.50 on full normal forms and Theorem 4.114 on head normal forms do not carry over to raw terms with the computation equations as axioms. `head_conversion` states the theorem for the native rules of (i) and the formula-wide corollary, and records 125 256 finite checks of the reading; the clause rules of (ii) and Theorem 4.128 are the application of the same reading to the calculi of Sections 6.5.1 and 6.5.3.

4.11 Full normalization

This section draws on `full_normalization` and `computation_normal`. The two packages arrive at the same six-rule calculus on full normal forms; they differ in the proof of termination, in the repair of the single introduction and in their measurements. Both computations are moved into the syntax, and no computation rule remains.

4.11.1 Termination and unique normal forms

The reduction $\to$ is that of Theorem 4.5. It is stable under substitution: if $t \to t'$, then $t[\sigma] \to t'[\sigma]$; and if $\sigma(x) \to^* \sigma'(x)$ for every x, then $t[\sigma] \to^* t[\sigma']$. A term is *strongly normalizing* if it has no infinite reduction sequence. A term has finitely many one-step reducts, so for a strongly normalizing term t the reduction sequences from t have a maximal length $d(t)$. The number of nodes of t is $|t|$.

Lemma 4.130 (Iterator closure). *Let n and a be strongly normalizing, and let $h[v/u]$ be strongly normalizing for every strongly normalizing term v. Then $I(n, a; u.\, h)$ is strongly normalizing.*

Proof. With $v \equiv u$ the body h is strongly normalizing. Argue by induction on the triple $(d(n), |n|, d(a)+d(h))$ in the lexicographic order; it suffices to show that every one-step reduct of $I(n, a; u.\, h)$ is strongly normalizing. A reduction in n decreases the first component. A reduction in a or in h leaves the first two components and decreases the third; if $h \to h'$, then $h'[v/u]$ is a reduct of $h[v/u]$ for every v, so the hypothesis on the body persists. If $n \equiv 0$, the root reduct is a. If $n \equiv Sp$, then $d(p) = d(n)$ and $|p| < |n|$, so $I(p, a; u.\, h)$ is strongly normalizing by the induction hypothesis, and the root reduct $h[I(p, a; u.\, h)/u]$ is strongly normalizing by the hypothesis on the body. □

Theorem 4.131 (Strong normalization). *If σ assigns strongly normalizing terms to the variables, then $t[\sigma]$ is strongly normalizing for every raw term t. In particular every raw term is strongly normalizing.*

Proof. By induction on t. Variables, 0 and successors are immediate. For $I(n, a; u.\, h)$, with u renamed away from σ, the instances $n[\sigma]$ and $a[\sigma]$ are strongly normalizing by the induction hypotheses. The induction hypothesis for h applies to every extension of σ by an assignment of a strongly normalizing v to u; this is the hypothesis on the body in Theorem 4.130 for the term $I(n[\sigma], a[\sigma]; u.\, h[\sigma])$. The last statement is the case of the identity substitution. □

Theorem 4.132 (Confluence)**.** *The reduction $\to$ is confluent. Hence every raw term t has a unique normal form* $\mathrm{nf}(t)$*, up to the renaming of bound variables.*

Proof. Define the *parallel reduction* $\Rightarrow$ inductively: $x \Rightarrow x$; $0 \Rightarrow 0$; if $t \Rightarrow t'$ then $St \Rightarrow St'$; and if $n \Rightarrow n'$, $a \Rightarrow a'$ and $h \Rightarrow h'$, then

$$I(n, a;\, u.\, h) \Rightarrow I(n', a';\, u.\, h'), \qquad I(0, a;\, u.\, h) \Rightarrow a', \qquad I(Sn, a;\, u.\, h) \Rightarrow h'[I(n', a';\, u.\, h')/u].$$

Then $t \Rightarrow t$, and $\to \subseteq \Rightarrow \subseteq \to^*$. By induction on $h \Rightarrow h'$, if also $r \Rightarrow r'$ then $h[r/u] \Rightarrow h'[r'/u]$; the case of a contracted successor redex uses the composition law of substitution.

Define the *complete development* $t^\bullet$ by $x^\bullet := x$, $0^\bullet := 0$, $(St)^\bullet := St^\bullet$, $(I(0, a;\, u.\, h))^\bullet := a^\bullet$, $(I(Sn, a;\, u.\, h))^\bullet := h^\bullet[I(n^\bullet, a^\bullet;\, u.\, h^\bullet)/u]$, and $(I(n, a;\, u.\, h))^\bullet := I(n^\bullet, a^\bullet;\, u.\, h^\bullet)$ for every other n. The claim is that $t \Rightarrow t'$ implies $t' \Rightarrow t^\bullet$; it is proved by induction on $t \Rightarrow t'$. Consider $I(n, a;\, u.\, h) \Rightarrow I(n', a';\, u.\, h')$, with $n' \Rightarrow n^\bullet$, $a' \Rightarrow a^\bullet$ and $h' \Rightarrow h^\bullet$ by the induction hypotheses. If $n \equiv 0$, then $n' \equiv 0$ and $I(0, a';\, u.\, h') \Rightarrow a^\bullet$. If $n \equiv Sm$, then $n' \equiv Sm'$ with $m' \Rightarrow m^\bullet$, and $I(Sm', a';\, u.\, h') \Rightarrow h^\bullet[I(m^\bullet, a^\bullet;\, u.\, h^\bullet)/u]$. Otherwise the congruence rule applies. For $I(0, a;\, u.\, h) \Rightarrow a'$ the induction hypothesis is $a' \Rightarrow a^\bullet$. For $I(Sn, a;\, u.\, h) \Rightarrow h'[I(n', a';\, u.\, h')/u]$ the substitution property, applied to $h' \Rightarrow h^\bullet$ and $I(n', a';\, u.\, h') \Rightarrow I(n^\bullet, a^\bullet;\, u.\, h^\bullet)$, gives the reduct $(I(Sn, a;\, u.\, h))^\bullet$. The remaining cases are immediate.

By the claim, two parallel reducts of t both reduce in parallel to $t^\bullet$, so $\Rightarrow$ has the diamond property, and its transitive closure $\to^*$ is confluent. With Theorem 4.131, every term has exactly one normal form. □

Remark 4.133. The proof of Theorem 4.131 is that of `full_normalization`, which obtains confluence from local confluence and termination. `computation_normal` obtains termination by an embedding into the simply typed calculus with a recursor at the type of numbers (Gödel's system T): the iterator is translated as a recursor whose step ignores its counter, a zero contraction becomes one recursor contraction, a successor contraction becomes one recursor contraction followed by two β-contractions, and the translation respects contexts and substitution, so an infinite reduction would contradict the strong normalization of T, for which the package cites the mechanized proof of S. Urciuoli (EPTCS 376, 2023); it obtains confluence by parallel reduction, as above. The Lean formalization defines the normalizer by evaluation: the term is evaluated in an environment of terms by structural recursion (`Tm.nbe`), the iterator by a recursion on the value of its counter that unfolds a numeral prefix and leaves a neutral counter in place (`Tm.recV`), and `Tm.full` is the value in the identity environment. Totality and uniqueness of normal forms then need no proof. The system T and the evaluation are used in the metatheory; neither is added to the arithmetic.

Lemma 4.134 (Full normal forms)**.** *The normal terms are generated by*

$$ne ::= x \mid I(ne, nt;\, u.\, nt), \qquad nt ::= ne \mid 0 \mid S\, nt.$$

The terms ne are the neutral *terms. The counter of every iterator of a normal term is neutral; it has neither* 0 *nor S at its root. Every closed normal term is a numeral.*

Proof. A term is normal exactly when its subterms are normal and no contraction applies at its root, which gives the grammar. A neutral term has a free variable: following the counters from its root leads, outside every binder, to a variable. Hence a closed normal term is built from 0 and S. □

For example $I(x, 0;\, u.\, 0)$ is normal, although its value is 0 for every x, while $I(Sx, 0;\, u.\, 0)$ has the normal form 0. Normalization does not decide equality.

4.11.2 The kernel without computation rules

Proposition 4.135. nf *is an admissible normalizer.*

Proof. (ν1): variables and 0 are normal, and the reducts of St are the terms St' with t' a reduct of t. (ν2): reduction commutes with renaming, the renaming of a normal term is normal by Theorem 4.134, and a contraction creates no free variable. (ν3) is immediate. (ν4): by the two stability properties, $t[\sigma] \to^* \mathrm{nf}(t)[\sigma] \to^* \mathrm{nf}(t)[\mathrm{nf} \circ \sigma]$, so both sides of ($\nu$4) are the normal form of $t[\sigma]$. (ν5) is Theorem 4.6. □

In the rest of this section terms and formulas are in full normal form and $t\langle\sigma\rangle$ is $\mathrm{nf}(t[\sigma])$. An expression $I(n, a;\, u.\, h)$ in a rule denotes the normal form of the displayed term, also when a substitution exposes a redex. Formula normalization changes terms and never removes Q_y or $D_{x,y}$.

Definition 4.136 (The kernel N^{f})**.** N^{f} has native formulas over full normal forms, normalizing substitution, and the seven rules HYP, REFL (for every normal term), TR, QE, DE, DI and QI.

Theorem 4.137 (Both computation rules are deleted)**.** *For every finite context* Γ *of raw native formulas and every raw native formula* A*,*

$$\Gamma \vdash_{\mathsf{N}} A \quad \Longleftrightarrow \quad \mathrm{nf}(\Gamma) \vdash_{\mathsf{N}^{\mathrm{f}}} \mathrm{nf}(A).$$

N^{f} *is closed under weakening, cut and normalizing substitution, and Theorem 4.31 applies to it.*

Proof. By Theorem 4.135 and Theorem 4.30 it suffices that N^{f} and N^{nf} have the same rules. The left side of an instance e of I0 or IS reduces to its right side, so $\mathrm{nf}(e)$ is literally $r = r$ for a normal r, an instance of REFL. Conversely, for a normal r the reflexivity $r = r$ is $\mathrm{nf}(e)$ for the instance $I(0, r;\, u.\, 0) = r$ of I0. □

Remark 4.138. The theorem is formalized as `fnprf_iff_nprf` for the calculus `FNPrf`; the laws of the normalizer are `Tm.full_subst_congr` and `NFm.full_subst_congr`, and (ν5) is `NPrf.eqFull`. Both packages state the theorem with the auxiliary seven-rule calculus as a bridge to their six rules, and both stress that the comparison uses properties of the raw calculus that are established before it: the equality rules, the Leibniz rule, cut and equation induction of N.

4.11.3 Six rules on full normal forms

Definition 4.139 (The calculus F)**.** F has the syntax and the substitution of N^{f} and the six rules HYP, REFL, TR, QE, DE and WI. The template of TR is any normal term with one distinguished variable, used any number of times; there is no restriction on the terms of its first premise. The test $t \equiv 0$ of WI is applied to the normal witness.

There is no computation axiom: both equations are enforced by the normalizing substitution. Weakening, injective renaming and cut are admissible in F by direct induction on derivations, since they preserve normality, the literal 0, the root of a matrix and freshness, and no guard depends on the context. The equality rules and transport in literal equations are derivable by Theorem 4.2(b), the instances matching by Theorem 4.26.

On full normal forms Theorem 4.3(ii) has no closed term Z to use: by Theorem 4.134 a closed normal term is a numeral, and the aliases of Theorem 4.40 have the normal form 0. The packages give two repairs.

Lemma 4.140 (A fresh name for zero)**.** *Let* $Z(x, y)$ *be the equation* $I(y, 0;\, u.\, x) = 0$*. Then* $\vdash_{\mathsf{F}} D_{x,y} Z(x, y)$*, and* $Q_y Z(b, y) \vdash_{\mathsf{F}} b = 0$ *for a variable* b*.*

Proof. Introduce $D_{x,y}Z$ by WI at the witness 0. The matrix is a literal equation, so the mode is progress. The conclusion of the premise is $Z(0, Sz)$, the normal form of $I(Sz, 0; u.\, 0) = 0$, which is $0 = 0$; reflexivity derives it without the assumption. For the second statement the base $Z(b, 0)$ is the normal form of $I(0, 0; u.\, b) = 0$, which is $0 = 0$. QE with this base, derived by reflexivity, at the target 1 gives $Z(b, 1)$, the normal form of $I(1, 0; u.\, b) = 0$, which is $b = 0$. □

The witness opened by DE is a variable, hence not literally 0, and it is provably 0. It is discharged before the conclusion; no constant and no element of the domain is added.

Theorem 4.141. *For every context* Γ *and formula* A *in full normal form,*

$$\Gamma \vdash_{\mathsf{F}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{N}^{\mathrm{f}}} A.$$

Hence $\Gamma \vdash_{\mathsf{N}} A$ *if and only if* $\mathrm{nf}(\Gamma) \vdash_{\mathsf{F}} \mathrm{nf}(A)$ *for raw* Γ, A*, and substitution and the Leibniz rule are admissible in* F*.*

Proof. From N^{f} to F: QI is admissible by Theorem 4.3(i). An instance of DI is an instance of WI unless its matrix is a literal equation $E(x, y)$ and its witness is 0. In that case let $\Gamma, E(0, 0) \vdash E(0, Sz)$ be given. Choose b fresh and work in the context $\Gamma' := \Gamma, Q_y Z(b, y)$, where $\Gamma' \vdash b = 0$ by Theorem 4.140. By Theorem 4.3(ii) with the term b, applied to the weakened premise, $\Gamma' \vdash D_{x,y}E$. DE with the first premise $\vdash D_{x,y}Z$ closes the scope of b, which occurs neither in Γ nor in $D_{x,y}E$. From F to N^{f}, WI is admissible by Theorem 4.3(iii). The remaining statements are Theorems 4.31 and 4.137. □

Only transport in literal equations, weakening, injective renaming and cut are used in the repair; the Leibniz rule and the substitution of terms in derivations of F are consequences of the theorem and not premises of it.

Lemma 4.142 (The compound envelope)**.** *For a literal equation* $E \equiv (s = t)$ *in full normal form put*

$$G(v, E) := \big(I(v, 0; u.\, s) = I(v, 0; u.\, t)\big), \qquad L(E) := Q_v G(v, E),$$

with u, v *not free in* E*. Then* $G(0, E) \equiv (0 = 0)$ *and* $G(Sw, E) \equiv E$*, and for every context* Γ *both* $\Gamma, E \vdash_{\mathsf{F}} L(E)$ *and* $\Gamma, L(E) \vdash_{\mathsf{F}} E$*.*

Proof. The two identities are computations of normal forms; in the second, u is not free in s and t. For the first derivation apply the closure by cases of Theorem 4.3(i) to the matrix $G(v, E)$: its premise asks for $G(Sz, E)$, which is the assumption E. For the second apply QE to $L(E)$ with the base $0 = 0$ at the target 1. Both are uniform in the parameters of E and use no induction. □

The envelope yields a second proof of the missing case in Theorem 4.141, without Theorem 4.140. Let $\Gamma, E(0, 0) \vdash E(0, Sz)$ and put $F(x, y) := L(E(x, y))$, a matrix with the root Q_v. In the context $\Gamma, F(0, 0)$ the envelope gives $E(0, 0)$, cut gives $E(0, Sz)$, and the envelope gives $F(0, Sz)$. WI at the witness 0 is in case mode, because F is not a literal equation, and derives $D_{x,y}F$. Open it by DE at a fresh b, retaining $Q_y F(b, y)$. To introduce $D_{x,y}E$ at the witness b, in case mode since a variable is not 0, assume $E(b, 0)$; the envelope gives $F(b, 0)$, QE gives $F(b, Sz)$, and the envelope gives $E(b, Sz)$. DE closes the derivation.

Remark 4.143. The fresh-zero lemma is the repair of `computation_normal`, and the compound envelope that of `full_normalization`. The Lean formalization of the six rules, `F6Prf` with `f6prf_iff_nprf`, uses a third repair, `F6Prf.dIZero`: with $T(x) := \mathrm{sel}(x, 0, 0)$, the formula $D_{x,y}B(T(x), y)$ is introduced at the witness 1, in case mode, from the given premise, because $T(1)$ has the normal form 0; it is opened at a fresh b; and $D_{x,y}B$ is introduced at the neutral witness $T(b)$ from the opened closure by QE.

Example 4.144. The formula $D_{x,y}(y = 1)$ is derivable: under its reading the antecedent is $0 = 1$. On raw terms it is introduced in case mode at the witness $I(0, 0; u.\, 0)$, from the premise $0 = 1 \vdash Sz = 1$. Full normalization turns this witness into 0, and the same inference would be in progress mode, with the premise $z = 1 \vdash Sz = 1$, which is not derivable. A derivation normalized rule by rule is therefore rejected, and the derivation produced by either repair is accepted. The package `full_normalization` ships both certificates.

4.11.4 What normalization costs

Proposition 4.145 (No primitive recursive uniform normalizer)**.** *The map $t \mapsto \mathrm{nf}(t)$ on codes of raw terms is total recursive and not primitive recursive.*

Proof. Totality is Theorems 4.131 and 4.132. Suppose that the map is primitive recursive. Enumerate the raw terms with one free variable, replacing a code that is not well formed by the code of the term 0. The check of well-formedness, the formation of the closed instance of a term at a numeral and the reading of the value of a numeral are primitive recursive. By Theorem 4.134 and (ν5) the normal form of a closed instance is the numeral of its value. Hence the function $V(e, n)$, the value at n of the term with the code e, is primitive recursive. By Theorem 1.102 every primitive recursive unary function is the value function of such a term. In particular $n \mapsto V(n, n) + 1$ has a code e, and $V(e, e) = V(e, e) + 1$, a contradiction. □

Each fixed term denotes a primitive recursive function; the obstruction is to one evaluator for all descriptions. A checker for F or N^{f} is a total recursive program that contains a normalizer, and its running time and the size of its intermediate terms are not bounded by a primitive recursive function of the input: by Theorem 4.134 every closed term is expanded to a numeral, so a short closed expression can have a normal form of enormous size. No primitive recursive bound and no proof in iPRA of a uniform normalization theorem is claimed. The statements of this section are external statements about each finite derivation; both interpretations of Theorem 4.31 act on fixed finite expressions.

Proposition 4.146 (Fuel)**.** *Let the checker of F be modified so that a certificate carries a number f, each successor contraction spends one unit, and exhaustion rejects. The relation "the certificate with fuel f is accepted" is primitive recursive, and a formula has an accepted certificate for some fuel exactly when it has an accepted certificate of the unbounded checker.*

Proof. Let m be the size of the certificate. An instance of a term or formula of size at most M at terms of size at most M has size at most M^2. A successor contraction replaces a term of size at most M by a term of size at most M^2, since the body and the recursive iterator are subterms of the redex and the iterator is copied once for each occurrence of the bound variable. A zero contraction and the traversals between contractions do not increase the size, and at most M zero contractions occur in a row. The checker performs at most m inferences, each with a number of substitutions bounded by a constant k, and at most f successor contractions. Hence every term that occurs has size at most $m^{2^{km+f}}$, and the length of the computation is bounded by a primitive recursive function of m and f; a computation with such bounds on its length and on the size of its configurations is primitive recursive. An accepting run of the unbounded checker is finite and performs finitely many successor contractions, so it is accepted with that much fuel; a run accepted with fuel is a run of the unbounded checker. □

`computation_normal` ships the fuelled checker and measures it (Section 4.13); `full_normalization` observes that a relation of this kind can also be obtained from explicit reduction traces checked step by step, and does not build that checker.

4.12 A smaller checker for the unchanged core

This section draws on the second result of `computation_normal`, which concerns a program and no calculus. The rules, the certificates and the kernel function of the checker are those of the package `validated_presentation`; only the validator of the public goal changes, by a composition of that validator with the reuse of the scope traversal of the package `scope_kernel`.

The input is a JSON array. Its last element is the closed public goal, in the arity coding of the native formulas, and the earlier elements are proof packets consumed from the end; an unused prefix is allowed. A lexical gate accepts the whole input only if every value in it is an integer or a list. The public validator of the baseline carries a depth d, even at formula positions and odd at numerical positions; a node with r children changes the depth by $5 - r$, a formula node has one, two or three children, of which the node with two children is the equation and has numerical children, and a numerical node has one or three children. A numerical leaf a is accepted if d is odd and $a = 0$ or a is odd with $2 < a < d$. The kernel, before it reads any packet, opens the goal by a scope traversal whose substitution argument is, in this initial call, the kernel function itself and not a term: at a leaf within the scope the traversal returns the leaf, and at a leaf outside the scope it attempts to use the substitution argument as a term and fails.

The new validator keeps only the Boolean "numerical position", which becomes true below a node with two children and is otherwise inherited, and accepts a numerical leaf a if $a = 0$ or a is odd with $a > 2$.

Theorem 4.147 (Same accepted inputs)**.** *In the execution model without resource limits, the new checker and the baseline accept exactly the same inputs.*

Proof. The lexical gate is the same. On inputs of integers and lists, induction on the tree shows that the Boolean of the new validator is the parity of the depth of the old one wherever the shape is valid, by the table of arities above. All old tests remain except the upper bound $a < d$ at numerical leaves, so every goal accepted by the old validator is accepted by the new one. Let a goal be accepted by the new validator and violate an upper bound. The kernel is unchanged and computes the initial opening of the goal before it tests membership in the context or reads a packet. Every numerical position has an odd depth at least three, and the leaf is 0 or an odd integer greater than two; at the violating leaf the opening uses its substitution argument, which is a function, as a term, and the operation raises an exception. An exception is a rejection in both programs, and no packet can prevent it. A formula-position integer is rejected by both validators. On goals accepted by the old validator the initial opening is the identity, and all later steps of the two programs are the same. □

Hence every soundness, completeness and interpretation result for the certificate system of the baseline holds for the new checker. The result is an implementation change for an unchanged calculus; its sizes are in Section 4.13.

4.13 Checker sizes

This section collects the measurements of all thirteen packages, by the protocol of Section 1.10. Every row of Table 4.2 is a complete standalone program, with its input handling, its validation of syntax and scope, its substitution and normalization, all rules and its output. The gzip column, at level 9 with the timestamp zero, is reported by the packages for information and is not an objective. The reference interpreter is CPython 3.12.14. `uniform_step`, `seed_evaluation` and `initializer_equality` remove docstrings before normalization; the other packages do not. A dash marks a number that the package does not state.

Table 4.2: Complete checkers and their matched controls, as stated by the packages.

Complete program	Tokens	AST nodes	Bytes	Gzip-9
`computational_equality` (tagged, native contexts)				
Archived seven-rule control	1024	1080	3145	964
Matched seven-rule control	1008	1064	3121	957
E_3	959	1026	3039	966
`equality_from_iteration` (same format)				
E_3, upstream	959	1026	3039	966
E_2	954	1019	2996	955
Zero-normal control with ROOT kept	987	1059	3064	982
U	979	1047	3011	965
`uniform_step` (clause contexts)				
Matched zero-normal control, computation and transport	860	949	3048	1018
Matched raw control, three-case EVAL	853	969	3094	1026
Incoming canonical clausal checker	946	1072	3305	1073
$\mathsf{C}_{\mathrm{Step}}$	829	902	2950	1005
Native contexts: matched zero-normal control	1010	1080	3118	–
Native contexts: REFL and STEP	981	1051	3029	–
`seed_evaluation` (clause contexts)				
Computation and transport control	860	949	3048	–
STEP, helper style	829	902	2950	–
SEEDEVAL, helper style	829	902	2950	–
STEP, shared forward traversal	758	815	2804	–
SEEDEVAL, shared forward traversal	756	815	2802	–
STEP, goal-directed	741	826	2283	–
SEEDEVAL, goal-directed	741	826	2283	–
STEP, shared goal-directed traversal	709	787	2265	–
SEEDEVAL, shared goal-directed traversal	709	787	2265	–
APP with the constructor U, shared forward traversal	803	864	2902	–
`zero_normal` (tagged, native contexts)				
Raw, two introductions (8 rules)	1084	1141	3322	984
N^0 (7 rules)	1073	1136	3272	996
Raw, single introduction (7 rules)	1024	1080	3145	964
Z_6 (6 rules)	1013	1075	3095	977
`reflexive_base` (same format)				
N^0 (7 rules)	1073	1136	3272	996
Z_6, the witness-selected control (6 rules)	1013	1075	3095	977
R (6 rules)	1013	1073	3084	981
`initializer_equality` (closed goals)				
One-template control	1202	1360	4029	1173
S, selected	1192	1347	3979	1175
S, explicit shape tests	1236	1404	4156	1226
Canonical recursor checker, inherited	1338	1485	4611	1205

Complete program	Tokens	AST nodes	Bytes	Gzip-9
Canonical recursor checker, shared traversal	1224	1378	4079	1188
`full_normalization` (goal-directed)				
Raw computations, matched control	895	930	2569	941
Zero normalization, matched control	890	930	2532	945
F, block form	904	962	2546	960
F, expression form	908	965	2529	963
`computation_normal`				
Compact arity envelope (reference)	265	306	516	332
Public goal, baseline	383	442	870	507
Public goal, scope reuse (Section 4.12)	377	433	860	508
Z_6, original	1013	1075	3095	977
Z_6, term as computation certificate	991	1049	3029	971
F, initial	993	1046	3048	965
F, neutral-counter test	992	1046	3044	964
F, explicit fuel	1024	1086	3204	1003
Z_6, goal-directed	972	1004	2982	965
F, goal-directed	982	1008	3063	961
F, goal-directed, fuel	1010	1044	3189	1001
`atomic_seed_refinement` (tagged, closed native goals)				
Matched raw control: identity normalizer, I0 as seventh rule	1319	1507	5348	–
A_6	1333	1521	5373	–
`head_conversion` (open native sequents, docstrings removed)				
Matched control: full normalization of the endpoints	1086	1150	3165	–
VH	1059	1119	3102	–
`bounded_head` (open raw sequents, explicit proof leaves, docstrings removed)				
VH, original interface	1059	1119	3102	–
Matched control: VH with explicit proof leaves	1062	1123	3119	–
BH_3	1056	1127	3089	–
`bounded_conversion` (open raw sequents with a declared environment, docstrings counted)				
Matched control: checker of VH of `head_conversion`	1060	1121	3336	–
BH_3, presented as $\mathcal{L}_3$	1075	1156	3522	–

The matched differences. The effect of each change of rules, against the control that shares everything else, is as follows.

- `computational_equality`: E_3 against the matched seven-rule control saves 82 bytes, 49 tokens and 38 AST nodes; gzip grows by 9 bytes. The matched control differs from the archived one only by the same use of the invariant on proved formulas.
- `equality_from_iteration`: E_2 against E_3 saves 5 tokens, 7 AST nodes, 43 bytes and 11 gzip bytes. U against its control with ROOT saves 8 tokens, 12 AST nodes and 53 bytes. U against E_3 has 28 fewer bytes, 20 more tokens and 21 more AST nodes.
- `uniform_step`: $\mathsf{C}_{\mathrm{Step}}$ against the matched zero-normal control saves 98 bytes, 31 tokens

and 47 AST nodes. The delivered file with documentation has 3507 bytes and 1274 gzip bytes. With native contexts the checker with REFL and STEP is smaller than its matched control and larger in tokens and AST nodes than the checker of E_3 (959, 1026, 3039).

- `seed_evaluation`: the change from STEP to SEEDEVAL ties in the helper style and in both goal-directed pairs, and saves 2 bytes and 2 tokens with the shared forward traversal; of the 148 bytes between the 2950-byte and the 2802-byte checker, 146 come from the shared traversal, which benefits both rules. The package reports gzip sizes of 818 and 822 bytes for the goal-directed pair and of 872 and 877 bytes for the shared-traversal pair, STEP first. The checker with the constructor U is larger than its STEP control in all three coordinates.

- `zero_normal`: the deletion of the zero computation rule saves 50 bytes, 11 tokens and 5 AST nodes in each of the two pairs; gzip grows by 12 and 13 bytes. The frozen upstream checkers have 3358 and 3382 bytes.

- `reflexive_base`: R against Z_6 saves 11 bytes and 2 AST nodes with the same number of tokens; gzip grows by 4 bytes.

- `initializer_equality`: S against the one-template control saves 50 bytes, 10 tokens and 13 AST nodes; gzip grows by 2 bytes. The two rows for the canonical recursor checker measure a refactoring of an inherited program and no change of rules.

- `full_normalization`: the expression form of F against the control with zero normalization has 3 fewer bytes, 18 more tokens, 35 more AST nodes and 18 more gzip bytes, and 40 fewer bytes than the raw control. Its auxiliary forward checker has 3100 bytes against 3095 for the forward checker of Z_6.

- `computation_normal`: the scope reuse saves 10 bytes, 6 tokens and 9 AST nodes; gzip grows by 1 byte. For the calculi, the apparent gain of 47 bytes of F over the original checker of Z_6 disappears once the latter also takes a term as its computation certificate: F with the neutral-counter test is then 15 bytes and 1 token larger, 3 AST nodes and 7 gzip bytes smaller. The goal-directed checker of F is larger than that of Z_6, and fuel adds to both.

- `atomic_seed_refinement`: A_6 against its matched raw control, which has the same syntax, raw metadata, validation and substitution, is larger by 14 tokens, 14 AST nodes and 25 bytes. The deletion of the zero computation rule does not pay for the normalizer.

- `head_conversion`: VH against its matched control, which has the same sequent interface, raw hints, rule families and command line and normalizes the endpoints fully, saves 27 tokens, 31 AST nodes and 63 bytes. The control is not the smallest earlier checker with full normalization.

- `bounded_head`: BH_3 against the checker of VH with the same explicit proof leaves saves 6 tokens and 30 bytes and has 4 more AST nodes; it does not dominate that control in all three coordinates.

- `bounded_conversion`: its checker of BH_3 is larger than its control, the unchanged checker of VH from `head_conversion` with its docstrings, by 15 tokens, 35 AST nodes and 186 bytes. The control normalizes the terms of its input, while the checker keeps raw syntax and accepts explicit conversions, so the raw certificate languages of the two differ. With docstrings counted, its rows are not comparable with those of `head_conversion` and `bounded_head`.

The checkers. All checkers are single Python modules that import only `json` and `sys`, read one JSON value, use explicit tests in place of `assert`, and accept by printing `ACCEPT` with exit status zero. Proof translators and test runners are not imported and are not trusted. The checkers of `computational_equality`, `equality_from_iteration`, `zero_normal` and `reflexive_base` take an object with the fields `variables`, `context`, `goal` and `proof`, with tagged terms in de Bruijn notation and six proof tags; they check open sequents. Their EVAL instruction takes two premise proofs and nothing else. It relies on the invariant that the inference function returns a well-scoped native formula, so that a proved formula of length three is an equation and a term of length four is an iterator; the matched controls use the same invariant. The checkers on zero-normal terms reject every supplied iterator with the literal counter 0, so that a malformed subterm cannot be erased before it is validated, and rebuild terms with the smart constructor during substitution; in `zero_normal` this is one conditional after the rebuilding of a node. The test of `reflexive_base` compares the slices of a validated formula after its tag, which is exact because an equation has three fields and a binder formula two.

The clause checkers of `uniform_step` and `seed_evaluation` take the fields `goal` and `proof` and check closed goals. Bound variables are positive indices and eigenvariables are negative levels; every substituted term has only negative levels free, so the opening of a binder needs no shift. In the helper style the change from STEP to SEEDEVAL is two lines: the comparison of the first source with the initializer in place of the predecessor iterate, and the predecessor iterate from the new seed inside the conclusion. The checker of `initializer_equality` implements SEED by one comparison of the two left sides and, failing that, one comparison with the initializer of an iterator; no substitution is performed by that instruction.

The selected checker of `full_normalization` is goal-directed: one recursive traversal checks arities and sorts, renames variables and contracts exposed redexes, and validation is the requirement that the identity action returns its input, with all children checked before a redex is erased. A goal that is a retained assumption or a reflexive equation closes without reading its proof field, in the checker and in its controls. In `computation_normal` a normal term is its own reflexivity certificate, and the fuelled variant adds the field `fuel`.

The checker of A_6 in `atomic_seed_refinement` takes the fields `goal` and `proof` and checks closed goals with $=$, Q and $D_{,}$. Terms and formulas are tagged, with zero-based de Bruijn indices: the body of an iterator binds the index 0, Q_y binds y at 0, and $D_{x,y}$ binds y at 0 and x at 1. The six proof tags carry the raw metadata of Theorem 4.97; the checker validates every raw field before it uses or normalizes it, since normalization can erase an ill-scoped subterm, forms the raw instance by capture-avoiding substitution, and normalizes afterwards. Contexts hold normalized formulas. The matched control has the identity normalizer and a seventh tag for I0; it accepts a primitive zero computation and rejects the derivation of Theorem 4.100, and the checker of A_6 rejects the tag for I0.

The checker of VH in `head_conversion` reads the number of free variables, a context, a goal and a proof, and certifies the open sequent; a theorem has the empty context. Formulas must have head normal endpoints; hints are raw and are validated for scope and sort before any evaluation, also in parts that a zero contraction later erases. Q binds one de Bruijn index, $D_{,}$ two, with the universal coordinate at 0, and the iterator one, in its body. The normalizer does not recurse below a successor and keeps the initializer and the body of a neutral iterator raw. The checker of BH_3 in `bounded_head` has the same syntax and the same four input fields, keeps every formula raw, and has one proof record per rule; a null record closes a goal that is an assumption or a reflexivity, and its matched control of VH uses the same explicit leaves. The record of HC_3 supplies the depth and the old left endpoint, and the checker performs one contraction at that depth in the direction that matches. The checker of `bounded_conversion` reads the same four fields, with the number of free variables as the declared environment, and validates the context and the goal before any proof is read; its conversion record is described

in Theorem 4.124. Every supplied redex is validated in full before it is contracted.

Executed checks. The packages record the following runs, none with an unexpected outcome; they support the implementations and are not proofs of the theorems. `computational_equality`: 38 inherited fixtures, 14 translation round trips within the bound $9N$, 29 further round trips, 51 adversarial cases and 206 process executions. `equality_from_iteration`: 460 process checks, 62 translation checks, 1280 normalization checks and 320 substitution checks. `uniform_step`: 376 process checks of the clause checker, 100 of the native checkers with 38 translation round trips, and 2104 comparisons of the opening operation. `seed_evaluation`: 398 process checks, 166 translation round trips and 92 rejections, among others, and 83 certificates for the calculus with U. `zero_normal`: 500 process checks, the translation of 64 inherited certificates and 4832 checks of the coherence with substitution. `reflexive_base`: 13 translation cases, 16 malformed inputs and 86 process checks, including two substitutions that turn the guard from false to true. `initializer_equality`: 600 process checks and 31 translation round trips. `full_normalization`: 680 process checks of the goal-directed family and 132 of the forward checker, with the two certificates of Theorem 4.144. `computation_normal`: 272 runs and 9000 generated goals for Theorem 4.147, and a recovered case introduction at the witness 0 with the context $a = b$ and the matrix $I(y, a;\, u.\, b) = I(y, x;\, u.\, 0)$. `atomic_seed_refinement`: 20 persisted certificates, among them an equation induction that uses its hypothesis, the zero computation at a compound initializer, and a case introduction at the raw witness $I(0, 0;\, u.\, 0)$ that uses its case assumption, rejected when the witness is changed to 0; with the malformed inputs and the control checks, 54 executions in ordinary and optimized Python. `head_conversion`: 99 construction, rejection and command-line checks of the two checkers, in ordinary and optimized Python; 6 397 checks of an independent normalizer (idempotence, left coherence, stability of single steps under substitution, packet rows, and the failure of the stronger laws); and 125 256 finite checks of the colour reading. `bounded_head`: 23 certificates and 68 command-line checks in ordinary and optimized Python, among them 11 positive certificates translated to the control of VH, and 18 checks of the depth, scope and capture boundaries. `bounded_conversion`: 37 certificates and 116 command-line checks in ordinary and optimized Python, with conversions in both directions at every depth from 0 to 3, rejections at depth 4 and of a descent into an initializer, scope and type errors in parts of a redex that a contraction would erase, a substitution whose replacement contains its source, equation progress that uses its hypothesis and a compound case introduction; each positive certificate translated, by erasing its conversions and normalizing its stored formulas, to a certificate accepted by the control; and 24 runs of the control on the certificates c_t of Theorem 4.122 for 12 closed terms t, each accepted exactly when t has a positive value.

4.14 Further questions

This section collects, from all thirteen packages, the questions that they leave open, the claims that they state without proof and the results that they report as not achieved.

4.14.1 What is not achieved

- No package proves a lower bound or a minimality statement, for rule counts or for checker sizes. Every package states that its checker does not improve on the compact arity-coded checker of the repository (516 bytes; 521 bytes at the snapshot used by `zero_normal`) or on the fully validated profiles of 866, 870 and 788 bytes that have different certificate formats and validation contracts. A port of EVAL, of the identity frame, of SEED or of zero normalization to those encodings is not established.

- Moving a computation into the syntax does not by itself reduce the size of a complete checker. The measurements of Section 4.13 for U against E_2, for the constructor U, and for F against Z_6 are mixed or negative, and gzip sizes grow in most matched pairs.
- `seed_evaluation` reports that SEEDEVAL gives no convincing reduction of checker size: two of its three matched pairs tie.
- `zero_normal` examines the rule of common-source congruence, from $s = t$ and $s = u$ infer $h(t) = h(u)$, in place of TR. With reflexivity it is interderivable with TR: the instance whose first premise is a reflexivity is congruence, and the instance with the identity template composes two equations with a common left side. An apparent saving of 36 bytes disappears after the same factoring is applied to the control; both programs measure 3322 bytes, 1084 tokens and 1141 AST nodes.
- `computation_normal` recommends that F not replace the zero-normal kernel on the ground of its shorter rule table: the matched checker size does not improve uniformly, normal forms can be enormous, and the uniform normalizer is not primitive recursive (Theorem 4.145).
- `atomic_seed_refinement` reports that its target, a complete checker of a calculus bi-interpretable with iPRA that is smaller than the compact checkers of the repository, is not reached. The checker of A_6 is larger than its matched control (Section 4.13), and the compact guarded checkers, with their different interfaces, attain the separate minima of 248 tokens, 270 AST nodes and 479 bytes in three different programs. No minimality of six rules is claimed: the chapter already contains other six-rule calculi with stronger normalization.
- `head_conversion` reports the same for VH: its checker is larger than the compact guarded programs, whose separate minima at its snapshot are 244 tokens, 267 AST nodes and 478 bytes.
- The Lean formalization covers the calculi as recorded in the remarks of this chapter. It does not cover, in the forms stated here: the clause calculi $\mathsf{C}_{\mathrm{Step}}$ and $\mathsf{C}_{\mathrm{Seed}}$ with their clause contexts, the calculus $\mathsf{C}_{\mathrm{App}}$ and the unit normalization, Z_6 and S with the single introduction, the fresh-zero lemma and the compound envelope, the fuelled relation of Theorem 4.146, the raw-metadata transfer of Theorem 4.90 and the calculus A_6, the head normalizer and VH, BH_3, Theorem 4.121 and Theorem 4.122, the colour reading of Theorem 4.127, and any of the Python checkers.
- `bounded_head` reports that BH_3 does not reach the size target either, and claims minimality neither of the depth three nor of the seven rule families.
- `bounded_conversion` reports the same: its checker is larger than its matched control (Section 4.13) and than the compact programs of the repository, whose separate minima at its snapshot, 244 tokens, 265 AST nodes and 477 bytes, belong to three different programs with different input contracts. It claims no minimum, neither of the depth three nor of the source size.

4.14.2 Claims stated without proof

- `seed_evaluation` states that the interpretation in which every literal equation is false obstructs the deletion of reflexivity from the clause calculi, and that moving reflexivity into an initial universal clause still leaves a reflexivity axiom to count. Theorem 4.23 proves the corresponding statement for E_2; the statement for the clause calculi is not proved in the package.

- Theorem 4.85(c) is asserted in the notes of `seed_evaluation` for arbitrary clause contexts, by reference to the completeness of the clause calculus for T_{eq}; the argument given covers contexts of assumption clauses. The general statement requires the completeness of $\mathsf{C}_{\mathrm{Step}}$ for arbitrary clauses.

- `full_normalization` argues that its executable traversal, which fuses substitution with normalization and passes the unevaluated recursive iterator into the substitution of the body, computes $\mathrm{nf}(t[\sigma])$: every step contracts a redex of the materialized instance, termination is Theorem 4.131, and the result is normal. The argument is given in prose and is not carried out as a proof about the program.

4.14.3 Open questions

Question 4.148 (Lower bounds)**.** For a fixed term grammar, certificate language, validation contract and set of primitive operations, what is the least size of a checker of a calculus bi-interpretable with iPRA? No package fixes such a class. `computational_equality` asks for an order of simplicity that separates rule counts, constructors of the syntax, tokens, bytes and certificate sizes; `initializer_equality` and `full_normalization` ask for lower bounds once a class of representations is declared.

Question 4.149 (A smaller basis for arithmetic and equality)**.** Can the interface of reflexivity and one evaluation rule be reduced further while a small normalization layer is counted in full? Theorems 4.22 and 4.23 show that no rule of E_2 can be deleted from that inventory; they are not lower bounds across term languages. `equality_from_iteration` notes that a different basis has to account for the freedom of the initializer, for the reflection of equality (Theorem 1.86) and for both round trips; `reflexive_base` asks whether another arithmetic or logical rule can be removed with a net reduction of the complete checking algorithm.

Question 4.150 (An introduction that is stable under substitution)**.** EVAL, STEP and SEED are stable under substitution (Theorems 4.21, 4.58 and 4.69), and WI is not (Theorem 4.4). Is there a calculus of six rules in which substitution in derivations is proved by a direct induction, with a single introduction that needs no test on the witness, without enlarging the complete checker and without induction beyond literal equations? The guard of Section 4.5 is preserved and not reflected. `uniform_step` asks whether the logical rules can be merged further while the distinction between coverage and equation induction and the eigenvariable conditions are kept.

Question 4.151 (Sizes of derivations)**.** The congruence of Theorem 4.15 adds eight nodes, that of Theorem 4.61 four, that of Theorem 4.67 four, and the elimination of ROOT adds two per instance (Theorem 4.12); in each case the labels contain copies of the template and of the endpoints. What are the least numbers of nodes for these fragments, for trees and for derivations with shared subproofs, and what are the sizes of serialized derivations and the checking times on an arithmetic library? Can sharing reduce the cost without a decoder that is larger than the rule it removes? Can an interface based on the initializer shorten broader families of derivations than the single computations of Theorem 4.78, without the sevenfold expansion of Theorem 4.76(c)?

Question 4.152 (Normalization between zero and full)**.** Is there a normalization between the zero normalization and the full normalization that reduces both the arithmetic interface and the complete trusted implementation? The unit normalization of Section 4.8.2 has a primitive recursive normalizer and a rule without a counter, and its measured checker is larger than its control.

Question 4.153 (Normalization below zero normalization)**.** For the atomic-seed normalizer, which violates ($\nu 4$) (Theorem 4.95), Theorem 4.102 is proved with raw rule metadata. Does the calculus $\mathsf{N}^{\mathrm{nf_a}}$, defined as in Theorem 4.28 with atomic-seed normal metadata and normalizing substitution, still derive exactly the normalized sequents of N, and is A_6 complete when its metadata are restricted to normal terms? `atomic_seed_refinement` does not claim that the atomic-seed normal terms with normalizing substitution form a free term language, and its derivation of the zero computation needs a body that is not normal. It also leaves open a compact implementation of atomic contraction in a level coding (Theorem 4.96), with its sort test and the treatment of dummy slots, that is smaller than the present compact checkers.

Question 4.154 (Substitution for a variable: theorems and translations)**.** Theorem 4.127 separates the calculi of Theorem 4.128 from N and $\mathcal{P}$ by a sequent with an assumption; their consequence relations are strictly weaker. The colour reading validates every theorem of N with the empty context (Theorem 4.129). Do these calculi derive every theorem of N with the empty context, or does a theorem separate them? `head_conversion` asks whether substitution for a variable on raw terms becomes complete through another public translation, which would need a faithful translation with both formula round trips.

Question 4.155 (The least conversion for the packet)**.** The proof of Theorem 4.114 uses the closed rows of the packet and left coherence under raw substitution, and head normalization supplies both. `head_conversion` asks whether a smaller certified reduction relation, which performs only the conversions used by a proof, suffices with its validation and its certificate contract counted, and whether head conversion can be combined with the compact arity coding for a joint improvement of all three coordinates. For conversion bounded in depth, is a bound smaller than three sufficient, with another parity term or another packet, and does a smaller bound shorten the complete checker of BH_3 once the longer certificates are accounted for? `bounded_conversion` asks the same, and notes that a proof that a smaller bound is incomplete would have to cover every encoding of the two rows of the packet. It asks further whether the one bounded conversion can be fused with an existing rule, such as VSUB or DE, while every raw hint is still validated before a contraction erases it and the packet is recovered without presupposing transport; and whether transport can be recovered with fewer explicit conversions and discharges, by shared subderivations or by a derived instruction whose verification is counted in the checker.

Question 4.156 (Checking VH on theorems)**.** Theorem 4.122 uses an open sequent whose context is false in the standard model. Is the relation "c is a derivation of $\vdash A$ in VH", restricted to the empty context, also not primitive recursive? `bounded_conversion` states its result for the open-sequent interface only. The corresponding questions arise for the derivation relations of F and N^{f}, whose checkers contain the full normalizer of Theorem 4.145.

Question 4.157 (Classes of provable bases)**.** Which decidable classes of bases that are derivable in every context give the best exchange between the size of a checker and the length of derivations for the guard of Theorem 4.45? The classes "$r = r$" and "$0 = 0$" of Theorem 4.54 already differ. Does $\mathsf{R}^{\mathrm{raw}}$, which needs no normalization, lower the size of a complete compact checker, and does the guard "$0 = 0$" suffice on raw terms?

Question 4.158 (Other positions of a recursor)**.** Which arithmetic primitives support both the identity and the application of an arbitrary template through one distinguished argument, as the initializer of the scalar iterator does at the counters 0 and 1 (Theorems 4.57 and 4.60)? `initializer_equality` requires equality interpreted by equality and both formula round trips in every such comparison.

Question 4.159 (Narrower induction)**.** `seed_evaluation` asks whether the approach by rigidity of the successor, which derives induction for equations with the right side 0, can be

completed to the reflection of equality without importing general equation induction at an intermediate step, and notes that a recursor whose body sees the counter gives a simpler predecessor (Theorem 1.65) and does not by itself supply the reflection argument.

Question 4.160 (Formalization and a common benchmark)**.** Formalize the statements listed above as not covered, in particular the conversion relation with finite witnesses and the fuelled relation, the fresh-zero lemma and the compound envelope next to `F6Prf.dIZero`, and the clause calculi with STEP, SEEDEVAL and APP. Separately, measure the compact, the goal-directed and the forward checkers under common contracts for the validity of syntax, open sequents, sharing in certificates and the behavior on errors, so that a bound for one encoding is not read as a bound for all presentations. `bounded_conversion` proposes as a formal target a derivation relation with a constructor for bounded conversion in a counter context and the equivalence with the derivation relation of N, to which the formalized bi-interpretation then applies. It notes that VSUB is not preserved under the substitution of arbitrary terms by an immediate induction, so that such a proof should keep the order of Theorem 4.120: structural operations, symmetry, equation progress, packet, transport. It asks for an explicit benchmark suite with a fixed reader contract, separating closed guarded formulas, full native formulas, open contexts, complete validation of the input and streaming input.

Chapter 5

Clause calculi

This chapter treats the presentations whose formulas are equations and the paired binder only. The closure binder Q_y is not a formula constructor in them. Its work is done by the context: a context entry is a reusable clause $[A;\, y.\, B(y)]$ (Section 1.9), and the elimination of $D_{x,y}B$ adds a clause where the kernel adds the formula $Q_yB(b,y)$. Section 5.1 relates sequents of the native scalar core N to clause sequents. Section 5.2 gives the five-rule calculus and its equivalence with N. The later sections vary one component at a time: the arithmetic interface (Section 5.3), the introduction rule (Section 5.4), the form of a context entry (Section 5.5), and the number of rule families (Section 5.6). Section 5.8 records a separate arithmetic observation of one of the packages, and Section 5.9 lists the checker sizes.

Every calculus of the chapter has the terms of Section 1.1, the designated contradiction $0 = S0$ and the ordinary reading of Section 1.2. Throughout, $\equiv$ is literal identity of syntax up to the names of bound variables. A matrix $B(x,y)$ displays the two variables that an enclosing $D_{x,y}$ binds, and $B(t,r)$ is the result of substituting t for x and r for y. Substitution avoids capture. A *literal equation* is a formula whose root is an equation; an equation below a binder D is not one. The numeral $S0$ is also written 1. The rules of N are cited by their names in Section 1.3: HYP, the two computation equations (I0, IS), TR, QE, DE, the introduction by cases (DI) and the introduction of Q by progress (QI).

5.1 From Q formulas to clauses

This section draws on `raw_clausal`, `clausal_two_case`, `reduced_clause_calculus` and `clause_guards`.

5.1.1 The Q-free form of a formula

A native formula is *Q-free* if it is built from equations by $D_{x,y}$ alone. The Q-free formulas are the formulas of every calculus of this chapter.

Definition 5.1 (Q-free form)**.** The translation τ from native formulas to Q-free formulas is

$$\tau(s=t) := (s=t), \qquad \tau(D_{x,y}B) := D_{x,y}\tau(B), \qquad \tau(Q_yB) := D_{w,y}\tau(B),$$

where each inserted w is fresh and does not occur in B. Terms are unchanged. The inclusion of Q-free formulas into native formulas is written j and is usually left implicit. For a matrix $B(y)$ and a fresh w the *closure abbreviation* is

$$q_yB(y) := D_{w,y}B(y), \qquad w \notin \mathrm{FV}(B).$$

The translation preserves free variables, commutes with substitution up to the names of bound variables, sends equations to equations and all other formulas to formulas with root D, and satisfies $\tau j F \equiv F$ for every Q-free F. The ordinary reading of $q_y B$ is $\exists w\,(B(0) \to \forall y\, B(y))$, which is equivalent to $B(0) \to \forall y\, B(y)$ because the domain is inhabited by 0. In the Lean development τ is `unQ`.

Lemma 5.2 (The round trip in `N`)**.** *The following hold in* `N`*, in every context.*

(i) $Q_y B \dashv\vdash D_{w,y} B$ *for* $w \notin \mathrm{FV}(B)$.

(ii) If $\Gamma, B \vdash B'$ *and* $\Gamma, B' \vdash B$*, where the variables* x, y *are not free in* Γ*, then* $\Gamma, D_{x,y} B \vdash D_{x,y} B'$ *and* $\Gamma, Q_y B \vdash Q_y B'$*, and conversely.*

(iii) $A \dashv\vdash j\tau A$ *for every native formula* A.

Proof. (i) From $Q_y B$: under the assumption $B(0)$ the rule QE gives $B(Sz)$, so the introduction by cases at an arbitrary witness gives $D_{w,y} B$. From $D_{w,y} B$: the rule DE opens $Q_y B(b, y)$, which is $Q_y B$ because w does not occur in B, and HYP closes the branch.

(ii) Assume $D_{x,y} B$. Apply DE at a fresh b; the branch has the assumption $Q_y B(b, y)$. Introduce $D_{x,y} B'$ by cases at the witness b. Its branch assumes $B'(b, 0)$. Substitution in the derivation of $\Gamma, B' \vdash B$ (Section 1.4) and cut give $B(b, 0)$, QE gives $B(b, Sz)$, and the substituted derivation of $\Gamma, B \vdash B'$ gives $B'(b, Sz)$. For Q, pass from $Q_y B$ to $D_{w,y} B$ by (i), apply the case just proved with the unused variable w in the place of x, and return by (i). The converse statements are symmetric.

(iii) Induction on A. Equations are unchanged. For $A \equiv D_{x,y} B$ use (ii) and the induction hypothesis for B. For $A \equiv Q_y B$ use (ii) for Q and then (i). □

5.1.2 Clause occurrences and their origins

A context of a clause calculus is a finite list of clause *occurrences* $[A;\, y.\, B(y)]$ with $y \notin \mathrm{FV}(A)$ (Section 1.9). The clause is a context object and not a formula. Its reading is $A \to \forall y\, B(y)$. A rule that uses a clause selects an occurrence, so two occurrences with the same text are distinct. An ordinary assumption F is the constant clause

$$\mathrm{ord}(F) := [\, 0 = 0;\, v.\, F\,], \qquad v \notin \mathrm{FV}(F),$$

and Γ, F abbreviates $\Gamma, \mathrm{ord}(F)$. For a list Δ of formulas, $\mathrm{ord}(\Delta)$ applies ord to each member.

In a derivation that starts from ordinary assumptions a clause occurrence arises in two ways only.

Definition 5.3 (Origins)**.** An *assumption occurrence* is an occurrence of the form $\mathrm{ord}(G)$. An *opened occurrence* is an occurrence of the form $[B(b, 0);\, y.\, B(b, y)]$ for a matrix $B(x, y)$ and a variable b. An *annotated* context is a context in which each occurrence is marked as one of the two and has the corresponding form. The *native image* $\Gamma^{\natural}$ of an annotated context replaces each assumption occurrence $\mathrm{ord}(G)$ by the formula G and each opened occurrence $[B(b, 0);\, y.\, B(b, y)]$ by the formula $Q_y B(b, y)$.

The mark belongs to the occurrence. A clause such as $[0 = 0;\, y.\, 0 = 0]$ has both forms, and either mark may be given to it. The rules of the calculi below add an opened occurrence at each elimination and an assumption occurrence at each introduction, so every context reachable from $\mathrm{ord}(\Delta)$ has a canonical annotation.

5.2 The five-rule calculus

This section draws on `raw_clausal`; the same lemmas are restated in `rootfree_clausal`, `reduced_clause_calculus`, `clausal_two_case` and `uniform_clauses`.

5.2.1 Rules

The evaluation operation $\Phi(s,t,b)$ on raw terms is that of Theorem 4.8. Its three rows are called the *root row* ($b \equiv s$, value t), the *zero row* ($b \equiv I(0,a;\,u.\,h)$, $s \equiv a$, value t) and the *successor row* ($b \equiv I(Sn,a;\,u.\,h)$, $s \equiv I(n,a;\,u.\,h)$, value $h[t/u]$); they are disjoint, and the three cases of the rule EVAL that they give are ROOT, EVAL_0 and EVAL_S of Theorem 4.9. The operation Φ_2 is Φ without the root row. Chapter 4 studies the rule with contexts of native formulas; here it is combined with clause contexts.

Definition 5.4 (The calculus C_3)**.** Formulas are Q-free, contexts are lists of clause occurrences, and the rules are the following five.

$$\frac{}{\Gamma \vdash s = s}\ \text{REFL} \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash b = r}{\Gamma \vdash r = \Phi(s,t,b)}\ \text{EVAL}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \text{USE} \qquad \frac{\Gamma \vdash D_{x,y}B(x,y) \quad \Gamma, [B(b,0);\, y.\, B(b,y)] \vdash F}{\Gamma \vdash F}\ \text{ELIM}$$

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \text{INTRO}$$

EVAL requires $\Phi(s,t,b)$ to be defined. USE requires an occurrence $[A;\, y.\, B(y)]$ in Γ and allows every term t. In ELIM the variable b is fresh for Γ, F and B. In INTRO the variable z is fresh for Γ, t and B, the witness t is an arbitrary term, and η is the term $\eta_{B,t}(z)$ of the witness-selected introduction (Theorem 2.15): it is z if B is a literal equation and $t \equiv 0$ (*progress mode*) and 0 otherwise (*case mode*). The calculus C is C_3 with Φ_2 in the place of Φ.

There is no hypothesis rule, no computation axiom, no transport rule, no induction rule and no structural rule. The three rows of Φ and the mode test of INTRO are part of the specification. The calculus C is `CPrf` in the Lean development.

5.2.2 Derived rules

The lemmas of this subsection hold in C_3 and in C under arbitrary clause contexts.

Lemma 5.5 (Structural operations)**.** *An ordinary assumption can be used:* $\Gamma, F \vdash F$*. Weakening by clause occurrences and injective renaming of free variables are admissible. Cut for ordinary assumptions is admissible: if* $\Gamma \vdash F$ *and* $\Gamma, \mathrm{ord}(F), \Gamma' \vdash A$ *then* $\Gamma, \Gamma' \vdash A$*.*

Proof. REFL proves $0 = 0$, and USE on $[0 = 0;\, v.\, F]$ at any term proves F. Weakening and injective renaming are inductions on the derivation after the eigenvariables are made fresh; an injective renaming preserves the root of a matrix and the test $t \equiv 0$, so it preserves the mode of INTRO, and it preserves the literal matches of Φ.

For cut, induct on the derivation that has the distinguished occurrence $\mathrm{ord}(F)$. At a USE that selects this occurrence, replace the node by the given derivation of F, weakened to the local context; the derivation of its premise $0 = 0$ is discarded. At every other node transform the premises and reapply the rule. The eigenvariables are made fresh first. No substitution of a term in a derivation is performed. □

Lemma 5.6 (Equality and computation)**.** *REFL and the zero and successor rows of EVAL derive, over every clause context, symmetry and transitivity of equality, the inference ROOT, the two computation equations*

$$I(0,a;\,u.\,h) = a, \qquad I(Sn,a;\,u.\,h) = h[I(n,a;\,u.\,h)/u],$$

congruence in every term template, and transport in a term and in a literal equation:

$$\frac{s = t}{h[s/u] = h[t/u]} \qquad \frac{s = t \quad h[s/u] = r}{r = h[t/u]} \qquad \frac{s = t \quad a[s/u] = b[s/u]}{a[t/u] = b[t/u]}$$

with no restriction on s.

Proof. The derivations of Section 4.2.3 (Theorems 4.11 and 4.13 to 4.16) consist of REFL, EVAL_0, EVAL_S and the given premises. They contain no rule that reads the context, so they are derivations of C and of C_3 over every clause context. Transport in an equation is the one-term transport with the template a, which gives $b[s/u] = a[t/u]$, followed by the one-term transport with the template b. □

Lemma 5.7 (All introductions by cases)**.** *For every matrix B and every witness t the rule*

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)} \qquad (z \text{ fresh})$$

is admissible.

Proof. INTRO has this form unless B is a literal equation and $t \equiv 0$. That case is repaired as in the proof of Theorem 2.17, at the second name $c \equiv I(0,0;\, u.\, 0)$ of zero (Theorem 2.16): in the context $\Gamma, B(c,0)$ transport along $c = 0$ (Theorem 5.6) gives $B(0,0)$, the cut of Theorem 5.5 with the given derivation gives $B(0,Sz)$, transport along $0 = c$ gives $B(c,Sz)$, and INTRO at the witness c is in case mode. The cut is the cut for a constant clause; only transport in an equation is used. □

Lemma 5.8 (Transport in every formula)**.** *For every Q-free formula F with a distinguished variable v, from $\Gamma \vdash s = t$ and $\Gamma \vdash F[s/v]$ one derives $\Gamma \vdash F[t/v]$.*

Proof. Induction on F. Equations are Theorem 5.6. Let $F \equiv D_{x,y}B(v,x,y)$. Apply ELIM to $F[s/v]$ at a fresh b, which adds $[B(s,b,0);\, y.\, B(s,b,y)]$. Introduce $F[t/v]$ by Theorem 5.7 at the witness b. Its branch assumes $B(t,b,0)$. The induction hypothesis along $t = s$ gives $B(s,b,0)$, USE at Sz gives $B(s,b,Sz)$, and the induction hypothesis along $s = t$ gives $B(t,b,Sz)$. Each use of the induction hypothesis concerns the matrix B. □

Lemma 5.9 (The three closure interfaces)**.** *With $q_yB \equiv D_{w,y}B$, w fresh, the following are derivable.*

(i) From $\Gamma, B(0) \vdash B(Sz)$, z fresh, derive $\Gamma \vdash q_yB$.

(ii) From $\Gamma \vdash q_yB$ and $\Gamma \vdash B(0)$ derive $\Gamma \vdash B(t)$ for every term t.

(iii) If Γ contains an occurrence $[B(0);\, y.\, B(y)]$ then $\Gamma \vdash q_yB$.

(iv) For a literal equation $E(y)$: from $\Gamma, E(z) \vdash E(Sz)$, z fresh, derive $\Gamma \vdash q_yE$.

Proof. (i) is Theorem 5.7 with the unused variable w in the place of x. For (ii) apply ELIM to q_yB; the fresh witness does not occur in B, so the added clause is $[B(0);\, y.\, B(y)]$, and USE with the given derivation of $B(0)$ proves $B(t)$. For (iii), under the assumption $B(0)$ use the retained clause at Sz and apply (i). For (iv) apply INTRO to the matrix $E(y)$, in which w does not occur, at the witness 0; the matrix is a literal equation and the witness is literally 0, so the rule is in progress mode. □

Corollary 5.10 (Equation induction and numerical cases)**.** *For a literal equation $E(y)$, from $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$ one derives $\Gamma \vdash E(t)$. For an arbitrary formula $F(y)$, from $\Gamma \vdash F(0)$ and $\Gamma \vdash F(Sz)$, z fresh, one derives $\Gamma \vdash F(t)$. For a literal equation $E(x,y)$ and a term t, from $\Gamma, E(t,z) \vdash E(t,Sz)$ one derives $\Gamma \vdash D_{x,y}E$.*

Proof. The first two statements are (iv) and (ii), respectively (i) and (ii), of Theorem 5.9. For the third, (iv) gives $q_y E(t, y)$, and Theorem 5.7 at the witness t gives $D_{x,y}E$, since under the assumption $E(t, 0)$ interface (ii) gives $E(t, Sz)$. □

Progress is available for literal equations only. Extending the progress mode to matrices with root D would give induction for arbitrary formulas by the same argument and would be a different theory.

5.2.3 Equivalence with the native scalar core

Theorem 5.11 (The five-rule calculus and N)**.** *Let* X *be* C_3 *or* C*.*

(i) If Γ *is an annotated clause context and* $\Gamma \vdash_{\mathsf{X}} F$*, then* $\Gamma^{\natural} \vdash_{\mathsf{N}} F$*.*

(ii) If Γ *is a native context and* $\Gamma \vdash_{\mathsf{N}} A$*, then* $\mathrm{ord}(\tau\Gamma) \vdash_{\mathsf{X}} \tau A$*.*

(iii) For Q-free Δ *and* F*:* $\mathrm{ord}(\Delta) \vdash_{\mathsf{X}} F$ *if and only if* $\Delta \vdash_{\mathsf{N}} F$*.*

(iv) For native Γ *and* A*:* $\mathrm{ord}(\tau\Gamma) \vdash_{\mathsf{X}} \tau A$ *if and only if* $\Gamma \vdash_{\mathsf{N}} A$*.*

All translations are effective on finite derivations and are the identity on terms.

Proof. (i) Induction on the derivation. N derives reflexivity, symmetry, transitivity and Leibniz replacement (Section 1.4). REFL is reflexivity. The three rows of EVAL are derived in N from I0, IS and TR as in the proof of Theorem 4.18. A USE of an assumption occurrence $\mathrm{ord}(G)$ is HYP. A USE of an opened occurrence $[B(b, 0); y.\, B(b, y)]$ is QE applied to the assumption $Q_y B(b, y)$ and the translated derivation of $B(b, 0)$. ELIM is DE, which adds $Q_y B(b, y)$, the image of the opened occurrence. INTRO in case mode is the introduction by cases, with the new occurrence marked as an assumption. In progress mode the matrix is a literal equation $E(x, y)$ and $t \equiv 0$; the translated premise is $\Gamma^{\natural}, E(0, z) \vdash E(0, Sz)$, the introduction of Q by progress gives $Q_y E(0, y)$, and the introduction by cases at the witness 0 gives $D_{x,y}E$, because under the assumption $E(0, 0)$ the rule QE gives $E(0, Sz')$.

(ii) Induction on the derivation in N. HYP becomes REFL at 0 followed by USE on the constant clause. The computation equations and TR are Theorem 5.6. QE is interface (ii) of Theorem 5.9, since $\tau(Q_y B) \equiv q_y \tau(B)$ and τ commutes with substitution. The introduction by cases is Theorem 5.7. The introduction of $Q_y E$ by progress is interface (iv), whose conclusion is $q_y E \equiv \tau(Q_y E)$. For DE with major premise $D_{x,y}B$ and branch $\Gamma, Q_y B(b, y) \vdash A$, translate the major premise and apply ELIM, which adds $[\tau B(b, 0); y.\, \tau B(b, y)]$. In the extended context interface (iii) derives $q_y \tau B(b, y)$, which is $\tau(Q_y B(b, y))$. Weaken the translated branch to the extended context and cut this derivation in for its ordinary assumption (Theorem 5.5). ELIM then concludes τA.

(iii) From left to right use (i) with the canonical annotation, for which $\mathrm{ord}(\Delta)^{\natural} = \Delta$. From right to left use (ii); τ is the identity on Q-free formulas.

(iv) From right to left is (ii). From left to right, (i) gives $\tau\Gamma \vdash_{\mathsf{N}} \tau A$, and Theorem 5.2(iii) with cut in N replaces $\tau\Gamma$ by Γ and τA by A. □

Corollary 5.12 (Both formula composites)**.** *The composite* τj *is the identity on Q-free formulas, and* $A \dashv\vdash_{\mathsf{N}} j\tau A$ *for every native formula* A*. Both term translations are identities. In particular* $\vdash_{\mathsf{C}} 0 = S0$ *if and only if* $\vdash_{\mathsf{N}} 0 = S0$*.*

Remark 5.13. In the Lean development (iv) is `cprf_iff_nprf` and (iii) is `cprf_iff_nprf_of_qfree`, for the calculus C (`NativeClausal`). There the clause calculus is defined over all native formulas, and `CPrf.qfree` states that it derives only Q-free formulas from Q-free clauses. `raw_clausal` states the theorem for C_3 against the six-rule calculus with REFL and EVAL of

`computational_equality`; `clausal_two_case` and `uniform_clauses` state it for C against the same calculus with two evaluation rows, and `clause_guards` against the six-rule calculus with the diagonal introduction. The proof above compares with N directly; the cases are the same. The Lean proof of completeness translates the six-rule calculus rule by rule.

Proposition 5.14 (Substitution)**.** *If $\Gamma \vdash_{\mathsf{X}} F$ and σ is a substitution of terms for free variables, then $\Gamma\sigma \vdash_{\mathsf{X}} F\sigma$, for X either of C_3, C and every clause context Γ. The rule at a node of the derivation is not always preserved.*

Proof. Induction on the derivation, with eigenvariables made fresh for σ. REFL is immediate, and $\Phi(s\sigma, t\sigma, b\sigma) \equiv \Phi(s,t,b)\sigma$ by the same row (Theorem 4.21). USE and ELIM commute with σ. An INTRO in progress mode keeps its equation root and its literal witness 0. An INTRO in case mode whose matrix has root D, or whose witness t has $t\sigma \not\equiv 0$, remains in case mode. In the remaining case the matrix is a literal equation, $t \not\equiv 0$ and $t\sigma \equiv 0$; the substituted premise is the premise of an introduction by cases at the witness 0, and Theorem 5.7 applies. □

A substitution can thus change the mode of an introduction, and the derivation after substitution may contain the nodes of Theorem 5.7 in the place of one INTRO. Theorem 5.36 gives a calculus in which the tree is preserved.

Remark 5.15 (Clauses that are not of the two origins)*.* Theorem 5.11(i) concerns annotated contexts. An arbitrary clause $[A;\, y.\, B]$ supplied from outside is represented in N by the derived implication and universal quantifier of the native logical interface (Theorem 1.119), as the formula that reads $A \to \forall y\, B$, and USE becomes the elimination of the implication followed by the elimination of the universal quantifier. The equivalences (iii) and (iv) do not need such clauses. They cover every sequent with ordinary assumptions and not only closed theorems.

5.3 Variants of the arithmetic interface

This section draws on `rootfree_clausal`, `reduced_clause_calculus`, `clausal_two_case`, `clausal_single_evaluation`, `raw_clausal`, `uniform_clauses` and `clause_contexts`; each subsection names its sources. The variants are: an evaluation rule with two rows, an evaluation rule with one scheme on zero-normal terms, and reflexivity stated as a clause, on raw and on zero-normal terms.

5.3.1 Two evaluation cases

This subsection draws on `rootfree_clausal`, `reduced_clause_calculus` and `clausal_two_case`, which present the same calculus C. Its two evaluation rows are the rules EVAL_0 and EVAL_S of Theorem 4.9. The root row is derived by the identity frame of Theorem 4.11: with $K(s) \equiv I(0, s;\, u.\, 0)$, the derivation

$$\mathrm{RC}(p,q) := \mathrm{Ev}_0(p,\ \mathrm{Ev}_0(q,\ \mathrm{R}_{K(s)}))$$

of Theorem 4.12 turns $p : s = t$ and $q : s = r$ into $r = t$ by two instances of EVAL_0 and one REFL leaf. It contains no rule that reads the context, so it is a derivation of C over every clause context. The body of $K(s)$ is never evaluated; any body serves.

Theorem 5.16 (Deletion of the root row)**.** *For every clause context Γ and every formula F,*

$$\Gamma \vdash_{\mathsf{C}_3} F \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathsf{C}} F.$$

The translation from C_3 to C leaves the end sequent unchanged and turns a derivation with N nodes, k of them uses of the root row, into a derivation with exactly $N + 2k \le 3N$ nodes. The converse translation is the inclusion.

Proof. Every rule of C is a rule of C_3. Conversely replace each use of the root row by RC and copy every other node, as in Theorem 4.12; the count is the count of that theorem. What the clause setting adds is that the replacement is made under arbitrary retained clauses, also those opened inside ELIM and INTRO: no variable is bound or freed, and the matrices and witnesses of INTRO are unchanged, so its modes and all freshness conditions are unchanged. □

The count is of inference nodes and bounds no serialized size (Theorem 4.12).

Proposition 5.17 (Three new nodes are necessary)**.** *Let x, y, z be distinct variables, and consider the fragment with the rules* REFL, EVAL_0 *and* EVAL_S *and with two given leaves $p : x = y$ and $q : x = z$. Every derivation of $z = y$ in this fragment contains at least two* EVAL *nodes and one* REFL *leaf.* $\mathrm{RC}(p, q)$ *attains the bound.*

Proof. The conclusion $z = y$ is not p, not q and not reflexive, so the last inference is an EVAL. In both rows the second premise has the form $b = r$ with b an iterator, and the conclusion has r as its left side. The second premise of the last inference is therefore $b = z$ with b an iterator. It is not p or q, whose left side is the variable x, and it is not a REFL leaf, since an iterator is not the variable z. So it ends in another EVAL. Follow second premises upwards from there. Each has an iterator as its left side, so the path does not end at p or q; it ends at a REFL leaf. □

The bound concerns this fragment and this cost convention. It does not bound derivations that use the logical rules or previously derived lemmas, and it is not a bound on the size of a checker.

Proposition 5.18 (Neither remaining row is redundant)**.** *In* C *without* EVAL_0 *the equation $I(0, 0; u.\, 0) = 0$ is not derivable. In* C *without* EVAL_S *the equation $I(S0, 0; u.\, Su) = S0$ is not derivable.*

Proof. The two interpretations of Theorem 4.22 apply, with a clause read as $A \to \forall y\, B(y)$: under any compositional interpretation of the iterator in the standard natural numbers REFL, USE, ELIM and both modes of INTRO are sound, each interpretation makes the remaining row sound, and the displayed equations are false, the second because its left side has the value 0. □

Remark 5.19. The three packages state Theorem 5.16 with the frame $I(0, s; u.\, 0)$; `uniform_clauses`, `clause_guards`, `one_body` and `prefix_clauses` use $I(0, s; u.\, u)$. The frame argument is that of `equality_from_iteration` (Theorem 4.11), applied inside the clause calculus. Theorem 5.17 is in `reduced_clause_calculus` and, in outline, in `clausal_two_case`; Theorem 5.18 is in `clausal_two_case`. `rootfree_clausal` also gives the direct proof of Theorem 5.14.

From here on Φ_2 has two rows, and the lemmas of Section 5.2 are used for C.

5.3.2 Zero-normal terms and one evaluation scheme

This subsection draws on `clausal_single_evaluation`, on the companion calculi of `reduced_clause_calcul` and `rootfree_clausal`, and on `clause_contexts`. The zero computation $I(0, a; u.\, h) = a$ is moved from the rules into the syntax.

Zero-normal terms, the smart constructor $\overline{I}$ and the normalization nf_0 are those of Theorem 4.32, and normal substitution is the normalizing substitution $t\langle s/x\rangle := \mathrm{nf}_0(t[s/x])$ of Theorem 4.25. The map nf_0 acts on formulas, clauses and contexts through their terms, and $B\langle t, r\rangle$ is the corresponding instance of a matrix. Two facts are used below. Normalization is coherent with substitution,

$$\mathrm{nf}_0(t\sigma) \equiv \mathrm{nf}_0\big(\mathrm{nf}_0(t)\,(\mathrm{nf}_0 \circ \sigma)\big)$$

(Theorem 4.34), so normal substitutions compose (Theorem 4.26). And N derives $t = \mathrm{nf}_0(t)$ for every raw term t (Theorem 4.6), so that $\Gamma, F \vdash_{\mathsf{N}} \mathrm{nf}_0(F)$ and $\Gamma, \mathrm{nf}_0(F) \vdash_{\mathsf{N}} F$ for every formula F and context Γ (Theorems 4.27 and 4.35).

Definition 5.20 (The calculus C^0)**.** Terms, formulas and clauses are zero-normal, contexts and the notion of origin are as before, and every substitution performed by a rule is normal substitution. The rules are

$$\frac{}{\Gamma \vdash s = s}\ \textsc{refl} \qquad \frac{\Gamma \vdash A}{\Gamma \vdash B\langle t/y\rangle}\ \textsc{use}$$

$$\frac{\Gamma \vdash \overline{I}(n, a;\, u.\, h) = t \quad \Gamma \vdash I(Sn, a;\, u.\, h) = r}{\Gamma \vdash r = h\langle t/u\rangle}\ \textsc{eval}$$

$$\frac{\Gamma \vdash D_{x,y}B \quad \Gamma, [B\langle b, 0\rangle;\, y.\, B\langle b, y\rangle] \vdash F}{\Gamma \vdash F}\ \textsc{elim}$$

$$\frac{\Gamma, B\langle t, \eta\rangle \vdash B\langle t, Sz\rangle}{\Gamma \vdash D_{x,y}B}\ \textsc{intro}$$

with the side conditions of Theorem 5.4; in particular η is z if B is a literal equation and $t \equiv 0$, and 0 otherwise.

There is one evaluation scheme, the rule STEP of Theorem 4.66; it has no root row and no zero row. The calculus is $\mathsf{C5}$ of `clausal_single_evaluation`, it is `ZCPrf` in the Lean development, and it coincides with the calculus $\mathsf{C}_{\mathrm{Step}}$ that Theorem 4.72 takes from `uniform_step`; Section 4.7.2 derives the native rules in it for that package and proves statement (iv) of Theorem 5.22 below through T_{eq}.

Lemma 5.21 (Derived rules of C^0)**.** *The following hold in C^0 under arbitrary clause contexts.*

(i) The structural operations of Theorem 5.5.

(ii) From $p : s = t$ and $q : s = r$ one derives $r = t$ by two EVAL *nodes and one* REFL *leaf. Hence symmetry and transitivity.*

(iii) The successor computation $I(Sn, a;\, u.\, h) = h\langle\overline{I}(n, a;\, u.\, h)/u\rangle$.

(iv) From $p : s = t$ one derives $h\langle s/u\rangle = h\langle t/u\rangle$ by two EVAL *nodes and two* REFL *leaves, with p used once.*

(v) From $p : s = t$ and $q : h\langle s/u\rangle = r$ one derives $r = h\langle t/u\rangle$ by four EVAL *nodes and three* REFL *leaves, with p and q used once each. Two such steps give transport in an equation.*

(vi) The introduction by cases of Theorem 5.7 for every matrix and every witness.

(vii) Transport in every formula (Theorem 5.8), the closure interfaces (Theorem 5.9) and Theorem 5.10.

Proof. (i) The proofs of Theorem 5.5 apply; injective renaming preserves normal terms.

(ii) to (v) The rule EVAL of C^0 is the rule STEP of Theorem 4.66, and the derivations of Theorem 4.67 consist of REFL, STEP and the given premises, so they are derivations of C^0 over every clause context: (ii) is the pivot with the frame $I(S0, s;\, u.\, u)$, with symmetry and transitivity; (iii) is the successor computation; (iv) is the congruence; (v) is the transport. Replacing one node of a one-term transport rule by the derivation of (v) adds six nodes and duplicates neither premise.

(vi) Only a literal equation at the witness 0 needs an argument. Let $Z \equiv I(S0, 0;\, u.\, 0)$. It is zero-normal and is not literally 0 (Theorem 4.40), and EVAL on the reflexivities of 0 and

of Z proves $Z = 0$. The proof of Theorem 5.7 applies with Z in the place of c. The raw term $I(0, 0;\, u.\, 0)$ cannot be used: it normalizes to 0.

(vii) The proofs of Theorems 5.8 and 5.9 use only the items above, USE, ELIM and INTRO. □

Theorem 5.22 (The zero-normal calculus and N)**.** *(i) For every raw clause context* Γ *and raw formula* F*: if* $\Gamma \vdash_{\mathsf{C}} F$ *then* $\mathrm{nf}_0(\Gamma) \vdash_{\mathsf{C}^0} \mathrm{nf}_0(F)$*.*

(ii) If Γ *is an annotated zero-normal context and* $\Gamma \vdash_{\mathsf{C}^0} F$*, then* $\Gamma^\natural \vdash_{\mathsf{N}} F$*. Here an opened occurrence is* $[B\langle b, 0\rangle;\, y.\, B\langle b, y\rangle]$ *and its native image is* $Q_y B(b, y)$*.*

(iii) For raw Q-free Δ *and* F*:* $\mathrm{ord}(\mathrm{nf}_0\, \Delta) \vdash_{\mathsf{C}^0} \mathrm{nf}_0(F)$ *if and only if* $\Delta \vdash_{\mathsf{N}} F$*. In particular, for zero-normal* Δ *and* F*,* $\mathrm{ord}(\Delta) \vdash_{\mathsf{C}^0} F$ *if and only if* $\Delta \vdash_{\mathsf{N}} F$*.*

(iv) For native Γ *and* A*:* $\mathrm{ord}(\mathrm{nf}_0\, \tau\Gamma) \vdash_{\mathsf{C}^0} \mathrm{nf}_0(\tau A)$ *if and only if* $\Gamma \vdash_{\mathsf{N}} A$*.*

Proof. (i) Induction on the derivation in C, normalizing every sequent. REFL is REFL. For EVAL_0 the normalized premises are $\mathrm{nf}_0(a) = \mathrm{nf}_0(t)$ and $\mathrm{nf}_0(a) = \mathrm{nf}_0(r)$, since $\mathrm{nf}_0(I(0, a;\, u.\, h)) \equiv \mathrm{nf}_0(a)$, and Theorem 5.21(ii) gives $\mathrm{nf}_0(r) = \mathrm{nf}_0(t)$. For EVAL_S the normalized premises are

$$\overline{I}(\mathrm{nf}_0\, n, \mathrm{nf}_0\, a;\, u.\, \mathrm{nf}_0\, h) = \mathrm{nf}_0(t), \qquad I(S\, \mathrm{nf}_0\, n, \mathrm{nf}_0\, a;\, u.\, \mathrm{nf}_0\, h) = \mathrm{nf}_0(r),$$

because the counter $S\, \mathrm{nf}_0(n)$ is not 0. EVAL of C^0 gives $\mathrm{nf}_0(r) = \mathrm{nf}_0(h)\langle \mathrm{nf}_0(t)/u\rangle$, which is $\mathrm{nf}_0(r) = \mathrm{nf}_0(h[t/u])$ by Theorem 4.34. A USE of $[A;\, y.\, B]$ at t becomes a USE of $[\mathrm{nf}_0\, A;\, y.\, \mathrm{nf}_0\, B]$ at $\mathrm{nf}_0(t)$, with conclusion $\mathrm{nf}_0(B)\langle \mathrm{nf}_0(t)/y\rangle \equiv \mathrm{nf}_0(B[t/y])$. For ELIM the normal form of the added clause $[B(b, 0);\, y.\, B(b, y)]$ is $[\mathrm{nf}_0(B)\langle b, 0\rangle;\, y.\, \mathrm{nf}_0(B)\langle b, y\rangle]$, the clause that ELIM of C^0 adds for $D_{x,y}\, \mathrm{nf}_0(B)$. For INTRO with matrix B and witness t the normalized premise is $\mathrm{nf}_0(\Gamma), \mathrm{nf}_0(B)\langle \mathrm{nf}_0\, t, \eta\rangle \vdash \mathrm{nf}_0(B)\langle \mathrm{nf}_0\, t, Sz\rangle$. The matrix $\mathrm{nf}_0(B)$ is a literal equation exactly when B is one. If the mode of INTRO for $(\mathrm{nf}_0\, B, \mathrm{nf}_0\, t)$ is the mode for (B, t), apply INTRO of C^0. The modes differ only when B is a literal equation, $t \not\equiv 0$ and $\mathrm{nf}_0(t) \equiv 0$. Then the source is in case mode, the normalized premise is the premise of an introduction by cases at the witness 0, and Theorem 5.21(vi) applies.

(ii) Induction on the derivation, as in Theorem 5.11(i). REFL is reflexivity. For EVAL, N derives $I(n, a;\, u.\, h) = \overline{I}(n, a;\, u.\, h)$ by the zero computation or by reflexivity, hence $I(n, a;\, u.\, h) = t$ from the first premise; the successor computation, Leibniz replacement and Theorem 4.27 give $I(Sn, a;\, u.\, h) = h[t/u] = \mathrm{nf}_0(h[t/u])$, and the second premise gives $r = h\langle t/u\rangle$. A USE of an assumption occurrence is HYP. For a USE of an opened occurrence the translated premise is $B\langle b, 0\rangle$; Theorem 4.27 gives $B(b, 0)$, QE on $Q_y B(b, y)$ gives $B(b, t)$, and Theorem 4.27 gives $B\langle b, t\rangle$. ELIM is DE. For INTRO the translated premise is $\Gamma^\natural, B\langle t, \eta\rangle \vdash B\langle t, Sz\rangle$; Theorem 4.27 and cut in N give $\Gamma^\natural, B(t, \eta) \vdash B(t, Sz)$, and the two modes are treated as in Theorem 5.11(i).

(iii) If $\Delta \vdash_{\mathsf{N}} F$ then $\mathrm{ord}(\Delta) \vdash_{\mathsf{C}} F$ by Theorem 5.11(iii), and (i) gives $\mathrm{ord}(\mathrm{nf}_0\, \Delta) \vdash_{\mathsf{C}^0} \mathrm{nf}_0(F)$, because $\mathrm{nf}_0(\mathrm{ord}(G)) \equiv \mathrm{ord}(\mathrm{nf}_0\, G)$. Conversely (ii) gives $\mathrm{nf}_0(\Delta) \vdash_{\mathsf{N}} \mathrm{nf}_0(F)$, and Theorem 4.27 with cut gives $\Delta \vdash_{\mathsf{N}} F$.

(iv) Combine (iii) for $\tau\Gamma$ and τA with Theorem 5.2(iii). □

Corollary 5.23 (Substitution)**.** *Normal substitution of terms for free variables is admissible in* C^0 *for sequents with ordinary assumptions.*

Proof. By Theorem 5.22(iii), substitution in N (Section 1.4) and $\mathrm{nf}_0(F\sigma) \equiv \mathrm{nf}_0(F)\langle \mathrm{nf}_0 \circ \sigma\rangle$ from Theorem 4.34. A witness may become the literal 0 under the substitution, so the rule at a node is not always preserved. □

Remark 5.24. In the Lean development (iv) is `zcprf_iff_nprf`, (iii) is `zcprf_iff_nprf_of_qfree` and the translation of (i) is `CPrf.toZC` (`NativeZeroClausal`). `clausal_single_evaluation`

proves (iii) for zero-normal sequents by comparison with the kernel N^0 on zero-normal terms (Theorem 4.38); the cases are those of Theorem 5.11 with Theorem 5.21 in the place of the lemmas of Section 5.2. The proof above normalizes derivations of C, which is the route of the companion of `reduced_clause_calculus` and of the Lean proof. The change of mode in (i) and its repair at the witness $I(S0, 0; u.\, 0)$ are stated in `reduced_clause_calculus` and `clausal_single_evaluation`. The count in Theorem 5.21(iv) and (v) is from `clausal_single_evaluation`; the congruence of Theorem 5.6 has four evaluations because it joins two separately evaluated endpoints. `rootfree_clausal` states Theorem 5.21(ii) to (iv) for its companion, whose introduction is that of Section 5.4.3.

5.3.3 Reflexivity as one fixed clause

This subsection draws on `raw_clausal` for the obstruction and on `uniform_clauses` for the calculus.

Theorem 5.25 (Closed seeds do not replace reflexivity)**.** *Let S be a finite set of closed equations that are true in the standard natural numbers, and let $M \subseteq \mathbb{N}$ be the finite set of values of their sides. Let C_3^S be C_3 with* REFL *deleted and the members of S added as axioms in every context. If $k \notin M$, then C_3^S does not derive $S^k0 = S^k0$ from the empty context. In particular the calculus with the single axiom $0 = 0$ in the place of* REFL *does not derive $S0 = S0$, and the calculus with* REFL *deleted derives no equation from the empty context.*

Proof. Evaluate terms in the standard natural numbers, the iterator by ordinary iteration. Give the equation atom the truth condition

$$[\![s = t]\!]_\nu \quad :\Leftrightarrow \quad [\![s]\!]_\nu = [\![t]\!]_\nu \in M.$$

Interpret $D_{x,y}B$ as $\exists x\,([\![B(x,0)]\!] \to \forall y\,[\![B(x,y)]\!])$ and a clause $[A;\, y.\, B]$ as $[\![A]\!] \to \forall y\,[\![B(y)]\!]$, with two-valued truth. A sequent is valid if its formula is true under every assignment that makes its clauses true. Every rule preserves validity.

Each seed is true. For EVAL let the premises $s = t$ and $b = r$ be true, so that $[\![s]\!] = [\![t]\!] \in M$ and $[\![b]\!] = [\![r]\!] \in M$. In each row $\Phi(s, t, b)$ has the value of b: in the root row $[\![t]\!] = [\![s]\!] = [\![b]\!]$; in the zero row $[\![t]\!] = [\![a]\!] = [\![I(0, a;\, u.\, h)]\!]$; in the successor row $[\![h[t/u]]\!] = [\![h[s/u]]\!] = [\![I(Sn, a;\, u.\, h)]\!]$, because $s \equiv I(n, a;\, u.\, h)$ and t have the same value. Hence the conclusion $r = \Phi(s, t, b)$ is true. USE is the elimination of an implication and of a universal quantifier. ELIM is existential elimination; the fresh variable takes the value of the witness, and neither the context nor the conclusion depends on it. For INTRO fix an assignment that satisfies Γ. The clause $\mathrm{ord}(F)$ is true whenever F is. In case mode, if $B(t, 0)$ is true then the premise makes $B(t, Sn)$ true for every standard n; every number is 0 or a successor, so every instance is true and the conditional closure holds at the value of t. If $B(t, 0)$ is false the closure holds trivially. In progress mode the premise states that $B(0, n)$ implies $B(0, Sn)$ for every standard n; induction on the standard numbers gives every instance from $B(0, 0)$. The argument does not use that the atoms mean identity.

So every theorem of C_3^S is true in the interpretation, and $S^k0 = S^k0$ is false for $k \notin M$. For the two special cases take $S = \{0 = 0\}$, $M = \{0\}$, and $S = \emptyset$. □

The interpretation is a counter-interpretation of an inference system. It is not a structure in which the equality symbol is identity, and it is not the arithmetic interpretation of the calculus. For contexts of native formulas the case $S = \emptyset$ is proved syntactically in Theorem 4.23; the theorem adds the closed seeds. The clause

$$[\, 0 = 0;\, y.\, y = y \,]$$

is false in it. Retained in every context, together with a derivation of $0 = 0$, it restores REFL: USE at t gives $t = t$. This relocates universal reflexivity into the initial context; the axiom and the cost of validating it remain part of the specification. The calculus that follows makes this choice and also removes the neutral antecedent.

Definition 5.26 (The calculus V)**.** A *uniform clause* is $[\Pi;\, y.\, B(y)]$ where Π is empty or one formula and y does not occur in Π; it reads $\forall y\, B(y)$, respectively $A \to \forall y\, B(y)$. The *reflexivity clause* and the clause of an assumption are

$$\mathcal{R} := [\,;\, y.\, y = y\,], \qquad \langle A\rangle := [\,;\, v.\, A\,] \quad (v \notin \mathrm{FV}(A)).$$

The rules are

$$\frac{\Gamma \vdash A_1 \;\cdots\; \Gamma \vdash A_k}{\Gamma \vdash B(t)}\ \textsc{use} \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash b = r}{\Gamma \vdash r = \Phi_2(s,t,b)}\ \textsc{eval}$$

$$\frac{\Gamma \vdash D_{x,y}B \quad \Gamma, [B(b,0);\, y.\, B(b,y)] \vdash F}{\Gamma \vdash F}\ \textsc{elim}$$

$$\frac{\Gamma, \langle B(t,\eta)\rangle \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B}\ \textsc{intro}$$

where USE selects an occurrence $[A_1, \ldots, A_k;\, y.\, B]$ of Γ with $k \in \{0, 1\}$, and the side conditions and η are those of Theorem 5.4. For a context Γ of uniform clauses, Γ° replaces each occurrence $[\,;\, y.\, B]$ by $[0 = 0;\, y.\, B]$.

There are four rule families and one fixed axiom clause. USE of $\mathcal{R}$ at t gives $t = t$, and USE of $\langle A\rangle$ gives A; both have no premise. Reflexivity is not eliminated as a principle; it is the clause $\mathcal{R}$. The calculus is `VPrf` in the Lean development.

Theorem 5.27 (Uniform clauses)**.** *For every context Γ of uniform clauses and every formula A,*

$$\mathcal{R}, \Gamma \vdash_{\mathsf{V}} A \quad \Longleftrightarrow \quad \Gamma^\circ \vdash_{\mathsf{C}} A.$$

Hence, for Q-free Δ and F, $\mathcal{R}, \langle\Delta\rangle \vdash_{\mathsf{V}} F$ if and only if $\Delta \vdash_{\mathsf{N}} F$. The translations leave terms and formulas unchanged. The translation from C to V does not increase the number of nodes; the converse adds at most one REFL node for each use of a clause without antecedent.

Proof. The occurrences of Γ and of Γ° correspond by position, and $\mathcal{R}$ is the first occurrence on the left. Both directions are inductions on derivations, over all Γ.

From right to left: REFL at t becomes USE of $\mathcal{R}$ at t. A USE of an occurrence $[0 = 0;\, y.\, B]$ that corresponds to $[\,;\, y.\, B]$ becomes the USE without premise, and the derivation of $0 = 0$ is discarded. A USE of an occurrence that corresponds to a clause with an antecedent keeps its translated premise. EVAL is copied. ELIM adds the same clause on both sides, and INTRO adds $\mathrm{ord}(B(t,\eta))$, which is $\langle B(t,\eta)\rangle^\circ$. Witnesses, modes and eigenvariables are unchanged.

From left to right: a USE of $\mathcal{R}$ at t becomes REFL at t. A USE of $[\,;\, y.\, B]$ becomes a USE of $[0 = 0;\, y.\, B]$ with premise REFL at 0. The other rules are copied.

The second statement follows from Theorem 5.11(iii), since $\langle\Delta\rangle^\circ = \mathrm{ord}(\Delta)$. □

Remark 5.28. In the Lean development the first statement is `vprf_iff_cprf` and the second is `vprf_iff_nprf_of_qfree` (`NativeUniformClauses`). In the native reading of a context of V that is reachable from assumptions, $\mathcal{R}$ supplies reflexivity, $\langle A\rangle$ is the assumption A and an opened clause is $Q_y B(b,y)$, so USE is reflexivity, HYP or QE.

5.3.4 Four schemes on zero-normal terms

This subsection draws on `clause_contexts`.

Definition 5.29 (The calculus V^0)**.** V^0 has zero-normal terms, normal substitution, uniform clauses and the clause $\mathcal{R}$, with the four schemes USE (as in V), STEP (the rule EVAL of C^0), OPEN (the rule ELIM of C^0) and CLOSE (the rule INTRO of C^0 with the assumption stored as $\langle B\langle t, \eta\rangle\rangle$).

Theorem 5.30 (Clause factorization)**.** *For every zero-normal context Γ of uniform clauses and every zero-normal formula A, $\mathcal{R}, \Gamma \vdash_{\mathsf{V}^0} A$ if and only if $\Gamma^\circ \vdash_{\mathsf{C}^0} A$. Hence, for raw Q-free Δ and F, $\mathcal{R}, \langle \mathrm{nf}_0\, \Delta\rangle \vdash_{\mathsf{V}^0} \mathrm{nf}_0(F)$ if and only if $\Delta \vdash_{\mathsf{N}} F$.*

Proof. The proof of Theorem 5.27 applies; it does not inspect the evaluation rule or the terms. The second statement is Theorem 5.22(iii). □

In the Lean development V^0 is `ZVPrf` and the two statements are `zvprf_iff_zcprf` and `zvprf_iff_nprf`.

Remark 5.31 (The atomic case without cut). `clause_contexts` recovers the introduction by cases of a literal equation $E(x, y)$ at the witness 0 without the cut of Theorem 5.5; Section 4.7.2 gives the same derivation for `uniform_step`. Let $q \equiv I(S0, 0; u.\, u)$, which is not literally 0 and satisfies $q = 0$ by Theorem 5.21(ii). Given $\Gamma, E(0,0) \vdash E(0, Sz)$, apply CLOSE to the matrix $E(0, y)$ with an unused first variable at the witness q; this is case mode, its premise is the given derivation, and it gives $D_{v,y}E(0, y)$. OPEN adds the clause $[E(0,0); y.\, E(0, y)]$. Now apply CLOSE to $E(x, y)$ at the witness q: its assumption $E(q, 0)$ transports to $E(0, 0)$, USE gives $E(0, Sz)$, and transport gives $E(q, Sz)$.

5.4 Variants of the introduction

This section draws on `clause_guards` and `rootfree_clausal`. The introduction of C selects its mode by the root of the matrix and by the literal witness. A substitution can change the second test (Theorem 5.14). This section gives introductions whose mode is selected by a guard on the matrix alone, and one whose mode is selected by the instantiated zero base.

5.4.1 The diagonal guard

This subsection draws on `clause_guards`. The diagonal shape, its instance at zero, its lift and its stability under substitution are those of the diagonal introduction with native contexts (Theorems 3.12, 3.48 and 3.55); what follows places that introduction in the clause calculus C.

Definition 5.32 (The calculus K)**.** Write $\mathrm{diag}_y(B)$ if the matrix $B(x, y)$ is diagonal in y (Theorem 3.12(c)), that is, literally $I(y, y; u.\, h) = I(y, y; v.\, k)$ with y the inner variable of the enclosing $D_{x,y}$ and h, k arbitrary. The test is made on the matrix before the witness is substituted. K is C with the introduction

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B}\ \text{INTRO}, \qquad \eta_B(z) := \begin{cases} z, & \mathrm{diag}_y(B), \\ 0, & \text{otherwise}, \end{cases}$$

with z fresh for Γ, t and B.

No rule of K inspects a witness. Theorem 5.5 and Theorem 5.6 hold in K with the same proofs: they use REFL, EVAL and USE only, and injective renaming preserves the diagonal test.

Lemma 5.33 (Derived rules of K). *(i) If* $\mathrm{diag}_y(B)$ *then* $\Gamma \vdash B(t,0)$ *for every* t *and* Γ.

(ii) The introduction by cases of Theorem 5.7 is admissible for every matrix and every witness.

(iii) The closure interfaces (i) to (iii) of Theorem 5.9, numerical cases for every formula, and transport in every formula (Theorem 5.8) hold.

Proof. (i) This is condition (S1) of Theorem 3.55(i): the instance is $I(0,0;\, u.\, h') = I(0,0;\, v.\, k')$, both sides equal 0 by the zero computation, and symmetry and transitivity join them. These are the derivations of Theorem 5.6, which read no clause.

(ii) For a matrix that is not diagonal this is INTRO. For a diagonal matrix cut the theorem $B(t,0)$ of (i) into the given branch $\Gamma, B(t,0) \vdash B(t,Sz)$. The result $\Gamma \vdash B(t,Sz)$ is weakened by the assumption $B(t,z)$, and INTRO in progress mode applies.

(iii) The proofs of Theorem 5.9(i) to (iii) and of Theorem 5.8 use the introduction by cases, USE and ELIM. Numerical cases follow from interfaces (i) and (ii). □

Equation induction is not immediate in K, since progress is restricted to diagonal matrices. For a literal equation $E(y) \equiv (s(y) = t(y))$ let $H_E(y) \equiv \Delta_y E$ be the diagonal lift of Theorem 3.12,

$$H_E(y) := (L_s(y) = L_t(y)), \qquad L_s(y) := I(y,y;\, u.\, s(y)), \quad L_t(y) := I(y,y;\, u.\, t(y)),$$

with u not in s, t. The matrix H_E is diagonal and $H_E(0)$ is a theorem by Theorem 5.33(i). The successor computation gives $L_s(Sz) = s(Sz)$ and $L_t(Sz) = t(Sz)$ (condition (S2) of Theorem 3.55(i)), hence

$$H_E(Sz) \dashv\vdash E(Sz). \tag{5.1}$$

Numerical cases prove the two equations of the equational decoder that follows Theorem 3.55,

$$\mathrm{sel}(y, s(0), L_s(y)) = s(y), \qquad \mathrm{sel}(y, t(0), L_t(y)) = t(y) \tag{5.2}$$

without induction: at 0 both sides of the first are $s(0)$ after the zero computation, and at Sz the selector and the iterator with constant body compute to $s(Sz)$; likewise for t. Under $E(0)$, congruence along $s(0) = t(0)$ and along $H_E(y)$ joins the left sides of (5.2), so

$$E(0),\ H_E(y) \vdash E(y). \tag{5.3}$$

Theorem 5.34 (Equation induction in K). *For every literal equation* $E(y)$*, every context* Γ *and every term* q*, derivations of* $\Gamma \vdash E(0)$ *and of* $\Gamma, E(z) \vdash E(Sz)$*,* z *fresh, yield a derivation of* $\Gamma \vdash E(q)$*. Consequently interface (iv) of Theorem 5.9 and Theorem 5.10 hold in* K.

Proof. Under the assumption $H_E(z)$, (5.3) and the given derivation of $E(0)$ give $E(z)$. Cut this into the given step derivation at its own variable z; the result is $E(Sz)$, and (5.1) gives $H_E(Sz)$. INTRO on the diagonal matrix H_E with an unused first variable proves $q_y H_E$. Its zero instance is a theorem, so interface (ii) gives $H_E(q)$, and (5.3) gives $E(q)$. No term is substituted in the given step derivation, and the induction that is being derived is not used for (5.2).

For interface (iv), work in $\Gamma, E(0)$. The first part gives $E(Sw)$ for a fresh w, and interface (i) gives $\Gamma \vdash q_y E$. With a witness, Theorem 5.33(ii) gives the third statement of Theorem 5.10. □

Theorem 5.35 (The diagonal clause calculus and N). *The four statements of Theorem 5.11 hold for* K.

Proof. (i) is proved as there, except for INTRO. An introduction with a matrix that is not diagonal is the introduction by cases of N. A diagonal matrix is a literal equation $E(x, y)$; the translated premise is $\Gamma^\natural, E(t, z) \vdash E(t, Sz)$, the introduction of Q by progress gives $Q_y E(t, y)$, and the introduction by cases at t with QE gives $D_{x,y}E$. (ii) is proved as there, with Theorem 5.33(ii) for the introduction by cases and Theorem 5.34 for the introduction of Q by progress. (iii) and (iv) follow as there. □

If σ substitutes terms for free variables of a matrix $B(x, y)$, lifted under x and y, then $\mathrm{diag}_y(B\sigma)$ if and only if $\mathrm{diag}_y(B)$: the root is preserved and reflected, and so is the diagonal shape of a literal equation (Theorem 3.15(i) and (iii)).

Theorem 5.36 (Substitution preserves the derivation tree)**.** *If $\Gamma \vdash_{\mathsf{K}} A$ and σ is a substitution of terms for free variables, then $\Gamma\sigma \vdash_{\mathsf{K}} A\sigma$ by a derivation with the same tree of rules, the same selected clause occurrences and the same evaluation rows, after a consistent renaming of eigenvariables.*

Proof. Induction on the derivation. REFL is immediate. The literal matches of each row of Φ_2 are preserved and substitution commutes with the instantiation of the body. USE keeps its occurrence and its antecedent. ELIM uses a fresh variable in the substituted context. For INTRO, Theorem 3.15(i) and (iii) fix the mode, so the substituted branch is the branch that the same rule requires. The argument is that of Theorem 3.55(iii) and Theorem 3.53(i); the clause setting adds only that USE and ELIM commute with σ and keep the selected occurrence. □

Remark 5.37. In the Lean development K is `KPrf`, the equivalences are `kprf_iff_nprf` and `kprf_iff_nprf_of_qfree`, and Theorem 5.36 is `KPrf.subst` (`NativeClausalDiagonal`). `clause_guards` compares K with the six-rule native calculus that has the diagonal introduction (`SPrf`), for which the translation τ preserves and reflects the diagonal shape, so that introductions are copied; the seven-rule native calculus with the same introduction is $\mathsf{N}_{\mathrm{diag}}$ of Theorem 3.55. C has substitution as an admissible rule (Theorem 5.14); the statement for K is that the derivation is not repaired. Theorem 5.34 differs from the derivation of QI in Theorem 3.52: it uses the decoder (5.3) and the closure interfaces of the clause calculus in place of the implication between equations of Theorem 3.51.

5.4.2 Guards with a derivable base

This subsection draws on `clause_guards`; the method of repairing an introduction after a one-way change of guard is that of `reflexive_base`. It is the clause form of the selector criterion with native contexts (Theorems 3.48, 3.52 and 3.53): conditions (a) and (b) below are the requirement on a selector and (S1), and (c) is (S2) together with a decoder of the lift.

Let $G_y(B)$ be a decidable predicate on matrices that is invariant under renaming of bound variables and under injective renaming of free variables. K_G is K with G in the place of diag. Consider the following conditions.

(a) Every matrix that satisfies G is a literal equation.

(b) For every matrix B that satisfies G and every t, the zero instance $B(t, 0)$ is derivable by REFL and EVAL.

(c) For every literal equation $E(y)$ there is a matrix $H_E(y)$ that satisfies G, with derivations of $H_E(Sz) \dashv\vdash E(Sz)$ and of $E(0), H_E(y) \vdash E(y)$ that use REFL, EVAL, USE, ELIM and introductions by cases only.

Theorem 5.38 (Guard criterion)**.** *Under (a), (b) and (c) the four statements of Theorem 5.11 hold for K_G. If moreover G is preserved by substitution of terms for free variables of the matrix,*

substitution is admissible in K_G by a direct transformation of derivations that inserts nodes only at introductions whose matrix acquires the guard. If G is also reflected by substitution, the transformation preserves the derivation tree.

Proof. Condition (b) gives the introduction by cases for guarded matrices as in Theorem 5.33(ii), hence the closure interfaces, numerical cases and transport in formulas. Condition (c) and the proof of Theorem 5.34, with the given lift in the place of the diagonal lift, give equation induction and interface (iv). A guarded introduction is sound in N because by (a) it is an instance of progress for a literal equation; an unguarded one is an introduction by cases. The proof of Theorem 5.35 then applies.

For substitution induct on the derivation. A guarded matrix remains guarded, and an unguarded matrix that remains unguarded keeps its rule. If an unguarded matrix becomes guarded, the substituted branch assumes the substituted zero instance; (b) proves that instance, cut removes the assumption, the branch is weakened by the progress assumption, and the guarded INTRO applies. If G is reflected this case does not occur. □

Proposition 5.39 (The mirror guard on the full language)**.** *Let* $\mathrm{mirror}_y(B)$ *hold if* $B \equiv (a = I(y, a;\, u.\, h))$*. The guard satisfies (a), (b) and (c) and is preserved by substitution. It is not reflected by substitution.*

Proof. The guard is the mirrored shape of Theorem 3.12(b). (a) holds by definition. (b) is condition (S1) of Theorem 3.56(i): the zero instance is $a' = I(0, a';\, u.\, h')$, the symmetric form of the zero computation. (c): for $E(y) \equiv (s(y) = t(y))$ the lift is the mirror gate $M_E(y) \equiv H_y E \equiv (s(y) = I(y, s(y);\, u.\, t(y)))$, $u \notin \mathrm{FV}(t)$, whose successor instance computes to $E(Sz)$, and $E(0), M_E(y) \vdash E(y)$ is the equational decoder of Theorem 3.57. Both derivations use the computation equations, equality and numerical cases only, which are available in the clause calculus by Theorem 5.6 and interfaces (i) and (ii) of Theorem 5.9, the latter from introductions by cases, USE and ELIM. Preservation and the failure of reflection are Theorem 3.15(ii); for distinct free variables p, q the matrix $p = I(y, q;\, u.\, h)$ is not guarded, and the substitution of p for q makes it guarded. □

Every matrix that is not guarded takes the case branch, so the mirror calculus is defined on the whole Q-free language, as the mirror selector with native contexts is on the full native language (Theorem 3.56); `mirror_guard` restricts the public language instead (Theorem 3.43). By Theorem 5.38 the mirror calculus is equivalent to N and has admissible substitution with a local repair, and it lacks the exact statement of Theorem 5.36. A guard that tests one iterator side without its relation to the initializer does not satisfy (b).

5.4.3 The fixed zero base on zero-normal terms

This subsection draws on `rootfree_clausal`; the guard is that of `reflexive_base` (Theorem 4.45), restricted from an arbitrary literally reflexive base $r = r$ to the fixed base $0 = 0$, and placed in the clause calculus C^0.

Definition 5.40 (The calculus $\mathsf{C}^0_{\mathrm{b}}$)**.** $\mathsf{C}^0_{\mathrm{b}}$ is C^0 with the introduction

$$\frac{\Gamma, B\langle t, \eta\rangle \vdash B\langle t, Sz\rangle}{\Gamma \vdash D_{x,y} B}\ \textsc{intro}$$

in which

$$\eta := \begin{cases} z, & B\langle t, 0\rangle \equiv (0 = 0), \\ 0, & \text{otherwise.} \end{cases}$$

The test is one comparison of the computed base with a fixed formula. Substitution and normalization preserve the root of a formula, so the test excludes matrices with root D; there is no separate test of the root or of the witness.

Theorem 5.41 (The fixed zero base)**.** *For every zero-normal clause context* Γ *and formula* F*,* $\Gamma \vdash_{\mathsf{C}^0_\mathrm{b}} F$ *if and only if* $\Gamma \vdash_{\mathsf{C}^0} F$*. Hence Theorem 5.22(iii) and (iv) hold for* C^0_b*.*

Proof. The two calculi differ in the introduction only. Theorem 5.21(i) to (v) hold in C^0_b with the same proofs.

Cases in C^0_b*.* Let $\Gamma, B\langle t, 0\rangle \vdash B\langle t, Sz\rangle$. If the base is not $0 = 0$ the rule applies. If it is $0 = 0$, cut REFL into the branch, weaken by the assumption $B\langle t, z\rangle$ and apply the rule in progress mode, as in Theorem 4.47. No auxiliary witness is needed. The closure interfaces (i) to (iii) and transport in formulas follow.

Equation progress in C^0_b*.* Let $E(y) \equiv (s(y) = t(y))$ and let $\Gamma, E(z) \vdash E(Sz)$ be given; the conclusion is $\Gamma \vdash q_y E$. The derivation is that of Theorem 4.50, with the progressive lift $H(y) \equiv (\mathrm{sel}(y, 0, s(y)) = \mathrm{sel}(y, 0, t(y)))$ and the two decoders Dec_{s,s_0} and Dec_{t,s_0} of Theorem 4.49, $s_0 \equiv s\langle 0\rangle$, the selector on zero-normal terms being $\mathrm{sel}(v, a, b) \equiv \overline{I}(v, a;\, u.\, b)$. It transfers to the clause calculus as follows. The closure by cases (4.1) is interface (i) of Theorem 5.9 with q_y in the place of Q_y, by the introduction by cases just derived; QE is interface (ii); equality, congruence, weakening and cut are Theorem 5.21(i) to (v). The zero instance $H\langle 0\rangle$ is literally $0 = 0$, so the introduction of C^0_b on the matrix H with an unused first variable is in progress mode, as RI is there, and it concludes $q_y H$ directly, without the DE step of that proof. As there, nothing is substituted into the given derivation and no equation induction is used before the guarded rule is applied.

From C^0 *to* C^0_b*.* An INTRO of C^0 in case mode is an introduction by cases. In progress mode the matrix is a literal equation and the witness is 0; equation progress gives $q_y B\langle 0, y\rangle$, and the introduction by cases at the witness 0 with interface (ii) gives $D_{x,y}B$.

From C^0_b *to* C^0*.* If the guard is false the rule is an introduction by cases (Theorem 5.21(vi)). If it is true the matrix is a literal equation, interface (iv) of C^0 applied to the matrix $B\langle t, y\rangle$ gives $q_y B\langle t, y\rangle$, its base is $0 = 0$, and interface (ii) gives $B\langle t, Sz\rangle$; the introduction by cases at the witness t concludes $D_{x,y}B$. □

If the base is literally $0 = 0$ it remains so under substitution, since $\mathrm{nf}_0(\mathrm{nf}_0(G)\sigma) \equiv \mathrm{nf}_0(G\sigma)$. A base that is not $0 = 0$ can become $0 = 0$, which changes an introduction by cases into a progressive one; the derived introduction by cases repairs that instance, as in Theorem 4.48. So the guard is preserved and not reflected, as for the mirror guard.

5.5 Variants of the context

This section draws on `prefix_clauses`, `clause_contexts` and `one_body`. A context entry of C has an antecedent and a body. In V the antecedent is optional (Section 5.3.3). This section gives the representation of a formula by its prefix, with clauses that keep one antecedent, and two calculi whose context entries have no antecedent field at all.

5.5.1 The prefix representation

This subsection draws on `prefix_clauses` and on the ranked representation of `clause_contexts`.

Lemma 5.42 (Unique prefix)**.** *Every Q-free formula is, in exactly one way up to the names of bound variables,*

$$D_{x_1,y_1} \cdots D_{x_k,y_k}(s = t), \qquad k \geq 0.$$

The number k *is unchanged by substitution of terms. A native formula has this form if and only if it is Q-free.*

Proof. An equation has $k = 0$. A formula $D_{x,y}B$ has exactly one formula child; the grammar has no branching constructor, so removing binders reaches exactly one equation, and restoring them in order reconstructs the formula. Substitution acts on the two terms of the leaf. □

The formula is therefore the record $[k, s, t]$, written $D^k(s = t)$: a number and two terms, with $2k$ bound variables in scope at the leaf. Only term syntax is recursive. The record is a bijective encoding of the Q-free formulas; it is not a prenex normal form theorem for ordinary formulas, and each D keeps its conditional reading. With de Bruijn indices starting at 1, the two outer variables of the matrix of $D^{k+1}(s = t)$ are the indices $2k + 2$ (the witness) and $2k + 1$ at the leaf, and an iterator body adds one to the index sought.

Lemma 5.43 (Opening commutes with packing)**.** *The record of an instance $B(t, r)$ of the matrix of $D^{k+1}(s = t)$ is obtained from $[k, s, t]$ by substituting at the index $2k + 2$ and then at the index $2k + 1$, with larger indices decreased when a variable is removed and the index sought increased by one under each iterator body.*

Proof. Each removed pair of binders adds two to the depth at which the substitution acts. At the leaf induct on the two terms: a variable is replaced at the indicated index, S adds no binder and the body of an iterator adds one. These are the clauses of capture-avoiding substitution on the nested representation. □

In the Lean development the predicate `NFm.Pre k A` states that A is k binders D followed by one equation, with `NFm.qfree_iff_pre`, `NFm.Pre.unique` and `NFm.Pre.subst`; `VPrf.pre` states that V derives only such formulas from Q-free assumptions (`NativePrefix`).

Definition 5.44 (The calculus PF)**.** Formulas are records $D^k(s = t)$ on raw terms. A formula is *reflexive at its leaf* if it is $D^k(r = r)$ with literally identical terms. Clauses are $[A;\, y.\, B(y)]$ with one antecedent, an assumption A is $[0 = 0;\, v.\, A]$, and every context contains the clause

$$\rho := [\, 0 = 0;\, y.\, y = y\,].$$

The rules are USE, EVAL with Φ_2, ELIM and INTRO of C, with one change: in USE of $[A;\, y.\, B]$ the premise is omitted when A is reflexive at its leaf. In INTRO the test that the matrix is a literal equation is the test that the conclusion has $k = 1$.

There are four rule families and the fixed clause ρ. The test of reflexivity at the leaf compares two term trees and evaluates nothing.

Lemma 5.45 (Reflexive prefixes)**.** *Every formula $D^k(r = r)$ is derivable in every context in C, in V with $\mathcal{R}$, and in PF.*

Proof. Induction on k. For $k = 0$ use REFL in C, USE of $\mathcal{R}$ in V, and USE of ρ in PF, where the antecedent $0 = 0$ is reflexive and the premise is omitted. For $k > 0$ apply INTRO at the witness $S0$, which is case mode whatever the matrix. The target at the fresh successor has $k - 1$ binders and identical terms at its leaf, since both receive the same substitutions; the induction hypothesis proves it, and the added clause is not used. □

Theorem 5.46 (The prefix calculus)**.** *For every clause context Γ and formula A, with formulas packed on the left,*

$$\rho, \Gamma \vdash_{\mathsf{PF}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{C}} A.$$

Hence $\rho, \mathrm{ord}(\Delta) \vdash_{\mathsf{PF}} F$ if and only if $\Delta \vdash_{\mathsf{N}} F$, for Q-free Δ and F. Both formula round trips between records and nested formulas are identities.

Proof. From right to left: pack all formulas and add ρ. REFL at t becomes USE of ρ at t without premise. The other rules translate directly; by Theorem 5.43 substitutions agree, and the free variables are unchanged, so the eigenvariable conditions agree. From left to right: a USE of the distinguished ρ becomes REFL. Any other USE keeps its translated premise; if the premise was omitted, Theorem 5.45 supplies a derivation of the antecedent in C. EVAL, ELIM and INTRO translate directly. No cut is needed. The second statement is Theorem 5.11(iii). □

By Theorem 5.27 the same holds against V, with the empty antecedent represented by $0 = 0$. `prefix_clauses` states the theorem against `CPrf` and `VPrf`.

5.5.2 Rank from variable support

This subsection draws on the optional notes of `clause_contexts`. It shows that the number k of a public record can be recovered from the variables of its leaf, at the price of identifying some theorems.

Work in V^0 with records and de Bruijn indices. For a term t let $\mu(t)$ be the greatest positive index that is free relative to the iterator binders of t, and 0 if there is none: $\mu(j) = \max(j, 0)$, $\mu(St) = \mu(t)$ and $\mu(I(n, a;\, u.\, h)) = \max(\mu(n), \mu(a), \max(\mu(h) - 1, 0))$. For a record $F = [k, s, t]$ let $r(F) := \lceil \max(\mu(s), \mu(t))/2 \rceil$, so that $r(F) \leq k$. Define

$$P[k, s, t] := \begin{cases} (s, t), & r(F) = k, \\ (0, 0), & r(F) < k, \end{cases} \qquad U(s, t) := \big[\lceil \max(\mu(s), \mu(t))/2 \rceil,\ s,\ t\big].$$

The pair $(0, 0)$ decodes to $0 = 0$. The quotient calculus has pairs as the formulas of its judgments and records in its clauses, which keep their rank: USE asks for derivations of $P(A_i)$ and returns $P(B\langle t/y\rangle)$; STEP acts on pairs of rank zero; OPEN decodes a major premise of nonzero rank, adds its clause as a record and derives the same pair in the branch; CLOSE decodes a goal of nonzero rank, adds its assumption as a record and asks for the packed successor instance. P is applied only to formulas all of whose clause variables are instantiated, and it is recomputed after every normal substitution.

Proposition 5.47 (Support quotient)**.** *For every record F, F and $U(P(F))$ are interderivable in V^0 in every context that contains $\mathcal{R}$, and $P(U(a)) = a$ for every pair a. The quotient calculus derives $P(F)$ from $\mathcal{R}$ if and only if V^0 derives F from $\mathcal{R}$.*

Proof. If $r(F) = k$ then $U(P(F))$ is F. If $r(F) < k$, the two variables of the outermost binder, with the indices $2k$ and $2k - 1$, do not occur in the leaf, so $F \equiv D_{x,y}B$ with x, y not in B. CLOSE at the witness 0 has B both as its assumption and as its target in either mode, so F is a theorem; $0 = 0$ is a theorem; and each follows from the other by weakening. $P(U(a)) = a$ because U uses the rank that P tests.

Soundness of the quotient rules: by the first part a derivation of $U(P(A))$ converts to one of A and conversely, by cut and weakening. USE converts its packed premises, applies USE of V^0 and converts the result. STEP involves rank zero only. CLOSE converts the local base and the target and applies CLOSE; the decoded goal has the rank derived from its support and packs to the same pair. OPEN decodes a record with root D and adds the clause of OPEN.

Completeness: transform a derivation of V^0, packing every conclusion. USE and STEP transform directly. A CLOSE whose goal keeps its rank is the same CLOSE; if the goal collapses, USE of $\mathcal{R}$ at 0 proves $0 = 0$ and the premise is discarded. OPEN transforms directly when its major premise keeps its rank. If the major premise $D_{x,y}B$ collapses, the added clause is $[B;\, y.\, B]$, and each use of it returns the formula of its own premise; delete those uses and keep the premise derivations, removing the clause from the branch. The branch may still contain the eigenvariable b in intermediate terms. To discharge it, use the theorem $T \equiv D_{x,y}(x = x)$,

which has rank 1 and support index 2 and does not collapse. CLOSE at the witness 0 proves T, both instances being $0 = 0$. OPEN of T at the fresh b adds the clause $[b = b;\ y.\, b = b]$, the transformed branch is weakened by it, and the rule concludes the goal in the outer scope. □

The quotient contracts a binder only when both its variables are absent; theorems such as $D_{x,y}(x = x)$ are kept, and no completeness of the test for theorems is claimed. A clause body has one more bound variable, the clause variable, so the same support can belong to a body of rank zero with a clause variable or to a formula of rank one; clause bodies therefore keep an explicit rank.

5.5.3 One body per context entry

This subsection draws on `one_body`. An opened occurrence $[B(b,0);\ y.\, B(b,y)]$ is determined by its body. An assumption can be stored in the same form.

Definition 5.48 (Gate and the calculus OB)**.** For $A \equiv D^d(s = t)$ and a variable v outside the $2d$ bound variables the *gate* is

$$\mathrm{gate}_v\, A := D^d\big(\mathrm{sel}(v, 0, s) = \mathrm{sel}(v, 0, t)\big).$$

A context of OB is a finite list of *bodies* $B(v)$, each a formula with one distinguished variable; a body reads $B(0) \to \forall v\, B(v)$, which is the reading of $Q_v B$. The rules are REFL for equations, EVAL_0 and EVAL_S, and

$$\frac{\Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{USE} \qquad \frac{\Gamma \vdash D_{x,y}B(x,y) \quad \Gamma, B(b,y) \vdash A}{\Gamma \vdash A}\ \text{OPEN}$$

$$\frac{\Gamma, \mathrm{gate}_v\big(B(w,\eta)\big) \vdash B(w, Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \text{INTRO}$$

where USE selects a body $B(v)$ of Γ, OPEN adds the body $y \mapsto B(b,y)$ with b fresh, and in INTRO the variables z and v are fresh, w is the witness, and η is z if B is a literal equation and $w \equiv 0$, and 0 otherwise.

A context entry has no antecedent field, and there is no entry for an ordinary assumption. The calculus is formalized in Lean as `OBPrf` (`NativeOneBody`); its theorems are listed in Theorem 5.54. The gate is a construction on the existing grammar. Its variable v lies outside the binders of A, and the terms of the leaf are placed under the new iterator binder of the selector, which in a de Bruijn representation requires one lifting.

The subsystem with REFL and the two evaluation rows gives the equality reasoning of Theorem 5.6 in every context. Write $s \simeq t$ if this subsystem proves $s = t$. Its derivations have no mode test and are stable under substitution. In particular

$$\mathrm{sel}(0, 0, s) \simeq 0, \qquad \mathrm{sel}(Sz, 0, s) \simeq s. \tag{5.4}$$

For formulas A, A' with the same prefix, aligned, write $A \approx A'$ if their left terms are related by $\simeq$ and their right terms are related by $\simeq$, the bound variables of the prefix being treated as free. The definition does not presuppose transport in formulas.

Lemma 5.49 (Deep reflexivity)**.** *Every $D^d(t = t)$ is derivable in* OB *in every context. Weakening and injective renaming of free variables are admissible.*

Proof. Induction on d as in Theorem 5.45: at a binder apply INTRO at the witness $S0$ and ignore the new gate body. Weakening and renaming are inductions on derivations. □

Lemma 5.50 (Conversion and access)**.** *For every d the following hold in* OB*.*

L_d *If $A \approx A'$ and both have d binders, a derivation of either in a context converts to a derivation of the other in that context.*

H_d *If A has d binders and $\mathrm{gate}_v A$ is a body of Γ, then $\Gamma \vdash A$.*

Proof. The statements are proved in the order $L_0, H_0, L_1, H_1, \ldots$.

L_0 is symmetry and transitivity. For L_d with $d > 0$ assume L_{d-1} and H_{d-1}, and let $A \equiv D_{x,y}B$, $A' \equiv D_{x,y}B'$. Apply OPEN to the given derivation of A at a fresh b, which adds the body $B(b, y)$. Apply INTRO for A' at the witness b. A variable is not the literal 0, so this is case mode, and the branch has the body $\mathrm{gate}_v B'(b, 0)$. H_{d-1} gives $B'(b, 0)$. L_{d-1}, with the endpoint derivations instantiated at $x := b$, $y := 0$, gives $B(b, 0)$. USE of the body $B(b, y)$ at Sz gives $B(b, Sz)$, and L_{d-1} gives $B'(b, Sz)$. INTRO concludes A' and OPEN discharges b. The other direction uses the symmetric endpoint equations.

For H_d, given L_d: let $T_d \equiv D^d(0 = 0)$ with the prefix of A. Theorem 5.49 proves T_d, and (5.4) gives $\mathrm{gate}_0 A \approx T_d$ and $\mathrm{gate}_1 A \approx A$. L_d gives $\mathrm{gate}_0 A$. USE of the body $\mathrm{gate}_v A$ at 1 gives $\mathrm{gate}_1 A$, and L_d gives A. □

Conversion at d binders uses access at $d-1$ binders only, so the induction is well founded. It is an induction on formula syntax; it adds no induction to the arithmetic and does not presuppose the access that it proves. The terms of the leaf may contain the bound variables of the prefix, so the endpoint equations live under the binders and cannot be replaced by one global substitution of equals.

Theorem 5.51 (From clauses to bodies)**.** *For Q-free $A_1, \ldots, A_n, F$: if $\mathrm{ord}(A_1), \ldots, \mathrm{ord}(A_n) \vdash_{\mathsf{C}} F$ then $\mathrm{gate}\, A_1, \ldots, \mathrm{gate}\, A_n \vdash_{\mathsf{OB}} F$.*

Proof. Induct on the derivation with its canonical annotation, translating an assumption occurrence $\mathrm{ord}(A)$ to the body $\mathrm{gate}_v A$ and an opened occurrence $[B(b, 0); y.\, B(b, y)]$ to the body $B(b, y)$. REFL and EVAL are unchanged. A USE of an assumption occurrence is replaced by H_d of Theorem 5.50; its premise $0 = 0$ is discarded. A USE of an opened occurrence has the premise $B(b, 0)$ and is USE of OB. ELIM adds an opened occurrence, as OPEN adds its body. INTRO adds the assumption $B(w, \eta)$, whose translation is the gate body that INTRO of OB adds; witness, mode and eigenvariables are unchanged. □

The translation uses no cut and no weakening of OB beyond Theorem 5.49, and no substitution in a derivation with INTRO.

Lemma 5.52 (The gate in N)**.** *For every Q-free A and fresh v, in every context, $A \vdash_{\mathsf{N}} Q_v\, \mathrm{gate}_v A$ and $Q_v\, \mathrm{gate}_v A \vdash_{\mathsf{N}} A$.*

Proof. The equations (5.4) hold in N, so Theorem 5.2(ii), applied along the prefix, gives $\mathrm{gate}_{Sz} A \dashv\vdash A$ and $\mathrm{gate}_0 A \dashv\vdash D^d(0 = 0)$; the latter formula is derivable by d introductions by cases. From A one obtains $\mathrm{gate}_{Sz} A$, the introduction by cases with an unused first variable gives $D_{w,v}\, \mathrm{gate}_v A$, and Theorem 5.2(i) gives $Q_v\, \mathrm{gate}_v A$. Conversely QE with the derivable base $\mathrm{gate}_0 A$ gives $\mathrm{gate}_1 A$, hence A. □

Theorem 5.53 (One-body contexts and N)**.** *If Γ is a context of bodies and $\Gamma \vdash_{\mathsf{OB}} A$, then $\Gamma^Q \vdash_{\mathsf{N}} A$, where Γ^Q replaces each body $B(v)$ by $Q_v B$. For Q-free $A_1, \ldots, A_n, F$ the following three statements are equivalent:*

$$\begin{aligned} &\mathrm{gate}\, A_1, \ldots, \mathrm{gate}\, A_n \vdash_{\mathsf{OB}} F, \\ &A_1, \ldots, A_n \vdash_{\mathsf{N}} F, \\ &\mathrm{ord}(A_1), \ldots, \mathrm{ord}(A_n) \vdash_{\mathsf{C}} F. \end{aligned}$$

Proof. Soundness is by induction on the derivation. REFL and EVAL are as in Theorem 5.11(i). USE is QE on the assumption Q_vB. OPEN is DE, which adds $Q_yB(b,y)$. For INTRO the translated premise is $\Gamma^Q, Q_v\,\mathrm{gate}_v(B(w,\eta)) \vdash B(w,Sz)$; Theorem 5.52 and cut give $\Gamma^Q, B(w,\eta) \vdash B(w,Sz)$, and the two modes are treated as in Theorem 5.11(i). For the equivalences: soundness and Theorem 5.52 with cut lead from the first statement to the second; the second and the third are equivalent by Theorem 5.11(iii); and Theorem 5.51 leads from the third to the first. □

Remark 5.54. In the Lean development the first equivalence is `obprf_iff_nprf`, its form for arbitrary native sequents through τ is `obprf_unQ_iff_nprf`, and the equivalence with C is `obprf_iff_cprf`; Theorem 5.50 is `OBPrf.conv` and `OBPrf.acc`, and Theorem 5.51 is `CPrf.toOB`. The Lean proof derives the gate at 0 directly (`OBPrf.gtZero`) and does not convert it from deep reflexivity. `one_body` proves soundness through T_{eq} and the bridge (Theorem 1.119); the proof above reads a body as a formula Q_vB of N, as the Lean proof does.

Remark 5.55 (A padding trap). To align the numbers of binders of two formulas, a formula A is padded as $D_{x,y}\,\mathrm{gate}_y\,A$ with x unused, which is equivalent to A by Theorem 5.52. Padding by a binder both of whose variables are unused is incorrect: $D_{x,y}A$ then reads $\exists x\,(A \to A)$ and is a theorem (compare Theorem 5.47).

5.5.4 Homogeneous contexts with an asymmetric gate

This subsection draws on the appendix of `prefix_clauses`. It removes the antecedent field as OB does, keeps reflexivity as a fixed entry, and uses a gate whose zero instance is literally reflexive.

Definition 5.56 (The calculus HB)**.** For a fresh variable v let

$$\kappa_v(s=t) := \big(I(0,s;\,u.\,t) = I(v,s;\,u.\,t)\big), \qquad \kappa_v(D_{x,y}B) := D_{x,y}\kappa_v B,$$

with u not in t. A context is a list of bodies and always contains the body $\rho(y) \equiv (y=y)$. The rules are EVAL with Φ_2, and

$$\frac{\Gamma \vdash B(0)}{\Gamma \vdash B(t)} \qquad \frac{B(0) \equiv D^j(r=r)}{\Gamma \vdash B(t)} \qquad \text{USE}$$

for a body $B(y)$ of Γ, the rule OPEN of OB, and INTRO of OB with $\kappa_v\big(B(t,\eta)\big)$ as the added body.

There are four rule families, no rule REFL and no hypothesis rule. The second form of USE omits the premise exactly when the raw base $B(0)$ is reflexive at its leaf, for some $j \geq 0$; a theorem with root D of another shape does not qualify. The gate satisfies

$$\kappa_0(s=t) \text{ is literally reflexive}, \qquad \kappa_{Sz}(s=t) \dashv\vdash (s=t),$$

the second by the two computation equations, since u is not in t. The gate changes neither the number of binders nor the root of a formula.

Lemma 5.57 (Gate retrieval)**.** *In* HB*, in every context:*

(i) $t=t$ is derivable, every $D^j(r=r)$ is derivable, and the equality rules of Theorem 5.6 hold;

(ii) $\kappa_0 A$ is derivable for every A;

(iii) a derivation of $\kappa_1 A$ converts to a derivation of A, and conversely;

(iv) if $\kappa_v A$ is a body of Γ then $\Gamma \vdash A$.

Proof. (i) USE of ρ at t has the base $0 = 0$, so its premise is omitted and it concludes $t = t$. Reflexive prefixes are proved as in Theorem 5.45, with INTRO at the witness $S0$ and the new gate body ignored. The equality rules use reflexivity and EVAL only. (ii) $\kappa_0 A$ has the prefix of A and identical terms at its leaf, so it is a reflexive prefix.

(iii) and (iv) are proved together by induction on the number of binders of A. For an equation, (iii) is the two computations and transport. For (iv), USE of the body $\kappa_v A$ at 1 has a base that is reflexive at its leaf, so its premise is omitted and it concludes $\kappa_1 A$; (iii) at the same number of binders gives A. For (iii) at $A \equiv D_{x,y} B$, assume (iii) and (iv) for formulas with fewer binders. Given $\kappa_1 A \equiv D_{x,y} \kappa_1 B$, apply OPEN at a fresh b, which adds the body $\kappa_1 B(b, y)$, and INTRO for A at the witness b, which is case mode and adds the body $\kappa_v(B(b, 0))$. (iv) gives $B(b, 0)$, (iii) gives $\kappa_1 B(b, 0)$, USE of the body $\kappa_1 B(b, y)$ at Sz gives $\kappa_1 B(b, Sz)$, and (iii) gives $B(b, Sz)$. The converse opens A, introduces $\kappa_1 A$ at the witness b, retrieves the local zero assumption by (iv), converts it by (iii), uses the body $B(b, y)$ at Sz and converts the result by (iii). □

Theorem 5.58 (Homogeneous contexts and N)**.** *For Q-free $A_1, \ldots, A_n, F$,*

$$\rho, \kappa A_1, \ldots, \kappa A_n \vdash_{\mathsf{HB}} F \quad \Longleftrightarrow \quad A_1, \ldots, A_n \vdash_{\mathsf{N}} F.$$

Proof. From right to left translate a derivation of C (Theorem 5.11(iii)) by the origins of its occurrences, as in Theorem 5.51: REFL becomes USE of ρ, a USE of an assumption occurrence becomes Theorem 5.57(iv), a USE of an opened occurrence becomes USE with its premise, and ELIM and INTRO become OPEN and INTRO.

From left to right read a body $B(y)$ as $Q_y B$ and induct as in Theorem 5.53. The reading $Q_y(y = y)$ of ρ is derivable in N by progress. A USE with omitted premise is QE whose base $D^j(r = r)$ is derivable in N by j introductions by cases. For INTRO use $A \vdash_{\mathsf{N}} Q_v \kappa_v A$, which is proved as Theorem 5.52 from $\kappa_{Sz} A \dashv\vdash A$. Finally $Q_v \kappa_v A_i$ follows from A_i, and cut gives the sequent on the right. □

Remark 5.59. `prefix_clauses` proves completeness from the seven-rule witness calculus of `witness_rule` by the same strengthened induction over contexts, and soundness through T_{eq}. The package states HB on records; the gate leaves k unchanged and acts on the leaf. This second theorem of the package is not formalized in Lean. OB and HB differ in three points: the gate (symmetric selector or asymmetric iterator), reflexivity (a rule for equations or the entry ρ with omitted premises), and the proof of access (conversion along $\approx$ from deep reflexivity or a literally reflexive zero gate).

5.6 Fusion: evaluation at the discharge

This section draws on `evaluation_discharge`, and Section 5.6.3 on `476_byte_presentation`. The two rows of EVAL and the elimination of D become one rule family, which leaves three: USE, INTRO and DISCHARGE.

5.6.1 The raw calculus

Definition 5.60 (One evaluation step)**.** For formulas E and F the formula $\varrho_E(F)$ is defined by three disjoint cases, every match being literal:

$$\begin{aligned} \varrho_{\,s=t}\big(I(0,s;\,u.\,h)=r\big) &:= (r=t),\\ \varrho_{\,I(c,a;\,u.\,h)=t}\big(I(Sc,a;\,u.\,h)=r\big) &:= (r=h[t/u]),\\ \varrho_E(F) &:= F \qquad \text{in every other case.} \end{aligned}$$

The first case has a literal zero counter and the second a successor, so they do not overlap, and ϱ is a total function of (E,F). The replacement term, the body and the counter are read from the two formulas. An E with root D always selects the third case. In the Lean development the graph of ϱ is a relation, with `Rho.functional` and `Rho.total`.

Definition 5.61 (The calculus ED)**.** Terms are raw, contexts are lists of uniform clauses, and the rules are USE and INTRO of V (Theorem 5.26) and

$$\frac{\Gamma \vdash D_{x,y}B \quad \Gamma, \mathcal{B}_b \vdash E \quad \Gamma, \mathcal{B}_b \vdash F}{\Gamma \vdash \varrho_E(F)}\ \textsc{discharge}$$

where $\mathcal{B}_b := [B(b,0);\, y.\,B(b,y)]$ and b is fresh for Γ and B and does not occur free in $\varrho_E(F)$. The formulas E and F may contain b.

The rule opens $D_{x,y}B$ at b, proves E and F in that scope, performs one evaluation step, and exports the result if the step has removed b or b was absent. The calculus is formalized in Lean as `E3Prf` (`NativeEvalDischarge`), with ϱ as `Rho`; its theorems are listed in Theorem 5.64. The third case does not derive F without a premise: the rule requires a derivation of F. The two patterns of ϱ and its identity case are part of the specification.

Lemma 5.62. *Weakening and injective renaming of free variables are admissible in* ED *and in* V*. The formula* $\Theta \equiv D_{x,y}(0=0)$ *is derivable in both, in every context, also in one without* $\mathcal{R}$*.*

Proof. Weakening and renaming are inductions on derivations with eigenvariables made fresh; an injective renaming preserves and reflects the literal matches of ϱ, the constructors 0 and S, and the mode of INTRO. For Θ apply INTRO at the witness 0. The matrix is a literal equation, the assumption and the target are both $0=0$, and USE of the added clause $\langle 0=0\rangle$ closes the branch. □

Theorem 5.63 (Evaluation at the discharge)**.** *For every context* Γ *of uniform clauses and every formula* A*,*

$$\Gamma \vdash_{\mathsf{ED}} A \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathsf{V}} A.$$

The translations leave the end sequent unchanged, and $\mathcal{R}$ *need not be in* Γ*. Hence, for* Q*-free* Δ *and* F*,* $\mathcal{R}, \langle\Delta\rangle \vdash_{\mathsf{ED}} F$ *if and only if* $\Delta \vdash_{\mathsf{N}} F$*.*

Proof. From left to right translate the three premises of a DISCHARGE. If $\varrho_E(F)$ is given by the first or the second case, apply the corresponding row of EVAL to the translated derivations of E and F in the context $\Gamma, \mathcal{B}_b$. If it is given by the third case, the translated derivation of F has the required conclusion. Then apply ELIM; its eigenvariable condition is the condition that b is not free in $\varrho_E(F)$. USE and INTRO are unchanged.

From right to left translate ELIM with major premise $D_{x,y}B$ and branch conclusion A by DISCHARGE with the same major premise, with $E := D_{x,y}B$, derived in the branch by weakening the translated major premise, and with $F := A$. Since E has root D, $\varrho_E(A) \equiv A$. Translate an EVAL with premises E and F by DISCHARGE with the major premise Θ

of Theorem 5.62 and with the translated derivations of E and F weakened to the context $\Gamma, [0 = 0;\, y.\, 0 = 0]$. The corresponding case of ϱ matches and gives the conclusion of the EVAL, which contains no b. The second statement is Theorem 5.27. □

Under the ordinary reading DISCHARGE is sound directly: eliminate the existential at b; in the first case the zero computation and $s = t$ turn the second equation into $r = t$, in the second case the successor computation and the equation for the preceding iterate give $r = h[t/u]$, and the third case keeps the formula; the conclusion leaves the scope because b does not occur in it.

Remark 5.64. In the Lean development the equivalences are `e3prf_iff_vprf` on every clause context and `e3prf_iff_nprf_of_qfree`. The Lean translation from V carries a renaming and an extension of the context through its induction, where the proof above uses Theorem 5.62. The translation of ELIM copies the derivation of its major premise into the scope of the witness.

Remark 5.65 (Checking without fuel). `evaluation_discharge` observes that the certificate predicate of ED is primitive recursive with no fuel field. On tree sizes raw substitution satisfies $|a[t/u]| \le |a| \cdot \max(1, |t|)$, a step of ϱ performs at most one substitution in a body, each recursive call consumes a proper subcertificate, and the work at a node is bounded by a fixed polynomial in the sizes of its data, its context and the conclusions of its subderivations. The zero-normal companion adds structural deletions and at most one constructed iterator.

5.6.2 The zero-normal companion

Definition 5.66 (Seed evaluation and the calculus ED^0)**.** On zero-normal terms let

$$\sigma_E(F) := \begin{cases} r = h\langle \overline{I}(c, t;\, u.\, h)/u \rangle, & E \equiv (s = t),\ F \equiv (I(Sc, s;\, u.\, h) = r), \\ F, & \text{otherwise.} \end{cases}$$

ED^0 has zero-normal terms, normal substitution, uniform clauses, USE and CLOSE of V^0, and DISCHARGE with $\sigma_E(F)$ in the place of $\varrho_E(F)$. The calculus $\mathsf{V}^0_{\mathrm{s}}$ is V^0 with STEP replaced by the rule SEEDEVAL of `seed_evaluation` (Theorem 4.75): from $s = t$ and $I(Sc, s;\, u.\, h) = r$ infer $r = h\langle \overline{I}(c, t;\, u.\, h)/u \rangle$.

The case of $\sigma_E(F)$ that is not the identity is exactly SEEDEVAL, with E and F as its two premises.

Lemma 5.67 (SEEDEVAL and STEP)**.** *On zero-normal terms, in every context that contains $\mathcal{R}$, SEEDEVAL is derivable from USE of $\mathcal{R}$ and STEP, and STEP is derivable from USE of $\mathcal{R}$ and SEEDEVAL. No logical rule is used.*

Proof. These are parts (b) and (c) of Theorem 4.76. Their derivations consist of reflexivity, STEP or SEEDEVAL, and the given premises; they read no clause, so they are derivations over every clause context once each reflexivity leaf is replaced by USE of $\mathcal{R}$ at the term in question. □

Theorem 5.68 (The zero-normal fusion)**.** *For every zero-normal context Γ of uniform clauses and every zero-normal formula A, $\Gamma \vdash_{\mathsf{ED}^0} A$ if and only if $\Gamma \vdash_{\mathsf{V}^0_{\mathrm{s}}} A$. If Γ contains $\mathcal{R}$, both are equivalent to $\Gamma \vdash_{\mathsf{V}^0} A$.*

Proof. The proof of Theorem 5.63 applies with SEEDEVAL in the place of the two rows of EVAL: a major premise with root D recovers OPEN, and the major premise Θ recovers SEEDEVAL. No substitution in a derivation is used. The second statement is Theorem 5.67. □

The companion and its equivalence are not formalized in Lean; the formalized calculus with SEEDEVAL has native contexts (`NVPrf` in `NativeZeroSeed`, Theorem 4.77). With clause contexts and a rule REFL, SEEDEVAL gives the calculus $\mathsf{C}_{\mathrm{Seed}}$ of Theorem 4.77, which is equivalent to C^0.

5.6.3 Partial evaluation and the diagonal introduction

This subsection draws on `476_byte_presentation`. Two rules of ED have instances that a substitution does not carry to instances of the same rule: the introduction selects its mode by the witness (Theorem 5.14), and the identity case of ϱ applies to every pair that matches no arithmetic case (Theorem 5.72). The calculus PD below takes the diagonal introduction of K and restricts the identity case to an E with root D. It keeps three rule families, and substitution preserves its derivation trees.

Definition 5.69 (Partial evaluation)**.** For formulas E and F the formula $\varrho^+_E(F)$ is

$$\begin{aligned} \varrho^+_{s=t}(b = r) &:= (r = \Phi_2(s,t,b)) \qquad \text{if } \Phi_2(s,t,b) \text{ is defined,} \\ \varrho^+_{D_{p,q}C}(F) &:= F, \end{aligned}$$

and $\varrho^+_E(F)$ is undefined in every other case.

Written out, the first line consists of the zero row and the successor row

$$\varrho^+_{s=t}(I(0,s;\, u.\, h) = r) \equiv (r = t), \qquad \varrho^+_{I(c,a;\, u.\, h)=t}(I(Sc,a;\, u.\, h) = r) \equiv (r = h[t/u]),$$

which are the first two cases of ϱ (Theorem 5.60); the second line, the *row* D, is the third case of ϱ restricted to an E with root D. The three rows are disjoint: in the first two E is an equation and the counter of the left side of F is 0, respectively a successor, and in row D the formula E has root D. Hence ϱ^+ is a partial function, and $\varrho^+_E(F) \equiv \varrho_E(F)$ wherever $\varrho^+_E(F)$ is defined. A pair of equations that matches no row of Φ_2 is not in its domain.

Definition 5.70 (The calculi K^u and PD)**.** K^u is the calculus V of Theorem 5.26 with the term η of INTRO replaced by the term $\eta_B(z)$ of the diagonal introduction (Theorem 5.32); its rule families are USE, EVAL, ELIM and INTRO. PD has raw terms, contexts of uniform clauses, the rules USE and INTRO of K^u and

$$\frac{\Gamma \vdash D_{x,y}B \quad \Gamma, \mathcal{B}_b \vdash E \quad \Gamma, \mathcal{B}_b \vdash F}{\Gamma \vdash \varrho^+_E(F)}\ \textsc{discharge}$$

where $\mathcal{B}_b := [B(b,0);\, y.\, B(b,y)]$, the formula $\varrho^+_E(F)$ is defined, and b is fresh for Γ and B and does not occur free in $\varrho^+_E(F)$.

K^u stands to K as V stands to C. The formulas E and F of a DISCHARGE may contain b. The specification of PD is the clause $\mathcal{R}$ in the initial context of a theorem, USE with zero or one premise, INTRO with the diagonal test made on the matrix before the witness is substituted, and DISCHARGE with the three rows of ϱ^+; there is no reflexivity, computation, transport, induction, cut or structural rule. Every DISCHARGE of PD is a DISCHARGE of ED, and the two calculi differ in the mode of INTRO and in the domain of the identity case.

Lemma 5.71 (Covariance of ϱ^+)**.** *Let σ be a capture-free substitution of terms for free variables. If $\varrho^+_E(F)$ is defined, then $\varrho^+_{E\sigma}(F\sigma)$ is defined by the same row and*

$$\varrho^+_{E\sigma}(F\sigma) \equiv \varrho^+_E(F)\,\sigma.$$

Proof. In the zero and the successor row $E \equiv (s = t)$ and $F \equiv (b = r)$, so $E\sigma \equiv (s\sigma = t\sigma)$ and $F\sigma \equiv (b\sigma = r\sigma)$. By Theorem 4.21, $\Phi_2(s\sigma, t\sigma, b\sigma)$ is defined by the same row and is $\Phi_2(s,t,b)\sigma$; hence $\varrho^+_{E\sigma}(F\sigma) \equiv (r\sigma = \Phi_2(s,t,b)\sigma) \equiv \varrho^+_E(F)\sigma$. In row D the formula $E\sigma$ has root D (Theorem 3.15(i)), and both sides are $F\sigma$. □

Definedness is not reflected: the substitution $x \mapsto 0$ takes the pair $(x = 0,\ I(0,0;\, u.\, 0) = 0)$, outside the domain, to a pair in the zero row. The theorems below need preservation only.

Remark 5.72 (The total identity case is not covariant). Let $E \equiv (x = 0)$, $F \equiv (I(0,0;\, u.\, 0) = 0)$ and $\sigma(x) := 0$. No arithmetic case of ϱ matches (E, F): the zero case needs the seed 0 of the iterator to be the left side x of E, and the successor case needs a successor counter. So $\varrho_E(F) \equiv F$. The pair $(E\sigma, F\sigma) \equiv (0 = 0,\ I(0,0;\, u.\, 0) = 0)$ matches the zero case, so

$$\varrho_{E\sigma}(F\sigma) \equiv (0 = 0) \not\equiv \big(I(0,0;\, u.\, 0) = 0\big) \equiv \varrho_E(F)\,\sigma.$$

A DISCHARGE of ED at this pair therefore does not become, under σ, an instance of the same rule with the substituted conclusion. This obstructs substitution that preserves the derivation tree of ED, not admissibility: for contexts $\mathcal{R}, \Gamma$, substitution is admissible in ED through Theorem 5.63, Theorem 5.27 and Theorem 5.14, since $(\Gamma\sigma)^\circ \equiv \Gamma^\circ\sigma$. The pair is outside the domain of ϱ^+.

For a substitution σ, the image of a uniform clause $[\Pi;\, y.\, C]$ is $[\Pi\sigma;\, y.\, C\sigma]$, with y renamed away from the variables and values of σ; then $(C[t/y])\sigma \equiv (C\sigma)[t\sigma/y]$, $\langle A\rangle\sigma \equiv \langle A\sigma\rangle$ and $\mathcal{R}\sigma \equiv \mathcal{R}$.

Theorem 5.73 (Substitution and weakening preserve the derivation tree)**.** *Let $\Gamma \vdash_{\mathsf{PD}} A$, let σ be a capture-free substitution of terms for free variables, and let Γ' be a context with a map ι from the occurrences of Γ to the occurrences of Γ' such that $\iota(c)$ is the clause $c\sigma$. Then $\Gamma' \vdash_{\mathsf{PD}} A\sigma$ by a derivation with the same tree of rules, in which every* USE *selects the image of its occurrence, every* INTRO *has the same mode and every* DISCHARGE *the same row of ϱ^+, after a renaming of eigenvariables. In particular substitution ($\Gamma' := \Gamma\sigma$), weakening and exchange (σ the identity) and injective renaming are admissible in* PD.

Proof. Induction on the derivation, simultaneously for all σ, Γ' and ι.

USE: the derivation selects $c \equiv [\Pi;\, y.\, C]$ at the term t and has a derivation of $\Gamma \vdash A'$ for each A' in Π. The induction hypothesis gives $\Gamma' \vdash A'\sigma$ for each member of $\Pi\sigma$, and USE of $\iota(c) \equiv [\Pi\sigma;\, y.\, C\sigma]$ at $t\sigma$ gives $(C\sigma)[t\sigma/y] \equiv (C[t/y])\sigma$.

INTRO: the premise is $\Gamma, \langle B(t, \eta_B(z))\rangle \vdash B(t, Sz)$ with z fresh for Γ, t and B. Choose z' fresh for Γ', $t\sigma$ and $B\sigma$, and let σ' be σ with $z \mapsto z'$ in addition. Since z does not occur in Γ, t and B, the substitution σ' acts on them as σ, so ι still satisfies the hypothesis for σ'. The diagonal test is preserved and reflected by the substitution, lifted under x and y (Theorem 3.15(iii)), so $\eta_B(z)\sigma' \equiv \eta_{B\sigma}(z')$ and the added occurrence has the image $\langle (B\sigma)(t\sigma, \eta_{B\sigma}(z'))\rangle$. Extend ι by sending the added occurrence to the occurrence added on the right. The induction hypothesis for σ' gives $\Gamma', \langle (B\sigma)(t\sigma, \eta_{B\sigma}(z'))\rangle \vdash (B\sigma)(t\sigma, Sz')$, and INTRO in the same mode with the eigenvariable z' gives $\Gamma' \vdash D_{x,y}(B\sigma) \equiv (D_{x,y}B)\sigma$.

DISCHARGE: the premises are $\Gamma \vdash D_{x,y}B$ and $\Gamma, \mathcal{B}_b \vdash E$, $\Gamma, \mathcal{B}_b \vdash F$, with $\varrho^+_E(F) \equiv R$ and b fresh for Γ, B and R. The induction hypothesis for σ and ι gives $\Gamma' \vdash D_{x,y}(B\sigma)$. Choose b' fresh for Γ', $B\sigma$ and $R\sigma$, and let σ' be σ with $b \mapsto b'$ in addition. Then $\mathcal{B}_b\sigma'$ is the clause $[(B\sigma)(b', 0);\, y.\, (B\sigma)(b', y)]$ that opens $D_{x,y}(B\sigma)$ at b'; extend ι accordingly. The induction hypothesis for σ' gives the two branches with the conclusions $E\sigma'$ and $F\sigma'$. By Theorem 5.71, $\varrho^+_{E\sigma'}(F\sigma')$ is defined by the same row and is $R\sigma' \equiv R\sigma$, because b does not occur in R. The eigenvariable condition holds for b', and DISCHARGE gives $\Gamma' \vdash R\sigma$.

For the special cases take ι positional for $\Gamma\sigma$, and for weakening and exchange the map that sends each occurrence of Γ to its copy in Γ'. □

No cut, normalization or arithmetic bridge is used. Substitution in K preserves the derivation tree as well (Theorem 5.36); K has four rule families and an explicit REFL, and PD has three and the clause $\mathcal{R}$.

Theorem 5.74 (Uniform clauses with the diagonal introduction)**.** *For every context* Γ *of uniform clauses and every formula* A*,*

$$\mathcal{R}, \Gamma \vdash_{\mathsf{K}^u} A \quad \Longleftrightarrow \quad \Gamma^\circ \vdash_{\mathsf{K}} A.$$

Hence, for Q*-free* Δ *and* F*,* $\mathcal{R}, \langle\Delta\rangle \vdash_{\mathsf{K}^u} F$ *if and only if* $\Delta \vdash_{\mathsf{N}} F$*.*

Proof. The proof of Theorem 5.27 applies. Its invariant is that each occurrence of the uniform context, other than $\mathcal{R}$, corresponds by position to an occurrence of the packed context with the same consequent and either the same antecedent or no antecedent in the place of $0 = 0$, and that USE of $\mathcal{R}$ corresponds to REFL. It copies every INTRO with the same matrix, witness and eigenvariable, and the added occurrences $\langle B(t,\eta)\rangle$ and $\mathrm{ord}(B(t,\eta)) \equiv \langle B(t,\eta)\rangle^\circ$ correspond. Here $\eta \equiv \eta_B(z)$ on both sides, since the diagonal test reads the matrix only. An ELIM adds the same opened occurrence on both sides, which has an antecedent. The second statement follows from Theorem 5.35, that is statement (iii) of Theorem 5.11 for K, since $\langle\Delta\rangle^\circ \equiv \mathrm{ord}(\Delta)$. □

Theorem 5.75 (Partial evaluation at the discharge)**.** *For every context* Γ *of uniform clauses and every formula* A*,*

$$\Gamma \vdash_{\mathsf{PD}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathsf{K}^u} A.$$

The translations leave the end sequent unchanged, and $\mathcal{R}$ *need not be in* Γ*. Hence, for* Q*-free* Δ *and* F*,* $\mathcal{R}, \langle\Delta\rangle \vdash_{\mathsf{PD}} F$ *if and only if* $\Delta \vdash_{\mathsf{N}} F$*.*

Proof. Both directions are inductions on derivations. USE and INTRO are the same rules in both calculi and are copied.

From left to right translate the three premises of a DISCHARGE with major premise $D_{x,y}B$ and conclusion $R \equiv \varrho^+_E(F)$. In the zero or the successor row, $E \equiv (s = t)$, $F \equiv (b = r)$ and $R \equiv (r = \Phi_2(s,t,b))$; EVAL applied to the translated derivations of E and F proves R in $\Gamma, \mathcal{B}_b$. In row D, $R \equiv F$, and the translated derivation of F proves it. Then apply ELIM to the translated major premise; its eigenvariable condition is the condition of DISCHARGE on b.

From right to left let $\Theta \equiv D_{x,y}(0 = 0)$. In PD it is derivable in every context, also without $\mathcal{R}$, by the two-node derivation of Theorem 5.62: the matrix $0 = 0$ is not diagonal, so INTRO at the witness 0 is in case mode, its assumption and its target are both $0 = 0$, and USE of $\langle 0 = 0\rangle$ closes the branch. Translate an ELIM with major premise $D_{x,y}B$ and branch $\Gamma, \mathcal{B}_b \vdash A$ by DISCHARGE with the translated major premise, with $E := \Theta$, derived in $\Gamma, \mathcal{B}_b$, and with $F := A$; row D gives A, and the eigenvariable condition is that of ELIM. Translate an EVAL with premises $\Gamma \vdash s = t$ and $\Gamma \vdash b = r$ by DISCHARGE with the major premise Θ, opened at a b_0 fresh for Γ and the four terms, which adds the occurrence $[0 = 0;\, y.\, 0 = 0]$. Weaken the translated derivations of the two premises into $\Gamma, [0 = 0;\, y.\, 0 = 0]$ by Theorem 5.73. The pair $(s = t,\ b = r)$ is in the zero or the successor row of ϱ^+ with the value $r = \Phi_2(s,t,b)$, the conclusion of the EVAL, in which b_0 does not occur.

The second statement is Theorem 5.74. □

Remark 5.76 (The Lean modules). `NativePartialDischarge` formalizes ϱ^+ (`RhoPlus`, with $\varrho^+ \subseteq \varrho$ as `RhoPlus.toRho`), K^u (`KUPrf`) and PD (`PDPrf`), and proves Theorem 5.75 for every clause context (`pdprf_iff_kuprf`), Theorem 5.74 with the reflexivity clause (`kuprf_iff_kprf`), and the bridge on Q-free sequents (`pdprf_iff_nprf_of_qfree`). The proofs follow those of the evaluating discharge (`E3Prf`) and of uniform clauses (`VPrf`): the diagonal start of the introduction depends on the matrix only and commutes with substitution (`NFm.dstart_subst`),

and the row D takes the place of the identity case. The translation from K^u carries an injective renaming and an extension of the context through the induction instead of a separate substitution lemma, so Theorem 5.73 is not needed for it and is not formalized.

Under the ordinary reading every rule of PD is sound in T_{eq} directly: DISCHARGE as the discharge of ED after Theorem 5.63, since ϱ^+ is a restriction of ϱ, and the diagonal INTRO by coverage in case mode and by induction for the literal equation under the assumed base in progress mode. By Theorem 5.75, PD and K^u are among the calculi of Theorem 5.78: through `kprf_iff_nprf_of_qfree` and Theorem 1.119 they are bi-interpretable with iPRA with whole domains, actual equality and identity numerical composites, and $\mathcal{R} \vdash_{\mathsf{PD}} 0 = S0$ if and only if iPRA derives falsity.

Remark 5.77. `476_byte_presentation` states Theorem 5.73 for substitution and, without proof details, for weakening and renaming, and proves Theorem 5.75 through K^u for contexts that contain $\mathcal{R}$; the proof above does not use $\mathcal{R}$. Its derivation of Θ is that of Theorem 5.62, its derivation of the standalone evaluation from DISCHARGE at Θ is that of the proof of Theorem 5.63, and its translation of DISCHARGE into EVAL and ELIM is the converse direction of that proof, restricted to the domain of ϱ^+. The one difference is the translation of ELIM: the proof of Theorem 5.63 takes as E the major premise weakened into the scope of the witness, and the proof above takes the two-node derivation of Θ, which does not copy the derivation of the major premise. Both choices are available in either calculus, since the weakened major premise also has root D. The package also recovers reflexivity by USE of $\mathcal{R}$ and, from the derived standalone evaluation, the two computation equations, the inference ROOT by the frame $I(0, s;\, u.\, u)$, symmetry, transitivity and one-template transport; these are the derivations of Theorem 5.6. The package compiles certificates in both directions between its checker and the matched four-family checker of K^u, preserving the goal; the compilers are untrusted and their outputs are rechecked.

5.7 The ordinary realization

This section draws on all twelve packages of the chapter: `raw_clausal`, `rootfree_clausal`, `reduced_clause_calculus`, `clausal_two_case`, `clausal_single_evaluation`, `uniform_clauses`, `clause_guards`, `clause_contexts`, `prefix_clauses`, `one_body`, `evaluation_discharge` and `476_byte_presentation`. Each restates the ordinary realization, the logical macros of the native interface (gate, selector and parity term, with q_y in the place of Q_y) and the bridge. These belong to the common core, and the statements reduce to it as follows.

Corollary 5.78 (Bi-interpretability)**.** *Let* X *be one of* C_3, C, C^0, $\mathsf{C}^0_{\mathrm{b}}$, V, V^0, K, *the mirror calculus,* PF, OB, HB, ED, ED^0, K^u, PD, *with its fixed reflexivity clause or entry where it has one. Then* X *derives* $0 = S0$ *from no assumptions if and only if* N *does, if and only if* iPRA *derives falsity. The ordinary realization of* X *is bi-interpretable with* iPRA *with whole domains, actual equality and identity numerical composites.*

Proof. Each calculus has been shown equivalent to N on Q-free sequents by translations that are the identity on terms, or the map nf_0 on terms for the zero-normal calculi, and the formula composites are the identity and $A \mapsto j\tau A$, respectively $A \mapsto \mathrm{nf}_0(j\tau A)$, which are provably equivalent to A in N (Theorems 4.27 and 5.2). The claim follows from Theorem 1.119. □

Under the ordinary reading every rule is sound in T_{eq} directly: REFL, EVAL, STEP and SEEDEVAL follow from the computation equations and equality, USE and the eliminations are ordinary eliminations, an introduction in case mode uses coverage, and an introduction in progress mode uses induction for a literal equation under the assumed base. The separation $0 = S0 \to \bot$ is an axiom of the ordinary realization and not a rule of a clause calculus, in

which $0 = S0$ is the designated contradiction; positive equations alone admit a one-element structure.

5.8 Successor rigidity

This section draws on `raw_clausal`. It concerns an arithmetic principle and not a clause calculus. The background theory is intuitionistic first-order logic with equality over the scalar terms, with the two computation equations and coverage $\forall n\,(n = 0 \vee \exists m\, n = Sm)$. Neither the injectivity of S nor the separation of 0 from $S0$ is used. The rule is

$$\frac{\Gamma \vdash f(0) = 0 \quad \Gamma \vdash f(Sz) = S(f(z))}{\Gamma \vdash f(t) = t}\ \mathrm{SR}_0$$

where $f(z)$ is a term with a distinguished variable, not a function variable, and z is fresh for Γ, t and the parameters of f. The second premise has no local assumption. The rule is the uniqueness of iteration UR (Section 2.5.1) restricted to the initializer 0 and the update S. Induction for literal equations derives SR_0: induct on $f(n) = n$, where the step is $f(Sz) = S(f(z)) = Sz$ under $f(z) = z$. Successor uniqueness SU (Theorem 2.74) also derives it: SU gives $f(t) = I(t, f(0);\, u.\, Su)$, the first premise turns the seed into 0, and $I(t, 0;\, u.\, Su) = t$ is Theorem 2.78(iii), which is derivable in T_{SU}.

Theorem 5.79 (Zero-equation induction from successor rigidity)**.** *Work in the background theory with* SR_0*. Let* $E(x)$ *be a formula with* $\Gamma \vdash E(0)$ *and* $\Gamma, E(z) \vdash E(Sz)$*,* z *fresh, and let* $e(x)$ *be a term such that, for a fresh* x*,*

$$\Gamma \vdash E(e(x)), \qquad \Gamma, E(x) \vdash e(x) = x.$$

Then $\Gamma \vdash E(t)$ *for every term* t*. In particular, for every term* $q(x)$ *the rule*

$$\frac{\Gamma \vdash q(0) = 0 \quad \Gamma, q(z) = 0 \vdash q(Sz) = 0}{\Gamma \vdash q(t) = 0}\ \mathrm{ZI}$$

is derivable.

Proof. Let $r(n) := I(n, 0;\, u.\, e(Su))$. First, $\Gamma \vdash E(r(n))$, by coverage on n. If $n = 0$ then $r(n) = 0$ and the base applies. If $n = Sm$ then $r(n) = e(S(r(m)))$, and the first condition on e at $S(r(m))$ applies. This step assumes nothing about $r(m)$ and uses no induction. Next instantiate the given step at $r(z)$; with $E(r(z))$ it gives $E(S(r(z)))$, and the second condition on e gives $e(S(r(z))) = S(r(z))$. The computation equations therefore prove $r(0) = 0$ and $r(Sz) = S(r(z))$ in the context Γ. SR_0 gives $r(t) = t$, and $E(r(t))$ transports to $E(t)$. The eigenvariable of SR_0 and the witness of the coverage elimination are fresh, so Γ may contain arbitrary formulas.

For ZI take $E(x) \equiv (q(x) = 0)$ and $e(x) := \mathrm{sel}(q(x), x, 0)$. If $q(x) = 0$ the selector computes to x, which is the second condition. For the first apply coverage to $q(x)$. If $q(x) = 0$ then $e(x) = x$ and $q(e(x)) = 0$. If $q(x) = Sv$ then $e(x) = 0$ and the base $q(0) = 0$ gives $q(e(x)) = 0$. These cases need no test of equality of two terms and no separation. □

The construction of e and r is effective on syntax, so the theorem is a transformation of derivations. The theorem does not give full scalar arithmetic from SR_0; see Theorem 5.81.

Remark 5.80 (Comparison with the rigidity of retractions). Theorems 2.80, 2.81 and 2.83 to 2.85 derive induction for zero equations from SU by a trajectory of the same kind, $r(n) \equiv I(n, 0;\, u.\, Se(u))$, for a term e with $e(0) = 0$ and $e(Se(x)) = Se(x)$; since `reductions` that derivation needs neither idempotence of e nor coverage (Theorem 2.80), and coverage and

separation enter only in Theorem 2.84. Since SR_0 is derivable from SU, Theorem 5.79 implies Theorem 2.85. What it adds is the following. Its rule SR_0 fixes the seed 0 and concludes the identity, so it does not presuppose Theorem 2.78(iii). Its two conditions on e concern the formula E and not e alone, so the general statement applies to every formula E with such an e. Its derivation of ZI uses coverage on $q(x)$ in place of a decided split (Theorem 2.83), so it needs neither the decidability of $q(n) = 0$ nor separation, which Theorem 2.84 uses.

5.9 Checker sizes

This section collects the measurements of the twelve packages named in Section 5.7. All of them measure complete standalone Python checkers by the protocol of Section 1.10, under CPython 3.12.14: the module without its docstring, printed by `ast.unparse`, with one final line feed. Each count includes parsing, validation of syntax and scope, substitution, the rules and the acceptance interface. The checkers read one JSON value, a closed goal and a proof, and accept closed theorems; open sequents occur in the branches. The universal statements of this chapter are the written proofs; the packages' finite test suites concern the implementations.

Representation. With three exceptions the checkers share one codec: 0 is the integer 0, a free parameter is a negative level, a bound variable is a positive index, St is `[0,t]`, the iterator is `[n,a,h]`, an equation is `[s,t]` and $D_{x,y}B$ is `[B]`, the expected sort separating a successor from an equation. Every inserted term has no loose positive index, so opening a binder needs no shifting; an elimination or introduction assigns one new negative level and validates its conclusion in the outer scope, which enforces the eigenvariable condition. `clausal_two_case` uses tagged lists and infers the conclusion of an elimination; it proves, by induction on a successful computation, that a returned formula is well scoped and derivable in C from the retained context, and that every derivation of a closed theorem has a certificate. `clause_contexts` writes St as `[t]` and $D_{x,y}B$ as `["d",B]`. `prefix_clauses` stores a formula as `[k,s,t]` and St as `[t]`; the expected sort separates a record from an iterator.

Protocols. `clausal_single_evaluation` and `clause_contexts` check goal-directed certificates in which a validated reflexive goal closes without a proof node. The selected checker of `clause_contexts` reads a JSON array of two-field packets followed by the goal, from right to left; the shape of the first field selects USE, CLOSE, OPEN or STEP, an unconsumed prefix is rejected, and an unconditional local clause is stored with the antecedent $0 = 0$, which closes without input. The package proves that a successful check yields a derivation of V^0 and that every derivation of a closed theorem has an accepted certificate, by induction on the call tree and by the invariant that a call consumes exactly its own segment. In `prefix_clauses` and `evaluation_discharge` the premise field of a USE whose premise is omitted or absent is an inert field of the format. `evaluation_discharge` infers every intermediate conclusion from the proof tree and supplies no template to DISCHARGE.

What the measurements show. The deletion of the root row saves 47 bytes, 5 tokens and 7 AST nodes in an unchanged format. Uniform clauses save 80 bytes against the same validator. Zero-normal syntax with one evaluation scheme is smaller than its raw control in `clausal_single_evaluation` and `rootfree_clausal` and larger in the companion of `reduced_clause_calculus`. The diagonal checker is 100 bytes larger than the witness-mode checker of `rootfree_clausal`. In `clause_contexts` the factorization of clauses alone saves 53 bytes, 2 tokens and 5 AST nodes against the strongest control, and the rest of its reduction comes from the packet protocol and shared validation; its ranked representation is larger than its recursive one. In `prefix_clauses` the prefix representation alone saves 3 tokens and 63 bytes and adds 10 AST nodes against its nested control. The one-body checker and the homogeneous checker are larger than their controls, by the cost of the gate and its lifting.

The fusion at the discharge saves 103 bytes, 22 tokens and 22 AST nodes on raw terms and 21 bytes, 12 tokens and 10 AST nodes on zero-normal terms. `one_body` also reports a refinement of an existing checker for the native calculus on the equation and D language that changes no rule. The checker of PD in `476_byte_presentation` measures 678 tokens, 727 AST nodes and 2087 bytes, against 688/735/2128 for its matched split control (the four families of K^{u}, same strict certificate contract) and 643/695/1892 for the inherited checker of ED, so it saves 10 tokens, 8 AST nodes and 41 bytes against its control and sets no size record.

Table 5.1: Checker sizes as stated by the packages: tokens, AST nodes, normalized bytes. A control is given as tokens/AST nodes/bytes.

Package	Program	Tok.	AST	Bytes	Matched control
`raw_clausal`	five rules, three rows (C_3)	853	993	3046	canonical five-rule control 903/1035/3175; raw six-rule control 930/1072/3252; upstream canonical checker 946/1072/3305
`rootfree_clausal`	raw, two rows (C)	837	977	2956	matched raw control 842/984/3003; incoming raw clausal 853/993/3046
`rootfree_clausal`	zero-normal, fixed zero base ($\mathsf{C}^{0}_{\mathrm{b}}$)	852	987	2948	zero-normal control with root row 864/1006/2993
`rootfree_clausal`	zero-normal, witness test (C^{0})	853	992	2948	the same control; tagged native normal comparator 979/1047/3011
`reduced_clause_calculus`	raw, two rows (C)	848	986	2999	exact raw clause control 853/993/3046
`reduced_clause_calculus`	companion, one scheme (C^{0})	871	1011	3050	zero-normal with root row 879/1023/3107
`clausal_two_case`	two-case checker (C)	794	868	2787	matched three-case control 808/888/2861; pinned three-case checker 853/993/3046; the same with the root row deleted 848/986/2999
`clausal_single_evaluation`	single evaluation (C^{0})	741	826	2283	matched raw control 744/842/2416; zero-normal control with root row 765/865/2426
`clausal_single_evaluation`	forward implementation	867	1008	3022	forward control 853/993/3046
`uniform_clauses`	uniform clauses (V)	807	931	2912	raw clause checker 853/993/3046; with shared validator 825/958/3039; two rows 848/986/2999; two rows, shared validator 820/951/2992

Package	Program	Tok.	AST	Bytes	Matched control
`clause_guards`	diagonal (K)	872	1013	3056	native diagonal control 975/1138/3395; witness-mode clause control 837/977/2956
`clause_guards`	mirror guard	874	1012	3059	the same
`clause_contexts`	four schemes, packets (V^0)	666	770	2282	pinned five-pattern tagged source 829/902/2950; five-pattern arity and scope control 753/828/2730
`clause_contexts`	packets, empty antecedent	665	770	2308	the same
`clause_contexts`	explicit tree checker	751	823	2677	the same
`clause_contexts`	ranked tree checker	781	853	2692	the same
`clause_contexts`	support-rank experiment, not selected	854	957	3069	none
`prefix_clauses`	prefix calculus (PF)	788	900	2700	matched nested control 791/890/2763; `uniform_clauses` 807/931/2912
`prefix_clauses`	homogeneous variant (HB)	840	965	2843	the same
`one_body`	one body (OB)	901	1041	3130	matched two-field control 848/986/2999
`one_body`	native checker, preferred refinement	357	412	778	existing native checker 358/413/788
`one_body`	native checker, byte-priority variant	357	415	777	the same
`evaluation_discharge`	raw fusion (ED)	643	695	1892	raw separate rules 665/717/1995
`evaluation_discharge`	zero-normal fusion (ED^0)	633	697	1858	zero-normal separate rules 645/707/1879; incoming controls 663/751/1983, 683/771/1990 and 636/708/2010

5.10 Further questions

Question 5.81 (Successor rigidity)**.** Do the computation equations, coverage, separation and SR_0 construct a term $\delta(a, b)$ with $\delta(a, b) = 0 \leftrightarrow a = b$ on the whole domain without using induction for arbitrary literal equations? Such a term would turn ZI into induction for every literal equation, and the bridge (Theorem 1.119) would apply. The known proofs of the predecessor (Theorem 1.65) and of the reflection of equality (Theorem 1.86) use inductions on general equations or uniqueness with updates other than S (Sections 2.5.4 to 2.5.7), so importing them is circular. `raw_clausal` claims no converse from ZI to SR_0, no equivalence

of SR_0 with unrestricted uniqueness, and no checker for SR_0.

Question 5.82 (A stable introduction at no cost)**.** Is there an introduction whose mode is preserved and reflected by substitution, as in Theorem 5.36, with a complete checker no larger than the witness-mode checker? The diagonal checker of `clause_guards` is larger by 100 bytes. Is there a guard that satisfies the conditions of Theorem 5.38 and is cheaper to test than the diagonal shape?

Question 5.83 (Tree-preserving substitution at no cost)**.** Does another syntax or certificate design turn the structural property of PD (Theorem 5.73) into a complete checker below the current minima? The checker of PD in `476_byte_presentation` is smaller than its matched four-family control but larger than the inherited checker of ED and than the compact checkers of other packages, and the package claims no size record for it.

Question 5.84 (One-body contexts without the cost of the gate)**.** Can the antecedent field be removed, as in Theorems 5.53 and 5.58, by a representation of assumptions that needs neither an origin tag nor a gate with lifting, so that a complete checker becomes smaller? The measured checkers of `one_body` and of the homogeneous variant of `prefix_clauses` are larger than their controls, and a variant of `one_body` with primitive deep reflexivity is larger still (3208 bytes).

Question 5.85 (The identity case of the discharge)**.** Can the identity case of ϱ be replaced by a single arithmetic pattern that is always active, without a supplied template and without enlarging the checker or the arithmetic assumptions? The partial operation ϱ^{+} of Section 5.6.3 keeps the identity case only for an E with root D; it does not remove the case.

Question 5.86 (Combinations not measured)**.** `prefix_clauses` and `one_body` ask for the diagonal introduction on the prefix representation and on one-body contexts, with a matched measurement. Neither combination is stated or measured in the packages.

Question 5.87 (Sizes of derivations)**.** Theorem 5.16 bounds inference nodes, not serialized size. `raw_clausal` reports that its translation from the six-rule raw calculus to C_3 produced at most nine times as many nodes on every executed example; no general bound is proved. No package bounds the serialized size of a translated derivation, the cost of closing and reopening a closure with an unused witness, or the duplication of the major premise in Theorem 5.63 (the translation of ELIM in Theorem 5.75 avoids that duplication by the derivation of Θ). `rootfree_clausal`, `reduced_clause_calculus` and `evaluation_discharge` ask whether certificates with sharing recover these costs once reference validation is charged.

The following statements of the packages are negative or unproved and are recorded as such.

- No package claims a smallest presentation or a lower bound over presentations, and none of the checkers of Table 5.1 is smaller than the compact codecs of other packages (516 and 508 normalized bytes under their own input contracts, 788, 780 and 778 bytes for validated public-goal checkers). The packages ask for a fixed common contract, comprising goal grammar, scope checking, normalization and measurement, before any such comparison.
- The formal statements not present in the Lean development are: the calculus $\mathsf{C}_{\mathrm{b}}^{0}$ on clauses (Theorem 5.41), the guard criterion and the mirror guard on clauses (Theorems 5.38 and 5.39), the support quotient (Theorem 5.47), the homogeneous calculus (Theorem 5.58), the zero-normal fusion (Theorem 5.68), the propositions Theorems 5.17 and 5.18, the obstruction Theorem 5.25, the theorem Theorem 5.79, the covariance Theorem 5.71 and the tree-preserving substitution Theorem 5.73 (`476_byte_presentation`

claims no Lean or Rocq verification of its results or of its proof compilers; Theorems 5.74 and 5.75 are formalized in this repository, see Theorem 5.76), and the correspondence of each Python checker with its calculus.

- `reduced_clause_calculus` implements the translation of certificates from raw to zero-normal syntax only; the converse compiler is not implemented, and the converse is the theorem Theorem 5.22.
- `clause_contexts` reports that neither the ranked representation nor the support quotient reduced its checker, and `prefix_clauses` that the prefix representation does not reduce every coordinate against its nested control.
- `reduced_clause_calculus` asks for a deletion that removes an arithmetic obligation, a term constructor or a logical operation while preserving both composites; the deletions of this chapter concern the proof interface and leave the arithmetic basis unchanged. Theorems 5.18 and 5.25 show that, with the other rules fixed, reflexivity cannot be replaced by finitely many closed seeds and neither evaluation row of C can be dropped.

Chapter 6

Discharge of a witness and restricted equality rules

A derivation in the native scalar core N (Section 1.3) supplies data at three places. The rule TR takes an equation $s = t$ together with a term template. The rule DE opens a paired formula at a fresh witness and is separate from every equality rule. The rule QE, and the use of a clause that replaces it in a clause calculus (Section 1.9), takes an arbitrary term. The calculi of this chapter fuse or restrict these interfaces.

In Section 6.2 one rule, NAME, discharges an opened witness b together with a proved equation $b = t$; it replaces both the equality rule and the elimination of $D_{x,y}$. In Section 6.3 neither the equation nor the term t is supplied: both are read off two rows of the matrix. In Section 6.4 the fusion is carried out on the canonical recursor syntax, where no normalization is available and reflexivity is derived. In Section 6.5 the equality rule substitutes for a variable only, for the newest variable only, or, on raw terms, for a free parameter; an obstruction shows that unary congruence is not enough. In Section 6.6 the use of a clause takes no term argument. In Section 6.7 a primitive fresh alias, whose discharge substitutes into the conclusion, and substitution for a parameter in the backward direction replace the equality rule; the source of equality and the target of every use of a clause are then the newest parameter, and both new rules can be restricted to literal equations. Section 6.8 records an induction interface by a constancy rule, with a recursor that sees its counter and with a scalar iterator, together with an obstruction for the scalar form without its comparison equation. Sections 6.9 and 6.10 summarize the checkers, and Section 6.11 lists what is left open.

Throughout, $\equiv$ is literal identity of syntax up to the names of bound variables, a *parameter* is a free numerical variable, and a *literal equation* is a formula whose root is an equation. The numerals $S0$ and $SS0$ are written 1 and 2, and H is the equation $0 = 0$.

6.1 Normal forms, clause contexts, and the reference calculus

This section fixes what is common to `witness_discharge`, `inferred_witness`, `deterministic_opening`, `first_instance`, `clause_discharge` and `variable_substitution`. Full normalization is the subject of Section 4.11 and the clause calculi on raw terms that of Section 5.2; they are cited, and only what is specific to the calculi of this chapter is stated here. The remark on the strict traversal is from `first_instance`.

6.1.1 Conventions on full normal forms

In Sections 6.1 to 6.3 and in Sections 6.5.1 and 6.5.2 every term and every formula is in full normal form: nf is the normal form for the reduction of Theorem 4.5, which exists and is

unique by Theorems 4.131 and 4.132, and every displayed instance $B(t)$, $B(t,r)$ or $F[t/b]$ is formed with the normalizing substitution of Theorem 4.25. Substitution is simultaneous: an inserted term is not traversed again, so that $F[t/b]$ is defined also when t contains b. By Theorem 4.135 the map nf is an admissible normalizer; in particular it commutes with injective renaming, it leaves the root of a formula unchanged, and it is coherent with substitution,

$$\mathrm{nf}(T\sigma) = \mathrm{nf}\big(\mathrm{nf}(T)[x \mapsto \mathrm{nf}(\sigma x)]\big), \tag{6.1}$$

so that instances compose (Theorem 4.26) and it is immaterial whether a compound expression is normalized before or after one of its variables is instantiated. The counter of an iterator in a normal form is neutral (Theorem 4.134).

Let P be the parity term of Theorem 1.3, written N there, $P(y) = I(y, 0;\, u.\, \mathrm{sel}(u, 1, 0))$. On normal forms the computations of Eq. (1.7) and of the selector are literal identities:

$$\mathrm{sel}(0, a, c) \equiv a, \qquad \mathrm{sel}(Sv, a, c) \equiv c, \qquad P(1) \equiv 1, \qquad P(2) \equiv 0. \tag{6.2}$$

No induction, and no statement about P at other arguments, is used anywhere in this chapter.

Remark 6.1 (Termination of a strict traversal). The checkers of this chapter normalize the counter, the seed and the body first and, at a successor counter, normalize the predecessor iterate $I(n, a;\, u.\, h)$ before inserting it into the body. If the body does not mention its state variable, that value is erased by the root contraction, so the strategy is not a reduction sequence of Theorem 4.5 as it stands. `first_instance` justifies it as follows. Add an administrative term $\mathsf{let}\ u = a\ \mathsf{in}\ h$ with the reduction to $h[a/u]$, and replace the successor contraction by the *strict expansion*

$$I(Sn, a;\, u.\, h) \longrightarrow \mathsf{let}\ v = I(n, a;\, u.\, h)\ \mathsf{in}\ h[v/u], \qquad v \text{ fresh},$$

reducing under all constructors and under both components of a let. No new normal forms arise: every let has a root contraction, and that contraction composes with a strict expansion to the original successor contraction. A let is strongly normalizing whenever its value is strongly normalizing and its body sends every strongly normalizing term substituted for u to a strongly normalizing term; this is proved by induction on the sum of the reduction heights of the two components, an internal contraction decreasing the sum and the root contraction being covered by the hypothesis on the body. The proof of Theorem 4.130 uses this fact at the successor root, after obtaining the predecessor iterate by the same lexicographic induction, and the structural induction of Theorem 4.131, extended by the new constructor, gives strong normalization of the auxiliary syntax. The strict traversal is an evaluation strategy of the auxiliary calculus: after a strict expansion it normalizes the value and then contracts the let. Hence it terminates, also when the value is erased, and since forgetting the administrative terms preserves conversion it returns the unique normal form $\mathrm{nf}(t)$.

Remark 6.2 (Bounded checkers). By Theorem 4.145 the normalizer is total recursive and not primitive recursive; it is part of the specification of every calculus on normal forms and is counted in every checker of such a calculus. `witness_discharge`, `inferred_witness`, `first_instance` and `clause_discharge` each supply a second checker with a fuel field, as in Theorem 4.146. The metering differs from the one there: one unit is consumed at every entry into the recursive traversal of syntax and into the recursive proof check, not at every successor contraction. The conclusions are the same, by the same argument. An accepting unbounded run with k metered entries is accepted with fuel k and rejected with fuel $k-1$, and erasing the fuel turns an accepting bounded run into an accepting unbounded run, so that

$$\exists p\ \ \mathrm{Accept}(A, p) \quad \Longleftrightarrow \quad \exists p\, \exists k\ \ \mathrm{Accept}_{\mathrm{fuel}}(A, p, k);$$

and since every cycle of the call graph passes through a metered entry, the bounded acceptance relation is primitive recursive. Nothing is asserted about a primitive recursive bound on the fuel that a derivable formula needs.

6.1.2 Formulas, clauses, and the two conventions

The formulas of Sections 6.1 to 6.3 are the Q-free formulas of Theorem 5.1 over normal forms, with the closure abbreviation $q_y A(y) = D_{w,y}A(y)$, w unused, of that definition. Contexts are lists of clause occurrences (Sections 1.9 and 5.1.2). For a matrix $B(x, y)$ and a term r the *opened clause* is

$$\mathcal{C}_B(r) := [B(r, 0);\ y.\, B(r, y)].$$

Two conventions for assumptions and reflexivity occur in the packages.

- *The convention with* REFL. Every clause has exactly one antecedent, an assumption F is the clause $\mathrm{ord}(F) = [H;\ v.\, F]$ of Section 5.1.2, and reflexivity is a rule.
- *The convention with the reflexivity clause.* Clauses are the uniform clauses of Theorem 5.26, with at most one antecedent; an assumption F is the unconditional clause $[\ ;\ v.\, F]$; every context contains one distinguished occurrence of the clause $\mathcal{R} = [\ ;\ y.\, y = y]$ of that definition; and there is no reflexivity rule: $t = t$ is a use of $\mathcal{R}$ at t.

In both, Γ, F denotes Γ extended by the assumption clause of F. Theorem 6.13 shows that the two conventions are interchangeable. All proofs below are written in the convention with REFL.

6.1.3 The shared rules and the reference calculus

The rules USE, ELIM and INTRO are those of Theorem 5.4, read on full normal forms with normalizing substitution; INTRO is the witness-selected introduction of Theorem 4.1 with a clause as its assumption. The equality rule of the reference calculus is TR, as in N.

Definition 6.3 (Shared rules)**.**

$$\frac{}{\Gamma \vdash t = t}\ \text{REFL} \tag{6.3}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \text{USE}, \qquad [A;\ y.\, B(y)] \text{ an occurrence in } \Gamma \tag{6.4}$$

$$\frac{\Gamma,\ B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}\ \text{INTRO}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise} \end{cases} \tag{6.5}$$

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma,\ \mathcal{C}_B(b) \vdash A}{\Gamma \vdash A}\ \text{ELIM} \tag{6.6}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}\ \text{TR} \tag{6.7}$$

In USE the term t is arbitrary; for an unconditional clause the rule has no premise. In INTRO the variable z is fresh for Γ, for the witness t and for $D_{x,y}B$; the instance with $\eta = 0$ is the *case mode* and the instance with $\eta = z$ the *progress mode*, and the mode is determined by the literal syntax of the normal forms, not by provable equality. In ELIM the variable b is fresh for Γ, A and $D_{x,y}B$. In TR, $h(w)$ is an arbitrary term template.

Definition 6.4 (The reference calculus)**.** $\mathcal{T}$ is the calculus with the five rules REFL, USE, INTRO, ELIM and TR, in the convention with REFL.

$\mathcal{T}$ is the fully normalized clause calculus with which the packages of this section compare themselves. It is the clause form of the six-rule calculus F of Theorem 4.139.

Proposition 6.5 (The reference calculus and the kernel)**.** *For a native formula A let $A^\flat := \tau(\mathrm{nf}(A))$, with τ the Q-free form of Theorem 5.1. Then* N *derives $\Gamma \vdash A$ if and only if $\mathcal{T}$ derives $A^\flat$ from the assumption clauses of the formulas $G^\flat$, $G \in \Gamma$; and* N *derives A from $A^\flat$ and conversely. The translation is the identity on normalized Q-free formulas and fixes $0 = S0$.*

Proof. N derives A from $\mathrm{nf}(A)$ and conversely by Theorem 4.6, and $\mathrm{nf}(A)$ from $A^\flat$ and conversely by Theorem 5.2(iii). By Theorem 4.137, N and the kernel N^f of Theorem 4.136 on full normal forms have the same consequences, and by Theorem 4.141 so does F; it remains to show, for Γ and A in full normal form, that $\Gamma \vdash_\mathsf{F} A$ if and only if $\mathrm{ord}(\tau\Gamma) \vdash_\mathcal{T} \tau A$. This is the argument of Theorem 5.11 with F in the place of N and $\mathcal{T}$ in the place of C_3; it is shorter, because F and $\mathcal{T}$ have the same equality rule and the same introduction.

From $\mathcal{T}$ to F, for an annotated context (Theorem 5.3) and its native image: REFL and TR are rules of F; a USE of an assumption occurrence is HYP, and a USE of an opened occurrence is QE; ELIM is DE; and INTRO is WI in the same mode, its new occurrence being an assumption.

From F to $\mathcal{T}$: HYP is REFL at 0 followed by USE; REFL and TR are rules; QE is the second rule of Theorem 6.7 below, since $\tau(Q_y B) \equiv q_y \tau B$; and WI is INTRO in the same mode, because τ sends literal equations to literal equations and all other formulas to formulas with root D. For DE apply ELIM to the translated major premise. In the extended context, under the assumption $\tau B(b, 0)$, the opened clause at Sz gives $\tau B(b, Sz)$, so the first rule of Theorem 6.7 gives $q_y \tau B(b, y)$, which is $\tau(Q_y B(b, y))$; weaken the translated branch and cut this formula into it (Theorem 6.6).

Finally, from $\mathrm{ord}(\tau\Gamma) \vdash_\mathcal{T} \tau A$ the first translation gives $\tau\Gamma \vdash_\mathsf{F} \tau A$, and Theorem 5.2(iii), which holds in F by Theorem 4.141, with cut replaces $\tau\Gamma$ and τA by Γ and A. □

6.1.4 Structural operations

Lemma 6.6 (Weakening, renaming, cut)**.** *Let $\mathcal{S}$ be a calculus whose rules are among those of Theorem 6.3 and the rules* NAME (6.10), OPEN (6.11), EXPORT (6.12) *and* VSUB (6.16) *introduced below, and which contains* REFL *and* USE*. The following are admissible in $\mathcal{S}$, by direct induction on derivations.*

(a) *Weakening: if $\Gamma \vdash A$ and every occurrence of Γ is an occurrence of Γ', then $\Gamma' \vdash A$.*

(b) *Injective renaming: if $\Gamma \vdash A$ and ι is an injective map of parameters, then $\Gamma\iota \vdash A\iota$.*

(c) *Cut of an assumption: if $\Gamma \vdash F$ and $\Gamma, F \vdash A$, then $\Gamma \vdash A$.*

None of the three substitutes a term for a parameter in a derivation.

Proof. The inductions are those of Theorem 5.5; for REFL, USE, ELIM, INTRO and TR nothing changes, since normalization commutes with injective renaming. Only the side conditions of the four new rules have to be checked under an injective renaming. Such a renaming preserves and reflects the property of being a variable, the literal identities tested by the selector of Theorem 6.23, and the absence of a parameter from a formula. Hence the selected witness of OPEN and EXPORT, the source of VSUB and the export conditions of NAME and EXPORT are preserved. For weakening and cut, no side condition of the new rules refers to the clauses of the context other than through freshness, and the selector depends on the matrix of the major premise alone. □

Substitution of a term for a parameter does not preserve rule instances in general. It can turn a witness into the literal 0 and so change the mode of INTRO (Theorems 4.144 and 5.14), it can make the test of the selector succeed, and it can turn the variable source of VSUB into a compound term. Admissibility of substitution is proved for the calculus $\mathcal{O}$ in Theorem 6.32 and follows for the others from the equivalences.

6.1.5 Closures, numerical cases, and the two introductions

The lemmas of this subsection and the next are the counterparts on full normal forms of Theorems 5.6, 5.7 and 5.9 and Theorem 5.8. They are restated because the calculi of Sections 6.2 and 6.3 do not have ELIM as a rule, and one of them has it at first for two kinds of matrices only; the hypotheses record what each argument uses.

Lemma 6.7 (Closure and numerical cases)**.** *Let $\mathcal{S}$ contain* USE *and* INTRO *and admit weakening, and suppose that* ELIM *is a primitive or derived rule of $\mathcal{S}$ for every matrix in which the witness coordinate does not occur. Then the following rules are derived, for every formula $A(y)$, every term t and z fresh:*

$$\frac{\Gamma,\ A(0) \vdash A(Sz)}{\Gamma \vdash q_y A(y)} \qquad \frac{\Gamma \vdash q_y A(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)} \qquad \frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}$$

No equality rule is used.

Proof. The second rule is Theorem 5.9(ii), whose proof eliminates a matrix with an unused witness coordinate, and the third follows from the first two as in Theorem 5.10. The first rule is obtained differently from Theorem 5.9(i), with no appeal to an unrestricted case introduction: apply INTRO to the matrix $A(y)$, with the unused witness coordinate, at the witness 1; since $1 \not\equiv 0$ the rule is in case mode at every root, and its premise is the given one. □

The third rule is a distinction of numerical cases, not an induction: its successor premise has no hypothesis.

For a literal equation E let $G(v, E)$ be the gate of Theorem 4.142, with $G(0, E) \equiv H$ and $G(Sw, E) \equiv E$, and let the *envelope* in the present formulas be $L(E) := q_v G(v, E)$.

Lemma 6.8 (The two unrestricted introductions)**.** *Let $\mathcal{S}$ contain* REFL, USE *and* INTRO*, admit weakening and cut, and have* ELIM *as a primitive or derived rule for every matrix in which the witness coordinate does not occur and for every matrix with root D. Then for every matrix B, every literal equation $E(x, y)$ and every witness t the rules*

$$\frac{\Gamma,\ B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y} B(x,y)} \qquad \frac{\Gamma,\ E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y} E(x,y)}$$

are derived, with z fresh as in INTRO*.*

Proof. Case introduction. INTRO is the first rule unless B is a literal equation $E(x, y)$ and $t \equiv 0$. For that case the derivation is the one given after Theorem 4.142 for the missing case of Theorem 4.141, read in the present formulas: Q_v becomes q_v; the two derivations $\Gamma, E \vdash L(E)$ and $\Gamma, L(E) \vdash E$ of Theorem 4.142 use the first and the second rule of Theorem 6.7, with REFL for the base H; the matrix $L(E(x, y))$ has root D, so it is introduced at the witness 0 in case mode and opened by ELIM at a fresh c; and the retained closure is the opened clause, used by USE at Sz. The derivation eliminates matrices with an unused witness coordinate and one matrix with root D, and no others.

Progress introduction. This is the third statement of Theorem 5.10, with the same proof: INTRO for the matrix $E(t, y)$ at the literal witness 0 of an unused coordinate is in progress mode and yields $q_y E(t, y)$; in the context $\Gamma, E(t, 0)$ the second rule of Theorem 6.7 gives $E(t, Sz')$; and the case introduction at the witness t concludes. □

6.1.6 Equality from one term transport

Uncrossed transport is the rule

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash h(t) = r}\ \textsc{utr} \tag{6.8}$$

for an arbitrary term template $h(w)$.

Lemma 6.9 (Equality algebra)**.** *Let $\mathcal{S}$ have* REFL*, as a rule or derived. If* UTR *is derivable in $\mathcal{S}$, or if* TR *is, then so are both, and so are symmetry, transitivity, the congruence $s = t \vdash h(s) = h(t)$, and transport in every equation template: $s = t,\ p(s) = q(s) \vdash p(t) = q(t)$.*

On raw terms the same consequences are Theorem 5.6; what is specific here is that one term transport, crossed or uncrossed, suffices once reflexivity is available.

Proof. From UTR: with $h(w) = w$ and $r \equiv s$ the premises are $s = t$ and the reflexive $s = s$, and the conclusion is $t = s$; this is symmetry, and UTR followed by symmetry is TR. From TR and reflexivity, Theorem 4.2(b) gives symmetry, transitivity, congruence and transport in an equation template, and TR followed by symmetry is UTR. □

Lemma 6.10 (Transport in a formula)**.** *Let $\mathcal{S}$ contain* USE *and* INTRO*, admit weakening, have* ELIM *as a primitive or derived rule for all matrices, and derive symmetry and transport in every equation template. Then for every formula template $F(w)$*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

is derived in $\mathcal{S}$.

Proof. The proof of Theorem 5.8 applies, by induction on the syntax of the template. Its one appeal to Theorem 5.7 is an INTRO at the fresh parameter opened by ELIM; a parameter is not the literal 0, so that instance is in case mode whatever the root of the matrix is, and the case introduction at the witness 0 of Theorem 6.8 is not needed. □

6.1.7 Two-row packets

The device by which a calculus without a general equality rule recovers TR is a paired formula whose matrix has prescribed rows at $y = 0, 1, 2$. It first occurs in `variable_substitution`.

Definition 6.11 (Packet)**.** Let $E_1(x) = (a_1 = c_1)$ and $E_2(x) = (a_2 = c_2)$ be equations in which x may occur and y does not. The *packet* with rows E_1, E_2 is the matrix

$$\Pi(x, y) := \Big(\mathrm{sel}\big(y, 0, \mathrm{sel}(P(y), a_2, a_1)\big) = \mathrm{sel}\big(y, 0, \mathrm{sel}(P(y), c_2, c_1)\big)\Big).$$

By (6.2) its rows are literally

$$\Pi(x, 0) \equiv H, \qquad \Pi(x, 1) \equiv E_1(x), \qquad \Pi(x, 2) \equiv E_2(x). \tag{6.9}$$

Lemma 6.12 (Introduction of a packet)**.** *Let $\mathcal{S}$ contain* USE *and* INTRO*, admit weakening, and have the numerical-case rule of Theorem 6.7. If $\Gamma \vdash E_1(s)$ and $\Gamma \vdash E_2(s)$, then $\Gamma \vdash D_{x,y}\Pi(x, y)$. In any context that contains the opened clause $\mathcal{C}_\Pi(b)$,* REFL *and* USE *derive $E_1(b)$ and $E_2(b)$.*

Proof. Apply INTRO at the witness s. Its goal is

$$\Pi(s, Sz) \equiv \big(\mathrm{sel}(P(Sz), a_2(s), a_1(s)) = \mathrm{sel}(P(Sz), c_2(s), c_1(s))\big).$$

Let $A(v) := \big(\mathrm{sel}(v, a_2(s), a_1(s)) = \mathrm{sel}(v, c_2(s), c_1(s))\big)$. Then $A(0) \equiv E_2(s)$, $A(Sz') \equiv E_1(s)$ and, by (6.1), $A(P(Sz)) \equiv \Pi(s, Sz)$. The numerical-case rule at the target $P(Sz)$, with the two given derivations weakened to the local context and an eigenparameter z' chosen fresh independently of z, proves the goal. The assumption of INTRO is not used, so the derivation is valid in either mode, in particular when $s \equiv 0$. The second statement is (6.9): the antecedent of $\mathcal{C}_\Pi(b)$ is H, proved by REFL, and USE at 1 and at 2 gives the two rows at b. □

6.1.8 Interchangeability of the two conventions

Lemma 6.13 (Reflexivity as a fixed clause)**.** *Let $\mathcal{S}$ be a calculus in the convention with* REFL *whose other rules are among* USE, INTRO, ELIM, TR, NAME, OPEN, EXPORT *and* VSUB*, and let $\mathcal{S}^{\mathcal{R}}$ have the same rules without* REFL*, in the convention with the reflexivity clause. For a context Δ of uniform clauses let $L(\Delta)$ be the context Δ° of Theorem 5.26, which replaces every unconditional occurrence $[\,;\, y.\, B]$ by $[H;\, y.\, B]$. Then for every formula F*

$$\mathcal{R}, \Delta \vdash_{\mathcal{S}^{\mathcal{R}}} F \quad \Longleftrightarrow \quad L(\Delta) \vdash_{\mathcal{S}} F.$$

The statement covers arbitrary contexts, not only those that arise in a derivation from $\mathcal{R}$ alone.

Proof. The two inductions of the proof of Theorem 5.27 apply. They translate only the uses of $\mathcal{R}$, the uses of unconditional clauses, REFL, and the assumption added by INTRO, and they copy every other rule; occurrences correspond by position, also when two have the same value. The rules that are copied here are TR, VSUB, and the rules ELIM, NAME, OPEN and EXPORT, each of which adds the clause $\mathcal{C}_B(r)$ with one antecedent on both sides and has the same premises and side conditions in the two conventions. □

6.2 Naming packets: equality at the discharge of a witness

This section draws on `witness_discharge`. The packet is that of `variable_substitution` (Section 6.5.1); its final substitution and the elimination of its witness become one inference here.

6.2.1 The rule name

Equality substitution is performed only while the fresh witness of a paired formula is discharged:

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma,\ \mathcal{C}_B(b) \vdash b = t \qquad \Gamma,\ \mathcal{C}_B(b) \vdash F}{\Gamma \vdash F[t/b]}\ \textsc{name} \tag{6.10}$$

The variable b is fresh for Γ and $D_{x,y}B$, and the exported formula $F[t/b]$ must not contain b. The two branch premises have the same extended context. The term t and the formula F may contain b; in particular $t \equiv b$ is allowed. The side condition concerns the exported normal form and is not a prohibition of b in t.

Definition 6.14 (The calculus of witness discharge)**.** $\mathcal{W}$ is the calculus with the four rules REFL, USE, INTRO and NAME, in the convention with REFL. $\mathcal{W}^{\mathcal{R}}$ has the three rule families USE, INTRO and NAME and the fixed clause $\mathcal{R}$. There is no further logical, equality, computation, case, induction, weakening or cut rule.

Lemma 6.15 (Soundness of NAME)**.** *NAME is sound for the ordinary reading in* T_{eq} *(Section 1.5), for every formula* F *and also when* t *contains* b*.*

Proof. Eliminate the existential quantifier in the reading of the major premise at a fresh b; the local assumption is the reading of $\mathcal{C}_B(b)$. The two branch derivations give $b = t$ and F. Equality substitution in the template $F[w/b]$ gives the raw instance at t, and the computation equations identify it with its normal form $F[t/b]$. This step does not need $b \notin \mathrm{FV}(t)$: an equation $b = t(b)$ supports the simultaneous substitution of $t(b)$ for b in a fixed formula. What the elimination of the quantifier needs is the absence of b from the result, which is the side condition. □

USE has the reading of its clause, and INTRO is sound by coverage in case mode and by induction for literal equations in progress mode.

Lemma 6.16 (Ordinary elimination)**.** *ELIM is a derived rule of* $\mathcal{W}$*.*

Proof. Take $t \equiv b$ in NAME. The equality premise is REFL, the other premise is the given derivation of A, and $A[b/b] \equiv A$ does not contain b. □

By Theorems 6.6 and 6.16 the closure rules, the numerical cases, the two unrestricted introductions and the introduction of packets (Theorems 6.7, 6.8 and 6.12) are available in $\mathcal{W}$. None of them uses an instance of NAME other than with $t \equiv b$.

6.2.2 Transport from one discharge

Theorem 6.17 (Transport in $\mathcal{W}$)**.** *For every term template* $h(w)$ *the rule* UTR (6.8) *is derived in* $\mathcal{W}$*. Apart from ordinary eliminations, its derivation contains one instance of* NAME*.*

Proof. Let $\Gamma \vdash s = t$ and $\Gamma \vdash h(s) = r$, let x be fresh for h, r and t, and let Π be the packet with the rows

$$E_1(x) = (h(x) = r), \qquad E_2(x) = (x = t).$$

The two given equations are $E_1(s)$ and $E_2(s)$, so $\Gamma \vdash D_{x,y}\Pi$ by Theorem 6.12. Apply NAME with a fresh b. In the context $\Gamma, \mathcal{C}_\Pi(b)$ the same lemma gives the row $b = t$, which is the equality premise, and the row $h(b) = r$, which is taken as F. The exported formula is

$$(h(b) = r)[t/b] \equiv (h(t) = r)$$

by (6.1), because b occurs neither in h nor in r; it does not contain b. The identity is one of normal forms and not an application of an equality rule. □

Corollary 6.18 (Equality in $\mathcal{W}$)**.** $\mathcal{W}$ *derives symmetry, transitivity, congruence,* TR*, and transport in every formula template.*

Proof. Theorem 6.9 applies by Theorem 6.17; in particular symmetry is obtained from the packet with the identity template and a reflexive minor premise, and is not assumed by it. Theorem 6.10 applies by Theorem 6.16. □

6.2.3 Equivalence

Theorem 6.19 ($\mathcal{W}$ and the reference calculus)**.** *For every context* Γ *and formula* F*,*

$$\Gamma \vdash_{\mathcal{W}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{T}} F.$$

Both directions are effective transformations of derivations that leave the end sequent unchanged.

Proof. From $\mathcal{T}$ to $\mathcal{W}$, translate a derivation recursively: ELIM is replaced by Theorem 6.16 and TR by Theorem 6.18; the other three rules are shared. The constructions that replace a TR node were derived without TR and without a translated node, so the recursion is on the given derivation alone.

From $\mathcal{W}$ to $\mathcal{T}$: $\mathcal{T}$ derives symmetry and UTR from TR by Theorem 6.9, and hence transport in every formula template by Theorem 6.10. At a NAME node the translated branch premises give $b = t$ and F in the context $\Gamma, \mathcal{C}_B(b)$. Transport in the template $F[w/b]$, w fresh, gives $F[t/b]$ in that context; the substitution is simultaneous, so this is correct also when t contains b. By the side condition the result does not contain b, and ELIM discharges the clause. The finitely many eigenparameters are renamed before each replacement. □

Corollary 6.20 (Three rule families)**.** *For every context Δ of clauses with at most one antecedent and every formula F, $\mathcal{R}, \Delta \vdash_{\mathcal{W}^{\mathcal{R}}} F$ if and only if $L(\Delta) \vdash_{\mathcal{T}} F$. Consequently* N *derives $\Gamma \vdash A$ if and only if $\mathcal{W}^{\mathcal{R}}$ derives $A^\flat$ from $\mathcal{R}$ and the unconditional assumption clauses of the formulas of $\Gamma^\flat$, and the ordinary reading of $\mathcal{W}^{\mathcal{R}}$ is bi-interpretable with* iPRA *in the sense of Theorem 1.119: whole domains, actual equality, both numerical composites with graph $x = y$, and both formula round trips.*

Proof. The first statement is Theorems 6.13 and 6.19; the second adds Theorem 6.5. The comparisons are the identity on normalized terms and formulas and fix $0 = S0$, so the interpretations and composites of Theorem 1.119 are unchanged. □

Remark 6.21 (What the count of three means; Lean). The number three counts inference patterns after reflexivity has been moved into one explicitly listed axiom clause; it does not say that reflexivity is proved from nothing. The reduction specific to `witness_discharge` is the localization of all equality elimination at the discharge of a witness; the clauses with at most one antecedent and the fixed clause are choices of presentation taken from `uniform_clauses`.

In Lean the kernel form is `NMPrf`: on full normal forms, with NAME in place of TR and DE and with the hypothesis $Q_y B(b, y)$ in the opened context, the kernel has the six rules HYP, REFL, NAME, QE, DI and QI, and `nmprf_iff_nprf` (`NativeWitnessDischarge`) proves it equivalent to N; the packet is `NMPrf.packet`. The clause form $\mathcal{W}^{\mathcal{R}}$ is `WDPrf`, with `wdprf_iff_nprf` (`NativeNameClauses`).

Remark 6.22 (Substitution in $\mathcal{W}$). The substitution inside NAME acts on the proved formula and is justified by the proved equation; it is not a substitution into the branch derivation. Substitution of a term for a parameter in a derivation of $\mathcal{W}$ is admissible through Theorem 6.19 and the admissibility in $\mathcal{O}$ (Theorems 6.32 and 6.36), not by an induction that preserves the rule at each node.

6.3 The inferred witness

This section draws on `inferred_witness`, `deterministic_opening`, `first_instance` and `clause_discharge`. The four packages contain one idea in two forms; Theorem 6.37 records how they differ.

6.3.1 The selector and the two forms of the rule

NAME has three premises, a supplied term t and a supplied formula F. The equation $b = t$ need not be proved: it can be read off the opened clause.

Definition 6.23 (Selector)**.** For a matrix $B(x, y)$ and a variable b that does not occur in $D_{x,y}B$, the *selected witness* is

$$\rho_B(b) := \begin{cases} r, & B(b,0) \equiv H \ \text{ and } \ B(b,1) \equiv (b = r), \\ b, & \text{otherwise.} \end{cases}$$

In the first case the test *succeeds*; in the second the selector *falls back*.

The test inspects two rows of the matrix, at $y = 0$ and at $y = 1$, and compares normal forms literally. The left side of the row at 1 must be the variable b itself; an equation $r = b$, an equation provably of that shape, and an equation whose left side merely contains b are not recognized. The right side r may contain b. There is no search, no decision of provable equality, and no choice left to the certificate.

Lemma 6.24 (Properties of the selector)**.** (a) *For an injective renaming ι, $\rho_{B\iota}(\iota b) \equiv \rho_B(b)\iota$, and the test succeeds for $B\iota$ at ιb if and only if it succeeds for B at b.*

(b) *If x does not occur in B, or if B has root D, the selector falls back.*

(c) *If the test succeeds for B with $\rho_B(b) \equiv r$, and σ is a substitution with $\sigma b \equiv b$ under which b remains fresh for $D_{x,y}B\sigma$, then it succeeds for $B\sigma$ with $\rho_{B\sigma}(b) \equiv r\sigma$.*

(d) *A fallback is not stable under substitution.*

Proof. (a) Normalization commutes with ι, and ι preserves and reflects literal identity. (b) If x does not occur in B, the row $B(b, 1)$ does not contain b and is not an equation with left side b. If B has root D, so has $B(b, 0)$, because normalization does not remove a formula constructor, and it is not H. (c) By (6.1), $(B\sigma)(b, 0) \equiv B(b, 0)\sigma \equiv H\sigma \equiv H$ and $(B\sigma)(b, 1) \equiv (b = r)\sigma \equiv (b = r\sigma)$. (d) For a parameter a and a term r let $B(x, y) := \big(\mathrm{sel}(y, a, x) = \mathrm{sel}(y, a, r)\big)$. Then $B(b, 0) \equiv (a = a) \not\equiv H$ and the selector falls back, while under $a \mapsto 0$ the rows are H and $b = r$ and the test succeeds. □

Lemma 6.25 (The selected equation)**.** *In every context that contains $\mathcal{C}_B(b)$, the equation $b = \rho_B(b)$ is derived by* REFL *and* USE*.*

Proof. If the selector falls back, the equation is $b = b$. If the test succeeds, the antecedent of $\mathcal{C}_B(b)$ is H, proved by REFL, and USE at 1 gives $B(b, 1) \equiv (b = \rho_B(b))$. □

The witness can be used in two ways. Either the clause is opened at the selected witness and the goal is unchanged,

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma,\ \mathcal{C}_B(\rho_B(b)) \vdash A}{\Gamma \vdash A}\ \textsc{open} \tag{6.11}$$

with b fresh for Γ, A and $D_{x,y}B$; or the clause is opened at the fresh variable and the selected witness is substituted in the exported formula,

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma,\ \mathcal{C}_B(b) \vdash F}{\Gamma \vdash F[\rho_B(b)/b]}\ \textsc{export} \tag{6.12}$$

with b fresh for Γ and $D_{x,y}B$ and b not in the exported formula. In OPEN the parameter b stays available in the branch whether or not the selector returns it, and the clause $\mathcal{C}_B(\rho_B(b)) = [B(\rho_B(b), 0);\ y.\, B(\rho_B(b), y)]$ may contain it. Both rules have two premises and no term argument. Against NAME, the rule EXPORT drops the equality premise and the term t; the rule OPEN drops in addition the formula F.

Definition 6.26 (The two calculi of the inferred witness)**.** $\mathcal{O}$ has the rules REFL, USE, INTRO and OPEN; $\mathcal{X}$ has the rules REFL, USE, INTRO and EXPORT. Their forms $\mathcal{O}^{\mathcal{R}}$ and $\mathcal{X}^{\mathcal{R}}$ with the reflexivity clause have three rule families each.

Proposition 6.27 (Soundness)**.** *OPEN and EXPORT are sound for the ordinary reading in* T_{eq}*, also when* $\rho_B(b)$ *contains* b*. Moreover every instance of EXPORT is derivable in* $\mathcal{W}$ *with the same end sequent.*

Proof. Eliminate the existential quantifier of the major premise at b; the local assumption is the reading of $\mathcal{C}_B(b)$, and Theorem 6.25 gives $b = \rho_B(b)$. For EXPORT, equality substitution gives $F[\rho_B(b)/b]$, which does not contain b. For OPEN, equality substitution in the template $\mathcal{C}_B(v)^\circ$ gives the reading of $\mathcal{C}_B(\rho_B(b))$, the branch gives A, and A does not contain b. No occurs check on the selected witness is needed: the rules do not assert that the equation $b = r(b)$ has a solution independently of the major premise. The last statement: keep the two premises of EXPORT, insert the derivation of Theorem 6.25, and apply NAME with $t \equiv \rho_B(b)$. □

6.3.2 The export form

Lemma 6.28 (Ordinary elimination in $\mathcal{X}$)**.** *ELIM is a derived rule of* $\mathcal{X}$*, by one EXPORT.*

Proof. Apply EXPORT with $F \equiv A$. Since A does not contain b, $A[\rho_B(b)/b] \equiv A$ whichever term the selector returns. The argument does not need the selector to fall back. □

Hence Theorems 6.7, 6.8 and 6.12 hold in $\mathcal{X}$.

Theorem 6.29 (Transport in $\mathcal{X}$)**.** *UTR is derived in* $\mathcal{X}$ *for every term template* $h(w)$*.*

Proof. Let $\Gamma \vdash s = t$ and $\Gamma \vdash h(s) = r$, and let Π be the packet with the rows, exchanged against Theorem 6.17,

$$E_1(x) = (x = t), \qquad E_2(x) = (h(x) = r).$$

Theorem 6.12 gives $\Gamma \vdash D_{x,y}\Pi$; the equation $s = t$ is used here, in the proof of the major premise, and not as a premise of the discharge. Apply EXPORT with a fresh b. By (6.9), $\Pi(b, 0) \equiv H$ and $\Pi(b, 1) \equiv (b = t)$, so the test succeeds and $\rho_\Pi(b) \equiv t$. In the branch, USE of $\mathcal{C}_\Pi(b)$ at 2 with the premise H gives $F \equiv (h(b) = r)$. The exported formula is $(h(b) = r)[t/b] \equiv (h(t) = r)$ by (6.1), and it does not contain b. □

The row that the rule must read is at the fixed position 1; the row that the branch supplies is read by USE at 2.

Theorem 6.30 ($\mathcal{X}$ and $\mathcal{W}$)**.** $\Gamma \vdash_{\mathcal{X}} F$ *if and only if* $\Gamma \vdash_{\mathcal{W}} F$*, for every context* Γ *and formula* F*, by effective transformations that leave the end sequent unchanged.*

Proof. From $\mathcal{X}$ to $\mathcal{W}$ use the last statement of Theorem 6.27 at every EXPORT node. From $\mathcal{W}$ to $\mathcal{X}$: by Theorems 6.9, 6.10, 6.28 and 6.29, $\mathcal{X}$ derives transport in every formula template. At a NAME node the translated branches give $b = t$ and F under the opened clause; transport in the template $F[w/b]$ gives $F[t/b]$; by the side condition of NAME it does not contain b, and Theorem 6.28 discharges the witness. This covers a term t that contains b. The order of the constructions is: ordinary elimination, numerical cases, the packet, transport in a term, transport in a formula; none uses a later one, and no substitution into a derivation is used. □

6.3.3 The open form

In $\mathcal{O}$ ordinary elimination is not immediate: the branch of OPEN assumes $\mathcal{C}_B(\rho_B(b))$ where ELIM assumes $\mathcal{C}_B(b)$, and it does not help that the goal is free of b. The assumption is changed by substituting in the branch derivation. The order of the argument avoids a circle: the instances of OPEN on which the selector necessarily falls back give numerical cases and the unrestricted case introduction; these give substitution; substitution gives ordinary elimination.

Lemma 6.31 (Fallback openings)**.** *If the witness coordinate does not occur in B, or if B has root D, then* OPEN *for B is literally* ELIM *for B. Consequently Theorems 6.7, 6.8 and 6.12 hold in $\mathcal{O}$.*

Proof. Theorem 6.24(b). The three lemmas require ELIM only for these two kinds of matrices, and weakening and cut hold by Theorem 6.6. □

Theorem 6.32 (Substitution in $\mathcal{O}$)**.** *If $\Gamma \vdash_{\mathcal{O}} A$ and σ is a simultaneous substitution of terms for parameters, then $\Gamma\sigma \vdash_{\mathcal{O}} A\sigma$. The transformation of derivations is effective. It does not preserve the rule instance at each node.*

Proof. By induction on the given derivation, with the statement quantified over all substitutions. The induction hypothesis is applied to proper subderivations of the given derivation only, never to a derivation that the construction has produced. A substitution may send a parameter to a term that contains that parameter; images are inserted once.

REFL is immediate. For USE of an occurrence $[A'; y.\, B]$ at t, the context $\Gamma\sigma$ contains $[A'\sigma; y.\, B\sigma]$, the induction hypothesis gives $A'\sigma$, and USE at $t\sigma$ gives $(B\sigma)(t\sigma) \equiv B(t)\sigma$ by (6.1).

For INTRO at the witness t with the matrix B, rename z away from the images of σ and let $B' \equiv B\sigma$, $t' \equiv t\sigma$. The induction hypothesis gives $\Gamma\sigma, B'(t', \eta) \vdash B'(t', Sz)$ with the old η. A progress instance remains one: the root is still an equation and the literal 0 is still 0. A case instance remains one unless B is a literal equation and $t \not\equiv 0 \equiv t'$. In that event the substituted premise is the premise of the unrestricted case introduction of Theorem 6.8, which holds in $\mathcal{O}$ by Theorem 6.31 and was proved before the present theorem.

For OPEN with the major premise $D_{x,y}B$ and the parameter b, rename b away from the images of σ, let $B' \equiv B\sigma$ and $\rho' \equiv \rho_{B'}(b)$, and translate the major premise by the induction hypothesis. There are two cases, according to the test for the *old* matrix.

If the old test succeeds with $\rho_B(b) \equiv r$, then by Theorem 6.24(c) the new test succeeds and $\rho' \equiv r\sigma$. Apply the induction hypothesis to the branch with σ, which fixes b. Its clause becomes

$$\mathcal{C}_B(r)\sigma \equiv [B'(r\sigma, 0);\, y.\, B'(r\sigma, y)] \equiv \mathcal{C}_{B'}(\rho')$$

by (6.1); this does not need r to be free of b.

If the old selector falls back, the old branch has the clause $\mathcal{C}_B(b)$. The new test may fail or succeed. In both events apply the induction hypothesis to the old branch with the substitution that agrees with σ on the other parameters and sends b to ρ'. The clause becomes $\mathcal{C}_{B'}(\rho')$, and the context and the goal become $\Gamma\sigma$ and $A\sigma$, because Γ and A do not contain b and no image of σ does.

In either case OPEN applies to the translated premises and proves $A\sigma$. □

The second case is necessary: by Theorem 6.24(d) the selector is not an invariant test, and an argument that treats it as one leaves a gap. The checkers of `inferred_witness` and `deterministic_opening` are tested on a derivation whose opening falls back and for which a substitution of a passive parameter makes the test succeed: the old branch is then rejected, and the old branch instantiated at the newly selected witness is accepted.

Corollary 6.33 (Ordinary elimination in $\mathcal{O}$)**.** *ELIM is admissible in $\mathcal{O}$ for every matrix.*

Proof. Given $\Gamma, \mathcal{C}_B(b) \vdash A$ with b fresh for Γ and A, apply Theorem 6.32 with the substitution $b \mapsto \rho_B(b)$ that fixes all other parameters. The result is $\Gamma, \mathcal{C}_B(\rho_B(b)) \vdash A$, the branch premise of OPEN. □

Theorem 6.34 (Transport in $\mathcal{O}$)**.** *UTR is derived in $\mathcal{O}$ for every term template $h(w)$.*

Proof. Take the packet of Theorem 6.29, with the rows $x = t$ and $h(x) = r$; by Theorems 6.12 and 6.31, $\Gamma \vdash D_{x,y}\Pi$. Apply OPEN with a fresh b. The test succeeds with $\rho_\Pi(b) \equiv t$, so the branch has the clause $\mathcal{C}_\Pi(t) = [H;\, y.\, \Pi(t, y)]$. USE at 2 with the premise H gives $\Pi(t, 2) \equiv (h(t) = r)$, the unchanged outer goal, which does not contain b. There is no equality premise and no substitution in an exported formula; the derivation does not use Theorem 6.32. □

Theorem 6.35 ($\mathcal{O}$ and the reference calculus)**.** *$\Gamma \vdash_{\mathcal{O}} A$ if and only if $\Gamma \vdash_{\mathcal{T}} A$, for every context Γ and formula A, by effective transformations that leave the end sequent unchanged.*

Proof. From $\mathcal{T}$ to $\mathcal{O}$: ELIM is replaced by Theorem 6.33, and TR by Theorem 6.34 followed by symmetry (Theorem 6.9).

From $\mathcal{O}$ to $\mathcal{T}$: $\mathcal{T}$ derives transport in every formula template. At an OPEN node translate the major premise and open it by ELIM at b, keeping the clause $\mathcal{C}_B(b)$. Write $\rho \equiv \rho_B(b)$; Theorem 6.25 gives $b = \rho$. Derive $q_y B(\rho, y)$ by the first closure rule of Theorem 6.7: from its assumption $B(\rho, 0)$ transport along $\rho = b$ in the template $B(w, 0)$ to $B(b, 0)$, use $\mathcal{C}_B(b)$ at Sz, and transport along $b = \rho$ in the template $B(w, Sz)$ to $B(\rho, Sz)$. ELIM for $q_y B(\rho, y)$, whose witness coordinate is unused, opens exactly the clause $[B(\rho, 0);\, y.\, B(\rho, y)] = \mathcal{C}_B(\rho)$. The translated branch, weakened by the clause $\mathcal{C}_B(b)$ that is still present, proves A. Both eliminations close, since A contains neither the unused witness nor b. The construction is valid when ρ contains b: throughout the inner part b is an available parameter, and only the outer elimination discharges it. □

6.3.4 Summary and comparison of the four packages

Theorem 6.36 (The inferred witness)**.** *The calculi $\mathcal{T}$, $\mathcal{W}$, $\mathcal{X}$ and $\mathcal{O}$ derive the same sequents. For every context Δ of clauses with at most one antecedent and every formula F the following are equivalent: $L(\Delta) \vdash_{\mathcal{T}} F$; $\mathcal{R}, \Delta \vdash_{\mathcal{W}^{\mathcal{R}}} F$; $\mathcal{R}, \Delta \vdash_{\mathcal{X}^{\mathcal{R}}} F$; $\mathcal{R}, \Delta \vdash_{\mathcal{O}^{\mathcal{R}}} F$. Substitution of terms for parameters is admissible in each of them.* N *derives $\Gamma \vdash A$ if and only if $\mathcal{O}^{\mathcal{R}}$, equivalently $\mathcal{X}^{\mathcal{R}}$, derives $A^\flat$ from $\mathcal{R}$ and the unconditional assumption clauses of $\Gamma^\flat$; and the ordinary readings of $\mathcal{O}^{\mathcal{R}}$ and $\mathcal{X}^{\mathcal{R}}$ are bi-interpretable with* iPRA *as in Theorem 6.20.*

Proof. Theorems 6.19, 6.30 and 6.35 give the first statement, Theorem 6.13 the second. Theorem 6.32 gives substitution in $\mathcal{O}$, and the other calculi inherit it through the equivalences, which leave end sequents unchanged. The rest is Theorem 6.5 and Theorem 1.119. □

In Lean the rule on the kernel, with the selected witness read off the normalized matrix in place of DE, is `FOPrf` with `foprf_iff_nprf` (`NativeInferredWitness`); its completeness proof translates a kernel derivation under an arbitrary substitution and at DE sends the fresh variable to the selected witness whatever it is, as in the second case of the proof of Theorem 6.32; no distinction of cases on the test is needed there. The clause form $\mathcal{O}^{\mathcal{R}}$ is `OCPrf` with `ocprf_iff_nprf` (`NativeOpenClauses`), and $\mathcal{X}^{\mathcal{R}}$ is `XCPrf` with `xcprf_iff_nprf` (`NativeExportClauses`).

Remark 6.37 (How the four packages differ). *Rows inspected.* There is no difference: all four inspect the rows $B(b,0)$ and $B(b,1)$ of the normalized matrix, require the first to be literally $0 = 0$ and the second to be literally an equation whose left side is the fresh variable, and take the right side of the second as the witness.

The reflexivity clause. There is no difference either: all four are stated in the convention with the fixed unconditional clause $[\ ;\ y.\, y = y]$ and clauses with at most one antecedent, and have three rule families.

Use of the witness. `inferred_witness` and `deterministic_opening` state OPEN: the clause is opened at the selected witness and the branch proves the unchanged goal. Their proofs are those of Theorems 6.31 to 6.35. `first_instance` and `clause_discharge` state EXPORT: the clause is opened at the fresh variable and the witness is substituted in the exported formula. Their proofs are those of Theorems 6.28 to 6.30, and they need no substitution theorem. `inferred_witness` keeps the export form as a second write-up and as a control checker, and `deterministic_opening` keeps it as an intermediate variant with its own checker.

Reference calculus. `inferred_witness` proves the equivalence with $\mathcal{T}$ in the convention with the reflexivity clause; the other three prove it with $\mathcal{W}^{\mathcal{R}}$. In the translation of OPEN back to the reference calculus, `inferred_witness` transports the base of the closure in both directions, as in the proof of Theorem 6.35; `deterministic_opening` distinguishes the two cases of the test and, when it succeeds, uses that the base is H and ignores the assumption of the closure.

Additional material. `first_instance` compares its calculus with the raw clause calculus directly instead of through $\mathcal{W}$, by the argument cited in Theorem 6.5, and contributes Theorem 6.1. `clause_discharge` delivers, besides the checker of $\mathcal{X}^{\mathcal{R}}$, a smaller checker of the unchanged $\mathcal{W}^{\mathcal{R}}$. `deterministic_opening` delivers a reordering of an inherited compact checker that is independent of the new rule (Section 6.9).

Names. The rule is called O* in `inferred_witness`, O in `deterministic_opening`, Name in `first_instance` and Row in `clause_discharge`; the Lean development calls EXPORT Discharge.

Remark 6.38 (The conditions of the test). Three conditions cannot be removed. `deterministic_opening` exhibits, for each, a certificate that the correspondingly weakened checker accepts: without the requirement that the left side of the row at 1 is the fresh witness, and without the requirement that the row at 0 is $0 = 0$, the checker accepts $1 = 0$; without the requirement that the outer goal is in the outer scope, it accepts a formula in which the fresh parameter escapes. In `clause_discharge` the source of the replacement must be the witness opened by the same inference: an equation whose left side is an older parameter, a constant or a compound term triggers no replacement. With an older parameter as source, the argument of Theorem 6.28 fails, since the substitution then acts on a goal that mentions that parameter; the generalization is not part of any of the calculi (Section 6.11).

6.4 Substitutive discharge on the canonical recursor syntax

This section draws on `substitutive_discharge`. Its term syntax, and the passage from that syntax to iterator terms, are those of `canonical_recursors`.

The calculi of Sections 6.2 and 6.3 depend on full normalization: the rows of a packet must be literal before any equality rule is available. If the discharge rule is given a formula template, so that the discharged equation may have an arbitrary source, no packet is needed, the only normalization is a structural erasure, and reflexivity is derived from one computation schema.

6.4.1 Syntax

In this section the convention on normal forms is not in force. The terms are those of the seeded recursor $R(n, a; u.h)$ with one bound variable of Theorem 9.1, with the defined successor $Sa := R(0, a; u.0)$ and the computation equations (9.1) and (9.2). The zero computation is made part of the syntax: terms and formulas of the calculus are zero-body canonical, and substitution is the normalized substitution $T\langle\sigma\rangle$, both in the sense of Theorem 9.2. The canonicalization ν does not unfold a successor iteration, does not execute a body and does not test whether a term denotes zero; it preserves the root constructor (9.5), so that $\nu(t) \equiv 0$ if and only if $t \equiv 0$; and it is coherent with substitution, which is therefore associative, commutes with renaming and introduces no free variables beyond those of its arguments (Theorem 9.3). The syntax is that of the calculi $\mathcal{C}_\nu$ and $\mathcal{K}_6$ of Theorems 9.61 and 9.67.

Formulas are equations and $D_{x,y}B$. A context is a finite list of clauses $[A;\, y.\, B(y)]$ with one antecedent, and an assumption A is the clause $[0 = 0;\, v.\, A]$. The truth of $0 = 0$ is derived, not assumed, and no clause is fixed in advance. The contradiction is $0 = S0$.

6.4.2 Four rule families

Definition 6.39 (The calculus of substitutive discharge)**.** $\mathcal{K}$ has the following four rules. All displayed expressions are canonical.

$$\frac{}{\Gamma \vdash R(Sn, a;\, u.\, h) = h[R(n, a;\, u.\, h)/u]}\ \textsc{calc} \tag{6.13}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \textsc{use}, \qquad [A;\, y.\, B(y)] \text{ an occurrence in } \Gamma$$

$$\frac{\Gamma,\ B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}\ \textsc{intro}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise} \end{cases}$$

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma,\ \mathcal{C}_B(b) \vdash s = t \qquad \Gamma,\ \mathcal{C}_B(b) \vdash F(s)}{\Gamma \vdash F(t)}\ \textsc{sdis} \tag{6.14}$$

CALC is the schema (R_+): the predecessor iterate on the right is canonicalized, and at $n \equiv 0$ it is Sa. In INTRO, z is fresh for Γ, t and $D_{x,y}B$, and the test is literal. In SDIS, b is fresh for Γ and $D_{x,y}B$, the exported formula $F(t)$ does not contain b, and $F(v)$ is an arbitrary formula template whose distinguished variable v is different from b; a constant template is allowed, and s, t and the template may contain b. The calculus $\mathcal{K}_{\text{sep}}$ has CALC, USE, INTRO, ELIM and the transport rule

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \textsc{trf} \tag{6.15}$$

for every formula template F.

$\mathcal{K}$ has no hypothesis rule, no reflexivity axiom, no separate equality rule, no separate elimination of $D_{x,y}$, no coverage or separation axiom and no structural rule. The one computation schema is CALC; the progress mode of INTRO carries the induction. Weakening, exchange, contraction, injective renaming and cut of an assumption are admissible by induction on derivations, as in Theorems 6.6 and 9.62; injective renaming preserves the literal test of INTRO. Substitution of a term for a parameter can turn a witness into the literal 0; it is admissible through Theorem 6.45, not by an induction that preserves the rules.

6.4.3 The bootstrap of equality

Let, for a term t and variables u, x, y that do not occur in it,

$$W_t := R(1, 0;\, u.\, t), \qquad J := (W_0 = 0), \qquad \Theta := D_{x,y}J.$$

CALC proves $W_t = t$: the body is constant and ignores the predecessor iterate.

Lemma 6.40 (A closed paired theorem)**.** Θ *is derived by* CALC *and* INTRO *alone.*

Proof. Apply INTRO at the witness 1. The matrix does not depend on its bound variables, so the goal of the premise is J, which CALC proves without the assumption. Neither the use of an assumption nor reflexivity is involved. □

Lemma 6.41 (Transport in a formula)**.** TRF *is a derived rule of* $\mathcal{K}$*.*

Proof. Use Θ as the major premise of SDIS, and weaken the two given derivations into the opened context; they do not mention the fresh witness, and $F(t)$ does not contain it. No law of equality is used. □

Lemma 6.42 (Reflexivity and assumptions)**.** $\Gamma \vdash t = t$ *for every term* t*, and an assumption can be used:* $\Gamma, A \vdash A$*.*

Proof. CALC gives $W_t = t$. Use it as both premises of TRF with the template $F(v) = (v = t)$: the instance at the source is $W_t = t$ and the instance at the target is $t = t$. For an assumption A, take $t \equiv 0$ and apply USE to $[0 = 0;\, v.\, A]$. □

Lemma 6.43 (Equality and ordinary elimination)**.** *Symmetry, transitivity, the congruence* $s = t \vdash h(s) = h(t)$ *and* ELIM *are derived rules of* $\mathcal{K}$*.*

Proof. From $s = t$, transport the reflexive $s = s$ in the template $(v = s)$ to $t = s$. From $s = t$ and $t = r$, transport $t = r$ along $t = s$ in the template $(v = r)$ to $s = r$. From $s = t$, transport the reflexive $h(s) = h(s)$ in the template $(h(s) = h(v))$ to $h(s) = h(t)$. For ELIM, let the opened branch prove a formula A without b. Apply SDIS with the constant template $F(v) = A$ and with the equation J, proved by CALC, as the equality premise; the third premise is the given branch, and the exported formula is A. No packet and no normalization of a selector is needed. □

Theorem 6.44 ($\mathcal{K}$ and the calculus with separate rules)**.** *For every context* Γ *of canonical clauses and every formula* A*,* $\Gamma \vdash_{\mathcal{K}} A$ *if and only if* $\Gamma \vdash_{\mathcal{K}_{\mathrm{sep}}} A$*. The translations are effective and preserve the end sequent, including its parameters.*

Proof. An instance of SDIS is TRF inside the opened context followed by ELIM, whose side condition is that of SDIS. Conversely TRF is Theorem 6.41 and ELIM is Theorem 6.43. The other rules are shared. Induct on derivations, renaming eigenparameters when the fixed auxiliary derivations are inserted. □

6.4.4 The canonical basis and the bridge

Proposition 6.45 (Recovered rules)**.** *The following are derived in* $\mathcal{K}$*, with* $q_y B := D_{w,y} B$ *for an unused* w*.*

(a) *Case introduction for every matrix and witness: from* $\Gamma, B(t,0) \vdash B(t, Sz)$ *infer* $\Gamma \vdash D_{x,y} B$*.*

(b) *From* $\Gamma, B(0) \vdash B(Sz)$ *infer* $\Gamma \vdash q_y B$*; from* $\Gamma \vdash q_y B$ *and* $\Gamma \vdash B(0)$ *infer* $\Gamma \vdash B(t)$*; and, for a literal equation* B*, from* $\Gamma, B(z) \vdash B(Sz)$ *infer* $\Gamma \vdash q_y B$*. Hence induction for literal equations under an arbitrary context and numerical cases for arbitrary formulas.*

(c) *A context that contains the clause* $[B(0);\, y.\, B(y)]$ *proves* $q_y B$*.*

Proof. (a) Only a literal equation at the witness 0 is missing from INTRO. The term $c := W_0$ is canonical, is not the literal 0, and satisfies $c = 0$ by CALC. Given the premise at the witness 0, start from the assumption $B(c, 0)$, transport it to $B(0, 0)$, insert the given derivation by cut, and transport its conclusion $B(0, Sz)$ back to $B(c, Sz)$. INTRO at c is in case mode and concludes $D_{x,y}B$. No induction is used.

(b) The first rule is (a) for the unused coordinate. For the second, ELIM opens $[B(0);\ y.\, B(y)]$, and USE with the proof of $B(0)$ gives $B(t)$. The third is INTRO at the literal witness 0 of the unused coordinate, in progress mode.

(c) Use (a) at the unused witness 1: under its assumption $B(0)$, use the clause at Sz. □

The term W_0 is the term $c = \mathrm{wr}(0)$ of Theorem 9.68, and (a) is the repair of Theorem 9.69, with TRF in place of the equality of $\mathcal{K}_6$; (b) and (c) are the clause forms of Theorem 9.63 and of Theorem 5.9(i)–(iv).

The Eq/Q/D basis of `canonical_recursors` is the six-rule calculus $\mathcal{K}_6$ of Theorem 9.67. By (c) a derivation of $\mathcal{K}_6$ whose elimination of $D_{x,y}$ keeps a formula $Q_y B(b, y)$ is translated by deriving $q_y B(b, y)$ from the opened clause and cutting it into the translated branch; the two representations of a context are related by derivations and are not identified by convention. Reflexivity, symmetry and TRF give the crossed equation transport (X) of $\mathcal{K}_6$, including its restriction on the source, and the crossed elimination (QX) is the second rule of (b) followed by symmetry when the conclusion is an equation. Conversely $\mathcal{K}_6$ derives formula transport (Theorem 9.71) and the elimination of $D_{x,y}$, which validate SDIS, a clause $[A;\ y.\, B]$ being represented by its ordinary reading. So $\mathcal{K}$ and $\mathcal{K}_6$ have the same consequences on formulas, and by Theorems 9.66 and 9.72 the same as $\mathcal{C}_\nu$ and $\mathcal{R}_{\mathrm{raw}}$.

Theorem 6.46 (Bridge for $\mathcal{K}$)**.** *The ordinary reading of* $\mathcal{K}$ *and* iPRA *are bi-interpretable with whole domains, actual equality, both numerical composites with graph* $x = y$*, and both formula round trips. In particular they are equiconsistent.*

Proof. By the comparison above, $\mathcal{K}$ and $\mathcal{K}_6$ derive the same formulas from the same canonical contexts, so the statement is Theorem 9.74, whose bi-interpretation is that of Theorem 9.25 for T^1_R. The direct interpretations, in the form described in Theorem 9.26, read as follows on the rules of $\mathcal{K}$.

From $\mathcal{K}$ to iPRA: interpret each fixed term $R(n, a; u.h)$ by primitive recursion with the seed Sa and the displayed step. The erasure of a body at the counter 0 preserves the value. CALC is a defining equation, USE is implication and universal elimination, INTRO is coverage in case mode and induction for an equation under its assumed base in progress mode, and SDIS is equality substitution inside the scope of an existential elimination.

From iPRA to $\mathcal{K}$: the predecessor p of (9.17) (Theorem 8.19, written P there) is constructed before any numerical selector, cancellation of successor or reflection of equality is available, and the iterator of Section 1.1 is the initialized iteration $\widehat{I}(n, a; u.h) = p(R(n, a; v.S(h[p(v)/u])))$ of Theorem 8.21 and Theorem 9.24: at the counter 0 its value is $p(Sa) = a$, and at a successor CALC exposes an outer successor that p removes. By Theorems 6.42, 6.43 and 6.45 the rules needed by the logical macros and by the arithmetic of Theorems 1.65, 1.86 and 1.102 are available in $\mathcal{K}$, with $\mathrm{sel}(v, a, b)$ read as $\widehat{I}(v, a; u.b)$. This gives the defining equations and the quantifier-free induction of iPRA.

Both numerical composites are the identity by the argument of Theorem 9.25(2): recurrence uniqueness (Theorem 9.21), applied to each fixed description in the order of its dependencies, with pointwise equality of bodies lifted to the iteration by body extensionality (Theorem 9.22) and not assumed as a congruence; the interpretation of successor returns Sa. The logical round trips of Theorem 1.119 identify the reading of $D_{x,y}$ with its returned macro, and induction on formulas extends the numerical identities to both formula composites. The contradiction $0 = S0$ corresponds to falsum in both directions. □

Remark 6.47 (Checking, the raw companion, Lean). The syntax recursion and the proof recursion of $\mathcal{K}$ are structural, and the only canonicalization erases a body at a literal counter 0. Under a standard coding of finite trees the certificate relation is therefore primitive recursive, with no fuel field, in contrast to Theorem 6.2.

The package contains a companion on raw iterator terms without any normalization: the two computation equations of Section 1.1, the use of a clause, INTRO, and the same discharge rule. An assumption F is $[H_0; v.\, F]$ with $H_0 := (I(0,0;\, u.\, 0) = 0)$, one instance of the computation schema; $D_{x,y}H_0$ is introduced at the witness $S0$ without its assumption; transport is the discharge of this formula; reflexivity is the transport of $I(0,t;\, u.\, 0) = t$ along itself in the template $(v = t)$; and ELIM is the discharge with a constant template and the equation H_0. The proofs are those of Theorems 6.40 to 6.43.

In Lean the discharge rule on the kernel, in place of TR and DE, gives the seven-rule calculus `SDPrf`, with `sdprf_iff_nprf` (`NativeSubstitutiveDischarge`): the closed theorem is $D_{x,y}(I(0,0;\, u.\, u) = 0)$, proved by cases from a computation; a discharge of it that does not use the opened context is transport in an arbitrary formula (`SDPrf.tr`), and a constant template gives DE (`SDPrf.dE`). The four families on the canonical recursor syntax are not formalized.

6.5 Restricted substitution

This section draws on `variable_substitution` (Sections 6.5.1 and 6.5.2), `parameter_substitution` (Section 6.5.3) and `substitutive_discharge` (Section 6.5.4).

6.5.1 Substitution for a variable

The convention on normal forms of Section 6.1 is in force. The equality rule substitutes for a parameter and does nothing else:

$$\frac{\Gamma \vdash x = t \qquad \Gamma \vdash u = v}{\Gamma \vdash v = u[t/x]}\ \text{VSUB}, \qquad x \text{ a parameter.} \tag{6.16}$$

The rule substitutes simultaneously for every occurrence of x in u. It does not remove x from the parameters and does not substitute in Γ or in v. The term t may contain x. No template, no selected occurrence, no compound source and no matching of a pattern is supplied. A variable bound by an iterator is not a source.

Definition 6.48. $\mathcal{V}$ is the calculus with the five rules REFL, VSUB, USE, ELIM and INTRO, in the convention with REFL.

Theorems 6.6, 6.7 and 6.12 apply to $\mathcal{V}$, in which ELIM is primitive. In particular the numerical cases are derived with no equality rule, and neither the case introduction at the witness 0 nor an envelope is needed for them.

Theorem 6.49 (Transport in $\mathcal{V}$)**.** *For every term template $h(w)$ the rule TR is derived in $\mathcal{V}$. The derivation contains exactly one instance of VSUB.*

Proof. Let $\Gamma \vdash s = t$ and $\Gamma \vdash h(s) = r$, and take the packet Π of Theorem 6.17, with the rows $h(x) = r$ and $x = t$. By Theorem 6.12, $\Gamma \vdash D_{x,y}\Pi$. Open it by ELIM at a fresh b; in the branch the same lemma gives $b = t$ and $h(b) = r$. One VSUB with the major premise $b = t$ and the minor premise $h(b) = r$ yields

$$r = (h(b))[t/b] \equiv h(t)$$

by (6.1). The conclusion does not contain b, and ELIM closes. □

Everything before the one VSUB uses the logical rules, reflexivity and literal computation only: no symmetry, transitivity or congruence, no substitution into a derivation, no reflection of equality and no induction. Occurrences of s in the passive part of h are not touched, because b is fresh for h; normalization may duplicate or erase the distinguished occurrences, and (6.1) accounts for this without counting them.

Theorem 6.50 ($\mathcal{V}$ and the reference calculus)**.** $\Gamma \vdash_{\mathcal{V}} A$ *if and only if* $\Gamma \vdash_{\mathcal{T}} A$*, for every context* Γ *and formula* A*, by effective transformations that are the identity on the end sequent.*

Proof. A VSUB node is one TR: with w fresh let $K(w) := u[w/x]$; then $K(x) \equiv u$ and $K(t) \equiv u[t/x]$, also when t contains x, and TR from $x = t$ and $K(x) = v$ concludes $v = K(t)$. Conversely a TR node is replaced by the derivation of Theorem 6.49 after its premises are translated. The fresh names of that derivation are chosen away from the context, the conclusion and the terms involved, so all conditions of ELIM and INTRO hold. □

Together with Theorem 6.5 and Theorem 1.119 this gives the comparison with N and the bridge to iPRA, as in Theorem 6.20.

Proposition 6.51 (The native form with six rules)**.** *On normal forms with the formulas* $s = t$*,* $Q_y A$*,* $D_{x,y} A$ *and contexts of formulas, the calculus with* HYP*,* REFL*,* VSUB*,* QE*,* DE *and the introduction* INTRO *of Theorem 6.3 derives the same sequents as the calculus with* TR *in place of* VSUB*, under the identity translation.*

Proof. VSUB is one TR as before. For the converse the numerical cases use no equality rule: introduce $D_{x,y}F(y)$, with x unused, at the witness 1, ignoring the assumption $F(0)$; the branch of DE has the hypothesis $Q_y F(y)$, and QE with the given proof of $F(0)$ gives $F(t)$. The packet is introduced and eliminated as in Theorem 6.49; the two uses of the clause $[H;\, y.\, \Pi(b, y)]$ become two applications of QE to the hypothesis $Q_y \Pi(b, y)$ with the reflexive premise H, and the final VSUB is unchanged. □

The calculus with TR in this proposition is the six-rule calculus of `full_normalization`. In Lean the kernel on full normal forms with VSUB in place of TR is `VSPrf`, with `vsprf_iff_nprf` (`NativeVariableSubstitution`) and the packet `VSPrf.packet`.

Remark 6.52 (What is reduced, and what the proof needs)*.* The source of the equality rule is a variable, and the action of the rule is the substitution that the binders already require. The freedom of an ordinary equality rule is recovered by introducing, using and eliminating a fresh name. Full normalization is what makes the argument short: the rows of the packet at 0, 1 and 2 are literal before any congruence is available. The proof uses the logical rules; no completeness statement is made for the equality rules in isolation.

6.5.2 The newest variable

In this subsection the parameters of a sequent form a finite ordered list, and the eigenparameter of ELIM or INTRO is put at the front of the list of the premise it belongs to. $\mathcal{V}_0$ is $\mathcal{V}$ with the source of VSUB restricted to the front parameter of the list at its node; with de Bruijn indices the source is the variable of index 0.

Weakening is not admissible in $\mathcal{V}_0$ by an induction that preserves the rule at each node: a parameter inserted in front turns the former front parameter into an older one. Theorem 6.6 is not available, and the completeness proof uses an invariant instead.

Definition 6.53 (Internally sourced derivations)**.** A derivation in $\mathcal{V}_0$ with the initial parameter list Ξ is *internally sourced* if at every VSUB node the source is the eigenparameter of an ELIM node of the derivation, the VSUB node lies in the branch of that node, and no other ELIM or INTRO node lies between the two.

So the source is never a member of Ξ, and it is the front parameter at its node.

Lemma 6.54 (Safe structural changes)**.** *Let an internally sourced derivation of $\Gamma \vdash A$ over Ξ be given, let ι be an injection of Ξ into a list Ξ', not necessarily monotone, and let Γ' be a context over Ξ' that contains every occurrence of $\Gamma\iota$. Then there is an internally sourced derivation of $\Gamma' \vdash A\iota$ over Ξ'.*

Proof. Rename the eigenparameters of the derivation away from Ξ' and Γ'. A node of the derivation has a sequent over a list $(e_k, \ldots, e_1, \Xi)$, where $e_1, \ldots, e_k$ are the eigenparameters of the ELIM and INTRO nodes on the path from the root, and a context Γ, Λ, where Λ consists of the clauses those nodes add. Replace it by the sequent over $(e_k, \ldots, e_1, \Xi')$ with the context $\Gamma', \Lambda\iota$ and the formula renamed by ι, which is extended by the identity on the e_i. The initial list is not reset when the transformation descends into a premise. Every instance of REFL, USE, ELIM and INTRO is carried to an instance, as in Theorem 6.6: normalization and substitution commute with injective renaming, the mode of INTRO is preserved, and the added clauses do not affect any side condition. At a VSUB node the source is e_k with $k \geq 1$, and it is still the front parameter of the new list and still bound by the same ELIM node; inserting, permuting or renaming members of the initial list changes neither. The result is internally sourced over Ξ'. □

The lemma does not give weakening for arbitrary derivations of $\mathcal{V}_0$.

Theorem 6.55 (The newest variable suffices)**.** *Every derivation in $\mathcal{T}$ over a parameter list Ξ has an internally sourced translation in $\mathcal{V}_0$ with the same end sequent and the same list. Consequently $\mathcal{V}_0$, $\mathcal{V}$ and $\mathcal{T}$ derive the same sequents, and weakening, substitution of terms for parameters and TR are admissible in $\mathcal{V}_0$.*

Proof. By induction on the derivation in $\mathcal{T}$. REFL, USE, ELIM and INTRO are copied. The translated premises are internally sourced over their own lists; a source that is bound by an ELIM node inside a premise is bound by a node inside the combined derivation, so the invariant is preserved.

At a TR node apply the construction of Theorem 6.49 to the two translated premises. The construction places copies of them under the eigenparameters and the assumption clauses of the INTRO of the packet and of the numerical-case rule; each copy is obtained by Theorem 6.54, which applies because the premises are internally sourced by the induction hypothesis. The remaining nodes of the construction are REFL, USE, ELIM, INTRO and the one VSUB. That node stands at the end of the branch of the ELIM which opens the packet at b; its premises $b = t$ and $h(b) = r$ are derived from the opened clause by REFL and USE, with no eigenparameter in between. So its source b is the front parameter and is bound by an ELIM node of the constructed derivation, with no node of ELIM or INTRO between them.

Conversely a restricted VSUB is an instance of VSUB and hence one TR (Theorem 6.50). So the three calculi agree. A derivation of $\mathcal{V}_0$ is weakened, or instantiated, or used as a premise of TR, by translating it to $\mathcal{T}$, operating there, and translating back. The completeness proof itself uses none of these operations on arbitrary derivations. □

The same invariant and translation apply to the six-rule form of Theorem 6.51, with QE for USE and DE for ELIM: the final VSUB stands under its own newest eigenparameter of DE. In Lean the kernel form with the newest variable is `V0Prf`, with `v0prf_iff_nprf` (`NativeNewestVariable`); the same proof applies, because the packet substitutes only for the witness it has just opened and no derivation is renamed. The clause form $\mathcal{V}_0$ is `VCPrf`, with `vcprf_iff_nprf` (`NativeVariableClauses`); there the premises needed under further variables are obtained from the induction hypothesis at a shifted renaming, which is the protected translation of Theorem 6.54.

6.5.3 Substitution for a parameter on raw terms

In this subsection terms are the raw terms of Section 1.1 and substitution is ordinary capture-avoiding substitution; $I(0, a;\, u.\, h)$ and a are different terms, and nothing is normalized. The rows of a packet are then not literal, and substitution for a variable has to be combined with the joining of two equations at the root. Formulas are equations and $D_{x,y}B$; contexts consist of clauses $[A;\, y.\, B(y)]$ with one antecedent; the assumption clause of F is $\mathrm{ord}(F) := [H;\, v.\, F]$.

For terms s, t, u define

$$\psi(s,t,u) := \begin{cases} u[t/s], & s \text{ is a parameter,} \\ t, & s \text{ is not a parameter and } u \equiv s, \\ u, & s \text{ is not a parameter and } u \not\equiv s. \end{cases}$$

The first case substitutes for a variable in one pass and is defined also when t contains s. For every s, $\psi(s,t,s) \equiv t$. For a source that is not a variable no proper subterm of u is inspected. Define also the partial one-step computation

$$\kappa(I(0,a;\, u.\, h)) := a, \qquad \kappa(I(Sn,a;\, u.\, h)) := h[I(n,a;\, u.\, h)/u],$$

undefined on all other terms; the iterator in the second result is left as it is.

Definition 6.56 (The calculus of parameter substitution)**.** $\mathcal{P}$ has the following five rule families.

$$\frac{}{\Gamma \vdash b = \kappa(b)}\ \textsc{compute}, \qquad \kappa(b) \text{ defined} \tag{6.17}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash u = r}{\Gamma \vdash r = \psi(s,t,u)}\ \textsc{param} \tag{6.18}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \textsc{use}, \qquad [A;\, y.\, B(y)] \text{ an occurrence in } \Gamma$$

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma,\ \mathcal{C}_B(b) \vdash F}{\Gamma \vdash F}\ \textsc{open}$$

$$\frac{\Gamma,\ B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \textsc{close}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise} \end{cases}$$

OPEN is ELIM and CLOSE is INTRO, on raw syntax and with the same freshness conditions; the tests of CLOSE are tests on raw syntax. In this subsection OPEN denotes this rule and not the rule of Eq. (6.11).

There is no reflexivity axiom, no hypothesis rule, no template, no constructor Q, no case rule and no induction schema. COMPUTE contains the two computation equations; counting it as one family is a grouping of the implementation and does not turn two equations into one.

Lemma 6.57 (Equality at the root)**.** *In $\mathcal{P}$ the following are derived under every context:*

$$s = t,\ s = r \vdash r = t; \qquad \vdash a = a; \qquad s = t \vdash t = s; \qquad s = t,\ t = r \vdash s = r.$$

Proof. The first, *joining at a common left side*, is PARAM with $u \equiv s$, since $\psi(s,t,s) \equiv t$. For reflexivity, COMPUTE proves $I(0,a;\, u.\, 0) = a$, and joining this equation with itself gives $a = a$. For symmetry join the reflexive $s = s$, as major premise, with $s = t$. For transitivity take the symmetric $t = s$ and join $t = r$, as major premise, with $t = s$. Only COMPUTE and joining at the root are used. □

Lemma 6.58 (Elementary operations)**.** *The use of an assumption, weakening, injective renaming and cut of an assumption are admissible in $\mathcal{P}$.*

Proof. Reflexivity proves H, and USE of $[H;\, v.\, F]$ proves F. Weakening and injective renaming are inductions on derivations; an injective renaming preserves being a parameter, literal identity of terms, the root of a formula and the literal 0, hence the case of ψ and the mode of CLOSE. For cut, replace every USE of the occurrence $[H;\, v.\, F]$ by the given derivation of F, weakened to the local context, after renaming eigenparameters. No substitution of a term for a parameter in a derivation is used. □

Lemma 6.59 (Fresh aliases)**.** *For a term a and a variable b fresh for Γ, F and a,*

$$\frac{\Gamma,\ b = a \vdash F}{\Gamma \vdash F}$$

is admissible in $\mathcal{P}$. Its derivation uses the two computation equations, joining at the root, the logical rules and cut; it uses no substitution in a proper subterm.

Proof. Let $B(x, y) := \big(\mathrm{sel}(y, a, x) = a\big)$, where $\mathrm{sel}(y, a, x) = I(y, a;\, u.\, x)$. CLOSE at the witness a proves $D_{x,y}B$: its goal $\mathrm{sel}(Sz, a, a) = a$ is the successor case of COMPUTE for the constant body a. The assumption is not used, so the derivation is valid in either mode, also when $a \equiv 0$.

Open at a fresh b. The clause is $[\mathrm{sel}(0, a, b) = a;\, y.\, \mathrm{sel}(y, a, b) = a]$. Its antecedent is the zero case of COMPUTE, and USE at 1 gives $\mathrm{sel}(1, a, b) = a$. The successor case of COMPUTE gives $\mathrm{sel}(1, a, b) = b$. Joining, with the former as major premise, gives $b = a$. Cut this into the given derivation, weakened by the clause; OPEN discharges b. □

A second derivation uses the case mode of CLOSE only: take the witness $I(0, a;\, v.\, 0)$, which is not the literal 0; the successor case of COMPUTE gives $\mathrm{sel}(Sz, a, I(0, a;\, v.\, 0)) = I(0, a;\, v.\, 0)$, the zero case gives $I(0, a;\, v.\, 0) = a$, and transitivity joins them. So the lemma does not depend on the progress mode. The alias is the device of the optional argument in `template_free`; the point here is that it needs neither substitution in a proper subterm nor transport in a template nor induction, and therefore cannot hide the principle it is used to derive.

Theorem 6.60 (Substitution for a parameter gives every template)**.** *For every term template $h(v)$, if $\Gamma \vdash_{\mathcal{P}} s = t$ then $\Gamma \vdash_{\mathcal{P}} h(s) = h(t)$. Repeated occurrences of v, occurrences under the binder of an iterator and the absence of v are all permitted.*

Proof. Work under the assumption $b = s$, with b fresh for Γ, h, s and t. Transitivity gives $b = t$. Reflexivity gives $h(b) = h(b)$. PARAM with the major premise $b = s$ and this minor premise gives $h(b) = h(s)$: the source is the parameter b, and substitution for b marks exactly the distinguished occurrences of the template. In the same way $b = t$ gives $h(b) = h(t)$. Joining $h(b) = h(t)$, as major premise, with $h(b) = h(s)$ gives $h(s) = h(t)$, which does not contain b, and Theorem 6.59 discharges the alias. □

The argument is not a pass that replaces every occurrence of the possibly compound term s in $h(s)$; such a pass would also replace occurrences of s in the passive part of the template. Only the parameter case of ψ acts on a proper subterm.

Corollary 6.61 (The equality interface of $\mathcal{P}$)**.** *$\mathcal{P}$ derives* TR*, $s = t,\ h(s) = r \vdash r = h(t)$, and transport in every equation template, $s = t,\ p(s) = q(s) \vdash p(t) = q(t)$. It derives the replacement rule of* `template_free`*: from $s = t$ and $u = v$ infer $v = u'$, where u' results from u by replacing, in one pass from the root, the occurrences of s by t.*

Proof. Join the congruence $h(s) = h(t)$, as major premise, with $h(s) = r$. Apply this to p and then to q, with symmetry, for the equation template. For the replacement rule, a pass from the root selects finitely many pairwise disjoint occurrences of s in u; replacing exactly these by a fresh variable gives a template h with $h(s) \equiv u$ and $h(t) \equiv u'$. The extraction of the template belongs to the producer of the proof and not to the checker. □

Lemma 6.62 (Every case introduction)**.** *For every matrix B and witness t, from $\Gamma, B(t,0) \vdash B(t,Sz)$ one derives $\Gamma \vdash D_{x,y}B(x,y)$ in $\mathcal{P}$.*

Proof. Only a literal equation E at the witness 0 is not already the case mode of CLOSE. Work under a fresh alias $b = 0$. From the assumption $E(b,0)$, transport in an equation template gives $E(0,0)$; cut in the given derivation to obtain $E(0,Sz)$; transport along the symmetric equation gives $E(b,Sz)$. CLOSE at the variable b is in case mode and concludes $D_{x,y}E$, which does not contain b; Theorem 6.59 discharges the alias. Only transport in equations is used. □

Lemma 6.63 (Closures, induction, and transport in formulas)**.** *With $q_yB := D_{w,y}B$ for an unused w, the following are derived in $\mathcal{P}$.*

(a) *From $\Gamma, B(0) \vdash B(Sz)$ infer $\Gamma \vdash q_yB$.*

(b) *From $\Gamma \vdash q_yB$ and $\Gamma \vdash B(0)$ infer $\Gamma \vdash B(t)$.*

(c) *A context that contains $[B(0);\, y.\, B(y)]$ proves q_yB.*

(d) *For a literal equation E, from $\Gamma, E(z) \vdash E(Sz)$ infer $\Gamma \vdash q_yE$; with (b), induction for literal equations under an arbitrary context.*

(e) *Transport in every formula template.*

Proof. (a) CLOSE at the unused witness 1 is in case mode. (b) OPEN for the unused witness adds $[B(0);\, y.\, B(y)]$, and USE gives $B(t)$. (c) In (a), under the assumption $B(0)$, use the clause at Sz. (d) CLOSE at the unused literal witness 0 is in progress mode. (e) By induction on the template, as in Theorem 6.10: the equation case is Theorem 6.61; for $D_{x,y}K(v,x,y)$ open the source at b and introduce the target at the same b in case mode, converting the base backwards and the successor instance forwards. □

Theorem 6.64 ($\mathcal{P}$ and the kernel)**.** *Let τ be the translation of native formulas that leaves terms unchanged and satisfies $\tau(s = t) = (s = t)$, $\tau(D_{x,y}B) = D_{x,y}\tau B$ and $\tau(Q_yB) = D_{w,y}\tau B$ with w fresh, and let j be the inclusion of the formulas of $\mathcal{P}$ into native formulas. For every context Γ of native formulas and every native formula A,*

$$\Gamma \vdash_{\mathsf{N}} A \quad \Longleftrightarrow \quad \mathrm{ord}(\tau\Gamma) \vdash_{\mathcal{P}} \tau A.$$

Moreover $\tau j F \equiv F$ for every formula F of $\mathcal{P}$, and N derives A from $j\tau A$ and conversely.

Proof. τ commutes with substitution and preserves the property of being a literal equation.

From N to $\mathcal{P}$, by induction on the derivation. HYP is the use of an assumption (Theorem 6.58). The two computation rules are the two cases of COMPUTE. TR is Theorem 6.61. QE is Theorem 6.63(b). DI is Theorem 6.62, and QI is Theorem 6.63(d). For DE, open the translated major premise at b; by Theorem 6.63(c) the opened clause proves $q_y\tau B(b,y)$, which is $\tau(Q_yB(b,y))$; weaken the translated branch and cut this formula into it; OPEN closes.

From $\mathcal{P}$ to N. Consider contexts each of whose occurrences is an assumption clause $\mathrm{ord}(G)$ or an opened clause $\mathcal{C}_B(b)$, with its origin recorded, and represent them by the native formulas jG and $Q_yjB(b,y)$ respectively. By induction on a derivation in $\mathcal{P}$ from such a context, N derives j of its conclusion from the representing formulas. A USE of an assumption clause

is HYP, and a USE of an opened clause is QE. COMPUTE is a computation rule. PARAM is an instance of TR: with the template $u[v/s]$, v fresh, if s is a parameter; with the identity template if s is not a parameter and $u \equiv s$; with the constant template u otherwise. OPEN is DE. A CLOSE in case mode is DI. A CLOSE in progress mode, at the literal witness 0 with the matrix E, is QI for $Q_y E(0, y)$, followed by QE at Sz under the assumption $E(0, 0)$ and by DI.

Round trips. τj is the identity. For $j\tau$ the essential equivalence in N is $Q_y B(y) \dashv\vdash D_{w,y} B(y)$ for an unused w: from left to right use QE under the base and DI at any witness; from right to left DE opens exactly $Q_y B(y)$. The equivalence lifts through both binders by the congruence of binders in N (Section 1.4), and induction on A gives the interderivability of A and $j\tau A$. With cut in N this turns the second translation into the implication from right to left of the displayed equivalence. □

The proof of completeness uses the parameter case of ψ and joining at the root, never the third case, which only reverses the minor premise.

Corollary 6.65 (Bridge for $\mathcal{P}$)**.** *The ordinary reading of $\mathcal{P}$ is bi-interpretable and equiconsistent with* iPRA*, with whole domains, actual equality, and both numerical composites with graph $x = y$.*

Proof. Compose Theorem 6.64 with Theorem 1.119. Terms are unchanged, so the numerical maps are identities, and $0 = S0$ is fixed by τ and j. □

Remark 6.66 (Substitution, checking, Lean)*.* Substitution of a term for a parameter does not preserve the rules of $\mathcal{P}$: it can turn the source of PARAM from a variable into a compound term and can turn a witness of CLOSE into the literal 0. Its admissibility follows from Theorem 6.64 and substitution in N (Section 1.4). The checker performs structural traversals, capture-avoiding substitutions, literal comparisons and one-step computations only, so its certificate relation is primitive recursive; proofs and substituted terms can still grow considerably, since the producer builds an alias around every transport in a template.

In Lean the kernel with TR replaced by PARAM is `PPrf`, with `pprf_iff_nprf` (`NativeParameter`); the alias is discharged through $D_{x,y}(\mathrm{sel}(y, s, x) = s)$, proved by cases with the witness s and eliminated at $y = 1$ (`PPrf.aliasE`), and the calculus is closed under injective renaming (`PPrf.ren`). The five families of $\mathcal{P}$, which combine the rule with clause contexts and the introduction with a mode, are not formalized together.

6.5.4 Unary congruence does not suffice

VSUB and PARAM are binary: they combine two equations. One may ask whether the equality rule can be the unary congruence. On raw and on zero-normal terms it cannot.

Proposition 6.67 (An obstruction)**.** *Let N^c be the calculus obtained from N by replacing TR with reflexivity, symmetry and the rule*

$$\frac{\Gamma \vdash s = t}{\Gamma \vdash h(s) = h(t)}$$

for every term template h, all other rules being kept. Then N^c does not derive $x = y,\ y = z \vdash x = z$. The same holds on zero-normal terms, where $I(0, a;\, u.\, h)$ is identified with a.

Proof. Let $V = \{0, m, 2\}$, ordered $0 < m < 2$, and call two elements *near* if they are equal or adjacent; so 0 and 2 are not near, and m is near to every element. Interpret 0 as 0, successor as the constant function with value 2, and

$$I(n, a;\, u.\, h) \ \text{ as } \ \begin{cases} a, & n = 0, \\ m, & n \neq 0, \end{cases}$$

the body being ignored. An equation $s = t$ is *true* under an assignment of elements of V to the variables if the values of s and t are near. This is a truth assignment to judgments and not a structure with actual equality. $Q_y B$ and $D_{x,y} B$ are given the Boolean readings of Section 1.2 over V. A sequent is valid if every assignment that makes its context true makes its conclusion true. The value of a term commutes with substitution, since bodies are ignored, and the value of a term is unchanged by zero-normalization.

The rules other than the introductions. HYP, reflexivity and symmetry are valid. The zero computation holds with equal values. In the successor computation the left side has the value m, which is near to every value. QE and DE are valid for the Boolean readings. For the congruence rule it suffices that the interpretation of the iterator preserves nearness in both arguments together: if both counters are 0 the outputs are the two seeds; if exactly one is 0 the outputs are a seed and m; otherwise both are m. Zero and successor are constants.

The shape of term functions. Every term denotes either the constant m, or a function that takes the value of a *payload*, which is 0, 2 or one variable, when all variables of a finite *gate* are 0, and the value m otherwise. This holds for variables, 0 and successors with an empty gate, and for $I(n, a; u.\, h)$ by cases on the counter: if n is constantly m or has the payload 2, the iterator is constantly m; if n has the payload 0 and the gate G, the iterator is a with G added to its gate; if n has a variable w as payload, the iterator is a with $G \cup \{w\}$ added to its gate. Consequently, as a function of one variable v with the others fixed, a term takes at $(0, m, 2)$ one of the triples

$$(0,0,0), \quad (m,m,m), \quad (2,2,2), \quad (0,m,2), \quad (0,m,m), \quad (2,m,m).$$

The key property. Every formula $B(v)$ satisfies, for every variable v and all values of the other variables,

$$B(0) \text{ and } B(2) \text{ imply } B(m). \tag{6.19}$$

For an equation: at $v = m$ each side has the value m or is constant in v. If one side has the value m there, the equation is true at m. If both are constant, its truth does not depend on v.

For $Q_y E$ with E an equation: the formula is false exactly if E is true at $y = 0$ and $y = m$ and false at $y = 2$. Falsity at 2 means that the two sides have the values 0 and 2 there, so neither side has y in its gate, all gate variables of both sides have the value 0, and each side is its payload. Inspecting the payloads, the formula is then false only if the payloads are y and 0, or y and a variable w whose value is 0. Hence $Q_y E$ is true, or false, or equivalent to the statement that some variable of a fixed finite set is different from 0. A formula of this last kind satisfies (6.19): if v belongs to the set it is true at $v = m$, and otherwise it does not depend on v. For $D_{x,y} E$, the reading is the existence of a value of x at which $Q_y E$ holds; if x is in the fixed finite set the formula is true, and otherwise it is the same statement as before. Finally, if B is true, false, or the statement that some variable of a finite set is different from 0, then $Q_y B$ is true: if y is not in the set the instances do not depend on y, and if it is, the instance at $y = 0$ implies all instances. So $D_{x,y} B$ is true as well, and so is every formula with more binders.

The introductions. For DI, take the value of the witness t for x. If $B(t, 0)$ is true, the premise gives $B(t, Sz)$, that is, B at $y = 2$, and (6.19) gives B at $y = m$. For QI, if $E(0)$ is true, the premise with z assigned 0 gives E at 2, and (6.19) gives E at m.

So every derivable sequent is valid. Under $x \mapsto 0$, $y \mapsto m$, $z \mapsto 2$ the equations $x = y$ and $y = z$ are true and $x = z$ is false. The contradiction $0 = S0$ is false as well. □

Full normalization of the successor computation is outside the obstruction: it identifies values that the interpretation only relates. The proposition excludes this one deletion in the neighborhood of the kernel; it does not exclude other presentations of arithmetic.

6.6 Specialization without a term

This section draws on `deterministic_specialization`. The calculus it restricts is the five-rule clause calculus of `uniform_step`.

6.6.1 The calculus

In this section terms are *zero-normal*: the normalization contracts $I(0, a;\, u.\, h)$ to a in all subterms and does not contract an iterator whose counter is a successor. Terms are the fixed points of this map, and every substitution is followed by it, including a substitution that makes a counter the literal 0; in particular $\mathrm{sel}(0, a, b) \equiv a$. Formulas are equations and $D_{x,y}F$, so that every formula is a chain of binders ending in one equation; its *rank* is the number of binders. A context is a finite list of clauses $[A;\, y.\, B(y)]$, written with the *newest* occurrence first, and the parameters of a sequent form a list whose last member is the newest. An assumption A is the clause $[0 = 0;\, v.\, A]$. The order is used only to choose the target of a specialization; clauses are reusable.

Definition 6.68 (Deterministic specialization)**.** The rules are

$$\frac{}{\Gamma \vdash t = t}\ \textsc{refl} \qquad \frac{\Gamma \vdash I(c, a;\, u.\, h) = t \qquad \Gamma \vdash I(Sc, a;\, u.\, h) = r}{\Gamma \vdash r = h[t/u]}\ \textsc{step} \tag{6.20}$$

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad [B(b, 0);\, y.\, B(b, y)],\ \Gamma \vdash F}{\Gamma \vdash F}\ \textsc{de}$$

$$\frac{[0 = 0;\, v.\, B(t, \eta)],\ \Gamma \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}\ \textsc{di}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise} \end{cases}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \textsc{use}, \qquad [A;\, y.\, B(y)] \text{ an occurrence in } \Gamma,\ t \text{ arbitrary}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(v_i)}\ \textsc{use}_*, \qquad [A;\, y.\, B(y)] = \Gamma_i, \quad v_i = \begin{cases} 1, & i = 0, \\ b_*, & i > 0. \end{cases} \tag{6.21}$$

In STEP the first iterator is zero-normal, so it is a when $c \equiv 0$; repeated expressions are literally identical. In DE the variable b is fresh for Γ, F and $D_{x,y}B$, and in DI the variable z is fresh for Γ, t and $D_{x,y}B$; each becomes the newest parameter of its premise, and the displayed clause becomes the newest clause. In $\textsc{use}_*$, Γ_0 is the newest clause and b_* is the newest parameter, or 0 if there is none. $\mathcal{U}$ is the calculus with REFL, STEP, USE, DE and DI; $\mathcal{U}_*$ has $\textsc{use}_*$ in place of USE.

So the newest clause is used at 1 and every older clause at the newest parameter. A certificate of $\textsc{use}_*$ consists of the index of the clause and the proof of its antecedent; it contains no term and no mode. The witness of DI is still supplied, and the completeness proof uses that facility to name the desired term.

$\mathcal{U}$ is the calculus of `uniform_step` with clause contexts; its comparison with the kernel belongs to that package, and its arithmetic interface on the kernel is `NSPrf`, with `nsprf_iff_nprf`. The facts about it that are used here are the following.

Lemma 6.69 (Arithmetic from STEP)**.** *Using REFL and STEP only, and opening no scope, both $\mathcal{U}$ and $\mathcal{U}_*$ derive: the pivot $s = t,\ s = r \vdash r = t$; symmetry and transitivity; the congruence $s = t \vdash h(s) = h(t)$; transport in every equation template; and the successor computation $I(Sc, a;\, u.\, h) = h[I(c, a;\, u.\, h)/u]$, in particular* $\mathrm{sel}(Sz, a, b) = b$.

Proof. Let $J_s := I(1, s;\, u.\, u)$. Since $I(0, s;\, u.\, u) \equiv s$, STEP with $c \equiv 0$ sends $s = t$ and $J_s = r$ to $r = t$. Given $p\colon s = t$ and $q\colon s = r$, STEP applied to q and the reflexive $J_s = J_s$ gives $J_s = r$, and STEP applied to p and this gives $r = t$. Symmetry is the pivot with the first premise $s = s$, and transitivity follows. For a template h let $K := I(1, s;\, u.\, h)$; STEP applied to the reflexive $s = s$ and $K = K$ gives $K = h(s)$, and STEP applied to $s = t$ and $K = h(s)$ gives $h(s) = h(t)$. An equation template $p(v) = q(v)$ is transported through $p(t) = p(s)$, $p(s) = q(s)$ and $q(s) = q(t)$. The successor computation is STEP applied to the reflexivities of $I(c, a;\, u.\, h)$ and $I(Sc, a;\, u.\, h)$. □

Conversely the two computation equations and transport give STEP: in the computation equation for $I(Sc, a;\, u.\, h)$ replace $I(c, a;\, u.\, h)$ by t in the body and the successor iterator by r. So STEP adds no arithmetic strength.

Lemma 6.70 (The unrestricted calculus)**.** *In $\mathcal{U}$ cut of an assumption is admissible, and the following are derived: case introduction for every matrix and witness; for a literal equation E and every witness t, the progress introduction from $\Gamma, E(t, z) \vdash E(t, Sz)$ to $\Gamma \vdash D_{x,y}E$; and transport in every formula template.*

Proof. Cut is an induction on the derivation: a use of the assumption clause is replaced by the given proof. The term $c := I(1, 0;\, u.\, u)$ is zero-normal, is not the literal 0, and STEP proves $c = 0$. Given a premise $\Gamma, E(0, 0) \vdash E(0, Sz)$ for a literal equation E, start from $E(c, 0)$, transport to $E(0, 0)$, insert the given derivation, and transport back to $E(c, Sz)$; DI at c is in case mode. For progress at t, introduce $D_{w,y}E(t, y)$ with an unused w at the witness 0; DE adds $[E(t, 0);\, y.\, E(t, y)]$; under the base $E(t, 0)$ use it at Sz and apply the case introduction at t. Transport through a binder is as in Theorem 6.10. A progress introduction for a compound matrix is never used; it would change the induction of the theory. □

6.6.2 Naming a term

For a term t let

$$\mathrm{Nm}_t := D_{x,y}\big(\mathrm{sel}(y, 0, x) = \mathrm{sel}(y, 0, t)\big), \qquad \mathcal{M}_{b,t} := [0 = 0;\, y.\, \mathrm{sel}(y, 0, b) = \mathrm{sel}(y, 0, t)].$$

Lemma 6.71 (Naming)**.** *In every context, $\mathcal{U}_*$ derives Nm_t. Opening it by DE at a fresh b adds the clause $\mathcal{M}_{b,t}$, and in the context of that premise, where $\mathcal{M}_{b,t}$ is the newest clause, $\mathcal{U}_*$ derives $b = t$.*

Proof. Apply DI at the witness t. The goal $\mathrm{sel}(Sz, 0, t) = \mathrm{sel}(Sz, 0, t)$ has literally equal sides and is proved by REFL, in either mode. The antecedent of the opened clause is $\mathrm{sel}(0, 0, b) = \mathrm{sel}(0, 0, t)$, that is, $0 = 0$. Since the clause is newest, USE_* applies it at 1, with REFL for its antecedent, and gives $\mathrm{sel}(1, 0, b) = \mathrm{sel}(1, 0, t)$; the computations $\mathrm{sel}(1, 0, b) = b$ and $\mathrm{sel}(1, 0, t) = t$ of Theorem 6.69 give $b = t$. □

This derivation of $b = t$ is valid immediately after the opening. It does not remain a derivation when the context is extended, because the clause then ceases to be newest. An extension of the scope changes b_*, and the insertion of a clause changes which clause is newest; so weakening is not an operation that preserves the rule at each node of a derivation of $\mathcal{U}_*$. The completeness proof replaces weakening by procedures that produce a proof again in every larger context.

6.6.3 Persistent proofs

A *situation* is a pair $(\Xi;\Gamma)$ of a parameter list and a context over it. An *extension* of $(\Xi;\Gamma)$ is a situation $(\Xi';\Gamma')$ together with an injective map ι of the members of Ξ to members of Ξ' and an injective map of the occurrences of Γ to occurrences of Γ' that sends an occurrence of a clause C to an occurrence of $C\iota$. Extensions compose, and neither map need preserve the order.

Definition 6.72 (Persistent proof)**.** A *persistent proof* of a formula A over $(\Xi;\Gamma)$ is a function that assigns to every extension of $(\Xi;\Gamma)$ a derivation in $\mathcal{U}_*$ of $\Gamma' \vdash A\iota$.

A persistent proof is a finite proof-producing procedure of the metatheory; it is not an additional rule.

Lemma 6.73 (Elementary persistence)**.** (a) *A persistent proof over a situation restricts to a persistent proof over each of its extensions.*

(b) *If* $[0 = 0;\, v.\, A]$ *occurs in* Γ*, then* A *has a persistent proof over* $(\Xi;\Gamma)$*.*

(c) *An equation* $t = t$ *has a persistent proof; if the two premises of a* STEP *have persistent proofs, so has its conclusion. Hence the conclusions of the derivations of Theorem 6.69 have persistent proofs when their premises have.*

Proof. (a) Compose extensions. (b) In every extension apply USE_* to the image of the occurrence, with REFL for the antecedent; the bound variable is unused, so either target gives $A\iota$. (c) Apply the rule in each extension; REFL and STEP do not depend on the context or on the order of the parameters. □

Lemma 6.74 (Specialization of an equation clause)**.** *Let* $[A;\, y.\, E(y)]$ *occur in* Γ *with* E *a literal equation, let* A *have a persistent proof over* $(\Xi;\Gamma)$*, and let* t *be a term over* Ξ*. Then* $E(t)$ *has a persistent proof over* $(\Xi;\Gamma)$*. Consequently, if* $\mathcal{M}_{b,t}$ *occurs in* Γ*, at any position, then* $b = t$ *has a persistent proof over* $(\Xi;\Gamma)$*.*

Proof. Let an extension be given; write t, E, A for the renamed expressions. Derive Nm_t and open it by DE at a fresh b (Theorem 6.71). In the premise, b is the newest parameter and $\mathcal{M}_{b,t}$ the newest clause. The persistent proof of A, taken at this larger situation, gives A. The given clause is now an older clause, so USE_* applies it at $b_* \equiv b$ and gives $E(b)$. Theorem 6.71 gives $b = t$, and transport in the equation template E (Theorem 6.69), which opens no scope, gives $E(t)$. This formula does not contain b, and DE closes. The construction is uniform in the extension, so it is a persistent proof.

For the second statement apply the first to the clause $\mathcal{M}_{b,t}$, the target 1 and the antecedent $0 = 0$, which has the persistent proof REFL; this gives $\text{sel}(1, 0, b) = \text{sel}(1, 0, t)$ persistently, and Theorem 6.73(c) with the selector computations gives $b = t$. The argument is not circular: the auxiliary naming clause of the first part is newest when it is used at 1, and the final transport there is between equations. □

Lemma 6.75 (Transport and specialization at every rank)**.** *For every* $r \geq 0$ *the following hold.*

(T_r) *Let* $F(w)$ *be a formula template of rank* r *and let* $s = t$ *have a persistent proof over* $(\Xi;\Gamma)$*. From a derivation in* $\mathcal{U}_*$ *of* $\Gamma \vdash F(s)$ *one obtains a derivation of* $\Gamma \vdash F(t)$*; and from a persistent proof of* $F(s)$ *over* $(\Xi;\Gamma)$ *a persistent proof of* $F(t)$*.*

(S_r) *Let* $[A;\, y.\, F(y)]$ *occur in* Γ *with* F *of rank* r*, let* A *have a persistent proof over* $(\Xi;\Gamma)$*, and let* t *be a term over* Ξ*. Then* $F(t)$ *has a persistent proof over* $(\Xi;\Gamma)$*.*

Proof. (T_0) is transport in an equation template (Theorem 6.69), with the persistent proof of $s = t$ taken at the identity extension; (S_0) is Theorem 6.74. In every (T_r) the second statement follows from the first, applied in each extension to the restricted persistent proofs.

(T_r) for $r > 0$, from (T_{r-1}) and (S_{r-1}). Let $F(w) = D_{x,y}G(w, x, y)$. Use the given derivation of $F(s)$ as the major premise of DE, with a fresh witness c; the context of the branch contains the clause $\mathcal{C} := [G(s, c, 0);\, y.\, G(s, c, y)]$. In the branch introduce $F(t)$ by DI at the witness c. As c is a parameter it is not the literal 0, so this is the case mode also when G is an equation, and the premise has the assumption clause of $G(t, c, 0)$ and the goal $G(t, c, Sz)$. In the situation of that premise:

- $G(t, c, 0)$ has a persistent proof by Theorem 6.73(b);
- $t = s$ has a persistent proof by Theorem 6.73(a),(c), so (T_{r-1}) for the template $G(w, c, 0)$ gives a persistent proof of $G(s, c, 0)$;
- (S_{r-1}) for the clause $\mathcal{C}$, this antecedent and the target Sz gives a persistent proof of $G(s, c, Sz)$;
- (T_{r-1}) for the template $G(w, c, Sz)$, applied to the derivation that this persistent proof yields at the identity extension, gives a derivation of $G(t, c, Sz)$.

DI and DE close; $F(t)$ does not contain c. The given derivation of $F(s)$ is used outside the new scopes and is not weakened.

(S_r) for $r > 0$, from (T_r) and Theorem 6.74. In a given extension derive Nm_t and open it at a fresh b. In the premise, the persistent proof of A gives A, and USE_* applies the given clause, now older, at b, which gives a derivation of $F(b)$. By Theorem 6.74, $b = t$ has a persistent proof over the situation of the premise; it is needed in persistent form because (T_r) opens further scopes. (T_r) gives $F(t)$, and DE closes. The construction is uniform in the extension.

The recursion is well founded: (T_r) uses rank $r - 1$ only, and (S_r) uses (T_r) and rank 0. □

6.6.4 Equivalence

Theorem 6.76 (Deterministic specialization)**.** *For every parameter list, every context* Γ *and every formula* A*,*

$$\Gamma \vdash_{\mathcal{U}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{U}_*} A.$$

The terms, the formulas and the ordinary reading are unchanged.

Proof. Every instance of USE_* is an instance of USE. Conversely, by induction on a derivation in $\mathcal{U}$ of $\Gamma \vdash A$ over Ξ, the formula A has a persistent proof over $(\Xi; \Gamma)$.

REFL and STEP: Theorem 6.73(c). A USE of $[A';\, y.\, B]$ at t: the induction hypothesis gives a persistent proof of A', and (S_r) of Theorem 6.75, with r the rank of B, gives a persistent proof of $B(t)$. DE with the major premise $D_{x,y}B$ and the branch over $(\Xi, b;\, \mathcal{C}_B(b), \Gamma)$: in a given extension choose a fresh b', take the derivation of the major premise that its persistent proof yields there, extend the extension by $b \mapsto b'$ and by sending the opened clause to the new opened clause, take the derivation that the persistent proof of the branch yields at this extension, and apply DE. DI is treated in the same way; an injective renaming preserves the root of the matrix and the literal test of the witness, hence the mode.

The compilation recurses on the height of the given derivation, and the auxiliary procedures recurse on the rank of formulas; there is no recursion on a derivation that the procedure has produced. So every derivation in $\mathcal{U}$ yields a finite derivation in $\mathcal{U}_*$: evaluate the persistent proof at the identity extension. □

Composed with the comparison of `uniform_step` between $\mathcal{U}$ and the kernel and with Theorem 1.119, the theorem gives the bi-interpretation of the ordinary reading of $\mathcal{U}_*$ with iPRA, with whole domains, actual equality and identity numerical composites; the equivalence itself changes no term, no formula and no reading.

Remark 6.77 (The cost). The argument of an arbitrary specialization is removed from the checker at the price of larger proofs: the translation wraps every use of a clause at a term in a naming formula, and at rank r it nests the constructions of ranks below r. In the test suite of the package the largest expansion is from 699 to 175,640 bytes of compact JSON. No polynomial bound on the size of the translation is claimed.

Remark 6.78 (The kernel counterpart in Lean). On the kernel the corresponding restriction concerns the term of QE. With QE allowed at a variable and at the numeral 1 only, the kernel derives the same sequents: this is `VQPrf` with `vqprf_iff_nprf` (`NativeVariableQE`). An arbitrary target t is named by a fresh alias $b = t$, whose discharge uses QE at 1 once; QE is applied at b, and $B(b)$ is moved to $B(t)$ by the Leibniz rule, which is proved together with QE at Sz by induction on the formula (`VQPrf.qELeib`, `VQPrf.leibniz`). The numeral cannot be dropped (`vqprf_var_incomplete`): with QE at variables only, the kernel does not derive $0 = 1$ from $Q_y(\mathrm{sel}(y, 0, 1) = 0)$, which N derives by QE at 1. The countermodel reads every variable as 0, every equation by its value in the natural numbers, and every formula with root Q or D, as true; each rule preserves truth, since $B(v)$ at a variable v has the truth value of $B(0)$. The form of the package, on zero-normal terms with clause contexts and the choice of the target by position, is not formalized; that choice is not stable under renaming, which is why the proof above regenerates the proofs of premises.

6.7 Fresh aliases and backward substitution

This section draws on `alias_back` (Sections 6.7.1 to 6.7.4) and `equation_aliases` (Section 6.7.5).

In Section 6.5.3 an alias $b = a$ is discharged only from a conclusion that does not contain b (Theorem 6.59), and the discharge is derived from the computations, joining at the root and the logical rules. Here the discharge of a fresh alias is a primitive rule, and it substitutes the defining term into the conclusion. Together with substitution for a parameter in the backward direction it gives reflexivity, symmetry and transport in every formula on raw terms, with no computation, normalization or induction (Theorem 6.83). With it the source of the equality rule and the target of every use of a clause can both be restricted to the newest parameter (Theorem 6.90), and, at the price of an induction on the rank of formulas, both new rules can be restricted to literal equations (Theorem 6.100). The arithmetic is unchanged: the two computation equations, the elimination OPEN and the introduction CLOSE with a supplied witness are retained.

In this section terms are the raw terms of Section 1.1, and substitution is the capture-avoiding substitution of raw syntax; it is simultaneous and performed in one pass, so that $F[t/b]$ is defined also when t contains b. Nothing is normalized. A judgment $\Pi; \Gamma \vdash A$ carries its numerical scope Π as in Theorem 7.1, and $\nu(\Pi)$ is the newest parameter of Π, or 0 if Π is empty; every rule with an eigenparameter appends it to the scope of its premise. Where the scope plays no role it is not written. The clause calculi of the section have the uniform clauses of Theorem 5.26, with at most one antecedent, the assumption clause $\langle A \rangle = [\ ;\ v.\, A]$, and no reflexivity clause; Γ, F denotes Γ extended by $\langle F \rangle$.

6.7.1 The two rules

Definition 6.79 (Alias and backward substitution)**.**

$$\frac{\Pi, b;\ \Gamma,\ b = t \vdash F}{\Pi; \Gamma \vdash F[t/b]}\ \textsc{alias} \tag{6.22}$$

$$\frac{\Pi; \Gamma \vdash b = t \qquad \Pi; \Gamma \vdash F[t/b]}{\Pi; \Gamma \vdash F}\ \textsc{back} \tag{6.23}$$

In ALIAS the parameter b is not in Π, hence fresh for Γ and for the *defining term* t, which is over Π; the formula F is over Π, b and may contain b, and every free occurrence of b in F is replaced in the conclusion. In a clause calculus the added assumption is the clause $\langle b = t \rangle$. In BACK the *source* b is a parameter of Π, the term t and the formula F are over Π, and t may contain b. The conclusion F is the goal: no template is supplied, and the second premise is computed from the goal and the first premise. The first premise is oriented; a derivation of $t = b$ does not take its place. The restricted forms are $\textsc{back}_\nu$, in which Π is not empty and $b \equiv \nu(\Pi)$; $\textsc{alias}^=$ and $\textsc{back}^=$, in which F is a literal equation; and $\textsc{back}^=_\nu$, with both restrictions. `equation_aliases` calls $\textsc{alias}^=$ and $\textsc{back}^=_\nu$ EQALIAS and EQBACK.

The essential difference from Theorem 6.59 is that the conclusion of ALIAS may change on discharge: from its own assumption the rule exports $(b = t)[t/b] \equiv (t = t)$. This is the step that is missing from the rule of parameter substitution in the reverse direction with an ordinary elimination of a fresh name, which `uniform_binding` considered and discarded (Section 9.15): once the goal no longer mentions the alias, that rule can require the goal itself as its premise. ALIAS is NAME (6.10) without its paired formula: the equation $b = t$ is assumed instead of being proved from an opened clause. BACK differs from VSUB (6.16) in its direction and in its scope: VSUB substitutes forward in the left side of an equation, and BACK computes its premise from a goal that is an arbitrary formula. Like BREP of Theorem 2.50 it is goal-directed; unlike BREP it substitutes for a parameter and does not replace occurrences of a compound term.

Proposition 6.80 (The freshness conditions)**.** *Neither freshness condition of* ALIAS *can be dropped.*

(a) *If b may be free in Γ, the rule derives $0 = S0$ from the assumption $b = S0$.*

(b) *If b may occur in t, the rule derives $Sb = SSb$ from the empty context.*

Both conclusions are false in the natural numbers under every assignment that makes the context true, while all other rules of the calculi of this section are sound for the ordinary reading in the natural numbers.

Proof. (a) Let Γ be the assumption $b = S0$ and $t \equiv 0$. The premise $\Gamma,\ b = 0 \vdash b = S0$ is a use of an assumption, and the discharge would export $(b = S0)[0/b] \equiv (0 = S0)$, although Γ is true at $b = 1$. (b) Let $t \equiv Sb$. The premise $b = Sb \vdash b = Sb$ is a use of the assumption, and the discharge would export $(b = Sb)[Sb/b] \equiv (Sb = SSb)$, which is false at every value of b. □

The checkers of `alias_back` validate the defining term in the outer scope before the alias parameter is added, so that both instances above are rejected. BACK has no freshness condition: its replacement may contain its source, because BACK discharges nothing.

Lemma 6.81 (Soundness in the kernel)**.** *Both rules are admissible in* N*. If $\Gamma,\ b = t \vdash_{\mathsf{N}} F$ and b is not free in Γ or in t, then $\Gamma \vdash_{\mathsf{N}} F[t/b]$. If $\Gamma \vdash_{\mathsf{N}} b = t$ and $\Gamma \vdash_{\mathsf{N}} F[t/b]$, then $\Gamma \vdash_{\mathsf{N}} F$.*

Proof. For ALIAS apply Theorem 1.10 with the substitution $\theta = [t/b]$ and the context $\Gamma,\ t = t$. Since b is not free in Γ, every member of Γ is unchanged by θ, and since b is not free in t, $(b = t)[\theta] \equiv (t = t)$; so $\Gamma,\ t = t \vdash F[t/b]$. Reflexivity (Theorem 1.18(i)) and Theorem 1.11, with HYP for the members of Γ, give $\Gamma \vdash F[t/b]$.

For BACK let w be fresh and $K := F[w/b]$. Then $K[b/w] \equiv F$ and $K[t/w] \equiv F[t/b]$, also when t contains b, because the substitution is simultaneous. By Theorem 1.22 with $\Gamma \vdash b = t$, the formulas $K[b/w]$ and $K[t/w]$ are interderivable over Γ. □

6.7.2 Equality from three rules

Lemma 6.82 (Structural operations with unrestricted sources)**.** *Let $\mathcal{S}$ have the rules* ALIAS *and* BACK*, or* ALIAS$^=$ *and* BACK$^=$*, together with either the rules of* N *other than* TR*, or the rules* I0*,* IS*,* USE *at an arbitrary term, and* OPEN *and* CLOSE *of Theorem 6.56 over uniform clauses, with $\langle B(t,\eta)\rangle$ as the assumption of* CLOSE*. Then weakening and injective renaming are admissible in $\mathcal{S}$: if $\Pi;\Gamma \vdash A$, every parameter of Π is in Π', in any order, and every occurrence of Γ is an occurrence of a context Γ' over Π', then $\Pi';\Gamma' \vdash A$; and if ι is an injective map of parameters, then $\Pi\iota;\Gamma\iota \vdash A\iota$. Both transformations preserve the rule at each node and the height.*

Proof. By induction on the derivation, after the eigenparameters and the alias parameters of the derivation have been renamed away from Π' and Γ', respectively from the range of ι. A use of an occurrence is a use of its image. The source of BACK is a parameter of the old scope, hence of the new one, and $(F\iota)[t\iota/b\iota] \equiv (F[t/b])\iota$ for an injective ι. The defining term of ALIAS is over the old scope, and its parameter is fresh for the new one. An injective renaming preserves the root of a matrix and the test $t \equiv 0$, hence the mode of CLOSE and of DI, and it preserves literal equations. The remaining rules are treated as in Theorem 1.10 and Theorem 6.58. □

The lemma fails for BACK$_\nu$ and for every rule whose argument is the newest parameter: inserting a newer parameter changes ν, as in Theorem 7.5.

Proposition 6.83 (Equality from HYP, ALIAS and BACK)**.** *Let $\mathcal{S}$ be as in Theorem 6.82. The following rules are derived in $\mathcal{S}$, under every context:*

$$(a)\ \frac{}{t = t} \qquad (b)\ \frac{s = t}{t = s} \qquad (c)\ \frac{s = t \qquad H(s)}{H(t)} \qquad (d)\ \frac{s = t \qquad h(s) = r}{r = h(t)}\ \textit{TR}$$

for every term t, every formula template $H(v)$, which is to be a literal equation if $\mathcal{S}$ has ALIAS$^=$ *and* BACK$^=$*, and every term template $h(v)$. The derivations consist of the given premises, weakened by Theorem 6.82, uses of assumptions, and instances of* ALIAS *and* BACK*. The source of every instance of* BACK *that they add is the parameter of the innermost* ALIAS *whose branch contains it, and it is the newest parameter at that node. In (a), (b) and (d), and in (c) when H is a literal equation, every added instance of* ALIAS *and* BACK *has a literal equation as its conclusion.*

Proof. (a) Choose b fresh. Under the alias $b = t$ the use of its assumption proves $b = t$, and ALIAS exports $(b = t)[t/b]$, which is $t = t$ because b does not occur in t.

(b) Let $\Gamma \vdash s = t$, and choose b fresh for Γ, s and t. Work under the alias $b = s$, where the given derivation is weakened. BACK with the first premise $b = s$ and the goal $b = t$ has the second premise $(b = t)[s/b] \equiv (s = t)$; so it proves $b = t$. By (a), $t = t$; the alias of that derivation is opened and discharged before the next step, so b is again the newest parameter. BACK with the first premise $b = t$ and the goal $t = b$ has the second premise $(t = b)[t/b] \equiv (t = t)$; so it proves $t = b$. ALIAS discharges $b = s$ and exports $(t = b)[s/b] \equiv (t = s)$.

(c) Let $\Gamma \vdash s = t$ and $\Gamma \vdash H(s)$. By (b), $\Gamma \vdash t = s$. Choose b fresh for Γ, H, s, t and the names of the given derivations, and work under the alias $b = t$. BACK with the first premise $b = t$ and the goal $b = s$ has the second premise $t = s$; so it proves $b = s$. BACK with the first premise $b = s$ and the goal $H(b)$ has the second premise $H(b)[s/b] \equiv H(s)$, because b does not occur in $H(v)$; so it proves $H(b)$. ALIAS exports $H(b)[t/b] \equiv H(t)$. The parameter b marks exactly the distinguished positions of the template; the occurrences of s and t in its passive part are not touched, the distinguished variable may occur several times, under the binder of an iterator or of $D_,$, or not at all, and capture is excluded by the choice of b and by simultaneous substitution. If H is a literal equation, the four conclusions $b = s$, $H(b)$, $H(t)$ and those of (b) are equations.

(d) Apply (c) to $H(v) \equiv (h(v) = r)$, which gives $h(t) = r$, and then (b). □

Transitivity is (c) with $H(v) \equiv (s = v)$, and joining at a common left side is (d) with the identity template; so every instance of PARAM (6.18) is derived.

Remark 6.84 (The equality fragment). The proposition is a completeness statement for an equality fragment alone, on raw terms: it uses neither a computation equation nor a logical rule, whereas the derivations of TR in Theorems 6.17, 6.49 and 6.60 use packets, computations or joining at the root, and Theorem 6.59 uses all three. It does not bear on the conjecture recorded in Section 6.11 that substitution for a variable alone is weaker on raw terms: ALIAS is an additional rule. In the raw calculus with substitution at a variable and no reflexivity, reflexivity is lost (Theorem 11.180); its invariant is not preserved by ALIAS, whose premise $b = t \vdash b = t$ is valid vacuously while its conclusion $t = t$ may fail the head test. `alias_back` contains an independent checker of the fragment with HYP, ALIAS and BACK only, with a schematic unary predicate for the template of (c); it accepts seven certificates, among them transport below $D_,$ and iterator binders, nested premise derivations and a replacement that contains its source, and rejects twelve malformed ones.

6.7.3 The kernel with aliases

Definition 6.85. N_{AB} is N with TR replaced by ALIAS and BACK; its rules are HYP, I0, IS, ALIAS, BACK, QE, DE, DI and QI. $\mathsf{N}^{=}_{\mathrm{AB}}$ has ALIAS$^{=}$ and BACK$^{=}$ in their place. Contexts are lists of native formulas.

Theorem 6.86 (The kernel with aliases)**.** *For every context* Γ *and every native formula* A*,*

$$\Gamma \vdash_{\mathsf{N}} A \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathsf{N}_{\mathrm{AB}}} A \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathsf{N}^{=}_{\mathrm{AB}}} A,$$

by effective transformations of derivations that are the identity on the end sequent. Consequently the ordinary readings of N_{AB} *and* $\mathsf{N}^{=}_{\mathrm{AB}}$ *are bi-interpretable and equiconsistent with* iPRA*, with whole domains, actual equality and both numerical composites with graph* $x = y$*.*

Proof. Every rule of $\mathsf{N}^{=}_{\mathrm{AB}}$ is a rule of N_{AB}. From N_{AB} to N: by induction on the derivation, with Theorem 6.81 at ALIAS and BACK; the other rules are rules of N. The substitution in that lemma is a substitution in derivations of N, where it is admissible; it is not assumed for the new calculi. From N to $\mathsf{N}^{=}_{\mathrm{AB}}$: by induction on the derivation, copying every rule other than TR and replacing every TR by the derivation of Theorem 6.83(d), whose alias parameters are chosen away from all names of the translated premises; the premises are weakened into the alias contexts by Theorem 6.82, applied to $\mathsf{N}^{=}_{\mathrm{AB}}$. The translations do not change terms, formulas or contexts, so the statement on iPRA follows from Theorem 1.119, and $0 = S0$ is fixed. □

One rule of transport is replaced by two schemes. The theorem is a different organization of equality, not a deletion of equality reasoning, and it does not reduce the number of rule families.

6.7.4 Seven rules on clause syntax and the newest parameter

Definition 6.87 (The raw template calculus)**.** $\mathcal{T}_\mathrm{r}$ is the calculus over uniform clauses with the rules I0 and IS, as axioms under every context, TR (6.7), USE at an arbitrary term, without premise for a clause without antecedent, and OPEN and CLOSE of Theorem 6.56, the assumption of CLOSE being $\langle B(t,\eta)\rangle$.

$\mathcal{T}_\mathrm{r}$ is the raw template calculus of `parameter_substitution` with assumptions without antecedent; both packages of this section use it as their matched control.

Lemma 6.88 (The raw template calculus)**.** (a) *For every context* Γ *of uniform clauses and every formula* A, $\Gamma \vdash_{\mathcal{T}_\mathrm{r}} A$ *if and only if* $\Gamma^\circ \vdash_{\mathcal{P}} A$, *where* Γ° *replaces every occurrence* $[\,;y.\,B]$ *by* $[H;y.\,B]$ *as in Theorem 5.26.*

(b) *For every context* Γ *of native formulas and every native formula* A, *with* τ *as in Theorem 6.64,* $\Gamma \vdash_{\mathsf{N}} A$ *if and only if* $\langle\tau\Gamma\rangle \vdash_{\mathcal{T}_\mathrm{r}} \tau A$.

(c) *In* $\mathcal{T}_\mathrm{r}$, *under every context of uniform clauses, reflexivity, symmetry, transport in every formula template and the case introduction for every matrix and every witness are derived, and weakening, injective renaming, cut of an assumption and substitution of terms for parameters (*$\Gamma \vdash A$ *implies* $\Gamma\sigma \vdash A\sigma$*) are admissible.*

Proof. (a) The occurrences of Γ and Γ° correspond by position. From $\mathcal{P}$ to $\mathcal{T}_\mathrm{r}$: COMPUTE is I0 or IS; every instance of PARAM is an instance of TR, with the template $u[w/s]$ if s is a parameter, the identity template if s is not a parameter and $u \equiv s$, and the constant template u otherwise (proof of Theorem 6.64); a USE of an occurrence $[H;\,y.\,B]$ that corresponds to $[\,;y.\,B]$ becomes the USE without premise, the derivation of H being discarded; every other USE keeps its translated premise; OPEN adds the same clause on both sides, and CLOSE adds $\mathrm{ord}(B(t,\eta))$, which is $\langle B(t,\eta)\rangle^\circ$. From $\mathcal{T}_\mathrm{r}$ to $\mathcal{P}$: I0 and IS are COMPUTE; TR is derived in $\mathcal{P}$ under every context (Theorem 6.61); a USE of $[\,;y.\,B]$ becomes a USE of $[H;\,y.\,B]$ whose premise is reflexivity (Theorem 6.57); the other rules are copied.

(b) $\langle\tau\Gamma\rangle^\circ = \mathrm{ord}(\tau\Gamma)$, so (a) and Theorem 6.64 give the claim.

(c) A rule derived in $\mathcal{P}$ under every context, whose premises have the context of the conclusion extended by assumption clauses, is derived in $\mathcal{T}_\mathrm{r}$ by (a), since $(\Gamma,\langle F\rangle)^\circ = \Gamma^\circ, \mathrm{ord}(F)$; the same holds for admissible weakening, renaming and cut. Reflexivity and symmetry are Theorem 6.57, transport is Theorem 6.63(e), the case introduction is Theorem 6.62, and weakening, renaming and cut are Theorem 6.58. Substitution is proved directly by induction on the derivation, as Theorem 5.14, with the eigenparameters made fresh for σ: the computation axioms go to computation axioms; for TR the template variable is chosen fixed by σ and not free in its values, as in Theorem 1.10; USE and OPEN commute with σ; a CLOSE keeps its mode unless it is in case mode with a literal equation as matrix and a witness t with $t\sigma \equiv 0$, and then the substituted premise is the premise of the case introduction at the witness 0, which is derived. □

Definition 6.89 (Clause calculi with aliases)**.** $\mathcal{B}$ is the calculus over uniform clauses with the rules I0, IS, USE at an arbitrary term, BACK, ALIAS, OPEN and CLOSE (of $\mathcal{T}_\mathrm{r}$). $\mathcal{B}_\nu$ has the seven rules I0, IS, USE_ν, BACK_ν, ALIAS, OPEN and CLOSE, where

$$\frac{\Pi;\Gamma \vdash A}{\Pi;\Gamma \vdash B(\nu(\Pi))}\ \mathrm{USE}_\nu, \qquad [A;\,y.\,B(y)] \text{ an occurrence in } \Gamma, \tag{6.24}$$

without premise for a clause without antecedent.

A certificate of USE_ν names the occurrence and contains no term, and a certificate of BACK_ν contains no source and no template. $\mathcal{B}_\nu$ has no primitive reflexivity, symmetry, transitivity, joining at the root, transport, template, normalization, predecessor or logical connective. I0 and IS are two equations even when a checker groups them. The witness of CLOSE is still supplied, and its progress mode is restricted to literal equations, so no induction for compound formulas is added. A use of an assumption $\langle A \rangle$ by USE_ν gives A whatever $\nu(\Pi)$ is.

Theorem 6.90 (The newest parameter suffices)**.** *Let p be a derivation of $\Pi; \Gamma \vdash A$ in $\mathcal{B}$. Let θ be an injective map of Π into a scope Π', in any order and with any further parameters, and let Δ be a context over Π' together with an injective map of the occurrences of Γ to occurrences of Δ that sends an occurrence of C to one of θC. Then a derivation of $\Pi'; \Delta \vdash \theta A$ in $\mathcal{B}_\nu$ is constructed effectively. If every BACK of p has a literal equation as its conclusion, so has every BACK_ν of the constructed derivation.*

In the terms of the chapter, every derivation of $\mathcal{B}$ yields a persistent proof (Theorem 6.72) in $\mathcal{B}_\nu$, that is, hereditary derivability in the sense of Theorem 7.27, with clause occurrences in the place of members of the context.

Proof. By induction on p, simultaneously for all Π', θ, Δ and occurrence maps. The induction hypothesis for a premise of the last rule is used with data constructed at that node, and no derivation produced by the construction is weakened, renamed or substituted into.

I0, IS. The image under θ is an instance of the same axiom.

OPEN. The major premise is translated at (Π', θ, Δ). For the branch, over Π, b with the opened clause $\mathcal{C}_B(b)$, choose $b' \notin \Pi'$, extend θ by $b \mapsto b'$ and the occurrence map by the opened clause to the new opened clause $\mathcal{C}_{\theta B}(b')$, and translate the branch at $(\Pi', b';\ \Delta, \mathcal{C}_{\theta B}(b'))$. OPEN concludes θA, which is over Π'.

CLOSE. Choose $z' \notin \Pi'$, extend θ by $z \mapsto z'$, append $\langle \theta B(\theta t, \eta') \rangle$, where η' is the image of η, and translate the branch there. CLOSE at the witness θt concludes $\theta(D_{x,y}B)$. The mode is the same, since an injective map of parameters preserves the root of the matrix, and $\theta t \equiv 0$ exactly if $t \equiv 0$.

ALIAS with the defining term t and the parameter b. Choose $b' \notin \Pi'$, extend θ by $b \mapsto b'$ to θ', append $\langle b' = \theta t \rangle$, and translate the branch there; it proves $\theta' F$. ALIAS exports $(\theta' F)[\theta t / b'] \equiv \theta(F[t/b])$.

BACK with the premises $v = t$ and $F[t/v]$ and the conclusion F, where v is a parameter of Π. Choose $b \notin \Pi'$ and work under the alias $b = \theta v$, in the scope Π', b and the context $\Delta, \langle b = \theta v \rangle$, where b is the newest parameter. Translate both premises there, by the induction hypothesis with the same θ and the occurrence map into the extended context; they give $\theta v = \theta t$ and $\theta(F[t/v])$. Then:

- BACK_ν with the first premise $b = \theta v$, the use of the alias assumption, and the goal $b = \theta t$ has the second premise $\theta v = \theta t$; it proves $b = \theta t$.
- Let $G := (\theta F)[b/\theta v]$. Since b does not occur in θF, $G[\theta t/b] \equiv (\theta F)[\theta t/\theta v] \equiv \theta(F[t/v])$, also when t contains v. BACK_ν with the first premise $b = \theta t$ and the goal G proves G.
- ALIAS exports $G[\theta v/b] \equiv \theta F$.

If F is a literal equation, so are $b = \theta t$ and G.

USE of an occurrence $[A'; y.\, B]$ at a term t. Choose $b \notin \Pi'$ and work under the alias $b = \theta t$. If the clause has an antecedent, translate the derivation of A' there. USE_ν of the image of the occurrence at the newest parameter b proves $(\theta B)(b)$, and ALIAS exports $(\theta B)(\theta t)$, which is $\theta(B(t))$. No BACK is used.

Every call of the induction hypothesis concerns a proper subderivation of p, so the construction terminates, although contexts and terms grow. □

Corollary 6.91 (Equivalence and bridge)**.** *For every scope, every context* Γ *of uniform clauses and every formula* A*,*

$$\Gamma \vdash_{\mathcal{B}_\nu} A \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathcal{B}} A \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathcal{T}_\mathrm{r}} A,$$

and for every native context Γ *and native formula* A*,* $\Gamma \vdash_{\mathsf{N}} A$ *if and only if* $\langle \tau\Gamma \rangle \vdash_{\mathcal{B}_\nu} \tau A$*. Weakening, renaming, substitution and cut are admissible in* $\mathcal{B}_\nu$ *as operations on derivable judgments. The ordinary reading of* $\mathcal{B}_\nu$ *is bi-interpretable and equiconsistent with* iPRA*, with whole domains, actual equality and both numerical composites with graph* $x = y$*.*

Proof. $\mathcal{B}_\nu \subseteq \mathcal{B}$: $\textsc{use}_\nu$ is USE at the term $\nu(\Pi)$, and $\textsc{back}_\nu$ is an instance of BACK.

$\mathcal{B} \subseteq \mathcal{T}_\mathrm{r}$: by induction on the derivation. I0, IS, USE, OPEN and CLOSE are rules of $\mathcal{T}_\mathrm{r}$. For BACK, transport along the symmetric $t = b$ in the template $F[w/b]$, w fresh, sends $F[t/b]$ to F (Theorem 6.88(c)). For ALIAS, the substitution $[t/b]$ in the translated branch gives $\Gamma, \langle t = t \rangle \vdash F[t/b]$, since b is not free in Γ or t, and cut with reflexivity gives $\Gamma \vdash F[t/b]$ (Theorem 6.88(c)).

$\mathcal{T}_\mathrm{r} \subseteq \mathcal{B}$: by induction on the derivation; TR is Theorem 6.83(d), with the premises weakened by Theorem 6.82, and the other rules are copied.

$\mathcal{B} \subseteq \mathcal{B}_\nu$: Theorem 6.90 at the identity map and the identity context.

The statement on N is Theorem 6.88(b). The structural operations are transferred through $\mathcal{T}_\mathrm{r}$; they are consequences of the equivalence, not steps of its proof. The bridge is Theorem 1.119 composed with these translations, which leave terms unchanged and fix $0 = S0$; both formula round trips are those of Theorem 6.64. □

Corollary 6.92 (Backward substitution for equations only)**.** *Theorem 6.91 holds with* $\textsc{back}^{=}_{\nu}$ *in the place of* $\textsc{back}_\nu$ *in* $\mathcal{B}_\nu$*,* ALIAS *still exporting arbitrary formulas.*

Proof. The translation from $\mathcal{T}_\mathrm{r}$ to $\mathcal{B}$ adds only the instances of BACK of Theorem 6.83(d), which have literal equations as conclusions, and weakening preserves this. The compiler of Theorem 6.90 then produces only instances of $\textsc{back}^{=}_{\nu}$. Its USE case exports compound formulas by ALIAS. □

Remark 6.93 (The kernel with the newest parameter)*.* The same induction applies to the kernel. Let $\mathsf{N}^{\nu}_{\mathrm{AB}}$ be N_{AB} with $\textsc{back}_\nu$ in the place of BACK and with QE restricted to the target $\nu(\Pi)$, the witness of DI being arbitrary. By induction on a derivation of N_{AB}, for all injective θ and all contexts $\Delta \supseteq \theta\Gamma$, $\mathsf{N}^{\nu}_{\mathrm{AB}}$ derives $\Pi'; \Delta \vdash \theta A$: HYP is kept; BACK is compiled as above; QE at t is compiled as USE, under the alias $b = \theta t$ with the translated major premise and base, QE at b and ALIAS; and DE, DI and QI append a fresh eigenparameter as OPEN and CLOSE do. So $\mathsf{N}^{\nu}_{\mathrm{AB}}$ and N derive the same sequents. If only QE is restricted and BACK is not, weakening in N_{AB} (Theorem 6.82) preserves the height of a derivation, and an induction on the height suffices: the source premises of QE are weakened into the alias context in N_{AB} and then translated, and no translated derivation is weakened. In the Lean development the target of QE can be restricted to a variable or the numeral 1 (Theorem 6.78), and the numeral cannot be dropped there (`vqprf_var_incomplete`). With ALIAS it can: the countermodel of Theorem 6.78 does not validate ALIAS. In the context $Q_y(\mathrm{sel}(y, 0, 1) = 0)$, under the alias $b = 1$, QE at b with the base $\mathrm{sel}(0, 0, 1) = 0$, an instance of I0, proves $\mathrm{sel}(b, 0, 1) = 0$; ALIAS exports $\mathrm{sel}(1, 0, 1) = 0$, and IS and the derived equality rules give $0 = 1$, which is false in the countermodel while the context is true there.

Remark 6.94 (Relation to the other restrictions of the chapter)*.* In $\mathcal{V}_0$ (Section 6.5.2) the source of the equality rule is the newest variable too, but completeness rests on full normal forms, which make the rows of the packet literal, and on internally sourced derivations (Theorem 6.53). In the derivations produced by Theorem 6.90 every source of $\textsc{back}_\nu$ is the

parameter of an ALIAS, so the alias plays the role of the packet, raw terms suffice, and the invariant is replaced by the induction over all target data. In $\mathcal{U}_*$ (Section 6.6) the use of a clause takes no term either; it uses the newest clause at 1 and every other clause at the newest parameter, on zero-normal terms, and needs the naming formula Nm_t and an induction on rank. With an ALIAS that exports compound formulas, USE_ν needs neither; Section 6.7.5 shows that the rank induction returns when ALIAS exports equations only. The selection $\nu(\Pi)$ is that of Theorem 7.1, and the families of derivations of the proof are those of Theorem 7.27 (Theorem 7.37). In Section 7.5 the witness of the introduction is the newest parameter and the target of QE is free; here the target of specialization and the source of equality are the newest parameter and the witness of CLOSE is supplied. The restriction of the target does not fall under Theorem 7.7, where the witness and the target are restricted together and no rule supplies a term; here the witness is arbitrary and ALIAS supplies an arbitrary term.

6.7.5 Equation-only aliases and the rank of formulas

Definition 6.95 (The calculus $\mathcal{A}$)**.** $\mathcal{A}$ has the seven rules I0, IS, USE_ν, $\mathrm{BACK}_\nu^=$, $\mathrm{ALIAS}^=$, OPEN and CLOSE over uniform clauses. The conclusion of OPEN is an arbitrary formula.

So $\mathcal{A}$ is $\mathcal{B}_\nu$ with both new rules restricted to literal equations. No rule of $\mathcal{A}$ exports a compound formula from an alias, and no rule takes a term argument except the witness of CLOSE. The rank of a formula is the number of its binders (Section 6.6); substitution of terms does not change it.

Situations $(\Pi;\Gamma)$, their extensions and persistent proofs are those of Theorem 6.72, with the parameter list written Π and $\mathcal{A}$ in the place of $\mathcal{U}_*$; a persistent proof is the hereditary family of Theorem 7.27, and `equation_aliases` calls it a *uniform proof procedure*. A *local derivation* over $(\Pi;\Gamma)$ is one derivation of $\Pi;\Gamma \vdash A$ in $\mathcal{A}$. Both USE_ν and $\mathrm{BACK}_\nu^=$ change their meaning when a newer parameter is added, so a local derivation is not in general a persistent proof.

Lemma 6.96 (Elementary persistence)**.** (a) *A persistent proof over a situation restricts to a persistent proof over each of its extensions.*

(b) *If $\langle A\rangle$ occurs in Γ, then A has a persistent proof over $(\Pi;\Gamma)$.*

(c) *The instances of I0 and IS over Π have persistent proofs.*

Proof. (a) Extensions compose. (b) In every extension apply USE_ν to the image of the occurrence; its body does not contain the bound variable, so the conclusion is the image of A whatever the newest parameter is. (c) The image of an instance is an instance. □

Lemma 6.97 (The equational base)**.** *Let $(\Pi;\Gamma)$ be a situation and s,t terms over Π.*

(i) *$t = t$ has a persistent proof.*

(ii) *If $s = t$ has a persistent proof, so has $t = s$.*

(iii) (T_0) *If $E(v)$ is a literal equation and $s = t$ and $E(s)$ have persistent proofs, so has $E(t)$. In particular, if $s = t$ and $h(s) = r$ have persistent proofs, so has $r = h(t)$.*

(iv) (S_0) *If $[A;\, y.\, E(y)]$ occurs in Γ with E a literal equation, and A has a persistent proof (no condition for a clause without antecedent), then $E(t)$ has a persistent proof.*

Proof. Let an extension be given, and write s,t,E,A for the renamed expressions. In each item the derivation is that of Theorem 6.83, constructed in the extension, with every premise taken from its persistent proof at the situation in which it is needed; this situation is an

extension of the given one, by (a) of Theorem 6.96. No derivation is weakened. Every ALIAS and BACK of the constructions has a literal equation as its conclusion, and every BACK uses the alias opened last, which is the newest parameter at its node; in (ii) the alias of the reflexivity is discharged before the second BACK. The second statement of (iii) is (iii) with $E(v) \equiv (h(v) = r)$ followed by (ii). For (iv) choose b fresh, open the alias $b = t$, take the antecedent from its persistent proof at the extended situation, apply USE_ν to the image of the occurrence at the newest parameter b, which proves $E(b)$, and export $E(t)$ by $\text{ALIAS}^=$. □

The antecedent A in (iv) may have any rank; its persistent proof is given, not constructed by an induction on the rank of E.

Lemma 6.98 (Names)**.** *For a term t over Π let*

$$\text{Name}(t) := D_{x,y}(\text{sel}(y,t,x) = t), \qquad \mathcal{N}_{b,t} := [\text{sel}(0,t,b) = t;\ y.\ \text{sel}(y,t,b) = t],$$

the bound variable of the selector not being free in t; the matrix is that of Theorem 6.59 with $a \equiv t$.

(a) $\text{Name}(t)$ *has a persistent proof over every situation $(\Pi;\Gamma)$, and opening it by* OPEN *at a fresh b adds the clause $\mathcal{N}_{b,t}$.*

(b) *If $\mathcal{N}_{b,t}$ occurs in Γ, at any position, and b is a parameter of Π not occurring in t, then $b = t$ has a persistent proof over $(\Pi;\Gamma)$.*

Proof. (a) In every extension apply CLOSE at the witness t. Its goal $\text{sel}(Sz,t,t) = t$ is the instance of IS with the constant body t, and its assumption is not used; so the derivation is valid in either mode, also when $t \equiv 0$ and the rule is in progress mode. The opened clause at b is $\mathcal{N}_{b,t}$.

(b) The antecedent $\text{sel}(0,t,b) = t$ is an instance of I0, with a persistent proof by Theorem 6.96(c). By Theorem 6.97(iv) at the term 1, $\text{sel}(1,t,b) = t$ has a persistent proof, and IS gives $\text{sel}(1,t,b) = b$ persistently. Transport (iii) along $\text{sel}(1,t,b) = t$ in the equation template $v = b$ sends $\text{sel}(1,t,b) = b$ to $t = b$, and (ii) gives $b = t$. The clause $\mathcal{N}_{b,t}$ need not be the newest clause and b need not be the newest parameter, because the specialization of (iv) opens its own alias. □

In contrast with Theorem 6.71, where $b = t$ is derived only while the naming clause is newest, the equality here is persistent from the start. This is what allows it to be used after later binders have added newer parameters.

Theorem 6.99 (Transport and specialization at every rank)**.** *For every $r \geq 0$ and every situation $(\Pi;\Gamma)$ the following hold in $\mathcal{A}$.*

(T_r) *Let $F(w)$ be a formula template of rank at most r. If $s = t$ and $F(s)$ have persistent proofs over $(\Pi;\Gamma)$, so has $F(t)$.*

(S_r) *Let $[A;\ y.\,F(y)]$ occur in Γ with F of rank at most r, let A have a persistent proof over $(\Pi;\Gamma)$ (no condition without antecedent), and let t be a term over Π. Then $F(t)$ has a persistent proof over $(\Pi;\Gamma)$.*

(L_r) *For $r > 0$, let $F(w)$ have rank exactly r. From a local derivation of $\Pi;\Gamma \vdash F(s)$ and a persistent proof of $s = t$ over $(\Pi;\Gamma)$, a local derivation of $\Pi;\Gamma \vdash F(t)$ is constructed.*

Proof. (T_0) and (S_0) are (iii) and (iv) of Theorem 6.97. No local form (L_0) is used. Let $r > 0$ and suppose (T_{r-1}) and (S_{r-1}).

(L_r). Write $F(w) = D_{x,y}B(w,x,y)$, with B of rank $r-1$. Use the given local derivation of $F(s)$ as the major premise of OPEN at a fresh c; the branch has the situation $\Theta_1 = (\Pi, c;\ \Gamma, \mathcal{C})$

with $\mathcal{C} := [B(s,c,0);\ y.\,B(s,c,y)]$. The local derivation is used in the situation in which it was given and is never moved under c or a later parameter. In Θ_1 introduce $F(t)$ by CLOSE at the witness c. A parameter is not the literal 0, so the rule is in case mode whatever B is; its premise has the situation $\Theta_2 = (\Pi, c, z;\ \Gamma, \mathcal{C}, \langle B(t,c,0)\rangle)$ and the goal $B(t,c,Sz)$. Over Θ_2, which is an extension of $(\Pi;\Gamma)$:

- $B(t,c,0)$ has a persistent proof by Theorem 6.96(b), and $t = s$ has one by restriction and Theorem 6.97(ii); (T_{r-1}) for the template $B(w,c,0)$ gives a persistent proof of $B(s,c,0)$;
- (S_{r-1}) for the occurrence of $\mathcal{C}$, with this antecedent, at the term Sz gives a persistent proof of $B(s,c,Sz)$;
- (T_{r-1}) for the template $B(w,c,Sz)$ and the restricted proof of $s = t$ gives a persistent proof of $B(t,c,Sz)$.

Its member at the identity extension is the premise of CLOSE, which proves $F(t)$ over Θ_1. The formula $F(t)$ does not contain c, and OPEN concludes $F(t)$ over $(\Pi;\Gamma)$. Even for $r = 1$ every appeal to transport meets the hypotheses of (T_0), since all its premises are persistent.

(T_r). Templates of rank below r are covered by the hypothesis. For rank r, given an extension, take the member of the persistent proof of $F(s)$ there, which is a local derivation of the image of $F(s)$, restrict the persistent proof of $s = t$ to the extension, and apply (L_r) there.

(S_r). Formulas of rank below r are covered by the hypothesis. For rank r, in a given extension, with renamed expressions, derive Name(t) (Theorem 6.98(a)) and open it at a fresh b. In the situation Θ of the branch, $b = t$ has a persistent proof by Theorem 6.98(b). Take A from its persistent proof at Θ. USE_ν of the image of the occurrence at the newest parameter b gives a local derivation of $F(b)$ over Θ, and (L_r) with the persistent equation $b = t$ gives a local derivation of $F(t)$ over Θ. The formula $F(t)$ does not contain b, and OPEN discharges it.

The recursion is well founded: (L_r) uses (T_{r-1}) and (S_{r-1}), (T_r) uses (L_r), and (S_r) uses (L_r) and the name, which uses rank 0 only. □

Theorem 6.100 (Equation-only aliases)**.** *Every derivation of $\Pi;\Gamma \vdash A$ in $\mathcal{T}_\mathrm{r}$ yields a persistent proof of A over $(\Pi;\Gamma)$ in $\mathcal{A}$, effectively; in particular a derivation in $\mathcal{A}$. Consequently $\mathcal{A}$, $\mathcal{B}_\nu$, $\mathcal{B}$ and $\mathcal{T}_\mathrm{r}$ derive the same judgments over every context of uniform clauses, $\Gamma \vdash_\mathsf{N} A$ if and only if $\langle\tau\Gamma\rangle \vdash_\mathcal{A} \tau A$ for native Γ and A, weakening, renaming, substitution and cut are admissible in $\mathcal{A}$ as operations on derivable judgments, and the ordinary reading of $\mathcal{A}$ is bi-interpretable and equiconsistent with* iPRA *with whole domains, actual equality and both numerical composites with graph $x = y$.*

Proof. By induction on the derivation in $\mathcal{T}_\mathrm{r}$.

- I0, IS: Theorem 6.96(c).
- TR: Theorem 6.97(iii), applied to the persistent proofs of the two premises.
- USE of $[A';\ y.\,B]$ at t: (S_r) of Theorem 6.99 with r the rank of B, applied to the persistent proof of the antecedent given by the induction hypothesis. The antecedent may have a larger rank than B; its persistent proof comes from the induction on the derivation, not from the induction on rank.
- OPEN with the major premise $D_{x,y}B$: in a given extension take the member of the persistent proof of the major premise, choose a fresh b', extend the extension by $b \mapsto b'$ and by the opened clause to the new opened clause, take the member of the persistent proof of the branch at this extension, and apply OPEN.

- CLOSE: in the same way, with the fresh z' and the renamed assumption; the mode is preserved by injective renaming.

The calls of the induction hypothesis concern proper subderivations, and the operations of Theorem 6.99 are constructed beforehand by the induction on rank, so the two recursions do not interact. No derivation of $\mathcal{A}$ is weakened, renamed or cut. The member at the identity extension is a derivation of $\Pi; \Gamma \vdash A$ in $\mathcal{A}$.

Every rule of $\mathcal{A}$ is an instance of a rule of $\mathcal{B}_\nu$, so $\mathcal{A} \subseteq \mathcal{B}_\nu$, and the remaining statements are Theorem 6.91. □

Remark 6.101 (Comparison and cost). Theorem 6.99 has the pattern of Theorem 6.75. There the transport at rank 0 is STEP, which opens no scope, so (T_0) has a local form. Here every operation at rank 0 opens an alias and is available only as a persistent proof; a local derivation of $F(b)$ obtained by USE_ν while b is newest is consumed as the major premise of OPEN before any newer parameter is introduced, as the local derivation at the case variable is in Section 7.5. The constructions (L_r) and (S_r) use OPEN with a compound conclusion; whether OPEN can be restricted as well is open (Section 6.11). The theorem answers the question of `alias_back` whether ALIAS can be restricted to equations while every clause is used at the newest parameter only. The compiled certificates of the two examples of `equation_aliases` with compound specialization at rank 1 and 2 have 1,308 and 5,302 bytes; no bound on the size of the translation is claimed.

6.8 A constancy rule in place of induction

This section draws on an appendix of `substitutive_discharge`, Section 6.8.4 on `boolean_constancy`, and Sections 6.8.5 and 6.8.6 on `scalar_constancy`. It is independent of the discharge rule: it presents an arithmetic whose one arithmetical inference has no induction hypothesis and no step function. The price is a primitive Boolean equality.

6.8.1 The theory

Definition 6.102 (The constancy theory)**.** The logic is intuitionistic first-order logic with equality over one sort. Terms are

$$t ::= x \mid 0 \mid S(t) \mid \delta(t,t) \mid R(n, a;\, i, u.\, h),$$

where the recursor binds i and u in h only. With $1 := S0$ and $s \neq t := (s = t \to \bot)$, the axioms are the universal closures of

$$R(0, a;\, i, u.\, h) = a, \tag{6.25}$$
$$R(Sn, a;\, i, u.\, h) = h[n/i,\ R(n, a;\, i, u.\, h)/u], \tag{6.26}$$
$$0 = 1 \to \bot, \tag{6.27}$$
$$\big(s = t \wedge \delta(s,t) = 0\big) \vee \big(s \neq t \wedge \delta(s,t) = 1\big), \tag{6.28}$$

the substitution in (6.26) being simultaneous, and the one further rule is

$$\frac{\Gamma \vdash f(Sz) = f(z)}{\Gamma \vdash f(t) = f(0)}\ \textsc{const} \tag{6.29}$$

for a term template $f(x)$, with z fresh for Γ, t and f. The context Γ consists of arbitrary first-order formulas. There is no coverage axiom, no injectivity axiom for successor, no induction schema and no assumed reflection of equality.

6.8.2 Recovery of quantifier-free induction

Define

$$\mathrm{sel}(v,a,b) := R(v,a;\, i,u.\, b), \qquad a+b := R(b,a;\, i,u.\, Su), \qquad pd(v) := R(v,0;\, i,u.\, i).$$

These are the counter-aware selector, addition and predecessor of Theorems 9.10 and 9.12, for a recursor whose value at the counter 0 is its initializer and not the successor of it: no outer decoder is needed, and each equation below is one computation step.

Lemma 6.103 (Elementary computations)**.** *Without* CONST*:* $\mathrm{sel}(0,a,b) = a$ *and* $\mathrm{sel}(Sv,a,b) = b$*;* $a+0=a$*,* $a+Sb = S(a+b)$ *and* $a+1=Sa$*;* $pd(0)=0$ *and* $pd(Sv)=v$*; successor is injective;* $Sv \neq 0$*; and*

$$\delta(s,t) = 0 \vee \delta(s,t) = 1, \qquad \delta(s,t) = 0 \leftrightarrow s = t. \tag{6.30}$$

Proof. The equations are instances of (6.25) and (6.26), and injectivity follows from $pd(Sv) = v$. If $Sv = 0$, applying $w \mapsto \mathrm{sel}(w,0,1)$ gives $1 = 0$, against (6.27); no distinction of cases on a number is used. The disjunction in (6.30) is read off (6.28). If $s = t$, the second disjunct of (6.28) contradicts it, so $\delta(s,t) = 0$. If $\delta(s,t) = 0$, the second disjunct would give $0 = 1$, so $s = t$. □

For a quantifier-free formula A define a term q_A, with the value 0 meaning that A holds:

$$\begin{aligned} q_{s=t} &:= \delta(s,t), & q_\bot &:= 1, \\ q_{A\wedge B} &:= \mathrm{sel}(q_A, q_B, 1), & q_{A\vee B} &:= \mathrm{sel}(q_A, 0, q_B), \\ q_{A\to B} &:= \mathrm{sel}(q_A, q_B, 0). \end{aligned}$$

Lemma 6.104 (Boolean terms)**.** *For every quantifier-free* A*, without* CONST*,*

$$q_A = 0 \vee q_A = 1, \qquad q_A = 0 \leftrightarrow A.$$

Proof. By induction on A in the metatheory, with the two computations of sel and (6.30). In the case of an implication distinguish the two values of q_A: if it is 0, then A holds and $A \to B$ is equivalent to B; if it is 1, then $\neg A$ follows and the implication holds. Only the decidability already proved for the quantifier-free antecedent is used. □

Theorem 6.105 (Quantifier-free induction from constancy)**.** *The theory of Theorem 6.102 derives*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma,\ A(z) \vdash A(Sz)}{\Gamma \vdash A(t)}$$

for every quantifier-free A *and every context* Γ *of first-order formulas,* z *fresh. In particular it derives induction for literal equations under an arbitrary context.*

Proof. Write $q(n) := q_{A(n)}$ and define the terms

$$e(n) := \mathrm{sel}\big(q(n), 0, \mathrm{sel}(q(Sn), 1, 0)\big), \qquad a(n) := R(n,0;\, i,u.\, u{+}e(i)), \qquad F(n) := a(n){+}q(n).$$

These are terms of the language: e and the addition are expanded before the outer recursor is formed. The computations give $a(0) = 0$ and $a(Sn) = a(n) + e(n)$. By Theorem 6.104 the pair $(q(n), q(Sn))$ has four possible values. Three of them give $F(Sn) = F(n)$ with no induction hypothesis:

$q(n)$	$q(Sn)$	$e(n)$	$F(Sn) = F(n)$
0	0	0	$a(n) = a(n)$
1	1	0	$Sa(n) = Sa(n)$
1	0	1	$Sa(n) = Sa(n)$

The fourth, $(0,1)$, is impossible under Γ: $q(n) = 0$ gives $A(n)$, the step premise gives $A(Sn)$, and Theorem 6.104 gives $q(Sn) = 0$, against $0 \neq 1$. By elimination of the finitely many disjunctions, $\Gamma \vdash F(Sz) = F(z)$. CONST gives $F(t) = F(0)$. The base $A(0)$ gives $q(0) = 0$, so $F(0) = 0$. If $q(t) = 1$, then $0 = F(t) = a(t) + 1 = Sa(t)$, against $Sv \neq 0$. Hence $q(t) = 0$, and $A(t)$ holds. □

The only identities of addition used are $a + 0 = a$ and $a + 1 = Sa$; commutativity and associativity are not used, so no induction is hidden in an algebraic rearrangement.

6.8.3 Both interpretations

Theorem 6.106 (The constancy theory and iPRA)**.** *The theory of Theorem 6.102 is bi-interpretable with* iPRA*, with whole domains, actual equality and both numerical composites provably the identity. In particular the two are equiconsistent.*

Proof. iPRA *in the constancy theory.* Theorem 6.105 applied to the quantifier-free formula $n = 0 \vee n = S(pd(n))$ gives coverage: the case 0 is immediate, and at a successor $pd(Sn) = n$ gives the second disjunct without the hypothesis. Injectivity of successor is in Theorem 6.103. Every primitive recursive definition compiles into the recursor with a counter, as in Theorem 9.12 with R in place of Rec: zero, successor, projections and composition are term operations, and $f(0, \vec{x}) = g(\vec{x})$, $f(Sn, \vec{x}) = h(n, f(n, \vec{x}), \vec{x})$ is represented by $R(n, g(\vec{x});\, i, u.\, h(i, u, \vec{x}))$ after g and h are compiled; the defining equations are instances of (6.25) and (6.26). With separation, injectivity and Theorem 6.105 this interprets iPRA.

The constancy theory in iPRA. Each fixed recursor term denotes a primitive recursive function, and its two equations hold by construction. CONST is admissible: under its premise, induction for the equation $f(n) = f(0)$ gives the conclusion. For δ an equality characteristic is constructed as follows. Let

$$d(a, b) := R(b, a;\, i, u.\, pd(u)).$$

The computation equations and induction for equations give

$$d(a, 0) = a, \quad d(a, Sb) = pd(d(a, b)), \quad d(Sa, Sb) = d(a, b), \quad d(Sn, n) = 1;$$

the third by induction on b at fixed a, with the base $pd(Sa) = a$ and the step obtained by applying pd to the preceding equation, the fourth by induction on n from the third. Inductions on the second argument of addition give $0 + b = b$ and $Sa + b = S(a + b)$. Quantifier-free induction on b, with a as a parameter, proves

$$d(a, b) \neq 0 \;\rightarrow\; a = b + d(a, b). \tag{6.31}$$

The base uses $0 + a = a$. For the step put $r := d(a, b)$ and suppose $pd(r) = d(a, Sb) \neq 0$. Coverage for r gives $r = S(pd(r))$, the case $r = 0$ contradicting $pd(r) \neq 0$. So $r \neq 0$, and the induction hypothesis gives $a = b + r = b + S(pd(r)) = Sb + pd(r)$. Now define

$$Z(a) := \mathrm{sel}(a, 1, 0), \qquad U(a) := \mathrm{sel}(a, 0, Z(pd(a))), \qquad E(a, b) := \mathrm{sel}(b, Z(a), U(d(a, pd(b)))).$$

Coverage and separation give that $Z(a)$ and $U(a)$ are 0 or 1, that $Z(a) = 1$ exactly if $a = 0$, and that $U(a) = 1$ exactly if $a = 1$. If $b = 0$, then $E(a, b) = 1$ is equivalent to $a = 0$. If $b = Sn$, then $E(a, b) = 1$ is equivalent to $d(a, n) = 1$, which implies $a = n + 1 = Sn = b$ by (6.31), and conversely $a = Sn$ gives $d(Sn, n) = 1$. So $E(a, b)$ is 0 or 1 and equals 1 exactly if $a = b$, and

$$\delta_*(a, b) := Z(E(a, b))$$

is 0 or 1 and equals 0 exactly if $a = b$. It satisfies (6.28). This derivation uses quantifier-free inductions of iPRA; it is not part of the bootstrap of Theorem 6.105. It is a variant of the reflection argument recorded in Theorem 9.28, with the control d and the test E in place of the sign and the reflection term used there.

Composites. In the constancy theory, induction for equations (Theorem 6.105) proves the uniqueness of every fixed primitive recursive definition under arbitrary parameters; by induction on terms the round trip of every recursor term has the value of the original. The same argument in iPRA treats the round trip of each primitive recursive function symbol. For the primitive δ, both δ and the translated δ_* satisfy (6.28) in the constancy theory, and the distinction of the two cases of the first shows $\delta(a, b) = \delta_*(a, b)$: both are 0 if $a = b$ and both are 1 otherwise. Congruence and induction on formulas give both formula round trips. The numerical isomorphisms have graph $x = y$. □

Remark 6.107 (What the rule does and does not simplify). The arithmetical inference concerns one term and one unconditional recurrence with the identity as step; it has no discharged hypothesis and no step template, and the accumulator $a(n)$ is a construction of the proof and not of the rule. Against the kernel the presentation adds a binary term constructor, a primitive equality split in place of coverage, and a second bound variable of the recursor. The split (6.28) is essential to the proof above: replacing it by the assertion that an internally defined equality test is faithful, without proving that assertion in the induction-free base, would be circular. Likewise, for a tail-recursive iterator the equation $d(a, Sb) = d(pd(a), b)$ does not by itself give $d(a, Sb) = pd(d(a, b))$; the missing commutation is a fusion property. No checker is supplied for this theory, and nothing is claimed about its size.

6.8.4 Constancy for Boolean tests

For a term template $g(x, \vec{p})$ put $b_g(x, \vec{p}) := \delta\big(g(x, \vec{p}), 1\big)$. The rule

$$\frac{\Gamma \vdash b_g(Sz, \vec{p}) = b_g(z, \vec{p})}{\Gamma \vdash b_g(t, \vec{p}) = b_g(0, \vec{p})}\ \textsc{bconst}, \tag{6.32}$$

with z fresh for Γ, t and the parameters $\vec{p}$, is the instance of CONST (6.29) for the templates $f \equiv b_g$, whose outermost constructor is the equality test against the numeral 1. The restriction is syntactic; it does not assert that an arbitrary term is Boolean. The *Boolean constancy theory* is the theory of Theorem 6.102 with BCONST in the place of CONST. Theorems 6.103 and 6.104 do not use CONST and hold in it.

Proposition 6.108 (Boolean tests suffice)**.** *Over the axioms* (6.25)–(6.28)*, under every context of first-order formulas:*

(a) *BCONST derives CONST;*

(b) *for every quantifier-free $A(n)$, BCONST derives the induction rule of Theorem 6.105 with one application of BCONST, by the accumulator*

$$a(n) := R\big(n, 1;\, i, u.\ \mathrm{sel}(q(i), u, 0)\big), \qquad F(n) := \mathrm{sel}\big(q(n), a(n), 0\big), \qquad q(n) := q_{A(n)}.$$

Consequently BCONST, CONST and quantifier-free induction are interderivable over this base, the Boolean constancy theory and the theory of Theorem 6.102 have the same theorems in the same language, and the Boolean constancy theory is bi-interpretable and equiconsistent with iPRA*, with whole domains, actual equality and both numerical composites provably the identity.*

Proof. All case distinctions below are eliminations of the disjunctions of Theorems 6.103 and 6.104; none is a distinction of cases on a number, and no induction is used.

(a) Let $\Gamma \vdash f(Sz) = f(z)$, and put $q(n) := \delta(f(n), f(0))$ and $g(n) := \mathrm{sel}(q(n), 1, 0)$. Congruence gives $q(Sz) = q(z)$, hence $g(Sz) = g(z)$ and $b_g(Sz) = b_g(z)$, with z fresh for the parameters of f, which are those of g. By (6.30), $q(0) = 0$, so $g(0) = 1$ and $b_g(0) = \delta(1, 1) = 0$. BCONST gives $\delta(g(t), 1) = 0$, and (6.30) gives $g(t) = 1$. If $q(t) = 1$, then $g(t) = \mathrm{sel}(1, 1, 0) = 0$ and $0 = 1$, against (6.27). So $q(t) = 0$, that is, $f(t) = f(0)$.

(b) Let $\Gamma \vdash A(0)$ and $\Gamma, A(z) \vdash A(Sz)$, the parameters of A being suppressed. The bound variables i, u do not occur in q, and the computations (6.25), (6.26) give

$$a(0) = 1, \qquad a(Sn) = \mathrm{sel}(q(n), a(n), 0) = F(n), \qquad F(Sn) = \mathrm{sel}\big(q(Sn), F(n), 0\big).$$

Stationarity, with no induction hypothesis: if $q(z) = 0$, then $A(z)$ by Theorem 6.104, the step gives $A(Sz)$, so $q(Sz) = 0$ and $F(Sz) = \mathrm{sel}(0, F(z), 0) = F(z)$. If $q(z) = 1$, then $F(z) = \mathrm{sel}(1, a(z), 0) = 0$, and $F(Sz) = \mathrm{sel}(q(Sz), 0, 0) = 0$ in both cases $q(Sz) = 0$ and $q(Sz) = 1$. So $\Gamma \vdash F(Sz) = F(z)$, and congruence gives $\Gamma \vdash b_F(Sz) = b_F(z)$. *Conclusion*: BCONST with $g \equiv F$ gives $\delta(F(t), 1) = \delta(F(0), 1)$. The base gives $q(0) = 0$, so $F(0) = a(0) = 1$ and $\delta(F(0), 1) = 0$; hence $F(t) = 1$ by (6.30). If $q(t) = 1$, then $F(t) = 0$ and $0 = 1$, against (6.27). So $q(t) = 0$, and $A(t)$ holds.

BCONST is an instance of CONST, and quantifier-free induction derives CONST by induction on the equation $f(n) = f(0)$, as in the proof of Theorem 6.106. So the three rules are interderivable, and the two theories, which have the same language and the same axioms, have the same theorems. The interpretations of Theorem 6.106 and the proofs of their composites therefore apply unchanged. □

Part (a) together with Theorem 6.105 already gives the equivalence; part (b) is a direct recovery of induction that replaces the proof of Theorem 6.105. Its accumulator absorbs the value 0: once a test fails, F stays 0, and while the tests hold it carries the initial value 1. Stationarity is a finite propositional case argument; it needs no statement that $a(n)$ is Boolean and no identity of addition. The proof of Theorem 6.105 instead adds a flag to an accumulator counting the downward changes of the test and excludes $Sa(t) = 0$.

Remark 6.109 (Stationarity over an interface). Theorem 10.130 proves the same equivalence between stationarity, which is CONST with passive parameters, and quantifier-free induction, over a different base: full primitive recursive arithmetic with coverage and with assumed truth flags χ_A. Its accumulator β is clamped, and a case split on n by coverage proves the Boolean invariant $(*)$ there before stationarity can be shown. Over (6.25)–(6.28) coverage is not available before induction is recovered, and the truth flags are the terms q_A of Theorem 6.104, derived from the split. The selector accumulator of Theorem 6.108(b) needs neither the coverage nor an invariant, and its rule is applied only to Boolean tests. The remark following Theorem 10.130 concerns the installation of the interface in the scalar kernel; it does not apply here, because the split (6.28) is an axiom of the base and is not derived from a reflection theorem proved with induction.

`boolean_constancy` supplies no checker for the Boolean constancy theory, and nothing is claimed about the size of one; its programs check the native kernel. Neither the proposition nor the theory is formalized in Lean.

6.8.5 A scalar iterator with one comparison equation

This subsection draws on `scalar_constancy`. The recursor of Theorem 6.102, whose body sees the old counter and the old state, is replaced by the iterator of Theorem 1.1, whose body sees only the state. The counter is paid for by one unconditional equation for the Boolean test.

Definition 6.110 (Scalar constancy)**.** The theory SC is formulated in intuitionistic first-order logic with equality over the terms

$$t ::= x \mid 0 \mid S(t) \mid \delta(t,t) \mid I(n,a;\, u.\, h),$$

where the iterator binds u in h only, as in Theorem 1.1. Its axioms are the universal closures of the computation equations (1.1) and (1.2) for all these terms, of the separation (6.27), of the split (6.28), and of the *comparison equation*

$$\delta(Ss, St) = \delta(s,t). \tag{6.33}$$

Its one further rule is CONST (6.29), for term templates f of this language, with z fresh for Γ, t and the parameters of f; contexts consist of arbitrary first-order formulas. The theory SC^- is SC without (6.33). Neither theory has an induction schema, a coverage axiom or an injectivity axiom.

The binder is a finite term constructor. Equivalently the language is a staged first-order signature: for each body $h(u, \vec{p})$ formed at an earlier stage there is a function symbol $I_h(n, a, \vec{p})$ with its two computation equations. No variable ranges over functions, and every formula and derivation uses finitely many of these symbols. The structures of Section 6.8.6 are structures for this signature. The selector $\mathrm{sel}(c, a, b) = I(c, a;\, v.\, b)$, v not in b, is that of the common core, and q_A is defined from it as before Theorem 6.104.

Lemma 6.111 (Elementary facts)**.** (a) *Without* CONST, SC^- *proves* $\mathrm{sel}(0,a,b) = a$, $\mathrm{sel}(Sv,a,b) = b$, $Sv \neq 0$, (6.30)*, and the two statements of Theorem 6.104 for every quantifier-free* A*.*

(b) *Over* SC^-*, without* CONST*, the comparison equation* (6.33) *and the injectivity* $Ss = St \to s = t$ *are interderivable.*

Proof. (a) The selector equations are instances of (1.1) and (1.2). The proofs of $Sv \neq 0$ and of (6.30) in Theorem 6.103, and the proof of Theorem 6.104, use only these selector equations, (6.27) and (6.28).

(b) Assume (6.33) and $Ss = St$. Then $\delta(Ss, St) = 0$ by (6.30), so $\delta(s,t) = 0$, and $s = t$ by (6.30). Conversely assume injectivity and eliminate the split (6.28) at (s,t). If $s = t$ and $\delta(s,t) = 0$, then $Ss = St$, so $\delta(Ss, St) = 0 = \delta(s,t)$. If $s \neq t$ and $\delta(s,t) = 1$, then $Ss = St$ would give $s = t$, so $Ss \neq St$, and the split at (Ss, St) gives $\delta(Ss, St) = 1 = \delta(s,t)$, its first disjunct being contradictory. Only eliminations of disjunctions are used. □

The rule SR_0 of Section 5.8 concludes $\Gamma \vdash f(t) = t$ from $\Gamma \vdash f(0) = 0$ and $\Gamma \vdash f(Sz) = Sf(z)$, z fresh for Γ, t and the parameters of f.

Lemma 6.112 (Successor rigidity from constancy)**.** SC *derives* SR_0*, with one application of* CONST*.*

Proof. Put $d(n) := \delta(f(n), n)$; its parameters are those of f. Under Γ, congruence and (6.33) give

$$d(Sz) = \delta(Sf(z), Sz) = \delta(f(z), z) = d(z).$$

CONST gives $d(t) = d(0) = \delta(f(0), 0) = \delta(0,0) = 0$, the last step by (6.30). Hence $f(t) = t$ by (6.30). □

Theorem 6.113 (Quantifier-free induction from scalar constancy)**.** SC *derives the contextual induction rule of Theorem 6.105 for every quantifier-free* A *and every context* Γ *of first-order formulas, with one application of* CONST*. The comparison equation enters only through Theorem 6.112.*

Proof. Let $\Gamma \vdash A(0)$ and $\Gamma, A(z) \vdash A(Sz)$ with z fresh; the parameters of A are suppressed. Put $q(n) := q_{A(n)}$ and

$$e(n) := \mathrm{sel}\big(q(n), n, 0\big),$$

and let x be a variable not free in Γ and A. By Theorem 6.111(a), $q(x) = 0 \vee q(x) = 1$. If $q(x) = 0$, then $A(x)$ and $e(x) = x$, so $A(e(x))$; if $q(x) = 1$, then $e(x) = 0$, and $\Gamma \vdash A(0)$ gives $A(e(x))$. Moreover $A(x)$ gives $q(x) = 0$ and so $e(x) = x$. Hence

$$\Gamma \vdash A(e(x)), \qquad \Gamma,\ A(x) \vdash e(x) = x,$$

and, x not being free in Γ, both hold with an arbitrary term in place of x. Define, with u fresh,

$$r(n) := I(n, 0;\, u.\, S(e(u))), \qquad f(n) := e(r(n)),$$

and let w be fresh. The second property at 0 and the base give $f(0) = e(0) = 0$. The first property at $r(w)$ gives $A(f(w))$, the step instantiated at $f(w)$ gives $A(Sf(w))$, and the second property at $Sf(w)$ gives $e(Sf(w)) = Sf(w)$. With (1.2),

$$f(Sw) = e\big(S(e(r(w)))\big) = e\big(Sf(w)\big) = Sf(w).$$

Theorem 6.112 gives $\Gamma \vdash f(t) = t$. The first property at $r(t)$ gives $A(f(t))$, and the Leibniz rule gives $A(t)$. □

For a parameter a define

$$F_a(u) := \mathrm{sel}\big(\delta(Su, a), u, Su\big), \qquad O_a(n) := I(n, 0;\, u.\, F_a(u)), \qquad P(a) := O_a(a),$$

so that $P(a) \equiv I(a, 0;\, u.\, I(\delta(Su, a), u;\, v.\, Su))$. The orbit O_a holds its state exactly when the successor of the state is the target a.

Proposition 6.114 (A predecessor and coverage)**.** *Over* SC^- *together with the contextual induction rule for quantifier-free formulas:*

(a) $O_a(n) = n \ \vee \ S(O_a(n)) = a$*;*

(b) $n = 0 \ \vee \ n = S(P(n))$*, and hence coverage* $n = 0 \vee \exists m\,(n = Sm)$*;*

(c) *if successor is injective, then* $P(0) = 0$ *and* $P(Sb) = b$*.*

Proof. (a) Induction on n with a fixed. At 0 the first disjunct holds by (1.1). For the step, $O_a(Sn) = F_a(O_a(n))$, and $\delta(SO_a(n), a)$ is 0 or 1. If it is 0, then $SO_a(n) = a$ and $F_a(O_a(n)) = O_a(n)$, so $SO_a(Sn) = a$. If it is 1, then $SO_a(n) \neq a$, the second disjunct of the hypothesis is refuted, so $O_a(n) = n$, and $O_a(Sn) = SO_a(n) = Sn$.

(b) Induction on n; the base is the first disjunct. The step needs no hypothesis: $P(Sn) = F_{Sn}(O_{Sn}(n))$, and (a) at $a := Sn$ gives two cases. If $O_{Sn}(n) = n$, then $F_{Sn}(n) = \mathrm{sel}(\delta(Sn, Sn), n, Sn) = n$, since $\delta(Sn, Sn) = 0$. If $SO_{Sn}(n) = Sn$, then the test is 0 and $P(Sn) = O_{Sn}(n)$. In both cases $SP(Sn) = Sn$. Coverage follows by $\exists$-introduction.

(c) $P(0) = O_0(0) = 0$ by (1.1). By (b) at Sb, either $Sb = 0$, which contradicts $Sv \neq 0$, or $Sb = SP(Sb)$, and injectivity gives $P(Sb) = b$. □

The term $P(a)$ has eleven nodes when each variable occurrence, constant, successor, test and iterator counts once and bound names are not counted. It uses the primitive test and its axioms, so it is not comparable with the predecessors of Theorem 1.65, which are built from the iterator alone.

Proposition 6.115 (Four equivalent completions)**.** *Over* SC^- *the following are interderivable: the comparison equation* (6.33)*; injectivity of successor; the rule* SR_0*; and the contextual induction rule for quantifier-free formulas. Only the passage from* (6.33) *to* SR_0 *uses* CONST*. In particular* SC *contains* T_{eq}*: every theorem of* T_{eq} *is a theorem of* SC*.*

Proof. (6.33) gives SR_0 by Theorem 6.112. The proof of Theorem 6.113 uses (6.33) only through SR_0, so SR_0 gives quantifier-free induction. Under quantifier-free induction every axiom of T_{eq} (Theorem 1.49) is a theorem: (T1) and (T3) are axioms, (T2) is Theorem 6.114(b), and (T4) follows from the rule with $\Gamma := \{E(0), \forall z\,(E(z) \to E(Sz))\}$, a fresh conclusion variable, $\forall$-introduction and two $\to$-introductions. Both theories use the same logic, and the language of T_{eq} is the part of the present language without δ, so every theorem of T_{eq} is a theorem; Theorem 1.84 gives injectivity. Injectivity gives (6.33) by Theorem 6.111(b). □

Theorem 6.116 (Scalar constancy and iPRA)**.** SC *and* iPRA *are bi-interpretable, with whole domains, actual equality and both numerical composites provably the identity, of graph* $x = y$*. In particular the two are equiconsistent.*

Proof. iPRA *in* SC. By Theorem 6.115, SC contains T_{eq}. The proof of Theorem 1.115 uses only theorems of T_{eq}, so σ is a term interpretation of iPRA in SC.

SC *in* iPRA. Let ε be the term of Theorem 1.86 and $\kappa(a,b) := \mathrm{sel}(\varepsilon(a,b),0,1)$. T_{eq} proves the split (6.28) for κ in the place of δ: by coverage on $\varepsilon(a,b)$, if it is 0, then $\kappa(a,b) = 0$ and $a = b$ by Theorem 1.86; if it is Sz, then $\kappa(a,b) = 1$, and $a = b$ would give $Sz = 0$ by Theorem 1.86, against Theorem 1.84. Extend the symbols $[\cdot]_{\vec{x}}$ and the translation τ of Theorem 1.111 by

$$[\delta(s,t)]_{\vec{x}} := \mathrm{comp}\big([\kappa]_{a\,b}; [s]_{\vec{x}}, [t]_{\vec{x}}\big), \qquad \tau(\delta(s,t)) := [\kappa]_{a\,b}\big(\tau(s), \tau(t)\big).$$

The proofs of Theorems 1.112 to 1.114 are inductions on terms, and the new case is the composition axiom; so τ interprets the computation equations for all bodies, and the separation. By Theorem 1.113(i), $\mathsf{iPRA} \vdash [\kappa]_{a\,b}(a,b) = \tau(\kappa(a,b))$, and by Theorem 1.119(ii) iPRA proves the image of the split for κ; this is the image of (6.28). The comparison equation follows from the split and injectivity in iPRA, by the argument of Theorem 6.111(b). CONST is admissible: under Γ^τ, with Theorem 1.113(ii), quantifier-free induction on a fresh n in the equation $\tau(f)(n) = \tau(f)(0)$ has a trivial base, and its step combines the premise at n with the hypothesis. Connectives, quantifiers and equality are translated identically.

Composites. $\mathsf{iPRA} \vdash \tau(\sigma(t)) = t$ is Theorem 1.117(ii), since $\sigma(t)$ contains no δ. For $\mathsf{SC} \vdash \sigma(\tau(t)) = t$, the two inductions on terms in the proof of Theorem 1.117(i) use the computation equations, congruence, Theorem 1.55 and Theorem 1.56(ii), whose proofs are inductions for equations; these hold in SC for all its terms by Theorem 6.113. The new case of both inductions is the equation $\ulcorner[\kappa]_{a\,b}\urcorner(a,b) = \delta(a,b)$, with congruence and the induction hypotheses for s and t. The first induction of that proof, applied to the term κ of T_{eq}, gives $\ulcorner[\kappa]_{a\,b}\urcorner(a,b) = \kappa(a,b)$. Both κ and δ satisfy the split in SC; eliminating the split for δ, if $a = b$ both values are 0, and if $a \neq b$, then $\delta(a,b) = 1$ and the first disjunct of the split for κ is contradictory, so $\kappa(a,b) = 1$. Induction on formulas gives both formula round trips, and both numerical isomorphisms have graph $x = y$. Consistency transfers in both directions, since $\bot$ is translated to $\bot$. □

Remark 6.117 (Constancy restricted to comparisons with the counter)*.* The recovery of induction applies CONST only to templates $\delta(f(n),n)$ (Theorem 6.112). The same theory therefore results if CONST is replaced by

$$\frac{\Gamma \vdash \delta(f(Sz),Sz) = \delta(f(z),z)}{\Gamma \vdash \delta(f(t),t) = \delta(f(0),0)}, \tag{6.34}$$

with the same freshness conditions: Theorem 6.113 holds with (6.34), and quantifier-free induction then derives CONST by induction on $f(n) = f(0)$. The restriction is syntactic, like BCONST (6.32), whose templates compare with the numeral 1 instead of the counter. Full CONST has the shorter statement, and no measurement favors either.

Remark 6.118 (Relation to the other derivations of induction). Against Theorem 6.102, the scalar theory has one bound variable fewer in the arithmetic term constructor and one more nonlogical equation, (6.33); it is not a deletion of an axiom, and Section 6.8.6 shows that the equation cannot be omitted. The induction proofs differ accordingly. Theorem 6.105 and Theorem 6.108(b) build accumulators with a recursor that reads the counter; Theorem 6.113 uses the retraction e and the trajectory r of Theorem 5.79 (`raw_clausal`) and of Theorem 2.80, with SR_0 obtained from CONST by comparing the trajectory with the identity. Theorem 5.79 assumes coverage to prove $E(r(n))$ for $r(n) = I(n, 0;\, u.\, e(Su))$; here the projection $f = e \circ r$ satisfies $A(f(n))$ outright, so coverage is not needed before it is derived in Theorem 6.114. The pairing and the compiler of primitive recursion are those of Theorem 1.102; `scalar_constancy` restates them in an appendix with the coding $\langle a, b\rangle = 2^{a+b} + a$ and the derived recursor $\pi_1\big(I(n, \langle 0, a\rangle;\, w.\, H(w))\big)$, which is the construction of the common core. A finite script of the package checks instances of the predecessor, its orbit invariant, the comparison equation and the equivariance of Theorem 6.121; these instances are not proofs. Nothing in this subsection is formalized in Lean.

6.8.6 Omitting the comparison equation

This subsection draws on `scalar_constancy`. It applies the symmetry method of Section 10.3 to an explicit model of SC^{-}. The obstruction concerns this chapter's theory and is stated here; its two general lemmas, Theorems 10.20 and 10.21, are cited. Besides (1.1)–(1.2) the following principles are considered, each read under an arbitrary context, with the freshness conditions stated with FUSION before Theorem 10.90:

- the *clock* scheme
$$I(t, 0;\, u.\, Su) = t; \tag{6.35}$$
- FUSION, the rule displayed before Theorem 10.90;
- *fixed points*: from $\Gamma \vdash h(a) = a$ infer $\Gamma \vdash I(t, a;\, u.\, h(u)) = a$;
- *seed shift*: $h\big(I(t, a;\, u.\, h(u))\big) = I(t, h(a);\, v.\, h(v))$;
- *extensionality*: from $\Gamma \vdash h(u) = h'(u)$, u not free in Γ, infer $\Gamma \vdash I(t, a;\, u.\, h) = I(t, a;\, u.\, h')$.

A rule is *valid* in a classical structure if, for every assignment that satisfies Γ, the premise for all values of the eigenvariable implies the conclusion. A classical structure in which every axiom holds and every rule is valid satisfies every theorem; intuitionistic derivability is sound for it.

Definition 6.119 (Three copies)**.** Let $M := \mathbb{N} \times \{0, 1, 2\}$, with *level* $\ell(n, c) := n$ and $\iota(n) := (n, 0)$. Put $0^M := \iota(0)$, $S^M(x) := \iota(\ell(x) + 1)$, and $\delta^M(x, y) := \iota(0)$ if $x = y$ and $\iota(1)$ otherwise; equality is identity. For an assignment ρ, terms are evaluated by recursion on their structure, the iterator by

$$[\![I(n, a;\, u.\, h)]\!]_\rho := \mathcal{I}\big([\![n]\!]_\rho, [\![a]\!]_\rho, G\big), \qquad G(v) := [\![h]\!]_{\rho[u \mapsto v]},$$

so that the body, a proper subterm, is evaluated first. The structure $\mathcal{M}$ has $\mathcal{I}(x,a,G) := G^{\ell(x)}(a)$. The structure $\mathcal{M}^{\mathrm{c}}$ has

$$\mathcal{I}(x,a,G) := \begin{cases} x, & \text{if } a = 0^M \text{ and } G = S^M \text{ as maps on } M, \\ G^{\ell(x)}(a), & \text{otherwise.} \end{cases}$$

Every exponent is a natural number, so nested iterators need no further construction. The condition $G = S^M$ is a condition of the metatheory, not an operation of the language.

Lemma 6.120 (Validity in the three copies)**.** (a) *The computation equations,* (6.27)*,* (6.28) *and* CONST *are valid in $\mathcal{M}$ and in $\mathcal{M}^{\mathrm{c}}$.* CONST *is valid in the strong form: every map $F : M \to M$ with $F(S^M x) = F(x)$ for all x is constant.*

(b) *Fixed points, seed shift and extensionality are valid in both structures;* FUSION *is valid in $\mathcal{M}$, and the clock* (6.35) *holds in $\mathcal{M}^{\mathrm{c}}$.*

(c) *In both structures successor is not injective and coverage fails. In $\mathcal{M}$ the clock fails, and with it an instance of the contextual induction rule for a literal equation whose base and step are derivable in SC^-.*

Proof. (a) Since $\ell(0^M) = 0$ and $\ell(S^M x) = \ell(x) + 1$, both equations hold for the ordinary case of $\mathcal{I}$. In the exceptional case of $\mathcal{M}^{\mathrm{c}}$ the seed and the update are 0^M and S^M on both sides, and $\mathcal{I}(0^M, 0^M, S^M) = 0^M$ and $\mathcal{I}(S^M x, 0^M, S^M) = S^M x = S^M(\mathcal{I}(x, 0^M, S^M))$. The split and the separation hold by the definition of δ^M, because $1^M = \iota(1) \neq \iota(0)$. If $F(S^M x) = F(x)$ for all x, then for $x = (n, c)$, $F(x) = F(\iota(n+1))$, and $n+1$ external steps along $\iota(k+1) = S^M \iota(k)$ give $F(\iota(n+1)) = F(\iota(0))$. For CONST, fix an assignment that satisfies Γ and apply this to $F(v) := [\![f(z)]\!]_{\rho[z \mapsto v]}$; the eigenvariable is fresh, so the premise holds for all v.

(b) In $\mathcal{M}$ the laws follow from $\mathcal{I}(x,a,G) = G^k(a)$, $k = \ell(x)$, by external induction on k: $G(a) = a$ gives $G^k(a) = a$; $G(G^k(a)) = G^k(G(a))$; and $F \circ G = G' \circ F$ gives $F(G^k(a)) = G'^k(F(a))$, which is FUSION. Extensionality holds in both structures, since $\mathcal{I}$ depends on the denoted map G only. In $\mathcal{M}^{\mathrm{c}}$, (6.35) is the exceptional case. For fixed points, $G(a) = a$ excludes the exceptional case, since S^M has no fixed point, and ordinary iteration applies. For seed shift, the two sides are computed by ordinary iteration unless $G = S^M$ and $a = 0^M$; the seed $S^M a$ is never 0^M. In that case, for $x = (k, c)$, the left side $S^M \mathcal{I}(x, 0^M, S^M) = S^M x$ and the right side $(S^M)^k(\iota(1))$ both equal $\iota(k+1)$.

(c) $S^M(n,1) = S^M(n,2)$ while $(n,1) \neq (n,2)$. The point $(0,1)$ is not 0^M and, having color 1, is not a successor. Let $E(x)$ be $I(x, 0;\, u.\, Su) = x$. Its base is an instance of (1.1), and its step follows from (1.2) and the Leibniz rule. In $\mathcal{M}$, $[\![I(x, 0;\, u.\, Su)]\!] = \iota(0) \neq (0,1)$ at $x \mapsto (0,1)$. □

By Theorem 6.115 and Theorem 6.120(c), SC^- derives neither (6.33), nor injectivity, nor SR_0, nor quantifier-free induction.

Lemma 6.121 (An involution at one level)**.** *For $m \in \mathbb{N}$ let σ_m exchange $(m,1)$ and $(m,2)$ and fix every other point. In $\mathcal{M}$ and in $\mathcal{M}^{\mathrm{c}}$, for every term t and every assignment ρ, $\sigma_m([\![t]\!]_\rho) = [\![t]\!]_{\sigma_m \circ \rho}$. Hence σ_m is an automorphism of both structures in the staged signature, and it preserves every formula.*

Proof. Write σ for σ_m; it is an involution, preserves levels and fixes every point of color 0. Induction on t. Variables are immediate, and 0^M is fixed. $S^M(\sigma x) = \iota(\ell(x)+1) = S^M x = \sigma(S^M x)$, so σ commutes with S^M. σ is injective and fixes $\iota(0)$ and $\iota(1)$, so it commutes with δ^M. For an iterator let G and G' be the update maps of ρ and $\sigma \circ \rho$. The induction hypothesis for h at $\rho[u \mapsto v]$ gives $\sigma(G(v)) = G'(\sigma v)$, so $G' = \sigma G \sigma^{-1}$; the hypotheses for n and a give

their values at $\sigma \circ \rho$ as σx and σa. In the ordinary case $G'^{\ell(\sigma x)}(\sigma a) = \sigma(G^{\ell(x)}(a))$, since $\ell(\sigma x) = \ell(x)$. In $\mathcal{M}^{\mathrm{c}}$ the exceptional condition is preserved and reflected: $\sigma a = 0^M$ if and only if $a = 0^M$, and $\sigma G \sigma^{-1} = S^M$ if and only if $G = \sigma^{-1} S^M \sigma = S^M$; in the exceptional case the value x becomes σx. Induction on formulas transfers the statement to satisfaction. □

Theorem 6.122 (Omitting the comparison equation)**.** *Let U be SC^- together with any of fixed points, seed shift, extensionality and* FUSION*, or SC^- together with the clock* (6.35) *and any of fixed points, seed shift and extensionality. Then U and* iPRA *are not bi-interpretable in the sense of Theorem 10.1: arbitrary definable domains, quotient equality and finite dimension are allowed, as are finite parameter tuples, provided the interpretations and the return isomorphism exist in $\mathcal{M}$, respectively $\mathcal{M}^{\mathrm{c}}$; this covers parameter conditions that U proves to be inhabited.*

Proof. By Theorem 6.120, $\mathcal{M}$ satisfies every theorem of a theory U of the first kind and $\mathcal{M}^{\mathrm{c}}$ every theorem of one of the second kind; write $\mathcal{M}_U$ for the structure. Suppose a bi-interpretation existed. Applied in $\mathcal{M}_U$, the interpretation of iPRA yields a classical model $\mathcal{N}$ of iPRA. Let $\vec{c}$ collect the parameters of that interpretation, of the source-side return isomorphism, and representatives in $\mathcal{M}_U$ of the parameters of the reverse interpretation, and choose m larger than every level of a component of $\vec{c}$. By Theorem 6.121, σ_m is an automorphism of $\mathcal{M}_U$ that fixes $\vec{c}$ and is not the identity. By Theorem 10.21 it induces an automorphism $\widehat{\sigma}_m$ of $\mathcal{N}$, and the induced homomorphism is injective. The automorphism $\widehat{\sigma}_m$ has order dividing two and preserves the parameter-free definable order of Theorem 10.20, so it is the identity. Hence σ_m is the identity, a contradiction. □

Remark 6.123 (Why the second model fails fusion)*.* In $\mathcal{M}^{\mathrm{c}}$ let $a := (k, 1)$, $b := (k, 2)$ be the values of two parameters, and

$$p(v) := \mathrm{sel}\big(\delta(v, a),\, b,\, \mathrm{sel}(\delta(v, b), a, v)\big).$$

A constant update is not S^M, so the selector has its two values at the counts 0^M and 1^M, and p exchanges a and b and fixes every other point. It fixes 0^M and commutes with S^M, since successors have color 0 and p preserves levels. The premise $p(Sz) = Sp(z)$ of FUSION, with both updates Su and the seed 0, therefore holds for every z, while its conclusion $p(I(t, 0;\, u.\, Su)) = I(t, p(0);\, v.\, Sv)$ reads $p(t) = t$ by (6.35) and fails at $t \mapsto a$. This failure is necessary. Over SC^- with (6.35), FUSION derives SR_0: from $\Gamma \vdash f(0) = 0$ and $\Gamma \vdash f(Sz) = Sf(z)$ it gives $f(I(t, 0;\, u.\, Su)) = I(t, f(0);\, v.\, Sv) = I(t, 0;\, v.\, Sv)$, and (6.35) on both sides gives $f(t) = t$. By Theorem 6.115 the theory then contains SC. Every principle listed above is derivable in SC by induction for equations, so the theory has the theorems of SC and is bi-interpretable with iPRA by Theorem 6.116. Thus $\mathcal{M}$ separates FUSION from the clock, and $\mathcal{M}^{\mathrm{c}}$ the clock from FUSION; together they recover the comparison equation.

The obstruction does not exclude equiconsistency or one-way interpretations; iPRA interprets SC^- through Theorem 6.116. It does not show that successor injectivity is the only principle whose addition to SC^- yields a bi-interpretation on another domain, and it does not concern constancy over the scalar interface with coverage and without a primitive test, which is item 4 of Theorem 9.159.

6.9 Checkers and program equivalences

This section summarizes the programs of `witness_discharge`, `inferred_witness`, `deterministic_opening`, `first_instance`, `clause_discharge`, `substitutive_discharge`, `variable_substitution`, `parameter_substitution`, `deterministic_specialization`, `alias_back` and `equation_aliases`. Sizes are in Section 6.10.

Common contract. Every checker is a standalone Python module that reads one JSON value, verifies a closed theorem, prints `ACCEPT` and exits with status zero on success, and rejects on any exception; no decision depends on an `assert`. The checkers of `variable_substitution` and `deterministic_specialization` code syntax by tagged lists. The others use integers and bare lists: 0 is zero, a negative integer is a free parameter level, a positive integer is a de Bruijn index, `[0,t]` is a successor, `[n,a,h]` an iterator or recursor binding one variable in its last entry, `[s,t]` an equation and `[B]` a paired formula in whose matrix index 1 is y and index 2 is x; the expected sort distinguishes a successor from an equation. Subterms are validated before a contraction can erase them. Inserted terms have no loose positive index, so insertion below a binder needs no lifting. The theorems of this chapter concern open sequents; the public interfaces check closed goals.

witness_discharge. A goal-directed checker of $\mathcal{W}^{\mathcal{R}}$ with three packets, for USE (clause index, term, premise), INTRO (witness, branch) and NAME (matrix, local formula, target, major premise, equality premise, branch). The exported formula is compared with the already validated outer goal, which enforces the absence of the eigenparameter also below binders and after normalization has erased an occurrence.

inferred_witness. A forward checker of $\mathcal{O}^{\mathcal{R}}$ that infers every conclusion once. It stores the antecedent $B(b, 0)$ of the opened clause as computed and substitutes the selected witness in the body only: if the test succeeds the antecedent is H and is unchanged by the substitution, and if it fails the substitution is the identity, so the stored clause is $\mathcal{C}_B(\rho_B(b))$ in both cases.

deterministic_opening. The checker of `witness_discharge` with the packet of the third rule shortened to matrix, major premise and branch. On validated syntax the test of the row at 0 is that both entries are zero, and the test that the first entry of the row at 1 is the integer of b also establishes that the row is an equation, a paired formula having a list as its only entry.

first_instance. A checker of $\mathcal{X}^{\mathcal{R}}$ in which every proof node is a triple; the depth of the clause context equals the number of parameters in scope.

clause_discharge. Two goal-directed checkers on the input `[goal, proof]`, one of $\mathcal{W}^{\mathcal{R}}$ and one of $\mathcal{X}^{\mathcal{R}}$. The next fresh parameter is computed from the length of the context, and one traversal performs shifting and opening. The replacement acts on a parameter leaf exactly if it is the left side of the row and is the fresh witness. A variant that infers conclusions was tried and was not smaller.

substitutive_discharge. A checker of $\mathcal{K}$ with four instructions and no fuel (Theorem 6.47), its matched control with separate rules, and the raw companion with its control.

variable_substitution. A goal-directed native checker of the six-rule form with the newest variable, obtained from the checker of `full_normalization` by changing the equality packet, and a forward checker of $\mathcal{V}_0$ whose equality packet consists of two premise proofs. In the native checker the leaves for hypothesis and reflexivity ignore an unused hint, as in its control.

parameter_substitution. A forward checker of $\mathcal{P}$ with five instructions. The certificate of $0 = 0$ joins two copies of the computation $I(0, 0; u.\, u) = 0$. Substitution in a proper subterm acts on negative integers only, so it cannot rewrite the zero that marks a successor.

deterministic_specialization. A goal-directed checker of $\mathcal{U}_*$ with one instruction for each of the five rules. The instruction for a clause contains the index and the proof of the antecedent; the target is computed from the index and the scope, and contexts are stored newest first. A translator in both directions is supplied and is not part of the checker.

alias_back. Two checkers of $\mathcal{B}_\nu$ on tagged trees: the input is an object with a closed goal and a proof, and the proof has one tag for each of the computations (grouped), for USE_ν (occurrence index and premise), for BACK_ν, ALIAS (defining term and branch), OPEN and CLOSE. The public goal and every active term, formula, clause index and scope are validated, and nothing is normalized. ALIAS validates its defining term in the outer scope

before it appends its parameter and the clause $\langle b = t \rangle$. A use of a clause without antecedent ignores its premise field, as in the control. The shorter checker computes the scope from the context by Theorem 6.124, which applies since ALIAS, OPEN and CLOSE each add one clause and one parameter. The matched controls check $\mathcal{T}_{\mathrm{r}}$ with the same coding and the same simplifications. Terms and formulas are coded by integers and bare lists as in the common contract. An untrusted translator compiles proofs of $\mathcal{T}_{\mathrm{r}}$ into $\mathcal{B}$ by Theorem 6.83 and then into $\mathcal{B}_\nu$ by Theorem 6.90; its results for 32 inherited closed certificates, among them compound binders, specialization, induction hypotheses, capture avoidance and zero sources, are replayed by the checkers, and one open example uses a parameter map that does not preserve the order and a non-contiguous occurrence map. The checker of the equality fragment of Theorem 6.84 is not a complete checker and has no size record.

`equation_aliases`. The two checkers of $\mathcal{B}_\nu$ of `alias_back` with the additional tests that the conclusions of ALIAS and BACK_ν are literal equations. The readable checker keeps the scope separate from the context, for open examples; the shorter one computes it from the context. An untrusted compiler implements Theorem 6.100; it compiles 34 closed derivations, the 32 inherited ones and two with compound specialization at rank 1 and 2, and three open examples with non-monotone parameter maps, non-contiguous occurrence maps, duplicated clauses and an antecedent of rank 2 against bodies of rank 0, 1 and 2. The matched controls are the checkers of `alias_back`, which accept the certificates with compound ALIAS and BACK that the new checkers reject.

Three statements about programs, independent of the rules, are proved in the packages.

Proposition 6.124 (Scope from the context)**.** *In a checker whose public entry starts from the empty context and scope* 0*, and in which the branch of every scope-extending rule adds exactly one clause and one parameter while no other call changes either, every reachable call satisfies* $scope = |context|$*. The scope argument can therefore be computed from the context.*

Proof. By induction on the tree of calls: the initial call satisfies the equation, the rules without a branch recur with both unchanged, and a branch increments both. □

`deterministic_specialization` applies this to the checker of $\mathcal{U}_*$ and to its control, and `clause_discharge` and `first_instance` build it into their checkers. The statement concerns the public entry; it is not claimed for a direct call of an internal function with a context and a scope that do not match.

Fusion of the rewrite with the continuation. `deterministic_opening` reorders, in three inherited compact checkers of an earlier guarded calculus, the rewriting of the goal by an elimination and the construction of the new hypothesis: the hypothesis is constructed first and the rewrite is an argument of the continuation call, evaluated before the scope is decremented by the left-to-right evaluation order of arguments. The two orders accept the same certificates. A rewrite changes the goal only if the major premise is a nonempty list, and then the construction of the hypothesis cannot perform the assignment of an introduction; conversely an introduction changes the goal only if the major premise is empty, and then the rewrite is the identity; so the two operations commute. The proof of the major premise runs in its own activation and does not affect the scope of the caller, and the rewrite consumes no proof packet. By induction on the number of packets each new program accepts exactly the certificates of its control and consumes the same suffix. These programs do not implement OPEN.

One traversal for grammar and scope. The supplement of `deterministic_specialization` modifies the validated compact checker of `scalar_uniqueness`, which checks the unchanged kernel. The control traverses the public goal twice, once for sorts and arities and once for scope; the new program does both in the first traversal, under a flag that is set for the public entry only and cleared before the first proof packet is read. By induction on the goal the

fused traversal succeeds exactly if both old traversals succeed and then returns the goal; with the flag cleared every added guard is skipped and the remaining computation is that of the control.

6.10 Checker sizes

This section lists the sizes reported by the eleven packages named at the beginning of Section 6.9. All figures follow the protocol of Section 1.10 and are those stated by the packages. A control is matched if it shares the term and formula coding, the normalizer, the context representation and the public contract with the program and differs in the rule under comparison; where a package compares with an unmatched program this is said.

Table 6.1: Complete checkers of the packages of this chapter.

Program	Tokens	AST	Bytes	Control (tokens / AST / bytes)
`witness_discharge`				
$\mathcal{W}^{\mathcal{R}}$	732	813	2,273	separate elimination and transport: 769 / 841 / 2,362
$\mathcal{W}^{\mathcal{R}}$ with fuel	777	868	2,486	the same with fuel: 814 / 896 / 2,575
`inferred_witness`				
$\mathcal{O}^{\mathcal{R}}$, forward	683	771	1,990	explicit equation at opening: 676 / 765 / 1,998; explicit equation at output: 663 / 751 / 1,983
$\mathcal{O}^{\mathcal{R}}$ with fuel	728	826	2,203	unmatched: `witness_discharge` with fuel, 777 / 868 / 2,486
`deterministic_opening`				
$\mathcal{O}^{\mathcal{R}}$, goal-directed	730	809	2,253	checker of `witness_discharge`: 732 / 813 / 2,273
intermediate export variant	745	828	2,296	the same control
compact profile, fused (inherited calculus)	267	302	508	265 / 306 / 516
AST profile, fused (inherited calculus)	263	276	580	260 / 278 / 602
validated native profile, fused (inherited calculus)	360	409	780	358 / 413 / 788
`first_instance`				
$\mathcal{X}^{\mathcal{R}}$, triples	704	804	2,157	$\mathcal{W}^{\mathcal{R}}$, triples: 701 / 813 / 2,228
$\mathcal{X}^{\mathcal{R}}$ with fuel	752	863	2,340	$\mathcal{W}^{\mathcal{R}}$ with fuel: 749 / 872 / 2,411
`clause_discharge`				
$\mathcal{W}^{\mathcal{R}}$ (explicit discharge)	636	708	2,010	checker of `witness_discharge`: 732 / 813 / 2,273
$\mathcal{X}^{\mathcal{R}}$ (deterministic discharge)	653	729	2,058	explicit discharge: 636 / 708 / 2,010
explicit discharge with fuel	673	755	2,139	
deterministic discharge with fuel	690	776	2,187	explicit discharge with fuel: 673 / 755 / 2,139
`substitutive_discharge`				
$\mathcal{K}$	713	798	2,267	$\mathcal{K}_{\mathrm{sep}}$: 726 / 809 / 2,318

Program	Tokens	AST	Bytes	Control (tokens / AST / bytes)
raw companion	744	828	2,550	separate rules: 756 / 837 / 2,606
`variable_substitution`				
native six rules, newest variable	899	951	2,499	checker of `full_normalization` with TR: 908 / 965 / 2,529
$\mathcal{V}_0$	898	968	2,826	clausal TR: 907 / 982 / 2,856
`parameter_substitution`				
$\mathcal{P}$	769	892	2,857	template rule, unused code removed: 765 / 881 / 2,907; template rule, shared opener: 777 / 895 / 2,937
`deterministic_specialization`				
$\mathcal{U}_*$, scope from context	819	896	2,914	$\mathcal{U}$, scope from context: 819 / 895 / 2,926
$\mathcal{U}_*$, explicit scope	829	903	2,938	$\mathcal{U}$, explicit scope (checker of `uniform_step`, newest first): 829 / 902 / 2,950
kernel, fused grammar and scope traversal	360	412	820	checker of `scalar_uniqueness`: 381 / 439 / 866
`alias_back`				
$\mathcal{B}_\nu$, readable	825	966	3,149	$\mathcal{T}_\mathrm{r}$, readable: 764 / 883 / 2,918
$\mathcal{B}_\nu$, scope from context, shorter interface	773	916	2,903	$\mathcal{T}_\mathrm{r}$, the same simplifications: 716 / 838 / 2,683
`equation_aliases`				
$\mathcal{A}$, readable	843	989	3,215	$\mathcal{B}_\nu$, readable (checker of `alias_back`): 825 / 966 / 3,149
$\mathcal{A}$, scope from context	791	939	2,969	$\mathcal{B}_\nu$, scope from context: 773 / 916 / 2,903

The comparisons are local. In its matched pair, NAME against separate rules is smaller in all three coordinates; OPEN against an explicit equation at the opening saves 8 bytes and costs 7 tokens and 6 nodes, and is 7 bytes larger than the control with an explicit equation at the output; EXPORT saves 71 bytes and 9 nodes and costs 3 tokens in `first_instance`, and costs 48 bytes, 17 tokens and 21 nodes in `clause_discharge`; SDIS is smaller in all three; VSUB for the newest variable saves 30 bytes, 9 tokens and 14 nodes in both profiles; PARAM saves 50 bytes and costs 4 tokens and 11 nodes; USE$_*$ saves 12 bytes, leaves the tokens unchanged and costs one node, and the computation of the scope from the context saves a further 24 bytes, 10 tokens and 7 nodes. ALIAS with the two restrictions to the newest parameter costs, against the template control, 61 tokens, 83 nodes and 231 bytes in the readable pair and 57 tokens, 78 nodes and 220 bytes in the shorter pair, and the restriction of both rules to equations costs a further 18 tokens, 23 nodes and 66 bytes in each pair; the interface loses its supplied template and specialization term and gains a rule. The programs of `alias_back` and `equation_aliases` read their input as tagged trees with a fully validated closed goal. `parameter_substitution` also supplies, as a comparison source and not as a new calculus, a refined checker of the two-case evaluation calculus with 741 tokens, 862 nodes and 2,731 bytes. Every package states that shorter checkers of the repository with other certificate contracts, in particular the compact checker of 516 bytes and, where it is named, the validated checker of 788 bytes, remain smaller than its programs, and none claims a minimum. For `alias_back` and `equation_aliases` the compact programs named are those of the coordinatewise envelope

of 248 tokens, 270 nodes and 487 bytes, respectively 479 bytes, attained by different programs under a promised input alphabet. `scalar_constancy` supplies no checker of SC; its two programs check the inherited guarded public calculus, with 247 tokens, 261 nodes and 557 bytes against 246/262/556, and 271 tokens, 305 nodes and 468 bytes against 272/306/469, and are recorded in Chapter 11.

6.11 Further questions

This section collects the questions, limits and unproved claims stated by the eleven packages named at the beginning of Section 6.9, by `boolean_constancy` and by `scalar_constancy`, and one conjecture recorded in the repository.

Question 6.125 (Substitution for a variable on raw terms: theorems)**.** On full normal forms substitution for a variable is the only equality rule needed (Theorem 6.50). On raw terms the rows of a packet are not literal, and PARAM, which also joins two equations at the root, suffices (Theorem 6.64). For open sequents the repository's conjecture is now settled: substitution for a variable alone, without joining at the root, is strictly weaker on raw terms (Theorems 4.127 and 4.128, from `head_conversion`); the sequent $I(0, x; u.u) = y \vdash x = y$ separates them. The reading used there validates every theorem of N with an empty context, so it does not separate theorems. Do the calculi with substitution for a variable alone derive every theorem of N in the empty context? With the additional rule ALIAS, substitution for a parameter in the backward direction suffices for all sequents (Theorem 6.91).

Question 6.126 (The least equality interface on raw terms)**.** The instances of PARAM that the completeness proof uses are joining at the root and substitution at a fresh variable; the third case of ψ is a reversal. Is there a smaller total operation that expresses exactly the needed instances without a larger test on syntax? An answer has to account for arbitrary equations between closed terms as assumptions, and not only for the computation equations.

Question 6.127 (Unary congruence on full normal forms)**.** Theorem 6.67 covers raw and zero-normal terms. Full normalization of the successor computation identifies values that its interpretation only relates. Whether unary congruence suffices on full normal forms is not decided.

Question 6.128 (The source of the selected equation)**.** In OPEN and EXPORT the source of the selected equation is the witness opened by the same inference. A generalization in which the row at 1 may name an older parameter was considered in `clause_discharge` and discarded; the derivation of ordinary elimination by one EXPORT fails for it. Whether a sound and complete rule of that kind exists is open.

Question 6.129 (The normalization that naming needs)**.** The packets use the two computations of sel and the two closed values $P(1)$ and $P(2)$. Is there a smaller discipline of normalization, coherent with substitution and with nested binders, that makes exactly these computations literal and keeps the theorems of Sections 6.2, 6.3 and 6.5.1? Can a syntax expose the rows at 0 and 1 of a matrix without full normalization and keep both formula round trips?

Question 6.130 (A term template in the substitutive discharge)**.** Can the formula template of SDIS be restricted to a term template, with transport in compound formulas recovered, while the complete checker stays smaller than that of $\mathcal{K}_{\mathrm{sep}}$? The occurrences of the witness that the rule permits have to be tracked.

Question 6.131 (An introduction that is stable under substitution)**.** The mode of the single introduction depends on the literal spelling of the witness, and this is the reason why substitution does not preserve the rules of the calculi of this chapter. Is there a test that does not depend on the supplied witness, permits the recovery of the case introduction, and does not cost more in the checker than it saves?

Question 6.132 (The equality split)**.** Can the axiom (6.28) of Theorem 6.102 be derived from a small recursion base and CONST alone? The split must not be assumed during such a bootstrap. `boolean_constancy` asks the same for BCONST (6.32); an equality test that is correct in the standard model is not enough, since its reflection theorem has to be proved without the induction that the rule is meant to recover.

Question 6.133 (A checker for Boolean constancy)**.** Do checkers of the theory of Theorem 6.102 and of the Boolean constancy theory, with the same logical kernel, recursor coding and input validation, differ in size, or does the test of the outer equality $\delta(g, 1)$ cost more than the restriction removes? No checker of either theory exists. (`boolean_constancy`.)

Question 6.134 (Scalar constancy)**.**

1. Over SC^- the comparison equation (6.33) is equivalent to injectivity, to SR_0 and to quantifier-free induction (Theorem 6.115), and Theorem 6.122 excludes every ordinary bi-interpretation of SC^-, also with the clock. Both models refute coverage. Is SC^- together with coverage $x = 0 \vee \exists y\,(x = Sy)$ bi-interpretable with iPRA, and does it derive (6.33)?

2. Is there a principle that SC^- does not derive, strictly weaker over SC^- than (6.33), whose addition gives a theory bi-interpretable with iPRA on another domain? By Theorem 6.115 such a theory does not derive quantifier-free induction on the original domain. The package states that it does not assert successor injectivity to be the only repair.

3. No checker of SC exists. Does a complete checker of SC, or of its restriction (6.34), differ in size from a checker of the theory of Theorem 6.102 with the same logical kernel, term coding and input validation, given that the scalar iterator binds one variable fewer and (6.33) adds an axiom?

Constancy over scalar computation with coverage and without a primitive test is item 4 of Theorem 9.159; the present results do not settle it, since both models refute coverage and Theorem 6.113 uses the split. (`scalar_constancy`.)

Question 6.135 (Size of the translations)**.** Theorem 6.76 produces a finite derivation with no polynomial bound (Theorem 6.77). The translations of Theorems 6.49 and 6.60 add one packet or one alias per transport. The compiler of Theorem 6.100 reconstructs persistent premises in every new scope and nests the constructions of lower ranks (Theorem 6.101). What are the least expansions, for certificates as trees and with explicit sharing? Shared proof graphs need a validation of their references whose cost has to be measured as well.

Question 6.136 (Restriction of opening)**.** In $\mathcal{A}$ the conclusion of OPEN is an arbitrary formula, and the constructions (L_r) and (S_r) of Theorem 6.99 use compound conclusions essentially. Can OPEN be restricted to literal equations as conclusions, with a different logical encoding, while ALIAS and BACK are restricted to equations and every clause is used at the newest parameter? (`equation_aliases`.)

Question 6.137 (The witness of CLOSE in the alias calculi)**.** In $\mathcal{B}_\nu$ and $\mathcal{A}$ the witness of CLOSE is an arbitrary supplied term. Can it be restricted to the newest parameter, as in Section 7.5, with a short generalized compiler? ALIAS supplies the names, but the assumption of the branch has to be transported through the new eigenparameter without a weakening of restricted derivations. (`alias_back`.)

Question 6.138 (The equality rule of $\mathcal{A}$)**.** Can $\textsc{back}_\nu^=$ be replaced by a crossed replacement of one term, keeping the goal-directed form without a template, the newest source and the restriction of ALIAS to equations, so that the checker becomes smaller? An answer has to account for raw computations and for the checks of scope. (`equation_aliases`.)

Question 6.139 (A measured benefit of the aliases)**.** The matched checkers of $\mathcal{B}_\nu$ and $\mathcal{A}$ are larger than their controls in every coordinate (Section 6.10). Can ALIAS and OPEN, both of which append a parameter and a clause, share an implementation without hiding their different freshness and export conditions, or can the equation tests of $\mathcal{A}$ share a representation with $\textsc{use}_\nu$, so that the restricted interface lowers the size of a complete checker? (`alias_back`, `equation_aliases`.)

Question 6.140 (Lower bounds)**.** A lower bound on the size of a checker needs a fixed public grammar, a fixed contract for malformed input, a fixed source language and a fixed measure. No package of this chapter fixes these, and none proves a lower bound. Is there one complete program that improves the token, node and byte coordinates of the repository simultaneously under one contract? Is there a presentation that removes a genuine arithmetic or logical principle and also has a smaller measured checker (`alias_back`)? Can the rules of this chapter be represented in the smallest certificate languages based on arities without losing their savings to normalization and to the bookkeeping of scope?

The following are stated by the packages as not achieved or not claimed.

- *Formalization.* No package contains a Lean or Rocq proof of its theorems. The repository proves the kernel forms and the clause forms named in Theorems 6.21, 6.47, 6.66 and 6.78 and after Theorems 6.36 and 6.55. Not formalized are: the four families of $\mathcal{K}$ on the canonical recursor syntax, the five families of $\mathcal{P}$ together, the positional calculus $\mathcal{U}_*$, the constancy theory of Section 6.8, its Boolean variant (Theorem 6.108), the scalar theory SC with Theorem 6.116 and the obstruction Theorem 6.122, and the calculi N_{AB}, $\mathcal{B}_\nu$ and $\mathcal{A}$ with the compilers of Theorems 6.90, 6.99 and 6.100. `alias_back` and `equation_aliases` ask for one formalization of the compiler with arbitrary parameter embeddings and occurrence maps, reusable for restrictions to the newest parameter, and of the distinction between persistent proofs and local derivations; their finite tests replay compiled certificates and are not a verification of the metatheorems.

- *Effectiveness.* The proof translations of this chapter are effective; none is claimed to be primitive recursive. No primitive recursive bound on normalization, and no bound on the fuel needed by a derivable formula, is asserted (Theorem 6.2). `clause_discharge` does not claim an arithmetized equivalence of consistency statements in a specified weak metatheory.

- *Executable translations.* The tests of `deterministic_opening` supply, for each tested theorem, a matching proof in $\mathcal{W}^{\mathcal{R}}$; they do not contain a general executable compiler from $\mathcal{O}^{\mathcal{R}}$, nor certified compilers between the calculi on normal forms and the raw kernel with normalization traces and the repair of the introduction mode.

- *The equality fragment.* The proofs of Theorems 6.17 and 6.49 use the logical rules; no completeness statement is made for the equality rules alone. The fragment of HYP, ALIAS and BACK is the exception: it derives reflexivity, symmetry and transport by itself (Theorem 6.83).

- *Size.* No package claims a smallest presentation or a smallest checker, and the matched comparisons of Section 6.10 are not improvements in every coordinate. The checkers of `alias_back` and `equation_aliases` are larger than their matched controls in every

coordinate; the packages claim a smaller equality and specialization interface with a proof of completeness, not a smaller checker.

- *Arithmetic.* The restrictions of this chapter change the equality interface and the use of clauses. They do not weaken the arithmetic: the construction of the predecessor and the reflection of equality (Theorems 1.65 and 1.86) remain what a proposal with weaker induction would have to redo, as `parameter_substitution` notes. The calculi of Section 6.7 keep the supplied witness of CLOSE and its progress mode for equations, so they keep the induction of the theory.

Chapter 7

Introduction without a supplied witness

The rule DI of the native scalar core takes a term: its witness. This chapter restricts that term. The witness is first required to be a variable or the literal 0, then to be the newest numerical parameter of the current scope, with 0 in an empty scope, so that the rule takes no term at all. Every restricted calculus of the chapter derives the same sequents as its unrestricted counterpart, on all open sequents and with the identity translation of terms and formulas. In N the restriction of the witness cannot be combined with a restriction of the targets of QE (Theorem 7.7); with a discharge that substitutes a term read off its major premise, both restrictions hold together (Section 7.9).

The chapter is organized by idea. Section 7.1 states the restricted calculi and what they lose: a calculus with variable witnesses is closed under renaming and not under substitution, and a calculus with a scope-determined witness is not closed under the extension of its scope. Section 7.2 contains the negative results. Section 7.3 reduces the completeness of a restricted calculus to the derivability of a name for every term, and Section 7.4 derives the names from a formula proved by numerical cases. Section 7.5 treats the newest parameter, where derivations must be regenerated instead of weakened; in the Alias/Back calculus (Section 7.5.8) a fresh alias takes the place of the names. Section 7.6 treats the raw guarded clause calculus, with and without a naming exception. Sections 7.7 and 7.8 treat two calculi on a recursor whose predecessor computes; the second combines the restricted introduction with an inferred discharge by means of conditional copy. Section 7.9 restricts the use of a clause to the newest parameter as well: first with an arbitrary witness of the introduction, and then, by a vacuous case of the selector, together with the introduction at the newest parameter, so that no rule takes a term.

Throughout the chapter 1 abbreviates $S0$ and 2 abbreviates $SS0$.

7.1 Restricted witnesses

This section draws on `no_witness`, `newest_parameter`, `parameter_witness` and `deterministic_introducti`

7.1.1 Scopes and the restricted rules

In N as stated in Section 1.3 the variables available to a derivation are left implicit. A restriction that mentions the variables in scope needs them explicit.

Definition 7.1 (Scopes)**.** A *numerical scope* is a finite list $\Pi = (a_1, \ldots, a_m)$ of distinct variables, called *parameters*. A term or formula is *over* Π if its free variables are in Π. The

newest parameter is

$$\nu(\Pi) := \begin{cases} a_m, & m > 0, \\ 0, & m = 0. \end{cases}$$

A judgment is $\Pi; \Gamma \vdash A$, where Γ and A are over Π. The scope and the context are separate lists: an assumption added to Γ does not change Π. The order of Π is proof syntax and has no numerical meaning.

With scopes the rules of N read as follows. HYP, I0, IS, TR and QE keep the scope, and the terms they mention are over Π. The three rules with an eigenvariable append it to the scope of their premise:

$$\frac{\Pi; \Gamma \vdash D_{x,y}B \qquad \Pi, b;\ \Gamma, Q_y B(b,y) \vdash A}{\Pi; \Gamma \vdash A}\ \text{DE}$$

$$\frac{\Pi, z;\ \Gamma, B(t,0) \vdash B(t,Sz)}{\Pi; \Gamma \vdash D_{x,y}B}\ \text{DI} \qquad \frac{\Pi, z;\ \Gamma, E(z) \vdash E(Sz)}{\Pi; \Gamma \vdash Q_y E(y)}\ \text{QI}$$

The conclusions are over Π, the witness t of DI is over Π, and E is a literal equation. A derivation of N in the sense of Section 1.3 is a derivation in this sense over any scope that contains its free variables, and conversely.

Definition 7.2 (Restrictions of the witness)**.** Each of the following calculi has the terms, formulas and rules of N, except that the witness t of DI is restricted.

(i) N_0: t is the literal 0.

(ii) N_v: t is a parameter of Π.

(iii) $\mathsf{N}_{0\mathrm{v}}$: t is the literal 0 or a parameter of Π.

(iv) N_ν: t is $\nu(\Pi)$. The rule takes no term:

$$\frac{\Pi, z;\ \Gamma, B(\nu(\Pi),0) \vdash B(\nu(\Pi),Sz)}{\Pi; \Gamma \vdash D_{x,y}B}\ \text{DI}_\nu$$

In DI_ν the witness is computed in the scope Π of the conclusion, before the eigenvariable z is appended. The eigenvariable is never the witness.

Every derivation of N_ν is a derivation of $\mathsf{N}_{0\mathrm{v}}$, and every derivation of N_0, N_v or $\mathsf{N}_{0\mathrm{v}}$ is a derivation of N. Hence on derivable judgments

$$\mathsf{N}_\nu \ \subseteq\ \mathsf{N}_{0\mathrm{v}} \ \subseteq\ \mathsf{N}. \tag{7.1}$$

The distinction between a variable and the constant 0 is syntactic: a term that is provably equal to 0 is not the literal 0.

The derivations of reflexivity, symmetry, transitivity and congruence of Section 1.4 use I0 and TR only, and equation induction is QI followed by QE. These rules are therefore available, with the same derivations, in every calculus of Theorem 7.2; equations are transformed by them without further comment. In particular the selector equations $\mathrm{sel}(0,a,b) = a$ and $\mathrm{sel}(Sc,a,b) = b$ are available.

7.1.2 What a restricted witness loses

Lemma 7.3 (Structural rules for variable witnesses)**.** *In* $\mathsf{N}_{0\mathrm{v}}$ *and in* N_{v} *the following are admissible.*

(i) *Renaming and weakening: if* $\Pi;\Gamma \vdash A$*,* ρ *is an injective map from* Π *into a scope* Π'*, and* Δ *is a context over* Π' *that contains* $\rho\Gamma$*, then* $\Pi';\Delta \vdash \rho A$*.*

(ii) *Cut: if* $\Pi;\Gamma \vdash A$ *and* $\Pi;\Gamma, A \vdash B$*, then* $\Pi;\Gamma \vdash B$*.*

Proof. (i) By induction on the derivation, for all ρ and Δ. A renaming sends a variable witness to a variable witness and fixes the literal 0. At a rule with an eigenvariable, choose a fresh parameter of the receiving scope, extend ρ by it and extend Δ by the renamed assumption. (ii) By induction on the derivation of $\Pi;\Gamma, A \vdash B$. A use of HYP for A is replaced by the derivation of A, renamed along the inclusion of Π into the scope of that leaf and weakened by the assumptions added on the way, by (i). All other rule data are unchanged. □

The proof of (i) does not extend to substitution. A derivation of $\mathsf{N}_{0\mathrm{v}}$ that introduces $D_{x,y}B$ at a parameter a is not turned into a derivation by substituting a compound term for a, because the image of the witness is not a variable. Closure under substitution holds for $\mathsf{N}_{0\mathrm{v}}$ (Theorem 7.21), but only as a consequence of its completeness. The arguments of the chapter therefore never substitute a term into a derivation of a restricted calculus; where a formula $B(t)$ is needed for a compound t, it is obtained by QE from a closure, or the derivation is constructed anew for that instance.

In N_{ν} less is available.

Lemma 7.4 (Weakening of the context at a fixed scope)**.** *If* $\Pi;\Gamma \vdash A$ *in* N_{ν} *and* $\Gamma \subseteq \Delta$ *with* Δ *over* Π*, then* $\Pi;\Delta \vdash A$ *in* N_{ν}*.*

Proof. By induction on the derivation. The scope of every node is unchanged, so every DI_{ν} selects the same witness; the eigenvariables are not in Π and hence not free in Δ. □

Example 7.5 (Extending the scope changes the witness)**.** In the empty scope N_{ν} derives $D_{x,y}(\mathrm{sel}(y,0,x) = 0)$: the selected witness is 0, and the premise asks for $\mathrm{sel}(Sz,0,0) = 0$, an instance of IS. The same tree is not a derivation over the scope (a). There the rule selects a and its premise asks for $\mathrm{sel}(Sz,0,a) = 0$ from $\mathrm{sel}(0,0,a) = 0$, that is, for $a = 0$, which is not derivable because it is false at $a = 1$ in the standard reading. The judgment itself remains derivable over (a), by Theorem 7.35; its derivation there is a different tree.

Thus a derivation of N_{ν} cannot be weakened to a larger scope by replaying its tree, and the usual proof of cut fails as well: when the receiving derivation enters a branch with a fresh parameter, the derivation to be inserted would have to be weakened to the larger scope. Section 7.5 replaces the weakening of derivations by families of derivations that are constructed anew in every scope; in the raw guarded calculus with a naming exception, weakening is recovered by naming the old selection anew (Theorem 7.53).

7.2 Obstructions

This section draws on `newest_parameter` for the two shapes and on `constructed_computation` for the simultaneous restriction.

7.2.1 Each shape alone is incomplete

Proposition 7.6 (One shape is not enough)**.** (i) N_0 *does not derive* $D_{x,y}(\mathrm{sel}(y,1,x)=1)$ *in the empty scope and context.* N *derives it.*

(ii) N_v *derives no formula* $D_{x,y}B$ *in the empty scope and the empty context.* N *derives* $D_{x,y}(0=0)$ *there.*

Proof. (i) Read the terms in the natural numbers, read Q_yB as $B(0)\to\forall y\,B(y)$, and read $D_{x,y}B$ at $x=0$, as $B(0,0)\to\forall y\,B(0,y)$. A judgment is valid if its conclusion is true under every assignment of its parameters that makes its context true. All rules of N_0 preserve validity. For HYP, the computation equations, TR and QE this is the usual verification, and QI is induction in the natural numbers. For DE, the premise is valid at $b=0$, where its assumption $Q_yB(0,y)$ is the reading of the major premise. For DI at the witness 0, the premise gives $B(0,Sz)$ for every z from $B(0,0)$, and $B(0,0)$ gives itself. The formula $D_{x,y}(\mathrm{sel}(y,1,x)=1)$ is false in this reading: its instance at $y=0$ is $\mathrm{sel}(0,1,0)=1$, which is true, and its instance at $y=1$ is $\mathrm{sel}(1,1,0)=1$, which says $0=1$. In N the rule DI with the witness 1 asks for $\mathrm{sel}(Sz,1,1)=1$, an instance of IS.

(ii) Call a formula *simple* if it is an equation or a formula Q_yE with E an equation. In N_v, every formula derived over the empty scope from a context of simple formulas is simple. This is proved by induction on the derivation, the claim being made only for the nodes whose scope is empty. HYP yields a member of the context. I0, IS and TR yield equations. The major premise Q_yB of QE has the same scope and context and is simple by the induction hypothesis, so B is an equation and so is $B(t)$. The major premise of DE would be a formula $D_{x,y}B$ derived over the empty scope from the same context, against the induction hypothesis; hence DE does not occur at such a node. DI needs a parameter of the empty scope as its witness and does not occur. The conclusion Q_yE of QI is simple by the form of the rule. In N the rule DI with the witness 0 derives $D_{x,y}(0=0)$, its premise being reflexivity. □

The Lean development contains both statements: the calculus with the witness restricted by shape is `DWPrf`, and the two parts are `dwprf_zero_incomplete` and `dwprf_var_incomplete` in the module `NativeWitnessShape`. Part (ii) explains why DI_ν uses 0 in the empty scope, and part (i) why the parameters matter. The proposition concerns the two shapes only; it is not a statement about all proof calculi.

7.2.2 The witness and the target cannot both be restricted

The target t of QE can be restricted to a variable or the numeral 1 without loss (Lean: `vqprf_iff_nprf`). The two restrictions do not combine.

Theorem 7.7 (Finite-range obstruction)**.** *Let* N_R *be* N *with the witness of* DI *restricted to* 0 *or a parameter in scope and the target of* QE *restricted to* 1 *or a parameter in scope. Then* N_R *does not derive*

$$D_{x,y}\big(\mathrm{sel}(y,1,x)=1\big),$$

which N *derives. The same holds when* N *is replaced by the calculus of the reports, with transport in formula templates and with the witnessed introduction by progress for literal equations, and for the introduction by progress with a compound matrix.*

Proof. Evaluate terms in the natural numbers. For a formula A and an assignment e of numbers to variables define a truth value $[A]_e$ by

$$\begin{aligned}[s=t]_e &:\Leftrightarrow s \text{ and } t \text{ have the same value under } e,\\ [Q_yB]_e &:\Leftrightarrow [B]_{e[y\mapsto 0]}\Rightarrow [B]_{e[y\mapsto 1]},\\ [D_{x,y}B]_e &:\Leftrightarrow [B]_{e[x\mapsto 0,\,y\mapsto 0]}\Rightarrow [B]_{e[x\mapsto 0,\,y\mapsto 1]}.\end{aligned}$$

These clauses define an invariant of derivations; they are not an ordinary first-order interpretation. By induction on B, the value $[B(t)]_e$ equals $[B]_{e[y\mapsto n]}$, where n is the value of t under e; bound variables are renamed away from t before the substitution. Call a judgment *valid* if its conclusion is true under the assignment e_0 that sends every parameter to 0 whenever its context is true under e_0. Every rule of N_R preserves validity.

HYP is immediate. The computation equations are true under every assignment, and TR, as well as transport in a formula template, preserves truth by the substitution property.

DI: an allowed witness has the value 0 under e_0, and the eigenvariable z has the value 0, so Sz has the value 1. The premise therefore supplies exactly the implication that defines $[D_{x,y}B]_{e_0}$. For the introduction by progress the premise has the assumption $B(t,z)$, which has the same value at $z=0$, and the same implication results; the matrix need not be an equation for this argument. QI is the same computation with the clause for Q. The context does not contain z and is unaffected.

DE: the fresh parameter b has the value 0. The assumption $Q_yB(b,y)$ then has the truth value of the major premise $D_{x,y}B$, so the validity of the second premise gives the conclusion.

QE: at a parameter, the conclusion $B(t)$ has the truth value of the premise $B(0)$. At the target 1 the rule is the elimination of the implication that defines $[Q_yB]_{e_0}$.

By induction on derivations every derivable judgment is valid. For the displayed formula the instance at $x=0$, $y=0$ is $\mathrm{sel}(0,1,0)=1$, which is true, and the instance at $x=0$, $y=1$ is $\mathrm{sel}(1,1,0)=1$, which is false. In N, DI with the witness 1 derives the formula from $\mathrm{sel}(Sz,1,1)=1$. □

Theorem 7.8 (Finitely many numeral targets do not help)**.** *Fix $N\geq 1$ and let $W=\{0,\dots,N\}$. Let $\mathsf{N}_{\mathrm{R},N}$ be N with the witness of* DI *restricted to* 0 *or a parameter and the target of* QE *restricted to a parameter or a numeral in W. Then $\mathsf{N}_{\mathrm{R},N}$ does not derive*

$$D_{x,y}\big(\mathrm{sel}(y,N+1,x)=N+1\big),$$

which N derives with the witness $N+1$.

Proof. Evaluate terms in the natural numbers. For an arbitrary assignment e of numbers to variables define $[s=t]_e$ as before and

$$[Q_yB]_e :\Leftrightarrow [B]_{e[y\mapsto 0]} \Rightarrow \bigwedge_{j\in W} [B]_{e[y\mapsto j]},$$

$$[D_{x,y}B]_e :\Leftrightarrow \bigvee_{i\in W}\Big([B]_{e[x\mapsto i,\,y\mapsto 0]} \Rightarrow \bigwedge_{j\in W} [B]_{e[x\mapsto i,\,y\mapsto j]}\Big).$$

The bound variables range over W; the values of terms, and hence the values at which a matrix is evaluated after a substitution, may lie outside W, and the substitution property holds as before. A judgment is *valid* if its conclusion is true under every assignment that sends its parameters into W and makes its context true. DI by cases preserves validity: the witness has a value $i\in W$, every positive $j\leq N$ is $S(j-1)$ with $j-1\in W$, and the case $j=0$ is the assumption. The introductions by progress, and QI, preserve validity by the finite induction from 0 to N. DE uses a witness $i\in W$ and the validity of its second premise at the assignment $b\mapsto i$. QE has a target with a value in W. Transport and the computation equations are as before. The displayed formula is false: for every $i\in W$ the instance at $y=0$ is true, and the instance at $y=1$, which belongs to the conjunction because $1\in W$, says $i=N+1$. □

Remark 7.9 (Scope of the obstruction)*.* The two theorems exclude the equality of the theorem sets in the identity language under the simultaneous restriction. They do not show that a differently interpreted theory with both restrictions fails to be bi-interpretable with iPRA.

The invariant of Theorem 7.7 is not the case $N = 1$ of Theorem 7.8: it fixes the witness at 0 and gives a smaller separating formula. Both invariants remain sound for the rule of `constructed_computation` that supplies a variable from a known instance at zero,

$$\frac{\Pi, w;\ \Gamma, H(w) \vdash A \qquad \Pi; \Gamma \vdash H(0)}{\Pi; \Gamma \vdash A} \qquad w \notin \mathrm{FV}(\Gamma, A).$$

In the first invariant $H(w)$ and $H(0)$ have the same truth value. In the second, extend the assignment by $w \mapsto 0$, which is allowed because $0 \in W$; the second premise makes $H(w)$ true there and the first supplies A. No rule that names an arbitrary term by a witness outside W is covered.

7.3 Names

This section draws on `deterministic_introduction` for the naming matrix, on `no_witness` and `parameter_witness` for the derived rules of the calculus with variable witnesses, and on the Lean module `NativeNamedWitness` for the organization.

A restricted introduction cannot introduce $D_{x,y}B$ at a compound term t. It can introduce it at a parameter b that is provably equal to t. A *name* of t is a formula whose opening at a fresh b yields the equation $b = t$.

7.3.1 Naming matrices

Definition 7.10 (Gate, naming matrix, alias matrix)**.** For an equation $E \equiv (s = t)$ and a term v the *gate* of E is

$$\gamma_v(E) := \big(\mathrm{sel}(v, 0, s) = \mathrm{sel}(v, 0, t)\big).$$

For a term r, and variables x, y not free in r, the *naming matrix* and the *alias matrix* of r are

$$\mathrm{Nm}_r(x, y) := \gamma_y(x = r) = \big(\mathrm{sel}(y, 0, x) = \mathrm{sel}(y, 0, r)\big), \qquad \mathrm{Al}_r(x, y) := \big(\mathrm{sel}(y, r, x) = r\big).$$

The formulas $D_{x,y}\mathrm{Nm}_r$ and $D_{x,y}\mathrm{Al}_r$ are the *names* of r.

By the selector equations, $\gamma_0(E)$ is derivable in every context, both sides being equal to 0, and $\gamma_{Sz}(E)$ is interderivable with E. The ordinary reading of either name is equivalent to $\exists x\,(x = r)$: the instance of the matrix at $y = 0$ is provable and its instance at $y = 1$ is equivalent to $x = r$. The gate cannot be omitted. The formula $D_{x,v}(x = r)$, whose matrix does not contain v, reads $\exists x\,(x = r \to x = r)$ and says nothing about r.

Lemma 7.11 (A name is derivable with the witness it names)**.** *For every term r over Π, N derives $\Pi; \Gamma \vdash D_{x,y}\mathrm{Nm}_r$ and $\Pi; \Gamma \vdash D_{x,y}\mathrm{Al}_r$ by DI with the witness r.*

Proof. The premise for the naming matrix asks for $\mathrm{sel}(Sz, 0, r) = \mathrm{sel}(Sz, 0, r)$, which is reflexivity. The premise for the alias matrix asks for $\mathrm{sel}(Sz, r, r) = r$, an instance of IS. Neither uses its assumption. □

Lemma 7.12 (Opening a name)**.** *Let Γ contain $Q_y\mathrm{Nm}_r(b, y)$ or $Q_y\mathrm{Al}_r(b, y)$. Then $\Gamma \vdash b = r$, by a derivation that uses HYP, I0, IS, TR and one QE with the target 1.*

Proof. For the naming matrix, the instance at 0 is $\mathrm{sel}(0, 0, b) = \mathrm{sel}(0, 0, r)$; both sides equal 0 by I0. QE at 1 gives $\mathrm{sel}(1, 0, b) = \mathrm{sel}(1, 0, r)$, and IS turns the two sides into b and r. For the alias matrix, the instance at 0 is $\mathrm{sel}(0, r, b) = r$, an instance of I0. QE at 1 gives $\mathrm{sel}(1, r, b) = r$, and IS gives $\mathrm{sel}(1, r, b) = b$. □

Opening the naming matrix uses reflexivity where opening the alias matrix uses one successor computation; otherwise the two are interchangeable. The naming matrix is the one that `deterministic_introduction` recognizes syntactically (Section 7.6); the alias matrix is the one of the Lean development.

7.3.2 Derived rules with variable witnesses

Let $\mathsf{N}_{\mathrm{v}}^{+}$ be N_{v} together with the axiom scheme

$$\Pi; \Gamma \vdash D_{x,y}\mathrm{Al}_a \qquad \text{for every term } a \text{ over } \Pi.$$

A renaming sends an instance of the scheme to an instance, so Theorem 7.3 holds for $\mathsf{N}_{\mathrm{v}}^{+}$ with the same proof. The next two lemmas are stated for $\mathsf{N}_{\mathrm{v}}^{+}$ and for $\mathsf{N}_{0\mathrm{v}}$ at once.

Lemma 7.13 (Closure by cases)**.** *If* $\Pi, z;\ \Gamma, B(0) \vdash B(Sz)$*, then* $\Pi; \Gamma \vdash Q_y B(y)$*. Here* B *is an arbitrary native formula.*

Proof. Choose r not free in B. In $\mathsf{N}_{0\mathrm{v}}$, DI with the witness 0 derives $D_{r,y}B(y)$ from the given premise. DE at a fresh b adds the assumption $Q_y B(y)$, which is the conclusion and does not contain b. In $\mathsf{N}_{\mathrm{v}}^{+}$ a variable is needed as the witness. Open the name $D_{x,y}\mathrm{Al}_0$ of 0 at a fresh parameter c and introduce $D_{r,y}B(y)$ at c; its premise is the given one, renamed along the inclusion of scopes and weakened by Theorem 7.3. Eliminate $D_{r,y}B(y)$ as before, and discharge c by DE: the conclusion $Q_y B(y)$ does not contain it. □

Lemma 7.14 (Leibniz rule)**.** *For every native formula* $F(v)$ *with a distinguished variable* v*: if* $\Pi; \Gamma \vdash s = t$ *and* $\Pi; \Gamma \vdash F(s)$*, then* $\Pi; \Gamma \vdash F(t)$*.*

Proof. By induction on the number of binders of F. The statement is taken for all scopes, contexts and terms at once, because the induction hypothesis is applied to new instances of the matrix; no term is substituted into a derivation.

If $F(v)$ is an equation $p(v) = q(v)$, congruence gives $p(s) = p(t)$ and $q(s) = q(t)$, and symmetry and transitivity give $p(t) = q(t)$.

Let $F(v) = Q_y B(v, y)$. By Theorem 7.13 it suffices to derive $B(t, Sz)$ over Π, z from $\Gamma, B(t, 0)$. The equation $t = s$ and the induction hypothesis for the matrix $B(v, 0)$ give $B(s, 0)$. QE at Sz, applied to the weakened premise $Q_y B(s, y)$, gives $B(s, Sz)$. The equation $s = t$ and the induction hypothesis for $B(v, Sz)$ give $B(t, Sz)$.

Let $F(v) = D_{x,y}B(v, x, y)$. Apply DE to the premise at a fresh parameter b, which adds $Q_y B(s, b, y)$. Introduce $D_{x,y}B(t, x, y)$ at the variable b. The premise of this introduction has the assumption $B(t, b, 0)$ and asks for $B(t, b, Sz)$: the induction hypothesis gives $B(s, b, 0)$, QE at Sz gives $B(s, b, Sz)$, and the induction hypothesis gives $B(t, b, Sz)$. The conclusion does not contain b, and DE discharges it. Where a premise is used below a new parameter it is renamed and weakened by Theorem 7.3. □

7.3.3 The introduction at a name

Theorem 7.15 (Variable witnesses and a name for every term)**.** *For every scope* Π*, context* Γ *and formula* A *over* Π*,*

$$\Pi; \Gamma \vdash_{\mathsf{N}_{\mathrm{v}}^{+}} A \quad \Longleftrightarrow \quad \Pi; \Gamma \vdash_{\mathsf{N}} A.$$

Proof. From left to right, an introduction at a variable is an instance of DI, and the axioms are derivable in N by Theorem 7.11.

From right to left it suffices to show that DI at an arbitrary term t over Π is admissible in $\mathsf{N}_{\mathrm{v}}^{+}$; the claim then follows by induction on the derivation in N. Let a derivation of

$$\Pi, z;\ \Gamma, B(t, 0) \vdash B(t, Sz)$$

be given. Take the axiom $D_{x,y}\mathrm{Al}_t$ and apply DE to it at a fresh parameter b. The branch has the scope Π, b and the context $\Gamma_b = \Gamma, Q_y \mathrm{Al}_t(b, y)$, and Theorem 7.12 gives $\Gamma_b \vdash b = t$. In this branch introduce $D_{x,y}B$ at the variable b. The premise of the introduction has the

scope Π, b, z and the context $\Gamma_b, B(b,0)$. The Leibniz rule gives $B(t,0)$ from $B(b,0)$ and $b = t$. Rename the given derivation along the inclusion of Π, z into Π, b, z, weaken it to the context $\Gamma_b, B(b,0), B(t,0)$, and cut $B(t,0)$; the result is $B(t,Sz)$. The Leibniz rule with $t = b$ gives $B(b,Sz)$. The introduction at b yields $\Pi, b; \Gamma_b \vdash D_{x,y}B$, a formula over Π, and DE discharges b. □

Corollary 7.16 (Completeness is the derivability of the names)**.** $\mathsf{N}_{0\mathrm{v}}$ *derives the same judgments as* N *if and only if it derives* $\Pi; \varnothing \vdash D_{x,y}\mathrm{Al}_a$ *for every scope* Π *and every term* a *over* Π*.*

Proof. If the two calculi agree, the names are derivable by Theorem 7.11. Conversely, suppose the names are derivable in $\mathsf{N}_{0\mathrm{v}}$. A derivation of $\mathsf{N}_{\mathrm{v}}^{+}$ becomes a derivation of $\mathsf{N}_{0\mathrm{v}}$ when each axiom leaf is replaced by the derivation of that name, weakened to the context of the leaf by Theorem 7.3. With Theorem 7.15 and (7.1) this gives $\mathsf{N} \subseteq \mathsf{N}_{\mathrm{v}}^{+} \subseteq \mathsf{N}_{0\mathrm{v}} \subseteq \mathsf{N}$. □

Remark 7.17 (Lean status). Theorem 7.15 is `dnprf_iff_nprf` for the calculus `DNPrf`, and Theorem 7.16 is `dwprf_complete_iff_names`; both are in `NativeNamedWitness`. The steps of the Lean proof are the ones above: the name, its opening, the introduction at the name, and a Leibniz rule proved over one context without substitution into derivations; a dummy witness in a scope without variables is a name of 0, as in Theorem 7.13. The derivation of the names inside the restricted calculus, which is the subject of the next section, is formalized as well (Theorem 7.23).

7.4 The coverage packet

This section draws on `no_witness` (its restriction note), `parameter_witness` (its appendix on restricted witnesses) and `newest_parameter`.

By Theorem 7.16 it remains to derive the name of an arbitrary term t with variable witnesses. The name cannot be introduced at t. The route is indirect: a formula $X(y)$ that says "y is 0 or a successor" is proved for all y by numerical cases, where in the successor case the witness is the case variable; it is then instantiated at $y = St$ by QE, whose target is unrestricted; and the injectivity of the successor turns the opened instance into a name of t.

7.4.1 The predecessor

Let $\mathcal{E}$ be the equational fragment of Theorem 1.53: HYP, the two computation equations, TR and induction for an equation, on judgments whose assumptions and conclusions are equations. By Theorem 1.65, whose equational derivation replaces every numerical case distinction by the interpolation and the two-point rule of Theorem 1.58, $\mathcal{E}$ derives, for the predecessor term written P there and pred in this chapter,

$$\mathrm{pred}(0) = 0, \qquad \mathrm{pred}(Sn) = n. \tag{7.2}$$

$\mathcal{E}$ is closed under substitution (Theorem 1.54), so for every term r it derives $\mathrm{pred}(Sr) = r$. In a calculus that has QI and QE, a derivation of $\mathcal{E}$ is a derivation without any introduction of $D_{,}$. Hence in N_0, $\mathsf{N}_{0\mathrm{v}}$ and N_ν, for all Π, Γ and terms r, s over Π,

$$\Pi; \Gamma \vdash \mathrm{pred}(Sr) = r, \qquad \Pi;\ \Gamma,\ Sr = Ss \vdash r = s, \tag{7.3}$$

the second by congruence with pred. The order of the argument matters. The predecessor is obtained before any term is named; obtaining it from the bridge of Theorem 1.119, which is proved for the unrestricted calculus, would be circular. That the derivation needs no case distinction under a hypothesis, hence no DI, is the contribution of the four packages recorded

in Theorem 1.66. The predecessor is formalized in this fragment: in Lean the fragment is `EqPrf` (hypothesis, the two computations, transport in one template, equation induction), with `EqPrf.pred_zero`, `EqPrf.pred_succ` and `EqPrf.succ_inj`, and `EqPrf.toDW` embeds it in the kernel under any restriction of the witness.

7.4.2 The packet and the names

Definition 7.18 (Coverage packet)**.** For a term y put

$$M_y(x,v) := \gamma_v\big(\mathrm{sel}(y,0,Sx)=y\big) = \big(\mathrm{sel}(v,0,\mathrm{sel}(y,0,Sx)) = \mathrm{sel}(v,0,y)\big), \qquad X(y) := D_{x,v}M_y(x,v).$$

The ordinary reading of $X(y)$ is equivalent to $\exists x\,(\mathrm{sel}(y,0,Sx)=y)$, which holds at $y=0$ for every x and says $y=Sx$ at a successor. The formula never asks for an introduction of $\exists x\,(x=t)$ at the witness t.

Lemma 7.19 (The packet with variable witnesses)**.** *In* $\mathsf{N}_{0\mathrm{v}}$*, for every term* t *over* Π*,* $\Pi;\Gamma \vdash X(t)$*.*

Proof. $X(0)$: introduce at the witness 0. The premise asks for $M_0(0,Sz)$, that is, for $\mathrm{sel}(Sz,0,\mathrm{sel}(0,0,1)) = \mathrm{sel}(Sz,0,0)$. By IS and I0 the left side equals $\mathrm{sel}(0,0,1)$ and then 0, and the right side equals 0.

$X(Sz)$ over the scope Π, z: introduce at the variable z. The premise asks for $M_{Sz}(z,Sz')$, that is, for $\mathrm{sel}(Sz',0,\mathrm{sel}(Sz,0,Sz)) = \mathrm{sel}(Sz',0,Sz)$; both sides equal Sz by IS. Neither premise uses its assumption.

By Theorem 7.13, with the assumption $X(0)$ unused, $\Pi;\Gamma \vdash Q_yX(y)$. QE with the base $X(0)$ and the target t gives $X(t)$. This is an application of QE, not a substitution into the derivation of $X(Sz)$. □

Lemma 7.20 (The alias)**.** *Let* t *be over* Π *and* $b \notin \Pi$*. If* Γ *contains* $Q_vM_{St}(b,v)$*, then* $\Gamma \vdash b = t$*, by a derivation in which no* $D_{,}$ *is introduced.*

Proof. The instance $M_{St}(b,0)$ is a gate at 0 and is derivable. QE at 1 gives $\mathrm{sel}(1,0,\mathrm{sel}(St,0,Sb)) = \mathrm{sel}(1,0,St)$, and IS, applied twice on the left and once on the right, gives $Sb = St$. By (7.3), $b = t$. □

Theorem 7.21 (The literal 0 and variables suffice)**.** *For every scope* Π*, context* Γ *and formula* A *over* Π*,*

$$\Pi;\Gamma \vdash_{\mathsf{N}_{0\mathrm{v}}} A \quad\Longleftrightarrow\quad \Pi;\Gamma \vdash_{\mathsf{N}} A.$$

Consequently $\mathsf{N}_{0\mathrm{v}}$ *is closed under substitution, and the bridge of Theorem 1.119 holds for it with the same translations.*

Proof. By Theorem 7.16 it suffices to derive $\Pi;\varnothing \vdash D_{x,y}\mathrm{Al}_t$ for t over Π. By Theorem 7.19, $X(St)$ is derivable. Apply DE to it at a fresh b; by Theorem 7.20 the branch derives $b = t$. Introduce $D_{x,y}\mathrm{Al}_t$ at the variable b: the premise asks for $\mathrm{sel}(Sz,t,b) = t$, and $\mathrm{sel}(Sz,t,b) = b$ by IS. The name does not contain b, and DE discharges it. The two calculi have the same terms and formulas and the comparison is the identity, so every property of the consequence relation of N transfers. □

Remark 7.22 (Versions of the packet and of the proof)*.* The formula $X(y)$ is $H(y)$ in `no_witness`, $X(y)$ in `newest_parameter` and $A(r)$ in `uniform_raw`. `parameter_witness` uses the ungated packet

$$T(v) := D_{x,y}\big(\mathrm{sel}(y,v,\mathrm{sel}(v,0,Sx)) = v\big),$$

whose instance at $y = 0$ is $v = v$ and whose instances at successors are $\mathrm{sel}(v, 0, Sx) = v$; it serves in a calculus where the mode of the introduction depends on the witness (Theorem 7.24). The restriction note of `no_witness` and the appendix of `parameter_witness` prove Theorem 7.21 directly, with the naming matrix $D_{x,v}\gamma_v(x = t)$ and a derived rule

$$\frac{\Pi, b;\ \Gamma, b = t \vdash F}{\Pi; \Gamma \vdash F} \qquad b \notin \mathrm{FV}(\Gamma, t, F),$$

obtained by opening $X(St)$ and cutting the alias. The proof above factors the same argument through Theorem 7.15. The theorem answers the question whether DI at the literal 0 or at a variable is enough.

Remark 7.23 (Lean status). Theorem 7.21 is formalized: `dwprf_iff_nprf`, in the module `NativeWitnessComplete`, states that `DWPrf true true` derives $\Gamma \vdash A$ if and only if `NPrf` does, that is, the kernel with DI by cases at the literal 0 or at a variable, and at no other term, derives exactly what the kernel derives. The names are derived there (`DWPrf.names`) from the coverage packet in a form in which the predecessor stands inside the matrix: the name $D_{x,v}\mathrm{Al}_{\mathrm{pred}(y)}$ of $\mathrm{pred}(y)$ holds at $y = 0$ with the witness 0 and at $y = Sz$ with the witness z, by the closure by cases with the dummy witness 0 (`DWPrf.namePQ`); it is used at $y = Sa$, and opening that name gives a variable equal to $\mathrm{pred}(Sa) = a$ (`DWPrf.nameOf`). The packet $X(y)$ applies the predecessor after the opening instead; the two differ in this order only. The deterministic selection of Section 7.5, with the regeneration of derivations under an extension of the scope, and the calculi built on it are not formalized (Theorem 7.101).

7.4.3 A mode selected by the witness

`parameter_witness` states the restriction for a calculus with one introduction whose mode is read off the witness. Let R be the seven-rule calculus W of Theorem 2.15, with scopes, in which the witness t of its introduction is the literal 0 or a parameter of Π:

$$\frac{\Pi, z;\ \Gamma, B(t, \eta_{B,t}(z)) \vdash B(t, Sz)}{\Pi; \Gamma \vdash D_{x,y}B(x, y)}\ \text{RDI},$$

where $\eta_{B,t}(z)$ is z if B is a literal equation and t is the literal 0, and 0 otherwise. There is no rule QI. Every introduction at a variable is by cases.

Theorem 7.24 (The calculus with a witness-selected mode)**.** R *and* N *derive the same judgments.*

Proof. Every derivation of R is a derivation of W, which is equivalent to N by Theorem 2.17. For the converse, the proof of Theorem 7.21 is repeated with the following changes.

Structural rules. Theorem 7.3 holds for R: a renaming sends variables to variables, fixes the literal 0 and preserves the root of a matrix, hence the mode.

Closure by progress. If $\Pi, z; \Gamma, E(z) \vdash E(Sz)$ for a literal equation E, then $\Pi; \Gamma \vdash Q_y E(y)$: RDI at the witness 0 on $D_{x,y}E(y)$ with x not free in E is in the mode by progress, and DE returns the opened assumption. With QE this is equation induction, so every derivation of $\mathcal{E}$ is a derivation of R rule by rule, and (7.3) holds in R.

Closure by cases. The introduction of $D_{r,y}B(y)$ at 0 is by progress when B is an equation, so a variable witness is used for every B. The formula $D_{r,w}(0 = 0)$ is derivable by RDI at 0, its premise being reflexivity. Open it at a fresh b and introduce $D_{r,y}B(y)$ at b; this is by cases. Eliminate the unused coordinate to obtain $Q_y B(y)$ and discharge b. This replaces Theorem 7.13.

Leibniz rule. Theorem 7.14 holds with the same proof; the reintroduction in the case of $D_{,}$ is at a variable and is by cases.

Packet. With T as in Theorem 7.22: $T(0)$ is introduced at 0 by progress, its premise asking for $\mathrm{sel}(Sz, 0, \mathrm{sel}(0, 0, 1)) = 0$, which holds by computation; $T(Sz)$ is introduced at z by cases, its premise asking for $\mathrm{sel}(Sz', Sz, \mathrm{sel}(Sz, 0, Sz)) = Sz$, which holds by computation. Closure by cases gives $Q_v T(v)$ and QE gives $T(t)$ for every t. Opening $T(St)$ at a fresh b gives a closure whose instance at 0 is $\mathrm{sel}(0, St, \cdot) = St$, an instance of I0; QE at 1 and two computations give $Sb = St$, hence $b = t$.

Recovery of the rules of N. QI is closure by progress. For DI at an arbitrary term t, with an arbitrary matrix, open $T(St)$ at b, introduce at b, which is by cases, and transport the assumption and the conclusion as in the proof of Theorem 7.15. This includes the introduction by cases of an equation at the literal 0, which RDI does not offer directly. □

7.5 The newest parameter

This section draws on `no_witness` and `newest_parameter` for the eight-rule calculus, on `no_witness` for the seven-rule diagonal variant, on `parameter_witness` for the one-bit rule and its clause calculus, and on `local_introduction` for the Alias/Back calculus.

In N_ν the introduction takes no term. The proof of Theorem 7.21 uses renaming, weakening to a larger scope and cut, none of which is available for the derivations of N_ν (Theorem 7.5). The remedy is to work with families of derivations that are constructed anew in every scope. A local derivation whose witness is a freshly chosen case variable z is used while z is the newest parameter; it is never moved to a scope in which z is older.

7.5.1 Stable derivations and hereditary families

Definition 7.25 (Stable derivations)**.** An application of DI_ν with the matrix $B(x, y)$ is *dummy* if x is not free in B. A derivation of N_ν is *stable* if all its introductions are dummy.

The premise of a dummy introduction does not depend on the selected witness.

Lemma 7.26 (Stability)**.** *Let* $\Pi; \Gamma \vdash A$ *have a stable derivation. Let* θ *assign to each parameter of* Π *a term over a scope* Π'*, and let* Δ *be a context over* Π' *that contains* $\Gamma[\theta]$*. Then* $\Pi'; \Delta \vdash A[\theta]$ *has a stable derivation. If* $\Pi; \Gamma \vdash A$ *and* $\Pi; \Gamma, A \vdash B$ *have stable derivations, so has* $\Pi; \Gamma \vdash B$*.*

Proof. The first statement is proved as the substitution lemma of Section 1.4, by induction on the derivation for all θ and Δ. At a dummy introduction the premise is $\Gamma, B(0) \vdash B(Sz)$ whatever the witness is, and the rule applies in the scope Π' to the matrix $B[\theta]$, in which x is still not free. The second statement is the usual induction on the derivation of $\Pi; \Gamma, A \vdash B$; the inserted derivation of A is moved to the scope of each leaf by the first statement. □

Derivations without any introduction of D, are stable. In particular the judgments (7.3) and the derivation of Theorem 7.20 are stable in N_ν.

Definition 7.27 (Hereditary derivability)**.** Write $\Pi; \Gamma \Vdash A$ if for every scope Π', every injective map ρ from Π into Π' and every context Δ over Π' that contains $\rho\Gamma$, the calculus N_ν derives $\Pi'; \Delta \vdash \rho A$.

Hereditary derivability is a property of a family of derivations, one for each (Π', ρ, Δ); it is not a rule of the calculus. The member of the family at the identity is a derivation of $\Pi; \Gamma \vdash A$. The families below are given by constructions that take the receiving scope and context as input and choose their eigenvariables there.

Lemma 7.28 (Closure properties)**.** (i) *If* $A \in \Gamma$*, then* $\Pi; \Gamma \Vdash A$*.*

(ii) *If* $\Pi;\Gamma \Vdash A$, *ρ is an injective map from Π into Π' and $\Delta \supseteq \rho\Gamma$ is over Π', then* $\Pi';\Delta \Vdash \rho A$.

(iii) *If* $\Pi;\Gamma \vdash A$ *has a stable derivation, then* $\Pi;\Gamma \Vdash A$.

(iv) *The rules* I0, IS, TR *and* QE *hold with* $\Vdash$ *in place of* $\vdash$.

(v) *If* $\Pi;\Gamma \Vdash D_{x,y}B$ *and* $\Pi, b;\ \Gamma, Q_yB(b,y) \Vdash A$ *with A over Π, then* $\Pi;\Gamma \Vdash A$.

(vi) *If* $\Pi, z;\ \Gamma, E(z) \Vdash E(Sz)$ *for a literal equation E, then* $\Pi;\Gamma \Vdash Q_yE(y)$.

Proof. (i) is HYP in every receiving context. (ii) holds because injective maps compose. (iii) is Theorem 7.26. For (iv), apply the rule to the members of the given families at (Π', ρ, Δ), with the terms renamed by ρ. For (v), let (Π', ρ, Δ) be given. Choose $b' \notin \Pi'$ and extend ρ by $b \mapsto b'$. The member of the second family at (Π', b') and the context $\Delta, Q_y\rho B(b', y)$ derives ρA, and DE applies to it and to the member of the first family. (vi) is proved in the same way with QI. □

Lemma 7.29 (Uniform cut)**.** *If* $\Pi;\Gamma \Vdash A$ *and* $\Pi;\Gamma, A \vdash B$ *in* N_ν, *then* $\Pi;\Gamma \vdash B$ *in* N_ν. *Consequently, if* $\Pi;\Gamma \Vdash A$ *and* $\Pi;\Gamma, A \Vdash B$, *then* $\Pi;\Gamma \Vdash B$.

Proof. By induction on the derivation of $\Pi;\Gamma, A \vdash B$, for all Π and Γ. At HYP for A, take the member of the family at the identity; any other HYP is kept. A rule whose premises have the same scope and context is reapplied to the results of the induction hypothesis. At a rule with an eigenvariable, say DI_ν with the premise $\Pi, z;\ \Gamma, A, B(w,0) \vdash B(w, Sz)$ and $w = \nu(\Pi)$, Theorem 7.28(ii) gives $\Pi, z;\ \Gamma, B(w,0) \Vdash A$, the induction hypothesis gives $\Pi, z;\ \Gamma, B(w,0) \vdash B(w, Sz)$, and the rule is reapplied in the scope Π, where it selects the same w. DE and QI are treated in the same way. Every node of the receiving derivation keeps its scope, and no derivation is moved to another scope: below a new parameter the family supplies a new member.

For the second statement let (Π', ρ, Δ) be given. The second family has a member deriving $\Pi';\Delta, \rho A \vdash \rho B$, and $\Pi';\Delta \Vdash \rho A$ by Theorem 7.28(ii). □

7.5.2 Closure by cases, the Leibniz rule and equation cases

Lemma 7.30 (Closure by cases)**.** *In* N_ν, *if* $\Pi, z;\ \Gamma, B(0) \vdash B(Sz)$, *then* $\Pi;\Gamma \vdash Q_yB(y)$; *if the given derivation is stable, so is the result. If* $\Pi, z;\ \Gamma, B(0) \Vdash B(Sz)$, *then* $\Pi;\Gamma \Vdash Q_yB(y)$.

Proof. Choose r not free in B and apply DI_ν to $D_{r,y}B(y)$. The introduction is dummy, and its premise is the given one whatever $\nu(\Pi)$ is. DE at a fresh b adds the assumption $Q_yB(y)$, which is the conclusion. For the hereditary statement, the member of the given family at (Π', z'), $\rho[z \mapsto z']$ and $\Delta, \rho B(0)$ is the premise needed in the scope Π'. □

Lemma 7.31 (Hereditary Leibniz rule)**.** *For every native formula $F(v)$: if* $\Pi;\Gamma \Vdash s = t$ *and* $\Pi;\Gamma \Vdash F(s)$, *then* $\Pi;\Gamma \Vdash F(t)$.

Proof. By induction on the number of binders of F, for all scopes, contexts and terms. For an equation the claim follows from Theorem 7.28(iv), as in Theorem 7.14.

Let $F(v) = Q_yB(v, y)$. By Theorem 7.30 it suffices to show $\Theta \Vdash B(t, Sz)$, where Θ stands for the scope Π, z with the context $\Gamma, B(t,0)$. By Theorem 7.28, $\Theta \Vdash t = s$, $\Theta \Vdash B(t,0)$ and $\Theta \Vdash Q_yB(s,y)$. The induction hypothesis gives $\Theta \Vdash B(s,0)$, QE at Sz gives $\Theta \Vdash B(s, Sz)$, and the induction hypothesis gives $\Theta \Vdash B(t, Sz)$.

Let $F(v) = D_{x,y}B(v,x,y)$, and let (Π', ρ, Δ) be given; write s', t', B' for the images under ρ. The first family gives $\Pi';\Delta \vdash D_{x,y}B'(s', x, y)$. Apply DE at a fresh b. In the scope Π', b the parameter b is the newest, so DI_ν for $D_{x,y}B'(t', x, y)$ selects b; its premise has the

scope Π', b, z and the context $\Delta, Q_y B'(s', b, y), B'(t', b, 0)$, denoted Θ. As in the previous case, $\Theta \Vdash B'(s', b, 0)$ by the induction hypothesis, $\Theta \Vdash B'(s', b, Sz)$ by QE, and $\Theta \Vdash B'(t', b, Sz)$ by the induction hypothesis. The member at the identity is the premise of the introduction. The conclusion does not contain b, and DE discharges it. Whatever the receiving scope is, the parameter b is generated after all its parameters, so the same recipe always introduces at the newest parameter. □

The derivation of (7.2) needs only equation induction. `newest_parameter` observes in addition that case distinctions on a term of the context are available in N_ν for equational conclusions, without DI at a chosen witness. The statement is Theorem 3.51 for hereditary families, with the implication $U \Rightarrow_= V$ between equations defined there.

Lemma 7.32 (Contextual equation cases)**.** *In N_ν, for equations U, V, F and a term e over Π:*

(i) *if $\Pi; \Gamma, U \Vdash V$, then $\Pi; \Gamma \Vdash U \Rightarrow_= V$;*

(ii) *if $\Pi; \Gamma \Vdash U \Rightarrow_= V$ and $\Pi; \Gamma \Vdash U$, then $\Pi; \Gamma \Vdash V$;*

(iii) *if $\Pi; \Gamma, e = 0 \Vdash F$ and $\Pi, z;\ \Gamma, e = Sz \Vdash F$, then $\Pi; \Gamma \Vdash F$.*

Proof. The proof of Theorem 3.51 applies, with Theorem 7.30 for the closure by cases and Theorem 7.29 for the cut; in (iii) the two hypotheses give $(e = y) \Rightarrow_= F$ at $y = 0$ and at $y = Sz$ by (i), the closure by cases and QE give it at $y = e$, and (ii) with $e = e$ gives F. □

All introductions in the constructions of Theorems 7.30 and 7.32 are dummy; applied to stable derivations they yield stable derivations.

7.5.3 The packet in every scope

Lemma 7.33 (The packet with the newest parameter)**.** *In N_ν, for every scope Π, every context Γ and every term t over Π: $\Pi; \Gamma \vdash X(St)$. Hence $\Pi; \Gamma \Vdash X(St)$.*

Proof. $X(0)$: the rule selects $w = \nu(\Pi)$, and its premise asks for $M_0(w, Sz)$, that is, for $\mathrm{sel}(Sz, 0, \mathrm{sel}(0, 0, Sw)) = \mathrm{sel}(Sz, 0, 0)$. Both sides equal 0 by computation, whatever w is. So $X(0)$ is derivable in the empty scope and also when w is an unrelated parameter.

$D_{r,y}X(y)$ with r unused: the premise of this introduction has the scope Π, z and asks for $X(Sz)$ from $X(0)$. In the scope Π, z the newest parameter is z, so the introduction of $X(Sz)$ selects z; its premise asks for $M_{Sz}(z, Sz')$, both sides of which equal Sz by computation. The assumptions of both premises are unused.

DE on $D_{r,y}X(y)$ gives $Q_y X(y)$, and QE with the base $X(0)$ and the target St gives $X(St)$. The construction works in every scope and every context, and $\rho X(St) = X(S\rho t)$; this is the hereditary statement. □

The construction is applied to the whole closure. It is not claimed that $\Pi, z; \Gamma \Vdash X(Sz)$: a derivation that introduces $X(Sz)$ at z is not a derivation in a scope in which z is no longer the newest parameter. In each receiving scope the case variable is chosen last.

Lemma 7.34 (The hereditary alias)**.** *Let Θ be a scope and context such that the context contains $Q_v M_{St}(b, v)$. Then $\Theta \Vdash b = t$.*

Proof. The derivation of Theorem 7.20 is stable. □

7.5.4 The equivalence theorem

Theorem 7.35 (No witness in the eight-rule calculus)**.** *For every scope* Π*, context* Γ *and formula* A *over* Π*,*

$$\Pi;\Gamma \vdash_{\mathsf{N}_\nu} A \quad\Longleftrightarrow\quad \Pi;\Gamma \vdash_{\mathsf{N}} A.$$

Both directions are effective transformations of derivations, and the terms and formulas of the judgment are unchanged.

Proof. From left to right every rule of N_ν is an instance of a rule of N. For the converse the following is proved by induction on a derivation of N:

$$\text{if } \Pi;\Gamma \vdash_{\mathsf{N}} A, \text{ then } \Pi;\Gamma \Vdash A.$$

HYP, I0, IS, TR and QE are Theorem 7.28(i) and (iv). DE and QI are Theorem 7.28(v) and (vi), applied to the families that the induction hypothesis gives for the premises.

Consider DI with the witness t and the matrix B. The induction hypothesis is

$$\Pi, z;\ \Gamma, B(t,0) \Vdash B(t,Sz). \tag{7.4}$$

Let (Π',ρ,Δ) be given and write $t'=\rho t$, $B'=\rho B$. By Theorem 7.33, $\Pi';\Delta \vdash X(St')$. Apply DE to it at a fresh b; the branch has the scope Π',b and the context $\Delta_b=\Delta,Q_vM_{St'}(b,v)$. In this branch apply DI_ν to $D_{x,y}B'$. The newest parameter is b, so the rule selects b, and its eigenvariable z' is appended after this selection. Let Θ be the scope Π',b,z' with the context $\Delta_b, B'(b,0)$; the premise to be derived is $\Theta \vdash B'(b,Sz')$.

(a) $\Theta \Vdash b=t'$ by Theorem 7.34, and $\Theta \Vdash B'(b,0)$ by membership. By Theorem 7.31, $\Theta \Vdash B'(t',0)$.

(b) The map $\rho[z\mapsto z']$ from Π,z into Π',b,z' is injective, and the context of Θ extended by $B'(t',0)$ contains the image of $\Gamma,B(t,0)$. By (7.4) and Theorem 7.28(ii), $\Theta, B'(t',0) \Vdash B'(t',Sz')$.

(c) By (a), (b) and Theorem 7.29, $\Theta \Vdash B'(t',Sz')$.

(d) By Theorem 7.31 with the equation $t'=b$, $\Theta \Vdash B'(b,Sz')$.

The member of the last family at the identity is the premise. The introduction yields $\Pi',b;\Delta_b \vdash D_{x,y}B'$, a formula over Π', and DE discharges b. This was done for an arbitrary (Π',ρ,Δ), so $\Pi;\Gamma \Vdash D_{x,y}B$.

The construction is well founded. The induction hypothesis is used for the premises of the given derivation only. Theorem 7.31 is an induction on formulas. The packet, the alias and the predecessor are fixed constructions that precede the theorem. No derivation of N_ν is moved to another scope and no term is substituted into one. Finally the member at the identity of $\Pi;\Gamma \Vdash A$ is the required derivation. □

Corollary 7.36 (Structural rules and the three calculi)**.** N_ν*,* $\mathsf{N}_{0\mathrm{v}}$ *and* N *derive the same judgments. Substitution, weakening to a larger scope and cut are admissible in* N_ν *as transformations of derivable judgments. The ordinary realization of* N_ν *is bi-interpretable and equiconsistent with* iPRA *by the translations of Theorem 1.119, with whole domains, actual equality and the identity graph* $x=y$ *in both numerical composites.*

Proof. The first statement is Theorem 7.35 and (7.1). For the structural rules translate to N, apply its rule (Section 1.4) and translate back; they are consequences of the theorem, not steps of its proof, and they do not say that an unchanged derivation tree is a derivation after an extension of the scope. The comparison with N is the identity on terms, formulas and contexts, so the translations of the bridge, the derivations of both formula round trips and the preservation of $0=S0$ transfer unchanged. □

Remark 7.37 (The three formulations of the proof families). `no_witness` calls the families *portable*: a procedure that, for every extension of the scope by fresh parameters and of the context by assumptions, constructs a derivation of the unchanged formula. `newest_parameter` calls them *persistent builders*, with the same definition. Both state the induction on the source derivation for an arbitrary substitution σ of terms over a target scope for the source parameters, a target context Δ, and a family for $E[\sigma]$ for each source assumption E; the result is a family for $A[\sigma]$. The substitution takes the place of the renaming ρ, and stability under extension takes the place of stability under injective maps. `parameter_witness` and `uniform_raw` use hereditary derivability as in Theorem 7.27, and the proof above follows them. Theorem 7.29 is the persistent cut of `newest_parameter` and the uniform cut of `parameter_witness`. `no_witness` also derives the standalone name $D_{x,v}\gamma_v(x = t)$ in N_ν: while $b = t$ is available and b is the newest parameter, introduce the name, and then discharge the opening of $X(St)$. None of the packages claims a Lean proof of Theorem 7.35; a formalization has to encode the families, or an equivalent protected translation, and cannot assume the weakening of restricted derivations.

7.5.5 The seven-rule diagonal variant

The diagonal calculus is the seven-rule calculus $\mathsf{N}_{\mathrm{diag}}$ of Theorem 3.55: the calculus N_P of Theorem 3.48 for the selector P that holds of the matrices diagonal in y (Theorem 3.12). Its one introduction $\textsc{di}_P$ takes a witness t and reads its mode off the raw matrix, before the witness is inserted; write $\eta_B(z)$ for $\eta^P_B(z)$. It derives the same judgments as N (Theorem 3.55; Lean: `gprf_iff_nprf`). Let $\mathsf{N}_{\mathrm{diag},\nu}$ be $\mathsf{N}_{\mathrm{diag}}$ with scopes and with $t = \nu(\Pi)$:

$$\frac{\Pi, z;\ \Gamma, B(\nu(\Pi), \eta_B(z)) \vdash B(\nu(\Pi), Sz)}{\Pi; \Gamma \vdash D_{x,y}B(x, y)}.$$

The matrix selects the mode and the scope selects the witness. No guard, axiom or auxiliary syntax is added.

Theorem 7.38 (No witness in the seven-rule calculus). *$\mathsf{N}_{\mathrm{diag},\nu}$, $\mathsf{N}_{\mathrm{diag}}$ and N derive the same judgments.*

Proof. Every rule of $\mathsf{N}_{\mathrm{diag},\nu}$ is a rule of $\mathsf{N}_{\mathrm{diag}}$. For the converse, Theorems 7.25 and 7.27 and Theorems 7.26, 7.28 and 7.29 are taken for $\mathsf{N}_{\mathrm{diag},\nu}$, with item (vi) of Theorem 7.28 omitted. Their proofs are unchanged, because the diagonal shape is preserved and reflected by the substitution of terms for parameters (Theorem 3.55(iii)). The following steps differ from the proof of Theorem 7.35; they are the steps of Theorem 3.52 carried out for families of derivations, since the structural rules of $\mathsf{N}_{\mathrm{diag}}$ are not available under the restriction.

Closure by cases. Let $\Pi, z; \Gamma, B(0) \vdash B(Sz)$ and r not free in B. If B is not diagonal, the introduction of $D_{r,y}B(y)$ is by cases and Theorem 7.30 applies. If B is diagonal, $B(0)$ is derivable in every scope and context by a stable derivation; this is condition (S1) of Theorem 3.55(i). Theorem 7.29 removes the assumption $B(0)$ from the given premise, and Theorem 7.4, which holds for $\mathsf{N}_{\mathrm{diag},\nu}$ with the same proof, adds the assumption $B(z)$. The result is the premise of the introduction by progress. In both cases $\textsc{de}$ yields $Q_yB(y)$.

Equation induction. There is no rule $\textsc{qi}$. For a term $s(y)$ let $L_s(y) := I(y, y; u.\, s(y))$ with u not free in s, so that $L_s(y) = L_t(y)$ is the diagonal lift $\Delta_y E$ of Theorem 3.12 for $E(y) \equiv (s(y) = t(y))$; then $L_s(0) = 0$ and $L_s(Sz) = s(Sz)$ by computation. The decoder equation

$$\mathrm{sel}(y, s(0), L_s(y)) = s(y) \tag{7.5}$$

holds at $y = 0$ by $\textsc{i}0$ and at $y = Sz$ by two successor computations; closure by cases and $\textsc{qe}$ give it at every term, in every scope and context. Let $E(y) \equiv (s(y) = t(y))$ and suppose

$\Pi, z; \Gamma, E(z) \Vdash E(Sz)$. Then $\Pi; \Gamma \Vdash Q_y E(y)$. Indeed, let (Π', ρ, Δ) be given, and drop ρ from the notation. By closure by cases it suffices to derive $E(Sz)$ in the scope Π', z from $\Delta_0 = \Delta, E(0)$. The matrix $L_s(y) = L_t(y)$ is diagonal. Introduce $D_{r,y}(L_s(y) = L_t(y))$ with r unused; this is by progress, and the premise has a further eigenvariable z_1 and the context $\Delta_0, L_s(z_1) = L_t(z_1)$, denoted Θ. By (7.5) for s and for t, the assumption and the base $E(0)$,

$$\Theta \Vdash s(z_1) = \mathrm{sel}(z_1, s(0), L_s(z_1)) = \mathrm{sel}(z_1, t(0), L_t(z_1)) = t(z_1).$$

The step family, at the map $z \mapsto z_1$, gives $\Theta, E(z_1) \Vdash E(Sz_1)$; uniform cut gives $\Theta \Vdash E(Sz_1)$, and computation gives $\Theta \Vdash L_s(Sz_1) = L_t(Sz_1)$. After the introduction, DE adds the closure $Q_y(L_s(y) = L_t(y))$. Its instance at 0 is derivable, QE at Sz gives $L_s(Sz) = L_t(Sz)$, and computation gives $E(Sz)$. With QE this is equation induction for hereditary families. Every rule of the fragment $\mathcal{E}$ therefore preserves $\Vdash$, and by induction on a derivation of $\mathcal{E}$ the judgments (7.3) hold with $\Vdash$. The construction uses closure by cases, computations and the equality rules; it uses no predecessor and no chosen witness.

Leibniz rule. In the case $F(v) = D_{x,y}B(v, x, y)$ of Theorem 7.31 the target is introduced at the opened parameter b. If its matrix is not diagonal, the proof is unchanged. If it is diagonal, the premise has the assumption $B'(t', b, z)$ in place of $B'(t', b, 0)$; the latter is derivable outright, and the same three steps give $B'(t', b, Sz)$ without the assumption.

Packet. The matrices $M_y(x, v)$ have outer iterators with the seed 0, and the diagonal shape requires the seed to be the bound variable; the matrix of $D_{r,y}X(y)$ is not an equation. All introductions in the proof of Theorem 7.33 are therefore by cases, and the proof is unchanged.

Translation. The induction is on a derivation of $\mathsf{N}_{\mathrm{diag}}$. The mode of an introduction is the same in source and target, because the alias changes the selected witness and not the raw matrix. In the mode by progress, step (a) of the proof of Theorem 7.35 transports $B'(b, z')$ to $B'(t', z')$, step (b) uses the induction hypothesis with the assumption $B(t, z)$, and steps (c) and (d) are unchanged.

The equivalence with N is the cited one for $\mathsf{N}_{\mathrm{diag}}$. □

7.5.6 One bit

Let R_ν be the calculus R of Section 7.4.3 with the variable witnesses restricted to $\nu(\Pi)$ in a nonempty scope; the literal 0 remains available, and in the empty scope it is the only witness. For an implementation both choices are made total. The calculus R_β has, in place of RDI, the rule

$$\frac{\Pi, z;\ \Gamma, B(t_\beta, \eta) \vdash B(t_\beta, Sz)}{\Pi; \Gamma \vdash D_{x,y}B(x, y)}\ \text{BIT-DI}$$

$$t_\beta = \begin{cases} 0, & \beta = 0, \\ \nu(\Pi), & \beta = 1, \end{cases} \qquad \eta = \begin{cases} z, & \beta = 0 \text{ and } B \text{ is a literal equation}, \\ 0, & \text{otherwise}, \end{cases}$$

whose only datum is the bit $\beta \in \{0, 1\}$. The mode is selected by the bit and not by an inspection of the selected term: with $\beta = 1$ in the empty scope the rule is an introduction by cases at 0, also for an equation.

Theorem 7.39 (Zero or the newest parameter)**.** R_ν, R_β *and* N *derive the same judgments.*

Proof. Every rule of R_ν is an instance of BIT-DI: an introduction at the literal 0 is the bit 0, with the same mode, and an introduction at the newest parameter is the bit 1. Every instance of BIT-DI is an introduction by cases or an introduction by progress at the witness 0, and both are rules of N, the second by Theorem 1.16. It remains to show $\mathsf{N} \subseteq \mathsf{R}_\nu$. Hereditary derivability and Theorems 7.28 and 7.29 are taken for R_ν; a renaming preserves the mode of

an introduction, and the proof of uniform cut is unchanged because each node keeps its scope. The following steps differ from the proof of Theorem 7.35.

Closure by progress and equation induction. If $\Pi, z; \Gamma, E(z) \Vdash E(Sz)$, then $\Pi; \Gamma \Vdash Q_y E(y)$, by the introduction of $D_{x,y}E(y)$ at the literal 0 in each receiving scope. This replaces Theorem 7.28(vi), and with QE it gives (7.3) with $\Vdash$ by induction on a derivation of $\mathcal{E}$.

Closure by cases. Let $\Pi, z; \Gamma, B(0) \Vdash B(Sz)$ and let (Π', ρ, Δ) be given. The formula $D_{r,w}(0 = 0)$ is derivable at the witness 0 by progress. Open it at a fresh b, which is then the newest parameter, and introduce $D_{r,y}\rho B(y)$ at b, by cases. Its premise is the member of the given family in the scope Π', b, z'; it is not obtained by renaming a derivation. Eliminate the two unused coordinates.

Leibniz rule. As Theorem 7.31. The reintroduction is at the parameter just opened, which is the newest, and it is by cases.

Packet. With T as in Theorem 7.22, $T(0)$ is derivable in every scope by progress at 0. For $Q_v T(v)$, expand the closure by cases: open $D_{r,w}(0 = 0)$ at a fresh b, introduce the dummy $D_{r,v}T(v)$ at b, and in its premise, where the eigenvariable z has just been appended and is the newest parameter although b was added before, introduce $T(Sz)$ at z by cases; the instance asked for holds by computation. Eliminate the two unused coordinates. QE gives $T(t)$ for every t, in every scope and context. It is not claimed that $T(Sz)$ has a hereditary derivation. Opening $T(St)$ at a fresh b gives $b = t$ hereditarily, as in the proof of Theorem 7.24.

Translation. The induction is on a derivation of N. QI is closure by progress. For DI at t open $T(S\rho t)$ at a fresh b and introduce at b, by cases; steps (a) to (d) of the proof of Theorem 7.35 are unchanged.

Finally $\mathsf{N} \subseteq \mathsf{R}_\nu \subseteq \mathsf{R}_\beta \subseteq \mathsf{N}$ on derivable judgments. □

7.5.7 The five-family clause calculus

The checker of `parameter_witness` uses the raw clause presentation without the binder Q, and the restriction is proved directly for it: a translation of N_ν into clauses may insert parameters and need not preserve which parameter is the newest. The comparison calculus C is the five-rule clause calculus C of Theorem 5.4, that is, C_3 with the two-case evaluation Φ_2: rules REFL, EVAL, USE, ELIM and INTRO, the last at an arbitrary witness t and by progress exactly when B is a literal equation and t is the literal 0. Its formulas are equations and $D_{x,y}B$, its contexts are lists of clauses (Section 1.9), and an assumption F is the clause $\mathsf{a}(F) := [0 = 0;\ v.F]$, used by REFL at 0 and USE. Here the scope is written explicitly (Theorem 7.1): ELIM appends its parameter b and the introduction its eigenvariable z, and the witness of INTRO is over Π. The calculus C_β has the rules REFL, EVAL, USE and ELIM of C and, in place of INTRO,

$$\frac{\Pi, z;\ \Gamma, \mathsf{a}(B(t_\beta, \eta)) \vdash B(t_\beta, Sz)}{\Pi; \Gamma \vdash D_{x,y}B}\ \text{BIT-INTRO}$$

with t_β and η as in BIT-DI. There are no other axioms or rules.

By Theorem 5.6, REFL and the two cases of EVAL derive, over every clause context, symmetry, transitivity, the two computation equations, congruence in every term template and transport in a term and in an equation. These derivations contain no ELIM and no introduction, so they are derivations of C_β as well and are valid in every scope.

Theorem 7.40 (The one-bit clause calculus)**.** *C_β and C derive the same judgments $\Pi; \Gamma \vdash A$, for all clause contexts Γ.*

Proof. From C_β to C. An introduction with the bit 0 is the introduction of C at the literal 0, with the same mode. With the bit 1 in a nonempty scope it is the introduction of C at a variable, by cases. With the bit 1 in the empty scope it is by cases at 0. For a compound

matrix this is the introduction of C at 0. For an equation matrix, C introduces at the term $w = I(0,0;\, u.\, 0)$, which is not the literal 0, so the mode is by cases; the assumption $B(w,0)$ is transported along $w = 0$ to $B(0,0)$, the given premise is applied, and its conclusion is transported back to $B(w,Sz)$. The calculus C is closed under injective renaming and weakening and admits the cut of an assumption (Theorem 5.5), so this composition is a derivation.

From C *to* C_β. Write $\Pi;\Gamma \Vdash_c A$ if for every injective map ρ from Π into a scope Π' and every clause context Δ over Π' that contains $\rho\Gamma$, C_β derives $\Pi';\Delta \vdash \rho A$. The renaming acts on the antecedent and on the body of each clause, and the receiving context retains the clauses themselves.

Cut of an assumption. If $\Pi;\Gamma,\mathsf{a}(A) \vdash F$ and $\Pi;\Gamma \Vdash_c A$, then $\Pi;\Gamma \vdash F$; this is proved as Theorem 7.29. A USE of the clause $\mathsf{a}(A)$ is replaced by the member of the family in the scope of that leaf, its premise $0 = 0$ being dropped; every other USE keeps its clause. ELIM and BIT-INTRO add their clauses in both contexts. No clause other than an assumption is cut.

Rules. REFL, EVAL and USE hold with $\Vdash_c$, the clause of a USE being a member of the renamed context; ELIM holds with $\Vdash_c$ by sending its eigenvariable to a fresh parameter of the receiving scope and retaining the new clause there.

Closures. Write $q_y A(y)$ for $D_{x,y}A(y)$ with x not free in A. Its introduction by cases is BIT-INTRO with the bit 1: the witness is unused, and the fallback in the empty scope is still by cases. For an equation E its introduction by progress is the bit 0. Its elimination is ELIM followed by USE of the clause $[A(0);\, y.A(y)]$. With hereditary premises these three operations hold for $\Vdash_c$, as in Theorem 7.30. In particular equation induction holds for $\Vdash_c$, and (7.3) holds with $\Vdash_c$ by induction on a derivation of $\mathcal{E}$.

Leibniz rule. If $\Pi;\Gamma \Vdash_c s = t$ and $\Pi;\Gamma \Vdash_c F(s)$, then $\Pi;\Gamma \Vdash_c F(t)$, by induction on F. For an equation this is the equality rules. For $D_{x,y}B(s,x,y)$, open the formula at a fresh b in the receiving scope, retaining the clause $[B(s,b,0);\, y.B(s,b,y)]$, and introduce the target with the bit 1, which selects b. From the assumption $B(t,b,0)$ the induction hypothesis gives $B(s,b,0)$, USE of the retained clause at Sz gives $B(s,b,Sz)$, and the induction hypothesis gives $B(t,b,Sz)$; the cut of an assumption composes these steps. ELIM discharges b.

Packet. With T as in Theorem 7.22, $T(0)$ is introduced with the bit 0 in every scope: the matrix is an equation, the introduction is by progress, and the instance asked for holds by computation. For $q_v T(v)$ use the bit 1 on the outer formula; in its premise the eigenvariable z is the newest parameter, and $T(Sz)$ is introduced with the bit 1, which selects z. The construction is repeated in every scope. ELIM and USE give $\Pi;\Gamma \Vdash_c T(t)$ for every t. Opening $T(St)$ at a fresh b and using its clause at 1 gives $Sb = St$, hence $b = t$, with $\Vdash_c$ from the opened clause context.

Translation. By induction on a derivation of C its conclusion is shown to be hereditarily derivable. REFL, EVAL, USE and ELIM are treated above. An introduction by progress at the literal 0 is the bit 0. For an introduction by cases at a term t, open $T(S\rho t)$ at a fresh b, retain $\mathsf{a}(b = \rho t)$ for the moment, and introduce the formula with the bit 1. In the premise the Leibniz rule turns the assumption $B(b,0)$ into $B(\rho t,0)$. The family given by the induction hypothesis is taken at the map that keeps the images of the source parameters, sends the source eigenvariable to the new one and skips b; the cut of an assumption supplies its assumption, and the Leibniz rule gives $B(b,Sz)$. After the introduction, the cut of an assumption replaces $\mathsf{a}(b = \rho t)$ by its hereditary derivation from the opened packet, and ELIM discharges b. □

Corollary 7.41. *The ordinary realizations of* R_ν, R_β, $\mathsf{N}_{\mathrm{diag},\nu}$ *and* C_β *are bi-interpretable and equiconsistent with* iPRA*, with whole numerical domains, actual equality and the identity graph in both numerical composites.*

Proof. Each calculus derives the same judgments, on the same terms and formulas, as a calculus for which the bridge of Theorem 1.119 is established: N for the first three, and the

unrestricted clause calculus C for the last, which derives the same sequents as N under the translation of Theorem 5.11. The translations and the derivations of both formula round trips transfer unchanged. □

7.5.8 The newest parameter in the Alias/Back calculus

This subsection draws on `local_introduction`; its equality interface is the Alias/Back basis of the chapter on discharge, from `alias_back`.

Theorem 7.38 removes the witness from the diagonal introduction in a calculus with the primitive transport TR, and it recovers an arbitrary witness t from the packet $X(St)$, whose opening supplies a parameter b with $b = t$. The Alias/Back basis has no primitive transport: equality is given by a fresh alias, discharged by substitution, and by a backward substitution at the newest parameter. There the alias itself supplies the parameter b with $b = t$, and the witness is removed without the packet, without names, without a predecessor and without normalization. The mode of the introduction is the diagonal mode of Section 7.5.5. The proof below has the shape of the proof of Theorem 7.35 and differs from it in three respects: equality on families is built from ALIAS and BACK (Theorem 7.45); the source clauses are given by realizers and not by literal membership, because the transported assumption of an introduction is not the image of the source assumption (this replaces the cut of an assumption in the proof of Theorem 7.40); and one induction removes, together with the witness, the arbitrary terms of USE and the arbitrary sources of BACK.

Definition 7.42 (The calculi A_ν and A_*)**.** Terms are those of Section 1.1; formulas are equations and $D_{x,y}B$, without Q. Substitution is capture avoiding and simultaneous, inserts each replacement once and performs no computation. A context is a list of clauses $[A;\, y.B(y)]$ and $[\,;\, y.B(y)]$, read as $A \to \forall y\, B(y)$ and $\forall y\, B(y)$; an assumption F is the clause $[\,;\, v.F]$ with v not in F. Judgments are $\Pi; \Gamma \vdash A$ as in Theorem 7.1, with the clauses of Γ over Π. Put

$$\epsilon_B(z) := \begin{cases} z, & B \text{ is diagonal in } y, \text{ that is, literally } I(y,y;\, u.\, h) = I(y,y;\, v.\, k), \\ 0, & \text{otherwise,} \end{cases}$$

the diagonal test of Theorem 3.12 and Theorem 5.32, made on the matrix $B(x,y)$ before a witness is substituted. The calculus A_ν has exactly the following seven rule patterns.

(i) I0: $\Pi; \Gamma \vdash I(0,a;\, u.\, h) = a$.

(ii) IS: $\Pi; \Gamma \vdash I(Sn,a;\, u.\, h) = h[I(n,a;\, u.\, h)/u]$.

(iii) USE: for a clause $[A;\, y.B]$ of Γ, from $\Pi; \Gamma \vdash A$ infer $\Pi; \Gamma \vdash B(\nu(\Pi))$; for a clause $[\,;\, y.B]$ of Γ, infer $\Pi; \Gamma \vdash B(\nu(\Pi))$ outright.

(iv) BACK: if Π is not empty and $b = \nu(\Pi)$, from $\Pi; \Gamma \vdash b = t$ and $\Pi; \Gamma \vdash F[t/b]$ infer $\Pi; \Gamma \vdash F$. The term t may contain b.

(v) ALIAS: if $b \notin \Pi$ and t is over Π, from $\Pi, b;\ \Gamma, b = t \vdash F$ infer $\Pi; \Gamma \vdash F[t/b]$. The formula F may contain b.

(vi) OPEN: if $b \notin \Pi$ and A is over Π, from $\Pi; \Gamma \vdash D_{x,y}B$ and $\Pi, b;\ \Gamma, [B(b,0);\, y.B(b,y)] \vdash A$ infer $\Pi; \Gamma \vdash A$.

(vii) CLOSE: if $z \notin \Pi$, from $\Pi, z;\ \Gamma, B(\nu(\Pi), \epsilon_B(z)) \vdash B(\nu(\Pi), Sz)$ infer $\Pi; \Gamma \vdash D_{x,y}B$.

The calculus A_* has the same syntax, I0, IS, ALIAS and OPEN, a USE at an arbitrary term r over Π in place of $\nu(\Pi)$, a BACK at an arbitrary parameter b of Π, and a CLOSE at an arbitrary witness t over Π in place of $\nu(\Pi)$, with the same mode ϵ_B.

CLOSE of A_ν takes no term. There is no reflexivity rule, transport rule, normalizer, primitive specialization at a chosen term or logical connective. The rules other than CLOSE are those of the Alias/Back basis, whose USE and BACK are already at the newest parameter; its introduction takes an arbitrary witness and is by progress exactly when the matrix is a literal equation and the witness is literally 0. A_ν changes only the introduction: the witness is the newest parameter, and the mode is read off the matrix alone. Every rule of A_ν is a rule of A_*. Every rule that opens a branch appends one parameter and one clause, so along a derivation of a closed judgment the length of the scope equals the length of the context.

Lemma 7.43 (Soundness)**.** *Every rule of A_* is valid in T_{eq} under the ordinary reading of Section 1.2 and of the clauses.*

Proof. The computations are axioms of T_{eq} (Section 1.5). USE is the instantiation of the universal quantifier of the clause, after modus ponens for a clause with an antecedent. BACK is the substitution of equals. For ALIAS, the ordinary derivation of the branch is closed under the substitution of t for the variable b, which occurs neither in Γ nor in t; the assumption becomes $t = t$, which reflexivity proves. OPEN is the elimination of the existential quantifier of $\exists x\,(B(x,0) \to \forall y\, B(x,y))$ at the fresh b. For CLOSE at the witness t: if B is diagonal it is a literal equation, its premise is a progressive step, and induction for literal equations proves $B(t,0) \to \forall y\, B(t,y)$; otherwise the premise is a case step, and coverage proves the same implication. The existential quantifier is introduced at t. □

Definition 7.44 (Persistent families and realizers)**.** A *world* is a pair $w = (\Pi_w; \Gamma_w)$ of a scope and a context over it. A world w' *extends* w, written $w' \succeq w$, if $\Pi_{w'}$ is Π_w followed by further parameters and $\Gamma_{w'}$ is Γ_w followed by further clauses over $\Pi_{w'}$. For a formula A over Π_w, write $w \Vdash A$ if there is a *persistent family* for A at w: an effective procedure that assigns to every $w' \succeq w$ a derivation of A_ν of $\Pi_{w'}; \Gamma_{w'} \vdash A$. A *persistent realizer* at w of a clause $[A;\, y.B]$ over Π_w is an effective procedure that assigns to every $w' \succeq w$, every term r over $\Pi_{w'}$ and every persistent family for A at w' a persistent family for $B(r)$ at w'; for a clause $[\,;\, y.B]$ the family argument is absent.

A family or realizer at w is one at every $w' \succeq w$, by the same procedure. A family for E at w is a realizer at w of the assumption $[\,;\, v.E]$: it ignores the term r, since $E[r/v] \equiv E$. Families are the hereditary derivability of Theorem 7.27 restricted to extensions; the injective map is carried by the induction statement of Theorem 7.46 instead. They and the realizers are constructions of the metatheory and are not rules of the calculus.

Lemma 7.45 (Clauses and equality on families)**.** *Let w be a world, s, t terms over Π_w and $H(v)$ a formula with a distinguished variable v, which may occur any number of times and below binders.*

(i) *Every clause of Γ_w has a persistent realizer at w.*

(ii) *$w \Vdash t = t$.*

(iii) *If $w \Vdash s = t$ and $w \Vdash H(t)$, then $w \Vdash H(s)$.*

(iv) *If $w \Vdash s = t$, then $w \Vdash t = s$; if moreover $w \Vdash H(s)$, then $w \Vdash H(t)$.*

Proof. In each item let $w' \succeq w$ be given, and let c be a parameter not in $\Pi_{w'}$ and not in s, t, H or the terms named below.

(i) Let $[A;\, y.B]$ be a clause of Γ_w, $w' \succeq w$, r a term over $\Pi_{w'}$, and a family for A at w' be given; let $w'' \succeq w'$. The world $w''_c = (\Pi_{w''}, c;\ \Gamma_{w''}, c = r)$ extends w', so the family gives A there. The clause is a member of $\Gamma_{w''_c}$ and c is the newest parameter, so USE gives $B(c)$,

and ALIAS gives $B(c)[r/c] \equiv B(r)$ at w''. For a clause without antecedent the family is not needed.

(ii) In $(\Pi_{w'}, c;\ \Gamma_{w'}, c = t)$, USE of the last clause gives $c = t$, and ALIAS gives $(c = t)[t/c] \equiv (t = t)$ at w'.

(iii) Work in $w'_c = (\Pi_{w'}, c;\ \Gamma_{w'}, c = s)$, which extends w. USE of the last clause gives $c = s$, and the first family gives $s = t \equiv (c = t)[s/c]$; BACK at c gives $c = t$. The second family gives $H(t) \equiv H(c)[t/c]$, and BACK at c with the major premise $c = t$ gives $H(c)$. ALIAS gives $H(c)[s/c] \equiv H(s)$ at w'.

(iv) Apply (iii) to the template $t = v$, whose instance $t = t$ holds by (ii); the result is $t = s$. For the second statement apply (iii) to $t = s$ and $H(s)$.

Both applications of BACK in (iii) are at c, which is the newest parameter of w'_c; the derivations of their premises are the members of the given families at w'_c itself, and no derivation is moved to another world. These constructions use no computation, induction, coverage, weakening of a completed derivation or substitution into one. □

Theorem 7.46 (No witness in the Alias/Back calculus)**.** *For every scope* Π*, clause context* Γ *and formula* A *over* Π*,*

$$\Pi; \Gamma \vdash_{\mathsf{A}_\nu} A \quad \Longleftrightarrow \quad \Pi; \Gamma \vdash_{\mathsf{A}_*} A,$$

by effective transformations of derivations that leave the terms, formulas and clauses of the judgment unchanged.

Proof. From left to right every rule of A_ν is a rule of A_*. For the converse the following *induction statement* is proved by induction on a derivation of A_* of $\Pi; \Gamma \vdash A$: for every world w, every injective map θ from Π into Π_w and every assignment of a persistent realizer at w of θC to each clause C of Γ, $w \Vdash \theta A$. Here θ acts on free parameters; since it is injective, $\theta(F[t/b]) \equiv (\theta F)[\theta t/\theta b]$, and $\epsilon_{\theta B} = \epsilon_B$, because the diagonal shape concerns the bound variable y and the literal identity of two iterator terms, both preserved and reflected by an injective renaming. The required derivation of $\Pi; \Gamma \vdash A$ is the member at $w = (\Pi; \Gamma)$ of the family that the induction statement gives for the identity map and the realizers of Theorem 7.45(i). In the cases below, $w' \succeq w$ is the world at which a member is constructed.

Computations. The image of an instance is an instance, in every world.

Use of a clause $C = [A;\ y.B]$ at the term r. The induction hypothesis gives a family for θA at w; the realizer of θC at the term θr turns it into a family for $\theta B(\theta r) \equiv \theta(B(r))$. For a clause without antecedent no family is needed.

Back at a parameter b of Π, with premises $b = t$ and $F[t/b]$. Let $H(v) := (\theta F)[v/\theta b]$ with v fresh; then $H(\theta t) \equiv \theta(F[t/b])$ and $H(\theta b) \equiv \theta F$. The induction hypothesis gives families for $\theta b = \theta t$ and for $H(\theta t)$, and Theorem 7.45(iii) gives $w \Vdash \theta F$.

Alias with the parameter b, the term t and the branch $\Pi, b; \Gamma, b = t \vdash F$. Choose $b' \notin \Pi_{w'}$ and let w'' be $(\Pi_{w'}, b';\ \Gamma_{w'}, b' = \theta t)$. The map $\theta' = \theta[b \mapsto b']$ is injective; the clauses of Γ keep their realizers, restricted to w'', and the new clause $b = t$, whose image is $b' = \theta t$, is realized by Theorem 7.45(i). The induction hypothesis gives $\theta' F$ at w'', and ALIAS gives $(\theta' F)[\theta t/b'] \equiv \theta(F[t/b])$ at w'.

Open with the major premise $D_{x,y}B$ and the parameter b. The induction hypothesis gives $\theta D_{x,y}B$ at w'. Choose $b' \notin \Pi_{w'}$ and extend θ by $b \mapsto b'$; the opened clause is realized in $w'' = (\Pi_{w'}, b';\ \Gamma_{w'}, [\theta B(b', 0);\ y.\theta B(b', y)])$ by Theorem 7.45(i). The induction hypothesis gives θA at w'', since A does not contain b, and OPEN applies.

Close at the witness t with the matrix B and the premise $\Pi, z;\ \Gamma, B(t, \epsilon_B(z)) \vdash B(t, Sz)$. Choose $b \notin \Pi_{w'}$ and apply ALIAS with the defining term θt; its branch is the world $w_b = (\Pi_{w'}, b;\ \Gamma_{w'}, b = \theta t)$. In w_b apply CLOSE to $D_{x,y}\theta B$. The newest parameter is b, so the rule selects b; let z' be its eigenvariable. Its premise is to be derived in

$$w_z = (\Pi_{w'}, b, z';\ \Gamma_{w'},\ b = \theta t,\ \theta B(b, \epsilon_B(z'))).$$

Put $\theta' = \theta[z \mapsto z']$, an injective map from Π, z into Π_{w_z}; the parameter b is not in its image. The clauses of Γ keep their realizers, restricted to w_z. The source assumption $B(t, \epsilon_B(z))$ has the image $\theta B(\theta t, \epsilon_B(z'))$, which is not a clause of w_z. By Theorem 7.45(i) for the last two clauses of w_z, which are assumptions, $w_z \Vdash b = \theta t$ and $w_z \Vdash \theta B(b, \epsilon_B(z'))$, and Theorem 7.45(iv) with the template $\theta B(v, \epsilon_B(z'))$ gives $w_z \Vdash \theta B(\theta t, \epsilon_B(z'))$; this family is the realizer of the source assumption. The induction hypothesis for the premise gives $w_z \Vdash \theta B(\theta t, Sz')$, and Theorem 7.45(iii) with the template $\theta B(v, Sz')$ and the equation $b = \theta t$ gives $w_z \Vdash \theta B(b, Sz')$. Its member at w_z is the premise of the target CLOSE, whose mode is $\epsilon_{\theta B} = \epsilon_B$. The conclusion $D_{x,y}\theta B$ does not contain b, so ALIAS yields $(D_{x,y}\theta B)[\theta t/b] \equiv \theta D_{x,y} B$ at w'.

The eigenvariable z' is newer than b; this does no harm, because every transport of Theorem 7.45 opens its own auxiliary parameter, which is then the newest. The construction is well founded. The realizer of the source assumption in the case of CLOSE is built from the actual clauses of w_z and does not call the translation of the premise. Every call of the induction hypothesis concerns a proper subderivation, although it may be made in several worlds; USE calls realizers on families of proper subderivations; the constructions of Theorem 7.45 call the given families finitely often. No derivation of A_ν is moved to another world, and no term is substituted into one. □

Corollary 7.47 (The Alias/Back calculi and iPRA)**.** *For a clause context* Γ *let* Γ° *be* Γ *with every clause* $[\ ; y.B]$ *replaced by* $[0 = 0; y.B]$*. Then*

$$\Pi; \Gamma \vdash_{\mathsf{A}_\nu} A \iff \Pi; \Gamma \vdash_{\mathsf{A}_*} A \iff \Gamma^\circ \vdash_{\mathsf{K}} A,$$

where K *is the diagonal clause calculus of Theorem 5.32. The ordinary realizations of* A_ν *and* A_* *are bi-interpretable and equiconsistent with* iPRA*, with whole numerical domains, actual equality and the identity graph in both numerical composites.*

Proof. The first equivalence is Theorem 7.46. In A_*, injective renaming and weakening by appended parameters and clauses are admissible, by induction on derivations after the eigenvariables are made fresh: no rule of A_* refers to the order of the scope, the source of a BACK is renamed to a parameter, and ϵ_B is preserved. A derivation of A_* therefore gives a derivation in every extension, and the constructions of Theorem 7.45, which are derivations of A_*, give reflexivity, symmetry, transitivity and transport in every formula over every scope and context. In A_* the clauses $[\ ; y.B]$ and $[0 = 0; y.B]$ are interchangeable: a USE of the second drops its premise when it becomes a USE of the first, and a USE of the first becomes a USE of the second with the premise $0 = 0$ by reflexivity.

From K *to* A_*. By induction on a derivation of $\Gamma^\circ \vdash_{\mathsf{K}} A$ in the scope Π. First its eigenvariables are made fresh for Π (Theorem 5.5, which holds in K), and every other variable that occurs in it and is not in Π is replaced by 0 (Theorem 5.36); the judgment is unchanged, and every term of the derivation is then over the scope of its node. REFL is reflexivity. The zero row of EVAL, from $a = t$ and $I(0, a; u.\, h) = r$, gives $r = t$ by I0, symmetry and transitivity; the successor row, from $I(n, a; u.\, h) = t$ and $I(Sn, a; u.\, h) = r$, gives $r = h[t/u]$ by IS, transport in the template $I(Sn, a; u.\, h) = h[v/u]$, symmetry and transitivity. USE, ELIM and INTRO are USE, OPEN and CLOSE at the same term, the same parameter and the same witness, with the same mode, since ϵ_B is the mode η_B of Theorem 5.32. The result is a derivation of $\Pi; \Gamma^\circ \vdash A$, hence of $\Pi; \Gamma \vdash A$.

From A_* *to* K. By induction on a derivation of $\Pi; \Gamma \vdash A$, with the context Γ°. The computation equations, symmetry and transport in every formula hold in K (Theorem 5.6 and Theorem 5.33(iii)). USE is USE, with the premise $0 = 0$ by REFL for a clause without antecedent. BACK at b is transport in the template $F[v/b]$ along $t = b$. For ALIAS, the branch gives $\Gamma^\circ, [0 = 0; v.\, b = t] \vdash_{\mathsf{K}} F$; substitution of t for b (Theorem 5.36) gives $\Gamma^\circ, [0 =$

$0;\, v.\, t = t] \vdash_{\mathsf{K}} F[t/b]$, since b is not in Γ, and the cut of the ordinary assumption against REFL (Theorem 5.5) removes the clause. OPEN is ELIM and CLOSE is INTRO at the same witness with the same mode; the side conditions of K on fresh variables follow from those on scopes.

By Theorem 5.35, K derives, for Q-free formulas and the translation τ of native formulas, the same judgments as N (`kprf_iff_nprf` and `kprf_iff_nprf_of_qfree`, Theorem 5.37). The bridge of Theorem 1.119 applies as in Theorem 7.41: the comparisons are the identity on terms and on Q-free formulas, $Q_y B$ is represented by $D_{x,y}B$ with x unused, and the translations, both formula round trips and the preservation of $0 = S0$ transfer unchanged. □

Remark 7.48 (What the proof uses of the mode). The proof of Theorem 7.46 uses the mode only in the case of CLOSE, and there only through $\epsilon_{\theta B} = \epsilon_B$ for injective renamings θ. The same proof therefore removes the witness from every introduction of the form $\Gamma, B(t, \epsilon_B(z)) \vdash B(t, Sz)$ over $\Gamma \vdash D_{x,y}B$, with $\epsilon_B(z) \in \{0, z\}$ determined by the uninstantiated matrix and preserved and reflected by injective renaming, in a calculus with the same ALIAS, USE and BACK. It is a theorem of relative derivability; soundness and arithmetic adequacy of the selected mode are separate obligations. The mirror guard of Theorem 5.39 qualifies: an injective renaming preserves and reflects the shape $a = I(y, a;\, u.\, h)$, although an arbitrary substitution can create it, and the proof uses no invariance under substitution.

The mode of the introduction of the Alias/Back basis does not qualify: it tests whether the witness is literally 0, and the alias b is a parameter. A source introduction at 0 in the progress mode would become a target introduction at b in the case mode, whose assumption $\theta B(b, 0)$ does not transport to the source assumption $\theta B(0, z')$, and the proof does not apply. Replacing the witness by the newest parameter while keeping that mode is not justified by it; the one-bit rule of Section 7.5.6 is a different repair. The proof uses BACK on compound formulas, in the two transports of the case of CLOSE, when B has a binder.

Remark 7.49 (The package). `local_introduction` states the theorem for the temporary calculus A_* with persistent families over worlds and realizers, as above, and the corollary as the equality of the open consequences of the two diagonal alias calculi together with the inherited bridge; the comparison with K is made explicit by the adapters for REFL, the two rows of EVAL and the clauses without antecedent, which are those of the proof above. The package includes an independent reading of the argument, which records the six assumptions the proof needs: BACK on compound formulas, the substitutive export of ALIAS, the mode tested on the raw matrix, the retention of old parameters in an extension, effective realizers quantified over all extensions, and capture-avoiding simultaneous substitution. The theorem is not formalized in Lean or Rocq.

7.6 The raw guarded calculus

This section draws on `deterministic_introduction` for the naming exception, and on `uniform_raw` and the guarded companion of `newest_parameter` for the calculus without it.

7.6.1 The calculus and its three introductions

Terms are the raw terms of Section 1.1; no computation is built into syntactic identity or into substitution. A *canonical formula* is an equation or a formula $D_{x,y}B$ in which B is canonical and, if B is an equation, it has the literal shape

$$I(y, 0;\, u.\, h) = I(y, 0;\, v.\, k). \tag{7.6}$$

The bodies h and k may contain x, y and parameters. An equation at the root of a formula is unrestricted. Substitution of terms for parameters preserves canonical formulas. Contexts are

lists of clauses $[A;\, y.B(y)]$ of canonical formulas, with y not free in A. Put $H := (I(0,0;\, u.\,0) = 0)$, an instance of I0; an assumption A is the clause $\mathsf{a}(A) := [H;\, v.A]$ and is used by USE with one computation.

The calculus G has the two computation equations as axioms in every context and four inference rules:

$$\frac{\Pi;\Gamma \vdash A}{\Pi;\Gamma \vdash B(t)}\ \textsc{use}\ ([A;\, y.B(y)] \in \Gamma) \qquad \frac{\Pi;\Gamma \vdash s = t \qquad \Pi;\Gamma \vdash F(s)}{\Pi;\Gamma \vdash F(t)}\ \textsc{tr}$$

$$\frac{\Pi;\Gamma \vdash D_{x,y}B \qquad \Pi, b;\ \Gamma, [B(b,0);\, y.B(b,y)] \vdash A}{\Pi;\Gamma \vdash A}\ \textsc{open} \qquad \frac{\Pi, z;\ \Gamma, \mathsf{a}(B(t,\eta_B(z))) \vdash B(t,Sz)}{\Pi;\Gamma \vdash D_{x,y}B}\ \textsc{intro}$$

with $\eta_B(z) = z$ if B is an equation and $\eta_B(z) = 0$ if B is compound. In TR, $F(v)$ is a canonical formula template. The mode of INTRO depends on the root of the matrix and not on the witness. There is no primitive hypothesis rule, reflexivity, weakening, cut, coverage or induction rule. The ordinary reading of $D_{x,y}B$ is that of Section 1.2; the comparison of G with N, by padding equation matrices, and hence the bridge to iPRA, is established for G in `guarded_core` and is inherited here.

Definition 7.50 (Three selections of the witness)**.** In G the witness t of INTRO is an arbitrary term over Π. In G_ν it is $\nu(\Pi)$. In G_d it is

$$\omega(B,\Pi) := \begin{cases} r, & B \text{ is literally } \mathrm{Nm}_r \text{ with } r \text{ over } \Pi, \\ \nu(\Pi), & \text{otherwise.} \end{cases}$$

A match with a naming matrix has priority, and it is literal up to the names of bound variables: r contains neither x, y nor the bound variables of the two iterators. It is not a match up to provable equality.

Every naming matrix has the shape (7.6). The derivations of G_ν and of G_d are derivations of G once the selected witnesses are written in. In all three calculi reflexivity and symmetry are derived from I0 and TR: with $q = I(0,r;\, u.\,0)$, the equation $q = r$ used as both premises of TR in the template $v = r$ gives $r = r$, and from $s = t$ the transport of $s = s$ in the template $v = s$ gives $t = s$.

7.6.2 The naming exception

In G_d the names are immediate, and the work is structural: weakening to a larger scope can change the selected witness, and the general cut needs the weakening of the inserted derivation. The order of the argument is a restricted cut, then weakening, then the general cut, then completeness.

Lemma 7.51 (Algebraic derivations and restricted cut)**.** *Call a derivation* algebraic *if it uses only the computation axioms,* USE *and* TR*.*

(i) *An algebraic derivation of $\Pi;\Gamma \vdash A$ yields an algebraic derivation of $\Pi';\Delta \vdash \rho A$ for every injective map ρ from Π into Π' and every clause context Δ over Π' into which $\rho\Gamma$ is embedded.*

(ii) *If $\Pi;\Gamma \vdash A$ has an algebraic derivation and $\Pi;\Gamma, \mathsf{a}(A) \vdash F$ in G_d, then $\Pi;\Gamma \vdash F$ in G_d.*

Proof. (i) By induction on the derivation; its rules have no choice that depends on the scope and no eigenvariable. (ii) By induction on the derivation of F. A USE of the clause $\mathsf{a}(A)$ is replaced by the algebraic derivation of A, moved to the scope and context of that leaf by (i). At INTRO and OPEN the induction continues in the extended branch. Deleting a clause does not change the scope, so every INTRO selects the same witness before and after. No weakening of a derivation that contains an INTRO is used. □

Lemma 7.52 (Every term has a name)**.** *For every term r over Π, G_d derives $\Pi;\Gamma \vdash D_{x,y}\mathrm{Nm}_r$. If a clause context contains $[\mathrm{Nm}_r(b,0);\, y.\mathrm{Nm}_r(b,y)]$, then $b = r$ and $r = b$ have algebraic derivations from it.*

Proof. The rule selects r by the naming alternative. The matrix is an equation, so the premise asks for $\mathrm{Nm}_r(r, Sz)$ from $\mathrm{Nm}_r(r,z)$; the conclusion is literally $\mathrm{sel}(Sz,0,r) = \mathrm{sel}(Sz,0,r)$ and is a reflexivity. For the second statement, both sides of $\mathrm{Nm}_r(b,0)$ equal 0 by I0, so the antecedent of the clause is derivable; USE at 1 gives $\mathrm{sel}(1,0,b) = \mathrm{sel}(1,0,r)$, and IS with TR gives $b = r$; symmetry gives $r = b$. □

Theorem 7.53 (Weakening by naming the old selection anew)**.** *Let $\Pi;\Gamma \vdash A$ be derivable in G_d. For every injective map ρ from Π into a scope Π' and every clause context Δ over Π' into which $\rho\Gamma$ is embedded, $\Pi';\Delta \vdash \rho A$ is derivable in G_d, by an effective construction.*

Proof. By induction on the height of the given derivation, for all ρ and Δ. The computation axioms, USE and TR are immediate. At OPEN, choose a fresh parameter of the receiving scope, extend ρ and the embedding of clauses, and apply the induction hypothesis to the two premises.

Consider an INTRO with the matrix B and the selected witness w, and let $B' = \rho B$, $w' = \rho w$. A match with a naming matrix is preserved and reflected by ρ: a bound variable does not become a parameter or conversely, and a parameter does not become a numeral or a compound term. If $B = \mathrm{Nm}_r$, the witness selected in the receiving scope is $\rho r = w'$, and the induction hypothesis for the premise gives the rule directly. The same holds if B is not a naming matrix and $\nu(\Pi') = w'$.

Suppose B' is not a naming matrix and $\nu(\Pi') \neq w'$. Derive $D_{x,y}\mathrm{Nm}_{w'}$ by Theorem 7.52 and open it at a fresh parameter e, which is then the newest; its clause gives algebraic derivations of $e = w'$ and $w' = e$. The formula $D_{x,y}B'$ will be introduced in the scope Π', e, where the rule selects e. Let v be the eigenvariable of that introduction and put

$$A_\mathrm{old} = B'(w', \eta_{B'}(v)), \qquad A_\mathrm{new} = B'(e, \eta_{B'}(v)).$$

Apply the induction hypothesis to the premise of the given INTRO, with ρ extended by sending its eigenvariable to v, and with the receiving context consisting of Δ, the clause of the name, $\mathsf{a}(A_\mathrm{old})$ and $\mathsf{a}(A_\mathrm{new})$. This is legitimate although e and v are new parameters of the receiving scope, because the induction statement quantifies over all injective maps of the smaller derivation. The result derives $B'(w', Sv)$. From A_new, TR along $e = w'$ gives A_old; this derivation is algebraic. Theorem 7.51(ii) removes the clause $\mathsf{a}(A_\mathrm{old})$. TR along $w' = e$ gives $B'(e, Sv)$. Apply INTRO, which selects e, and discharge the name by OPEN; the conclusion does not contain e. If the old selection was 0 in an empty scope, the name is that of 0.

Every use of the induction hypothesis is for a proper subderivation of the given derivation. The naming and transport derivations built here are never fed back into the induction, and the restricted cut was established before. The construction is therefore well founded and uses neither the general cut nor weakening. □

Corollary 7.54 (Cut of an assumption)**.** *If $\Pi;\Gamma \vdash A$ and $\Pi;\Gamma, \mathsf{a}(A) \vdash F$ in G_d, then $\Pi;\Gamma \vdash F$ in G_d.*

Proof. By induction on the derivation of F. A USE of $\mathsf{a}(A)$ is replaced by the derivation of A, moved to the scope and context of that leaf by Theorem 7.53. Deleting a clause leaves the scope unchanged, so the introductions of the receiving derivation keep their selections. □

Theorem 7.55 (The introduction with a naming exception is complete)**.** *For every scope Π, canonical clause context Γ and canonical formula A,*

$$\Pi;\Gamma \vdash_{\mathsf{G}_\mathrm{d}} A \quad\Longleftrightarrow\quad \Pi;\Gamma \vdash_{\mathsf{G}} A.$$

Proof. From left to right the selected witness is a witness of INTRO in G. For the converse induct on the derivation in G. At an INTRO whose matrix is a naming matrix Nm_r, discard the supplied witness and the premise and use Theorem 7.52. At any other INTRO, with the witness t, translate the premise by the induction hypothesis, derive $D_{x,y}\mathrm{Nm}_t$ and open it at a fresh b. In the context with the opened name and the assumption $B(b, \eta_B(z))$, TR along $b = t$ gives $B(t, \eta_B(z))$. Theorems 7.53 and 7.54 insert the translated premise and give $B(t, Sz)$, and TR along $t = b$ gives $B(b, Sz)$. The newest parameter of the scope is b and B is not a naming matrix, so the rule selects b. Close the introduction and the opening of the name. □

Remark 7.56 (Why the qualifications matter). The simulation changes the witness from t to b and must not change the assumption of the premise. In G the mode depends on the root of the matrix only. A rule that selects progress exactly when the witness is the literal 0 does not fit this argument as it stands: at $t = 0$ it has the assumption $B(0, z)$ and at b the assumption $B(b, 0)$; the calculus R of Section 7.4.3 is treated by recovering the two modes separately. The diagonal selection of Section 7.5.5 is independent of the witness as well. The recognition of a naming matrix is a fixed syntactic match. Replacing it by the condition that some instance of the matrix names the witness would need additional data or a search, and a rule about a provable naming instance would reintroduce a premise. The proof uses transport in arbitrary canonical formulas: the donor of the restricted cut converts $B(e, \eta)$, which may be compound. Through the comparison, substitution is admissible in G_d as a transformation of derivable judgments; a certificate is not claimed to remain valid after a substitution.

7.6.3 Uniform introduction: the stable fragment

In G_ν the name $D_{x,y}\mathrm{Nm}_r$ is no longer immediate, and the names are generated as in Section 7.4. The guarded calculus has no rule for equation induction, so the predecessor has to be obtained first, in a part of the calculus that is insensitive to the selected witness.

Definition 7.57. An INTRO of G_ν with the matrix $B(x, y)$ is *dummy* if x is not free in B; a derivation is *stable* if all its introductions are dummy.

Lemma 7.58 (Stability)**.** *Stable derivations of G_ν are closed under the substitution of terms over a scope Π' for the parameters, with any clause context over Π' that contains the substituted clauses, and under the cut of an assumption by a stable derivation.*

Proof. As Theorem 7.26. The premise of a dummy introduction does not depend on the witness; a substitution does not change whether the matrix is an equation, and it preserves the shape (7.6). □

For an equation E let the *padding* be $L(E) := D_{r,v}\gamma_v(E)$ with r, v not free in E; it is canonical. For equations $U \equiv (s = t)$ and $V \equiv (p = q)$ let

$$U \Rightarrow_g V := D_{r,w}L(\mathrm{sel}(w, s, p) = \mathrm{sel}(w, t, q)),$$

a canonical formula with a compound matrix and an unused coordinate r.

Lemma 7.59 (Padding, implication and cases without names)**.** *The following hold for stable derivations of G_ν, in every scope and context.*

(i) *E and $L(E)$ are interderivable.*

(ii) *If $\Pi; \Gamma, \mathsf{a}(U) \vdash V$, then $\Pi; \Gamma \vdash U \Rightarrow_g V$; and $U \Rightarrow_g V$ together with U gives V.*

(iii) *If $\Pi; \Gamma \vdash F(0)$ and $\Pi, z; \Gamma \vdash F(Sz)$, then $\Pi; \Gamma \vdash F(t)$ for every term t; here $F(y)$ is a compound formula or an equation.*

(iv) *If E is an equation and t a term over Π, and $\Pi;\Gamma,\mathsf{a}(t=0)\vdash E$ and $\Pi,z;\ \Gamma,\mathsf{a}(t=Sz)\vdash E$, then $\Pi;\Gamma\vdash E$.*

Proof. (i) From E: introduce $L(E)$. The matrix $\gamma_v(E)$ is an equation, so the premise asks for $\gamma_{Sz}(E)$ from $\gamma_z(E)$; it follows from E by IS and TR, and the assumption is unused. From $L(E)$: open it at a fresh b; the antecedent $\gamma_0(E)$ of the clause holds by I0, USE at 1 gives $\gamma_1(E)$, and IS gives E, which does not contain b.

(ii) Write row_w for $\mathrm{sel}(w,s,p)=\mathrm{sel}(w,t,q)$. The matrix is compound, so the introduction is by cases and asks for $L(\mathrm{row}_{Sz})$ from $L(\mathrm{row}_0)$. By (i) and I0 the assumption gives U; the cut of the given derivation gives V; IS gives row_{Sz}, and (i) pads it. For the elimination open the formula at a fresh b, derive $L(\mathrm{row}_0)$ from U by I0 and (i), use the clause at 1, and unpad and compute to obtain V.

(iii) For a compound F, introduce $D_{r,y}F(y)$ with r unused; it is by cases, and its premise is the given derivation of $F(Sz)$ weakened by the assumption $F(0)$. Open it at a fresh b and use the clause $[F(0);\ y.F(y)]$ at t, the antecedent being the given derivation moved to the scope Π,b by Theorem 7.58. For an equation $E(y)$ apply this to $L(E(y))$ and use (i).

(iv) Let $F(y)$ be $(t=y)\Rightarrow_g E$. By (ii) the hypotheses give $F(0)$ and $F(Sz)$, by (iii) $F(t)$, and by (ii) with the reflexivity $t=t$ the equation E. □

Lemma 7.60 (Equation induction in the stable fragment)**.** *Let $E(y)\equiv(s(y)=t(y))$. If $\Pi;\Gamma\vdash E(0)$ and $\Pi,z;\ \Gamma,\mathsf{a}(E(z))\vdash E(Sz)$ have stable derivations in G_ν, then so has $\Pi;\Gamma\vdash E(a)$ for every term a over Π. Consequently every derivation of the equational fragment $\mathcal{E}$ is translated into a stable derivation of G_ν, and for all terms r,s*

$$\Pi;\Gamma\vdash \mathrm{pred}(Sr)=r,\qquad \Pi;\ \Gamma,\mathsf{a}(Sr=Ss)\vdash r=s$$

by stable derivations.

Proof. Let $G(y):=\gamma_y(E(y))$, an equation of the shape (7.6). First, $E(z)$ is derivable from $G(z)$: by Theorem 7.59(iv) with the term z, in the branch $z=0$ transport the base $E(0)$ along $0=z$, and in the branch $z=Sw$ transport $G(z)$ to $G(Sw)$, compute the two gates to obtain $E(Sw)$, and transport back. The step then gives $E(Sz)$, by Theorem 7.58, and computation gives $G(Sz)$. This is the premise of the introduction of $D_{r,y}G(y)$ with r unused, which is by progress. Open it at a fresh b; the antecedent $G(0)$ of its clause holds by computation, and USE at a gives $G(a)$. A second application of Theorem 7.59(iv), with the term a and with $E(0)$ in the branch $a=0$, gives $E(a)$. All introductions are dummy.

For the translation of $\mathcal{E}$, an assumption E is the clause $\mathsf{a}(E)$, the equality rules are instances of TR and reflexivity, and induction is the first statement; the instances of earlier lemmas that an equational derivation uses are available by Theorem 7.58. The last statement follows from Theorem 1.65. □

7.6.4 Uniform introduction: names and completeness

Hereditary derivability $\Pi;\Gamma\Vdash A$ is defined for G_ν as in Theorem 7.27, with clause contexts. The computation axioms, USE and TR hold with $\Vdash$; OPEN holds with $\Vdash$ by allocating its parameter in each receiving scope; a stable derivation is a member of a hereditary family; and, as in Theorem 7.29,

$$\Pi;\Gamma\Vdash A\ \text{ and }\ \Pi;\Gamma,\mathsf{a}(A)\vdash B\quad\text{imply}\quad\Pi;\Gamma\vdash B. \tag{7.7}$$

In the proof of (7.7) the donor is generated at each USE of the removed clause in the scope and context of that leaf, the other clause addresses are adjusted, and every other node keeps its scope and therefore its selected witness. The hypotheses of the donor must remain available after the deletion.

The packet $X(y)$ of Theorem 7.18 is canonical: both sides of its matrix are iterators with the counter v and the seed 0.

Lemma 7.61 (The packet in the guarded calculus)**.** *In G_ν, for every scope Π, clause context Γ and term t over Π: $\Pi;\Gamma \vdash D_{a,r}X(r)$ with a unused, and $\Pi;\Gamma \vdash X(St)$.*

Proof. $X(0)$ is introduced at whatever witness the scope selects. Its matrix is an equation, the introduction is by progress, and the instance asked for computes to $0 = 0$ whatever the witness and the assumption are. The matrix of $D_{a,r}X(r)$ is compound, so its introduction is by cases and asks for $X(Sz)$ with a new eigenvariable z. In that branch the scope ends in z, so the introduction of $X(Sz)$ selects z, and the instance asked for computes to $Sz = Sz$. The assumption $X(0)$ of the outer introduction is unused. Open $D_{a,r}X(r)$ at a fresh parameter; its clause is $[X(0);\, r.X(r)]$. Derive $X(0)$ again in the extended scope and use the clause at St. The result does not contain the opened parameter, and OPEN discharges it. □

The lemma is about the whole construction, which allocates z anew in each receiving scope. A separate derivation of $X(Sz)$ is not claimed to remain a derivation after z ceases to be the newest parameter.

Lemma 7.62 (The alias in the guarded calculus)**.** *If a clause context over a scope contains $[M_{St}(b,0);\, v.M_{St}(b,v)]$, then $b = t$ is hereditarily derivable from it.*

Proof. The antecedent holds by computation. USE at 1 and the successor computations give $Sb = St$, and Theorem 7.60 gives $b = t$. The derivation is stable. □

Theorem 7.63 (Uniform introduction on raw terms)**.** *For every scope Π, canonical clause context Γ and canonical formula A,*

$$\Pi;\Gamma \vdash_{\mathsf{G}_\nu} A \quad\Longleftrightarrow\quad \Pi;\Gamma \vdash_{\mathsf{G}} A \quad\Longleftrightarrow\quad \Pi;\Gamma \vdash_{\mathsf{G}_\mathrm{d}} A.$$

The naming exception of G_d is not needed.

Proof. The second equivalence is Theorem 7.55, and $\mathsf{G}_\nu \subseteq \mathsf{G}$ is immediate. For $\mathsf{G} \subseteq \mathsf{G}_\nu$ it is shown by induction on a derivation of G that $\Pi;\Gamma \Vdash A$. The computation axioms, USE and TR are copied with their terms renamed. For OPEN, allocate a parameter in the receiving scope and apply the induction hypothesis to the branch with the extended map.

Consider an INTRO at a term t with the matrix B. The induction hypothesis is $\Pi, z;\ \Gamma, \mathsf{a}(B(t, \eta_B(z))) \Vdash B(t, Sz)$. Let (Π', ρ, Δ) be given and write t', B' for the images. Derive $X(St')$ by Theorem 7.61 and open it at a fresh b. Introduce $D_{x,y}B'$ in the scope Π', b; the rule selects b and then allocates its eigenvariable z'. Let Θ be the scope Π', b, z' with the context consisting of Δ, the clause of the packet and $\mathsf{a}(B'(b, \eta_{B'}(z')))$. By Theorem 7.62, $\Theta \Vdash b = t'$, and TR gives $\Theta \Vdash B'(t', \eta_{B'}(z'))$. The induction hypothesis at the map $\rho[z \mapsto z']$, with (7.7), gives $\Theta \Vdash B'(t', Sz')$, and TR along $t' = b$ gives $\Theta \Vdash B'(b, Sz')$. Its member at the identity is the premise of the introduction. The conclusion does not contain b, and OPEN discharges it. The two modes agree because η_B depends on the root of the matrix, which neither ρ nor the change of witness alters. All uses of the induction hypothesis are for proper subderivations, and the packet and the alias were constructed before. □

Corollary 7.64 (The literal naming theorem is derived)**.** *For every term t, the formula $D_{x,y}\mathrm{Nm}_t$ has a hereditary derivation in G_ν, without a selection by pattern.*

Proof. Derive $X(St)$, open it at a fresh b and extract $b = t$. Introduce $D_{x,y}\mathrm{Nm}_t$; the selected witness is b. In the premise the alias and the selector computations give $\mathrm{Nm}_t(b, Sz)$, the progress assumption being unused. Discharge the packet. □

By Theorem 7.63 and the inherited comparison of G with N, the ordinary realizations of G_ν and of G_d are bi-interpretable and equiconsistent with iPRA, with whole domains, actual equality and the identity graph in both numerical composites; weakening, renaming, cut and substitution are admissible in G_ν as consequences of the theorem.

Remark 7.65 (Versions). The stable fragment and the route to equation induction through Theorem 7.59(iv) are those of the guarded companion of `newest_parameter`, which states the translation with persistent builders and an arbitrary substitution, as in Theorem 7.37. `uniform_raw` proves instead, by induction on a derivation of $\mathcal{E}$ and for all substitutions σ at once, that $\Gamma\sigma \Vdash E\sigma$. At an induction with the invariant $E(y)$ it regenerates the step at $y = 0$ and at $y = Sw$ from the source derivation, proves $\Phi(y) := G(y) \Rightarrow_g E(Sy)$ at 0 and at Sw, obtains $\Phi(z)$ by cases, and concludes as above; no derivation of the target is substituted into. A technical note of `uniform_raw` also observes that, once the names are available, the proof of Theorem 7.53 applies to G_ν with no case for a naming matrix, which gives weakening and then cut directly. The checkers of `uniform_raw` and of `newest_parameter` for G_ν are obtained from the checker of G_d by deleting the function that selects the witness (Section 7.10).

7.7 A recursor whose predecessor computes

This section draws on `parameter_intro`; the normalization lemma and the extensionality lemma are also in `conditional_copy`. The term language, its normal forms, the decoder and the ordinary theory with its bridge to iPRA are those of Theorems 9.1, 9.7, 9.10 and 9.25; the section states what the witness-free introduction adds to them. The calculi on the same terms with a supplied witness are those of Section 9.9.

On iterator terms no term p satisfies $p(Sz) = z$ by computation, and the names of Section 7.4 need the derived predecessor and an equality rule. With a recursor whose body receives the counter, and with terms taken in normal form, the predecessor computes, and the name of every term is obtained with clause specialization alone. This makes the restricted introduction available in a calculus of three rule families in which equality transport is itself part of the discharge rule and is not available beforehand.

7.7.1 Terms in normal form

The raw terms are those of the counter-aware seeded recursor of Theorem 9.1(2): variables, 0 and $R(n, a; i, u.\, h)$, with i, u bound in h only, the successor $Sa := R(0, a; i, u.\, 0)$ and the computation equations (9.3) and (9.4). The *public terms* are the normal terms of Theorem 9.5, and every substitution performed by a rule is followed by normalization nf (Theorem 9.6). Computation is thus part of the syntax, and the normalizer is part of a checker and of its measured size; it is an external operation on finite terms, with no primitive recursive bound uniform in the term (Theorem 9.9). By Theorem 9.7, normal forms are unique, normalization respects both computation equations in every context, and

$$\mathrm{nf}(t[\sigma]) = \mathrm{nf}\big(\mathrm{nf}(t)[\mathrm{nf} \circ \sigma]\big)$$

for terms, formulas and templates with one distinguished variable (9.7). Normalization introduces no free variable: by induction on t, the free variables of $\mathrm{ev}(t, \rho)$ are among those of the values $\rho(x)$ for x free in t.

Remark 7.66 (The normalization lemma in the packages). `parameter_intro` and `conditional_copy` define the normal forms by rewriting, taking 0, S and R as distinct raw constructors (to avoid the circular rewrite of $R(0, a; i, u.\, 0)$ to itself), orienting the two equations from left to right, reducing in every subterm including bodies, and printing the normal constructor Sa as $R(0, a; i, u.\, 0)$. They prove strong normalization and confluence by the reducibility argument

for the base-type recursor of Gödel's system T recorded in Theorem 9.8; the resulting normal forms are those of Theorem 9.6.

The abbreviations are those of (9.10): the decoder P, the selector C, the predecessor p, and $J(n, a;\, i, u.\, h)$ for $\mathrm{Rec}(n, a;\, i, u.\, h)$. By Theorem 9.10, on normal terms

$$P(0) \equiv 1, \quad P(Sn) \equiv n, \qquad C(0, a, b) \equiv a, \quad C(Sn, a, b) \equiv b, \qquad p(0) \equiv 0, \quad p(Sn) \equiv n,$$
$$J(0, a;\, i, u.\, h) \equiv a, \qquad J(Sn, a;\, i, u.\, h) \equiv h[n/i,\ J(n, a;\, i, u.\, h)/u],$$

where $\equiv$ is literal identity of normal forms. The iterator of Section 1.1 is J with the counter coordinate unused (Theorem 9.12). In this section and the next, C takes the place of sel in the gate and in the naming matrix of Theorem 7.10.

7.7.2 Three rule families

Formulas are equations and $D_{x,y}B$. A clause is $[\ ;\, y.B]$ or $[A;\, y.B]$ with y not free in A; the first reads $\forall y\, B$. Every derivation starts from the one clause

$$\mathsf{Rf} := [\ ;\, y.\, y = y],$$

and an assumption A is the clause $[\ ;\, v.A]$. For a matrix B and a term r write $B_r(y)$ for $\mathrm{nf}(B(r, y))$. The calculus P_ν has three rule families.

(i) USE: for a clause of the context and a term t over Π, derive the antecedent of the clause, if it has one, and conclude the normalized body at t.

(ii) INTRO: with $\ell = \nu(\Pi)$ and a fresh z,

$$\frac{\Pi, z;\ \Gamma, [\ ;\, v.B_\ell(\eta)] \vdash B_\ell(Sz)}{\Pi; \Gamma \vdash D_{x,y}B} \qquad \eta = \begin{cases} z, & B_\ell(0) \text{ is literally an equation } r = r, \\ 0, & \text{otherwise.} \end{cases}$$

The instruction supplies no witness, no mode and no proof of the instance at 0.

(iii) DISCHARGE: with a fresh b, the clause $\mathcal{C}_B(b) := [\mathrm{nf}(B(b, 0));\, y.\, \mathrm{nf}(B(b, y))]$ and a template $F(v)$,

$$\frac{\Pi; \Gamma \vdash D_{x,y}B \qquad \Pi, b;\ \Gamma, \mathcal{C}_B(b) \vdash s = t \qquad \Pi, b;\ \Gamma, \mathcal{C}_B(b) \vdash \mathrm{nf}(F(s))}{\Pi; \Gamma \vdash \mathrm{nf}(F(t))}$$

provided $\mathrm{nf}(F(t))$ does not contain b. The terms s, t and the template may contain b.

There are no other logical, equality, arithmetic or structural rules; the computations (9.3) and (9.4) belong to the syntax. The designated contradiction is $0 = 1$.

Proposition 7.67 (Soundness)**.** *Every rule of* P_ν *is valid under the ordinary reading in intuitionistic logic with equality, the equations* (9.3) *and* (9.4)*, numerical coverage,* $0 = 1 \to \bot$ *and induction for literal equations.*

Proof. USE is the application of a clause. DISCHARGE is the elimination of the existential quantifier followed by the substitution of equals in the branch; the condition on $\mathrm{nf}(F(t))$ is the eigenvariable condition, and the rule is sound also when t contains b. For INTRO: if the instance at 0 is not reflexive, the mode is by cases, and coverage combines the retained instance at 0 with the derived instance at successors. If it is reflexive, the matrix is a literal equation, because normalization does not remove a binder of a formula; the instance at 0 is provable and induction for that equation gives all instances. In both cases ℓ is a witness of the existential quantifier. No induction on a quantified formula is used. □

A premise by cases can be used at any witness. If the rule selects progress instead, the discarded assumption at 0 is literally reflexive: derive it by USE of Rf and ignore the progress assumption. In the constructions below this is an operation on *builders*, that is, on constructions that can be carried out again after fresh parameters and clauses have been adjoined; it is not a weakening principle for derivation trees. A builder is called *persistent*.

7.7.3 Constructions that precede equality

Write $q_y A$ for $D_{w,y}A$ with w not free in A. The witness of its introduction is unused, so its instances do not depend on which parameter is the newest. The elimination of $D_{x,y}B$ without substitution is DISCHARGE with $s = t = 0$, the equation $0 = 0$ by USE of Rf, and the constant template whose value is the conclusion. Hence the introduction by cases and the elimination of q are available from the three rules, with the repair of the previous paragraph when the instance at 0 is reflexive; this gives numerical cases for all formulas, with the branches constructed in their own scopes. Neither equality transport nor equation induction is needed for these constructions.

For a formula A let $G_v A$ be A with each side s of its final equation replaced by $C(v, 0, s)$, the binders being kept. Then $G_{Sz}A \equiv A$, and $G_0 A$ is a tower of binders over $0 = 0$, which is derivable in every scope by INTRO and reflexivity, its witnesses being unused. For persistent premises the padding $L(A) := q_y G_y A$ is interderivable with A: one direction ignores its assumption at 0 and uses A, the other uses the closure at 1 with the provable instance at 0. Padding two formulas to the same number of binders permits their selector, sidewise by $C(v, \cdot, \cdot)$, and the implication $q_y \operatorname{Sel}(y, A, B)$ with its introduction and elimination. Every introduction in these constructions has an unused witness, except those of the towers over $0 = 0$, whose result does not depend on the witness.

Lemma 7.68 (A name without equality)**.** *For every term t over Π there is a persistent derivation in P_ν of*

$$\operatorname{Name}(t) := D_{x,u}\big(C(u, 0, x) = C(u, 0, t)\big),$$

and the clause obtained by opening $\operatorname{Name}(t)$ *at a fresh b gives $b = t$ by one* USE*.*

Proof. The opened clause is $[0 = 0;\ u.\, C(u, 0, b) = C(u, 0, t)]$; USE at 1, with the antecedent from Rf, gives $b = t$. For the derivation let

$$F(y) := D_{x,u}\Big(C(y, 0, C(u, 0, x)) = C(y, 0, C(u, 0, P(y)))\Big).$$

$F(0)$ normalizes to $D_{x,u}(0 = 0)$ and is introduced at whatever parameter is the newest. In the branch for $F(Sz)$ the parameter z is the newest, and $F(Sz)$ normalizes to $D_{x,u}(C(u, 0, x) = C(u, 0, P(Sz)))$, that is, to $\operatorname{Name}(z)$, because $P(Sz) \equiv z$. Its introduction selects the witness z; the instance at 0 is $0 = 0$, so the mode is by progress, and the instance asked for is $z = z$, by USE of Rf. Numerical cases give $q_y F(y)$, and its elimination at the term St gives $\operatorname{nf}(F(St))$, which is literally $\operatorname{Name}(t)$, since $C(St, 0, b) \equiv b$ and $P(St) \equiv t$. The derivation is constructed anew in each scope. □

The proof uses no arbitrary witness, no equality substitution and no predecessor theorem. The shift by one is necessary: the value $P(0) = 1$ is erased by the outer selector.

7.7.4 Completeness

The source of the translation is the diagonal calculus $\mathsf{N}_{\mathrm{diag}}$ of Theorem 3.55 (see also Section 7.5.5), over the terms of Section 1.1 extended by the fixed descriptions R with the equations (9.3) and (9.4); it is stated with transport in formula templates, which is a derived

rule of it. The iterator is translated to J with an unused counter coordinate, equations and binders are translated compositionally, $Q_y B$ is translated to $q_y B$, and all terms are normalized. A diagonal matrix has the normalized instance $0 = 0$ at $y = 0$, whatever its witness and its bodies are.

Theorem 7.69 (Completeness of the three families)**.** *Let $\Gamma \vdash A$ be derivable in $\mathsf{N}_{\mathrm{diag}}$ over the extended vocabulary. In every scope that contains the parameters, and from the clauses $[\ ;\ v.B^*]$ for $B \in \Gamma$, P_ν derives the normalized translation A^*. In particular closed theorems and the contradiction are preserved.*

Proof. By induction on the source derivation the following is shown: for every target scope and context, every assignment of target terms to the source parameters, and persistent builders of the translations of the source assumptions, there is a persistent builder of the translation of the conclusion. At the root the builder of an assumption is its use. Every replacement of a local source assumption made below uses only the use of a clause, reflexivity, and transport through the dummy major premise $D_{x,y}(0 = 0)$, whose introduction does not depend on the selected witness; the builders therefore remain persistent.

Computation axioms translate to reflexivity, their two sides having the same normal form. *Hypotheses* invoke their builders.

Elimination of $D_{,}$. Translate the major premise and apply DISCHARGE with a constant template and $0 = 0$. The source assumption $Q_y B(b, y)$ of the branch is represented by the clause $[B(b, 0);\ y.B(b, y)]$. Turn the clause into the formula $q_y B(b, y)$ by an introduction with an unused witness: in its premise use the clause at Sz, the antecedent being the local assumption, or reflexivity if the rule selected progress. This builder is persistent and realizes the assumption that the induction hypothesis for the branch requires.

Elimination of Q. Translate the major premise to $q_y B(y)$ and open it first. Then apply the induction hypothesis to the derivation of $B(0)$ in the extended context, use the new clause at the required term, and close the DISCHARGE with a constant template and $0 = 0$. The premise $B(0)$ is not translated outside the opening and then moved into the branch.

Transport. Open the dummy major premise $D_{x,y}(0 = 0)$, apply the induction hypothesis to the two source premises in the extended context, and finish the DISCHARGE with the template of the source inference.

Introduction at a witness t. Derive Name(t) by Theorem 7.68 and open it at a fresh b. The branch has the persistent equation $b = t$, and b is the newest parameter, so INTRO selects b. If the source introduction is by progress, its matrix is diagonal, its normalized instance at 0 is $0 = 0$ also at b, and the target rule selects progress: transport the assumption at b to the assumption at t, apply the induction hypothesis to the source premise, and transport the result back to b. If the source introduction is by cases and the target rule selects cases, do the same with the assumption at 0. If the target rule selects progress, the instance at 0 at the witness b is reflexive: derive it by USE of Rf, transport it to the instance at 0 at the witness t, apply the induction hypothesis, and transport back; the progress assumption is unused. For each transport back, the dummy major premise is opened before the induction hypothesis is applied to the source premise, so that the recursive call is made in the extended context; the builders of its assumptions are persistent there. The introduction yields $D_{x,y}B$, which does not contain b, and the opening of the name is closed with a constant template and $0 = 0$.

Every recursive call is for a proper subderivation of the source, and no completed derivation of the target is moved into a new scope. □

The translation covers formulas that contain R essentially and not only the image of the iterator terms: the logical comparison and the diagonal argument treat the two sides of an equation as arbitrary term templates, and the additional computation axioms translate to reflexivity.

7.7.5 The ordinary theory and the bridge

Let T_R be the theory T^2_R of Theorem 9.20: intuitionistic first-order logic with equality over the fixed descriptions R, with the equations (9.3) and (9.4), numerical coverage, the separation axiom $0 = 1 \to \bot$ and contextual induction for literal equations. By body extensionality (Theorem 9.22) T_R proves $t = \mathrm{nf}(t)$ for every term, also with reductions inside bodies, and every formula is provably equivalent to its normalization (Theorem 9.23). No truth or evaluation predicate is used.

Theorem 7.70 (Bridge for the recursor calculus)**.** P_ν *is sound and complete for* T_R*, and* T_R *is bi-interpretable with* iPRA*: both domains are the whole numerical sort, equality is equality, both numerical comparison maps have the graph* $x = y$*, and every formula is provably equivalent to its double translation. In particular* P_ν *derives* $0 = 1$ *if and only if* iPRA *proves* $\bot$*.*

Proof. Soundness is Theorem 7.67 with Theorem 9.23. Completeness is Theorem 7.69 together with the equivalence of $\mathsf{N}_{\mathrm{diag}}$ with N (Theorem 3.55) and the presentation of the ordinary theory by N (Theorem 1.119), which are parametric in the additional fixed descriptions. The formula round trips between P_ν and T_R follow by induction on formulas from the reading of $D_,$ and the logical macros (Section 7.8.5).

The bi-interpretation of T_R with iPRA, with the stated domains, equality, composites and double translations, is Theorem 9.25; quantifier-free induction in T_R is Theorem 9.27 (Theorem 7.81). The packages state the interpretations directly, as recorded in Theorem 9.26: a description is interpreted by the primitive recursion with the base Sa and the step h, and primitive recursion is compiled by J without pairing (Theorem 9.12); P applied to $Sx = Sy$ gives the injectivity of the successor, and $C(\cdot, 0, 1)$ applied to $Sx = 0$ gives $1 = 0$. Falsity corresponds to $0 = 1$, which T_R refutes by separation, and derivations of a contradiction translate in both directions. □

7.7.6 Checking a template through its two instances

In DISCHARGE the checker receives a template $F(v)$, terms s, t and derivations of $s = t$ and $\mathrm{nf}(F(s))$, and its result $\mathrm{nf}(F(t))$ must equal the conclusion, which is checked. A separate check that F is well-scoped and normal is redundant.

Lemma 7.71 (Completion of an accepted template)**.** *Let* F *be a finite expression such that the substitution of the checked terms* s *and* t *for its distinguished variable, followed by normalization, succeeds, and such that the two results are well-scoped formulas in the local scope. Then there is a well-scoped template* $F'(v)$ *in normal form with*

$$\mathrm{nf}(F'(s)) = \mathrm{nf}(F(s)), \qquad \mathrm{nf}(F'(t)) = \mathrm{nf}(F(t))$$

literally.

Proof. Sorts. The traversal that substitutes and normalizes checks every proper child at its required sort, term or formula, before any reduction that erases it. At the root it may accept an unclassified expression. A root that is neither an equation, a binder nor a recursor does not reduce and yields no formula; a numerical root stays numerical under reduction. Since an instance is a formula, the root of F is an equation or a binder, and F is a sorted formula template that may contain additional numerical parameters and reducible subterms.

Additional parameters. Read every parameter of F outside the local scope, and every bound-variable index that exceeds the binders above it by more than the one position of the distinguished variable, as an additional free parameter; the reading is consistent under a binder because the excess is measured relative to the depth. Let ρ replace all additional

parameters by 0 and fix v, the local parameters and the bound variables. Neither s nor t contains an additional parameter. For $r \in \{s, t\}$, by Theorem 9.7,

$$\mathrm{nf}\big((\rho F)[r/v]\big) = \mathrm{nf}\big(\rho(F[r/v])\big) = \mathrm{nf}\big(\rho\ \mathrm{nf}(F[r/v])\big) = \mathrm{nf}(F[r/v]),$$

the last step because the checked instance is normal and contains no additional parameter. A reduction that becomes possible after an additional parameter is closed does not affect the argument, which uses the law of Theorem 9.7 and not an identity of reduction sequences.

The template. Put $F' := \mathrm{nf}(\rho F)$. It is sorted, well-scoped and normal, and a second application of Theorem 9.7 gives the two identities. The distinguished variable may occur any number of times. □

Replacing F by F' leaves the two premises and the conclusion unchanged, so every accepted use of the relaxed interface is an instance of DISCHARGE, and every canonical template is still accepted. The equation $s = t$ must be checked before the template is instantiated, so that s and t are known to be terms, and the instance in the branch and the conclusion keep their checks; the lemma does not justify omitting either.

7.8 Conditional copy

This section draws on `conditional_copy`; the example of an unsound probe is from `deterministic_introduct`

The calculus P_ν takes no witness at its introduction and still takes an equation, with its derivation, and a template at its discharge. Other calculi infer the substitution of a discharge from two instances of the opened matrix and take an arbitrary witness at their introduction (Theorems 6.23 and 6.36). The two restrictions are combined here. The composition is not immediate: with an inferred discharge, equality transport is recovered from packets that are introduced at arbitrary witnesses, and the names of Theorem 7.68 give an equation $b = t$, whose use would need the transport that is being recovered. The combination rests on a formula that copies an instance of a formula to a fresh parameter without any equation.

7.8.1 Use, Close and Export

Terms, normal forms, the abbreviations P, C, J, formulas, clauses and the initial clause Rf are those of Section 7.7. Write $\top_* := (0 = 0)$ and $\bot_* := (0 = 1)$. The calculus K has three rule families; all displayed instances are normalized.

(i) USE: as in P_ν.

(ii) CLOSE: with $\ell = \nu(\Pi)$ and a fresh z,

$$\frac{\Pi, z;\ \Gamma, [\ ;\ v.\,\mathrm{nf}(B(\ell, \eta))] \vdash \mathrm{nf}(B(\ell, Sz))}{\Pi; \Gamma \vdash D_{x,y}B} \qquad \eta = \begin{cases} z, & \mathrm{nf}(B(\ell, 0)) \text{ is literally } \top_*, \\ 0, & \text{otherwise.} \end{cases}$$

The test for $\top_*$ is narrower than the test for a reflexive equation in P_ν and suffices.

(iii) EXPORT: with a fresh b, put

$$B_0 = \mathrm{nf}(B(b, 0)), \qquad B_1 = \mathrm{nf}(B(b, 1)), \qquad r_B = \begin{cases} t, & B_0 \equiv \top_* \text{ and } B_1 \equiv (b = t), \\ b, & \text{otherwise,} \end{cases}$$

where $\equiv$ is literal identity, including the orientation of the equation, and let $\mathcal{C}_B(b) = [B_0;\ y.\,\mathrm{nf}(B(b, y))]$. Then

$$\frac{\Pi; \Gamma \vdash D_{x,y}B \qquad \Pi, b;\ \Gamma, \mathcal{C}_B(b) \vdash F}{\Pi; \Gamma \vdash \mathrm{nf}(F[r_B/b])} \qquad b \notin \mathrm{FV}\big(\mathrm{nf}(F[r_B/b])\big).$$

F may contain b, and so may r_B; the substitution is one simultaneous pass. No equation, replacement term or template is supplied.

Only USE takes a term. The term r_B is the selected witness $\rho_B(b)$ of Theorem 6.23, and EXPORT is the rule (6.12) of the inferred discharge, here on the normal terms of the counter-aware recursor (Theorem 6.36 treats it with a supplied witness at the introduction, on iterator terms).

Lemma 7.72 (Soundness)**.** *Every rule of* K *is valid in* T_R *under the ordinary reading.*

Proof. USE and CLOSE are as in Theorem 7.67; if the instance at 0 is $\top_*$, the matrix is a literal equation, since normalization does not remove a binder. For EXPORT the argument is that of Theorem 6.27: open the existential quantifier; in the selected case reflexivity proves B_0, the clause at 1 gives $b = r_B$, and the substitution of equals turns F into $F[r_B/b]$; in the default case the substitution is the identity. The elimination of the quantifier is legitimate because the normalized result does not contain b. Normalization is justified in T_R by Theorem 9.23. □

Example 7.73 (The instance at 0 must be tested)**.** An EXPORT that inspected only the instance at 1 would be unsound. Let f be a term with $f(0) = f(1) = 0$ and $f(2) = 1$, and let $B(x, y)$ be $x = f(y)$. The reading of $D_{x,y}B$, $\exists x\,(x = f(0) \to \forall y\, x = f(y))$, is provable with the witness 1, whose antecedent is false. The instance at 1 at a fresh b is $b = 0$. A rule that replaced b by 0 on this evidence would produce the clause $0 = 0 \to \forall y\, 0 = f(y)$, which gives $0 = 1$ at $y = 2$. The test $B_0 \equiv \top_*$ excludes this matrix; Theorem 6.38 records a certificate for $1 = 0$ accepted without it. For the same reason the name $D_{x,y}\mathrm{Nm}_t$ of Section 7.6 is not silently combined with an inferred opening: opening a name directly at t removes the fresh alias $b = t$ on which the simulation of an arbitrary witness relies.

7.8.2 Preparations without equality

If F does not contain the opened parameter b, the substitution of EXPORT does nothing to it; so the ordinary elimination of $D_{x,y}B$ is available whatever r_B is. As in Section 7.7, $q_yA(y)$ is $D_{d,y}A(y)$ with d unused; opening it and using its clause gives $A(t)$ from q_yA and $A(0)$, and the witness selected by CLOSE for it is irrelevant. Given persistent constructions of $A(0)$ and of $A(Sz)$, CLOSE introduces $q_yA(y)$: if the instance at 0 is $\top_*$, supply reflexivity in place of the assumption at 0 and ignore the progress assumption; otherwise the rule is by cases. The same holds for a premise that assumes $A(0)$.

Every formula is a chain of binders ending in an equation. For a formula A with d binders let G_vA be as in Section 7.7; then

$$G_0A \equiv T_d, \qquad G_{Sz}A \equiv A, \qquad T_0 = \top_*, \quad T_{d+1} = D_{x,y}T_d.$$

Every tower T_d is derivable in every scope, by reflexivity at depth 0 and CLOSE above it; none of its witnesses occurs in its body. For formulas A and B with the same number of binders, align the binders and replace corresponding sides s, t of the final equations by $C(v, s, t)$. This defines $\mathrm{Sel}(v, A, B)$, with

$$\mathrm{Sel}(0, A, B) \equiv A, \qquad \mathrm{Sel}(Sz, A, B) \equiv B.$$

A substitution in terms does not change the number of binders, so $H(s)$ and $H(x)$ always have the same depth. For formulas of different depth the padding $L(A) = q_vG_vA$, interderivable with A, equalizes the depths. All of these are constructions carried out in the scope where they are needed.

7.8.3 The conditional-copy theorem

Theorem 7.74 (Conditional copy)**.** *For every formula $H(x)$ with a distinguished variable and every term t over Π, the formula*

$$\Delta_t(H) := D_{x,y}\,\mathrm{Sel}(y, H(t), H(x))$$

is derivable in K *in every scope and context, by a construction that uses no equality transport and no equation other than reflexivity.*

Proof. Construct the closure

$$q_v G_v \Delta_{P(v)}(H). \tag{7.8}$$

Its instance at $v = 0$ is a tower. In the branch for $v = Sz$, with z fresh, the conclusion normalizes to $\Delta_z(H)$, because $P(Sz) \equiv z$ and the gate disappears. The parameter z is the newest, so CLOSE for $\Delta_z(H)$ selects $x = z$. Its instance at 0 and its instance at a successor are both literally $H(z)$:

$$\mathrm{Sel}(0, H(z), H(z)) \equiv H(z), \qquad \mathrm{Sel}(Sw, H(z), H(z)) \equiv H(z).$$

If $H(z) \not\equiv \top_*$, the rule is by cases and its assumption is its conclusion. If $H(z) \equiv \top_*$, the rule is by progress and reflexivity derives the instance asked for. The outer introduction is by cases, since its instance at 0 has at least one binder; its assumption is unused. Open (7.8) by the ordinary elimination, supply the tower as the instance at 0, and use the clause at St. The gate disappears and $P(St) \equiv t$, so the result is literally $\Delta_t(H)$; it does not contain the opened parameter. Every step can be carried out in every scope. □

The ordinary reading of $\Delta_t(H)$ is $\exists x\,(H(t) \to H(x))$. The theorem neither proves $H(t)$ nor identifies x with t. Opening $\Delta_t(H)$ gives a fresh parameter b and the clause $[H(t);\, v.\,\mathrm{Sel}(v, H(t), H(b))]$, which yields $H(b)$ at $v = 1$ once $H(t)$ is supplied.

7.8.4 Removal of the witness

Let L, the *liberal* calculus, have the syntax, USE and EXPORT of K and a CLOSE that takes an arbitrary term t over Π in place of ℓ, with the same test: progress exactly when $\mathrm{nf}(B(t, 0)) \equiv \top_*$.

Theorem 7.75 (Witness removal)**.** *For every scope Π, clause context Γ and formula A, the calculi K and L derive the same judgments $\Pi; \Gamma \vdash A$.*

Proof. A CLOSE of K is a CLOSE of L. For the converse a derivation of L is translated by induction on its tree. The induction statement admits an injective map θ from the source parameters into an arbitrary target scope and a map from the positions of the source clauses to positions of target clauses, such that the clause at each image position is literally the normalized θ-image of the source clause; further target parameters and clauses are allowed. This is an invariant of membership; it is not an assumption that completed target derivations can be weakened.

USE follows the map of positions and translates the derivation of the antecedent. EXPORT translates its major premise, sends its eigenvariable to the new target parameter and extends the map of positions by the opened clause; its two tests are preserved and reflected by injective renaming and Theorem 9.7.

Consider a CLOSE at a term t with the matrix B and the premise $\Gamma, [\ ;\, v.\,\mathrm{nf}(B(t, \eta))] \vdash \mathrm{nf}(B(t, Sz))$. Write t, B also for their images, and put $H(x) := q_y B(x, y)$.

(a) Derive $\Delta_t(H)$ by Theorem 7.74 and open it at a fresh b; its clause is $[H(t);\, v.\,\mathrm{Sel}(v, H(t), H(b))]$.

(b) Apply CLOSE to $D_{x,y}B$. The newest parameter is b, so the rule selects b; let z' be its eigenvariable.

(c) In the premise derive $H(t)$ by a CLOSE with an unused witness. Its test is the source test at t, and its premise, with a further eigenvariable, is the image of the source premise: the induction hypothesis applies with the source eigenvariable sent to the new one and the source assumption sent to the local clause of this CLOSE.

(d) USE of the clause of (a) at 1 gives $H(b)$.

(e) Open $H(b)$; its clause is $[B(b,0);\ y.B(b,y)]$. If the CLOSE of (b) is by cases, its assumption is $B(b,0)$. If it is by progress, $B(b,0)$ is literally $\top_*$ and reflexivity supplies it. USE at Sz' gives $B(b,Sz')$, and the unused witness of $H(b)$ is discharged.

The CLOSE of (b) yields $D_{x,y}B$, which does not contain b, and the ordinary elimination discharges b. Every recursive call is for the premise of the source CLOSE, a proper subderivation, although it may be made several times; the packet and the towers are explicit constructions. At each call the tests agree, because they are made on the same instantiated formula. □

Remark 7.76 (The modes need not agree). The test at t and the test at b may differ. The proof transfers the whole conditional closure H and not a progressive premise; at b a progressive mode has a reflexive instance at 0, so the closure still yields the instance at a successor. No equality reasoning repairs a failed test. The same proof applies with the rule NAME of Section 7.8.7 in place of EXPORT.

7.8.5 The liberal calculus presents the ordinary theory

Lemma 7.77 (Structural facts of the liberal calculus). *In* L*, weakening by clauses, injective renaming of parameters and the cut of an assumption* $[\ ;\ v.A]$ *are admissible.*

Proof. By induction on derivations. A supplied witness remains available after a renaming, an injective renaming preserves and reflects the literal tests of CLOSE and EXPORT, and normalization commutes with renaming. For the cut, a USE of the clause is replaced by the given derivation, weakened and renamed into the scope of the leaf. No equality transport and no substitution of terms into derivations is used. The lemma is about L and is not claimed for K at this stage. □

Lemma 7.78 (Transport from an inferred discharge). *In* L*, from* $s = t$ *and* $h(s) = u$ *one derives* $h(t) = u$*. Consequently symmetry, transitivity, congruence and transport in arbitrary formulas are derived.*

Proof. Let

$$E(x,y) := \Big(C(y,0,C(P(y),x,h(x))) = C(y,0,C(P(y),t,u))\Big).$$

Its normalized instances are $E(x,0) \equiv \top_*$, $E(x,1) \equiv (x = t)$ and $E(x,SSv) \equiv (h(x) = u)$. Introduce $D_{x,y}E$ at the witness s; the mode is by progress, and the instance asked for, $C(z,s,h(s)) = C(z,t,u)$, is derived by numerical cases on z from the two given equations, the progress assumption being unused. Open the formula at b: the first two instances make EXPORT select t. USE at 2, with the reflexive antecedent, gives $h(b) = u$, and EXPORT concludes $h(t) = u$. This is the argument of Theorem 6.29, with the decoder packet (9.28) of Theorem 9.107 in place of the packet on iterator terms.

With reflexivity this rule gives symmetry, transitivity and congruence, and transport in an equation template is two such replacements. Transport in a formula $D_{x,y}B(v,x,y)$ follows by induction on the number of binders: open the given formula at its witness, introduce the

target at that witness, transport the assumption at 0 backwards through the matrix, use the opened clause at the successor and transport the result forwards; if the new instance at 0 is $\top_*$, supply reflexivity instead. The result does not contain the opened witness. The premises are moved into the local scopes by Theorem 7.77. □

Lemma 7.79 (Cases and equation induction in the liberal calculus)**.** *In* L *the introduction by cases is available for every matrix and witness, and for a literal equation $E(y) \equiv (s(y) = t(y))$, from $E(0)$ and a derivation of $E(Sz)$ from $E(z)$ one derives $E(a)$ for every term a.*

Proof. If an introduction by cases selects progress because its instance at 0 is $\top_*$, replace the assumption at 0 by reflexivity and ignore the progress assumption. For the induction let

$$K(y) := \big(C(y, 0, s(y)) = C(y, 0, t(y))\big),$$

whose instance at 0 is literally $\top_*$. Numerical cases, without induction, give the two decoder equations

$$C(y, s(0), C(y, 0, s(y))) = s(y), \qquad C(y, s(0), C(y, 0, t(y))) = t(y);$$

the first has reflexive branches, and the second uses $E(0)$ at 0 and is reflexive at successors. From $K(z)$, congruence with $v \mapsto C(z, s(0), v)$ and the decoders give $E(z)$. The cut of this into the step gives $E(Sz)$, which is literally $K(Sz)$. The introduction by progress gives $q_y K(y)$, USE with the reflexive antecedent gives $K(a)$, and the decoders give $E(a)$. No compound term is substituted into a derivation. A conditional closure $q_y E(y)$ is introduced by cases, with $E(0)$ as its assumption and the induction just derived for its successor instance. □

The ordinary logical vocabulary is given by the macros (9.24) of Theorem 9.91, with $\mathrm{Sel}(v, A, B)$ in the role of $M_v(A, B)$, the depths equalized by L before each binary selector and all bound variables fresh; in particular $(A \to B)^* = q_y \,\mathrm{Sel}(y, A^*, B^*)$, $(A \wedge B)^* = q_y G_y \,\mathrm{Sel}(P(y), A^*, B^*)$, $(\forall x\, A)^* = q_y G_y A^*(P(y))$ and $\bot^* = \bot_*$. The proofs of Theorems 9.92 and 9.93 use USE, the ordinary elimination, numerical cases, the introduction by cases at an arbitrary witness, the introduction by progress, transport in formulas and binder congruence (proved as Theorem 9.84(5)). All of these are available in L by Theorems 7.77 to 7.79 and, for the ordinary elimination, Section 7.8.2, so the proofs apply unchanged: the macros admit the introduction and elimination rules of intuitionistic logic under arbitrary assumptions, with explosion for $\bot_*$, and by induction on a formula the ordinary reading of each macro is equivalent to the intended connective (Theorem 9.95(3)). `parameter_intro` uses the same macros with two differences: the conjunction selects by a parity term, $(A \wedge B)^* = q_y G_y \,\mathrm{Sel}(\mathrm{par}(y), A^*, B^*)$, and the universal quantifier is $(\forall x\, A)^* = A^*(0) \wedge q_x A^*(x)$.

Proposition 7.80 (The liberal calculus)**.** L *is sound and complete for T_R. Hence so is* K*.*

Proof. Soundness is Theorem 7.72 with a supplied witness. For completeness translate the diagonal calculus $\mathsf{N}_{\mathrm{diag}}$ over the vocabulary with the fixed descriptions R, as in Theorem 7.69: computation axioms become reflexivity after normalization; hypotheses are used; equality is Theorem 7.78; the eliminations of Q and D, are the derived ones; a diagonal matrix has the instance $0 = 0$ at 0, so its introduction by progress is the progress mode of CLOSE; an introduction by cases is Theorem 7.79. The structural facts of Theorem 7.77 move derivations between scopes. This gives the full consequence relation and not only closed equations. The statement for K follows by Theorem 7.75, which regenerates the branches in the target scopes. □

7.8.6 Quantifier-free induction and the bridge

Remark 7.81 (Reflection of equality in T_R). T_R proves the successor injective and equality decidable, and it proves every instance of contextual quantifier-free induction (Theorem 9.27); the reflection term of Theorem 1.86 transfers along Theorem 9.25. `parameter_intro` and `conditional_copy` prove this directly in T_R, with the terms of Section 9.2, by the argument recorded in Theorem 9.28: addition and truncated subtraction d compiled by J, four inductions on literal equations, one induction on the residual equation $C(\mathrm{sg}(d(x,y)), x, y + d(x,y)) = x$ whose case splits are all coverage, the reflection term $e(a,b) = \mathrm{sg}(K(d(Sa,b)))$ with $K(u) = C(u, 1, p(u))$, and characteristic terms b_A with 0 for truth.

Theorem 7.82 (Introduction and discharge without supplied terms)**.** K *is a sound and complete presentation of* T_R*, and* T_R *is bi-interpretable with* iPRA*, with the domains* $x = x$*, equality as equality, the graph* $x = y$ *in both numerical composites and a provable equivalence of every formula with its double translation. Hence* K *and* iPRA *are equiconsistent.*

Proof. The first statement is Theorem 7.80. The bi-interpretation of T_R with iPRA, with the stated domains, equality, composites and double translations, is Theorem 9.25, as in the proof of Theorem 7.70; quantifier-free induction in T_R is Theorem 9.27; the formula round trips between K and T_R follow from the macros of Section 7.8.5 by induction on formulas. Falsity is translated to $0 = 1$, whose reading implies $\bot$ by the separation axiom of T_R. □

Here iPRA has quantifier-free induction, and T_R has induction for literal equations only; the words "first-order" refer to the logical language.

7.8.7 Discharge by a supplied name

Let $\mathsf{K_N}$ have the syntax, the clause Rf, USE and CLOSE of K, and in place of EXPORT the rule

$$\frac{\Pi;\Gamma \vdash D_{x,y}B \qquad \Pi, b;\ \Gamma, \mathcal{C}_B(b) \vdash b = t \qquad \Pi, b;\ \Gamma, \mathcal{C}_B(b) \vdash F}{\Pi;\Gamma \vdash \mathrm{nf}(F[t/b])}\ \textsc{name} \qquad b \notin \mathrm{FV}\big(\mathrm{nf}(F[t/b])\big).$$

The term t is over Π, b and may contain b; it is inserted once. The left side of the supplied equation is fixed to the opened parameter, whereas DISCHARGE of P_ν takes both sides of an equation and a template.

Proposition 7.83 (The name variant)**.** $\mathsf{K_N}$*,* K *and* T_R *have the same open consequences under their readings.*

Proof. NAME is sound in T_R: equality gives the substituted formula and the condition on the scope permits the elimination of the quantifier. Every instance of EXPORT is an instance of NAME in the same scope: in the default case with the reflexivity $b = b$, in the selected case with the equation $b = t$ obtained from the opened clause by reflexivity and USE at 1. The replacement adds no parameter, so no CLOSE changes its witness or its mode, and induction on the derivation gives $\mathsf{K} \subseteq \mathsf{K_N}$. The completeness of K and the soundness of $\mathsf{K_N}$ give the rest. □

A backward checker for $\mathsf{K_N}$ receives a template $T(v)$ and a term t, checks the conclusions $b = t$ and $F = \mathrm{nf}(T[b/v])$ in the branch, and requires the goal to be $A = \mathrm{nf}(T[t/v])$. It does not forbid occurrences of b in T.

Lemma 7.84 (Template representation)**.** *Let* $T(v)$ *be a well-scoped template and* t *a term of the local scope, and put* $F = \mathrm{nf}(T[b/v])$ *and* $A = \mathrm{nf}(T[t/v])$*. If* A *does not contain* b*, then* $\mathrm{nf}(F[t/b]) = A$ *literally.*

Proof. A renaming of parameters, injective or not, preserves normal forms: the reductions test only for the constructors 0 and S at a counter, and an identification of parameters creates neither; no reduction turns an equation with equal sides into $0 = 0$. Choose a parameter c not in T or t and let $U = \mathrm{nf}(T[v := t,\, b := c])$, with simultaneous substitution. Identifying c with b gives

$$U[c \mapsto b] = \mathrm{nf}(U[c \mapsto b]) = \mathrm{nf}(T[t/v]) = A.$$

Since A does not contain b, U contains neither b nor c, and $U = A$. Substituting t for c and (9.7) give $\mathrm{nf}(T[v := t,\, b := t]) = A$. Finally

$$\mathrm{nf}(F[t/b]) = \mathrm{nf}((T[b/v])[t/b]) = \mathrm{nf}(T[v := t,\, b := t]) = A.$$

No substitution is made inside an inserted copy of t. □

The checker validates a template through its two normalized instances, as in Theorem 7.71, and the same completion applies: the excess parameters of an accepted raw template are replaced by 0, which preserves both checked instances literally because neither they nor the checked arguments b, t contain an excess parameter; the normalized completed template is well-scoped, and Theorem 7.84 applies to it. The equation premise validates t, and each instance is validated in its own scope; these checks are needed. Conversely, abstracting all free occurrences of b in F into v represents every instance of NAME. The backward representation is therefore exact.

7.9 Specialization at the newest parameter

This section draws on `export_specialization` for Section 7.9.1 and on `vacuous_specialization` for Sections 7.9.2 and 7.9.3.

In every calculus considered so far the use of a clause takes an arbitrary term. Here it is restricted to the newest parameter, with 0 in an empty scope, so that it takes no term. The term is delivered instead by an inferred discharge, which substitutes a term read off the opened matrix. In Section 7.9.1 the introduction keeps an arbitrary witness, and the names introduced at that witness carry the term. In Sections 7.9.2 and 7.9.3 the introduction is that of K, at the newest parameter, so that no name can be introduced at an arbitrary term; a closure whose matrix does not depend on its second variable carries the term, and the selector of EXPORT receives one additional case. Neither result needs a primitive alias rule (Section 6.7.4) or the supplied witness of the deterministic specialization (Theorem 6.68).

7.9.1 Newest specialization with the inferred discharge

The calculus $\mathcal{X}$ of Theorem 6.26 has the rules REFL, USE, INTRO and EXPORT of Theorem 6.3 and (6.12), on full normal forms of iterator terms with normalizing substitution, in the convention with REFL. Its scopes are written explicitly as in Theorem 7.1: INTRO appends its eigenvariable z and EXPORT its parameter b.

Definition 7.85 (Newest specialization)**.** $\mathcal{X}_\nu$ is $\mathcal{X}$ with USE replaced by

$$\frac{\Pi;\Gamma \vdash A}{\Pi;\Gamma \vdash \mathrm{nf}(B(\nu(\Pi)))}\ \mathrm{USE}_\nu, \qquad [A;\, y.\, B(y)] \text{ an occurrence in } \Gamma.$$

REFL is kept at every term over Π, and INTRO at every witness over Π, with the mode of (6.5).

A certificate of USE_ν consists of the occurrence and the derivation of its antecedent. For a term t over Π let

$$\text{Nm}_t := D_{x,y}(\text{sel}(y, 0, x) = \text{sel}(y, 0, t)),$$

the name of Section 6.6.

Lemma 7.86 (A name without USE)**.** *In every scope* Π *over which* t *is a term, and in every context,* $\mathcal{X}_\nu$ *derives* Nm_t *by one* INTRO *and one* REFL*.* EXPORT *of* Nm_t *at a fresh* b *selects* t*.*

Proof. Introduce Nm_t at the witness t. The instance asked for is $\text{sel}(Sz, 0, t) = \text{sel}(Sz, 0, t)$, whose normal form is $t = t$; it is proved by REFL, and the assumption of the premise is unused. This holds in both modes, in particular for $t \equiv 0$, when the rule is by progress. The normalized rows at a fresh b are $\text{sel}(0, 0, b) = \text{sel}(0, 0, t)$, that is $0 = 0$, and $\text{sel}(1, 0, b) = \text{sel}(1, 0, t)$, that is $b = t$; so the selector of Theorem 6.23 returns t. □

Theorem 7.87 (Newest specialization)**.** *For every scope* Π*, clause context* Γ *and formula* A*,*

$$\Pi; \Gamma \vdash_{\mathcal{X}_\nu} A \quad \Longleftrightarrow \quad \Pi; \Gamma \vdash_{\mathcal{X}} A,$$

by effective transformations that leave the end sequent unchanged. In the convention with REFL*, a derivation of* $\mathcal{X}$ *with* n *inference nodes,* u *of them uses of a clause, is translated into a derivation of* $\mathcal{X}_\nu$ *with exactly* $n + 3u$ *nodes.*

Proof. Every instance of USE_ν is an instance of USE. For the converse the following is shown by induction on a derivation π of $\Pi; \Gamma \vdash_{\mathcal{X}} A$: for every injective map θ of Π into a scope Π', and every context Δ over Π' with an injective map of the occurrences of Γ to occurrences of Δ that sends each clause to its normalized θ-image, $\mathcal{X}_\nu$ derives $\Pi'; \Delta \vdash \theta A$. Further parameters and clauses of Δ are arbitrary, and θ need not preserve which parameter is the newest. This is a construction carried out again in each target situation; it is not a claim that a derivation of $\mathcal{X}_\nu$ can be weakened, which is false node by node, because an extension of the scope changes the newest parameter.

REFL is translated literally. At INTRO extend θ by the eigenvariable sent to a fresh target parameter, extend the map of occurrences by the assumption, and translate the premise; an injective renaming preserves the root of the matrix and the literal 0, so the mode is unchanged. At EXPORT translate the major premise, open at a fresh target b', extend θ by $b \mapsto b'$ and the map of occurrences by the opened clause, and translate the branch; by Theorem 6.24(a) the selected term is the image of the source selection, and by (6.1) the exported formula is the image of the source conclusion, which does not contain b'.

At a USE of the occurrence $[A_0; y.\, B(y)]$ at a term t, put $t' \equiv \theta t$. Derive $\text{Nm}_{t'}$ by Theorem 7.86 and apply EXPORT at a fresh b, which is the newest parameter of the branch. In the branch apply the induction hypothesis to the source derivation of A_0, with the same θ and the same map of occurrences, the opened clause and b being extras; this gives θA_0. USE_ν of the image of the occurrence gives $\text{nf}((\theta B)(b))$, and EXPORT concludes

$$\text{nf}\big((\theta B)(b)[t'/b]\big) \equiv \theta\ \text{nf}\big(B(t)\big)$$

by (6.1). Neither the context nor t' contains b, so the side condition holds. For a clause without antecedent the inductive call is omitted. The source derivation of A_0 is a proper subderivation of π, and it is translated in the situation where it is used. The identity map gives the equivalence.

For the count, every node other than a USE is copied once with its translated premises, and a USE becomes four nodes: the EXPORT, the INTRO of the name with its REFL leaf, and the USE_ν, above the translated antecedent, which is translated once. Induction on the derivation gives $n + 3u$. □

Remark 7.88 (The Lean module). `NativeNewestUse` formalizes $\mathcal{X}_\nu$ (`XNPrf`) over the calculus `XCPrf` of $\mathcal{X}$: USE at `Tm.newest`, the variable 0 of the de Bruijn scope or the literal 0 in an empty scope, and REFL at every term. `xnprf_iff_xcprf` is Theorem 7.87 with the reflexivity clause in the context, and `xnprf_iff_nprf_of_qfree` the bridge on Q-free sequents. The translation is the one of the proof above: it carries a renaming and an extension of the context (`XCPrf.toXN`); the name formula is introduced at the normal form of the witness (`XNPrf.nm`, `NFm.nm_step`), its rows at the fresh variable select it (`NFm.nm_otest`), and the exported formula is the image of the source conclusion up to full normal forms. The count $n + 3u$ is not formalized.

The count is in inference nodes. A name copies its term, and normalization can enlarge formulas, so no bound on the size of certificates in bytes or on the time of checking follows.

Corollary 7.89 (The bridge for $\mathcal{X}_\nu$)**.** *$\mathcal{X}_\nu$ derives the same sequents as the reference calculus $\mathcal{T}$ of Theorem 6.4; weakening, injective renaming, substitution of terms for parameters and cut are admissible in it on derivable judgments; and its ordinary reading is bi-interpretable with* iPRA*, with whole domains, equality as equality and the graph $x = y$ in both numerical composites, as for $\mathcal{X}^{\mathcal{R}}$ in Theorem 6.36.*

Proof. Theorem 7.87, Theorem 6.36 and, for the two conventions, Theorem 6.13. The admissible rules hold for the derivable judgments of $\mathcal{X}$ and transfer through the equivalence; they are not preserved node by node. □

Remark 7.90 (What the proof uses). The name is introduced at the arbitrary witness t, and its instance is the reflexivity $t = t$ at a compound term. The proof therefore restricts neither the witness of INTRO nor REFL. In the convention with the reflexivity clause $\mathcal{R} = [\,;\, y.\, y = y]$ the restriction of all uses of $\mathcal{R}$ to $\nu(\Pi) = \nu(\Pi)$ would leave only $\nu(\Pi) = \nu(\Pi)$, and a derivation of $t = t$ from a name would presuppose the naming lemma; the theorem is therefore stated in the convention with REFL, and its form $\mathcal{X}_\nu^{\mathcal{R}}$ with the reflexivity clause exempts $\mathcal{R}$ from the restriction. The opened clause of the name is never used: EXPORT reads t from its row at 1. This is the difference from Theorem 6.71, where $b = t$ is derived from the naming clause by USE_* at 1, and from Theorem 6.91, where a primitive ALIAS supplies the equation. Section 7.9.2 restricts the witness of the introduction as well.

The same argument applies without a normalizer. Let C be the five-rule clause calculus of Theorem 5.4, with REFL, EVAL, USE, ELIM and INTRO on raw terms with ordinary substitution, and with scopes written explicitly. The *raw selector* returns t if the matrix $B(x, y)$ is literally $\mathrm{sel}(y, 0, x) = \mathrm{sel}(y, 0, t)$ with t free of x, y and the variables bound by the two selectors, and returns b otherwise. EXPORT_r is (6.12) with this selector and ordinary substitution, the exported formula being required to be over the old scope.

Proposition 7.91 (The raw companion)**.** *The calculus with the five rule families REFL, EVAL, USE_ν, INTRO and EXPORT_r derives the same judgments $\Pi; \Gamma \vdash A$ as C.*

Proof. Every instance of EXPORT_r is derivable in C. If the selector returns b, the exported formula is the branch formula, which does not contain b, and the rule is ELIM. Otherwise open the major premise by ELIM at b; the opened clause is $[\mathrm{sel}(0, 0, b) = \mathrm{sel}(0, 0, t);\, y.\, \mathrm{sel}(y, 0, b) = \mathrm{sel}(y, 0, t)]$. Both sides of its antecedent are equal to 0 by the computation equations of Theorem 5.6, so the antecedent is derived by symmetry and transitivity; USE at 1 gives $\mathrm{sel}(1, 0, b) = \mathrm{sel}(1, 0, t)$, and the computation equations give $b = t$. Transport in the formula $F[v/b]$ (Theorem 5.8) turns the branch conclusion F into $F[t/b]$, and ELIM discharges b, since $F[t/b]$ does not contain it. USE_ν is an instance of USE.

Conversely translate a derivation of C with the induction statement of the proof of Theorem 7.87. REFL, EVAL and INTRO are copied; an injective renaming preserves the literal matches of EVAL, the root of the matrix and the literal 0. ELIM becomes EXPORT_r:

its conclusion does not contain b, so the substitution of the selected term changes nothing. A USE at t is simulated as before: $\mathrm{Nm}_{t'}$ is introduced at t', where the raw instance $\mathrm{sel}(Sz,0,t') = \mathrm{sel}(Sz,0,t')$ has literally equal sides and is proved by REFL in either mode; its matrix is the pattern of the raw selector, which therefore returns t'; the antecedent is translated in the branch, USE_ν gives $(\theta B)(b)$, and $\mathrm{EXPORT}_\mathrm{r}$ gives $(\theta B)(b)[t'/b] \equiv \theta(B(t))$, since B does not contain b. The pattern and its condition on free variables are preserved and reflected by injective renaming. □

No checker of the raw companion is measured.

7.9.2 The vacuous selector

Terms, normal forms, the abbreviations P, C, J, formulas, clauses, $\top_*$, $\bot_*$ and the initial clause Rf are those of Sections 7.7 and 7.8. For a matrix B and a fresh b write

$$\widehat{B}(y) := \mathrm{nf}\big(B(b,y)\big), \qquad B_0 := \mathrm{nf}\big(\widehat{B}(0)\big), \qquad B_1 := \mathrm{nf}\big(\widehat{B}(1)\big).$$

Since normalization introduces no free variable (Section 7.7), $\widehat{B} \equiv B_0$ holds exactly when y does not occur in $\widehat{B}$; in that case $\widehat{B} \equiv B_0 \equiv B_1$.

Definition 7.92 (The calculus V)**.** V has three rule families.

(i) USE_ν: for an occurrence $[A;\, y.\, B]$ in Γ, from $\Pi;\Gamma \vdash A$ infer $\Pi;\Gamma \vdash \mathrm{nf}(B(\nu(\Pi)))$; for a clause without antecedent the premise is omitted.

(ii) CLOSE: the rule CLOSE of K (Section 7.8), at $\ell = \nu(\Pi)$, by progress exactly when $\mathrm{nf}(B(\ell,0)) \equiv \top_*$.

(iii) EXPORT: the rule EXPORT of K with the selector

$$r^{\mathsf{V}}_B := \begin{cases} r, & B_1 \equiv (b = r), \text{ and } B_0 \equiv \top_* \text{ or } \widehat{B} \equiv B_0, \\ b, & \text{otherwise,} \end{cases}$$

in place of r_B:

$$\frac{\Pi;\Gamma \vdash D_{x,y}B \qquad \Pi, b;\ \Gamma, [B_0;\, y.\, \widehat{B}(y)] \vdash F}{\Pi;\Gamma \vdash \mathrm{nf}\big(F[r^{\mathsf{V}}_B/b]\big)} \qquad b \notin \mathrm{FV}\big(\mathrm{nf}(F[r^{\mathsf{V}}_B/b])\big).$$

There are no other rules; every derivation starts from the context Rf.

No rule of V takes a term: USE takes an occurrence and the derivation of its antecedent, CLOSE a matrix and the derivation of its premise, EXPORT two derivations. Terms still occur in the matrices supplied to CLOSE; what is removed is the separate term argument, not the information it carries. The first case of r^{V}_B is r_B of K; the second is new, and r may contain b. The two cases exclude each other, because in the second B_0 is the equation $b = r$, whose left side is a parameter.

Lemma 7.93 (Soundness of the vacuous case)**.** *Every rule of* V *is valid in* T_R *under the ordinary reading, also when the selected term contains* b*.*

Proof. USE_ν, CLOSE and the first case of EXPORT are covered by Theorem 7.72. In the second case the opened clause reads $(b = r) \to \forall y\,(b = r)$, with y not free, which is a theorem of intuitionistic logic. Suppose the reading of the branch is derivable in T_R from the readings of Γ and of this clause; cutting the clause gives a derivation of F from Γ in which b is a free variable not free in Γ. Substituting the term r for b throughout this derivation gives $F[r/b]$ from Γ, also when r contains b, and T_R proves it equivalent to $\mathrm{nf}(F[r/b])$ by Theorem 9.23. This formula does not contain b. The major premise is not needed in this case. □

Lemma 7.94 (The vacuous packet)**.** *For every scope* Π*, context* Γ *and term* t *over* Π*, the formula*

$$\mathsf{V}_t := D_{x,y}(x = t)$$

is derived in V *by one* CLOSE *and one* USE_ν*.*

Proof. CLOSE selects $\ell = \nu(\Pi)$; its matrix at ℓ is the equation $\ell = t$, which does not contain y. In either mode the assumption of the premise and the instance asked for are both $\ell = t$, the mode being by progress only when $\ell \equiv t \equiv 0$. USE_ν of the assumption clause $[\ ;\ v.\, \ell = t]$ derives the instance. □

The reading of V_t is $\exists x\,(x = t \to \forall y\,(x = t))$, a tautology. It does not assert that an opened witness equals t, and its opened clause $[b = t;\ y.\, b = t]$ gives no equation $b = t$; it is the selector that substitutes t for b in the exported formula.

Lemma 7.95 (Specialization at a term)**.** *Let* $[A;\ y.\, M(y)]$ *be an occurrence in* Γ *and* t *a term over* Π*, and suppose a derivation of* A *can be constructed over the scope* Π, b *and the context* $\Gamma, [b = t;\ y.\, b = t]$ *for a fresh* b*. Then* V *derives* $\Pi; \Gamma \vdash \mathrm{nf}(M(t))$ *by four inferences above that derivation: the two of Theorem 7.94, one* EXPORT *and one* USE_ν*. For a clause without antecedent no derivation of* A *is needed.*

Proof. Derive V_t by Theorem 7.94 and apply EXPORT at the fresh b. The opened matrix is $b = t$, so $\widehat{B} \equiv B_0 \equiv B_1 \equiv (b = t)$ and $r_B^{\mathsf{V}} \equiv t$. In the branch b is the newest parameter; USE_ν of the occurrence, with the given derivation of A, gives $\mathrm{nf}(M(b))$. The exported formula is $\mathrm{nf}(\mathrm{nf}(M(b))[t/b]) \equiv \mathrm{nf}(M(t))$ by Theorem 9.7, and it does not contain b because neither M nor t does. □

The derivation of A must be constructed in the extended scope: a derivation of A over Π is in general not a derivation over Π, b, since its uses of clauses and its introductions select $\nu(\Pi)$, which is no longer the newest parameter (Theorem 7.5). The lemma is therefore applied to persistent constructions. Applied to Rf, it gives $b = b$ in the branch and $t = t$ as the exported formula: reflexivity at every term, in every scope, by four inferences and without a term argument. This is the *persistent reflexivity* used below.

Remark 7.96 (The obstruction does not apply)*.* At the closed root both USE_ν and CLOSE use 0, and in every scope both use the same parameter. This does not contradict Theorem 7.7, which concerns the rules of N, whose elimination DE substitutes nothing. The validity invariant of that proof, which sends every parameter to 0, is not preserved by the vacuous case of EXPORT: the branch is valid at $b \mapsto 0$, while the conclusion is the branch formula at the term r, whose value can be any number. The restriction of the targets is compensated by a discharge that substitutes a term read off the major premise.

7.9.3 Conditional copy and the removal of both terms

The constructions of Section 7.8.2 transfer to V, with every use of a clause at a term other than the newest parameter replaced by Theorem 7.95 and every reflexivity by the persistent reflexivity. The ordinary elimination of $D_{x,y}B$ is EXPORT with a branch conclusion F that does not contain b: then $\mathrm{nf}(F[r/b]) \equiv F$ for every selected r, so the extended selector does no harm. The tower $T_0 = \top_*$ is the persistent reflexivity at 0, and T_{d+1} is derived by CLOSE with an unused witness, whose assumption in either mode is T_d, the instance asked for. The closure $q_y A$ is introduced as in Section 7.8.2, with the persistent reflexivity for $A(0)$ when $A(0) \equiv \top_*$, and $A(t)$ is obtained from $q_y A$ and $A(0)$ by opening $q_y A$ and applying Theorem 7.95 to its clause at t, with $A(0)$ as the antecedent.

Lemma 7.97 (Conditional copy in V)**.** *For every formula $H(x)$ with a distinguished variable and every term t over Π, V derives $\Delta_t(H)$ of Theorem 7.74 in every scope and context, by a construction that is carried out anew in each scope.*

Proof. The proof of Theorem 7.74 applies with two changes. The outer introduction of the closure (7.8) is by cases, since its instance at 0 is the tower T_{d+1}, where d is the depth of H. In the branch for Sz the conclusion normalizes to $\Delta_z(H)$, and z is the newest parameter, so CLOSE for $\Delta_z(H)$ selects z and both of its instances are $H(z)$. If $H(z) \not\equiv \top_*$ the rule is by cases and USE_ν of the assumption gives the instance; if $H(z) \equiv \top_*$ the instance is the persistent reflexivity at 0, the progress assumption being unused; this is the first change. Then open the closure at a fresh d'; its clause is $[T_{d+1};\, v.\, G_v \Delta_{P(v)}(H)]$. The second change: in place of the use of this clause at St, apply Theorem 7.95 at St, with the persistent tower T_{d+1} as the antecedent, constructed in the scope of its use. The result is $\mathrm{nf}(G_{St}\Delta_{P(St)}(H)) \equiv \Delta_t(H)$, which contains neither d' nor the parameter of the packet, and the ordinary elimination discharges d'. The construction is explicit and makes no recursive call to a translation of derivations. □

Let L be the liberal calculus of Theorem 7.75: USE at every term, CLOSE at every witness, and EXPORT with r_B.

Theorem 7.98 (Both terms removed)**.** *For every scope Π, clause context Γ and formula A the calculi V, K and L derive the same judgments $\Pi; \Gamma \vdash A$. The transformations are effective.*

Proof. K and L derive the same judgments by Theorem 7.75.

From V to L, by induction on the derivation. USE_ν is USE at $\nu(\Pi)$, CLOSE is CLOSE at the witness $\nu(\Pi)$, and an EXPORT whose selector is not in the vacuous case is an EXPORT of L with the same conclusion, because then $r_B^{\mathsf{V}} \equiv r_B$. In the vacuous case the induction hypothesis gives derivations in L of $\Pi; \Gamma \vdash D_{x,y}B$ and of

$$\Pi, b;\ \Gamma, [b = r;\ y.\, b = r] \vdash F,$$

and $G := \mathrm{nf}(F[r/b])$ does not contain b. (1) Every use of the clause $[b = r;\ y.\, b = r]$ concludes its antecedent $b = r$; replacing each such node by the derivation of its premise, and deleting the occurrence from all contexts, gives $\Pi, b; \Gamma \vdash F$ in L; no rule of L depends on the length of the context. (2) For a fresh c, the injective renaming $b \mapsto c$ into the scope Π, b, c and weakening by a clause (Theorem 7.77) give $\Pi, b, c; \Gamma, \mathcal{C} \vdash F[c/b]$ for every clause $\mathcal{C}$ over Π, b, c. (3) Over Π, b the name $D_{x,y}(C(y, 0, x) = C(y, 0, r))$ is derived by CLOSE at the witness r: the instance at 0 is $\top_*$, the mode is progress, and the instance asked for normalizes to $r = r$, a USE of Rf at r. EXPORT of the name at c has the rows $\top_*$ and $c = r$, selects r, and with the branch of (2), $\mathcal{C}$ being the opened clause, concludes $\mathrm{nf}(F[c/b][r/c]) \equiv G$, which contains neither c nor b. (4) Weakening by the clause $[b = r;\ y.\, b = r]$ and EXPORT of the given major premise at b with this branch conclude G over Π, since G does not contain b.

From L to V, by induction on a derivation π of L, with the statement of the proof of Theorem 7.75: for every injective map θ of the source parameters into a target scope and every map of the source clause occurrences to target occurrences whose clauses are literally the normalized θ-images, V derives the normalized θ-image of the conclusion; further parameters and clauses are allowed, and Rf is sent to Rf. The images of t, B and so on are written with the same letters.

USE at a term t: apply Theorem 7.95 at t to the image occurrence; the derivation of the antecedent in the extended situation is the induction hypothesis for the source derivation of the antecedent, with the same maps, the packet parameter and its clause being extras.

EXPORT: translate the major premise and the branch, the source eigenvariable sent to the fresh target parameter and the opened clause to the opened clause. An injective renaming

preserves and reflects the conditions $B_0 \equiv \top_*$, $B_1 \equiv (b = r)$ and $\widehat{B} \equiv B_0$, by Theorem 9.7. If the source selector succeeds, so does the first case of r^{V}, with the image of the source term. If neither case applies, both fall back. If the vacuous case applies, the source selector falls back, because B_0 is the equation $b = r$ and not $\top_*$; the source branch conclusion therefore does not contain the source eigenvariable, its image F does not contain b, and $\mathrm{nf}(F[r/b]) \equiv F$. In each case the target conclusion is the image of the source conclusion.

CLOSE at a witness t with the matrix B and the premise $\Gamma, [\,;\, v.\ \mathrm{nf}(B(t,\eta))] \vdash \mathrm{nf}(B(t, Sz_s))$: put $H(x) := q_y B(x,y)$ and follow the steps (a) to (e) of the proof of Theorem 7.75.

(a) Derive $\Delta_t(H)$ by Theorem 7.97 and open it at a fresh b, with the clause $[H(t);\ v.\,\mathrm{Sel}(v, H(t), H(b))]$.

(b) Apply CLOSE to $D_{x,y}B$; the newest parameter is b, so the rule selects b; let z be its eigenvariable.

(c) In the premise derive $H(t)$ by a CLOSE with an unused witness and a further eigenvariable w. Its test is the source test at t, so its assumption clause is literally the image of the source assumption with $z_s \mapsto w$; the induction hypothesis for the source premise, with these maps, gives $\mathrm{nf}(B(t, Sw))$. This construction is carried out anew wherever $H(t)$ is needed.

(d) Apply Theorem 7.95 at 1 to the clause of (a), with the construction (c) of $H(t)$, made in the scope of the packet, as the antecedent: this gives $\mathrm{Sel}(1, H(t), H(b)) \equiv H(b)$.

(e) Open $H(b)$ at a fresh d, with the clause $[B(b,0);\ y.\,B(b,y)]$, and apply Theorem 7.95 at Sz to it. Its antecedent $B(b,0)$ is USE_ν of the assumption of the CLOSE of (b) if that rule is by cases, and the persistent reflexivity at 0 if it is by progress, since then $B(b,0) \equiv \top_*$. The result $\mathrm{nf}(B(b,Sz))$ does not contain d, and the ordinary elimination discharges d.

The CLOSE of (b) yields $D_{x,y}B$, which does not contain b, and the ordinary elimination discharges the opening of (a). The recursive calls are made for the premise of the source CLOSE and for the derivation of a source antecedent, both proper subderivations, possibly several times and each time in the situation where the result is used; the packets, the towers and the conditional copy are explicit constructions, which are larger than the source premise but are not sent through the translation. The tests at t and at b may differ, as in Theorem 7.76. □

Corollary 7.99 (The bridge for V)**.** V *is a sound and complete presentation of* T_R*, and its ordinary reading is bi-interpretable with* iPRA*, with whole domains, equality as equality, the graph* $x = y$ *in both numerical composites and a provable equivalence of every formula with its double translation. Hence* V *and* iPRA *are equiconsistent.*

Proof. Theorems 7.82 and 7.98. V has the formulas, terms and readings of K; only derivations change, so the formula round trips of Theorem 7.82 hold unchanged. □

Remark 7.100 (The two packages)*.* `export_specialization` proves Theorem 7.87 and Theorem 7.91 and the count of nodes; it keeps the witness of the introduction and REFL unrestricted and asks whether both restrictions can be combined, and whether arbitrary reflexivity survives a restriction of the reflexivity clause. On the recursor syntax with normalization, `vacuous_specialization` answers both: Theorem 7.98 restricts the witness, the target of every use, including the uses of Rf, and reflexivity at every term is the persistent reflexivity. The conditional copy of Theorem 7.74 with the restricted introduction is from `conditional_copy`; the new ingredient is the vacuous case of the selector. The package proves the inclusion of V in L semantically; the proof above derives the vacuous case in L. On iterator terms no term satisfies $P(Sz) \equiv z$ by computation, and the combination there is open (Theorem 7.103).

7.10 Checkers and sizes

This section draws on the checker reports of `no_witness`, `parameter_witness`, `deterministic_introduction` `newest_parameter`, `uniform_raw`, `parameter_intro`, `conditional_copy`, `local_introduction`, `export_specialization` and `vacuous_specialization`. The checker of `constructed_computation` and the compact checkers of `export_specialization` and `vacuous_specialization`, which check the guarded scalar calculus and not the calculi of this chapter, belong to another chapter.

All checkers are standalone Python modules that read one JSON certificate, validate syntax, scopes and every inference, and accept or reject; none calls a prover. They share the encoding of syntax: the integer 0 is zero, negative integers are parameters counted from the outside, positive integers are de Bruijn indices, and lists encode the constructors. The sizes are measured by the protocol of Section 1.10 under CPython 3.12.14.

Table 7.1: Checkers of the chapter: significant tokens, AST nodes and normalized bytes, with the matched control as tokens / nodes / bytes.

Program	Tokens	Nodes	Bytes	Matched control
`no_witness`				
eight rules, N_ν	971	1078	2817	supplied witness: 979 / 1086 / 2834
seven rules, $\mathsf{N}_{\mathrm{diag},\nu}$	924	1021	2753	supplied witness: 932 / 1029 / 2770
`parameter_witness`				
five families, C_β	833	971	2947	two-case raw clause checker: 837 / 977 / 2956
`deterministic_introduction`				
G_d	1159	1339	3688	G: 1078 / 1252 / 3442
inherited native calculus, shared first traversal	333	379	656	incoming native control: 360 / 409 / 780
inherited guarded compact module	267	303	506	before the change: 267 / 302 / 508
`newest_parameter`				
G_ν	1070	1243	3425	G: 1078 / 1252 / 3442; G_d: 1159 / 1339 / 3688
`uniform_raw`				
G_ν	1073	1248	3439	G_d: 1159 / 1339 / 3688
`parameter_intro`				
P_ν	630	701	1925	supplied witness: 636 / 707 / 1967
P_ν with a separate template check	646	717	1985	
P_ν with fuel	667	748	2056	supplied witness, with fuel: 673 / 754 / 2098
`conditional_copy`				
K	646	748	1844	L: 651 / 755 / 1863
K_N, backward	619	686	1907	P_ν of `parameter_intro`: 630 / 701 / 1925
explicit equation at NAME	634	742	1835	
K with fuel	685	796	1995	
`local_introduction`				

Program	Tokens	Nodes	Bytes	Matched control
A_{ν}	862	1011	3245	A_{ν} with a supplied CLOSE witness: 870 / 1022 / 3274
`export_specialization`				
$\mathcal{X}_{\nu}$, REFL primitive	663	748	2093	$\mathcal{X}$, REFL primitive: 669 / 754 / 2117
$\mathcal{X}_{\nu}^{\mathcal{R}}$, target of $\mathcal{R}$ read off the goal	654	731	2049	$\mathcal{X}^{\mathcal{R}}$ of `clause_discharge`: 653 / 729 / 2058
`vacuous_specialization`				
V	645	745	1830	K of `conditional_copy`: 646 / 748 / 1844

The packages compare their programs with further programs of the corpus. `parameter_intro` states 636 / 708 / 2010 for the incoming checker with an explicit name and 683 / 771 / 1990 for the incoming checker with an inferred opening; its gzip sizes are 778 for its checker, 790 for the matched control, 792 with the separate template check, and 826 and 838 for the two variants with fuel. `conditional_copy` states 662 / 741 / 1950 for the checker of `uniform_recursion`. The compact programs of the corpus use other input contracts and are smaller: 267 tokens, 302 nodes and 508 bytes; 256, 283 and 594; 263, 276 and 580; and, with a validator of the public goal, 360, 409 and 780. The three minima 256, 276 and 508 belong to different programs. No package claims a global minimum or a lower bound.

`no_witness.` An introduction is the instruction `['di', matrix, premise]`; its witness is the integer $-n$ in a scope of n parameters, which is zero in the empty scope. The controls differ only by a supplied and validated witness term. Each matched pair differs by 8 tokens, 8 nodes and 17 bytes. In the tagged syntax of the pinned programs the same change saves 3 bytes and 1 node and adds 2 tokens. Eighteen accepted and 33 rejected certificates are run in two modes; the certificate for $X(2)$ checks the part of the naming that uses numerical cases and not the predecessor or the equivalence theorem.

`parameter_witness.` The introduction field is a bit, decoded by the truth value of the JSON value; it is never inserted as a term, and the witness is constructed from the scope. Certificates for the packet T at 0, for its closure and for its instances at 1 and 2 are checked, and a certificate that uses the eigenvariable of an introduction as its witness is rejected. A separate evaluator checks 447 numerical equalities of the predecessor construction, among them the values of the predecessor at $0, \dots, 7$.

`deterministic_introduction.` The naming matrix is recognized as the literal list pattern of Nm_{r} with r valid in the parent scope. The checker of G_{d} is larger than its control by 81 tokens, 87 nodes and 246 bytes: the rule removes a datum from each introduction and costs more code to recognize the exception. The fixtures contain an introduction that is valid before an extension of the scope, the rejected replay of the same tree after the extension, and the repaired derivation of Theorem 7.53. The package also contains two changes to inherited programs that do not implement G_{d}. One checks the grammar of the public goal during the first scope traversal, using the parity of the traversal depth as the distinction between formulas and terms; an induction on the goal shows that the shared pass succeeds exactly when the two separate passes do and that the subsequent traces are equal. The other replaces $k-1$ by the bitwise complement of $-k$ in the compact module, which saves two bytes and adds one node.

`newest_parameter` and `uniform_raw`. Both obtain the checker of G_ν from the checker of G_d by deleting the function that selects the witness and instantiating the matrix at $-n$. `newest_parameter` also contains an unmeasured checker of the eight rules of N_ν, with certificates for $X(St)$ at a parameter t in scopes of zero, one, two and five parameters, and a rejected replay of a closed derivation in a larger scope; it also replays symbolic certificates of seven equation-case lemmas of the predecessor construction. `uniform_raw` checks the whole closure $D_{a,r}X(r)$ and its instance at the successor of a compound term.

`parameter_intro`. The recursor and the binder $D_,$ both bind two variables in their last child, so one traversal serves terms, formulas, scopes, substitution and normalization. The reduction of a successor counter substitutes the previous counter first, into the outer of the two bound positions, and the previous value second; the opposite order without a shift would capture variables. The witness of an introduction is the integer $1 - n$ for a context of n clauses. Omitting the witness field saves 42 bytes, 6 tokens and 6 nodes; omitting the separate template check, by Theorem 7.71, saves 60 bytes, 16 tokens and 16 nodes. The variant with fuel decrements a budget at every call of the traversal and of the inference checker; every accepted certificate has a sufficient budget, so the two interfaces have the same theorems, and the bounded relation is primitive recursive. The certificates that name 1 and 2 without a witness field need 1155 and 1345 units.

`conditional_copy`. The forward checker of K infers every conclusion and compares the goal with it. Omitting the witness saves 19 bytes, 5 tokens and 7 nodes against L. The checker of K has fewer bytes and more tokens and nodes than that of `parameter_intro`; the backward checker of K_N is smaller in all three coordinates, by 18 bytes, 11 tokens and 15 nodes. The package contains a translator from L to K that implements the proof of Theorem 7.75 on arbitrary derivations. Its outputs show the growth of certificates, in inference nodes and bytes of JSON: an introduction at the witness 1 grows from 2 nodes and 74 bytes to 15 and 529; a conditional copy of depth one from 7 and 368 to 46 and 2527; a derivation with a used progress assumption from 15 and 421 to 80 and 2868; the congruence of the successor under two universal quantifiers from 25 and 1335 to 142 and 8909; a derivation in an open scope from 4 and 76 to 17 and 501.

`local_introduction`. The input is a JSON object with exactly the keys `goal` and `proof`; the goal is a closed formula with equations and $D_,$, validated in full, and the proof is a tagged tree with six tags: one for each of USE, BACK, ALIAS, OPEN and CLOSE, and one for the two computations, given by a counter, a seed and a body. Every consumed term is validated in its scope, including an iterator body that a zero computation does not evaluate; the source of BACK must be the newest parameter and not 0; the unused premise field of a USE of a clause without antecedent is ignored. The checker of A_ν and its control share all code, including the diagonal test, and differ only in the witness field of CLOSE, supplied and validated in the control and computed from the scope in A_ν, by the invariant that the scope and the context have the same length. The difference is 8 tokens, 11 nodes and 29 bytes. The diagonal test is absent from the witness-selected checkers of the Alias/Back basis; the short raw Alias/Back checker of `alias_back`, 773 / 916 / 2903, is smaller than both programs of the row. The package remeasures the compact incumbents of the corpus under their own contract, a closed guarded public goal without a grammar validator and a different certificate protocol: 248 / 277 / 609, 254 / 270 / 608 and 272 / 304 / 479, and 344 / 392 / 637 with the full native goal and its alphabet validated; the envelope 248 / 270 / 479 is attained by three different programs. An untrusted compiler implements the proof of Theorem 7.46 for sources in the control calculus, in which USE and BACK are already at the newest parameter, and replays

every output through the checker of A_ν; it does not parse the larger calculus A_*. Eighteen source certificates are compiled and checked, eighteen adversarial certificates are rejected, and 108 process runs in the normal and the optimized interpreter mode pass; one open example uses an embedding of parameters that does not preserve their order and a map of clauses that is not contiguous.

`export_specialization`. All four programs are complete closed-theorem checkers of the normalized calculus on iterator terms: they read one JSON pair of a closed goal and a proof, validate syntax and scopes, normalize, check every inference and print `ACCEPT`; an exception rejects. The matched pair with primitive REFL isolates the restriction of USE: a use packet loses its term field and its validation, and the checker instantiates at the newest parameter. The second pair keeps the reflexivity clause at context position 0 as in the control of `clause_discharge`; a use of position 0 takes its term from the left side of the validated equational goal, and every other use specializes at the newest parameter. This pair saves 9 bytes and adds one token and two nodes; the first saves 24 bytes, 6 tokens and 6 nodes. The validation of the witness of INTRO is kept in all four. An untrusted translator compiles the nine certificates of `clause_discharge` (congruence, symmetry, transitivity, induction, nested binders, the repair at the witness 0) to all four protocols and back; five open targets, an injective renaming with clauses on both sides of the embedded clause, a regression for capture under formula and iterator binders, and thirteen negative controls are replayed.

`vacuous_specialization`. The checker of V and its control, the checker of K of `conditional_copy`, read one JSON pair of a goal and a proof. Terms are 0, rigid parameters $-1, -2, \ldots$, positive de Bruijn indices and $[n, a, h]$ for $R(n, a;\, i, u.\, h)$, with u of index 1 and i of index 2 in h; formulas are $[s, t]$ and $[B]$ for $D_{x,y}B$. The initial context is Rf, and a context of length n has the parameters $1-n, \ldots, -1$, so that $1-n$ is the newest parameter, and 0 at the root. A use is $[j, q]$, a CLOSE is $[B, q]$, an EXPORT is $[[\,], p, q]$ and opens at $-n$; the vacuous case compares the whole body of the opened matrix with its instance at 0. The only differences from the control are the deletion of the term of a use with its validation and the enlarged selector. The program `control_liberal.py` of the package is the checker of L, the source of an untrusted compiler that implements the translation of Theorem 7.98 and whose outputs are rechecked; 39 focused checks and 8 command-line checks under normal and optimized interpretation pass, among them the packet, reflexivity at a compound term, conditional copy at three depths, a used induction hypothesis, nontrivial embeddings of parameters and clauses, and the rejection of a surviving fresh parameter and of rows at 0 and 1 that agree without a constant matrix.

The restricted rules reduce the data of a certificate and, except for G_d, the measured checker by a small amount. They increase the length of certificates, because a witness term is replaced by a derivation of its name.

7.11 Further questions

The following items are stated by the packages without proof, are open, or mark what the results of the chapter do not establish.

Question 7.101 (Formalization of the scope-determined witness)**.** The Lean development formalizes the restriction of the witness to the literal 0 or a variable, with the derivation of the names (Theorem 7.23), and the predecessor in the equational fragment (Section 7.4). It does not formalize the deterministic selection of the witness, the newest parameter or 0 in an empty scope, with the regeneration of derivations under an extension of the scope. Hence none of Theorems 7.35, 7.38 to 7.40, 7.46, 7.53, 7.55 and 7.63 is formalized, and nothing in Sections 7.7 and 7.8 on the counter-aware recursor is formalized, in particular Theorems 7.69,

7.74 and 7.75. A formalization of the first has to take derivations of the source with realized contexts, or hereditary families, as its induction interface. For G_{d} the selection with priority for a naming matrix, the clause contexts and the guarded language are not formalized. For the recursor calculi a syntax with a normalizer and its bi-interpretation with the iterator terms (Theorems 9.7 and 9.25) would be needed first; `conditional_copy` proposes a small interface for Theorem 7.74 consisting of a predecessor that computes on successors, gates, clause specialization, CLOSE at the newest parameter and the ordinary elimination, so that the theorem can be reused. Theorem 7.71 and the translation of Theorem 7.69 are named by `parameter_intro` as the next formalization task. `local_introduction` proposes a Lean definition of the calculus A_{ν} with ordered scopes and the induction statement of Theorem 7.46 over clause realizers, kept apart from the invariant of closed certificates that the scope and the context have the same length.

Question 7.102 (Equation-only BACK with the newest CLOSE)**.** The proof of Theorem 7.46 transports the assumption and the conclusion of a CLOSE, which are compound when the matrix has a binder, by BACK on compound formulas (Theorem 7.48). `alias_back` states that its presentation with a supplied witness admits the restriction of BACK to equations. Is A_{ν} with BACK restricted to equations still equivalent to A_{*}? A positive answer needs a noncircular recovery of formula transport from equational BACK, the newest USE and the newest CLOSE.

Question 7.103 (Three families on raw iterator terms)**.** Can the introduction at the newest parameter be combined, on raw iterator terms without normalization, with a three-family calculus whose equality transport is recovered from packets? The proofs of Section 7.5 keep TR as a primitive rule, and transferring them unchanged would be circular. Sections 7.7 and 7.8 answer the question on the recursor with normalization, where $P(Sz) \equiv z$ holds by computation; on iterator terms no term has this property. A technical note of `uniform_raw` sketches a raw calculus with a universal reflexivity clause, a deterministic introduction, USE and a discharge that evaluates, in which the opening and the evaluation rules are recovered through the dummy major premise $D_{x,y}(I(y,0;\,u.\,0) = I(y,0;\,u.\,0))$ and formula transport is proved before the names. The note is not part of the proved results of that package.

Question 7.104 (Transport in terms only)**.** In G_{d} and G_{ν} the rule TR acts on arbitrary canonical formulas, and the restricted cut of Theorem 7.51 uses this. Can it be weakened to transport in terms before the bootstrap, with formula transport proved without unrestricted witnesses and without unrepaired weakening?

Question 7.105 (The size of certificates)**.** The translations are effective. No linear or polynomial bound on the size of the translated certificates is claimed; the translations regenerate a premise inside additional scopes and may duplicate it. Determine the growth, and whether sharing of regenerated subderivations in a checked directed acyclic graph reduces it without enlarging the trusted rules. `local_introduction` asks the same for the repeated production of persistent families in the proof of Theorem 7.46, and whether the sharing can be checked without erasing the saving of the interface.

Question 7.106 (Modes without a guard)**.** Can the scope determine the witness while the mode of the introduction is also determined without a test on the matrix, with a direct proof of all native consequences? In N_{ν} the progress rule QI is kept; the result does not remove it, does not reduce the number of rules and does not settle the independence of progress.

Question 7.107 (The specialization term)**.** In the calculi of the chapter up to Section 7.8, QE, or USE, still takes an arbitrary term; the proofs specialize a closure at St. By Theorems 7.7 and 7.8 the target cannot be restricted to a parameter or finitely many numerals together with

the witness in the identity language of N, whose elimination substitutes nothing. Theorem 7.98 delivers the term by the vacuous case of an inferred discharge instead. Whether a calculus with the native elimination DE and both restrictions, differently interpreted, is bi-interpretable with iPRA is not decided.

Question 7.108 (Newest specialization)**.** (i) Theorem 7.87 is formalized against `XCPrf`, as `export_specialization` proposes (Theorem 7.88), without the node count; Theorem 7.98 is not formalized, and `vacuous_specialization` one of its compiler with the invariant on parameter and clause embeddings, beyond the soundness of the vacuous case. Their translators are untrusted programs whose outputs are rechecked; their correctness is supported by finite tests only.

(ii) The size of translated certificates. Theorem 7.87 bounds the inference nodes by $n + 3u$, but not the size of the normalized terms, the bytes of a certificate or the time of checking. In V a use of a clause costs four inferences before its antecedent, and an introduction at an arbitrary witness invokes conditional copy. A benchmark that reports proof size and checking time beside checker size is proposed.

(iii) Is there a larger class of cheaply recognized tautological closure matrices than those with $\widehat{B} \equiv B_0$ that keeps the discharge deterministic and makes the complete checker smaller?

(iv) Does the raw selector of Theorem 7.91, with its condition on free variables, cost less in a complete checker than the selector on normal forms? No raw checker is measured.

(v) Can the newest specialization, or the vacuous selector, be built into the arity-coded compact checkers, so that one program checks the new calculus and is smaller than the compact records? Can one module improve all three coordinates of the compact envelope at once under a fixed contract? The packages ask, with `local_introduction`, for a representation-stable measure that charges normalization, parsing and all decoding and admits nontrivial lower bounds; neither package proves minimality or a lower bound.

(vi) Membership-only majors. `export_specialization` considers the calculus in which the major premise of QE and of DE must be a literal member of the context instead of a derived formula, any computation equation may be added to the context, and all other rules are unchanged. It argues that this calculus does not derive $\vdash I(v, 0; u.\, u) = 0$, which N derives by equation induction: from equation-only contexts an introduction cannot end a derivation of an equation and neither elimination finds a compound major, so only hypotheses, computation equations and transport remain, and the free term algebra modulo the computation equations separates the two sides. The article does not state this as a theorem; a complete proof would have to fix the induction on derivations and the equational countermodel exactly. It bounds which checker shortcuts preserve the open consequences of N, not the size of every certificate design.

Question 7.109 (Less normalization)**.** The proofs of Sections 7.7 and 7.8 use the literal identities $P(St) \equiv t$ and the two instances of the selector. Is there a canonical syntax that guarantees these reductions without full normalization, with its own coherence law in place of Theorem 9.7?

Further remarks of the packages on what is not shown:

- No package proves that its presentation is minimal, and no checker of a calculus of the chapter improves the compact programs of the corpus; the compact checkers of `export_specialization` and `vacuous_specialization` check another calculus. Whether

the restriction of the witness improves the compact records under a matched contract is open, and a comparison that fixes one validation contract for all competing calculi is proposed. The question of `deterministic_introduction` whether the selector by pattern can be recognized more cheaply is superseded by Theorem 7.63, which removes the selector.

- `local_introduction` records that its target, a presentation whose complete checker lies below the current compact programs, is not attained: the checker of A_{ν} is smaller than its matched control only, and the diagonal test it needs is absent from the witness-selected Alias/Back checkers. Whether the removal of the witness can reduce the smallest complete checker, with the diagonal or the mirror test and the alias machinery made cheaper while exact freshness and a stated input contract are kept, is open; no measurement of a mirror variant is made. The package also asks for a formulation of a genuine lower bound on checker size, which has to fix the implementation language, the input contract, the accounting of dependencies, the convention for success and the resource model; the measured envelope and the obstruction theorems of the article do not provide one.

- `deterministic_introduction` supplies a complete elaborator from certificates of G_{d} to certificates of G and illustrates the converse by one translated pair; `uniform_raw` supplies no translator; the translator of `conditional_copy` is not verified, and the compiler of `local_introduction` is checked only by the replay of its outputs. The correctness of the Python programs is supported by finite tests only.

- In Theorems 7.70 and 7.80 the equivalence of the diagonal calculus with N and the bridge are used over the vocabulary extended by the fixed descriptions R. The packages argue that the proofs are parametric in additional terms; the Lean development covers the iterator terms.

Chapter 8

Recursors with a built-in seed

The iterator $I(n, a;\, u.\, h)$ of Section 1.1 takes its initial value a as an independent argument and returns it unchanged at the counter 0. This chapter treats three arithmetic bases in which the value at 0 is tied to other data of the recursor.

- The *successor-seeded* recursor $R(n, a; u.h)$ starts at the successor of its initializer, $R(0, a; u.h) = Sa$. The successor is then the abbreviation $Sa := R(0, a; u.0)$, and the term language is $x \mid 0 \mid R(n, a; u.h)$.
- The *tied-seed* recursor $B(n; u, v.h)$ has no initializer. Its update sees the old counter u and the old state v, and its value at 0 is $S(h[0/u][0/v])$.
- The *self-seeded* recursor $F(n; v.h)$ has no initializer and its update sees only the old state; its value at 0 is $S(h[0/v])$.

The hypotheses differ from base to base, and Table 8.1 records which properties are axioms and which are derived. In every case the logical base is either ordinary intuitionistic first-order logic with equality or the native binders Q_y, $D_{x,y}$ of Section 1.3.

Theory	Logic	Axioms beyond the two recursor equations	Derived
N_R	native	none (the eight rule forms of N)	predecessor, iterator, hence everything derived in N
T_R	ordinary	coverage, $0 = S0 \to \bot$, induction for literal equations	predecessor, injectivity of S, iterator
$\mathsf{T}_{B\varepsilon}$	ordinary	Boolean equality test ε (range, reflection), $0 = S0 \to \bot$, induction for literal equations	predecessor, injectivity, quantifier-free induction, coverage
T_{Bd}	ordinary	two equations for d, coverage, $0 = S0 \to \bot$, induction for literal equations	predecessor, injectivity, reflection of equality, quantifier-free induction
N_{Bd}	native	two equations for d	the same, and the ordinary connectives
T_F	ordinary	injectivity of S (redundant by Theorem 8.69), $0 = S0 \to \bot$, Boolean equality test, induction for literal equations	coverage, predecessor (as a finite-case graph)
T_F^-	ordinary	as T_F without injectivity	coverage, $S^k a \neq a$ for each fixed $k \geq 1$, injectivity of S, hence every theorem of T_F (Theorem 8.70)

Table 8.1: Axioms and derived properties of the theories of this chapter.

All recursors are staged: the body h of a recursor term is a term built earlier, and each fixed body may be read as a function symbol whose arguments are the counter, the initializer where there is one, and the free variables of the body. No theory of this chapter quantifies over bodies or has an evaluator for them. Substitution is capture-avoiding. The numerals are $1 := S0$, $2 := S1$, and so on; $S^k t$ denotes k written applications of S, and $\bar{k} := S^k 0$.

8.1 The successor-seeded recursor

This section draws on `scalar_successor`, `transient_seed` and `successor_seed_completion`, three write-ups of the same recursor.

8.1.1 Terms, equations and the two theories

Definition 8.1. The R-terms are

$$t ::= x \mid 0 \mid R(n, a; u.h),$$

where $R(n, a; u.h)$ binds u in h and nowhere else. The successor is the abbreviation $Sa := R(0, a; u.0)$. The two computation equations are

$$R(0, a; u.h) = Sa, \tag{8.1}$$
$$R(Sn, a; u.h) = h[R(n, a; u.h)/u]. \tag{8.2}$$

With the abbreviation expanded, (8.1) reads $R(0, a; u.h) = R(0, a; u.0)$ and the counter on the left of (8.2) is $R(0, n; u.0)$; no successor constructor is hidden.

(a) N_R is the native calculus with the eight rule forms of N (Section 1.3) over the R-terms: HYP, (8.1) and (8.2) as the two computation equations, TR, QE, DE, DI by cases and QI by progress for literal equations. The contradiction is $0 = S0$.

(b) T_R is ordinary intuitionistic first-order logic with equality over the R-terms, with (8.1), (8.2), coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$, separation $0 = S0 \to \bot$, and induction for literal equations E with side assumptions: from $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$, with z fresh, infer $\Gamma \vdash E(t)$.

Neither theory has an axiom of successor injectivity, a predecessor, a selector or an initializer. The term language is that of the seeded recursor of Theorem 9.1, where it is studied on canonical terms with normalizing substitution; here the terms are raw and both computation equations are rules. Weakening, substitution and cut (Theorems 1.10 and 1.11) hold for N_R with the same proofs: the only cases that mention a term constructor are those of the two computation equations, and (8.1), (8.2) are closed under substitution like (1.1), (1.2). In the same way every result of Section 1.4 whose proof does not use a computation equation holds for N_R, and for the other native calculi of this chapter; these results are cited below for such calculi without further comment.

Throughout the section

$$\mathrm{sel}^+(n, a, b) := R(n, a; u.b) \quad (u \notin \mathrm{FV}(b)), \qquad Z(n) := \mathrm{sel}^+(n, 0, 0).$$

The computation equations give

$$\mathrm{sel}^+(0, a, b) = Sa, \qquad \mathrm{sel}^+(Sn, a, b) = b, \qquad Z(0) = 1, \qquad Z(Sn) = 0. \tag{8.3}$$

The zero branch of sel^+ adds one to the supplied value, so sel^+ is not the selector sel of Section 1.1.

8.1.2 Equality, cases and induction before the predecessor

Lemma 8.2. N_R *derives* $t = t$ *for every term* t*, symmetry and transitivity of equality, congruence* $s = t \vdash h(s) = h(t)$ *for every term template* h*, and transport of literal equations: from* $s = t$ *and* $l(s) = r(s)$ *infer* $l(t) = r(t)$*. It admits the rules*

$$\frac{\Gamma \vdash B(0) \qquad \Gamma \vdash B(Sz)}{\Gamma \vdash B(t)}\ (\textit{NC}), \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}\ (\textit{EI}),$$

where B *is any native formula,* E *is a literal equation and* z *is fresh for* Γ*,* t *and the parameters of the matrix. Under the hypothesis* $0 = S0$ *it derives every equation.*

Proof. Let $r := R(S0, 0; u.t)$ with $u \notin \mathrm{FV}(t)$. Equation (8.2) is $r = t$. Rule TR (from $s = t$ and $h(s) = u$ infer $u = h(t)$) with the source equation $r = t$, the template $h(v) := v$ and the premise $r = t$ gives $t = t$. This replaces the two instances of (1.1) in Theorem 1.18(i). The proofs of Theorem 1.18(ii)–(v) use TR and reflexivity only, and give symmetry, transitivity, congruence and transport of equations. Rule NC is Theorem 1.14 and rule EI is Theorem 1.20; their proofs use DI, DE, QI, QE and HYP and no computation equation.

Under $0 = S0$, congruence gives $\mathrm{sel}^+(0, 0, s) = \mathrm{sel}^+(S0, 0, s)$, that is $S0 = s$ by (8.3), for every s. Hence $s = t$ for all s, t. □

In T_R reflexivity and replacement are rules of the logic, NC follows from coverage, EI is the induction rule, and every formula follows from $0 = S0$ by separation.

Lemma 8.3 (Uniqueness of iteration)**.** *Let* u *be a variable not free in* Γ*. If* $\Gamma \vdash h = k$*, then* $\Gamma \vdash R(n, a; u.h) = R(n, a; u.k)$ *for all terms* n*,* a*. More generally, if* $\Gamma \vdash Sa = Sb$ *and* $\Gamma \vdash h = k$*, then* $\Gamma \vdash R(n, a; u.h) = R(n, b; u.k)$*.*

Proof. By EI on a fresh counter z for the literal equation $R(z, a; u.h) = R(z, b; u.k)$. At 0 both sides are computed by (8.1) to Sa and Sb. At Sz the left side is $h[R(z, a; u.h)/u]$, which equals $h[R(z, b; u.k)/u]$ by the induction hypothesis and congruence, and this equals $k[R(z, b; u.k)/u]$ by the instance of $\Gamma \vdash h = k$ at $u := R(z, b; u.k)$, which exists because u is not free in Γ. Then substitute n for z. □

Two bodies with different syntax are thus identified by a proof, not by a primitive extensionality rule. The lemma is the counterpart for R of Theorem 1.55, proved by the same equation induction with (8.1), (8.2) in place of the iterator equations.

Lemma 8.4 (Replacement)**.** *Write* $A \approx A'$ *if* A *and* A' *are interderivable over the empty context; by weakening and substitution they are then interderivable over every context and under every substitution.*

(i) If $\vdash t = t'$ *is derivable in the empty context, and* A' *arises from* A *by replacing occurrences of* t *by* t'*, also in the scope of binders that bind variables of* t*, then* $A \approx A'$*, in* N_R *and in* N*.*

(ii) The Leibniz rule (Theorem 1.22) and transport in a formula template (Theorem 1.23) hold in N_R*.*

Proof. (ii) The proofs of Theorems 1.19, 1.21 and 1.22 use the equality rules, QE, DE, DI and the structural properties, and no computation equation.

(i) By induction on A. At a literal equation the bound variables of A that occur in t are free, and transport of equations along $t = t'$ applies. At a binder the matrices are interderivable over the empty context by the induction hypothesis, and Theorem 1.21 applies, since no variable is free in the empty context. Statement (i) is not an instance of Theorem 1.22, whose substitution is capture-avoiding. □

8.1.3 Boolean terms and contextual case analysis

The proof of the predecessor theorem splits on the value of a two-valued term inside an induction step, where the induction hypothesis is in the context and mentions that term. Rule NC does not allow this: its premises carry no equations $t = 0$, $t = Sz$, and its context may not mention the case variable. In T_R the split is a disjunction elimination. In N_R it is derived as follows, before any selector with unshifted branches exists.

Lemma 8.5 (Functional range)**.** *For every formula template $K(w)$ and every term r: if $\Gamma \vdash K(1)$ and $\Gamma \vdash K(0)$ then $\Gamma \vdash K(Z(r))$. The context may mention r.*

Proof. Apply NC to the template $w \mapsto K(Z(w))$ with w fresh. Its zero premise $K(Z(0))$ follows from $K(1)$ and $Z(0) = 1$, and its successor premise $K(Z(Sz))$ from $K(0)$ and $Z(Sz) = 0$, by Theorem 8.4(ii). Then instantiate w by r. □

Define the parity terms

$$U(n) := R(n, 0; u.Z(u)), \qquad N(n) := Z(U(n)).$$

Lemma 8.6 (Parity and the Boolean decoder)**.** N_R *derives*

$$Z(Z(Z(r))) = Z(r), \qquad N(0) = 0, \qquad N(Sn) = Z(N(n)), \qquad N(SSn) = N(n),$$
$$N(Z(r)) = Z(r), \qquad N(SSZ(r)) = Z(r).$$

Proof. The first equation is Theorem 8.5 for the template $Z(Z(w)) = w$: its instances are $Z(Z(1)) = Z(0) = 1$ and $Z(Z(0)) = Z(1) = 0$. By (8.1), $U(0) = 1$, so $N(0) = Z(1) = 0$. By (8.2), $U(Sn) = Z(U(n))$, so $N(Sn) = Z(Z(U(n))) = Z(N(n))$, and $N(SSn) = Z(Z(Z(U(n)))) = Z(U(n)) = N(n)$. In particular $N(1) = Z(0) = 1$. The equation $N(Z(r)) = Z(r)$ is Theorem 8.5 for the template $N(w) = w$, with instances $N(1) = 1$ and $N(0) = 0$. The last equation follows from the fourth and the fifth. □

Definition 8.7. A term b is *Boolean in* Γ if $\Gamma \vdash b = Z(r)$ for some term r. The numerals 0 and 1 are Boolean in every context, since $Z(1) = 0$ and $Z(0) = 1$; so is every term $N(n)$.

If b and c are Boolean in Γ, then $\Gamma, Sb = Sc \vdash b = c$: apply the template $w \mapsto N(Sw)$ to the hypothesis and use the last equation of Theorem 8.6 on both sides. This cancels a successor on Boolean terms only; it is not injectivity of S.

Lemma 8.8 (Restricted implication)**.** *Let b, c be Boolean in Γ, let $s = t$ be an equation, and put*

$$\mathcal{I}(b = c;\ s = t) := D_{x,y}(\mathrm{sel}^+(y, b, s) = \mathrm{sel}^+(y, c, t))$$

with x, y fresh. Then

(i) if $\Gamma, b = c \vdash s = t$, then $\Gamma \vdash \mathcal{I}(b = c;\ s = t)$;

(ii) if $\Gamma \vdash \mathcal{I}(b = c;\ s = t)$ and $\Gamma \vdash b = c$, then $\Gamma \vdash s = t$.

Proof. (i) Use DI by cases with witness 0. The local assumption is the matrix at $y = 0$, which by (8.3) is $Sb = Sc$; the decoder gives $b = c$, and cutting in the given derivation gives $s = t$. By (8.3) this is the matrix at $y = Sz$.

(ii) Rule DE opens $Q_y(\mathrm{sel}^+(y, b, s) = \mathrm{sel}^+(y, c, t))$; the opened parameter does not occur. Its zero instance is $Sb = Sc$, which follows from $b = c$ by congruence. Rule QE at the term 1 gives $\mathrm{sel}^+(1, b, s) = \mathrm{sel}^+(1, c, t)$, that is $s = t$. □

Lemma 8.9 (Contextual Boolean cases)**.** *Let b be Boolean in Γ and let E be a literal equation. If $\Gamma, b = 0 \vdash E$ and $\Gamma, b = 1 \vdash E$, then $\Gamma \vdash E$. The context and E may mention b.*

Proof. Let $\Gamma \vdash b = Z(r)$ and let w be a variable not in Γ, b, r, E. Apply NC to the native formula $B(w) := \mathcal{I}(b = Z(w); E)$. Its zero premise is Theorem 8.8(i) applied to $\Gamma, b = Z(0) \vdash E$, which is the second given branch because $Z(0) = 1$. Its successor premise is Theorem 8.8(i) applied to $\Gamma, b = Z(Sz) \vdash E$, which is the first given branch because $Z(Sz) = 0$. Rule NC gives $\Gamma \vdash B(r)$, and Theorem 8.8(ii) with $\Gamma \vdash b = Z(r)$ gives E. The context does not change during the argument; the variable that varies is the fresh w. □

A branch whose hypotheses yield $0 = S0$ is closed by the last statement of Theorem 8.2.

Remark 8.10. Theorems 8.6, 8.8 and 8.9 are the native bootstrap of `transient_seed`. `successor_seed_compl`
proves the same case rule for terms with values among $0, 1, 2$: a term b has the *functional range rule* for a finite list V of numerals if $\Gamma \vdash K(r)$ for all $r \in V$ implies $\Gamma \vdash K(b)$; successor is cancelled on such terms by applying the three-cycle M of Theorem 8.12 and then j twice, using $M(w) = w$ and $j(j(j(w))) = w$ on $0, 1, 2$; and the guard formula is $D_{a,y}(\mathrm{sel}^+(y, x, u) = \mathrm{sel}^+(y, b, v))$ evaluated at $x = r$ for each $r \in V$. The predecessor proof below needs the two-valued case only. `scalar_successor` states its finite case analyses as abbreviations for repeated NC and substitution; inside an induction step they are instances of Theorem 8.9.

8.1.4 The predecessor

All statements of this subsection are derivable in N_R, by computation, the equality laws, NC, EI for the displayed literal equation with its other parameters fixed, and Theorem 8.9. They are therefore derivable in T_R. No step cancels a successor or uses a selector with unshifted branches. The plan is that of Theorem 1.65: parity, a three-cycle, a ramp, a factor map, a comparator and a saturating counter. The steps that Theorem 1.65 takes by equational interpolation and the two-point rule (Theorem 1.58(iv), (v)) need the selector sel with unshifted branches; here they are replaced by the contextual cases of Theorem 8.9, which split a Boolean value under the induction hypothesis, and every term is adjusted to the seed Sa. The conjugacy and non-saturation inductions of Theorem 8.17 and the last comparator equation have the shape of the corresponding steps of Theorems 1.64 and 1.65, with the shifted values.

Lemma 8.11 (Shifted sum)**.** *Let $a \oplus b := R(b, a; u.Su)$. Then $a \oplus 0 = Sa$, $a \oplus Sb = S(a \oplus b)$, and*

$$0 \oplus a = Sa, \qquad Sa \oplus b = S(a \oplus b), \qquad a \oplus b = b \oplus a, \qquad (a \oplus b) \oplus c = a \oplus (b \oplus c).$$

Proof. The first two equations are (8.1) and (8.2). For $0 \oplus a = Sa$ use EI on a: $0 \oplus 0 = 1$ and $0 \oplus Sa = S(0 \oplus a) = SSa$. For $Sa \oplus b = S(a \oplus b)$ use EI on b: $Sa \oplus 0 = SSa = S(a \oplus 0)$ and $Sa \oplus Sb = S(Sa \oplus b) = SS(a \oplus b) = S(a \oplus Sb)$. For commutativity use EI on b: $a \oplus 0 = Sa = 0 \oplus a$ and $a \oplus Sb = S(a \oplus b) = S(b \oplus a) = Sb \oplus a$. For associativity use EI on c: both sides are $S(a \oplus b)$ at 0, and $(a \oplus b) \oplus Sc = S((a \oplus b) \oplus c) = S(a \oplus (b \oplus c)) = a \oplus (b \oplus Sc)$. □

In the standard model $a \oplus b = a + b + 1$. Define the affine terms

$$A_2(a) := S^2(a \oplus a), \qquad A_3(a) := S^4((a \oplus a) \oplus a), \qquad A_2^\circ(a) := S^3(a \oplus a),$$

with standard values $2a + 3$, $3a + 6$ and $2a + 4$. Theorem 8.11 gives

$$A_2(0) = 3, \qquad A_2(Sa) = S^2 A_2(a), \qquad A_3(0) = 6, \qquad A_3(Sa) = S^3 A_3(a). \tag{8.4}$$

Lemma 8.12 (A three-cycle)**.** *Let*

$$j(u) := \mathrm{sel}^+(u, 0, \mathrm{sel}^+(Z(N(u)), 1, 0)), \qquad X(n) := R(n, 0; u.j(u)), \qquad M(n) := j(j(X(n))).$$

Then $j(0) = 1$, $j(1) = 2$, $j(2) = 0$, $M(0) = 0$ and $M(Sn) = j(M(n))$.

Proof. By (8.3), $j(0) = S0$ and $j(Sz) = \mathrm{sel}^+(Z(N(Sz)), 1, 0)$. Since $N(1) = 1$ and $N(2) = N(0) = 0$, this gives $j(1) = \mathrm{sel}^+(0, 1, 0) = 2$ and $j(2) = \mathrm{sel}^+(1, 1, 0) = 0$. Further $X(0) = 1$ and $X(Sn) = j(X(n))$, so $M(0) = j(j(1)) = 0$ and $M(Sn) = j(j(j(X(n)))) = j(M(n))$. □

Lemma 8.13 (The ramp)**.** *Let* $G(n) := R(n, 0; u.\, u \oplus M(S^2 u))$. *Then*

$$M(G(n)) = S(N(n)), \tag{8.5}$$

$$N(n) = 0 \vdash G(Sn) = SG(n), \qquad N(n) = 1 \vdash G(Sn) = S^2 G(n), \tag{8.6}$$

$$G(S^2 n) = S^3 G(n), \tag{8.7}$$

$$N(n) = 0 \vdash G(S^3 n) = S^4 G(n), \qquad N(n) = 1 \vdash G(S^3 n) = S^5 G(n). \tag{8.8}$$

Proof. By the computation equations, $G(0) = 1$ and $G(Sn) = G(n) \oplus M(S^2 G(n)) = G(n) \oplus j(j(M(G(n))))$.

Equation (8.5) is proved by EI on n. The base is $M(1) = j(M(0)) = j(0) = 1 = S(N(0))$. For the step keep the induction hypothesis $M(G(n)) = S(N(n))$ in the context and apply Theorem 8.9 to the Boolean term $N(n)$. If $N(n) = 0$, then $M(G(n)) = 1$, $j(j(1)) = 0$, so $G(Sn) = G(n) \oplus 0 = SG(n)$ and $M(G(Sn)) = j(M(G(n))) = j(1) = 2 = S(Z(0)) = S(N(Sn))$. If $N(n) = 1$, then $M(G(n)) = 2$, $j(j(2)) = 1$, so $G(Sn) = G(n) \oplus 1 = S^2 G(n)$ and $M(G(Sn)) = j(j(M(G(n)))) = j(j(2)) = 1 = S(Z(1)) = S(N(Sn))$.

The two computations of $G(Sn)$ just made, now with (8.5) as a theorem, are (8.6). For (8.7) apply Theorem 8.9 to $N(n)$: if $N(n) = 0$ then $N(Sn) = 1$ and $G(S^2 n) = S^2 G(Sn) = S^3 G(n)$; if $N(n) = 1$ then $N(Sn) = 0$ and $G(S^2 n) = SG(Sn) = S^3 G(n)$. Finally $G(S^3 n) = S^3 G(Sn)$ by (8.7), and (8.6) gives (8.8). □

From $G(0) = 1$, $N(0) = 0$ and (8.6), (8.7) one computes $G(1) = 2$, $G(2) = 4$, $G(3) = 5$, $G(4) = 7$, $G(6) = 10$. In the standard model $G(n) = \lfloor 3n/2 \rfloor + 1$.

Lemma 8.14 (Affine identities)**.**

$$N(A_2(a)) = 1, \qquad N(A_3(a)) = N(a), \qquad SG(A_2(a)) = A_3(a),$$
$$G(A_3(a)) = A_3(G(a)) \oplus N(a).$$

Proof. Each equation is proved by EI on a with (8.4).

$N(A_2(0)) = N(3) = N(1) = 1$, and $N(A_2(Sa)) = N(S^2 A_2(a)) = N(A_2(a))$.

$N(A_3(0)) = N(6) = N(0)$, and $N(A_3(Sa)) = N(S^3 A_3(a)) = Z(N(A_3(a))) = Z(N(a)) = N(Sa)$.

$SG(A_2(0)) = SG(3) = 6 = A_3(0)$, and $SG(A_2(Sa)) = SG(S^2 A_2(a)) = S^3 SG(A_2(a)) = S^3 A_3(a) = A_3(Sa)$ by (8.7).

For the last equation the base is $G(A_3(0)) = G(6) = 10$ and $A_3(G(0)) \oplus N(0) = A_3(1) \oplus 0 = S9 = 10$. For the step keep the induction hypothesis $G(A_3(a)) = A_3(G(a)) \oplus N(a)$ and apply Theorem 8.9 to $N(a)$; the left side of the goal is $G(S^3 A_3(a))$ and $N(A_3(a)) = N(a)$.

If $N(a) = 0$, then by (8.8) and the hypothesis

$$G(S^3 A_3(a)) = S^4 G(A_3(a)) = S^4(A_3(G(a)) \oplus 0) = S^5 A_3(G(a)),$$

while $G(Sa) = SG(a)$, $N(Sa) = 1$ and

$$A_3(G(Sa)) \oplus N(Sa) = S^3 A_3(G(a)) \oplus 1 = S^5 A_3(G(a)).$$

If $N(a) = 1$, then

$$G(S^3 A_3(a)) = S^5 G(A_3(a)) = S^5(A_3(G(a)) \oplus 1) = S^7 A_3(G(a)),$$

while $G(Sa) = S^2 G(a)$, $N(Sa) = 0$ and

$$A_3(G(Sa)) \oplus N(Sa) = S^6 A_3(G(a)) \oplus 0 = S^7 A_3(G(a)).$$ □

Lemma 8.15 (The factor map)**.** *Let* $H(u) := \mathrm{sel}^+(Z(N(u)), G(u), u)$*. Then*

$$N(u) = 0 \vdash H(u) = u, \qquad N(u) = 1 \vdash H(u) = SG(u),$$
$$H(2) = 2, \qquad H(A_2(a)) = A_3(a), \qquad H(A_3(a)) = A_3(H(a)).$$

Proof. The first line is (8.3) with $Z(0) = 1$ and $Z(1) = 0$. Since $N(2) = 0$, $H(2) = 2$. Since $N(A_2(a)) = 1$, $H(A_2(a)) = SG(A_2(a)) = A_3(a)$. For the last equation apply Theorem 8.9 to $N(a)$, using $N(A_3(a)) = N(a)$. If $N(a) = 0$, both sides are $A_3(a)$. If $N(a) = 1$, then by Theorem 8.14

$$H(A_3(a)) = SG(A_3(a)) = S(A_3(G(a)) \oplus 1) = S^3 A_3(G(a)) = A_3(SG(a)) = A_3(H(a)). \quad \square$$

On standard numbers H fixes even inputs and sends an odd u to $3(u+1)/2$. In the coordinate $u+3$ the terms A_2 and A_3 are doubling and tripling, and an active step of H multiplies by $3/2$. This explains the comparator below; the proof does not use it.

Lemma 8.16 (A comparator)**.** *Let*

$$\alpha(x) := R(x, 0; u.A_2^\circ(u)), \qquad e(x) := S\alpha(x), \qquad \Phi(x,y) := R(y, \alpha(x); u.H(u)), \qquad L(x,y) := Z(N(\Phi(x,y)))$$

Then $L(x,y)$ *is Boolean, and*

$$L(0,y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x,y), \qquad L(Sn, n) = 0.$$

Proof. By computation and Theorem 8.11, $e(0) = S\alpha(0) = 2$ and

$$e(Sx) = SA_2^\circ(\alpha(x)) = S^4(\alpha(x) \oplus \alpha(x)) = S^2(S\alpha(x) \oplus S\alpha(x)) = A_2(e(x)).$$

Also $\Phi(x,0) = S\alpha(x) = e(x)$ by (8.1), and $\Phi(x, Sy) = H(\Phi(x,y))$. The initializer of Φ is chosen so that the successor-seeded initial value is $e(x)$; no predecessor is needed to start the orbit there.

By EI on y, $\Phi(0,y) = 2$: the base is $e(0) = 2$ and the step is $H(2) = 2$. Hence $L(0,y) = Z(N(2)) = Z(0) = 1$. Next $\Phi(Sx, 0) = A_2(e(x))$ has N-value 1, so $L(Sx, 0) = Z(1) = 0$.

By EI on y with x fixed, $\Phi(Sx, Sy) = A_3(\Phi(x,y))$. At $y = 0$ the left side is $H(e(Sx)) = H(A_2(e(x))) = A_3(e(x)) = A_3(\Phi(x,0))$. In the step both orbits apply H, and $H(A_3(v)) = A_3(H(v))$ turns $H(A_3(\Phi(x,y)))$ into $A_3(\Phi(x, Sy))$. Applying $Z \circ N$ and $N(A_3(v)) = N(v)$ gives $L(Sx, Sy) = L(x,y)$.

Finally $L(Sn, n) = 0$ by EI on n from the second and third equations, as in Theorem 1.64. $\square$

The standard value of $L(x,y)$ is 1 if $x \le y$ and 0 otherwise.

Lemma 8.17 (Saturation)**.** *Let*

$$K(c,u) := \mathrm{sel}^+(c,u,u), \qquad \beta_m(u) := K(Z(L(Su, m)), u), \qquad V(t,m) := R(t, 0; u.\beta_m(u)).$$

Then

$$V(St, S^2 k) = SV(t, Sk), \qquad V(n, Sn) = Sn, \qquad V(Sn, Sn) = V(S^2 n, Sn) = Sn.$$

Proof. $K(0,u) = Su$ and $K(Sv, u) = u$, so NC on c gives $K(c, Su) = SK(c,u)$. With the comparator equations,

$$\beta_{Sm}(Su) = K(Z(L(Su,m)), Su) = S\beta_m(u), \qquad \beta_{S^2 k}(1) = K(Z(L(0,k)), 1) = K(0,1) = 2.$$

The first equation of the lemma is EI on t with k fixed. At $t = 0$, $V(1, S^2k) = \beta_{S^2k}(V(0, S^2k)) = \beta_{S^2k}(1) = 2 = SV(0, Sk)$. In the step, $V(S^2t, S^2k) = \beta_{S^2k}(SV(t, Sk)) = S\beta_{Sk}(V(t, Sk)) = SV(St, Sk)$.

The second equation is a separate EI on n: $V(0, 1) = 1$, and $V(Sn, S^2n) = SV(n, Sn)$ by the first equation at $t = k = n$, an instance of a completed lemma. For the third,

$$\beta_{Sn}(Sn) = K(Z(L(S^2n, Sn)), Sn) = K(Z(L(Sn, n)), Sn) = K(1, Sn) = Sn,$$

so one and then two further steps of the recursor from $V(n, Sn) = Sn$ leave the value unchanged. □

Lemma 8.18 (A capped predecessor)**.** *Let* $Q(m) := R(m, 0; u.K(Z(L(S^2u, m)), u))$. *Then* $Q(0) = Q(1) = 1$ *and* $Q(S^2k) = Sk$.

Proof. $Q(0) = 1$ by (8.1), and $Q(1) = K(Z(L(3, 1)), 1) = K(Z(L(2, 0)), 1) = K(1, 1) = 1$. For $m = S^2k$ the comparator equations give $L(S^2u, S^2k) = L(u, k) = L(Su, Sk)$, so the body of Q at this parameter equals $\beta_{Sk}(u)$ for a generic u. By Theorem 8.3, at an independent counter t with k fixed,

$$R(t, 0; u.K(Z(L(S^2u, S^2k)), u)) = V(t, Sk).$$

Substituting $t = S^2k$ and Theorem 8.17 give $Q(S^2k) = V(S^2k, Sk) = Sk$. □

Theorem 8.19 (Predecessor)**.** *Let*

$$P(m) := \mathrm{sel}^+(Z(L(3, m)),\ Q(Q(m)),\ L(2, m)).$$

N_R *and* T_R *derive*

$$P(0) = 0, \qquad P(Sn) = n.$$

Consequently both derive $Sx = Sy \vdash x = y$.

Proof. For $m \in \{0, 1, 2\}$ the comparator equations give $L(3, m) = 0$, so $Z(L(3, m)) = 1$ and $P(m) = L(2, m)$ by (8.3); and $L(2, 0) = 0$, $L(2, 1) = L(1, 0) = 0$, $L(2, 2) = L(0, 0) = 1$. Hence $P(0) = 0$, $P(1) = 0$ and $P(2) = 1$. For $m = S^3k$, $L(3, m) = L(0, k) = 1$, so the zero branch of sel^+ applies and, by Theorem 8.18,

$$P(S^3k) = SQ(Q(S^3k)) = SQ(S^2k) = S^2k.$$

Rule NC on n, and in its successor case NC once more, give $P(Sn) = n$ from $P(1) = 0$, $P(2) = 1$ and $P(S^3k) = S^2k$. Injectivity follows by applying P to $Sx = Sy$. □

Remark 8.20 (The three constructions). The construction above is that of `successor_seed_completion`: the orbit G is started at the seed 1 that (8.1) provides, the affine terms absorb the shift, and the capped predecessor Q is corrected at three small arguments. The two other packages reach Theorem 8.19 by different terms.

`scalar_successor` first builds a term τ with $\tau(0) = 1$, $\tau(1) = 0$, $\tau(S^2n) = S^2n$ and $\tau(\tau(n)) = n$, from the parity term U, a Boolean conjunction $Z(\mathrm{sel}^+(a, 0, Z(b)))$ and two finite orbits. Conjugation gives an iterator that starts at zero,

$$K_0(n; u.h) := \tau\big(R(n, 0; v.\tau(h[\tau(v)/u]))\big), \qquad K_0(0; u.h) = 0, \quad K_0(Sn; u.h) = h[K_0(n; u.h)/u],$$

for every body h. With the shifted sum $a \oplus b$, the terms $a \oplus a$ and $(a \oplus a) \oplus a$, a ramp $F(n) := K_0(n; u.\, u \oplus M'(u))$ over a three-state orbit M' with the invariant $M'(F(n)) = N(n)$, and a factor map, it obtains a strict comparator V with $V(x, 0) = 0$, $V(0, Sy) =$

1, $V(Sx,Sy) = V(x,y)$. The predecessor is then the saturated zero-start orbit $P(m) := K_0(m; u.\,\mathrm{sel}^+(Z(V(Su,m)),u,u))$, with no correction.

`transient_seed` works in T_R and observes that $f(R(n,a;u.f))$ starts at $f(Sa)$: a seed Sa outside an intended orbit is used once to enter it. Doubling and tripling orbits $d_2(Sn) = S^2 d_2(n)$, $d_3(Sn) = S^3 d_3(n)$ that start at zero are obtained in this way, then a ramp and a factor map, and a comparator L with the equations of Theorem 8.16 and the persistence law $L(x,y) = 1 \to L(x,Sy) = 1$. The predecessor is a reset cycle: with $F_m(u) := \mathrm{sel}^+(Z(L(Su,m)),u,0)$, which increments below m and resets to 0 from m on, and

$$J_m(t) := F_m\big(R(t,m;u.F_m(u))\big), \qquad K_m(t) := F_m\big(R(t,m;u.F_m(F_m(u)))\big),$$

one has $J_m(0) = K_m(0) = 0$ because $F_m(Sm) = 0$. Induction on the literal equation $\mathrm{sel}^+(L(t,m),0,J_m(t)) = \mathrm{sel}^+(L(t,m),0,t)$ gives $L(t,m) = 1 \to J_m(t) = t$; two further inductions give the period $J_m(m \oplus t) = J_m(t)$ and $K_m(Sa) = J_m(a \oplus Sa)$; and $P(n) := K_n(n)$ satisfies $P(Sa) = J_{Sa}(a) = a$. The package then transfers the proof to N_R by Theorem 8.9.

The three proofs use the same resources: literal-equation induction with fixed parameters, numerical cases, and Boolean cases in a context.

8.1.5 Recovery of the iterator

Theorem 8.21 (Initialized iteration)**.** *For every body h and a fresh variable v put*

$$\widehat{I}(n,a;u.h) := P\big(R(n,a;\, v.\,S(h[P(v)/u]))\big).$$

N_R and T_R derive

$$\widehat{I}(0,a;u.h) = a, \qquad \widehat{I}(Sn,a;u.h) = h[\widehat{I}(n,a;u.h)/u].$$

In particular $\widehat{I}(c,a;u.b)$ with $u \notin \mathrm{FV}(b)$ is a selector with the unshifted values a and b.

Proof. Write $r(n) := R(n,a;v.S(h[P(v)/u]))$. By (8.1) and Theorem 8.19, $\widehat{I}(0,a;u.h) = P(Sa) = a$. By (8.2), $r(Sn) = S(h[P(r(n))/u])$, so $\widehat{I}(Sn,a;u.h) = P(S(h[P(r(n))/u])) = h[P(r(n))/u] = h[\widehat{I}(n,a;u.h)/u]$. The equation $P(Sx) = x$ is used at arbitrary terms; no invariant about the state $r(n)$ is needed. □

Remark 8.22. The recursor, the candidate equations and the compiler $\widehat{I}$ are stated in an earlier draft of the series, which leaves the term P open; the three packages supply it. In the opposite direction the predecessor is the only obstacle: a successor-seeded recursor whose update also receives the old counter has an immediate predecessor (see Theorem 8.28 for the analogous computation).

8.1.6 Comparison of the two term languages

Definition 8.23. The translation $t \mapsto t^\iota$ from R-terms to the terms of Section 1.1 and the translation $t \mapsto t^\rho$ in the opposite direction are defined by

$$x^\iota := x, \qquad 0^\iota := 0, \qquad R(n,a;u.h)^\iota := I(n^\iota, S(a^\iota);\, u.\,h^\iota),$$
$$x^\rho := x, \qquad 0^\rho := 0, \qquad (St)^\rho := S(t^\rho), \qquad I(n,a;\, u.\,h)^\rho := \widehat{I}(n^\rho, a^\rho; u.h^\rho),$$

where $S(t^\rho)$ on the right is the abbreviation $R(0,t^\rho;u.0)$. Formulas, native or ordinary, are translated by translating their terms. Both translations commute with substitution and preserve free variables and the property of being a literal equation.

Note that $(Sa)^\iota = I(0,S(a^\iota);\,u.\,0)$, which is not literally $S(a^\iota)$ but equals it by the zero computation equation of the iterator.

Theorem 8.24 (Equivalence of the term languages)**.**

(a) If $\Gamma \vdash A$ is derivable in N_R, then $\Gamma^\iota \vdash A^\iota$ is derivable in N. If $\Gamma \vdash A$ is derivable in T_R, then $\Gamma^\iota \vdash A^\iota$ is derivable in T_{eq}.

(b) If $\Gamma \vdash A$ is derivable in N, then $\Gamma^\rho \vdash A^\rho$ is derivable in N_R. If $\Gamma \vdash A$ is derivable in T_{eq}, then $\Gamma^\rho \vdash A^\rho$ is derivable in T_R.

(c) N_R and T_R derive $t^{\iota\rho} = t$ for every R-term t; N and T_{eq} derive $t^{\rho\iota} = t$ for every iterator term t. Hence $A^{\iota\rho} \approx A$ in N_R and $A^{\rho\iota} \approx A$ in N for native formulas, and $A^{\iota\rho} \leftrightarrow A$, $A^{\rho\iota} \leftrightarrow A$ are provable in T_R, T_{eq} for ordinary formulas.

(d) Consequently $\Gamma \vdash A$ is derivable in N_R if and only if $\Gamma^\iota \vdash A^\iota$ is derivable in N, and likewise for T_R and T_{eq}, and for $(\cdot)^\rho$ in the other direction. The contradictions correspond: $(0 = S0)^\iota$ is interderivable with $0 = S0$, and $(0 = S0)^\rho$ is $0 = S0$.

Proof. (a) By induction on the derivation. Each rule instance other than a computation equation is translated into an instance of the same rule, because the translation commutes with substitution, keeps literal equations literal and keeps parameters fresh. One adjustment is needed where a rule displays a successor: the premise of DI by cases translates to $\Gamma^\iota, B^\iota(t^\iota, 0) \vdash B^\iota(t^\iota, I(0, Sz;\, u.\,0))$, and the premise of QI, or of the induction rule of T_R, to $\Gamma^\iota, E^\iota(z) \vdash E^\iota(I(0, Sz;\, u.\,0))$. Replacing $I(0, Sz;\, u.\,0)$ by Sz along the zero computation equation, by the Leibniz rule Theorem 1.22, gives the premise of the same rule in N or T_{eq}. The translation of (8.1) is $I(0, Sa^\iota;\, u.\,h^\iota) = I(0, Sa^\iota;\, u.\,0)$; both sides equal Sa^ι by the zero computation equation. The translation of (8.2) is

$$I(I(0, Sn^\iota;\, u.\,0), Sa^\iota;\, u.\,h^\iota) = h^\iota[I(n^\iota, Sa^\iota;\, u.\,h^\iota)/u],$$

which follows from $I(0, Sn^\iota;\, u.\,0) = Sn^\iota$, congruence and the successor computation equation. For T_R the logical rules are translated identically; coverage translates to $\forall x\,(x = 0 \vee \exists y\, x = I(0, Sy;\, u.\,0))$ and separation to $0 = I(0, S0;\, u.\,0) \to \bot$, both equivalent in T_{eq} to the corresponding axioms.

(b) By induction on the derivation. Here $(St)^\rho$ is literally $S(t^\rho)$, so every rule instance is translated into an instance of the same rule, and coverage, separation and induction are translated into themselves. The two computation equations of the iterator are translated into $\widehat{I}(0, a^\rho; u.h^\rho) = a^\rho$ and $\widehat{I}(Sn^\rho, a^\rho; u.h^\rho) = h^\rho[\widehat{I}(n^\rho, a^\rho; u.h^\rho)/u]$, which are instances of Theorem 8.21.

(c) First, $t^{\iota\rho} = t$ by induction on the R-term t. The cases of variables and 0 are trivial. For $t = R(n, a; u.h)$,

$$t^{\iota\rho} = P\Big(R(n^{\iota\rho}, S(a^{\iota\rho});\, v.\, S(h^{\iota\rho}[P(v)/u]))\Big).$$

By the induction hypothesis, congruence and Theorem 8.3 (for the body, whose induction hypothesis $h^{\iota\rho} = h$ is derivable in the empty context with u free), this equals $P\big(R(n, Sa; v.S(h[P(v)/u]))\big)$. The equation

$$P(R(z, Sa;\, v.\, S(h[P(v)/u]))) = R(z, a; u.h)$$

is proved by EI on z: at 0 the left side is $P(SSa) = Sa = R(0, a; u.h)$; at Sz it is $P(S(h[P(R(z, Sa; \ldots))/u])) = h[P(R(z, Sa; \ldots))/u]$, which by the induction hypothesis is $h[R(z, a; u.h)/u] = R(Sz, a; u.h)$.

Second, $t^{\rho\iota} = t$ by induction on the iterator term t. By (a) applied to Theorem 8.19, the term template $P^\iota(m) := (P(m))^\iota$ satisfies $P^\iota(0) = 0$ and $P^\iota(I(0, Sx;\, u.\,0)) = x$, hence $P^\iota(Sx) = x$, in N and T_{eq}. For $t = Ss$, $t^{\rho\iota} = I(0, S(s^{\rho\iota});\, u.\,0) = S(s^{\rho\iota}) = Ss$. For $t = I(n, a;\, u.\,h)$,

$$t^{\rho\iota} = P^\iota\Big(I(n^{\rho\iota}, S(a^{\rho\iota});\, v.\, I(0, S(h^{\rho\iota}[P^\iota(v)/u]);\, w.\,0))\Big),$$

which by the induction hypothesis, the zero computation equation and uniqueness of iteration for the iterator (Theorem 1.55, applied to the iteration with the first body as the template f) equals $P^\iota\big(I(n, Sa;\, v.\, S(h[P^\iota(v)/u]))\big)$. Induction for the literal equation $P^\iota\big(I(z, Sa;\, v.\, S(h[P^\iota(v)/u]))\big) =$

$I(z, a;\, u.\, h)$ proves the rest: at 0 both sides are a, and at Sz the left side is $h[P^\iota(I(z, Sa;\, v.\, \ldots))/u]$, which by the induction hypothesis is $h[I(z, a;\, u.\, h)/u] = I(Sz, a;\, u.\, h)$.

The statements on formulas follow from Theorem 8.4(i), in N_R and in N, and from replacement of provably equal terms under quantifiers in ordinary logic.

(d) If $\Gamma^\iota \vdash A^\iota$ is derivable in N, then $\Gamma^{\iota\rho} \vdash A^{\iota\rho}$ is derivable in N_R by (b), and $\Gamma \vdash A$ follows by (c) and cut. The other cases are the same. For the contradiction, $(0 = S0)^\iota$ is $0 = I(0, S0;\, u.\, 0)$. □

Corollary 8.25. *N_R and T_R are bi-interpretable with N, T_{eq} and iPRA: the interpretations are the translations of Theorem 8.23 composed with those of Theorem 1.119; the domains are the whole domain, equality is actual equality, each term is provably equal to its double translation, so that the graph $x = y$ is the isomorphism of both numerical composites, and every formula is provably equivalent to its double translation. The theories are equiconsistent. In iPRA the recursor with body h is the function defined by primitive recursion with initial value Sa and a step that ignores the counter.*

Proof. Theorem 8.24 gives the statement for N and T_{eq} with the identity as numerical map. Composition with Theorem 1.119 gives the rest; the description in iPRA is the composite of $(\cdot)^\iota$ with the interpretation of the iterator. □

Consequently every result that Theorems 1.65, 1.86 and 1.102 state for T_{eq} holds in T_R for the translated terms: reflection of equality into a term, induction for quantifier-free formulas, pairing, and a term with the defining equations of each primitive recursive description. The predecessor of Theorem 1.65 is not needed: Theorem 8.19 provides one directly.

Remark 8.26 (Rule forms in the packages). The three packages state the native calculus with two rule forms that differ from Section 1.3: transport is formula transport, from $s = t$ and $F(s)$ infer $F(t)$ for every native formula template F; and progress introduces the paired binder, from $\Gamma, E(t, z) \vdash E(t, Sz)$ infer $\Gamma \vdash D_{x,y}E(x, y)$ for a literal equation E. Both calculi derive the same sequents. Formula transport is Theorem 8.4(ii). The progress rule on $D_{x,y}$ is the witnessed introduction Theorem 1.16(i), derived from QI and Theorem 1.15. Conversely QI is derived from the progress rule on $D_{v,y}E(y)$ with v unused, followed by DE and HYP; and the transport rule of Section 1.3 is a case of formula transport combined with symmetry. `successor_seed_completion` and `scalar_successor` also describe a *computation-supply* presentation used by their checkers: instead of a computation leaf, a rule passes from $\Gamma, H \vdash A$ to $\Gamma \vdash A$ for a computation equation H. The two presentations derive the same sequents by cut in one direction and HYP in the other.

8.2 Tied-seed recursors

This section draws on `bounded_initializer`, `monus_and_neutral_major` and the obstruction section of `transient_seed`.

8.2.1 The recursor and its elementary terms

Definition 8.27. The tied-seed recursor $B(n; u, v.h)$ binds the distinct variables u (old counter) and v (old state) in h. Its computation equations are

$$B(0; u, v.h) = S(h[0/u][0/v]), \tag{8.9}$$

$$B(Sn; u, v.h) = h[n/u][B(n; u, v.h)/v]. \tag{8.10}$$

The same body determines the value at 0 and every later transition. Put

$$C(n, a) := B(n; u, v.a), \quad W(n) := B(n; u, v.u), \quad P(n) := W(C(n, n)), \quad Z(n) := C(n, 0),$$

with $u, v \notin \mathrm{FV}(a)$.

Lemma 8.28. *From* (8.9), (8.10) *and the equality laws alone,*

$$C(0,a) = Sa, \quad C(Sn,a) = a, \quad W(0) = 1, \quad W(Sn) = n, \quad P(0) = 0, \quad P(Sn) = n, \quad Z(0) = 1, \quad Z(Sn) = 0$$

Hence $Sx = Sy \vdash x = y$, *and* $Sx = 0 \vdash 0 = S0$.

Proof. The first four equations are instances of the computation equations. $P(0) = W(C(0,0)) = W(1) = 0$ and $P(Sn) = W(C(Sn,Sn)) = W(Sn) = n$. Applying P to $Sx = Sy$ gives $x = y$. From $Sx = 0$, applying P gives $x = 0$, and substitution gives $S0 = 0$. □

The successor can be removed from the primitive signature: $\sigma(a) := C(0,a)$ satisfies $\sigma(a) = Sa$, and writing σ for S in (8.9), (8.10) and in every axiom gives an equivalent presentation on the terms $x \mid 0 \mid B(n; u, v.h)$ and the other primitive operations. The constant-body instance of (8.9) becomes tautological; (8.10) for it and all other instances keep their content. The section keeps S for readability.

Lemma 8.29 (Gate and zero-start recursion)**.** *Assume induction for literal equations (rule* EI *of Theorem 8.2). Let*

$$G(n,a) := P\big(P(B(a; u, v.C(Z(n), v)))\big), \qquad B_0(n; u, v.h) := B\big(Sn;\, r, s.\, G(r, h[P(r)/u][s/v])\big).$$

Then

$$G(0,a) = 0, \qquad G(Sn,a) = a, \qquad B_0(0; u, v.h) = 0, \qquad B_0(Sn; u, v.h) = h[n/u][B_0(n; u, v.h)/v].$$

Proof. For the control 0 the body of the inner recursor is $C(1,v) = v$, its value at 0 is $S(C(1,0)) = 1$, and EI on a shows that the value at a is 1; so $G(0,a) = P(P(1)) = 0$. For the control Sn the body is $C(0,v) = Sv$, the value at 0 is $S(S0) = 2$, and EI on a shows that the value at a is SSa; so $G(Sn,a) = P(P(SSa)) = a$. In both inductions n is a fixed parameter.

Let $k(r,s) := G(r, h[P(r)/u][s/v])$, so that $k(0,s) = 0$ and $k(Sr,s) = h[r/u][s/v]$. Then $B(0; r, s.k) = S(k(0,0)) = 1$, $B(1; r, s.k) = k(0,1) = 0$, and $B(SSn; r, s.k) = k(Sn, B(Sn; r, s.k)) = h[n/u][B(Sn; r, s.k)/v]$. These are the equations for B_0. □

The gate G is a one-sided selector: it returns 0 or its second argument. The term $P(P(B(a; u, v.C(n,v))))$ is the complementary mask, with values a at $n = 0$ and 0 at $n = Sm$.

8.2.2 The one-coordinate bound

The independent initializer cannot be recovered by a term. A function has a *bounding coordinate* in the sense of Theorem 10.29: some argument x_i and a nondecreasing unary primitive recursive F with $f(\vec{x}) \le F(x_i)$; the general obstruction for tied seeds is Theorem 10.41, and its two generalizations are Theorems 10.36 and 10.39, both instances of the barrier for adaptive query programs, Theorem 10.33. The case of this chapter is the following instance.

Corollary 8.30 (One-coordinate bound)**.** *Consider the terms built from variables,* 0, *S, B, and optionally a binary* ε *with values in* $\{0,1\}$ *and a binary* d *with* $d(x,y) \le x$, *in the standard model with* (8.9), (8.10). *The function denoted by a term is bounded by a constant or has a bounding coordinate among its free variables, with a nondecreasing primitive recursive bound that is obtained by recursion on the term.*

Proof. The recursor B is the constructor B of the tied-seed language of Theorem 10.3(b), with the same computation equations (10.1), (10.2). The successor is a unary primitive recursive operation; ε is globally bounded by 1; and d has the bounding coordinate x with the bound

$F(z) = z$. By Theorem 10.41 every term denotes a function that is globally bounded or has a bounding coordinate. The proof of that corollary, a recursion on the term with the four cases of a constant body and of a passive, counter or state coordinate of the body, composed with the bound of the counter term by Theorem 10.30(iii), constructs the nondecreasing primitive recursive bound. □

Corollary 8.31. *No term of these languages denotes, in the standard model, any of the following functions: the selector* $\mathrm{sel}(n, a, b)$ *with* $\mathrm{sel}(0, a, b) = a$ *and* $\mathrm{sel}(n+1, a, b) = b$*; addition; an injection* $\mathbb{N}^2 \to \mathbb{N}$.

Proof. By Theorem 8.30 every such function would be globally bounded or have a bounding coordinate, and the argument of Theorem 10.42 shows that none of the three has either property. □

Remark 8.32. The bound is in `transient_seed` for the terms $x \mid 0 \mid B$, and in `bounded_initializer` with S and ε; the case of d is immediate from $d(x, y) \leq x$. The recursor $J(0; h) = 1$, $J(Sn; h) = h(n, J(n; h))$, which `scalar_successor` mentions as the reason for keeping the initializer in R, is a single-body constructor scheme of Theorem 10.35 (with the constant seed combiner 1), so Theorem 10.36 gives the same bound for it. The bound concerns the functions denoted by terms. Sound logical rules cannot change it, so there is no compiler from the iterator language to these terms. It does not show that a theory on these terms fails to be bi-interpretable with iPRA: Theorem 8.43 interprets each primitive recursive function by a graph with finitely many value terms. The successor-seeded recursor $R(n, a; u.h)$ escapes the bound because $a \oplus b$ is unbounded in each of its two arguments. Related obstructions for tied seeds are in the package `local_scopes` and in Theorems 10.36 and 10.39.

8.2.3 The theory with a Boolean equality test

Definition 8.33. $\mathsf{T}_{B\varepsilon}$ is ordinary intuitionistic first-order logic with equality over the terms $t ::= x \mid 0 \mid St \mid \varepsilon(t, t) \mid B(n; u, v.h)$, with the axioms (8.9), (8.10),

$$\varepsilon(x, y) = 0 \vee \varepsilon(x, y) = 1, \tag{8.11}$$

$$\varepsilon(x, y) = 1 \leftrightarrow x = y, \tag{8.12}$$

$$0 = 1 \to \bot, \tag{8.13}$$

universally closed, and induction for literal equations with side assumptions. Coverage, injectivity of the successor and induction for compound formulas are not axioms.

The equality on the right of (8.12) is the equality of the logic on the whole domain. The axioms make numerical equality decidable and give $\varepsilon(s, t) = 0 \leftrightarrow s \neq t$; they do not give excluded middle for quantified formulas.

Lemma 8.34 (Quantifier-free induction and coverage)**.** *In* $\mathsf{T}_{B\varepsilon}$ *every quantifier-free formula* A *has a term* χ_A *with* $\chi_A = 0 \vee \chi_A = 1$ *and* $A \leftrightarrow \chi_A = 1$*. Induction with side assumptions holds for quantifier-free formulas, and* $x = 0 \vee x = S(P(x))$ *is provable.*

Proof. By recursion on A: $\chi_{s=t} := \varepsilon(s, t)$, $\chi_\bot := 0$, and on Boolean values Z is negation and G is conjunction by Theorems 8.28 and 8.29; disjunction is $Z(G(Z(b), Z(c)))$ and implication is $Z(G(b, Z(c)))$. The truth tables are checked on the two values. The clause for implication is valid intuitionistically because A is decidable by its characteristic term. Given a base and a step for $A(n)$, pass to the literal equation $\chi_{A(n)} = 1$, apply the induction rule and pass back. Apply the result to $x = 0 \vee x = S(P(x))$: the base is immediate, and the step holds by $P(Sx) = x$ without the induction hypothesis. □

For a Boolean b let $\mathrm{inc}(a,b) := C(Z(b),a)$, so that $\mathrm{inc}(a,0) = a$ and $\mathrm{inc}(a,1) = Sa$. Reflection and Theorem 8.28 give

$$\varepsilon(x,x) = 1, \qquad \varepsilon(x,y) = \varepsilon(y,x), \qquad \varepsilon(Sx,Sy) = \varepsilon(x,y), \qquad \varepsilon(0,Sx) = 0.$$

Define the *hit term* and the order relations

$$\mathrm{hit}(n,y) := B_0(n; r, v.\, \mathrm{inc}(v, \varepsilon(r,y))), \qquad x < y :\Leftrightarrow \mathrm{hit}(y,x) = 1, \qquad x \le y :\Leftrightarrow \mathrm{hit}(Sy,x) = 1.$$

Thus $\mathrm{hit}(0,y) = 0$ and $\mathrm{hit}(Sn,y) = \mathrm{inc}(\mathrm{hit}(n,y), \varepsilon(n,y))$.

Lemma 8.35 (Hit identities)**.** $\mathsf{T}_{B\varepsilon}$ *proves*

$$\mathrm{hit}(Sn,0) = 1, \quad \mathrm{hit}(Sn,Sy) = \mathrm{hit}(n,y), \quad \mathrm{hit}(n,n) = 0, \quad \mathrm{hit}(Sn,n) = 1,$$
$$\mathrm{hit}(n,y) = 0 \vee \mathrm{hit}(n,y) = 1, \qquad \mathrm{hit}(Sy,x) = Z(\mathrm{hit}(x,y)),$$
$$\mathrm{hit}(n,y) = 1 \to \mathrm{hit}(Sn,y) = 1, \qquad \mathrm{hit}(Sn,y) = 1 \to \mathrm{hit}(n,y) = 1 \vee n = y.$$

Proof. $\mathrm{hit}(Sn,0) = 1$ by induction on n: the first step tests $\varepsilon(0,0) = 1$, and every later test $\varepsilon(Sn,0)$ is 0. $\mathrm{hit}(Sn,Sy) = \mathrm{hit}(n,y)$ by induction on n with y fixed: the base is $\mathrm{inc}(0,\varepsilon(0,Sy)) = 0$, and the step uses $\varepsilon(Sn,Sy) = \varepsilon(n,y)$. Then $\mathrm{hit}(n,n) = 0$ by induction, and $\mathrm{hit}(Sn,n) = \mathrm{inc}(0,1) = 1$.

For the range use quantifier-free induction on n: if $\varepsilon(n,y) = 0$ the state is unchanged; if $\varepsilon(n,y) = 1$ then $n = y$, the old state is $\mathrm{hit}(n,n) = 0$ and the new state is 1. If $\mathrm{hit}(n,y) = 1$, then $n \neq y$ by the diagonal value, so the next test is 0 and the hit persists. The same case split gives the last implication.

The duality $\mathrm{hit}(Sy,x) = Z(\mathrm{hit}(x,y))$ is proved by induction on x with y fixed. At $x = 0$ both sides are 1. In the step the left side is $\mathrm{hit}(Sy,Sx) = \mathrm{hit}(y,x)$ and the right side is $Z(\mathrm{inc}(\mathrm{hit}(x,y),\varepsilon(x,y)))$. If $\varepsilon(x,y) = 0$, the right side is $Z(\mathrm{hit}(x,y))$, and the induction hypothesis reads $\mathrm{inc}(\mathrm{hit}(y,x),0) = \mathrm{hit}(y,x) = Z(\mathrm{hit}(x,y))$. If $\varepsilon(x,y) = 1$, then $x = y$, the left side is 0 and the right side is $Z(\mathrm{hit}(Sx,x)) = Z(1) = 0$. □

Lemma 8.36 (Order)**.** *In* $\mathsf{T}_{B\varepsilon}$ *the relations* $<$ *and* $\le$ *form a discrete linear order with least element* 0*:* $x \le y \leftrightarrow (x = y \vee x < y)$*,* $x < Sy \leftrightarrow x \le y$*, exactly one of* $x < y$*,* $x = y$*,* $y < x$ *holds, both relations are transitive, the successor preserves and reflects them, and* $x \le Sy$ *with* $x \neq Sy$ *implies* $x \le y$*.*

Proof. The diagonal identities give $x \le x$, $x \not< x$ and $x < Sx$. The recurrence for $\mathrm{hit}(Sy,x)$, split on $\varepsilon(y,x)$, gives the first equivalence. If $x \neq y$, then $\mathrm{hit}(Sy,x) = \mathrm{hit}(y,x)$, and by duality $\mathrm{hit}(x,y)$ is its Boolean complement; this is trichotomy. For transitivity of $<$ keep $\mathrm{hit}(y,x) = 1$ and induct on z in the quantifier-free formula $\mathrm{hit}(z,y) = 1 \to \mathrm{hit}(z,x) = 1$. The base has a false antecedent. At Sz the last implication of Theorem 8.35 gives $\mathrm{hit}(z,y) = 1$, and then the induction hypothesis and persistence apply, or $z = y$, and then the retained $\mathrm{hit}(y,x) = 1$ and persistence apply. Transitivity of $\le$ and the mixed laws follow by splitting equality from strict comparison, and antisymmetry from trichotomy. The identity $\mathrm{hit}(Sn,Sy) = \mathrm{hit}(n,y)$ gives preservation and reflection by the successor. $x < Sy \leftrightarrow x \le y$ holds by definition, and the last claim follows from it and the first equivalence. Finally $\mathrm{hit}(Sx,0) = 1$ is $0 \le x$, and $x \le 0$ gives $x = 0$ by trichotomy. □

Define the difference, the minimum, doubling and bounded addition by

$$\widehat{d}(x,y) := B_0(x; u, v.\, \mathrm{inc}(v, \mathrm{hit}(Su,y))), \qquad \mathrm{mn}(x,y) := \widehat{d}(x, \widehat{d}(x,y)), \qquad \mathrm{db}(n) := B_0(n; u, v.SSv),$$
$$K_b(a) := \widehat{d}(\mathrm{db}(b), \widehat{d}(b,a)), \qquad \mathrm{ad}_b(a,c) := \widehat{d}(K_b(a), \widehat{d}(b,c)).$$

Thus $\widehat{d}(0,y) = 0$ and $\widehat{d}(Sx,y) = \mathrm{inc}(\widehat{d}(x,y), \mathrm{hit}(Sx,y))$.

Lemma 8.37 (Difference)**.** $\mathsf{T}_{B\varepsilon}$ *proves*

$$\widehat{d}(x,0)=x,\qquad \widehat{d}(Sx,Sy)=\widehat{d}(x,y),\qquad \widehat{d}(x,x)=0,\qquad \widehat{d}(x,y)=0\leftrightarrow x\leq y,$$
$$\widehat{d}(x,Sy)=P(\widehat{d}(x,y)),\qquad \widehat{d}(x,y)\leq x,$$
$$y\leq x\to\widehat{d}(Sx,y)=S(\widehat{d}(x,y)),\qquad 0<y\leq x\to\widehat{d}(x,P(y))=S(\widehat{d}(x,y)),$$

and that $\widehat{d}$ is monotone in its first argument and antitone in its second.

Proof. $\widehat{d}(x,0)=x$ by induction, since $\mathrm{hit}(Sx,0)=1$. For $\widehat{d}(Sx,Sy)=\widehat{d}(x,y)$ induct on x with y fixed: the base uses $\mathrm{hit}(1,Sy)=0$, and the step uses $\mathrm{hit}(SSx,Sy)=\mathrm{hit}(Sx,y)$.

Quantifier-free induction on x with y fixed proves $\widehat{d}(x,y)=0\leftrightarrow\mathrm{hit}(x,y)=0$. In the step, if $\mathrm{hit}(Sx,y)=1$ the new difference is a successor; if $\mathrm{hit}(Sx,y)=0$, then $\mathrm{hit}(x,y)=0$ by persistence, the old difference is 0 and stays 0. By duality $\mathrm{hit}(x,y)=0$ is $\mathrm{hit}(Sy,x)=1$, which gives the characterization of $\leq$ and $\widehat{d}(x,x)=0$.

For $\widehat{d}(x,Sy)=P(\widehat{d}(x,y))$ induct on x with y fixed. In the step the left side is $\widehat{d}(x,y)$ and the right side is $P(\mathrm{inc}(\widehat{d}(x,y),\mathrm{hit}(Sx,y)))$. If the Boolean is 1 the predecessor cancels the successor; if it is 0 the characterization gives $\widehat{d}(Sx,y)=0$, hence $\widehat{d}(x,y)=0$ and both sides are 0.

Quantifier-free induction on x proves $\widehat{d}(x,y)\leq x$: the new state is the old one or its successor. For monotonicity in the first argument induct on v in $u\leq v\to\widehat{d}(u,z)\leq\widehat{d}(v,z)$ with u, z fixed: at Sv either $u=Sv$, or $u\leq v$ by discreteness and the new state is at least the old one; the base uses $u\leq 0\to u=0$. For antitonicity keep $y\leq z$ and induct on x in $\widehat{d}(x,z)\leq\widehat{d}(x,y)$: if $\mathrm{hit}(Sx,z)=0$ the left state stays and the right state does not decrease; if it is 1 then $z\leq x$, so $y\leq x$ and both states increase.

The first of the two final implications is the defining recurrence, since $y\leq x$ means $\mathrm{hit}(Sx,y)=1$. For the second, $y=S(P(y))$ by coverage, $P(y)<x$, so $\widehat{d}(x,P(y))\neq 0$, and coverage at this difference with $\widehat{d}(x,S(P(y)))=P(\widehat{d}(x,P(y)))$ gives the claim. □

Lemma 8.38 (Minimum, doubling, bounded addition)**.** $\mathsf{T}_{B\varepsilon}$ *proves:*

(a) $x\leq y\to\mathrm{mn}(x,y)=x$ *and* $y\leq x\to\mathrm{mn}(x,y)=y$*;*
(b) db *is monotone,* $n\leq\mathrm{db}(n)$ *and* $\widehat{d}(\mathrm{db}(n),n)=n$*;*
(c) $a\leq b\to\mathrm{ad}_b(a,0)=a$*;*
(d) $a\leq b\wedge Sc\leq b\to\mathrm{ad}_b(a,Sc)=S(\mathrm{ad}_b(a,c))$*;*
(e) $c\leq b\to\mathrm{ad}_b(0,c)=c$*;*
(f) $\mathrm{ad}_b(b,b)=\mathrm{db}(b)$ *and* $\mathrm{ad}_b(a,c)\leq\mathrm{db}(b)$ *for all* a*,* c*;*
(g) ad_b *is monotone in its second argument;*
(h) if $a,c\leq b$ *and* $a,c\leq b'$ *then* $\mathrm{ad}_b(a,c)=\mathrm{ad}_{b'}(a,c)$*.*

Proof. (a) The first claim uses $\widehat{d}(x,y)=0$ and $\widehat{d}(x,0)=x$. For the second induct on y in $y\leq x\to\widehat{d}(x,\widehat{d}(x,y))=y$ with x fixed. The base is $\widehat{d}(x,x)=0$. If $Sy\leq x$, then $y<x$ and $0<\widehat{d}(x,y)\leq x$, so by Theorem 8.37 $\widehat{d}(x,\widehat{d}(x,Sy))=\widehat{d}(x,P(\widehat{d}(x,y)))=S(\widehat{d}(x,\widehat{d}(x,y)))=Sy$.

(b) $\mathrm{db}(0)=0$ and $\mathrm{db}(Sn)=SS\,\mathrm{db}(n)$ give $n\leq\mathrm{db}(n)$ by quantifier-free induction; monotonicity is an induction on the larger argument that splits equality at the successor from the discrete predecessor case. The identity is an induction on a literal equation: $\widehat{d}(\mathrm{db}(Sn),Sn)=\widehat{d}(S\,\mathrm{db}(n),n)=S(\widehat{d}(\mathrm{db}(n),n))=Sn$, using $n\leq\mathrm{db}(n)$.

(c)–(h) Since $\widehat{d}(b,a)\leq b$, antitonicity and (b) give $b=\widehat{d}(\mathrm{db}(b),b)\leq K_b(a)\leq\mathrm{db}(b)$, and $K_b(0)=b$. If $Sa\leq b$, put $q:=\widehat{d}(b,a)$; then $0<q\leq b\leq\mathrm{db}(b)$ and $\widehat{d}(b,Sa)=P(q)$, so $K_b(Sa)=\widehat{d}(\mathrm{db}(b),P(q))=S(K_b(a))$. Quantifier-free induction on a proves (c) under $a\leq b$: the base is $\widehat{d}(b,b)=0$, and the step increments $\widehat{d}(K_b(a),b)$ by the last identity and $b\leq K_b(a)$. For (d) put $q:=\widehat{d}(b,c)$; then $0<q\leq b\leq K_b(a)$ and $\mathrm{ad}_b(a,Sc)=\widehat{d}(K_b(a),P(q))=S(\widehat{d}(K_b(a),q))$. Statement (e) is a quantifier-free induction on $c\leq b$ from (c) and (d). For (f) substitute $a=c=b$ and use $\widehat{d}(b,b)=0$ twice; the bound is $\widehat{d}(K_b(a),\widehat{d}(b,c))\leq K_b(a)\leq\mathrm{db}(b)$.

For (g), $c \le c'$ gives $\widehat{d}(b,c') \le \widehat{d}(b,c)$, and antitonicity of the outer difference gives the claim. For (h) keep $a \le b, b'$ and induct on c in $c \le \mathrm{mn}(b,b') \to \mathrm{ad}_b(a,c) = \mathrm{ad}_{b'}(a,c)$: the base gives a on both sides, and (d) applies to both sides whenever $Sc \le \mathrm{mn}(b,b')$. □

Every induction in Theorems 8.35 to 8.38 is on a quantifier-free formula with the other variables fixed. There is no total binary addition term; ad_b adds below the bound b.

Lemma 8.39 (Clipped initialized recursion)**.** *Let*

$$\mathrm{sel}_b(c,a,e) := \mathrm{ad}_b\big(G(Z(c),a), G(c,e)\big), \qquad \mathrm{Rec}_b(n,a;u,v.h) := B_0\Big(Sn;\, r, s.\ \mathrm{sel}_b\big(r, \mathrm{mn}(a,b), \mathrm{mn}(h[P(r)/u][s/$$

If $a, e \le b$, then $\mathrm{sel}_b(0,a,e) = a$ and $\mathrm{sel}_b(Sc,a,e) = e$. For all arguments,

$$\mathrm{Rec}_b(0,a;u,v.h) = \mathrm{mn}(a,b), \qquad \mathrm{Rec}_b(Sn,a;u,v.h) = \mathrm{mn}\big(h[n/u][\mathrm{Rec}_b(n,a;u,v.h)/v],\, b\big),$$

and $\mathrm{Rec}_b(n,a;u,v.h) \le b$.

Proof. By Theorem 8.29 one of the two summands of sel_b is 0 and the other is a at $c = 0$ and e at a successor; Theorem 8.38(c) and (e) give the values. The equations for Rec_b follow from those of B_0 and of sel_b, since both arguments $\mathrm{mn}(\cdot, b)$ are at most b; the bound is a quantifier-free induction. □

The cap b is a parameter that stays fixed during the recursion. Arbitrary initialization is thus available once a cap has been fixed, in agreement with Theorem 8.30: the value is bounded in the coordinate b.

8.2.4 A compiler with caps

Fix the primitive recursive descriptions built from zero, successor, projections, composition and primitive recursion. The following construction is a recursion on a description outside the theory; it produces terms and proofs of $\mathsf{T}_{B\varepsilon}$ and adds no evaluator for codes of descriptions.

Theorem 8.40 (Bounded realization)**.** *For every k-ary primitive recursive description f there are terms $\tau_f(b,x_1,\dots,x_k)$ and $M_f(b)$ such that $\mathsf{T}_{B\varepsilon}$ proves:*
(i) M_f is nondecreasing and $b \le M_f(b)$;
(ii) $\tau_f(b,\vec{x}) \le M_f(b)$ for all b, $\vec{x}$;
(iii) if $x_i \le b$ and $x_i \le c$ for all i, then $\tau_f(b,\vec{x}) = \tau_f(c,\vec{x})$.

Proof. By recursion on f.

Initial descriptions. $\tau_0 := 0$, $\tau_{\pi_i}(b,\vec{x}) := \mathrm{mn}(x_i,b)$, $\tau_S(b,x) := S\,\mathrm{mn}(x,b)$, with $M(b) := Sb$. The properties follow from Theorem 8.38(a).

Composition. Let $f(\vec{x}) = h(g_1(\vec{x}),\dots,g_m(\vec{x}))$. Put $Q := M_{g_1} \circ \cdots \circ M_{g_m}$ (the identity if $m = 0$), $M_f := M_h \circ Q$ and

$$\tau_f(b,\vec{x}) := \tau_h\big(Q(b), \tau_{g_1}(b,\vec{x}),\dots,\tau_{g_m}(b,\vec{x})\big).$$

Since the M_{g_i} are nondecreasing and $z \le M_{g_i}(z)$, $M_{g_i}(b) \le Q(b)$. (ii) is the bound for τ_h. For (iii) the inner values agree at the caps b and c by the hypothesis for the g_i, and they are bounded by $Q(b)$ and by $Q(c)$; (iii) for h gives the claim. (i) holds by composition.

Primitive recursion. Let $f(0,\vec{x}) = g(\vec{x})$ and $f(Sn,\vec{x}) = h(n, f(n,\vec{x}),\vec{x})$. Put

$$F := M_h \circ M_g \circ S, \qquad I_F(t) := B_0(t;u,v.F(v)), \qquad C_b := I_F(S\,\mathrm{db}(b)), \qquad M_f(b) := C_b.$$

F is nondecreasing with $Sz \le F(z)$ and $M_h(z) \le F(z)$. Then $I_F(0) = 0$, $I_F(St) = F(I_F(t))$, $t \le I_F(t)$ by induction, and I_F is nondecreasing by a quantifier-free induction that uses

discreteness of the order; so $b \mapsto C_b$ is nondecreasing and $b \le C_b$. For $j \le b$ define the stage bound

$$\kappa_b(j) := I_F(S\,\mathrm{ad}_b(b, j)).$$

Theorem 8.38 gives

$$b \le \kappa_b(0), \quad M_g(b) \le \kappa_b(0), \qquad Sj \le b \to \kappa_b(Sj) = F(\kappa_b(j)), \qquad j \le b \to b \le \kappa_b(j) \le C_b. \tag{8.14}$$

Indeed $\kappa_b(0) = I_F(Sb) = F(I_F(b)) \ge F(b) \ge M_g(b)$; the recurrence is Theorem 8.38(d); the upper bound is Theorem 8.38(f) and the lower bound is monotonicity.

Write $n_b := \mathrm{mn}(n, b)$ and $\vec{x}_b := (\mathrm{mn}(x_i, b))_i$, and define

$$\tau_f(b, n, \vec{x}) := \mathrm{Rec}_{C_b}(n_b, \tau_g(b, \vec{x}_b);\, u, v.\, \tau_h(C_b, u, v, \vec{x}_b)).$$

(ii) is the bound of Theorem 8.39.

No clipping on bounded inputs. Fix $\vec{x} \le b$ and let $r_b(j) := \mathrm{Rec}_{C_b}(j, \tau_g(b, \vec{x}); u, v.\tau_h(C_b, u, v, \vec{x}))$. Quantifier-free induction on j proves

$$j \le b \to r_b(j) \le \kappa_b(j). \tag{8.15}$$

At 0, $\tau_g(b, \vec{x}) \le M_g(b) \le \kappa_b(0) \le C_b$, so the seed is not clipped. Let $Sj \le b$. The arguments j, $r_b(j)$, $\vec{x}$ of the step are bounded by $\kappa_b(j)$ and by C_b, so (iii) for h gives

$$\tau_h(C_b, j, r_b(j), \vec{x}) = \tau_h(\kappa_b(j), j, r_b(j), \vec{x}) \le M_h(\kappa_b(j)) \le F(\kappa_b(j)) = \kappa_b(Sj) \le C_b.$$

The value is not clipped, which proves (8.15) and

$$Sj \le b \to r_b(Sj) = \tau_h(C_b, j, r_b(j), \vec{x}). \tag{8.16}$$

Coherence. Fix b, c and $n, \vec{x}$ bounded by both. The minimum identities remove the clipping of the inputs. Quantifier-free induction on j proves $(j \le b \wedge j \le c) \to r_b(j) = r_c(j)$: the base is (iii) for g and the absence of seed clipping; if $Sj \le b, c$, the two states at j are equal and bounded by C_b and C_c, as are the other arguments, so (iii) for h equates the two step values and (8.16) removes their clipping. At $j = n$ this is (iii) for f. The parameters $b, c, n, \vec{x}$ are fixed and both induction formulas are quantifier-free.

Every body inserted into B_0 or Rec_b is the already compiled term τ_h of the original language. No recursion over a function introduced later by a graph is used. □

For a nonempty tuple $\vec{x} = (x_1, \ldots, x_k)$ let

$$C_i(\vec{x}) := \bigwedge_{j<i} x_j < x_i \wedge \bigwedge_{j>i} x_j \le x_i$$

say that i is the first largest coordinate. By Theorem 8.36 these k quantifier-free cases are exhaustive and pairwise disjoint. Define the graph

$$\Phi_f(\vec{x}, z) := \bigvee_{i=1}^{k} (C_i(\vec{x}) \wedge z = \tau_f(x_i, \vec{x})), \tag{8.17}$$

and $z = \tau_f(0)$ for a nullary f.

Example 8.41. For addition, without the general compiler: $\Phi_+(x, y, z) := (y \le x \wedge z = \mathrm{ad}_x(x, y)) \vee (x < y \wedge z = \mathrm{ad}_y(x, y))$. In each case the chosen cap bounds both arguments, and Theorem 8.38 gives the addition equations. Each of the two value terms is bounded by twice one input, whereas addition has no bounding coordinate.

Lemma 8.42 (Graphs)**.** $\mathsf{T}_{B\varepsilon}$ *proves that each* Φ_f *is total and single-valued. In the definitional extension by the functions* f^* *with graph* Φ_f*:*

(a) $f^*(\vec{x}) = \tau_f(b, \vec{x})$ *whenever* b *bounds all* x_i*;*

(b) the defining equations of the description f *hold;*

(c) induction holds for quantifier-free formulas that contain finitely many of the f^**.*

Proof. Exhaustiveness gives a value term in each case, and disjointness gives uniqueness. (a) On the case C_i both b and x_i bound all arguments, so Theorem 8.40(iii) applies.

(b) The initial equations follow from the initial terms. For composition take a cap b for the inputs; the inner values are bounded by $Q(b)$, and (a) at $Q(b)$ for the outer function gives the equation from the definition of τ_f. For recursion at the counter 0 take a cap b for $0, \vec{x}$: the seed is not clipped, so the value is $\tau_g(b, \vec{x}) = g^*(\vec{x})$. At Sn take a cap b for $Sn, \vec{x}$: then $n \leq b$, (8.16) applies, (a) identifies the previous state with $f^*(n, \vec{x})$, all arguments of the step are bounded by C_b, and (a) for h identifies the step value with $h^*(n, f^*(n, \vec{x}), \vec{x})$.

(c) Conservativity of the extension is not enough for this. By recursion on a term with occurrences of the f^* one obtains finitely many pairs (A_j, t_j) of a quantifier-free guard and a term, both in the original language, such that the guards cover all inputs and the term equals t_j on A_j: at a composition take the product of the cases of the arguments and then the cases of the outer graph. The guards need not be disjoint; the values agree on overlaps. For a quantifier-free formula take a common refinement for all its terms, replace the terms by their values on each guard, and keep the connectives. The disjunction of the guarded formulas is quantifier-free in the original language and provably equivalent to the given formula, also under implication and negation, because the guards cover all inputs. Theorem 8.34 applied to it gives the induction instance. A graph with an existential quantifier would not allow this step. □

Theorem 8.43 (Bi-interpretation)**.** $\mathsf{T}_{B\varepsilon}$ *and* iPRA *are bi-interpretable. Both interpretations use the whole domain and actual equality, the logic is translated identically, and the identity is the isomorphism of both composites. The theories are equiconsistent.*

Proof. In iPRA. Interpret 0, S as themselves, ε as the primitive recursive characteristic of equality, and each staged recursor with body h by primitive recursion with initial value $S(h(0, 0, \vec{p}))$ and step $h(n, v, \vec{p})$, after translating the body. The equations (8.9), (8.10) become defining equations, and (8.11)–(8.13) and induction for literal equations are provable.

In $\mathsf{T}_{B\varepsilon}$. Interpret each description f by its graph Φ_f, or work in the extension by the f^*. Theorem 8.42 gives the defining equations and every quantifier-free induction instance; Theorem 8.28 and (8.13) give injectivity of the successor and $0 \neq Sx$.

The composite on the side of iPRA. By recursion on a description, with quantifier-free induction on the counter at each recursion node, the double interpretation of a function equals the function: both satisfy the same initial and step equations.

The composite on the side of $\mathsf{T}_{B\varepsilon}$. By recursion on the stage of a term. 0 and S are preserved. The interpreted characteristic of equality and ε are both Boolean and both reflect equality, so they agree by cases. For a recursor with body h the hypothesis for the earlier stage identifies the interpreted body with h; the recursor and its double interpretation satisfy (8.9), (8.10) for the same body, and induction on the equation between them identifies them at every counter and parameter. Where the double interpretation is expressed with graph-defined functions, Theorem 8.42(c) provides the induction instance.

All operations and equality are preserved by the identity, so every formula is provably equivalent to its double translation by induction on the formula. Falsity is preserved in both directions. □

8.2.5 The functions denoted by terms

For $k \geq 1$ let $\mathcal{A}_k$ be the set of primitive recursive $f\colon \mathbb{N}^k \to \mathbb{N}$ for which there are a coordinate i and a unary primitive recursive F with $f(\vec{x}) \leq F(x_i)$ for all $\vec{x}$, and let $\mathcal{A}_0$ be the constants.

Theorem 8.44 (Exact class of term functions)**.** *The functions denoted in the standard model by the terms of $\mathsf{T}_{B\varepsilon}$ with k free variables are exactly the functions in $\mathcal{A}_k$.*

Proof. Every term denotes a primitive recursive function, and Theorem 8.30 gives the bound. Conversely let $f(\vec{x}) \leq F(x_i)$. The compiler provides the cases $C_j(\vec{x})$ and the value terms $v_j(\vec{x}) := \tau_f(x_j, \vec{x})$, with value $f(\vec{x})$ on C_j. Let $a_j(\vec{x}) := G(\chi_{C_j}(\vec{x}), v_j(\vec{x}))$: it is 0 off its case and $f(\vec{x})$ on it. Exactly one case holds, so each a_j is at most $F(x_i)$ and at most one is nonzero. Every unary primitive recursive function is a term: for one variable the only case is automatic and the value is $\tau_F(x, x)$. Put $b := \tau_F(x_i, x_i)$, $s_0 := 0$ and $s_j := \mathrm{ad}_b(s_{j-1}, a_j)$ for $1 \leq j \leq k$. Every partial sum is 0 or $f(\vec{x})$, so both arguments of each bounded addition are at most b, and s_k denotes f. For $k = 0$ use the numeral. □

The converse direction assumes that the majorant is valid in the standard model. To prove inside $\mathsf{T}_{B\varepsilon}$ that the single term equals the graph-defined function one needs a proof of the majorant in the theory; Theorem 8.43 needs no such hypothesis.

Proposition 8.45 (Number of value terms)**.** *The graph* (8.17) *of a k-ary function has k value terms. The k-ary sum is not, in the standard model, always equal to one of $r < k$ terms.*

Proof. Each of the r terms is bounded by a constant or by a function of one input. Choose an input that controls none of them, fix the others at 0 and let the chosen one grow: the terms stay bounded and the sum does not. □

This concerns a choice among finitely many value terms. It is not a bound on the number of clauses of a graph formula: the characteristic function of a primitive recursive graph is bounded, hence a term by Theorem 8.44, so every such graph has a definition by one equation. The value terms are what gives totality and the transfer of induction in Theorem 8.42.

Example 8.46 (The majorant must be primitive recursive)**.** $\mathcal{A}_2$ is strictly smaller than the set of primitive recursive functions that are bounded by an arbitrary function of one input. Let $u_0, u_1, \ldots$ enumerate the unary primitive recursive descriptions and let $g(x) := 1 + \max_{e \leq x} u_e(x)$. This g is total recursive and has no primitive recursive majorant, since $g(x) > u_e(x)$ for $x \geq e$. Fix a machine for g and let $s(x, t)$ be its output if it halts within t steps on x, and 0 otherwise; s is primitive recursive. Then $f(x, t) := x + s(x, t)$ is primitive recursive, and for fixed x its values lie in $\{x, x + g(x)\}$, so it is bounded by a function of x. A primitive recursive bound in x would majorize g, and no bound in t exists because $f(x, t) \geq x$. Hence $f \notin \mathcal{A}_2$.

8.2.6 Truncated subtraction in place of the equality test

Definition 8.47. The terms are $t ::= x \mid 0 \mid St \mid d(t, t) \mid B(n; u, v.h)$. The computation equations are (8.9), (8.10) and

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)), \tag{8.18}$$

where P is the term of Theorem 8.27.

(a) T_{Bd} is ordinary intuitionistic first-order logic with equality over these terms, with the four computation equations, coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$, separation $0 = S0 \to \bot$ and induction for literal equations with side assumptions.

(b) N_{Bd} is the native calculus over these terms with the four computation equations and the rule forms of Section 1.3; the contradiction is $0 = S0$.

In T_{Bd} coverage is an axiom, in contrast to $\mathsf{T}_{B\varepsilon}$. It is used in the proof of Theorem 8.49, and it does not follow from the computation equations and induction for literal equations: `monus_and_neutral_major` refers to the coordinatewise structure on $\mathbb{N}\times\mathbb{N}$, treated in `scope_kernel`, which satisfies those and refutes coverage. In N_{Bd} the case rule NC supplies this resource.

By Theorems 8.28 and 8.29 the terms P, Z, G and B_0 and their equations are available in both theories. In T_{Bd}, coverage and the values of Z give

$$Z(x) = 0 \vee Z(x) = 1, \qquad Z(x) = 1 \leftrightarrow x = 0. \tag{8.19}$$

Lemma 8.48. *With induction for literal equations only,*

$$d(0,y) = 0, \qquad P(d(Sx,y)) = d(x,y), \qquad d(Sx,Sy) = d(x,y), \qquad d(x,x) = 0, \qquad d(Sx,x) = 1.$$

Proof. $d(0,y) = 0$ by induction on y with $P(0) = 0$. For the second equation fix x and induct on y: the base is $P(Sx) = x$, and $P(d(Sx,Sy)) = P(P(d(Sx,y))) = P(d(x,y)) = d(x,Sy)$. The third equation follows from (8.18) and the second. The last two are inductions on x with bases $d(0,0) = 0$, $d(1,0) = 1$ and the third equation as step. □

Lemma 8.49 (Gated complement)**.** T_{Bd} *proves*

$$G\big(Z(d(y,x)),\, d(x,d(x,y))\big) = G\big(Z(d(y,x)),\, y\big), \tag{8.20}$$

and consequently $d(y,x) = 0 \to d(x,d(x,y)) = y$.

Proof. Fix x and apply induction for literal equations to (8.20) as an equation in y. At $y = 0$ the control is $Z(d(0,x)) = 1$, and both second arguments are 0, by $d(x,0) = x$ and $d(x,x) = 0$.

For the step keep the induction hypothesis and put $a := d(Sy,x)$. By coverage a is a successor or 0. If a is a successor, $Z(a) = 0$ and both sides of the goal are 0. Let $a = 0$. By Theorem 8.48, $d(y,x) = P(d(Sy,x)) = 0$, so the induction hypothesis, with both gates at the control 1, gives

$$d(x,t) = y, \qquad t := d(x,y).$$

If $t = 0$, then $d(x,t) = x$, so $x = y$ and $a = d(Sy,y) = 1$, contradicting $a = 0$. Hence $t = Sw$ for some w by coverage, and $w = P(t) = d(x,Sy)$ by (8.18). Put $v := d(x,w)$. Then $P(v) = d(x,Sw) = d(x,t) = y$. If $v = 0$, then $y = 0$, so $t = d(x,0) = x = Sw$ and $v = d(Sw,w) = 1$, a contradiction. Hence $v = Sz$ by coverage, $z = P(v) = y$ and $v = Sy$. With $w = d(x,Sy)$ this reads

$$d(x,d(x,Sy)) = Sy,$$

which is the goal at the control $Z(a) = 1$. The contradictions use $0 \neq 1$ only. The only induction formula is the literal equation (8.20). If $d(y,x) = 0$, both gates have the control 1. □

Theorem 8.50 (Reflection of equality)**.** *Let* $\varepsilon(x,y) := G\big(Z(d(x,y)), Z(d(y,x))\big)$. T_{Bd} *proves*

$$d(x,y) = 0 \wedge d(y,x) = 0 \to x = y, \qquad \varepsilon(x,y) = 0 \vee \varepsilon(x,y) = 1, \qquad \varepsilon(x,y) = 1 \leftrightarrow x = y.$$

Proof. From $d(y,x) = 0$, Theorem 8.49 gives $d(x,d(x,y)) = y$; substituting $d(x,y) = 0$ and $d(x,0) = x$ gives $x = y$. Each input of the gate is 0 or 1 by (8.19), and the gate equations give the range and $\varepsilon(x,y) = 1 \leftrightarrow d(x,y) = 0 \wedge d(y,x) = 0$: if the first input is 0 the output is 0, and otherwise it is the second input. The first statement gives $x = y$ from the right side; conversely $x = y$ makes both differences 0 by $d(x,x) = 0$. □

Corollary 8.51. T_{Bd} *is bi-interpretable with* iPRA*, with whole domains, actual equality, the logic translated identically and the identity as isomorphism of both composites.*

Proof. By Theorem 8.50 the term ε satisfies (8.11) and (8.12), so every derivation of $\mathsf{T}_{B\varepsilon}$ becomes a derivation of T_{Bd} when this term is substituted for the symbol; in particular Theorems 8.40 and 8.42 hold. In the other direction iPRA interprets the recursors as in Theorem 8.43 and d as truncated subtraction; it proves (8.18), coverage, separation and induction for literal equations.

For the composite on the side of T_{Bd}, first the recursors C and W, which do not contain d, are identified with their double interpretations by induction on the equation between them, and hence P is preserved. The double interpretation of d is the graph of the primitive recursive truncated subtraction; it satisfies the two equations (8.18) with the preserved P, and induction on the second argument identifies it with d. Likewise the term $\widehat{d}$ of Theorem 8.37, with ε expanded, satisfies $\widehat{d}(x,0) = x$ and $\widehat{d}(x,Sy) = P(\widehat{d}(x,y))$, so $\widehat{d}(x,y) = d(x,y)$ by induction on y for this literal equation of the original language. The remaining recursors are identified as in Theorem 8.43, and the composite on the side of iPRA is unchanged. □

8.2.7 The native calculus with truncated subtraction

The proof of Theorem 8.49 splits on $t = 0$ or $t = Sw$ while an induction hypothesis that mentions t is in the context, and Theorem 8.31 excludes a selector term. N_{Bd} derives the split by a selector whose zero branch is Boolean.

Lemma 8.52. N_{Bd} *derives the equality laws,* NC *and* EI *of Theorem 8.2, the equations of Theorems 8.28, 8.29 and 8.48, every equation from* $0 = S0$*, and the functional range rule: if* $\Gamma \vdash K(0)$ *and* $\Gamma \vdash K(1)$ *then* $\Gamma \vdash K(Z(t))$*, for every formula template* K *and term* t*.*

Proof. Reflexivity follows from the computation $C(S0,p) = p$ as in Theorem 8.2, and the other equality laws, NC and EI are derived as there; Theorem 8.4 holds as well. The equations of the three lemmas use computation, equality and EI only. Under $0 = S0$, transport between $C(0,a) = Sa$ and $C(1,a) = a$ gives $Sa = a$; then EI on the equation $a = 0$ gives $a = 0$ for every a, hence every equation. The functional range rule is NC on the template $w \mapsto K(Z(w))$. □

By recursion on a native formula, with DI by cases at each $D_{x,y}$ and the representation of $Q_y B$ by $D_{v,y}B$ with v unused, every native formula follows from $0 = S0$.

A term b is called *two-valued* in Γ if the functional range rule holds for it: $\Gamma \vdash K(0)$ and $\Gamma \vdash K(1)$ imply $\Gamma \vdash K(b)$ for every template K. Terms $Z(t)$ are two-valued.

Lemma 8.53 (A selector with a Boolean zero branch)**.** *Let* $\mathrm{inc}(a,b) := C(Z(b),a)$ *and*

$$\eta(y,b,u) := \mathrm{inc}(G(y,u),\, G(Z(y),b)).$$

Then $\eta(Sy,b,u) = u$ *for all terms, and* $\eta(0,b,u) = b$ *for every two-valued* b*.*

Proof. $\eta(Sy,b,u) = \mathrm{inc}(u,G(0,b)) = \mathrm{inc}(u,0) = C(1,u) = u$. $\eta(0,b,u) = \mathrm{inc}(0,G(1,b)) = C(Z(b),0)$, which is $C(1,0) = 0$ at $b = 0$ and $C(0,0) = 1$ at $b = 1$; the functional range rule for the template $C(Z(w),0) = w$ gives $\eta(0,b,u) = b$. □

The zero branch of η is two-valued and the successor branch is arbitrary, so η does not contradict Theorem 8.30.

Lemma 8.54 (Contextual cases)**.** *In* N_{Bd}*, let* $s = t$ *be a literal equation.*

(a) If b *is two-valued in* Γ*,* $\Gamma, b = 0 \vdash s = t$ *and* $\Gamma, b = 1 \vdash s = t$*, then* $\Gamma \vdash s = t$*.*

(b) If z *is fresh,* $\Gamma, r = 0 \vdash s = t$ *and* $\Gamma, r = Sz \vdash s = t$*, then* $\Gamma \vdash s = t$*.*

The context and the equation may mention b *and* r*.*

Proof. (a) Let $K(x) := D_{a,y}(\eta(y,x,s) = \eta(y,b,t))$ with a unused. For $c \in \{0,1\}$ derive $K(c)$ by DI by cases: the local assumption $\eta(0,c,s) = \eta(0,b,t)$ is $c = b$ by Theorem 8.53, the given branch gives $s = t$, and this is the matrix at Sz. The range rule for b gives $K(b)$. Rule DE opens its closure, whose zero instance $\eta(0,b,s) = \eta(0,b,t)$ holds because both sides equal b, and QE at 1 gives $s = t$.

(b) Rule NC on fresh variables in literal equations gives

$$G(Z(r),r) = 0, \qquad G(Z(Z(r)),r) = G\big(Z(Z(r)), S(P(r))\big) :$$

for the first, the zero case is $G(1,0) = 0$ and a successor case has the control 0; for the second, the zero case has the control 0, and a successor case has the control 1 and $P(Sx) = x$. Apply (a) to $b := Z(r)$. Under $Z(r) = 1$ the first identity gives $r = 0$. Under $Z(r) = 0$ the second gives $r = S(P(r))$; substitute $P(r)$ for z in the second given branch. □

Theorem 8.55. N_{Bd} *derives* (8.20)*, the three statements of Theorem 8.50 in the forms*

$$d(x,y) = 0,\ d(y,x) = 0 \vdash x = y, \qquad s = t \vdash \varepsilon(s,t) = 1, \qquad \varepsilon(s,t) = 1 \vdash s = t,$$

with $\varepsilon(s,t)$ two-valued, and the introduction and elimination rules of intuitionistic implication, conjunction, disjunction, both quantifiers and falsity for the formulas defined in the proof. It is bi-interpretable with T_{Bd} and with iPRA*, with whole domains, actual equality, identity composites and both formula round trips.*

Proof. Every case split in the proof of Theorem 8.49 has an equation as its conclusion, so it is an instance of Theorem 8.54(b); the branches that end in $0 = 1$ are closed by Theorem 8.52. The induction is EI for the same literal equation. The term $\varepsilon(s,t)$ is two-valued by the range rules of its two inputs and the gate equations.

For a literal equation E, namely $s = t$, define the gate $\mathcal{G}_v(E) := \big(G(v,\varepsilon(s,t)) = G(v,1)\big)$; it is derivable at $v = 0$ and interderivable with E at $v = Sw$. For literal equations E $(s = t)$ and F $(r = w)$ define the formula selector $\mathcal{M}_v(E,F) := \big(\eta(v,\varepsilon(s,t),\varepsilon(r,w)) = 1\big)$; by Theorem 8.53, $\mathcal{M}_0(E,F)$ is interderivable with E and $\mathcal{M}_{Sz}(E,F)$ with F. Extend the gate through $D_{x,y}$ by $\mathcal{G}_v(D_{x,y}A) := D_{x,y}\mathcal{G}_vA$, replace each Q_yA by $D_{a,y}A$ with a unused, align binder depths by the padding $A \approx D_{a,y}\mathcal{G}_yA$, and extend the selector through matched binders by recursion on the formulas; the endpoint properties are preserved by the binder congruence of Theorem 8.4. With $p(z) := B_0(z;u,v.Z(v))$, so that $p(1) = 1$ and $p(2) = 0$, and with unused coordinates fresh, put

$$\begin{aligned}
A \Rightarrow_* B &:= D_{a,z}\mathcal{M}_z(A,B), & A \wedge_* B &:= D_{a,z}\mathcal{G}_z\mathcal{M}_{p(z)}(A,B),\\
\exists_* x\, A(x) &:= D_{x,y}\mathcal{G}_yA(x), & A \vee_* B &:= \exists_* x\, \mathcal{M}_x(A,B),\\
\forall_* x\, A(x) &:= A(0) \wedge_* D_{a,z}\mathcal{G}_zA(z), & \bot_* &:= (0 = S0).
\end{aligned}$$

Implication introduction is DI by cases, whose local assumption is the zero instance of the selector; implication elimination opens the closure and specializes at 1. The derivable zero instance of the gate gives the introductions of conjunction and existence, and specialization at 1 and 2 their eliminations. Once implication is available, NC on an implication template gives numerical cases in a context for arbitrary formulas, hence disjunction elimination and universal elimination. These are the constructions of Theorem 1.119 with the atomic gate and selector above in place of the numerical selector sel, which does not exist here.

Every rule of N_{Bd} is sound for T_{Bd} under the readings of Section 1.2: DI by cases uses coverage and QI uses induction for literal equations. Conversely N_{Bd} derives the translation of every theorem of T_{Bd} by the rules just established. The round trips are

$$Q_yB \approx B(0) \Rightarrow_* \forall_* y\, B(y), \qquad D_{x,y}B \approx \exists_* x\, \big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big) :$$

forward, DE opens the witness, and under the zero instance QE gives every instance; backward, eliminate the existential, keep the zero instance of the matrix, and use the implication and the universal at Sz for DI by cases. Induction on formulas gives both formula composites, and the numerical composites are identities. The bi-interpretation with iPRA is Theorem 8.51. □

8.3 The self-seeded recursor

This section draws on `selfseed_scalar`, `scope_and_coverage`, `coverage_and_kernel` and `selfseed_injectivity`.

8.3.1 The theories

Definition 8.56. The terms are $t ::= x \mid 0 \mid St \mid \varepsilon(t,t) \mid F(n; v.h)$, where $F(n; v.h)$ binds v in h. The computation equations are

$$F(0; v.h) = S(h[0/v]), \qquad F(Sn; v.h) = h[F(n; v.h)/v]. \tag{8.21}$$

T_F^- is ordinary intuitionistic first-order logic with equality over these terms with (8.21), the axioms (8.11), (8.12), (8.13) for ε and separation, and induction for literal equations with side assumptions. T_F is T_F^- with the additional axiom $Sx = Sy \to x = y$. Coverage is not an axiom of either theory.

The update sees neither an initializer nor the old counter. Compared with $\mathsf{T}_{B\varepsilon}$, which derives injectivity from the counter argument (Theorem 8.28), T_F postulates it; Theorem 8.69 shows that the postulate is redundant, so that T_F^- and T_F have the same theorems. As in Section 8.2, S can be removed from the signature by $Sa := F(0; v.a)$. Put

$$C(n,a) := F(n; v.a), \qquad Z(n) := C(n,0),$$

so that $C(0,a) = Sa$, $C(Sn,a) = a$, $Z(0) = 1$, $Z(Sn) = 0$.

Lemma 8.57. *T_F^- proves $Sn \neq 0$. The terms*

$$\neg_B b := \varepsilon(b,0), \qquad b \wedge_B c := \varepsilon\big(C(b, \varepsilon(c,0)), 0\big), \qquad b \vee_B c := \neg_B(\neg_B b \wedge_B \neg_B c)$$

have the truth tables of negation, conjunction and disjunction on the values $0, 1$. Every quantifier-free formula A has a term χ_A with $\chi_A = 0 \vee \chi_A = 1$ and $\chi_A = 1 \leftrightarrow A$, and T_F^- admits induction with side assumptions for quantifier-free formulas.

Proof. If $Sn = 0$, then $0 = Z(Sn) = Z(0) = 1$. Reflection and the range axiom give $\varepsilon(0,0) = 1$ and $\varepsilon(1,0) = 0$. For the conjunction, $C(0, \varepsilon(c,0)) = S(\varepsilon(c,0)) \neq 0$, so the value at $b = 0$ is 0; and $C(1, \varepsilon(c,0)) = \varepsilon(c,0)$, so the value at $b = 1$ is $\neg_B \neg_B c = c$. The only non-Boolean intermediate value is a successor, and its distinctness from 0 does not cancel a successor. Define $\chi_{s=t} := \varepsilon(s,t)$, $\chi_\bot := 0$, and the connectives by the Boolean terms, implication as $\neg_B \chi_A \vee_B \chi_B$; the Boolean range comes from the range axiom and the truth tables. Induction for $A(n)$ is induction for the literal equation $\chi_{A(n)} = 1$. □

Definition 8.58. A *finite-case graph* is a formula $\bigvee_i (A_i(\vec{x}) \wedge y = t_i(\vec{x}))$ with quantifier-free guards A_i and terms t_i of the language, such that the guards are provably exhaustive and the values provably agree where guards overlap.

Lemma 8.59 (Finite-case elimination)**.** *A finite-case graph is provably total and single-valued. In the definitional extension by the functions of finitely many finite-case graphs, every quantifier-free formula is provably equivalent to a quantifier-free formula of the original language, and induction for quantifier-free formulas of the extension is admissible. Compositions of finite-case functions are finite-case.*

Proof. As in Theorem 8.42(c): expand each term into finitely many guarded terms of the original language by refining the cases of its arguments and then the cases of the outer graph, take a common refinement for a formula, and apply Theorem 8.57 to the resulting disjunction. □

The lemma concerns function terms in formulas. It does not allow a new body h of F that contains a function defined by a graph; bodies are always terms of the original language.

8.3.2 Without injectivity of the successor

All statements of this subsection are theorems of T_F^-.

Lemma 8.60 (No fixed point)**.** *$Sa \neq a$ for every a.*

Proof. The term $h(v) := \varepsilon(v, 0)$ has no fixed point: $h(v) = v$ and the range of h give $v = 0$ or $v = 1$, contradicting $h(0) = 1$ and $h(1) = 0$. If $Sa = a$, then $F(a; v.h) = F(Sa; v.h) = h(F(a; v.h))$. □

Theorem 8.61 (No cycle of a fixed length)**.** *For each fixed integer $k \geq 1$, the equations* (8.21) *and the axioms* (8.11)*,* (8.12)*,* (8.13) *prove $\forall a\ S^k a \neq a$, without any instance of induction. Consequently distinct numerals are provably distinct, and $S^i a \neq S^j a$ for fixed $i < j$.*

Proof. The proofs are constructed by recursion on k outside the theory. Assume the statement for all lengths below k. Then $\bar{0}, \ldots, \bar{k}$ are pairwise distinct: $\bar{0}$ differs from a positive numeral by Theorem 8.57, and $\bar{i} = \bar{j}$ with $1 \leq i < j \leq k$ would be $S^{j-i}(\bar{i}) = \bar{i}$ with $1 \leq j - i < k$. For $k = 1$ only separation is used.

For a term b with $b = 0 \vee b = 1$ let $\mathrm{Inc}(b, a) := C(\varepsilon(b, 0), a)$, so that $\mathrm{Inc}(0, a) = a$ and $\mathrm{Inc}(1, a) = Sa$. Define, as a written finite expansion,

$$T_0(v) := 0, \qquad T_{j+1}(v) := \mathrm{Inc}\big(\varepsilon(v, \bar{j}), \cdot\big)^{j+1}\big(T_j(v)\big) \quad (0 \leq j < k), \qquad c_k(v) := T_k(v),$$

where the exponent denotes $j + 1$ written compositions. At $v = \bar{i}$ with $i < k$ the test with index i is 1 and the others are 0, so $i + 1$ increments are performed; at $v = \bar{k}$ all tests are 0. Hence

$$c_k(\bar{i}) = \overline{i+1}\ (0 \leq i < k), \qquad c_k(\bar{k}) = 0, \qquad c_k(v) = \bar{0} \vee \cdots \vee c_k(v) = \bar{k},$$

the last by the finitely many cases of (8.11): if some test is 1, reflection identifies v with a numeral and distinctness makes the other tests 0.

If $c_k^k(w) = w$ for the k-fold composition, then w is one of $\bar{0}, \ldots, \bar{k}$ by the range, since $k \geq 1$. On these $k+1$ distinct values c_k is one cycle, and k applications of it end at a different numeral. So $c_k^k(w) \neq w$ for every w.

Now let $S^k a = a$ and $w := F(a; v.c_k)$. By k uses of the successor equation and congruence, $w = F(S^k a; v.c_k) = c_k^k(F(a; v.c_k)) = c_k^k(w)$, a contradiction. The last statement applies the theorem to $S^i a$ with the length $j - i$. □

The theorem excludes a cycle along one successor chain. It does not derive $a = b$ from $Sa = Sb$: two predecessor chains may merge without a cycle. The term c_k depends on the fixed k, and there is no term for cycles of a length given by a variable.

For the next results define, with a parameter a,

$$g_a(v) := \varepsilon(v, 0) \vee_B \varepsilon(Sv, a), \qquad \theta_a(v) := C(g_a(v), v), \qquad U(j, a) := F(j; v.\theta_a(v)), \qquad p_*(a) := U(a, a).$$

The update θ_a holds the state if it is 0 or if its successor is a, and increments it otherwise. Since $\theta_a(0) = 0$, the orbit starts at $U(0, a) = 1$, and $U(Sj, a) = \theta_a(U(j, a))$.

Lemma 8.62 (Protected orbit)**.** *$U(j,a) = Sj \vee S(U(j,a)) = a$, and $S(p_*(S^2m)) = S^2m$.*

Proof. The first statement is a quantifier-free induction on j with a fixed. The base is $U(0,a) = S0$. In the step put $v := U(j,a)$ and decide $Sv = a$. If it holds, θ_a fixes v and the second alternative persists. Otherwise the first alternative of the hypothesis gives $v = Sj \neq 0$, the guard is false and $U(Sj,a) = Sv = SSj$.

Specialize the proved statement to $j = m$, $a = S^2m$. Both alternatives give $S(U(m,S^2m)) = S^2m$, so the state is fixed by θ_a and two further computation steps give $U(S^2m,S^2m) = U(m,S^2m)$. No successor is cancelled. □

Theorem 8.63 (Coverage and a right predecessor)**.** *Let P be the function of the finite-case graph*

$$(n = 0 \wedge p = 0) \vee (n = 1 \wedge p = 0) \vee (n \neq 0 \wedge n \neq 1 \wedge p = p_*(n)).$$

T_F^- proves, by Theorem 8.59,

$$P(0) = P(1) = 0, \qquad n = 0 \vee n = S(P(n)), \qquad S(P(Sn)) = Sn,$$

and hence coverage $n = 0 \vee \exists m\, n = Sm$.

Proof. The graph is finite-case because equality is decidable and $0 \neq 1$. Apply quantifier-free induction to

$$n = 0 \ \vee \ n = 1 \ \vee \ S(p_*(n)) = n.$$

The base holds. In the step, $n = 0$ gives $Sn = 1$. If $n = 1$, then $Sn = S^20$ and Theorem 8.62 with $m = 0$ gives $S(p_*(Sn)) = Sn$. If $S(p_*(n)) = n$, then $Sn = S^2(p_*(n))$ and Theorem 8.62 with $m = p_*(n)$ gives $S(p_*(Sn)) = Sn$. This proves the displayed disjunction for every n. To pass to P, decide $n = 0$ and $n = 1$: if $n = 0$ the first alternative of $n = 0 \vee n = S(P(n))$ holds; if $n = 1$ then $n = S0 = S(P(1))$; and if $n \neq 0, 1$ the third alternative of the disjunction holds and $P(n) = p_*(n)$, so $n = S(P(n))$. Applying $n = 0 \vee n = S(P(n))$ to Sn and excluding its zero alternative gives $S(P(Sn)) = Sn$. □

The proof holds for all elements of a model, standard or not. The result is a right inverse: it chooses one predecessor of each nonzero element and does not prove that $P(Sa) = a$.

Proposition 8.64 (Standard numerals have unique predecessors)**.** *For every fixed integer $k \geq 1$, T_F^- proves*

$$Sx = \bar{k} \leftrightarrow x = \overline{k-1}. \tag{8.22}$$

Proof. By recursion on k outside the theory. The data carried are the pairwise distinctness of $\bar{0}, \ldots, \overline{k+1}$ and (8.22) for all smaller positive indices. Initially $0, 1, 2$ are distinct by Theorems 8.57 and 8.60.

Let h_k be a term with the values

$$h_k(v) = \begin{cases} \overline{i-1}, & v = \bar{i},\ 1 \leq i \leq k-1, \\ \overline{k-1}, & v = \overline{k+1}, \\ \bar{k}, & \text{otherwise.} \end{cases}$$

Such a term is obtained as follows. For $r = 1, \ldots, k$ let $b_r(v)$ be the Boolean negation of the disjunction of $\varepsilon(v,\bar{i})$ for $1 \leq i \leq \min(r, k-1)$, together with $\varepsilon(v,\overline{k+1})$ if $r = k$; it is the characteristic term of the condition that the displayed value is at least r. Put $t_0(v) := 0$, $t_r(v) := C(\neg_B b_r(v), t_{r-1}(v))$ and $h_k := t_k$: a true condition contributes one successor. For $k = 1$ the values of h_1 are those of $\neg_B \varepsilon(v,2)$.

Let $A_k(n) := F(n; v.h_k)$ and let $q_0 := \overline{k+1}$, $q_j := \overline{k-j}$ for $1 \le j \le k$; these numerals are distinct. Since $h_k(0) = \bar{k}$, $A_k(0) = q_0$, and computation gives $A_k(\bar{j}) = q_j$ for $0 \le j \le k$. The term h_k never has the value q_0, and for $1 \le j \le k$,

$$h_k(v) = q_j \leftrightarrow v = q_{j-1}.$$

Quantifier-free induction gives $A_k(n) = q_0 \leftrightarrow n = 0$: at a successor both sides are false. For $1 \le j < k$, recursion on j gives $A_k(n) = q_j \leftrightarrow n = \bar{j}$: by coverage (Theorem 8.63) $n = 0$ or $n = Sm$; at 0 both sides are false; at Sm the left side is equivalent to $A_k(m) = q_{j-1}$, hence to $m = \overline{j-1}$, and so is the right side by (8.22) for $j < k$.

Now let $Sx = \bar{k}$. Then $h_k(A_k(x)) = A_k(\bar{k}) = q_k$, so $A_k(x) = q_{k-1}$ and $x = \overline{k-1}$. The converse is immediate. For the distinctness of $\overline{k+2}$: an equality $\overline{k+2} = \bar{j}$ with $1 \le j \le k$ reduces by (8.22) for j to $\overline{k+1} = \overline{j-1}$; equality with 0 is excluded by Theorem 8.57 and equality with $\overline{k+1}$ by Theorem 8.60. □

This is a family of derivations, one for each k, and not one theorem that quantifies over k. It does not exclude a collision $Sx = Sy$, $x \neq y$, over a nonstandard target; Theorem 8.69 does.

Lemma 8.65 (An equivalent form of injectivity)**.** *In the extension by P put $T(n) := P(Sn)$. Then*

$$S(T(n)) = Sn, \qquad T(T(n)) = T(n), \qquad T(0) = 0,$$

and over T_F^- injectivity of the successor is equivalent to $\forall n\,(T(n) = n \to T(Sn) = Sn)$.

Proof. The first equation is Theorem 8.63; applying P to it gives the second; the third is $P(1) = 0$. If the fixed points of T are closed under successor, quantifier-free induction on $T(n) = n$ gives $T(n) = n$ for all n, and from $Sa = Sb$ one gets $a = T(a) = T(b) = b$. Conversely injectivity applied to $S(T(n)) = Sn$ gives $T(n) = n$ for all n. □

Remark 8.66. `scope_and_coverage` and `coverage_and_kernel` contain the protected orbit with the same guard, under the names V, W and U, p_*; the first derives $n = 0 \vee n = S(P(n))$ by induction on that formula in the extension by P, the second by the induction of the proof above. Theorems 8.61 and 8.65 are from the first package and Theorems 8.60 and 8.64 from the second. Neither derives injectivity; Section 8.3.3 does, with a different orbit. The first package records why a comparison of the orbits with targets Sn and S^2n does not close the gap: for $v \neq 0$ the term $S(\theta_{Sn}(v))$ stops when $Sv = Sn$ and $\theta_{S^2n}(Sv)$ stops when $S^2v = S^2n$, so the intertwining of the two orbits needs $S^2v = S^2n \to Sv = Sn$ at the visited states, which is an instance of the missing cancellation.

8.3.3 Injectivity from paired stopping orbits

This subsection draws on `selfseed_injectivity`. All statements are theorems of T_F^-, and no successor is cancelled before Theorem 8.69.

The protected orbit of Theorem 8.62 stops when the successor of the state reaches the target. The orbit below stops when the state itself reaches the target. For a parameter a define

$$e_a(v) := \varepsilon(v, 0) \vee_B \varepsilon(v, a), \qquad s_a(v) := C(e_a(v), v), \qquad O_a(n) := F(n; v.s_a(v)).$$

After expansion of C and $\vee_B$ these are terms of the original language; the body of the recursor in O_a contains no function given by a finite-case graph. By reflection, the range axiom and

the truth table of $\vee_B$ (Theorem 8.57), $e_a(v) = 1$ if $v = 0$ or $v = a$, and $e_a(v) = 0$ if $v \neq 0$ and $v \neq a$. With $C(1, v) = v$ and $C(0, v) = Sv$ this gives

$$s_a(0) = 0, \qquad s_a(a) = a, \qquad v \neq 0 \wedge v \neq a \;\to\; s_a(v) = Sv, \tag{8.23}$$

and (8.21) gives

$$O_a(0) = S(s_a(0)) = 1, \qquad O_a(Sn) = s_a(O_a(n)). \tag{8.24}$$

The value 1 at 0 comes from $s_a(0) = 0$ and the base equation; no initializer is supplied.

Lemma 8.67 (Stopping invariant)**.** *$O_a(n) = Sn \vee O_a(n) = a$.*

Proof. Quantifier-free induction on n with a fixed (Theorem 8.57). The base is $O_a(0) = 1 = S0$. In the step put $v := O_a(n)$ and decide $v = a$. If $v = a$, then $O_a(Sn) = s_a(a) = a$. If $v \neq a$, the hypothesis gives $v = Sn$, so $v \neq 0$ by Theorem 8.57, and $O_a(Sn) = s_a(v) = Sv = SSn$ by (8.23). □

Lemma 8.68 (Absorption)**.** *Let $a \neq 0$.*
(a) $O_a(a) = a$ and $O_a(Sa) = a$.
(b) If $Sa = Sb$, then $O_a(b) = a$.

Proof. (a) By coverage (Theorem 8.63) and $a \neq 0$ there is p with $a = Sp$. Both alternatives of Theorem 8.67 at $n = p$ give $O_a(p) = a$. Hence $O_a(a) = O_a(Sp) = s_a(a) = a$ and $O_a(Sa) = s_a(O_a(a)) = s_a(a) = a$.

(b) Put $c := Sa$, so that also $c = Sb$. By Theorem 8.67 at $n = b$, $O_a(b) = c$ or $O_a(b) = a$. In the first case $c \neq 0$ by Theorem 8.57 and $c \neq a$ by Theorem 8.60, so (8.23), (8.24) and (a) give

$$a = O_a(Sa) = O_a(Sb) = s_a(O_a(b)) = s_a(c) = Sc = SSa,$$

which contradicts Theorem 8.61 with $k = 2$. So the second case holds. □

Theorem 8.69 (Injectivity of the successor)**.** T_F^- *proves $\forall a \forall b\, (Sa = Sb \to a = b)$.*

Proof. Assume $Sa = Sb$ and decide $a = b$ (Theorem 8.57). If $a = b$ there is nothing to prove. Let $a \neq b$; it suffices to derive $\bot$. If $a = 0$, then $Sb = 1$, and Theorem 8.64 with $k = 1$ gives $b = 0 = a$; so $a \neq 0$, and in the same way $b \neq 0$. Theorem 8.68, applied to a with $Sa = Sb$ and to b with $Sb = Sa$, gives

$$O_a(a) = a, \qquad O_a(b) = a, \qquad O_b(b) = b, \qquad O_b(a) = b. \tag{8.25}$$

Each orbit avoids the stop of the other: for every n,

$$O_a(n) \neq b, \qquad O_b(n) \neq a. \tag{8.26}$$

Suppose $O_a(n) = b$. By Theorem 8.67, $O_a(n) = Sn$ or $O_a(n) = a$; the second alternative contradicts $a \neq b$, so $Sn = b$. Since $b \neq 0$ and $b \neq a$, (8.23) and (8.24) give

$$O_a(b) = O_a(Sn) = s_a(O_a(n)) = s_a(b) = Sb,$$

and (8.25) turns this into $a = Sb = Sa$, against Theorem 8.60. The second inequality follows by exchanging a and b, which preserves the assumptions.

The two orbits agree: apply induction for the literal equation

$$O_a(n) = O_b(n)$$

in n, with the parameters a, b and the side assumptions $Sa = Sb$ and $a \neq b$; the variable z of the step is fresh for them. The base is $O_a(0) = 1 = O_b(0)$. In the step assume $O_a(z) = O_b(z)$

and put $v := O_a(z)$. By (8.26), derived from the side assumptions, $v \neq b$ and $v = O_b(z) \neq a$. Decide $v = 0$. If $v = 0$, then $s_a(v) = 0 = s_b(v)$. Otherwise $s_a(v) = Sv$ by (8.23) with $v \neq a$, and $s_b(v) = Sv$ with $v \neq b$. In both cases

$$O_a(Sz) = s_a(v) = s_b(v) = s_b(O_b(z)) = O_b(Sz)$$

by (8.24). The conclusion of the induction at $n = a$ and (8.25) give $a = O_a(a) = O_b(a) = b$, which contradicts $a \neq b$.

Every case distinction of the proof is a decision of an equation by Theorem 8.57; no other instance of excluded middle is used. □

Corollary 8.70. *For every context Γ and formula A, $\Gamma \vdash A$ holds in T_F^- if and only if it holds in T_F. The identity translation of terms and formulas is an interpretation in both directions, and the two composites are the identity.*

Proof. T_F extends T_F^- by one axiom. Conversely replace each instance $Ss = St \to s = t$ of that axiom in a derivation of T_F by the derivation of Theorem 8.69 followed by two universal eliminations at s and t; all other axioms and rules are shared. □

Remark 8.71 (Where the collision is used). The intertwining of Theorem 8.66 relates one orbit to its shift and needs a cancellation at every visited state. The proof of Theorem 8.69 uses the collision once, in the cross absorption of Theorem 8.68(b). Cross absorption gives the avoidance (8.26); once neither orbit visits the stop of the other, the two updates agree at every state that both orbits reach, and the final induction compares two terms by a literal equation, which is the strength of the induction rule of T_F^-. The proof uses no order, no least-number principle, no predecessor and not the compiler of Theorem 8.82. Of the results above it uses Theorems 8.57 and 8.60, the case $k = 2$ of Theorem 8.61, Theorem 8.63, and the case $k = 1$ of Theorem 8.64, all proved without injectivity.

Remark 8.72 (The version of the package). `selfseed_injectivity` restates T_F^- with a complete inventory of the natural-deduction rules (hypothesis, elimination of $\bot$, introduction and elimination of $\wedge$, $\vee$, $\to$, $\forall$, $\exists$, reflexivity and substitution of equality), which is the ordinary logic of Theorem 8.56, and reproves the preparations it needs. Two of them differ from the proofs above. For $SSa \neq a$ it uses, in place of the term c_2 of Theorem 8.61, the term $H(v) := C(q(v), C(q(v), \varepsilon(v, 0)))$ with $q(v) := Z(\varepsilon(v, 1))$, which has the values $H(0) = 1$, $H(1) = 2$ and $H(v) = 0$ for $v \neq 0, 1$, the same three-cycle on $0, 1, 2$ as c_2. For $Sb = 1 \to b = 0$ it uses, in place of the case $k = 1$ of Theorem 8.64, cross absorption: if $b \neq 0$, then Theorem 8.68(b) with b and 0 in the roles of a and b gives $b = O_b(0) = 1$, so $Sb = 1 = b$, against Theorem 8.60.

Remark 8.73 (A Lean check of the abstract core). `selfseed_injectivity` ships the file `formal/StoppingOrbits.lean`, which imports nothing and is stated to compile with Lean 4.24.0; it is not part of the repository's Lean library. It declares a structure `SelfSeed.StoppingOrbits` over a type α with a zero, a successor, a two-argument update and a two-argument orbit, and with fields for $Sn \neq 0$, $Sn \neq n$, $SSn \neq n$, coverage, the three statements (8.23), the two equations (8.24), Theorem 8.67, and induction restricted to the equations $O_a(n) = O_b(n)$ between two orbits. Assuming decidable equality on α, `SelfSeed.successor_injective` derives $\forall a\, b,\ Sa = Sb \to a = b$ from these fields through absorption, the predecessor of 1 and avoidance, and `#print axioms` reports no axioms. The fields are the statements derived in T_F^- above, so the file checks the argument from Theorem 8.67 on; the term constructions, the derivations of the fields and the logic of T_F^- are not formalized.

The package also contains a compact checker of 468 normalized bytes for the iterator calculus N, which reconstructs byte for byte the existing program of `468_presentation` and does not check T_F^-.

8.3.4 With injectivity: predecessor and order

From here on the theory is T_F; by Theorem 8.70 every result below is also a theorem of T_F^-.

Theorem 8.74 (Predecessor)**.** T_F *proves* $p_*(S^2m) = Sm$*, and for the finite-case function* P *of Theorem 8.63*

$$P(0) = 0, \qquad P(Sn) = n, \qquad n = 0 \vee n = S(P(n)).$$

Proof. Injectivity applied to Theorem 8.62 gives the first equation. By coverage $n = 0$ or $n = Sm$. In the first case $P(Sn) = P(1) = 0$. In the second S^2m differs from 0 and, by injectivity and $Sm \neq 0$, from 1, so $P(S^2m) = p_*(S^2m) = Sm$. □

Write $w(a) := p_*(a)$. The constructions below use this term only through

$$w(S^3m) = S^2m, \tag{8.27}$$

that is, w removes one successor from every value that is at least 3, and $w(w(\cdot))$ removes two from every value that is at least 4.

Remark 8.75. This order of proof, coverage first and then the predecessor with corrections at 0 and 1, is that of `coverage_and_kernel`. `selfseed_scalar` uses the orbit without the guard at zero, $V(j,a) := F(j; v.C(\varepsilon(Sv,a),v))$ and $W(a) := V(a,a)$, with the invariant $a \neq 1 \to (V(j,a) = S^2j \vee S(V(j,a)) = a)$, which gives (8.27) for W and the values $W(0) = 2$, $W(1) = 2$, $W(2) = 4$. Its predecessor graph has the exceptional inputs $0, 1, 2$, and predecessor and coverage are proved together by quantifier-free induction on $(n = 0 \vee n = S(P(n))) \wedge P(Sn) = n$. Both proofs cancel a successor at the step from $S(V(m,a)) = a$ to the value of $V(m,a)$, which is where T_F uses its injectivity axiom.

For a parameter a define the capped counter and the comparison term

$$\lambda_a(v) := C(\varepsilon(v, S^2a), v), \qquad \mathrm{Cap}(n,a) := F(n; v.\lambda_a(v)), \qquad L(a,n) := \varepsilon(\mathrm{Cap}(n,a), S^2a),$$

and write $a \leq n$ for $L(a,n) = 1$. The update λ_a holds the value S^2a and increments every other value; since $S^2a \neq 0$, $\mathrm{Cap}(0,a) = 2$.

Lemma 8.76 (Capped counting and order)**.** T_F *proves*

$$\mathrm{Cap}(n,a) = S^2n \vee \mathrm{Cap}(n,a) = S^2a, \qquad \mathrm{Cap}(a,a) = S^2a, \qquad \mathrm{Cap}(Sn,Sa) = S(\mathrm{Cap}(n,a)),$$
$$L(0,n) = 1, \qquad L(Sa,0) = 0, \qquad L(Sa,Sn) = L(a,n),$$
$$a \leq Sb \leftrightarrow (a \leq b \vee a = Sb),$$

and that $\leq$ *is a discrete linear order with least element* 0 *which the successor preserves and reflects. The finite-case function* $\min(n,a)$*, with value* n *if* $n \leq a$ *and* a *otherwise, satisfies* $\mathrm{Cap}(n,a) = S^2\min(n,a)$.

Proof. The first statement is a quantifier-free induction on n with a fixed: the base is $\mathrm{Cap}(0,a) = S^20$; a state S^2a is held; and a state $S^2n \neq S^2a$ is incremented to S^2Sn. At $n = a$ both alternatives give the second statement. In particular $\mathrm{Cap}(n,0) = 2$.

Coverage gives $C(c,Sv) = S(C(c,v))$, and reflection with injectivity gives $\varepsilon(Sv,S^3a) = \varepsilon(v,S^2a)$; hence $\lambda_{Sa}(Sv) = S(\lambda_a(v))$. Also $\mathrm{Cap}(1,Sa) = \lambda_{Sa}(2) = 3$, because $S^3a \neq 2$. Induction on n for the literal equation, with a fixed, gives the third statement: the base is $3 = S2$ and the step applies λ_{Sa} and the last identity.

$L(0,n) = 1$ by $\mathrm{Cap}(n,0) = 2$; $L(Sa,0) = \varepsilon(2,S^3a) = 0$; and $L(Sa,Sn) = \varepsilon(S\,\mathrm{Cap}(n,a), SS^2a) = L(a,n)$.

For the equivalence, let $\mathrm{Cap}(Sb, a) = S^2a$. If $\mathrm{Cap}(b, a) = S^2a$, then $a \le b$. Otherwise $\mathrm{Cap}(b, a) = S^2b$ by the first statement, the next value is S^3b, and $S^3b = S^2a$ gives $a = Sb$. Conversely a reached cap persists, and $\mathrm{Cap}(Sb, Sb) = S^2Sb$.

Linearity. Fix a and induct on n in $a \le n \vee n \le a$. The base is $0 \le a$. In the step, $a \le n$ persists. If $n \le a$, split $a = 0$ or $a = Sb$ by coverage. If $a = 0$, then $\mathrm{Cap}(0, n) = 2 = S^2n$ gives $n = 0$ and $a \le Sn$. If $a = Sb$, the equivalence gives $n \le b$, and then $Sn \le Sb = a$, or $n = a$, and then $a \le Sn$.

Irreflexivity and descent. $Sn \not\le n$ by induction on the literal equation $L(Sn, n) = 0$. $Sn \le a \to n \le a$: by coverage $a = 0$ contradicts $L(Sn, 0) = 0$, and $a = Sb$ gives $n \le b$ and then $n \le Sb$ by persistence.

Antisymmetry. Fix a and induct on n in $(a \le n \wedge n \le a) \to a = n$. At Sn the equivalence splits $a \le Sn$ into $a = Sn$, or $a \le n$; in the latter case $Sn \le a$ gives $n \le a$, the hypothesis gives $a = n$, and $Sn \le n$ is a contradiction.

Transitivity. Fix a, b and induct on c in $(a \le b \wedge b \le c) \to a \le c$. At Sc the equivalence splits $b \le Sc$ into $b \le c$, where the hypothesis and persistence apply, or $b = Sc$, where substitution applies.

For the last statement: if $n \le a$, the first statement gives $\mathrm{Cap}(n, a) = S^2n$ directly, or $\mathrm{Cap}(n, a) = S^2a$, which is $a \le n$ and gives $a = n$ by antisymmetry. If $n \not\le a$, then $a \le n$ by linearity, which is $\mathrm{Cap}(n, a) = S^2a$. □

Lemma 8.77 (Bounded selection)**.** *For Boolean c, p, q let $\mu(c, p, q) := (c \wedge_B p) \vee_B (\neg_B c \wedge_B q)$, and let*

$$\Theta(b, c, a, d) := F\big(b;\, v.\, C(\mu(c, \varepsilon(v, S^2a), \varepsilon(v, S^2d)), v)\big), \qquad \mathrm{Sel}(b, c, a, d) := w(w(\Theta(b, c, a, d))).$$

If $a = S^2a_0$ and $a \le b$, then $c = 1 \to \mathrm{Sel}(b, c, a, d) = a$. If $d = S^2d_0$ and $d \le b$, then $c = 0 \to \mathrm{Sel}(b, c, a, d) = d$.

Proof. If $c = 1$ the body equals $\lambda_a(v)$ for a generic v, so $\Theta(b, c, a, d) = \mathrm{Cap}(b, a)$ by induction on the equation between the two recursors at a counter variable; similarly $\Theta(b, c, a, d) = \mathrm{Cap}(b, d)$ if $c = 0$. Under the hypotheses the selected value is $S^2a = S^4a_0$ or $S^2d = S^4d_0$, and two applications of (8.27) remove two successors. □

8.3.5 Bounded iteration of finite-case functions

Write $e(x) := S^2x$. On an input $u = e(x)$ the finite cases

$$u = 2 :\ 0; \qquad u = 3 :\ 1; \qquad u \ne 2, 3 :\ w(w(u)) \tag{8.28}$$

return x: in the last case $u = S^4x'$ by coverage. For $B = e(b)$ let $\mathrm{clip}_B(v) := w(w(\mathrm{Cap}(v, B)))$. For $v \ge 2$, Theorem 8.76 and (8.27) give $\mathrm{clip}_B(v) = \min(v, B)$ and $2 \le \mathrm{clip}_B(v) \le B$. The term is applied only to arguments $e(t)$ or to values in $[2, B]$.

Proposition 8.78 (Bounded iteration)**.** *Let $g(\vec{p})$ and $h(v, \vec{p})$ be finite-case functions. For every cap variable b there is a finite-case function $\mathrm{It}_b(n, \vec{p})$, obtained from terms of the original language by the cases* (8.28)*, such that T_F proves*

$$\mathrm{It}_b(0, \vec{p}) = \min(g(\vec{p}), b), \qquad \mathrm{It}_b(Sn, \vec{p}) = \min\big(h(\mathrm{It}_b(n, \vec{p}), \vec{p}), b\big), \qquad \mathrm{It}_b(n, \vec{p}) \le b.$$

Every occurrence of F in the construction has a body that is a term of the original language.

Proof. Put $B := e(b)$. Let g have the cases (A_i, t_i). Encode and clip each candidate as $\mathrm{Cap}(t_i, b) = e(\min(t_i, b))$, which lies in $[2, B]$ for all inputs, and fold the cases by nested

$\mathrm{Sel}(B, \chi_{A_i}, \cdot, \cdot)$ with the default value 2. By Theorem 8.77 this gives a term $a_B(\vec{p})$ in $[2, B]$ with $a_B(\vec{p}) = e(\min(g(\vec{p}), b))$.

For the update, compose the cases of h with the cases (8.28) for its state argument, refine the guards, and encode, clip and select in the same way. The result is a term $h_B(u, \vec{p})$ of the original language with $2 \le h_B(u, \vec{p}) \le B$ for every u, because every candidate is a value of $\mathrm{Cap}(\cdot, b)$, and

$$h_B(e(v), \vec{p}) = e(\min(h(v, \vec{p}), b)).$$

All guard tests are Boolean terms; nothing depends on an order of evaluation.

The seed of the recursor is tied to the body, so the initial value a_B is entered through two transient states. Let

$$H_B(u) := \mathrm{Sel}\Big(S^2B, \varepsilon(u, 0), B,\ \mathrm{Sel}(S^2B, \varepsilon(u, SB), S^2B,\ \mathrm{Sel}(S^2B, \varepsilon(u, S^2B), a_B, h_B(u)))\Big).$$

All candidates lie in $[2, S^2B]$. The sentinels SB and S^2B are distinct and exceed B, so

$$H_B(0) = B, \qquad H_B(SB) = S^2B, \qquad H_B(S^2B) = a_B, \qquad 2 \le u \le B \to H_B(u) = h_B(u),$$

and therefore $F(0; u.H_B) = SB$, $F(1; u.H_B) = S^2B$, $F(2; u.H_B) = a_B$. Put $Y_b(n, \vec{p}) := F(e(n); u.H_B(u))$. Quantifier-free induction gives $2 \le Y_b(n, \vec{p}) \le B$, $Y_b(0, \vec{p}) = a_B$ and $Y_b(Sn, \vec{p}) = h_B(Y_b(n, \vec{p}), \vec{p})$. Applying the cases (8.28) to Y_b gives It_b. □

The proposition gives initialized, capped iteration of finite-case functions whose update sees the state only. Each use below supplies a cap that is a term of the original language in one variable and a proof that clipping does not occur on the intended arguments.

8.3.6 Arithmetic by finite-case graphs

All functions introduced in this subsection are finite-case, every induction is quantifier-free after Theorem 8.59, and Theorem 8.78 is used whenever an update refers to one of them.

Lemma 8.79 (Difference and doubling)**.** *There is a finite-case function d with*

$$d(x, 0) = x, \qquad d(x, Sn) = P(d(x, n)), \qquad d(x, n) \le x, \qquad d(Sx, Sy) = d(x, y), \qquad d(n, n) = 0.$$

The term $Q(n) := F(n; v.S^2v)$ and the finite-case function $\mathrm{db}(n) := P(P(P(Q(n))))$ satisfy

$$Q(0) = 3, \quad Q(Sn) = S^2Q(n), \qquad \mathrm{db}(0) = 0, \quad \mathrm{db}(Sn) = S^2\,\mathrm{db}(n), \quad Q(n) = S^3\,\mathrm{db}(n),$$

and db *is monotone with $n \le \mathrm{db}(n)$.*

Proof. Let $d(x, n)$ be Theorem 8.78 with initial value x, update P and cap x. Since $P(v) \le v$, clipping is never active, which gives the first three statements. $d(Sx, Sy) = d(x, y)$ by induction on y with x fixed: the base is $P(Sx) = x$ and the step applies P. Then $d(n, n) = 0$ by induction. The equations for Q are (8.21). By induction $Q(n) \ge 3$, so the functional coverage $n = 0 \vee n = S(P(n))$ cancels three successors and gives the equations for db. $n \le \mathrm{db}(n)$ is a quantifier-free induction, and monotonicity is an induction on the larger argument in $a \le b \to \mathrm{db}(a) \le \mathrm{db}(b)$ with the equivalence of Theorem 8.76. □

Lemma 8.80 (Addition)**.** *There is a finite-case function $+$ with $0+a = a$, $Sn+a = S(n+a)$, commutative, associative and monotone, and with $n + a < Q(b)$ whenever $n, a \le b$.*

Proof. For a cap c let $A_c(n,a)$ be Theorem 8.78 with initial value a and update S, and write $s_c(v) := \min(Sv, c)$:

$$A_c(0,a) = \min(a,c), \qquad A_c(Sn,a) = s_c(A_c(n,a)).$$

For $a \le c$, induction on the equation gives the seed commutation $A_c(n, s_c(a)) = s_c(A_c(n,a))$: both sides have the same value at 0 and the same update. The minimum satisfies $\min(0,c) = 0$ and $\min(Sn,c) = s_c(\min(n,c))$ by the order laws, so $A_c(n,0) = \min(n,c)$ by uniqueness of iteration.

Fix $a \le c$ and induct on n in $n \le c \to A_c(n,a) = A_c(a,n)$. The base is the last identity. If $Sn \le c$, the step applies s_c on the left and the seed commutation on the right. Since the update increases and s_c preserves order, A_c is monotone in the counter (induction on the larger counter) and in the initial value within $[0,c]$ (induction on the common counter).

Fix an input bound b and take the cap $c := Q(b) = S^3\,\mathrm{db}(b)$, a term of the original language. Induct on n in $n \le b \to A_c(n,n) = \mathrm{db}(n)$: at a successor the seed commutation gives $A_c(Sn,Sn) = s_c(s_c(A_c(n,n)))$, and since $\mathrm{db}(Sn) \le \mathrm{db}(b) < c$ neither step is clipped, so the value is $S^2\,\mathrm{db}(n)$. By monotonicity

$$n, a \le b \to A_{Q(b)}(n,a) \le A_{Q(b)}(b,b) = \mathrm{db}(b) < Q(b),$$

so there is no clipping on bounded inputs. For the caps of two input bounds b, b', induction on n restricted by $n \le b, b'$ shows that the results agree for $a \le b, b'$: the values at 0 are a and both steps are unclipped successors.

Take b to be the first largest of the two arguments, by finitely many order cases; this defines the finite-case function $n + a$. Coherence across the caps proves $0 + a = a$ and $Sn + a = S(n + a)$, bounded commutativity proves commutativity, and associativity and monotonicity are quantifier-free inductions with fixed parameters. □

Lemma 8.81 (Multiplication and pairing)**.** *Let $E(n) := F(n; v.Q(v))$, so that $E(0) = 4$ and $E(Sn) = Q(E(n))$. There are finite-case functions $\cdot$, κ, π_0, π_1 with*

$$n \cdot a \le E(Q(b)) \text{ for } n, a \le b, \qquad \kappa(x,y) := (x+y)^2 + x \le Q(E(Q(Q(b)))) \text{ for } x, y \le b,$$
$$\pi_0(\kappa(x,y)) = x, \qquad \pi_1(\kappa(x,y)) = y, \qquad \pi_0(z) \le z, \qquad \pi_1(z) \le z,$$

and the semiring identities for $+$ and $\cdot$.

Proof. E is monotone, $E(n) \ge n$ and $E(Sn) = \mathrm{db}(E(n)) + 3$. Fix $n, a \le b$ and apply Theorem 8.78 with initial value 0, update $v \mapsto v + a$ and cap $E(Q(b))$. For the state v_j at a stage $j \le b$ the invariant $v_j \le E(b + j)$ holds: the base is immediate, and

$$v_j + a \le E(b+j) + b \le \mathrm{db}(E(b+j)) < E(S(b+j)),$$

using $E(b+j) \ge b$. Since $b + j \le \mathrm{db}(b) < Q(b)$, the stage bound is at most the cap, so clipping is inactive. Cap coherence as for addition gives a finite-case multiplication with the bound $E(Q(b))$; its equations at 0 and at a successor give the semiring identities by quantifier-free induction, in particular $w^2 + w < (Sw)^2$.

κ is finite-case with the stated bound, which is monotone and at least its argument, and both arguments of a code are at most the code. For the root, apply Theorem 8.78 with initial value 0, the update that sends u to Su if $(Su)^2 \le z$ and to u otherwise, the cap z and the counter z. The update preserves $u \le z$ and $u^2 \le z$, and the quantifier-free invariant "$u_j = j$ or the update fixes u_j" shows that the state at $j = z$ is fixed: if it equals z, then $(Sz)^2 > z$. So the result $r(z)$ satisfies $r(z)^2 \le z < (Sr(z))^2$, and it is unique with this property by monotonicity of squares. Put

$$\pi_0(z) := d(z, r(z)^2), \qquad \pi_1(z) := d(r(z), \pi_0(z)).$$

The identity $d(x+y,y) = x$ follows by induction on y from $d(Sx,Sy) = d(x,y)$ and the addition equations. For $z = \kappa(x,y)$ and $w := x+y$, the inequalities $w^2 \le z \le w^2+w < (Sw)^2$ give $r(z) = w$, hence the projection equations. The projections are total on all arguments and bounded by their argument, since a difference is bounded by its first argument. □

Theorem 8.82 (Primitive recursion)**.** *For every primitive recursive description f there are a finite-case function f^* in T_F, proofs of the defining equations of f for f^*, and a term M_f in one variable, monotone and with $b \le M_f(b)$, such that $\vec{x} \le b \to f^*(\vec{x}) \le M_f(b)$ is provable.*

Proof. By recursion on f. Zero, successor and projections have these data with the bound Q. For a composition $f = h(g_1,\dots,g_m)$ take a common refinement of the cases and the bound $M_h \circ M_{g_1} \circ \cdots \circ M_{g_m}$. Nullary descriptions have numeral values and constant shifts of b as bounds.

Let $f(0,\vec{p}) = g(\vec{p})$ and $f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p})$. Write K_κ for the bound of κ in Theorem 8.81 and put

$$\Delta := K_\kappa \circ M_h \circ M_g \circ Q, \qquad \Upsilon(t) := F(t; v.\Delta(v)), \qquad \beta(b) := \Upsilon(SQ(b)).$$

Each factor is monotone and at least its argument, and $\Delta(v) \ge Sv$; so Υ is monotone with $\Upsilon(t) \ge St$, and β is monotone with $\beta(b) \ge b$, by quantifier-free inductions. The state update is the finite-case function

$$J(z,\vec{p}) := \kappa\big(S\pi_0(z),\, h^*(\pi_0(z),\pi_1(z),\vec{p})\big),$$

with the initial value $\kappa(0, g^*(\vec{p}))$. Apply Theorem 8.78 to them with the cap $\beta(b)$ and let z_j be the state at the stage j. For $n,\vec{p} \le b$ and $j \le b$,

$$z_j \le \Upsilon(Sb+j). \tag{8.29}$$

The seed satisfies $\kappa(0,g^*(\vec{p})) \le K_\kappa(M_g(b)) \le \Delta(b) \le \Delta(\Upsilon(b)) = \Upsilon(Sb)$. For the step write $s := \Upsilon(Sb+j)$. Then $s \ge b$, both projections of z_j are at most s, the next counter is at most $Ss \le Q(s)$ and the next value is at most $M_h(M_g(Q(s)))$, so $J(z_j,\vec{p}) \le \Delta(s) = \Upsilon(S(Sb+j))$. Since $j \le b$, the argument $Sb+j$ is at most $S\,\mathrm{db}(b) \le SQ(b)$, so every stage bound is at most $\beta(b)$ and no clipping occurs. The invariant (8.29) is quantifier-free in the extension.

By the projection equations and a quantifier-free induction guarded by $j \le b$, the first component of z_j is j, and the second component satisfies the base equation and, for $Sj \le b$, the step equation. Two caps give the same run up to their common bound on the counter: the seeds agree and the unclipped updates are identical. Take b to be the first largest among $n,\vec{p}$ and define $f^*(n,\vec{p}) := \pi_1(z_n)$. For any larger input bound, cap coherence identifies this value with the run at that cap, so $f^*(n,\vec{p}) \le \beta(b)$ for every valid b; put $M_f := \beta$. Cap coherence proves the defining equations also when a successor input changes the first largest coordinate.

Theorem 8.78 expands every use of g^*, h^*, κ, π_i inside the update into terms of the original language, encoded cases and bounded selections. There is no evaluator for codes of descriptions and no recursor over functions defined by graphs. □

Theorem 8.83 (Bi-interpretation)**.** T_F *and* iPRA *are bi-interpretable, with whole domains, actual equality, the logic translated identically and the identity as isomorphism of both composites. They are equiconsistent.*

Proof. In iPRA keep 0, S and equality, interpret ε as the primitive recursive characteristic of equality, and each recursor with body h by primitive recursion with initial value $S(h(0))$ and a step h that ignores the counter. All axioms of T_F and its induction rule hold. Conversely interpret each description by the finite-case function of Theorem 8.82; totality, single-valuedness, the defining equations and the quantifier-free induction instances are proved in T_F by Theorem 8.59, and injectivity and separation are axioms.

On the side of iPRA, recursion on a description reduces the identification of a function with its double interpretation to uniqueness from the base and step equations, which is a quantifier-free induction. On the side of T_F, 0 and S are unchanged; the two characteristics of equality are two-valued and reflect equality, so they agree by cases; and for a recursor with body h, the earlier stage identifies the body, both functions satisfy (8.21) for it, and induction on the equation between them identifies them, with Theorem 8.59 where graph-defined functions occur. Induction on formulas extends this to all formulas, and falsity is preserved. □

Corollary 8.84. T_F^- *and* iPRA *are bi-interpretable with the data of Theorem 8.83: whole domains, actual equality, the logic translated identically and the identity as isomorphism of both composites. They are equiconsistent.*

Proof. Compose the two interpretations of Theorem 8.83 with the identity interpretations of Theorem 8.70. The axioms of T_F^- are axioms of T_F, so their translations are provable in iPRA. The translations of the axioms of iPRA and the equivalences of both round trips are proved in T_F, hence in T_F^- by Theorem 8.70; the round trips on the side of iPRA are those of Theorem 8.83, since the formula translations are unchanged. Preservation of $\bot$ gives equiconsistency. □

`selfseed_injectivity` obtains Theorem 8.84 in this way and does not reprove the compiler of Theorem 8.82.

8.4 Three observations on the native core

The packages of this chapter contain three results about the calculus N on iterator terms that do not concern a seeded recursor. They are recorded here from `coverage_and_kernel`, `transient_seed` and `monus_and_neutral_major`. The second coincides with Theorem 2.13(ii) and is recorded by a reference; the first and the third differ from the results of Section 2.3.2 and Section 5.5.1 with similar names, and the remarks below relate them.

8.4.1 One native binder over a literal equation

In this subsection formulas are read in iPRA through Section 1.2 and Theorem 1.119, and sel, the predecessor P of Theorem 1.65, the characteristic terms of Theorem 1.86 and the pairing of Theorem 1.102 are available. A formula is called Σ_2^{qf} if it has the form $\exists x\,\forall y\,\theta(x,y,\vec{p})$ with θ quantifier-free, and Π_1^{qf} if it has the form $\forall y\,\theta(y,\vec{p})$. These are classes of prenex formulas; no prenex normal form theorem for intuitionistic formulas is used.

Proposition 8.85 (One binder)**.** *Up to provable equivalence, the native formulas $D_{x,z}E(x,z,\vec{p})$ with E a literal equation are exactly the Σ_2^{qf} formulas, and the native formulas $Q_zE(z,\vec{p})$ with E a literal equation are exactly the Π_1^{qf} formulas.*

Proof. The reading of $D_{x,z}E$ is $\exists x\,(E(x,0)\to\forall z\,E(x,z))$, which is equivalent to $\exists x\,\forall z\,(E(x,0)\to E(x,z))$ because z does not occur in $E(x,0)$; the matrix is quantifier-free. Without the existential quantifier this is the statement for Q_z.

Conversely let $r(x,y,\vec{p})$ be a characteristic term with $r=0\leftrightarrow\theta$, and let $E(x,z,\vec{p})$ be the literal equation

$$\mathrm{sel}(z,0,r(x,P(z),\vec{p}))=0.$$

Its instance at $z=0$ is provable, and its instance at $z=Sy$ is equivalent to $r(x,y,\vec{p})=0$ by $P(Sy)=y$. By coverage, $\forall z\,E(x,z,\vec{p})\leftrightarrow\forall y\,\theta(x,y,\vec{p})$, and the provable antecedent $E(x,0,\vec{p})$ drops out of the reading of $D_{x,z}E$. □

Corollary 8.86. *If rule* QI *is extended to matrices that consist of one $D_{x,y}$ over a literal equation, the extended calculus derives every induction instance for Σ_2^{qf} formulas. If it is extended to matrices Q_yE with E a literal equation, it derives every induction instance for Π_1^{qf} formulas.*

Proof. Let $B(n,\vec{p})$ be the native formula of Theorem 8.85 for the given formula, and let $\Gamma \vdash B(0,\vec{p})$ and $\Gamma, B(z,\vec{p}) \vdash B(Sz,\vec{p})$. The extended rule gives $Q_nB(n,\vec{p})$, and QE with the base gives $B(t,\vec{p})$ for every term t. The equivalence of the proposition transports the derivation to the given formula. □

A guard "at most one binder" is therefore not equivalent to the guard "literal equation" unless an elimination or conservativity theorem for the additional induction instances is proved. No statement on the strength of the extended calculus relative to iPRA is made.

For a native formula let $\delta(s = t) := 0$ and $\delta(Q_yA) := \delta(D_{x,y}A) := 1 + \delta(A)$. For a fresh z let G_zA be A with every equation $s = t$ replaced by $\mathrm{sel}(z,0,s) = \mathrm{sel}(z,0,t)$. Then $\delta(G_zA) = \delta(A)$, G_0A is provable, and $G_{Su}A \leftrightarrow A$: for equations by computation, and for a binder by induction on the formula, with the witness 0 for $D_{x,y}$.

Theorem 8.87 (Alternating prefixes)**.** *For every $k \geq 0$, every formula $\exists x_1 \forall y_1 \cdots \exists x_k \forall y_k\, \theta(\vec{x},\vec{y},\vec{p})$ with θ quantifier-free is provably equivalent in* iPRA *to the reading of a native formula of depth $\delta \leq k$. Consequently an extension of* QI *to all matrices of depth at most k derives induction for this class.*

Proof. For $k = 0$ take the characteristic equation. If $A(x,y,\vec{p})$ translates the remaining prefix at depth at most $k-1$, put $\mathcal{B}(A) := D_{x,z}\, G_zA(x,P(z),\vec{p})$. Its matrix at $z = 0$ is provable and its matrix at $z = Sy$ is equivalent to $A(x,y,\vec{p})$, so by coverage $\mathcal{B}(A) \leftrightarrow \exists x \forall y\, A(x,y,\vec{p})$, and $\delta(\mathcal{B}(A)) \leq k$. The induction statement is the argument of Theorem 8.86. □

Blocks of several variables are packed by pairing, and dummy blocks can be added because the domain is inhabited; so the inclusion covers the prenex Σ_{2k} formulas and, for $k \geq 1$, the prenex Π_{2k-1} formulas. At depth greater than one only this inclusion is claimed: the reading of a nested native formula contains implications with quantified antecedents, and moving quantifiers through them is not valid intuitionistically.

Remark 8.88 (Relation to the prefix representation)*.* Theorems 5.42 and 5.46 concern a different statement with a similar name: every Q-free native formula is, uniquely and stably under substitution, a record $D^k(s = t)$, and a clause calculus on such records is equivalent to N. That is a statement about syntax and derivations. Theorems 8.85 and 8.87 concern what native formulas express under the reading in iPRA. The two fit together: the formulas built in the proof of Theorem 8.87, and those for the Σ_2^{qf} case of Theorem 8.85, use $D_{,}$, an equation and the gate G_z only, so they are Q-free and are records $D^j(s = t)$ with $j \leq k$ in the sense of Theorem 5.42.

8.4.2 The case rule at a literal equation cannot be deleted

Remark 8.89 (The atomic case introduction)*.* `transient_seed` considers the calculus on iterator terms with paired introduction by cases and the progress rule on $D_{x,y}$ of Theorem 8.26, with the introduction restricted so that its local assumption is $E(t,z)$ whenever the matrix is a literal equation E, and $B(t,0)$ otherwise, and shows that the restricted calculus does not derive $D_{x,y}(y = S0)$, which the unrestricted calculus derives. At a literal equation the restricted introduction is the progress rule DP, so the restricted calculus is that of Theorem 2.13(ii) with transport in a formula template, and the statement and its proof, by the reading of an atomic $D_{x,y}E$ as $\exists x \forall y\, (E(x,y) \to E(x,Sy))$, are given there. Theorem 3.10 is the same statement

for the root-determined introduction DI_{root}, and Theorems 2.12 and 2.13(i) are the companion results for dropping DI altogether and for keeping QI in place of progress.

Under the standard reading the same formula is true, since its zero instance is false. Hence the two readings of a $D_{x,y}$ over an equation, closure and progress, cannot be exchanged by padding the equation into a compound formula without losing one direction of the formula round trip.

8.4.3 Transport from the literal zero alone

Proposition 8.90. *In the calculus on iterator terms with formula transport (Theorem 8.26) and no reflexivity axiom, replace transport by the rule: from* $\Gamma \vdash 0 = t$ *and* $\Gamma \vdash F(0)$ *infer* $\Gamma \vdash F(t)$*. The resulting calculus does not derive* $0 = 0$*.*

Proof. Interpret terms in $\{0, 1\}$ with $S0 = S1 = 1$ and every iterator term equal to 1, read an equation $s = t$ as "the value of s is 1", and read Q_y, $D_{x,y}$ by the clauses of Section 1.2. Every computation equation holds, because its left side is an iterator. No equation $0 = t$ holds, so the restricted rule is sound. Every element is 0 or a successor, and induction holds for every predicate on the domain: the base gives the value at 0 and the step at 0 gives the value at 1. So DI by cases and the progress rule are sound, as are HYP, QE and DE. The equation $0 = 0$ does not hold. □

The relation used in the proof is not equality. With unrestricted transport $0 = 0$ follows from the computation $I(0, 0; v.\, 0) = 0$ used as both premises with the template $w = 0$. The proposition does not apply to a calculus that has $0 = 0$ as an axiom or as an initial context entry.

Remark 8.91 (The opposite restriction of the source). The proposition restricts transport to the source 0, which is the complement of the restriction $s \not\equiv 0$ of Section 2.3.2. That restriction is harmless: reflexivity and the unrestricted rule TR are derivable from I0 and TR^- (Theorem 2.25), and for transport in a formula template the calculus Z, in which a source 0 is served by QE, is equivalent to N (Theorem 2.35). The restriction $s \not\equiv 0$ is harmful only inside the strictly guarded composition of Theorem 3.37, where a zero-left hypothesis is inert. Thus the sources other than 0 suffice for equality in N, and the source 0 alone does not.

8.5 Checker sizes

All nine packages report sizes of complete standalone checkers under the protocol of Section 1.10. Table 8.2 lists them as stated, with the control each package measures against. Only the programs of the first three packages that are marked R check the successor-seeded calculus N_R, and only the two programs of `selfseed_injectivity` check the self-seeded theories T_F^- and T_F. All other programs check the calculus N on iterator terms; no package reports a checker for $\mathsf{T}_{B\varepsilon}$, T_{Bd} or N_{Bd}, and the sizes of the iterator programs are not sizes of those theories.

Table 8.2: Checker sizes as stated by the packages (tokens, AST nodes, normalized bytes).

Package	Program	Tokens	AST
`scalar_successor`	`checker.py` (R)	279	319
	`checker_bytes.py` (R)	279	321
	`checker_ast.py` (R)	282	314

Package	Program	Tokens	AST
	control: generated assumptions, main	281	318
	control: generated assumptions, AST profile	283	315
	control: guarded elimination, token profile	281	325
transient_seed	checker_sr_bytes.py (R)	284	322
	checker_bytes.py (iterator)	282	320
	checker_structure.py (iterator)	280	311
	checker_joint.py (iterator, predecessor packets)	280	314
	control: generated assumptions, main	281	318
	control: generated assumptions, AST profile	283	315
successor_seed_completion	checker_direct.py (R)	283	320
	checker.py (R)	284	322
	checker_ast.py (R)	282	313
	checker_ast_total.py (R)	287	314
	control: prior generated-assumption checker	281	318
	control: prior AST-oriented checker	283	315
bounded_initializer	checker.py	266	310
	checker_compact.py	266	312
	checker_ast.py	267	298
	control: byte leader	269	311
	control: AST and token leader	268	300
monus_and_neutral_major	checker.py	263	307
	checker_bytes.py	263	309
	checker_joint.py	261	295
	control: certified zero, explicit list test	264	308
	control: certified zero, compact list test	264	310
	control: certified zero, generic traversal	262	296
selfseed_scalar	local_argument.py	267	309
	local_argument_bytes.py	267	311
	local_loop_joint.py	263	291
	local_loop_joint_separate.py	262	293
	control: baseline_checker.py	264	308
	control: baseline_checker_bytes.py	264	310
	control: baseline_checker_joint.py	262	296
scope_and_coverage	checker_compact.py	266	310
	checker_explicit.py	266	308
	control: neutral major, global floor (byte incumbent)	263	309
	control: guarded major, lexical floor	267	311
	control: neutral major, lexical floor, conditional dispatch	266	310
coverage_and_kernel	checker.py	266	308
	checker_bytes.py	266	310
	checker_ast.py	263	289
	control: byte leader	263	309
	control: token leader	261	295
	control: AST leader	263	291
selfseed_injectivity	checker_without_injectivity.py (T_F^-)	1897	2192

In every package of iterator or R programs the minima over the three coordinates are attained by different programs, and each package states that no single one of its programs is below all three coordinates of its control, with one exception: `checker_joint.py` of `transient_seed`, at 280/314/684, is below 281/315/685, the coordinatewise minimum of its two controls. The packages state that these minima are sizes attained in a fixed corpus of earlier checkers and not lower bounds. The checker of `selfseed_injectivity` for T_F^- is below its matched control in all three coordinates, by 50 tokens, 55 AST nodes and 198 bytes, because the two programs differ only by the branch for the injectivity axiom of T_F; the theorem sets agree by Theorem 8.70.

Self-seeded checkers. The two programs of `selfseed_injectivity` are readable natural-deduction checkers, measured on the whole module like all others, with the input contract: one JSON object with exactly the keys `goal` (a closed formula) and `proof` (a proof tree), validated completely by the program. Terms and formulas are arrays with de Bruijn indices; the counter of F lies outside its state binder. The proof trees have one node for each rule of ordinary intuitionistic natural deduction with equality, one node for induction on a literal equation, and axiom nodes for the two computation equations, range, reflection and separation; the control alone also accepts an axiom node for injectivity. Both programs check the eigenvariable conditions of universal introduction and existential elimination and the freshness of the induction variable. The package tests them on 35 certificates and gives a written soundness and completeness argument for the implemented calculus; it contains no certificate of Theorem 8.69, so the control accepts the primitive injectivity certificate that the other program rejects.

Successor-seeded checkers. The R programs derive from the checker with generated assumptions of an earlier draft: a certificate is a JSON array whose last element is the closed goal and whose earlier four-field packets are consumed from the end; a computation equation enters as a local assumption (Theorem 8.26). An arithmetic packet supplies the data $[n, a, h]$ and a flag, and the checker itself builds the counter 0 or Sn of the generated equation, so no shape test on a supplied counter is needed. The same change applied to the iterator gives the matched iterator programs of `transient_seed`, whose joint profile uses a changed packet format for predecessor computations. The AST profiles accept additional raw aliases and reject malformed nonempty strings only through the recursion limit of the interpreter; `checker_ast_total.py` rejects them directly. `transient_seed` ships kernel certificates for the Boolean decoder (45 commands) and for a restricted implication (149 commands); `successor_seed_completion` states that the full predecessor derivation has not been emitted as a kernel certificate.

Iterator checkers in the other packages. The programs of `bounded_initializer` compose an arity encoding of the raw syntax with a predecessor supply. Those of `monus_and_neutral_major` implement its neutral-major certificate protocol for N, in which every instruction proves a major premise and a derivable truth serves where none is needed; the protocol is not described in this chapter. Those of `selfseed_scalar`, `scope_and_coverage` and `coverage_and_kernel` compose lexical proof scopes, the arity encoding, the certified-zero dispatch and, in the last two, neutral majors; the loop variants keep contexts as immutable tuples. `selfseed_scalar` records two failed variants: a loop with in-place mutation of the context list accepts a certificate of $0 = S0$ by leaking an assumption of a subproof, and a variant with opaque objects as fresh parameters accepts a certificate of $0 = S0$ because a floating-point leaf 3.0 is treated

as a rigid parameter while comparing equal to the bound index 3; the stricter repair of the latter loses the canonical certificate of universal reflexivity.

8.6 Further questions

No result of this chapter has a Lean formalization recorded in the repository; the packages present written proofs with finite numerical checks of the defining terms on standard numbers. Outside the repository's Lean library, the abstract core of Theorem 8.69 is checked in the Lean file of `selfseed_injectivity` described in Theorem 8.73.

The question whether T_F^- proves injectivity of the successor, left open by `selfseed_scalar`, `scope_and_coverage` and `coverage_and_kernel`, is answered by Theorem 8.69, and its bi-interpretability with iPRA by Theorem 8.84.

Question 8.92 (Certificates and formalization of the axiom deletion)**.** `selfseed_injectivity` contains no certificate of Theorem 8.69 for its checker of T_F^-, and its Lean file assumes the statements of Theorem 8.67, coverage, Theorem 8.60, the case $k = 2$ of Theorem 8.61 and the computation of the stopping update as fields. The size of a complete natural-deduction certificate of injectivity in T_F^-, and a formalization that derives the fields from the terms of Section 8.3.3 in a deep embedding of T_F^-, are not given.

Question 8.93 (A native kernel for the self-seeded terms)**.** `selfseed_injectivity` names as a next target a native logical interface for the terms $x \mid 0 \mid St \mid \varepsilon(t,t) \mid F(n; v.h)$, in which the equality test and every logical rule are charged in one small kernel, in the manner of N for the iterator. Such a calculus, and its relation to T_F^-, are not given.

Question 8.94 (Uniform rigidity)**.** Theorem 8.61 is a family of derivations indexed by a fixed integer. Is there a single derivation with a variable index, without a general compiler for primitive recursion in T_F^-? For Theorem 8.64 the uniform statement $Sx = Sy \to x = y$ is Theorem 8.69, a single derivation that uses only the cases $k = 1$ of Theorem 8.64 and $k = 2$ of Theorem 8.61.

Question 8.95 (Shorter predecessor terms)**.** Is there a substantially shorter predecessor term for the successor-seeded recursor than those of Theorems 8.19 and 8.20, for instance a reset cycle that needs no Boolean comparator? The three packages prove existence and correctness on the whole domain and make no claim on size.

Question 8.96 (Other seed maps)**.** For which maps σ in place of the successor does the recursor with $R(0, a; u.h) = \sigma(a)$ recover arbitrary initialization from its two equations and induction for literal equations? A criterion has to provide a proved inverse and not assume one. Related: is there a general class of finite reset mechanisms for which the transient-seed method works, and is the correction of a capped predecessor at finitely many arguments an instance of a general repair principle?

Question 8.97 (Cancellation on finite-valued terms)**.** In N_R and N_{Bd} a restricted implication and contextual case analysis are obtained from cancellation of the successor on two-valued terms, before a selector exists (Theorems 8.9 and 8.54). Which other weak term languages admit this route to the full logical interface?

Question 8.98 (Tied seeds without an additional operation)**.** Theorem 8.30 excludes terms for addition, selection and pairing. It does not exclude interpretations of iPRA by relations, quotients or several coordinates in a theory on the terms $x \mid 0 \mid B(n; u, v.h)$ alone; Theorem 8.43 needs ε and Theorem 8.51 needs d and the coverage axiom. An invariant that is stable under such interpretations is not known from the packages of this chapter.

Question 8.99 (Fewer axioms for the equality test)**.** T_{Bd} replaces the range and reflection axioms of $\mathsf{T}_{B\varepsilon}$ by two recursive equations, at the price of coverage as an axiom. Can range and reflection be replaced by recursive equations from which coverage is still derived? `monus_and_neutral_major` investigates a basis with three equations for the minimum, in which parity$(\min(2x, 2y + 1))$ has the standard meaning of a comparator; the derivation of the equations of d and of reflection on the whole domain from that basis, without induction over a shifted parameter, is left open there.

Question 8.100 (Internal majorants and sizes of the compilers)**.** The converse direction of Theorem 8.44 produces a single term for a function with a majorant that is valid in the standard model; a proof in $\mathsf{T}_{B\varepsilon}$ that this term equals the graph-defined function needs a proof of the majorant in the theory. Bounds on the size of the terms and proofs produced by Theorems 8.40, 8.42, 8.78 and 8.82, on the branching of the case expansions for nested terms, and on the cost of translating the graph-based proofs into a native calculus are not given.

Question 8.101 (Extended progress)**.** Theorems 8.86 and 8.87 show that an extension of QI to matrices with binders derives induction for prenex classes. Whether such an extension is conservative over N, or strictly stronger, is not decided; at depth greater than one only the inclusion of Theorem 8.87 is proved.

Question 8.102 (Variants of the two obstructions)**.** Theorem 2.13(ii), in the form of Theorem 8.89, refutes the deletion of case introduction at literal equations for the two displayed readings; it does not exclude a calculus in which a $D_{x,y}$ over an equation is an auxiliary judgment and the object formulas are padded. Theorem 8.90 does not exclude transport from the literal zero together with a reflexivity axiom. Neither variant is decided in the packages of this chapter; calculi with a single introduction rule are the subject of `witness_rule` and `guarded_core` (Section 2.2.4 and Theorem 3.10), and restrictions on the source of transport of `source_dispatch` (Theorem 2.35).

Question 8.103 (Checkers for the new bases)**.** No package measures a complete checker for $\mathsf{T}_{B\varepsilon}$, T_{Bd} or N_{Bd}, in which the equality test or the subtraction equations, the ordinary logic where it is used, and the validation of the staged bodies are all charged. For $\mathsf{T}_{\bar{F}}$ and T_F only the readable checkers of `selfseed_injectivity` are measured, which are several times larger in every coordinate than the iterator checkers of Table 8.2 and have a different input contract; no compact checker is measured. Whether any of these bases, or N_R, admits a single checker below the iterator controls of Table 8.2 in all three coordinates is open, as are lower bounds for a fixed checker language and input grammar.

Question 8.104 (Formalization)**.** The packages name as first targets for a proof assistant: the contextual case lemmas (Theorems 8.9 and 8.54), the factor map identities and the predecessor (Theorems 8.15 and 8.19), the gated complement (Theorem 8.49), cap coherence and finite-case elimination (Theorems 8.40, 8.42 and 8.59), and the coverage and rigidity proofs of Theorems 8.63 and 8.64; for Theorem 8.69 see the first question of this section.

Chapter 9

Canonical and counter-aware recursors

The common core takes the successor S and the iterator $I(n, a; u.\, h)$ as term constructors and states both computation equations as rules (Sections 1.1 and 1.3). This chapter treats the term languages in which the successor is an instance of the recursor, or the body of the recursor sees its counter, or both, and in which part of the computation is moved into a canonical grammar of terms with a normalizing substitution. Section 9.1 fixes the term languages and their normal forms. Section 9.2 shows what a counter-aware body provides: a decoder that computes, a selector, and a compiler for primitive recursion that uses no pairing. Section 9.3 gives the translations to and from iterator terms and transfers the bi-interpretation of Theorem 1.119. Section 9.4 removes the constant 0 from the register language and defines zero by an equation. Section 9.5 restricts the bodies of a register with one bound variable so that two unconditional equations replace its graph axiom, and proves the resulting native calculus equivalent to N. Sections 9.6 to 9.9 treat the proof calculi stated on these syntaxes. Section 9.10 keeps the successor as a constructor, gives the recursor the old counter and the initializer as its value at 0, and shows that on these raw terms one computation step at the root of the left endpoint of an equation, with substitution for a variable as the only equality rule, gives a complete calculus. Section 9.11 records a bi-interpretation of iPRA with an arithmetic of binary trees whose comparison maps are not the identity. Section 9.12 treats the concrete syntax in which the recursor and the paired binder bind the same number of variables. Section 9.13 collects the boundary results.

9.1 Term languages and normal forms

This section draws on `register_graph`, `canonical_recursors`, `clausal_canonical`, `counter_evaluation`, `uniform_binding`, `uniform_recursion` and `single_constructor`.

9.1.1 Three recursors without a successor constructor

Definition 9.1 (Term languages)**.** Each of the following grammars has one numerical sort, variables, the constant 0 and one binding constructor.

1. The *seeded recursor*. Terms are $t ::= x \mid 0 \mid R(n, a; u.h)$; the variable u is bound in h only. Put $Sa := R(0, a; u.0)$ and $1 := S0$. The computation equations are

$$R(0, a; u.h) = Sa, \tag{9.1}$$
$$R(Sn, a; u.h) = h[R(n, a; u.h)/u]. \tag{9.2}$$

2. The *counter-aware seeded recursor.* Terms are $t ::= x \mid 0 \mid R(n, a; i, u.h)$; the variables i, u are bound in h only. The first receives the old counter and the second the preceding state. Put $Sa := R(0, a; i, u.0)$ and $1 := S0$. The computation equations are

$$R(0, a; i, u.h) = Sa, \tag{9.3}$$
$$R(Sn, a; i, u.h) = h[n/i,\ R(n, a; i, u.h)/u], \tag{9.4}$$

with simultaneous capture-avoiding substitution on the right.

3. The *register iterator.* Terms are $t ::= x \mid 0 \mid J(n; i, r.h)$; the variables i, r are bound in h only. Put $Sa := J(a; i, r.0)$ and $1 := S0$. The term $J(n; i, r.h)$ executes n instructions on a register that starts at 1. The body receives the old counter i and the register r and returns an instruction: the value 0 increments the register, and a value St loads t. The axioms are stated in Theorem 9.13.

The number of bound variables distinguishes the first two constructors, so both are written R. In every language the variables of a body other than the bound ones are passive numerical parameters, and substitution is capture avoiding.

In all three languages the successor is a defined operation, not a constructor. The initializer of the seeded recursors is an argument, but the iteration starts at its successor; this is what allows the successor to be an instance of the recursor. The register iterator has no initializer argument at all.

The binding notation is a notation for finite descriptions in a first-order language, as in Section 1.1. Each fixed body, built from earlier descriptions, determines one function symbol whose arguments are the counter, the initializer where there is one, and the passive parameters of the body. No variable ranges over bodies, and no evaluator for descriptions is postulated inside any theory of this chapter.

9.1.2 Zero-body canonical terms

The equations (9.1) and (9.3) say that a recursor with counter 0 does not depend on its body. A canonical grammar records this by one spelling.

Definition 9.2 (Zero-body normalization)**.** For raw terms of either seeded language define ν by $\nu(x) = x$, $\nu(0) = 0$ and

$$\nu\big(R(n, a; \vec{u}.h)\big) = \begin{cases} R(0, \nu a; \vec{u}.0), & n \equiv 0, \\ R(\nu n, \nu a; \vec{u}.\nu h), & n \not\equiv 0, \end{cases}$$

where $\vec{u}$ is u or i, u and $\equiv$ is literal identity of syntax. It is not a test for provable equality with 0. A term is *zero-body canonical* if it is a fixed point of ν, that is, if every subterm with literal counter 0 has literal body 0. The map ν extends to formulas, clauses and contexts by normalizing all terms. For zero-body canonical T and a substitution σ the *normalized substitution* is

$$T\langle\sigma\rangle := \nu(T[\sigma]).$$

The map ν is a terminating traversal of a finite tree. It preserves the root constructor of a term and of a formula, so

$$\nu(t) \equiv 0 \iff t \equiv 0, \qquad \nu(Sa) = S(\nu a). \tag{9.5}$$

It is idempotent, commutes with renaming of free and bound variables, and introduces no free variable. It performs no successor computation.

Lemma 9.3 (Substitution coherence)**.** *For every raw term or formula T and every capture-free simultaneous substitution σ,*

$$\nu(T[\sigma]) = \nu(\nu(T)[\nu \circ \sigma]).$$

Consequently $(T\langle\sigma\rangle)\langle\tau\rangle = T\langle\sigma\tau\rangle$ for zero-body canonical T, where $(\sigma\tau)(x) = \sigma(x)\langle\tau\rangle$.

Proof. Induct on T with bound names chosen fresh for σ. For a variable both sides are $\nu(\sigma(x))$ by idempotence, and the case 0 is trivial. Let $T = R(n, a; \vec{u}.h)$.

If $n \equiv 0$, then $n[\sigma] \equiv 0$ and both sides erase the body: the left side is $R(0, \nu(a[\sigma]); \vec{u}.0)$, the right side is $R(0, \nu(\nu(a)[\nu \circ \sigma]); \vec{u}.0)$, and the induction hypothesis for a applies.

If $n \not\equiv 0$ and $n[\sigma] \equiv 0$, then n is a variable x with $\sigma(x) \equiv 0$, because a substitution instance of a recursor node is a recursor node. The left side erases the body. On the right, $\nu(T) = R(x, \nu a; \vec{u}.\nu h)$, and after the substitution $\nu \circ \sigma$ the counter is $\nu(\sigma(x)) \equiv 0$ by (9.5), so the outer ν erases the body as well. The initializers agree by the induction hypothesis.

If $n[\sigma] \not\equiv 0$, then $n \not\equiv 0$, and by (9.5) neither $\nu(n[\sigma])$ nor $\nu(n)[\nu \circ \sigma]$ is literally 0. Both sides retain the node, and the induction hypotheses for the counter, the initializer and the body give the equality; for the body the substitution is extended by the identity on $\vec{u}$.

Formula constructors bind variables but introduce no further case. For the consequence, apply the identity twice: $\nu(\nu(T[\sigma])[\tau]) = \nu(T[\sigma][\tau])$ since τ is canonical, and $\nu(T[\sigma\tau]) = \nu(T[\sigma][\tau])$ since T is canonical and $\nu(\sigma(x)[\tau]) = (\sigma\tau)(x)$. □

A variable replaced by 0 may expose a zero counter. Normalizing only the original syntax, or testing the counter before substitution, is therefore not sufficient, and rejecting noncanonical input is not sufficient either: instances generated by a rule have to be normalized. In a display that belongs to a canonical calculus, an expression $R(n, a; \vec{u}.h)$ denotes its normalization; the convention matters only when $n \equiv 0$, where the written body is discarded.

Remark 9.4. Theorem 9.3 is the substitution-coherence lemma of `canonical_recursors` for the seeded recursor. `counter_evaluation` and `uniform_binding` state it for the counter-aware recursor in the form $\nu(t\sigma) = \nu(\nu(t)\,\nu(\sigma))$ with the same proof, and `clausal_canonical` states it for the opening operation of a de Bruijn representation (Theorem 9.136). The proof above covers both binding arities.

9.1.3 Full normal forms

The counter-aware language also admits a grammar in which both computation equations are built into the syntax.

Definition 9.5 (Normal terms)**.** In the counter-aware language, a *canonical successor* is a term $R(0, a; i, u.0)$ with a normal. A *neutral* term is a variable or a term $R(c, a; i, u.h)$ with c neutral and a, h normal. A term is *normal* if it is 0, a canonical successor or neutral. Thus a retained recursor node has a counter that is neither 0 nor a successor.

Every normal term has the form $S^k(c_0)$ with $c_0 \equiv 0$ or c_0 neutral; k is the length of its successor prefix. Every normal term is zero-body canonical.

Definition 9.6 (Normalization by evaluation)**.** An environment ρ maps variables to normal terms. Define $\mathrm{ev}(t, \rho)$ by recursion on the raw term t: $\mathrm{ev}(x, \rho) = \rho(x)$, $\mathrm{ev}(0, \rho) = 0$, and

$$\mathrm{ev}\big(R(n, a; i, u.h), \rho\big) = \mathrm{run}\big(\mathrm{ev}(n, \rho), \mathrm{ev}(a, \rho), H\big), \qquad H(d, w) = \mathrm{ev}\big(h, \rho[i \mapsto d, u \mapsto w]\big),$$

where, for normal c, v,

$$\mathrm{run}(c, v, H) = \begin{cases} Sv, & c \equiv 0, \\ H\big(d, \mathrm{run}(d, v, H)\big), & c \equiv Sd, \\ R\big(c, v; i, u.H(i, u)\big), & c \text{ neutral}, \end{cases} \tag{9.6}$$

with i, u fresh in the third clause. The notation $H(i,u)$ is evaluation of the body in the environment extended by the two fresh variables; it introduces no function object into the term language. Put $\mathrm{nf}(t) = \mathrm{ev}(t, \mathrm{id})$, and for formulas, clauses and substitutions normalize all terms.

Lemma 9.7 (Normalization and substitution)**.** *Evaluation is total on raw terms and normal environments, and its values are normal. Moreover:*

1. $\mathrm{nf}(t) = t$ *for every normal* t*; hence* nf *is idempotent.*
2. nf *respects* (9.3) *and* (9.4)*: both sides of every instance have the same normal form, and so do two terms that differ by an instance inside any context, including under a binder.*
3. *For every raw* t *and substitution* σ*,*

$$\mathrm{nf}(t[\sigma]) = \mathrm{nf}\big(\mathrm{nf}(t)[\mathrm{nf} \circ \sigma]\big). \tag{9.7}$$

The same statements hold for formulas and clauses; no formula binder is erased.

Proof. Totality. Induct on t. At a recursor node the values c, v are normal and the body evaluator H is total with normal values, by the induction hypothesis for the proper subterms. For fixed total H, induct on the successor prefix of c in (9.6): the first and third clauses return a normal term, and in the second clause the recursive call has a shorter prefix and H is total.

Item 1. Induct on a normal term. A canonical successor evaluates to $\mathrm{run}(0, a, H) = Sa$. A neutral $R(c, a; i, u.h)$ evaluates by the third clause to $R(c, a; i, u.H(i,u))$, and $H(i,u) = h$ by the induction hypothesis.

Two environment laws. For a substitution σ and environments ρ, τ write $(\rho \circ \sigma)(x) = \mathrm{ev}(\sigma(x), \rho)$ and $(\tau \bullet \rho)(x) = \mathrm{ev}(\rho(x), \tau)$. Then

$$\mathrm{ev}(t[\sigma], \rho) = \mathrm{ev}(t, \rho \circ \sigma), \tag{9.8}$$

$$\mathrm{ev}\big(\mathrm{ev}(t, \rho), \tau\big) = \mathrm{ev}(t, \tau \bullet \rho). \tag{9.9}$$

Both are proved by induction on t with bound names fresh. For (9.8) the recursor case holds because the body evaluator of $h[\sigma]$ in ρ is the body evaluator of h in $\rho \circ \sigma$, by the induction hypothesis. For (9.9) let H be the body evaluator of h in ρ and H^τ the one in $\tau \bullet \rho$; the induction hypothesis for h gives $\mathrm{ev}(H(d,w), \tau) = H^\tau(\mathrm{ev}(d,\tau), \mathrm{ev}(w,\tau))$. It suffices to show

$$\mathrm{ev}\big(\mathrm{run}(c, v, H), \tau\big) = \mathrm{run}\big(\mathrm{ev}(c,\tau), \mathrm{ev}(v,\tau), H^\tau\big)$$

by induction on the successor prefix of c. If $c \equiv 0$ both sides are $S(\mathrm{ev}(v,\tau))$. If $c \equiv Sd$ the left side is $H^\tau(\mathrm{ev}(d,\tau), \mathrm{ev}(\mathrm{run}(d, v, H), \tau))$, which by the hypothesis for d is the second clause of (9.6) at the counter $S(\mathrm{ev}(d,\tau)) = \mathrm{ev}(Sd, \tau)$. If c is neutral, the left side evaluates the residual node $R(c, v; i, u.H(i,u))$ in τ; its body evaluator is $(d,w) \mapsto \mathrm{ev}(H(i,u), \tau[i \mapsto d, u \mapsto w]) = H^\tau(d,w)$, so the left side is $\mathrm{run}(\mathrm{ev}(c,\tau), \mathrm{ev}(v,\tau), H^\tau)$ whichever clause applies to $\mathrm{ev}(c,\tau)$. This is the case in which a substitution exposes a new redex.

Item 2. In every environment, $\mathrm{ev}(R(0, a; i, u.h), \rho) = S(\mathrm{ev}(a,\rho))$ independently of h, and $\mathrm{ev}(R(Sn, a; i, u.h), \rho) = H(c, \mathrm{run}(c, v, H))$ with $c = \mathrm{ev}(n,\rho)$, which by (9.8) is the value of $h[n/i, R(n, a; i, u.h)/u]$ in ρ. Evaluation is compositional: the value of a term in all environments is determined by the values of its immediate subterms in all environments. Hence replacing a subterm by one with the same values does not change the value.

Item 3. By (9.8), $\mathrm{nf}(t[\sigma]) = \mathrm{ev}(t, \mathrm{nf} \circ \sigma)$. By (9.8), item 1 and (9.9), $\mathrm{nf}(\mathrm{nf}(t)[\mathrm{nf} \circ \sigma]) = \mathrm{ev}(\mathrm{ev}(t, \mathrm{id}), \mathrm{nf} \circ \sigma) = \mathrm{ev}(t, \mathrm{nf} \circ \sigma)$. □

Remark 9.8. `single_constructor` defines the normalizer by evaluation, as above. `uniform_recursion` obtains the same normal forms from rewriting: it reads $R(n, a; i, u.h)$ as the base-type recursor of Gödel's system T with initial value Sa and update $h(i, u)$, derives strong normalization from the reducibility argument for that recursor (a numerical expression is computable when it is strongly normalizing, an update is computable when it maps computable arguments to computable expressions, and the recursor case is an induction on the successor spine of a normal counter), observes that the zero and successor rules have disjoint head patterns and commute with reductions in their arguments and with erasure of an unused body, and concludes unique normal forms from termination and local confluence. The two definitions agree, since by item 2 of Theorem 9.7 every reduction step preserves nf and by item 1 the normal terms are exactly the irreducible ones.

Remark 9.9 (No uniform primitive recursive normalizer). Each fixed term denotes a primitive recursive function of its parameters. It does not follow that normalization of arbitrary term descriptions is primitive recursive uniformly; a universal evaluator of that kind leads to the usual diagonal contradiction. The normalizer is an external total recursive operation on finite syntax, and no assertion that all descriptions normalize is an axiom of any theory here. `uniform_recursion` supplies in addition a checker with an explicit budget. Bounded simulation gives a primitive recursive certificate predicate with the same set of provable formulas, since every finite successful run has a sufficient budget.

9.1.4 The status of the successor and of each computation

The languages of this chapter differ from the iterator terms of Section 1.1, and from one another, in which of three items is a constructor or a rule and which is carried by the syntax. The following table summarizes the presentations treated below.

Terms	Successor	Zero computation	Successor computation
$0, S, I$ (Section 1.1)	constructor	rule	rule
raw seeded R, either arity	$R(0, a; \vec{u}.0)$	rule	rule
zero-body canonical R, one binder (Sections 9.6 and 9.7)	$R(0, a; u.0)$	identity of spelling	the one computation rule
zero-body canonical R, two binders (Section 9.8)	$R(0, a; i, u.0)$	identity of spelling	derived from reflexivity and counter evaluation
normal R, two binders (Section 9.9)	$R(0, a; i, u.0)$	identity of spelling	identity of spelling
raw $0, S, \hat{R}$, two binders (Section 9.10)	constructor	one root step, either direction	one root step, either direction
register J (Section 9.2)	$J(a; i, r.0)$	axiom $J(0; i, r.h) = 1$	graph biconditional
guarded scalar J (Section 9.5)	$J(a; v.0)$	rule $J(0; u.h) = S(h[0/u])$	rule at successor-headed bodies, derived at state-free bodies

A computation that is an identity of spelling has not left the trusted specification: the normalizer is applied at every substitution and comparison, and it is counted in every checker of Section 9.14. Its soundness in ordinary arithmetic is Theorem 9.23; it rests on an extensionality property of recursor bodies (Theorem 9.22) and not on syntactic idempotence alone.

9.2 What the counter gives

This section draws on `counter_evaluation`, `uniform_binding`, `uniform_recursion`, `single_constructor`, `register_graph` and, for the contrast with a body that does not see its counter, `canonical_recursors`.

9.2.1 A decoder that computes

In the counter-aware language define, with all auxiliary bound variables fresh,

$$\begin{aligned} P(n) &:= R(n,0;i,u.i),\\ C(n,a,b) &:= P\big(R(n,a;i,u.Sb)\big),\\ \mathrm{Rec}(n,a;i,u.h) &:= P\big(R(n,a;i,v.S(h[P(v)/u]))\big),\\ p(n) &:= C(n,0,P(n)), \qquad \mathrm{sg}(n) := C(n,0,1). \end{aligned} \tag{9.10}$$

Lemma 9.10 (Decoder, selector and recursion)**.** *The following equations are derivable from* (9.3)*,* (9.4)*, reflexivity and congruence, for arbitrary terms and passive parameters:*

$$\begin{aligned} P(0) &= 1, & P(Sn) &= n,\\ C(0,a,b) &= a, & C(Sn,a,b) &= b,\\ \mathrm{Rec}(0,a;i,u.h) &= a, & \mathrm{Rec}(Sn,a;i,u.h) &= h[n/i,\mathrm{Rec}(n,a;i,u.h)/u],\\ p(0) &= 0, & p(Sn) &= n,\\ \mathrm{sg}(0) &= 0, & \mathrm{sg}(Sn) &= 1. \end{aligned}$$

No induction, numerical case distinction or logical rule is used. On normal terms each equation is a literal identity of normal forms.

Proof. The equations for P are the two computations with the body that returns the old counter and ignores the state. In $C(0,a,b)$ the inner recursor is Sa and in $C(Sn,a,b)$ it is Sb; then apply $P(Sx)=x$. For recursion put $T(n)=R(n,a;i,v.S(h[P(v)/u]))$. Then $T(0)=Sa$ and $T(Sn)=S(h[n/i,P(T(n))/u])$, and applying P gives both equations. The equations for p and sg are instances of those for C and P. On normal terms each step is a computation, so both sides have the same normal form by Theorem 9.7. □

The decoder P is a left inverse of the successor obtained by one computation step, for an arbitrary open argument. Its value at 0 is 1, not 0; the predecessor of Theorem 1.65 is p. The state of Rec is stored as a successor and the outer P removes that wrapper after each observation, so the successor seed of R does not restrict the initial value.

Remark 9.11. `counter_evaluation` and `uniform_binding` use the names of (9.10). `uniform_recursion` writes W for the decoder, P for the predecessor p and J for Rec, and uses the term $K(y) := C(W(y),1,0)$, with $K(1)=1$ and $K(S1)=0$. `single_constructor` defines the predecessor and the selector from the recursion macro, as $p(t) := \mathrm{Rec}(t,0;i,v.i)$ and $C(t,a,b) := \mathrm{Rec}(t,a;i,v.b)$; they satisfy the same equations.

9.2.2 Primitive recursion without pairing

Theorem 9.12 (Direct compiler)**.** *Every fixed primitive recursive description compiles to a term of the counter-aware language whose defining equations are derivable from the computation equations and equality reasoning alone. Zero, successor and projections are terms, composition is substitution, and a recursion $f(0,\vec{p}) = g(\vec{p})$, $f(Sn,\vec{p}) = h(n,f(n,\vec{p}),\vec{p})$ compiles to*

$$f^*(n,\vec{p}) = \mathrm{Rec}(n,g^*(\vec{p});i,u.h^*(i,u,\vec{p})).$$

The compiler uses no pairing, no projection functions, no induction and no coverage. The iterator of Section 1.1 is the instance $I(n,a;\,u.\,h) := \mathrm{Rec}(n,a;i,u.h)$ with $i \notin \mathrm{FV}(h)$.

Proof. By external induction on the description. The recursion case is Theorem 9.10; the other cases are immediate. □

For example $x + y = \mathrm{Rec}(y, x; i, v.Sv)$, $x \cdot y = \mathrm{Rec}(y, 0; i, v.v + x)$ and the truncated subtraction is $d(x, y) = \mathrm{Rec}(y, x; i, v.p(v))$. The compiler is an external recursion on each finite description. It is not a universal evaluator for description codes inside the theory. The compiler of Theorem 1.102 for iterator terms stores the counter beside the state by a pairing function and therefore comes after the construction of pairing; here the counter is an argument of the body.

9.2.3 The register iterator

The register iterator carries the same information in a different form: the body returns an instruction, and the axioms describe the transition graph.

Definition 9.13 (The theory T_J)**.** The language has the terms of Theorem 9.1(3) and the formulas

$$A ::= (s = t) \mid \bot \mid (A \wedge A) \mid (A \vee A) \mid (A \to A) \mid \forall x\, A \mid \exists x\, A.$$

The logical rules are those of intuitionistic natural deduction with equality: hypothesis, reflexivity, transport of an arbitrary formula along a proved equation, ex falso, and the introduction and elimination rules of $\wedge, \vee, \to, \forall, \exists$ with the usual eigenvariable conditions. There is no classical rule. For a fixed body $h(i, r, \vec{p})$ write $j_n = J(n; i, r.h)$ and $e_n = h(n, j_n, \vec{p})$. The arithmetic axioms are the universal closures of

$$J(0; i, r.h) = 1, \tag{J0}$$

$$J(Sn; i, r.h) = b \;\leftrightarrow\; \big((e_n = 0 \wedge b = Sj_n) \vee e_n = Sb\big), \tag{STEP}$$

$$0 = 1 \to \bot, \tag{Sep}$$

where b is a variable distinct from the passive parameters. The one arithmetic rule is induction for literal equations with an arbitrary context:

$$\frac{\Gamma \vdash s(0) = t(0) \qquad \Gamma,\ s(z) = t(z) \vdash s(Sz) = t(Sz)}{\Gamma \vdash s(a) = t(a)} \qquad (z \text{ fresh for } \Gamma, a \text{ and the parameters}). \tag{9.11}$$

An equation under a quantifier or a connective is not eligible as an invariant.

The right-to-left direction of (STEP) gives two conditional computation rules, whose premises are proved equations and not tests performed by an evaluator:

$$e_n = 0 \implies J(Sn; i, r.h) = Sj_n, \tag{9.12}$$

$$e_n = St \implies J(Sn; i, r.h) = t. \tag{9.13}$$

Lemma 9.14 (Functional coverage)**.** *Let $f(n, a) := J(n; i, r.a)$ with i, r fresh for a, and $W(a) := f(1, a)$. Without induction, $\mathsf{T}_J \vdash a = 0 \vee a = S(W(a))$. In particular $\forall a\,(a = 0 \vee \exists x\, a = Sx)$ is derivable.*

Proof. Take the constant body a, the counter $n = 0$ and $b = f(1, a)$ in (STEP). Its left side holds by reflexivity. Its right side is $(a = 0 \wedge b = S1) \vee a = Sb$; drop the second conjunct of the first alternative. □

Proposition 9.15 (The graph axiom)**.** *Over intuitionistic logic with equality, and without* (Sep)*, induction or injectivity of the successor:*

1. (J0) *and* (STEP) *are equivalent to* (J0), (9.12), (9.13) *and numerical coverage* $\forall a\,(a = 0 \vee \exists x\, a = Sx)$*;*

2. *both are equivalent to the single schema*

$$J(n; i, r.h) = b \;\leftrightarrow\; (n = 0 \wedge b = 1) \vee \exists m\,\big(n = Sm \wedge ((e_m = 0 \wedge b = Sj_m) \vee e_m = Sb)\big). \tag{9.14}$$

Proof. (1) From (STEP) the two conditional rules follow by its right-to-left direction and coverage is Theorem 9.14. Conversely the right-to-left direction of (STEP) is the two conditional rules. For the left-to-right direction assume $J(Sn; i, r.h) = b$ and split coverage for e_n. If $e_n = 0$, then (9.12) gives $b = Sj_n$. If $e_n = St$, then (9.13) gives $b = t$ and hence $e_n = Sb$.

(2) The right-to-left direction of (9.14) gives (J0) at $n = 0$, $b = 1$ and both conditional rules at $n = Sm$. Its left-to-right direction applied to the reflexive equation $J(n; i, r.0) = J(n; i, r.0)$ gives $n = 0 \vee \exists m\, n = Sm$. By (1) this proves (STEP). Conversely, assume (J0) and (STEP). The right-to-left direction of (9.14) is (J0) and the right-to-left direction of (STEP) after transport along $n = Sm$. For the left-to-right direction split coverage for n: at $n = 0$ use (J0), and at $n = Sm$ use (STEP). □

The biconditional is essential: the left-to-right direction of (STEP) alone does not assert that the computation is deterministic. A packaging such as (9.14) is also available for the iterator of Section 1.1, so a smaller number of named schemas is not by itself a simplification.

Lemma 9.16 (Bootstrap without induction)**.** *Using only equality,* (J0) *and the conditional rules,* T_J *proves*

$$f(0, a) = 1, \qquad f(n, 0) = Sn, \qquad f(Sn, Sa) = a, \tag{9.15}$$

and for

$$Z(x) := f(x, 1), \quad W(x) := f(1, x), \quad D(x) := f(x, Sx), \quad P(x) := W(D(x)),$$

$$\begin{aligned} Z(0) &= 1, & Z(Sx) &= 0, \\ W(0) &= S1, & W(Sx) &= x, \\ D(0) &= 1, & D(Sx) &= Sx, \\ P(0) &= 0, & P(Sx) &= x. \end{aligned}$$

Further, for

$$G(c, a) := P^2 f(Sa, Z(c)), \qquad C(c, a, b) := P^2 f(Sa,\, G(c, S^3 b)),$$

where P^2 *and* S^3 *are finite compositions,*

$$G(0, a) = 0, \quad G(Sn, a) = a, \qquad C(0, a, b) = a, \quad C(Sn, a, b) = b. \tag{9.16}$$

Consequently the successor is injective, and with (Sep) *no successor is* 0*.*

Proof. The first equation of (9.15) is (J0), the second is the definition of S, and the third is a load step whose premise is the reflexive equation $Sa = Sa$. The eight endpoint equations are instances: for example $Z(Sx) = f(Sx, S0) = 0$, $W(0) = f(1, 0) = S1$, $P(0) = W(1) = W(S0) = 0$ and $P(Sx) = W(Sx) = x$. For G: at $c = 0$ the inner term is $f(Sa, S0) = 0$ and $P^2(0) = 0$; at $c = Sn$ it is $f(Sa, 0) = S^2 a$ and P^2 returns a. For C: at $c = 0$ the instruction $G(c, S^3 b)$ is 0, so the inner term is $S^2 a$; at $c = Sn$ the instruction is $S^3 b$, and the counter Sa, which is visibly a successor, loads $S^2 b$. Applying P to $Sx = Sy$ gives $x = y$. An equation $Sx = 0$ gives $Z(Sx) = Z(0)$, that is $0 = 1$. □

The selector C returns arbitrary numerical outputs. It is not limited to Boolean results or to values bounded by a preselected initializer; the direct use of the old register in the increment branch is what makes this possible.

Theorem 9.17 (Register compiler)**.** *For terms a and $h(u,v)$ of the register language, with i, r fresh, put*

$$H(i,r) := S\big(C(i,a,h(P(i),r))\big), \qquad \mathrm{Rec}_J(n,a;u,v.h) := J(Sn;i,r.H(i,r)).$$

Without induction, coverage, pairing or reflection of equality,

$$\mathrm{Rec}_J(0,a;u,v.h) = a, \qquad \mathrm{Rec}_J(Sn,a;u,v.h) = h\big(n,\mathrm{Rec}_J(n,a;u,v.h)\big).$$

Hence every fixed primitive recursive description compiles to a term of the register language with derivable defining equations.

Proof. The instruction is always a successor, so every update is a load. The first update has counter $i = 0$ and old register 1; by (9.16) its instruction is Sa and (9.13) gives $J(1;i,r.H) = a$. For the second equation put $r_0 = J(Sn;i,r.H)$. The old counter is Sn, so (9.16) and $P(Sn) = n$ give $H(Sn,r_0) = S(h(n,r_0))$, and one load gives $J(S(Sn);i,r.H) = h(n,r_0)$. Initial functions and composition translate by substitution. □

The compact source grammar does not guarantee small expanded terms. The compiler shipped with `register_graph` uses the instruction $S(C(Z(i),h(P(i),r),a))$, which has the same endpoint values and a smaller expansion.

9.2.4 A register with one bound variable

Proposition 9.18 (One-variable register)**.** *Let $t ::= x \mid 0 \mid J_s(n;r.h)$ with $Sa := J_s(a;r.0)$, and let the axioms be* (J0)*,* (STEP) *with $e_n = h(j_n,\vec{p})$,* (Sep) *and* (9.11)*. Then Theorems 9.14 and 9.16 hold unchanged, and for*

$$H(v) := S^3\big(C(P(v),a,h(P^2v))\big), \qquad K(n) := J_s(Sn;v.H(v)), \qquad \widehat{I}(n,a;r.h) := P^2K(n)$$

the equations $\widehat{I}(0,a;r.h) = a$ and $\widehat{I}(Sn,a;r.h) = h(\widehat{I}(n,a;r.h))$ are derivable without induction or coverage.

Proof. The bootstrap uses constant bodies only. One load step gives, at every counter,

$$J_s(Sn;v.H) = S^2\big(C(P(J_s(n;v.H)),a,h(P^2J_s(n;v.H)))\big),$$

so every state after the initial one is visibly a double successor. At the first step the previous register is 1 and $P(1) = 0$, so $K(0) = S^2a$. At a later step $P(K(n))$ is a successor and the selector takes the update branch, so $K(Sn) = S^2h(P^2K(n))$. Applying P twice gives both equations. The shape of the state is a consequence of one computation, not an inferred invariant. □

The one-variable register keeps the binding arity of the iterator and still has neither a successor constructor nor an initializer. Its simulation of general primitive recursion passes through the iterator and therefore through the pairing of Theorem 1.102. The counter variable of the two-variable register is the cost of the direct compiler of Theorem 9.17.

Proposition 9.19 (Reflexivity from conditional computation)**.** *Consider* (J0)*,* (9.12) *and* (9.13) *as rules whose premises are derivations, together with transport of an equation along an equation. Then $t = t$ is derivable for every term t, without a primitive reflexivity rule.*

Proof. (i) (J0) for the zero body is literally $1 = 1$. (ii) A load step with the constant instruction $1 = S0$, whose premise is (i), gives $J(1; i, r.1) = 0$; transporting this equation along itself gives $0 = 0$. (iii) An increment step with the zero body at the counter t, whose premise is (ii), gives the literal equation $S(St) = S(St)$, since $J(t; i, r.0)$ is St by definition. (iv) A load of the constant instruction $S(St)$, whose premise is (iii), gives an equation with right side St, and transport along itself gives $St = St$. (v) A load of the constant instruction St gives an equation with right side t, and transport along itself gives $t = t$. □

By Theorem 9.19 the register terms are compatible with a native kernel in the style of Section 1.3, in which a conditional computation step takes a proved equation as premise. No checker for such a kernel is supplied (Section 9.15).

9.2.5 Without a counter

The seeded recursor with one bound variable has the partial selector $B(n, a, b) := R(n, a; u.b)$, $u \notin \mathrm{FV}(b)$, with $B(0, a, b) = Sa$ and $B(Sn, a, b) = b$, but its body has no access to the counter, and the decoder of Theorem 9.10 is not available. A predecessor exists nevertheless: by Theorem 8.19 there is a fixed term P of the seeded language with

$$P(0) = 0, \qquad P(Sa) = a \tag{9.17}$$

derivable in the native calculus and in the ordinary theory of the seeded recursor, by native numerical cases, equality reasoning and induction on literal equations. By Theorem 8.21 the term

$$\widehat{I}(n, a; u.h) := P\big(R(n, a; v.S(h[P(v)/u]))\big)$$

then satisfies the two equations of the iterator, and $C(n, a, b) := \widehat{I}(n, a; u.b)$ is a selector with $C(0, a, b) = a$ and $C(Sn, a, b) = b$.

The difference between the two arities is therefore the length of the bootstrap and the need for pairing, not expressive strength: with the counter the predecessor is one computation step and primitive recursion is Theorem 9.12; without it the predecessor is the construction of Section 8.1 and primitive recursion is compiled as in Theorem 1.102. If the independent initializer is removed as well and no equality test is added, no term denotes the predecessor (Theorem 9.140). The witness-free introductions that use the computing predecessor of the counter-aware recursor are treated in Sections 7.7 and 7.8, on the normal terms of Theorem 9.5.

9.3 Translations to iterator terms and the transfer of the bridge

This section draws on `register_graph`, `canonical_recursors`, `clausal_canonical`, `counter_evaluation`, `uniform_binding`, `uniform_recursion` and `single_constructor`: each of them proves, for its own syntax, a bi-interpretation with iPRA. Here the statement is proved once, relative to T_{eq}, and Theorem 1.119 is then applied.

9.3.1 The ordinary theories

Definition 9.20 (Ordinary theories)**.** The theories T_R^1 and T_R^2 have the raw terms of the seeded recursor with one and with two bound variables, the formulas and logical rules of Theorem 9.13, and the following arithmetic part: the two computation equations ((9.1), (9.2), respectively (9.3), (9.4)); coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$; separation $0 = S0 \to \bot$; and the rule (9.11) of induction for literal equations with an arbitrary context. The theory T_J is that of Theorem 9.13; coverage is a theorem of it (Theorem 9.14). The theory T_R^1 is the theory T_R of Theorem 8.1.

These are the theory T_{eq} of Section 1.5 with the terms exchanged. The separation axiom is part of each theory: the positive formulas read in isolation admit a one-element structure. Since a fixed body is a function symbol of its parameters, ordinary equality logic gives congruence in the parameters of a body. Replacing a body by a different body that is provably equal to it pointwise is not an instance of congruence; it is the next lemma.

Lemma 9.21 (Recurrence uniqueness)**.** *Let* T *be* T_{eq}, T^1_R *or* T^2_R*, and let* $F(z)$, $G(z)$, $k(z,w)$, $k'(z,w)$ *be terms with* z, w *fresh for* Γ*. If*

$$\Gamma \vdash F(0) = G(0), \quad \Gamma \vdash F(Sz) = k(z, F(z)), \quad \Gamma \vdash G(Sz) = k'(z, G(z)), \quad \Gamma \vdash k(z,w) = k'(z,w),$$

then $\Gamma \vdash F(t) = G(t)$ *for every term* t*.*

Proof. Apply (9.11) to the literal equation $F(z) = G(z)$, with all parameters fixed. The base is given. Under $F(z) = G(z)$,

$$F(Sz) = k(z, F(z)) = k(z, G(z)) = k'(z, G(z)) = G(Sz),$$

by congruence in the argument w of k and the instance $w := G(z)$ of the fourth hypothesis. □

Lemma 9.22 (Body extensionality)**.** *In* T^1_R *and* T^2_R*, if* $\Gamma \vdash a = b$ *and* $\Gamma \vdash h = k$ *with the bound variables* $\vec{u}$ *free in this premise and fresh for* Γ*, then* $\Gamma \vdash R(n, a; \vec{u}.h) = R(n, b; \vec{u}.k)$ *for every* n*.*

Proof. For one bound variable this is Theorem 8.3. In general take $F(z) = R(z, a; \vec{u}.h)$ and $G(z) = R(z, b; \vec{u}.k)$ in Theorem 9.21. At 0 the two sides are Sa and Sb, equal by congruence. The successor equations are the computation equations, and the updates agree by the second premise. □

Lemma 9.23 (Normalization is sound)**.** T^1_R *and* T^2_R *prove* $t = \nu(t)$ *for every raw term* t*, and* T^2_R *proves* $t = \mathrm{nf}(t)$*. Hence every formula is provably equivalent to its normalization, and a raw substitution instance is provably equivalent to the normalized one.*

Proof. For ν induct on t. The counter and the initializer change by congruence, the body by Theorem 9.22 and the induction hypothesis with the bound variables free. If the counter is literally 0, the zero computation, applied to the term and to its normalization, identifies both with Sa.

For nf prove $\mathsf{T}^2_R \vdash t[\rho] = \mathrm{ev}(t, \rho)$ for every normal environment ρ, by induction on t. At a recursor node the induction hypotheses give $n[\rho] = c$, $a[\rho] = v$ and, at the fresh variables, $h[\rho] = H(i,u)$; hence $R(n, a; i, u.h)[\rho] = R(c, v; i, u.H(i,u))$ by Theorem 9.22. It remains to prove $R(c, v; i, u.H(i,u)) = \mathrm{run}(c, v, H)$ by induction on the successor prefix of c. If $c \equiv 0$ this is the zero computation. If c is neutral the two sides are identical. If $c \equiv Sd$, the successor computation gives $H(i,u)[d/i, R(d, v; i, u.H(i,u))/u]$; replace the state by $\mathrm{run}(d, v, H) =: w$ using the hypothesis for d, and observe that $H(i,u)[d/i, w/u] = h[\rho][d/i, w/u] = H(d, w)$ by the induction hypothesis for h in the two environments. The statements on formulas follow by transport of formulas along equations. □

9.3.2 The translations

Definition 9.24 (Translations)**.** Let T be one of T^1_R, T^2_R, T_J. The translation $t \mapsto t^\iota$ from T to T_{eq} fixes variables and 0 and is defined on the binding constructor as follows.

1. For T^1_R: $R(n, a; u.h)^\iota = I(n^\iota, Sa^\iota; u.\, h^\iota)$, as in Theorem 8.23.
2. For T^2_R: $R(n, a; i, u.h)^\iota = F(n^\iota)$, where F is the term given by the compiler of Theorem 1.102 for the recursion $F(0) = Sa^\iota$, $F(Sz) = h^\iota[z/i, F(z)/u]$.

3. For T_J: $J(n;i,r.h)^\iota = F(n^\iota)$, where F is given by the same compiler for the recursion

$$F(0) = S0, \qquad F(Sz) = \mathrm{sel}(e,\ SF(z),\ \mathrm{pd}(e)), \qquad e = h^\iota[z/i, F(z)/r],$$

and pd is the predecessor term of Theorem 1.65, written P there.

The compiler of Theorem 1.102 provides, for terms a and $g(y,v)$ of T_{eq}, a defined term with the value a at 0 and the value $g(c,\cdot)$ of the preceding value at Sc; it is built with pairing. That derived recursor of T_{eq} is not the constructor R of this chapter, whose value at 0 is Sa. The translation $s \mapsto s^\kappa$ from T_{eq} to T fixes variables and 0, sends Ss to the defined successor of s^κ, and sends the iterator to the initialized iteration of T:

$$I(n,a;\,u.\,h)^\kappa = \begin{cases} \widehat{I}(n^\kappa, a^\kappa; u.h^\kappa) & \text{in } \mathsf{T}^1_R \text{ (Theorem 8.21; the translation } \rho \text{ of Theorem 8.23)},\\ \mathrm{Rec}(n^\kappa, a^\kappa; i, u.h^\kappa),\ i \text{ fresh}, & \text{in } \mathsf{T}^2_R \text{ (Theorem 9.10)},\\ \mathrm{Rec}_J(n^\kappa, a^\kappa; i, u.h^\kappa),\ i \text{ fresh}, & \text{in } \mathsf{T}_J \text{ (Theorem 9.17)}. \end{cases}$$

Both translations commute with substitution and are extended to formulas so that they commute with every connective and quantifier.

Theorem 9.25 (Transfer of the bridge)**.** *Let* T *be* T^1_R, T^2_R *or* T_J.

1. *If* $\mathsf{T} \vdash A$ *then* $\mathsf{T}_{\mathrm{eq}} \vdash A^\iota$, *and if* $\mathsf{T}_{\mathrm{eq}} \vdash B$ *then* $\mathsf{T} \vdash B^\kappa$.
2. $\mathsf{T} \vdash t^{\iota\kappa} = t$ *for every term* t *of* T, *and* $\mathsf{T}_{\mathrm{eq}} \vdash s^{\kappa\iota} = s$ *for every term* s *of* T_{eq}.
3. *Consequently* $\mathsf{T} \vdash A^{\iota\kappa} \leftrightarrow A$ *and* $\mathsf{T}_{\mathrm{eq}} \vdash B^{\kappa\iota} \leftrightarrow B$; *both translations are faithful; and* T *is bi-interpretable with* T_{eq}, *hence by Theorem 1.119 with* N *and with* iPRA, *by interpretations whose domains are the whole numerical domain, whose equality is actual equality and whose numerical composites have the graph* $x = y$. *The four theories are equiconsistent.*

The same holds for the one-variable register of Theorem 9.18, with $J_s(n;r.h)^\iota = I(n^\iota, S0;\, v.\ \mathrm{sel}(h^\iota(v), Sv, \mathrm{pd}(h^\iota$ *and the iteration* $\widehat{I}$ *of that proposition for* κ.

Proof. For T^1_R the three statements are Theorems 8.24 and 8.25. The proof is written for T^2_R and T_J.

(1) *From* T *to* T_{eq}. The logical rules translate to themselves. The translation of the defined successor satisfies

$$\mathsf{T}_{\mathrm{eq}} \vdash (St)^\iota = S(t^\iota). \tag{9.18}$$

For the seeded recursor this is the base equation of the translated recursion. For the register, $(St)^\iota = F_0(t^\iota)$, where F_0 translates the zero body and satisfies $F_0(0) = S0$ and $F_0(Sz) = \mathrm{sel}(0, SF_0(z), \mathrm{pd}(0)) = SF_0(z)$; induction on the literal equation $F_0(z) = Sz$ gives (9.18). Thus the defined successor is interpreted as the successor of T_{eq} at every element of the domain, not only at numerals. With (9.18) and transport, the computation equations of the seeded recursor translate to the defining equations of the translated recursion, coverage and separation translate to the corresponding axioms of T_{eq}, and an instance of (9.11) translates to an instance of the induction rule of T_{eq} for a literal equation. For T_J, (J0) is the base equation. For (STEP) write $j = F(n)$ and e for the translated instruction, so that $F(Sn) = \mathrm{sel}(e, Sj, \mathrm{pd}(e))$, and split coverage for e. If $e = 0$ then $F(Sn) = Sj$, and both sides of the biconditional are equivalent to $b = Sj$, the second alternative being excluded by separation. If $e = Sy$ then $F(Sn) = y$, and both sides are equivalent to $b = y$, by injectivity of the successor. (Sep) is separation.

From T_{eq} *to* T. The two computation equations of the iterator translate to the equations of Rec and Rec_J, which are derivable by Theorems 9.10 and 9.17. Coverage and separation are axioms or theorems of T, and the induction rule for literal equations is (9.11).

(2) Induct externally on the term. Let $t = R(n, a; i, u.h)$ in T^2_R and let $F(z)$ be the κ-translation of the compiled recursion of Theorem 9.24(2). By part (1), T^2_R proves $F(0) = Sa^{\iota\kappa}$ and $F(Sz) = h^{\iota\kappa}[z/i, F(z)/u]$. By the induction hypotheses, $a^{\iota\kappa} = a$ and $h^{\iota\kappa} = h$ with i, u free. The term $G(z) = R(z, a; i, u.h)$ has the same base and the same update, so Theorem 9.21 gives $F(z) = G(z)$, and the instance at $n^{\iota\kappa} = n$ is the claim. In the other direction, for $s = I(n, a;\, u.\, h)$ the term $s^{\kappa\iota}$ is $F(n^{\kappa\iota})$ for a term F of T_{eq} that satisfies the two equations of the iterator with initial value $a^{\kappa\iota}$ and update $h^{\kappa\iota}$, by part (1); Theorem 9.21 in T_{eq} and the induction hypotheses give $s^{\kappa\iota} = s$. For $s = Ss_0$ use (9.18).

For $t = J(n; i, r.h)$ in T_J the double translation is $F(n^{\iota\kappa})$ with $F(0) = 1$ and $F(Sz) = C'(e', SF(z), P'(e'))$, where $e' = h^{\iota\kappa}[z/i, F(z)/r]$ and C', P' are the translated selector and predecessor, which have their endpoint equations in T_J. Apply (9.11) to the equation $F(z) = J(z; i, r.h)$. The base is (J0). Under the induction equation and the induction hypothesis for h, the two instructions e' and e_z are equal. Split coverage for this common value. In the zero branch both sides increment equal registers, by (9.12); in the successor branch both load the same number, by (9.13). The invariant is a single equation, and the case split is supplied by coverage inside the step.

(3) Since the translations commute with the logical operations, part (2) and transport give the two formula equivalences by induction on formulas. Faithfulness follows: if $\mathsf{T}_{\mathrm{eq}} \vdash A^{\iota}$ then $\mathsf{T} \vdash A^{\iota\kappa}$ and hence $\mathsf{T} \vdash A$. Both translations fix the domain, equality and $\bot$, so the identity graph is a provably total bijection that preserves every operation in both composites. The one-variable register is treated in the same way: coverage validates its graph axiom for the displayed iterator, induction on a literal equation shows that its successor abbreviation agrees with the successor, and recurrence uniqueness gives both composites. □

Remark 9.26 (The direct interpretations of the packages). Each package states the theorem against iPRA directly, with the same two maps: a recursor or register description is interpreted by the primitive recursive function symbol with the same base and step, and a function symbol of iPRA is interpreted by the compiler of Theorem 9.12 or Theorem 9.17, without pairing. By Theorem 9.21 these maps agree provably with the composites of ι and κ with the interpretations of Theorem 1.119. `canonical_recursors` and `clausal_canonical` use the first clause of Theorem 9.24 and the iteration $\widehat{I}$, and compile primitive recursion with pairing as in Theorem 1.102. The mere agreement of two theories on standard numerals, or mutual interpretability, would not give Theorem 9.25; the composites are proved inside the theories by Theorem 9.21.

Proposition 9.27 (Quantifier-free induction). *Each of* T^1_R, T^2_R *and* T_J *proves the successor to be injective, proves equality decidable, and admits induction for all quantifier-free formulas with an arbitrary context.*

Proof. Let $A(z)$ be quantifier-free in the language of T. By Theorem 1.95, T_{eq} proves the induction principle for the quantifier-free formula A^{ι} as a sentence, with its parameters universally quantified. By Theorem 9.25(1) the theory T proves the κ-translation of this sentence, in which the successor is the defined successor, and by Theorem 9.25(3) the formula $A^{\iota\kappa}$ may be replaced by A. A sentence of this form yields the rule with an arbitrary context. Injectivity and decidability of equality transfer in the same way from Theorems 1.65 and 1.86. □

Remark 9.28 (The reflection argument in the packages). `register_graph`, `counter_evaluation`, `uniform_binding`, `uniform_recursion` and `single_constructor` do not transfer Theorems 1.86 and 1.95; they repeat the proofs with the terms of Section 9.2, where the predecessor and the

selector are available before any induction. With addition and truncated subtraction compiled by Theorem 9.12, four inductions on literal equations give $0 + y = y$, $Sx + y = S(x + y)$, $d(Sx, Sy) = d(x, y)$ and $d(Sa, a) = 1$; a fifth, on the single equation

$$C(\mathrm{sg}(d(x, y)),\, x,\, y + d(x, y)) = x$$

with x fixed, in whose step every case split is supplied by coverage, gives $d(Sa, b) = 1 \leftrightarrow a = b$. The reflection term is $\epsilon(a, b) = \mathrm{sg}(K(d(Sa, b)))$ with $K(n) = C(n, 1, p(n))$, and the characteristic term of a quantifier-free formula is defined with 0 for truth by $b_{s=t} = \epsilon(s, t)$, $b_{\bot} = 1$, $b_{A\wedge B} = C(b_A, b_B, 1)$, $b_{A\vee B} = C(b_A, 0, b_B)$, $b_{A\to B} = C(b_A, b_B, 0)$. `register_graph` uses the control $d(x, y)$ in place of its sign, the term $e(a, b) = Z(W(d(Sa, b)))$ and characteristic terms with 1 for truth. An independent audit shipped with `uniform_binding` confirms that the only induction instances in this argument are literal equations. The argument is that of Theorems 1.86 and 1.95 and is not repeated.

9.4 A register language without constants

This section draws on `constantfree`, which builds on `register_graph`.

The register language of Theorem 9.1(3) has one constant, 0. It can be removed. Three occurrences of J over a variable form a term that is provably constant, and an equation $z = Z(z)$ then defines zero without a closed term. This section states the theory without constants, proves that it defines zero and the successor, translates it to and from T_J, and obtains the bi-interpretation with iPRA from Theorem 9.25. Only what the removal of the constant requires is proved here; the bootstrap, the compiler and the reflection argument are those of Section 9.2.3 and Theorem 9.28.

9.4.1 The language and the theory

Definition 9.29 (Terms without constants)**.** The terms are

$$t ::= x \mid J(n; i, r.h),$$

with i, r bound in h only. There is no numerical constant, no successor, no initializer and no pairing. With fresh bound variables in every abbreviation put

$$\mathsf{K}(t) := J(t; i, r.i), \qquad \mathsf{U}(t) := J(t; i, r.\mathsf{K}(r)),$$
$$\mathsf{Z}(t) := J(\mathsf{U}(t); i, r.r), \qquad \mathsf{S}(t) := J(t; i, r.\mathsf{Z}(r)).$$

Fully expanded, $\mathsf{Z}(t) = J(J(t; i, r.J(r; j, s.j)); k, v.v)$; thus U has two occurrences of J and Z has three. The successor S is a fixed term with the single free variable t. For an arbitrary term a, the *anchor*, put

$$0_a := \mathsf{Z}(a), \qquad 1_a := \mathsf{S}(0_a), \qquad 2_a := \mathsf{S}(1_a).$$

Here a is not a distinguished element; it is a passive parameter.

The sans-serif letters distinguish these terms from the successor S of T_J and from the term $Z(x) = f(x, 1)$ of Theorem 9.16. `constantfree` writes K, U, Z, S for $\mathsf{K}, \mathsf{U}, \mathsf{Z}, \mathsf{S}$ and q for the term Z of Theorem 9.16.

Lemma 9.30 (No closed terms)**.** *Every term of Theorem 9.29 has a free variable.*

Proof. By induction on the term. A variable is free in itself. The counter n of $J(n; i, r.h)$ is outside the binder, so a free variable of n is free in $J(n; i, r.h)$. □

The schemas of T_J are written once more, with the zero and the successor as parameters, so that T_J and the theory without constants are two instances.

Definition 9.31 (Anchored register schemas)**.** Let o be a term and $\sigma(x)$ a term with a distinguished variable x. For a fixed body $h(i, r, \vec{p})$ write $j_n = J(n; i, r.h)$ and $e_n = h(n, j_n, \vec{p})$. The *anchored schemas at* $[o, \sigma]$ are the universal closures of

$$J(o; i, r.h) = \sigma(o), \tag{J0}[o,\sigma]$$

$$J(\sigma(n); i, r.h) = b \;\leftrightarrow\; \big((e_n = o \wedge b = \sigma(j_n)) \vee e_n = \sigma(b)\big), \tag{STEP}[o,\sigma]$$

$$o = \sigma(o) \to \bot, \tag{Sep}[o,\sigma]$$

with b a variable distinct from the parameters, together with the rule

$$\frac{\Gamma \vdash E(o) \qquad \Gamma,\ E(z) \vdash E(\sigma(z))}{\Gamma \vdash E(t)} \tag{EI}[o,\sigma]$$

for literal equations E, with z fresh for Γ, t, o and the parameters.

The theory T_J of Theorem 9.13 is ordinary logic with the anchored schemas at $[0, S]$, where $Sx = J(x; i, r.0)$: the instances are literally (J0), (STEP), (Sep) and (9.11).

Definition 9.32 (The theory $\mathsf{T}_J^{\varnothing}$)**.** The formulas of $\mathsf{T}_J^{\varnothing}$ are those of Theorem 9.13 over the terms of Theorem 9.29, and its logical rules are those of Theorem 9.13. Its arithmetic part is the anchored schemas at $[\mathsf{Z}(a), \mathsf{S}]$ for every term a, with bodies ranging over the terms without constants:

$$J(0_a; i, r.h) = 1_a, \qquad J(\mathsf{S}n; i, r.h) = b \leftrightarrow \big((e_n = 0_a \wedge b = \mathsf{S}j_n) \vee e_n = \mathsf{S}b\big), \qquad 0_a = 1_a \to \bot,$$

and the rule (EI$[o, \sigma]$) with base $E(0_a)$, step $E(z) \vdash E(\mathsf{S}z)$ and z fresh also for the anchor. The anchor is part of each instance and is universally quantified in the axioms.

As in every first-order logic with a nonempty domain, a derivation may contain a free variable that occurs neither in its context nor in its conclusion. When the variables in scope are recorded in the judgment, as in the checker of `constantfree`, this is the explicit rule

$$\frac{\Gamma \vdash_{V \cup \{a\}} A}{\Gamma \vdash_V A} \qquad a \notin V,\ a \notin \mathrm{FV}(\Gamma, A). \tag{Param}$$

It discharges an arbitrary element, not an assumption. By Theorem 9.30 it cannot be replaced by the instantiation of a closed term.

Proposition 9.33 (Param is needed in an empty scope)**.** *In the calculus of Theorem 9.32 with variable scopes, in which every term of a derivation is in scope and every axiom instance is a universal closure or an instance of one at terms in scope, the sentence* $\exists x\,(x = x)$ *is not derivable in the empty scope without* (Param)*. With* (Param) *it is derivable.*

Proof. Call a judgment $\Gamma \vdash_V A$ valid if $V \neq \varnothing$, or if $V = \varnothing$ and A is true in the structure with the empty domain whenever all formulas of Γ are. Every rule other than (Param) preserves validity. A rule with a conclusion in a nonempty scope has a valid conclusion. In the empty scope there is no term, by Theorem 9.30, so reflexivity, transport, $\forall$-elimination, $\exists$-introduction, the instances of axioms at terms, and (EI$[o, \sigma]$), whose conclusion and anchor are terms, do not occur there. Universal closures are true in the empty structure, and so is every universal conclusion of $\forall$-introduction. The propositional rules preserve truth. For $\exists$-elimination in the empty scope, the major premise $\exists x\, A$ is false in the empty structure, so its validity makes the context false and the conclusion valid. Since $\exists x\,(x = x)$ is false in the empty structure, it has no derivation in the empty scope. With (Param): in scope $\{a\}$, reflexivity gives $a = a$ and $\exists$-introduction gives $\exists x\,(x = x)$; (Param) discharges a. □

Remark 9.34 (Bodies as function symbols). As in Section 9.3, a fixed body is a function symbol $J_h(n, \vec{p})$ of its parameters, and the schemas are supplied for each one. A capture-avoiding substitution into a body yields another symbol; its agreement with the substituted old one is not an axiom. It follows by external induction on the body: inner occurrences of J are handled by the induction hypothesis, and the outer occurrence by Theorem 9.36 below. `constantfree` gives this argument; its auditors add that it needs neither injectivity of the successor nor coverage.

9.4.2 Zero and the successor

The following two lemmas hold for every instance of Theorem 9.31. They are applied to $\mathsf{T}_J^{\varnothing}$ at $[\mathsf{Z}(a), \mathsf{S}]$ and to T_J at $[0, S]$.

Lemma 9.35 (Constant one and zero)**.** *Let a theory contain ordinary logic with equality, prove every instance of* (J0$[o, \sigma]$) *and of the right-to-left direction of* (STEP$[o, \sigma]$)*, and be closed under* (EI$[o, \sigma]$)*, at one pair* $[o, \sigma]$*. Then it proves, for every term* t*,*

$$\mathsf{K}(o) = \sigma(o), \quad \mathsf{K}(\sigma(o)) = \sigma(\sigma(o)), \quad \mathsf{U}(t) = \sigma(o), \quad J(\sigma(o); i, r.r) = o, \quad \mathsf{Z}(t) = o.$$

Neither (Sep$[o, \sigma]$)*, nor coverage, nor the left-to-right direction of* (STEP$[o, \sigma]$) *is used, and the only induction is one instance of* (EI$[o, \sigma]$) *on the literal equation* $\mathsf{U}(x) = \sigma(o)$*.*

Proof. (J0$[o, \sigma]$) for the body i gives $\mathsf{K}(o) = \sigma(o)$. In (STEP$[o, \sigma]$) for the body i at the counter $n := o$ the instruction is $e_o = o$, literally, and the old register is $j_o = \mathsf{K}(o)$. Take $b := \sigma(\sigma(o))$. The increment disjunct $e_o = o \wedge b = \sigma(\mathsf{K}(o))$ holds by reflexivity and by congruence from $\mathsf{K}(o) = \sigma(o)$, so the right-to-left direction gives $\mathsf{K}(\sigma(o)) = \sigma(\sigma(o))$.

(J0$[o, \sigma]$) for the body $\mathsf{K}(r)$ gives $\mathsf{U}(o) = \sigma(o)$. Assume $\mathsf{U}(z) = \sigma(o)$. In (STEP$[o, \sigma]$) for the body $\mathsf{K}(r)$ at the counter z the instruction is $\mathsf{K}(\mathsf{U}(z))$, which equals $\mathsf{K}(\sigma(o)) = \sigma(\sigma(o))$ by congruence and the preceding paragraph. With $b := \sigma(o)$ the load disjunct $e_z = \sigma(b)$ holds, and the right-to-left direction gives $\mathsf{U}(\sigma(z)) = \sigma(o)$. Now (EI$[o, \sigma]$), applied to the literal equation $\mathsf{U}(x) = \sigma(o)$ in the empty context, gives $\mathsf{U}(t) = \sigma(o)$.

Let $B(n) := J(n; i, r.r)$. (J0$[o, \sigma]$) gives $B(o) = \sigma(o)$. In (STEP$[o, \sigma]$) for the body r at the counter o the instruction is the old register $B(o)$; with $b := o$ the load disjunct $B(o) = \sigma(o)$ holds, so $B(\sigma(o)) = o$. Finally $\mathsf{Z}(t) = B(\mathsf{U}(t))$ literally, and transport along $\mathsf{U}(t) = \sigma(o)$ gives $\mathsf{Z}(t) = B(\sigma(o)) = o$. □

Lemma 9.36 (Register body extensionality)**.** *Let a theory contain ordinary logic with equality, prove every instance of* (J0$[o, \sigma]$) *and of both directions of* (STEP$[o, \sigma]$)*, and be closed under* (EI$[o, \sigma]$)*, at one pair* $[o, \sigma]$*. Let* $h(i, r)$ *and* $k(i, r)$ *be bodies, with arbitrary parameters, and let* i, r *be fresh for* Γ*. If* $\Gamma \vdash h = k$*, then* $\Gamma \vdash J(n; i, r.h) = J(n; i, r.k)$ *for every term* n*. Coverage, separation and injectivity of the successor are not used.*

Proof. Apply (EI$[o, \sigma]$) to the literal equation $E(x)$: $J(x; i, r.h) = J(x; i, r.k)$ in the context Γ, with z fresh. The base holds since both sides equal $\sigma(o)$ by (J0$[o, \sigma]$). For the step write $j = J(z; i, r.h)$, $j' = J(z; i, r.k)$, $e = h[z/i, j/r]$ and $e' = k[z/i, j'/r]$, and assume $j = j'$. Since i, r are not free in Γ, the premise gives $h[z/i, j/r] = k[z/i, j/r]$, and congruence along $j = j'$ gives $e = e'$. Put $c := J(\sigma(z); i, r.h)$. The left-to-right direction of (STEP$[o, \sigma]$) for h at the counter z, applied to the reflexive equation $c = c$, gives

$$(e = o \wedge c = \sigma(j)) \vee e = \sigma(c).$$

Transport along $e = e'$ and $j = j'$ gives $(e' = o \wedge c = \sigma(j')) \vee e' = \sigma(c)$, and the right-to-left direction of (STEP$[o, \sigma]$) for k at the counter z gives $J(\sigma(z); i, r.k) = c$, which is $E(\sigma(z))$ after symmetry. □

In $\mathsf{T}_J^{\varnothing}$ the lemma is applied with an anchor a that is any term in scope, for instance the counter n itself; by Theorem 9.30 the conclusion always contains one. For T_J, Theorem 9.36 is a proof of the identification of two register computations without the coverage split used in part (2) of the proof of Theorem 9.25.

Corollary 9.37 (Zero without a zero term)**.** *Write* $\mathrm{Zero}(z) := (z = \mathsf{Z}(z))$. *The theory* $\mathsf{T}_J^{\varnothing}$ *proves, for all terms* a, b, t,

$$\mathsf{U}(t) = \mathsf{S}(\mathsf{Z}(a)), \qquad \mathsf{Z}(t) = \mathsf{Z}(a), \qquad \mathsf{S}(t) = J(t; i, r.\mathsf{Z}(a)),$$

and the sentences

$$\forall a\, \forall b\ \mathsf{Z}(a) = \mathsf{Z}(b), \qquad \exists z\ \mathrm{Zero}(z), \qquad \forall x\, \forall y\, (\mathrm{Zero}(x) \wedge \mathrm{Zero}(y) \to x = y).$$

Proof. The first two equations are Theorem 9.35 at $[\mathsf{Z}(a), \mathsf{S}]$. The third is Theorem 9.36 for the bodies $\mathsf{Z}(r)$ and $\mathsf{Z}(a)$, whose premise $\mathsf{Z}(r) = \mathsf{Z}(a)$ is the second equation with r free; the anchor a may be taken for the lemma. The first sentence follows by $\forall$-introduction. For existence, the second equation at $t := \mathsf{Z}(a)$ gives $\mathsf{Z}(\mathsf{Z}(a)) = \mathsf{Z}(a)$, so $\mathsf{Z}(a)$ is a witness; the free variable a is discharged as in (Param). For uniqueness, $x = \mathsf{Z}(x) = \mathsf{Z}(y) = y$. □

Corollary 9.38 (The terms in T_J)**.** T_J *proves* $\mathsf{U}(t) = 1$, $\mathsf{Z}(t) = 0$, $\mathsf{S}(t) = St$ *and* $\mathrm{Zero}(z) \leftrightarrow z = 0$ *for all terms* t *and variables* z.

Proof. Theorem 9.35 at $[0, S]$ gives the first two. Theorem 9.36 for the bodies $\mathsf{Z}(r)$ and 0, with the premise $\mathsf{Z}(r) = 0$, gives $\mathsf{S}(t) = J(t; i, r.0) = St$. Finally $\mathsf{Z}(z) = 0$ gives the last equivalence. □

Remark 9.39 (The bootstrap at an anchor). `constantfree` also carries out, at a fixed anchor, the bootstrap of Theorem 9.16 with $f(n, 0) = \mathsf{S}n$ obtained from the third equation of Theorem 9.37, coverage as in Theorem 9.14, the compiler of Theorem 9.17 in the variant with the instruction $\mathsf{S}(C(i, b, h(p(i), r)))$, and the reflection argument of Theorem 9.28 with characteristic terms that use 1 for truth. These are the constructions of Section 9.2.3 with $\mathsf{Z}(a)$ for 0 and S for S. By Theorem 9.44 below they are not needed separately: they hold in $\mathsf{T}_J^{\varnothing}$ as translations of theorems of T_J.

9.4.3 Translations and the bi-interpretation

Definition 9.40 (Translations)**.** The translation from $\mathsf{T}_J^{\varnothing}$ to T_J is the inclusion: every term and formula without constants is a term or formula of T_J. For a term or formula X of T_J and a variable z that does not occur in X, let X_z be X with every occurrence of the constant 0 replaced by z, including the occurrences inside the abbreviations S and 1; thus $(St)_z = J(t_z; i, r.z)$. The translation from T_J to $\mathsf{T}_J^{\varnothing}$ is

$$A^{\flat} := \exists z\, (\mathrm{Zero}(z) \wedge A_z),$$

and it interprets a term t of T_J by the graph $\exists z\, (\mathrm{Zero}(z) \wedge y = t_z)$, with z chosen fresh. Both translations fix the variables, the domain and equality, and $\bot^{\flat}$ is provably equivalent to $\bot$ (Theorem 9.43(3) below).

The replacement commutes with substitution, $(X[s/x])_z = X_z[s_z/x]$, and with every connective and quantifier whose bound variable is not z.

Lemma 9.41 (Inclusion)**.** *If* $\Gamma \vdash A$ *in* $\mathsf{T}_J^{\varnothing}$, *then* $\Gamma \vdash A$ *in* T_J.

Proof. By induction on the derivation. The logical rules are shared. A use of (Param) is admissible in T_J: substitute 0 for the discharged variable throughout the derivation of its premise, renaming eigenvariables. By Theorem 9.38, T_J proves $\mathsf{Z}(a) = 0$ and $\mathsf{S}(x) = Sx$ for all terms a, x. Transport along these equations, at each occurrence, turns every instance of an anchored schema at $[\mathsf{Z}(a), \mathsf{S}]$ into the instance with the same body at $[0, S]$, which is an axiom of T_J. For an instance of (EI$[o, \sigma]$) at $[\mathsf{Z}(a), \mathsf{S}]$ the induction hypotheses give $\Gamma \vdash E(\mathsf{Z}(a))$ and $\Gamma, E(z) \vdash E(\mathsf{S}z)$ in T_J; transport gives $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$, and (9.11) gives $\Gamma \vdash E(t)$. □

Lemma 9.42 (Zero as a parameter)**.** *Let $\Gamma \vdash A$ in T_J and let z be a variable that occurs nowhere in the derivation. Then $\Gamma_z, \mathrm{Zero}(z) \vdash A_z$ in $\mathsf{T}_J^{\varnothing}$.*

Proof. Under the hypothesis $\mathrm{Zero}(z)$ the theory $\mathsf{T}_J^{\varnothing}$ proves

$$\mathsf{Z}(a) = z, \qquad J(t; i, r.z) = \mathsf{S}(t) \tag{9.19}$$

for all terms a, t: the first is $\mathsf{Z}(a) = \mathsf{Z}(z) = z$ by Theorem 9.37, and the second is Theorem 9.36 for the bodies z and $\mathsf{Z}(r)$, with the premise $z = \mathsf{Z}(r)$ given by the first. Now induct on the derivation. A logical rule maps to the same rule with $\mathrm{Zero}(z)$ added to the context; eigenvariables differ from z and do not occur in Γ, hence not in $\Gamma_z, \mathrm{Zero}(z)$; transport and $\forall$-elimination commute with the replacement.

Axioms. $(J(0; i, r.h) = S0)_z$ is $J(z; i, r.h_z) = J(z; i, r.z)$. The instance of (J0$[o, \sigma]$) at the anchor z is $J(\mathsf{Z}(z); i, r.h_z) = \mathsf{S}(\mathsf{Z}(z))$, and transport along $\mathsf{Z}(z) = z$ and $\mathsf{S}(z) = J(z; i, r.z)$ from (9.19) gives the translated axiom. The translation of (STEP) is

$$J\big(J(n_z; i, r.z); i, r.h_z\big) = b \leftrightarrow \big((e' = z \wedge b = J(j'; i, r.z)) \vee e' = J(b; i, r.z)\big),$$

with $j' = J(n_z; i, r.h_z)$ and $e' = h_z[n_z/i, j'/r]$, since the replacement commutes with substitution. The instance of (STEP$[o, \sigma]$) at the anchor z for the body h_z and the counter n_z differs from it only by $\mathsf{Z}(z)$ in place of z and $\mathsf{S}(x)$ in place of $J(x; i, r.z)$ for $x = n_z, j', b$, so transport along (9.19) proves it. The translation of (Sep) is $z = J(z; i, r.z) \to \bot$, and transport turns the instance $\mathsf{Z}(z) = \mathsf{S}(\mathsf{Z}(z)) \to \bot$ of (Sep$[o, \sigma]$) into it.

Induction. For an instance of (9.11) with premises $\Gamma \vdash E(0)$ and $\Gamma, E(y) \vdash E(Sy)$, the induction hypotheses give $\Gamma_z, \mathrm{Zero}(z) \vdash E_z(z)$ and $\Gamma_z, \mathrm{Zero}(z), E_z(y) \vdash E_z(J(y; i, r.z))$. Transport along (9.19) gives $E_z(\mathsf{Z}(z))$ and $E_z(\mathsf{S}y)$, and (EI$[o, \sigma]$) at the anchor z gives $E_z(t_z)$. The invariant E_z is a literal equation, and y differs from z and is fresh for the extended context and the target. □

Lemma 9.43 (The translation $\flat$)**.** *In $\mathsf{T}_J^{\varnothing}$:*

1. *$\mathrm{Zero}(z) \vdash A^{\flat} \leftrightarrow A_z$ for every formula A of T_J and every variable z not in A;*
2. *$\flat$ commutes with every connective and quantifier up to provable equivalence: $(A \circ B)^{\flat} \leftrightarrow A^{\flat} \circ B^{\flat}$ for $\circ \in \{\wedge, \vee, \to\}$, $\bot^{\flat} \leftrightarrow \bot$, $(\forall x\, A)^{\flat} \leftrightarrow \forall x\, A^{\flat}$ and $(\exists x\, A)^{\flat} \leftrightarrow \exists x\, A^{\flat}$;*
3. *$A^{\flat} \leftrightarrow A$ for every formula A without constants.*

Proof. (1) If $\mathrm{Zero}(w)$ and A_w, uniqueness in Theorem 9.37 gives $w = z$, and transport gives A_z. Conversely z is a witness. (2) Let z be a variable free in neither side. By existence in Theorem 9.37 and $\exists$-elimination it suffices to prove the equivalence under $\mathrm{Zero}(z)$. There, by (1), applied under the quantifier for $\forall x$ and $\exists x$ since x is not z, both sides are equivalent to $(A \circ B)_z = A_z \circ B_z$, respectively to $\forall x\, A_z$ and $\exists x\, A_z$. (3) Here A_z is A, and $\exists z\, (\mathrm{Zero}(z) \wedge A) \leftrightarrow A$ by existence. □

Theorem 9.44 (Constant-free presentation)**.**

1. *If* $\mathsf{T}_J^{\varnothing} \vdash A$ *then* $\mathsf{T}_J \vdash A$. *If* $\Gamma \vdash A$ *in* T_J *then* $\Gamma^\flat \vdash A^\flat$ *in* $\mathsf{T}_J^{\varnothing}$.

2. $\mathsf{T}_J^{\varnothing} \vdash A^\flat \leftrightarrow A$ *for every formula* A *of* $\mathsf{T}_J^{\varnothing}$, *and* $\mathsf{T}_J \vdash A^\flat \leftrightarrow A$ *for every formula* A *of* T_J. *The graph of each numerical composite is* $x = y$.

3. *With* $\iota_{\varnothing}(A) := A^\iota$ *and* $\kappa_{\varnothing}(B) := (B^\kappa)^\flat$, *where* ι, κ *are the translations of Theorem 9.24 for* T_J, *the theory* $\mathsf{T}_J^{\varnothing}$ *is bi-interpretable with* T_{eq}, *and hence by Theorem 1.119 with* N *and with* iPRA, *by interpretations whose domains are the whole numerical domain, whose equality is actual equality and whose numerical composites have the graph* $x = y$. *All these theories are equiconsistent.*

Proof. (1) The first statement is Theorem 9.41. For the second take z fresh. By Theorem 9.42, $\Gamma_z, \mathrm{Zero}(z) \vdash A_z$. Under $\mathrm{Zero}(z)$, Theorem 9.43(1) turns every formula of $\Gamma^\flat$ into the corresponding one of Γ_z and A_z into $A^\flat$. Hence $\Gamma^\flat, \mathrm{Zero}(z) \vdash A^\flat$, and $\exists$-elimination on $\exists z\, \mathrm{Zero}(z)$ removes the hypothesis, since z is free in neither $\Gamma^\flat$ nor $A^\flat$.

(2) The first composite is Theorem 9.43(3). For the second, T_J proves $\mathrm{Zero}(z) \leftrightarrow z = 0$ by Theorem 9.38, so $A^\flat \leftrightarrow \exists z\, (z = 0 \wedge A_z) \leftrightarrow A$ by transport, since $A_z[0/z]$ is A. For a term t of T_J the composite graph $\exists z\, (\mathrm{Zero}(z) \wedge y = t_z)$ is equivalent in T_J to $y = t$ in the same way, and a term without constants is mapped to itself by the inclusion and has the graph $\exists z\, (\mathrm{Zero}(z) \wedge y = t)$, equivalent to $y = t$ by Theorem 9.43(3).

(3) By (1) and Theorem 9.25(1), $\mathsf{T}_J^{\varnothing} \vdash A$ implies $\mathsf{T}_{\mathrm{eq}} \vdash A^\iota$, and $\mathsf{T}_{\mathrm{eq}} \vdash B$ implies $\mathsf{T}_J \vdash B^\kappa$ and $\mathsf{T}_J^{\varnothing} \vdash (B^\kappa)^\flat$. For a formula A of $\mathsf{T}_J^{\varnothing}$, Theorem 9.25(3) gives $\mathsf{T}_J \vdash A^{\iota\kappa} \leftrightarrow A$; by (1) and Theorem 9.43(2) the theory $\mathsf{T}_J^{\varnothing}$ proves $(A^{\iota\kappa})^\flat \leftrightarrow A^\flat$, and $A^\flat \leftrightarrow A$ by (2). For a formula B of T_{eq}, (2) gives $\mathsf{T}_J \vdash (B^\kappa)^\flat \leftrightarrow B^\kappa$; by Theorem 9.25(1), since ι commutes with the connectives, $\mathsf{T}_{\mathrm{eq}} \vdash ((B^\kappa)^\flat)^\iota \leftrightarrow B^{\kappa\iota}$, and $B^{\kappa\iota} \leftrightarrow B$ by Theorem 9.25(3). The four translations fix the domain, equality and the variables, so the numerical composites have the graph $x = y$, by (2) and Theorem 9.25(2). Both translations preserve derivability and map $\bot$ to $\bot$ up to provable equivalence ($\bot^\iota = \bot$ and $\bot^\flat \leftrightarrow \bot$), so a derivation of $\bot$ in one theory yields one in the other. □

Remark 9.45 (The direct proof of the package). `constantfree` proves Theorem 9.44(3) against iPRA directly. A body is interpreted by the primitive recursive function j_h with $j_h(0, \vec{p}) = 1$ and $j_h(n+1, \vec{p}) = C(e,\, j_h(n, \vec{p}) + 1,\, p(e))$, $e = h(n, j_h(n, \vec{p}), \vec{p})$; in iPRA the defining equations give $\mathsf{K}(1) = 2$ and $J(1; i, r.r) = 0$, one induction on an equation gives $\mathsf{U}(t) = 1$ and $\mathsf{Z}(t) = 0$, and a second gives $\mathsf{S}(n) = n + 1$, before the anchored schemas are verified. In the other direction it adds a constant c with $c = \mathsf{Z}(c)$, which is a conservative definitional extension by Theorem 9.37, compiles primitive recursion at the anchor c, and removes c by the translation $\flat$. Theorem 9.42 is the same argument with c kept as a variable z. The two routes give the same maps up to provable equality, by Theorem 9.21 in T_{eq} and Theorem 9.36 in $\mathsf{T}_J^{\varnothing}$. The package presents the bi-interpretation as a written proof, with an independent written audit; it is not formalized.

9.4.4 Sizes of the constant terms

Let the terms of Theorem 9.29 denote functions of their free variables on the natural numbers: $J(n; i, r.h)$ is r_n, where $r_0 = 1$, and $r_{m+1} = r_m + 1$ if $h(m, r_m) = 0$ and $r_{m+1} = h(m, r_m) - 1$ otherwise. This is the standard model of T_J, so by Theorem 9.41 it satisfies $\mathsf{T}_J^{\varnothing}$.

Proposition 9.46 (Sharp sizes)**.** *Count occurrences of* J *in fully expanded terms. A term that denotes the constant function* 1 *has at least two occurrences, and a term that denotes the constant function* 0 *has at least three. The terms* $\mathsf{U}(x)$ *and* $\mathsf{Z}(x)$ *attain these bounds.*

Proof. A term without J is a variable and is not constant. A term with exactly one J is $J(x; i, r.v)$ with x a free variable and v a variable. It takes the value 1 at $x = 0$. It also takes the value 0: if v is r, at $x = 1$ the instruction $r_0 = 1$ loads 0; if v is i, at $x = 2$ the instructions 0 and 1 increment to 2 and then load 0; if v is free, possibly x itself, at $x = v = 1$ the instruction 1 loads 0. Hence no term with at most one J is constant, and every such term takes the value 0 at some valuation.

A term with exactly two occurrences is $J(n; i, r.h)$ with n a variable or a term with one occurrence. The counter n is outside the binder, so its free variables are free in the whole term, and by the preceding paragraph some valuation makes $n = 0$. There the term takes the value 1, whatever h is. A term with at most two occurrences therefore takes the value 1 somewhere and is not constantly 0.

$\mathsf{U}(x)$ has two occurrences and $\mathsf{Z}(x)$ three, and by Theorem 9.38 they denote 1 and 0. □

The bound concerns raw term size only. It is not a bound on the size of an axiomatization or of a checker.

9.4.5 The reference checker

The checker of `constantfree` is a standalone program for $\mathsf{T}_J^{\varnothing}$ in ordinary logic. Its numerical term tags are a variable and J with a counter, two bound names and a body; zero and the successor are expanded raw terms. The input is a JSON object with a closed goal and a proof tree in which every node states its conclusion. Every arithmetic instance names its anchor, which is checked to be in scope; the anchor of an induction may not depend on its eigenvariable, and the invariant must be a literal equation. The biconditional (STEP$[o, \sigma]$) is expanded into a conjunction of implications, so increments and loads are checked by logical rules and not by evaluation. (Param) is a primitive instruction. Four certificates are supplied: $\forall a\, \forall t\ \mathsf{U}(t) = \mathsf{S}(\mathsf{Z}(a))$, $\forall a\, \forall t\ \mathsf{Z}(t) = \mathsf{Z}(a)$, $\exists z\, (z = \mathsf{Z}(z))$, which uses (Param), and $\forall x\, \forall y\, ((x = \mathsf{Z}(x) \wedge y = \mathsf{Z}(y)) \to x = y)$. The validation runs 19 certificate cases in ordinary and optimized Python, 38 executions, including the purely logical theorem $\exists x\, (x = x)$ and fourteen rejected variants (an escaping anchor, an anchor depending on the induction variable, a zero constructor, induction on a conjunction, malformed input), and 6 checks of capture-avoiding substitution. These are finite evidence for the implementation. The compact refinements shipped in the same package concern the scalar kernel and are treated with the certificate checkers.

9.5 Guarded scalar registers

This section draws on `guarded_register`.

The register languages of Sections 9.2.3 and 9.4 state a computation step by the graph biconditional (STEP), whose right-to-left direction is the pair of conditional rules (9.12) and (9.13). A restriction on the bodies of the register with one bound variable makes both computations unconditional equations: a body either does not depend on the state, or is literally a successor. The resulting native calculus $\mathcal{K}_{\mathrm{G}}$ has the formulas and the six logical rules of N and two computation equations in place of I0 and IS. It has no successor constructor, no initializer, no reflexivity rule and no conditional increment or load. Theorem 9.57 proves it equivalent to N, with both composites.

9.5.1 Terms and the guard

Definition 9.47 (Guarded terms)**.** The raw terms are

$$t ::= x \mid 0 \mid J(n; u.h),$$

with u bound in h only; the counter n is outside the binder. Put

$$Sa := J(a; v.0), \qquad 1 := S0, \qquad 2 := S1, \qquad S^{k+1}a := S(S^k a),$$

with v unused. A raw term is *guarded* if at every occurrence $J(n; u.h)$ the terms n and h are guarded and at least one of the following holds:

1. $u \notin \mathrm{FV}(h)$: the body is *state-free*; passive parameters are allowed;
2. $h \equiv S(g)$, that is $h \equiv J(g; v.0)$, for some term g, which may contain u: the body is *successor-headed*.

The two conditions may overlap. They are tests on literal syntax, not on provable equality or on values. Below every term is guarded, and a *template* $h(v)$ is a guarded term with a distinguished variable v.

As with R in Theorem 9.1, the number of bound variables distinguishes this J from the register $J(n; i, r.h)$ of Section 9.2.3; the one-variable register of Theorem 9.18 is written J_s. On the natural numbers $J(n; u.h)$ denotes r_n, where

$$r_0 = h(0) + 1, \qquad r_{m+1} = \begin{cases} r_m + 1, & h(r_m) = 0, \\ h(r_m) - 1, & h(r_m) > 0. \end{cases}$$

The transition is that of J_s: the instruction 0 increments the register and an instruction $b+1$ loads b. The start differs. The register J_s starts at 1, while $J(0; u.h)$ is $S(h(0))$, which is the base equation of the self-seeded constructor F of Section 9.13. That constructor has no term for the predecessor (Theorem 9.140); the guarded register has one (Theorem 9.51), because its step at a successor-headed body removes the successor head. The guard excludes the bodies of J_s that depend on the state and are not literally successors, for instance the identity body u; the definition of zero in Theorem 9.29 uses bodies of that kind, and the guarded language keeps the constant 0. At a successor-headed body only the load branch of the transition can occur, and the equation (GR) below states it. At a state-free body the instruction a does not change along the computation, and the step at an arbitrary a follows from the two cases $a = 0$ and $a = Sz$ (Theorem 9.54). Neither case needs the case distinction on a state-dependent instruction that (STEP) provides.

Lemma 9.48 (Substitution closure)**.** *If t and s are guarded, then the capture-avoiding substitution $t[s/x]$ is guarded. In particular $h[s/u]$ is guarded for a guarded body h and a guarded term s, and renaming of bound variables preserves the guard.*

Proof. Induct on t, with the bound variable of each occurrence of J renamed away from x and from $\mathrm{FV}(s)$. Variables and 0 are immediate. For $t = J(n; u.h)$, $t[s/x] = J(n[s/x]; u.h[s/x])$, and the counter and the body are guarded by the induction hypothesis. If $u \notin \mathrm{FV}(h)$, then $u \notin \mathrm{FV}(h[s/x])$, since u is not free in s. If $h \equiv J(g; v.0)$, then $h[s/x] \equiv J(g[s/x]; v.0)$, since the zero body has no free variable. Renaming changes neither the set of free variables of a body other than its bound variable nor the shape $J(g; v.0)$. □

Definition 9.49 (The calculus $\mathcal{K}_{\mathrm{G}}$)**.** The formulas are the native formulas of Section 1.3 over guarded terms, with the reading of Section 1.2, and contexts are finite lists of formulas. The rules are HYP, TR, QE, DE, DI and QI of Theorem 1.7, with the same side conditions, where the template of TR is a guarded template and Sz in DI and QI is the defined successor $J(z; v.0)$, together with the two computation equations, in every context,

$$J(0; u.h) = S(h[0/u]), \tag{GB}$$

$$J(Sn; u.Sg) = g[J(n; u.Sg)/u], \tag{GR}$$

for every guarded $J(0; u.h)$ and all guarded n and g. The counter of (GR) is literally $J(n; v.0)$, and its body Sg is admitted whatever g is. The designated contradiction is $0 = 1$.

Compared with N, the terms $0, S, I$ are replaced by $0, J$, and the computations I0, IS by (GB), (GR); the other six rules are literally those of Theorem 1.7. As in N (Theorem 1.8) there is no reflexivity rule. Compared with T_J, the conditional rules (9.12) and (9.13) are not primitive (Theorem 9.52). The guard, the test that the state does not occur and the test of the successor head belong to the trusted specification; by Theorem 9.48 they are stable under substitution.

Lemma 9.50 (Structural rules and equality)**.** *In $\mathcal{K}_{\mathrm{G}}$:*

1. *weakening, substitution and cut are admissible, as in Theorems 1.10 and 1.11;*
2. *$\Gamma \vdash J(1; u.St) = t$ for $u \notin \mathrm{FV}(t)$, and the equality rules of Theorem 1.18 hold: reflexivity, symmetry, transitivity, congruence in every guarded template, transport in equation templates, and simultaneous congruence;*
3. *equation induction (Theorem 1.20), numerical cases (Theorem 1.14), congruence of Q and D (Theorem 1.21) and the Leibniz rule (Theorem 1.22) hold, with Sz the defined successor.*

Proof. The proofs in Section 1.4 use the computation rules only in the instance cases of Theorems 1.10 and 1.11 and in Theorem 1.18(i). (1) For a substitution θ with u fresh for θ,

$$J(0; u.h)[\theta] = J(0; u.h[\theta]), \qquad S(h[0/u])[\theta] = S(h[\theta][0/u]),$$

and $J(Sn; u.Sg)[\theta] = J(S(n[\theta]); u.S(g[\theta]))$, $(g[J(n; u.Sg)/u])[\theta] = g[\theta][J(n[\theta]; u.S(g[\theta]))/u]$. The substituted terms are guarded by Theorem 9.48, so an instance of (GB) or (GR) becomes an instance of the same equation, and the remaining cases are those of the two base lemmas. (2) (GR) at the counter 0, with the constant $g := t$ and $u \notin \mathrm{FV}(t)$, is $J(1; u.St) = t$, since $1 \equiv S0$. TR with the template v, applied to this equation as both premises, gives $t = t$. This replaces the instances of I0 in Theorem 1.18(i); parts (ii)–(v) and Theorem 1.19 are proved by TR alone, with templates that are guarded by Theorem 9.48. (3) The proofs of these statements use HYP, TR, QE, DE, DI, QI, parts (1) and (2), and the term Sz only as it occurs in the conclusions of DI and QI. They apply verbatim. □

9.5.2 Arithmetic from the two equations

With fresh bound variables, and $u \notin \mathrm{FV}(a)$ in the first definition, put

$$f(n, a) := J(n; u.a), \qquad W(x) := f(1, x), \qquad Y(n) := f(n, 1),$$
$$Z(c) := W^2(Y(Y(c))), \quad G(c, a) := W^3(f(S^2a, Z(c))), \quad C(c, a, b) := W^3(f(S^2a, G(c, S^4b))),$$

and $P(c) := C(c, 0, W(c))$, where the powers are finite compositions. In this section the letters f, W, Z, G, C, P denote these terms; the terms of the same names in Theorem 9.16 are their analogues for the register with two bound variables. `guarded_register` writes Z_2 for Y. Every occurrence of J in these terms has a state-free body, so they are guarded for all guarded arguments, and a template such as $f(S^2a, v) = J(S^2a; u.v)$, with the hole in the body, is guarded.

Lemma 9.51 (Bootstrap from two equations)**.** *Using only* (GB), (GR) *and the equality rules of Theorem 9.50(2), $\mathcal{K}_{\mathrm{G}}$ derives in every context, for all terms,*

$$f(0, a) = Sa, \qquad f(n, 0) \equiv Sn, \qquad f(Sn, Sa) = a, \tag{9.20}$$

and

$$\begin{array}{llll} W(0)=2, & W(Sx)=x, & Y(0)=2, & Y(Sn)=0, \\ W^3(0)=0, & W^3(S^3a)=a, & Z(0)=1, & Z(Sn)=0, \\ G(0,a)=0, & G(Sn,a)=a, & C(0,a,b)=a, & C(Sn,a,b)=b, \\ P(0)=0, & P(Sn)=n. & & \end{array}$$

Consequently $Sx = Sy \vdash x = y$ *and* $0 = 1 \vdash a = b$ *for all terms* a, b.

Proof. In (9.20) the first equation is (GB) for the state-free body a, the second is the definition of S up to the name of the bound variable, and the third is (GR) with the constant $g := a$. Hence $W(0) = f(1,0) \equiv S1 = 2$, $W(Sx) = f(S0, Sx) = x$, $Y(0) = f(0,1) = S1$ and $Y(Sn) = f(Sn, S0) = 0$. The map W removes one successor and cycles through $0, 2, 1$: $W(0) = 2, W(2) = W(S1) = 1$ and $W(1) = W(S0) = 0$, with congruence at each composition. Thus $W^3(0) = 0$ and $W^3(S^3a) = a$.

At $c = 0$ the two inner applications of Y give $Y(Y(0)) = Y(S1) = 0$, and $W^2(0) = W(2) = 1$; at $c = Sn$ they give $Y(0) = 2$, and $W^2(2) = W(1) = 0$. For $G(0,a)$ the instruction is $Z(0) = 1 = S0$, so congruence in the template $W^3(f(S^2a, v))$ and (GR) give $W^3(f(S(Sa), S0)) = W^3(0) = 0$. For $G(Sn,a)$ the instruction is 0, and $f(S^2a, 0) \equiv S^3a$, so the value is $W^3(S^3a) = a$. For $C(0,a,b)$ the instruction $G(0, S^4b)$ is 0, giving $W^3(S^3a) = a$. For $C(Sn,a,b)$ the instruction is S^4b, and $f(S(Sa), S(S^3b)) = S^3b$ by the third equation of (9.20), so the value is $W^3(S^3b) = b$. The rows of P follow: $P(0) = C(0,0,W(0)) = 0$ and $P(Sn) = C(Sn, 0, W(Sn)) = W(Sn) = n$.

From $Sx = Sy$ congruence in W gives $W(Sx) = W(Sy)$, hence $x = y$. From $0 = 1$, TR with the template $C(v,a,b)$ and the minor premise $C(0,a,b) = a$ gives $a = C(1,a,b)$, and $C(1,a,b) = C(S0,a,b) = b$. □

No induction, numerical case distinction, coverage or previously available selector is used. The finite decoder W^3 fixes 0 and removes three successors; this is the property used in G and C.

Proposition 9.52 (The conditional rules are derivable)**.** *For a guarded term* $J(n; u.h)$ *write* $j_n := J(n; u.h)$ *and* $j_{Sn} := J(Sn; u.h)$. *The rules*

$$\frac{\Gamma \vdash h[j_n/u] = Sb}{\Gamma \vdash j_{Sn} = b}\ \textit{LOAD}, \qquad \frac{\Gamma \vdash h[j_n/u] = 0}{\Gamma \vdash j_{Sn} = Sj_n}\ \textit{INC}$$

are derivable in $\mathcal{K}_\mathrm{G}$ *from* (GB), (GR) *and* TR, *without* QE, DE, DI *and* QI. *They are the native forms of* (9.13) *and* (9.12).

Proof. LOAD. If $h \equiv Sg$, (GR) gives $j_{Sn} = g[j_n/u]$, and the premise $S(g[j_n/u]) = Sb$ gives $g[j_n/u] = b$ by injectivity (Theorem 9.51). If $u \notin \mathrm{FV}(h)$, write $h \equiv a$; the premise is $a = Sb$, and congruence in the template $f(Sn, v)$ gives $j_{Sn} = f(Sn, a) = f(Sn, Sb) = b$ by (9.20).

INC. If $u \notin \mathrm{FV}(h)$, write $h \equiv a$; the premise is $a = 0$. Congruence in $f(n, v)$ and in $f(Sn, v)$ gives $j_n = f(n, 0) \equiv Sn$ and $j_{Sn} = f(Sn, 0) \equiv S(Sn)$, and congruence in Sv gives $Sj_n = S(Sn)$; transitivity concludes. If $h \equiv Sg$, the premise is $S(g[j_n/u]) = 0$; congruence in Z and Theorem 9.51 give $0 = Z(S(g[j_n/u])) = Z(0) = 1$, and the equational explosion of Theorem 9.51 gives the conclusion. □

Theorem 9.19 derives reflexivity from the conditional rules; in $\mathcal{K}_\mathrm{G}$ reflexivity comes from (GR) (Theorem 9.50), and the conditional rules come from the two equations and TR. The body restriction is what permits their removal from the rules.

9.5.3 Initialized iteration

Theorem 9.53 (Initialized iteration)**.** *Let a be a guarded term and $h(v)$ a guarded template, with all passive parameters fixed. With w fresh put*

$$H(w) := S\Big(C\big(w,\, a,\, S^2(h(P^2(w)))\big)\Big), \qquad A(k) := J(k; w.H(w)), \qquad I_{\mathrm{G}}(n, a; v.h) := P^2\big(A(n)\big).$$

These terms are guarded, since H is literally successor-headed, even when h depends on its variable. $\mathcal{K}_{\mathrm{G}}$ derives in every context, for every term n,

$$I_{\mathrm{G}}(0, a; v.h) = a, \qquad I_{\mathrm{G}}(Sn, a; v.h) = h\big(I_{\mathrm{G}}(n, a; v.h)\big). \tag{9.21}$$

The proof uses (GB)*,* (GR)*, equality and one equation induction, and no coverage, pairing or logical connective.*

Proof. By the zero row of C, $H(0) = Sa$, and (GB) gives $A(0) = S(H(0)) = S^2a$; hence $P^2(A(0)) = a$ by the rows of P. This is the first equation. Let k be fresh for the context, n, a and h, and consider the literal equation

$$A(k) = S^2\big(P^2(A(k))\big). \tag{9.22}$$

At $k = 0$ both sides equal S^2a. For the step assume (9.22) and write $q := P^2(A(k))$. (GR) for the successor-headed body H gives

$$A(Sk) = C\big(A(k),\, a,\, S^2(h(q))\big). \tag{9.23}$$

Transport of (9.22) in the first argument of C makes the control $S(Sq)$, and the successor row of C gives $A(Sk) = S^2(h(q))$. Applying P twice gives $P^2(A(Sk)) = h(q)$, and congruence in S^2v gives (9.22) at Sk. Equation induction (Theorem 9.50(3)) proves (9.22) at every term, in particular at n. Repeating the step calculation with this instance at n in place of the induction hypothesis gives $A(Sn) = S^2(h(P^2(A(n))))$, and applying P twice gives the second equation of (9.21). □

If the counter n also occurs in a or h, those occurrences stay passive: the induction is on the fresh k, and n is substituted only in the proved instance. In Theorem 9.18 the shape of the state is a consequence of one computation; here it is the invariant (9.22), proved by one induction on a literal equation.

Lemma 9.54 (Unconditional recursion)**.** *Let* $\mathrm{Sel}(c, a, b)$ *and* $\mathrm{Pd}(c)$ *be guarded templates such that $\mathcal{K}_{\mathrm{G}}$ proves* $\mathrm{Sel}(0, a, b) = a$*,* $\mathrm{Sel}(Sc, a, b) = b$ *and* $\mathrm{Pd}(Sc) = c$ *for all terms a, b, c. For every guarded term $J(n; u.h)$, with $j := J(n; u.h)$, $\mathcal{K}_{\mathrm{G}}$ derives in every context*

$$J(0; u.h) = S(h[0/u]), \qquad J(Sn; u.h) = \mathrm{Sel}(h[j/u],\, Sj,\, \mathrm{Pd}(h[j/u])).$$

In particular this holds for $\mathrm{Sel} = C$ *and* $\mathrm{Pd} = P$*. The proof uses numerical cases and no coverage.*

Proof. The first equation is (GB). If $h \equiv Sg$, put $g' := g[j/u]$; (GR) gives $J(Sn; u.h) = g'$, and $\mathrm{Sel}(Sg', Sj, \mathrm{Pd}(Sg')) = \mathrm{Pd}(Sg') = g'$. Otherwise $u \notin \mathrm{FV}(h)$; write $h \equiv a$, so that $h[j/u] \equiv a$. Let x be a fresh variable and consider the literal equation

$$E(x)\colon \quad \mathrm{Sel}\big(x,\, S(J(n; u.x)),\, \mathrm{Pd}(x)\big) = J(Sn; u.x),$$

whose bodies are state-free. At $x = 0$ the zero row of Sel makes the left side equal to $S(J(n; u.0))$, which is literally $S(Sn)$; the right side $J(Sn; u.0)$ is the same term up to the name of the bound variable. At $x = Sz$, with z fresh, the left side is $\mathrm{Pd}(Sz) = z$ and the right side is z by (GR) with the constant $g := z$. Numerical cases (Theorem 9.50(3)) give $E(a)$, which is the claim, since $J(n; u.x)[a/x] \equiv J(n; u.a) = j$ and $u \notin \mathrm{FV}(a)$. □

9.5.4 Equivalence with the native core

Definition 9.55 (Translations)**.** The translation $s \mapsto s^\lambda$ from terms of N to guarded terms fixes variables and 0 and sets

$$(Ss)^\lambda := J(s^\lambda; v.0), \qquad I(n, a;\, u.\, h)^\lambda := I_{\mathrm{G}}(n^\lambda, a^\lambda; u.h^\lambda).$$

The translation $t \mapsto t^\mu$ from guarded terms to terms of N fixes variables and 0 and sets, with pd the predecessor term of Theorem 1.65 (written P there) and sel the selector of Section 1.1,

$$J(n; u.h)^\mu := I(n^\mu, S(h^\mu[0/u]);\, u.\, \mathrm{sel}(h^\mu, Su, \mathrm{pd}(h^\mu))).$$

Both are extended to native formulas and contexts so that they fix $=$, Q_y and $D_{x,y}$. Put $\sigma(x) := (Sx)^\mu = I(x, S0;\, u.\, \mathrm{sel}(0, Su, \mathrm{pd}(0)))$.

Both translations are compositional, so they commute with capture-avoiding substitution and preserve the free variables of a term; they map literal equations to literal equations. Every λ-image is guarded, by Theorems 9.48 and 9.53. The term $(Ss)^\lambda$ is literally the defined successor of s^λ, while $(St)^\mu$ is $\sigma(t^\mu)$, which is only provably the successor.

Lemma 9.56 (The reverse translation of the axioms)**.** N *proves* $\sigma(t) = St$ *for every term* t*, and every* μ*-translation of an instance of* (GB) *or* (GR)*, in every context.*

Proof. Successor. Apply Theorem 1.20 to $\sigma(z) = Sz$. The base is I0. In the step IS and (1.4) give $\sigma(Sz) = \mathrm{sel}(0, S\sigma(z), \mathrm{pd}(0)) = S\sigma(z)$, and congruence along the hypothesis gives SSz.

(GB). The left side translates to a term whose counter is 0, so I0 gives $S(h^\mu[0/u])$. The right side translates to $\sigma(h^\mu[0/u])$, since $(h[0/u])^\mu = h^\mu[0/u]$, and this equals $S(h^\mu[0/u])$ by the first part.

(GR). Put $s := (Sg)^\mu = \sigma(g^\mu)$ and $K(c) := I(c, S(s[0/u]);\, u.\, \mathrm{sel}(s, Su, \mathrm{pd}(s)))$, so that $J(m; u.Sg)^\mu = K(m^\mu)$ for every m. The left side translates to $K(\sigma(n^\mu))$. By the first part and congruence it equals $K(Sn^\mu)$, and IS gives, with $k := K(n^\mu)$ and $g' := g^\mu[k/u]$,

$$K(Sn^\mu) = \mathrm{sel}(\sigma(g'), Sk, \mathrm{pd}(\sigma(g'))) = \mathrm{sel}(Sg', Sk, \mathrm{pd}(Sg')) = \mathrm{pd}(Sg') = g',$$

by the first part, (1.4) and Theorem 1.65. The right side translates to $g^\mu[K(n^\mu)/u] = g'$. □

Theorem 9.57 (Guarded registers and the native core)**.**

1. *If* $\Gamma \vdash_{\mathsf{N}} A$ *then* $\Gamma^\lambda \vdash_{\mathcal{K}_{\mathrm{G}}} A^\lambda$*. If* $\Gamma \vdash_{\mathcal{K}_{\mathrm{G}}} A$ *then* $\Gamma^\mu \vdash_{\mathsf{N}} A^\mu$*.*
2. $\mathcal{K}_{\mathrm{G}} \vdash t^{\mu\lambda} = t$ *for every guarded term* t*, and* $\mathsf{N} \vdash s^{\lambda\mu} = s$ *for every term* s *of* N*. Every formula is interderivable with its double translation, in* $\mathcal{K}_{\mathrm{G}}$ *and in* N*.*
3. $\Gamma \vdash_{\mathcal{K}_{\mathrm{G}}} A$ *if and only if* $\Gamma^\mu \vdash_{\mathsf{N}} A^\mu$*, and* $\Gamma \vdash_{\mathsf{N}} A$ *if and only if* $\Gamma^\lambda \vdash_{\mathcal{K}_{\mathrm{G}}} A^\lambda$*. Moreover* $\vdash_{\mathcal{K}_{\mathrm{G}}} 0 = 1$ *if and only if* $\vdash_{\mathsf{N}} 0 = S0$*.*
4. *Consequently* $\mathcal{K}_{\mathrm{G}}$ *is bi-interpretable with* T_{eq} *and with* iPRA*, by the compositions of* λ *and* μ *with the translations of Theorem 1.119, with whole domains, actual equality and numerical composites of graph* $x = y$*, and* $\mathcal{K}_{\mathrm{G}}$*,* N*,* T_{eq} *and* iPRA *are equiconsistent.*

Proof. (1) *From* N *to* $\mathcal{K}_{\mathrm{G}}$*.* Induct on the derivation. HYP is unchanged. An instance of I0 translates to the first equation of (9.21). An instance of IS translates to $I_{\mathrm{G}}(J(n^\lambda; v.0), a^\lambda; u.h^\lambda) = h^\lambda[I_{\mathrm{G}}(n^\lambda, a^\lambda; u.h^\lambda)/u]$, since λ commutes with substitution, and this is the second equation of (9.21). TR with the template h translates to TR with the guarded template h^λ. QE and DE translate to themselves, with the side condition of DE preserved since λ preserves free variables. In DI and QI the translated conclusion of the premise is $B^\lambda(t^\lambda, J(z; v.0))$, respectively $E^\lambda(J(z; v.0))$, which is literally the premise of the rule of $\mathcal{K}_{\mathrm{G}}$; E^λ is a literal equation.

From $\mathcal{K}_{\mathrm{G}}$ to N. Induct on the derivation. HYP, TR, QE and DE translate to themselves, as above. The computation equations are Theorem 9.56. In DI the induction hypothesis gives $\Gamma^\mu, B^\mu(t^\mu, 0) \vdash_{\mathsf{N}} B^\mu(t^\mu, \sigma(z))$. The Leibniz rule (Theorem 1.22) along $\sigma(z) = Sz$ (Theorem 9.56) gives $B^\mu(t^\mu, Sz)$, and DI of N applies; z is fresh since μ preserves free variables. QI is the same with the literal equation E^μ.

(2) *Terms of* N. Induct on s. For $s = Ss_0$, $s^{\lambda\mu} = \sigma(s_0^{\lambda\mu})$, which equals Ss_0 by Theorem 9.56 and the induction hypothesis. For $s = I(n, a;\, u.\, h)$ let $\Phi(k) := I_{\mathrm{G}}(k, a^\lambda; u.h^\lambda)^\mu$ with k fresh, so that $s^{\lambda\mu} = \Phi(n^{\lambda\mu})$. The μ-translations of the two equations (9.21) at k are derivable in N by part (1); they are $\Phi(0) = a^{\lambda\mu}$ and $\Phi(\sigma(k)) = h^{\lambda\mu}[\Phi(k)/u]$, and Theorem 9.56 turns the second into $\Phi(Sk) = h^{\lambda\mu}[\Phi(k)/u]$. By the induction hypotheses $a^{\lambda\mu} = a$ and $h^{\lambda\mu} = h$, the latter with u free. The term $I(k, a;\, u.\, h)$ has the base a and the update h. The proof of Theorem 9.21 uses equality, an instance of a derived equation and equation induction, and is available in N (Theorems 1.18 and 1.20); it gives $\Phi(k) = I(k, a;\, u.\, h)$, and the instance at $n^{\lambda\mu} = n$ is the claim.

Guarded terms. Induct on t; variables and 0 are fixed. Let $t = J(n; u.h)$. With $\mathrm{Sel}(c, a, b) := I_{\mathrm{G}}(c, a; w.b)$, w not free in b, and $\mathrm{Pd}(c) := \big(\mathrm{pd}(x)\big)^\lambda[c/x]$,

$$t^{\mu\lambda} = \Psi(n^{\mu\lambda}), \qquad \Psi(k) := I_{\mathrm{G}}\Big(k,\ S\big(h^{\mu\lambda}[0/u]\big);\ u.\,\mathrm{Sel}\big(h^{\mu\lambda}, Su, \mathrm{Pd}(h^{\mu\lambda})\big)\Big),$$

because λ commutes with substitution and $\mathrm{sel}(c, a, b)$ is $I(c, a;\, w.\, b)$. By (9.21), $\mathrm{Sel}(0, a, b) = a$ and $\mathrm{Sel}(Sc, a, b) = b$. By Theorem 1.65, $\mathsf{N} \vdash \mathrm{pd}(Sx) = x$, and part (1) gives $\mathcal{K}_{\mathrm{G}} \vdash \mathrm{Pd}(Sx) = x$, since $(Sx)^\lambda$ is literally Sx; substitution gives it for every c. So Theorem 9.54 applies to Sel and Pd. Apply equation induction to $\Psi(k) = J(k; u.h)$ with k fresh. At 0, (9.21), the induction hypothesis $h^{\mu\lambda} = h$ with u free, substitution of 0 for u and (GB) give $\Psi(0) = S(h[0/u]) = J(0; u.h)$. In the step, with $j := J(k; u.h)$ and the hypothesis $\Psi(k) = j$, (9.21), congruence and the induction hypothesis substituted at j give $\Psi(Sk) = \mathrm{Sel}(h[j/u], Sj, \mathrm{Pd}(h[j/u]))$, which is $J(Sk; u.h)$ by Theorem 9.54. The instance at $n^{\mu\lambda} = n$ is the claim. This compares the recurrence of the double translation directly with the original register; no intermediate term $J(n; u.h^{\mu\lambda})$, which need not be guarded, is formed.

Formulas. Induct on the formula. For an equation use the term identities with symmetry and transitivity. For $Q_y B$ and $D_{x,y} B$ the induction hypothesis gives interderivability of B and its double translation, with x, y free, and the congruence of Q and D (Theorem 1.21, and Theorem 9.50(3) for $\mathcal{K}_{\mathrm{G}}$) applies.

(3) If $\Gamma^\mu \vdash_{\mathsf{N}} A^\mu$, then $\Gamma^{\mu\lambda} \vdash_{\mathcal{K}_{\mathrm{G}}} A^{\mu\lambda}$ by (1), and (2) with cut (Theorem 9.50(1)) gives $\Gamma \vdash_{\mathcal{K}_{\mathrm{G}}} A$. The other equivalence is symmetric, with Theorem 1.11. The formula $(0 = S0)^\lambda$ is literally $0 = 1$, and $(0 = 1)^\mu$ is $0 = \sigma(0)$, which is interderivable in N with $0 = S0$ by Theorem 9.56.

(4) By (3) and Theorem 1.119(i) and (iii), derivability in $\mathcal{K}_{\mathrm{G}}$ corresponds to derivability in T_{eq} and in iPRA under the composite translations. For the round trips, a formula A of $\mathcal{K}_{\mathrm{G}}$ is interderivable with $A^{\mu\lambda}$ by (2), and the round trip of Theorem 1.119 makes A^μ interderivable in N with its double translation through T_{eq} or iPRA; by (1) the λ-images of interderivable formulas are interderivable. In the other direction a formula ψ of iPRA is equivalent to its round trip through N by Theorem 1.119(iii), and the formula of N obtained on the way is interderivable with its $\lambda\mu$-translation by (2); since derivability in N is preserved by the translation to iPRA, iPRA proves the equivalence of ψ with its round trip through $\mathcal{K}_{\mathrm{G}}$. The same argument applies to T_{eq} with Theorem 1.119(i). The translations λ and μ fix the variables, the binders and equality, and on terms the composites are provably the identity by (2); with Theorem 1.119(iv) the numerical composites have the graph $x = y$. Equiconsistency is (3) with Theorem 1.119(v). □

Remark 9.58 (The route of the package). `guarded_register` proves Theorem 9.57 with the same two translations. It interprets $\mathcal{K}_{\mathrm{G}}$ in the ordinary scalar theory by μ, reading DI by coverage and QI by equation induction; it reconstructs the logical connectives inside $\mathcal{K}_{\mathrm{G}}$ by gates and a selector of formulas built from the term C, which is the construction of Theorem 1.28 with C in place of sel; and it proves Theorem 9.54 for a state-free body by numerical coverage of the instruction and elimination of the derived disjunction. The proof above replaces coverage by numerical cases at a fresh variable, so the logical interface is not needed for the equivalence; the logic of $\mathcal{K}_{\mathrm{G}}$ is that of N, transferred by Theorem 9.57(3). The two composites are proved inside the theories, by equation induction, and not by agreement on the standard model.

Remark 9.59 (The ordinary realization). `guarded_register` also states an ordinary theory T_{G}: intuitionistic first-order logic with equality and an independent $\bot$ over guarded terms, the universal closures of (GB) and (GR), coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$, the separation $0 = 1 \to \bot$, and the rule (9.11) of induction for literal equations. With λ and μ extended to commute with the connectives, it is bi-interpretable with T_{eq} by the argument of Theorem 9.57. In T_{eq} the μ-translations of the two equations hold by Theorem 9.56 and Theorem 1.119(i), and coverage, separation and induction translate to the corresponding principles of T_{eq} after transport along $\sigma(x) = Sx$. In T_{G} the proofs of Theorems 9.51 and 9.53 use only equality, the two equations and induction on a literal equation; the axioms of T_{eq} translate to (9.21), to coverage and to separation, the last two literally. The term composites are proved as in Theorem 9.57(2), with coverage in place of numerical cases, and the formula composites follow by transport. The separation axiom is needed: a one-element structure satisfies the two equations, coverage and every instance of induction, and refutes $0 = 1 \to \bot$.

Remark 9.60 (A closure meaning that the eliminations do not fit). `guarded_register` records an obstruction for a different shortcut. Keep the elimination QE and the introduction QI of N, and read the closure $Q_y E$ of a literal equation progressively, as $\forall y\,(E(y) \to E(Sy))$. The rules are sound for this reading, but not complete for it: under the reading of Section 1.2, $Q_y E$ is $E(0) \to \forall y\, E(y)$, every rule of N is sound, and for $E(y) := (y = 1)$ the formulas $Q_y E$ and $E(1)$ are true on the natural numbers while $E(2)$ is false. Hence $Q_y E, E(1) \nvdash_{\mathsf{N}} E(2)$, although the sequent is valid under the progressive reading. A faithful progressive reading requires a corresponding change of the elimination rule.

9.5.5 The strict checker

The checker of `guarded_register` is a standalone module that imports only the JSON library, reads a certificate $[goal, proof]$ with a closed goal and prints `ACCEPT` only after checking the goal and the proof; every check is a runtime condition, so optimized Python does not remove it. Variables are one-based de Bruijn indices and the integer 0 is the constant. A guarded term is $[\mathtt{j}, n, h]$, with h binding the index 1; the state-free test is `subst(h,(0,),1)==h`, which replaces the bound state by 0 and keeps all outer indices, and the alternative test is that h is a J node with literal zero body. Both computation nodes validate the whole constructed J term, so an unguarded body cannot enter through an axiom, and the node for (GR) takes n and g, has no equation premise and no output witness. The six logical nodes are those of the matched control for iterator terms, which has the term tags S and I and no body restriction. The validation runs 108 subprocess checks in ordinary and optimized Python (48 acceptances, 60 rejections, no mismatch), among them rejected certificates that use the removed increment and conditional load, unguarded bodies in auxiliary nodes and quantified induction, and 12 parameterized certificates of the equations of Theorem 9.51 for f, W, Z, G, C and P, checked with an empty context. These are finite evidence for the implementation.

9.6 The seeded recursor: unrestricted calculi and six rules

This section draws on `canonical_recursors`.

The native formulas are those of Section 1.3, $A ::= (s = t) \mid Q_y A \mid D_{x,y} A$, over the terms of the seeded recursor with one bound variable, with the reading of Section 1.2. Contexts are finite reusable lists of formulas, and the designated contradiction is $0 = S0$. The six-rule calculus has two tests on literal syntax, which are not stable under substitution. The comparison with the raw calculus is therefore established first for two unrestricted calculi, and the six rules are recovered afterwards.

9.6.1 The raw and the canonical unrestricted calculus

Definition 9.61. The *raw calculus* $\mathcal{R}_{\mathrm{raw}}$ uses raw terms and ordinary substitution. Its rules are:

1. HYP: $\Gamma \vdash A$ if $A \in \Gamma$;
2. the two computation equations (9.1) and (9.2), in every context;
3. transport in every native formula template: from $s = t$ and $F[s/v]$ infer $F[t/v]$;
4. QE: from $Q_y B(y)$ and $B(0)$ infer $B(t)$;
5. DE: from $D_{x,y} B(x, y)$ and a derivation of A from $\Gamma, Q_y B(b, y)$ infer A, where b is fresh for Γ, A and the parameters of B;
6. introduction by cases: from $\Gamma, B(t, 0) \vdash B(t, Sz)$ infer $D_{x,y} B(x, y)$;
7. introduction by progress: from $\Gamma, E(t, z) \vdash E(t, Sz)$ infer $D_{x,y} E(x, y)$, for a literal equation E.

In the two introductions z is fresh for Γ, the witness t and the parameters of the matrix. The *canonical calculus* $\mathcal{C}_\nu$ has the same rules on zero-body canonical formulas with normalized substitution, except that its only computation schema is

$$R(Sn, a; u.h) = h\langle R(n, a; u.h)/u \rangle. \tag{R_+}$$

The left counter of (R_+) is never literally 0, so all displayed terms are canonical. The raw zero computation has become an identity of spelling. The raw calculus is the kernel N on the seeded terms in an equivalent rule set: transport of formulas is the Leibniz rule of Section 1.4, and progress concludes a paired formula instead of a closure.

Lemma 9.62 (Structural rules)**.** *Weakening, renaming of eigenvariables, cut of an ordinary assumption and capture-free numerical substitution are admissible in $\mathcal{R}_{\mathrm{raw}}$ and $\mathcal{C}_\nu$.*

Proof. Induct on derivations after freshening the local eigenparameters. For $\mathcal{C}_\nu$, Theorem 9.3 shows that normalized substitution maps every rule instance to a rule instance, including the instances of (R_+). The distinction between the two introductions depends only on the root of the matrix and is stable. □

Lemma 9.63 (Elementary resources)**.** *Both unrestricted calculi derive reflexivity, symmetry, transitivity and congruence of equality, introduction of $Q_y B$ by cases, numerical cases, and induction on literal equations with an arbitrary context.*

Proof. Let $\mathrm{wr}(a) := R(S0, 0; u.a)$ with $u \notin \mathrm{FV}(a)$. The successor computation gives $\mathrm{wr}(a) = a$, also in $\mathcal{C}_\nu$. Transport of this equation along itself in the template $v = a$ gives $a = a$. The templates $v = s$, $s = v$ and $f(s) = f(v)$ give symmetry, transitivity and congruence.

From $\Gamma, B(0) \vdash B(Sz)$, introduction by cases with an unused witness coordinate gives $D_{w,y}B(y)$, and DE followed by HYP exports $Q_yB(y)$. With proofs of $B(0)$ and $B(Sz)$, QE gives $B(t)$; this is numerical cases. The same construction with introduction by progress gives $Q_yE(y)$ from $\Gamma, E(z) \vdash E(Sz)$, and with a proof of $E(0)$ it gives $E(t)$. □

Lemma 9.64 (Body extensionality, native form)**.** *In both unrestricted calculi, if* $\Gamma \vdash h(v) = k(v)$ *for a variable* v *fresh for* Γ*, then* $\Gamma \vdash R(n, a; u.h(u)) = R(n, a; u.k(u))$.

Proof. Apply induction on literal equations to the equation $R(z, a; u.h) = R(z, a; u.k)$ at a fresh counter z, with the initializer and the parameters fixed. At 0 both terms are Sa, by the zero computation in $\mathcal{R}_{\mathrm{raw}}$ and by identity of canonical spelling and reflexivity in $\mathcal{C}_\nu$. At a successor the computation exposes the two updates. Replace the old state in h by the induction equation, and use the premise at the second old state; this instance of the premise is justified by Theorem 9.62. □

The lemma is necessary. Changing a normalized body changes a finite description; it is not congruence for one fixed function symbol.

Lemma 9.65 (Normalization is provable in the raw calculus)**.** *For every raw term* t*,* $\mathcal{R}_{\mathrm{raw}} \vdash t = \nu(t)$*. For every raw native formula* A*, the raw calculus derives* $A \vdash \nu(A)$ *and* $\nu(A) \vdash A$*, uniformly in the free parameters.*

Proof. For terms induct on t. The counter and the initializer change by congruence. The body changes by Theorem 9.64 and the induction hypothesis with the bound variable free. If the counter is literally 0, the raw zero computation replaces the body by 0.

For formulas induct on the formula. An equation follows from the two endpoint identities. For $Q_yA(y)$, under this assumption introduce $Q_y\nu(A)(y)$ by cases: convert the assumed normalized base to the old base, apply QE at the fresh successor and convert back. The reverse direction is identical. For $D_{x,y}A$, eliminate the old formula at a fresh witness, introduce the new one at that witness by cases, convert the new base to the old base, use the exported closure, and convert the successor conclusion. These are derived congruence arguments for the two binders; no ordinary quantifier logic is assumed. □

Theorem 9.66 (Canonical and raw consequence)**.** *For zero-body canonical* Γ, A*,*

$$\Gamma \vdash_{\mathcal{C}_\nu} A \iff \Gamma \vdash_{\mathcal{R}_{\mathrm{raw}}} A.$$

More generally, ν *sends every raw derivation to a canonical derivation of the normalized sequent.*

Proof. From raw to canonical, normalize every sequent of the derivation. By Theorem 9.3 an instance of transport, QE, DE or an introduction becomes an instance of the same rule with normalized substitution. A raw zero computation becomes $Sa = Sa$, which is derivable by Theorem 9.63, and a successor computation becomes an instance of (R_+).

Conversely, regard canonical expressions as raw ones and recurse on the canonical derivation. A canonical rule instance differs from the raw instance only by normalization of substituted expressions. Theorem 9.65 converts premises, local assumptions and conclusions in both directions, and cut handles the local assumptions. For example, canonical transport converts the minor premise $\nu(E[s/v])$ to $E[s/v]$, applies raw transport and converts $E[t/v]$ to its normalization. Freshness is preserved because ν introduces no free variable. No substitution theorem for a restricted rule enters either direction. □

9.6.2 The six rules

For a native formula define the root reversal by $(s = t)^\dagger = (t = s)$, $(Q_y B)^\dagger = Q_y B$ and $(D_{x,y} B)^\dagger = D_{x,y} B$. It commutes with normalized substitution.

Definition 9.67 (The calculus $\mathcal{K}_6$)**.** Contexts are finite reusable lists of zero-body canonical native formulas. The rules are:

1. HYP: $\Gamma \vdash A$ if $A \in \Gamma$.
2. Successor computation: every instance of (R_+), in every context.
3. Crossed equational transport, for a template $E(v)$ that is a literal equation (constant templates are allowed):

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash E\langle s/v\rangle}{\Gamma \vdash (E\langle t/v\rangle)^\dagger} \qquad (s \not\equiv 0). \tag{X}$$

4. Crossed closure elimination:

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B\langle 0/y\rangle}{\Gamma \vdash (B\langle t/y\rangle)^\dagger}. \tag{QX}$$

5. Paired elimination, with b fresh for Γ, A and the parameters of B:

$$\frac{\Gamma \vdash D_{x,y} B(x,y) \qquad \Gamma,\ Q_y B\langle b/x\rangle \vdash A}{\Gamma \vdash A}. \tag{DE}$$

6. One introduction, with z fresh for Γ, t and the parameters of B:

$$\frac{\Gamma,\ B\langle t/x, \eta/y\rangle \vdash B\langle t/x, Sz/y\rangle}{\Gamma \vdash D_{x,y} B} \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{WI}$$

There is no primitive reflexivity, symmetry, numerical case rule, induction rule, zero-computation rule, logical connective or rule of numerical substitution. On iterator terms, one-term transport at sources other than the literal 0 together with the single witness-directed introduction is the seven-rule calculus `MPrf`, which is formalized and proved equivalent to the kernel (`mprf_iff_nprf`). $\mathcal{K}_6$ crosses its two eliminations, takes equation templates, and has no zero computation; what is specific to it is the canonical syntax. Weakening, renaming of eigenvariables and cut of an ordinary assumption are admissible in $\mathcal{K}_6$ by induction on derivations: cut replaces uses of an assumption by proofs and changes no witness, root or source term, and renaming preserves literal zero and roots. Numerical substitution is not claimed at this stage.

Lemma 9.68 (Equality in $\mathcal{K}_6$)**.** *$\mathcal{K}_6$ derives reflexivity, symmetry, transitivity, congruence, uncrossed transport in every literal-equation template without the restriction on the source, and the ordinary rule* QE.

Proof. Keep $\mathrm{wr}(a) = R(S0, 0; u.a)$ and put $c := \mathrm{wr}(0)$. Each $\mathrm{wr}(a)$ is not literally 0, and $\mathrm{wr}(a) = a$ is an instance of (R_+).

Symmetry. (X) with major premise $c = 0$ and a constant template reverses any proved equation. Since the reversal of a compound formula is the identity, (QX) followed by a reversal is ordinary QE, and (X) followed by a reversal is uncrossed transport at sources that are not literally 0.

Reflexivity. Use $\mathrm{wr}(a) = a$ as both premises of (X) with the template $v = a$.

Zero join. Let $p, q \not\equiv 0$ with $p = 0$ and $q = 0$. (X) with major $p = 0$, minor $\mathrm{wr}(p) = p$ and template $\mathrm{wr}(v) = p$ gives $p = c$; similarly $q = c$. Reverse the second and transport $p = c$ along $c = q$, whose source c is allowed, to get $p = q$.

Transitivity. Given $s = t$ and $t = r$: if $t \not\equiv 0$ transport $s = t$ along $t = r$. If $t \equiv 0$ and both endpoints are not literally 0, reverse the second equation and use the zero join. If an endpoint is literally 0, the conclusion is one of the premises.

Congruence. Along $s = t$ with $s \not\equiv 0$, transport the reflexive equation $f(s) = f(s)$ in the template $f(s) = f(v)$. For $0 = t$ with $t \not\equiv 0$, reverse to $t = 0$ and transport $f(t) = f(t)$ in the template $f(v) = f(t)$, which gives $f(0) = f(t)$. If both endpoints are literally 0 use reflexivity.

Transport. From $s = t$ and $a(s) = b(s)$, congruence and transitivity give $a(t) = a(s) = b(s) = b(t)$. Every step instantiates a schema at the terms at hand; none substitutes into a derivation. □

Lemma 9.69 (Introduction by cases)**.** *Every instance of introduction by cases is derivable in* $\mathcal{K}_6$*.*

Proof. (WI) is introduction by cases when the matrix is compound or the witness is not literally 0. Let E be an equation matrix with the premise $\Gamma, E(0,0) \vdash E(0, Sz)$. Use the witness c, which is not literally 0 and is provably 0. Under the assumption $E(c, 0)$, Theorem 9.68 gives $E(0, 0)$; cut with the premise gives $E(0, Sz)$, and transport back gives $E(c, Sz)$. (WI) at the witness c is in case mode and concludes the unchanged formula $D_{x,y}E(x, y)$. The witness c does not contain z. □

Lemma 9.70 (Introduction by progress)**.** *Every instance of introduction by progress for a literal equation is derivable in* $\mathcal{K}_6$*.*

Proof. Let the premise be $\Gamma, E(t, z) \vdash E(t, Sz)$ and let w be unused in $E(t, y)$. (WI) with witness 0 is in progress mode and proves $\Gamma \vdash D_{w,y}E(t, y)$. Under the additional assumption $E(t, 0)$ eliminate this formula. The exported closure is $Q_y E(t, y)$, independent of the fresh witness, and ordinary QE gives $E(t, Sz)$. Closing (DE) gives the premise of Theorem 9.69 at the witness t. □

Lemma 9.71 (Transport in compound templates)**.** $\mathcal{K}_6$ *derives transport in every native formula template.*

Proof. Induct on the template. Equations are Theorem 9.68. For $Q_y B$, introduce the new closure by cases with an unused witness as in Theorem 9.63: convert its base back along the equation by the induction hypothesis, apply the old closure at the fresh successor, and convert forward. For $D_{x,y}B$, (DE) opens the old formula at a fresh witness; introduce the new formula by cases at that witness, convert its base back, use the exported closure, and convert the successor forward. The other direction of the equation is obtained by symmetry. Only symmetry, cut and renaming are used. □

Theorem 9.72 (Six-rule equivalence)**.** *For zero-body canonical* Γ, A*,*

$$\Gamma \vdash_{\mathcal{K}_6} A \iff \Gamma \vdash_{\mathcal{C}_\nu} A.$$

Proof. Theorems 9.68 to 9.71 provide every rule of $\mathcal{C}_\nu$ inside $\mathcal{K}_6$. Conversely an instance of (WI) is one of the two unrestricted introductions, and (X) and (QX) are transport and QE followed by the derived symmetry when the conclusion is an equation; the other rules are common. No admissibility of numerical substitution in $\mathcal{K}_6$ is presupposed; it now transfers through the equivalence. □

Remark 9.73. Normalization preserves the literal-zero status of a whole term and the root of a formula, so it does not alter a test of (X) or (WI). An arbitrary numerical substitution can alter such a test: a variable witness can become 0. Keeping the unrestricted equivalence of Theorem 9.66 and the recovery of the restricted rules separate avoids an invalid rule-by-rule substitution argument.

9.6.3 The bridge for the seeded calculi

Theorem 9.74. *Extend ι and κ of Theorem 9.24(1) to native formulas, fixing $=$, Q_y and $D_{x,y}$.*

1. *If $\Gamma \vdash_{\mathcal{R}_{\mathrm{raw}}} A$ then $\Gamma^{\iota} \vdash_{\mathsf{N}} A^{\iota}$, and if $\Gamma \vdash_{\mathsf{N}} A$ then $\Gamma^{\kappa} \vdash_{\mathcal{R}_{\mathrm{raw}}} A^{\kappa}$.*
2. *$\mathcal{R}_{\mathrm{raw}} \vdash t^{\iota\kappa} = t$ and $\mathsf{N} \vdash s^{\kappa\iota} = s$; every native formula is interderivable with its double translation in both calculi.*
3. *For zero-body canonical Γ, A: $\Gamma \vdash_{\mathcal{K}_6} A$ if and only if $\Gamma^{\iota} \vdash_{\mathsf{N}} A^{\iota}$.*

Consequently $\mathcal{K}_6$, $\mathcal{C}_{\nu}$ and $\mathcal{R}_{\mathrm{raw}}$ are sound and faithful for T^1_R under the reading of Section 1.2, and each is equiconsistent and bi-interpretable with iPRA*, with whole domains, actual equality and numerical composites of graph $x = y$. On raw syntax the composite through canonical syntax is provably, not literally, the identity; on canonical syntax ν is literally idempotent.*

Proof. (1) From $\mathcal{R}_{\mathrm{raw}}$ to N: the computation equations translate to the computation equations of the iterator, using $\mathsf{N} \vdash (St)^{\iota} = S(t^{\iota})$ and the Leibniz rule of Section 1.4; transport of formulas is that Leibniz rule; QE, DE and introduction by cases are rules of N; and introduction by progress is derived by forming $Q_y E(t, y)$ by progress, applying QE under the assumption $E(t, 0)$ and introducing by cases at t. From N to $\mathcal{R}_{\mathrm{raw}}$: the computation equations of the iterator are Theorem 8.21 with (9.17); the one-term transport of N is transport followed by symmetry; and the progress rule of N is introduction by progress with an unused witness followed by DE and HYP, as in Theorem 9.63.

(2) By Theorems 9.62 and 9.63 the proof of Theorem 9.21 is available in $\mathcal{R}_{\mathrm{raw}}$ and in N: it uses equality reasoning, an instance of a derived equation and induction on a literal equation. The argument of Theorem 9.25(2) then gives the two term identities. A formula differs from its double translation by these provable identities at its terms, with the bound variables of the formula free; the binder-congruence argument of Theorem 9.65 gives interderivability.

(3) By Theorems 9.66 and 9.72, derivability in $\mathcal{K}_6$ is derivability in $\mathcal{R}_{\mathrm{raw}}$. One direction is (1). For the other, a derivation of $\Gamma^{\iota} \vdash_{\mathsf{N}} A^{\iota}$ gives $\Gamma^{\iota\kappa} \vdash A^{\iota\kappa}$ in $\mathcal{R}_{\mathrm{raw}}$, and (2) with cut gives $\Gamma \vdash A$. The consequences follow from Theorem 1.119 and Theorem 9.25. □

The mathematical change in this section is the canonical syntax and the proof that it preserves substitution, derivability and both composites. The inventory decreases from seven rules to six, and the successor constructor is gone, but the omitted zero computation is the law $R(0, a; u.h) \equiv R(0, a; u.0)$ of the syntax, which every substitution and comparison of a complete checker enforces.

9.7 Five rules with clause contexts

This section draws on `clausal_canonical`.

The calculus of this section is the clause presentation of Section 1.9 with the one-term transport of Section 1.3, stated on the zero-body canonical terms of the seeded recursor with one bound variable. The closure binder is no longer a formula constructor, and the hypothesis rule and the closure elimination become one rule of clause use. Only what the canonical syntax and the missing reflexivity rule require is proved.

9.7.1 The calculus

Definition 9.75 (The calculus $\mathcal{K}_5$)**.** Terms are zero-body canonical terms of the seeded recursor, and all substitutions are normalized. Formulas are $F ::= (s = t) \mid D_{x,y}F$. A context is a finite reusable list of clauses $[A;\, y.\, B(y)]$ with $y \notin \mathrm{FV}(A)$; clauses are context objects, not formulas or goals. Let

$$\mathrm{wr}(a) := R(S0, 0; u.a), \qquad c := \mathrm{wr}(0), \qquad H := (c = 0).$$

An ordinary assumption F is the constant clause $[H;\, v.\, F]$ with v unused; this applies also to the local assumption of rule (I). The rules are:

1. Computation, in every context:

$$R(Sn, a; u.h) = h\langle R(n, a; u.h)/u\rangle. \tag{R}$$

2. One-term crossed transport, for a term template $h(v)$ and an arbitrary source:

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}. \tag{T}$$

3. Clause use, for $[A;\, y.\, B(y)] \in \Gamma$:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}. \tag{U}$$

4. Paired elimination, with b fresh for Γ, F and the parameters of B:

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma,\ [B(b, 0);\, y.\, B(b, y)] \vdash F}{\Gamma \vdash F}. \tag{E}$$

5. Witness-directed introduction, with z fresh for Γ, t and the parameters of B:

$$\frac{\Gamma,\ B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)} \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{I}$$

There is no separate hypothesis rule, zero-computation rule, closure constructor, closure elimination, reflexivity axiom, case rule or induction rule. The neutral antecedent is H and not $0 = 0$, because H is an instance of (R) and reflexivity is not yet available. The clause pair, its bound variable and its substitution are part of the specification and of the implementation; they are not a saving that goes uncounted.

Computation proves $\mathrm{wr}(a) = a$ in every context, in particular H. Use of an ordinary assumption is therefore computation of H followed by (U). Weakening, renaming of fresh parameters and cut of an ordinary assumption follow by induction on derivations; in a cut each use of the constant clause is replaced by the supplied proof of its conclusion. No substitution theorem for rule (I) is presupposed.

Lemma 9.76 (Equality from one term template)**.** $\mathcal{K}_5$ *derives reflexivity, symmetry, transitivity, congruence and transport in every literal-equation template.*

Proof. Use $\mathrm{wr}(a) = a$ as both premises of (T) with the identity template to get $a = a$. To reverse $s = t$, take the major premise H and the constant template $h(v) = s$; (T) returns $t = s$. Given $s = t$ and $t = r$, reverse the former and apply (T) with the identity template to the major $t = r$ and the minor $t = s$, which gives $s = r$. Congruence is (T) with the major $s = t$ and the reflexive minor $h(s) = h(s)$. For a template $a(v) = b(v)$ the two applications

$$s = t,\ a(s) = b(s) \implies b(s) = a(t), \qquad s = t,\ b(s) = a(t) \implies a(t) = b(t)$$

give transport. Each template is fixed finite syntax; no function variable occurs. □

Lemma 9.77 (All case introductions)**.** *For every formula B and term t the rule "from $\Gamma, B(t,0) \vdash B(t,Sz)$ infer $D_{x,y}B(x,y)$" is admissible in $\mathcal{K}_5$.*

Proof. This is the repair of Theorem 5.7, at the second name $c = \mathrm{wr}(0)$ of zero. The only instance that is not primitive has an equation matrix E and the witness 0. Under $E(c,0)$, the theorem $c = 0$ and Theorem 9.76 give $E(0,0)$; cut with the supplied branch gives $E(0,Sz)$; transport back gives $E(c,Sz)$. Rule (I) at the witness c is in case mode and proves the original goal. □

Lemma 9.78 (Transport in every formula)**.** *For every template $F(v)$, from $s = t$ and $F(s)$ one derives $F(t)$ in $\mathcal{K}_5$, under arbitrary retained clauses.*

Proof. The proof of Theorem 5.8, with rules (E) and (U) for ELIM and USE, and Theorems 9.76 and 9.77 for Theorems 5.6 and 5.7. □

9.7.2 Closure as an abbreviation

For a fresh unused variable w write $q_y B(y) := D_{w,y}B(y)$.

Lemma 9.79. *In $\mathcal{K}_5$:*

1. *from $\Gamma, B(0) \vdash B(Sz)$ one derives $\Gamma \vdash q_y B$;*
2. *from $q_y B$ and $B(0)$ one derives $B(t)$;*
3. *in every context that contains $[B(0);\, y.\, B(y)]$ one derives $q_y B$;*
4. *for a literal equation E, from $\Gamma, E(z) \vdash E(Sz)$ one derives $\Gamma \vdash q_y E$; hence induction on literal equations with an arbitrary context, numerical cases for all formulas, and introduction by progress at every witness are derivable.*

Proof. Items (1)–(3) and the first part of (4) are Theorem 5.9, and the derived rules of (4) are Theorem 5.10; their proofs use only (E), (U), (I) and Theorem 9.77, which here play the parts of ELIM, USE, INTRO and Theorem 5.7. □

Items (1)–(3) are introduction, elimination and eta expansion of a closure resource. The third is what translates a branch of (DE) in $\mathcal{K}_6$, which assumes a closure formula.

9.7.3 Comparison with the six rules

Define T from the formulas of $\mathcal{K}_6$ to those of $\mathcal{K}_5$ by

$$T(s = t) = (s = t), \qquad T(D_{x,y}B) = D_{x,y}T(B), \qquad T(Q_y B) = q_y T(B),$$

with every inserted coordinate fresh and unused. T preserves free variables, the distinction between equation and compound roots, and normalized substitution. Let U be the inclusion of the formulas of $\mathcal{K}_5$ into the native formulas, and let $\mathrm{ord}(\Delta)$ replace each formula of a list Δ by its constant clause.

Theorem 9.80 (Preservation in both directions)**.** *For every native context Γ and formula A,*

$$\Gamma \vdash_{\mathcal{K}_6} A \implies \mathrm{ord}(T\Gamma) \vdash_{\mathcal{K}_5} T(A),$$

and for formulas Δ, F of $\mathcal{K}_5$,

$$\mathrm{ord}(\Delta) \vdash_{\mathcal{K}_5} F \implies \Delta \vdash_{\mathcal{K}_6} F.$$

Proof. Forward. Induct on the derivation in $\mathcal{K}_6$. HYP uses H and the constant clause. Computation is unchanged. (X) is derived transport followed by symmetry. (QX) is Theorem 9.79(2), followed by symmetry when the conclusion is an equation. (WI) is (I): the translation preserves roots and the witness is unchanged. For (DE), the translated major premise proves $D_{x,y}T(B)$; rule (E) opens $[T(B)(b,0);\, y.\, T(B)(b,y)]$; Theorem 9.79(3) derives the translated assumption $q_y T(B)(b,y)$ of the old branch, and cut inserts this proof into the translated branch. All new variables are freshened first.

Backward. Track each context occurrence by the rule that introduced it. A constant clause is represented by its formula as an assumption of $\mathcal{K}_6$, and a clause opened by (E) by the assumption $Q_y B(b,y)$. (U) on a constant clause is HYP, and (U) on an opened clause is ordinary QE, which $\mathcal{K}_6$ derives (Theorem 9.68). (R) is the same computation, (T) is derived one-term transport, (E) is (DE), and (I) is (WI). Induction on the derivation gives the claim. □

The two directions are those of Theorem 5.11(ii) and (i), with $\mathcal{K}_6$ in the place of N. A clause supplied from outside, which is neither constant nor opened by (E), is treated as in Theorem 5.15; a checker for closed certificates needs no such clause.

Theorem 9.81 (Both composites)**.** *$T(U(F)) = F$ literally for every formula F of $\mathcal{K}_5$, and $A \dashv\vdash_{\mathcal{K}_6} U(T(A))$ for every native formula A. Both numerical translations are the identity. Consequently $\mathcal{K}_5$ is equiconsistent and bi-interpretable with* iPRA*, with whole domains, actual equality and numerical composites of graph $x = y$; the designated contradiction $0 = S0$ is unchanged.*

Proof. The first identity is immediate from the clauses of T. For the second use $Q_y B(y) \dashv\vdash D_{w,y}B(y)$ in $\mathcal{K}_6$ with w unused: forward, under $B(0)$ specialize the closure at Sz and introduce the paired formula by cases; backward, (DE) opens exactly $Q_y B$ at an unused witness and HYP closes the branch. The binder congruence of Theorem 9.65 lifts this through Q_y and $D_{x,y}$, by induction on the formula. No quotient, retract or reinterpretation of equality is introduced. The consequence follows from Theorems 9.74 and 9.80. □

Under the reading $[A;\, y.\, B]^\circ = A^\circ \to \forall y\, B^\circ(y)$ the five rules are directly sound in T^1_R: (R) is a defining equation, (T) is transport followed by symmetry, (U) is implication and universal elimination, (E) is existential elimination, and (I) is coverage in case mode and induction on an equation under the assumed zero instance in progress mode, followed by existential introduction. Certificates of $\mathcal{K}_5$ can be longer than those of $\mathcal{K}_6$ while the checker is shorter: an ordinary assumption needs the neutral computation premise, and the removal of the closure constructor adds elimination and eta-expansion steps.

9.8 Counter evaluation

This section draws on `counter_evaluation` and `uniform_binding`.

On the counter-aware recursor, the computation rule and the transport rule merge into one rule that places the supplied equation in the counter. The rule has no analogue on iterator terms, since it needs a body that sees the counter.

9.8.1 The clause syntax and a liberal calculus

Throughout this section and the next, terms are canonical terms of the counter-aware language, either zero-body canonical (with ν) or normal (with nf), and every displayed substitution is normalized accordingly. Formulas are $A ::= (s = t) \mid D_{x,y}B$ with the reading of Section 1.2. A context is a finite reusable list of clauses $[A;\, y.\, B(y)]$, $y \notin \mathrm{FV}(A)$, read as

$A^\circ \to \forall y\, B^\circ(y)$ as in Section 1.9; a clause may also have an empty antecedent, $[\,;\, y.\, B(y)]$, read as $\forall y\, B^\circ(y)$. An ordinary assumption A is the constant clause $[0 = 0;\, v.\, A]$, or $[\,;\, v.\, A]$ where empty antecedents are used, with v unused. The designated contradiction is $0 = 1$.

The calculi below have rules with tests on literal syntax. The following calculus has none and serves as the common reference.

Definition 9.82 (The liberal clause calculus $\mathcal{L}$)**.** The rules of $\mathcal{L}$ are:

1. REFL: $\Gamma \vdash t = t$.
2. COMP: $\Gamma \vdash R(Sn, a; i, u.h) = h\langle n/i, R(n, a; i, u.h)/u\rangle$.
3. TR: from $s = t$ and $h\langle s/v\rangle = r$ infer $r = h\langle t/v\rangle$, for a term template h.
4. USE: for $[A;\, y.\, B(y)] \in \Gamma$, from $\Gamma \vdash A$ infer $\Gamma \vdash B\langle t/y\rangle$; for a clause with empty antecedent there is no premise.
5. DE: from $D_{x,y}B(x, y)$ and a derivation of A from $\Gamma, [B(b, 0);\, y.\, B(b, y)]$, with b fresh for Γ, A and the parameters of B, infer A.
6. CASE: from $\Gamma, B(t, 0) \vdash B(t, Sz)$ infer $D_{x,y}B(x, y)$, for every matrix and witness.
7. PROG: from $\Gamma, E(t, z) \vdash E(t, Sz)$ infer $D_{x,y}E(x, y)$, for a literal equation E.

In the last two rules z is fresh for Γ, t and the parameters of the matrix. On normal terms both sides of COMP have the same spelling, and the rule is an instance of REFL.

Lemma 9.83 (Structure of $\mathcal{L}$)**.** *The following are admissible in $\mathcal{L}$: weakening; injective renaming of parameters; numerical substitution; cut of an ordinary assumption; and cut of a clause, that is, from $\Gamma, [A;\, y.\, B(y)] \vdash F$ and $\Gamma, A \vdash B(y)$ with y fresh for Γ, A infer $\Gamma \vdash F$.*

Proof. Induct on derivations after freshening eigenparameters. Every rule is a schema without a test on the shape of a term, so Theorem 9.3, respectively Theorem 9.7, shows that a substitution instance of a rule instance is a rule instance. For the cut of an assumption replace each use of its constant clause by the supplied proof. For the cut of a clause replace each use of the clause at a term t, whose premise proves A, by the second derivation with t substituted for y, followed by a cut of the assumption A. □

Lemma 9.84 (Derived rules of $\mathcal{L}$)**.** *$\mathcal{L}$ derives:*

1. *symmetry, transitivity, congruence, and transport in every equation template;*
2. *numerical cases: from $\Gamma \vdash F(0)$ and $\Gamma \vdash F(Sz)$, z fresh, infer $\Gamma \vdash F(t)$;*
3. *equation induction: from $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$, for a literal equation E and z fresh, infer $\Gamma \vdash E(t)$;*
4. *transport in every formula template: from $s = t$ and $F(s)$ infer $F(t)$;*
5. *binder congruence: if $B(x, y)$ and $B'(x, y)$ are interderivable in every context uniformly in x, y, then so are $D_{x,y}B$ and $D_{x,y}B'$.*

Proof. (1) As in Theorem 9.76, with REFL in place of the two uses of wr: to reverse $s = t$ apply TR to the major $0 = 0$ and the constant template.

(2), (3) are Theorem 5.10, with CASE for Theorem 5.7 and PROG at the witness 0 for the progress closure: for an unused coordinate w and $q_yA := D_{w,y}A(y)$, CASE introduces q_yF from $\Gamma \vdash F(Sz)$, ignoring the assumed base, PROG introduces q_yE from the step, and DE and USE at t with the given base finish.

(4) is Theorem 5.8, with (1) for the equations and CASE at the fresh witness.

(5) The construction of (4) at the outer binder, with the two conversions given by the hypothesis. □

9.8.2 The five patterns of counter evaluation

Definition 9.85 (The calculus $\mathcal{K}_{\mathrm{CE}}$)**.** Terms are zero-body canonical. Write $\varrho(t,a,h)$ for the canonical term $R(t,a;i,u.h)$, which is Sa when $t \equiv 0$. Every clause has an antecedent, and an ordinary assumption A is $[0=0;\, v.\, A]$. The rules are REFL, USE and DE of Theorem 9.82, the rule of counter evaluation

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash R(Ss,a;i,u.h) = r}{\Gamma \vdash r = h\langle t/i, \varrho(t,a,h)/u\rangle} \tag{CE}$$

and the witness-directed introduction, with z fresh for Γ, w and the parameters of B,

$$\frac{\Gamma,\ B(w,\eta) \vdash B(w,Sz)}{\Gamma \vdash D_{x,y}B(x,y)} \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } w \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{DI}$$

In (CE) the two substitutions in h are simultaneous. The seed, the body and the passive parameters are read from the second premise and are kept fixed: occurrences of s inside a or h are not replaced. The first endpoint s is an arbitrary term, including the literal 0. There is no further computation axiom, equality rule, induction rule, successor axiom or logical connective. The count of five patterns counts (DI) as one schema with two modes; it is not a claim under every convention of counting rules. `uniform_binding` calls DE and (DI) OPEN and CLOSE.

Theorem 9.86 (Arithmetic reconstruction)**.** *REFL and* (CE) *alone derive the successor computation* COMP*, symmetry, transitivity, congruence in every term template, and the crossed transport* TR*. The derivations use no logical rule, no numerical cases and no induction.*

Proof. Computation. Apply (CE) to the reflexive premises $n = n$ and $B = B$ with $B = R(Sn,a;i,u.h)$. The conclusion is $B = h\langle n/i, \varrho(n,a,h)/u\rangle$.

Root composition. Let p prove $s = t$ and q prove $s = r$, and put $K_s := R(Ss,0;i,u.i)$. (CE) applied to q and $K_s = K_s$ gives $K_s = r$, and applied to p and this result it gives $r = t$. Write $\mathsf{J}(p,q)$ for this derivation of $r = t$ from $s = t$ and $s = r$.

Symmetry and transitivity. $\mathsf{J}(\mathrm{REFL}(s),p)$ proves $t = s$ from $p\colon s = t$. Given $p\colon s = t$ and $q\colon t = v$, the derivation $\mathsf{J}(q,\mathrm{Sym}(p))$ proves $s = v$.

Congruence. For a template $F(v)$ choose the binders fresh and put $B_F := R(Ss,0;i,u.F(i))$. (CE) on two reflexive premises gives $B_F = F(s)$, and with first premise $p\colon s = t$ and this second premise it gives $F(s) = F(t)$. The template may have repeated occurrences and occurrences under the binders of nested recursors.

Crossed transport. Apply J to $F(s) = F(t)$ and the supplied $q\colon F(s) = r$; the result is $r = F(t)$. Every substitution in the argument is an instance of (CE); no congruence is presumed in order to establish congruence. □

Proposition 9.87 (Proof costs)**.** *Count every primitive inference, including a* REFL *leaf, as one node and count repeated subtrees repeatedly. Root composition adds three nodes to its premises, congruence adds four, and crossed transport adds seven. Each supplied premise occurs once.*

Proof. $\mathsf{J}(p,q)$ is two applications of (CE) and one leaf. Congruence is two applications and two leaves. Crossed transport is J applied to a congruence, $4+3=7$. The term labels contain copies of F and of the endpoints, so these are bounds on inference nodes, not on bytes or running time. □

Theorem 9.88 (Equality-interface equivalence)**.** *Let $\mathcal{K}_{\mathrm{CT}}$ be $\mathcal{K}_{\mathrm{CE}}$ with* REFL *and* (CE) *replaced by* COMP *and* TR*. A sequent is derivable in $\mathcal{K}_{\mathrm{CE}}$ if and only if it is derivable in $\mathcal{K}_{\mathrm{CT}}$, and the two proof translations are effective and preserve end sequents literally.*

Proof. Theorem 9.86 replaces each COMP or TR node. Conversely, in $\mathcal{K}_{\mathrm{CT}}$ the computation with the counter projection gives $K_s = s$, and TR applied to two copies of this equation with the identity template gives $s = s$. TR with a constant template gives symmetry, with the identity template and a reversed premise it gives transitivity, and with a reflexive minor premise it gives congruence. For (CE): from $s = t$, congruence in the counter gives $R(Ss, a; i, u.h) = R(St, a; i, u.h)$, and COMP gives $R(St, a; i, u.h) = h\langle t/i, \varrho(t, a, h)/u\rangle$; reverse the second premise $R(Ss, a; i, u.h) = r$ and compose. The three logical rules are common. The use of ordinary equality in the converse is legitimate, since it is derived in $\mathcal{K}_{\mathrm{CT}}$ before (CE) is simulated; the forward direction never assumes it. □

Lemma 9.89 (Both introductions)**.** *CASE and PROG are derivable in $\mathcal{K}_{\mathrm{CE}}$ for every matrix and witness. Before this, weakening, renaming of fresh parameters and cut of an ordinary assumption are admissible in $\mathcal{K}_{\mathrm{CE}}$.*

Proof. The three structural operations do not change the literal-zero test of (DI); for cut, replace each USE of the constant clause $[0 = 0; v.\, A]$ by the supplied proof of A and discard the proof of its antecedent.

The only instance of CASE that is not an instance of (DI) has an equation matrix E and the witness 0. It is repaired as in Theorem 5.7, at the second name $Z := R(S0, 0; i, u.0)$ of zero, which is not literally 0 and for which Theorem 9.86 proves $Z = 0$: under $E(Z, 0)$, transport in the equation gives $E(0, 0)$; cut with the premise gives $E(0, Sz)$; transport back gives $E(Z, Sz)$; and (DI) at the witness Z is in case mode. Only transport in an equation is used, not transport through a binder.

PROG at a witness w is the third statement of Theorem 5.10, with the same proof: (DI) in progress mode at the witness 0 of an unused coordinate gives $D_{v,y}E(w, y)$, DE opens $[E(w, 0); y.\, E(w, y)]$, and under $E(w, 0)$ USE and CASE at w conclude. □

Theorem 9.90. *On zero-body canonical terms, $\mathcal{K}_{\mathrm{CE}}$, $\mathcal{K}_{\mathrm{CT}}$ and $\mathcal{L}$ derive the same sequents whose clauses have antecedents. In particular numerical substitution is admissible in $\mathcal{K}_{\mathrm{CE}}$.*

Proof. Every rule of $\mathcal{L}$ is derivable in $\mathcal{K}_{\mathrm{CE}}$ by Theorems 9.86 and 9.89. Conversely (CE) is derivable in $\mathcal{L}$ as in Theorem 9.88, and an instance of (DI) is an instance of CASE or of PROG. Substitution is Theorem 9.83. □

The test of (DI) is not invariant under substitution: a variable witness can become 0. A progress introduction keeps its equation root and its literal witness 0, but a case introduction whose witness becomes 0 has to be replaced by the derived CASE. Admissibility of substitution is obtained through $\mathcal{L}$, not by a rule-by-rule argument.

9.8.3 Ordinary logic with the decoder

The derivation of intuitionistic logic from the paired binder is that of the gates, selectors and connectives of Section 1.4; it needs a selector with the two endpoint equations. With the decoder P of Theorem 9.10 the conjunction and the universal quantifier take a shorter form: no parity iterator and no separate zero instance are needed. The construction is given in full for $\mathcal{L}$, on either kind of canonical terms.

Definition 9.91 (Gates, selection and the logical macros)**.** With binders renamed away from the control v, put

$$G_v(s = t) := \big(C(v, 0, s) = C(v, 0, t)\big), \qquad G_v(D_{x,y}B) := D_{x,y}G_vB,$$

and $L(F) := D_{w,y}G_yF$ with w, y fresh. Every formula is a finite chain of paired binders ending in an equation. For formulas A, B apply L to the shorter one until the depths agree, align the bound names, and at the equations put

$$M_v((s = t), (a = b)) := (C(v, s, a) = C(v, t, b)),$$

carrying the matched binders outside. With all auxiliary binders fresh and w unused define

$$\begin{aligned} \bot_* &:= (0 = 1), & A \to_* B &:= D_{w,y}M_y(A, B), \\ A \wedge_* B &:= D_{w,y}G_yM_{P(y)}(A, B), & \exists_* x\, A(x) &:= D_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), & \forall_* x\, A(x) &:= D_{w,y}G_yA(P(y)). \end{aligned} \tag{9.24}$$

These are finite formulas of the same syntax, not further constructors.

Lemma 9.92 (Gates and selection)**.** *In $\mathcal{L}$, for all formulas and in every context: $\vdash G_0F$; $G_{Sz}F \dashv\vdash F$; $L(F) \dashv\vdash F$; $M_0(A, B) \dashv\vdash A$; and $M_{Sz}(A, B) \dashv\vdash B$.*

Proof. For an equation the first two statements are the endpoint equations of C, reflexivity and transport. For $D_{x,y}B$ the first is CASE with the gate theorem of the matrix, the assumed base being unused, and the second is binder congruence (Theorem 9.84). For $L(F)$: forward, CASE uses F to prove the successor gate; backward, DE, the theorem G_0F and USE at 1 give G_1F and hence F. For M: at the equations the endpoint equations of C give the two statements for the padded formulas, binder congruence carries them through the common binders, and $L(F) \dashv\vdash F$ removes the padding. □

Lemma 9.93 (Natural deduction)**.** *In $\mathcal{L}$ the macros* (9.24) *admit every introduction and elimination rule of intuitionistic first-order natural deduction, with the usual discharge and eigenvariable conditions, and ex falso for $\bot_*$. Moreover $\mathcal{L}$ proves coverage $n = 0 \vee_* \exists_* m\,(n = Sm)$.*

Proof. Implication. Given a derivation of B under the assumption A, CASE introduces $A \to_* B$: the assumed zero instance gives A and the successor instance is equivalent to B. For elimination, DE retains the selector clause, the supplied A proves its antecedent, and USE at 1 gives B.

Conjunction. From A and B, numerical cases prove $M_v(A, B)$ for every term v, in particular for $v = P(Sz)$. The successor gate and CASE give $A \wedge_* B$; the antecedent of its clause is a gate theorem. For elimination, DE and USE at 1 and at $S1$ give the selector at $P(1) = 0$ and at $P(S1) = 1$, hence A and B by transport. No parity or range theorem is used.

Existential quantifier. From $A(t)$, CASE at the witness t introduces the gated formula. Conversely DE at a fresh b, the zero gate and USE at 1 give $A(b)$; cut this into a branch whose conclusion and context do not contain b, and close DE.

Disjunction. The witnesses 0 and 1 give the two injections. For elimination, discharge the two branches as $A \to_* F$ and $B \to_* F$. Numerical cases, in the original context, prove $M_v(A, B) \to_* F$ for every v. Existential elimination supplies $M_b(A, B)$ at a fresh tag b, and implication elimination gives F. The tag is not substituted into a retained assumption, since the numerical cases are performed before that assumption is added.

Universal quantifier. A derivation of $A(z)$ with z fresh gives $G_{Sz}A(P(Sz))$ by $P(Sz) = z$ and the gate equivalence, and CASE with an unused witness gives $\forall_* x\, A(x)$. For elimination, DE, the zero gate and USE at St give $G_{St}A(P(St))$ and hence $A(t)$. Every numerical target is covered directly; the value $P(0) = 1$ plays no role, because the outer gate makes the zero instance provable.

Falsity. Under $0 = 1$, transport in the control of $C(v, s, t)$ and the two endpoint equations give $s = t$ for arbitrary terms. Induction on the number of binders, with CASE, gives every formula.

Coverage. Numerical cases on n: the left injection at 0, and the right injection with the witness z at Sz. □

Remark 9.94 (Variants of the macros). The form (9.24) is that of `uniform_binding` and `single_constructor`. `counter_evaluation` uses a term π with $\pi(1) = 1$ and $\pi(S1) = 0$ in the conjunction and defines $\forall_* x\, A(x) := A(0) \wedge_* D_{w,x}A(x)$. `uniform_recursion` uses $K(y) = C(P(y), 1, 0)$ in the conjunction and the form above for the universal quantifier. `clausal_canonical`, which has no decoder, uses $p(z) = \widehat{I}(z, 0; u.C(u, 1, 0))$ with $p(1) = 1$, $p(S1) = 0$ and the universal quantifier with a separate zero instance. `single_constructor` defines M_v by recursion on the pair of formulas, padding an equation E against a paired formula by $G_y E$ under the aligned binder; the result is interderivable with the definition above.

9.8.4 Adequacy and the main theorem

Theorem 9.95 (Adequacy of $\mathcal{L}$)**.** *Let $A \mapsto A^*$ translate the formulas of T^2_R to the clause syntax: an equation is normalized, $\bot$ becomes $\bot_*$, and the connectives and quantifiers become the macros* (9.24)*.*

1. *If $\Gamma \vdash_{\mathcal{L}} A$ then T^2_R proves A° from the readings of the clauses of Γ.*
2. *If T^2_R proves B from assumptions Δ, then $\Delta^* \vdash_{\mathcal{L}} B^*$, the formulas of Δ^* being ordinary assumptions.*
3. *$\mathsf{T}^2_R \vdash (B^*)^\circ \leftrightarrow B$ for every formula B of T^2_R, and $(A^\circ)^* \dashv\vdash_{\mathcal{L}} A$ for every formula A of the clause syntax, under arbitrary retained clauses.*

Hence $\mathcal{L}$ is faithful for T^2_R in both directions, and both numerical maps of this comparison are the identity.

Proof. (1) Induct on the derivation. By Theorem 9.23 a normalized instance is provably equivalent to the raw instance. REFL, COMP and TR are equality reasoning and a computation equation. USE is implication and universal elimination, and DE is existential elimination. For CASE, assume $B(t, 0)$; coverage and the premise give $B(t, y)$ for every y; then introduce the witness t. For PROG, assume $E(t, 0)$ and apply (9.11) with the premise as step.

(2) The logical rules are Theorem 9.93 and the equality rules are Theorem 9.84. The translation of the zero computation has the same canonical term on both sides, and that of the successor computation is COMP. Coverage is Theorem 9.93. Separation translates to $(0 = 1) \to_* (0 = 1)$. The rule (9.11) is Theorem 9.84(3), with the unchanged context.

(3) For the first statement induct on B: in T^2_R the gate and selector endpoints, coverage for an arbitrary control and $P(St) = t$ show that the reading of each macro is equivalent to the connective or quantifier it represents; for example $\forall y\, (M_y(A, B))^\circ$ is equivalent to $A^\circ \wedge B^\circ$ by coverage, so the reading of $A \to_* B$ is equivalent to $A^\circ \to B^\circ$. For the second induct on A. Equations are unchanged. For $D_{x,y}B$ apply the induction hypothesis to the matrix, uniformly, and prove

$$D_{x,y}B(x, y) \dashv\vdash \exists_* x(B(x, 0) \to_* \forall_* y\, B(x, y)).$$

Forward, open the left side at b; under $B(b, 0)$ its clause gives $B(b, z)$ for a fresh z; universal, implication and existential introduction build the right side; close DE. Backward, existential elimination gives a fresh b and the implication; introduce $D_{x,y}B$ by CASE at b: the local base feeds the implication, and universal elimination at Sz gives the successor instance. Faithfulness follows: if T^2_R proves A° from the readings of Γ, then by (2) and (3) $\mathcal{L}$ derives A from Γ, each clause $[A_0; y.\, B_0]$ providing the translation of its reading by implication and universal introduction. □

Theorem 9.96 (Counter evaluation)**.** *The five-pattern calculus $\mathcal{K}_{\mathrm{CE}}$ is sound and faithful for T^2_R, and it is equiconsistent and bi-interpretable with* iPRA*: both numerical domains are the whole domain, equality is actual equality, the numerical composites have the graph $x = y$, and every formula is provably equivalent to its double translation. The translations preserve the full consequence relation, including open sequents and formulas with unbounded quantifiers.*

Proof. Combine Theorems 9.25, 9.90 and 9.95. The contradiction $0 = 1$ reads as an equation that T^2_R refutes by separation and that implies every formula, so inconsistency transfers in both directions. □

Remark 9.97. `counter_evaluation` and `uniform_binding` present the same calculus. `counter_evaluation` contributes Theorems 9.86 to 9.88 and a checker that synthesizes the conclusion of each proof node; its ordinary theory has the zero computation as an identity of canonical spelling. `uniform_binding` takes the ordinary theory over raw terms with both computation equations, reads canonical terms as raw ones, justifies the normalization by Theorem 9.23, and contributes the macros (9.24) and the goal-directed checker with one traversal (Section 9.12). The equivalence of the two certificate formats is Theorem 9.139.

9.9 Three families on normal forms

This section draws on `uniform_recursion` and `single_constructor`.

On normal terms (Theorem 9.5) both computation equations are identities of spelling, and the equations of Theorem 9.10 are literal identities of normal forms. A calculus then needs neither a computation rule nor, with a suitable elimination, a transport rule. Two such calculi are stated on the counter-aware recursor. Both take the clause syntax of Section 9.8 with empty antecedents, and both contain in every context the fixed clause

$$\mathcal{A} := [\,;\, y.\, y = y], \tag{9.25}$$

which is the initial context of a closed proof; it is the reflexivity clause $\mathcal{R}$ of Theorem 5.26, and the convention is that of Theorem 6.13. Reflexivity is thus an axiom clause; it has not been deleted without replacement. An ordinary assumption F is $[\,;\, v.\, F]$. On iterator terms in full normal form, the two calculi are the calculi $\mathcal{W}^{\mathcal{R}}$ and $\mathcal{O}^{\mathcal{R}}$ of `witness_discharge` and `inferred_witness`, treated with their Lean formalizations in Sections 6.2 and 6.3 (Theorems 6.19 and 6.36); the counter-aware forms below are not formalized. The arguments of that chapter are stated for calculi with the listed rules; apart from the packets, they use from the term syntax only the literal test for 0, the endpoint identities of the selector and the coherence of normalization with substitution. On the normal terms of Theorem 9.5 these are Theorem 9.10 and (9.7), so such arguments are cited and not repeated. What the new syntax requires is proved here: the packets that carry transport use the decoder, and the repair of the introduction at the witness 0 cannot use a closed term that is provably but not literally 0, because normal terms have none; the compound envelope of Theorem 4.142 is used instead.

9.9.1 Use, Close and Name

Definition 9.98 (The calculus $\mathcal{K}_{\mathrm{Name}}$)**.** Terms are normal and all displayed instances are normalized by nf. The rules are:

1. USE: for $[A;\, y.\, B(y)] \in \Gamma$ and a term t, from $\Gamma \vdash A$ infer $\Gamma \vdash B(t)$; for an empty antecedent there is no premise. In particular $\mathcal{A}$ gives $t = t$.

2. CLOSE: the rule (DI), its local assumption being an unconditional constant clause.

3. NAME: for a fresh parameter b and the clause $\mathcal{B}_b := [B(b,0);\, y.\, B(b,y)]$,

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \mathcal{B}_b \vdash b = t \qquad \Gamma, \mathcal{B}_b \vdash F}{\Gamma \vdash F\langle t/b\rangle} \qquad b \notin \mathrm{FV}(\Gamma, F\langle t/b\rangle). \tag{9.26}$$

   The term t and the formula F may contain b. The substitution replaces the free occurrences of b once and does not revisit an inserted copy of t. The case $t \equiv b$ is permitted.

There is no transport rule, no hypothesis rule apart from USE, no computation inference, no successor axiom and no separate induction rule. The rule (9.26) is the rule NAME of Section 6.2 on the present normal terms, with the same soundness argument (Theorem 6.15). Weakening, exchange, contraction, injective renaming of parameters and cut of an ordinary assumption are admissible by induction on derivations, as in Theorem 6.6; these operations preserve the test of CLOSE.

Lemma 9.99 (Elimination, closure and cases)**.** *In $\mathcal{K}_{\mathrm{Name}}$:*

1. *from $\Gamma \vdash D_{x,y}B$ and $\Gamma, \mathcal{B}_b \vdash A$ with $b \notin \mathrm{FV}(\Gamma, A)$ one infers $\Gamma \vdash A$;*
2. *writing $q_y A(y) := D_{x,y}A(y)$ for an unused x: from $\Gamma, A(0) \vdash A(Sz)$ one infers $\Gamma \vdash q_y A$, and from $q_y A$ and $A(0)$ one infers $A(t)$;*
3. *numerical cases: from $\Gamma \vdash A(0)$ and $\Gamma \vdash A(Sz)$, z fresh, one infers $\Gamma \vdash A(t)$.*

Proof. (1) is the proof of Theorem 6.16, with $\mathcal{A}$ for REFL: take $t \equiv b$ in (9.26). (2) and (3) are Theorem 6.7, whose proof uses USE, the introduction at the unused witness 1 (in case mode, since 1 is not literally 0), weakening, and (1) for a matrix with an unused witness coordinate. □

Lemma 9.100 (Transport from one discharge)**.** *For every term template $h(w)$, from $\Gamma \vdash s = t$ and $\Gamma \vdash h(s) = r$ one infers $\Gamma \vdash h(t) = r$ in $\mathcal{K}_{\mathrm{Name}}$. Hence symmetry, transitivity, congruence and the crossed transport TR are derivable.*

Proof. Let $K(y) := C(P(y), 1, 0)$, so that $K(1) \equiv 1$ and $K(S1) \equiv 0$ as normal forms. Choose x, y fresh and put

$$B(x,y) := \Big(C(y, 0, C(K(y), x, h(x))) = C(y, 0, C(K(y), t, r))\Big).$$

Its normalized rows are $B(x,0) \equiv (0 = 0)$, $B(x,1) \equiv (h(x) = r)$ and $B(x, S1) \equiv (x = t)$, and its row at Sz is $C(K(Sz), x, h(x)) = C(K(Sz), t, r)$. To introduce $D_{x,y}B$ at the witness s, apply numerical cases to the equation $C(v, s, h(s)) = C(v, t, r)$ and instantiate it at $v = K(Sz)$: its zero case is $s = t$ and its successor case is $h(s) = r$. This derivation ignores the local assumption of CLOSE and is valid in either mode, including $s \equiv 0$. Now apply (9.26) with a fresh b. In the opened context the antecedent $0 = 0$ holds by $\mathcal{A}$; USE at $S1$ gives the equality premise $b = t$, and USE at 1 gives the formula premise $h(b) = r$. The export is $h(t) = r$ by (9.7); the passive data do not contain b. This is the derivation of Theorem 6.17 with the packet above in place of that of Theorem 6.11. The remaining equality rules follow from this uncrossed transport and $\mathcal{A}$ by Theorem 6.9. □

The argument is not circular: ordinary elimination, closure and numerical cases (Theorem 9.99) are available before any equality rule, and the packet needs only the literal endpoint identities and no range theorem for K.

Lemma 9.101 (Formula transport)**.** *From $s = t$ and $F(s)$ one derives $F(t)$ in $\mathcal{K}_{\mathrm{Name}}$, for every formula template F.*

Proof. Theorem 6.10, whose hypotheses hold by Theorems 9.99 and 9.100: the atomic case is transport in an equation template, and at a paired binder the introduction is at a fresh parameter, which is not literally 0, so that CLOSE is in case mode. □

Lemma 9.102 (The introduction at the witness 0)**.** *For an equation $E = (s = t)$ put $G_vE := (C(v, 0, s) = C(v, 0, t))$ and $L(E) := q_vG_vE$. Then $G_0E \equiv (0 = 0)$ and $G_{Sw}E \equiv E$ literally, and $E \dashv\vdash L(E)$ in every context. Consequently CASE is derivable in $\mathcal{K}_{\mathrm{Name}}$ for every matrix and witness, and so are PROG and induction on literal equations.*

Proof. The two literal identities are the endpoint equations of C on normal forms (Theorem 9.10). With them, G_vE is a gate in the sense of Theorem 4.142, and the proof of that lemma gives $E \dashv\vdash L(E)$, with the closure rules of Theorem 9.99(2) in place of the closure by cases and QE, and $\mathcal{A}$ for the base $0 = 0$; no transport is used. The two unrestricted introductions are then Theorem 6.8, whose hypotheses hold in $\mathcal{K}_{\mathrm{Name}}$ by Theorem 9.99(1): in the only instance of CASE that CLOSE does not provide, the compound matrix $L(E(x, y))$ is introduced at the witness 0 in case mode, opened at a fresh c, and $D_{x,y}E$ is introduced at the witness c, which is not literally 0. Induction on literal equations is CLOSE in progress mode at the witness 0 of an unused coordinate, followed by the elimination of Theorem 9.99(2) with the supplied base. □

Theorem 9.103. *On normal terms, with $\mathcal{A}$ in every context, $\mathcal{K}_{\mathrm{Name}}$ and $\mathcal{L}$ derive the same sequents. In particular numerical substitution is admissible in $\mathcal{K}_{\mathrm{Name}}$.*

Proof. REFL is USE of $\mathcal{A}$, COMP is an instance of it on normal terms, and TR, DE, CASE and PROG are Theorems 9.99, 9.100 and 9.102. Conversely CLOSE is an instance of CASE or PROG. For (9.26), apply DE; in the opened context derive F and $b = t$; transport in the template obtained from F by replacing the free occurrences of b by a new variable (Theorem 9.84) gives $F\langle t/b\rangle$, which does not contain b; close DE. This is the second half of the proof of Theorem 6.19. □

9.9.2 Use, Introduce and Open with an inferred witness

Definition 9.104 (The calculus $\mathcal{K}_{\mathrm{Open}}$)**.** Terms are normal and all displayed instances are normalized. The rules are USE and the introduction (DI), as in Theorem 9.98, and the following elimination. For a major premise $D_{x,y}B(x, y)$ and a parameter b fresh for Γ and the parameters of B, let $B_b(y)$ be the normalized instance $B(b, y)$ and

$$\tau_B(b) := \begin{cases} r, & B_b(0) \equiv (0 = 0) \text{ and } B_b(1) \equiv (b = r), \\ b, & \text{otherwise.} \end{cases} \tag{9.27}$$

This is the selector ρ_B of Theorem 6.23 with the neutral antecedent $0 = 0$: the variable b has to be the entire left side of the row at 1, and no search for a provable equality takes place. The rule is

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma,\ [B(\tau_B(b), 0);\ y.\, B(\tau_B(b), y)] \vdash A}{\Gamma \vdash A} \qquad b \notin \mathrm{FV}(A). \tag{Open}$$

The branch is checked in the scope that contains b, also when the inferred term contains b. The replacement of b by $\tau_B(b)$ is simultaneous and occurs once.

The rule (Open) is the rule OPEN of Section 6.3 on the present normal terms. It is sound for the ordinary reading by the proof of Theorem 6.27, and weakening, injective renaming of parameters and cut of an ordinary assumption are admissible by Theorem 6.6; injective renaming preserves the literal tests.

Lemma 9.105 (Safe openings and the repair of cases)**.** *In $\mathcal{K}_{\mathrm{Open}}$:*

1. *if the witness coordinate is unused in B, or the root of B is compound, then $\tau_B(b) \equiv b$ and* (Open) *is the ordinary elimination* DE*;*
2. *the closure rules and numerical cases of Theorem 9.99(2),(3) hold;*
3. CASE *is derivable for every matrix and witness.*

None of these uses a substitution theorem or an equality rule.

Proof. (1) is Theorem 6.24(b): if x is unused, the row at 1 does not contain the fresh b; if the root of B is compound, the row at 0 is compound and is not $0 = 0$. (2) and (3) follow as in Theorem 6.31: Theorem 6.7 and Theorem 6.8, in the form of Theorem 9.102, require the ordinary elimination only for the two kinds of matrices of (1). □

Lemma 9.106 (Substitution)**.** *If $\Gamma \vdash A$ in $\mathcal{K}_{\mathrm{Open}}$, then $\Gamma\langle\sigma\rangle \vdash A\langle\sigma\rangle$ for every simultaneous numerical substitution σ.*

Proof. The proof of Theorem 6.32 applies without change, with (9.7) in place of the coherence of normalization with substitution on iterator terms, Theorem 9.105(3) for a case introduction whose witness becomes 0 under an equation matrix, and the preservation of a successful probe under a substitution that fixes b (Theorem 6.24(c), whose proof uses only that coherence). As there, a failed probe is not stable under substitution, and in that case the induction hypothesis is applied to the original branch with the substitution that extends σ by $b \mapsto \tau_{B'}(b)$, B' the normalized instance of B under σ; the induction is on proper subderivations of the original derivation only. □

Lemma 9.107 (Ordinary elimination, transport and induction)**.** *In $\mathcal{K}_{\mathrm{Open}}$:*

1. DE *is admissible;*
2. *for every term template $h(w)$, from $s = t$ and $h(s) = r$ one infers $h(t) = r$; hence symmetry, transitivity, congruence and transport in every formula template are derivable;*
3. PROG *and induction on literal equations are derivable.*

Proof. (1) As in Theorem 6.33: given a branch under the clause at b, apply Theorem 9.106 to it with $b \mapsto \tau_B(b)$ and then (Open); the context and the conclusion do not contain b.

(2) Use the matrix

$$B(x, y) := \Big(C(y, 0, C(P(y), x, h(x))) = C(y, 0, C(P(y), t, r))\Big). \tag{9.28}$$

Its normalized rows are $B(x, 0) \equiv (0 = 0)$, $B(x, 1) \equiv (x = t)$ and $B(x, S1) \equiv (h(x) = r)$, and its row at Sz is literally $C(z, x, h(x)) = C(z, t, r)$, because $P(Sz)$ computes to z. At the witness s, numerical cases on z prove this row: the zero case is $s = t$ and the successor case is $h(s) = r$. The derivation ignores the local assumption of the introduction, so it is valid in either mode and proves $D_{x,y}B$. Now (Open) infers the witness t: the row at 0 is $0 = 0$ and the row at 1 is $b = t$. The opened clause has the base $0 = 0$, which $\mathcal{A}$ proves, and USE at $S1$ gives $h(t) = r$, which is in the outer scope. This is the derivation of Theorem 6.34 with the packet (9.28). The remaining statements follow by Theorems 6.9 and 6.10, with (1) for the eliminations.

(3) is Theorem 6.8 (progress introduction) and Theorem 5.10: the introduction at the witness 0 with an unused witness coordinate gives $q_y E(y)$ from $\Gamma, E(z) \vdash E(Sz)$, its opening is safe by Theorem 9.105(1), and USE with the base gives $E(t)$; PROG at a witness t follows with CASE. □

The decoder makes the packet (9.28) simpler than the packet of Theorem 9.100 and than the parity packet of Theorem 6.11 on iterator terms: one successor computation exposes the variable on which the numerical cases are taken.

Theorem 9.108. *On normal terms, with $\mathcal{A}$ in every context, $\mathcal{K}_{\mathrm{Open}}$ and $\mathcal{L}$ derive the same sequents.*

Proof. The rules of $\mathcal{L}$ are derivable by Theorems 9.105 and 9.107. Conversely the introduction is an instance of CASE or PROG. For (Open) with a successful probe at r, apply DE at b; the opened clause has the antecedent $0 = 0$, and USE at 1 gives $b = r$. For a fresh y, USE gives $B(b, y)$ and transport along $b = r$ gives $B(r, y)$; hence the clause $[B(r, 0);\, y.\, B(r, y)]$ of the branch can be cut (Theorem 9.83), and the branch gives A. With a failed probe the rule is DE. The comparison of $\mathcal{O}$ with the reference calculus in Theorem 6.35 is the same argument with the clause cut replaced by a closure. □

9.9.3 The scalar normal forms embed exactly

The calculus of `inferred_witness` is $\mathcal{K}_{\mathrm{Open}}$ on iterator terms $0, S, I$ in full normal form: a normal iterator term $I(n, a;\, v.\, h)$ has a neutral counter, and the normalizer nf_I applies both computation equations of Section 1.1. Let T_{raw} translate variables and 0 identically, the successor by the defined successor, and $I(n, a;\, v.\, h)$ by $\mathrm{Rec}(n, a; i, v.h)$ with i fresh, and for a normal scalar term m put $T(m) := \mathrm{nf}(T_{\mathrm{raw}}(m))$.

Lemma 9.109 (Exact embedding of normal forms)**.** *For every scalar term t, $\mathrm{nf}(T_{\mathrm{raw}}(t)) = T(\mathrm{nf}_I(t))$. The raw translation commutes with substitution and T commutes with normalized substitution. On normal scalar terms T preserves and reflects the literal 0, each variable and the property of being a successor, and it preserves the set of free variables.*

Proof. The two scalar computation equations translate to the computations of Theorem 9.10, so $T_{\mathrm{raw}}(t)$ and $T_{\mathrm{raw}}(\mathrm{nf}_I(t))$ have the same normal form by Theorem 9.7. For a normal scalar term describe T by induction. Zero, variables and successors are immediate. A normal iterator has a neutral counter, and its image is

$$P(R(Tn, Ta; i, u.S((Th)[P(u)/v]))).$$

Here Tn is neutral, so the inner recursor is retained and neutral, and the outer P is retained. Substituting the neutral term $P(u)$ for the body variable does not turn a neutral counter into 0 or a successor, so the whole image is normal. The raw translation commutes with substitution by induction on terms, with the two new variables fresh at every translated iterator, and (9.7) gives the normalized statement. The four kinds of normal source term have distinct kinds of image: 0, the same variable, a canonical successor, a neutral recursor. No free variable of a normal source term is erased. □

Proposition 9.110 (Translation of the scalar calculus)**.** *Extended to formulas without change of roots, T maps every instance of* USE*, of the introduction and of* (Open) *of the normalized scalar calculus to an instance of the same rule of $\mathcal{K}_{\mathrm{Open}}$.*

Proof. For USE this is commutation with normalized substitution. For the introduction, reflection of the literal 0 preserves the mode, and the translated successor row is the required successor row. For (Open) the two tests on rows are preserved and reflected, so $\tau_{T(B)}(b) = T(\tau_B(b))$, and normalized substitution gives the translated branch clause. Preservation of free variables gives the scope conditions. □

The translation concerns derivations of the already normalized scalar calculus. If raw scalar proofs are normalized first, a witness can become 0; that separate step is handled by Theorem 9.105(3). The embedding preserves more than standard numerical values: a row becomes $0 = 0$, or has a given fresh variable as its entire left side, exactly when the source row did.

9.9.4 The main theorem for the two calculi

Theorem 9.111. *$\mathcal{K}_{\mathrm{Name}}$ and $\mathcal{K}_{\mathrm{Open}}$ are sound and faithful for T^2_R and are equiconsistent and bi-interpretable with* iPRA*, with whole domains, actual equality, numerical composites of graph $x = y$, and every formula provably equivalent to its double translation. Every fixed primitive recursive description compiles to a term whose defining equations hold by computation (Theorem 9.12).*

Proof. By Theorems 9.103 and 9.108 both calculi are $\mathcal{L}$ on normal terms, and Theorems 9.25 and 9.95 apply. In particular $\mathcal{K} \vdash 0 = 1$ if and only if $\mathsf{iPRA} \vdash \bot$; this is a metatheorem and not an assertion that either system proves its own consistency. □

On normal terms the coverage theorem of Theorem 9.93 strengthens to $t = 0 \vee_* t = S(p(t))$, since $p(Sz) \equiv z$; this is the form used by `uniform_recursion` and `single_constructor`.

9.10 Root conversion on the counter-aware recursor with successor

This section draws on `root_conversion` and on a second, independent delivery of the same calculus, `root_conversion_raw` (Theorem 9.128).

The term language of this section keeps the successor as a constructor and gives the body of the recursor the old counter; the value of the recursor at 0 is its initializer, not the successor of it. The recursor is the primitive form of the recursor that Theorem 1.101 defines in T_{eq} by pairing. On these raw terms the bounded conversion of Section 4.10.3 can be restricted to depth 0: one computation step at the root of the left endpoint of an equation, in either direction, together with substitution for a variable as the only equality rule and the single introduction, gives a complete calculus with seven rules. The counter of the recursor is what makes depth 0 sufficient: a staged packet exposes each premise of transport at a root that an earlier root step has uncovered, so that no step inside a counter is needed. Whether depth 0 suffices on the iterator terms of Section 1.1, whose bodies do not see the counter, is not settled (Section 9.15).

9.10.1 Terms and root steps

Definition 9.112 (The counter-aware recursor with successor)**.** Terms are $t ::= x \mid 0 \mid St \mid \hat{R}(n, a; i, u.h)$; the variables i, u are bound in h only, i receiving the old counter and u the preceding value. The computation equations are

$$\hat{R}(0, a; i, u.h) = a, \tag{9.29}$$

$$\hat{R}(Sn, a; i, u.h) = h[n/i,\ \hat{R}(n, a; i, u.h)/u], \tag{9.30}$$

with simultaneous capture-avoiding substitution on the right. Terms are raw syntax up to the renaming of bound variables. In particular $\hat{R}(0, a; i, u.h)$ is not identical to a, and no normalization is part of the identity of terms. As in Theorem 9.1, each fixed body is a

function symbol of the counter, the initializer and the passive parameters of the body. With all bound variables fresh, and $i, u \notin \mathrm{FV}(b)$ in the selector, put

$$\hat{C}(n,a,b) := \hat{R}(n,a;i,u.b), \qquad \hat{p}(n) := \hat{R}(n,0;i,u.i), \qquad I(n,a;\,u.\,h) := \hat{R}(n,a;i,u.h) \quad (i \notin \mathrm{FV}(h)). \tag{9.31}$$

The last abbreviation embeds the iterator terms of Section 1.1: with i unused, (9.29) and (9.30) are literally (1.1) and (1.2).

Definition 9.113 (Root steps)**.** For terms r, r' write $r \to_0 r'$ if r is the left side of an instance of (9.29) or (9.30) and r' is its displayed right side, and $r \leftrightarrow_0 r'$ if $r \to_0 r'$ or $r' \to_0 r$.

The counter of the redex has to be literally 0 or literally a successor Sn. There is no step below a successor, in the counter, initializer or body of a recursor, or below a formula binder, and $\leftrightarrow_0$ is a single step, not its reflexive and transitive closure. For example $S(\hat{R}(0,0;i,u.u))$ and $S0$ are not related by $\leftrightarrow_0$; the equation between them is derived in Theorem 9.124. The abbreviations (9.31) have the root steps

$$\hat{C}(0,a,b) \to_0 a, \qquad \hat{C}(Sn,a,b) \to_0 b, \qquad \hat{p}(0) \to_0 0, \qquad \hat{p}(Sn) \to_0 n.$$

Proposition 9.114 (Direct compiler)**.** *Every fixed primitive recursive description compiles to a term of Theorem 9.112 whose defining equations are instances of* (9.29) *and* (9.30)*: zero, successor and projections are terms, composition is substitution, and a recursion* $f(0,\vec{p}) = g(\vec{p})$*,* $f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p})$ *compiles to*

$$f^*(n,\vec{p}) := \hat{R}\big(n, g^*(\vec{p}); i, u.h^*(i,u,\vec{p})\big).$$

Proof. By external induction on the description. At a recursion, $f^*(0,\vec{p}) = g^*(\vec{p})$ is an instance of (9.29), and $h^*(i,u,\vec{p})[n/i, f^*(n,\vec{p})/u]$ is $h^*(n, f^*(n,\vec{p}),\vec{p})$, so the successor equation is an instance of (9.30). □

For example $\hat{p}(Sn) = n$, $x + y := \hat{R}(y,x;i,u.Su)$ and $x \mathbin{\dot{-}} y := \hat{R}(y,x;i,u.\hat{p}(u))$. The compiler of Theorem 9.12 needs the decoder because the seeded recursor starts at Sa; here no wrapper is needed, and no pairing as in Theorem 1.102. The compiler is an external recursion on finite descriptions, not an evaluator inside a theory.

9.10.2 The calculus

Definition 9.115 ($\mathcal{K}_{\mathrm{Root}}$ and $\mathsf{N}_{\hat{R}}$)**.** Formulas are the native formulas $A ::= (s = t) \mid Q_y A \mid D_{x,y} A$ of Section 1.2 over the terms of Theorem 9.112, with the reading of that section. Contexts are finite reusable lists of formulas, and the designated contradiction is $0 = S0$. The calculus $\mathcal{K}_{\mathrm{Root}}$ has the following seven rules:

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \textsc{hyp} \qquad \frac{}{\Gamma \vdash t = t}\ \textsc{refl} \qquad \frac{\Gamma \vdash r = v}{\Gamma \vdash r' = v}\ \textsc{hc}_0 \quad (r \leftrightarrow_0 r')$$

$$\frac{\Gamma \vdash x = t \qquad \Gamma \vdash u = v}{\Gamma \vdash v = u[t/x]}\ \textsc{vsub} \qquad \frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \textsc{qe}$$

$$\frac{\Gamma \vdash D_{x,y} B(x,y) \qquad \Gamma,\ Q_y B(b,y) \vdash A}{\Gamma \vdash A}\ \textsc{de} \qquad \frac{\Gamma,\ B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y} B(x,y)}\ \textsc{wi}$$

In VSUB the source x is a variable; the term t is arbitrary and may contain x, and the replacement is performed once. In DE the variable b is not free in Γ, A or $D_{x,y}B$. In WI the variable z is not free in Γ, in the witness t or in $D_{x,y}B$, and $\eta = z$ if B is a literal equation and $t \equiv 0$, otherwise $\eta = 0$. The displayed assumptions are discharged. The kernel $\mathsf{N}_{\hat{R}}$ is the calculus N of Theorem 1.7 over the same terms and formulas, with (9.29) and (9.30) in place of I0 and IS.

There is no primitive computation equation, transport, symmetry, case rule, induction rule, successor axiom, logical connective or structural rule. HC_0 changes only the left endpoint, by one step at its root; the right endpoint and formulas below binders are not converted. $\mathcal{K}_{\text{Root}}$ is the calculus BH_3 of Theorem 4.116 with HC_3 replaced by its instances of depth 0, on the terms of Theorem 9.112. The package names the rules Hyp, Refl, Root, Var, Use, Open and Intro. Its eight-rule control has DI and QI of Theorem 1.7 in place of WI; the passage between the two inventories is Theorem 4.3 and is not new.

9.10.3 Equality from root steps

Lemma 9.116 (Structure, symmetry and cases before transport)**.** *In $\mathcal{K}_{\text{Root}}$:*

1. *weakening, injective renaming of free variables and cut on an assumption are admissible;*
2. *symmetry is derivable without HC_0; hence HC_0 at the right endpoint is admissible, and every instance of* (9.29) *and* (9.30) *is derivable in every context;*
3. *numerical cases: if $\Gamma \vdash A(0)$ and $\Gamma \vdash A(Sz)$ with z not free in Γ and $A(y)$, then $\Gamma \vdash A(t)$ for every term t, for every native formula A;*
4. *QI of Theorem 1.7 is derivable.*

None of these derivations uses transport.

Proof. (1) and the first part of (2) are Theorem 4.117, whose proof applies word for word: an injective renaming commutes with capture-free substitution, so it sends a root step to a root step, a variable to a variable and the literal 0 only to itself, and symmetry is REFL at a variable x followed by VSUB with the source x, the replacement x and the minor premise $s = t$, whose conclusion is $t = s[x/x] \equiv t = s$. A step at the right endpoint is symmetry, HC_0 and symmetry. For $r \to_0 r'$, REFL gives $r' = r'$ and HC_0 with $r' \leftrightarrow_0 r$ gives $r = r'$.

(3) Let w be a variable not free in A. The matrix $A(y)$ of $D_{w,y}A(y)$ at the witness 1, which is not literally 0, puts WI in case mode, with the premise $\Gamma, A(0) \vdash A(Sz)$; it is the second given derivation weakened by (1). This gives $\Gamma \vdash D_{w,y}A(y)$. DE at a fresh b adds $Q_y A(y)$, and QE with this hypothesis and the first given derivation, weakened, gives $A(t)$, which does not contain b.

(4) For a literal equation $E(y)$ and the premise $\Gamma, E(z) \vdash E(Sz)$, WI for $D_{w,y}E(y)$, w unused, at the witness 0 is in progress mode and has this premise; DE with HYP exports $Q_y E(y)$, which does not contain the fresh witness. These are the constructions of Theorem 4.3(i). □

The package `root_conversion` derives symmetry in the scoped form that Theorem 4.118 describes for `bounded_conversion`: it introduces $D_{a,y}(0 = 0)$, opens it at a fresh b, applies VSUB to $b = b$ and $u = v$, and discharges b. Its checker declares the numerical scope of every sequent, so that a closed sequent has no variable to serve as the source.

Definition 9.117 (The staged packet)**.** Let s, t, r be terms and $h(w)$ a term template, let x, y be variables not free in s, t, r, h, and let all bound variables be fresh. Put

$$\hat{F}(c; a, b) := \hat{R}(c, 0; i, u.\hat{C}(i, a, b)), \qquad \Pi(x, y) := \Big(\hat{F}(y; x, h(x)) = \hat{F}(y; t, r)\Big).$$

The bodies ignore the state u, and the old counter i is the control of the inner selector. Since i, u are not free in a, b,

$$\hat{F}(0; a, b) \to_0 0, \qquad \hat{F}(Sn; a, b) \to_0 \hat{C}(n, a, b).$$

Hence, on each endpoint, the rows of the packet are reached by root steps only:

Row	Equation reached	Root steps per endpoint
$\Pi(x,0)$	$0=0$	one
$\Pi(x,1)$	$x=t$	two: $\hat{F}(1;a,b)\to_0\hat{C}(0,a,b)\to_0 a$
$\Pi(x,SSz)$	$h(x)=r$	two: $\hat{F}(SSz;a,b)\to_0\hat{C}(Sz,a,b)\to_0 b$
$\Pi(x,Sz)$	$\hat{C}(z,x,h(x))=\hat{C}(z,t,r)$	one

The second step of a row acts at the root that the first step exposes. No step acts inside a counter or inside the payloads $x, h(x), t, r$.

Lemma 9.118 (Packet introduction)**.** *If $\Gamma\vdash s=t$ and $\Gamma\vdash h(s)=r$ in $\mathcal{K}_{\mathrm{Root}}$, then $\Gamma\vdash D_{x,y}\Pi(x,y)$.*

Proof. Let z be fresh for Γ, s, t, r, h and put $E(v):=(\hat{C}(v,s,h(s))=\hat{C}(v,t,r))$ with v fresh. From $s=t$, a step at the right endpoint (Theorem 9.116(2)) with $\hat{C}(0,t,r)\to_0 t$ and a step at the left endpoint with $\hat{C}(0,s,h(s))\to_0 s$, both read as expansions, give $E(0)$. From $h(s)=r$ the same two expansions with $\hat{C}(Sw,\cdot,\cdot)$, w fresh, give $E(Sw)$. Numerical cases (Theorem 9.116(3)) give $E(z)$. Two further expansions, with $\hat{F}(Sz;s,h(s))\to_0\hat{C}(z,s,h(s))$ on the left and $\hat{F}(Sz;t,r)\to_0\hat{C}(z,t,r)$ on the right, give $\Gamma\vdash\Pi(s,Sz)$. Weaken by the local assumption $\Pi(s,\eta)$ of WI at the witness s, which is not used, and apply WI; z is fresh for Γ, s and $D_{x,y}\Pi$. The derivation is the same in both modes, so the case $s\equiv 0$, where WI selects progress, is not exceptional. □

Theorem 9.119 (Transport from root steps)**.** *For every term template h, the rule TR of Theorem 1.7,*

$$\frac{\Gamma\vdash s=t\qquad\Gamma\vdash h(s)=r}{\Gamma\vdash r=h(t)},$$

is admissible in $\mathcal{K}_{\mathrm{Root}}$. Its derivation uses VSUB only in symmetry and in its last step, and no transport, congruence or transitivity.

Proof. Theorem 9.118 gives $D_{x,y}\Pi$. Apply DE at a variable b fresh for Γ, s, t, r, h; the added hypothesis is $Q_y\Pi(b,y)$. REFL gives $0=0$, and one expansion at each endpoint gives $\Pi(b,0)$. QE at 1 and at $S1$, followed by the two contractions of each row on each endpoint (Theorem 9.116(2) for the right endpoint), gives

$$b=t\qquad\text{and}\qquad h(b)=r.$$

VSUB with the source b, the major premise $b=t$ and the minor premise $h(b)=r$ concludes $r=h(b)[t/b]$, which is $r=h(t)$ because b is not free in h and r. The conclusion does not contain b, and DE discharges it. □

Remark 9.120 (The packet and its predecessors). The three rows $0=0$, $x=t$ and $h(x)=r$ and the use of a fresh name with VSUB are those of the packets of Theorem 6.11, of Theorem 4.110 and of (9.28). In (9.28) the rows are literal identities of normal forms, through the decoder $P(Sz)=z$ on normal terms; on raw iterator terms Theorem 4.119 reaches them by steps at depths up to 3 inside the counter of a selector, which computes a parity. The packet of Theorem 9.117 moves the test of the row into the counter variable of the body, so that every step is at a root. The proof uses expansions as well as contractions: the rows $\Pi(s,Sz)$ and $\Pi(b,0)$ are built from smaller equations. It does not show that contractions alone suffice, and since it uses the counter of the body, it says nothing about the iterator terms of Section 1.1 (Section 9.15).

The packet is the case of three entries of a family of finite tables, all of whose entries are reached by root steps; the family is from `root_conversion_raw`.

Lemma 9.121 (Finite tables)**.** *For an external number k and terms $a_0, \dots, a_k$ define, by recursion on k and with all bound variables fresh for the entries and for one another,*

$$T_0(y; a_0) := a_0, \qquad T_{k+1}(y; a_0, \dots, a_{k+1}) := \hat{R}(y, a_0;\, i, u.\, T_k(i; a_1, \dots, a_{k+1})).$$

Write $\bar{\jmath} := S^j 0$. For every $m \le k$ and every term z there is a chain of m root steps

$$T_k(S^m z; a_0, \dots, a_k) \to_0 \cdots \to_0 T_{k-m}(z; a_m, \dots, a_k).$$

Consequently $T_k(\bar{\jmath}; a_0, \dots, a_k)$ reaches a_j by $j+1$ root steps if $j < k$, and $T_k(S^k z; a_0, \dots, a_k)$ reaches a_k by k root steps for every term z. Each step acts at the root of the term that the preceding step produced, and no step acts inside an entry. In $\mathcal{K}_{\mathrm{Root}}$, for every term v and every context, $\Gamma \vdash a_j = v$ and $\Gamma \vdash T_k(\bar{\jmath}; a_0, \dots, a_k) = v$ are interderivable for $j < k$, and so are $\Gamma \vdash v = a_j$ and $\Gamma \vdash v = T_k(\bar{\jmath}; a_0, \dots, a_k)$.

Proof. Induction on m, for all $k \ge m$ simultaneously. For $m = 0$ the chain is empty. Let $m + 1 \le k$. Then $T_k(S^{m+1} z; a_0, \dots, a_k)$ is $\hat{R}(S(S^m z), a_0;\, i, u.\, T_{k-1}(i; a_1, \dots, a_k))$, and (9.30) gives the root step to $T_{k-1}(i; a_1, \dots, a_k)[S^m z/i,\ \dots/u]$. The variable u is not free in the body, so the recursive call is discarded without being computed; i is not free in the entries, and it occurs in $T_{k-1}(i; \dots)$ only as the counter of the outer recursor if $k - 1 \ge 1$, and not at all if $k - 1 = 0$; after renaming the inner bound variables away from z the result is $T_{k-1}(S^m z; a_1, \dots, a_k)$. The induction hypothesis for $k - 1$ and m supplies the remaining m steps. For $j < k$ take $m = j$ and $z = 0$: the term $T_{k-j}(0; a_j, \dots, a_k)$ has $k - j \ge 1$, so it is a recursor with the counter 0 and the initializer a_j, and (9.29) is one more step. For $m = k$ the last term is $T_0(z; a_k) \equiv a_k$. In $\mathcal{K}_{\mathrm{Root}}$ each step of the chain is one application of HC_0 at the left endpoint, read as a contraction or as an expansion, and at the right endpoint by Theorem 9.116(2). □

The proof is an external induction on the syntax of the table, not an induction inside the calculus. $T_1(y; a, b) \equiv \hat{C}(y, a, b)$, and the table of the packet of Theorem 9.117 is $\hat{F}(c; a, b) \equiv T_2(c; 0, a, b)$. The lemma gives every fixed finite set of observations, not an evaluator of coded tables of variable length.

9.10.4 Equivalence with the native kernel

Lemma 9.122 (The structural theory of $\mathsf{N}_{\hat{R}}$)**.** *Theorems 1.10, 1.11, 1.14 and 1.18 to 1.22 hold for $\mathsf{N}_{\hat{R}}$, with the same proofs; in Theorem 1.18(i) the instance $\hat{R}(0, t; i, u.u) = t$ of* (9.29) *replaces* I0*. Moreover:*

1. Body congruence. *If $\Gamma \vdash h = k$ with i, u not free in Γ, then $\Gamma \vdash \hat{R}(n, a; i, u.h) = \hat{R}(n, a; i, u.k)$ for all terms n, a.*
2. Conversion. *Let $\to$ be the closure of $\to_0$ under all term contexts, including the bodies of recursors. If $t \to t'$, then $\Gamma \vdash t = t'$; if A' arises from A by replacing terms by such reducts, then A and A' are interderivable over every Γ.*

Proof. The cited proofs use of the term language only that a substitution instance of a computation equation is an instance of it, which holds for (9.30) by the composition of simultaneous capture-free substitutions as in Theorem 1.2, with i, u renamed away from the substitution; all other steps are applications of the rules.

(1) Rename i, u away from n, a and let z be fresh. Apply Theorem 1.20 to the literal equation $\hat{R}(z, a; i, u.h) = \hat{R}(z, a; i, u.k)$ and then substitute n for z. At 0 both sides equal a by (9.29). Under the induction hypothesis,

$$\hat{R}(Sz, a; i, u.h) = h[z/i, \hat{R}(z, a; i, u.h)/u] = h[z/i, \hat{R}(z, a; i, u.k)/u] = k[z/i, \hat{R}(z, a; i, u.k)/u] = \hat{R}(Sz, a; i, u.k),$$

by (9.30), congruence in the argument u of h, the instance of the premise under $[z/i, \hat{R}(z,a;i,u.k)/u]$, which fixes Γ (Theorem 1.10), and (9.30).

(2) As in Theorem 4.6: induct on the position of the redex. At the root the equation is a computation equation; below a successor, in a counter or in an initializer use congruence; in a body use (1) with the induction hypothesis, derived in the empty context with i, u free and weakened. For formulas use Theorem 1.21. □

Theorem 9.123 (Root conversion and the native kernel)**.** *For every context* Γ *and native formula* A *over the terms of Theorem 9.112,*

$$\Gamma \vdash_{\mathcal{K}_{\mathrm{Root}}} A \iff \Gamma \vdash_{\mathsf{N}_{\hat{R}}} A,$$

by effective translations of derivations that leave the end sequent unchanged.

Proof. From $\mathsf{N}_{\hat{R}}$ *to* $\mathcal{K}_{\mathrm{Root}}$, rule by rule. HYP, QE and DE are common. The computation equations are Theorem 9.116(2), TR is Theorem 9.119 and QI is Theorem 9.116(4). With TR and REFL, symmetry, transitivity, congruence and transport in literal equations are admissible (Theorem 4.2(b)). An instance of DI is an instance of WI unless its matrix is a literal equation and its witness is literally 0. In that case apply Theorem 4.3(ii) with the term $Z := \hat{R}(0,0;i,u.0)$, which is not literally 0 and for which REFL at 0 and one expansion at the left endpoint give $Z = 0$; the lemma needs only HYP, QE, DE, WI, the structural operations of Theorem 9.116(1), symmetry and transport in literal equations.

From $\mathcal{K}_{\mathrm{Root}}$ *to* $\mathsf{N}_{\hat{R}}$, rule by rule. HYP, QE and DE are common. REFL is Theorem 1.18(i). A step HC_0 from $r = v$ to $r' = v$ is the computation equation $r = r'$ or $r' = r$, symmetry if needed, and transitivity. VSUB is one TR, as in the proof of Theorem 6.50: with w fresh and $K(w) := u[w/x]$, TR from $x = t$ and $K(x) \equiv u = v$ concludes $v = K(t) \equiv u[t/x]$, also when t contains x. WI is Theorem 4.3(iii): in case mode it is an instance of DI; in progress mode its premise is $\Gamma, E(0,z) \vdash E(0,Sz)$, and QI for $E(0,y)$, QE under the assumption $E(0,0)$ and DI at the witness 0 derive the conclusion. □

Corollary 9.124. *In* $\mathcal{K}_{\mathrm{Root}}$ *the substitution of arbitrary terms into derivations, transport in every native formula template, and conversion at every position of a term and below binders (Theorem 9.122(2)) are admissible, and so are all instances of* DI *and* QI*. In particular* $\vdash S(\hat{R}(0,0;i,u.u)) = S0$*, which no single root step proves.*

Proof. These hold in $\mathsf{N}_{\hat{R}}$ by Theorem 9.122, and Theorem 9.123 carries them over with the end sequents unchanged. The example is congruence in the template Sv along the computation $\hat{R}(0,0;i,u.u) = 0$. □

The stronger operations of the corollary are not used in the proofs of Theorems 9.116, 9.118 and 9.119. Substitution is not proved for $\mathcal{K}_{\mathrm{Root}}$ by a direct induction on derivations: a substituted witness can become the literal 0, and VSUB needs a variable source (Theorem 4.4).

9.10.5 The bridge

Definition 9.125 (Translations and the ordinary theory)**.** The translation $t \mapsto t^{\iota}$ to the iterator terms of Section 1.1 fixes variables, 0 and S, and sends $\hat{R}(n,a;i,u.h)$ to the term $R(n^{\iota}, a^{\iota}; i\,u.\,h^{\iota})$ of Theorem 1.101. The translation $s \mapsto s^{\kappa}$ in the other direction fixes variables, 0 and S, and sends $I(n,a;\,u.\,h)$ to $\hat{R}(n^{\kappa}, a^{\kappa}; i, u.h^{\kappa})$ with i fresh. Both are extended to native formulas, fixing $=$, Q_y and $D_{x,y}$, and to ordinary formulas, fixing every connective and quantifier. The ordinary theory $\mathsf{T}_{\hat{R}}$ has the terms of Theorem 9.112, the formulas and logical rules of Theorem 9.13, the computation equations (9.29) and (9.30), coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$, separation $0 = S0 \to \bot$, and the rule (9.11) of induction for literal equations with an arbitrary context.

The term of Theorem 1.101 is built from a and g by substitution and pairing, with its own bound variables renamed away, so ι commutes with capture-free substitution and $\mathrm{FV}(t^\iota) \subseteq \mathrm{FV}(t)$; κ commutes with substitution and preserves free variables. Under κ, (1.1) and (1.2) become instances of (9.29) and (9.30), because i is not free in h^κ.

Theorem 9.126 (Bridge for root conversion)**.** *1. If $\Gamma \vdash_{\mathsf{N}_{\hat{R}}} A$ then $\Gamma^\iota \vdash_{\mathsf{N}} A^\iota$, and if $\Gamma \vdash_{\mathsf{N}} A$ then $\Gamma^\kappa \vdash_{\mathsf{N}_{\hat{R}}} A^\kappa$.*

2. $\mathsf{N}_{\hat{R}} \vdash t^{\iota\kappa} = t$ for every term t of Theorem 9.112, and $\mathsf{N} \vdash s^{\kappa\iota} = s$ for every iterator term s.

3. Every native formula is interderivable with its double translation, in $\mathsf{N}_{\hat{R}}$ and in N, and $\Gamma \vdash_{\mathsf{N}_{\hat{R}}} A$ if and only if $\Gamma^\iota \vdash_{\mathsf{N}} A^\iota$.

4. The same statements hold for $\mathsf{T}_{\hat{R}}$ and T_{eq} with ordinary formulas, and $\Gamma \vdash_{\mathsf{N}_{\hat{R}}} A$ if and only if $\mathsf{T}_{\hat{R}}$ proves A° from Γ°.

Consequently $\mathcal{K}_{\mathrm{Root}}$ and $\mathsf{N}_{\hat{R}}$ are sound and faithful for $\mathsf{T}_{\hat{R}}$ under the ordinary reading, and the three systems are equiconsistent and bi-interpretable with iPRA*, by interpretations whose domains are the whole numerical domain, whose equality is actual equality and whose numerical composites have the graph $x = y$; every formula is provably equivalent to its double translation.*

Proof. (1) Under κ every instance of a rule of N is an instance of the same rule of $\mathsf{N}_{\hat{R}}$: the computation equations by the remark before the theorem, the other rules because κ commutes with substitution, preserves free variables and preserves the root of a formula. Under ι, the instances of (9.29) and (9.30) become, by commutation with substitution,

$$R(0, a^\iota;\, i\,u.\,h^\iota) = a^\iota, \qquad R(Sn^\iota, a^\iota;\, i\,u.\,h^\iota) = h^\iota\big[n^\iota/i,\ R(n^\iota, a^\iota;\, i\,u.\,h^\iota)/u\big],$$

which T_{eq} proves by Theorem 1.102(i), hence N proves them by Theorem 1.107, since the reading and its inverse fix equations; weakening gives them in every context. The other rules translate to the same rules; the freshness conditions survive because $\mathrm{FV}(X^\iota) \subseteq \mathrm{FV}(X)$.

(2) Induct on the term. Variables, 0 and S are fixed. For $t = \hat{R}(n, a; i, u.h)$, the counter of the term of Theorem 1.101 occurs only as the counter of its iterator, so $t^{\iota\kappa} = G(n^{\iota\kappa})$ for $G(z) := R(z, a^\iota;\, i\,u.\,h^\iota)^\kappa$. By (1) applied to the derivations in N of the two equations displayed above, $\mathsf{N}_{\hat{R}}$ proves $G(0) = a^{\iota\kappa}$ and $G(Sz) = h^{\iota\kappa}[z/i, G(z)/u]$. By the induction hypotheses, $a^{\iota\kappa} = a$ and $h^{\iota\kappa} = h$ with i, u free. The term $\hat{R}(z, a; i, u.h)$ has the base a and the update h, so the proof of Theorem 9.21, which uses induction on a literal equation, congruence and an instance of a derived equation and is available in $\mathsf{N}_{\hat{R}}$ by Theorem 9.122, gives $G(z) = \hat{R}(z, a; i, u.h)$; the instance at $n^{\iota\kappa}$ and congruence with $n^{\iota\kappa} = n$ give the claim. For $s = I(n, a;\, u.\,h)$, the term $s^{\kappa\iota}$ is $R(n^{\kappa\iota}, a^{\kappa\iota};\, i\,u.\,h^{\kappa\iota})$ with i not free in $h^{\kappa\iota}$. By Theorem 1.102(i) and Theorem 1.107 it satisfies the two iterator equations with the base $a^{\kappa\iota}$ and the update $h^{\kappa\iota}$, and the proof of Theorem 9.21 in N, with the induction hypotheses, gives $s^{\kappa\iota} = s$.

(3) Induct on the formula: an equation differs from its double translation by the term identities of (2) and equality reasoning, and the binder cases are Theorem 1.21, in N and, by Theorem 9.122, in $\mathsf{N}_{\hat{R}}$, with the induction hypothesis for the matrix with its bound variables free. If $\Gamma^\iota \vdash_{\mathsf{N}} A^\iota$, then (1) gives $\Gamma^{\iota\kappa} \vdash_{\mathsf{N}_{\hat{R}}} A^{\iota\kappa}$, and the interderivabilities with cut give $\Gamma \vdash A$. The other direction is (1).

(4) For $\mathsf{T}_{\hat{R}}$ the proof of Theorem 9.25 applies with the present ι and κ in place of those of Theorem 9.24(2): the successor is fixed, so (9.18) is an identity; the computation equations translate as in (1); coverage, separation and (9.11) translate to the corresponding axioms and rule of T_{eq}; and the term round trips are those of (2), with Theorem 9.21 for $\mathsf{T}_{\hat{R}}$, whose proof applies unchanged. The reading commutes with ι, since it does not change terms, so

$(A^\iota)^\circ = (A^\circ)^\iota$. Hence $\Gamma \vdash_{\mathsf{N}_{\hat{R}}} A$ if and only if $\Gamma^\iota \vdash_{\mathsf{N}} A^\iota$ by (3), if and only if $(\Gamma^\circ)^\iota \vdash_{\mathsf{T}_{\mathrm{eq}}} (A^\circ)^\iota$ by Theorem 1.107, if and only if $\Gamma^\circ \vdash_{\mathsf{T}_{\hat{R}}} A^\circ$ by the faithfulness of ι for $\mathsf{T}_{\hat{R}}$, used with the deduction theorem for the finite context.

The consequences follow from Theorem 9.123 and Theorem 1.119: the translations fix variables, the domain, equality and the contradiction $0 = S0$, so the composites with the interpretations of Theorem 1.119 have the stated shape, and inconsistency transfers in both directions. □

Remark 9.127 (The route of the package). The package `root_conversion` proves the equivalence of Theorem 9.123 in the same way, except that it simulates VSUB by transport in the equation template $u[w/x] = v$ followed by symmetry and recovers the zero-witness case at the alias $\hat{R}(0, 0; i, u.0)$ by transport in equations. For the bridge it states the realization $\mathsf{T}_{\hat{R}}$, checks the soundness of each rule of $\mathcal{K}_{\mathrm{Root}}$ in it (introduction by cases from coverage, the progress mode from induction on equations, VSUB from actual equality, a root step from one computation equation and symmetry), and invokes the inherited adequacy of the native binders. It interprets iPRA directly, a recursion by Theorem 9.114 and a fixed body by its primitive recursive description, with term round trips by recurrence uniqueness. It also writes the connectives as the macros (9.24) with $\hat{C}$ for C and $\hat{p}$ for P, since $\hat{p}(1) = 0$, $\hat{p}(S1) = 1$ and $\hat{p}(St) = t$ hold by root steps, and states quantifier-free induction through characteristic terms built from $+$, $\dot{-}$ and the selector, as in Theorem 9.28. The article obtains the bridge through N instead, by Theorem 9.126, and does not need these constructions.

Remark 9.128 (The second delivery). `root_conversion_raw` presents the same calculus $\mathcal{K}_{\mathrm{Root}}$, with the same kernel and the same bridge; it differs from `root_conversion` only in presentation.

1. *Terms.* The raw terms of Theorem 9.112, with the recursor $R(n, a; i, u.h)$, i the old counter and u the state, the root steps of Theorem 9.113, and no normal forms; the same abbreviations $\hat{C}$ and $\hat{p}$.

2. *Rules.* The seven rules of Theorem 9.115, named Hyp, Refl, VSub, Root, QE, DE and WI, with the same side conditions; Root is HC_0. The full kernel N_R of the package is $\mathsf{N}_{\hat{R}}$ of Theorem 9.115.

3. *Packet.* The table $T(y; a, b, c) := \hat{R}(y, a; i, u.\hat{C}(i, b, c))$, which is T_2 of Theorem 9.121, and the matrix $T(y; 0, h(x), x) = T(y; 0, r, t)$. This is $\Pi(x, y)$ of Theorem 9.117 with the rows at 1 and $S1$ exchanged: the row at 1 is $h(x) = r$ and the row at $S1$ is $x = t$. In the introduction the zero instance of the selector equation comes from $h(s) = r$ and the successor instance from $s = t$; the proofs of Theorems 9.118 and 9.119 apply with the two roles exchanged.

4. *Proof order.* Symmetry is derived as in the scoped form described after Theorem 9.116, the theorem $D_{x,y}(0 = 0)$ coming from WI at the witness 0 in progress mode, whose premise $0 = 0 \vdash 0 = 0$ is HYP. Numerical cases are stated for literal equations only and obtained either as in Theorem 9.116(3) or from Theorem 9.116(4): weaken the successor premise by the unused assumption $E(z)$, derive $Q_y E(y)$, and apply QE with $E(0)$. The zero-witness case of DI uses the alias $\hat{R}(0, 0; i, u.0)$ of the proof of Theorem 9.123, transporting both endpoints of the atomic matrix between the alias and 0 before and after a cut with the given premise.

5. *Bridge.* The ordinary realization is $\mathsf{T}_{\hat{R}}$ of Theorem 9.125, read as a staged signature with one function symbol for each fixed body, as in Theorem 9.1; the connectives are the macros (9.24) with $\hat{C}$ for C and $\hat{p}$ for P, and the bi-interpretation is the direct route

of Theorem 9.127, with quantifier-free induction through the reflection of Theorem 1.86 applied after substituting recursor terms for variables. Successor injectivity is $\hat{p}(Sx) = x$, and $0 = Sx$ gives $1 = 0$ through $\hat{C}(\cdot, 1, 0)$.

The only mathematical addition is Theorem 9.121. The checker differs from that of `root_conversion` in its public interface and in its matched control (Sections 9.10.6 and 9.14).

9.10.6 The checkers

The package `root_conversion` supplies a standalone checker for $\mathcal{K}_{\mathrm{Root}}$ and an eight-rule control with DI and QI in place of WI. Terms and formulas are JSON arrays with the tags `z`, `s`, `r` (two bound variables in the body, u at index 0 and i at index 1), `=`, `q` and `d`, and variables are de Bruijn indices; Boolean values are rejected as indices. The public interface accepts an object with a closed goal and a proof, validates syntax and scope, and checks a root step by one comparison with a contraction, without normalizing; the branch of DE must return the shifted outer conclusion, so the witness cannot escape. Open sequents are checked through an internal interface with an explicit scope and context. The untrusted generator builds five positive certificates for each checker: open transport from $x = y$ and $Sx = z$ to $z = Sy$; transport at the literal witness 0, where WI is in progress mode; transport inside a recursor body with a retained parameter; the closed theorem $S(\hat{R}(0, 0; i, u.u)) = S0$ of Theorem 9.124, whose direct derivation by one root step is rejected; and the progress closure of induction for $\hat{R}(n, 0; i, u.u) = 0$. Nine negative controls are rejected: an unjustified false equation, a nested conversion presented as a root step, a Boolean index, an out-of-scope variable, an invalid induction mode, a source of VSUB that is not a variable, an escaping witness, an incorrect zero computation, and a compound induction matrix (in the seven-rule checker, the removed tag of QI). These finite runs are evidence for the implementation, not a proof of its soundness; the metatheorems above are written proofs.

The checker of `root_conversion_raw` checks open sequents through its public interface: the input is one object with a number of free variables, a context, a goal and a proof, and all formulas of the context and the goal are validated before the proof. A leaf is HYP or REFL. A VSUB node carries the source index and the two terms of the premises; a QE node carries the matrix and the instance, which are validated in their scopes even where the substitution erases them; a DE node carries the matrix and checks the branch at a new outermost variable against the shifted goal; a WI node carries the witness and selects the mode by the literal test of Theorem 9.115. A root node carries the previous left endpoint r' and checks that one root contraction leads from r' to the current left endpoint or from the current left endpoint to r'; a contraction at the right endpoint is rejected as a primitive node and derived by symmetry. The untrusted producer builds 29 positive and negative fixtures, replayed in 58 separate runs, with and without the optimization flag of the interpreter. The main positive certificate is open transport in a template with several occurrences of the variable below nested recursor binders, including the counter binder (4268 bytes, 18 root steps); further fixtures cover a source that is literally 0, which puts WI in progress mode, a replacement with older free parameters, the two computation equations and the projections to the counter and to the state, the alias of 0, derived symmetry and derived conversion at the right endpoint, and a replacement that contains its source. The negative fixtures cover exchanged binders of the recursor, malformed and erased scopes, wrong sorts, unbound variables of the interface, unused but active hints, a wrong introduction mode, attempted steps inside a counter, an initializer, a successor or the right endpoint, and an unproved $0 = S0$.

9.11 Binary trees with a paramorphism

This section draws on the appendix of `root_conversion`.

The section records an ordinary theory BT of finite binary trees and a bi-interpretation of BT with iPRA in which neither comparison map is the identity: iPRA is interpreted in BT on the quantifier-free definable subdomain of unary trees, BT is interpreted in iPRA on the whole domain by a pairing code, and the comparison maps are internally definable and provably inverse. It is not a countermodel and not a semantics of a native calculus. No calculus or checker of the article uses it, and no checker for it is supplied. Binary-tree domains are listed among the directions that `uniform_predecessor` examined without result, in the chapter on obstructions.

Definition 9.129 (The theory BT)**.** Terms are $t ::= x \mid e \mid N(s,t) \mid \Phi(t;a;u,v,p,q.h)$; the four variables u,v,p,q are bound in h only. The logic is intuitionistic logic with equality (Theorem 1.48). The axioms are the universal closures of

$$\Phi(e;a;u,v,p,q.h) = a, \tag{9.32}$$

$$\Phi(N(s,t);a;u,v,p,q.h) = h\big[s/u,\ t/v,\ \Phi(s;a;u,v,p,q.h)/p,\ \Phi(t;a;u,v,p,q.h)/q\big], \tag{9.33}$$

and $e = N(e,e) \to \bot$. The one further rule is quantifier-free structural induction: for a quantifier-free formula $A(x)$, which may contain every connective,

$$\frac{\Gamma \vdash A(e) \qquad \Gamma,\ A(u),\ A(v) \vdash A(N(u,v))}{\Gamma \vdash A(t)} \qquad (u,v \text{ distinct and not free in } \Gamma,\ t,\ A). \tag{SI}$$

Induction on formulas with quantifiers is not part of BT.

Write $\underline{1} := N(e,e)$ and, with the four bound variables fresh,

$$H(x) := \Phi(x;e;u,v,p,q.u), \qquad T(x) := \Phi(x;e;u,v,p,q.v), \qquad Z(x;a,b) := \Phi(x;a;u,v,p,q.b).$$

Lemma 9.130 (Constructors)**.** BT *proves* $H(e) = T(e) = e$, $H(N(s,t)) = s$, $T(N(s,t)) = t$, $Z(e;a,b) = a$, $Z(N(s,t);a,b) = b$, *the injectivity of* N, $e \neq N(s,t)$, *and coverage* $x = e \vee x = N(H(x),T(x))$.

Proof. The equations are instances of (9.32) and (9.33). Applying H and T to $N(s,t) = N(s',t')$ gives injectivity. From $e = N(s,t)$, applying $Z(\,\cdot\,;e,\underline{1})$ gives $e = \underline{1}$, which separation refutes. Coverage is (SI) for the quantifier-free formula $x = e \vee x = N(H(x),T(x))$: the base is the left disjunct, and at $N(u,v)$ the right disjunct holds by the equations for H and T; the induction hypotheses are not used. □

Lemma 9.131 (Unary trees)**.** *Put* $f(x) := \Phi\big(x;e;u,v,p,q.Z(u;q,\underline{1})\big)$, $U(x) :\Leftrightarrow f(x) = e$ *and* $S_U(x) := N(e,x)$. *Then* BT *proves* $U(e)$ *and* $U(N(u,v)) \leftrightarrow (u = e \wedge U(v))$.

Proof. $f(e) = e$ and $f(N(u,v)) = Z(u;f(v),\underline{1})$. Split coverage for u. If $u = e$, then $f(N(u,v)) = f(v)$, and both sides of the biconditional are equivalent to $U(v)$. If $u = N(u_1,u_2)$, then $f(N(u,v)) = \underline{1}$, so $U(N(u,v))$ gives $e = \underline{1}$, which is refuted, and $u = e$ is refuted by Theorem 9.130; both sides are false. □

Proposition 9.132 (iPRA in BT)**.** *Let* J *interpret* iPRA *in* BT *with the domain* U, *actual equality,* $0^J := e$, $\mathrm{succ}^J := S_U$, *projections and composition by themselves, and*

$$\mathrm{prec}(g_0,g_1)^J(x,\vec{z}) := \Phi\big(x;\ g_0^J(\vec{z});\ u,v,p,q.\ g_1^J(v,q,\vec{z})\big),$$

defined by external recursion on descriptions, quantifiers relativized to U. *Then* BT *proves* $U(\vec{z}) \to U(f^J(\vec{z}))$ *for every function symbol* f, *and if* iPRA $\vdash \varphi$ *then* BT *proves* φ^J *under the assumptions* $U(x)$ *for the free variables* x *of* φ.

Proof. The defining equations of a recursion hold for all trees: at e by (9.32), and at $S_U(n) = N(e,n)$ by (9.33), since the body ignores u and p. Closure under U is proved by (SI) on $U(x) \to U(f^J(x,\vec{z}))$ with $U(\vec{z})$ in the context: the base is closure of g_0^J, and at $N(u,v)$, Theorem 9.131 gives $u = e$ and $U(v)$, the induction hypothesis for v and closure of g_1^J give the claim. Separation of the successor is $N(e,x) \neq e$, and injectivity is the injectivity of N (Theorem 9.130). A quantifier-free formula translates to a quantifier-free formula. For induction, assume $\varphi^J(e)$ and $\forall x\,(U(x) \to \varphi^J(x) \to \varphi^J(S_U x))$, and apply (SI) to the quantifier-free formula $U(x) \to \varphi^J(x)$: at $N(u,v)$, the hypothesis $U(N(u,v))$ gives $u = e$ and $U(v)$, the induction hypothesis for v gives $\varphi^J(v)$, the assumed step gives $\varphi^J(N(e,v))$, and transport along $e = u$ gives $\varphi^J(N(u,v))$. The logical rules translate under relativization, since $U(e)$ holds and U is closed under the translated function symbols. □

Proposition 9.133 (BT in iPRA)**.** *Let I interpret* BT *in* iPRA *with the whole domain, actual equality, $e^I := 0$, $N^I(a,b) := \pi(a,b) := 2^a(2b+1)$, and $\Phi(t;a;u,v,p,q.h)^I := \phi(t^I)$, where ϕ is defined by course-of-values recursion, by external recursion on the term, from*

$$\phi(0) = a^I, \qquad \phi(\pi(i,j)) = h^I[i/u, j/v, \phi(i)/p, \phi(j)/q],$$

with the passive parameters as arguments. Then every axiom of BT *is translated to a theorem of* iPRA*, and (SI) to a derivable rule.*

Proof. The proof uses the following standard facts of primitive recursive arithmetic. (a) π is primitive recursive; iPRA proves that it is injective and never 0, and there are primitive recursive α, β, defined by bounded search, with $n \neq 0 \to n = \pi(\alpha(n), \beta(n)) \wedge \alpha(n) < n \wedge \beta(n) < n$. (b) A function defined by course-of-values recursion from fixed primitive recursive functions is primitive recursive, through the history function, and iPRA proves its recursion equations. (c) For a quantifier-free A, the bounded formula $\forall k < n\, A(k)$ has a primitive recursive characteristic function, so quantifier-free induction on it gives strong induction for A. With these, (9.32) and (9.33) are the recursion equations of ϕ, and separation is $\pi(0,0) = 1 \neq 0$. For (SI), assume $A(0)$ and $A(i) \wedge A(j) \to A(\pi(i,j))$; by (a) every n is 0 or $\pi(\alpha(n), \beta(n))$ with smaller components, and strong induction (c) gives $A(n)$. □

Theorem 9.134 (A bi-interpretation on a changed domain)**.** *I and J form a bi-interpretation of* BT *and* iPRA*, with the following comparison maps.*

1. *In* BT*, for the composite $J \circ I$: the map c from all trees to U and its inverse d defined below;* BT *proves $U(c(x))$, $d(c(x)) = x$, and $U(n) \to c(d(n)) = n$.*
2. *In* iPRA*, for the composite $I \circ J$: the map $\gamma(n) := 2^n - 1$ onto the codes of unary trees and its inverse, the binary length ℓ.*

Both maps preserve the interpreted constants, constructors and function symbols, and every formula is provably equivalent to its double translation, including formulas with quantifiers. Hence BT *and* iPRA *are equiconsistent.*

Proof. All arithmetic in BT below is on U, through J; by Theorem 9.132 the theorems of iPRA about it hold, and induction on U applies to every quantifier-free formula of BT, also when it compares arbitrary trees, since the proof of the induction case of Theorem 9.132 uses only (SI). Write π_U, α_U, β_U, $+$ and $\dot{-}$ for the J-images.

The maps c and d. Put $c(x) := \Phi(x;e;u,v,p,q.\pi_U(p,q))$, so that $c(e) = e$ and $c(N(s,t)) = \pi_U(c(s), c(t))$; (SI) proves $U(c(x))$. Represent lists of trees by e and $N(\text{item}, \text{rest})$, and put

$$\text{drop}(k,L) := \Phi(k;L;u,v,p,q.T(q)), \qquad \text{get}(L,k) := H(\text{drop}(k,L)),$$

so that $\mathrm{drop}(0, L) = L$ and $\mathrm{drop}(S_U k, L) = T(\mathrm{drop}(k, L))$. Define the table τ by recursion on a unary argument with a tree-valued state,

$$\tau(0) = \underline{1}, \qquad \tau(S_U n) = N\big(N(\mathrm{get}(\tau(n), n \dot{-} \alpha_U(S_U n)),\ \mathrm{get}(\tau(n), n \dot{-} \beta_U(S_U n))),\ \tau(n)\big),$$

that is, $\tau(n) := \Phi(n; \underline{1}; u, v, p, q.N(N(\mathrm{get}(q, v \dot{-} \alpha_U(S_U v)), \mathrm{get}(q, v \dot{-} \beta_U(S_U v))), q))$, and put $d(n) := H(\tau(n))$. Unary induction on k proves $\mathrm{drop}(S_U k, N(x, L)) = \mathrm{drop}(k, L)$. Unary induction on n, with k as a parameter, proves the table identity

$$k \leq n \ \rightarrow\ \mathrm{get}(\tau(n), n \dot{-} k) = d(k) :$$

at $S_U n$, if $k = S_U n$ the left side is $H(\tau(S_U n))$; if $k \leq n$, then $S_U n \dot{-} k = S_U(n \dot{-} k)$, the shift lemma removes the new head, and the induction hypothesis applies. This case distinction is decidable arithmetic on U. Since $\alpha_U(S_U n), \beta_U(S_U n) \leq n$, it gives $d(0) = e$ and, for $U(a)$ and $U(b)$, $d(\pi_U(a, b)) = N(d(a), d(b))$, because $\pi_U(a, b)$ is a successor $S_U m$ with $\alpha_U(S_U m) = a$ and $\beta_U(S_U m) = b$.

$d(c(x)) = x$. (SI) on this equation: $d(c(e)) = d(e) = e$, and $d(c(N(u, v))) = d(\pi_U(c(u), c(v))) = N(d(c(u)), d(c(v))) = N(u, v)$, using $U(c(u))$ and $U(c(v))$.

$c(d(n)) = n$ *for* $U(n)$. Put $\mathrm{Mc}(x) := \Phi(x; e; u, v, p, q.N(c(u), q))$, so that $\mathrm{Mc}(e) = e$ and $\mathrm{Mc}(N(x, L)) = N(c(x), \mathrm{Mc}(L))$, and the list $\varepsilon(0) := N(e, e)$, $\varepsilon(S_U n) := N(S_U n, \varepsilon(n))$. Coverage for L and $c(e) = e$ give $H(\mathrm{Mc}(L)) = c(H(L))$ and $\mathrm{Mc}(T(L)) = T(\mathrm{Mc}(L))$ for every tree L; unary induction on k with L as a parameter then gives $\mathrm{Mc}(\mathrm{drop}(k, L)) = \mathrm{drop}(k, \mathrm{Mc}(L))$, hence $c(\mathrm{get}(L, k)) = \mathrm{get}(\mathrm{Mc}(L), k)$. Unary induction gives $k \leq n \rightarrow \mathrm{get}(\varepsilon(n), n \dot{-} k) = k$, as for the table identity. Now unary induction on the single equation $\mathrm{Mc}(\tau(n)) = \varepsilon(n)$: at 0 both sides are $N(e, e)$; at $S_U n$ the new head of $\tau(S_U n)$ has the code

$$\pi_U\big(\mathrm{get}(\varepsilon(n), n \dot{-} \alpha_U(S_U n)),\ \mathrm{get}(\varepsilon(n), n \dot{-} \beta_U(S_U n))\big) = \pi_U\big(\alpha_U(S_U n), \beta_U(S_U n)\big) = S_U n,$$

and the tail is the induction hypothesis. Applying H gives $c(d(n)) = H(\mathrm{Mc}(\tau(n))) = H(\varepsilon(n)) = n$. The induction formulas are equations; no universally quantified statement about lists is used as an invariant.

Preservation by c. For the composite $J \circ I$, the tree e goes to e, the constructor N to π_U, and Φ to the J-image ϕ^J of the course-of-values function of Theorem 9.133, which satisfies the J-images of its recursion equations. The equations $c(e) = e$ and $c(N(s, t)) = \pi_U(c(s), c(t))$ are those above, and $c(\Phi(t; a; u, v, p, q.h)) = \phi^J(c(t))$, with c applied to the parameters, follows by (SI) on t from the two recursion equations and the corresponding statement for a and h, by external induction on the term.

The map γ. Under $I \circ J$, the number 0 goes to 0, the successor to $x \mapsto \pi(0, x) = 2x + 1$, and the domain is the set of codes m with $U^I(m)$, the I-image of U. By Theorem 9.131 translated by I, $U^I(0)$ and $U^I(\pi(i, j)) \leftrightarrow (i = 0 \wedge U^I(j))$. Quantifier-free induction proves $\gamma(Sn) = 2\gamma(n) + 1$, $U^I(\gamma(n))$ and $\ell(\gamma(n)) = n$, and strong quantifier-free induction (fact (c) of Theorem 9.133) proves $U^I(m) \rightarrow \gamma(\ell(m)) = m$. That γ commutes with each function symbol and its $I \circ J$-image follows by induction on descriptions, with recurrence uniqueness at a recursion; for a recursion the image satisfies the translated recursion equations by Theorems 9.132 and 9.133.

Formulas. By induction on formulas: an atomic formula is preserved because the maps preserve the terms and are injective; connectives commute with the translations; a quantifier over the whole domain corresponds to the relativized quantifier, because the maps are bijections onto the relativizing domain with provable inverses (d and ℓ). The contradiction is preserved, so inconsistency transfers. □

Each fixed paramorphism and each fixed recursion is translated by external recursion on finite syntax; no evaluator for descriptions is assumed in either theory. The bi-interpretation

is of a different shape from those of Theorems 1.119 and 9.25, whose numerical composites have the graph $x = y$: here the composites are the definable bijections c and γ. A checker for a calculus on BT would have to charge the four bound variables of Φ, the binary step of (SI) and the logical rules; the package does not claim an implementation or a size advantage.

9.12 Uniform binding and one traversal

This section draws on `clausal_canonical`, `counter_evaluation`, `uniform_binding`, `uniform_recursion` and `single_constructor`, and for the separate refinements on `canonical_recursors` and `register_graph`.

9.12.1 The concrete syntax

In the checkers of these packages the integer 0 is the numerical constant, a negative integer is a free parameter (a level), and a positive integer is a de Bruijn index of a bound variable. Without a successor constructor and without a closure constructor, the array arities distinguish the constructors:

Object	Array	Child depths, one binder in R	Child depths, two binders in R
$R(n, a; \vec{u}.h)$	$[n, a, h]$	$(d, d, d+1)$	$(d, d, d+2)$
$s = t$	$[s, t]$	(d, d)	(d, d)
$D_{x,y}B$	$[B]$	$(d+2)$	$(d+2)$

In a paired matrix y has index 1 and x index 2; in the body of the counter-aware recursor the state has index 1 and the counter index 2. With two binders in R, every constructor other than the equation binds exactly two variables in its last child, so that for child number j of a node of arity k the depth increment is

$$2\,[\,k \neq 2 \text{ and } j = k - 1\,]. \tag{9.34}$$

The sort is still checked: formula arities are 1 and 2, and the only arity of a compound term is 3. The arity convention does not identify a formula with a term. A successor is $[0, a, 0]$, and a compound canonical counter is a successor exactly when its first child is 0; the test that distinguishes a successor node from an iterator node on the terms of Section 1.1 is not needed.

A context clause is a pair (A, B) whose second component has one more bound variable. It is a separate data structure and is never passed to the opening operation as one object; clause use opens B alone.

Definition 9.135 (Opening)**.** For an expression e and a term v, the operation $O_d(e, v)$ removes the index $d + 1$ and preserves the d nearer indices: on an integer it replaces $d + 1$ by v, decrements an integer greater than $d + 1$ and leaves every other integer unchanged; on an array it processes the children at the depths of the table; and at each resulting recursor node it erases the body if the resulting counter is 0. O_d^{raw} is the same operation without the erasure.

Lemma 9.136 (Opening)**.** *Let e have free scope m and ambient depth $k > d$, and let v be a term of free scope m with no free positive index.*

1. *$O_d(e, v)$ has the sort of e, ambient depth $k-1$ and free scope m; it contains no parameter beyond those of e and v; and it is zero-body canonical if e and v are.*

2. *For canonical e, v, $O_d(e, v) = \nu(O_d^{\mathrm{raw}}(e, v))$, and for arbitrary raw e, v, $\nu(O_d^{\mathrm{raw}}(\nu e, \nu v)) = \nu(O_d^{\mathrm{raw}}(e, v))$.*

3. *For a matrix B and terms t, w without free positive indices, $O_0(O_1(B,t),w) = \nu(B[t/x, w/y])$, and the two openings may be performed in the opposite order.*

Proof. (1) Induct on e. The integer cases remove the selected index, lower an outer index, or keep a nearer index or a parameter. A replacement without free positive indices stays well scoped under any additional binders. The array cases use the binder increments of the table. Erasing a body removes occurrences only. (2) is Theorem 9.3 for the substitution that opening performs: a body discarded before the opening had a counter that was literally 0 and stays 0; a counter that becomes 0 only by the opening is erased on both sides by the outer ν; and $\nu(t)$ is 0 only if t is. (3) follows from the simultaneous-substitution lemma, since neither replacement contains a free positive index, and from (2). □

The hypothesis on v is essential. Because free parameters are negative levels and do not depend on the binder depth, inserting such a v under binders needs no lifting; inserting a term with a free positive index without lifting is incorrect. For the same reason an old context needs no shifting when a fresh parameter is allocated: the new negative integer is absent from everything validated before. A branch can use it, but its discharged result cannot contain it, because the outer goal and the witness are validated at the old scope.

Remark 9.137 (Unfolding under binders)*.* With full normalization a successor contraction can occur under binders, and the inserted terms can then have free positive indices. The simultaneous substitution of (9.4) is implemented counter first: remove the counter slot at depth 1, under the retained state slot and with lifting of the inserted term, and then remove the state slot. Substituting the previous state first without lifting captures an ambient index under the counter binder that is still present. For example, under one ambient binder $R(1, v; j, w.w)$ normalizes to Sv, that is $[[0,0,0],1,1]$ normalizes to $[0,1,0]$, whereas the state-first order returns $S0$. This correction is recorded in the mathematical review shipped with `uniform_recursion`.

9.12.2 One traversal

Lemma 9.138 (The shared walker)**.** *Let* walk *visit a tree with the depth increments* (9.34)*, check the sort and arity of every node, apply a given operation to the numerical leaves, and erase the body at every recursor node whose resulting counter is* 0*.*

1. *With the leaf operation that checks $-n \le i \le d$ for an exact integer i at scope n and depth d and returns i, the test that the result equals the input holds exactly for the members of the scoped zero-body canonical grammar.*

2. *With the leaf operation that maps $d+1$ to v, an index $i > d+1$ to $i-1$ and every other leaf to itself, and with v canonical without free positive indices, the result is the canonical capture-avoiding opening O_d.*

Proof. Induct on the tree. The integer clause checks the sort and the scope interval. At an array the admissible arity determines whether the children are terms or formulas, and the last child of a recursor or a paired formula receives two more protected indices. Every child is visited before the erasure, so a malformed body is rejected before it can be discarded. In (1), equality with the returned tree excludes every zero-counter node with a nonzero body. In (2) the three index cases are opening of one slot, and the final erasure is that of O_d. □

Thus one traversal owns grammar validation, the canonical-form check and opening. For a certificate that starts from the empty context every branch of an elimination or introduction adds exactly one clause and one fresh parameter, and no other rule adds either; by induction on checker calls the scope equals the length of the context, so no separate scope counter is passed. This is an invariant of certificates built from the empty root, not a statement

about abstract clause contexts. In the rule (CE) the checker opens the state slot with the canonical term $R(t, a; i, u.h)$ and then the former counter slot with t, which is the simultaneous substitution; the opening is computed before the two premises are checked, and this does not bypass validation, because proving the major premise validates both endpoints and proving the minor premise validates the seed and the original body, whose outer counter is a successor and cannot have been erased.

Proposition 9.139 (Checker correspondence)**.** *Disregarding physical resource bounds:*

1. *A closed canonical formula has a certificate accepted by the goal-directed checker of* `uniform_binding` *if and only if it is a theorem of* $\mathcal{K}_{\mathrm{CE}}$*, and accepted certificates translate effectively to and from the certificates of the conclusion-synthesizing checker of* `counter_evaluation`*, with literally identical final formulas.*

2. *The checker of* `uniform_recursion` *accepts a certificate for a closed normal goal exactly when the certificate describes a derivation in* $\mathcal{K}_{\mathrm{Name}}$*, and every finite derivation has an accepted certificate; the same holds for* `single_constructor` *and* $\mathcal{K}_{\mathrm{Open}}$*.*

Proof. (1) Use the scope invariant and Theorem 9.138. An accepted reflexive goal is a REFL node, and every other accepted branch is the displayed instance of (CE), USE, DE or (DI) after translation of its premises; this is soundness, by induction on the accepted call tree. Conversely induct on a derivation: insert the derived endpoints and the seed and body of the minor premise as the hints of a (CE) node, copy the clause index and target of USE, the matrix of DE and the witness of the introduction. The opening invariant identifies every required premise goal, and each construction preserves the end formula.

(2) Induct on the proof packets. The formula synthesized at each node is checked for grammar, normal form and scope. In USE the term is checked before instantiation and a proof of the antecedent has to synthesize exactly the retained antecedent. In the introduction the matrix and the witness are checked, the generated assumption is the row selected by the rule, and the result of the branch has to equal the generated successor row. In NAME the major premise has to be a paired formula, the equality has to have the freshly allocated parameter as its left endpoint, and the final scope check excludes that parameter from the normalized export. In (Open) the major premise is checked before the context is extended, the probe (9.27) is computed, and the conclusion of the branch is validated again in the outer scope; when the probe succeeds the stored base is $0 = 0$ and coincides with the substituted base of the rule. Theorem 9.7 and Theorem 9.137 identify the computed instances with the normalized substitutions. Conversely a derivation is encoded by its rule data and the certificates of its premises, with fresh negative levels in order of scope creation. □

9.12.3 The checkers

All checkers are standalone Python modules that read one JSON object with exactly the keys `goal` and `proof`, validate a closed goal against the full grammar, and accept by printing `ACCEPT` with exit status zero; none imports a proof producer or a theorem table. Exact integer tests exclude Booleans, and recursion and memory limits of the interpreter can reject a large valid certificate. Sizes are in Section 9.14.

`register_graph`. The checker implements the natural deduction of Theorem 9.13 with named variables, simultaneous substitution, alpha equivalence, the three arithmetic schemas and (9.11). It is a readable reference and is much larger than the compact checkers of the kernel. A separate ancillary module is a profile of the kernel N on iterator terms, not a checker for T_J: it replaces a recursive continuation of its parent by a loop and uses immutable tuple contexts, which lowers the node count of the inspected leader by one and raises its bytes. Its generic opener inherits a qualification on malformed input: in an interpreter without limits

certain nonempty strings cause unbounded traversal, so soundness concerns finite successful executions.

canonical_recursors. Three modules share one proof-tree format, context handling, de Bruijn convention and driver: iterator terms with a successor and seven rules, raw seeded terms with seven rules, and $\mathcal{K}_6$. The canonical module applies the constructor $[n, a,\ 0 \text{ if } n = 0 \text{ else } h]$ at every newly built recursor node after its children have been normalized, validates canonical input and has no zero-computation rule. The scalar control uses crossed eliminations and the witness-directed introduction and is an equivalent presentation of the formalized seven-rule calculus named in Section 9.6, not a transcription of it.

clausal_canonical. The checker for $\mathcal{K}_5$ has the instruction tags `r`, `x`, `u`, `e`, `i` and one opening function for terms and formulas. Its matched control is $\mathcal{K}_6$ ported to the same variable convention and opening function, with the closure constructor, the restricted transport and the root-reversing closure elimination retained, so that the comparison isolates the change of rules and formulas from the change of representation.

counter_evaluation. The checker for $\mathcal{K}_{\mathrm{CE}}$ has the tags `r` (reflexivity), `v` (counter evaluation), `u`, `e`, `i` and synthesizes the conclusion of every node. Its matched control is $\mathcal{K}_{\mathrm{CT}}$ on the same syntax. An untrusted translator implements Theorem 9.88: a computation node becomes (CE) on two reflexive leaves and a transport node becomes the construction of Theorem 9.87.

uniform_binding. The checker for $\mathcal{K}_{\mathrm{CE}}$ is goal directed: a validated reflexive goal closes at once and its payload is ignored, and the other instructions are `v`, `u`, `e`, `i` with hints. Validation and opening are the two uses of Theorem 9.138, and the scope is the length of the context. Its matched control applies the same organization to iterator terms.

uniform_recursion. The checker for $\mathcal{K}_{\mathrm{Name}}$ has the packets `u`, `i`, `e`; the fixed clause (9.25) is the initial context $([\,], [1, 1])$, so that at context length n the parameters are $-1, \ldots, -(n-1)$ and the next one is $-n$. The minimal proof is `{"goal":[0,0],"proof":["u",0,0,null]}`. A variant carries the budget of Theorem 9.9; its control is the scalar control with the same budget instrumentation.

single_constructor. The checker for $\mathcal{K}_{\mathrm{Open}}$ has the packets `u`, `i`, `e`, with the same three logical tests as its control on iterator terms; the changes are confined to the arithmetic syntax, the shared binder increment, the recursor computation and the successor abbreviation in the rules.

9.12.4 Three refinements of checkers for the kernel on iterator terms

Three packages of this chapter also contain refinements of existing checkers that do not concern the calculi of this chapter; no size of these programs is a size of a calculus on the new syntaxes.

`canonical_recursors` refines the compact guarded codec: it pairs the comparisons of the major premise and of the target that are already checked, uses a shorter discriminator between integers and lists, and composes the administrative truth convention of the scalar zero with the guarded checker. The byte and node improvements belong to different modules. The compact modules do not validate the public grammar themselves; a strict front end is counted separately. The eager paired comparison may inspect instruction fields that were unused before, so the completeness statement is made for the specified well-scoped language of integer and list certificates, and no equality of behavior on malformed JSON is claimed.

`uniform_recursion` fuses the grammar validation of the strict native checker of `deterministic_opening` into its first scope traversal. That walker increases its depth by $5 - k$ at a node of arity k. At a formula position under p protected slots the depth is $2p$, and at a term position it is $3 + 2p$; hence even depth means formula and odd depth means term, and the admissible arities at depth d are exactly 1 and $2 + (d \bmod 2)$. A numerical leaf is 0 or an odd integer at least 3 within the scope interval. The grammar predicate is enabled for the initial traversal

without substitution image and disabled before any proof instruction is read. By induction on the tree the fused traversal succeeds exactly when the old grammar test and the old scope traversal both succeed, and it returns the same tree; every later transition of the checker is unchanged, so the translation of certificates in both directions is the identity. Thresholds of resource exhaustion and failure diagnostics are not equated. One variant states the predicate explicitly; a smaller one folds it into the existing cases.

`register_graph` supplies the ancillary profile described above.

9.13 Boundary results

This section draws on `canonical_recursors` and, for Theorem 9.149, on `clausal_canonical`.

The constructions above move a computation into the syntax or change the representation of the terms. The results of this section show that three nearby simplifications are not available: deleting the initializer together with the counter, deleting one of the two computation equations of the iterator, and restricting induction to equations with 0 on one side. A fourth shows that the single computation rule of $\mathcal{K}_5$ cannot be deleted without replacement.

9.13.1 A recursor with neither counter nor initializer

Consider the terms generated by variables, 0, S and a self-seeded constructor $F(n; v.h)$ with the standard interpretation

$$F(0; v.h) = S(h(0)), \qquad F(Sn; v.h) = h\big(F(n; v.h)\big),$$

with arbitrary nesting and passive parameters, and no further operation. The successor may be taken as the abbreviation $F(0; v.a)$, $v \notin \mathrm{FV}(a)$. This is the unary-update variant F of the tied-seed language of Theorem 10.3(b), the instance $\mathsf{B}(n; u, v.h)$ with u not in h. Its terms are almost unary by Theorem 10.41, which is no restriction on unary functions; the theorem below is a lower bound for unary terms and uses the finite-control invariant of Theorem 10.52.

Theorem 9.140 (Unary terms of the self-seeded language)**.** *For every term $t(n)$ of this language with one free variable there is a natural number C such that*

$$\forall n \in \mathbb{N} \quad \big(t(n) \le C \ \vee \ t(n) \ge n\big).$$

Consequently no term defines the predecessor $\max(n-1, 0)$*, integer halving, or any function with unbounded values strictly below its argument.*

Let $\mathcal{U}$ be the class of functions $f\colon \mathbb{N} \to \mathbb{N}$ for which there is C with $f(n) \le C$ or $f(n) \ge n$ for every n.

Lemma 9.141 (Closure)**.** *Constants, the identity and the successor belong to $\mathcal{U}$. The class is closed under composition and under pointwise selection from a finite family of its members.*

Proof. For a pointwise selection take the maximum of the finitely many bounds. For $f \circ g$ with bounds C_f, C_g: if $g(n) \le C_g$ the value is at most $\max_{j \le C_g} f(j)$; otherwise $g(n) \ge n$, and the value is at most C_f or at least $g(n) \ge n$. □

Lemma 9.142 (Orbits)**.** *If $h \in \mathcal{U}$, then the sequence $r(0) = 1 + h(0)$, $r(n+1) = h(r(n))$ is a member of $\mathcal{U}$ as a function of n.*

Proof. If the orbit repeats a value it is eventually periodic and hence bounded. Otherwise it is injective. Let C be a bound for h. A step with $r(n+1) < r(n)$ ends in $\{0, \dots, C\}$, so an injective orbit has finitely many such steps. After the last one the orbit increases strictly, and

it eventually exceeds every value of the earlier finite prefix. For all sufficiently large n the $n+1$ distinct values $r(0), \dots, r(n)$ lie in $[0, r(n)]$, so $r(n) \geq n$. Increase the constant to cover the finitely many earlier arguments. □

Proof of Theorem 9.140. Say that a function $g\colon \mathbb{N}^k \to \mathbb{N}$, $k \geq 1$, has *$\mathcal{U}$-control* if it has finite control in the sense of Theorem 10.50 with every profile in $\mathcal{U}$. A finite-valued feature of the active coordinate may be added to the index: for fixed features of the other coordinates the function is then a pointwise selection from finitely many profiles, which belongs to $\mathcal{U}$ by Theorem 9.141. A unary function with $\mathcal{U}$-control is its own profile and belongs to $\mathcal{U}$. It therefore suffices to show that every term, as a function of any nonempty list of variables that contains its free variables, has $\mathcal{U}$-control.

Composition. Functions with $\mathcal{U}$-control are closed under composition. The proof of Theorem 10.51(ii) shows that the composite has finite control, and that for fixed combined controls its profile is $x_k \mapsto F_{a(x_k)}(G(x_k))$, where G is a profile of the inner function, the F_a are profiles of the outer function, and the index $a(x_k)$ ranges over finitely many values determined by finite features of x_k (Theorem 10.51(i)). This profile is a pointwise selection from finitely many compositions $F_a \circ G$ of members of $\mathcal{U}$, and belongs to $\mathcal{U}$ by Theorem 9.141.

Induction on terms. Variables, 0 and successors are immediate, the last by composition with the successor. For $F(n; v.h)$ first replace the counter by a fresh variable m. By the induction hypothesis $h(v, \vec{x})$ has $\mathcal{U}$-control with an active coordinate y; write $c = \beta(\vec{x})$ for the features of the passive coordinates other than y and β_v for the feature of v when v is not active.

If y is the state v, fix c. The update $h_c(v) = H_c(v)$ is a profile and belongs to $\mathcal{U}$, and by Theorem 9.142 so does its orbit r_c. Then $F(m; v.h) = r_{\beta(\vec{x})}(m)$ has $\mathcal{U}$-control with the counter active.

If y is a passive parameter p, the catalogue argument of the third case of the proof of Theorem 10.52 applies to $F(m; v.h) = \mathsf{B}(m; u, v.h)$, with the old counter u absent, so that no feature of the counter enters the transitions. The catalogue consists of the update profiles $H_{(b,c)}(p)$ and the initial profiles $1 + H_{(\beta_v(0),c)}(p)$; it yields

$$F(m; v.h) = V_{\eta(m)_{\beta(\vec{x}),\kappa(p)}}(p), \qquad \kappa(p) = \big(\beta_v(V_k(p))\big)_k,$$

with a finite-valued feature η of the counter and with every catalogue function V_k in $\mathcal{U}$ by Theorem 9.141. This is $\mathcal{U}$-control with p active. The argument uses only finite observations and no test of numerical equality.

Finally substitute the actual counter term n for m by the composition step. This completes the induction and proves the bound. For the predecessor choose $n > C + 1$: its value exceeds C and is less than n. Halving fails at every $n > 2C + 1$. □

Remark 9.143. The theorem concerns definitions by terms under the standard interpretation. It is not an obstruction to interpretations by graphs or on changed domains. The counter-aware tied-seed language of the package `local_scopes` has the finite-control obstruction of Theorem 10.52 but defines the predecessor (Theorem 10.4); the unary bound is specific to removing the access to the old counter together with the independent initializer. The majorant obstructions of Theorems 10.36 and 10.41 bound terms from above by a function of one coordinate and say nothing about unary terms; the theorem above is a dichotomy for unary terms, which they do not give.

9.13.2 The two computation equations of the iterator are independent

Proposition 9.144. *Neither computation equation of the iterator of Section 1.1 follows from the other, even over the standard natural numbers with coverage, separation and every first-order induction instance.*

Proof. Interpret every fixed body compositionally in $\mathbb{N}$. If $I(n, a;\, u.\, h)$ is interpreted as $h^n(0)$, the successor equation holds and $I(0, 1;\, u.\, u) = 0 \neq 1$. If it is interpreted as a, the zero equation holds and the successor equation fails for $h(u) = Su$. □

Thus the canonical calculi succeed by changing the representation; they do not derive a deleted law from the remaining axioms.

9.13.3 Induction on zero equations does not give induction on equations

Let T_0 be the theory T_{eq} of Section 1.5 with its induction rule restricted to equations of the form $0 = t(n, \vec{p})$, or equivalently $t(n, \vec{p}) = 0$, with an arbitrary context. A calculus that uses the progress introduction only for matrix equations with the literal 0 on one side has this induction. The following theorem shows that such a restriction does not preserve the consequence relation.

Theorem 9.145 (A model separating the two inductions)**.** *There is a classical model of T_0, in which the successor is injective, and in which the sentence*

$$\forall n\, \forall a\ I(n, a;\, u.\, u) = a$$

is false, although its instance at $n = 0$ and its induction step are provable from the computation equations and equality logic. Consequently induction on arbitrary literal equations is not admissible in T_0. The same model satisfies induction for every equation $\bar{k} = t(n, \vec{p})$ with a fixed standard numeral $\bar{k}$ on one side.

A classical model is in particular a model of intuitionistic logic. Its construction uses compactness and choices of compatible paths in the metatheory; these are not principles granted to T_0.

Lemma 9.146 (Paths through finite fibers)**.** *Let $(B_j)_{j\in\mathbb{Z}}$ be finite nonempty sets and $f_j \colon B_j \to B_{j+1}$ total maps. There is a sequence $(b_j)_{j\in\mathbb{Z}}$ with $b_j \in B_j$ and $b_{j+1} = f_j(b_j)$ for every j.*

Proof. The product $\prod_j B_j$ of finite discrete spaces is compact, and each equation between adjacent coordinates is a closed condition. Every finite set of these conditions is satisfiable: choose a value at the least index involved, propagate it forward, and choose arbitrary values elsewhere. The finite intersection property gives a common solution. □

Proof of Theorem 9.145. The base. Fix a nonstandard model M of Peano arithmetic expanded by symbols for all primitive recursive descriptions; it exists by compactness. For $m \in M$ let $A_m = \{0\}$ if m is a standard numeral and $A_m = \{0, 1\}$ otherwise; the distinction is external. Put

$$D = \{(m, i) : m \in M,\ i \in A_m\}, \qquad \pi(m, i) = m, \qquad 0_D = (0_M, 0), \qquad S_D(m, i) = (S_M m, i).$$

The successor is well defined because it preserves standardness and nonstandardness. The map π is surjective and preserves zero and successor. Every standard numeral has a one-element fiber and every nonstandard element has two lifts. On D the successor is injective, zero is not a successor, and coverage holds: a nonstandard (m, i) has the predecessor $(m - 1, i)$. The nonstandard part of M is a disjoint union of chains of the order type of $\mathbb{Z}$ under successor, and the label i is constant along each chain of D.

The iterator on actual functions. Let $\mathscr{H}$ be the set of functions $h \colon D \to D$ for which there is a unary function $f \colon M \to M$, primitive recursive with parameters, such that

$$\pi(h(x)) = f(\pi(x)) \qquad (x \in D). \tag{9.35}$$

By surjectivity of π the function f is unique. Write $\mathrm{It}_f(m,b)$ for its iteration in M with initial value b. This does not depend on the description chosen for f: two descriptions of the same function give iterations with the same base and pointwise equal updates, and an induction in M proves them equal at every counter. This induction is used in the base model, not in T_0. Define an operation $\mathcal{I}\colon D \times D \times \mathscr{H} \to D$ as follows. For a standard counter k put $\mathcal{I}(k,a,h) = h^k(a)$, the finite iteration on D. For fixed a, h and a nonstandard chain $n_j = (m_j, i)$, $j \in \mathbb{Z}$, with $m_{j+1} = S_M m_j$, put

$$r_j = \mathrm{It}_f(m_j, \pi(a)), \qquad B_j = \pi^{-1}(r_j).$$

The sets B_j have one or two elements, $r_{j+1} = f(r_j)$ by the computation law in M, and (9.35) makes h a total map $B_j \to B_{j+1}$. Choose a path $b_j \in B_j$ by Theorem 9.146 and put $\mathcal{I}(n_j, a, h) = b_j$. Make these choices for all triples of a function, an initializer and a chain, with one specified family: for $h = \mathrm{id}_D$ choose on every nonstandard chain the constant path

$$\mathcal{I}(n, a, \mathrm{id}_D) = (\pi(a), 0). \tag{9.36}$$

This is legitimate, since the projected iteration is constantly $\pi(a)$, the point with label 0 always exists, and the identity fixes it. The choices depend on the actual function h, so every description of the identity body receives the same family. By construction

$$\mathcal{I}(0_D, a, h) = a, \qquad \mathcal{I}(S_D n, a, h) = h(\mathcal{I}(n, a, h)), \qquad \pi(\mathcal{I}(n, a, h)) = \mathrm{It}_f(\pi(n), \pi(a)),$$

the standard and the nonstandard counters never meeting under a finite number of successor steps.

Terms. Interpret $I(n, a;\, u.\, h)$ in D under a valuation ρ as $\mathcal{I}(\llbracket n \rrbracket, \llbracket a \rrbracket, v \mapsto \llbracket h \rrbracket_{\rho[u:=v]})$. By induction on the term, with a bound variable of a body treated as a further free variable,

$$\pi(\llbracket t \rrbracket_{D,\rho}) = \llbracket t \rrbracket_{M, \pi \circ \rho}, \tag{9.37}$$

where the right side uses the iterator of M; in particular every body function supplied to $\mathcal{I}$ satisfies (9.35) with the interpretation of the body in M and belongs to $\mathscr{H}$. The operation $\mathcal{I}$ is chosen for actual functions before this induction, so nested terms create no circular definition. Equal parameter values determine the same body function and the same iterator; hence D is a first-order structure for the staged language, the substitution lemma holds, and both computation schemas hold for every fixed body.

Zero equations. Since the fiber over 0_M has one element, $\llbracket t \rrbracket_D = 0_D$ if and only if $\llbracket t \rrbracket_M = 0_M$ at the projected valuation. Fix parameters $\vec{a}$ in D and suppose that in D

$$t(0, \vec{a}) = 0, \qquad \forall n \, \big(t(n, \vec{a}) = 0 \to t(Sn, \vec{a}) = 0\big).$$

By (9.37) and surjectivity of π the same two statements hold in M for the primitive recursive interpretation of t at $\pi\vec{a}$. Induction in M and reflection of zero along π give $\forall n \in D \; t(n, \vec{a}) = 0$. The rule with a context is valid as well: at a valuation that makes the context true the two premises supply the displayed base and universal step, and no projection of the context is needed. The same argument applies to an equation $\bar{k} = t(n, \vec{a})$, because every standard numeral has a one-element fiber.

Failure of the full rule. Choose a nonstandard $b \in M$ and $a = (b, 1)$, and let $E(n)$ be $I(n, a;\, u.\, u) = a$. Its base is a computation equation, and its step is provable from $I(Sn, a;\, u.\, u) = I(n, a;\, u.\, u)$ and transitivity. For every nonstandard counter n, however, (9.36) gives $I(n, a;\, u.\, u) = (b, 0) \neq (b, 1) = a$. The two elements have the same projection; tests against zero cannot distinguish the labels. □

Corollary 9.147 (No reflection of equality)**.** T_0 *proves for no term* $e(x, y)$ *the law* $e(x, y) = 0 \leftrightarrow x = y$.

Proof. In the model take $x = (b, 0)$ and $y = (b, 1)$ over a nonstandard b. By (9.37) the terms $e(x, y)$ and $e(x, x)$ have the same projection. If the law held, $e(x, x) = 0$, and the one-element fiber over zero would force $e(x, y) = 0$, contradicting $x \neq y$. □

The usual conversion of an equation into a zero equation therefore cannot justify the restriction: the reflection of equality of Theorem 1.86 is exactly the missing conclusion. The theorem refutes the recovery of the induction rule of T_{eq} in the presence of full intuitionistic logic with equality, all numerical cases, both computation schemas and injectivity of the successor. Under the reading of Section 1.2, the rules QE, DE, introduction by cases and progress for zero equations are valid in the same model, so the native rules do not evade the counterexample. The construction does not show that T_0 has no interpretation of arithmetic on a quotient or on a changed domain (Section 9.15).

Remark 9.148. Theorems 9.145 and 9.147 are term-level model constructions for a restricted induction rule; the obstructions of Section 10.6 concern term-definability, and the nonstandard model of Section 10.9 concerns the deletion of the progress rule, not its restriction to zero equations; neither contains them. The model above refutes the rule of successor uniqueness of the package `orbit_closure`, as recorded in Theorem 2.104, and therefore does not decide Theorem 2.168.

9.13.4 The last computation rule

Proposition 9.149. *The calculus $\mathcal{K}_5$ without rule (R) derives no equation from the empty context, not even $0 = 0$.*

Proof. Assign the value false to every equation and the value true to every paired formula, and read a clause $[A; y.\, B]$ as the implication from the value of A to the value of B. This agrees with the reading of the paired binder as a conditional closure when all equations are false, and it does not impose the equality axioms. By induction on derivations, if all clauses of Γ are true, then the conclusion of $\Gamma \vdash A$ is true. Rule (T) has equations as premises, so it applies only when some clause is false. Rule (U) is modus ponens. In rule (E) the opened clause $[B(b, 0); y.\, B(b, y)]$ is true, because its two components have the same root. Rule (I) concludes a paired formula. With the empty context the conclusion is true and is therefore not an equation. □

Adding a reflexivity rule or another seed of equations repairs the loss, as in Sections 9.8 and 9.9, but that is a replacement and not a deletion. The proposition concerns this one deletion; it is not a theorem that a presentation of iPRA needs five rules.

9.14 Checker sizes

This section draws on the measurements stated by `register_graph`, `canonical_recursors`, `clausal_canonical`, `counter_evaluation`, `uniform_binding`, `uniform_recursion`, `single_constructor`, `constantfree`, `guarded_register`, `root_conversion` and `root_conversion_raw`.

Each row is a complete standalone module in the sense of Section 1.10: imports, JSON decoding, validation of syntax and scope, normalization, substitution, every rule and the final comparison are counted. All measurements are stated by the packages for CPython 3.12.14, after removal of docstrings and `ast.unparse` with one final line feed. The role column names the new program of a package, its matched control, and the other programs that the package reports for orientation. The programs marked † implement the kernel on iterator terms and no calculus of this chapter.

Program	Tokens	AST	Bytes	Role
`register_graph`				
Reference checker for T_J	2814	3060	13633	new
Ancillary profile of the kernel†	261	288	604	new
Direct parent of the ancillary profile†	261	292	576	control
Earlier leader in nodes†	263	289	613	other
Earlier leader in bytes†	266	310	529	other
`canonical_recursors`, strict family				
$\mathcal{K}_6$ on canonical seeded terms, six rules	1338	1485	4611	new
Iterator terms with successor, seven rules	1433	1595	4923	control
Raw seeded terms, seven rules	1378	1532	4753	control
`canonical_recursors`, compact guarded codec†				
Refined compact byte profile	265	306	516	new
Frozen compact byte control	265	303	522	control
Refined compact node profile	263	280	602	new
Frozen compact node control	262	281	606	control
Refined strict guarded checker	542	624	1376	new
Matched strict guarded control	541	620	1381	control
`clausal_canonical`				
$\mathcal{K}_5$, five rules	946	1072	3305	new
$\mathcal{K}_6$ in the same variable convention	1107	1266	3820	control
$\mathcal{K}_6$ as shipped by `canonical_recursors`	1338	1485	4611	other
Six rules on zero-normal iterator terms (`zero_normal`)	1013	1075	3095	other
`counter_evaluation`				
$\mathcal{K}_{\mathrm{CE}}$	854	987	3004	new
$\mathcal{K}_{\mathrm{CT}}$, same terms, computation and transport	929	1063	3272	control
Scalar zero-normal, evaluation with one case	884	1025	3061	other
Raw scalar clause checker with five rules, unchanged	853	993	3046	other
The same with the known deletion of the root case	848	986	2999	other
$\mathcal{K}_5$ of `clausal_canonical`, unchanged	946	1072	3305	other
`uniform_binding`				
$\mathcal{K}_{\mathrm{CE}}$, goal directed, one traversal	693	764	2238	new
Scalar checker with the same shared traversal	699	773	2284	control
Incoming scalar checker, exact source	741	826	2283	control
Conclusion-synthesizing checker of `counter_evaluation`	854	987	3004	other
`uniform_recursion`				
$\mathcal{K}_{\mathrm{Name}}$	662	741	1950	new
Scalar control with NAME	663	751	1983	control
$\mathcal{K}_{\mathrm{Name}}$ with a budget	710	800	2136	new
Scalar control with a budget	711	810	2169	control
Strict native checker, explicit fusion†	340	385	685	new
Strict native checker, integrated cases†	336	381	661	new
Strict native control†	360	409	780	control
Compact control with a different contract†	267	302	508	other
`single_constructor`				

Program	Tokens	AST	Bytes	Role
$\mathcal{K}_{\mathrm{Open}}$	683	764	1965	new
Iterator terms with inferred opening	683	771	1990	control
`constantfree`				
Reference checker for $\mathsf{T}_J^{\varnothing}$	3072	3349	13723	new
Reference checker for T_J, as measured by `constantfree`	2811	3054	12626	control
`guarded_register`				
Strict checker for $\mathcal{K}_{\mathrm{G}}$	1041	1181	3467	new
Strict checker for N on iterator terms, same interface†	1045	1193	3566	control
`root_conversion_raw`, open-sequent interface				
$\mathcal{K}_{\mathrm{Root}}$, seven rules	1029	1096	3003	new
BH_3 of Theorem 4.116 on iterator terms (`bounded_head`)	1056	1127	3089	control

The comparisons that isolate a change of calculus are those against a matched control. The checker for $\mathcal{K}_6$ is 312 bytes (6.34%) below the scalar control and 142 bytes below the raw control, with normalization counted. The checker for $\mathcal{K}_5$ is 515 bytes (13.48%), 161 tokens and 194 nodes below its matched control; the difference of 1306 bytes to the shipped checker for $\mathcal{K}_6$ combines the change of interface with the simpler variable representation and is not to be attributed to the deleted rule. The checker for $\mathcal{K}_{\mathrm{CE}}$ is 268 bytes (8.19%), 75 tokens and 76 nodes below $\mathcal{K}_{\mathrm{CT}}$, and the node overheads of the translation are those of Theorem 9.87. The goal-directed checker with one traversal is 46 bytes, 6 tokens and 9 nodes below the matched scalar control and 45 bytes, 48 tokens and 62 nodes below the exact incoming checker. The checker for $\mathcal{K}_{\mathrm{Name}}$ is 33 bytes, one token and 10 nodes below its control, and the checker for $\mathcal{K}_{\mathrm{Open}}$ is 25 bytes and 7 nodes below its control with the same token count. The checker for $\mathsf{T}_J^{\varnothing}$ is 261 tokens, 295 nodes and 1097 bytes above its control: removing the constant enlarges this implementation, through the construction of the expanded terms, the checks of anchors and the scoped rule (Param). The control is the reference checker of `register_graph`, but the figures that `constantfree` measures for it differ from the figures 2814, 3060, 13633 that `register_graph` states; the packages do not reconcile the two measurements. The strict checker for $\mathcal{K}_{\mathrm{G}}$ is 4 tokens, 12 nodes and 99 bytes (2.78%) below its control, which has the same JSON interface, variable representation, logical nodes, scope checks and acceptance convention; the guard, the state-free test and the successor-head test are counted in it. The checker of `root_conversion` for $\mathcal{K}_{\mathrm{Root}}$ (Section 9.10), which validates the complete syntax and scope of a closed goal and its proof, measures 1108 tokens, 1301 nodes and 3875 normalized bytes, against 1173, 1379 and 4096 for its matched eight-rule control with DI and QI, both measured by the package with docstrings retained. The checker of `root_conversion_raw` for the same calculus, which checks open sequents with full validation of the goal, the context and every active hint, is 27 tokens, 31 nodes and 86 bytes below the checker of BH_3 of `bounded_head`, which has the same interface, scope discipline and proof architecture but the scalar iterator and conversion up to three counter edges, so that the two programs differ in the term language as well as in the computation rule; the new module is counted whole, the control without its docstring, and the difference to the checker of `root_conversion`, a separate program with another interface, is not attributed to the calculus. Compressed sizes are reported by the packages as supplementary data and are not an optimization target; they are omitted here.

9.15 Further questions

This section draws on `register_graph`, `canonical_recursors`, `clausal_canonical`, `counter_evaluation`, `uniform_binding`, `uniform_recursion`, `single_constructor`, `constantfree`, `guarded_register`, `root_conversion` and `root_conversion_raw`. It collects their open questions, the claims that they state without proof, and what they report as not achieved.

Status of formalization. None of the calculi and term languages of this chapter is formalized in Lean or Rocq. The theorems of Sections 9.1 to 9.11 and 9.13 are written proofs, and the executable tests of the packages are finite evidence for the implementations. The four certificates of `constantfree` for Theorems 9.35 and 9.37, and the twelve certificates of `guarded_register` for Theorem 9.51, are checked by the Python checkers of the packages only; Theorem 9.57 rests on the formalized Theorem 1.119, whose Lean build `guarded_register` did not rerun. The formalized results named in Sections 9.6 and 9.9 concern iterator terms.

Question 9.150 (Formalization)**.** Each term language of this chapter needs its own syntax, its normalizer where it has one, and the bi-interpretation with iterator terms of Theorem 9.25. The packages propose the following orders of work. For Theorems 9.66 and 9.72: body extensionality before the soundness of normalization, and the tests on literal sources and witnesses after the unrestricted equivalence. For $\mathcal{K}_5$: the invariant on context occurrences and the eta expansion of the closure, proved directly. For $\mathcal{K}_{\mathrm{CE}}$: a deep embedding with a checked equivalence to the kernel; the arithmetic reconstruction is small and the main work is the syntax with two binders and its canonicalization. For normal terms: extend normalization by evaluation from iterator terms to the counter-aware recursor, prove the environment laws (9.8)–(9.9), and lift the comparison of the inferred opening through Theorem 9.109. For the checkers: the invariant of the sorted walker (Theorem 9.138) together with the interpretation of the counter-aware recursor. For the register: the compiler and both round trips as translations of finite descriptions, keeping fixed syntactic bodies apart from arbitrary functions of the metatheory. Can these be carried out by extending the existing development?

Question 9.151 (A native register kernel)**.** Can a native checker with Q_y and $D_{x,y}$ share its verified equality premise with the increment and load steps (9.12), (9.13), so that the register grammar also reduces the complete trusted source, when all conditional premises, scope checks and syntax checks are counted? Theorem 9.19 shows compatibility of the basis with such a kernel; no checker for it is supplied. The guarded calculus $\mathcal{K}_{\mathrm{G}}$ of Section 9.5 is a native register kernel of a different form: its bodies are restricted, its computations are unconditional, and the conditional rules are derived (Theorem 9.52). Its strict checker is below its matched control (Section 9.14). For the unrestricted register with conditional rules the question remains open.

Question 9.152 (Guarded registers)**.**

1. Can $\mathcal{K}_{\mathrm{G}}$ and Theorem 9.57 be formalized on top of the existing development of N? `guarded_register` gives the equivalence as a written proof only.

2. Is there a complete checker for $\mathcal{K}_{\mathrm{G}}$, or for a variant of it, below the compact envelope of 256 tokens, 276 nodes and 506 bytes? That envelope combines three different programs, a token profile of `kernel_refinement` (256, 283, 594), a node profile of `deterministic_opening` (263, 276, 580) and a byte profile of `deterministic_introduction` (267, 303, 506), which share a guarded public-language contract that differs from the complete validation of the strict checkers. The package states that a claim of this kind has to implement the whole certificate contract, charge the guard and every scope operation, and compare complete programs under the same normalization and acceptance conventions.

3. `guarded_register` reports two shortcuts that it examined and did not use, and proves neither claim. Restricting both the introduction of a witness and the specialization of a clause at once is said to need a joint completeness argument, since the separate completeness arguments may each depend on the unrestricted form of the other rule. Removing the scope information before the phase that chooses fresh names is complete is said to admit future witnesses.

Question 9.153 (The register selector and the cost of the counter)**.** Is there a substantially shorter numerical selector for the register basis than the term C of Theorem 9.16, which has a short proof and a large expansion? Source brevity, term size and certificate size are different objectives. Does the register with two bound variables, or the one-variable register of Theorem 9.18, yield the smaller complete checker after comparable implementations?

Question 9.154 (Weakening the graph axiom)**.** Which parts of (STEP) can be weakened while actual equality and the formula round trips on the whole domain are retained? `register_graph` states without proof that one-sided operational membership and agreement on the standard model are insufficient. `constantfree` states in the same way, without proof, that replacing the biconditional of $\mathsf{T}_J^{\varnothing}$ by an unconditional evaluation equation gives a different theory, and asks whether fewer or cheaper inference patterns can supply both the computation and the coverage that the graph schema provides.

Question 9.155 (The constant-free presentation)**.**

1. Can the anchored schemas of Theorem 9.32 be compiled into a native calculus with Q_y and $D_{x,y}$, without a large selector, normalizer or second certificate language, and what is the size of the complete standalone checker, including validation of the public grammar?

2. Can a certificate protocol recover the anchor of each arithmetic instance from a numerical parameter already in scope, instead of naming it? By Theorem 9.33 an empty scope still needs (Param) or a definable constant.

3. What is the least complete cost of eliminating the constant? Theorem 9.46 fixes the size of the zero term, but not the cost of the parser, the scoping discipline, the expansion of the abbreviations, the rules and the interpretation.

4. Can Theorem 9.44, in particular the equivalence with T_J and the definition of zero by Zero, be formalized in Lean or Rocq? The register theory T_J itself is not among the formalized syntaxes, so that dependency has to be formalized as well.

5. Is there one checker, for any presentation, below every coordinate of the currently observed compact minima, with its input contract and the theory it recognizes stated?

Question 9.156 (Zero-equation induction)**.** Which principle beyond induction on zero equations excludes the two-point fibers of Theorem 9.145? Reflection of actual equality suffices, at a cost that has to be counted. Does a narrowly stated uniqueness or extensionality principle for the identity iterator already give full induction on equations, and what does it cost in a complete checker? Does T_0 interpret arithmetic on a quotient, or admit a bi-interpretation with iPRA on a changed domain? The countermodel decides neither.

Question 9.157 (Successor rigidity)**.** The package on raw clause calculi with ordinary substitution states a restricted successor-rigidity principle that yields induction on zero equations and leaves a faithful bootstrap of equality and predecessor unresolved. Does that narrower principle suffice? `counter_evaluation` records the question and does not use the unproved conclusion.

Question 9.158 (The self-seeded language)**.** Can Theorem 9.140 be strengthened to a normal form or an effective classification of the functions defined by self-seeded terms? The proof gives the existence of a bound and no algorithm that extracts an optimal bound from a term. Is there an obstruction for interpretations by graphs or on changed domains?

Question 9.159 (Root conversion)**.** 1. Is root-only conversion complete on the iterator terms of Section 1.1, whose bodies do not see the counter: does BH_3 of Theorem 4.116 with HC_3 restricted to depth 0, or to depth 1, derive the sequents of N? The package `root_conversion` leaves this open; the proof of Theorem 9.119 uses the old counter of the body, and the embedding (9.31) of the iterator does not transfer it. It is a special case of the question in the computation chapter whether a bound smaller than three suffices.

2. Is $\mathcal{K}_{\mathrm{Root}}$ complete when HC_0 performs contractions only? The proofs of Theorems 9.118 and 9.119 expand the left endpoint, and the package states that the reverse direction must not be omitted without a new proof.

3. Can Theorems 9.123 and 9.126 be formalized on the existing development of N? No Lean or Rocq proof is claimed by either package, and neither rebuilt the inherited Lean development.

4. `root_conversion_raw` records two routes to equation induction that it examined and did not establish. Whether successor uniqueness alone recovers induction for all literal equations is Theorem 2.168. For the constancy rule (6.29) without the equality split, over scalar computation, numerical cases with the case hypothesis and the selector sel: if $\Gamma \vdash q(0) = 0$ and $\Gamma,\ q(x) = 0 \vdash q(Sx) = 0$, put $h(u) := \mathrm{sel}(q(u), Su, u)$ and $r(n) := I(n, 0; u.\, h(u))$. Cases on $q(u)$ give $q(h(u)) = q(u)$, by the step when $q(u) = 0$ and because $h(u) = u$ otherwise; hence $q(r(Sn)) = q(r(n))$, the constancy rule gives $q(r(n)) = q(r(0)) = 0$, and then $r(Sn) = Sr(n)$. So r is a successor-preserving map from 0 into the set $\{x : q(x) = 0\}$. Does a rule of this kind prove $r(n) = n$, or otherwise recover induction for literal equations, without successor rigidity or a reflection of equality assumed beforehand? With the primitive equality split and the comparison equation the answer is yes (Theorems 6.112 and 6.113); without the comparison equation the corresponding theory is not bi-interpretable with iPRA (Theorem 6.122), but its countermodels refute coverage, so they do not decide the question in the form asked here.

5. Does the theory BT of Section 9.11, or a native calculus on binary trees, yield a smaller complete checker once the four bound variables of the paramorphism, the binary induction step and the logical rules are charged? No implementation exists.

Question 9.160 (Tests on literal syntax)**.** The witness-directed introduction is concise but not stable under substitution: a substituted witness can become the literal 0. Is there a substitution-stable presentation with a comparable complete checker? In particular, guarded atomic matrices make the introduction depend on the root of the formula only and guarantee a reflexive zero row for the opening (Open); is the resulting complete grammar with its checker simpler than the unrestricted $\mathcal{K}_{\mathrm{Open}}$? Is there a uniform counter rule on raw syntax, with the zero computation stated explicitly instead of placed in the canonical substitution, whose checker is of comparable size?

Question 9.161 (Compact encodings)**.** Can an encoding of canonical seeded terms absorb the erasure of zero-counter bodies into a traversal that is needed anyway and improve the compact source minima, while validating the complete public language? Does the grammar

with disjoint arities, without a successor constructor and without a closure constructor, yield a smaller fully validated compact program once normalization and every case of malformed input are counted? Can the combination of the counter-aware arithmetic with the three families be ported to the compact certificate architecture below the existing profiles? Can a uniform representation remove the partial opening of binders, given that the recursor and the paired binder bind two variables and a clause one, while reducing the whole checker? Can the two instantiations of the counter body in (CE) be given a simpler complete scope discipline? Can a pointfree representation of proofs reduce the substitution machinery without introducing pairing structure, when the typing and conversion costs of external tuple contexts are counted?

Question 9.162 (Proof size)**.** What are the optimal translations of proof graphs between $\mathcal{K}_6$ and $\mathcal{K}_5$, and how much sharing avoids repeated neutral premises? Can a comparison of proof complexity between counter evaluation and the evaluation rule on the state of an iterator quantify certificate lengths and label sizes in addition to the inference nodes of Theorem 9.87? Should checkers with an explicit budget be compared only with controls that carry the same budget?

Question 9.163 (Measures and lower bounds)**.** The coordinates used in Section 9.14 and the counts of constructors, rules and proof expansion disagree with one another. Is there a robust measure that compares different proof interfaces? Can a lower bound be proved for a restricted class of checkers, with the term grammar, the class of decidable operations on syntax, the permitted runtime primitives and the certificate interface fixed in advance? A count of named rules alone changes too easily, by merging cases or by moving work into the syntax.

Question 9.164 (A new arithmetic reduction)**.** Eliminating the initializer, changing the domain or replacing the recursor may still give a simpler basis. A candidate has to respect the self-seeded constructions and the obstruction of Theorem 9.140, and it has to supply both interpretation composites, not only expressiveness on the standard model.

Not achieved. The packages state the following limits.

- No program of Section 9.14 for a calculus of this chapter is below the compact codecs of the kernel (516 and 508 bytes), and no minimality of an axiomatization or of a checker is claimed. An observed minimum among programs is an upper bound on the optimum.
- The register theory does not improve the size of a complete checker: its graph axiom is richer, its body has two bound variables, and its reference checker has 13633 bytes. The ancillary profile lowers the node count by one and is larger in bytes.
- The refined compact byte profile does not improve the token count of its control, and the token frontier of 258 reported by `canonical_recursors` is not beaten.
- The checker for $\mathcal{K}_5$ has fewer rules and tokens but more bytes than the six-rule checker on zero-normal iterator terms of `zero_normal`; its positive claim in bytes is the matched comparison.
- The checker for $\mathcal{K}_{\mathrm{CE}}$ is smaller in all three coordinates than the scalar control with one evaluation case, but the raw scalar clause checker with the root case deleted is five bytes smaller.
- For the checker of `uniform_binding` the compressed size is larger than that of the exact incoming scalar checker.

- Two directions were examined in `uniform_binding` and discarded. A formula of the clause syntax is a chain of paired binders ending in an equation and can be flattened to a triple of a depth and two terms; the change is exact, but the implementation grew, because checking of ranks and opening of endpoints outweighed the saved traversal. A rule of parameter substitution used only in the reverse direction is sound, but the intended completeness argument by a fresh alias fails: once a goal no longer mentions the alias, the rule can require that same goal as its premise.

- The translator of test fixtures of `uniform_recursion` declines an inherited fixture whose witness becomes the literal 0 under normalization; a direct proof of the same formula is supplied, and no universal executable proof compiler is claimed.

- Removing the constant reduces the numerical syntax and not the checker: the reference checker for $\mathsf{T}_J^{\varnothing}$ (3072 tokens, 3349 nodes, 13723 bytes) is larger in every coordinate than its control. No minimality of the axiomatization is claimed, Theorem 9.46 is not a bound on checker size, and the measurement of the control differs from the one stated by `register_graph` (Section 9.14).

- The strict checker for $\mathcal{K}_{\mathrm{G}}$ (1041 tokens, 1181 nodes, 3467 bytes) improves only its matched control; it is larger in every coordinate than each of the three compact programs named in the question on guarded registers, and `guarded_register` does not reach its stated target of a program below their envelope. Its survey of the existing programs measured the 442 distinct Python sources of at most 1500 source bytes in the ninety package trees; the threshold limits the audit, and it is not a lower bound for larger sources that normalize to smaller programs. No minimality of the axiomatization or of the checker is claimed, and a smaller number of constructors or rule names is not by itself a smaller checker.

- The checker for $\mathcal{K}_{\mathrm{Root}}$ (1108 tokens, 1301 nodes, 3875 bytes) improves only its eight-rule control, by 221 bytes, through the inherited single introduction. `root_conversion` measured the compact profiles of the incoming shared-leaf package, a token profile (242, 268, 565), a node profile (246, 262, 556) and a byte profile (272, 306, 469), whose coordinatewise minima 242, 262 and 469 are attained by different programs with a compact input contract in which the public goal already has the canonical guarded grammar; the integrated packages alone give 244, 265 and 477. The package does not reach its stated target of a checker below these minima, which are upper bounds on the optimum and not lower bounds. Theorem 9.119 restricts the computation that the calculus trusts; it is not a smaller axiomatization, and it changes the term language from the counter-blind iterator to the counter-aware recursor.

- The checker of `root_conversion_raw` for $\mathcal{K}_{\mathrm{Root}}$ (1029 tokens, 1096 nodes, 3003 bytes) improves only its matched control. The package compares it with the incoming-inclusive compact envelope 244, 265, 477, attained by different programs (a token program of `boolean_constancy`, 244/272/604; a node program of `fused_iteration`, 249/265/605; byte programs of `head_conversion`, 272/306/477, and of `checked_reductions`, 273/305/477), and with the whole-input envelope 249, 271, 477, and reaches neither. Its compact program (274/306/476) is the composition of Theorem 11.146(b) and implements the inherited guarded scalar presentation, not $\mathcal{K}_{\mathrm{Root}}$. No minimality of the axiomatization or of a checker is claimed.

- The bi-interpretation of Theorem 9.134 has no checker and no size advantage, and the underlying arithmetic of trees is standard; it is recorded to show what comparison maps that are not the identity require.

- Concrete limits of the interpreter on recursion depth and memory can reject a valid large certificate for every checker of the chapter; completeness is a statement about the calculi and about the checkers without such limits.

Chapter 10

Obstructions

The chapter collects the negative results: what a smaller numerical basis or a smaller rule inventory cannot do, together with the model or invariant that proves it. Each result names the exact class of theories or presentations that it excludes and states what it does not exclude. The results are grouped by method. Section 10.2 uses coordinatewise powers of the standard model and exact truth transfer for conjunctive formulas. Section 10.3 uses the automorphisms of such powers. Sections 10.4 and 10.5 use growth invariants of term languages and a dimension argument for term interpretations. Section 10.6 counts definable values and reaches arbitrary relational bi-interpretations through witness extraction. Section 10.7 classifies zero sets. Section 10.8 collects small counter-interpretations of individual rule deletions. Section 10.9 builds a nonstandard model of all iterator computations, by a tower of ultrapowers, in which the progress rule fails, and a refinement in which preservation of fixed points and seed shift hold as well; it ends with two covers of a nonstandard model of arithmetic which show that neither iterator fusion nor the successor numbering law suffices alone, even with induction against fixed numerals. The packages that prove the obstructions also contain positive statements about the presentations that the obstructions are measured against; these are in Sections 10.10 to 10.13.

None of the results is a lower bound on the size of a proof checker, and none excludes equiconsistency. Table 10.1 lists the main statements.

Table 10.1: The obstructions of the chapter.

Result	Excluded	Not excluded
Theorem 10.16	case introduction with target z in place of Sz, with all other native rules	other readings of the weaker calculus
Theorem 10.17	deletion of Boolean cases from the progressive calculus	a weaker case principle
Theorem 10.22	ordinary bi-interpretation with iPRA of any theory true in an eventually constant power: equation induction, conjunctive induction, all true conjunctive sequents, many sorts	one-way interpretations, equiconsistency, disjunctive axioms
Theorem 10.28	a term compiler into the zero-seeded recursor	relational interpretations

Result	Excluded	Not excluded
Theorem 10.33, with Theorems 10.36 and 10.39	terms for addition, selection and pairing when seed and update are finite adaptive query programs on one body	an independent input, retained registers, relational interpretations
Theorem 10.45	finite-dimensional interpretations with a term-defined addition, any quotient equality	graph-defined operations
Theorem 10.52	terms for an equality test, minimum and truncated subtraction in the tied-seed language	formula-defined relations
Theorem 10.58	every finite-dimensional bi-interpretation of iPRA with an existential-free theory true in the standard tied-seed algebra	one-way interpretations, theories with positive existence axioms
Theorem 10.62	terms for predecessor, zero test, addition and initialized iteration from positive iteration	graph definitions
Theorems 10.65, 10.67 to 10.69 and 10.117	five individual rule deletions or replacements	redesigned calculi with their own interpretation proofs
Theorems 10.79 and 10.80	deletion of QI by progress from the core: the other seven rules do not derive $Q_y(I(y,0;\,u.\,u) = 0)$, nor does classical extensional computation with coverage derive its induction instance	fusions of QI with another rule, weaker induction interfaces, equivalence of uniqueness of iteration with induction
Theorems 10.87 and 10.88	replacement of equation induction by preservation of fixed points and seed shift, with the canonical successor iterator and cancellative addition; QI over the seven rules with FP and SHIFT	general fusion (Theorem 10.90), a derivable step over the identity scheme alone, the uniqueness rule for the successor update
Theorems 10.91, 10.93 and 10.95	recovery of equation induction from numeral induction with the identity law, all numbering laws $I(n,\overline{k};\,u.\,Su) = S^k n$ and extensionality, or with FUSION without $I(n,0;\,u.\,Su) = n$; QI over the seven rules with the identity scheme	FUSION together with $I(n,0;\,u.\,Su) = n$, which gives induction
Theorem 10.94	bi-interpretation with iPRA of the fusion theory with finite parameters, and of the numbering theory with standard parameters	nonstandard parameters for the numbering theory, one-way interpretations, equiconsistency

10.1 Conventions and the presentations under test

This section fixes the vocabulary shared by `adaptive_seed`, `research`, `local_scopes`, `checker_and_barriers`, `compact_kernel`, `uniform_predecessor`, `scope_kernel`, `constructed_computation` and `counter_congruen`

10.1.1 Three kinds of exclusion

The statements of the chapter have three strengths.

Term definability. A reduced term language is given a standard interpretation: every fixed term denotes a primitive recursive function on $\mathbb{N}$, by structural recursion on the term followed by numerical primitive recursion. No uniform evaluator of all descriptions is used. A term-definability obstruction is a statement about these functions. It excludes a direct compiler of the primitive recursive signature into terms with their usual meanings. It does not exclude a definition of the missing function by a formula.

Derivability in a specified rule system. A rule system is shown to lose a theorem of another one by a reading under which every rule of the first is sound and the theorem is false. A classical structure is a one-node Kripke model, so soundness in a classical structure is a sound test of intuitionistic derivability. Such a result concerns the displayed rules with the identity translation of formulas. It does not exclude a different translation.

Ordinary bi-interpretability. This is the strongest kind.

Definition 10.1 (Ordinary interpretations)**.** An *ordinary interpretation* $\mathcal{I}$ of a theory U in a theory T has a dimension $d \geq 0$ and consists of formulas of T: a domain formula $\delta(\vec{x})$ in d variables for each sort of U, an equality formula $\varepsilon(\vec{x}, \vec{y})$, and a graph formula for each function and relation symbol of U. The formulas may contain finitely many parameters, subject to a parameter formula. T proves that ε is an equivalence on δ respected by the graphs, that the function graphs are total and single-valued modulo ε, and the translation of every axiom of U. The domain, the graphs and the quotient equality are arbitrary formulas.

A *bi-interpretation* of T and U is a pair of ordinary interpretations $\mathcal{I}$ of U in T and $\mathcal{J}$ of T in U such that T proves that a formula η defines an isomorphism of the identity interpretation of T onto $\mathcal{J}$ interpreted through $\mathcal{I}$, and U proves the corresponding statement for the other composite. The formula η of T is the *source-side return isomorphism.*

The bridge of Theorem 1.119 is a bi-interpretation in this sense, of dimension one, with identity domains, actual equality and return isomorphisms with graph $x = y$. The obstructions of Sections 10.3 and 10.6 exclude all bi-interpretations in the sense of Theorem 10.1, not only those of this special form. They use only the source-side return isomorphism.

10.1.2 The paired-introduction form of the native core

The packages of this chapter state the native calculus with a progress introduction that concludes a paired formula:

$$\frac{\Gamma,\ E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)}\ \textsc{di-progress}, \qquad E \text{ a literal equation},\ z \text{ fresh for } \Gamma, t.$$

The core N of Section 1.3 has QI by progress instead. The *paired-introduction form* N_{p} is N with QI replaced by DI-PROGRESS. Several packages also supply a computation instance as a discharged local assumption (rule COMP, called SUPPLY in some packages): from $\Gamma, H \vdash A$ with H an instance of a computation equation infer $\Gamma \vdash A$.

Lemma 10.2. N *and* N_{p} *derive the same sequents.* COMP *is admissible in both, and the computation equations are derivable from* COMP *and* HYP*.*

Proof. Let $\Gamma, E(t,z) \vdash E(t,Sz)$ with z fresh. In N, QI for the matrix $y \mapsto E(t,y)$ gives $\Gamma \vdash Q_y E(t,y)$. Weaken by $E(t,0)$. HYP gives $E(t,0)$, and QE at the target Sz' gives $\Gamma, E(t,0) \vdash E(t,Sz')$. DI by cases with witness t gives $\Gamma \vdash D_{x,y}E(x,y)$. Conversely, let $\Gamma, E(z) \vdash E(Sz)$. In N_p, DI-PROGRESS with a witness coordinate u that does not occur in E gives $\Gamma \vdash D_{u,y}E(y)$. DE supplies the clause $Q_y E(y)$, which does not contain the fresh witness, and HYP concludes $Q_y E(y)$. COMP is a cut with a computation equation (Section 1.4); conversely COMP followed by HYP derives the equation. □

In the paired-introduction form the formula $Q_y B$ is interderivable with $D_{u,y}B$ for an unused u: DE followed by HYP gives one direction, and DI by cases at the witness 0 with QE gives the other. The rule-level statements of Section 10.8 refer to N_p where the difference matters.

10.1.3 Reduced term languages

Definition 10.3 (Seedless recursors)**.** Each of the following languages has variables, 0 and one recursion constructor. Only the update body binds the displayed bound variables; its other variables are passive parameters, fixed through the recursion. Bodies are fixed finite earlier terms.

(a) *Zero-seeded recursion* has, besides 0 and S, the constructor $\mathsf{L}(n; u, v.h)$ with

$$\mathsf{L}(0; u, v.h) = 0, \qquad \mathsf{L}(Sn; u, v.h) = h[n/u][\mathsf{L}(n; u, v.h)/v].$$

(b) *Tied-seed recursion* has the constructor $\mathsf{B}(n; u, v.h)$ and the abbreviation $Sa := \mathsf{B}(0; u, v.a)$ for u, v not in a, with

$$\mathsf{B}(0; u, v.h) = S(h[0/u][0/v]), \tag{10.1}$$
$$\mathsf{B}(Sn; u, v.h) = h[n/u][\mathsf{B}(n; u, v.h)/v]. \tag{10.2}$$

The two substitutions are simultaneous. The initial state is tied to the update; there is no independent initializer. The standard interpretation is

$$\mathsf{B}_h(0, \vec{p}) = 1 + h(0, 0, \vec{p}), \qquad \mathsf{B}_h(m+1, \vec{p}) = h(m, \mathsf{B}_h(m, \vec{p}), \vec{p}),$$

so that S denotes the successor. The *unary-update* variant $\mathsf{F}(n; v.h)$ is the case in which u does not occur in h.

(c) *Positive iteration* has, besides 0 and S, the constructor $\mathsf{R}^+(n, a; v.h)$, standing for $h^{n+1}(a)$:

$$\mathsf{R}^+(0, a; v.h) = h[a/v], \qquad \mathsf{R}^+(Sn, a; v.h) = h[\mathsf{R}^+(n, a; v.h)/v].$$

For reference, the *counter-aware initialized recursor* is $\mathsf{R}(n, a; u, v.h)$ with $\mathsf{R}(0, a; u, v.h) = a$ and $\mathsf{R}(Sn, a; u, v.h) = h[n/u][\mathsf{R}(n, a; u, v.h)/v]$. It is ordinary primitive recursion and compiles every primitive recursive description directly.

The tied-seed language is the most capable of the three. The next two lemmas record what it does define; they are the positive boundary of the obstructions of Sections 10.4 and 10.6.

Lemma 10.4 (Terms of the tied-seed language)**.** *Put*

$$W(n) := \mathsf{B}(n; u, v.u), \quad Z(n) := \mathsf{B}(n; u, v.0), \quad H(n, a) := \mathsf{B}(n; u, v.a), \quad P(n) := W(H(n, n)).$$

The computation equations (10.1), (10.2) *and equality reasoning prove*

$$W(0) = 1,\ W(Sn) = n, \quad Z(0) = 1,\ Z(Sn) = 0, \quad H(0,a) = Sa,\ H(Sn,a) = a, \quad P(0) = 0,\ P(Sn) = n.$$

With induction for literal equations, the two gates

$$G(n,a) := P\big(P(\mathsf{B}(a;u,v.H(Z(n),v)))\big), \qquad G'(n,a) := P\big(P(\mathsf{B}(a;u,v.H(n,v)))\big)$$

satisfy $G(0,a) = 0$, $G(Sq,a) = a$ *and* $G'(0,a) = a$, $G'(Sq,a) = 0$.

Proof. The equations for W, Z and H are instances of (10.1) and (10.2). Then $P(0) = W(H(0,0)) = W(S0) = 0$ and $P(Sn) = W(H(Sn,Sn)) = W(Sn) = n$. For G at the control 0, the update is $H(Z(0),v) = H(1,v) = v$. Induction on the literal equation $\mathsf{B}(a;u,v.H(Z(0),v)) = 1$ has the base $S(H(1,0)) = S0$ and a successor step that repeats the previous state. Two predecessors give $G(0,a) = 0$. At the control Sq the update is $H(0,v) = Sv$. Induction on $\mathsf{B}(a;u,v.H(Z(Sq),v)) = SSa$ has the base $S(H(0,0)) = SS0$ and a successor step that applies S. Two predecessors give $G(Sq,a) = a$. The argument for G' exchanges the two cases. The inductions keep the actual update bodies and the control parameter fixed; no extensionality principle for recursion descriptions and no induction on a compound formula is used. □

Lemma 10.5 (Zero-start counter recursion is a macro)**.** *For a body h with bound variables u, v and fresh r, s, put*

$$L(n;u,v.h) := \mathsf{B}\big(Sn;\ r,s.\,G(r,\ h[Pr/u][s/v])\big).$$

Then $L(0;u,v.h) = 0$ and $L(Sn;u,v.h) = h[n/u][L(n;u,v.h)/v]$ follow from Theorem 10.4 without further induction.

Proof. Let $K(r,s) = G(r, h[Pr/u][s/v])$. The gate equations and $P(Sr) = r$ give $K(0,s) = 0$ and $K(Sr,s) = h[r/u][s/v]$. Hence $\mathsf{B}_K(0) = S(K(0,0)) = 1$ and $\mathsf{B}_K(1) = K(0,\mathsf{B}_K(0)) = 0$, which is $L(0) = 0$. At a successor, $L(Sn) = \mathsf{B}_K(SSn) = K(Sn,\mathsf{B}_K(Sn)) = h[n/u][L(n)/v]$. □

Remark 10.6. Theorem 10.4 is in `counter_congruence` (with the gate G'), `research` (with $P(n) = W(H(Z(n),W(n)))$, which computes the same function) and `local_scopes` (with the gate G and the internal proof). Theorem 10.5 is in `local_scopes`. Predecessor, a zero test, a one-sided gate and zero-start recursion are therefore not what the tied-seed language lacks. The obstruction is to combining two unbounded inputs.

10.1.4 The progressive calculus

Definition 10.7 (Progressive binder calculus)**.** Numerical terms are variables, 0, St and the counter-aware recursor $\mathsf{R}(n,a;u,v.h)$ with its two computation equations. Object formulas are

$$A ::= (s = t) \mid \mathsf{H}_{x,y}A.$$

Internal clauses are $\mathsf{P}_y A(y)$ with A an object formula. Contexts are finite reusable lists of object formulas and clauses. Write $\Gamma \triangleright M$ when M is a member of Γ or a proved object formula; clauses are not goals. The ordinary readings are

$$(\mathsf{P}_y B)^\circ = \forall y\,(B^\circ(y) \to B^\circ(Sy)), \qquad (\mathsf{H}_{x,y}B)^\circ = \exists x\,\forall y\,(B^\circ(x,y) \to B^\circ(x,Sy)).$$

Besides the computation equations, the rules are

$$\frac{\Gamma \rhd s = t \quad \Gamma \vdash A(s)}{\Gamma \vdash A(t)}\ \text{TR} \qquad \frac{\Gamma,\ B(t,z) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{H}_{x,y}B(x,y)}\ \text{HI}$$

$$\frac{\Gamma \rhd \mathsf{H}_{x,y}B(x,y) \quad \Gamma,\ \mathsf{P}_y B(a,y) \vdash A}{\Gamma \vdash A}\ \text{HE} \qquad \frac{\Gamma \rhd \mathsf{P}_y B(y) \quad \Gamma \vdash B(t)}{\Gamma \vdash B(St)}\ \text{STEP}$$

$$\frac{\Gamma \rhd \mathsf{P}_y E(y) \quad \Gamma \vdash E(0)}{\Gamma \vdash E(t)}\ \text{ITER} \qquad \frac{\Gamma,\ \mathrm{sg}(t) = 0 \vdash A \quad \Gamma,\ \mathrm{sg}(t) = 1 \vdash A}{\Gamma \vdash A}\ \text{BOOLCASES}$$

In ITER the matrix E is a literal equation. In HI the variable z is fresh for Γ, t and the passive parameters; in HE the variable a is fresh for Γ, A and the passive parameters. Here $1 = S0$ and $\mathrm{sg}(n) = \mathsf{R}(n, 0; u, v.1)$, so $\mathrm{sg}(0) = 0$ and $\mathrm{sg}(Sn) = 1$. There is no primitive hypothesis rule and no other induction rule. The *case-free progressive calculus* omits BOOLCASES.

HI is an unrestricted logical introduction; the only induction is ITER, at literal equations, in an elimination. The calculus with BOOLCASES is a complete presentation of iPRA (Theorem 10.107); Theorem 10.17 shows that BOOLCASES cannot be deleted.

10.2 Coordinatewise powers and truth transfer

This section draws on `research`, `compact_kernel`, `scope_kernel` and `checker_and_barriers`. One family of structures serves four results: equation induction does not imply numerical cases (Theorem 10.16), Boolean cases are not admissible in the progressive calculus (Theorem 10.17), and, in Section 10.3, positive axioms and conjunctive induction do not give a theory bi-interpretable with iPRA.

10.2.1 The powers

Definition 10.8 (Coordinatewise powers)**.** Let $\mathcal{N}$ be a finitary expansion of the standard structure $(\mathbb{N}, 0, S)$ by total functions and relations. The signature may be infinite; in particular it may contain a symbol for every primitive recursive description. $\mathcal{N}$ may be many-sorted, with nonempty sorts and with $(\mathbb{N}, 0, S)$ as a distinguished numerical sort. For a nonempty index set J, the structure $\mathcal{N}^{(J)}$ has, in each sort, the families $a = (a(i))_{i \in J}$ of elements of that sort that are constant outside a finite subset of J. Functions are interpreted coordinatewise. A relation holds of families exactly when it holds at every coordinate. Equality is actual equality of families, and constants are constant families.

For $J = \{1, 2\}$ the structure is the full product $\mathcal{A} = \mathcal{N} \times \mathcal{N}$. For $J = \mathbb{N}$ it is the structure $\mathcal{E}$ of eventually constant sequences. For $i \in J$, the family e_i of the numerical sort has the value 1 at i and 0 elsewhere.

Finitely many families are simultaneously constant outside the union of their exceptional sets, so coordinatewise functions preserve the domain. Each element of $\mathcal{E}$ has a finite description, so $\mathcal{E}$ is countable when the sorts of $\mathcal{N}$ are.

Definition 10.9 (Conjunctive formulas)**.** The class $\mathcal{C}$ of *conjunctive formulas* is generated from atomic formulas and $\top$ by $\wedge$, $\exists$ and $\forall$, with unrestricted quantifier alternation. A conjunctive formula without $\forall$ is *primitive positive*. No disjunction, implication or negation occurs inside a conjunctive formula.

Theorem 10.10 (Conjunctive transfer)**.** *For every $\varphi(\vec{x}) \in \mathcal{C}$, every nonempty J and every tuple $\vec{a}$ of $\mathcal{N}^{(J)}$,*

$$\mathcal{N}^{(J)} \models \varphi(\vec{a}) \quad \Longleftrightarrow \quad \mathcal{N} \models \varphi(\vec{a}(i)) \ \text{ for every } i \in J.$$

Proof. Induction on the formula. Terms evaluate coordinatewise, so atomic formulas transfer by the definition of the relations and of equality. Conjunction is immediate.

Existential quantifier. If $\mathcal{N}^{(J)} \models \theta(b, \vec{a})$, the induction hypothesis gives $\mathcal{N} \models \theta(b(i), \vec{a}(i))$ at every i. Conversely, suppose $\mathcal{N} \models \exists y\, \theta(y, \vec{a}(i))$ at every i. There is a finite set $F \subseteq J$ such that $\vec{a}(i)$ is one fixed tuple $\vec{a}_\infty$ for all $i \notin F$. Choose a witness b_i for each of the finitely many $i \in F$ and one witness b_∞ for $\vec{a}_\infty$ if $J \neq F$. The family with the values b_i on F and b_∞ elsewhere belongs to $\mathcal{N}^{(J)}$, and the induction hypothesis gives $\mathcal{N}^{(J)} \models \theta(b, \vec{a})$. Only finitely many witnesses are chosen.

Universal quantifier. Suppose $\mathcal{N}^{(J)} \models \forall y\, \theta(y, \vec{a})$. Fix i and an element b of the relevant sort of $\mathcal{N}$. The constant family $\overline{b}$ is an element of $\mathcal{N}^{(J)}$; instantiate at $\overline{b}$ and apply the induction hypothesis at i to get $\mathcal{N} \models \theta(b, \vec{a}(i))$. Conversely, if every coordinate satisfies $\forall y\, \theta(y, \vec{a}(i))$, then for every family b every coordinate satisfies $\theta(b(i), \vec{a}(i))$, and the induction hypothesis gives $\mathcal{N}^{(J)} \models \theta(b, \vec{a})$. □

Corollary 10.11 (Regular axioms and conjunctive induction)**.** *Let J be nonempty.*

(i) $\mathcal{N}^{(J)}$ satisfies every sentence $\forall \vec{x}\, (\varphi \to \psi)$ and every sentence $\forall \vec{x}\, (\varphi \to \bot)$ with $\varphi, \psi \in \mathcal{C}$ that is true in $\mathcal{N}$.

(ii) For every $A(n, \vec{p}) \in \mathcal{C}$, with n of the numerical sort, $\mathcal{N}^{(J)}$ satisfies

$$\forall \vec{p} \Big[\big(A(0, \vec{p}) \wedge \forall n\, (A(n, \vec{p}) \to A(Sn, \vec{p})) \big) \to \forall n\, A(n, \vec{p}) \Big].$$

Consequently the contextual induction rule for matrices in $\mathcal{C}$ is sound in $\mathcal{N}^{(J)}$ under arbitrary retained assumptions.

Proof. (i) If $\varphi(\vec{a})$ holds in $\mathcal{N}^{(J)}$, then $\varphi(\vec{a}(i))$ holds in $\mathcal{N}$ at every i by Theorem 10.10; truth of the sentence in $\mathcal{N}$ gives $\psi(\vec{a}(i))$ at every i, and the converse direction of the theorem gives $\psi(\vec{a})$. For a negative constraint, a true antecedent in $\mathcal{N}^{(J)}$ gives a true antecedent in $\mathcal{N}$ at any one coordinate, which contradicts the constraint.

(ii) Fix $\vec{p}$ and suppose the base and the progress hypothesis hold in $\mathcal{N}^{(J)}$. Fix $i \in J$ and put $U_i = \{k : \mathcal{N} \models A(k, \vec{p}(i))\}$. Transfer of the base gives $0 \in U_j$ for every j. Let $k \in U_i$ and let a be the family with the value k at i and 0 elsewhere. At i the formula $A(a(i), \vec{p}(i))$ holds by the choice of k, and at every other coordinate it is the transferred base; hence $\mathcal{N}^{(J)} \models A(a, \vec{p})$. The progress hypothesis gives $A(Sa, \vec{p})$, and transfer at i gives $k+1 \in U_i$. Ordinary induction in $\mathbb{N}$ gives $U_i = \mathbb{N}$ for every i, and the converse direction of the theorem gives $A(a, \vec{p})$ for every a.

For the contextual rule, fix an assignment that makes the retained assumptions true. The induction variable is fresh, so changing its value does not change their truth, and the argument applies under that assignment. The retained assumptions may be arbitrary formulas. Witnesses for A are not assumed to be terms. □

In the primitive recursive signature the corollary covers every defining equation, the injectivity of S, the separation $0 = Sx \to \bot$, every universal Horn sentence $\forall \vec{x}\, (E_1 \wedge \cdots \wedge E_r \to F)$ with equations E_k and an equation or $\bot$ as F that is true in $\mathbb{N}$, and induction for literal equations with parameters.

Definition 10.12. T_{alg} is intuitionistic first-order logic with equality in the signature with a symbol for every primitive recursive description, with all defining equations, the separation axioms and induction for every literal equation. Coverage is absent. For a structure $\mathcal{N}$ as in Theorem 10.8, $T_{\mathcal{C}}(\mathcal{N})$ is intuitionistic logic with equality together with all sentences of Theorem 10.11(i) that are true in $\mathcal{N}$ and all induction sentences of Theorem 10.11(ii); $T_{\mathrm{pp}}(\mathcal{N})$ is its restriction to primitive positive φ, ψ, A.

By Theorem 10.11, every $\mathcal{N}^{(J)}$ is a model of $T_{\mathcal{C}}(\mathcal{N})$, hence of $T_{\mathrm{pp}}(\mathcal{N})$, and, for the primitive recursive signature, of T_{alg} together with all universal Horn sentences true in $\mathbb{N}$.

Proposition 10.13 (What fails in a power)**.** *Let J have at least two elements and let the signature contain* sg*. In $\mathcal{N}^{(J)}$:*

(i) e_i is neither 0 nor a successor, so coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$ fails;

(ii) the quantifier-free induction instance for $\varphi(n) := (\mathrm{sg}(n) = 0 \vee \mathrm{sg}(n) = 1)$ fails: $\varphi(0)$ and $\varphi(Sa)$ hold for every a, and $\varphi(e_i)$ fails;

(iii) no formula with finitely many parameters defines a linear order of the numerical sort when J is infinite (Theorem 10.19).

Proof. Every successor family is positive at every coordinate, and e_i is 0 at some coordinate and 1 at i. The family $\mathrm{sg}(Sa)$ is the constant 1, while $\mathrm{sg}(e_i) = e_i$ is neither the constant 0 nor the constant 1. For (iii), an order defined over a finite tuple of parameters is preserved by the involution of Theorem 10.19 that fixes the tuple, and an order-preserving involution of a linear order is the identity (Theorem 10.20). □

The same holds for the relation $x \leq y$ of the primitive recursive signature: for $i \neq j$ neither $e_i \leq e_j$ nor $e_j \leq e_i$ holds, since each fails at one coordinate.

Remark 10.14. The missing feature is a disjunctive range assertion, not numerical computation. A primitive recursive characteristic of equality still reflects equality in a power: its value is the zero family exactly when its arguments agree at every coordinate. What fails is the assertion that the value is globally 0 or globally 1. In the same way, equation induction proves the algebraic identities $t(\mathrm{sel}(n, a, b)) = \mathrm{sel}(n, t(a), t(b))$ for each fixed term context t, and $\mathrm{sel}(b, 0, 1) = b$ for the Boolean value $b = \mathrm{sel}(n, 0, 1)$; these hold for the mixed values $(0, 1)$ and $(1, 0)$ of $\mathcal{A}$ and do not entail $b = 0 \vee b = 1$. Coordinatewise predecessor satisfies $P(0) = 0$ and $P(Sx) = x$ in every power, so adding valid predecessor equations does not restore coverage.

Remark 10.15. The product $\mathcal{A}$ with literal-equation induction is in `scope_kernel` and `checker_and_barriers`; `research` attributes the product model of non-coverage to an earlier draft. The structure $\mathcal{E}$ with induction for equations and for finite conjunctions of atoms, universal Horn sentences and arbitrary finitary expansions is in `research`. Primitive positive transfer, the universal quantifier and the many-sorted form are in `compact_kernel`. The statement for an arbitrary index set is the same proof.

10.2.2 Equation induction does not replace numerical cases

Consider the *direct-target* replacement of case introduction:

$$\frac{\Gamma,\ B(t, 0) \vdash B(t, z)}{\Gamma \vdash D_{x,y} B(x, y)} \qquad z \text{ fresh for } \Gamma, t \text{ and the parameters of } B.$$

It is sound for the ordinary reading of $D_{x,y}$ in every structure, with no coverage axiom. Let N^{dt} be N (or N_{p}) with DI by cases replaced by this rule.

Theorem 10.16 (`scope_kernel`)**.** *Every theorem of N^{dt} is valid in every power $\mathcal{N}^{(J)}$ of the standard model of the iterator language, under the ordinary reading. Write $1 = S0$ and define*

$$E(n, y, p) := \big(\mathrm{sel}(p,\, \mathrm{sel}(n, y, 1),\, \mathrm{sel}(n, 1, 0)) = 0\big), \qquad K(n, p) := Q_y E(n, y, p),$$
$$M(p) := Q_n K(n, p), \qquad F := Q_p M(p).$$

The closed formula F is derivable in N without the progress rule and is false in $\mathcal{A}$. Hence N^{dt} does not derive F.

Proof. Soundness. Interpret Q_yB and $D_{x,y}B$ by their ordinary readings. HYP, transport, QE and DE are sound in every structure with actual equality. The computation equations hold coordinatewise. The progress rule is sound by Theorem 10.11(ii). The direct-target rule is sound because its fresh variable already ranges over the whole domain.

Derivation of F in N. A closure $Q_yB(y)$ follows from a derivation of $B(Sz)$ with z fresh: DI by cases with an unused witness gives $D_{u,y}B(y)$, and DE with HYP gives $Q_yB(y)$. At $n = Sz$ the computation equations give $\mathrm{sel}(Sz, y, 1) = 1$ and $\mathrm{sel}(Sz, 1, 0) = 0$, so $E(Sz, y, p)$ is interderivable with $\mathrm{sel}(p, 1, 0) = 0$, independently of y. Hence $E(Sz, 0, p) \vdash E(Sz, Sw, p)$ by transport, and $K(Sz, p)$ is derivable. This is the successor instance of $M(p)$, so $M(p)$ is derivable for a free p; its instance $M(Sw)$ is the successor instance of F. Only case introduction, DE, QE, the computations and transport are used.

F is false in $\mathcal{A}$. Let $p_* = (0, 1)$. Coordinatewise computation gives

$$\mathcal{A} \models E(n, y, p_*) \quad \Longleftrightarrow \quad n_1 = 0,\ n_2 > 0,\ y_1 = 0.$$

Hence $K(0, p_*)$ holds, since its antecedent $E(0, 0, p_*)$ is false, and $K((0, 1), p_*)$ fails: its antecedent holds at $y = 0$ and its consequent fails at $y = (1, 0)$. So $M(p_*)$ is false. At $p = 0$ the formula $E(0, y, 0)$ is $y = 0$, so $K(0, 0)$, which reads $0 = 0 \to \forall y\,(y = 0)$, is false and $M(0)$ is true. Therefore F, which reads $M(0) \to \forall p\, M(p)$, is false. □

The theorem excludes this replacement with the identity translation of formulas. It does not show that every interpretation of the weaker calculus in iPRA fails, and it does not show that no restricted form of numerical cases suffices.

10.2.3 Boolean cases are not admissible in the progressive calculus

Theorem 10.17 (`checker_and_barriers`)**.** *Let C and Z be the terms $C(n, a, b) = \mathsf{R}(n, a; u, v.b)$ and $Z(n) = \mathsf{R}(n, 1; u, v.0)$, so that $C(0, a, b) = a$, $C(Sn, a, b) = b$, $Z(0) = 1$ and $Z(Sn) = 0$. For a numerical term t put*

$$F(t) := \mathsf{H}_{u,y}\, \mathsf{H}_{v,z}(C(t, Z(y), z) = 0),$$

with u, v unused. The case-free progressive calculus of Theorem 10.7 derives $F(0)$ and $F(1)$ and does not derive $F(\mathrm{sg}(t))$. Hence BOOLCASES, with an arbitrary object formula as conclusion, is not admissible in the case-free calculus, in the presence of all primitive recursive symbols, their defining equations and contextual induction for literal equations.

Proof. The endpoints. The derivations use the derived identity $\Gamma, A \vdash A$ of Theorem 10.101, which needs no cases. At $t = 0$ the inner matrix is interderivable with $Z(y) = 0$, independently of z. Its progress step is therefore an identity, and HI gives $\mathsf{H}_{v,z}(C(0, Z(y), z) = 0)$ for arbitrary y. The outer HI ignores its local assumption. At $t = 1$ the inner matrix is interderivable with $z = 0$, independently of y. For the outer HI, open the assumed inner formula at y by HE, introduce the inner formula at Sy by HI with the same witness, convert its local assumption to the matrix at y by the computation equations, apply STEP, and convert back.

The model. Let $\mathcal{A}$ be the product of the standard model of the primitive recursive signature. Under the readings of Theorem 10.7, TR, HI, HE and STEP are logical rules, valid in every structure with actual equality, and ITER is induction for a literal equation, valid by Theorem 10.11(ii). The computation equations hold. So every theorem of the case-free calculus is valid in $\mathcal{A}$.

Let $e = (0, 1)$, so $\mathrm{sg}(e) = e$. For $y = (y_1, y_2)$ and $z = (z_1, z_2)$ the inner equation $C(e, Z(y), z) = 0$ says $(Z(y_1), z_2) = (0, 0)$. If $y_1 = 0$ it is false for every z, so the inner progress statement is vacuously true. If $y_1 > 0$ it says $z_2 = 0$; it holds at $z = (0, 0)$ and fails at $Sz = (1, 1)$, so the inner statement is false. Thus the inner formula is true at $y = (0, 0)$ and

false at $Sy = (1, 1)$, which refutes the outer progress statement for every u. Hence $\mathcal{A} \not\models F(e)$, and $F(\mathrm{sg}(t))$ is not derivable.

Non-admissibility. From the assumption $\mathrm{sg}(t) = 0$, transport of $F(0)$ gives $F(\mathrm{sg}(t))$; from $\mathrm{sg}(t) = 1$, transport of $F(1)$ gives the same. An admissible BOOLCASES would derive $F(\mathrm{sg}(t))$. □

Proposition 10.18 (The binder depth is sharp for this separation)**.** *Let $G(t)$ be an equation or a formula $\mathsf{H}_{x,y}E(x, y, t)$ with E a literal equation. If $G(0)$ and $G(1)$ hold in the standard model, then $G((0, 1))$ holds in $\mathcal{A}$.*

Proof. For an equation this is coordinatewise evaluation. Otherwise let a_0 and a_1 be standard witnesses for $t = 0$ and $t = 1$. If $E((a_0, a_1), (y_1, y_2), (0, 1))$ holds in $\mathcal{A}$, then $E(a_0, y_1, 0)$ and $E(a_1, y_2, 1)$ hold in $\mathbb{N}$, so $E(a_0, y_1 + 1, 0)$ and $E(a_1, y_2 + 1, 1)$ hold, which is E at Sy in $\mathcal{A}$. □

The first failure arises when an implication tests a compound premise: one coordinate makes an inner implication vacuous while the other would fail it. Theorem 10.17 rejects the deletion of BOOLCASES. It is not a statement about other logical bases, other primitive operations or other interpretations; the stronger statement about bi-interpretations of the case-free calculus is Theorem 10.25.

10.3 Symmetry

This section draws on `research` and `compact_kernel`; `checker_and_barriers` applies the result to the case-free progressive calculus. `research` attributes the method, a finite-parameter involution against a definable linear order, to earlier drafts on unary arithmetic. The obstruction of Theorem 10.13 concerns the original domain. An interpretation on a different definable domain is not touched by it; the symmetry argument handles that freedom.

Lemma 10.19 (Finite parameters leave an involution)**.** *Let J be infinite. For every finite tuple $\vec{c}$ of $\mathcal{N}^{(J)}$, the expansion $(\mathcal{N}^{(J)}, \vec{c})$ has an automorphism of order two that is not the identity on the numerical sort.*

Proof. Outside a finite set $F \subseteq J$ every component of $\vec{c}$ is constant. Choose distinct $i, j \notin F$ and exchange the coordinates i and j in every family of every sort. The exchange preserves the domain, commutes with every coordinatewise function and preserves every coordinatewise relation, whatever the size of the signature. It fixes $\vec{c}$, has order two, and moves e_i to e_j. □

Lemma 10.20 (Order rigidity)**.** *Every classical model of* iPRA *has a parameter-free definable strict linear order, for example $x < y :\Leftrightarrow Sx \dot{-} y = 0$ with truncated subtraction. An automorphism of finite order of a linearly ordered structure that preserves the order is the identity.*

Proof. The order laws are theorems of primitive recursive arithmetic with quantifier-free induction. If $a < \sigma(a)$, repeated application gives $a < \sigma(a) < \cdots < \sigma^r(a) = a$ for the order r of σ; the case $\sigma(a) < a$ is symmetric. Linearity gives $\sigma(a) = a$. □

Lemma 10.21 (Automorphisms under a bi-interpretation)**.** *Let a structure M interpret a structure N by an ordinary interpretation with parameters among $\vec{c}$. Every automorphism of M that fixes $\vec{c}$ acts coordinatewise on representing tuples and induces an automorphism of N; this is a group homomorphism $\mathrm{Aut}(M/\vec{c}) \to \mathrm{Aut}(N)$. If in addition N interprets a copy of M, and an isomorphism η of M onto that copy is definable in M, and $\vec{c}$ contains the parameters of η and representatives in M of the parameters of the second interpretation, then the homomorphism is injective.*

Proof. The domain, the equality and the graphs of the interpretation are definable over $\vec{c}$, so they are preserved by $\sigma \in \mathrm{Aut}(M/\vec{c})$; this gives the induced automorphism $\widehat{\sigma}$ and the homomorphism. The automorphism $\widehat{\sigma}$ fixes the parameters of the second interpretation, since σ fixes their representatives, and so induces an automorphism $\widehat{\widehat{\sigma}}$ of the copy of M. Since η is definable over parameters fixed by σ, $\eta(\sigma(a)) = \widehat{\widehat{\sigma}}(\eta(a))$. If $\widehat{\sigma}$ is the identity, so is $\widehat{\widehat{\sigma}}$, hence $\eta(\sigma(a)) = \eta(a)$, and injectivity of η gives $\sigma(a) = a$. Only this one composite is used. The interpretations may have any finite dimension, quotient equality and domains different from the original ones. □

Theorem 10.22 (Symmetry obstruction)**.** *Let J be infinite and let T be a theory, in the language of a structure $\mathcal{N}$ as in Theorem 10.8, that has $\mathcal{N}^{(J)}$ as a model. Then T and* iPRA *are not bi-interpretable in the sense of Theorem 10.1. Arbitrary formula-defined domains, graphs and quotient equality are allowed, in any finite dimension and over finite products of the sorts. Finite parameter tuples are allowed, including parameters of the return isomorphism, provided the interpretations and the isomorphism exist in $\mathcal{N}^{(J)}$; this covers a parameter condition that T proves to be inhabited and under which T proves all the required facts.*

Proof. Apply a hypothetical bi-interpretation in $\mathcal{N}^{(J)}$, which yields a classical model $\mathcal{M}$ of iPRA. Let $\vec{c}$ collect the parameters of the first interpretation, of the return isomorphism, and representatives in $\mathcal{N}^{(J)}$ of the parameters of the second interpretation. Theorem 10.19 gives a non-identity involution σ of $(\mathcal{N}^{(J)}, \vec{c})$. Its image in $\mathrm{Aut}(\mathcal{M})$ has order dividing two and preserves the definable order, so it is the identity by Theorem 10.20. By Theorem 10.21 the homomorphism is injective, so σ is the identity on every sort, which contradicts its choice on the numerical sort. Faithfulness of the action on the other sorts is not needed. □

Corollary 10.23. *None of the following theories is bi-interpretable with* iPRA *in the sense of Theorem 10.1:*

(i) T_{alg}*, also with every set of universal Horn sentences true in* $\mathbb{N}$ *adjoined;*

(ii) for every finitary expansion $\mathcal{N}$ of $(\mathbb{N}, 0, S)$, the theory of all universal Horn sentences true in $\mathcal{N}$ with induction for atomic formulas or for finite conjunctions of atomic formulas;

(iii) $T_{\mathrm{pp}}(\mathcal{N})$ *and* $T_{\mathcal{C}}(\mathcal{N})$*, and every subtheory of them, recursively axiomatized or not;*

(iv) the many-sorted theories $T_{\mathcal{C}}(\mathcal{A}')$ for a many-sorted $\mathcal{A}'$ with numerical sort $(\mathbb{N}, 0, S)$, with induction on the numerical sort for conjunctive matrices whose parameters and witnesses may range over the other sorts.

Proof. Each theory is true in $\mathcal{E}$ by Theorem 10.11. In the many-sorted case the family varied in the induction argument belongs to the numerical sort and the parameters of the other sorts stay fixed. □

Remark 10.24 (Function objects)*.* The many-sorted case applies when a sort consists of numerical functions and the signature has a total application operation. Extensionality, $(\forall n\ \mathrm{app}(f, n) = \mathrm{app}(g, n)) \to f = g$, has a conjunctive antecedent and is preserved. Adding function objects, application, extensionality and equational combinator laws does not by itself remove the symmetry.

Corollary 10.25 (The case-free progressive calculus)**.** *Every rule of the case-free progressive calculus is sound in T_{alg} under the readings of Theorem 10.7;* ITER *is induction for a literal equation. Hence $\mathcal{E}$ is a model of its ordinary first-order realization, and that realization is not bi-interpretable with* iPRA *in the sense of Theorem 10.1.*

Remark 10.26 (Scope). Theorem 10.22 is about ordinary model-theoretic bi-interpretations, or about a native proposal whose claimed interpretations are proved to induce such interpretations under the displayed readings. Soundness of a native calculus in an ordinary theory does not show that the calculus derives the logical rules of that theory, and a proof translation that changes the meaning of the connectives need not give an ordinary interpretation; the theorem does not address such translations. It does not exclude equiconsistency, one-way interpretations, disjunctive axioms such as coverage or a Boolean range assertion, non-regular rules, or expansions that are not interpreted coordinatewise. A parameter condition that excludes $\mathcal{E}$ and is not proved inhabited there is outside the claim. Changing a term-defined operation to a positive existential graph does not escape the theorem, since existential witnesses and conjunctive induction of arbitrary alternation are allowed.

Remark 10.27. `research` proves Theorem 10.22 for T_{alg}, universal Horn sentences and conjunctions of atoms. `compact_kernel` proves it for T_{pp}, for $T_{\mathcal{C}}$ and for many sorts, with the order $x \le y :\Leftrightarrow \exists z\,(x + z = y)$ in place of the one in Theorem 10.20. `checker_and_barriers` restates the argument for Theorem 10.25.

10.4 Majorants: almost unary term languages

This section draws on `counter_congruence`, `research`, `local_scopes`, `adaptive_seed`, `uniform_predecesso` and `validated_leaves`. All statements concern the standard interpretation of fixed finite terms. The bounding functions belong to the metatheory; they are not asserted to be terms of the reduced languages.

10.4.1 Zero-seeded recursion is monotone

Proposition 10.28 (`counter_congruence`)**.** *Every function denoted by a term of the zero-seeded language of Theorem 10.3(a) is nondecreasing in each coordinate. Hence no term denotes the zero test $z(0) = 1$, $z(Sn) = 0$, and the terms do not denote all primitive recursive functions.*

Proof. Induction on terms. Variables, 0 and S are monotone. For $\mathsf{L}(n; u, v.h)$ the update $h(u, v, \vec{p})$ is coordinatewise nondecreasing by the induction hypothesis. Fix $\vec{p}$ and write $r_0 = 0$, $r_{k+1} = h(k, r_k, \vec{p})$. Then $r_0 \le r_1$, and $r_k \le r_{k+1}$ gives $r_{k+1} = h(k, r_k, \vec{p}) \le h(k+1, r_{k+1}, \vec{p}) = r_{k+2}$. So the value is nondecreasing in the counter. Induction on k gives $r_k(\vec{p}) \le r_k(\vec{p}\,')$ for $\vec{p} \le \vec{p}\,'$ coordinatewise. Substitution of the counter term and of parameter terms preserves monotonicity by the induction hypothesis. The zero test is not monotone at $0 < 1$. □

The statement is formalized as `ZTm.mono` and `ZTm.no_zero_test` in the module `ZeroSeedObstruction`; `Tm.val_zf` records that $\mathrm{sel}(x, 1, 0)$ of the iterator language is the zero test. The proposition excludes the direct term compiler only. The tied-seed language of Theorem 10.3(b) escapes it, since Z of Theorem 10.4 is the zero test.

10.4.2 Bounding coordinates

Definition 10.29. A function $f\colon \mathbb{N}^r \to \mathbb{N}$ has the *bounding coordinate* i if there is a nondecreasing unary primitive recursive function F with $f(x_1, \dots, x_r) \le F(x_i)$ for all arguments. A function is *almost unary* if it is globally bounded or has a bounding coordinate.

The name is the one used in clone theory for functions bounded by a unary function of one coordinate, whose closure under composition is standard (Pinsker, *Clones containing all almost unary functions*, Algebra Universalis 51 (2004)). The primitive recursive majorants are the additional content here.

Lemma 10.30 (Envelopes and composition)**.** *(i) For a primitive recursive map $g\colon \mathbb{N}^r \to \mathbb{N}^s$ the function $\widehat{g}(z) = \max\{g(\vec{z})_k : z_1, \dots, z_r \le z,\ k \le s\}$ is a nondecreasing unary primitive recursive function that bounds every output coordinate of g whenever all inputs are at most z. In particular a unary bound F may be replaced by its nondecreasing envelope $\widehat{F}(z) = \max_{m \le z} F(m)$.*

(ii) Constants, projections, unary primitive recursive functions and globally bounded functions are almost unary.

(iii) If f and $g_1, \dots, g_r$ are almost unary, so is $f(g_1(\vec{x}), \dots, g_r(\vec{x}))$.

Proof. (i) and (ii) are immediate. For (iii), if f is globally bounded so is the composite. If $f \le F(x_j)$, only g_j matters: a constant bound c of g_j gives the constant bound $F(c)$, and $g_j \le N(x_i)$ gives $F(N(x_i))$, by monotonicity of F. □

10.4.3 Adaptive query programs with several registers

This subsection is from `validated_leaves`. It gives one barrier for constructors whose seed and update both query one supplied body adaptively, with several registers and with query locations that depend on the target and the elapsed counter; Theorems 10.36 and 10.39 below are special cases. The restriction that makes the barrier work is on the data interface: numerical outputs are functions of oracle answers only.

Definition 10.31 (`validated_leaves`)**.** Fix integers $q, d \ge 1$ and $c \ge 0$. A *query program* with c control inputs, q query coordinates and output dimension d is defined together with the number j of answers available at a node: a program with j answers is

(a) a *leaf* F, where $F\colon \mathbb{N}^j \to \mathbb{N}^d$ is a fixed primitive recursive map; or

(b) a *query node* (Q, P'), where $Q\colon \mathbb{N}^{c+j} \to \mathbb{N}^q$ is a fixed primitive recursive map and P' is a program with $j+1$ answers; or

(c) a *test node* (T, P', P''), where $T \subseteq \mathbb{N}^{c+j}$ is a fixed primitive recursive predicate and P', P'' are programs with j answers.

A query program is a program with 0 answers; it is a finite tree. For an oracle $O\colon \mathbb{N}^q \to \mathbb{N}$ and controls $\vec{c} \in \mathbb{N}^c$, the value $P^O(\vec{c}) \in \mathbb{N}^d$ is computed from the root with the empty answer list: a query node (Q, P') reached with answers $a_1, \dots, a_j$ sets $a_{j+1} = O\big(Q(\vec{c}, a_1, \dots, a_j)\big)$ and continues with P'; a test node continues with P' if $T(\vec{c}, a_1, \dots, a_j)$ holds and with P'' otherwise; a leaf returns $F(a_1, \dots, a_j)$.

An *adaptive-program scheme* is given by $q, d \ge 1$, a seed program P_0 with one control input, an update program P_1 with $d+2$ control inputs, both with q query coordinates and output dimension d, and a fixed primitive recursive $G\colon \mathbb{N}^d \to \mathbb{N}$. Its supplied subdescriptions are a counter term $t(\vec{p})$ and one body $h(\vec{u}, \vec{p})$ with q local variables $\vec{u}$ and passive variables $\vec{p}$. With $n = t(\vec{p})$ and $h_{\vec{p}}(\vec{a}) = h(\vec{a}, \vec{p})$, the constructor denotes $G(\vec{r}_n)$, where

$$\vec{r}_0 = P_0^{h_{\vec{p}}}(n), \qquad \vec{r}_{m+1} = P_1^{h_{\vec{p}}}(n, m, \vec{r}_m). \tag{10.3}$$

The *adaptive-program term language* has variables, any primitive operations that are almost unary, composition, and the constructors of all adaptive-program schemes, arbitrarily nested; the body and the counter term of a constructor are terms of the same language.

The programs reach $\vec{p}$ only through the oracle $h_{\vec{p}}$. The controls n, m and the old registers $\vec{r}$ may determine query locations and tests, but no leaf reads them: a leaf is a function of the answers alone, so a test on m may choose between two fixed output maps but cannot

copy m into a register. The number of queries on every path is bounded by the depth of the tree, independently of the controls and of the answers; the only unbounded iteration is the displayed recursion on m. A program with no test whose query maps ignore the earlier answers is *non-adaptive*.

Lemma 10.32 (Bounds for a query program)**.** *Let P be a query program with c control inputs and q query coordinates.*

(i) There is a nondecreasing unary primitive recursive B_P such that every output coordinate of $P^O(\vec{c})$ is at most $B_P(M)$, for all $\vec{c}$, whenever $O(\vec{u}) \le M$ for all $\vec{u}$.

(ii) Let H be a nondecreasing unary primitive recursive function and $\ell \le q$. There is a nondecreasing unary primitive recursive $K_{P,H,\ell}$ such that every output coordinate of $P^O(\vec{c})$ is at most $K_{P,H,\ell}(s)$ whenever all coordinates of $\vec{c}$ are at most s and $O(\vec{u}) \le H(u_\ell)$ for all $\vec{u}$.

Proof. Write $\widehat{g}$ for the envelope of Theorem 10.30(i); for a map without inputs it is the largest output coordinate.

(i) Every answer on the path taken is at most M. A leaf F reached with j answers returns coordinates at most $\widehat{F}(M)$. Put $B_P(M) = \max_F \widehat{F}(M)$ over the finitely many leaves. Tests select the leaf and query locations select the answers, but neither enters the bound; a query location may be arbitrarily large while its answer is at most M.

(ii) Let b be the largest number of query nodes on a path of P, and let $L(s) = \max_Q \widehat{Q}(s)$ over the finitely many query nodes. Put

$$A_0(s) = s, \qquad A_{j+1}(s) = \max\big(A_j(s),\, H(L(A_j(s)))\big) \quad (j < b).$$

Each A_j is nondecreasing and primitive recursive, and $A_j \le A_{j+1}$. By induction along the path taken, at a node reached with j answers all controls and all answers are at most $A_j(s)$. For $j = 0$ this is the hypothesis on $\vec{c}$; a test changes nothing; at a query node (Q, P') every coordinate of $Q(\vec{c}, a_1, \dots, a_j)$ is at most $\widehat{Q}(A_j(s)) \le L(A_j(s))$, so the new answer is at most $H(L(A_j(s))) \le A_{j+1}(s)$, since H is nondecreasing. A leaf F reached with $j \le b$ answers returns coordinates at most $\widehat{F}(A_j(s)) \le \widehat{F}(A_b(s))$. Put $K_{P,H,\ell}(s) = \max_F \widehat{F}(A_b(s))$ over the leaves. □

Theorem 10.33 (Adaptive iteration with several registers)**.** *Every term of the adaptive-program term language denotes an almost unary function.*

Proof. Induction on descriptions; the local and passive variables of a body are independent inputs for the induction hypothesis. Variables are projections, the primitive cases hold by assumption and composition is Theorem 10.30(iii). Consider a constructor with programs P_0, P_1 and output map G, and let $\widehat{G}$ be the envelope of G.

h is bounded by a constant M. By Theorem 10.32(i) every coordinate of $\vec{r}_0$ is at most $B_{P_0}(M)$ and every coordinate of $\vec{r}_{m+1}$ is at most $B_{P_1}(M)$, so the result is at most the constant $\widehat{G}\big(\max(B_{P_0}(M), B_{P_1}(M))\big)$.

$h(\vec{u}, \vec{p}) \le H(p_i)$ for a passive coordinate. For fixed $\vec{p}$ the oracle $h_{\vec{p}}$ is bounded by $M = H(p_i)$, whatever its local arguments. The same argument bounds the result by $\widehat{G}\big(\max(B_{P_0}(H(p_i)), B_{P_1}(H(p_i))\big)$, a nondecreasing primitive recursive function of p_i alone; the counter is not used.

$h(\vec{u}, \vec{p}) \le H(u_\ell)$ for a local coordinate. Let $K_0 = K_{P_0,H,\ell}$ and $K_1 = K_{P_1,H,\ell}$ be the bounds of Theorem 10.32(ii), and define by primitive recursion

$$W(0, n) = K_0(n), \qquad W(m+1, n) = \max\big(W(m, n),\, K_1(n + m + W(m, n))\big).$$

Induction on m shows that every coordinate of $\vec{r}_m$ is at most $W(m, n)$: the seed has the control n; at the step the number $n + m + W(m, n)$ bounds the target counter, the elapsed

counter and every old register, which are the controls of P_1. Hence the result is at most $\widehat{G}(W(n,n))$, and $K(n) = \max_{z \le n} \widehat{G}(W(z,z))$ is a nondecreasing unary primitive recursive bound, independent of every passive parameter.

In the last case apply the induction hypothesis to the counter term: a constant bound c of t gives the bound $K(c)$, and $t(\vec{p}) \le N(p_i)$ gives $K(N(p_i))$. The cases exhaust the induction hypotheses for h. □

If the primitive operations are primitive recursive, so is every term: a fixed query program is a finite composition with definition by cases, primitive recursive relative to its body, and the outer loop is primitive recursion on m.

Remark 10.34 (Effectiveness). Given bounding-coordinate or global-bound witnesses for the primitive operations of a description, the proof constructs a coordinate and a majorant effectively. A bound of an arbitrary globally bounded primitive recursive function cannot be computed from its bare description: for a machine e, the function $b_e(x,y)$ equal to the first halting time of e if it is at most $\max(x,y)$ and 0 otherwise is globally bounded, and a uniform extractor of bounds would decide halting. Unary primitives have computable envelopes and Boolean predicates have the bound 1, so these need no witness. The existence statement of the theorem needs no witnesses.

10.4.4 Single-body constructors

Definition 10.35 (`uniform_predecessor`)**.** A *single-body constructor scheme* is given by integers $a, b, d, q \ge 1$ and fixed primitive recursive maps

$$A_i \colon \mathbb{N} \to \mathbb{N}^q \ (i \le a), \quad U_j \colon \mathbb{N}^{d+2} \to \mathbb{N}^q \ (j \le b), \quad F_0 \colon \mathbb{N}^a \to \mathbb{N}^d, \quad F_1 \colon \mathbb{N}^b \to \mathbb{N}^d, \quad G \colon \mathbb{N}^d \to \mathbb{N}.$$

Its supplied subdescriptions are a counter term t and one body $h(\vec{u}, \vec{p})$ with q local variables $\vec{u}$ and passive variables $\vec{p}$. With $n = t(\vec{p})$, the constructor denotes $G(\vec{r}_n)$, where

$$\vec{r}_0 = F_0\big(h(A_1(n), \vec{p}), \ldots, h(A_a(n), \vec{p})\big), \tag{10.4}$$

$$\vec{r}_{m+1} = F_1\big(h(U_1(n, m, \vec{r}_m), \vec{p}), \ldots, h(U_b(n, m, \vec{r}_m), \vec{p})\big). \tag{10.5}$$

The maps are fixed before the body is supplied. No passive parameter and no retained register enters F_0, F_1 or G directly: passive data and retained state reach the seed and update combiners only through evaluations of the one body h. The target counter n and the elapsed counter m may enter the probe locations. The *single-body term language* has variables, 0, any primitive operations that are almost unary, composition, and the constructors of all schemes, arbitrarily nested.

The tied-seed recursor is the scheme $d = 1$, $q = 2$, $a = b = 1$, $A_1(n) = (0,0)$, $U_1(n,m,r) = (m,r)$, $F_0(z) = z + 1$, $F_1(z) = z$, $G(r) = r$.

Theorem 10.36 (Single-body information barrier)**.** *Every term of the single-body term language denotes an almost unary function.*

Proof. A single-body constructor is an adaptive-program constructor with the same counter term, body, d, q and G. The seed program has no test, queries $h_{\vec{p}}$ at $A_1(n), \ldots, A_a(n)$ and has the leaf F_0; the update program has no test, queries $h_{\vec{p}}$ at $U_1(n,m,\vec{r}), \ldots, U_b(n,m,\vec{r})$ and has the leaf F_1. Both are non-adaptive, and Eqs. (10.4) and (10.5) are Eq. (10.3) for them. By induction on descriptions every term of the single-body term language is therefore a term of the adaptive-program term language with the same denotation, and Theorem 10.33 applies. □

10.4.5 Adaptive query circuits

Definition 10.37 (`adaptive_seed`)**.** A *query circuit* P is a finite acyclic circuit with no input nodes. Its nodes are numerals, applications of fixed primitive recursive functions to earlier nodes, and queries $Q(a,b)$ whose two arguments are earlier nodes; one node is the output. For an oracle $Q\colon \mathbb{N}^2 \to \mathbb{N}$ the output is P^Q. Later queries may depend on earlier answers. For an update term $h(u,v,\vec{p})$ put $h_{\vec{p}}(a,b) = h(a,b,\vec{p})$; the *tied-seed recursor with circuit* P is

$$\mathsf{R}_P(0;u,v.h) = P^{h_{\vec{p}}}, \qquad \mathsf{R}_P(n+1;u,v.h) = h(n,\mathsf{R}_P(n;u,v.h),\vec{p}).$$

The circuit reaches $\vec{p}$ only through queries to $h_{\vec{p}}$; it receives no separate initializer, no counter and no passive parameter. The *circuit term language* has constants, projections, any primitive operations that are almost unary, substitution, and the constructors R_P for all circuits P, arbitrarily nested. The combining operations inside a circuit are arbitrary primitive recursive functions.

The tied-seed recursor B is the circuit with one query $Q(0,0)$ followed by successor. A circuit may, for example, compute $q = h(0,0,\vec{p})$ and return $q + h(q,q+1,\vec{p})^2$.

Lemma 10.38. *Every query circuit P is a query program without control inputs, with $q = 2$ and $d = 1$, in the sense of Theorem 10.31: there are fixed primitive recursive maps $V_1,\dots,V_k$ and E such that P^Q is computed by the queries $a_j = Q(V_j(a_1,\dots,a_{j-1}))$ for $j \le k$ followed by the leaf $E(a_1,\dots,a_k)$, for every oracle Q. Consequently*

(i) there is a unary primitive recursive C_P such that $P^Q \le C_P(M)$ whenever $Q(a,b) \le M$ for all a,b;

(ii) if $Q(a,b) \le F(a)$ for all a,b, with F a fixed unary primitive recursive function, then P^Q is bounded by a constant that depends on P and F only. The same holds with $F(b)$.

Proof. List the query nodes in a topological order of the circuit. By induction along that order, the value of every node is a fixed primitive recursive function of the answers of the query nodes before it: a numeral is a constant function, an operation node composes a fixed primitive recursive function with the functions of its inputs, and a query node is its own answer. Applied to the two arguments of the jth query node this gives V_j, and applied to the output node it gives E. Then (i) is Theorem 10.32(i), and (ii) is Theorem 10.32(ii) with $\ell = 1$ or $\ell = 2$, $H = \widehat{F}$ and $s = 0$, since there are no controls. □

Theorem 10.39 (Tied-seed circuit obstruction)**.** *Every term of the circuit term language denotes an almost unary function.*

Proof. $\mathsf{R}_P(n;u,v.h)$ is the adaptive-program constructor with the counter term n, the body h, $q = 2$, $d = 1$, $G(r) = r$, the seed program given for P by Theorem 10.38 (its control n unused), and the update program with controls (n,m,r), the single query location (m,r) and the leaf a_1. Indeed Eq. (10.3) then reads $r_0 = P^{h_{\vec{p}}}$ and $r_{m+1} = h(m,r_m,\vec{p})$, which is the definition of R_P at 0 and $m+1$. Constants, projections and substitution are variables, primitive operations and composition. So every term of the circuit term language is a term of the adaptive-program term language with the same denotation, and Theorem 10.33 applies. □

Remark 10.40. The direct proof of `adaptive_seed` gives sharper bounds in the two local cases. If $h(u,v,\vec{p}) \le F(u)$ or $h(u,v,\vec{p}) \le F(v)$, then r_0 is at most a constant b_0 by Theorem 10.38(ii). In the first case $r_{j+1} \le F(j)$, so $\max(b_0, \widehat{F}(n))$ bounds r_n. In the second case $K(z) = \max(z, \widehat{F}(z))$ is nondecreasing with $K \ge \mathrm{id}$, and induction gives $r_n \le K^n(b_0)$. If $h \le c$ or $h \le F(p_i)$, writing M for the bound, $\max(C_P(M), M)$ bounds the result.

Corollary 10.41 (The tied-seed language)**.** *Every term of the tied-seed language of Theorem 10.3(b), and of its unary-update variant, denotes an almost unary function. This remains true after adjoining all unary primitive recursive functions and any operations that are globally bounded or have a bounding coordinate, for example a primitive characteristic of equality with values in* $\{0,1\}$*, predecessor, zero tests and minimum.*

Proof. The language is a sublanguage of both the single-body and the circuit term language. Directly: if $h \le c$ the values are at most $c+1$; if $h \le F(p_i)$ they are at most $1+F(p_i)$; if $h \le F(u)$ the value at the counter m is at most $1+F(m)$; if $h \le F(v)$ it is at most $\kappa(m)$ with $\kappa(0) = 1+F(0)$ and $\kappa(m+1) = \max(\kappa(m), F(\kappa(m)))$. □

Corollary 10.42 (What almost unary terms cannot denote)**.** *No term of the adaptive-program term language, and hence none of the single-body or of the circuit term language, denotes addition, maximum, the selector with* $\mathrm{sel}(0,a,b) = a$ *and* $\mathrm{sel}(Sn,a,b) = b$*, or an injection* $\mathbb{N}^2 \to \mathbb{N}$*. In particular the tied-seed language has no such term and cannot compile the initialized iterator, since* $I(b,a;\, v.\, Sv) = a+b$*.*

Proof. By Theorem 10.33 it suffices that none of these functions is almost unary. For addition or maximum, fix the supposed bounding coordinate and increase the other argument; a global bound fails likewise. For the selector, a bound in the control fails at the control 0 as a grows; a bound in a fails at $a = 0$ and control 1 as b grows; a bound in b fails at $b = 0$ and control 0 as a grows. Additional dummy coordinates can be fixed. For an injection, fixing the bounding coordinate at 0 leaves infinitely many pairs whose values lie in one finite interval. □

Remark 10.43 (The boundary of the hypotheses). The data interface is what the theorems restrict. (i) If the initializer circuit receives one further passive input y, the update $h(u,v,x,y) = x$ and the initializer $y + h(0,0,x,y)$ give $x+y$ at the counter 0. (ii) A circuit with two independent update oracles queries the constant updates x and y and adds the answers. (iii) If a passive initializer enters a register directly, $r_0 = a$, $r_{m+1} = Sr_m$ computes $a+n$. (iv) Direct register retention escapes Theorems 10.33 and 10.36 even with a tied seed: $r_0 = h(0,\vec{p})$, $r_{m+1} = Sr_m$ with the constant body $h(u,a) = a$ computes $a+n$; here the old register reaches the update combiner without passing through the body. (v) Finiteness of the circuit matters: for the oracle that is 1 exactly at $(u,v) = (x,y)$, every answer is at most 1, and an unbounded search over all pairs finds (x,y) and returns $x+y$; such a search is not a fixed finite circuit and is not total for all oracles. (vi) In Theorem 10.33 a leaf may not read a control numerically. If the seed and the update program both return $n + h(0,\vec{p})$, with the body $h(u,a) = a$ and the counter term x, then $\vec{r}_m = x+a$ at every stage and the output is $x+a$; no register is retained, and finite tests on n do not allow this use of n. (vii) The query budget must be fixed in advance. If both programs make n queries of the body $h(u,a) = a$ and return the sum of the answers, with the counter term x, the output is xa: every answer is at most a, but xa is bounded neither through a nor through x. Saying only that every execution makes finitely many queries is not enough; (v) is the same failure for a budget that depends on the answers. These examples mark the boundary. None of them is claimed to give a complete basis or a smaller checker.

Remark 10.44 (Scope and sources). The theorems are term-definability obstructions. They do not exclude quantified graph definitions, a relational interpretation, a different interpretation domain or additional axioms; an independent unbounded initializer is one escape and is not asserted to be the only one. The one-coordinate bound for B and the exclusion of addition and selection are in `counter_congruence`; `research` rederives them for F and B; `local_scopes` adds the bounded-valued primitives. Theorem 10.36 is from `uniform_predecessor` and Theorem 10.39 from `adaptive_seed`, each with a direct majorant induction. The two classes

are incomparable: the first has register vectors, several probes in the update and counter-dependent probe locations, but a non-adaptive seed; the second has an adaptive seed, but a scalar state and the fixed update interface. Theorem 10.33, from `validated_leaves`, is a common extension: later queries may depend on earlier answers in the seed and in every update, with several registers and counter-dependent locations. Its proof is the same majorant induction, and the article derives the two earlier theorems from it. Globally bounded primitives such as an equality characteristic are allowed in it, so it asserts none of the finite-profile invariants of Section 10.6. Theorem 10.47 applies to its standard algebra as well.

10.5 Finite-dimensional term interpretations

This section draws on `adaptive_seed` and `local_scopes`, which prove the same theorem independently. It turns the bound of Section 10.4 into an obstruction to interpretations that use finite tuples and a quotient equality, as long as the interpreted addition is given by a tuple of terms. The theorem is about functions on $\mathbb{N}$; the domain and the equivalence need not be definable.

Theorem 10.45 (Finite quotient from coordinate bounds)**.** *Let $d \geq 1$, $D \subseteq \mathbb{N}^d$, let $\sim$ be an equivalence relation on D and $e \in D$. Let $f\colon D \times D \to D$ satisfy*

$$f(e,z) \sim z \sim f(z,e) \qquad (z \in D).$$

Suppose that for each output coordinate i there are an input coordinate $j_i \leq d$ and a nondecreasing unary g_i such that, uniformly on $D \times D$, either $f(x,y)_i \leq g_i(x_{j_i})$ or $f(x,y)_i \leq g_i(y_{j_i})$, or that $f(x,y)_i$ has a constant bound. Then $D/\!\sim$ is finite. More precisely, with $L(z) = f(e,z)$ and $R(z) = f(z,e)$, a composition of at most $\ell_d = \lceil \log_2(d+1) \rceil$ maps chosen from L, R sends every element of D to a $\sim$-equivalent element of one finite box $\prod_{i \leq d}\{0,\dots,C_i\}$, and $D/\!\sim$ has at most $\prod_i (C_i+1)$ elements.

Proof. A coordinate bounded through an x-input has the constant bound $g_i(e_{j_i})$ under L and the bound $g_i(z_{j_i})$ under R; for a coordinate bounded through a y-input the roles are exchanged; a constantly bounded coordinate is bounded under both. Regard $\{1,\dots,d\}$ as a set of states that track dependencies of bounds. A letter sends a state i to the state j_i if it leaves the dependence, and to an absorbing terminal state if it gives a constant bound. Each state is terminated by at least one of the two letters.

Choose the letters from the outside of the composition inward. Let S be the current set of live states, initially all d states. The number of states of S terminated by L plus the number terminated by R is at least $|S|$, so one letter terminates at least $\lceil |S|/2 \rceil$ of them. Choose it and replace S by the set of images of the survivors; each survivor has one image, so the new set has at most $\lfloor |S|/2 \rfloor$ elements. After at most ℓ_d choices S is empty, since $2^{\ell_d} > d$.

Let the chosen letters be $V_1,\dots,V_k$ and $W = V_1 \circ \cdots \circ V_k$, so that V_k is applied first. By induction on the number of letters, each coordinate of the partial composition $V_1 \circ \cdots \circ V_m$ is either uniformly bounded or bounded by a composition of the functions g_i along one live dependency path, applied to a live coordinate of the argument; appending the next letter on the inside replaces that coordinate by its bound, which is legitimate because the g_i are nondecreasing. When the path terminates the bound is a constant. The empty final set gives constants $C_1,\dots,C_d$ with $W(D) \subseteq \prod_i \{0,\dots,C_i\}$.

Both L and R map D into D and send every element to an equivalent one, so $W(z) \sim z$ by transitivity. Every class meets the finite set $D \cap \prod_i \{0,\dots,C_i\}$. □

Associativity, commutativity and compatibility of f with $\sim$ are not used. A unary bound may be replaced by its nondecreasing envelope, so monotonicity is no extra assumption for term majorants. For $d = 0$ the domain has at most one element.

Proposition 10.46 (The word length is sharp)**.** *For every $d \geq 1$ there are $D = \mathbb{N}^d$, the universal equivalence, e the zero tuple and an f whose coordinates are projections or constants, such that the shortest composition of L, R with finite image has length exactly $\lceil \log_2(d+1) \rceil$.*

Proof. Let $k = \lfloor \log_2 d \rfloor$. If $k = 0$ then $d = 1$; take $f(x, y) = x$, so R is the identity and L is constant, and the minimum is 1. Let $k \geq 1$. Label 2^k of the coordinates by words $w \in \{L, R\}^k$ and let $\sigma(w)$ be w with its first letter removed and R appended. Put $f(x, y)_w = y_{\sigma(w)}$ if w starts with L and $f(x, y)_w = x_{\sigma(w)}$ if w starts with R; the remaining coordinates are 0. Under a letter, the state w survives exactly when the letter is its first letter, and then moves to $\sigma(w)$. For a word $\alpha \in \{L, R\}^k$, read from the outside inward, the state α survives all k letters, and the coordinate α of the composition is the projection to the coordinate R^k of the argument. So no word of length k has finite image, and neither has a shorter word, since extending a word with finite image on the inside keeps the image finite. The word L^{k+1} has finite image: a state with an R among its k letters is terminated when that letter is reached, and the state L^k has become R^k after k letters and is terminated by the last L. The minimum is $k + 1 = \lceil \log_2(d+1) \rceil$. □

Corollary 10.47 (No finite-dimensional term interpretation of iPRA)**.** *Let T be a theory in a numerical language whose standard model on $\mathbb{N}$ satisfies T and in which every term denotes an almost unary function; for example a theory true in the standard tied-seed algebra, in the single-body algebra, in the circuit algebra or in the adaptive-program algebra of Theorem 10.33. There is no interpretation of* iPRA *in T such that the interpreted domain is a nonempty subset of a finite power of the source domain, the interpreted equality is an arbitrary equivalence relation on it, and the interpreted addition has one fixed tuple of terms as realizers. Fixed numerical parameters are allowed, and successor may be interpreted in any way consistent with the axioms. In particular there is no bi-interpretation with these properties.*

Proof. Apply the interpretation in the standard model. Let e represent the interpreted zero. The two zero laws of addition are theorems of iPRA, so the addition tuple satisfies the unit hypothesis of Theorem 10.45; each of its coordinate terms is almost unary in the $2d$ input variables, and a majorant in a fixed parameter is a constant. So the interpreted quotient is finite. The interpreted successor is injective on the quotient and its image omits the class of zero, which is impossible on a finite nonempty set. □

Remark 10.48 (Limits). (i) Both unit laws are needed: on $\mathbb{N}$ with equality, $f(x, y) = x$ satisfies $f(z, e) = z$ and the coordinate bound, and the quotient is infinite. (ii) The realizers must be terms. With decidable equality to zero the quantifier-free graph $(x = 0 \wedge z = y) \vee (x \neq 0 \wedge z = x)$ is total and single-valued and defines an associative operation on $\mathbb{N}$ with the two-sided unit 0, which selects the first nonzero argument; it is not almost unary. Likewise $\max(x, y)$ has the unit 0 and is defined by cases that choose different controlling coordinates. Neither example is an interpretation of iPRA; they show that the relational route is not closed by Theorem 10.45. (iii) The qualifier about the standard model is needed: a theory whose axioms exclude the standard algebra is not covered. (iv) The logarithmic bound counts layers of unit multiplication in a metatheoretic construction; it is not a bound on term size, evaluation time or checker size.

Remark 10.49. `adaptive_seed` proves Theorem 10.45 by the halving argument given here and proves Theorem 10.46. `local_scopes` proves the theorem by counting: from each starting coordinate at most one binary word of length m keeps a dependence alive, so at most d of the 2^m words are admitted, and a word admitted from no coordinate exists when $2^m > d$.

10.6 Finite profiles and relational bi-interpretations

This section draws on `local_scopes`. The coordinate bound leaves two questions open: whether the tied-seed language computes a bounded comparison of two unbounded inputs, and whether an interpretation that defines its operations by formulas escapes. A stronger invariant of tied-seed terms answers the first, and, combined with witness extraction from proofs, answers the second for a specified class of theories.

10.6.1 Finite control

Definition 10.50. A function $f\colon \mathbb{N}^r \to \mathbb{N}$, $r \geq 1$, has *finite control* if there are an active coordinate i, finite nonempty sets A_j and unary maps $\alpha_j\colon \mathbb{N} \to A_j$ for $j \neq i$, and unary functions F_a indexed by $a \in \prod_{j\neq i} A_j$, such that

$$f(x_1, \ldots, x_r) = F_{(\alpha_j(x_j))_{j\neq i}}(x_i).$$

The value may depend without bound on one coordinate; every other coordinate supplies only a member of a fixed finite set. The functions F_a are the *profiles*. For primitive recursive f all displayed maps are taken primitive recursive, with finite encodings of the A_j; specified finite codomains replace exact ranges, and no decision procedure for attained ranges is assumed.

A constant function has finite control with any active coordinate, and the definition imposes no restriction at arity one.

Lemma 10.51. *(i) If f has finite control and ρ is unary with finite range, then $\rho \circ f$ factors through finite-valued unary maps of all input coordinates.*

(ii) The functions with finite control contain the constants and projections and are closed under composition.

Both preserve primitive recursiveness of the displayed maps.

Proof. (i) Attach to the active coordinate the finite feature $\beta_i(x) = (\rho(F_a(x)))_a$. Together with the $\alpha_j(x_j)$ it determines $\rho(f(\vec{x}))$.

(ii) Consider $f(g_1(\vec{x}), \ldots, g_r(\vec{x}))$ where f has the active input i and g_i has the active input k. Every other g_j enters f through the finite-valued $\alpha_j \circ g_j$, which by (i) is determined by finite unary features of all coordinates of $\vec{x}$. Combine the features of the coordinates other than k with the finite controls of g_i. With these combined controls fixed, the expression is one unary function of x_k; the features that depend on x_k are evaluated inside it. There are finitely many combined controls. □

Theorem 10.52 (Finite control of tied-seed terms)**.** *Every function of positive arity denoted by a term of the tied-seed language of Theorem 10.3(b) has finite control, with primitive recursive feature maps and profiles obtained by recursion on the term. A closed term is a constant.*

Proof. Variables and 0 are covered by Theorem 10.51. It suffices to show that $\mathsf{B}_h(n, \vec{p})$ has finite control when the update $h(u, v, \vec{p})$ has; the actual counter term is then substituted by Theorem 10.51(ii). Write $\gamma(\vec{p})$ for a tuple of finite unary features, one for each passive coordinate.

The active input of h is the previous state: $h(u, v, \vec{p}) = F_{\alpha(u),\gamma(\vec{p})}(v)$. For each of the finitely many values c of γ define $R_c(0) = 1 + F_{\alpha(0),c}(0)$ and $R_c(m+1) = F_{\alpha(m),c}(R_c(m))$. Then $\mathsf{B}_h(n, \vec{p}) = R_{\gamma(\vec{p})}(n)$, with active coordinate n.

The active input of h is the old counter: $h(u, v, \vec{p}) = F_{\beta(v),\gamma(\vec{p})}(u)$. Define $R_c(0) = 1 + F_{\beta(0),c}(0)$ and $R_c(m+1) = F_{\beta(R_c(m)),c}(m)$. Again $\mathsf{B}_h(n, \vec{p}) = R_{\gamma(\vec{p})}(n)$.

The active input of h is a passive parameter p. Write $\vec{q}$ for the other passive parameters and $h(u, v, p, \vec{q}) = F_{\alpha(u),\beta(v),\gamma(\vec{q})}(p)$. Form a finite catalogue $(V_j)_{j\in I}$ of unary functions that contains every profile $F_{a,b,c}$ and every initial profile $V_{0,c}(p) = 1 + F_{\alpha(0),\beta(0),c}(p)$, with separate indices for separate occurrences. Every state of the recursion is the value at p of a catalogue function. The only observation of a state used by the update is its β-value; record all of them in the finite feature

$$\kappa(p) = \big(\beta(V_j(p))\big)_{j\in I}.$$

For each control value c and each possible vector k, define a finite-state process: its initial state is the index of $V_{0,c}$, and at the counter m the state j is followed by the index of $F_{\alpha(m),k_j,c}$. Let $\theta_{c,k}(n) \in I$ be its state after n transitions and pack the finitely many processes into the finite-valued feature $\eta(n) = (\theta_{c,k}(n))_{c,k}$. Induction on the counter gives

$$\mathsf{B}_h(n, p, \vec{q}) = V_{\eta(n)_{\gamma(\vec{q}),\kappa(p)}}(p) :$$

the initial case selects $V_{0,c}$, and the transition uses exactly the recorded observation $\beta(V_j(p)) = \kappa(p)_j$. With $H_{e,c}(p) = V_{e_{c,\kappa(p)}}(p)$ this reads $\mathsf{B}_h(n, p, \vec{q}) = H_{\eta(n),\gamma(\vec{q})}(p)$, with the active coordinate p and finite controls on n and on each remaining parameter. The processes are defined for all vectors k, including unrealized ones; only the realized ones are used. □

The bound of Theorem 10.41 follows from the theorem by taking the maximum of the finitely many profiles and its running maximum. The new content is that the other inputs supply only finitely many distinct profiles.

Corollary 10.53 (No equality test, minimum or truncated subtraction)**.** *No term of the tied-seed language denotes the characteristic function of equality or of inequality,* $\min(a, b)$*, or* $a \mathbin{\dot{-}} b = \max(a - b, 0)$.

Proof. A binary function with finite control and active first coordinate has finitely many distinct sections $a \mapsto f(a, b)$; with active second coordinate it has finitely many sections $b \mapsto f(a, b)$. Each displayed function has infinitely many distinct sections in both directions: for equality the section at b is the characteristic function of $\{b\}$; the functions $x \mapsto \min(x, b)$ differ at an x above both parameters; for truncated subtraction, distinct second arguments are separated at a large first argument, and distinct first arguments at the second argument 0. □

Remark 10.54. The theorem does not decide whether every unary primitive recursive function is denoted by a tied-seed term, since finite control is no restriction at arity one. It does not concern formula-defined relations: the source has logical equality $a = b$, which is not a term that returns its characteristic value. Bounded-valued primitives preserve the coordinate bound (Theorem 10.41) and need not preserve finite control; an adjoined equality characteristic violates it.

10.6.2 An existential-free presentation

Let T_{B} be intuitionistic first-order logic with equality in the staged signature of the tied-seed language, with the computation equations (10.1), (10.2), coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$, separation $0 = S0 \to \bot$, and the universal closures of

$$\big(E(0, \vec{p}) \wedge \forall n\,(E(n, \vec{p}) \to E(Sn, \vec{p}))\big) \to \forall n\, E(n, \vec{p})$$

for literal equations E. By Theorem 10.4 the term P satisfies $P(0) = 0$ and $P(Sy) = y$ by computation, so coverage is equivalent, without induction, to the existential-free sentence

$\forall x\,(x = 0 \vee x = S(Px))$. Equality is treated as a binary predicate with universal axioms of reflexivity, symmetry, transitivity and congruence for every function symbol; transport through formulas is derived. Thus T_{B} has an existential-free axiomatization in its own language. An infinite axiom scheme causes no difficulty, since a proof uses finitely many axioms.

Lemma 10.55 (Finite witness extraction)**.** *Let the language contain the closed term* 0 *and let* Γ *be a finite set of existential-free formulas. If intuitionistic first-order logic proves* $\Gamma \vdash \exists z_1 \cdots \exists z_d\, R(\vec{x}, \vec{z})$*, then there are finitely many tuples of terms* $\vec{t}_1(\vec{x}), \ldots, \vec{t}_M(\vec{x})$ *with*

$$\Gamma \vdash \bigvee_{\ell \le M} R(\vec{x}, \vec{t}_\ell(\vec{x})).$$

The formula R *is arbitrary; it may contain existential quantifiers. No uniqueness of witnesses is assumed.*

Proof. Take a cut-free intuitionistic sequent proof; cut elimination for intuitionistic first-order logic is standard (for instance Avigad, *Algebraic proofs of cut elimination*, J. Log. Algebr. Program. 49 (2001), Theorem 7.6). Equality is handled by the existential-free axioms, so no equality extension of the theorem is needed.

Starting at the end sequent, trace the branches whose succedent carries the displayed leading existential quantifiers, and keep the prefix of witness terms already selected. At a left implication rule keep the proof of the first premise unchanged and follow the second premise; at a left disjunction rule follow both premises; the other left and structural rules preserve the tracked succedent. Every antecedent formula on a tracked branch is a subformula instance of a formula of Γ and is existential-free, so no left existential rule occurs there. While a leading existential quantifier remains, an initial sequent cannot close the branch, because the succedent is not an antecedent formula; the only applicable right rule is existential introduction. Record its witness term and continue. When all d witnesses are introduced, stop and keep the remaining proof of the matrix instance unchanged; the matrix is not analyzed. If a tracked branch closes by falsity elimination, fill the unselected witnesses with 0; the matrix instance follows from the same falsity. There are finitely many branches, so finitely many complete tuples $\vec{t}_\ell$ are recorded.

Replace each stopped branch by its kept proof of $R(\vec{x}, \vec{t}_\ell)$ followed by introductions into the common disjunction, and reassemble the tracked left and structural inferences. An existential introduction that supplied a recorded component is omitted, since the disjunction is already the succedent of its transformed premise. Left implication inferences are reconstructed from the unchanged first premise and the transformed second premise.

No numerical parameter is bound on a tracked branch: left existential rules are absent, and right universal introduction cannot have the tracked succedent. Extraneous free variables of the proof are replaced by 0 after renaming eigenvariables, so the witness terms contain only $\vec{x}$ and the retained parameters. □

The lemma does not give a single uniform witness term; a finite disjunction is what the next theorem needs.

10.6.3 No compression of scalars

Theorem 10.56 (No provable compression from k scalars to d coordinates)**.** *Let* T *be an intuitionistic theory in the tied-seed language with an existential-free axiomatization and with the standard tied-seed algebra as a model. Let* $k > d \ge 1$ *and let* $R(\vec{x}, \vec{z})$ *be any formula with* k *variables* $\vec{x}$ *and* d *variables* $\vec{z}$*. If* $T \vdash \forall \vec{x}\, \exists \vec{z}\, R(\vec{x}, \vec{z})$*, then in the standard model some output tuple* $\vec{z}$ *occurs with two different input tuples. In particular* T *does not also prove*

$$R(\vec{x}, \vec{z}) \wedge R(\vec{x}\,', \vec{z}) \to \bigwedge_{i \le k} x_i = x'_i.$$

The relation may have many outputs for each input.

Proof. Instantiate at free $\vec{x}$ and collect the finitely many axioms used. Theorem 10.55 gives d-tuples of terms $\vec{t}_1, \dots, \vec{t}_M$ such that for every input in the standard model some $\vec{t}_\ell(\vec{x})$ is an R-output. By Theorem 10.52 each coordinate term $t_{\ell j}$ has finite control; let $q_{\ell j}$ be the number of its profiles. On the grid $X_N = \{0, \dots, N-1\}^k$ it takes at most $q_{\ell j} N$ values, since its active argument takes at most N values; constant terms are included by adding an unused active argument. So $\vec{t}_\ell$ takes at most $N^d \prod_j q_{\ell j}$ values on X_N, and the union of the images has at most CN^d elements with $C = \sum_\ell \prod_j q_{\ell j}$. If every output belonged to at most one input, the N^k inputs of X_N would need N^k distinct outputs in this union, which fails for $N > C$. Soundness in the standard model turns a proof of the displayed implication into exactly that condition. □

Corollary 10.57 (Scalar pairing is not provably total)**.** *In T_{B} no formula is provably a total coding of pairs of numbers by single numbers with unique decoding. This includes a pairing relation $\pi_1(c) = a \wedge \pi_2(c) = b$ with unary terms as projections, and arbitrary formulas as projection relations, provided each code determines at most one pair.*

Proof. A provably total coding relation $R(a, b, c)$ with provably unique decoding is the case $k = 2$, $d = 1$ of Theorem 10.56. □

Even a proof that every unary primitive recursive function is a tied-seed term would therefore not give pairing inside T_{B}.

10.6.4 No finite-dimensional bi-interpretation

Theorem 10.58. *Let T be an intuitionistic theory in the tied-seed language with an existential-free axiomatization and with the standard tied-seed algebra as a model; for example T_{B}, or T_{B} with further existential-free axioms true in the standard algebra. Then T and* iPRA *are not bi-interpretable in the sense of Theorem 10.1. The interpreted domains, the operation graphs and the quotient equality are arbitrary formulas. The statement is parameter-free, and it holds when fixed parameters are retained uniformly in the interpretation proofs without assumptions that are not existential-free.*

Proof. Let $\mathcal{I}$ interpret iPRA in T with dimension d, domain $D \subseteq \mathbb{N}^d$ and interpreted equality E, and let $\mathcal{J}$ interpret T in iPRA with dimension e. A zero-dimensional domain has finitely many tuples and cannot take part in the source-side isomorphism with the infinite source domain, so $d, e \geq 1$.

The composite represents source numbers by e-tuples of interpreted arithmetic elements, that is, by de source coordinates. The source-side return isomorphism provides a formula $\eta(a, \vec{u})$ of T, with $\vec{u} \in D^e$ and the domain conditions included, such that

$$T \vdash \forall a\, \exists \vec{u}\, \eta(a, \vec{u}), \qquad T \vdash \eta(a, \vec{u}) \wedge \eta(a', \vec{u}\,') \wedge E^{\mathcal{I}}_{\mathcal{J}}(\vec{u}, \vec{u}\,') \to a = a',$$

where $E^{\mathcal{I}}_{\mathcal{J}}$ is the translation under $\mathcal{I}$ of the equality of $\mathcal{J}$. Only this part of the bi-interpretation is used.

Fix $k \geq 1$. iPRA has a primitive recursive injective tupling of ke numbers with primitive recursive projections, and proves its decoding equations. Its translation under $\mathcal{I}$ is a provably total coding relation from $(D/E)^{ke}$ to D/E that is injective on input tuples. Define $R_k(a_1, \dots, a_k, \vec{c})$ in T: there are η-representatives $\vec{u}_i \in D^e$ of all a_i, and the interpreted tupling codes the ke arithmetic components of $(\vec{u}_1, \dots, \vec{u}_k)$ by $\vec{c} \in D$. From the totality of η and of the tupling, finite existential reasoning gives

$$T \vdash \forall a_1 \cdots \forall a_k\, \exists c_1 \cdots \exists c_d\, R_k(\vec{a}, \vec{c}).$$

The inputs a_i range over the whole source domain; no choice is used.

Suppose $R_k(\vec{a}, \vec{c})$ and $R_k(\vec{a}\,', \vec{c})$. The same tuple $\vec{c}$ represents the same arithmetic element, by reflexivity of E on D. Injectivity of the interpreted tupling gives componentwise E-equality of the arithmetic components of $\vec{u}_i$ and $\vec{u}'_i$, hence $E^{\mathcal{I}}_{\mathcal{J}}(\vec{u}_i, \vec{u}'_i)$ for each i, since interpretations respect equality. The second property of η gives $a_i = a'_i$. So T proves that the output fibers of R_k are disjoint. Surjectivity of η and uniqueness of representatives are not needed.

With $k = d + 1$ this contradicts Theorem 10.56. □

Corollary 10.59 (Provably inhabited parameter families)**.** *The exclusion holds for a bi-interpretation with a finite tuple $\vec{p}$ of r source parameters subject to a condition $\Theta(\vec{p})$, when $T \vdash \exists \vec{p}\,\Theta(\vec{p})$ and T proves the interpretation and return-isomorphism facts uniformly under $\Theta(\vec{p})$.*

Proof. The construction with the parameters retained proves totality and disjoint fibers of $R_{k,\vec{p}}$ under $\Theta(\vec{p})$. Put $\widehat{R}_k(\vec{a}, \vec{p}, \vec{c}) := \Theta(\vec{p}) \wedge R_{k,\vec{p}}(\vec{a}, \vec{c})$. Then $T \vdash \forall \vec{a}\,\exists \vec{p}\,\exists \vec{c}\,\widehat{R}_k$, and an identical output $(\vec{p}, \vec{c})$ uses identical parameters, so the fibers are disjoint. There are $d + r$ output coordinates; take $k = d + r + 1$. The formula Θ is inside the output matrix, so no restriction on its quantifiers is needed. □

Remark 10.60 (Scope). Positive existential domain formulas do not rescue a bi-interpretation: they occur inside the matrix R_k, which witness extraction does not analyze, and the return isomorphism removes every domain condition from the unrestricted source inputs. The theorem does not exclude equiconsistency or one-way interpretations on complicated domains; the return isomorphism is essential. It does not cover a parameter condition that T does not prove inhabited, a theory with positive existence axioms that cannot be eliminated in the tied-seed language, or every axiomatization in that language. It does not extend automatically to additional bounded numerical primitives, which preserve the coordinate bound and need not preserve finite control.

10.6.5 The selector boundary

The obstruction has an exact positive counterpart: a bounded primitive does not help (Theorem 10.41), and one unbounded selector restores primitive recursion.

Proposition 10.61 (`local_scopes`)**.** *Adjoin to the tied-seed language an operation C with $C(0, a, b) = a$ and $C(Sn, a, b) = b$, allowed inside update bodies. For terms $g(\vec{p})$ and $h(n, z, \vec{p})$ and fresh u, v put*

$$K(u, v, \vec{p}) := C(u,\, g(\vec{p}),\, h(W(u), v, \vec{p})), \qquad J(n, \vec{p}) := \mathsf{B}(Sn; u, v.K(u, v, \vec{p})),$$

with W as in Theorem 10.4. Equality reasoning and the computation equations prove $J(0, \vec{p}) = g(\vec{p})$ and $J(Sn, \vec{p}) = h(n, J(n, \vec{p}), \vec{p})$, with no induction, predecessor search, equality characteristic, pairing or independent initializer.

Proof. Suppress $\vec{p}$. The first update discards the tied initial state: $J(0) = \mathsf{B}(S0; u, v.K) = K(0, \mathsf{B}(0; u, v.K)) = C(0, g, \dots) = g$. At a successor, $J(Sn) = \mathsf{B}(SSn; u, v.K) = K(Sn, \mathsf{B}(Sn; u, v.K)) = C(Sn, g, h(W(Sn), J(n))) = h(n, J(n))$. These hold for arbitrary parameters. □

Zero, successor and projections are terms and composition is substitution, so every primitive recursive description compiles into the enlarged language, and conversely each description of the enlarged language is primitive recursive. This is numerical completeness only; the logical and interpretation claims for such a basis need their own proofs and their own size accounting.

A formula definition of C does not supply this compiler. With coverage and $0 \neq S0$, the term Z decides $z = 0$, and $(z = 0 \wedge w = a) \vee (z \neq 0 \wedge w = b)$ is provably total and single-valued, so a symbol with this graph is a definitional extension. Two things do not follow. The tied-seed signature has recursion symbols for its original update terms only; a recursion whose update contains the new symbol is a new graph whose totality and uniqueness need proof. And literal-equation induction for terms with the new symbol translates into induction for formulas that are not literal equations of the old language.

10.7 Zero profiles of positive iteration

This section draws on `checker_and_barriers`. Positive iteration $\mathsf{R}^+(n, a; v.h) = h^{n+1}(a)$ of Theorem 10.3(c) is not almost unary: $\mathsf{R}^+(n, a; v.Sv) = n + a + 1$ grows in both arguments. A different invariant applies.

Theorem 10.62 (Zero-profile theorem)**.** *For every term $t(x_1, \ldots, x_r)$ built from variables, 0, S and R^+, the zero set of t in the standard interpretation is $\varnothing$, $\mathbb{N}^r$, or $\{\vec{x} : x_j = 0\}$ for a single j. The profile and the coordinate are obtained by recursion on the term.*

Proof. Induction on terms, with bound update variables fresh. 0 is always zero, a successor never, and x_j exactly when $x_j = 0$. Consider $\mathsf{R}^+(n, a; v.h)$ with update $h(v, \vec{x})$. By the induction hypothesis the zero set of h has one of four forms. If h is never zero, the iterator is never zero. If h is always zero, the iterator is always zero. If h is zero exactly when a passive variable x_j is zero, the iterator has the same profile. These three cases use that at least one update is applied. If h is zero exactly when its state v is zero, every update preserves the zero status of the state, so the final state has the zero profile of the seed a, which is classified by the induction hypothesis. The zero profile of the counter is not used: the number of updates is positive. A variable that occurs both in the counter and in the seed or the update keeps its role in the respective profile. □

Corollary 10.63. *No term of this language denotes the predecessor (zero set $\{0, 1\}$), the zero test that is 1 at 0 and 0 elsewhere (zero set $\mathbb{N} \setminus \{0\}$), addition (zero set $\{(0, 0)\}$), or the initialized iterator, whose constant-update case $I(n, a;\, v.\, b)$ is the selector and gives the zero test at $a = 1$, $b = 0$.*

The definable shifted addition $n + a + 1$ is never zero and does not contradict the corollary.

Remark 10.64 (Scope). This is a standard-model term-definability obstruction. In intuitionistic logic with coverage, injectivity of S and separation $0 \neq Sm$, the formula

$$(n = 0 \wedge z = a) \;\vee\; \exists m\, (n = Sm \wedge z = \mathsf{R}^+(m, a; v.h))$$

defines a total single-valued graph of the initialized iteration of each update h of the old language. Separation is needed to exclude overlap of the two branches. The observation does not give a bi-interpretation with iPRA: induction for the graph-defined expanded language and closure under nesting of newly defined updates remain to be proved, and in the native calculus the connectives of the graph formula are themselves reconstructed from a selector, so assuming them at that point is circular. The invariant does not survive a zero-test primitive: $\mathsf{R}^+(n, 0; v.Zv)$ alternates between 1 and 0, so zero status then depends on the iteration count.

The motivation is a raw encoding in which a two-field numerical array $[a, h]$ has the value a and a three-field array $[n, a, h]$ the value $\mathsf{R}^+(n, a; v.h)$, which would allow a uniform predecessor expansion in place of a zero/successor conditional. The theorem shows that this encoding cannot inherit the term compiler or the selector-based logical bootstrap.

10.8 Rule-level obstructions

This section draws on `research`, `adaptive_seed`, `local_scopes`, `counter_congruence` and `constructed_computation`. Each result rejects one tempting deletion or replacement of a rule by a reading that validates the remaining rules. Each concerns the stated rule system with the identity translation of formulas.

10.8.1 Atomic case introduction cannot be made automatic

In N_p a certificate chooses between the two introductions of $D_{x,y}$. The *forced-progress* variant removes the choice: it uses DI-PROGRESS when the matrix is a literal equation and DI by cases when the matrix is compound. Equivalently, it deletes DI by cases at literal-equation matrices.

Proposition 10.65. N_p *derives* $D_{x,y}(y = 1)$*, with* x *unused. The forced-progress variant does not.*

Proof. In N_p, DI by cases needs $0 = 1 \vdash Sz = 1$. From $0 = 1$, transport through the selector gives $s = \mathrm{sel}(0, s, t) = \mathrm{sel}(1, s, t) = t$ for all terms s, t, in particular $Sz = 1$. No induction on a compound formula is used.

For the variant, evaluate terms in $\mathbb{N}$, read equations as equality and $Q_y B$ as $B(0) \to \forall y\, B(y)$, and read $D_{x,y}$ by

$$(D_{x,y}B)^\dagger = \begin{cases} \exists x\, \forall y\, (B^\dagger(x, y) \to B^\dagger(x, Sy)), & B \text{ a literal equation,} \\ \exists x\, (B^\dagger(x, 0) \to \forall y\, B^\dagger(x, y)), & B \text{ compound.} \end{cases}$$

The reading is defined on each fixed formula. Forced progress introduction is sound by its direct meaning. Case introduction at a compound matrix is sound by the zero-or-successor property of $\mathbb{N}$. Paired elimination at an atomic matrix is sound because induction in $\mathbb{N}$ turns the supplied progressiveness into the conditional closure $Q_y B(a, y)$; at a compound matrix it is sound directly. QE, HYP and the computation equations are sound. Transport is sound because numerical substitution does not change whether a matrix is a literal equation. Under $\dagger$ the formula $D_{x,y}(y = 1)$ says $\exists x\, \forall y\, (y = 1 \to Sy = 1)$, which is false at $y = 1$. □

The padding equivalence between a formula and a gated paired closure does not repair the counterexample: padding moves a proof obligation into a compound matrix, and the elimination returns only the conditional closure, not the progressiveness needed to introduce a raw atomic paired formula. A conditional closure can hold because its zero antecedent is false, without its matrix being progressive. A public language in which every atomic matrix is gated evades the obstruction at the cost of new grammar and decoder obligations.

Remark 10.66. `research` rederives the proposition from an earlier draft on flat computation certificates; `compact_kernel` and `uniform_predecessor` restate the conclusion. For the root-determined introduction with unrestricted transport the corresponding strictness is formalized as `rdprf_not_cases` and `rdprf_strictly_weaker`, with the progressive reading over the natural numbers as the sound reading.

10.8.2 Elimination majors must be proved, not only retained

Proposition 10.67 (`adaptive_seed`)**.** N_p *derives* $Q_y(0 = 0)$*. Restrict* N_p *so that the major premise of* QE *and of* DE *and the equation of a transport must be members of the retained context, keep* COMP*, the two introductions and, optionally, direct induction at equation goals, and add no cut rule. The restricted calculus does not derive* $Q_y(0 = 0)$*.*

Proof. In N_p, DI by cases with the witness 0 derives $D_{x,y}(0 = 0)$ from $0 = 0 \vdash 0 = 0$; DE applied to this derived major supplies $Q_y(0 = 0)$, and HYP concludes.

In the restricted calculus consider a goal that is a closure $Q_y B$ in a context of equations only, initially empty. HYP does not close it. An introduction concludes a paired formula. COMP adds an equation and keeps the goal; induction applies to equation goals only. Transport along a retained equation leaves a minor that is again a closure, in the same context. QE and DE need a retained closure or paired formula, and there is none. Induction on the height of a derivation shows that no derivation exists. □

The proposition concerns this deletion with closures as public formulas and the paired-introduction form; in N the formula is immediate by QI. It does not show that the closure binder must remain an object connective, and an unrestricted cut repairs the displayed theorem. The package verifies the example by a mutation of a checker in which the recursive check of a major is replaced by a membership test.

10.8.3 Root replacement does not give congruence

A further merger encodes an equation $s = t$ as a clause that sends $r = s$ to $r = t$, so that equality transport and closure use become one clause elimination. This yields replacement at the root of an equation only.

Proposition 10.68 (`local_scopes`)**.** *The computation equations, zero/successor cases, induction, reflexivity and root replacement do not derive congruence of S for a primitive equation predicate.*

Proof. Take $\mathbb{N}$ with the standard functions and interpret the equation predicate by the equivalence relation $\approx$ whose only non-singleton class is $\{2, 3\}$. Every computation equation holds because both sides have the same value. Reflexivity and root replacement are sound for an equivalence relation. Cases and induction are sound on $\mathbb{N}$ for every interpreted formula. But $2 \approx 3$ and $S2 \not\approx S3$. □

The counter-model is not the equality of the complete theory, which is actual equality. Theorem 10.113 shows what must be added: congruence in one fixed position, the counter of a recursor, suffices. A Leibniz clause $P(s) \to P(t)$ schematic in a formula context also merges the two eliminations, and moves the work into context application and higher-order substitution; no smaller checker for that route is established.

10.8.4 The witness order of the paired binder

A reversed binder $\mathsf{E}_{x,y}B := \forall y\, \exists x\, (B(x, 0) \to B(x, y))$ would merge the two eliminations and remove the reusable clause. Consider the naive introduction that fixes a witness term $t(z)$ in the instruction and checks $B(t(z), 0) \vdash B(t(z), Sz)$, its progress variant with a witness independent of z at a literal equation, and the ordinary instance elimination.

Proposition 10.69 (`counter_congruence`)**.** *There is a literal equation matrix B such that the ordinary reading $\forall y\, \exists x\, (B(x, 0) \to B(x, y))$ is true in $\mathbb{N}$ and $\mathsf{E}_{x,y}B$ is false under a reading that validates the naive rules. Hence the naive rules do not derive the equivalence of every E formula with its ordinary reading.*

Proof. Read $\mathsf{E}_{x,y}B$, with free parameters fixed, as: there is a unary primitive recursive f with $\forall y\, (B(f(y), 0) \to B(f(y), y))$. The naive case rule is sound with $f(0) = 0$ and $f(Sz) = t(z)$; the progress rule is sound with a constant f; instance elimination and transport are sound. The gated macros for the connectives keep their two-valued meanings: an existential macro needs a constant selector, and the universal macro has an unused existential coordinate.

Let F be a total computable function that is not primitive recursive and let $T(n, c)$ be the primitive recursive predicate saying that c is a halting computation trace of a fixed program for F on the input n. Choose a literal equation B with $B(x, 0) \equiv \top$ and $B(x, Sn) \equiv T(n, x)$, by a primitive recursive characteristic and a case selector. The ordinary reading is true. The selector reading is false: a primitive recursive f would give the trace $f(Sn)$, and the primitive recursive output decoder would compute $F(n)$. □

The source witness obtained at a fresh universal parameter may have to be chosen inside the proof scope of that parameter, after an existential elimination; a witness fixed before the subproof does not allow this interleaving. The proposition excludes this rule package with this reading. A repair with delayed witness choice or an intermediate introduction judgment is not excluded.

10.8.5 Two side conditions that cannot be dropped

Example 10.70 (Freshness of the step variable for the witness)**.** The formula $D_{x,y}(I(y, 0;\, v.\, Sx) = y)$ is false in $\mathbb{N}$: its zero instance is true, and its reading requires every positive number to equal one fixed Sx. If an introduction could take the step variable z as its witness, the step $I(Sz, 0;\, v.\, Sz) = Sz$ would be a computation instance. The condition that z is fresh for the witness is therefore necessary (`local_scopes`).

Example 10.71 (Local assumptions do not escape a major)**.** Case introduction derives $D_{x,y}(0 = 1)$ while locally assuming $0 = 1$. A checker that lets this assumption leak from the subproof of a major into the context of the caller accepts $0 = 1$ (`research`).

`constructed_computation` proves, by a finite-range invariant, that restricting the witnesses of DI to 0 or variables and at the same time the targets of QE to 1, to finitely many numerals, or to variables loses the theorem $D_{x,y}(\mathrm{sel}(y, N + 1, x) = N + 1)$ for a suitable N; the statement and its proof are given with the specialization rules in another chapter.

10.9 Independence of the progress rule

This section draws on `progress_independence` and, in Section 10.9.4, on `ipra_fixed_points_and_checker`, and in Section 10.9.5 on `iteration_barriers`. The first package has two independent write-ups of the same result; the section follows the first, takes the explicit derivation of the positive side and the extensional corollary from the second, and compares the two constructions in Theorem 10.83. It shows that QI by progress is not admissible over the other seven rules of the core. Write N^- for N without QI.

The README of the repository records three facts that shape a countermodel. Each of the other seven rules is independent of the rest, by readings of the language over the natural numbers (`NativeModels`). Such a reading cannot work for QI: over the natural numbers the soundness conditions for DI and DE already imply the condition for QI, whatever an equation means (`Sem.Sound.qIInd_of`); so the numerical domain must be nonstandard. And N^- derives $0 = 1$ from every successor cycle of odd length (`NoInd.no_odd_cycle`), so the model has no such cycle; Theorem 10.99 extends this to every length. The structures used earlier in the chapter do not serve. Every power $\mathcal{N}^{(J)}$ satisfies induction for literal equations (Theorem 10.11), so QI is sound in it; what fails there is coverage, hence DI by cases (Theorem 10.13). An ultrapower of the standard model with its internal iterator satisfies induction as well, by the theorem of Łoś. The countermodel has to do the opposite of a power: keep coverage and both computation equations at every count, nonstandard counts included, and lose induction for one equation. Coverage and acyclicity force the nonstandard part of the successor graph to consist of chains infinite in both directions, so every step

function that is definable with parameters needs an orbit infinite in both directions along each chain.

All arguments of this section are carried out in classical metamathematics with the axiom of choice. This adds no classical axiom to the object calculus: a classical structure is a sound test of intuitionistic derivability (Section 10.1.1).

10.9.1 Coherent iterator structures

Definition 10.72. An *iterator language* is a first-order language L with 0, S and iterator symbols $I_h(n, a, \vec{p})$, one for each member $h(u, \vec{p})$ of a set of *descriptors*: terms of L with a designated state variable u and a finite tuple of passive variables $\vec{p}$. The variable u need not occur in h. The language is built in stages, so that the body of an iterator symbol is a term of an earlier stage. An L-structure is *coherent* if it satisfies the following first-order sentences.

(a) *Successor conditions.* S is injective, 0 is not a successor, every element is 0 or a successor, and $S^k x \neq x$ for each $k \geq 1$.

(b) *Computation.* For every iterator symbol, $I_h(0, a, \vec{p}) = a$ and $I_h(Sn, a, \vec{p}) = h(I_h(n, a, \vec{p}), \vec{p})$.

(c) *Cross-extensionality.* For every two iterator symbols I_h, I_g of L, the case $h = g$ included,

$$\forall \vec{p} \forall \vec{q} \Big[\forall u \, \big(h(u, \vec{p}) = g(u, \vec{q})\big) \to \forall n \, \forall a \, \big(I_h(n, a, \vec{p}) = I_g(n, a, \vec{q})\big) \Big]. \tag{Ext}$$

By (Ext) an iterator depends on the unary function $u \mapsto h(u, \vec{p})$ and on its two numerical inputs, not on the presentation of the function; repetition, omission and reordering of parameters are included. An independent choice of an orbit for each written body would not have this property: substitution of terms for the passive parameters changes the written body without changing the function, and the substitution lemma for the native syntax would fail (Theorem 10.78).

Lemma 10.73 (Orbits in one ultrapower)**.** *Let A be a nonempty L-structure, $\mathcal{U}$ a nonprincipal ultrafilter on $\mathbb{N}$ and $B = A^{\mathbb{N}}/\mathcal{U}$. For every term $h(u, \vec{p})$ of L and every tuple $\vec{p}$ of B, the function $f(v) = h^B(v, \vec{p})$ has a two-sided orbit: a family $(x_j)_{j \in \mathbb{Z}}$ of elements of B with $f(x_j) = x_{j+1}$ for all j.*

Proof. Let $\vec{p} = [\vec{p}_i]$, put $f_i(v) = h^A(v, \vec{p}_i)$ and choose any $a_i \in A$. For $j \in \mathbb{Z}$ let $x_j = [f_i^{\max(0, i+j)}(a_i)]$, with finite external iteration in each coordinate. Term functions of B are computed coordinatewise, and $\max(0, i + j) + 1 = \max(0, i + j + 1)$ for all $i \geq -j$, a cofinite set of coordinates. Hence $f(x_j) = x_{j+1}$. □

The lemma concerns functions defined by terms with parameters. No induction axiom of A is used, and the orbit need not pass through a prescribed element. In the structure $\mathfrak{M}_0$ of Theorem 10.76 the function $u \mapsto S(I_{\mathrm{id}}(u, u))$ maps everything into the standard ray and has no two-sided orbit; this is why the ultrapower precedes every adjunction of iterators.

Lemma 10.74 (Chains)**.** *In a structure with the successor conditions, the elements $S^k 0$, $k \in \mathbb{N}$, are pairwise distinct and form one component of the successor graph, the* standard ray*; every other component is a copy of $(\mathbb{Z}, +1)$.*

Proof. Every nonzero element has exactly one predecessor and 0 has none. An element outside the standard ray does not reach 0 after finitely many predecessor steps, so its iterated predecessors and successors are all defined; they are pairwise distinct by injectivity and the absence of cycles. The same two properties make the numerals distinct. □

Lemma 10.75 (Extension of a coherent structure)**.** *Let A be a coherent L-structure, $B = A^{\mathbb{N}}/\mathcal{U}$ as in Theorem 10.73, and let $\mathcal{D}$ be a set of terms of L with designated state variables that have no iterator symbol in L. Then B is a coherent elementary extension of A, and B has an expansion to $L \cup \{I_h : h \in \mathcal{D}\}$ that is coherent for the enlarged language. The expansion changes no element and no function of B.*

Proof. The conditions of Theorem 10.72 are first-order, so they hold in B by the theorem of Łoś, also for the parameter tuples that are new in B, and the diagonal embedding is elementary.

By Theorem 10.74 the successor graph of B is the standard ray together with copies of $\mathbb{Z}$; choose an origin b in each copy. Call a function $f\colon B \to B$ *presented* if $f = h^B(-, \vec{p})$ for a term h of L and a tuple $\vec{p}$ of B, and *old* if h can be taken with an iterator symbol in L. Assign to every presented function f one map $It_f\colon B \times B \to B$ as follows.

If f is old, say $f = g^B(-, \vec{q})$ with I_g in L, put $It_f(n, a) = I_g^B(n, a, \vec{q})$. This does not depend on the choice of g and $\vec{q}$, by (Ext) in B.

If f is not old, put $It_f(S^k 0, a) = f^k(a)$ on the standard ray. For each copy of $\mathbb{Z}$ with origin b and each seed a choose a two-sided orbit $(x_j)_{j\in\mathbb{Z}}$ of f by Theorem 10.73 and put $It_f(S^j b, a) = x_j$, where negative powers are iterated predecessors. The axiom of choice provides the choices.

In both cases $It_f(0, a) = a$ and $It_f(Sn, a) = f(It_f(n, a))$: for old f by the computation axioms of I_g, on the standard ray by the definition, and on a copy of $\mathbb{Z}$ because $x_{j+1} = f(x_j)$. No condition links the values on a copy of $\mathbb{Z}$ to the seed, and none is required.

For $h \in \mathcal{D}$ define $I_h(n, a, \vec{p}) = It_f(n, a)$ with $f = h^B(-, \vec{p})$. The computation axioms of the new symbols hold. (Ext) holds for every pair of symbols of the enlarged language: for two old symbols it holds in B already, and in the remaining cases two instances with the same function f are both equal to It_f, an old instance by the first clause of the definition. □

Lemma 10.76 (The seed)**.** *Let $L_0 = \{0, S, I_{\mathrm{id}}\}$, where I_{id} is the iterator symbol of the descriptor u without parameters. Let $\mathfrak{M}_0 = \mathbb{N} \sqcup \mathbb{Z}$, with 0 and $1 = S0$ in the first summand, the ordinary successor on each summand, and*

$$I_{\mathrm{id}}(n, a) = \begin{cases} a, & n \in \mathbb{N}, \\ 1, & n \in \mathbb{Z}. \end{cases}$$

Then $\mathfrak{M}_0$ is coherent, and $I_{\mathrm{id}}(c, 0) = 1 \neq 0$ for every c in the second summand.

Proof. The successor conditions are evident. $I_{\mathrm{id}}(0, a) = a$, and $I_{\mathrm{id}}(Sn, a) = I_{\mathrm{id}}(n, a)$ because S preserves the summands. (Ext) for the one symbol without parameters is trivial. No predicate that distinguishes the summands is in the language. □

Proposition 10.77 (The tower)**.** *Let L_{i+1} be L_i together with an iterator symbol for every term of L_i with a designated state variable that has no symbol in L_i, and let $L_\omega = \bigcup_i L_i$. There is a coherent L_ω-structure $\mathfrak{M}_\omega$ that extends $\mathfrak{M}_0$, in which every term of L_ω with a designated state variable has an iterator symbol, and in which $I_{\mathrm{id}}(c, 0) = 1 \neq 0$ for the elements c of the second summand of $\mathfrak{M}_0$.*

Proof. Given the coherent L_i-structure $\mathfrak{M}_i$, let $\mathfrak{M}_{i+1}$ be the expansion to L_{i+1} of its ultrapower given by Theorem 10.75; the ultrapower is taken in the whole language L_i, and only descriptors over L_i receive symbols at this stage, so no body contains a symbol that is being introduced. Replace each ultrapower by an isomorphic copy that contains the preceding structure. Then for every j the reducts $(\mathfrak{M}_i | L_j)_{i \geq j}$ form an elementary chain, and by the elementary chain theorem the reduct to L_j of the union $\mathfrak{M}_\omega$ is an elementary extension of each of them. Every successor condition, every computation axiom and every instance of (Ext) mentions finitely

many symbols, hence is a sentence of some L_j that holds in $\mathfrak{M}_j$, and so holds in $\mathfrak{M}_\omega$. The equation $I_{\mathrm{id}}(c,0)=1$ and the inequation $1\neq 0$ hold in $\mathfrak{M}_0$ and are preserved. A term of L_ω is a term of some L_i and has a symbol in L_{i+1}. □

10.9.2 Native terms in the tower

Translate native terms into L_ω by $x^*=x$, $0^*=0$, $(St)^*=St^*$ and

$$I(n,a;\,u.\,h)^* = I_{h^*}(n^*,a^*,\vec{p}),$$

where $\vec{p}$ lists the free variables of h other than u in a fixed order. Inner iterators are translated first, so h^* is a term of some L_i and its symbol belongs to L_{i+1}; the number of stages is the nesting depth of iterators. Parameters are numerical variables; there is no sort of functions and no internal evaluator. The value $[\![t]\!]_\rho$ of a native term under an assignment ρ in $\mathfrak{M}_\omega$ is the value of t^*. The body u is the descriptor of I_{id}, so $[\![I(y,0;\,u.\,u)]\!]_\rho = I_{\mathrm{id}}(\rho(y),0)$.

Lemma 10.78 (Substitution)**.** *For native terms t,s, capture-avoiding substitution and every assignment ρ in $\mathfrak{M}_\omega$,*

$$[\![t[s/x]]\!]_\rho = [\![t]\!]_{\rho[x\mapsto [\![s]\!]_\rho]},$$

and the value of a term does not change under renaming of bound variables. The same holds for native formulas under the ordinary readings of Q_y and $D_{x,y}$ with actual equality.

Proof. Induction on t. The cases of variables, 0 and S are immediate. Let $t=I(n,a;\,u.\,h)$ with u renamed away from x and s. The translation of $t[s/x]$ is $I_g(\ldots,\vec{q})$ for the descriptor $g=(h[s/x])^*$, whose passive variables $\vec{q}$ are those of h other than x together with those of s; in general g is a different descriptor from h^*. By the induction hypothesis the counter and the seed have the required values, and for every element v

$$[\![h[s/x]]\!]_{\rho[u\mapsto v]} = [\![h]\!]_{\rho[x\mapsto [\![s]\!]_\rho][u\mapsto v]},$$

so the unary function of g at the parameters $\rho(\vec{q})$ equals the unary function of h^* at the parameters given by $\rho[x\mapsto [\![s]\!]_\rho]$. (Ext) for the pair I_g, I_{h^*} gives equal iterator values. Renaming of the bound variable is the same argument with equal functions. The statement for formulas follows by induction on formulas. □

This is the point at which independent choices for written bodies would be insufficient: the two descriptors are syntactically unrelated, and only (Ext), imposed for all parameter tuples and preserved through all stages, identifies their iterators.

10.9.3 The theorem

Theorem 10.79 (Progress is not admissible)**.** *Let $E(y) := (I(y,0;\,u.\,u)=0)$.*

(i) N^- *derives $E(0)$ and the sequent $E(z)\vdash E(Sz)$.*

(ii) Every sequent derivable in N^- *is valid in $\mathfrak{M}_\omega$ under the ordinary readings, with actual equality.*

(iii) $Q_yE(y)$ is false in $\mathfrak{M}_\omega$, and so is $D_{x,y}E(y)$ with x unused.

Hence N^- *does not derive $Q_yE(y)$, which* N *derives by one application of* QI*: the rule* QI *is not admissible over the other seven rules, and* DI-PROGRESS *is not admissible over the other rules of the paired-introduction form. The countermodel is a classical structure with actual equality in which the successor is injective, coverage and separation hold, there is no successor cycle, and both computation equations hold for every finite iterator term at every count.*

Proof. (i) $E(0)$ is the zero computation. For the step write $a := I(z, 0; u.\, u)$ and $b := I(Sz, 0; u.\, u)$, and work in the context $E(z)$, that is $a = 0$. The derivation uses only primitive rules:

(1) HYP: $a = 0$.

(2), (3) The successor computation, twice: $b = a$, since the body u with a substituted for u is a.

(4) TR with $s = b$, $t = a$, the constant template $h(v) = b$ and $r = a$; both premises, $s = t$ and $h(s) = r$, are the equation $b = a$ of (2) and (3), and the conclusion $r = h(t)$ is $a = b$.

(5) TR with $s = a$, $t = 0$, the identity template $h(v) = v$ and $r = b$; the premises are $a = 0$ from (1) and $a = b$ from (4), and the conclusion $r = h(t)$ is $b = 0$, that is $E(Sz)$.

One application of QI to (5) gives $Q_y E(y)$ in N. As a proof tree this has six nodes: one HYP leaf, two successor computation leaves, two TR nodes and one QI node. No derived equality rule and no induction lemma is used; derived reflexivity, symmetry and transitivity (Section 1.4) are not needed.

(ii) A sequent is valid if every assignment that satisfies its context satisfies its conclusion. HYP is immediate. The zero computation is the computation axiom for I_{h^*}. For the successor computation, the left side has the value $h^*(I_{h^*}(n, a, \vec{p}), \vec{p})$ by the computation axiom, and the right side $h[I(n, a; u.\, h)/u]$ has the same value by Theorem 10.78. Transport, from $s = t$ and $h(s) = r$ to $r = h(t)$, is sound because equality is actual and $[\![h(s)]\!]_\rho = [\![h(t)]\!]_\rho$ by Theorem 10.78. QE is modus ponens and instantiation, with Theorem 10.78 for the instance $B(t)$. DE is existential elimination: the value of the fresh variable is the witness, and freshness keeps the context and the conclusion unchanged. For DI by cases let ρ satisfy Γ and take the value of t as witness. Suppose $B(t, 0)$ holds under ρ. Every element is 0 or Sd for some d; in the second case $\rho[z \mapsto d]$ still satisfies Γ and $B(t, 0)$ and gives t the same value, since z is fresh, so the premise gives $B(t, Sz)$ there. Hence $B(t, y)$ holds for every y. All seven arguments are first-order soundness arguments in one structure; coverage is used for DI only.

(iii) By the translation, $E(y)$ says $I_{\mathrm{id}}(y, 0) = 0$. It holds at $y = 0$ and fails at $y = c$ by Theorem 10.77. So $E(0) \to \forall y\, E(y)$ is false, and with an unused witness coordinate so is $\exists x\, (E(0) \to \forall y\, E(y))$. □

With the seven independence results recorded in the README (`NativeModels`), each of the eight rule families of N is independent of the other seven.

The model also satisfies extensionality in the update, stated in the native term language. This strengthens the list of laws at the end of Theorem 10.79 from the computation equations to the whole first-order theory of extensional computation.

Corollary 10.80 (Extensional computation does not give induction)**.** *Let T_{ext} be the classical first-order theory with equality over the native terms whose axioms are: $Sx = Sy \to x = y$, $Sx \neq 0$, $x = 0 \vee \exists y\, x = Sy$, $S^k x \neq x$ for each $k \geq 1$, the two computation equations for every iterator term, and, for every two native bodies h and g with the free variables $\vec{p}$ of h and $\vec{q}$ of g other than u,*

$$\forall \vec{p} \forall \vec{q} \Big[\forall u\, (h = g) \to \forall n \forall a\, \big(I(n, a; u.\, h) = I(n, a; u.\, g)\big) \Big].$$

Then T_{ext} does not prove $E(0) \wedge \forall y\, (E(y) \to E(Sy)) \to \forall y\, E(y)$ for $E(y) := \big(I(y, 0; u.\, u) = 0\big)$. A fortiori no intuitionistic subtheory of T_{ext} proves it.

Proof. All axioms hold in $\mathfrak{M}_\omega$ under the translation of native terms. The successor laws and coverage are successor conditions of Theorem 10.72, and the computation equations were verified in part (ii) of the proof of Theorem 10.79. For the extensionality axiom fix values

of $\vec{p}$ and $\vec{q}$ under an assignment ρ. The value of h at $\rho[u \mapsto v]$ is $(h^*)^{\mathfrak{M}_\omega}(v, \rho(\vec{p}))$, and similarly for g, so the antecedent says that the unary functions of h^* at $\rho(\vec{p})$ and of g^* at $\rho(\vec{q})$ coincide. The translations of the two iterator terms are $I_{h^*}(n, a, \rho(\vec{p}))$ and $I_{g^*}(n, a, \rho(\vec{q}))$, and (Ext) for the pair I_{h^*}, I_{g^*} gives their equality. The displayed induction implication is false in $\mathfrak{M}_\omega$: $E(0)$ holds, every $E(y) \to E(Sy)$ holds because $I_{\mathrm{id}}(Sy, 0) = I_{\mathrm{id}}(y, 0)$, and $E(c)$ fails by Theorem 10.77. □

Equality in $\mathfrak{M}_\omega$ is actual, so transport of equals in arbitrary formulas is also sound there; strengthening TR to full equality substitution leaves the countermodel intact.

Remark 10.81 (What the construction needs). Four requirements carry the proof, and dropping any of them leaves a gap. The iterator of a function that is not old is chosen once for the function, not for a written body. (Ext) is imposed for all pairs of symbols and all parameter tuples, so that it transfers to the parameters of later ultrapowers. Each ultrapower is taken in the whole current language, so that the earlier iterators and their coherence are carried along. And a stage adds symbols only for descriptors over the preceding language, so that the orbit lemma applies to them. The construction makes no assertion about iteration of arbitrary functions; Theorem 10.73 is false for arbitrary maps.

Remark 10.82 (A countable model). The languages L_i and L_ω are countable. By the downward Löwenheim–Skolem theorem $\mathfrak{M}_\omega$ has a countable elementary substructure that contains c; it satisfies the same sentences, hence is coherent and refutes $E(c)$, and the soundness argument applies to it. This does not contradict the obstruction to readings over the natural numbers: the successor of the countable carrier has nonstandard chains and is not the map $n \mapsto n+1$ under any enumeration that would make it a reading in that sense. No computable presentation of the model is claimed. The second write-up of the package proves countability by the same argument, and so does not add to the statement here.

Remark 10.83 (The second construction: saturation in place of ultrapowers). The second write-up of `progress_independence` builds the same kind of model with three differences, none of which changes the theorem.

(a) The seed is $\mathbb{N} \sqcup \mathbb{Z}$ with the identity iterator $G(n, a) = a$ on the first summand and $G(n, a) = Sa$ on the second, and with a constant c for an element of the second summand. The computation equations hold because S preserves the summands, and $G(c, 0) = S0 \neq 0$.

(b) Two-sided orbits come from ω-saturation instead of Theorem 10.73. Let M be ω-saturated and f a unary function defined by a term with finitely many parameters. The type $\{\exists x\, f^m(x) = y : m \geq 1\}$ is finitely satisfiable, by $y = f^N(w)$ for any w, so some b_0 realizes it. If b_j realizes it, the type $\{f(x) = b_j\} \cup \{\exists w\, f^m(w) = x : m \geq 1\}$ over the parameters of f and b_j is finitely satisfiable as well: for the requirements up to N choose w with $f^{N+1}(w) = b_j$ and put $x = f^N(w)$, so that $x = f^m(f^{N-m}(w))$ for $m \leq N$. A realization is b_{j-1}, again with predecessors of every finite depth, and dependent choice gives all negative indices; the positive indices are $b_{j+1} = f(b_j)$ for $j \geq 0$. The orbit need not be injective.

(c) Each stage takes an ω-saturated elementary extension of the preceding expansion, and the extension step groups the presented functions by equality: a class that contains a function with an old iterator keeps it, which is unambiguous by (Ext), and every other class receives one iterator $J_f(S^m 0, a) = f^m(a)$ on the standard ray and $J_f(S^i d_C, a) = b_i$ on each copy C of $\mathbb{Z}$ with origin d_C, for one orbit of f chosen independently of the seed. All templates without a symbol receive symbols interpreted by these iterators, so (Ext) holds between old and new symbols and among new ones, as in Theorem 10.75.

The write-up states the outcome in the generality of Theorem 10.75 and Theorem 10.77: a structure in a countable language with the successor conditions and a countable coherent family of iterators has an elementary extension with an expansion by coherent iterators for all finite-depth templates, preserving the old iterators. The difference shows that two-sided orbits of term-defined functions are a compactness phenomenon: every ω-saturated structure has them, and the ultrapower of Theorem 10.73, which is $\aleph_1$-saturated for a countable language, is one instance. The proof of the second write-up is complete; each of its steps is checked above or coincides with a step of Theorems 10.75, 10.77 and 10.78 and of the soundness argument in the proof of Theorem 10.79.

Proposition 10.84 (A seed with cancellative addition)**.** *Let $M_0 = \{(0,n) : n \in \mathbb{N}\} \cup \{(k,z) : k \geq 1,\ z \in \mathbb{Z}\}$ with componentwise addition, $0 = (0,0)$, $1 = (0,1)$ and $S(k,z) = (k, z+1)$. Let $I_{\mathrm{id}}(n,a)$ be a if the first coordinate of n is 0 and 1 otherwise, and let $K(n,a) = a+n$ be the iterator symbol of the descriptor Su. Then M_0 is coherent for the language $\{0, S, I_{\mathrm{id}}, K\}$, $K(n,0) = n$ holds, addition $a+n = K(n,a)$ is associative, commutative and cancellative with the unit 0, and $I_{\mathrm{id}}((1,0),0) = 1$. The tower over M_0 gives a countermodel as in Theorem 10.79 that satisfies these laws in addition.*

Proof. M_0 is a submonoid of the group $\mathbb{Z}^2$, so addition is associative, commutative and cancellative; it is conical, and $Sx = x+1$. The successor is injective, omits 0, has no cycle, and every nonzero element has a predecessor in M_0. The computation axioms of I_{id} hold because S preserves the first coordinate, and those of K are $a+0 = a$ and $a+Sn = S(a+n)$. (Ext) between the two symbols has a false antecedent, since the identity and the successor are different functions. The displayed laws are first-order sentences of the seed language and survive the tower. □

The stronger seed shows that the countermodel does not depend on a defect of addition. It does not separate uniqueness of iteration from induction: in M_0 the term $f(n) = n + I_{\mathrm{id}}(n,0)$ satisfies $f(0) = 0$ and $f(Sn) = Sf(n)$, and $f(c) = Sc \neq c = K(c,0)$ at $c = (1,0)$, so the instance of the rule UR of Section 10.13 for the update Su fails there. The instance of UR for the identity update and the constant template $f(z) = 0$ already excludes the witness of the theorem, since its conclusion is $0 = I(t, 0;\, u.\, u)$.

10.9.4 Fixed points and seed shift do not give induction

This subsection draws on the mathematical part of `ipra_fixed_points_and_checker`. The models of Theorems 10.79 and 10.84 refute the identity-iterator scheme $I(n, a;\, u.\, u) = a$. The subsection refines the extension step so that two laws of iteration survive the tower, both theorems of iPRA proved by induction on the counter, and starts the tower from a seed in which an induction instance fails whose base and step follow from these laws. For a descriptor $h(u, \vec{p})$ write $I_h(n, a, \vec{p})$ as in Theorem 10.72. The two schemes are

$$\forall \vec{p} \forall a \left[h(a, \vec{p}) = a \to \forall n \left(I_h(n, a, \vec{p}) = a \right) \right], \tag{FP}$$

$$\forall \vec{p} \forall n \forall a \left[I_h(n, h(a, \vec{p}), \vec{p}) = h(I_h(n, a, \vec{p}), \vec{p}) \right]. \tag{Shift}$$

(FP) says that iteration preserves every fixed seed of its update; for the body u it is the scheme $I(n, a;\, u.\, u) = a$. (Shift) moves one application of the update from the seed to the output. Both compare iterators of one update; fusion, which compares iterators of different updates, is treated after the theorem. A coherent structure is *fixed-point coherent* if it satisfies every instance of (FP) and (Shift) for its iterator symbols.

Lemma 10.85 (Extension by orbit maps)**.** *Let A be a fixed-point coherent L-structure, and let $B = A^{\mathbb{N}}/\mathcal{U}$ and $\mathcal{D}$ be as in Theorem 10.75. Then B is fixed-point coherent, and B has an expansion to $L \cup \{I_h : h \in \mathcal{D}\}$ that is fixed-point coherent for the enlarged language. The expansion changes no element and no function of B.*

Proof. (FP) and (Shift) are schemes of first-order sentences with all parameters bound, so, like the conditions of Theorem 10.72, they hold in B by the theorem of Łoś, also at the parameter tuples that are new in B.

Orbit maps. Let $f = h^B(-, \vec{p})$ be a presented function, with h a term of L and $\vec{p} = [\vec{p}_i]$, and put $f_i = h^A(-, \vec{p}_i)$. For $j \in \mathbb{Z}$ define $H_j \colon B \to B$ by

$$H_j([a_i]) = [f_i^{\max(0,i+j)}(a_i)],$$

with finite external iteration in each coordinate. If $a_i = a_i'$ on a member of $\mathcal{U}$, the finite iterates agree on that member, and the same holds for a change of the representatives $\vec{p}_i$; so H_j is a well-defined map. It depends only on f: if also $f = g^B(-, \vec{q})$, Łoś applied to $\forall u\, h(u, \vec{p}) = g(u, \vec{q})$ gives a member of $\mathcal{U}$ on which f_i and g_i agree at every argument, hence so do their iterates of every finite length. The maps have three properties.

(a) $H_{j+1} = f \circ H_j$. Term functions of B are computed coordinatewise, and $\max(0, i+j)+1 = \max(0, i+j+1)$ for all $i \geq -j$; this cofinite set of coordinates is the same for every seed.

(b) $H_j \circ f = f \circ H_j$, since finite iterates of f_i commute with f_i.

(c) If $f(a) = a$, then $H_j(a) = a$: for $a = [a_i]$ the equation $f_i(a_i) = a_i$ holds on a member of $\mathcal{U}$, and there every finite iterate fixes a_i.

For a fixed seed a the family $(H_j(a))_{j \in \mathbb{Z}}$ is a two-sided orbit of f as in Theorem 10.73; the difference is that one family of maps serves all seeds at once.

The expansion. Assign It_f to every presented function f as in the proof of Theorem 10.75, with one change: if f is not old, put $It_f(S^k 0, a) = f^k(a)$ for $k \in \mathbb{N}$ and $It_f(S^j b, a) = H_j(a)$ for $j \in \mathbb{Z}$, for each copy of $\mathbb{Z}$ with origin b (Theorem 10.74), negative powers being iterated predecessors. Define I_h for $h \in \mathcal{D}$ as there. The computation axioms and (Ext) hold by the argument of Theorem 10.75: on a copy of $\mathbb{Z}$ property (a) replaces $x_{j+1} = f(x_j)$, and (Ext) uses only that It_f is assigned once for each function.

(FP) and (Shift). A symbol I_h at the parameters $\vec{p}$ is It_f for $f = h^B(-, \vec{p})$, and $h(a, \vec{p}) = f(a)$. It therefore suffices to show, for every presented f and all n, a,

$$It_f(n, f(a)) = f(It_f(n, a)), \qquad f(a) = a \;\Rightarrow\; It_f(n, a) = a.$$

If f is old, $f = g^B(-, \vec{q})$ with I_g in L and $It_f = I_g^B(-, -, \vec{q})$, and the two statements are instances of (Shift) and (FP) for I_g at $\vec{q}$, which hold in B. If f is not old, on the standard ray $f^k(f(a)) = f(f^k(a))$, and $f(a) = a$ gives $f^k(a) = a$; at $n = S^j b$ the two statements are (b) and (c). □

The choice of orbits in Theorem 10.75, made separately for each seed, does not give (Shift): the orbits chosen at a and at $f(a)$ are unrelated. The ultrapower supplies the maps H_j because the iteration length $\max(0, i+j)$ depends on the coordinate only, not on the seed.

Proposition 10.86 (The tower over the doubling seed)**.** *Let M_0 be the carrier of Theorem 10.84, $M_0 = \{(0, z) : z \geq 0\} \cup \{(q, z) : q \geq 1,\ z \in \mathbb{Z}\}$ with componentwise addition, $0 = (0, 0)$ and $S(q, z) = (q, z+1)$. Let $L_0 = \{0, S, J, K\}$, where J and K are the iterator symbols of the descriptors Su and $S(Su)$ without parameters, and put*

$$d(q, z) = (q, 2z), \qquad J(n, a) = a + n, \qquad K(n, a) = a + d(n).$$

Then M_0 is fixed-point coherent, it satisfies the sentences

$$\begin{aligned} &\forall n\ J(n,0)=n, \qquad \forall a\ J(0,a)=a, \qquad \forall a\,\forall b\ J(b,a)=J(a,b),\\ &\forall a\,\forall b\,\forall c\ J(c,J(b,a))=J(J(c,b),a), \qquad \forall a\,\forall b\,\forall c\ \big(J(c,a)=J(c,b)\to a=b\big), \end{aligned} \tag{10.6}$$

and $K(c,0)\neq J(c,c)$ at $c=(1,0)$. The tower of Theorem 10.77 over M_0, with Theorem 10.85 in place of Theorem 10.75 at every stage, gives a fixed-point coherent L_ω-structure $\mathfrak{M}_\omega^{\mathrm{fp}}$ that extends M_0, in which every term of L_ω with a designated state variable has an iterator symbol, and which satisfies (10.6) *and $K(c,0)\neq J(c,c)$.*

Proof. The successor conditions hold in M_0 as in the proof of Theorem 10.84. The map d keeps the first coordinate and keeps the second coordinate nonnegative when the first is 0, so it maps M_0 into M_0; moreover $d(0)=0$ and $d(Sn)=S^2d(n)$. Since $Sx=x+(0,1)$,

$$J(0,a)=a, \quad J(Sn,a)=SJ(n,a), \qquad K(0,a)=a, \quad K(Sn,a)=a+S^2d(n)=S^2K(n,a),$$

which are the computation axioms for the bodies Su and $S(Su)$. (Shift) holds: $J(n,Sa)=SJ(n,a)$ and $K(n,S^2a)=S^2K(n,a)$. (FP) is vacuous, since S and S^2 have no fixed point. The mixed instances of (Ext) have a false antecedent, because $S0\neq S^20$; the two self-instances have no parameters and are tautologies. The sentences (10.6) say that $J(n,0)=0+n=n$ and that $a+b=J(b,a)$ is the addition of a submonoid of $\mathbb{Z}^2$, which is commutative, associative and cancellative with unit 0. Finally $K(c,0)=d(1,0)=(1,0)$ and $J(c,c)=(2,0)$.

The proof of Theorem 10.77 applies without change: every instance of (FP) and (Shift) mentions finitely many symbols, so it is a sentence of some L_j that holds in $\mathfrak{M}_j$ and is preserved by the elementary chain. The sentences (10.6) and the inequation, with the parameter c of M_0, hold in M_0 and are preserved in the same way. □

Native terms are translated into L_ω as in the subsection on native terms in the tower; the body Su translates to the descriptor of J, the body $S(Su)$ to that of K, and the body u receives its symbol at the first stage. Theorem 10.78 and its proof hold for $\mathfrak{M}_\omega^{\mathrm{fp}}$ unchanged, since they use only the computation axioms and (Ext).

Theorem 10.87 (Fixed points and seed shift do not give induction)**.** *In the native term language put*

$$J(n,a):=I(n,a;\,u.\,Su), \qquad K(n,a):=I(n,a;\,u.\,S(Su)), \qquad a{+}b:=J(b,a), \qquad E(n):=(K(n,0)=J(n,n$$

(i) Under the translation and with actual equality, $\mathfrak{M}_\omega^{\mathrm{fp}}$ satisfies: $Sx=Sy\to x=y$, $Sx\neq 0$, $x=0\vee\exists y\,x=Sy$ and $S^kx\neq x$ for each $k\geq 1$; both computation equations for every iterator term; the extensionality axioms of Theorem 10.80; for every native body h, the universal closures of

$$h[a/u]=a\to I(n,a;\,u.\,h)=a, \qquad I(n,h[a/u];\,u.\,h)=h[I(n,a;\,u.\,h)/u];$$

the law $I(n,0;\,u.\,Su)=n$; and the laws of a commutative cancellative monoid with unit 0 for $+$.

(ii) The computation equations, the second scheme of (i) for the body Su, and equality reasoning prove $E(0)$ and $E(n)\to E(Sn)$, without induction.

(iii) $E(c)$ is false in $\mathfrak{M}_\omega^{\mathrm{fp}}$ for $c=(1,0)$.

Consequently the classical first-order theory T_{fix} whose axioms are the statements listed in (i) proves $E(0)$ and $\forall n\,(E(n)\to E(Sn))$ but not $\forall n\,E(n)$. It does not prove the induction instance $E(0)\wedge\forall n\,(E(n)\to E(Sn))\to\forall n\,E(n)$, and neither does any intuitionistic subtheory of T_{fix}. Preservation of all fixed seeds, which contains $I(n,a;\,u.\,u)=a$, seed shift for every update, the canonical successor iterator and cancellative addition do not replace induction for literal equations over computation, coverage and separation.

Proof. (i) The successor laws and coverage are successor conditions of Theorem 10.72. The computation equations follow from the computation axioms and Theorem 10.78 as in part (ii) of the proof of Theorem 10.79, and the extensionality axioms as in the proof of Theorem 10.80. For the two schemes fix an assignment ρ and let $f = (h^*)^{\mathfrak{M}^{\mathrm{fp}}_\omega}(-, \rho(\vec{p}))$, where $\vec{p}$ are the free variables of h other than u. By Theorem 10.78 the value of $h[a/u]$ is $f(\bar{a})$, where $\bar{a}$ is the value of a, and the value of $I(n, a;\, u.\, h)$ is $I_{h^*}(\bar{n}, \bar{a}, \rho(\vec{p}))$; the value of $h[I(n, a;\, u.\, h)/u]$ is f applied to it. The two schemes are therefore the instances of (FP) and (Shift) for I_{h^*} at $\rho(\vec{p})$. The terms $I(n, 0;\, u.\, Su)$ and $a + b$ translate to $J(n, 0)$ and $J(b, a)$, so the last two items are (10.6).

(ii) $E(0)$: the zero computations give $K(0, 0) = 0$ and $J(0, 0) = 0$, and symmetry and transitivity give $K(0, 0) = J(0, 0)$. Step: the successor computation for J and the scheme for Su at the seed n give

$$J(Sn, Sn) = SJ(n, Sn) = SSJ(n, n).$$

The successor computation for K, the hypothesis $E(n)$ and congruence give

$$K(Sn, 0) = SSK(n, 0) = SSJ(n, n) = J(Sn, Sn).$$

The seed-shift instance can be replaced by commutativity of $+$ and the computation equation: $J(n, Sa) = J(Sa, n) = SJ(a, n) = SJ(n, a)$.

(iii) The translation of $E(c)$ is $K(c, 0) = J(c, c)$, which is false by Theorem 10.86.

The classical structure $\mathfrak{M}^{\mathrm{fp}}_\omega$ is a model of T_{fix} by (i), and a sound test of intuitionistic derivability (Section 10.1.1). If T_{fix} or a subtheory proved the induction instance, then with (ii) T_{fix} would prove $\forall n\, E(n)$, which is false by (iii). □

Corollary 10.88 (The native form)**.** *Let* $\mathsf{N}^-_{\mathrm{fix}}$ *be* N^- *with two further rule families, for every native body* h *and all terms* n, a*:*

$$\Gamma \vdash I(n, h[a/u];\, u.\, h) = h[I(n, a;\, u.\, h)/u] \quad \textit{SHIFT}, \qquad \frac{\Gamma \vdash h[a/u] = a}{\Gamma \vdash I(n, a;\, u.\, h) = a} \quad \textit{FP}.$$

With E *as in Theorem 10.87:*

(i) $\mathsf{N}^-_{\mathrm{fix}}$ *derives* $E(0)$ *and the sequent* $E(z) \vdash E(Sz)$*.*

(ii) Every sequent derivable in $\mathsf{N}^-_{\mathrm{fix}}$ *is valid in* $\mathfrak{M}^{\mathrm{fp}}_\omega$ *under the ordinary readings, with actual equality.*

(iii) $Q_y E(y)$ *is false in* $\mathfrak{M}^{\mathrm{fp}}_\omega$*.*

Hence $\mathsf{N}^-_{\mathrm{fix}}$ *does not derive* $Q_y E(y)$*, and* QI *is not admissible over* $\mathsf{N}^-_{\mathrm{fix}}$*. In particular* N^- *extended by the leaves* $\Gamma \vdash I(n, a;\, u.\, u) = a$*, which* $\mathsf{N}^-_{\mathrm{fix}}$ *derives by* FP *from reflexivity (Theorem 1.18(i)), does not derive* $Q_y E(y)$*.*

Proof. (i) The equality rules of Theorem 1.18 use only TR and I0, so they are available in N^-. $E(0)$: two I0 leaves, symmetry and transitivity. For the step, in the context $E(z)$: IS gives $K(Sz, 0) = SSK(z, 0)$; HYP and congruence in the template SSv give $SSK(z, 0) = SSJ(z, z)$; IS gives $J(Sz, Sz) = SJ(z, Sz)$; the SHIFT leaf for the body Su, the counter z and the seed z gives $J(z, Sz) = SJ(z, z)$, and congruence in the template Sv gives $SJ(z, Sz) = SSJ(z, z)$; symmetry and transitivity give $K(Sz, 0) = J(Sz, Sz)$.

(ii) For the seven rules of N^- the argument of part (ii) of the proof of Theorem 10.79 applies verbatim; it uses only the coherence of the structure, Theorem 10.78 and coverage. The SHIFT leaves are valid by Theorem 10.87(i). For FP, if ρ satisfies Γ, the premise gives $h[a/u] = a$ at ρ, and the first scheme of Theorem 10.87(i) gives $I(n, a;\, u.\, h) = a$ at ρ.

(iii) $Q_y E(y)$ reads $E(0) \to \forall y\, E(y)$; $E(0)$ is true and $E(c)$ is false. □

Remark 10.89 (Scope of the fixed-point model). The step of Theorem 10.88(i) uses the SHIFT leaf for the body Su, or, by the alternative in the proof of Theorem 10.87, commutativity of $+$. Neither is shown derivable in N^- extended by the identity scheme $I(n, a;\, u.\, u) = a$ alone. The corollary shows that the conclusion $Q_y E(y)$ is underivable in that extension, but not that the premise $E(z) \vdash E(Sz)$ is derivable there, so it does not by itself show that QI fails to be admissible over N^- with the identity scheme; that is shown with another equation and another model in Theorem 10.95. In $\mathfrak{M}_\omega^{\mathrm{fp}}$ the term $n + I(n, 0;\, u.\, u)$ that refutes the uniqueness rule for the successor update over the seed of Theorem 10.84 equals n by (FP), so that refutation does not apply; whether SU, the uniqueness rule for the update Su (Theorem 2.74), is valid in $\mathfrak{M}_\omega^{\mathrm{fp}}$ is not known. The result is a separation of principles over the native scalar interface under the ordinary reading. It is not a smaller axiomatization of iPRA and not a lower bound on the size of a checker, and it does not exclude a bi-interpretation through a different definable domain. The package checks the seed algebra of Theorem 10.86 on finitely many coordinates by a script; the tower and the native interpretation rest on the written proof alone, and no Lean or Rocq verification is claimed.

The missing strength sits at the comparison of different updates. For unary term templates f, g, h, which may have parameters, the *fusion rule* is

$$\frac{\Gamma \vdash f(g(z)) = h(f(z))}{\Gamma \vdash f\big(I(t, a;\, u.\, g(u))\big) = I(t, f(a);\, v.\, h(v))}\ \textsc{fusion},$$

where z is not free in Γ, t, a or the parameters, and the three templates contain z only through the displayed argument. It is read, like UR (Section 10.13), in intuitionistic logic with equality and the two computation equations.

Proposition 10.90 (Fusion and uniqueness)**.** *Over intuitionistic logic with equality and the computation equations:*

(i) UR *derives* FUSION*;*

(ii) FUSION *and the scheme* $I(t, 0;\, u.\, Su) = t$ *derive* UR*;*

(iii) FUSION *derives both schemes of Theorem 10.87(i): from* $\Gamma \vdash h(a) = a$ *it derives* $\Gamma \vdash I(t, a;\, u.\, h(u)) = a$*, and it derives* $h\big(I(t, a;\, u.\, h(u))\big) = I(t, h(a);\, v.\, h(v))$*.*

Hence, in the presence of $I(t, 0;\, u.\, Su) = t$*,* FUSION *and* UR *are interderivable.*

Proof. (i) Assume the premise of FUSION. Let w be a variable not free in Γ, t, a or the templates, and put $r(n) := f\big(I(n, a;\, u.\, g(u))\big)$. The successor computation and congruence give $r(Sw) = f\big(g(I(w, a;\, u.\, g(u)))\big)$. Substituting $I(w, a;\, u.\, g(u))$ for the variable z, which is not free in Γ, in the premise gives $f\big(g(I(w, a;\, u.\, g(u)))\big) = h\big(f(I(w, a;\, u.\, g(u)))\big) = h(r(w))$. So $r(Sw) = h(r(w))$, and UR for the template r gives $r(t) = I(t, r(0);\, v.\, h(v))$. The zero computation gives $r(0) = f(a)$, and congruence in the seed gives the conclusion of FUSION.

(ii) Take $g(u) := Su$ and $a := 0$. The premise of FUSION is then $f(Sz) = h(f(z))$, the premise of UR. Its conclusion is $f\big(I(t, 0;\, u.\, Su)\big) = I(t, f(0);\, v.\, h(v))$, and the scheme and congruence turn the left side into $f(t)$.

(iii) For fixed points take the constant template $f(w) := a$, any g, for instance $g(u) := Su$, and the seed a. The premise $a = h(a)$ follows from $h(a) = a$ by symmetry, and the conclusion is $a = I(t, a;\, v.\, h(v))$. For seed shift take $f = g = h$: the premise $h(h(z)) = h(h(z))$ is reflexivity, and the conclusion is the displayed equation. □

With $I(t, 0;\, u.\, Su) = t$, which UR derives (Theorem 10.125(iii)), the proposition and Theorem 2.76 show that T_{eq} with induction for literal equations replaced by FUSION and this

scheme has the same theorems as T_{eq}: FUSION follows in T_{eq} from Theorem 2.75 and (i). Consequently no structure with actual equality that satisfies the computation equations, coverage, separation, the scheme $I(t, 0;\, u.\, Su) = t$ and every instance of FUSION refutes an induction instance for a literal equation; an instance is satisfied if, at each value of its parameters, the premise at all values of z implies the conclusion at all values of t and a, and then UR is sound with arbitrary contexts because z is fresh. So (FP) and (Shift), which FUSION derives, are strictly weaker over this background, and Theorem 10.87 is the witness. The model violates one mixed instance: the doubling template $f(w) := J(w, w)$ satisfies $f(Sz) = SSf(z)$ by Theorem 10.87(ii), which is the premise of FUSION for $g(u) := Su$, $h(v) := S(Sv)$ and $a := 0$, and also the premise of UR for the update $S(Su)$. With $f(0) = 0$ and $I(t, 0;\, u.\, Su) = t$, both conclusions read $J(t, t) = K(t, 0)$, which fails at c. Adding this one instance excludes the model; it is not shown to give induction. Neither FUSION nor the scheme $I(t, 0;\, u.\, Su) = t$ can be omitted from part (ii) of Theorem 10.90, even in the presence of induction against every fixed standard numeral (Section 10.9.5).

10.9.5 Numeral induction, numbering and fusion

This subsection draws on `iteration_barriers`. It separates the two hypotheses of Theorem 10.90(ii) by two classical structures with actual equality, coverage, separation and an injective successor, both built over a nonstandard model of Peano arithmetic, and it turns their automorphisms into obstructions to bi-interpretation with iPRA. The first structure refines the two-fiber model of Theorem 9.145; the second has no counterpart earlier in the article.

The theories. Let B be the ordinary theory over the native terms with intuitionistic first-order logic with equality (Section 1.5), the two computation equations, coverage and separation; it is T_{eq} without induction. Write $\overline{k} = S^k 0$. T_{num} is B with $Sx = Sy \to x = y$ and, for every term $t(n, \vec{p})$ and every $k \in \mathbb{N}$ fixed in the metatheory, the numeral induction rule

$$\frac{\Gamma \vdash t(0, \vec{p}) = \overline{k} \qquad \Gamma,\ t(z, \vec{p}) = \overline{k} \vdash t(Sz, \vec{p}) = \overline{k}}{\Gamma \vdash t(s, \vec{p}) = \overline{k}}$$

with z fresh for Γ, s and $\vec{p}$; there is no predicate for standardness in the language. Consider the laws

$$I(n, a;\, u.\, u) = a \quad (\mathsf{Id}), \qquad I(n, \overline{k};\, u.\, Su) = S^k n \quad (\mathsf{N}_k,\ k \in \mathbb{N}),$$

and the extensionality axioms (Ext) of Theorem 10.80. N_0 is the scheme $I(t, 0;\, u.\, Su) = t$ of Theorem 10.90(ii). Put

$$T_{\mathrm{cal}} := T_{\mathrm{num}} + \mathsf{Id} + \{\mathsf{N}_k : k \in \mathbb{N}\} + \mathsf{Ext}, \qquad T_{\mathrm{fus}} := T_{\mathrm{num}} + \mathsf{Id} + \mathsf{Ext} + \text{FUSION},$$

with FUSION as displayed before Theorem 10.90. All these axioms and rules are derivable in T_{eq}: Id, N_k and Ext by induction on the displayed equation, FUSION by Theorem 2.75 and Theorem 10.90(i), and injectivity from the predecessor of Theorem 1.65. In T_{fus} the laws Id and Ext are redundant: Id is Theorem 10.90(iii) for the body u, whose premise $a = a$ is reflexivity, and under the antecedent $\forall u\,(h = g)$ of an extensionality axiom, instantiation at a fresh z gives $h(z) = g(z)$, which is the premise of FUSION for the identity template f and the two updates h, g. A classical structure *validates* a theory if it satisfies its axioms and every rule is valid in it in the sense stated before Theorem 6.119; it then satisfies every theorem, and is a sound test of intuitionistic derivability (Section 10.1.1).

Theorem 10.91 (Numbering does not repair numeral induction)**.** *Let* $E(p, n) := \big(I(n, Sp;\, u.\, Su) = SI(n, p;\, u.\, Su)\big)$.

(i) B *proves* $E(p,0)$ *and* $E(p,n) \to E(p,Sn)$*, using only the computation equations and equality reasoning.*

(ii) There is a classical structure $\mathcal{D}$ *with actual equality that validates* T_{cal} *and in which* $\forall n\, E(p,n)$ *is false at some value of* p*.*

(iii) $\mathcal{D}$ *has an automorphism* τ *of order two, of the full staged language, that fixes every standard numeral and is not the identity.*

Hence T_{cal} *does not prove* $\forall p \forall n\, E(p,n)$*, and induction for literal equations is not admissible in* T_{cal}*.*

Proof. (i) $E(p,0)$: the zero computations give $I(0,Sp;\, u.\, Su) = Sp$ and $I(0,p;\, u.\, Su) = p$, and congruence gives $Sp = SI(0,p;\, u.\, Su)$. Step: by the successor computation, the hypothesis $E(p,n)$ under S, and the successor computation again,

$$I(Sn,Sp;\, u.\, Su) = SI(n,Sp;\, u.\, Su) = SSI(n,p;\, u.\, Su) = SI(Sn,p;\, u.\, Su).$$

(ii) Let M, D, π, 0_D, S_D, $\mathscr{H}$, It_f, the nonstandard chains $C = (n_j)_{j\in\mathbb{Z}}$ and the fibers $B_j = \pi^{-1}(\mathrm{It}_f(\pi(n_j),\pi(a)))$ be as in the proof of Theorem 9.145; a *path* for a triple (h,a,C) is a family $b_j \in B_j$ with $b_{j+1} = h(b_j)$. Only the choice of paths changes. Let $\tau\colon D \to D$ fix $(m,0)$ for standard m and exchange $(m,0)$ and $(m,1)$ for nonstandard m. Then $\tau^2 = \mathrm{id}$, $\pi\tau = \pi$, $\tau(0_D) = 0_D$ and $\tau S_D = S_D\tau$. If $h \in \mathscr{H}$ with projected function f, then $\tau h\tau \in \mathscr{H}$ with the same f. The label is constant along a nonstandard chain C, so τC is the chain over the same chain of M with the other label, and $\tau C \neq C$.

Prescribed paths. Fix a nonstandard $b \in M$ and put $a_0 = (b,0)$, $a_1 = (b,1)$. On every nonstandard chain $C = (n_j)$ prescribe:

(a) for $h = \mathrm{id}_D$ and every seed a, the constant path $b_j = a$;

(b) for $h = S_D$ and the seed $\overline{k}_D = S_D^k 0_D$, $k \in \mathbb{N}$, the path $b_j = S_D^k n_j$;

(c) for $h = S_D$ and the four seeds a_0, $S_D a_0$, a_1, $S_D a_1$, the paths $b_j = (\pi(n_j)+b,0)$, $(\pi(n_j)+Sb,1)$, $(\pi(n_j)+b,1)$ and $(\pi(n_j)+Sb,0)$ respectively, with the addition of M.

These are paths. In (a) $\mathrm{It}_{\mathrm{id}}(\pi(n_j),\pi(a)) = \pi(a)$ and the identity fixes a. In (b) and (c) the projected iteration of the successor from k, b or Sb is $k+\pi(n_j)$, $b+\pi(n_j)$ or $Sb+\pi(n_j)$; in (c) these first coordinates are nonstandard, so both labels exist; and S_D raises the first coordinate by one and keeps the label. The prescribed triples are pairwise distinct: the identity and the successor are different functions, the seeds of (b) are standard, and the four seeds of (c) are distinct and nonstandard. The involution $(h,a,C) \mapsto (\tau h\tau, \tau a, \tau C)$ maps the set of prescribed triples onto itself, and the prescription of the image is the image under τ of the prescription: $\tau\,\mathrm{id}_D\,\tau = \mathrm{id}_D$ and the constant path at τa is τ of the constant path at a; $\tau S_D\tau = S_D$, $\tau\overline{k}_D = \overline{k}_D$ and $S_D^k(\tau n_j) = \tau(S_D^k n_j)$; and in (c) the involution exchanges a_0 with a_1 and $S_D a_0$ with $S_D a_1$, whose prescribed values have equal first coordinates and opposite labels.

The remaining paths. The involution on triples has no fixed point, since $\tau C \neq C$. Choose one triple (h,a,C) from each two-element orbit that is not prescribed, a path (b_j) for it by Theorem 9.146, and for the partner $(\tau h\tau, \tau a, \tau C)$ the family that takes the value τb_j at τn_j. It is a path: $\tau h\tau(\tau b_j) = \tau b_{j+1}$, and $\pi(\tau b_j) = \pi(b_j) = \mathrm{It}_f(\pi(\tau n_j),\pi(\tau a))$. Define $\mathcal{I}(n,a,h)$ as $h^q(a)$ at a standard counter $\overline{q}_D$ and as the chosen path at nonstandard counters. Then $\mathcal{I}$ satisfies the three displayed equations of the proof of Theorem 9.145, and

$$\mathcal{I}(\tau n, \tau a, \tau h\tau) = \tau\,\mathcal{I}(n,a,h) \qquad (n,a \in D,\ h \in \mathscr{H}), \tag{10.7}$$

at standard counters because $\tau\overline{q}_D = \overline{q}_D$ and $(\tau h\tau)^q(\tau a) = \tau(h^q(a))$, and at nonstandard counters by the construction.

Validity. Terms are interpreted as in the proof of Theorem 9.145; (9.37) holds by the same induction, every body function lies in $\mathscr{H}$, the substitution lemma and both computation schemas hold, and the argument given there for an equation $\overline{k} = t(n, \vec{a})$, with an arbitrary context, validates the numeral induction rule. Injectivity, coverage and separation are shown there. The value of an iterator depends only on the actual body function, so Ext holds. Id holds at standard counters by finite iteration and at nonstandard ones by (a). N_k holds at $\overline{q}_D$ because $S^q_D \overline{k}_D = S^k_D \overline{q}_D$, and at nonstandard counters by (b). Finally let $p \mapsto a_0$ and let n be nonstandard. By (c) the left side of $E(a_0, n)$ is $(\pi(n) + Sb, 1)$, and the right side is $S_D(\pi(n) + b, 0) = (\pi(n) + Sb, 0)$.

(iii) By induction on terms, $\tau[\![t]\!]_\rho = [\![t]\!]_{\tau\circ\rho}$ for every assignment ρ. Variables, 0 and S are immediate. For $t = I(n, a;\, u.\, h)$ let G and G' be the body functions at ρ and at $\tau \circ \rho$. The induction hypothesis for h at $\rho[u \mapsto v]$ gives $G'(\tau v) = \tau G(v)$, so $G' = \tau G \tau$; the counter and the seed have the values $\tau[\![n]\!]_\rho$ and $\tau[\![a]\!]_\rho$; and (10.7) gives the claim. So τ commutes with the interpretation of every function symbol of the staged language. It fixes the standard numerals and moves a_0. □

The proof of Theorem 9.147 applies to $\mathcal{D}$ verbatim, since it uses only (9.37), the one-element fiber over zero and two distinct points with a common projection. Hence T_{cal} proves for no term $e(x, y)$ the law $e(x, y) = 0 \leftrightarrow x = y$, and the reduction of equation induction to zero-equation induction through such a term is not available in T_{cal}.

The second structure lives on pairs. Fix a nonstandard model M of Peano arithmetic expanded by symbols for all primitive recursive descriptions, write St for its standard part, and put

$$D_2 := \{(m, m) : m \in \mathrm{St}\} \cup \{(m, m') : m, m' \in M \setminus \mathrm{St}\} \subseteq M^2,$$

with $0 = (0, 0)$ and the coordinatewise successor. Native terms are first interpreted on all of M^2: variables, 0 and S as stated, and $I(n, a;\, u.\, h)$ at an assignment ρ as the internal iterate in M, of length the first coordinate n_1 of $[\![n]\!]_\rho$, of the function $v \mapsto [\![h]\!]_{\rho[u\mapsto v]}$, starting at $[\![a]\!]_\rho$. By induction on the term every term defines, with its parameters, a function on M^2 given by a primitive recursive description on the two coordinates, so the internal iterate exists at every count in M. Write $x_{i,1}$, $x_{i,2}$ for the coordinates of a pair x_i.

Lemma 10.92 (Provenance)**.** *For every term $t(x_1, \ldots, x_r)$ there are functions σ and d of $x_{1,1}, \ldots, x_{r,1}$ alone, primitive recursive in M with parameters, with $\sigma \in \{0, \ldots, r\}$ and $d \in M$, such that on all of M^2*

$$[\![t]\!] = (x_{\sigma,1} + d,\ x_{\sigma,2} + d), \qquad x_0 := (0, 0).$$

Proof. Induction on t. A variable x_i has $\sigma = i$, $d = 0$; 0 has $\sigma = 0$, $d = 0$; S adds 1 to d. Let $t = I(n, a;\, u.\, h)$, and apply the induction hypothesis to a and to h, where h has the additional input u, read as the index $r+1$. Define internally, by primitive recursion on $j \in M$, a pair (σ_j, d_j): (σ_0, d_0) is the provenance of a; given (σ_j, d_j), evaluate the provenance (σ_h, d_h) of h at the first coordinates of the inputs and at the first coordinate $x_{\sigma_j,1} + d_j$ of the state; if $\sigma_h = r + 1$ put $(\sigma_{j+1}, d_{j+1}) = (\sigma_j, d_j + d_h)$, and otherwise (σ_h, d_h). All of this depends on first coordinates only. Induction in M on j proves that the j-th internal iterate equals $(x_{\sigma_j,1} + d_j,\ x_{\sigma_j,2} + d_j)$: in the first case h adds d_h to both coordinates of the state, and in the second it returns $x_{\sigma_h} + (d_h, d_h)$. Evaluate at $j = n_1$, the first coordinate of the counter, which by the induction hypothesis for n is again a function of first coordinates. □

Three consequences. First, every term function maps tuples of D_2 into D_2: adding a common d to a standard diagonal pair gives a diagonal pair, whose coordinates are both standard or both nonstandard, adding it to a pair of nonstandard coordinates keeps both nonstandard, and x_0 is diagonal. Second, for a body h with parameters in D_2 and a seed

$a \in D_2$, every internal iterate $h^j(a)$, $j \in M$, lies in D_2, since it is the value of the iterator at the counter $(j, j) \in D_2$. Third, by induction on t, the first coordinate of $[\![t]\!]$ is the value in M of t at the first coordinates, with the iterator of M; in the iterator case this is an induction in M on the count. Restricted to D_2 the interpretation is therefore a structure for the staged language, with actual equality.

Theorem 10.93 (Fusion does not give numbering)**.** *The structure D_2 validates T_{fus}, and $I(n, 0; u.\, Su) = n$ is false at every counter (c, c') with $c \neq c'$ nonstandard. For every finite tuple $\vec{c}$ of elements of D_2 there is an automorphism of D_2 of order two, of the full staged language, that fixes $\vec{c}$ and is not the identity.*

Proof. Background. The successor is injective and 0 is not a successor. A nonzero standard diagonal pair (m, m) is the successor of $(m-1, m-1)$, and a pair of nonstandard coordinates is the successor of the pair of their predecessors, again nonstandard; so coverage holds. The computation equations are the recursion equations of the internal iterate at the counts 0 and $n_1 + 1$. The interpretation is compositional on M^2, so the substitution lemma holds by induction on terms: substituting s for x in a body does not change the function on M^2 that it defines once x takes the value of s. Id holds because every internal iterate of the identity is the identity.

Numeral induction. The first projection maps D_2 onto M, through the diagonal, and its fiber over a standard k is $\{(k, k)\} = \{\overline{k}\}$. By the third consequence of Theorem 10.92, the argument for an equation $\overline{k} = t(n, \vec{a})$ in the proof of Theorem 9.145, with the first projection in place of π, validates the numeral induction rule with an arbitrary context.

Fusion. Fix an assignment that satisfies Γ; it fixes the values of the parameters of f, g, h and of t and a. Since the eigenvariable z is fresh, the premise gives $f(g(z)) = h(f(z))$ for every value $z \in D_2$. Write $a_j = g^j(a)$ for the internal iterates, $j \in M$. By the second consequence of Theorem 10.92 every a_j lies in D_2, so $f(g(a_j)) = h(f(a_j))$ for every $j \in M$. With the primitive recursive descriptions of f, g, h and of $j \mapsto a_j$ substituted, this is a statement of M that mentions no predicate for D_2, and it is true in M. Induction in M on j gives $f(a_j) = h^j(f(a))$: at $j = 0$ both sides are $f(a)$, and $f(a_{j+1}) = f(g(a_j)) = h(f(a_j)) = h(h^j(f(a)))$. At j the first coordinate of the value of t, this is the conclusion $f(I(t, a; u.\, g(u))) = I(t, f(a); v.\, h(v))$. Ext is derivable from FUSION, hence valid.

Numbering fails. At $n = (c, c')$ the iterator $I(n, 0; u.\, Su)$ is the c-fold internal successor of $(0, 0)$, that is $(c, c) \neq (c, c')$.

Involutions. For distinct positive standard r, s define on M^2

$$\tau_{r,s}(m, m') = \begin{cases} (m, m + s), & m' = m + r, \\ (m, m + r), & m' = m + s, \\ (m, m'), & \text{otherwise.} \end{cases}$$

The cases are disjoint, $\tau_{r,s}$ is an involution, it fixes the first coordinate and every diagonal pair, and by cancellation in M it commutes with adding a common $d \in M$ to both coordinates. It maps D_2 onto D_2: the standard elements of D_2 are diagonal and fixed, and a moved pair has a nonstandard first coordinate, so both second coordinates are nonstandard. By Theorem 10.92, applying $\tau_{r,s}$ to all arguments of a term function changes neither σ nor d, which depend on first coordinates, so $[\![t]\!](\tau_{r,s}\vec{x}) = \tau_{r,s}(x_\sigma) + (d, d) = \tau_{r,s}([\![t]\!](\vec{x}))$, also when $\sigma = 0$. Hence the restriction to D_2 is an automorphism of the staged language. Given $\vec{c}$, let Q be the finite set of positive standard q such that some component of $\vec{c}$ has the form $(m, m + q)$, and choose $r \neq s$ outside Q. Then $\tau_{r,s}$ fixes $\vec{c}$, and it exchanges $(m, m + r)$ and $(m, m + s)$ for every nonstandard m. □

Corollary 10.94 (Obstructions to bi-interpretation). *(i) No theory validated by D_2, in particular T_{fus}, is bi-interpretable with* iPRA *in the sense of Theorem 10.1, with arbitrary definable domains, quotient equality, finite dimension and finite parameter tuples, provided the interpretations and the return isomorphism exist in D_2; this covers a parameter condition that the theory proves inhabited.*

(ii) No theory validated by $\mathcal{D}$, in particular T_{cal}, is bi-interpretable with iPRA *in that sense by a bi-interpretation whose parameters, those of the first interpretation, of the return isomorphism and representatives of those of the second interpretation, can be chosen in $\mathcal{D}$ among the standard numerals; in particular by no parameter-free bi-interpretation.*

Proof. As in the proof of Theorem 10.22: a hypothetical bi-interpretation, applied in the structure, yields a classical model of iPRA; collect the parameters $\vec{c}$ as stated. In (i) Theorem 10.93, and in (ii) Theorem 10.91(iii), gives an automorphism of order two that fixes $\vec{c}$ and is not the identity. Its image under the homomorphism of Theorem 10.21 has order dividing two and preserves the parameter-free definable order of Theorem 10.20, so it is the identity, and injectivity of the homomorphism gives a contradiction. □

The automorphism τ of $\mathcal{D}$ moves every nonstandard element, so (ii) says nothing about bi-interpretations that need a nonstandard parameter. Neither part excludes equiconsistency or a one-way interpretation.

The witness of Theorem 10.93 is parameter-free and its step needs no auxiliary law, which settles the question left open in Theorem 10.89. Let $\mathsf{N}^-_{\mathrm{id}}$ be N^- with the leaves $\Gamma \vdash I(n, a; u.\, u) = a$ for all terms n, a.

Corollary 10.95 (Progress over the identity scheme). *Let $E(y) := \big(I(y, 0; u.\, Su) = y\big)$.*

(i) N^- *derives $E(0)$ and the sequent $E(z) \vdash E(Sz)$.*

(ii) Every sequent derivable in $\mathsf{N}^-_{\mathrm{id}}$, also when the rule FUSION *with native contexts and the rule* QI *restricted to matrices $t(y) = \overline{k}$, $k \in \mathbb{N}$, are added, is valid in D_2 under the ordinary readings, with actual equality.*

(iii) $Q_y E(y)$ is false in D_2.

Hence $\mathsf{N}^-_{\mathrm{id}}$ does not derive $Q_y E(y)$, and QI *is not admissible over N^- with the identity scheme, nor over that extension.*

Proof. (i) $E(0)$ is a zero computation leaf. In the context $E(z)$: the successor computation gives $I(Sz, 0; u.\, Su) = SI(z, 0; u.\, Su)$, HYP and congruence in the template Sv give $SI(z, 0; u.\, Su) = Sz$, and transitivity gives $E(Sz)$. The equality rules of Theorem 1.18 use only TR and I0, as in the proof of Theorem 10.88.

(ii) For the seven rules of N^- the argument of part (ii) of the proof of Theorem 10.79 applies: it uses actual equality, the computation equations, the substitution lemma and coverage, which hold in D_2 by the proof of Theorem 10.93. The identity leaves hold since D_2 satisfies Id. FUSION is valid by Theorem 10.93. For QI at a matrix $t(y) = \overline{k}$, let ρ satisfy Γ; the premise holds at every value of the fresh z, so if $t(0) = \overline{k}$ holds at ρ then numeral induction gives $t(y) = \overline{k}$ at every value of y, which is the reading of Q_y.

(iii) $Q_y E(y)$ reads $E(0) \to \forall y\, E(y)$; $E(0)$ holds and E fails at every (c, c') with $c \neq c'$ nonstandard. □

The structure $\mathcal{D}$ gives a second witness with a parameter: N^- derives $E(p, z) \vdash E(p, Sz)$ for the equation of Theorem 10.91, by the derivation in part (i) of its proof, and N^- with the identity leaves, the leaves $\Gamma \vdash I(n, \overline{k}; u.\, Su) = S^k n$ and QI restricted to matrices $t(y) = \overline{k}$ is sound in $\mathcal{D}$ by the same argument, while $Q_y E(p, y)$ fails at $p \mapsto a_0$.

Remark 10.96 (Comparison and scope). The equation $E(p,n)$ of Theorem 10.91 is the instance of (Shift) for the body Su. It is also an instance of SU (Theorem 2.74): for $f(z) := SI(z,p;\, u.\, Su)$ the successor computation and congruence give $f(Sz) = Sf(z)$, and $f(0) = Sp$, so SU gives $SI(t,p;\, u.\, Su) = I(t, Sp;\, u.\, Su)$. Hence $\mathcal{D}$ refutes (Shift) and SU; it is no witness for Theorem 2.168, and it is complementary to the model of Theorem 10.87, which satisfies (FP), (Shift) and N_0 and is not shown to satisfy numeral induction. In Theorem 9.145 the identity iterator is the specified family $(\pi(a), 0)$, so Id fails there; Theorem 10.91 keeps the base and adds Id, every N_k and the equivariance. The structure $\mathcal{M}$ of Theorem 6.119 also validates FUSION and refutes N_0, but its successor is not injective and coverage fails there; D_2 has both, together with numeral induction. With Theorems 10.87 and 10.90 the picture over B is: FUSION with N_0 recovers induction for literal equations; FUSION, Id, Ext and numeral induction without N_0 do not; N_0 with every N_k, Id, Ext and numeral induction does not; and N_0 with (FP), (Shift), Ext and cancellative addition does not.

The package also states the positive equivalence: over B, equation induction proves FUSION and N_0, and these two prove UR, hence equation induction by Theorem 2.76. This is Theorem 10.90(i), (ii) with the discussion after it and is not restated; the package derives FUSION from equation induction directly, by induction on $f(I(n,a;\, u.\, h(u))) = I(n, f(a);\, u.\, k(u))$, in place of the route through UR, and it notes that the converse half of Theorem 2.76 is a written proof that is not part of the Lean formalization. The constructions of the subsection are written proofs in classical metamathematics with a nonstandard model of Peano arithmetic and, for $\mathcal{D}$, the axiom of choice and Theorem 9.146; none of these principles enters the object theories. No Lean or Rocq verification is claimed, and the package's test suites concern inherited checkers only.

10.9.6 No successor cycle without progress

The countermodel has no successor cycle, and it cannot have one. Let T^- be intuitionistic logic with equality, the two computation equations and coverage; there is no induction. Write $Z(x) := \mathrm{sel}(x, 1, 0)$, so $Z(0) = 1$ and $Z(Sx) = 0$, write $\overline{j}$ for $S^j 0$, and $h^{[r]}$ for the r-fold composition of a unary term, which introduces no iterator. All inductions below are on the integer that specifies a term.

Lemma 10.97 (Finite clocks)**.** *For every $m \geq 2$ there is a term $h_m(x)$, built from the leaves $\overline{0}, \ldots, \overline{m-1}$ by the selector with arbitrary counter terms, such that T^- proves*

$$\bigvee_{j<m} h_m(x) = \overline{j}, \qquad h_m(\overline{j}) = \overline{j+1}\ (j < m-1), \qquad h_m(\overline{m-1}) = 0.$$

Proof. Let $h_2 = Z$; coverage gives $Z(x) = 0 \vee Z(x) = 1$. Given h_m, put $T_m(x) := I(x, 0;\, u.\, h_m(u))$. By coverage on x and the two computations, $T_m(x)$ is 0 or a value of h_m, so it is one of $\overline{0}, \ldots, \overline{m-1}$; nothing is asserted about the preceding iterator value. Finite computation gives $T_m(\overline{j}) = \overline{j}$ for $j < m$ and $T_m(\overline{m}) = 0$. For $1 \leq r < m$ put

$$d_{m,r}(z) := Z\big(h_m^{[m-r]}(z)\big), \qquad V_{m,0}(z) := 0, \qquad V_{m,r}(z) := \mathrm{sel}\big(d_{m,r}(z),\, V_{m,r-1}(z),\, \overline{r+1}\big),$$

and $V_m := V_{m,m-1}$. Ground computation gives $d_{m,r}(\overline{s}) = 1$ for $s = r$ and $d_{m,r}(\overline{s}) = 0$ for $s < m$, $s \neq r$; hence $V_m(0) = 0$ and $V_m(\overline{r}) = \overline{r+1}$ for $1 \leq r < m$. At an arbitrary argument V_m takes one of the values $0, \overline{2}, \ldots, \overline{m}$, by coverage on the selector counters. Put

$$h_{m+1}(x) := \mathrm{sel}\big(x,\, 1,\, V_m(T_m(x))\big).$$

Its values are among $\overline{0}, \ldots, \overline{m}$. At 0 its value is 1; at $\overline{j}$ with $1 \leq j < m$ it is $V_m(\overline{j}) = \overline{j+1}$; at $\overline{m}$ it is $V_m(0) = 0$. □

For example $h_3(x) = \mathrm{sel}(x, 1, \mathrm{sel}(Z(Z(T_2(x))), 0, 2))$ with $T_2(x) = I(x, 0;\, u.\, Z(u))$.

Lemma 10.98 (Separation of numerals)**.** *For $m \geq 2$ and distinct $i, j < m$, T^- proves $\overline{i} = \overline{j} \to 0 = 1$, without injectivity of the successor.*

Proof. Apply $h_m^{[r]}$ with $r \equiv -i \pmod m$, $0 \leq r < m$. The result is $0 = \overline{s}$ with $1 \leq s < m$, and Z gives $1 = 0$. □

Theorem 10.99 (No finite successor cycle)**.** *For every $k \geq 1$ and every term c, T^- proves $S^k c = c \to 0 = 1$, and N^- derives the sequent $S^k c = c \vdash 0 = 1$.*

Proof. Let $m = k + 1$ and $v = T_m(c)$. The hypothesis and k successor computations give $v = T_m(S^k c) = h_m^{[k]}(v)$. By Theorem 10.97, $v = \overline{j}$ for some $j < m$, and the equation becomes $\overline{j} = \overline{(j + k) \bmod m}$. Since $k = m - 1$ the two residues differ, and Theorem 10.98 gives $0 = 1$.

For N^- the finite disjunctions are removed. The cases rule for equation templates, from $E(0)$ and $E(Sz)$ to $E(t)$, is derivable in N^-: DI by cases with an unused witness, DE and QE (`NoInd.cases`). First, if $E(\ell)$ is derivable for each leaf ℓ of a term built from finitely many leaves by the selector, then E of the term is derivable, by induction on the term with the cases rule at each selector counter. Second, since h_m is such a term over $\overline{0}, \ldots, \overline{m-1}$, the cases rule at the counter of T_m gives: if $E(\overline{j})$ is derivable for every $j < m$, then $E(T_m(c))$ is derivable; the zero branch is $E(0)$ and the successor branch is $E(h_m(T_m(z)))$. Third, put $b_r(a) := Z(h_m^{[(-r) \bmod m]}(a))$ for $r < m$, so that $b_r(\overline{j}) = 1$ exactly when $r = j$, and

$$\mathrm{xor}(p, q) := \mathrm{sel}(p, q, Z(q)), \qquad \mathrm{or}(p, q) := \mathrm{sel}(p, q, 1),$$

and let $D_m(a, b)$ be the left-nested or, starting at 0, of the terms $\mathrm{xor}(b_r(a), b_r(b))$, $r < m$. The cases rule for the template $\mathrm{xor}(Z(w), Z(w)) = 0$, instantiated at the arguments of Z in the terms $b_r(a)$, and the computation $\mathrm{or}(0, 0) = 0$ give $D_m(a, a) = 0$ for every a. Ground computation gives $D_m(\overline{j}, h_m^{[k]}(\overline{j})) = 1$ for $j < m$, so the second step gives $D_m(v, h_m^{[k]}(v)) = 1$ at $v = T_m(c)$. Under the hypothesis $v = h_m^{[k]}(v)$, and transport gives $D_m(v, v) = 1$, hence $0 = 1$. No disjunction, no characteristic of equality of arbitrary numbers and no QI is used. □

The README records the odd case in Lean (`NoInd.no_odd_cycle`, `NoInd.no_succ_fixed`) and leaves the even case open, because it needs a counter of another period whose behavior has to be established without induction; the clocks h_m are such counters. The theorem is a constraint on countermodels and does not make QI redundant.

Remark 10.100 (What is proved and what is not). Theorem 10.79 is an independence result for one rule of one fixed kernel, with the identity translation of formulas. It is proved by a written model construction that uses a nonprincipal ultrafilter and the axiom of choice. It is not formalized: the README lists the package as filed and not formalized, and the Lean readings of `NativeModels` fix the standard successor, so they cannot express the model. The result does not bound the size of a checker, does not concern presentations in which QI is fused with another rule or replaced by a weaker induction interface, and does not show that the eight rules are a minimum: equivalent presentations with fewer rule families exist. It does not decide whether the uniqueness rule for the successor update is equivalent to induction for literal equations, since the seed of Theorem 10.84 refutes that uniqueness instance; the refined model of Theorem 10.87 is not known to satisfy it (Theorem 10.89).

10.10 The progressive binder presentation

This section draws on `research`. It shows that the calculus of Theorem 10.7, with BOOL-CASES, is a complete presentation of iPRA, so that Theorem 10.17 isolates the one rule that

cannot be removed. Structural weakening, contraction, exchange, substitution, renaming and assumption cut are admissible by induction on derivations, with eigenparameters renamed first.

Lemma 10.101 (Identity and equation induction, without cases)**.** *The case-free calculus derives $t = t$, symmetry, transitivity and numerical congruence; $\Gamma, A \vdash A$ for every object formula A; and contextual induction for literal equations: from $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$ with z fresh, $\Gamma \vdash E(t)$.*

Proof. The computation $\mathsf{R}(0, t; u, v.0) = t$, used as the major and as the minor of TR with the template $w = t$, gives $t = t$; the templates $w = s$, $s = w$ and $f(s) = f(w)$ give the other equality rules. An available equation $s = t$ transports $s = s$ to $s = t$. For an available $\mathsf{H}_{x,y}B$, induct on the binder depth: HE supplies a fresh a and $\mathsf{P}_y B(a, y)$; reintroduce the formula by HI at the witness a, where the local assumption $B(a, z)$ yields a proof of itself by the induction hypothesis and STEP gives $B(a, Sz)$. For induction, HI with an unused witness gives $\mathsf{H}_{x,y}E(y)$, HE supplies $\mathsf{P}_y E(y)$, and ITER gives $E(t)$. □

Lemma 10.102 (Boolean cases and constructor cases)**.** *Put $P(n) = \mathsf{R}(n, 0; u, v.u)$ and $C(n, a, b) = \mathsf{R}(n, a; u, v.b)$. The case-free calculus derives*

$$C(\mathrm{sg}(x), 0, x) = x, \qquad C(\mathrm{sg}(x), x, S(Px)) = x.$$

Over the case-free calculus, BOOLCASES is equivalent to the rule

$$\frac{\Gamma,\ t = 0 \vdash A \quad \Gamma,\ t = Sz \vdash A}{\Gamma \vdash A}\ \textit{CASES}, \qquad z \text{ fresh for } \Gamma, t, A.$$

Proof. Both identities follow by literal-equation induction; each base is a zero computation and each successor step is a successor computation that does not use its induction hypothesis. Hence $\mathrm{sg}(t) = 0$ gives $t = 0$ and $\mathrm{sg}(t) = 1$ gives $t = S(Pt)$. For CASES from BOOLCASES, substitute $P(t)$ for z in the second premise. Conversely CASES gives BOOLCASES by the equations of sg. □

The missing principle is thus the two-value range of sg; equation induction supplies the numerical identities.

Lemma 10.103 (Binder congruence)**.** *If $A(x, y)$ and $B(x, y)$ are interderivable uniformly in x, y, with x, y not in the retained context, then $\mathsf{H}_{x,y}A$ and $\mathsf{H}_{x,y}B$ are interderivable. No cases and no compound induction are used.*

Proof. From $\mathsf{H}_{x,y}A$, HE gives $\mathsf{P}_y A(a, y)$. Introduce $\mathsf{H}_{x,y}B$ at the witness a: convert the local $B(a, z)$ to $A(a, z)$, apply STEP, convert $A(a, Sz)$ to $B(a, Sz)$. □

Definition 10.104 (Gates, selectors and macros)**.** With bound variables renamed apart, define by recursion on formulas

$$\begin{aligned} G_v(s = t) &:= (C(v, 0, s) = C(v, 0, t)), & G_v(\mathsf{H}_{x,y}A) &:= \mathsf{H}_{x,y}G_v A, \\ M_v(s = t,\ r = w) &:= (C(v, s, r) = C(v, t, w)), & M_v(\mathsf{H}_{x,y}A, \mathsf{H}_{x,y}B) &:= \mathsf{H}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{H}_{x,y}B) &:= \mathsf{H}_{x,y}M_v(G_y E, B), & M_v(\mathsf{H}_{x,y}A, E) &:= \mathsf{H}_{x,y}M_v(A, G_y E), \end{aligned}$$

for equations E; the maximal binder depth decreases in each recursive call. Let $\mathsf{T} = (0 = 0)$ and let p, q, d be the terms with

$$p(0) = 0,\ p(Sn) = C(p(n), 1, 0); \quad q(0) = 0,\ q(Sn) = C(p(n), q(n), S(q(n))); \quad d(0) = 0,\ d(Sn) = SS(d(n)),$$

which are parity, half and doubling in $\mathbb{N}$. The logical macros are

$$\begin{aligned} A \Rightarrow_{\mathsf{H}} B &:= \mathsf{H}_{u,z} M_z(A,B), & \exists_{\mathsf{H}} x\, A(x) &:= \mathsf{H}_{x,z} G_z A(x), \\ A \wedge_{\mathsf{H}} B &:= \mathsf{H}_{u,z} G_z M_{p(z)}(A,B), & A \vee_{\mathsf{H}} B &:= \exists_{\mathsf{H}} x\, M_x(A,B), \\ \forall_{\mathsf{H}} x\, A(x) &:= \mathsf{H}_{u,z} M_{p(z)}(\mathsf{T}, A(q(z))), & \bot_{\mathsf{H}} &:= (0 = S0), \end{aligned}$$

with all unused coordinates fresh. These are translations of fixed finite syntax, not truth predicates.

Lemma 10.105. *The calculus of Theorem 10.7 derives:*

(i) $G_0 A$, *and* $G_{Sv} A$ *is interderivable with* A;

(ii) A *is interderivable with* $\mathsf{H}_{u,y} G_y A$ *for fresh* u, y;

(iii) $M_0(A,B)$ *is interderivable with* A *and* $M_{Sv}(A,B)$ *with* B;

(iv) $p(d(n)) = 0$, $p(Sd(n)) = 1$, $q(d(n)) = n$ *and* $q(Sd(n)) = n$.

Items (i)–(iii) hold in the case-free calculus.

Proof. (i) By induction on the formula, with the computations of C at equations and Theorem 10.103 at binders; for G_0 at a binder, HI ignores its local assumption and uses the induction hypothesis for the successor matrix. (ii) Forward, HI ignores its assumption and uses $G_{Sz} A$ from A. Backward, HE supplies the progress clause, $G_0 A$ is a theorem, and one STEP gives $G_1 A$, hence A. (iii) By computation, (ii) and congruence. (iv) Induct on the single equation $p(d(n)) = 0$: at a successor two parity updates send 0 to 1 and back. Then $p(Sd(n)) = 1$ by one computation. Induct on the single equation $q(d(n)) = n$: the two half updates at $d(n)$ and $Sd(n)$ first keep the value and then increment it, by the parity equations. Then $q(Sd(n)) = n$ by its defining update. Two single-equation inductions are used in sequence; no induction on a conjunction or a compound formula. □

Proposition 10.106 (All intuitionistic rules)**.** *The calculus of Theorem 10.7 derives the introduction and elimination rules of intuitionistic natural deduction for the macros of Theorem 10.104, with the usual eigenvariable conditions, explosion from* $\bot_{\mathsf{H}}$, *and coverage* $(n = 0) \vee_{\mathsf{H}} \exists_{\mathsf{H}} v\, (n = Sv)$. *No excluded middle for quantified formulas is used.*

Proof. By Theorem 10.102, CASES is available, and with Theorem 10.105(iii) it selects either endpoint of a selector at an arbitrary control, also when the context mentions the control; in particular proofs of A and of B give a proof of $M_t(A,B)$.

Implication. Given $\Gamma, A \vdash B$, apply HI; the local assumption is $M_z(A,B)$, and CASES on z gives A and then B at $z = 0$, and B at $z = Sw$; in both branches the successor matrix is B. For elimination, HE supplies the clause, A is its zero matrix, and STEP at 0 gives B.

Existential quantifier. From $A(t)$, HI at the witness t ignores the local gate. For elimination, HE gives a fresh a and the clause; the zero gate is a theorem and STEP at 0 gives $A(a)$.

Conjunction. If A and B are proved, CASES and the selector endpoints prove every successor matrix. Conversely the zero gate is a theorem, STEP at 0 gives the matrix at 1, which is B since $p(1) = 1$, and a second STEP gives the matrix at 2, which is A since $p(2) = 0$.

Disjunction. The introductions use the witnesses 0 and 1. For elimination, eliminate the existential at a fresh a and apply CASES to a.

Universal quantifier. Let $\Gamma \vdash A(x)$ with x fresh. Then T and $A(q(Sz))$ are proved, so CASES and the selector endpoints prove the successor matrix, and HI ignores its assumption. For specialization, HE gives the clause; the matrix at $d(t)$ is a theorem by Theorem 10.105(iv),

and STEP at $d(t)$ gives the matrix at $Sd(t)$, which is $A(t)$. No compound progress clause is iterated.

Falsity and coverage. Under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$; induction on formulas then gives every object formula, since at a binder HI ignores its assumption. CASES proves coverage by its two introductions. □

Theorem 10.107 (`research`)**.** *Let T_{H} be intuitionistic first-order logic with equality in the staged signature of* R*, with the computation equations, induction for literal equations, coverage and $0 = 1 \to \bot$. The calculus of Theorem 10.7 is sound in T_{H} under its ordinary readings and derives the translations of all rules and axioms of T_{H}. For every matrix B,*

$$\mathsf{H}_{x,y} B(x, y) \;\dashv\vdash\; \exists_{\mathsf{H}} x \, \forall_{\mathsf{H}} y \, (B(x, y) \Rightarrow_{\mathsf{H}} B(x, Sy)).$$

T_{H} and iPRA *have the same theorems in the staged primitive recursive language. The progressive calculus is equiconsistent and bi-interpretable with* iPRA*, with whole domains, actual equality and return isomorphisms with graph $x = y$.*

Proof. Soundness. HI, HE and STEP are logical rules under the readings; ITER is equation induction under the retained base; BOOLCASES is the range of sg, which follows from coverage.

Completeness. Theorem 10.106 gives the logical rules and coverage, Theorem 10.101 gives equation induction, and separation translates to the identity implication $(0 = 1) \Rightarrow_{\mathsf{H}} (0 = 1)$.

The composite. Forward, HE supplies a fresh a and the clause; at a fresh y assume $B(a, y)$ and use STEP; introduce the implication, the universal and the existential macro and close HE. Backward, eliminate the existential at a fresh a, specialize the universal at the eigenparameter z of HI, and eliminate the implication under the assumption $B(a, z)$. The clause composite is the universal progress formula. In T_{H} the gate, selector and clock equations prove the intended meaning of every macro; for the universal macro, the even-to-odd progress implications give every instance $A(n)$, and all instances of A make every progress implication true, with no induction on A. Induction on formulas, with Theorem 10.103, gives both formula composites.

Arithmetic. T_{H} has the computation equations of ordinary counter-aware primitive recursion, coverage, separation and equation induction. The scalar iterator is the case of R whose update ignores its counter, so the construction of a characteristic of actual equality and the derivation of quantifier-free induction from equation induction (Theorem 1.86) apply to the terms of the larger language, and T_{H} proves every quantifier-free induction instance. Counter-aware recursion compiles every primitive recursive description directly, in place of the scalar compiler. Numerical descriptions are identified with themselves, and equation induction identifies two fixed recursions with the same equations, so both domain isomorphisms have the graph $x = y$. □

Remark 10.108. The presentation has one unrestricted introduction and pays for it with the explicit BOOLCASES and with the distinction between STEP and ITER. The universal quantifier is recovered with one progress step through the clock p, q, d. No smaller complete checker for this presentation is established, and the presentation is not assigned the measurements of the checkers of its package.

10.11 The counter-congruence form of the calculus

This section draws on `counter_congruence`. General equality transport carries an arbitrary formula or term context as rule data. Here it is recovered from two specific operations: replacement of the complete left side of an equation, and congruence in the counter of a recursor. The result is a restriction of the primitive interface; it does not by itself give a shorter checker. Theorem 10.68 shows that root replacement alone is not enough, and Theorem 10.117 that counter congruence alone is not enough.

Definition 10.109 (The calculus $\mathcal{K}_{\mathrm{ctr}}$)**.** Terms are variables, 0 and the successor-seeded counter-aware recursor $\mathsf{R}_S(n, a; u, v.h)$, with $Sa := \mathsf{R}_S(0, a; u, v.0)$ and $1 := S0$. The computation schemas are

$$\mathsf{R}_S(0, a; u, v.h) = Sa, \tag{R0}$$
$$\mathsf{R}_S(Sn, a; u, v.h) = h[n/u][\mathsf{R}_S(n, a; u, v.h)/v]. \tag{RS}$$

Object formulas are $A ::= (s = t) \mid D_{x,y}A$. Contexts are finite lists of object formulas and internal clauses $Q_y B$; a clause is available by membership only, and an object formula by membership or by a proof, written $\Gamma \triangleright M$. The rules are HYP for object formulas, R0, RS,

$$\frac{\Gamma \triangleright s = t \quad \Gamma \vdash s = r}{\Gamma \vdash t = r}\ \mathrm{RL} \qquad \frac{\Gamma \triangleright s = t}{\Gamma \vdash \mathsf{R}_S(s, a; u, v.h) = \mathsf{R}_S(t, a; u, v.h)}\ \mathrm{CC}$$

and QE, DE, DI by cases and DI-PROGRESS as in N_{p}, with majors available in the sense of $\triangleright$. In CC the initializer, the update and the passive parameters are the same on both sides. There is no primitive reflexivity, no general transport and no other congruence. The designated contradiction is $0 = 1$.

Weakening, exchange, contraction, renaming, substitution of a term for a parameter and cut of an object assumption are admissible by induction on derivations; substitution preserves the matching initializer and update of CC and the matching root of RL. These are operations on proof syntax and do not replace equal terms inside a formula.

Lemma 10.110 (Equality algebra from root replacement)**.** *Reflexivity, symmetry and transitivity are derivable from RS and RL, without CC.*

Proof. For a term s put $c_s := \mathsf{R}_S(Ss, 0; u, v.u)$; RS gives $c_s = s$. With one copy as the major and one as the minor, RL gives $s = s$. An available $s = t$ with the minor $s = s$ gives $t = s$. For $s = t$ and $t = r$, symmetry gives $t = s$, and RL with this major and the minor $t = r$ gives $s = r$. Paths of equations are composed at their roots; no proper subterm changes. □

Lemma 10.111 (Numerical cases)**.** *For every object template $A(y)$, from $\Gamma \vdash A(0)$ and $\Gamma \vdash A(Sz)$ with z fresh, $\Gamma \vdash A(t)$ is derivable without either equality rule and without DI-PROGRESS.*

Proof. Weaken the second premise by $A(0)$ and apply DI by cases with the witness 0 and an unused coordinate x, which gives $D_{x,y}A(y)$. DE supplies the clause $Q_y A(y)$, and QE with the first premise gives $A(t)$. □

Lemma 10.112 (Counter representation of a context)**.** *Let $W(n) := \mathsf{R}_S(n, 0; u, v.u)$, so that $W(0) = S0$ and $W(Sn) = n$ by R0 and RS. For a numerical template $F(x)$ put*

$$H_F(n) := \mathsf{R}_S(n, F(0); u, v.S(F(Su))), \qquad J_F(n) := W(H_F(n)),$$

with fresh bound variables. Then $\mathcal{K}_{\mathrm{ctr}}$ derives $J_F(n) = F(n)$ for arbitrary n in every context, by cases and without DI-PROGRESS.

Proof. The update of H_F does not contain v, so R0 and RS give literally $H_F(0) = S(F(0))$ and $H_F(Sz) = S(F(Sz))$. Let q be 0 or Sz. CC for the recursor W gives $W(H_F(q)) = W(S(F(q)))$, and RS gives $W(S(F(q))) = F(q)$. Composition by Theorem 10.110 gives $J_F(q) = F(q)$. Theorem 10.111 for the equation $J_F(y) = F(y)$ gives the claim. The only change of a proper subterm is the one CC instance through W; replacing $H_F(q)$ by $S(F(q))$ under W without it would assume the congruence that is being derived. □

Theorem 10.113 (All numerical congruence from counter congruence)**.** *For every numerical template $F(x)$, from $\Gamma \triangleright s = t$ the calculus $\mathcal{K}_{\mathrm{ctr}}$ derives $\Gamma \vdash F(s) = F(t)$. Hence for templates U, V, from $\Gamma \triangleright s = t$ and $\Gamma \vdash U(s) = V(s)$ it derives $\Gamma \vdash U(t) = V(t)$.*

Proof. CC with the initializer $F(0)$ and the update $S(F(Su))$ gives $H_F(s) = H_F(t)$; a second CC for W gives $J_F(s) = J_F(t)$. Theorem 10.112 and Theorem 10.110 give $F(s) = J_F(s) = J_F(t) = F(t)$. For the second claim, $U(t) = U(s) = V(s) = V(t)$. □

Proposition 10.114 (Constant overhead in inference nodes)**.** *Count one node for each primitive inference, hypothesis lookups included, and count repeated subproofs repeatedly. If the supplied proof of $s = t$ has p nodes, the derivation of $F(s) = F(t)$ has $p+56$ nodes, independently of the syntactic depth of F.*

Proof. Reflexivity has three nodes; symmetry adds four nodes to its premise and transitivity five to the sum of its premises. Each of the two proofs of $J_F(q) = F(q)$ has eight nodes: a computation, a CC lifting, a second computation and the transitivity overhead. The cases construction adds DE, DI, QE and one clause lookup, so $J_F(t) = F(t)$ has twenty nodes. The two CC liftings of $s = t$ have $p + 2$ nodes, the reversed left representation $20 + 4$, the right one 20, and two transitivities give $(p + 2) + 24 + 20 + 5 + 5 = p + 56$. □

The count is not a bound on bytes or running time: term and formula labels contain copies of F. It is not a constant bound for the lifting to compound formulas below.

Theorem 10.115 (Transport in object formulas)**.** *For every object template $A(w)$, from $\Gamma \triangleright s = t$ and $\Gamma \vdash A(s)$ the calculus $\mathcal{K}_{\mathrm{ctr}}$ derives $\Gamma \vdash A(t)$, with no progress instance in the expansion. An available clause $Q_y B(s, y)$ together with an available $s = t$ has the force of $Q_y B(t, y)$: from a proof of $B(t, 0)$ one obtains $B(t, r)$ for every r.*

Proof. Induction on the number of binders of A, uniformly in contexts, terms and parameters. An equation is Theorem 10.113. Let $A(w) = D_{x,y} B(w, x, y)$ with the bound variables renamed apart. Eliminate $A(s)$ at a fresh a, which retains $Q_y B(s, a, y)$. Introduce $D_{x,y} B(t, x, y)$ by cases with the witness a and a fresh z: the local assumption $B(t, a, 0)$ is transported backward along $t = s$ by the induction hypothesis to $B(s, a, 0)$; the clause at Sz gives $B(s, a, Sz)$; the induction hypothesis transports it forward to $B(t, a, Sz)$. Close the introduction and the elimination. Both recursive uses concern the smaller matrix B, and the only introductions added are case introductions. For the clause, transport the base backward, specialize, and transport forward. □

Corollary 10.116 (Identity-language equivalence)**.** *Let $\mathcal{K}_{\mathrm{tr}}$ be $\mathcal{K}_{\mathrm{ctr}}$ with RL and CC replaced by unrestricted transport in object formulas. For every context Γ of object formulas and clauses and every object formula A, $\Gamma \vdash A$ in $\mathcal{K}_{\mathrm{ctr}}$ if and only if $\Gamma \vdash A$ in $\mathcal{K}_{\mathrm{tr}}$. The translations are effective and preserve end sequents literally.*

Proof. From $\mathcal{K}_{\mathrm{tr}}$, replace each transport by Theorem 10.115. Conversely RL is transport with the template $w = r$; reflexivity follows from a computation by the same instance; and CC is obtained by transporting the left occurrence of s in $\mathsf{R}_S(s, a; u, v.h) = \mathsf{R}_S(s, a; u, v.h)$ to t and applying symmetry. □

Since the language and the meaning of every primitive are unchanged, the bi-interpretation of $\mathcal{K}_{\mathrm{tr}}$ with iPRA, with whole domains, actual equality and identity return isomorphisms, transfers to $\mathcal{K}_{\mathrm{ctr}}$. In the successor-seeded basis the numerical bridge is short: with $C(n, a, b) := W(\mathsf{R}_S(n, a; u, v.Sb))$ and $P(n) := C(n, 0, W(n))$ one has $C(0, a, b) = a$, $C(Sn, a, b) = b$, $P(0) = 0$, $P(Sn) = n$, and for terms g and h the term $W(\mathsf{R}_S(n, g(\vec{p}); u, v.Sh(u, W(v), \vec{p})))$ satisfies the equations of the primitive recursion with base g and step h, without predecessor

search, pairing or induction. The package attributes this compiler, the logical macros and the lifting of Theorem 10.115 to earlier drafts; its own contribution is Theorem 10.112 and Theorem 10.113, with the computation schemas unchanged and the counter as the only congruence position. An earlier draft on fixed-position transport uses replacement in the initializer of a scalar iterator together with an additional first-step computation schema.

Theorem 10.117 (Root replacement is not derivable)**.** *Let $\mathcal{K}^{-}$ be $\mathcal{K}_{\mathrm{ctr}}$ without* RL*, with no reflexivity or other equality rule added. Then $\mathcal{K}^{-}$ does not derive $0 = 0$ from the empty context. In particular* RL *is not admissible in $\mathcal{K}^{-}$.*

Proof. Interpret the atom $s = t$ by a binary predicate, not assumed to be equality. The domain is $\{0, 1\}$, 0 denotes 0, and every recursion term denotes 1 under every assignment, whatever its counter, initializer, update and parameters; this is compatible with substitution and renaming. So Sa is constantly 1. Let $s = t$ be true exactly when s has the value 1; the right side is immaterial. Read $Q_y B$ and $D_{x,y} B$ by their ordinary readings.

R0, RS and the conclusion of CC have a recursion term on the left and are true. HYP, availability, QE and DE are sound as logical rules. For DI by cases let b be the value of the witness; a sound premise gives $B(b, 0) \to B(b, 1)$, because Sz denotes 1, and on the two-element domain this is $B(b, 0) \to \forall y\, B(b, y)$. For DI-PROGRESS, with parameters and witness fixed, a term $U(y)$ has one of the value functions $y \mapsto 0$, $y \mapsto 1$, $y \mapsto y$, so the truth set of a literal equation $U(y) = V(y)$ is $\varnothing$, $\{0, 1\}$ or $\{1\}$; in each case truth at 0 implies truth everywhere, and the closure holds for every witness. So every rule of $\mathcal{K}^{-}$ is sound, and $0 = 0$ is false. In $\mathcal{K}_{\mathrm{ctr}}$ two copies of $\mathsf{R}_S(S0, 0; u, v.u) = 0$ as major and minor of RL prove $0 = 0$. □

The theorem concerns the specified inference system. The interpretation is not a model of an ordinary theory whose equality carries its usual axioms, and the theorem does not exclude a bi-interpretation with a different equality or domain.

10.12 Constructed sources and certificate interfaces

This section draws on `constructed_computation` and `uniform_predecessor` for the computation interface, and on `checker_and_barriers`, `compact_kernel`, `local_scopes` and `research` for statements about their checkers. The object calculus is N_{p} with COMP throughout; the results change the certificate format, not the theory.

10.12.1 Successor sources constructed from predecessors

Earlier certificate formats submit an iterator source $[c, a, h]$ and must check that the counter c has successor shape before using a successor computation; deleting that check admits a false equation. The constructed-source formats submit the predecessor.

Definition 10.118. A *constructed computation packet* supplies a predecessor p, an initializer a, an update h and a mode. In base mode the checker generates $I(0, a;\, v.\, h) = a$. In step mode it constructs the successor itself and generates

$$I(Sp, a;\, v.\, h) = h[I(p, a;\, v.\, h)/v].$$

In `constructed_computation` the mode is an explicit flag in a previously unused packet field, and the generated equation is opened at the fresh parameter of the continuation. In `uniform_predecessor` the mode is read off the raw representation of p (an empty representation selects the base equation), the packet is marked by a null major, and the generated equation is opened with the image zero. In both, the recursive image $I(p, a;\, v.\, h)$ is validated as a numerical term only when opening the update requests it.

Lemma 10.119 (Known-instance discharge)**.** *If $w \notin \mathrm{FV}(\Gamma, A)$, $\Gamma \vdash H(0)$ and $\Gamma, H(w) \vdash A$, then $\Gamma \vdash A$.*

Proof. Rename the eigenparameters of the second derivation, substitute 0 for w, which leaves Γ and A unchanged, and cut with the first derivation. □

Lemma 10.120 (Boxed zero)**.** *Put $B_0(p) := I(0, p;\, w.\, 0)$. The base computation and the successor computations at predecessors of the form $B_0(p)$ derive every instance of the successor computation.*

Proof. The base computation gives $B_0(p) = p$. Compute at $B_0(p)$ and transport along $B_0(p) = p$ with the template $I(Sw, a;\, u.\, h) = h[I(w, a;\, u.\, h)/u]$. The argument holds for every element of the domain. □

Theorem 10.121 (Constructed computation supply)**.** *(i) In the format of* `constructed_computation`*, every successfully generated equation $H(w)$ has a zero instance $H(0)$ derivable from the computation equations and transport. Hence its continuation is admissible by Theorem 10.119.*

(ii) In the format of `uniform_predecessor`*, every successfully supplied equation is a theorem uniformly in every newly admitted parameter; no known-instance discharge is needed.*

(iii) Both formats derive exactly the closed theorems of N_{p} with COMP*. In the first, the step at the predecessor 0 is the packet with $p = 0$ in step mode. In the second, a literal 0 selects the base equation, and the step at 0 is obtained by Theorem 10.120.*

Proof. Validation separates into two obligations: the source counter is a successor, and a recursive image used in the result is a valid term. Construction establishes the first, and demand-driven substitution the second.

(i) In base mode, after $w = 0$ the initializer and its copy on the right agree by structural zero completion, and the equation is the base computation. In step mode, suppose first that opening the update requests the recursive image. The image $[p, a, h]$ is then validated at numerical root; successful validation is idempotent, so validating on demand gives the same substituted value as validating eagerly. The additional dummy slot below the constructed successor introduces no dependency, since root validation of the image rejects a meaningful reference to it. The two occurrences of the predecessor agree and the zero instance is the successor computation. If the update never requests its image, it is independent of the state on every meaningful path, and after zero completion the equation is $I(Sp_0, a_0;\, v.\, h_0) = h_0$ with h_0 independent of v, which is a successor computation without the unused image. The old goal and context were checked before w was allocated and do not contain it.

(ii) With a null major the continuation opens the equation with the image zero, and the newly admitted parameters stay rigid. For a predecessor with empty representation the decoded counter is zero and the two occurrences of the initializer agree, so the equation is a base computation. Otherwise the source counter is the constructed successor, a successor whatever the value of the predecessor. If the update uses its state, the image is checked at the old parameter floor and the opening lemma of the inherited codec identifies the right side with the update at the preceding iterator. If not, the update is independent of the state, and the successor computation with a state-independent update holds for every predecessor and initializer, including new parameters.

(iii) Soundness is (i), (ii) and the unchanged logical branches. For completeness, encode each instance of COMP by its packet and close its use by HYP; the other instructions are unchanged. □

The raw codec is inherited. Input is a JSON array that ends with the closed goal; four-field packets are consumed from the end; membership in the context closes a goal without a packet. One list grammar is read in two sorts, with arities standing for truth or zero, closure or successor, equation, and paired binder or iterator. Negative coordinates name allocated rigid parameters and positive coordinates name protected binder slots. An opener substitutes a checked image at exposed numerical coordinates. These conventions are syntax, not arithmetic operations.

The uniform format shortens certificates: a step at a scalar parameter is one record where the earlier flat format needs three (boxing, computation, transport). For the same named derivations and the delivered encoders the record counts are as follows; they are not lower bounds on shortest proofs.

Named derivation	Earlier format	Uniform predecessor
Nonvacuous zero witness	4	2
Atomic progress closure	6	4
Arbitrary-term specialization	8	6
Contextual induction with seed congruence	48	40

10.12.2 Statements about the checkers of the packages

Composition of the arity codec with predecessor packets (`checker_and_barriers`, `compact_kernel`). The arity codec encodes $s = t$ as $[s, t]$, $D_{x,y}B$ as $[B]$ and Q_yB as $[B, B, B]$, with the depth increment $5 - \mathrm{len}$ for every array, numerical root depth three, and fixed truth and zero aliases at the other arities. The composed checker uses the packet $[\texttt{null}, p, a, h]$ of Theorem 10.118. For a truthy p it corresponds to the arity packet $[[\,], [p, a, h], 1, 0]$ and for a falsy p to $[[\,], [0, a, h], p, 0]$: the generated raw equation, the image checks at the old floor, the next floor and the opened goal and hypothesis are identical in the two executions, so the inherited supply proof applies. The literal 0 stays available as a witness of an introduction, and the step at the predecessor 0 is the three-node derivation of Theorem 10.120.

Theorem 10.122 (Composed verifier)**.** *Every successful execution of each of the supplied composed checkers has a decoded conclusion derivable in* N_{p}*, and every closed theorem of* N_{p} *has a finite accepted canonical certificate, given sufficient resources.*

Proof. Induction on the successful trace. Membership is HYP, by raw equality and the opening invariant: at a meaningful sorted root a successful opening implements capture-free filling of the distinguished hole, by induction on the tree, and the interpretation of a constant alias is unchanged by traversal of its children. A proved equality or closure major with matching endpoints is transport or QE; a mismatch leaves the goal and adds truth. Triplication of a unary major is DE at a fresh parameter. The positional argument check makes an introduction a genuine instance, with the restriction of progress to literal equations. Arithmetic supply is Theorem 10.121. Parameters admitted inside a completed major and unused by its end sequent are replaced by 0. For completeness, encode a derivation rule by rule, with the boxed step at the predecessor 0. □

The invariant is conditional: opening is not idempotent on every raw tree below ignored annotations. The two packages obtain the same 612-byte program.

Local eigenparameter scopes (`local_scopes`). The inherited checker decreases one global integer at every allocation, so the proof of a major permanently advances the allocator of the continuation. The local-scope checker passes the current floor to each call by value.

Theorem 10.123 (Local scope preserves the calculus)**.** *With the same sorted decoding and rule correspondence, the local-scope checker is sound for* N_{p} *with* COMP*, and every finite canonical derivation has a local-scope certificate.*

Proof. At the entry of a call with floor k and context g the scope contains the labels $k, \dots, -1$, and $k = -|g|$, since each continuation appends one context entry and decreases the floor by one. Induct on a successful trace. The entry traversal checks the goal at the current scope before a name is allocated. A major is checked at the same scope, so its conclusion contains no name allocated inside its subproof, and context extension builds a new list, so its local assumptions do not escape. The caller then allocates $k-1$; the discharged side proof imposes no freshness requirement on this independent binding. The openings, the transport, the closure use, the paired rules and the generated arithmetic assumption are justified as before; the generated assumption needs only a derivable zero instance, by Theorem 10.119. For completeness, rename the binders of a named derivation, label the active eigenparameters at a branch with d context entries by members of $-1, \dots, -d$, serialize a major with the same environment and discard its private extensions on return. □

The theorem states preservation of the calculus under scope-respecting renaming, not equality of accepted byte strings: two sibling subproofs may reuse a raw label, and an old packet may mention a name from a finished side proof.

Iterative continuations (`research`). The loop checker keeps recursion for majors and runs continuations in a loop over an immutable context.

Lemma 10.124 (Continuation simulation)**.** *On canonical certificates and with unbounded resources, the iterative checker has the same successful derivations as the recursive flat checker, with the same order of allocation of fresh names.*

Proof. The invariant is that the current goal, the contents of the context, the global freshness floor and the unconsumed instruction stack equal those of the corresponding recursive call. Hypothesis closure is the same membership test. The recursive call for a major uses the same preceding context, and its allocations persist in both programs. Every branch constructs the same next goal and added assumption; tuple concatenation has the contents of list concatenation and changes no earlier context. Opening the goal and opening the assumption have no effect on the floor, so their order is immaterial. Induction on the trace gives both directions. □

Persistence of the context is necessary by Theorem 10.71. The loop removes the dependence of recursion depth on the number of continuation steps, not on nested majors, syntax traversal or parsing.

10.13 Partial routes

This section draws on the second research note of `scope_kernel`, on `local_scopes` and on the appendix on stationarity of the second write-up of `progress_independence`. All three results are conditional: each isolates the statement that is missing for a complete replacement of a rule.

10.13.1 From uniqueness of iteration toward induction

Work in intuitionistic logic with equality over the scalar iterator, with the two computation equations. The *uniqueness rule* is

$$\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash f(t) = I(t, f(0);\, u.\, h(u))}\ \mathrm{UR},$$

where f and h are fixed term templates, z is fresh for Γ, t and the parameters, and $h(u)$ does not contain z. The premise has no discharged induction hypothesis and there is no separate base premise. Uniqueness of recursion as an arithmetic principle is classical for richer languages with counter-aware recursion and product types; the question here is the scalar update.

Lemma 10.125. *(i) Induction for literal equations derives* UR*, without coverage or an equality test.*

(ii) UR *derives: from* $f(0) = g(0)$*,* $f(Sz) = h(f(z))$ *and* $g(Sz) = h(g(z))$ *with* z *fresh,* $f(t) = g(t)$*.*

(iii) UR *derives* $I(t, 0; u.\, Su) = t$*.*

Proof. (i) Induct on $f(n) = I(n, f(0); u.\, h(u))$: the base is the zero computation, and $f(Sn) = h(f(n)) = h(I(n, f(0); u.\, h(u))) = I(Sn, f(0); u.\, h(u))$. (ii) Apply UR to f and to g and use congruence in the initializer. (iii) Take $f(z) = z$ and $h(u) = Su$; the premise is reflexivity. □

Part (i) is formalized for the kernel as `NPrf.uniq`.

Lemma 10.126 (Retraction lemma)**.** *Let* $e(x)$ *be a term template such that* Γ *proves, with* x *fresh,*

$$e(0) = 0, \qquad e(e(x)) = e(x), \qquad e(S(e(x))) = S(e(x)).$$

Then the computation equations, equality reasoning and UR *prove* $\Gamma \vdash e(t) = t$*. No induction is used.*

Proof. Put $h(u) = e(S(e(u)))$ and $r(n) = I(n, 0; u.\, h(u))$. Idempotence at $S(e(u))$ gives $e(h(u)) = h(u)$, and idempotence under S and e gives $h(e(u)) = h(u)$; the third hypothesis is $h(u) = S(e(u))$. For $f(n) = e(r(n))$: $f(0) = e(0) = 0$ and $f(Sn) = e(h(r(n))) = h(r(n)) = h(e(r(n))) = h(f(n))$. UR and replacement of the equal initializers give $e(r(n)) = r(n)$; so the whole orbit of h lies in the image of the retraction. Then $r(Sn) = h(r(n)) = S(e(r(n))) = S(r(n))$ and $r(0) = 0$, so UR for r with the update Su and Theorem 10.125(iii) give $r(n) = n$. Hence $e(n) = e(r(n)) = r(n) = n$. □

Proposition 10.127 (Induction from a term-defined split)**.** *Let* $E(n)$ *be a formula with* $\Gamma \vdash E(0)$ *and* $\Gamma, E(z) \vdash E(Sz)$*,* z *fresh. Suppose there is a term* $e(n)$ *such that, uniformly in a fresh* n*,* $\Gamma \vdash E(n) \vee \neg E(n)$*,* $\Gamma, E(n) \vdash e(n) = n$ *and* $\Gamma, \neg E(n) \vdash e(n) = 0$*. Then* UR *proves* $\Gamma \vdash E(t)$*.*

Proof. A case split proves $E(e(n))$: in the first branch by $e(n) = n$, in the second by $e(n) = 0$ and the base. The true-branch identity at $e(n)$ gives $e(e(n)) = e(n)$. The step at $e(n)$ gives $E(S(e(n)))$, and the true-branch identity gives $e(S(e(n))) = S(e(n))$. The base gives $e(0) = 0$. Theorem 10.126 gives $e(n) = n$, and transport of $E(e(n))$ gives $E(n)$. □

Corollary 10.128 (Induction for zero equations)**.** *With coverage and separation,* UR *derives: from* $\Gamma \vdash q(0) = 0$ *and* $\Gamma, q(z) = 0 \vdash q(Sz) = 0$*,* $\Gamma \vdash q(t) = 0$*. Only coverage, separation, the selector computations, equality reasoning and* UR *are used.*

Proof. Coverage gives the case distinction for every term, and $Sx = 0$ is contradictory: the template $v \mapsto \mathrm{sel}(v, 0, S0)$ gives $S0 = 0$. So $q(n) = 0$ is decidable. Put $e(n) = \mathrm{sel}(q(n), n, 0)$. If $q(n) = 0$ then $e(n) = n$; otherwise coverage for $q(n)$ leaves the successor alternative, where $e(n) = 0$. Apply Theorem 10.127. □

Coverage decides whether one number is zero; it does not give a term $\delta(a,b)$ with $\delta(a,b) = 0 \leftrightarrow a = b$. If the arithmetic with UR constructed such a term and proved the equivalence on its whole domain, Theorem 10.128 applied to $\delta(s(n), t(n)) = 0$ would recover induction for literal equations, and with Theorem 10.125(i) the two principles would be equivalent over the common background. The note does not supply this: the available proofs of predecessor and of reflection of equality (Theorems 1.65 and 1.86) use literal-equation inductions with non-constant right sides, and importing them would be circular. The note records one step of a translation. With $Z(u) = \mathrm{sel}(u, 1, 0)$, $N(n) = I(n, 0; u.\, Z(u))$, $a + b = I(b, a; u.\, Su)$, $j(u) = Z(u) + 2N(u)$, $M(n) = I(n, 0; u.\, j(u))$ and $G(n) = I(n, 0; u.\, u + M(u) + 1)$, the comparison $M(G(n)) = N(n)$ can be set up as a comparison of two recurrences with the common update $k(v) = I(Sv, v; u.\, j(u))$, and finite Boolean computations give $k(N(n)) = Z(N(n))$. The README of the repository records that the package `scalar_uniqueness` derives predecessor, reflection of equality and equation induction from UR and so closes this gap; that argument is not part of the note. A native kernel with UR in place of the progress rule would in addition have to derive its equality logic, case distinction and cut without the removed rule.

10.13.2 What fixed countdown uniqueness determines

A draft on scalar countdown uses a bouncing predecessor W and a countdown L with $L(0, a) = a$ and $L(Sn, a) = W(L(n, a))$, and the contextual rule

$$\frac{\Gamma \vdash g(Sz, \vec{p}) = W(g(z, \vec{p}))}{\Gamma \vdash g(t, \vec{p}) = L(t, g(0, \vec{p}))}\ (U_W),$$

with g a fixed template and z fresh. It completes its theory with an additional faithfulness sentence.

Lemma 10.129 (`local_scopes`). *If $\Gamma \vdash f(0) = 0$ and $\Gamma \vdash f(Sz) = S(f(z))$ with z fresh, then $\Gamma \vdash L(f(t), a) = L(t, a)$ for every t and a, by U_W alone. Consequently, if the theory also proves*

$$\forall x\, \forall y\, \big((\forall a\, L(x, a) = L(y, a)) \to x = y\big),$$

it derives $f(t) = t$ for every such f.

Proof. Take z fresh for a and $g(n) = L(f(n), a)$. Then $g(Sz) = L(S(f(z)), a) = W(L(f(z), a)) = W(g(z))$ and $g(0) = L(0, a) = a$. Apply U_W and substitute $g(0) = a$. The second claim instantiates the displayed sentence at $x = f(t)$, $y = t$. □

For example $L(A(t, 0), a) = L(t, a)$ for an initialized successor iterator A. Equality of all countdown observations is a statement about an action and is not equality of the counters. No proof of the displayed faithfulness sentence from U_W is given, and no equivalence with the stronger single-observation sentence $\forall a\, \forall b\, (L(b, SSa) = 2 \to a = b)$ of the draft is claimed.

10.13.3 Stationarity over an explicit primitive recursive interface

The *stationarity rule* is

$$\frac{\Gamma \vdash f(Sz, \vec{p}) = f(z, \vec{p})}{\Gamma \vdash f(t, \vec{p}) = f(0, \vec{p})}\ \textsc{stat},$$

for a term f of the language below, with z fresh for Γ, t and $\vec{p}$. It has no induction hypothesis, and its conclusion mentions no iterator. Read in the native language over the computation equations, it is equivalent to the instance of UR for the identity update, $\Gamma \vdash f(t) = I(t, f(0); u.\, u)$, together with the scheme $I(t, a; u.\, u) = a$; the scheme is STAT for

$f(z) = I(z, a; u.\, u)$, whose premise is a successor computation. The scheme fails at $t = c$, $a = 0$ in the countermodel of Theorem 10.79, so STAT is not sound there.

The *interface* is intuitionistic first-order logic with equality over the terms of primitive recursive arithmetic, with the defining equations of all primitive recursive function symbols of fixed description, coverage $x = 0 \vee \exists y\, x = Sy$, separation $0 \neq 1$, and, for each quantifier-free formula $A(\vec{x})$, a fixed primitive recursive term χ_A with the truth-flag axioms

$$\chi_A(\vec{x}) = 0 \leftrightarrow A(\vec{x}), \qquad \chi_A(\vec{x}) = 0 \vee \chi_A(\vec{x}) = 1.$$

All interface axioms are theorems of iPRA. They are not claimed to be derivable from the scalar computation equations without induction.

Proposition 10.130 (Stationarity recovers quantifier-free induction)**.** *Over the interface,* STAT *and induction for quantifier-free formulas derive each other, with arbitrary contexts. Hence the interface with* STAT *has the same theorems as* iPRA *in its definitional expansion by the symbols χ_A, with the identity numerical domain and actual equality.*

Proof. Let A be quantifier-free with $\Gamma \vdash A(0)$ and $\Gamma, A(z) \vdash A(Sz)$, z fresh; the parameters of A are kept and suppressed. Write $e(n) := \chi_A(n)$ and take primitive recursive symbols with

$$\kappa(0) = 0, \quad \kappa(Sx) = 1, \qquad C(0, a, b) = a, \quad C(Sx, a, b) = b,$$
$$\beta(0) = 0, \quad \beta(Sn) = C\big(e(Sn),\, \kappa(\beta(n)),\, 1\big).$$

Coverage at v and the equations of the clamp κ give $\kappa(v) = 0 \vee \kappa(v) = 1$. For a fresh n, a case split on n by coverage, with no induction, proves

$$\beta(n) = 0 \vee \beta(n) = 1, \qquad \beta(n) = 0 \to A(n). \tag{$*$}$$

At $n = 0$, $\beta(0) = 0$ and the base gives $A(0)$. At $n = Sm$ split on the flag $e(Sm)$. If $e(Sm) = 0$, then $\beta(Sm) = \kappa(\beta(m))$, which is 0 or 1, and $A(Sm)$ holds by the truth-flag axiom. If $e(Sm) = 1$, then $\beta(Sm) = 1$, and $\beta(Sm) = 0$ gives $0 = 1$, hence $A(Sm)$ by separation. Only the outermost recursion equation of β is used.

The premise of STAT, $\beta(Sn) = \beta(n)$ for fresh n, follows by cases on the first part of $(*)$. If $\beta(n) = 1$, then $\kappa(\beta(n)) = 1$, and both flag cases give $\beta(Sn) = 1$. If $\beta(n) = 0$, the second part of $(*)$ gives $A(n)$, the step gives $A(Sn)$, so $e(Sn) = 0$ and $\beta(Sn) = \kappa(0) = 0$. STAT gives $\beta(t) = \beta(0) = 0$, and $(*)$ at t gives $A(t)$.

Conversely, induction on the equation $f(n, \vec{p}) = f(0, \vec{p})$ derives STAT: the base is reflexivity, and the step composes the premise $f(Sz, \vec{p}) = f(z, \vec{p})$ with the hypothesis $f(z, \vec{p}) = f(0, \vec{p})$. Since the interface axioms are iPRA theorems, the two theories prove the same formulas. □

The proposition is an alternative induction principle, not a smaller checker. Installing the interface inside the scalar kernel without QI would need predecessor and reflection of equality, and their proofs (Theorems 1.65 and 1.86) use QI, so citing them would be circular. The rule UR of `scalar_uniqueness` has an arbitrary update and does not by itself justify the restriction to the identity update.

10.14 Checker sizes

Table 10.2 lists the complete standalone programs of the packages of this chapter with the sizes that the packages state, under the protocol of Section 1.10. The sizes of `scope_kernel`, `progress_independence` and `ipra_fixed_points_and_checker`, whose 241-token compact checker is a trace-equivalent reimplementation of an inherited calculus, belong to their checker work and are listed with it in other chapters, the last in Chapter 11. `iteration_barriers`

delivers no new program; it re-measures five unchanged programs of `whole_json_refinement` and `shared_leaf` under CPython 3.12.14, with the compact proof interface of Chapter 11 as input contract (a promised exact-integer and list alphabet and a canonical closed guarded public goal, not a validator of arbitrary JSON values). Its values agree with Section 11.11. It reports the envelope 248/268/469 for whole-input programs and 242/262/469 when JSON-line streams are admitted, each attained by three different programs. These envelopes are stated for the programs the package inspects; Section 11.11 also records a stream program of `ipra_fixed_points_and_checker` with 241 tokens and later programs with smaller envelopes. The observed minima at the snapshots of the packages are: 277 tokens, 311 nodes and 672 bytes, attained by different programs, for `adaptive_seed`, `research`, `local_scopes` and `uniform_predecessor`; 268, 300 and 616 for `checker_and_barriers` and `compact_kernel`; 281, 315 and 685 for `constructed_computation` and `counter_congruence`. These minima are attained sizes, hence upper bounds on an unknown optimum; they are not lower bounds.

Table 10.2: Checker sizes as stated by the packages. The control column gives tokens / nodes / bytes of the matched control.

Program	Tokens	Nodes	Bytes	Matched control
`adaptive_seed`				
rejected local-scope prototype	279	322	685	277 / 317 / 684
`research`				
iterative checker	276	305	749	278 / 314 / 679 (flat computation)
byte profile	280	319	652	same
explanatory checker	303	339	1401	—
`local_scopes`				
local-scope checker	284	323	668	281 / 322 / 672 (zero-product control)
readable checker	320	363	1426	—
`checker_and_barriers`				
explicit list dispatch	266	310	612	265 / 312 / 616 (old arity, explicit lists, same driver)
compact list predicate	266	312	604	265 / 314 / 608 (old arity, compact lists, same driver)
Boolean-expression profile	267	315	593	274 / 318 / 601 (old affine, same driver)
guarded traversal, tuple context	269	300	690	see below
`compact_kernel`				
selected composition	266	310	612	269 / 306 / 628 (matched old-flat control)
shorter-byte composition	267	315	597	—
smaller-AST composition	267	298	639	—
immutable-context loop	266	296	689	—
`uniform_predecessor`				
selected checker	269	307	629	269 / 306 / 628 (flat computation, matched driver)
predecessor packet, two verdicts	278	315	680	278 / 314 / 679 (flat computation)
ancillary byte profile	269	311	616	see below
ancillary AST profile	268	300	649	see below
readable checker	312	359	1531	—
`constructed_computation`				
selected profile	279	314	682	283 / 315 / 683 (old generator, same initialization profile)
explicit assignments	277	317	684	281 / 318 / 685 (generated assumptions, principal)

Program	Tokens	Nodes	Bytes	Matched control
AST profile	279	314	709	283 / 315 / 710 (generated assumptions, AST profile)
`counter_congruence`				
readable checker of $\mathcal{K}_{\mathrm{ctr}}$	1617	1822	6834	—
module import only	281	320	680	281 / 318 / 685 (frozen principal baseline)
module import, exact class	280	321	680	same
module import, exact type	281	322	676	same
module import, zero-product test	281	322	672	same
inverted hypothesis guard	283	314	751	283 / 315 / 710 (frozen AST baseline)
compact, separate RL, CC, QE	343	395	881	296 / 333 / 704 (general transport)
compact, CC fused into local facts	341	388	892	same
`iteration_barriers` (unchanged inherited programs, re-measured)				
whole input, token profile (`whole_json_refinement`)	248	277	615	— (no new program)
whole input, AST profile (`whole_json_refinement`)	254	268	613	— (no new program)
whole input, byte profile (`shared_leaf`)	272	306	469	— (no new program)
JSON-line stream, token profile (`shared_leaf`)	242	268	565	— (no new program)
JSON-line stream, AST profile (`shared_leaf`)	246	262	556	— (no new program)

For `checker_and_barriers` the further same-driver controls are the old arity token profile, 264 / 303 / 641, and the old research loop, 267 / 297 / 698; a screen of 105 old profiles under the same rejection driver gives the matched minima 264 tokens, 297 nodes and 601 bytes, of which only the byte minimum is improved, by the 593-byte profile. For `uniform_predecessor` the matched-driver controls are: token leader 268 / 309 / 633, AST leader 271 / 303 / 654, byte leader 272 / 314 / 621, flat computation 269 / 306 / 628, predecessor joint profile 271 / 306 / 637; the selected checker is one node and one byte larger than the matched flat checker, and its position below the unchanged minima comes from rejection by exit status in place of a printed verdict. In `constructed_computation` the arithmetic generator alone changes the explicit program from 281 / 318 / 685 to 277 / 317 / 684. The 612-byte programs of `checker_and_barriers` and `compact_kernel` are the same file.

The implementation devices are inherited source idioms and are not arithmetic principles. In short: the list test `a * 0 == []` on parsed JSON values; a short-circuit expression for guarded elimination; exception-based openers, which reject nonempty strings only through the recursion limit of the runtime and are therefore not compared with structurally terminating parsers; folding of initialization into a default argument; module import in place of a name import; and rejection by nonzero exit status. All programs import only the standard JSON library, and none imports a producer or a theorem library.

10.15 Further questions

The following statements of the packages are not results. First, what was not achieved.

- No package of this chapter obtains a complete axiomatization that is simpler than the common core under the measurement protocol. Where a checker improves a coordinate of the observed minima, the gain is an implementation change to an inherited theory. No delivered program is below all three coordinates in `research`, `local_scopes`,

`checker_and_barriers`, `compact_kernel` or `counter_congruence`; `constructed_computation` is below its three minima by two tokens, one node and three bytes; `uniform_predecessor` is below the unchanged minima only through the rejection convention.

- No obstruction of the chapter is a lower bound on the size of a checker, and none excludes equiconsistency or all one-way interpretations. Apart from Theorem 10.28, Theorem 10.125(i), the strictness statement cited after Theorem 10.65 and the odd case of Theorem 10.99, the README of the repository lists none of them as formalized in Lean. In particular the countermodel of Theorem 10.79 and the finite clocks of Theorem 10.97 are written proofs only.

- `progress_independence`: the result is the independence of one rule of the fixed kernel. No smaller arithmetic basis follows from it, and the package does not decide whether uniqueness of iteration for the successor update is equivalent to induction for literal equations; its strengthened countermodel refutes that uniqueness instance and so gives no answer. The second write-up likewise claims no smaller axiomatization, no new checker-size record and no global minimality of the eight rules, and Theorem 10.130 gives no smaller native checker, since the interface it assumes is not derived in the scalar kernel without QI.

- `ipra_fixed_points_and_checker`: Theorem 10.87 is a separation of principles and deletes no arithmetic axiom; it is not a smaller axiomatization, not a lower bound on checker size, and does not exclude a bi-interpretation through a different definable domain. The package does not show that QI fails to be admissible over the seven rules with the identity scheme alone (Theorem 10.89); this is settled by Theorem 10.95 of `iteration_barriers`. It does not decide the uniqueness rule for the successor update in its model.

- `iteration_barriers`: no axiomatization below the observed minima, no new checker and no minimality statement is obtained; the positive equivalence of FUSION with $I(n, 0; u.\, Su) = n$ and equation induction is Theorem 10.90 and rests on the written proof of Theorem 2.76. Theorem 10.94(ii) does not address bi-interpretations with nonstandard parameters, and neither part excludes equiconsistency or one-way interpretations. Theorem 10.91 concerns the identity law for the designated iterator, not the stronger rule that every term satisfying a constant-step recurrence is constant, and it settles no restricted uniqueness scheme. The package also states, without a model or a derivation, that scope validation of inserted terms in the compact openers cannot be deleted, that packing a checker into a decoded string or a universal interpreter only relocates the trusted program, and that QI for arbitrary formulas changes the target theory; these are assessments of implementation proposals, not theorems. The countermodels are not formalized in Lean or Rocq. Its finite check of the seed algebra is supplementary; no Lean or Rocq verification of the construction is claimed.

- No complete checker is given for the progressive calculus of Section 10.10. No implementation of $\mathcal{K}_{\mathrm{ctr}}$ is below the inherited programs; the compact attempts of the package have no universal audit of their raw aliases.

- `adaptive_seed`: the deletion of derived majors is rejected only in the form of Theorem 10.67. For a redesigned calculus without public closure goals, logical normalization does not justify deleting all derived majors, because equation induction can produce an equality that is then used for transport into a compound formula, and at an arbitrary parameter that induction is not a reducible successor case. This is an open proof obligation, not an incompleteness theorem.

- `compact_kernel`: two directions were examined without result. Replacing computation supply by numerical definitional conversion requires a reducer that respects dummy slots, the state binder, neutral counters and rigid parameters, and gave no smaller checker. Binders in the certificate grammar, with eigenparameters allocated by the opener, require a new proof-opening discipline, and no complete smaller construction was obtained.

- `uniform_predecessor`: binary-tree domains and arithmetic whose behavior depends on the syntax of a body were examined; no reduction of the complete checking mechanism and no valid smaller presentation was found.

- `scope_kernel`: the route from UR to induction is unfinished in the note (Section 10.13); the note claims no bi-interpretation and no smaller checker for a calculus with UR.

- `validated_leaves`: Theorem 10.33 is a written proof; no Lean or Rocq verification of it is claimed. It is a term-definability obstruction and does not exclude interpretations of iPRA whose operations are defined by formulas or finite-case graphs. The package deletes no arithmetic axiom, and its checkers do not go below the compact minima.

Question 10.131 (Relational escapes)**.** Theorem 10.45 and the theorems of Section 10.4 exclude term realizers only, and Theorem 10.58 requires an existential-free axiomatization and finite control. Is there a seedless basis, with positive existence axioms or with an additional primitive, that is bi-interpretable with iPRA through operations defined by graphs, and whose complete checker, with the defining equations and scope rules of the addition counted, is smaller? Under which witness-extraction property do graph interpretations reduce to the term case?

Question 10.132 (Leaving the almost unary class)**.** An independent initializer is one way to combine two unbounded inputs. Is there another operation, such as pairing or addition, that restores completeness of a seedless basis more cheaply, with both interpretation composites proved internally? Theorem 10.61 gives numerical completeness for an unbounded selector and no more.

Question 10.133 (Extent of finite control)**.** Does finite control, and with it Theorem 10.58, survive the adjunction of bounded numerical primitives other than an equality characteristic? Is every unary primitive recursive function denoted by a tied-seed term? Neither is decided by Theorem 10.52.

Question 10.134 (Restricted access to the state)**.** Theorem 10.33 answers the question of a common barrier for Theorems 10.36 and 10.39: adaptive queries to one body in seed and update, register vectors and counter-dependent locations do not leave the almost unary class. It is broken at once by unrestricted numerical retention of an old register (Theorem 10.43(iv)). Which restricted retention interfaces suffice to define addition and a selector, support their internal proofs, and give both interpretation round trips? Defining the functions in the standard model is not enough (`validated_leaves`).

Question 10.135 (Weaker case principles)**.** Is there a primitive rule weaker in syntax than BOOLCASES that supplies the mixed-parameter elimination refuted by the separator of Theorem 10.17, supports both formula round trips and admits a smaller checker? Is there a restriction of numerical cases that avoids the product model of Theorem 10.16 and still derives cases for all native formulas? A proposal must say where the global case distinction enters; predecessor identities do not supply it.

Question 10.136 (Positive iteration)**.** Is there a small operation, cheaper than the split into base and step, that destroys the zero-profile invariant of Theorem 10.62 and recovers a selector and the logical bootstrap? Can initialized iteration be introduced relationally from positive iteration without assuming the selector-based logic that it is meant to reconstruct, and how does induction transfer through the graph definitions when new updates are nested?

Question 10.137 (Majors and cuts)**.** Can atomic cuts be compiled away without induction on compound formulas, so that a focused calculus with fewer proved majors is complete? A normalization theorem would have to handle equation-induction proofs used for transport in arbitrary formulas, or an explicit generator of local induction clauses would have to be counted in the checker.

Question 10.138 (Uniqueness and countdown)**.** Does a complete checker for a calculus with UR improve the size coordinates? Removing an induction hypothesis does not decide this, since the conclusion of UR constructs several term instances and an iterator. Does U_W alone prove the faithfulness sentence of Theorem 10.129, so that the additional observation axiom of the countdown draft can be removed?

Question 10.139 (The countermodel to progress)**.** Can Theorem 10.75 and the tower of Theorem 10.77 be formalized in the Lean development, with arbitrary carriers and an interpreted successor in place of readings over the natural numbers? The same is asked for the orbit-map extension of Theorem 10.85 (`ipra_fixed_points_and_checker`). Which parts of the extension lemma survive when further laws of iteration, such as fusion or uniqueness, are imposed on the coherent structures? Computation and extensionality in the update are compatible with the countermodel; uniqueness for the identity update excludes its particular witness, and excluding one model does not give arithmetic strength. This part is answered in part by Section 10.9.4: the extension survives preservation of all fixed points and seed shift for every update (Theorem 10.85), and the seed can carry the canonical successor iterator and commutative cancellative addition (Theorem 10.86); it cannot survive general fusion together with the canonical successor iterator, because the two give UR and hence equation induction (Theorem 10.90 and the discussion after it). General fusion alone is compatible with a failure of equation induction: the pair structure of Theorem 10.93, which is not a tower, refutes the canonical successor iterator instead. Whether the tower itself can be extended so as to keep all instances of FUSION is not decided. Is there a counterpart of the model that satisfies the uniqueness rule for the successor update and still refutes an induction for a literal equation, which would separate the two principles? The model of Theorem 10.87 satisfies $I(n, 0;\, u.\, Su) = n$, which is the instance of that rule at the template z, but it is not shown to satisfy the rule, so the question stays open. Is there a presentation that combines computational equality with an induction interface weaker than QI without moving the cost into syntax validation?

Question 10.140 (After the countermodel to progress)**.** The second write-up of `progress_independence` asks three further questions. Is there an explicitly specified, for instance computable, algebra that supports all native substitutions and refutes $I(t, 0;\, u.\, u) = 0$ at some t? It would have to solve the problem of two-sided orbits for every update term, including updates that use earlier nonstandard iterators. `ipra_fixed_points_and_checker` asks the same for Theorem 10.87: is there a direct model, without a nonprincipal ultrafilter and an elementary tower, that keeps all passive-parameter and native-substitution laws together with (FP) and (Shift)? The second question, whether some induction instance for a literal equation remains underivable after the scheme $I(n, a;\, u.\, u) = a$ is added to the seven retained rules, is answered by Theorem 10.88 in the following exact form: the conclusion $Q_y E(y)$ of the instance for $E(n) \equiv \big(I(n, 0;\, u.\, S(Su)) = I(n, n;\, u.\, Su)\big)$ is underivable even after the rule FP, which

contains the scheme, and the leaves SHIFT are added, and over these additions its premise $E(z) \vdash E(Sz)$ is derivable, so QI is not admissible there. The stronger form, whether QI is admissible over N^- with the identity scheme, is answered negatively by Theorem 10.95 (`iteration_barriers`): for $E(y) \equiv (I(y, 0;\, u.\, Su) = y)$ the base and the step are derivable in N^- alone, and $Q_y E(y)$ is not derivable even after the identity leaves, FUSION and QI for matrices $t(y) = \overline{k}$ are added. After the identity scheme and every $I(n, \overline{k};\, u.\, Su) = S^k n$ are added instead, the model of Theorem 10.91 gives an underivable instance whose step is derivable in N^-, with a parameter in the equation. Both answers use nonstandard models, not explicit ones, so the first question of this paragraph stays open for them as well. Any such scheme would have to be counted in a complete checker. Can predecessor and reflection of equality be derived from STAT of Theorem 10.130 in the bare scalar language, without QI, so that the interface of that proposition is supplied inside the kernel? Equivalently in spirit, does STAT recover the strength missing from (FP) over the scalar interface? The model of Theorem 10.87 satisfies the identity scheme, which is the instance of STAT at $f(z) := I(z, a;\, u.\, u)$, but it is not shown to satisfy STAT, so it does not settle this (`ipra_fixed_points_and_checker`).

Question 10.141 (Restricted fusion)**.** Which proper fragment of FUSION recovers induction for literal equations over computation, coverage and separation? By Theorem 10.90 the whole rule does so with the canonical successor iterator, and (FP) and (Shift) do not (Theorem 10.87). The doubling instance excludes the model of Theorem 10.87, but excluding one witness is not completeness. (`ipra_fixed_points_and_checker`.) The scheme $I(t, 0;\, u.\, Su) = t$ cannot be dropped from Theorem 10.90(ii): the whole rule FUSION, with the identity law, extensionality, successor injectivity and induction against every fixed numeral, does not recover induction (Theorem 10.93); conversely the scheme with every $I(n, \overline{k};\, u.\, Su) = S^k n$, the identity law, extensionality and numeral induction does not either (Theorem 10.91). The question therefore concerns fragments of FUSION taken together with the scheme (`iteration_barriers`).

Question 10.142 (Parameters for the numbering theory)**.** Is T_{cal} of Section 10.9.5 bi-interpretable with iPRA by a bi-interpretation that uses nonstandard parameters? The involution of Theorem 10.91 moves every nonstandard element, so Theorem 10.94(ii) does not decide this; a model of T_{cal} with involutions fixing arbitrary finite tuples, as for T_{fus}, would. Do T_{cal} and T_{fus} interpret iPRA at all, and are they equiconsistent with it? Both are subtheories of T_{eq}, so they are interpreted in iPRA; the converse is not addressed (`iteration_barriers`).

Question 10.143 (Counter congruence and the reversed binder)**.** Can RL, CC and numerical computation be encoded through one checked representation that deletes a trusted operation, so that the restricted interface of Section 10.11 removes code? Can a staged introduction with delayed witness choice repair the reversed binder of Theorem 10.69, with the reverse formula composite proved for nested formulas?

Question 10.144 (Formalization)**.** The packages name as formalization targets: the bi-interpretation of the native calculus with its ordinary realization, with formula composites, freshness and arbitrary contexts; the scalar predecessor and the reflection of equality, without reintroducing quantifier-free induction before the characteristic is proved correct; the derivation transformer for $J_F = F$ of Theorem 10.112, without use of the ambient equality substitution; the sorted codec with its opener, freshness conditions and input contract; and a proved decoder that connects the arity checker with the Lean semantics of the native core. None is supplied.

Question 10.145 (Metric)**.** A coordinatewise envelope is not one program and not a lower bound. Would a fixed implementation grammar, a fixed semantic contract, a fixed treatment of the host runtime and a policy on aliases make strict-improvement claims more informative? Triplicated closures make explicitly serialized certificates grow as 3^r in closure depth, and the

protocol charges checker source only; how should certificate size and checking time enter the comparison? The empirical target left by `checker_and_barriers` is one structurally terminating complete program below 593 bytes, 264 tokens and 297 nodes under matched drivers.

Chapter 11

Certificates and checkers

The packages of this chapter do not change a rule basis. Each of them fixes a calculus that is established elsewhere in the article, specifies a format of proof certificates for it, and gives a program that accepts exactly the certificates of derivable sequents. The mathematical content is the specification of the format, the invariants of successful runs, and two kinds of theorem: a checker is sound and complete for its calculus, or two checkers of one calculus accept the same certificates. The programs are small, and many of their steps are safe only because of an invariant of the whole run; these invariants are stated and proved here. Program text is quoted only where an invariant refers to it.

All checkers of this chapter share one encoding of syntax (Section 11.2) and one instruction format (Section 11.3). They differ in a small number of independent devices, each of which is introduced once:

Device	Section	Introduced by
initializer dispatch, certified zero	Section 11.3	`certified_zero`
uniform zero, constructed majors	Section 11.4	`uniform_zero`
neutral major premise (empty array)	Section 11.5	`uniform_premise`, `monus_and_neutral_major`
local floors, shared hypothesis expression	Section 11.5	`neutral_scope`
elimination fixed before its major	Section 11.6	`prevalidated_elimination`
scalar neutral premise, tuple loop	Section 11.7	`kernel_refinement`, `compact_update`
validators of goal and certificate	Section 11.8	`validated_core` and successors
inferred major, eager selection, remainder selector, phase by context	Section 11.9	`inferred_major`, `smaller_envelope`, `frontier_review`, `context_phase`
three fields, source reused as the major	Section 11.9.9	`three_field`
allocation before the elimination test	Section 11.9.10	`context_order`, through `three_field` and `constantfree`
retained major, forward elimination	Section 11.9.3	`retained_premises`
truth-valued result, inverted entry test and phase by an empty context on the allocation-first programs	Section 11.9.11	`checker_refinement`, `single_binder`, `verified_refinements`
immediate application of the opener	Section 11.9.11	`immediate_application`
floor recomputed from the length of the context	Section 11.9.12	`scope_and_frontier`
two fields, computation equations recognized from the goal	Section 11.9.14	`refinement`
delayed export of a retained target	Section 11.9.3	`checked_frontier`
truth-valued result with the context extended inside an expression, on the native goal-validated program	Section 11.9.13	`checked_frontier`, `reductions`, `checked_presentation` (second delivery)
exact-integer leaves of the goal without the test of the printed form	Section 11.9.13	`479_predecessor57`
one parity comparison for the leaves of the goal	Section 11.9.13	`atomic_seed_refinement`
initial floor supplied with the goal	Section 11.9.13	`zero_test_split`
products of bits in the opener of the strict programs	Section 11.9.13	`equation_aliases`
two-field stream for the conditional-copy calculus	Section 11.9.13	`alias_back`
test of the printed form as a disjunction; eager mode selector	Section 11.9.13	`absorbing_orbit`; `boolean_constancy`
compositions of these rewritings	Section 11.9.13	`checked_reductions`, `refinement` (second delivery)
zero as a defined alias in the public goal	Section 11.9.13	`zero_alias`
terminating array test and era-	Sec-	`tolerant_certificates`

The packages `neutral_scope`, `scope_and_closure` (in its checker part), `compact_update`, `kernel_refinement`, `checked_refinement`, `validated_core`, `validated_presentation`, `scope_kernel`, `immediate_application`, `refinement`, `scope_and_frontier`, `immediate_opening`, `479_predecessor57` (in its checker part), the second delivery of `checked_presentation`, and the checker parts of `checked_frontier`, `zero_test_split`, `atomic_seed_refinement`, `reductions`, `equation_aliases`, `alias_back`, `absorbing_orbit`, `absorbing_predecessor`, `boolean_constancy`, `zero_alias`, `checked_reductions`, `fused_iteration`, `pred50`, `tolerant_certificates`, the second delivery of `refinement`, `offset472`, `checker_refinements`, `468_presentation`, `checked_presentations` and the checker parts of `fixed_points_and_checker`, `scope_refinement`, `validated_leaves`, `orbit_equality`, `ast261` and `root_conversion_raw`, and `468_shared_leaf`, `shifted_shared_leaf`, `direct_packets`, `procedure_presentation` and the checker part of `scalar_constancy`, and `prefix_conditional`, the third delivery of `checked_presentation` and the checker part of `fused_blocks_and_checkers` state that they contain no new rule and no new arithmetic; each composes devices of this list, and the composition is what is proved. The mathematical part of `scalar_constancy` belongs to the chapter on discharge. The compiler of finite blocks of conjunctions and disjunctions of `fused_blocks_and_checkers` concerns the base theories and is not treated in this chapter. The mathematical parts of `fixed_points_and_checker`, `validated_leaves` and `orbit_equality` belong to other chapters, and so does the native calculus of `root_conversion_raw` (the chapter on canonical and counter-aware recursors); the rigidity theorem of `scope_refinement` is Theorem 11.184, and that of `ast261` is Theorem 11.185. The mathematical parts of `479_predecessor57`, `zero_test_split`, `atomic_seed_refinement`, `reductions`, `equation_aliases`, `alias_back`, `absorbing_orbit`, `absorbing_predecessor`, `boolean_constancy`, `zero_alias`, `checked_reductions`, `fused_iteration`, `pred50` and the second delivery of `refinement` concern the rule basis and the arithmetic and are not treated in this chapter.

Throughout, a checker reads one finite array of JSON values, except for the framings of Section 11.11.1. Its last element is the *goal*; the earlier elements are *packets*, consumed from the end. A run *accepts* if it terminates normally; a failed operation (a comparison of incomparable values, a missing packet, a wrong number of fields) rejects. An unused prefix of packets is permitted. Completeness statements refer to ideal executions without bounds on memory or recursion depth; a physical run can reject a valid large certificate, and no statement below equates the resource limits of two programs.

11.1 The calculi that are checked

This section collects, from `certified_zero`, `kernel_refinement`, `compact_update` and `validated_presenta` the three rule bases for which the checkers of this chapter are written, and two admissible rules on which every soundness proof relies.

Definition 11.1 (The calculus $\mathcal{A}$)**.** Terms and native formulas are those of Sections 1.1 and 1.3; a context is a finite list of formulas. The calculus $\mathcal{A}$ has the rules

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma) \qquad \frac{}{\Gamma \vdash I(0,a;\,u.\,h) = a} \qquad \frac{}{\Gamma \vdash I(Sn,a;\,u.\,h) = h[I(n,a;\,u.\,h)/u]}$$

$$\frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \frac{\Gamma \vdash Q_y B(y) \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \qquad \frac{\Gamma \vdash D_{x,y} B(x,y) \quad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A}$$

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y} B(x,y)} \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y} E(x,y)}$$

named HYP, I0, IS, TR, QE, DE, DI-case and DI-progress. In TR, $F(v)$ is an arbitrary native formula template with one designated numerical variable, which may occur any number of

times and below binders. In DE the parameter b is fresh for Γ, A and the parameters of B. In both introductions z is fresh for Γ, t and the parameters of the matrix, and in DI-progress the matrix E is a literal equation: its outermost constructor is equality.

The calculus $\mathcal{A}$ is the calculus `IPrf` of the Lean development, and `iprf_iff_nprf` states that it derives the same sequents as the native core N of Section 1.3. Consequently weakening, renaming of parameters, capture-free numerical substitution and cut of an assumption are admissible for $\mathcal{A}$ (Section 1.4), and $\mathcal{A}$ inherits the bridge of Theorem 1.119. The formula $0 = 0$ is derivable: with $r := I(0,0;\, u.\,0)$, I0 proves $r = 0$, and TR with $F(v) \equiv (v = 0)$ applied to $r = 0$ and $F(r)$ gives $0 = 0$.

Lemma 11.2 (Zero-instance supply)**.** *Let $\vec{b}$ be parameters that do not occur in Γ or A. If $\vdash H(\vec{0})$ and $\Gamma, H(\vec{b}) \vdash A$, then $\Gamma \vdash A$.*

Proof. Rename the eigenparameters of the second derivation away from $\vec{b}$ and substitute 0 for every member of $\vec{b}$. Since $\vec{b}$ does not occur in Γ or A, the result is a derivation of $\Gamma, H(\vec{0}) \vdash A$. Cut the assumption $H(\vec{0})$ against its derivation. □

Lemma 11.3 (Neutral resource)**.** *If T is a closed theorem, then $\Gamma, T \vdash A$ if and only if $\Gamma \vdash A$.*

Proof. Weakening gives one direction and cut gives the other. Only renaming of eigenparameters is used, no numerical substitution. □

Theorem 11.2 is the reason why a checker may add a computation equation to its context without recognizing the equation as an axiom: it is enough that the instance at $\vec{0}$ of the added formula is an instance of I0 or IS. The open formula $H(\vec{b})$ need not be true. For example $H(b) \equiv (I(0,0;\, v.\,0) = b)$ has a computation instance at $b = 0$ and is false at $b = S0$.

Definition 11.4 (Source-selected elimination)**.** For a template $F(v)$ and terms s, t put

$$M(s,t;F) := \begin{cases} Q_v F(v), & s \text{ is literally } 0, \\ s = t, & \text{otherwise.} \end{cases}$$

The rule SE infers $F(t)$ from $M(s,t;F)$ and $F(s)$. The calculus $\mathcal{A}^-$ is $\mathcal{A}$ with TR and QE replaced by SE.

Definition 11.5 (The guarded-source calculus $\mathcal{U}$)**.** A formula is *public* if it is an equation, or $D_{x,y}B$ with B public and compound, or $D_{x,y}E$ with E an equation of the shape

$$I(y,0;\, u.\,h) = I(y,0;\, u.\,k). \tag{11.1}$$

The calculus $\mathcal{U}$ has HYP, I0, IS, SE, DE and one introduction

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \eta_B(z) := \begin{cases} z, & B \text{ a literal equation}, \\ 0, & B \text{ compound}, \end{cases}$$

with z fresh as before. Derivations may use all native formulas, including Q and unguarded atomic matrices, as auxiliary formulas; a *public sequent* has public assumptions and a public conclusion. The *auxiliary reading* A^* of a formula in T_{eq} is the ordinary reading of Section 1.2 except that $(D_{x,y}E)^* := \exists x\, \forall y\, (E(x,y) \to E(x,Sy))$ when E is a literal equation.

Remark 11.6 (Imported equivalences). The checkers of Sections 11.7 and 11.8 are written for $\mathcal{A}^-$ and $\mathcal{U}$, whose equivalence with N is not a result of this chapter. The package `source_dispatch` proves that $\mathcal{A}^-$ and $\mathcal{A}$ derive the same sequents (in Lean: `zprf_iff_nprf`); every omitted instance of TR, with source the literal 0, is derivable. The packages `guarded_core`

and `guarded_source` prove that every rule of $\mathcal{U}$ is valid in T_{eq} under the auxiliary reading, that A^* and the ordinary reading of A are equivalent in T_{eq} for public A (the zero instance of a guarded equation (11.1) is a theorem), and that $\mathcal{U}$ derives every public sequent whose ordinary reading is derivable in T_{eq}; the guarded public language is formalized in Lean in the form `pgprf_pub_iff`. Numerical substitution into a derivation is not admissible in $\mathcal{A}^-$ or $\mathcal{U}$ as a transformation of derivations, because it can turn a source into the literal 0. For this reason soundness of a checker of $\mathcal{U}$ is proved below through the reading in T_{eq}, where Theorem 11.2 is available, and not by a direct appeal to Theorem 11.2 in $\mathcal{U}$.

11.2 The sorted arity encoding

The encoding of this section is used by every compact checker of the chapter; it is restated, from an earlier checker, by `certified_zero`, `uniform_premise`, `prevalidated_elimination` and `monus_and_neutral_major`, whose statements are merged here.

Definition 11.7 (Raw trees)**.** A *raw tree* is a scalar or a finite array of raw trees. The *arity* of an array is its length. A scalar is *numeric* if it is an integer, a floating-point number or a Boolean; the Booleans are compared and added as 0 and 1. A raw value is *false-valued* if it is numerically zero, the empty array, or one of the remaining empty JSON values (`null`, the empty string, the empty object); otherwise it is *true-valued*.

A raw tree is read at a position of one of two sorts, term or formula, below some number $p \geq 0$ of *protected slots*. The *depth* of the position is $3 + 2p$ at a term position and $2p$ at a formula position, so that its parity records the sort. Slots are numbered from the innermost, starting with 0. A *floor* is an integer $k \leq 0$.

Definition 11.8 (Sorted reading)**.** Fix a floor k. At a term position below p slots a numeric scalar r reads as

$$\begin{cases} 0, & 0 \leq r < 3, \\ \text{slot } j, & 3 + 2j \leq r < 5 + 2j \text{ and } j < p, \\ \text{the rigid parameter named } r, & k \leq r < 0, \\ \text{the hole } w, & \text{otherwise}, \end{cases}$$

where w is one distinguished variable. Arrays read as follows.

Arity	At a term position	At a formula position
0	0	$0 = 0$
1	S of the child	$D_{x,y}$ of the child
2	0	equality of the two children
3	$I(n, a;\, u.\, h)$ of the children n, a, h	Q_y of the last child
≥ 4	0	$0 = 0$

The child of a unary term array is read below two additional *dummy* slots, which read as 0. The children n and a of a ternary term array are read below one additional dummy slot and the child h below one additional slot, the state u. The children of a binary formula array are read at term positions below the same slots. The child of a unary formula array is read below two additional slots, y innermost and x next; the last child of a ternary formula array is read below one additional slot y. A scalar at a formula position reads as $0 = 0$. Children that the table does not mention (those of a binary term array, of an array of arity at least 4, and the first two children of a ternary formula array) are *annotations* and have no reading.

With child depth $d+5-r$ for a child of an array of arity r at depth d, all the environments of Theorem 11.8 are obtained from one law: the increments are 4, 3 and 2 at arities 1, 2 and 3, that is, two slots, a change of sort, and one slot.

Definition 11.9 (Canonical encoding)**.** The *canonical encoding* of a term or formula whose free variables are rigid parameters with negative integer names writes 0 as the scalar 0, a parameter as its name, the slot j as $3+2j$, and

$$St \mapsto [t], \qquad I(n,a;\,u.\,h) \mapsto [n,a,h], \qquad s=t \mapsto [s,t], \qquad D_{x,y}B \mapsto [B], \qquad Q_yB \mapsto [B,B,B],$$

where the slot numbers of variables are counted through the dummy slots of Theorem 11.8. A canonical encoding contains no reference to a dummy slot, and its reading is the encoded expression.

The triplication of the matrix of Q serves the export step of the checker: repeating a unary array $[B]$ three times yields exactly the canonical closure $[B,B,B]$, and repeating any array of arity at least 2 yields an array of arity at least 6, which reads as $0=0$.

Definition 11.10 (Opener)**.** For a floor k, a raw tree a, a raw *image* t or the missing image $\perp$, and a depth d, the opening $O_k(a;t,d)$ is defined by

$$O_k(a;t,d) := \begin{cases} [\,O_k(b;t,d+5-r) : b \in a\,], & a \text{ an array of arity } r, \\ a, & a \text{ a scalar with } k \le a < d, \\ O_k(t;\perp,3), & a \text{ another scalar and } d \text{ odd}, \\ 0, & a \text{ another scalar and } d \text{ even}. \end{cases}$$

It is undefined if a comparison is applied to a value that is not numeric, or to $\perp$. The tree a is *validated at depth d and floor k* if $O_k(a;\perp,d)$ is defined, and *closed at depth d and floor k* if its reading at a position of depth d and floor k contains no hole. The depth is omitted when it is 0, the depth of the formula root.

The image is inserted after being validated at the root term depth 3 with a missing image, so an insertion never starts a further substitution; the opener terminates by structural recursion on finite trees. The checkers differ in the test that recognizes an array (an explicit type test, the equation `a*0 == []`, which separates arrays from the other JSON values on which it is defined, or an attempted traversal that falls back to the scalar case), and in the spelling of $\perp$ (`None`, the complex number `0j`, or a function object); each spelling makes every comparison with $\perp$ fail. The profile that recognizes arrays by attempted traversal accepts a few further empty iterables as aliases of the empty array and has one qualification: on a nonempty string an ideal execution does not terminate, and an actual one rejects through its recursion limit. This qualification is attached to that profile wherever its size is quoted (Section 11.11).

Lemma 11.11 (Semantic opening)**.** *Let a be read at a position of depth d and floor k, and suppose that $a' = O_k(a;t,d)$ is defined.*

(a) The tree a' is closed at depth d and floor k.

(b) If $t=\perp$, then a is closed at depth d and floor k, and a' and a have the same reading.

(c) If $t \neq \perp$ and the hole occurs in the reading of a, then $t' = O_k(t;\perp,3)$ is defined, the reading τ of t at the root term position is a term whose only free variables are parameters with names in $[k,0)$, and the reading of a' is the reading of a with τ substituted, without capture, for the hole. If the hole does not occur, a' and a have the same reading.

Proof. Statement (b) is proved first, by induction on a. A scalar at a term position is retained exactly when it lies in $[k, d)$, that is, when it does not read as the hole; otherwise the comparison with $\perp$ fails. A scalar at a formula position is retained or replaced by 0, and both read as $0 = 0$. At an array of arity 1, 2 or 3 the children that have a reading are opened at exactly the depths of their positions, and the induction hypothesis applies; annotations are traversed, which can make the opening undefined but does not affect a reading. Arrays of other arities have a constant reading and keep their arity.

For (c), the same induction applies, with one new case: a scalar that reads as the hole is replaced by t'. By (b) for t at depth 3, the tree t' reads as τ at the root term position, and every scalar of t' at its root level lies in $[k, 3)$, so it reads as 0 or as a parameter in $[k, 0)$; a scalar of t' below q of its own binders lies in $[k, 3 + 2q)$ and refers to 0, to a parameter, or to one of these q binders. Hence t' has the reading τ at every term position, below any number of slots: copying it below further binders captures nothing and moves none of its indices. Statement (a) follows from (b) and (c), since every retained scalar at a term position lies in $[k, d)$. □

No claim is made that an opened tree is unchanged by a second opening at another depth, floor or image: annotations below arrays of arity at least 4 are traversed at depths that may decrease, and a scalar at a formula position is not a fixed point of every traversal. At one floor and one depth, without an image, opening is idempotent (Theorem 11.136). Only readings at positions that have one are used. Since the reading is a function of the raw tree, equal raw trees have equal readings.

Lemma 11.12 (Exposure of the outermost slot)**.** *If a is closed at depth $d + 2$ and floor k, then its reading at depth d is its reading at depth $d + 2$ with the outermost slot replaced by the hole. Consequently $O_k(a; t, d)$, when defined, reads as the reading of a at depth $d + 2$ with τ substituted for the outermost slot.*

Proof. Let the position of depth $d + 2$ lie below $p + 1$ slots. A scalar r at a term position with a reading, below q further binders, lies in $[k, 3 + 2(p + 1 + q))$ because the reading has no hole. At depth d the same position has depth $3 + 2(p + q)$: a scalar in $[k, 3 + 2(p + q))$ keeps its reading, and a scalar in $[3 + 2(p + q), 5 + 2(p + q))$, which read as slot $p + q$, the outermost of the $p + 1$ slots, now reads as the hole. The second statement is Theorem 11.11(c). □

Theorem 11.12 covers the three openings of a checker. For a closed goal $[B]$, the matrix B is closed at depth 4 with slots y and x; $O_k(B; t, 2)$ instantiates the witness slot x and keeps y. For a matrix B' closed at depth 2, $O_k(B'; t, 0)$ instantiates its index slot y. For a closed major $[B]$, the opening $O_k([B, B, B]; t, 0)$ opens each copy of B at depth 2: it instantiates x and leaves y bound by the closure.

Lemma 11.13 (Canonical opening)**.** *Let a be the canonical encoding of an expression below $p + 1$ slots whose parameters have names in $[k, 0)$, read at depth d below p slots, and let t be the canonical encoding of a term whose free variables are parameters with names in $[k, 0)$. Then $O_k(a; t, d)$ is the canonical encoding of the expression with that term substituted for the outermost slot, and $O_k(a; \perp, d + 2) = a$.*

Proof. A canonical encoding uses the scalar 0, parameter names in $[k, 0)$ and slot scalars $3 + 2j$, all retained when in range; the exposed slot scalars are replaced by $O_k(t; \perp, 3) = t$. The encoding of a term without free slots does not depend on the number of slots above it, so the result is canonical. □

Lemma 11.14 (Freshness)**.** *If a tree is closed at depth d and floor k, its reading contains only parameters with names in $[k, 0)$, and it is closed at depth d with the same reading at*

every floor $k' \le k$. Hence the names in $[k', k)$ are fresh for every formula that is the reading of a tree closed at floor k.

Proof. Lowering the floor changes the reading only of scalars below k, and a tree closed at floor k has none at a position with a reading, since such a scalar reads as the hole. □

For canonical data there is exactly one name in $[k-1, k)$, the integer $k-1$. An arbitrary certificate may contain fractional scalars; then finitely many names lie in $[k-1, k)$, they are distinct parameters, and the arguments below treat them as a finite family $\vec{b}$.

11.3 The instruction format and the certified zero

This section follows `certified_zero`: the base protocol of all compact checkers, the restriction of computation that lets the initializer field select the computation branch, and the correspondence between the checker and the calculus $\mathcal{A}$.

11.3.1 The base protocol

Definition 11.15 (Base protocol)**.** The state of a run consists of a *global floor* k, initially 0, and the stack of unread packets. The procedure Check(a, g), for a raw goal a and a list g of raw trees, performs the following steps.

(1) Compute $a^\circ = O_k(a; \bot, 0)$. If a° is a member of g, return.

(2) Remove the last packet. It has exactly four fields $[m, s, u, t]$.

(3) If m is true-valued, run Check(m, g). Afterwards, if $m = [s, u]$, or if s is false-valued and $m = [t, t, t]$, and if moreover $O_k(t; u, 0) = a$, replace a by $O_k(t; s, 0)$.

(4) Form the hypothesis h.

   (i) If m is true-valued, h is the threefold repetition of m.

   (ii) If m is false-valued and u is true-valued, then $h = [[s, u, t], u]$ when s is false-valued, and $h = [[[s], u, t], O_k(t; [s, u, t], 3)]$ when s is true-valued.

   (iii) If m and u are false-valued, a is an array $[B]$ of arity one; replace a by $O_k(B; t, 2)$ and put $h = a$.

(5) Decrease k by 1.

(6) Let $\iota = k$ if m is true-valued, or if m is numerically zero and the arity of a is even; otherwise let ι be the scalar zero. Run Check$(O_k(a; [k], 0),\ g +\!\!+ [O_k(h; \iota, 0)])$.

The checker reads the goal a_0 and runs Check$(a_0, [\,])$.

The three branches of step (4) are selected by data: a true-valued major selects an elimination, a true-valued initializer field u selects a computation, and otherwise the packet is an introduction. There is no operation code. A packet is used as follows on canonical encodings; fields without a role are written 0, and $[\,]$ stands for any false-valued marker that is not numerically zero.

Inference	Packet $[m, s, u, t]$	Effect
TR with major $s = u$, template F	$[[s, u], s, u, F]$	goal $F(u)$ becomes $F(s)$
QE with major $Q_y B$	$[[B, B, B], 0, u, B]$	goal $B(u)$ becomes $B(0)$
DE with major $D_{x,y} B$	$[[B], 0, 0, 0]$	$Q_y B(b, y)$ is added
I0, initializer a, update h	$[[\,], 0, a, h]$	$I(0, a;\, v.\, h) = a$ is added
IS, predecessor n	$[[\,], n, a, h]$	the IS equation is added
DI-case, witness w	$[[\,], 0, 0, w]$	goal $D_{x,y} B$ becomes $B(w, Sz)$
DI-progress, witness w	$[0, 0, 0, w]$	the same, with $E(w, z)$ added

In a template F the designated variable is encoded as the outermost slot of a formula read below one slot, so that Theorem 11.12 applies; for QE the template is the matrix itself. A derivation is serialized with the packets of the major premise after the packet of the inference and the packets of the continuation after those, and the whole sequence is reversed, since packets are consumed from the end.

Two points of the definition depend on arguments given below. First, in the computation branch the field u must be true-valued, so the equations I0 and IS are available only for an initializer that is not the scalar 0; Theorem 11.17 shows that nothing is lost. Second, step (6) may open a generated computation equation at the fresh parameter instead of at zero (this happens when m is numerically zero and the goal has even arity); Theorem 11.18 and Theorem 11.2 show that this is sound.

11.3.2 Restricted computation and the certified zero

Definition 11.16. The calculus $\mathcal{A}_{\mathrm{r}}$ is $\mathcal{A}$ with its computation rules restricted by the syntax of their arguments: I0 is available only when the initializer a is not the literal term 0, and IS only when neither the predecessor n nor the initializer a is the literal term 0. A term that is not the literal 0 is permitted even if it denotes zero.

The restriction on the predecessor is present in earlier checkers of the same family (`uniform_predecessor`); the restriction on the initializer is the one introduced by `certified_zero`. The term

$$C := I(SS0, S0;\, v.\, 0)$$

is a permitted argument, and the instance of IS with predecessor $S0$, initializer $S0$ and update 0 is permitted and reads $C = 0$.

Theorem 11.17 (Certified zero)**.** *Every instance of I0 and IS is derivable in $\mathcal{A}_{\mathrm{r}}$, by one application of TR to two permitted computation instances. Hence $\mathcal{A}_{\mathrm{r}}$ and $\mathcal{A}$ derive the same sequents, over the same terms and formulas.*

Proof. A permitted instance is immediate. For an excluded instance choose a fresh variable w, with the bound state variable renamed away from w, and an equation $E(w)$ as follows.

I0, initializer 0 :	$E(w) \equiv (I(0, w;\, v.\, h) = w)$;
IS, initializer 0, predecessor $n \neq 0$:	$E(w) \equiv (I(Sn, w;\, v.\, h) = h[I(n, w;\, v.\, h)/v])$;
IS, predecessor 0, initializer $a \neq 0$:	$E(w) \equiv (I(Sw, a;\, v.\, h) = h[I(w, a;\, v.\, h)/v])$;
IS, both 0 :	$E(w) \equiv (I(Sw, w;\, v.\, h) = h[I(w, w;\, v.\, h)/v])$.

Only the displayed occurrences of the initializer and the predecessor are replaced by w; occurrences of 0 inside h stay. In each case $E(C)$ is a permitted computation instance, because C is not the literal 0, and $E(0)$ is the excluded instance. TR with the template E, applied to the permitted instance $C = 0$ and to $E(C)$, gives $E(0)$. Replacing every excluded leaf of a derivation in this way gives the inclusion of $\mathcal{A}$ in $\mathcal{A}_{\mathrm{r}}$; the other inclusion is trivial. Contexts and end sequents are unchanged. □

The construction uses no induction, no earlier computation with initializer 0 and no decision about the value of a term. Its cost is explicit: if a tree derivation has N inference nodes of which r are excluded computation leaves, the repaired derivation has $N + 2r$ nodes; if the node proving $C = 0$ is shared in a directed acyclic representation and $r > 0$, it has $N + r + 1$ nodes. These are counts for this construction, not lower bounds.

11.3.3 Correspondence with the calculus

Lemma 11.18 (Zero-instance computation)**.** *In step (4ii) of Theorem 11.15, suppose that the opening $h' = O_{k-1}(h; \iota, 0)$ of step (6) is defined. Let $\vec{b}$ be the parameters with names in $[k-1, k)$ that occur in the reading $H(\vec{b})$ of h'. Then $H(\vec{0})$ is an instance of I0 or IS.*

Proof. The tree h is a binary array, so h' reads as an equation; its left side is a ternary term array and reads as an iterator with update η, the reading of t below the state slot.

Let s be false-valued, so $h = [[s, u, t], u]$. The counter s is traversed by the opening; a false-valued tree for which this succeeds is a numeric zero or the empty array and reads as 0. The initializer u is read on the left below the dummy slot of the iterator and on the right at the root. The two readings agree except that a scalar referring to the dummy slot reads as 0 on the left and as the hole, hence as the reading of ι, on the right. The reading of ι is 0 or the parameter named $k-1$. After the substitution of 0 for $\vec{b}$ both sides have the same initializer α, and $H(\vec{0})$ is $I(0, \alpha; v.\,\eta) = \alpha$.

Let s be true-valued, so $h = [[[s], u, t], O_k(t; [s, u, t], 3)]$. If the reading of t at depth 3 contains the hole, that is, if the update uses its state, then by Theorem 11.11(c) the image $[s, u, t]$ is validated at depth 3 and floor k. This validates s and u below the dummy slot and t below exactly one state slot, so no coordinate of t beyond its state is left unbound. The right side of h reads as $\eta[I(\sigma, \alpha; v.\,\eta)/v]$, where σ and α are the readings of s and u. On the left, s occurs below two further dummy slots of the constructed successor; all its scalars lie in $[k, 5)$, so its reading is again σ. Thus h is closed at floor k and reads as the instance of IS with predecessor σ. If the update does not use its state, the right side $O_k(t; [s, u, t], 3)$ is t validated at depth 3, and η does not contain v. The left side reads, after the substitution of 0 for $\vec{b}$ and for the hole, as $I(S\sigma^0, \alpha^0; v.\,\eta)$ for some terms σ^0, α^0, and $\eta = \eta[I(\sigma^0, \alpha^0; v.\,\eta)/v]$ because v does not occur in η. This is again an instance of IS. □

Theorem 11.19 (Soundness of the base protocol)**.** *Let every member of g be closed at floor k. If* Check(a, g)*, started at floor k, returns, then a is closed at floor k and the reading of a is derivable in $\mathcal{A}$ from the readings of the members of g. In particular, if the checker accepts a goal a_0, the reading of a_0 is a theorem of $\mathcal{A}$; for a canonical encoding this reading is the encoded formula.*

Proof. The proof is by induction on the number of calls in the run. Floors only decrease, and by Theorem 11.14 a tree that is closed at some floor keeps its reading at all lower floors; readings are written without mention of the floor. Step (1) validates a, so a is closed by Theorem 11.11(b). If the run returns in step (1), a° has the reading of a and is a member of g, and HYP applies.

Elimination. Let m be true-valued and let $k' \le k$ be the floor after step (3). By the induction hypothesis the reading of m is derivable from g, and m is closed at floor k. Suppose that the goal is replaced. If $m = [s, u]$, then s and u are closed at depth 3 with readings σ and υ, the major reads $\sigma = \upsilon$, and by Theorem 11.11(c) the old goal $O_{k'}(t; u, 0)$ and the new goal $O_{k'}(t; s, 0)$ read as $F(\upsilon)$ and $F(\sigma)$ for the template $F(w)$ that is the reading of t at floor k'. By TR it suffices to derive $F(\sigma)$. If s is false-valued and $m = [t, t, t]$, then t is closed at depth 2, the major reads $Q_y B(y)$ for the reading B of t below the slot y, and by Theorem 11.12 the old goal reads $B(\upsilon)$. A false-valued image that is used by an opening is a numeric zero or the

empty array and reads 0, so the new goal reads $B(0)$, and by QE it suffices to derive $B(0)$. If the goal is not replaced there is nothing to show at this point. In each case the current goal a_1 is closed at floor k', and it remains to derive its reading from g.

The final call runs at floor $k'-1$ with goal $O_{k'-1}(a_1;[k'-1],0)$, which has the reading of a_1 since a_1 contains no hole, and with the additional hypothesis $h' = O_{k'-1}(h;k'-1,0)$, where h is the threefold repetition of m. All members of the extended context are closed at floor $k'-1$ by Theorem 11.11(a), so the induction hypothesis applies. If $m = [B]$ is unary, then $h = [B,B,B]$, and by Theorem 11.12 h' reads $Q_y B(b,y)$, where $D_{x,y}B$ is the reading of m and b is the parameter named $k'-1$. By Theorem 11.14, b is fresh for g, for a_1 and for the parameters of B, and DE concludes. If m is an array of arity at least 2, then h has arity at least 6 and h' reads $0 = 0$; Theorem 11.3 concludes. If m is a true-valued scalar for which the run continues, it is numeric, h is a numeric scalar and h' reads $0 = 0$ as well; moreover no replacement of the goal has occurred, since a scalar is not equal to an array. (The program writes the second alternative of the test as a value that is the Boolean `False` when s is true-valued; a true-valued scalar is not equal to it either.)

Computation. Let m be false-valued and u true-valued. The goal is unchanged, and the final call derives its reading from g and the reading $H(\vec{b})$ of h', where $\vec{b}$ are the parameters with names in $[k-1,k)$. They are fresh for g and a by Theorem 11.14. By Theorem 11.18 $H(\vec{0})$ is an axiom of $\mathcal{A}$, and Theorem 11.2 concludes.

Introduction. Let m and u be false-valued and $a = [B]$. Then B is closed at depth 4, and by Theorem 11.12 $a_1 = O_k(B;t,2)$ reads, below the slot y, as $B(\tau,y)$, where $D_{x,y}B(x,y)$ is the reading of a and τ is the reading of t if the witness slot occurs, and 0 otherwise; the parameters of τ have names in $[k,0)$. Let z be the parameter named $k-1$. In the final call the goal $O_{k-1}(a_1;[k-1],0)$ reads $B(\tau,Sz)$ and the hypothesis $O_{k-1}(a_1;\iota,0)$ reads $B(\tau,z)$ if $\iota = k-1$ and $B(\tau,0)$ if ι is zero. The parameter z is fresh for g, for τ and for the parameters of B. The opener preserves arities, so the arity of a_1 is the arity of B. If it is 2, B is a literal equation and DI-progress or DI-case applies. If it is 1 or 3, the arity is odd, ι is zero, and DI-case applies to the compound matrix. If it is 0 or at least 4, the conclusion reads $D_{x,y}(0=0)$, which is derivable by DI-case from the theorem $0 = 0$. If B is a scalar, step (6) is undefined and the run rejects. □

Remark 11.20. Three details of the programs lie outside Theorem 11.15 and do not affect Theorem 11.19. On the empty goal [ ] an introduction packet is processed with shifted arguments; the reading of that goal is the theorem $0 = 0$. In two of the three profiles the replacement of step (3) is written as a Boolean expression that keeps the old goal when the new one is false-valued; then the stronger, unchanged obligation is checked. Canonical formulas are nonempty arrays, so neither detail affects canonical certificates. Finally a major is always proved before it is used: a mismatch of the pattern of step (3) leaves the goal unchanged but does not remove the obligation to prove m.

Theorem 11.21 (Completeness of the base protocol)**.** *Let A be a closed formula. If A is derivable in $\mathcal{A}$, there is a stack of packets on which the checker accepts the canonical encoding of A.*

Proof. By Theorem 11.17 there is a derivation in $\mathcal{A}_{\mathrm{r}}$. The following statement is proved by induction on the height of derivations: for every floor k, every list g of canonical encodings of formulas with parameters named in $[k,0)$, and every derivation of $\Gamma \vdash A'$ in $\mathcal{A}_{\mathrm{r}}$ with Γ among the formulas encoded in g and parameters named in $[k,0)$, there is a stack on which Check, called at floor k with the canonical encoding of A' and with g, returns. Weakening and renaming of parameters do not increase the height of a derivation, so the induction hypothesis may be applied to premises in a larger context and with renamed eigenparameters. By Theorem 11.13 the goal is unchanged by step (1). If it is a member of g, the empty stack

suffices; this covers HYP and also every sequent whose conclusion happens to be retained already. Otherwise take the last inference of the derivation and the packet of the table after Theorem 11.15.

For TR, QE and DE the major is a nonempty array, and the stack for its derivation, which exists by the induction hypothesis, is consumed by step (3) and leaves a floor $k' \le k$. By Theorem 11.13 the pattern test of step (3) succeeds for TR and QE and replaces the goal by the canonical encoding of the minor premise; for DE it fails, because a unary array is equal neither to $[0,0]$ nor to $[0,0,0]$. The final call runs at floor $k'-1$ with the same goal and a context extended by a formula that reads $0 = 0$, or by the canonical encoding of $Q_y B(b,y)$ with the eigenparameter renamed to $k'-1$. The induction hypothesis for the remaining premise, weakened and renamed, supplies the rest of the stack.

For a computation leaf, which in $\mathcal{A}_{\mathrm{r}}$ has an initializer that is not the literal 0 and, for IS, such a predecessor, the fields u and, for IS, s are true-valued: the canonical encoding of a term other than the literal 0 is a nonempty array or a negative integer. Step (4ii) forms $[[0,a,h],a]$ or $[[[n],a,h],O_k(h;[n,a,h],3)]$, which by Theorem 11.13 is the canonical encoding of the computation equation; the canonical encoding of a term without free slots is the same below the dummy slots. This is the goal, so the final call returns in its step (1).

For an introduction with witness w the packet has false-valued m and u, step (4iii) opens the matrix at the canonical encoding of w, and step (6) forms the canonical encodings of $B(w,Sz)$ and of $B(w,z)$ or $B(w,0)$ with z renamed to $k-1$: the marker $m = 0$ selects $B(w,z)$ exactly when the matrix is an equation, as DI-progress requires, and the marker $[\,]$ selects $B(w,0)$. The induction hypothesis supplies the stack for the premise. In all cases the stack consists of the stack of the continuation, followed by the stack of the major if there is one, followed by the packet of the last inference. The packet is then consumed first, the major call consumes exactly its own stack, and the final call consumes the rest; that each call consumes exactly the stack constructed for it, when this stack is the end of a longer one, is part of the statement proved by the induction. □

Remark 11.22 (Profiles). `certified_zero` delivers three complete programs of the base protocol, which differ only in the recognition of arrays described after Theorem 11.10: an explicit type test, the zero-product test, and attempted traversal. The first two terminate on every finite input; the third has the qualification on nonempty strings. All three are complete for canonical certificates. The package also isolates the effect of the initializer dispatch from other changes of the source: restoring the earlier test that the major is the null value, in the three final programs, costs exactly eight bytes, two tokens and three syntax-tree nodes in each.

11.4 Uniform zero and constructed majors

This section follows `uniform_zero`. The certified zero of Theorem 11.17 repairs a restriction that exists only because the scalar 0 is at once the canonical zero and a false-valued marker. A different canonical spelling of zero removes the restriction, and the same observation lets an elimination packet omit its major formula.

11.4.1 The recoding

By Theorem 11.8 the scalars 0, 1 and 2 all read as zero at a term position. Let R map a canonical encoding to the tree obtained by replacing each scalar 0 by the scalar 1; arrays, slot scalars and parameter names are unchanged. The trees $R(A)$ are the *canonical encodings with uniform zero*: the term 0 is the scalar 1, the term $S0$ is $[1]$. The recoding applies to expression fields of packets only; the markers 0 and $[\,]$ in the major field keep their role.

Lemma 11.23 (Uniform truth value)**.** *Every canonical encoding with uniform zero of a term is true-valued, every such encoding of a formula is a nonempty array, and $R(A)$ has the same reading as A.*

Proof. The scalar 1 is true-valued, a parameter name is negative, a slot scalar is at least 3, and the encodings of S, of the iterator, of equations, of D and of Q are nonempty arrays. The scalars 0 and 1 have the same reading at every term position, and R changes nothing else. □

Truth value is a property of the syntax: a true-valued term may denote zero.

Let O_k^+ be the opener of Theorem 11.10 with one change: a scalar a with $k \le a < d$ is returned as a if it is true-valued and as the scalar 1 otherwise.

Lemma 11.24 (Opening commutes with the recoding)**.** *For canonical encodings A and t as in Theorem 11.13,*

$$O_k^+\big(R(A); R(t), d\big) = R\big(O_k(A; t, d)\big),$$

and likewise with the missing image when no image is used. Opening a canonical template with uniform zero at the raw image 0 *gives the same result as opening it at the raw image* 1*.*

Proof. First let no image be used; the proof is by induction on A. A scalar 0 of A occurs at a term position, where it is retained by O_k; in $R(A)$ it is the scalar 1, retained by O_k^+. Every other retained scalar is true-valued and unchanged on both sides. Arrays are traversed at the same depths on both sides. A canonical formula has no scalar at a formula position, so the formula-position clause does not arise. With an image, the only new case is an exposed scalar, replaced on the left by $O_k^+(R(t); \bot, 3)$ and on the right by $R(O_k(t; \bot, 3))$, which are equal by the first part. For the last statement, $O_k^+(0; \bot, 3) = 1 = O_k^+(1; \bot, 3)$ at every floor. □

Lemma 11.25 (Semantic opening for O^+)**.** *Theorems 11.11, 11.12 and 11.14 hold for O_k^+.*

Proof. The only changed result is a false-valued numeric scalar retained by the range test (the integer or floating-point zero, or the Boolean `False`), which is returned as 1. At a term position both read as 0; at a formula position both read as $0 = 0$. All other clauses are those of O_k. □

The identity of Theorem 11.24 is restricted to canonical encodings; for arbitrary raw trees only the statement about readings in Theorem 11.25 is available, and only it is used for soundness. In contrast with a definitional reduction of $I(0, a; v.\, h)$ to a, the recoding deletes no node and no slot, so no index is shifted.

11.4.2 Constructed majors and local floors

Definition 11.26 (Uniform-zero protocols)**.** The *uniform-zero protocol* is the base protocol with O^+ in place of O and with *local floors*: the floor is an argument $\mathrm{Check}(a, g, k)$ of the procedure, the call for the major in step (3) receives the caller's floor k and leaves it unchanged, and step (5) decreases the caller's own floor. The *constructed-major protocol* changes, in addition, step (3): if the field m is true-valued, the major is

$$M(m, s, u, t) := \begin{cases} [s, u], & s \text{ true-valued}, \\ [t, t, t], & s \text{ false-valued and } t \text{ true-valued}, \\ m, & s, t \text{ false-valued}; \end{cases}$$

the checker runs $\mathrm{Check}(M, g, k)$, and afterwards, if $O_k^+(t; u, 0) = a$ and $O_k^+(t; s, 0)$ is true-valued, replaces a by $O_k^+(t; s, 0)$. Steps (4)–(6) use M in place of m.

Local floors are taken over from the package `local_scopes`. They are justified by the same induction as the global floor.

Lemma 11.27 (Local floors)**.** *Theorems 11.19 and 11.21 hold for the base protocol with local floors.*

Proof. In the proof of Theorem 11.19 the induction hypothesis for the major call is the statement that the reading of m is derivable from g; it does not mention the names that the call allocates, which are eigenparameters of that derivation. With local floors $k' = k$ in the elimination case, the templates are opened at the caller's floor, and the final call at floor $k - 1$ has an eigenparameter that is fresh for g and for the goal, both closed at floor k. The rest of the proof is unchanged. For completeness the stack of the major derivation is produced at floor k and that of the continuation at floor $k-1$; names used inside the major derivation are reused, which is a renaming of eigenparameters in disjoint subderivations. No call changes the context of its caller, since a context is extended by forming a new list. □

With local floors and an initial context of c entries, every call satisfies $k = c - |g|$: a major call keeps both, and a continuation appends one entry and decreases k. The floor is nevertheless not a function of the context that the program could recompute at entry for free; see Theorem 11.56.

Proposition 11.28 (Soundness of constructed majors)**.** *Theorem 11.19 holds for the constructed-major protocol.*

Proof. By Theorems 11.25 and 11.27 only step (3) needs an argument. If s is true-valued, the proved major is $[s, u]$ and reads $\sigma = \upsilon$; a successful comparison identifies the goal with $F(\upsilon)$ for the reading F of the template t, the new goal is $F(\sigma)$, and TR applies. No separately supplied formula is claimed to have these endpoints. If s is false-valued and t true-valued, the proved major is $[t, t, t]$ and reads $Q_y F(y)$; a false-valued image that is used reads 0, so the new goal is $F(0)$ and QE applies. If s and t are false-valued, the supplied m is proved and exported as in the base protocol; the comparison $O_k^+(t; u, 0) = a$ can then succeed only for a template t that is a false-valued scalar or the empty array, in which case $O_k^+(t; s, 0)$ is false-valued and the goal is not replaced. □

Theorem 11.29 (Direct canonical compilation)**.** *Every derivation in $\mathcal{A}$, with unrestricted I0 and IS, of a closed formula has a certificate in the constructed-major protocol on canonical encodings with uniform zero. No computation leaf is replaced, and the compilation is the identity on readings.*

Proof. The induction of Theorem 11.21 is repeated with Theorem 11.24 in place of Theorem 11.13 and with the packets

Inference	Packet $[m, s, u, t]$
TR through F, major $s = u$	$[1, R(s), R(u), R(F)]$
QE from $Q_y B$, target u	$[1, 0, R(u), R(B)]$
DE from $D_{x,y} B$	$[R(D_{x,y}B), 0, 0, 0]$
I0, initializer a, update h	$[0, 0, R(a), R(h)]$
IS, predecessor n	$[0, R(n), R(a), R(h)]$
DI-case, witness w	$[[\,], 0, 0, R(w)]$
DI-progress, witness w	$[0, 0, 0, R(w)]$

By Theorem 11.23 the source $R(s)$ of a transport is true-valued even when s is the term 0, and a matrix $R(B)$ is a nonempty array, so the constructed major is exactly the equation or the closure of the inference. The initializer $R(a)$ is true-valued for every term a, so every

computation leaf selects the computation branch. In I0 the generated counter is the raw marker 0, which the opening of the hypothesis returns as the canonical 1; in IS the predecessor $R(n)$ is true-valued even for $n = 0$, and the generated counter [1] is the canonical successor of zero. The zero images of step (6) become the canonical 1 as well. The recoding does not change the root of a matrix, so the restriction of progress to literal equations is unchanged. The arrays of arity 6 or 9 that equality and closure majors add to the context are not used by the certificate. □

By Theorem 11.23 the consequence relation presented on canonical encodings is that of $\mathcal{A}$, read through the identity, so the bridge of Theorem 1.119 applies without change.

11.4.3 Sizes of certificates

Let $|X|$ be the length in bytes of the JSON text of X without whitespace. Then $|[X, Y]| = |X| + |Y| + 3$ and $|[B, B, B]| = 3|B| + 4$. In a transport packet the major field $[s, u]$ is replaced by the marker 1 while s and u remain in their own fields, which saves exactly $|s| + |u| + 2$ bytes; in a closure packet the field $[B, B, B]$ is replaced by 1, which saves exactly $3|B| + 3$ bytes. No inference and no premise is removed by these two changes, and for one derivation with one allocation of names the savings add over its packets, since the recoding of zero replaces one digit by another.

Separately, the direct serialization of computation leaves removes the repairs of Theorem 11.17: a derivation with N nodes and r excluded leaves has N packets instead of $N + 2r \le 3N$, when neither serialization closes an additional goal by a retained hypothesis. The constructed major copies fields that are present at most three times and C is a fixed term, so both translations are linear in the encoded proof; the triplication of matrices in the encoding of Q, which can be exponential relative to a shared representation, is not removed.

Example 11.30. The equation $I(0, 0; v.\, 0) = 0$ has, in the base protocol with the repair of Theorem 11.17, a certificate of three packets and 97 bytes. With uniform zero the certificate is

```
[[0,0,1,1],[[1,1,1],1]],
```

one packet and 23 bytes: the goal $[[1, 1, 1], 1]$ and the packet of I0 with initializer and update 1. The package reports, for sixteen selected examples, 120 packets and 5029 bytes in the base protocol, 90 packets and 3801 bytes with uniform zero, and 90 packets and 2955 bytes with constructed majors. These are measurements of particular certificates, not minimal sizes.

The gain is in the certificates. Under a comparison with a matched control that already uses local floors and the shared hypothesis expression of Section 11.5, uniform zero costs five bytes of checker source and constructed majors a further six (Section 11.11).

11.5 Neutral major premises

This section merges `uniform_premise` and the certificate protocol of `monus_and_neutral_major`, which are two write-ups of one device, and adds the composition with local floors of `neutral_scope`. In the base protocol a packet has a major premise only if its first field is true-valued. Here every packet has one: a computation or an introduction names a fixed proved truth, and the test for the presence of a major disappears. The safety of the resulting program rests on an invariant of successful runs.

11.5.1 The protocol and its hazard

Definition 11.31 (Neutral-major protocol)**.** The neutral-major protocol is the base protocol of Theorem 11.15 with the following changes.

(N1) The initial call is Check$(a_0, [[\,]])$: the context contains the empty array, which reads $0 = 0$.

(N2) In step (3), Check(m, g) is run for every packet, whether or not m is true-valued. The replacement test that follows is unchanged.

(N3) In step (4), branch (i) is taken if the threefold repetition of m is true-valued, branch (ii) if it is not and u is true-valued, and branch (iii) otherwise.

(N4) In step (6), $\iota = k$ if m is true-valued, or if the arity of a taken modulo 2 is numerically equal to u; otherwise ι is zero.

On canonical certificates a packet without a substantive major has $m = [\,]$, an introduction by cases has $u = [\,]$, and an introduction by progress has $u = 0$.

The call Check$([\,], g)$ returns in its step (1) without reading a packet and without changing the floor, because $[\,]$ is a member of every context of the run. The mode of an introduction moves from the first field to the initializer field, which is false-valued in an introduction.

The program evaluates the test of step (3) as membership of m in the pair consisting of $[s, u]$ and of the value that is $[t, t, t]$ if s is false-valued and the Boolean `False` otherwise. In the base protocol the test is reached only for a true-valued m. Now it is reached for every m, and the Boolean `False` is equal to the numeric scalar 0. If a numeric zero could be a proved major, a packet with $m = 0$ and a true-valued s would replace the goal $F(u)$ by $F(s)$ with no equation proved. The protocol is therefore sound only if a numeric scalar is never a major of a returning call, and this is what the invariant below establishes. It holds because the initial context is $[[\,]]$ and not $[0]$.

11.5.2 The invariant of successful runs

Lemma 11.32 (Root openings)**.** *For a numeric scalar r and any image t,*

$$O_k(r; t, 0) = \begin{cases} r, & k \le r < 0, \\ 0, & \text{otherwise}, \end{cases}$$

which is numeric and does not depend on t. If $O_k(a; t, 0)$ is defined and a is an array, the result is an array; if the result is numeric, a is numeric.

Proof. The depth 0 is even, so a scalar outside $[k, 0)$ is replaced by 0 and no image is consulted. An array is mapped to an array by the first clause of Theorem 11.10, and a scalar that is not numeric fails the comparison. □

Lemma 11.33 (No numeric goal)**.** *In the neutral-major protocol there is no floor, no stack and no list g of arrays for which* Check(a, g) *returns with a numeric goal a.*

Proof. Suppose there is such a run, and choose one with the least number of calls, over all floors, stacks and contexts. By Theorem 11.32 the tree a° is numeric, and no numeric scalar is equal to an array, so the run does not return in step (1). It reads a packet, and Check(m, g) returns. If m were numeric, this call would be a smaller run of the excluded kind; so m is not numeric, and since its opening is defined, m is an array.

If the replacement test of step (3) succeeds, then $O_k(t; u, 0) = a$ is numeric, so t is numeric by Theorem 11.32, and $O_k(t; s, 0) = O_k(t; u, 0)$ by the same lemma. The goal is numeric after step (3) in every case.

If m is nonempty, h is its threefold repetition, an array. If m is empty and u is true-valued, h is a binary array. In the remaining case step (4iii) requires the goal to be an array of arity one, which a numeric scalar is not, and the run rejects. So h is an array, its opening in step (6) is an array, and the final call has a numeric goal $O_k(a; [k], 0)$ and a context of arrays. It is a smaller run of the excluded kind, a contradiction. □

Lemma 11.34 (Contexts of arrays)**.** *In a run of the neutral-major protocol from* Check$(a_0, [[\,]])$*, every call has a context all of whose members are arrays, and every major m whose call returns is an array.*

Proof. The initial context consists of the array $[\,]$. Suppose a call with a context of arrays reaches step (3). The major call receives the same context. If it returns, m is an array by Theorem 11.33. The hypothesis h is then the threefold repetition of a nonempty array, or a binary array, or, in branch (iii), the opened matrix $a = O_k(B; t, 2)$. In the last case m is the empty array, so step (6) evaluates the arity of a, which is undefined for a scalar; if the final call is reached, $h = a$ is an array. The opening of an array is an array, and the new context is formed by appending it to a copy of the old one, so no call changes the context of another. □

Corollary 11.35 (The unguarded test)**.** *At every point of a run that is reached after the return of a major call, the test of step (3) succeeds if and only if m is a nonempty array and either $m = [s, u]$, or s is false-valued and $m = [t, t, t]$.*

Proof. By Theorem 11.34 m is an array. An array is not equal to the Boolean `False`, and the empty array is equal neither to $[s, u]$ nor to $[t, t, t]$. □

The corollary states that, on the states that can occur, the test has the value of the same test guarded by the condition that m is true-valued, although the two differ on arbitrary values. Two mutations show that the hypotheses matter. If the initial context is [0], the numeric zero is a proved major, and a certificate of $S0 = 0$ is accepted. If the comparison in (N4) uses the field s in place of u, an introduction with $s = 1$ obtains the hypothesis $B(t, z)$ for a matrix of odd arity, that is, progress for a compound matrix; the field s is unrestricted in an introduction, whereas u is false-valued there and no false-valued value is numerically equal to 1. Both mutants are constructed and observed by the packages.

11.5.3 Correspondence

Theorem 11.36 (Neutral-major protocol)**.** *If the checker of the neutral-major protocol accepts a goal, the reading of the goal is a theorem of $\mathcal{A}$. If a closed formula is a theorem of $\mathcal{A}$, its canonical encoding is accepted on some stack.*

Proof. Soundness. The statement of Theorem 11.19 is proved for contexts whose members are arrays closed at the current floor, by the same induction. By Theorem 11.35 the elimination case applies verbatim when m is a nonempty array. If m is the empty array, its reading is the theorem $0 = 0$, the goal is not replaced, and the packet is a computation or an introduction. In an introduction u is false-valued, and by (N4) $\iota = k$ exactly when the arity of the opened matrix is even and u is numerically zero; so the hypothesis $B(\tau, z)$ is granted only to a matrix of even arity, and the case analysis on the arity in the proof of Theorem 11.19 applies. In a computation u is true-valued, and (N4) can give $\iota = k$ when u is numerically 1; Theorem 11.18 holds for either value of ι, so Theorem 11.2 applies as before. The result is a derivation of the reading of the goal from $0 = 0$, and Theorem 11.3 removes this assumption.

Completeness. By Theorem 11.17 take a derivation in $\mathcal{A}_{\mathrm{r}}$ and repeat the induction of Theorem 11.21 with contexts that contain, in addition, the empty array. The packets are those of the base protocol with [] in the major field of computations and introductions and with the mode in the field u. A canonical goal is a nonempty array and is never equal to [], so the additional member of the context closes no goal of the derivation; the major call of a packet without substantive major returns at once and consumes nothing. □

Remark 11.37 (A second sufficient argument). One may read every numeric scalar at a formula position as the contradiction $0 = S0$ instead of $0 = 0$. Theorem 11.11 remains true, since such a scalar is retained or replaced by the scalar 0. Under this reading a returning call with a numeric major would prove the contradiction, and the conclusion of its parent would follow by explosion whatever the test of step (3) does. This gives soundness without Theorem 11.33. The invariant is the stronger statement: numeric majors do not occur, so the reading of scalars need not be changed.

Remark 11.38 (Sources). `uniform_premise` proves Theorems 11.33 and 11.34 as one simultaneous induction on successful runs (every goal and every appended hypothesis is an array), for the profiles with an exact test for arrays, and states Theorem 11.37. `monus_and_neutral_major` proves them by the minimal counterexample used above and treats, in addition, the profile with attempted traversal: there the empty string and the empty object open to the empty array, an object whose only key is the empty string opens to [[]], a true-valued object cannot be a major because its threefold repetition is undefined, and a nonempty string has no finite successful opening. The statements above then hold with “array” read as “array or one of these empty aliases”. The two packages report different programs (Section 11.11); their protocols coincide.

11.5.4 Composition with local floors

Definition 11.39. The *local neutral-major protocol* is the neutral-major protocol with local floors (Theorem 11.26).

Theorem 11.40 (Composed checker)**.** *Theorems 11.33 to 11.35 and Theorem 11.36 hold for the local neutral-major protocol.*

Proof. The proofs of the three invariants do not refer to the floor except through Theorem 11.32, which holds at every floor; the minimal counterexample is chosen over all floors. The proof of Theorem 11.36 combines with Theorem 11.27. The context of a caller is not changed by a major call, because every extension forms a new list. □

The programs of `neutral_scope` write step (4), as modified by (N3), as one Boolean chain: the threefold repetition of m, or else the computation equation if u is true-valued, or else the introduction. By Theorem 11.34 the threefold repetition is true-valued exactly for a nonempty array m, and the computation equation is a binary array, hence true-valued; so the chain takes the branch that (N3) prescribes, and the introduction is reached exactly when m and u are false-valued. The missing image is spelled `0j`, a value that the JSON reader cannot produce and that fails every comparison. A further profile of the package replaces the final call by a loop over an immutable context; this transformation is treated once, in Theorem 11.51. The package constructs two mutants: the one with initial context [0] described above, and one in which the loop extends a shared mutable list, which accepts a certificate of $0 = S0$.

11.6 Fixing an elimination before its major is proved

This section follows `prevalidated_elimination` and the checker part of `scope_and_closure`. The order of two steps of the base protocol is exchanged: the template of an elimination is

opened, and the goal replaced, before the major premise is proved. The proof of the major can then be folded into the expression that forms the hypothesis.

11.6.1 The global-floor version

Definition 11.41 (Prevalidated protocol)**.** The prevalidated protocol is the base protocol of Theorem 11.15 with step (3) replaced by the following two steps.

($3'$) If $m = [s, u]$, or if s is false-valued and $m = [t, t, t]$, and if moreover $O_k(t; u, 0) = a$ and $O_k(t; s, 0)$ is true-valued, replace a by $O_k(t; s, 0)$.

($3''$) If m is true-valued, run Check(m, g).

The program evaluates the test of ($3'$) as membership of m in the pair consisting of $[s, u]$ and of the value that is $[t, t, t]$ if s is false-valued and the complex number `1j` otherwise, without a test that m is true-valued; and it forms the hypothesis of branch (4i) as `c(m,g) or m*3`. One of the three delivered profiles replaces the goal also when $O_k(t; s, 0)$ is false-valued; Theorem 11.43 covers both forms, and canonical minor premises are nonempty arrays.

Lemma 11.42 (Two properties of the program text)**.** *(a) No value produced by the JSON reader is equal to `1j`. Hence the membership test succeeds exactly under the condition of ($3'$), and then m is a nonempty array.*

(b) Every returning call of the checking procedure returns the null value. Hence `c(m,g) or m*3` proves the major and then has the value of the threefold repetition of m.

Proof. (a) The values of the reader are arrays, strings, objects, the null value, Booleans and real numbers, including its nonfinite numbers; none is equal to a complex number with nonzero imaginary part. If the test succeeds, m is $[s, u]$ or $[t, t, t]$, a nonempty array. (b) A call returns in step (1) with no value or after its final call with no value; this is an induction on the run. A failure inside the major call propagates and the hypothesis is not formed. □

Writing the Boolean `False` for the excluded alternative in (a) is unsound here: it is equal to the numeric 0, the marker of progress, so a packet with $m = 0$ and a true-valued s would pass the test and, having a false-valued major, would prove nothing. The package exhibits a false closed equation accepted after this replacement. Likewise, spelling the missing image as the integer 0 instead of a value that fails comparisons fills unbound holes and accepts a goal that contains an unallocated name.

Lemma 11.43 (Frozen elimination)**.** *In step ($3'$), let the goal a and the context be closed at floor k, and suppose that the goal is replaced by $a_1 = O_k(t; s, 0)$. If the major call of step ($3''$) returns, leaving a floor $k' \le k$, then a_1 is closed at floor k' with the same reading as at floor k, and the reading of a is derivable in $\mathcal{A}$ from the readings of m and a_1.*

Proof. By Theorem 11.11(a) a_1 is closed at floor k: every name in it was admitted at floor k, and so were the names of the inserted image. By Theorem 11.14 its reading is the same at floor k'. The major call returns, and its step (1) validates m at floor k, so m is closed at floor k. If $m = [s, u]$, the two openings of t at the one floor k read $F(\upsilon)$ and $F(\sigma)$ for one template F, and the reading of m is $\sigma = \upsilon$; TR applies. If s is false-valued and $m = [t, t, t]$, the openings read $B(\upsilon)$ and $B(0)$ by Theorem 11.12, the reading of m is $Q_y B(y)$, and QE applies. □

Theorem 11.44 (Prevalidated protocol)**.** *The checker of the prevalidated protocol is sound and complete for $\mathcal{A}$ in the sense of Theorems 11.19 and 11.21.*

Proof. For soundness, the induction of Theorem 11.19 is repeated; its elimination case uses Theorem 11.43 in place of the argument about openings at the floor k', and the major is proved in step $(3'')$ by the induction hypothesis, by Theorem 11.42. A replacement in step $(3')$ is followed by step $(3'')$, since m is then true-valued; if the major call fails, the run rejects.

For completeness, the stacks of Theorem 11.21 are accepted without change. In a canonical packet the designated variable of a template is an exposed slot scalar, which is a hole at every floor, and all other names are admitted at floor k; so the openings of step $(3')$ at floor k are the trees that the base protocol computes at floor k'. Canonical minor premises are nonempty arrays, so the condition that $O_k(t; s, 0)$ is true-valued removes no canonical inference. □

Proposition 11.45 (Raw acceptance is not preserved)**.** *There is a stack that the prevalidated checker accepts and the base checker rejects.*

Proof. Let $o = [0]$, $r = [0, o, 0]$, $M = [r, o]$ and $c = [0, 0, o, 0]$, and consider the stack $[\,c,\ c,\ [M, r, o, [-1, o]],\ [o, o]\,]$ with goal $[o, o]$, which reads $S0 = S0$. The term r reads $I(0, S0;\, v.\, 0)$, and the packet c supplies the equation $r = S0$. In the prevalidated order the template $[-1, o]$ is opened at floor 0, where the scalar -1 is a hole: the opening at $u = o$ is the goal, and the goal becomes $[r, o]$. The major $M = [r, o]$ is then proved by one packet c, and the continuation proves the new goal $[r, o]$ by the other. In the base order the major is proved first, which lowers the floor to -1; the scalar -1 of the template is then an admitted name, the opening at u is $[-1, o]$, the test fails, the goal remains $[o, o]$, and the remaining packet does not prove it. □

The package states that the two sets of accepted stacks are incomparable and supplies a stack accepted only in the base order; that stack is not reproduced in its report (Section 11.12). The two checkers agree on canonical certificates and present the same consequence relation, by Theorem 11.44.

11.6.2 The local-floor version

With local floors the exchange is an identity of accepted inputs. This is the checker result of `scope_and_closure`, which applies it to the local neutral-major programs of Theorem 11.39 and to a loop profile of the same protocol.

Theorem 11.46 (Equality of accepted certificates)**.** *Consider a checker with local floors in which the major call is a separate statement before the replacement test, and the checker obtained from it by evaluating the replacement test first and placing the major call at the head of the Boolean chain that forms the hypothesis. The two checkers accept the same finite inputs.*

Proof. The replacement test reads the packet, the current goal and the opener at the caller's floor. It reads no packet from the stack and changes neither the context nor the floor; its only effect is the replacement of the caller's goal. The major call receives the same arguments m, g and k in both programs, has its own floor and its own binding of the context, does not refer to the caller's goal, and extends contexts only by forming new lists or tuples. Its one shared effect is the consumption of packets, which the replacement test does not touch. Hence the two operations commute on every state. By Theorem 11.42(b) the major call contributes a false value to the chain, so the hypothesis is formed from the same remaining operands. An induction on the returning run, applied to the major calls and the continuations, shows that each program returns exactly when the other does, with the same goal, floor and remaining stack. A failure of either operation prevents acceptance in both orders; only the point of rejection can differ. □

Theorem 11.45 does not contradict Theorem 11.46: with a global floor the major call lowers the floor at which the test is evaluated, and with local floors it does not. The decrement of step (5) is not moved in either version; the witness of an introduction is opened before the fresh name is allocated.

11.7 Scalar neutral premises and loops

This section covers the checker parts of `kernel_refinement` and `compact_update`, the compact profiles of `validated_core`, and the compact changes of `checked_refinement` and `deterministic_introduction`. The programs are checkers of the guarded-source calculus $\mathcal{U}$ of Theorem 11.5. Their protocol is taken from `guarded_source`; the scalar spelling of the neutral premise is taken from `reversal_core`. `kernel_refinement` and `compact_update` apply that spelling to the same programs independently and report the same sizes; both describe the result as a composition.

11.7.1 The guarded-source protocol

Definition 11.47 (Guarded-source protocol)**.** The guarded-source protocol has local floors and the following steps for Check(a, g, k); steps (1), (2) and (5) are those of Theorem 11.15.

(3) Let M° be $[s, u]$ if s is true-valued and $[t, t, t]$ otherwise. If $m = M^\circ$, $O_k(t; u, 0) = a$ and $O_k(t; s, 0)$ is true-valued, replace a by $O_k(t; s, 0)$.

(4) Run Check(m, g, k). Form h as in (N3) of Theorem 11.31.

(6) Let $\iota = k$ if m is true-valued or the arity of a is 2, and zero otherwise. Run the final call as in Theorem 11.15.

The initial context is $[0]$, and a packet without substantive major has $m = 0$. The *native protocol* is the same with (N4) of Theorem 11.31 in place of step (6).

Step (3) implements SE: the source field selects the shape of the major, an equation for a true-valued source and a closure for a false-valued one. Step (6) implements the single introduction of $\mathcal{U}$: its mode is read off the root of the matrix, and a packet carries no mode. The native protocol keeps the free choice of mode and is a checker of $\mathcal{A}^-$; it is the proof kernel of the validated checkers of Section 11.8.

Proposition 11.48 (Scalar neutral premise)**.** *In the guarded-source protocol let every member of g be closed at floor k. If* Check(a, g, k) *returns, then a is closed at floor k and the auxiliary reading of a is derivable in T_{eq} from the auxiliary readings of the members of g. Conversely, every closed theorem of $\mathcal{U}$ has a certificate on which its canonical encoding is accepted. The same holds for the native protocol with the ordinary reading and $\mathcal{A}^-$.*

Proof. Soundness. A scalar at the formula root reads $0 = 0$ (Theorem 11.8), so the initial context is a theorem and a scalar goal is trivially derivable; no invariant on the roots of goals is needed. The induction is that of Theorem 11.19, carried out in T_{eq}. The tree M° is an array in every case. Hence the test of step (3) succeeds only for a nonempty array m that is $[s, u]$ with s true-valued, or $[t, t, t]$ with s false-valued; a scalar major never passes it. In the first case the major reads $\sigma = \upsilon$ and the replacement is a transport of equals, valid in T_{eq}. In the second the major reads $Q_y B(y)$, the goal $B(\upsilon)$ is replaced by $B(0)$, and the reading $B(0) \to \forall y\, B(y)$ of the closure justifies it. The openings are evaluated at the caller's floor, which the major call does not change.

By the induction hypothesis the reading of m is derivable. If $m = [B]$, the exported hypothesis reads $Q_y B(b, y)$ with b fresh. For a compound B the auxiliary reading of $D_{x,y} B$

is $\exists x$ of the reading of $Q_y B(x, y)$, and existential elimination applies. For a literal equation B it is $\exists x \, \forall y \, (B(x, y) \to B(x, Sy))$, from which $B(b, 0) \to \forall y \, B(b, y)$ follows by induction for the equation $B(b, y)$. If m is an array of arity at least 2, or a true-valued numeric scalar, the exported hypothesis reads $0 = 0$. If the threefold repetition of m is false-valued, the packet is a computation, treated by Theorems 11.2 and 11.18 in T_{eq}, or an introduction. In an introduction the hypothesis is $B(\tau, z)$ exactly when the opened matrix has arity 2. For a literal equation the premise gives $\forall y \, (B(\tau, y) \to B(\tau, Sy))$, the auxiliary reading with witness τ. For a matrix of arity 1 or 3 the premise gives $B(\tau, Sz)$ from $B(\tau, 0)$ for fresh z, hence $B(\tau, 0) \to \forall y \, B(\tau, y)$ by coverage. For other arities the matrix reads $0 = 0$ and the conclusion is valid outright. In the native protocol the ordinary reading is used throughout and the case analysis is that of Theorem 11.19.

Completeness. The induction of Theorem 11.21 is repeated with local floors (Theorem 11.27) and the packets

$$[[s, u], s, u, F], \qquad [[B, B, B], 0, u, B], \qquad [[B], 0, 0, 0], \qquad [0, s, u, t], \qquad [0, 0, 0, w]$$

for SE at a source other than the literal 0, SE at the literal 0, DE, a computation and the introduction; in the native protocol the introduction by cases has $u = [\,]$. The canonical encoding of a source other than the literal 0 is true-valued, so M° is the intended major in both cases of SE, and for DE the unary major differs from $M^\circ = [0, 0, 0]$. A neutral major 0 is closed by the initial member of the context without a packet. Computations with a literal zero initializer or predecessor are first replaced as in Theorem 11.17; the transport used there has the source C, which is not the literal 0, so the replacement is available in $\mathcal{U}$ and $\mathcal{A}^-$. A canonical goal is a nonempty array and is not closed by the scalar member of the context. □

By Theorem 11.6, for a canonical public formula A the checker of the guarded-source protocol accepts some stack with goal A if and only if A is a theorem of the public theory. The checker also accepts goals that are not canonical public formulas, for example scalars and auxiliary formulas; the compact programs assume that the goal offered to them is canonical, and this assumption is their input contract (Section 11.8).

Remark 11.49 (Comparison with the empty-array premise). The predecessor programs have the initial context $[[\,]]$ and the neutral major $[\,]$. The scalar spelling is shorter by one byte and one syntax-tree node. It does not need the invariant of Theorem 11.33: in the test of step (3) the candidate M° is always an array, whereas the test of Theorem 11.31 has a Boolean candidate that a numeric zero matches. The two spellings do not accept the same raw stacks, since a stack with the old neutral major can be rejected. The translation of certificates replaces the major field of a packet when that field is exactly the empty array and changes no other occurrence of an empty array; in particular the mode field of an introduction in the native protocol is kept.

Remark 11.50 (Substitution in the restricted calculus). In the soundness proof the fresh names of a generated computation equation are specialized to zero by Theorem 11.2 applied in T_{eq}, after the continuation has been interpreted there. Applying a substitution theorem to derivations of $\mathcal{U}$ would be incorrect, as noted in Theorem 11.6.

11.7.2 Loops over immutable contexts

Theorem 11.51 (Tail call and loop). *In any of the protocols of this chapter, replace the final call of step (6) by a return to step (1) with the new goal and the new context, and represent contexts by immutable tuples that are extended by forming a new tuple. The resulting checker has the same returning runs, with the same consumption of packets and the same floors, as the recursive checker.*

Proof. Relate a list and a tuple when they have the same members in the same order; membership tests agree on related contexts. A major call receives related contexts, the same stack and the same floor, and by induction on its run it returns after consuming the same packets and leaving the same floor. The recursive checker then calls itself on the opened goal and the extended context; the loop continues with exactly these objects at the same floor. The extension forms a new tuple and rebinds a local name, so the context held by the caller of a major call is unchanged. The two openings of step (6) are evaluated in the opposite order in the two programs, but they are pure at the fixed new floor. When step (1) closes the goal, the loop ends and the recursive checker returns through its pending frames, which do nothing further. Unrolling a finite run of the loop gives a run of the recursive checker, and conversely. □

The loop can check longer continuations than the recursion under a fixed recursion limit; the theorem compares ideal executions. The loop profile with the fewest tokens, in `kernel_refinement` and `validated_core`, is not a program of Theorem 11.47: it keeps a global floor, an empty initial context and the optional major of Theorem 11.41, with the test of step (3) and the mode of step (6) of Theorem 11.47. Its soundness is that of Theorem 11.44, read in T_{eq} as in Theorem 11.48. Certificates of the global and of the local protocols are not interchangeable, since they allocate names differently. Immutability is necessary. If the context is a list extended in place, a hypothesis that is local to the proof of a major premise remains in the context of the caller. `kernel_refinement` gives the stack

```
[[[],0,[],[0]],[[[0,[0]]],0,0,0],[0,[0]]]
```

with goal $0 = S0$: the major $D_{x,y}(0 = S0)$ is introduced by cases, the case assumption $0 = S0$ is appended during its proof, and with a shared list it closes the goal of the caller. The checker with tuples rejects this stack.

11.7.3 The root test is a logical boundary

Step (6) of Theorem 11.47 tests both the major and the arity of the matrix. A shorter test that inspects only the arity of the generated hypothesis grants the hypothesis $B(t, z)$ also to matrices of arity 3, that is, to matrices that are closures. `kernel_refinement` shows that this changes the theory.

Proposition 11.52 (Progress at closures gives full induction)**.** *Add to $\mathcal{A}$ the rule that infers $D_{x,y}B$ from $\Gamma, B(t, z) \vdash B(t, Sz)$, z fresh, for matrices B whose root is Q. Then induction is derivable for every native formula.*

Proof. Let $P(n)$ be a native formula. The logical reconstruction of Section 1.2 provides a gate $G_v A$ with $\vdash G_0 A$ and $G_{Sv} A \dashv\vdash A$. Put $G(n) := Q_z G_z P(n)$. Then $G(n) \dashv\vdash P(n)$: from $G(n)$ and the theorem $G_0 P(n)$, QE gives $G_{S0} P(n)$ and hence $P(n)$; conversely $P(n)$ gives $G_{Sz} P(n)$, so DI-case with an unused witness gives $D_{u,z} G_z P(n)$, and DE exports the closure $Q_z G_z P(n)$, in which the fresh parameter does not occur. A derivation of $P(n) \vdash P(Sn)$ therefore yields $G(n) \vdash G(Sn)$. The added rule, with a matrix $B(x, y) \equiv G(y)$ that does not contain x, gives $D_{x,y} G(y)$; DE gives $Q_y G(y)$, and QE with the base $G(0)$ gives $G(t)$. Translating back, $P(0)$ and $P(n) \vdash P(Sn)$ give $P(t)$. □

The strengthening reaches public formulas through the translation of the guarded language, so a restriction of the goal grammar does not repair it. The same observation applies to an arbitrary quantified matrix in the progress rule (`checked_refinement`).

Remark 11.53 (A second source). `verified_refinements` proves Theorem 11.52 again, for a shorter selector that grants progress whenever the generated hypothesis has arity greater than 1, with the formula $Q_vG_vA(y)$ in place of $Q_zG_zP(n)$ and the same two directions of the interderivability. It adds that the gate conversions use only computation equations whose left sides are iterator-headed, so the restriction of transport sources in $\mathcal{U}$ does not block the derivation either. It claims no executable certificate for a modified program. `immediate_application` gives the same argument, with $Q_vG_vF(x)$, against the selector of the three-field profiles that opens the retained resource h at the fresh parameter whenever h does not have arity 1: the selector agrees with the control on the canonical branches (an equation matrix has arity 2, a paired matrix arity 1, an exported closure arity 3), but a goal $D_{x,y}B$ whose matrix is a closure, which an auxiliary subproof may contain, then receives progress. Restricting the public goal to equality and D does not exclude such a subproof. `468_presentation` gives the argument once more, with $K(y) := Q_vG_vA(y)$, against the selector of the stream programs that reads the mode of an introduction from the generated hypothesis alone; the collision of the two generated hypotheses that motivates it is Theorem 11.155.

The same package shows that the strengthening disappears if a closure may be concluded only by a hypothesis.

Lemma 11.54 (Rigidity of raw substitution)**.** *Let $B(y)$ be a raw formula and z a variable that does not occur in B. If $B(z)$ and $B(Sz)$ are equal as trees, or if $B(Sz)$ is equal to a tree in which z does not occur, then y does not occur free in B.*

Proof. Bound occurrences are not replaced. Each free occurrence of y is replaced by z in $B(z)$ and by Sz in $B(Sz)$, so $B(Sz)$ has one more node than $B(z)$ for each free occurrence. If there is one, the two trees differ in size, and $B(Sz)$ contains z. □

Proposition 11.55 (Closures concluded only by hypothesis)**.** *Add to $\mathcal{A}$ the rule of Theorem 11.52 under the side condition that every subderivation whose conclusion has the root Q is an instance of* HYP*. Every derivation of the extended calculus that satisfies the side condition can be transformed into a derivation of $\mathcal{A}$ with the same conclusion and the same open assumptions.*

Proof. Consider an instance of the added rule, with premise $\Gamma, B(t,z) \vdash B(t,Sz)$, z fresh, and B with root Q. The conclusion of the premise has root Q, so by the side condition it is a hypothesis: either $B(t,Sz)$ is $B(t,z)$, or it is a member of Γ, in which z does not occur. In both cases Theorem 11.54 shows that y does not occur free in $B(t,y)$. Then $B(t,0)$, $B(t,z)$ and $B(t,Sz)$ are the same formula, and $\Gamma, B(t,0) \vdash B(t,Sz)$ holds by HYP. DI by cases gives the same conclusion $D_{x,y}B$. Replacing every instance in this way, by induction on the derivation, gives the claim. □

The proposition concerns typed raw syntax. For the permissive arrays of the compact checkers it would need an argument about the decoder, and with normalizing substitution a different rigidity lemma; the implementation that `verified_refinements` examined was larger than the current token and node minima and is not delivered.

11.7.4 Local rewritings of the program text

Three reported changes are identities of the host language and involve no invariant of runs.

In the compact guarded checker, `checked_refinement` writes M° as `s and [s,u] or [t]*3`. If s is false-valued both spellings give $[t,t,t]$; if s is true-valued, $[s,u]$ is a nonempty array, hence true-valued, and both give $[s,u]$. The values agree for all JSON values. The change saves one byte and adds three syntax-tree nodes.

In a compact guarded checker with the decrement written inside the final call, `deterministic_introductio` writes $k-1$ as `~-k`. On integers the complement satisfies $\sim x = -x-1$, so $\sim(-k) = k-1$, and every floor of a run is an integer, being obtained from 0 by decrements. The change saves two bytes and adds one node.

Remark 11.56 (The floor is not a free function of the context). With local floors and initial context [0], every call satisfies $k = 1-|g|$. `checked_refinement` and `validated_presentation` record that this does not shorten the program. The opener depends on the current floor, and the floor is decreased in step (5) before the two openings of step (6), which must admit the new name. A variant that fixes the floor from the length of the context at entry and drops the decrement accepts simple computation certificates and rejects valid certificates with induction under a retained hypothesis or with transport below a binder. A correct variant that recomputes the floor at entry and keeps the decrement is 8 bytes longer than the program it was meant to shorten (529 bytes against 521; an invalid variant has 514). `immediate_application` records the invariant again and notes that with a global floor the counter advances during a major call while the length of the caller's context does not change, so the reconstruction must keep those transitions; its measured compositions were larger. The invariant is proved, the correct variant is shown to have exactly the runs of its control, and the inclusive single bound is refuted in Section 11.9.12.

11.8 Validated presentations

This section covers `validated_core`, `validated_presentation`, `scope_kernel` (its checker part), `checked_refinement`, and the separate implementation refinements reported in `deterministic_introd` `one_body` and `uniform_recursion`. The compact checkers accept goals that are not canonical encodings. A validated checker adds a test of its input, and the results of this section say that the test can share work with the proof kernel. Every result compares a program with a control that satisfies the same input contract.

11.8.1 Input contracts and the public grammar

Definition 11.57 (Public grammar)**.** For a raw tree a and a depth $e \geq 0$ the predicate $P(a,e)$ holds if

- a is an exact integer, e is odd, and $a = 0$ or a is odd with $3 \leq a < e$; or
- a is an array of arity r, where $r \in \{1,\ 2+(e \bmod 2)\}$ or $a = [a_0, a_0, a_0]$, and $P(b, e+5-r)$ holds for every child b.

The *shape predicate* $S(a,e)$ is defined in the same way with the condition $3 \leq a < e$ weakened to $3 \leq a$. The predicates P^{ed} and S^{ed} omit the alternative $a = [a_0, a_0, a_0]$. A raw tree is a *canonical closed formula* of the language with equality, Q and D if $P(a,0)$ holds, and of the language with equality and D if $P^{\mathrm{ed}}(a,0)$ holds. It is a canonical closed *guarded* formula if $P^{\mathrm{ed}}(a,0)$ holds and, in every unary array $[B]$ at a formula position whose child B is binary, each of the two children of B is a ternary array whose first two entries are 5 and 0.

At an even depth the admitted arities are 1 and 2, for D and equality, together with the diagonal triple for Q; at an odd depth they are 1 and 3, for S and the iterator, where the diagonal alternative adds nothing. The depth law of Section 11.2 makes one arity table serve both sorts, since the parity of the depth is the sort. A leaf is the scalar 0 or the odd scalar of a slot in scope. In the guarded condition the entries 5 and 0 are the encodings of the index y, below the dummy slot of the iterator, and of the initializer 0, so the condition is the guard (11.1). The language with equality and D expresses the language with Q: a closure $Q_y B$ is

interderivable with $D_{x,y}B$ for x not in B, by DE in one direction and by DI-case and QE in the other; closures remain available as auxiliary formulas in both cases.

Definition 11.58 (Input contracts)**.** A checker satisfies

- the *raw contract* if it tests nothing about its goal beyond what the proof kernel computes; its claims concern goals that are canonical encodings;
- the *goal contract* for a grammar if it accepts only inputs whose goal is a canonical closed formula of that grammar;
- the *typed contract* for a grammar if it accepts only inputs that are nonempty arrays, all of whose nodes, including those of unused packets, are exact integers or arrays, and whose goal is a canonical closed formula of that grammar.

Sizes of programs under different contracts, or under one contract for different grammars, are not comparable; Section 11.11 lists the contract of every program. Let $J(a)$ state that every node of a is an exact integer or an array. Booleans are not exact integers.

11.8.2 Scope is checked by the first opening

Lemma 11.59 (The first opening supplies the upper bounds)**.** *For every raw tree a with $J(a)$ and every depth e,*

$$P(a,e) \iff S(a,e) \text{ and } O_0(a;\bot,e) \text{ is defined,}$$

and if this holds then $O_0(a;\bot,e) = a$. The same is true of P^{ed} and S^{ed}.

Proof. By induction on a. For an integer at an odd depth, S requires $a = 0$ or a odd with $a \geq 3$; the opening at floor 0 is defined exactly if $0 \leq a < e$, which under S is the missing bound $a < e$, and then the scalar is retained. At an even depth S and P fail for every integer. For an array, S and P impose the same condition on the arity, the children are opened at the depths $e + 5 - r$ at which the predicates are evaluated, and the induction hypothesis applies; the opening rebuilds the array from unchanged children. □

A kernel run starts with $O_0(a_0;\bot,0)$ at floor 0, in step (1), before any packet is read and before the membership test. If a leaf violates its upper bound, the opener consults the missing image; in these programs the missing image is the checking function itself, to which the array test of the opener cannot be applied, and the run rejects. Theorem 11.59 therefore lets a validator omit every scope bound and keep only the shape.

11.8.3 Typed checkers

The typed checkers consist of the native protocol of Theorem 11.47 as proof kernel, preceded by a validation of the whole input. The four programs considered are the control of `validated_core`, an incoming checker with separate validators for raw types and for the goal, and three successive refinements.

Lemma 11.60 (Persistent type invariant)**.** *If the input satisfies J, every raw tree that the kernel handles afterwards satisfies J, with the exception of the missing image. On trees satisfying J the tests `a*0 == []` and `a*0 != 0` both hold exactly for arrays.*

Proof. The kernel constructs trees by forming arrays, repeating an array, decreasing the integer floor, and opening; each preserves J, by induction on the run. The product of an array with 0 is the empty array and that of an integer is 0. □

Theorem 11.61 (One validator with a raw mode)**.** *Let $V(a,d)$ be defined for integers a by: $d=0$, or d is odd and ($a=0$ or a is odd with $2<a<d-2$); and for other values by:* `a*0 == []`*, and ($d=0$ or the arity is in $\{1,2+(d \bmod 2)\}$ or $a=[a_0,a_0,a_0]$), and $V(b,d')$ for all children b, where $d'=0$ if $d=0$ and $d'=d+5-r$ otherwise. Then, for every JSON value a,*

$$V(a,0) \iff J(a), \qquad V(a,\delta+2) \iff J(a) \text{ and } P(a,\delta) \quad (\delta \geq 0).$$

Consequently the program that removes the goal a_0 from the input q, requires $V(q,0)$ and $V(a_0,2)$, and then runs the kernel satisfies the typed contract and accepts the same inputs as its control.

Proof. Here V holds when the evaluation returns a true value; a failed operation counts as false. In raw mode, exact integers pass; among the other values of the JSON reader only arrays satisfy `a*0 == []`, the others giving a different product or a failure; the arity condition is vacuous and the children stay in raw mode. Induction gives $V(a,0) \iff J(a)$. In scoped mode with $d=\delta+2$, an integer passes exactly at odd δ with $a=0$ or a odd and $2<a<\delta$, which is the integer clause of $P(a,\delta)$; an array passes the arity condition of P, and its children are evaluated at $d+5-r=(\delta+5-r)+2$, so the induction hypothesis applies. On an empty array the diagonal alternative fails by an undefined access, which is a rejection.

For the program: an input that is not an array has no last element to remove, and an empty array has none either. For a nonempty array the two requirements say that the prefix and the goal satisfy J and that $P(a_0,0)$ holds. The control tests the same three conditions with two separate recursive functions and then runs the same kernel, up to the spelling of the array test, which agrees on J by Theorem 11.60. □

The control has 1049 bytes and the program of Theorem 11.61 has 897. Of the difference, 117 bytes are obtained by sharing the arity table between the sorts and by dropping a repeated type test, with two separate guards; the remaining 35 come from merging the two input tasks into the one predicate V and one diagnostic.

Theorem 11.62 (Typing by the printed form)**.** *Let Σ consist of the ten digits, the two square brackets, the comma, the space and the minus sign. For a value x of the JSON reader, $J(x)$ holds if and only if the printed representation of x in the host language contains only characters of Σ. Consequently the program that requires this of the whole input, removes the goal a_0, requires $v(a_0,0)$, and runs the unchanged kernel accepts the same inputs as the program of Theorem 11.61. Here $v(a,d)$ requires, for a value a whose product with 0 is 0, that d is odd and that $a=0$ or a is odd with $2<a<d$; and for any other value a, of arity r, that $r \in \{1,2+(d \bmod 2)\}$ or $a=[a_0,a_0,a_0]$, and that $v(b,d+5-r)$ holds for all children b.*

Proof. An exact integer prints as decimal digits with an optional minus sign, and an array prints as the representations of its elements separated by a comma and a space, within brackets; so $J(x)$ implies the condition, by induction. For the converse, each other kind of node contributes a character outside Σ that enclosing arrays preserve: a string a quotation mark, an object a brace, the Booleans and the null value letters, a finite floating-point number a decimal point or an exponent marker, a nonfinite one letters. The program tests the condition by removing characters of Σ from both ends of the printed form and requiring that nothing remains, which holds exactly when no other character occurs anywhere. The test is applied to the whole input before the goal is removed, so an unused packet cannot hide a value of another type.

On values satisfying J the product with 0 is 0 exactly for integers, so v dispatches correctly, and $v(a,d)$ is $V(a,d+2)$ with the raw mode removed: the bound $a<(d+2)-2$ is $a<d$, parities agree, and the children are evaluated at depths that differ by 2. Hence $v(a,0) \iff P(a,0)$ on such values. The goal, the stack and the kernel are then the same in both programs. □

The statement is about values of the standard reader, not about arbitrary objects of the host language. The two programs differ at resource limits: the recursive raw validator fails on an unused packet nested 600 levels deep that the printed-form test accepts. A variant that filters the input text before parsing, with the same characters and the JSON whitespace, has a direct lexical proof and is larger (885 bytes against 870).

Theorem 11.63 (Scope reuse and direct splitting)**.** *In the program of Theorem 11.61, replace the condition $2 < a < d - 2$ of V by $2 < a$, and replace the removal of the last element by a destructuring of the input into a prefix and a last element. The resulting program accepts the same inputs.*

Proof. The changed predicate tests J and S in place of J and P. A goal accepted by the old validator is accepted by the new one and leads to the same first state of the kernel. A goal accepted only by the new validator satisfies S and not P; by Theorem 11.59 its first opening is undefined, and the run rejects before a packet is read; no hypothesis can intervene, because the whole goal is traversed before the membership test. All later operations are unchanged.

The destructuring produces, for a nonempty array, the same prefix and last element. An empty array and a value that cannot be iterated are rejected. A nonempty string or object is destructured into pieces whose last element is a string, which fails the type test of the goal validator. So no input that was rejected is accepted. □

The three refinements have 897, 870 and 881 bytes; the second and the third are alternatives, each measured against the first.

11.8.4 Goal-validated checkers

Theorem 11.64 (Guarded goal validation)**.** *Let $W(a)$ state that every node of a is an exact integer or an exact array, that $S^{\mathrm{ed}}(a, 0)$ holds, and that for every unary array $[B]$ at a formula position with B binary, both children b of B satisfy* `b[:2] == [5,0]`*. A goal a is a canonical closed guarded formula if and only if $W(a)$ holds and $O_0(a; \bot, 0)$ is defined. Hence the checker that tests W and runs the compact guarded kernel accepts the same inputs as the checker with a complete separate validator of guarded formulas.*

Proof. On a child of a binary formula array that has passed the shape test, the prefix test has exactly the successes of the explicit guard test: such a child is an integer, for which the slice is undefined, or a unary array, whose prefix has length one, or a ternary array, for which equality of the prefix with $[5, 0]$ states the two coordinate conditions; its third child has been validated as a term. With Theorem 11.59 this gives the stated equivalence, and the opening returns the goal unchanged. For the programs: the reader, the removal of the goal, the kernel and the verdict are the same. If the complete validator accepts, so does W, and the kernel starts in the same state. If W holds and the complete validator fails, the first opening is undefined and the run rejects before a packet is read. □

The complete validator carries the number of protected slots through the recursion and tests the iterator shape explicitly; the program with W has 921 bytes against 1201.

For the native calculus with public goals of equality and D, the goal contract is satisfied by a kernel preceded by a validator of S^{ed}, by Theorem 11.59. Two controls of this form occur, of 803 and of 780 bytes; they differ in the arrangement of the kernel (in the second the replacement test is evaluated inside the final call). The reported refinements are the following.

Theorem 11.65 (Grammar checked during the first opening)**.** *Give the opener a flag v. For $v = 0$ it is the opener of Theorem 11.10. For $v = 1$ an array is traversed only if its arity lies in $\{1, 2 + (d \bmod 2)\}$; a scalar is retained only if, in addition to $k \le a < d$, it is an exact*

integer, d is odd, and $a = 0$ or a is odd with $a > 1$; and in every other case the missing image is consulted, also at an even depth. Let the checker call the kernel on the goal with $v = 1$, and let the kernel set $v = 0$ after the membership test of step (1) of that call; every other call has $v = 0$. This checker accepts the same inputs as the control that validates S^{ed} by a separate function and then runs the kernel.

Proof. By induction on the goal, the opening with $v = 1$ at floor 0 and depth e is defined exactly if $P^{\mathrm{ed}}(a, e)$ holds, and then returns a. For an integer the conditions are those of P^{ed}: at an odd depth, $a = 0$ or a odd with $1 < a < e$. An integer that fails them, or any scalar at an even depth, leads to the missing image, and the opening is undefined; the ordinary opener would return the scalar 0 at an even depth, which is why the flag forces the failure there. An array of a forbidden arity is not traversed and is handed to the scalar comparison, which is undefined for an array. An array of an admitted arity is traversed at the depths of P^{ed}. By Theorem 11.59 the control accepts the goal exactly under the same condition and starts its kernel in the same state. A goal with $P^{\mathrm{ed}}(a, 0)$ is a nonempty array; whether or not it is a member of the initial context, both programs agree on step (1). After step (1) of the first call the flag is 0 in every call, the opener is the ordinary one, and all packets, majors, contexts and floors coincide. An induction on the consumed stack gives the same verdict and, on acceptance, the same unused prefix. □

The parity of the depth takes the place of the sort argument of a separate validator, as in Theorem 11.57. `deterministic_introduction` realizes Theorem 11.65 against the control of 780 bytes with 656 bytes; `uniform_recursion` realizes it against the same control twice, with an explicit grammar guard (685 bytes) and with the guard merged into the cases of the opener (661 bytes), and notes that malformed goals are rejected at once and not through exhaustion of the recursion limit. Both packages state that the fusion of grammar and scope traversals is not new with them and claim only the measured refinement.

Two further changes are rewritings of a validator of S^{ed} that preserve its successes. `checked_refinement` replaces an early return on integers followed by a return on arrays by one conditional expression with the same two branches (803 to 788 bytes). `one_body` takes the result as control and changes, under the exact-integer branch, $a = 0$ to the negation of a and $a \geq 3$ to $a > 2$; replaces the Boolean sort flag by the numbers 0 and 1, of which only the truth value and the sum with 2 are used; and replaces the exact array test, after integers have been excluded, by `a*0 == []`, which holds exactly for arrays among the values of the reader and fails or is undefined otherwise (788 to 778 bytes). A variant writes M° as in Section 11.7 in addition (777 bytes, with two more syntax-tree nodes than the control). Failures may occur at a different operation than in the control; the accepted inputs are the same.

11.8.5 A rejected saving

The opener returns the scalar 0 for an out-of-range scalar at an even depth and consults the image only at an odd depth. `validated_core` records that removing this distinction of parities gives a shorter program that accepts the canonical guarded formula

$$D_{x,y}\big(I(y, 0;\, u.\, 0) = I(y, 0;\, u.\, S0)\big),$$

which is false: its instance at $y = 0$ holds, and its instance at $y = S0$ is $0 = S0$. The accepted certificate uses a scalar template, which without the parity test is opened both as a formula and as a term. The parity test is the separation of the two sorts and cannot be removed.

`whole_json_refinement` adapts the counterexample to the program of 473 bytes of Theorem 11.146(a), which uses the eager opener of Theorem 11.133, reads four-field packets with

the header $[A, 0]$ and closes computation goals as leaves. Delete the factor $d \bmod 2$ from its fallback, so that an out-of-range scalar is replaced by the opened image at every depth. Let

$$E := \texttt{[[5,0,0],[5,0,[0]]]}, \qquad A := [E], \qquad B := [A, A, A], \qquad M := [B, A],$$

so that A is the canonical encoding of the false formula displayed above. The resulting program (`negative/REJECTED_UNSOUND_no_parity.py`, 268/301/465) accepts the stack

$$[\,[[\,], 0, [\,], 0],\ [[A], 0, 0, 0],\ [[\,], E, A, A],\ [M, B, A, 0],\ [A, 0]\,]$$

of integers and arrays (`negative/false_guarded_goal.json`). The run is as follows. The first packet $[M, B, A, 0]$ has the major M. As a formula M is the equation of the terms read from B and A; at a term position E, of arity 2, reads 0, the tree A reads $S0$, and B reads $I(S0, S0; u.\, S0)$, so M reads $I(S0, S0; u.\, S0) = S0$, an instance of IS. The packet $[[\,], E, A, A]$ proves it as a computation leaf: its candidate is $[[[E], A, A], A] = [B, A] = M$, since the update A contains no reference to the state. The elimination test then compares the goal A with the opening of the scalar template 0 at the image A at the formula depth 0. Without the parity factor the scalar 0, which is out of range at depth 0, is replaced by the image, so the opening is A itself, the test succeeds, and the goal becomes the opening of 0 at the image B, that is, B. As a formula B reads $Q_z A$ with z not occurring in A; the packets $[[A], 0, 0, 0]$ and $[[\,], 0, [\,], 0]$ prove it by DE from $D_{x',y'}A$, which is introduced by cases with the hypothesis A closing the premise. So the scalar template acts as the identity template on whole formulas, and the term equation M licenses the replacement of the false formula A by the theorem $Q_z A$. With the parity factor the opening of 0 at the formula depth is the product of 0 with the opened image, the empty array, which differs from A, and since the program of 473 bytes is sound (Theorem 11.146) and A is false, it rejects the stack. The package reports that the original template 100 of `validated_core` makes the run fail in the computation probe of the newer protocol, which opens the template at depth 3 before the elimination test; the template 0 lies in range there and is retained. The package claims no new obstruction, only the adaptation.

`shifted_shared_leaf` and `468_shared_leaf` adapt the counterexample once more, with the same false formula and with one and the same certificate, to the program T_+ of Theorem 11.153, which uses the 1-shifted encoding, the uniform terminals and the scalar 0 as neutral major. Delete the factor $d \bmod 2$ from the fallback of its opener. In the 1-shifted encoding the variable y, below the dummy slot of an iterator, is written 7, and the scalar template is 2, which is in range at the root term depth 5 and reads 0 there, and out of range at the formula root depth 2. Let

$$E' := \texttt{[[7,0,0],[7,0,[0]]]}, \qquad A' := [E'], \qquad B' := [A', A', A'], \qquad M' := [B', A'],$$

so that A' is the 1-shifted canonical encoding of the false formula. The resulting program (`negative/REJECTED_UNSOUND_shared_no_parity.py` of `shifted_shared_leaf`; `468_shared_leaf` constructs it in its review and does not deliver it) accepts the stack

$$[\,[0, 0, 0, 0],\ [0, 0, 0, 0],\ [0, 0, 0, 0],\ [0, 0, [\,], 0],\ [[A'], 0, 0, 0],\ [0, E', A', A'],\ [M', B', A', 2],\ [A', 0]\,]$$

(`negative/false_guarded_goal_shifted_shared.json` of `shifted_shared_leaf` and `boundary_review/fa` of `468_shared_leaf`, with the same content). The run is that above. The first packet $[M', B', A', 2]$ has the major M', which reads $I(S0, S0; u.\, S0) = S0$; the major call removes $[0, E', A', A']$, whose candidate $[[[E'], A', A'], A']$ is M', and closes. The hypothesis generated by the transport is the sixfold repetition of M', which reads $0 = 0$. Without the parity factor the opening of the template 2 at the image A' at the formula depth 2 is A', the test succeeds, and the goal becomes the opening of 2 at B', that is B', which reads

$Q_z A'$. The packet $[[A'], 0, 0, 0]$ eliminates $D_{x',y'} A'$, whose major call removes the introduction $[0, 0, [\,], 0]$; its neutral major 0 and its premise A', a hypothesis, close by membership with a dummy packet $[0, 0, 0, 0]$ each, and the generated hypothesis B' closes the continuation with the third. With the parity factor the opening of 2 at the formula depth is the empty array, which differs from A', and since T_+ is sound and A' is false, T_+ rejects the stack. `shifted_shared_leaf` also gives the stack without the three dummy packets, $[[0, 0, [\,], 0], [[A'], 0, 0, 0], [0, E', A', A'], [M', B', A', 2], [A', 0]]$ (`negative/false_direct_shifted.json`), which P_+ of Theorem 11.151 without the parity factor (`negative/direct_no_parity.py`) accepts according to the package's replay. Neither package claims a new obstruction, and neither unsound program has a reported size.

11.9 Inferred majors, eager selection and the phase of validation

This section covers `inferred_major` and `smaller_envelope`, which overlap, `frontier_review`, `checked_presentation`, `context_phase`, the two checkers of `progress_independence`, the compact checker of `crossed_uniqueness`, the certificates and programs of `three_field`, the compact part of `constantfree`, the programs of `context_order`, the checkers of `retained_premises`, `checker_refinement`, `502_scope_audit` and `verified_refinements`, the checker part of `single_binder` (its single-binder universal, Theorem 1.31, is not used here), and the audits of sources in `orbit_closure`, `progress_independence` and `verified_refinements`. The archive `context_order` is not one of the packages of the article; its two compact programs and its audit of the schedule of Section 11.9.10 are reproduced, with hashes, in `three_field` and `constantfree`, and are cited from there. All programs are checkers of $\mathcal{U}$ or $\mathcal{A}^-$ in the protocols of Theorem 11.47; no rule is changed. Each device is compared with a control, and for each device the set of accepted stacks is unchanged, or shrinks, or changes in a way that is described exactly, or, where only the accepted canonical goals are compared, this is said. All statements are about stacks whose nodes are integers and arrays.

11.9.1 The controls

The compact controls of this section are programs of the guarded-source protocol of Theorem 11.47 with two differences, both taken from packages outside this chapter. First, step (6) reads: $\iota = k$ if m is true-valued or the arity of a is even. This *parity selector* agrees with the test "the arity of a is 2" at the arities 1, 2 and 3; at the arities 0 and at least 4 the matrix reads $0 = 0$, and the proof of Theorem 11.48 applies without change. Second, there are two arrangements of step (3). In the control of 516 bytes the replacement test precedes step (4), as in Theorem 11.47. In the control of 506 bytes it follows step (4): it is evaluated at the floor k, with the goal as step (4) leaves it, immediately before the decrement of step (5). The control of 506 bytes is the compact program of `deterministic_introduction` in the table of Section 11.11.

Lemma 11.66 (Delayed replacement)**.** *In a checker with local floors, the two arrangements of the replacement test of step (3) have the same returning runs.*

Proof. Let m be true-valued. If the major call returns, the threefold repetition of m is true-valued, so step (4) takes branch (i) and does not change the goal. The test reads the packet, the goal and the opener at the caller's floor; the major call changes none of them, and the opener is pure. So the test has the same value and the same effect before and after step (4). Let m be false-valued. The candidate M° is a nonempty array and is not equal to m, so the test fails in both positions, although branch (iii) of step (4) may have replaced the goal

in between; the opening $O_k(t; u, 0)$ that the test evaluates does not depend on the goal. An opening that is undefined in one arrangement is evaluated, and undefined, in the other. □

11.9.2 The inferred major

`inferred_major` and `smaller_envelope` deliver the same two programs for this device, a loop profile with a global floor and a loop profile with local floors and the scalar neutral premise; their controls are the two loop profiles of Section 11.7. In these controls step (3) tests $m = M^\circ$ and $O_k(t; u, 0) = a$. The first field of an elimination packet is thus determined by its other fields.

Definition 11.67 (Inferred-major protocol)**.** The inferred-major protocol is obtained from its control by replacing step (3) with: if $O_k(t; u, 0) = a$, put $m := M^\circ$ and replace a by $O_k(t; s, 0)$. All later steps use the new value of m; in particular the major call of step (4) is $\mathrm{Check}(M^\circ, g, k)$. On a canonical certificate the first field of an elimination by SE is arbitrary, for instance 0; the first field of a DE packet is the major $[B]$ as before, and its template field is 0.

The major is reconstructed, not omitted: its proof is checked as before, in the unchanged context. The device differs from the constructed major of Theorem 11.26, which is triggered by a true-valued first field; here the trigger is the agreement of the target instance with the goal.

Theorem 11.68 (Repair)**.** *Let the inferred-major checker accept a stack. In each consumed packet at which the test $O_k(t; u, 0) = a$ succeeded, replace the first field by M°, and leave all other fields and packets unchanged. The control accepts the resulting stack, with the same goals, contexts, floors and consumption of packets. Hence the soundness statement of Theorem 11.48 holds for the inferred-major protocol with local floors, and the statement made after Theorem 11.51 for the profile with a global floor.*

Proof. At a packet where the test succeeded, the control finds $m = M^\circ$ and $O_k(t; u, 0) = a$ and replaces the goal by $O_k(t; s, 0)$, as the inferred run does. At a packet where the test failed, the conjunction tested by the control fails, whatever m is, and the first field is unchanged. In both cases the two runs continue with the same major, the same goal and the same floor; the reconstruction of M° is pure and precedes the major call, so with a global floor the floor seen by the major call is the same too. The major call consumes the repaired packets of its own subrun, by induction on the run, and the hypothesis, the decrement and the final call coincide. The repair is determined by the accepted run. □

Lemma 11.69 (Separation of terms and goals)**.** *Let t be the canonical encoding of a term below any number of slots. Then $O_k(t; u, 0)$ is defined for every u, without the image being consulted, and contains no binary array. Every canonical encoding of a formula contains a binary array. Hence $O_k(t; u, 0)$ is not a canonical formula.*

Proof. The arrays of t have arity 1 or 3, so from the depth 0 every node of t is traversed at an even depth, where a scalar is retained or replaced by 0 and the image is not consulted. The traversal preserves arities. A canonical formula ends in an equation, which is a binary array. □

Theorem 11.70 (Completeness with inferred majors)**.** *Every closed theorem of $\mathcal{U}$ has a certificate on which the inferred-major checker accepts its canonical encoding.*

Proof. Take the certificate of Theorem 11.48, with the template field of every DE packet equal to 0. At a packet of SE the test succeeds and M° is the major of the inference, so the run is

that of the control. At a computation packet the field t is the encoding of the update, and at an introduction packet that of the witness; by Theorem 11.69 the test is defined and fails, since every goal of the run is a canonical formula. At a DE packet $O_k(0; u, 0)$ is the scalar 0, which is not a canonical formula. So no packet is taken for an elimination that is not one, and the run is that of the control. A neutral major 0 is closed by the initial context in the local profile and is not checked in the global profile. □

Proposition 11.71 (Change of the accepted stacks)**.** *Compare the inferred-major checker with its control on one stack.*

(a) If at every consumed packet $O_k(t; u, 0)$ is defined, and is equal to the goal only when $m = M^\circ$, the two runs coincide. This holds for the certificates of Theorem 11.70.

(b) A stack accepted by the inferred-major checker and rejected by the control becomes a stack accepted by the control under the repair of Theorem 11.68. Such stacks exist: both packages replace the major field of eliminations by 0 *in certificates of the control and obtain stacks that the inferred-major checker accepts and the control rejects.*

(c) A stack accepted by the control can be rejected by the inferred-major checker only if, at some consumed packet with $m \neq M^\circ$, the opening $O_k(t; u, 0)$ is undefined or is equal to the goal.

Proof. (a) Under the hypothesis the two tests have the same value at every packet, and where they succeed the assignment $m := M^\circ$ changes nothing; an induction on the run gives the claim. The control evaluates $O_k(t; u, 0)$ only when $m = M^\circ$ in some profiles; the hypothesis makes the additional evaluation harmless. Statement (c) is the contrapositive of (a), and (b) is Theorem 11.68 together with the cited certificates. □

In case (c) with equality, the control treats the packet as a DE, a computation or an introduction with the supplied m, while the inferred-major checker replaces the major and the goal; the packages exhibit no such stack and do not claim equality of the accepted sets. The set of accepted canonical goals is the same, by Theorems 11.68 and 11.70. The two programs improve the token and the node coordinate of their controls by three each and are five bytes longer.

Remark 11.72 (A third delivery). `retained_premises` delivers, as its token and node profiles, two programs whose text is identical to the two programs of `inferred_major`; it calls the device goal-directed reconstruction of the major and gives the proofs of Theorems 11.68 to 11.70 in the same form (a repair of the first field for soundness, the odd arities of canonical terms for completeness). It measures them against the controls 256/283/594 and 260/278/602 and its new byte program against 265/306/516, and states the envelope 253/275/513 against the baseline 256/278/516 of its revision; Section 11.11 records why that baseline was not the minimum of the corpus. It adds two focused tests: a reflexivity proved after a wrong supplied major is reconstructed, and a matched target whose inferred major has no proof, which is rejected.

11.9.3 Retained majors

This subsection follows `retained_premises`. Its control is the control of 516 bytes: the guarded-source protocol with local floors, the parity selector, and the replacement test before step (4). The device replaces the backward use of a supplied major by a forward use of a major that the context already contains.

Definition 11.73 (Retained-major protocol)**.** Let M° be formed from the fields s, u, t of the packet $[m, s, u, t]$ as in step (3) of Theorem 11.47. The retained-major protocol is obtained from its control by replacing steps (3) and (4) with:

(3r) If M° is a member of g, run $\mathrm{Check}(O_k(t; s, 0), g, k)$ and put $h := O_k(t; u, 0)$; the goal is not changed, and step (4) is skipped.

(4r) Otherwise perform step (4) of Theorem 11.47: run $\mathrm{Check}(m, g, k)$ and form h as in (N3) of Theorem 11.31. There is no replacement test.

Steps (1), (2), (5) and (6) are those of the control; in step (6), $\iota = k$ if m is true-valued or the arity of a is even, and zero otherwise.

In a step (3r) the first field m has no role except in the selector of step (6). Both openings of (3r) are evaluated at the floor k of the caller, before and after the call on the minor premise, which with local floors does not change it. In terms of the calculus, (3r) is the forward rule

$$\frac{\sigma = \upsilon \in \Gamma \qquad \Gamma \vdash F(\sigma) \qquad \Gamma, F(\upsilon) \vdash A}{\Gamma \vdash A}$$

or its analogue with $Q_y F(y) \in \Gamma$ and the minor premise $F(0)$, which follow from SE and cut.

Theorem 11.74 (Soundness of retained majors)**.** *In the retained-major protocol, let every member of g be closed at floor k. If* $\mathrm{Check}(a, g, k)$ *returns, then a is closed at floor k and the auxiliary reading of a is derivable in* T_{eq} *from the auxiliary readings of the members of g.*

Proof. By induction on the run, as in Theorem 11.48. Packets that take (4r) are treated there: the branch is step (4) of the control, and the absence of a replacement test only removes a case. Consider a packet that takes (3r). The member M° of g is closed at floor k.

Let s be true-valued. Then $M^\circ = [s, u]$ reads $\sigma = \upsilon$, where σ and υ are the readings of s and u at the root term position; they are closed and contain only parameters with names in $[k, 0)$. By Theorem 11.11(c), if the hole occurs in the reading F of t, the trees $O_k(t; s, 0)$ and $O_k(t; u, 0)$ read $F(\sigma)$ and $F(\upsilon)$; if it does not occur, both read F. The call on the minor premise returns, so by the induction hypothesis $F(\sigma)$ is derivable from g, and transport along $\sigma = \upsilon$ in T_{eq} derives $F(\upsilon)$.

Let s be false-valued. Then $M^\circ = [t, t, t]$ reads $Q_y B(y)$, where B is the reading of t at depth 2, and t is closed at depth 2. By Theorem 11.12 the tree $O_k(t; s, 0)$ reads $B(\tau)$ for the reading τ of s, which is 0 for every false-valued spelling for which the opening is defined, and $O_k(t; u, 0)$ reads $B(\upsilon)$. The minor premise gives $B(0)$, and the reading $B(0) \to \forall y\, B(y)$ of the closure gives $B(\upsilon)$.

In both cases $h = O_k(t; u, 0)$ is closed at floor k (Theorem 11.11(a)), hence closed at floor $k - 1$ with the same reading (Theorem 11.14). Its opening in step (6) contains no hole, so by Theorem 11.11(c) it has the reading of h whatever ι is; the traversal of annotations can change the raw tree or make the opening undefined, but not the reading. The goal is closed at floor k by step (1) and Theorem 11.11(b), and so is its opening at $[k - 1]$, with the same reading. The final call returns, and by the induction hypothesis the reading of a is derivable from g and the reading of h, hence from g by cut. □

The argument needs both openings of (3r) at one floor. A program with a global floor that opens the target only after the call on the minor premise can see a lower floor there: a negative scalar of the template that is a hole at the floor of the test becomes a retained parameter. `retained_premises` makes no claim for such a program, and its token profile is the global program with inferred majors instead.

`checked_frontier` shows that this delay is not by itself a source of unsoundness: the delayed target can be used through its existential closure. The package delivers no global-floor

program with retained majors, and the checkers of Section 11.9.13 do not use the following results. Consider a packet $[m, s, u, t]$ that takes (3r) in a call at the floor K, and suppose that the call on the minor premise $O_K(t; s, 0)$ returns at a floor $L \leq K$, the names in $[L, K)$ having been allocated inside the minor proof, and that the target $h' := O_L(t; u, 0)$ is opened only then, at the floor L. Let Γ be the readings of the members of g, all closed at the floor K, and let $\vec{j}$ be the finitely many parameters with names in $[L, K)$ that occur in the reading of h'. By Theorem 11.14 no member of $\vec{j}$ occurs in Γ, in the reading of M°, or in the reading of a goal closed at the floor K.

Lemma 11.75 (Fresh-instance supply)**.** *Let $\vec{j} = (j_1, \ldots, j_r)$ be distinct variables that do not occur free in Γ or in A, and let $w_1, \ldots, w_r$ be terms. If $\Gamma \vdash H[\vec{w}/\vec{j}]$ and $\Gamma, H \vdash A$ in T_{eq}, then $\Gamma \vdash A$ in T_{eq}.*

Proof. Existential introduction with the witnesses $\vec{w}$ gives $\Gamma \vdash \exists \vec{j}\, H$. Existential elimination of the variables $\vec{j}$, one at a time, with the derivation of $\Gamma, H \vdash A$ as minor premise, gives $\Gamma \vdash A$; its eigenvariable condition holds because $\vec{j}$ does not occur free in Γ or A. Equivalently, substitute $\vec{w}$ for $\vec{j}$ simultaneously throughout the second derivation, after renaming its eigenvariables away from the variables of $\vec{w}$, and cut with the first. For $r = 0$ the step is a cut. No induction and no arithmetic axiom is used. □

Proposition 11.76 (Delayed export from a retained equation)**.** *In the situation above let s be true-valued, so that $M^\circ = [s, u]$ reads $\sigma = \upsilon$. Let $F(w)$ be the reading of t at the formula root and the floor K, with the hole w, and suppose that $\Gamma \vdash F(\sigma)$ in T_{eq}. Then the reading H of h' satisfies*

$$H[\upsilon/\vec{j}] = F(\upsilon), \qquad \Gamma \vdash \exists \vec{j}\, H,$$

and for every formula A whose reading contains no member of $\vec{j}$, $\Gamma, H \vdash A$ implies $\Gamma \vdash A$. The same holds for the auxiliary readings.

Proof. The trees s and u are closed at the floor K, since M° is, and by Theorem 11.14 they are closed at the floor L with the same readings σ and υ, which contain only 0 and parameters with names in $[K, 0)$, and no slot.

The equation $H[\upsilon/\vec{j}] = F(\upsilon)$ is proved by induction on t through the positions that have a reading; scalars at formula positions read $0 = 0$ at both floors, and annotations have no reading. Let r be a scalar of t at a term position of depth d. If r reads at the floor K as 0, a slot or a parameter with a name in $[K, 0)$, it lies in $[K, d)$, it reads in the same way at the floor L, and it is retained by O_L. If $L \leq r < K$, it is the hole at the floor K, so it contributes υ to $F(\upsilon)$; at the floor L it lies in $[L, d)$, it is retained by O_L, and it reads as the parameter j_r of $\vec{j}$, which the substitution replaces by υ. Otherwise r is the hole at both floors; O_L replaces it by $O_L(u; \bot, 3)$, which reads as υ at every term position (Theorem 11.11(b),(c)), and $F(\upsilon)$ has υ there as well. Arrays are opened at the depths of their positions, and the arities, hence the binders, are unchanged. Since υ contains no slot and no member of $\vec{j}$, the substitution captures nothing and the two sides agree.

The member $\sigma = \upsilon$ of Γ and $\Gamma \vdash F(\sigma)$ give $\Gamma \vdash F(\upsilon)$ by transport, which is valid in T_{eq} for every template. Hence $\Gamma \vdash H[\upsilon/\vec{j}]$, and existential introduction gives $\Gamma \vdash \exists \vec{j}\, H$. The last statement is Theorem 11.75 with $w_i = \upsilon$. The auxiliary reading of Theorem 11.5 depends only on the shape of a formula, which a substitution of terms for parameters preserves, so the argument applies to it unchanged. □

Proposition 11.77 (Delayed export from a retained closure)**.** *In the situation above let s be false-valued, so that $M^\circ = [t, t, t]$ reads $Q_y B(y)$, and suppose that $\Gamma \vdash B(0)$. Then $\Gamma \vdash H$ for the reading H of h', and $\Gamma, H \vdash A$ implies $\Gamma \vdash A$ for every A.*

Proof. The tree t is closed at depth 2 and the floor K, hence at the floor L with the same reading B (Theorem 11.14). By Theorem 11.12 at the floor L, H is $B(\upsilon_L)$, where υ_L is the reading of u at the root term position and the floor L; it may contain members of $\vec{j}$. The closure and $B(0)$ give $\forall y\, B(y)$, and universal elimination at υ_L gives $\Gamma \vdash H$. A cut gives the rest. □

The existential closure in Theorem 11.76 cannot be dropped. If $t = [b, c]$ with a scalar $b \in [L, K)$ and a scalar $c \geq 3$, both are holes at the floor K, so $F(w)$ is $w = w$ and the minor premise is trivial; at the floor L the target reads $j_b = \upsilon$, which holds for the witness $j_b := \upsilon$ and not for an arbitrary value of j_b. The delayed target is therefore a hypothesis about names that are fresh for the context and the goal, not a theorem in those names. `checked_frontier` states that a complete global-floor checker would also need its other branches, the repair of collisions (Theorem 11.79), the invariants of the global floor and completeness; none of these is proved by a package.

Completeness needs a normal form of certificates, because the dispatch of (3r) depends on the context: a packet meant for (4r) takes (3r) if its candidate M° happens to be in the context.

Definition 11.78 (Compiler-normal certificates)**.** A certificate produced from a derivation is *compiler-normal* if every computation hypothesis and every hypothesis added by (3r) is closed at the floor at which it is formed, before the opening of step (6): every numerical parameter of a computation equation is a parameter already allocated, and no field of a computation packet contains a hole. Fresh markers and aliases introduced below have the same property.

Lemma 11.79 (No collision)**.** *Let g consist of the scalar 0 and of canonical encodings of formulas, and let r be the canonical encoding of a numeral S^n0, $n \geq 1$, that is not a subtree of a member of g.*

(a) A packet $[m, r, 0, t]$ does not take (3r).

(b) A computation packet $[0, 0, u, t]$ whose field t is the canonical encoding of a term does not take (3r).

(c) For a computation packet $[0, s, u, t]$ with s true-valued, put $U := I(0, u;\, v.\, r)$, encoded $[0, u, r]$. In the context g extended by $U = u$, the packet $[0, s, U, t]$ does not take (3r).

Proof. (a) $M^\circ = [r, 0]$, and a member of g equal to it would contain r. (b) $M^\circ = [t, t, t]$ is not the scalar 0; if it were a member $[B, B, B]$, the canonical formula B would equal the term t, but a canonical formula contains a binary array and a canonical term has only arrays of arity 1 and 3. (c) $M^\circ = [s, U]$. A member of g equal to it would contain r. The new member $[U, u]$ is not equal to it, since U has u as a proper subtree and so differs from u. □

Theorem 11.80 (Completeness with retained majors)**.** *Every closed theorem of $\mathcal{U}$ has a compiler-normal certificate on which the retained-major checker accepts its canonical encoding.*

Proof. Prove, by induction on a derivation of $\Gamma \vdash A$ in $\mathcal{U}$, the following strengthened statement: for every context g that consists of 0 and of canonical encodings closed at the current floor k and contains the encodings of Γ, and for every injective renaming of the parameters already allocated, there is a compiler-normal certificate on which $\mathrm{Check}(A, g, k)$ returns; and if A is an equation or a closure, there is moreover a compiler-normal *continuation* for A, a sequence of packets that, run in front of the certificate of any other goal A' in the same scope, adds the encoding of A to the context. Allocated parameters are kept, and new ones are taken at the current floor as local floors require.

Continuations. The principal continuation of the certificate of an equation or closure A (the packets read by the chain of final calls, not by major calls) cannot contain an introduction: an introduction needs a goal of arity 1, and the goal stays an equation or a closure. It therefore consists of computation packets, DE packets and (3r) packets, and ends when A is a member of the context. Each of them adds a hypothesis that does not depend on the goal: a computation hypothesis is closed, so its opening in step (6) does not depend on ι, which is where the arity of the goal enters; a DE packet exports its major with $\iota = k$; a (3r) packet adds a closed tree. The same packets, read while the goal is another closed A', therefore add the same hypotheses; if A' is met as a member earlier, its certificate ends there. This is the continuation for A.

The rules. HYP, DE and the introduction are compiled as in Theorem 11.48, with the unused fields chosen by Theorem 11.79(a): $s = r$ and $u = 0$ for a fresh marker r, so that the packet takes (4r). A zero computation takes (4r) by Theorem 11.79(b). A successor computation with predecessor s, seed u and update t, whose candidate $[s, u]$ is in the context, is replaced by three packets: the zero computation $U = u$ for $U = I(0, u;\, v.\, r)$ with a fresh marker r; the successor computation with seed U, which takes (4r) by Theorem 11.79(c) and adds $F(U)$, where $F(z)$ is the computation equation with its seed replaced by z; and a (3r) packet with source U, target u and template F, whose minor premise $F(U)$ is closed by the context and which adds $F(u)$. A computation with a literal zero seed is first replaced as in Theorem 11.17, so the seed field is true-valued. SE with major $\sigma = \upsilon$ and minor premise $F(\sigma)$, at the goal $F(\upsilon)$, is compiled as the continuation for $\sigma = \upsilon$, followed by the (3r) packet with source σ, target υ and template F, followed by nothing: the call on the minor premise receives the certificate of $F(\sigma)$ in the extended context, which the strengthened induction hypothesis supplies, and the added hypothesis $F(\upsilon)$ is the goal, which the final call finds in the context. SE at the literal source 0 with major $Q_y B(y)$ is compiled in the same way with the continuation for the closure and the minor premise $B(0)$. Every packet is formed with parameters that are already allocated, so the certificate is compiler-normal, and the renaming clause lets the induction hypothesis be applied below the parameters allocated by the continuations. □

The normal form is used for completeness only. An arbitrary computation packet accepted by the checker may contain holes and needs only a known zero instance (Theorem 11.18); its opening in step (6) can depend on the arity of the goal, and such a packet cannot in general be moved in front of another goal. The statement that a continuation can be appended is not claimed for arbitrary accepted runs.

Proposition 11.81 (Relation to the control)**.** *The retained-major checker and the control of* 516 *bytes accept the same canonical goals, the closed theorems of* $\mathcal{U}$*. Some stacks accepted by the control are rejected by the retained-major checker. The converse inclusion is not claimed.*

Proof. The first statement is Theorems 11.74 and 11.80 with Theorem 11.48. For the second, a packet of the control whose candidate M° is in the context by chance is processed by (3r) instead of its intended branch, and a backward elimination packet whose major is not in the context is processed by (4r), which exports the major and leaves the goal unchanged. The package exhibits three certificates accepted by the control that the retained-major checker rejects, each with a successor computation whose candidate is in the context, and translates them by the repair of Theorem 11.79(c) into certificates that it accepts. Certificates that use (3r) are written for a branch that the control does not have, and the package does not compare them with the control. □

The translator of certificates is untrusted; it is tested on the canonical certificates of the package and on the three certificates with collisions, and support of arbitrary aliased certificates is not claimed. The byte program has 265 tokens, 301 nodes and 513 bytes against 265/306/516; its readable expansion is a separate program.

Remark 11.82 (Validated companions). `retained_premises` adds two validated programs. The first adds to the byte program the test of the printed form of the whole input (Theorem 11.62) and a recursive test of the guarded goal grammar, and satisfies the typed contract for canonical guarded formulas; its correctness is that of the kernel together with Theorem 11.62 and Theorem 11.64. The second keeps the kernel of the native goal-validated program of `checked_refinement` unchanged and changes its validator in three ways that preserve the accepted inputs. Arrays are recognized by $a \cdot 0 = [\,]$, which on JSON values is true for lists, false for strings, floats and Booleans, and undefined for `null` and objects; the depth flag is the integer 0 or 1 instead of a Boolean. The leaf condition becomes $a \bmod 2 = [a > 1]$, which for $a \geq 0$ holds exactly when $a = 0$ or $a \geq 3$ is odd, as before, and admits in addition the even negative integers. These cannot be accepted: in the first opening at floor 0 every numerical leaf of a well-shaped goal lies at an odd depth, a negative leaf fails $0 \leq a < d$, and the missing image rejects. Finally a goal that fails the validator is replaced by the checking procedure itself, on which the first opening fails, instead of raising an error with a message; this accounts for 65 of the 88 bytes saved. The validator alone no longer recognizes the grammar exactly; the excluded negative leaves are delegated to the scope traversal.

11.9.4 Eager selection of the goal

`frontier_review` changes the form of the replacement in the control of 516 bytes; `checked_presentation`, `progress_independence` and `crossed_uniqueness` apply the same change to the control of 506 bytes. Write C for the condition $m = M^{\circ}$ and $O_k(t; u, 0) = a$, which these controls evaluate as one comparison of pairs, and R for $O_k(t; s, 0)$. The control takes the new goal to be R if C holds and R is true-valued, and a otherwise, and evaluates R only if C holds. The refinement evaluates R in every case and takes the new goal to be R if C holds and a otherwise; in the program this is the indexing of the pair (a, R) by the truth value of C.

Theorem 11.83 (Eager selection). *(a) Every stack accepted by the refined checker is accepted by its control, with the same goals, contexts, floors and consumption of packets.*

(b) The converse fails: there is a stack accepted by the control and rejected by the refined checker.

(c) Every closed theorem of $\mathcal{U}$ has a certificate accepted by the refined checker.

Proof. (a) Whenever R is defined, the two forms select the same goal. This is clear if C fails, and if C holds and R is true-valued. Let C hold and R be false-valued. By Theorem 11.32 an opening of an array at the depth 0 is an array of the same arity, so a false-valued R arises only from the empty array t or from a numeric t; in both cases the opening does not depend on the image, so $R = O_k(t; u, 0) = a$. The opener is pure, so on an accepted run of the refined checker every operation of the control is available with the same value, and an induction on the run gives (a).

(b) Insert, before a certificate of a computation, the packet $[0, 3, [0], [5, 0]]$. It has $m = 0$, so C fails, and the control processes it as a computation packet that adds a hypothesis. The refined checker evaluates $R = O_k([5, 0]; 3, 0)$: the scalar 5 is exposed at the depth 3, the image 3 is validated at the depth 3, where it is out of range, and the run rejects.

(c) If $O_k(s; \bot, 3)$ is defined, then $O_k(t; s, d)$ is defined for every tree t of integers and arrays and every depth d: by induction on t, an array leads to finitely many calls on its children, and a scalar is retained, replaced by 0, or replaced by the validated image, which starts no further substitution. In the certificate of Theorem 11.48 choose the inert fields as follows: $s = u = t = 0$ in a DE packet and $s = 0$ in an introduction packet. Then in every packet s is 0 or the canonical encoding of a term, $O_k(s; \bot, 3)$ is defined, and so is R. The run of the refined checker is then the run of the control. □

Since C is evaluated as a comparison of pairs, $O_k(t; u, 0)$ is evaluated at every packet already in these controls; the refinement adds only the evaluation of R. The accepted canonical goals are unchanged by (a) and (c). The refinement saves three bytes in both controls. In the arrangement of Theorem 11.66 with the test after step (4), all openings of the test are evaluated at the floor k, before the decrement that the same expression performs.

11.9.5 The remainder selector

`smaller_envelope` replaces the parity selector of step (6) by the remainder of 3 modulo the arity of a: $\iota = k$ if m is true-valued or this remainder is not 0.

Proposition 11.84 (Remainder selector)**.** *The checker with the remainder selector satisfies Theorem 11.48: it is sound for the auxiliary reading and accepts a certificate of every closed theorem of $\mathcal{U}$. It rejects every stack on which step (6) is reached with a false-valued m and a goal of arity* 0*. Equality of the accepted stacks with the control is not claimed.*

Proof. At the arities 1, 2 and 3 both selectors have the values 0, 1 and 0, so every packet whose matrix or goal is a meaningful formula is treated as in the control; in particular a matrix that is a closure receives the zero instance, as Theorem 11.52 requires. At the arity 0 the remainder is undefined and the run rejects. At an arity of at least 4 the remainder is 3 and $\iota = k$; the parity selector gives $\iota = k$ at even and zero at odd arities. In an introduction such a matrix reads $0 = 0$, and the conclusion is valid for either hypothesis; in a computation Theorem 11.18 holds for either value of ι. So the soundness induction is unchanged. Canonical certificates contain only matrices and goals of arity 1, 2 or 3 at step (6), so the completeness part is unchanged. □

At an arity of at least 4 the two selectors can retain different raw hypotheses with the same reading $0 = 0$, and a later membership test can then differ; this is why only soundness and the equality of the accepted canonical goals are stated. The selector saves one byte. `checked_presentation` examines a further selector, which inspects the arity of the generated hypothesis, and rejects it for the reason given in Theorem 11.52; it also notes that in the native protocol the two false-valued modes 0 and [] of the field u must remain distinct, since identifying them removes the introduction by cases at equations.

The program of 502 bytes, which `checked_presentation`, `progress_independence` and `crossed_uniqueness` deliver with identical text, is the control of 506 bytes with eager selection and the remainder selector. By Theorems 11.83 and 11.84 it satisfies Theorem 11.48. Its accepted stacks are included in those of its control as long as no formula of arity 0 or at least 4 reaches step (6); without this restriction neither inclusion is claimed. `502_scope_audit` delivers a program of 502 bytes with the same two changes and the same sizes, together with the intermediate program with the remainder selector only (266/301/505), and proves them in the same two steps: the remainder selector by the argument of Theorem 11.84, and eager selection by the inclusion of successful runs of Theorem 11.83(a) and the definedness of the extra opening on canonical packets of Theorem 11.83(c). Its fixtures show one difference of raw acceptance among the three programs, the empty array as an administrative truth, which the remainder selector rejects.

11.9.6 Truth-valued results and the tail of a packet

Two changes of `inferred_major` and `progress_independence` preserve every run.

Proposition 11.85 (Truth-valued result)**.** *Let the checking procedure of the control of* 506 *bytes return the truth value of the membership test of step (1) if it is true, and otherwise a one-element list whose element is the pair formed by the hypothesis of step (4) and the result*

of the final call; and let branch (i) of step (4) be written as the conjunction of the major call and the threefold repetition of m. The resulting checker has the same returning runs, with the same consumption of packets and the same floors, as the control.

Proof. By induction on the run, every returning call returns a true value: the value of a successful membership test, or a nonempty list. The control forms the hypothesis as the disjunction of the result of the major call, which is the null value, and the threefold repetition; the new form is the conjunction of a true value and the threefold repetition. Both have the value of the threefold repetition, and if that is false-valued both continue with the same later alternatives. The list has one element, obtained from one removal of a packet, and its pair is evaluated from left to right, so the hypothesis is formed before the goal is selected, the floor decreased and the final call made, in the order of the control. A failure in either component propagates. □

The new results retain data that the control discards, so the two programs need not behave alike under a fixed bound on memory. The program has 503 bytes with the complement spelling of the decrement (Section 11.7) and 505 with the ordinary one.

In the global loop profile with inferred majors, `progress_independence` destructures a packet into m and the list r of the remaining fields, then r into s, u and t, and uses r for the image $[s, u, t]$ of step (4ii) and its first two entries for the major $[s, u]$. The second destructuring succeeds exactly if r has three entries, the list r is not modified, and a packet of another arity is rejected by one of the two destructurings as it is by the control. All runs are preserved. The change saves two tokens and costs three nodes and six bytes.

11.9.7 The phase of validation

The control is the native goal-validated checker of Theorem 11.65 with 656 bytes, whose opener has the flag v.

Proposition 11.86 (Flag as an order)**.** *In the checker of Theorem 11.65 write each guard "$v = 0$ or p" as $v \le p$, and the disjunction of the parity of the depth with v as their bitwise disjunction. The accepted inputs and the unused prefixes are unchanged.*

Proof. The flag is 0 or 1, and so is the parity of the depth, which starts at 0 and changes by integers. Each guarded condition p has the value 0, `False` or `True`: the arity condition is a Boolean and is evaluated only for arrays, and the leaf condition is evaluated only after the range test has succeeded and begins with an exact-integer test. For such values $v \le p$ holds exactly if $v = 0$ or p is true, and the bitwise disjunction of two bits is their disjunction. The new form evaluates p also when $v = 0$; by the preceding remarks this evaluation is defined wherever the control reaches the guard. □

`context_phase` removes the flag. The context is an immutable tuple and is initially empty. The opener applies the grammar conditions of Theorem 11.65 exactly when the context is empty. After the membership test of step (1), every call that does not return appends the scalar 0 to its context, by forming a new tuple.

Theorem 11.87 (Phase by context)**.** *The checker of `context_phase` accepts the same inputs as the control of Theorem 11.65, with the same consumption of packets and the same unused prefix.*

Proof. In the first call the context is empty and the control has $v = 1$ and the context $[0]$. The guards of the two openers have the same truth values, so the first opening is defined in one program exactly if it is in the other, and returns the goal unchanged. A goal that passes is a nonempty array; it is a member neither of the empty context nor of $[0]$. Both programs

therefore continue; the control sets $v = 0$, and the new checker appends 0. The opener of the first call refers to the rebound context and from then on applies no grammar condition; every later call receives a nonempty context. So there is one validating traversal in each program.

For the remaining run, relate a call of the control and a call of the new checker if their floors and stacks agree, their goals agree up to the replacement of the integer 0 by the Boolean `False` at formula positions, and the tuple has the members of the list up to the same replacement, together with additional copies of 0. At an even depth the new opener can return `False` where the control returns 0; the two are equal, have the same truth value, and behave alike under every operation of the kernel, and the exact-integer test that separates them is applied only in the first opening. Related calls agree on the membership test, since membership does not depend on multiplicity. Otherwise both remove the same packet, their major calls are related, they select the same branch and form related hypotheses, and they decrease the floor at the same point. The additional 0 is a formula that the context already contains. The final calls are related, and an induction on the run in both directions gives the claim. □

The floor of this checker is not a function of the length of the context, because of the additional copies of 0; the observation after Theorem 11.27 does not apply to it, and the argument would fail for a variant that computed the floor from the context. The shortest variant tests exact integers by the identity of a leaf with its conversion to an integer, which is correct for the values of the standard reader of the measured runtime and is not a statement about other objects; a variant with the explicit type test is three bytes longer. A further variant extends a mutable list by the scalar 0 only. Since 0 is already a member after the first step, a major call that appends to the list of its caller adds no formula; actual hypotheses are still added by forming a new list. This variant is correct for one invocation of the program and is not offered as a reusable procedure, because its default context is no longer empty after the first call.

The checker of Theorem 11.86 and the checker of Theorem 11.87 have the same three sizes and are different programs.

11.9.8 A typed native checker and typed guarded checkers

`checked_presentation` composes devices of Section 11.8 and of the present section into a checker with the typed contract for the grammar with equality and D. The driver applies the printed-form test of Theorem 11.62 to the whole input and, if it fails, has no procedure with which to remove the goal. The kernel is that of Theorem 11.65, with the replacement test after step (4) and with eager selection. Because every node of the input is an exact integer or an array, the exact-integer test in the flagged opener is omitted and arrays are recognized by a product with 0 that differs from 0 (Theorem 11.60).

Theorem 11.88 (Typed native checker)**.** *This checker satisfies the typed contract for the grammar with equality and D. Every input that it accepts is accepted by the typed checker of Theorem 11.62 for that grammar, with the same run, and every closed theorem of $\mathcal{A}^{-}$ in that grammar has a certificate that it accepts.*

Proof. By Theorem 11.62 the driver reaches the kernel exactly for nonempty arrays whose nodes satisfy J; an input that is not an array has no last element to remove. By Theorems 11.59 and 11.65 the first opening is defined exactly if the goal satisfies P^{ed} at the depth 0, which is the condition that the control tests by a separate predicate; the omitted exact-integer test is implied by J for nodes of the input, and values generated by the kernel reach the opener only after the flag is 0. By Theorem 11.66 the position of the replacement test does not change the returning runs. By Theorem 11.83(a) every accepted run is a run of the checker without eager selection, and by part (c) of that theorem, in the form for the native

protocol, canonical certificates are accepted. The decrement is written with the complement, which is the same integer. □

The inclusion is proper in general: eager selection can reject a stack with an ill-scoped inert field. The program has 673 bytes against 849 for its control. The program of 656 bytes from which its kernel is taken satisfies the goal contract only and is not its control.

Four programs add the typed contract to a compact guarded kernel; they test J on the whole input, by a recursive predicate or by the printed form, test the guarded goal grammar as in Theorem 11.64, and run the kernel. Their correctness is that of the kernel together with Theorem 11.62 or the definition of J, and Theorem 11.64. `frontier_review` wraps its kernel of 513 bytes (1183 bytes against 1186 for the same wrapper around the control). `checked_presentation` and `crossed_uniqueness` wrap the kernel of 502 bytes with a recursive validator (1172 bytes; against 1176 for the wrapper around the control of 506 bytes in the first package and against the 1183 bytes of `frontier_review` in the second). `progress_independence` wraps the same kernel with the printed-form test (869 bytes, without a stated control). `smaller_envelope` supplies a readable program of this contract (2473 bytes) as an interface for audit.

11.9.9 Three-field certificates

This subsection follows `three_field`. Its controls are the two loop profiles with inferred majors: the profile with a global floor of `progress_independence`, which reuses the tail of a packet (the paragraph after Theorem 11.85), and the profile with local floors and the scalar neutral premise of `inferred_major` and `smaller_envelope` (Theorem 11.67). In these controls the first field m of a packet is overwritten whenever the test of step (3) succeeds; otherwise it is the major of a DE packet or a false-valued field without role, and in a DE packet the fields s, u, t are inert. Three fields suffice: the major of a DE packet moves into the source field, and the inferred major is stored in the source field as well.

Definition 11.89 (Three-field protocol)**.** A packet is an array $r = [s, u, t]$ of exactly three fields. The three-field protocol is obtained from its control by the following changes.

(3′) If $O_k(t; u, 0) = a$, perform in this order: replace a by $O_k(t; s, 0)$; replace s by M°, that is, by $[s, u]$ if the field s of r is true-valued and by $[t, t, t]$ otherwise; replace u by the scalar 0.

(4′) If u is true-valued, form h from r as in step (4ii) of Theorem 11.15. If u is false-valued, run the major call $\mathrm{Check}(s, g, k)$ (with local floors always; with the global floor only if s is true-valued) and let h be the threefold repetition of s if this is true-valued; otherwise the packet is an introduction: the goal $a = [B]$ is replaced by $O_k(B; t, 2)$ and $h = a$.

(6′) Let $\iota = k$ if s is true-valued or the arity of a is 2, and zero otherwise; the final step is that of the control.

The *global three-field profile* keeps the global floor, the empty initial context and the loop of the global control; the *local three-field profile* keeps local floors, the initial context [0] and the loop of the local control (Theorem 11.51). The *expansion* of a packet is

$$E([s, u, t]) := \begin{cases} [0, s, u, t], & u \text{ true-valued}, \\ [s, s, u, t], & u \text{ false-valued}, \end{cases}$$

and $E(\sigma)$ is the stack σ with every consumed packet expanded.

After a successful test, s holds the nonempty array M° and u is 0, so the packet takes the major branch of step $(4')$: the reconstructed major is proved in the unchanged context, not assumed. The new goal is formed from the original source before s is overwritten. Without a successful test the fields keep their values, so a computation packet has the predecessor in s and a DE packet its major. The only place where the three-field protocol and its control, run on the expanded packets, can differ is step $(6')$ at a computation packet: with a true-valued predecessor s and a goal of arity other than 2, the hypothesis is opened at the fresh parameter, where the control, whose m is 0, opens it at zero.

Theorem 11.90 (Soundness of three fields)**.** *In either three-field profile, let every member of g be closed at floor k. If* Check(a, g, k) *returns, then a is closed at floor k, and the auxiliary reading of a is derivable in* T_{eq} *from the auxiliary readings of the members of g.*

Proof. By induction on the run, as in Theorem 11.48; for the global profile the induction is the one stated after Theorem 11.51. Step (1) is unchanged. Consider a packet that is consumed.

If the test of $(3')$ succeeds, the major that is proved is M°, the goal is replaced exactly as in the control with $m = M^\circ$, and the hypothesis is the threefold repetition of M°, an array of arity 6 or 9 that reads $0 = 0$. The analysis of step (3) in the proof of Theorem 11.48 applies: for a true-valued source the major reads $\sigma = \upsilon$ and the replacement is a transport, for a false-valued one it reads $Q_y B(y)$ and the new goal reads $B(0)$. The hypothesis is discharged by Theorem 11.3, and $\iota = k$ since s is true-valued.

If the test fails and u is false-valued, the packet is processed exactly as the control processes $[s, s, u, t]$: the major call, the hypothesis, the introduction and the selector depend on s in the same way as there on m. The cases of Theorem 11.48 for a unary major (DE through the auxiliary reading), for a major of arity at least 2 or a numeric scalar (the hypothesis reads $0 = 0$), and for an introduction apply.

If the test fails and u is true-valued, the packet is a computation and h is the raw tree formed in step (4ii) of Theorem 11.15 from r. Whether ι is the new floor or zero, Theorem 11.18 applies: its proof treats both readings of ι. Hence the zero instance $H(\vec{0})$ of the reading $H(\vec{b})$ of the opened hypothesis is an instance of I0 or IS, where $\vec{b}$ are the parameters with names in $[k-1, k)$, which are fresh for g and a by Theorem 11.14. Theorem 11.2, applied in T_{eq} as in Theorem 11.48, derives the reading of a from g, whatever the shape of a; it is not an induction rule and does not depend on the goal being an equation. □

Lemma 11.91 (Stable computation hypotheses)**.** *Let h be the canonical encoding of a closed computation equation whose parameters have names in $[k, 0)$. Then $O_{k-1}(h; \iota, 0) = h$ for every image ι, in particular for $\iota = k - 1$ and $\iota = 0$.*

Proof. In the canonical encoding of a formula without free slots there is no scalar at a formula position, and every scalar at a term position of depth d is 0, a parameter name in $[k, 0)$, or a slot scalar $3 + 2j$ below more than j slots, hence in $[k-1, d)$. Every scalar is retained, no hole occurs, the image is never consulted, and arrays are rebuilt with the same children. □

Theorem 11.92 (Completeness with three fields)**.** *Every closed theorem of $\mathcal{U}$ has, for each three-field profile, a stack of triples on which the checker accepts its canonical encoding. The packets are*

$$[s, u, F], \qquad [0, u, B], \qquad [[B], 0, 0], \qquad [s, u, t] \ (u \textit{ true-valued}), \qquad [0, 0, w]$$

for SE at a source other than the literal 0, SE at the literal 0, DE, a computation and the introduction.

Proof. Take the certificate of Theorem 11.70 for the control of the same profile and replace each packet by the triple of the same inference. The two runs agree packet by packet, with the same goals and contexts as raw trees and the same floors. At SE the test succeeds as in the control, $(3')$ installs the major of the inference in s, and the run continues as the control's with $m = M^\circ$. At DE the test compares $O_k(0;0,0) = 0$ with a canonical formula and fails; $s = [B]$ is proved and exported as $[B, B, B]$ with $\iota = k$, as in the control with $m = [B]$. At an introduction and at a computation the field t is a canonical term, so by Theorem 11.69 the test is defined and fails. An introduction then proceeds as in the control (the major call on $s = 0$ is closed by the initial context in the local profile and omitted in the global one). At a computation h is the raw tree formed by the control, which is the canonical encoding of the computation equation (proof of Theorem 11.21); by Theorem 11.91 the opening of h at the fresh parameter and at zero coincide, so the change of step $(6')$ has no effect. Computations with a literal zero initializer or predecessor are first replaced as in Theorem 11.17, so u is true-valued. □

Proposition 11.93 (Relation to the inferred-major checker)**.** *Compare a three-field profile with its control.*

(a) For a stack σ of triples, the run of the three-field checker on σ and the run of the control on $E(\sigma)$ agree, with the same goals, contexts, floors and consumption of packets, up to the first computation packet without a successful test at which s is true-valued, the goal has an arity other than 2, and the two openings $O_{k-1}(h; k-1, 0)$ and $O_{k-1}(h; 0, 0)$ are not both defined and equal. If neither run reaches such a packet, the three-field checker accepts σ if and only if the control accepts $E(\sigma)$.

(b) The certificates of Theorem 11.92 reach no such packet; the control accepts their expansions, in which a DE packet reads $[[B], [B], 0, 0]$ instead of $[[B], 0, 0, 0]$.

(c) The accepted canonical goals are the same for the three-field checker and its control: the closed theorems of $\mathcal{U}$, and for canonical public formulas exactly the public theorems (Theorem 11.6).

(d) Deleting the first field of an accepted four-field stack is not a translation: the DE packet $[[B], 0, 0, 0]$ becomes the introduction packet $[0, 0, 0]$. A stack in which a consumed packet has four fields is rejected by a three-field checker, and conversely.

Proof. (a) The test of step (3) of the control does not read m. If it succeeds, both checkers prove M°, select the same goal and export the same hypothesis with $\iota = k$. If it fails and u is false-valued, E puts s into m, and the two checkers act identically. If it fails and u is true-valued, E puts 0 into m: the control closes the major 0 by its initial context or omits it, and both form h from the same fields; the selectors agree unless s is true-valued and the arity of the goal is not 2. An induction on the run gives (a). Statement (b) is Theorem 11.91 applied to the run of Theorem 11.92, (c) follows from Theorems 11.90 and 11.92 and Theorems 11.48 and 11.70, and (d) is immediate from the destructuring of a packet. □

At a packet excluded in (a) the two checkers extend the context by different raw trees, both sound, and a later membership test can differ. The package exhibits no stack accepted by one side and rejected by the other, and claims neither inclusion; it claims the equality of the accepted canonical goals, from which the bridge of Theorem 1.119 transfers unchanged. The translation of certificates in the package is derivation-aware: it regenerates each triple from the role of the packet, as in Theorem 11.92.

Remark 11.94 (Sources and evidence). `three_field` states the separation of terms and formulas for trees of arity 0, 1 and 3 opened at images of the same shape, which contains Theorem 11.69. It passes through an *intermediate protocol*, which keeps an explicit temporary major, 0 for a true-valued u and s otherwise, and uses it in steps $(4')$ and $(6')$; by the proof of (a) of Theorem 11.93 the intermediate protocol on σ is exactly the control on $E(\sigma)$. Its regression evidence regenerates 26 canonical proofs (9782 consumed packets: 174 introductions, 158 DE, 4716 computations, 4734 eliminations by SE) and checks seven rejected stacks, among them a four-field packet and a leak of a context; these are finite tests. Both programs pass the context as variadic arguments of the procedure, which changes no run (Theorem 11.97).

The global three-field profile has 249 tokens, 278 nodes and 607 bytes against 251/283/605 for its control; the local profile has 255/271/606 against 257/275/607, below its control in all three coordinates.

11.9.10 Allocation before the elimination test

This subsection follows the audit of `context_order` reproduced in `three_field`, the compositions of `three_field`, and the compact part of `constantfree`. In the arrangement of Theorem 11.66 with the replacement test after step (4), the test is evaluated at the floor k, immediately before the decrement. The schedule of `context_order` moves it after the decrement.

Definition 11.95 (Allocation first)**.** A checker of the guarded-source protocol with local floors and the replacement test after step (4) *allocates first* if it performs, in this order: step (4) at the floor k, including the major call, the formation of a computation hypothesis with its image check, and the opening of an introduction; the decrement to $b := k - 1$; the opening $O_b(h; \iota, 0)$ of the hypothesis; and only then the test $m = M^\circ$ and $O_b(t; u, 0) = a$, the selected goal $O_b(t; s, 0)$ or a, and its opening at $[b]$. The native protocol allocates first in the same way.

Compared with the arrangement of Theorem 11.66, only the two openings of the test are evaluated at a different floor. A scalar b of the template at a term position is a hole at the floor k and the parameter b at the floor b, so the test can change its value, not only its definedness.

Proposition 11.96 (Allocation first)**.** *(a) A checker that allocates first satisfies the soundness statement of Theorem 11.48, for the auxiliary reading and $\mathcal{U}$, or for the ordinary reading and $\mathcal{A}^-$ in the native protocol.*

(b) The certificates of Theorem 11.48, with all inert fields 0*, are accepted, with the run of the checker that tests at the floor k.*

(c) The stack

```
[[0,0,[0],0],[0,0,0,0],[[[]],0,-1,[-1,0]],[[0,[0],0],[0]]]
```

with goal $I(0, S0; u.\, 0) = S0$ is accepted by the checker that allocates first and rejected by the control of 506 *bytes.*

Proof. (a) Everything before the test is the computation of the control at the floor k, and the cases of Theorem 11.48 for the hypothesis apply unchanged; the hypothesis is opened at the floor b as before. It remains to treat a successful test at the floor b. The major $m = M^\circ$ was proved by a call at the floor k and is closed there, and so is a; by Theorem 11.14 both are closed at the floor b with the same readings, which contain no parameter named b. If s is true-valued, the major reads $\sigma = \upsilon$, and by Theorem 11.11(c) at the floor b the goal $a = O_b(t; u, 0)$ reads $F(\upsilon)$ and the new goal $O_b(t; s, 0)$ reads $F(\sigma)$, for the reading F of t at

the floor b. Transport along $\sigma = \upsilon$ is valid in T_{eq} for every template, so the reading of a follows from that of the new goal. If s is false-valued, the major reads $Q_y B(y)$; by Theorem 11.12 at the floor b the goal reads $B(\tau)$ for some term τ and the new goal reads $B(0)$, and $B(\tau)$ follows from $B(0)$ and the closure. In both cases the hypothesis is the threefold repetition of M° and reads $0 = 0$ (Theorem 11.3).

(b) In these certificates every field is 0 or a canonical encoding whose parameters have names in $[k, 0)$. No scalar of such a tree lies in $[k-1, k)$, so every opening at the floor $k-1$ returns the same raw tree as at the floor k, and the test has the same value as in the control.

(c) The packet consumed first, $[[[\,]], 0, -1, [-1, 0]]$, has the major $[[\,]]$, which reads $D_{x,y}(0 = 0)$, and the template $[-1, 0]$. The control evaluates $O_0([-1, 0]; -1, 0)$ at the floor 0: the scalar -1 is a hole, and the image -1 fails its validation at depth 3, so the run rejects. The checker that allocates first evaluates the same opening at the floor -1, where -1 is retained; the test fails, since $m \neq M^\circ$, and the run proves the major with the introduction packet $[0, 0, 0, 0]$ and then the goal with the computation packet $[0, 0, [0], 0]$. □

When the test succeeds, Theorem 11.11(c) shows in addition that the reading of the template contains no parameter b and that a used target image contains none: every occurrence of b in the template or in a used image persists in the result, which is the goal a. The audit of `context_order` states soundness in this form, as an instance of SE with data of the old scope; the proof above does not need it. By (c), the accepted stacks are not included in those of the control. The converse inclusion is not claimed either.

Proposition 11.97 (Compositions)**.** *Each of the programs (i)–(iv) below satisfies Theorem 11.48 for $\mathcal{U}$, and the program (v) satisfies Theorem 11.88 for the typed contract, in both cases with completeness for the certificates of Theorem 11.48 with inert fields* 0*.*

(i) The byte profile of `context_order`*: the control of* 506 *bytes allocating first, with the goal and the context exchanged in the argument list.*

(ii) The loop profile of `context_order`*: the loop profile of* `deterministic_opening` *allocating first.*

(iii) The byte program of `constantfree`*: (i) with the remainder selector of Theorem 11.84.*

(iv) The byte program of `three_field`*: (i) with eager selection (Theorem 11.83) and the remainder selector.*

(v) The native program of `three_field`*: the typed native checker of Theorem 11.88 allocating first.*

(vi) The token program of `constantfree`*: the global loop profile of* `progress_independence` *with the context passed as variadic arguments.*

The program (vi) has the same returning runs as its control.

Proof. (i) and (ii) are Theorem 11.96, with Theorem 11.51 for the loop; exchanging two arguments changes nothing. For (iii) the proof of Theorem 11.84 applies verbatim, since it concerns only the value of ι, which allocation first does not change: at the arities 1, 2 and 3 the two selectors agree, at the arity 0 the remainder rejects, and at arities of at least 4 the matrix reads $0 = 0$. For (iv), the proof of Theorem 11.83(a) uses only that a false-valued result of an opening at the root arises from an empty array or a numeric template, whose opening does not depend on the image (Theorem 11.32); this holds at every floor, and both openings of the selection are evaluated at the same floor b. The proof of (c) of that theorem needs $O(s; \bot, 3)$ to be defined for the inert fields, which holds at the floor b since it holds

at k. Selection and selector act on different steps, so the arguments compose. For (v), the first opening of the goal, with the flag of Theorem 11.65 set, precedes every packet; every later opening, including the two of the test at the floor b, is unflagged, and the native mode test (N4) is unchanged; the proof of Theorem 11.88 goes through with Theorem 11.96 in its native form. For (vi), relate a call Check(a, g) of the control with the call that receives the members of the tuple g as separate arguments; inside the call they form the same tuple, which is extended by forming a new tuple, so no caller's context changes, and the same packets are read in the same order at the same floor. □

Raw acceptance is not compared for (i)–(v): allocation first accepts further stacks, eager selection rejects further stacks, and the remainder selector treats surplus arities differently; the accepted canonical goals are those of $\mathcal{U}$, or of the typed contract for (v). An intermediate program with eager selection only, of 496 bytes, occurs in a receipt of `three_field` and is not one of its delivered programs. The program (v) retains the test of the whole input by its printed form and the full native goal grammar, and keeps the two false-valued modes 0 and [] of the field u distinct.

11.9.11 Refinements of the allocation-first programs

This subsection follows `checker_refinement`, the checker part of `single_binder`, `verified_refinements`, the compact part of `refinement`, and `immediate_application`. Their controls are the byte program (iv) and the native program (v) of Theorem 11.97, the list variant of `context_phase` (the paragraph after Theorem 11.87), and, for `immediate_application`, two programs of this subsection. Every change below preserves the returning runs of its control exactly, except in Theorem 11.104, where the accepted canonical goals are preserved.

Proposition 11.98 (Truth-valued result after allocation first)**.** *In the program (iv), let the procedure return the truth value of the membership test of step (1) if it is true, and otherwise the one-element list formed by a comprehension over the one-element list that contains the removed packet, whose element is the result of the final call; and let branch (i) of step (4) be written as the conjunction of the major call and the threefold repetition of m. The resulting checker has the same returning runs, with the same consumption of packets, contexts, goals and floors, as (iv).*

Proof. By induction on the run, every returning call returns a true value: the value of a successful membership test, or a list with one element. The control writes branch (i) as the disjunction of the major call, which returns the null value, and the threefold repetition; the new form is the conjunction of a true value and the threefold repetition. After a returning major call both have the value of the threefold repetition, and if that is false-valued both continue with the same later alternatives; a failed major call fails in both. The comprehension iterates exactly once: it removes one packet, destructures its four fields, and evaluates the final call with its arguments in the order of the control, the extended context first (major call, hypothesis, decrement, opening of the hypothesis) and the selected goal second. The assignments to the goal and to the floor inside the comprehension bind the variables of the enclosing procedure, which is the rule of the host language for assignment expressions in comprehensions, so the opener, which reads the floor of the procedure, sees the decremented floor as in the control. When the membership test succeeds, the comprehension is not evaluated and no packet is read. □

This is Theorem 11.85 on the newer control. `checker_refinement` and `single_binder` deliver the resulting program with identical text, 271/304/488 against 267/299/495. `single_binder` also writes the procedure as an anonymous function whose body first binds the opener by an

assignment expression and then evaluates the same expression; the function object is true-valued, the opener becomes a local of the anonymous function with the same free variables, and the runs are those of the program of 488 bytes (276/308/487). Both packages note that the new results retain data that the control discards, so memory use can differ.

Remark 11.99 (A third delivery of the program of 488 bytes). `refinement` delivers the program of Theorem 11.98 once more, with text identical to that of `checker_refinement` and `single_binder` and against the same control (271/304/488 against 267/299/495), and proves the same correspondence by the same argument: the comprehension is evaluated eagerly, so the final call finishes before the one-element list exists, and the assignments to the goal and the floor bind the enclosing procedure. It adds two observations. First, writing the comprehension as the application of an anonymous function to the unpacked packet is not covered by the proof: the assignments to the goal and the floor would then bind in that function, and would either be read before assignment or no longer reach the cell that the opener reads; the package discards such a variant. Second, it attaches the validator of the typed wrapper of `progress_independence` (the test of the printed form of Theorem 11.62 and the shape and guard test of Theorem 11.64) to both kernels. The two programs differ only in the kernel, which each enters after the same reader, the same test of the printed form, the same goal validator and the same initial call, so by Theorem 11.98 the program on the kernel of 488 bytes accepts exactly the inputs that the program on the kernel of 495 bytes accepts. Both satisfy the typed contract for canonical closed guarded formulas, as the wrapper of `progress_independence` does; the fields of unused packets are typed by J only and keep the permissive sorted reading. They have 423/474/855 and 419/469/862, against 421/471/869 for the wrapper of `progress_independence`; the saving of seven bytes costs four tokens and five nodes, as in the compact pair.

Proposition 11.100 (Immediate application of the opener)**.** *Let L be the expression that forms the opener of a checking procedure and R the expression that removes a packet and performs the remaining steps.*

(a) In the anonymous-function program of `single_binder`*, whose body is* `(i := L) and (i(a) in g or R)`*, replace the body by* `(i := L)(a) in g or R`*.*

(b) In the native program of Theorem 11.102, whose procedure binds `i = L` *and returns* `i(a) in g or R`*, replace the procedure by an anonymous function with the same parameters and defaults whose body is* `(i := L)(a) in g or R`*.*

In both cases the resulting checker accepts exactly the inputs that the original accepts, with the same consumption of packets and the same contexts, goals, floors and, in (b), values of the flag v.

Proof. An assignment expression evaluates its right side, binds its target to the value and returns the value. So `(i := L)(a)` forms the opener, binds i, and applies that opener to a; in both forms the opener is bound before its first application, in particular before any recursive use of i inside L. The opener is a function object, which is true-valued, so in (a) the conjunction of the control always evaluates its second operand and returns its value; the removed test selects no branch and has no effect. The missing image of L is a default value, the checking procedure itself; it is evaluated when L is formed during a call, when the global name of the checker already denotes the installed checker in both programs, and no operation of either program inspects the name, code or defaults of a function, so the change from a named procedure to an anonymous function in (b) is not observable through it. The parameters are local to the procedure in both forms, and i is a cell captured by L in both forms. The assignments to a, k and, in (b), v inside the comprehension of R bind

in the enclosing function also when that function is anonymous, which is the rule of the host language for assignment expressions in comprehensions; so they update the cells that L reads, as before. The expression R, the reader, the test of the printed form, the initial call and the verdict are unchanged.

Relate states of the two programs with equal parsed input and equal remaining stack, and corresponding calls with equal arguments. By the observations above, related calls form openers with the same behavior, evaluate the same membership test, and, if it fails, remove the same packet and make related recursive calls in the same order; an operation fails in one exactly if it fails in the other, and a call that does not terminate in one does not terminate in the other. The only step without a counterpart is the truth test of the opener in the control of (a), which can be skipped. Induction on the tree of calls, in both directions, gives the same verdict and the same remaining stack. □

The programs, of `immediate_application`, have 272/304/479 against 276/308/487 for (a) and 344/392/637 against 343/392/646 for (b). By (a) and the paragraph after Theorem 11.98, the program of 479 bytes has exactly the runs of the program (iv) of Theorem 11.97, so its accepted stacks are those of the program of 495 bytes; by (b) and Theorem 11.102, the program of 637 bytes accepts exactly the inputs of (v) and satisfies the typed contract for the grammar with equality and D. The package checks by a comparison of syntax trees that, after the displayed rewriting is inverted, the parameter lists, the expression L, the expression R with the first application, and every other statement coincide with those of the controls, and it runs 121 fixtures (65 on the compact pair, 56 on the native pair) on candidate and control in ordinary and optimized execution (484 executions). The program of 479 bytes is below its control in all three coordinates; the program of 637 bytes has one token more than its control and the same number of nodes.

Proposition 11.101 (Inverted entry test)**.** *Replace, in a checking procedure whose body ends with the final call, the step "if a° is a member of g, return" followed by the remaining steps by "if a° is not a member of g, perform the remaining steps". The returning runs are unchanged.*

Proof. The membership test is evaluated once in both forms and fails in both forms on the same arguments. If it holds, both return the null value without further effect. Otherwise both perform the same steps in the same order, and then both return the null value, since nothing follows the final call. □

`verified_refinements` applies this to (iv) and obtains 267/298/492 against 267/299/495; its verifier compares the syntax trees and confirms that this is the only change. The rewriting acts on the same lines as Theorem 11.98, which has no return statement left to invert; no package combines the two.

Proposition 11.102 (Native program with the flag as an order)**.** *In the program (v), write the guards of the opener as in Theorem 11.86, apply the change of Theorem 11.98, and reset the flag v to 0 as an argument of the major call instead of immediately after the membership test. The resulting checker accepts exactly the inputs that (v) accepts, with the same consumption of packets and the same floors.*

Proof. The guards of (v) are those of Theorem 11.65, so Theorem 11.86 applies: the flag and the parity are bits, the guarded conditions have the values 0, `False` or `True`, and they are defined wherever the control reaches them, because the test of the printed form admits only integers and arrays and the leaf condition is reached only after a successful range test. Between the membership test and the major call the program removes and destructures a packet, which does not call the opener, and the major call is the first subexpression evaluated in the arguments of the final call; so every opening after the first is evaluated with $v = 0$ in

both programs, and the major call receives $v = 0$ explicitly where the control's receives it by default. The rest is Theorem 11.98. □

The program, of `checker_refinement`, has 343/392/646 against 342/391/666 and keeps the test of the whole input by its printed form, so it satisfies the typed contract for the grammar with equality and D. The package attributes the truth-valued result and the flag as an order to `inferred_major`; the new element is their composition with the allocation-first schedule.

Proposition 11.103 (Phase by an empty context)**.** *In the program (v), remove the flag v, start with the empty context instead of* $[0]$*, read "$v = 1$" as "the context is empty", and replace the reset of the flag by an extension of the context by the scalar* 0 *in place. The resulting checker accepts exactly the inputs that (v) accepts, with the same consumption of packets and the same floors.*

Proof. The test of the printed form is unchanged. In the first call the guards of the two openers have the same truth values, so the first opening is defined in one program exactly if it is in the other, with the same result. A successful first opening at depth 0 and floor 0 is an array of arity 1 or 2: a scalar at the root is not in $[0, 0)$, and the missing image rejects. It is a member neither of $[0]$ nor of the empty list, so both programs continue. The control then sets $v = 0$, and the new program appends 0 to its list; the opener reads the updated variable in both. Every later call receives a nonempty context.

Relate two calls if their goals, floors and stacks agree and their contexts agree up to additional copies of 0, each containing at least one. Membership does not depend on multiplicity, and the contexts are inspected only by membership and by truth value. A major call receives the list of its caller and may append copies of 0 to it; a hypothesis is added by forming a new list, so nothing else changes in the context of a caller. Related calls therefore remove the same packets, make related major calls, form the same hypotheses, and decrease the floor at the same point, and their final calls are related. Induction on the run in both directions gives the claim. This is the argument of Theorem 11.87 for the list variant of `context_phase`. □

The program, of `verified_refinements`, has 334/387/654 against 342/391/666, under the same typed contract. Its context is passed explicitly at the top-level call, so the qualification of the list variant of `context_phase`, whose default context is no longer empty after one invocation, does not arise. The audit in the package also measures the same program with the inverted entry test of Theorem 11.101 (334/386/655), which is not delivered.

Proposition 11.104 (A composed native goal-validated checker)**.** *Starting from the list variant of* `context_phase`*, let the checker allocate first (Theorem 11.95), select the new goal as R if the condition C of Theorem 11.83 holds and as a otherwise, with C evaluated as a comparison of pairs and R evaluated only when C holds, recognize arrays by $a \cdot 0 \neq 0$, use the inverted entry test, and pass the empty initial context explicitly. The resulting checker satisfies the soundness statement of Theorem 11.48 for the ordinary reading and $\mathcal{A}^{-}$, validates the native goal as the control does, and accepts a certificate of every closed theorem of $\mathcal{A}^{-}$ in the grammar with equality and D.*

Proof. On integers and arrays $a \cdot 0 \neq 0$ and $a \cdot 0 = [\,]$ have the same value. The first opening, with the empty context, is that of Theorem 11.87, and all later openings are unflagged. The control selects R if C holds and R is true-valued, and a otherwise. If C holds and R is false-valued, then $R = a$ by the argument of Theorem 11.83(a), which concerns only the root of the template and holds at every floor; so the two selections agree wherever R is evaluated. The new checker evaluates $O(t; u, 0)$ also when $m \neq M^{\circ}$, which can only add a rejection. Soundness follows from Theorem 11.96(a) in its native form, as for Theorem 11.97(v), and

from Theorem 11.87. Completeness follows from Theorem 11.96(b) with inert fields 0, on which $O(t; u, 0)$ is defined, and from Theorem 11.101. □

The program, of `verified_refinements`, has 324/374/629 against 325/375/641. Its correctness statement uses integer and array certificates, and unused prefixes may contain other JSON values; it is a goal-validated program and does not satisfy the typed contract. It tests exact integers by the identity of a leaf with its conversion to an integer, with the qualification about one runtime stated after Theorem 11.87. Raw acceptance is not compared with the control, since allocation first accepts further stacks and the evaluation of $O(t; u, 0)$ rejects further stacks.

`single_binder` supplies, besides, a program that tests every node of the input to be an exact integer or array and the goal to be a closed guarded formula, by recursive predicates with explicit error messages, and then runs the kernel of 488 bytes (556/633/1544, without a stated control), and `502_scope_audit` a readable program of the same contract for the kernel of 502 bytes (605/700/2629). Their correctness is that of the kernel together with the definition of J and Theorem 11.64.

11.9.12 The floor recomputed from the context

This subsection follows `scope_and_frontier`. Theorem 11.56 records that with local floors and the initial context [0] the floor of a call is determined by the length of its context. The package proves this for the inverted-entry program of Theorem 11.101, derives an exact transformation, and shows that the invariant holds at the entry of a call and not inside it.

Lemma 11.105 (Floor at entry)**.** *Let $C(g, a, k)$ be the inverted-entry program of Theorem 11.101. Every call of C reached from the initial call $C([0], a_0, 0)$ satisfies $k = 1 - |g|$, where $|g|$ counts the members of g with repetitions.*

Proof. The initial call has $|[0]| = 1$ and $k = 0$. A call makes recursive calls at exactly two places. The major call $C(g, m, k)$ receives the context and the floor of its caller unchanged; it is evaluated before the decrement of the caller's floor. The final call receives the new list $g + [H]$ and the floor $k - 1$, since $\sim(-k) = k - 1$ on integers; and $k - 1 = (1 - |g|) - 1 = 1 - |g + [H]|$. Induction on the depth of the call gives the claim. The value of H plays no role; it may repeat a member of g or be a neutral resource. □

Proposition 11.106 (Floor recomputed at entry)**.** *Let $\widehat{C}(g, a)$ be the program that differs from C only in that every call begins with the assignment $k := 1 - |g|$ and that the floor is not passed to the two recursive calls; the decrement before the final call is kept in place. On every finite input $\widehat{C}$ accepts exactly when C accepts. Corresponding calls have the same contexts, goals and remaining stacks and, after the assignment, the same floors, and they remove the same packets in the same order.*

Proof. Relate a call of C with arguments (g, a, k) and a call of $\widehat{C}$ with arguments (g, a) if their remaining stacks are equal and $k = 1 - |g|$. The initial calls are related, and by Theorem 11.105 this is no restriction on the reachable calls of C. The two bodies are equal except for the assignment at entry and the floor argument of the two recursive calls. The opener reads the floor of its call when it is applied, so in related calls it behaves identically at every application: before the decrement both floors are $1 - |g|$, and the decrement is the same assignment in both. Hence the membership test, the removal of a packet, the choices of the Boolean operators, the construction of the hypothesis and of the new goal, and their openings agree. The major calls are related, with the context unchanged, and the final calls are related, with the context extended by one member and the floor decreased by one. The context is extended by forming a new list and never in place, and each call has its own floor, so the only shared mutable state

is the stack, from which related calls remove the same packets. This is a simulation step by step of every finite prefix of an evaluation, including evaluations that fail or do not terminate, so the verdicts agree on every input and not only on certificates of derivations. □

The proposition does not identify $C(g, a, k)$ with $\widehat{C}(g, a)$ for an arbitrary k supplied from outside, and it does not allow the context to be replaced by a set or equal members to be merged: the length of the list counts the extensions, which is what the floor records. The program, of `scope_and_frontier`, has 267/303/500 against 267/298/492; the assignment costs five nodes and eight bytes, as the corresponding variant of Theorem 11.56 cost eight bytes. Its accepted inputs are those of the program of 492 bytes, hence those of the program (iv) of Theorem 11.97. The package compares the syntax trees of the two programs and runs 42 inherited inputs on them and on a readable version, in ordinary and optimized execution (252 executions).

A call whose context has n members uses the floor $1 - n$ for its entry scan, its major call and the opening of an introduction, and the floor $-n$, after the decrement, for the openings of the hypothesis and of the new goal. One bound written in terms of the length of the context cannot serve both phases: the strict bound $-|g| < a$ never admits the new parameter where it must be admitted, and the inclusive bound admits it too early.

Proposition 11.107 (Admitting the new parameter early)**.** *Let $C^{\leq}$ be the program C of Theorem 11.105 without the floor argument, with the scope test $k \leq a < d$ of the opener replaced by $-|g| \leq a < d$, and with the decremented floor in the opening of the hypothesis and in the image of the new goal replaced by $-|g|$, for the context g of the call. Let*

$$F' := D_{x,y}(\mathrm{sel}(y, 0, x) = \mathrm{sel}(y, 0, y)),$$

a closed guarded formula with the canonical encoding `[[[5,0,7],[5,0,5]]]`, and put

$$R := \texttt{[[-1],0,[-1]]}, \quad P := \texttt{[0,0,R,0]}, \quad T := \texttt{[[[0,R,0],R],[0,R,0],R,[3,R]]}.$$

Then iPRA *refutes F'; the program $C^{\leq}$ accepts the stack $[P, P, T, \texttt{[0,0,0,[-1]]}, F']$ of four packets; and C and $\widehat{C}$ reject it.*

Proof. The ordinary reading of F' is $\exists x\,(\mathrm{sel}(0, 0, x) = \mathrm{sel}(0, 0, 0) \to \forall y\ \mathrm{sel}(y, 0, x) = \mathrm{sel}(y, 0, y))$. The antecedent reads $0 = 0$; for $y = S0$ and $y = SS0$ the consequent gives $x = S0$ and $x = SS0$, hence $S0 = SS0$, and by the injectivity of the successor $0 = S0$. So iPRA proves $\neg F'$.

Write z for the parameter named -1. At a term position R reads as $\mathrm{sel}(Sz, 0, Sz)$. The packets are consumed from the end.

1. The goal F' is closed, and it is not a member of $[0]$. The packet $[0, 0, 0, [-1]]$ has the neutral major 0, which is proved by membership in $[0]$, and the field u is false-valued, so the packet is an introduction at the witness $[-1]$. In C and $\widehat{C}$ the opening of the matrix at this witness is evaluated at the floor 0: the image is validated at depth 3, where the scalar -1 is out of range, the missing image is consulted, and the run rejects. In $C^{\leq}$ the bound is $-|[0]| = -1$, the scalar is retained, and the witness is Sz, where z is the name that this introduction allocates as its eigenparameter. The matrix is an equation, so the hypothesis is $\mathrm{sel}(z, 0, Sz) = \mathrm{sel}(z, 0, z)$, and the new goal is the reflexive equation $\mathrm{sel}(Sz, 0, Sz) = \mathrm{sel}(Sz, 0, Sz)$, encoded $[R, R]$.

2. The packet T has the major $I(0, \rho;\, u.\, 0) = \rho$, where ρ is the reading of R. The major call consumes the computation packet P, which adds this instance of ı0 to the context of the major call, whose goal is then a member and returns. In the caller, the major equals $[s, u]$ for $s = [0, R, 0]$ and $u = R$, and the template $[3, R]$ opened at u is $[R, R]$,

the goal; so the elimination test succeeds, and the new goal is the template opened at s, the equation $I(0,\rho;\,u.\,0)=\rho$. The exported resource is the sixfold array, which reads $0=0$.

3. The second packet P adds the same instance of I0 to the context of the final call, and its goal is a member.

So $C^{\leq}$ accepts. The derivation is transport of the reflexive equation from the computation equation; what makes the conclusion false is the first step, in which the witness names the eigenparameter of its own introduction. □

The stack is the fixture of future witness capture that earlier packages already use; its application to this shortcut is the contribution of `scope_and_frontier`, which records the outcomes in ordinary and optimized execution and gives no size for the refuted program. The stack of Theorem 11.176 is rejected by $C^{\leq}$ and is not evidence against it.

The review of `468_shared_leaf` adapts the stack to T_+ of Theorem 11.153, with the scope test $k \leq a < d$ of its opener replaced by $k-1 \leq a < d$; call the result $T_+^{\leq}$. In the 1-shifted encoding F' is `[[[7,0,9],[7,0,7]]]`, and with R as above and $s_R :=$ `[0,`R`,0]` the program $T_+^{\leq}$ accepts the stack

$$[\,[0,0,R,0],\ [0,0,R,0],\ [[s_R,R],s_R,R,[5,R]],\ [0,0,0,0],\ [0,0,0,[-1]],\ [F',0]\,]$$

of five packets (`boundary_review/false_early_floor_goal.json`), and T_+ rejects it. The root call removes $[0,0,0,[-1]]$; its neutral major 0 closes by membership with the packet $[0,0,0,0]$. The matrix is opened at the witness $[-1]$ at the floor 0, and the image is validated at the depth 5: in T_+ the scalar -1 is out of range there, the missing image is consulted and the run rejects before the new name is allocated; in $T_+^{\leq}$ the bound $-1 \leq -1$ admits it, so the witness is Sz for the name $z=-1$ that the introduction allocates next. As in the proof above, the hypothesis is $\mathrm{sel}(z,0,Sz)=\mathrm{sel}(z,0,z)$ and the new goal is $[R,R]$. The packet $[[s_R,R],s_R,R,[5,R]]$ has the major $[s_R,R]$, the instance $I(0,\rho;\,u.\,0)=\rho$ of I0, which its major call closes as a computation leaf with one packet $[0,0,R,0]$, whose candidate is $[[0,R,0],R]$; the template $[5,R]$ opened at R is $[R,R]$, so the elimination test succeeds, and the new goal $[s_R,R]$ is closed in the same way by the other packet $[0,0,R,0]$. The certificate was constructed for the review; the package gives no size for the weakened program and does not deliver it.

11.9.13 Refinements of the native goal-validated and compact programs

This subsection follows the checker parts of `checked_frontier`, `reductions`, the second delivery of `checked_presentation`, `479_predecessor57`, `atomic_seed_refinement`, `zero_test_split`, `equation_aliases`, `alias_back`, `absorbing_orbit`, `boolean_constancy`, `checked_reductions`, `zero_alias`, `tolerant_certificates` and the second delivery of `refinement`, and records the deliveries of `immediate_opening` and `absorbing_predecessor`. The controls are programs of Section 11.9.11: the composed native goal-validated program of Theorem 11.104 (629 bytes), the native immediate application of Theorem 11.100(b) (637 bytes), the native program of `inferred_major` with the flag as an order (Theorem 11.86, 643 bytes), and the immediate application of Theorem 11.100(a) (479 bytes). Every statement below is either an equality of runs with the control or an equality of the accepted canonical goals, and it is said which.

Remark 11.108 (Repeated deliveries). The program of 479 bytes of Theorem 11.100(a) is delivered with identical text by `immediate_opening` (`code/checker479.py`), by the second delivery of `checked_presentation` (`code/checker_bytes.py`), by `479_predecessor57`

(`checker/code/compact479.py`) and by `alias_back` (`code/checker_479.py`), each against the same control of 487 bytes and with the argument of Theorem 11.100(a). The program of 637 bytes of Theorem 11.100(b) is delivered with identical text by the second delivery of `checked_presentation` (`code/checker_strict.py`) and by `479_predecessor57` (`checker/code/strict637.py`), with the argument of Theorem 11.100(b). `immediate_opening` proves Theorem 11.105 once more, for the program of 479 bytes, with the observation that the relation fails between the decrement and the end of a call, which is the observation after Theorem 11.106. It also supplies a readable expansion of the program of 479 bytes and a program that tests J on the whole input and the goal by the recursive predicates of `single_binder` and then runs the kernel of 479 bytes; the correctness of the latter is that of the kernel together with the definition of J and Theorem 11.64, as for the corresponding program of `single_binder` on the kernel of 488 bytes. Both are measured with their docstrings. `absorbing_predecessor` contains no new checker: its five baseline programs (`baseline/code/`) are, with identical text, the programs of 479 and 637 bytes, the program of 607 bytes of `479_predecessor57` and the two stream programs of the second delivery of `checked_presentation`.

The truth-valued result on the program of 629 **bytes**

The control of 629 bytes is a procedure $c(g, a, k)$ that binds the opener $i := L$, and, if $i(a)$ is not a member of g, extends g in place by the scalar 0, removes a packet $[m, s, u, t]$, and makes the final call, whose first argument begins with the major call $c(g, m, k)$ in the expression $c(g, m, k) \vee 3m$ (written `c(g, m, k) or m * 3`); it returns the null value. The initial call is $c([\,], a_0)$, so the flagged first opening of Theorem 11.87 is the opening with the empty context.

Proposition 11.109 (Truth-valued result with the context extended inside an expression)**.** *Consider the following programs.*

(a) The procedure returns $i(a) \in g$ **`or`** g.append(0) **`or`** *a comprehension over the one-element list* $[q()]$ *that unpacks* m, s, u, t *and whose element is the final call of the control, with* $c(g, m, k) \vee 3m$ *replaced by* $c(g, m, k) \wedge 3m$ *(`checked_frontier`,* 330/379/623*).*

(b) The program (a) written as an anonymous function whose body is $(i := L)$ **`and`** *the returned expression of (a) (`checked_frontier`,* 335/383/622*).*

(c) The program (a) without the call of append*, in which the first argument of the final call begins with the assignment expression* $g := [*g, 0]$*, a new list (`reductions`,* 332/381/621*).*

(d) The program (c) written as in (b) (`reductions`, 337/385/620*).*

(e) The program (c) with $g := g + [0]$ *in place of* $g := [*g, 0]$*, written as an anonymous function whose body is* $(i := L)(a) \in g$ **`or`** *the comprehension, as in Theorem 11.100 (the second delivery of `checked_presentation`,* 334/381/614*).*

Each accepts exactly the inputs that the control accepts, with the same consumption of packets, the same goals and the same floors. In (a) and (b) the contexts are the same lists as in the control; in (c), (d) and (e) they agree with those of the control up to additional copies of the scalar 0*.*

Proof. Results. By induction on the run, every returning call of a variant returns a true value: `True` from a successful membership test, or a list with one element. A returning call of the control returns the null value. After a returning major call, $c(g, m, k) \vee 3m$ in the control and $c(g, m, k) \wedge 3m$ in a variant both have the value $3m$; a failing major call fails in both. This is the argument of Theorem 11.98.

Outer forms. The opener L is a function object, which is true-valued, so $(i := L)$ `and` E evaluates E with i bound and returns its value; $(i := L)(a)$ binds i before its first application. The missing image of L is the global checking function, evaluated when L is formed during a call, when that name already denotes the installed checker; no operation inspects the name, code or defaults of a function, and no JSON value is a function, so a comparison with the missing image fails in both programs. This is the argument of Theorem 11.100(b).

Contexts in (a) and (b). The method append extends the list object in place, as the augmented assignment of the control does, and returns the null value, so the disjunction continues to the comprehension. The order is that of the control: membership test, extension, removal of the packet.

Contexts in (c), (d) and (e). The comprehension first evaluates its iterable, removing the packet, then unpacks it, and then evaluates the final call, whose first subexpression is the assignment to g. By the rule of the host language for assignment expressions in comprehensions this binds g in the enclosing function, so the opener, which reads g of its call, sees the extended list in every later opening of the call. Between the membership test and the assignment only the removal and the unpacking of a packet occur, which neither call the opener nor read g. The major call receives the extended list, as in the control. The difference is that a major call of the control extends the list of its caller in place, while a major call of a variant extends a copy; so after the major call the context of the caller in the control can contain further copies of 0. Relate calls whose goals, floors and remaining stacks are equal and whose contexts are either both empty or both contain 0 and agree after the deletion of all members equal to the scalar 0. A context is inspected only by membership and by its truth value, in the test of the opener that selects the grammar of the first opening; both are invariant under this relation, and a hypothesis is added by forming a new list $g + [h]$, which preserves it. The initial calls have the empty context in both programs, so the first opening is the same flagged opening, before any extension.

Relating the remaining operations as in Theorem 11.103, related calls evaluate the same membership test, remove the same packet, make related major calls, form the same hypothesis and the same new goal, decrease the floor at the same point, which is the same assignment expression in all programs, and make related final calls. A failing operation fails in both, and a nonterminating call does not terminate in both. Induction on the tree of calls in both directions gives the claim. □

All five programs keep the test of exact integers by the identity of a leaf with its conversion to an integer, with the qualification about one runtime stated after Theorem 11.87, and they satisfy the goal contract of their control. The second delivery of `checked_presentation` states that a variant of (e) that keeps the flag of the strict programs instead of the phase by context has 640 bytes; it is not delivered. `checked_frontier` also writes the native program of `checker_refinement` (343/392/646) as an anonymous function whose body is $(i := L)$ `and` the returned expression of the procedure; by the argument for the outer forms above it has the runs of its control, and it has 348/396/645. The program of 637 bytes of Theorem 11.100(b) is the same rewriting with the immediate application and is smaller in all three coordinates.

Exact-integer leaves without the test of the printed form

Proposition 11.110 (Native program from the program of 637 bytes)**.** *Let N be the program of* 637 *bytes of Theorem 11.100(b) with the test of the printed form of the whole input deleted and with the leaf condition of the opener preceded by the conjunct "the leaf is identical with its conversion to an integer". Then:*

(a) on every input that satisfies J, N has exactly the runs of the program of 637 *bytes;*

(b) N accepts no input whose goal is not a canonical closed formula of the language with equality and D; on such an input it rejects, or, for some goals that contain a nonempty string, it does not terminate;

(c) on inputs whose consumed packets satisfy J, N satisfies the soundness statement of Theorem 11.48 for the ordinary reading and $\mathcal{A}^-$, and it accepts a certificate of every closed theorem of $\mathcal{A}^-$ in that language.

Proof. (a) If the input satisfies J, its printed form consists of digits, minus signs, brackets, commas and spaces, so the deleted test succeeds in the program of 637 bytes, and the stack is the same. A leaf that reaches the leaf condition is an exact integer by Theorem 11.60, and the conversion of an exact integer to an integer returns the same object in the measured runtime, so the new conjunct is true and the condition has the value it has in the program of 637 bytes. Every other expression is unchanged.

(b) In the first opening the flag is 1 and the floor 0. The array test is $a \cdot 0 \neq 0$, as in the program of 629 bytes. A leaf that is a float or a Boolean fails the range test or the new conjunct; `null` and objects make the array test fail; a string passes the array test, and if the arity condition rejects its length the range test then fails, while a string of an admitted length is traversed as an array of one-character strings, without end. In every failing case the opener reaches the missing image, which is the checking function, or a failed operation, and the run rejects. These are the tests, and the qualification, of the program of 629 bytes in its flagged phase. So a goal that the first opening validates satisfies J, and by Theorems 11.59 and 11.65 it satisfies P^{ed} at the depth 0.

(c) In every later opening the flag is 0, and the guarded conditions are true whatever the value of the conjunct, which is defined wherever it is reached: a value that passes the range test is a finite number. So, after the first opening, N performs the operations of the kernel of the program of 637 bytes, and the soundness and completeness of that kernel (Theorem 11.100(b), Theorem 11.102 and Theorem 11.97(v)) apply on consumed packets that satisfy J. Unused packets are not inspected. Canonical certificates satisfy J. □

The program, of `479_predecessor57`, has 337/383/607 against 324/374/629. Its kernel is that of the program (v) of Theorem 11.97, with eager selection, while the program of 629 bytes selects its goal as in Theorem 11.104; the two programs accept the same canonical goals, and raw acceptance is not compared. The exact-integer test carries the qualification about one runtime of the program of 629 bytes. The package also records that the selector of the native mode cannot be replaced by the remainder selector (Theorem 11.84): with $u = [\,]$ the native protocol requests the introduction by cases at an equation, which the remainder selector does not provide, and the formula $D_{x,y}(y = S0)$, whose ordinary reading follows from separation, would no longer be accepted.

One parity comparison

Proposition 11.111 (One parity comparison)**.** *In the native program of* 643 *bytes of* `inferred_major` *and in the program of* 637 *bytes of Theorem 11.100(b), replace the leaf condition "$a = 0$, or $a > 1$ and a is odd" (written* `not a or a > 1 == a % 2`*) by $a \bmod 2 = [a > 1]$ (written* `a % 2 == (a > 1)`*). Each resulting program has exactly the runs of its control.*

Proof. For an integer $a \geq 0$ the two conditions have the same truth value: at $a = 0$ both hold, at $a = 1$ both fail, and for $a \geq 2$ both say that a is odd. They differ exactly on the even negative integers (Theorem 11.82). The leaf condition is the guarded condition p of a guard $v \leq p$ (Theorem 11.86). With the flag $v = 1$, in the first opening, the floor is 0, so the leaf has passed the range test $0 \leq a$, and it is an exact integer: by the explicit type test that precedes the condition in the program of 643 bytes, and by the test of the printed form and

Theorem 11.60 in the program of 637 bytes. So the two conditions agree there. With $v = 0$ the guard holds whatever the value of p, which is 0, `False` or `True` for both conditions, and both conditions are defined on integers; in the program of 643 bytes a leaf that is not an exact integer stops the conjunction at the type test before either condition is evaluated, and in the program of 637 bytes every leaf is an exact integer. So the opener returns the same value or fails at the same point in both programs, every other expression is unchanged, and induction on the run gives the claim. □

The programs, of `atomic_seed_refinement`, have 326/370/636 against 327/375/643, under the goal contract, and 341/387/628 against 344/392/637, under the typed contract for the language with equality and D. The first tests exact integers by an explicit type test and carries no qualification about the runtime. The same identity of conditions was used by `retained_premises` in its refined native validator (Theorem 11.82). The package states two further compositions with a one-line lexical front end, of sizes 324/369/639 and 334/377/607, which have a different raw input grammar and are not delivered.

Products of bits in the strict programs

`equation_aliases` gives three programs of the typed contract for the language with equality and D; they implement the native calculus, not the calculus with restricted aliases of that package, which belongs to another chapter. Write P for the leaf condition `not a or a > 1 == a % 2` and e for the parity d mod 2 of the depth. Let C be the program (d) of Theorem 11.109, of 620 bytes, preceded by the test of the printed form of the whole input (Theorem 11.62); the package constructs it as the control of two of its programs (351/403/666).

Proposition 11.112 (Products of bits)**.** *(a) C satisfies the typed contract for the language with equality and D, and on inputs that satisfy J it accepts exactly the inputs that the program of* 629 *bytes accepts.*

(b) Let T be C with the conjunct "the leaf is identical with its conversion to an integer" deleted, the condition $e \wedge P$ written as the product $e \cdot P$, the condition $e \vee \neg g$ of the fallback written as the bitwise disjunction $e \mid \neg g$, and the opener applied immediately as in Theorem 11.100. Then T accepts exactly the inputs that C accepts, with the same consumption of packets and the same floors; goals and contexts agree up to the replacement of `False` by the integer 0 *at formula positions.*

(c) Let A be T with the parsing of the input, the test of the printed form and the binding of the removal function written as assignment expressions inside the initial call. Then A has the runs of T.

(d) Let B be the program of 637 *bytes of Theorem 11.100(b) with the guard $v \leq (e \wedge P)$ written as $v \leq e \cdot P$. Then B has the runs of the program of* 637 *bytes.*

Proof. (a) By Theorem 11.62 the kernel is reached exactly for nonempty arrays that satisfy J, and on these the kernel of C is the program of 620 bytes, which has the accepted inputs of the program of 629 bytes by Theorem 11.109. That program validates the goal (Theorem 11.104), and canonical certificates satisfy J.

(b) The deleted conjunct stands inside the alternative "g or …", which is evaluated only while the context is empty, that is, in the first opening of the goal; there every leaf is an exact integer of the input, for which the conversion returns the same object, so the conjunct is true and its deletion changes no value. (This also removes the qualification about one runtime.) The parity e is 0 or 1, and P, which is evaluated only for a number that has passed the range test, is a Boolean and has no effect; $e \wedge P$ and $e \cdot P$ are both 0 if $e = 0$ and have the truth

value of P if $e = 1$, and they are used only for their truth value. Evaluating P also when $e = 0$ is harmless. The condition of the fallback has the same truth value in both forms; it selects the same branch, and where it is false the fallback returns its value, which is `False` in C and the integer 0 in T. This happens only at an even depth, at a formula position, after the first opening. The two values are equal, have the same truth value, behave alike under multiplication, comparison and membership, make the array test fail in the same way, and no exact-type test remains in T that could separate them; the test of the printed form is applied to the input only. Relate states of C and T in which the remaining stacks, floors and contexts agree and the trees agree up to this replacement, as in the proof of Theorem 11.87; related calls take the same branches and remove the same packets. The immediate application is Theorem 11.100.

(c) The initial call looks up the global checking function, which has no effect, and then evaluates its arguments in order: the empty context, then the parsing of the input, the test of the printed form, the binding of the removal function to a global name, and the removal of the goal. These are the operations of the three statements of T, in the same order, with the same global bindings; the checking function is unchanged.

(d) As in (b), $e \wedge P$ and $e \cdot P$ have the same truth value, and with $v \in \{0, 1\}$ the guards $v \le e \wedge P$ and $v \le e \cdot P$ have the same value. Every leaf that reaches P is an exact integer by the test of the printed form and Theorem 11.60, so P is defined; nothing else changes. □

The programs have 338/390/637 (T, `strict/checkers/tokens338.py`), 342/386/641 (A, `strict/checkers/ast386.py`) and 342/392/633 (B, `strict/checkers/bytes633.py`), against 351/403/666 for C and 344/392/637 for the program of 637 bytes. The package states the envelope 338/386/633 against 342/391/637, which it takes from the native program of `three_field` and the program of 637 bytes; its census does not include the phase by an empty context (334/387/654) or the parity program of 628 bytes. No package combines the product of bits with the parity comparison of Theorem 11.111.

The test of the printed form as a disjunction, and an eager mode selector

`absorbing_orbit` and `boolean_constancy` make two further changes to programs of this section; their arithmetic parts belong to another chapter. Write r for the printed form of the parsed input with the admitted characters (digits, minus sign, brackets, comma, space) stripped from both ends; the test of the printed form of the program of 637 bytes binds the removal function as $q := (\neg r) \wedge p.\mathrm{pop}$.

Proposition 11.113 (Test of the printed form as a disjunction)**.** *Replace in the program of* 637 *bytes the binding by* $q := r \vee p.\mathrm{pop}$. *The resulting program accepts exactly the parsed inputs that the program of* 637 *bytes accepts, with the same run.*

Proof. If r is empty, both bindings evaluate $p.\mathrm{pop}$ and bind the same method, or both fail when p has no such method; everything after the binding is identical. If r is not empty, neither evaluates $p.\mathrm{pop}$; the old binding gives `False` and the new one the nonempty string r, and the next operation, the call $q()$ that removes the goal, fails on both, since neither value can be called. Malformed text is rejected by the parser before either binding. □

The program, of `absorbing_orbit` (`checker/checkers/strict632.py`), has 343/390/632 against 344/392/637, under the typed contract for the language with equality and D. The package states the envelope 334/387/632, at a revision without the programs of 628 and 641 bytes.

Proposition 11.114 (Eager mode selector)**.** *In the program of* 479 *bytes replace the selector* $\mathrm{bool}(m \vee e)$ *of the floor of the new hypothesis, with* $e = 3 \bmod |a|$, *by* $\mathrm{any}((m, e))$, *which*

evaluates e also when m is true-valued; in the program of 637 *bytes make the same change with e the test $|a| \bmod 2 = u$. In each case:*

(a) every input accepted by the new program is accepted by the old one, with the same run;

(b) every certificate of Theorem 11.48, with inert fields 0*, that the old program accepts is accepted by the new one, with the same run;*

(c) for the program of 479 *bytes, some inputs accepted by the old program are rejected by the new one.*

Hence the two programs accept the same canonical goals.

Proof. Where e is defined, $\mathrm{bool}(m \vee e)$ and $\mathrm{any}((m, e))$ are the same truth value, and the evaluation of e (a length, a remainder, an equality) changes no state. The selector is evaluated after the hypothesis and the new goal of an introduction are formed and before the decrement of the floor, in both programs.

(a) In a returning run of the new program every evaluation of e is defined. Replacing each of them by the old selector, including in major calls, gives a returning run of the old program with the same consumed packets.

(b) The two programs differ only where m is true-valued and e is undefined, that is, where the goal a at the selector has no length or, in the program of 479 bytes, the length 0. In the certificates of Theorem 11.48 every goal that reaches a selector is the canonical encoding of a formula, or, after an introduction, the opened matrix, which is again a canonical formula; these are arrays of arity 1, 2 or 3, and the neutral major 0 is closed by membership before any selector. So e is defined, and the argument of (a) in the other direction applies.

(c) With $E = \texttt{[[0,[0],0],[0]]}$, the encoding of $I(0, S0;\, u.\, 0) = S0$, the stack

$$\texttt{[[0,0,0,}E\texttt{],[0,0,[0],0],[}E\texttt{,0,0,0],[[],0,[0],0],}E\texttt{]}$$

(the boundary certificate of the package) is accepted by the program of 479 bytes and rejected by the new program: the first packet $[[\,], 0, [0], 0]$ has the major $[\,]$, and the call that proves it has the empty array as goal and reaches a selector with the true-valued major E, where the old selector does not evaluate $3 \bmod |[\,]|$ and the new one fails on it. The direct certificate $[[0, 0, [0], 0], E]$ of the same goal is accepted by both. □

The programs, of `boolean_constancy`, have 274/304/478 (`checkers/code/checker478.py`) against 272/304/479, and 346/392/636 (`checkers/code/checker636.py`) against 344/392/637, under the typed contract for the language with equality and D. The program of 478 bytes is not the program of `zero_test_split` of the same number of bytes (271/305/478): it reads the unchanged four-field format with the goal as last element, and its accepted stacks are a proper subset of those of the program of 479 bytes, while the program of `zero_test_split` has, with the header $[A, 0]$, exactly the runs of the program of 479 bytes.

An initial floor supplied with the goal

Lemma 11.115 (A zero in every canonical public goal)**.** *The canonical encoding (Theorem 11.9) of a closed public formula of $\mathcal{U}$ contains the scalar* 0 *at a term position, that is, at an odd depth of the first opening.*

Proof. If the formula is $D_{x,y}B$, descend through compound matrices to a guarded equation (11.1); the initializer 0 of its iterator is encoded by the scalar 0 at a term position. If the formula is a closed equation, descend in one of its terms through a successor or through the counter or the initializer of an iterator; these positions are not below the state binder, so

every term on the path is closed, and the descent ends at an atom that is not a variable, the constant 0. A canonical encoding contains no reference to a dummy slot, so this 0 is encoded by the scalar 0. Term positions have odd depth. □

Proposition 11.116 (Initial floor in the certificate)**.** *Let H be the program of* 479 *bytes in which the floor parameter of the checking function has no default value, and the initial call $c([0], q())$ is replaced by $c([0], *q())$, so that the last element of the input is a pair $[A, k_0]$ of the goal and an initial floor.*

(a) For every stack σ with last element A, let σ' be σ with its last element replaced by $[A, 0]$. Then H accepts σ' exactly when the program of 479 *bytes accepts σ, with the same run.*

(b) Let the input consist of integers and arrays and let A be the canonical encoding of a closed public formula of $\mathcal{U}$. If H accepts an input with last element $[A, k_0]$, then k_0 is an integer, $k_0 \leq 0$, and A is a theorem of $\mathcal{U}$.

Hence H and the program of 479 *bytes accept the same canonical goals.*

Proof. (a) The program of 479 bytes enters $c([0], A, 0)$ with its default floor, and H enters the same call with the same remaining stack. The bodies are identical.

(b) The checking function has exactly three positional parameters, so the last element must be an array with exactly two elements; otherwise the call fails. In particular the header cannot add a member to the initial context [0]. The first operation is the opening of A at the floor k_0; unless it has failed earlier, it reaches the scalar 0 of Theorem 11.115 at an odd depth $d \geq 3$ and compares it with k_0. If k_0 is an array, the comparison fails. If $k_0 > 0$, then $0 \notin [k_0, d)$, the depth is odd, the missing image, the checking function, is consulted, and the array test fails on it. In both cases the run rejects before a packet is read. So k_0 is an integer and $k_0 \leq 0$, a floor in the sense of Section 11.2. The program of 479 bytes has, call by call, the runs of the program (iv) of Theorem 11.97 (Theorem 11.100(a) and Theorem 11.98), and the soundness statement of Theorem 11.48 for that program is proved for an arbitrary floor and an arbitrary context whose members are closed at that floor; the scalar 0 is closed at every floor. So the auxiliary reading of A at the floor k_0 is derivable in T_{eq} from $0 = 0$. The canonical encoding A contains no negative scalar, so it is closed at the floor 0, and by Theorem 11.14 its reading at the floor k_0 is the closed formula that it encodes; the parameters with names in $[k_0, 0)$ carry no assumption and do not occur in it. By Theorem 11.6 the formula is a theorem of $\mathcal{U}$. The converse is (a) with the completeness of Theorem 11.97(iv). □

The program, of `zero_test_split`, has 271/305/478 against 272/304/479: deleting the default saves two bytes, the unpacking costs one byte and one node. Its input format differs from that of its control in the last element, and an untrusted converter wraps an old goal as $[A, 0]$; with a nonpositive offset $-r$ it also shifts every negative name of a canonical certificate by $-r$, and the package proves for canonical certificates that openings and allocation commute with this shift. The argument of (b) uses the scalar 0 of a canonical goal, and the program does not validate its goal; it is a program of the raw contract with a canonical goal assumed.

Remark 11.117 (The header in the native programs)*.* The same change applied to the programs of 607 and 637 bytes (giving 606 and 636 bytes) is unsound, and neither program is delivered. The native language has closed formulas without the scalar 0, such as $D_{x,y}E(y)$ with

$$E(y)\colon \quad I(y, y;\, u.\, Su) = I(y, y;\, u.\, I(y, u;\, v.\, Sv)),$$

which says $2y = y{+}y^2$; it holds at 0 and 1 and fails at 2, so its ordinary reading $E(0) \to \forall y\, E(y)$ is false. At the initial floor 1 the first name allocated is the scalar 0, which reads as the constant 0 and not as a fresh parameter, so the premise of an introduction by progress at this name

becomes $E(0) \vdash E(S0)$, whose conclusion is a true closed equation. `zero_test_split` reports a certificate of 30 packets that both modified programs accept and that the unmodified native program and the program of 478 bytes at the floor 0 reject; the certificate is not displayed in its report (Section 11.12).

Compositions of the preceding rewritings

`checked_reductions` and the second delivery of `refinement` combine the rewritings of Theorems 11.101, 11.111 to 11.114 and 11.116; their arithmetic parts belong to another chapter. The proofs of these propositions use, for the parity comparison and the product of bits, only that the leaf condition is evaluated in the first opening at the floor 0 on exact integers and afterwards either not at all or inside a guard that holds whatever its value; for the disjunctive gate only the line that binds the removal function; for the eager selector only the definedness of e on canonical certificates; and for the header only the first opening of a canonical guarded goal.

Proposition 11.118 (Compositions)**.** *Each of the following programs has exactly the runs of its control, except (i), whose relation to its control is that of Theorem 11.114.*

(a) The program of 628 *bytes (Theorem 11.111) with the gate of Theorem 11.113 (*`checked_reductions`*,* 340/385/623*).*

*(b) The phase by an empty context of Theorem 11.103 (*334/387/654*) with the parity comparison, the gate of Theorem 11.113 and the inverted entry test of Theorem 11.101 (*`checked_reductions`*,* 330/379/641*; without the inverted entry test* 330/380/640*, an intermediate program of the package).*

(c) The program of 607 *bytes (Theorem 11.110) with the parity comparison (*`checked_reductions`*,* 336/378/600*).*

(d) The program of 629 *bytes with the parity comparison (*`checked_reductions` *and* `refinement`*, with identical text,* 323/369/622*).*

(e) The program of 629 *bytes with the parity comparison and the product of bits* $e \cdot (\text{exact-integer test} \wedge P)$ *(*`refinement`*,* 321/371/618*).*

(f) The program of 607 *bytes with the same product (*`refinement`*,* 334/380/596*).*

(g) The phase by an empty context with the parity comparison, the product of bits and the gate of Theorem 11.113 (`refinement`*,* 330/380/638*).*

(h) The program of 628 *bytes, that is, the program of* 637 *bytes with the parity comparison, with the product of bits and the gate of Theorem 11.113 (*`refinement`*,* 340/385/621*).*

(i) The program of 478 *bytes of* `zero_test_split` *(Theorem 11.116) with the eager selector of Theorem 11.114 (*`checked_reductions`*,* 273/305/477*).*

The programs (a), (b), (g) and (h) satisfy the typed contract for the language with equality and D, the programs (c)–(f) the goal contract, and (i) the raw contract with a canonical guarded goal assumed and the header of Theorem 11.116.

Proof. The rewritings act on different expressions, so it suffices to check that each hypothesis used in its proof holds in the composed program.

Parity and products. In the programs with the phase by context ((b), (d), (e), (g)) the leaf condition stands inside "g or ..." and is evaluated only while the context is empty, in

the first opening, at the floor 0; the range test gives $a \geq 0$, and a is an exact integer by the test of the printed form ((b), (g)) or by the exact-integer conjunct that precedes the parity comparison ((d), (e)). In the programs with the flag ((a), (c), (f), (h)) the condition is the guarded condition of $v \leq p$, which holds whatever p is when $v = 0$; when $v = 1$ the floor is 0 and the leaf is an exact integer, by the printed form ((a), (h)) or by the exact-integer conjunct ((c), (f)). In the products of (e) and (f) the factor that is now evaluated also when $e = 0$ is the conjunction of the exact-integer test with the parity comparison; the exact-integer test was already evaluated first in the controls, it is defined on every number that passes the range test, and the parity comparison is evaluated only after it has succeeded, on an exact integer. In (g) and (h) the factor is the parity comparison on an exact integer. So the identity of Theorem 11.111 and the argument of Theorem 11.112 apply, the values of the guards agree, and no new operation fails.

Gate and entry test. Theorem 11.113 concerns only the binding of the removal function before the first call, and Theorem 11.101 only the form of the entry test in a procedure whose body ends with the final call, which is the form of the program of 654 bytes.

Header and eager selector. With the header $[A, 0]$ the program of 478 bytes of `zero_test_split` has the runs of the program of 479 bytes, and Theorem 11.116(b) shows that an accepted header has a floor $k_0 \leq 0$. The proof of Theorem 11.114(a) is a replacement of selectors in a returning run and does not depend on the initial floor; so every input accepted by (i) is accepted by its control with the same run, and the soundness of Theorem 11.116 applies. The certificates of Theorem 11.48, with the last element $[A, 0]$, are accepted by (i) by Theorem 11.114(b). The stack of Theorem 11.114(c) with its last element written $[E, 0]$ is accepted by the control and rejected by (i). □

Zero as a defined alias in the public goal

`zero_alias` changes the encoding of public goals. By Theorem 11.177 the scalar 1 at a term position behaves in the raw protocol as a constant c about which nothing is known, and Theorem 11.178 shows that a validator that merely admits it as a spelling of zero loses completeness. The remark after that corollary names an identification of spellings as a possible repair whose cost must be counted; `zero_alias` implements it by an assumption in the initial context.

Definition 11.119 (Zero-alias encoding and checker)**.** For a formula B of the language with equality and D let $B^\sharp$ be its canonical encoding with every scalar 0 at a term position replaced by the scalar 1; let P_1^{ed} be the predicate P^{ed} of Theorem 11.57 with the leaf alternative "$a = 0$, or a odd with $3 \leq a < e$" replaced by "a odd with $1 \leq a < e$". The *zero-alias checker* is the program (v) of Theorem 11.97 with the guards of Theorem 11.86, the reset of the flag in the major call of Theorem 11.102, the inverted entry test of Theorem 11.101 and the gate of Theorem 11.113, with the leaf condition of the flagged opener replaced by $e \,\&\, a$ (the depth is odd and a is odd), and with the initial context $[0, [1, 0]]$ instead of $[0]$.

Proposition 11.120 (Zero as a defined alias)**.** *(a) The zero-alias checker accepts only inputs that satisfy J and whose goal a satisfies $P_1^{\mathrm{ed}}(a, 0)$; in particular it accepts no goal that contains the scalar 0 at a term position.*

(b) If it accepts an input with goal a, then the sorted reading of a, in which the scalar 1 reads as 0, is a closed theorem of $\mathcal{A}^-$, and the parametric reading of Theorem 11.177, in which 1 reads as c_1, is derivable in $\mathcal{A}^-$ over the language with c_1 from $c_1 = 0$.

(c) For every closed theorem B of $\mathcal{A}^-$ in the language with equality and D, the checker accepts some certificate with goal $B^\sharp$.

Hence the accepted canonical goals are exactly the encodings $B^\sharp$ of the closed theorems B.

Proof. (a) The gate gives J (Theorems 11.62 and 11.113). In the first opening, with the flag 1 and the floor 0, a leaf at an odd depth d is retained exactly when $0 \le a < d$ and a is odd, that is, a odd with $1 \le a < d$; at an even depth the guarded condition is 0, and the fallback consults the missing image and rejects. The arity conditions are those of the control. The argument of Theorem 11.59 gives P_1^{ed}, and the opening returns the goal unchanged.

(b) After the first opening the program is the kernel of the program (v), whose runs are not changed by the devices listed, and its soundness statement (Theorem 11.96(a) in its native form) holds for an arbitrary initial context of trees closed at the floor 0. The member $[1, 0]$ is closed at every floor, since 1 lies in the range of every term position, and its sorted reading is $0 = 0$; so the sorted reading of the goal is derivable from $0 = 0$, hence outright. For the parametric reading, Theorem 11.177 gives derivability from the parametric readings of the context, which are $0 = 0$ and $c_1 = 0$.

(c) Let z be a parameter and $F(v)$ the template obtained from B by writing v for each occurrence of the constant 0 at a term position, so that $B = F(0)$. In $\mathcal{A}$ (Theorem 11.1), TR accepts every source, including 0: from $z = 0$ it derives $0 = z$ (transport with the template $v = z$ and the reflexive $z = z$), and then $F(z)$ from $F(0)$, which is derivable since B is a theorem. So $z = 0 \vdash F(z)$ in $\mathcal{A}$, hence in $\mathcal{A}^-$ (Theorem 11.6). Its eigenparameters are fresh, so they differ from z, and a source z of SE is not the literal 0. Compile this derivation by the completeness proof of the program (v) (Theorem 11.97, with Theorem 11.96(b)), with z encoded by the scalar 1 in place of a negative name. The compilation uses of a parameter only that it is retained by every opening, is never the hole and never equals a fresh name; the scalar 1 has these properties, since every term position has depth at least 3 and fresh names are negative (Theorem 11.177). The source field 1 is true-valued, as the encoding of a parameter source is. The hypothesis $z = 0$ is the member $[1, 0]$ of the initial context. The resulting certificate proves the goal $F(1)$ in the encoding, which is $B^\sharp$; it satisfies P_1^{ed}, so the first opening passes, and the run is that of the kernel. □

The program, of `zero_alias` (`implementation/alias_inverted.py`), has 331/373/625; its readable companion has 430/499/2322. Its contract is not that of T/ed: it accepts the encodings $B^\sharp$ and rejects every canonical goal of T/ed that contains the scalar 0, so it satisfies neither the completeness of T/ed nor the converse. It is listed under its own code. The package states that it is below the envelope 334/386/628 of T/ed in all three coordinates, which is true of the figures and is a comparison across contracts related by the bijection $B \mapsto B^\sharp$ on goals; completeness is claimed for canonical encodings only, as under T/ed. The package's negative control, the same program with the initial context [0], rejects a translated certificate of a computation, as Theorem 11.178 predicts.

Erasing the alphabet precondition

`tolerant_certificates` removes the last assumption on certificates from a goal-validated program. The programs of the goal contract recognize arrays by $a \cdot 0 \neq 0$, which is true for strings as well; their soundness is stated for consumed packets that are integers and arrays, and a nonempty string at an active position makes an ideal execution traverse it without end, as noted in Theorem 11.110(b). The package replaces the test by $a \cdot 0 = [\,]$, which on values of the JSON reader holds exactly for arrays, is false for numbers and strings, and fails for `null` and objects.

Definition 11.121 (Numeric respelling; tolerant contract)**.** A *numeric respelling* of a raw tree replaces leaves by values of the JSON reader with the same numerical value: integral finite floats, and, for the leaf 0, also `False` or the negative floating zero. A checker satisfies the *tolerant goal contract* for a grammar if every input accepted by the JSON reader leads to a terminating ideal run, every accepted goal is a numeric respelling of a canonical closed formula

of the grammar whose reading is a theorem, and every numeric respelling of the canonical encoding of a closed theorem is accepted with some certificate. No property of unused or consumed certificate data is assumed.

Let $\widetilde{N}$ be the program (e) of Theorem 11.118 (321/371/618) with the exact-integer conjunct deleted from its leaf condition and with the array test $a \cdot 0 \neq 0$ replaced by $a \cdot 0 = [\,]$.

Lemma 11.122 (First opening of $\widetilde{N}$)**.** *The first opening of $\widetilde{N}$, with the empty context and the floor 0, is defined exactly when the goal is a numeric respelling of a tree b with $P^{\mathrm{ed}}(b, 0)$, and it then returns the goal unchanged.*

Proof. The array test separates arrays from all other values of the reader, and the arity condition is that of the control. A string reaches the scalar branch and fails the range test; `null` and objects fail the array test. A numeric leaf passes exactly when $0 \leq a < d$, the depth d is odd, and $a \bmod 2 = [a > 1]$. For a finite nonnegative number the last condition holds exactly at 0 and at the odd integers of at least 3: for $0 < a < 2$ the remainder is a, which is neither 0 nor 1 when $a \neq 1$, while $[a > 1]$ is 0 or 1, and at $a = 1$ the two sides are 1 and 0; for $a \geq 2$ the side $[a > 1]$ is 1, and the floating remainder modulo 2 is exact, so it is 1 exactly at the odd integers; `True` has the value 1 and fails, `False` has the value 0 and passes. Infinite values and NaN fail the range test. Every failure consults the missing image, the checking function, and fails. The opening retains every leaf it admits, so it returns the goal. Induction on the goal gives the claim, as in Theorem 11.59. □

Lemma 11.123 (Later openings of $\widetilde{N}$)**.** *After the first opening, every opening of $\widetilde{N}$ terminates; it fails when it reaches a string, `null` or an object; a nonfinite number behaves as a scalar outside every range, that is, as the hole at a term position and as $0 = 0$ at a formula position; and a returning opening has the properties of Theorems 11.11, 11.12 and 11.14 for the raw trees of Theorem 11.7, whose numeric scalars include floats and Booleans.*

Proof. With a nonempty context the arity condition is inactive, and the array test recognizes exactly arrays. A string reaches the range test and fails; it is no longer traversed. `null` and objects fail the array test. A nonfinite number fails the range test and is replaced by the image at an odd depth or by the scalar 0 at an even depth, which is the treatment of a hole. Traversal is structural on a finite tree, and the image is opened once with the missing image, after which a further hole fails; so every opening terminates. A returning opening has met only arrays and finite or nonfinite numbers at the positions it traversed, and the proofs of the three lemmas apply verbatim; for freshness, finitely many names lie in $[k-1, k)$, the case treated after Theorem 11.14. □

Theorem 11.124 (Erasure of the alphabet precondition)**.** *$\widetilde{N}$ satisfies the tolerant goal contract for the language with equality and D and the calculus $\mathcal{A}^{-}$:*

(a) on every input accepted by the JSON reader its ideal run terminates;

(b) if it accepts, the goal is a numeric respelling of a tree b with $P^{\mathrm{ed}}(b, 0)$ whose reading is a closed theorem of $\mathcal{A}^{-}$;

(c) for every closed theorem B with canonical encoding b and every numeric respelling b' of b, some certificate with goal b' is accepted.

Proof. (a) The input must be an array, or the removal of the goal fails. Every call that does not return at its membership test removes a packet from the finite stack before any recursive call, and every opening terminates (Theorems 11.122 and 11.123). So the tree of calls is finite.

(b) By Theorem 11.122 the goal has the stated form. For the run after the first opening, the soundness proof of the control (the program of 629 bytes, Theorem 11.104, with Theorems 11.109 and 11.118) is an induction on a returning run over raw trees; it uses only that the operations it relies on are defined, and by Theorem 11.123 an opening that reaches a value other than an array or a number fails. The remaining places where such a value could enter are these. A packet that is a string or an object of four entries is destructured into strings, and its major, a string, fails at the opening that begins the major call. The comparison of the major with $[s, u]$ or $[t, t, t]$ and the membership test compare a fully opened tree, which contains only arrays and numbers, with another tree; equal trees then contain only arrays and numbers, and equal numbers have equal readings, so the identities used by the soundness proof hold. A field that is never opened or compared has no role in the induction. In an introduction the mode field u is false-valued, and the test $|a| \bmod 2 = u$ selects progress only if u is numerically 0; no false-valued value equals 1, so a compound matrix of arity 1 or 3 never receives progress. A false-valued source used as an image must open to the reading 0; numeric zeros and [ ] do, and other false-valued values fail if they are used. The extra copies of 0 in the contexts of callers (Theorem 11.109) read $0 = 0$. Hence the reading of the goal is derivable.

(c) On inputs of integers and arrays the two array tests agree, and the deleted conjunct is true on exact integers, so $\widetilde{N}$ has the runs of its control; the canonical certificate of B (Theorem 11.104) is accepted with goal b. With goal b' relate trees of equal shape whose leaves are numerically equal. The range tests, the parity condition, the truth values, the equalities and the membership tests of the program have the same values on related trees, and openings map related trees to related trees; the goal is not multiplied, decremented or tested for its type. So the run with goal b' is related to the run with goal b step by step and accepts. □

The program, of `tolerant_certificates` (`code/checker.py`), has 315/361/603 against 321/371/618 for its control under the goal contract. It satisfies neither the goal contract, since it accepts respelled goals, nor the typed contract, since it does not test the alphabet; and the programs of those contracts do not satisfy its contract: the programs with the array test $a \cdot 0 \neq 0$ do not terminate, in ideal execution, on some inputs with a nonempty string at an active position (in an actual run they reject at the recursion limit), and their soundness is not established for consumed data that are not integers and arrays. It is listed under its own code. The package states that it is below the envelope 330/380/621 of the typed contract in all three coordinates; this compares different contracts. Completeness is claimed, as elsewhere, for canonical encodings and their respellings and not for every spelling with references to dummy slots.

Two fields for the conditional-copy calculus

`alias_back` gives, besides a further delivery of the program of 479 bytes (Theorem 11.108), a certificate format for the calculus K of conditional copy (`conditional_copy`; the calculus, its terms with the counter-aware recursor in full normal form, its clauses and its rules USE, CLOSE and EXPORT are those of the chapter on witnesses, where its tree checker is measured). The tree checker reads a pair $[A, p]$ of a goal and a proof tree, infers the formula proved by p in the initial context g_0 of the reflexivity clause, and compares it with A: a node $[j, t, q]$ uses the clause j at the term t, with q a proof of its antecedent, ignored when the antecedent is empty; a node $[B, r]$ with B nonempty is CLOSE for the goal $[B]$; a node $[[\,], q, r]$ is EXPORT, with q a proof of a formula $[D]$ and r a proof in the context extended by the opened clause. Every active term and formula is validated for exact integers and arrays, sort, scope and normal form.

Definition 11.125 (Two-field stream for K)**.** A certificate is an array whose last element is the goal and whose earlier elements are pairs, consumed from the end; the whole array must be consumed. The goal-directed procedure Prove(A, g) validates A as a normal closed formula in the scope of g, removes a pair and

(i) for $[j, t]$ with j an integer, $0 \le j < |g|$: validates t, runs Prove on the antecedent of the clause j if it is nonempty, and requires the normalized instance of the clause at t to be A;

(ii) for $[[\,], [\,]]$: requires $A = [B]$ and computes the matrix, the new hypothesis and the successor instance from B and $|g|$ exactly as the tree checker does for $[B, r]$, and runs Prove on the successor instance in the extended context;

(iii) for $[B, F]$ with B nonempty: runs Prove$([B], g)$, opens B at the new parameter as the tree checker does for EXPORT, runs Prove(F, g') in the extended context g', and requires the replacement of the tree checker, applied once to F, to give A.

Proposition 11.126 (Two fields for K)**.** *For a context g reachable from g_0 and a formula A, a call* Prove(A, g) *returns after consuming a finite segment of the stack exactly when there is a proof tree p on which the tree checker, in the context g, infers A. Hence the two checkers accept the same goals.*

Proof. From a returning stream call to a tree, by induction on the call, which is finite since every call removes a pair. For $[j, t]$ the tree is $[j, t, q]$ with q the tree given by the induction hypothesis for the antecedent, or any value if the antecedent is empty; the tree checker computes the instance at t, which the stream checker has compared with A, and it validates that instance, which the stream checker has validated as A. For $[[\,], [\,]]$ the goal is $[B]$ and the tree is $[B, r]$ with r the tree for the successor instance; both checkers perform the same substitutions and the same test of the zero instance, which decides between the hypothesis $B(b)$ and the zero instance, and extend the context by the same clause. For $[B, F]$ the tree is $[[\,], q, r]$ with q the tree for $[B]$ and r the tree for F; the tree checker infers $[B]$ from q, opens B at the same parameter, retains the same clause, infers F from r and applies the same replacement, whose result is A; its final validation of A is the validation that the stream checker performs at entry.

From a tree to a stream, by induction on the tree: emit $[j, t]$ followed by the stream of the antecedent, $[[\,], [\,]]$ followed by the stream of the successor instance, and $[B, F]$, with $[B]$ and F the formulas inferred for q and r, followed by the stream of q and then that of r, in the order in which the stream checker consumes them. The same computations succeed. At the root, reverse the emitted pairs and append A; the stream checker consumes the whole array. □

The tree format admits an ignored field and the stream format does not, and an EXPORT pair carries the two formulas that the tree checker infers, so a stream can be longer than the tree; no statement about the size of certificates is made. The stream checker has 609/699/1644 (`supplement/two_field/code/two_fields_compact.py`). Its control (629/732/1715) is the tree checker of `conditional_copy` with the same stylistic simplifications, constructed by the package; the delivered tree checker has 646/748/1844. Untrusted converters translate in both directions, and each translated proof is replayed by the other checker.

11.9.14 Two fields, and computation equations recognized from the goal

This subsection follows the two-field protocol of `refinement` and its independent review, which the package includes. It changes the format of certificates and does not improve a size: computation equations consume no packet, and every other packet has two fields.

Definition 11.127 (The calculus $\mathcal{U}^{\mathrm{Tr}}$)**.** $\mathcal{U}^{\mathrm{Tr}}$ is $\mathcal{U}$ (Theorem 11.5) with SE replaced by TR and QE. Equivalently, it is $\mathcal{A}$ (Theorem 11.1) without the instances of DI-case whose matrix is a literal equation.

The two descriptions agree: both calculi have HYP, I0, IS, TR, QE and DE, and the introduction of $\mathcal{U}$ is DI-progress at a literal equation and DI-case at a compound matrix. Every rule of $\mathcal{U}^{\mathrm{Tr}}$ is a rule of $\mathcal{A}$, and SE is an instance of QE when its source is the literal 0 and of TR otherwise. Hence every public sequent derivable in $\mathcal{U}$ is derivable in $\mathcal{U}^{\mathrm{Tr}}$, and every sequent derivable in $\mathcal{U}^{\mathrm{Tr}}$ is derivable in $\mathcal{A}$. By Theorem 11.6 and the equivalence of $\mathcal{A}$ with N, a closed public formula is a theorem of $\mathcal{U}^{\mathrm{Tr}}$ exactly if it is a theorem of $\mathcal{U}$, exactly if its ordinary reading is derivable in T_{eq}.

Definition 11.128 (Two-field protocol)**.** A packet is a pair $[m, t]$. A call with goal a, context g and floor k, initially [0] and 0, performs the following steps.

(T1) If $O_k(a; \bot, 0)$ is a member of g, return.

(T2) If $a = [[n, b, h], r]$ with n false-valued and $b = r$, return. If $a = [[n, b, h], r]$ with $n = [n']$ and $O_k(h; [n', b, h], 3) = r$, return. A failed operation in this step does not reject; it passes to step (T3).

(T3) Remove the last packet $[m, t]$ and prove m by a call with context g and floor k.

(T4) If $m = [s, u]$ is binary, require $O_k(t; u, 0) = a$ and replace a by $O_k(t; s, 0)$. If m is ternary with last entry B, require $O_k(B; t, 0) = a$ and replace a by $O_k(B; 0, 0)$. If m is a true-valued scalar, reject.

(T5) If m is true-valued, let h be the threefold repetition of m. Otherwise replace a by the opening $O_k(a_0; t, 2)$ of its single child a_0 at the witness t, and let $h := a$.

(T6) Decrease the floor to $k - 1$ and call recursively with the goal $O_{k-1}(a; [k - 1], 0)$ and the context $g + [O_{k-1}(h; \iota, 0)]$, where $\iota = k - 1$ if m is true-valued or a is binary, and $\iota = 0$ otherwise.

The checker accepts if the initial call on the goal returns. Its contract is the raw contract with a canonical guarded goal assumed: finite trees of integers and arrays, and claims about canonical closed guarded goals.

Step (T2) takes the place of the computation branch of the four-field protocols. It introduces no parameter, so the zero-instance supply of Theorems 11.2 and 11.18 is not used, and no false-valued alias of zero is needed to select a mode.

Lemma 11.129 (Erasure of dummy slots)**.** *Let a raw tree a be validated at the term depth $3 + 2p$ and floor k. Then at every term depth $3 + 2(p + q)$ at which the opening of a is defined, the reading of a, with the q additional outer slots read as 0, is its reading at depth $3 + 2p$ with the p slots read as 0.*

Proof. By induction on a. Below r binders of a itself, a scalar with a reading lies in $[k, 3 + 2(p + r))$ at the shallower depth, because a is validated there; at the deeper depth it lies in the same interval and refers to the same object: 0, a parameter, one of the r inner slots, which are innermost in both readings, or one of the p outer slots, read as 0; it never refers to one of the q added slots. Annotations have no reading. At a unary or ternary term array the induction hypothesis applies to the children below their prescribed slots. □

The lemma makes no claim when an outer slot that is present in both readings carries a value other than 0. Below, the zero computation uses it with $p = 0$, and the successor computation with $p = 1$.

Lemma 11.130 (Computation leaves)**.** *Let a be a goal validated at floor k for which step (T2) returns. Then the reading of a is an instance of* I0 *or of* IS*.*

Proof. The goal is an equation $[[n, b, h], r]$ whose left side is an iterator; its children n, b are read at depth 5 below one dummy slot, h at depth 5 below the state slot, and r at depth 3. All of them are validated, since a is.

If n is false-valued, then on trees of integers and arrays it is 0 or the empty array, which read as 0. The tree b equals r, which is validated at depth 3; by Theorem 11.129 with $p = 0$ and $q = 1$, the reading of b at depth 5 is the reading of r. So a reads as $I(0, \beta;\, u.\, \eta) = \beta$.

If $n = [n']$, the left side reads as $I(S\nu', \beta;\, u.\, \eta)$, where ν' is the reading of n' at depth 9, below the dummy slot of the counter and the two dummy slots of the successor. Suppose first that the state occurs in the reading η of h. Then the opening $O_k(h; [n', b, h], 3)$ validates its image at the root term depth 3, so n', b and h are validated at depth 5 as children of the image. By Theorem 11.129 with $p = 1$ and $q = 2$ the reading of n' at depth 9 is its reading ν' at depth 5, so the image reads as $I(\nu', \beta;\, u.\, \eta)$, and by Theorem 11.12 the opening reads as $\eta[I(\nu', \beta;\, u.\, \eta)/u]$. This second validation is necessary: a scalar of n' that is admitted only because of the two dummy slots of the successor would otherwise escape into the predecessor term. If the state does not occur in η, the opening reads as η, which does not depend on the state, and the equation $I(S\nu', \beta;\, u.\, \eta) = \eta$ is the instance of IS for every ν'. In both cases the opening equals r as a tree, and equal trees at the same depth have the same reading; so a is an instance of IS. □

Theorem 11.131 (Soundness of two fields)**.** *If a call of the two-field protocol with goal a, context g and floor k returns, where a and the members of g are validated at floor k, then the reading of a is derivable in $\mathcal{U}^{\mathrm{Tr}}$ from the readings of the members of g. In particular a canonical closed public goal that the checker accepts is a theorem of $\mathcal{U}^{\mathrm{Tr}}$, and its ordinary reading is derivable in T_{eq}.*

Proof. By induction on the accepting call. In step (T1) the reading of a is that of a member of g (Theorem 11.11(b)), an instance of HYP. In step (T2) it is an axiom, by Theorem 11.130. Otherwise the major call returns; its entry scan validates m at floor k, and by the induction hypothesis the reading M of m is derivable.

If $m = [s, u]$, then M is $\sigma = \upsilon$; by Theorem 11.11(c) the goal $a = O_k(t; u, 0)$ reads as $F(\upsilon)$ and the new goal $O_k(t; s, 0)$ as $F(\sigma)$, for the reading F of the template, and TR gives $F(\upsilon)$ from the major and $F(\sigma)$. If m is ternary, M is $Q_y B'(y)$ for the reading B' of its last child B, whose first two entries are annotations and need not be copies of B; by Theorem 11.12 the goal reads as $B'(\tau)$ and the new goal as $B'(0)$, and QE applies. In both cases the threefold repetition of m has arity 6 or 9 and reads $0 = 0$, a neutral resource (Theorem 11.3); the new goal is validated at floor k by Theorem 11.11(a), so by Theorem 11.14 its opening at floor $k - 1$ has the same reading. If m is unary, $m = [B]$ reads as $D_{x,y}B'$, its repetition is the closure $[B, B, B]$, and its opening at the image $k - 1$ and floor $k - 1$ reads as $Q_y B'(b, y)$ with b the parameter $k - 1$ (Theorem 11.12); the goal is unchanged. By Theorem 11.14 the name b does not occur in the readings of g, a and m, all validated at floor k, and DE applies. A major of arity at least 4 reads $0 = 0$ and leaves the goal unchanged, with a neutral resource.

If m is false-valued, it reads $0 = 0$. A goal of arity other than 1 that reaches step (T5) either rejects at the unpacking of its children or reads as $0 = 0$, which is derivable without assumptions. For $a = [a_0]$, reading as $D_{x,y}B$, the opening $O_k(a_0; t, 2)$ reads as $B(\tau, y)$ with y the hole, where the witness τ is validated at floor k (Theorem 11.12). The new goal reads as $B(\tau, Sz)$ with z the parameter $k - 1$, and the added hypothesis as $B(\tau, z)$ if the matrix is an equation and as $B(\tau, 0)$ otherwise. The name z is fresh for g, for τ and for $D_{x,y}B$, which are validated at floor k. So the introduction of $\mathcal{U}^{\mathrm{Tr}}$ applies; a matrix of arity 0 or at least 4, or a scalar, reads $0 = 0$, and DI-case applies to it. In every case the members of the extended

context and the new goal are validated at floor $k-1$, and the induction hypothesis applies to the final call. The last statement follows from the equivalences after Theorem 11.127. □

Theorem 11.132 (Completeness with two fields)**.** *For every canonical closed public formula that is a theorem of $\mathcal{U}$ there is a stack of two-field packets that the protocol accepts with that goal.*

Proof. By Theorem 11.127 the formula has a derivation in $\mathcal{U}^{\mathrm{Tr}}$. Compile it along the deterministic schedule of the checker, writing every expression in its canonical encoding (Theorem 11.9) and naming the eigenparameter of the inference at a call with floor k by $k-1$. At a call whose goal, after the entry scan, is a member of the context, or is recognized by step (T2), emit no packet and discard the subderivation; the names of later eigenparameters then follow the pruned tree of calls and not the original derivation, so no packet is left in the stack to misalign a later major or continuation. Otherwise emit the packet of the last inference and compile the derivation of its major and then that of its continuation: TR with major $s = u$ and template F gives $[[s, u], F]$, the designated variable of F encoded as the outermost slot of a formula read below one slot; QE with major $Q_y B$ at the target τ gives $[[B, B, B], \tau]$; DE with major $D_{x,y}B$ gives $[[B], 0]$; the introduction at the witness w gives $[0, w]$, whose major 0 is proved by membership in the initial context. The stack is the reverse of the emitted sequence. A computation equation is written in the form that step (T2) recognizes: for I0 the seed is copied unchanged to the right side, since the encoding of a term without free slots does not depend on the number of slots above it, and for IS the counter is the successor array of the predecessor and the right side is the opening of the update at the recursive image, which is canonical by Theorem 11.13; a literal zero predecessor is recognized directly. Since all parameters are named in $[k, 0)$ at a call with floor k, every opening of the run returns the canonical encodings of Theorem 11.13, every test of step (T4) succeeds, and the run follows the compiled derivation. □

Step (T2) need not recognize every raw spelling of a computation equation; completeness needs only the canonical spellings. The entry scan of step (T1) stays necessary: auxiliary certificates are checked by the program itself. The certificates are not interchangeable with four-field certificates, and the protocol has no control; its accepted canonical goals are those of every checker of $\mathcal{U}$ in this chapter. The reference program of `refinement` has 295/352/742, above every minimum of Section 11.11.2: the packets carry less data, but the checker has to reconstruct the role of the major, and the package states that this does not improve the size. The package supports the proof by finite evidence: of 20,000 generated candidate leaves, 4,065 pass the entry scan and step (T2), and all of them read as true equations under a sorted numerical evaluator with one fixed assignment of the parameters; of 8,008 further candidates aimed at dummy slots and nested states, 3,608 are accepted and true and 4,400 are rejected. The program runs 26 of the 898 executions of the package's suite.

11.9.15 Computation leaves, uniform terminals and their compositions

This subsection draws on `computation_leaves`, `shared_leaf`, `uniform_terminals`, `smaller_presentations`, `vacuous_specialization`, `whole_json_refinement`, `ast269_successor_rigidity`, `selective_normalizat`, `export_specialization`, `476_byte_presentation`, `abstraction_and_refinements`, `checker_refinements`, `orbit_equality`, `fixed_points_and_checker`, `offset472`, `scope_refinement`, `validated_leaves`, `ast261`, `checked_presentations`, `root_conversion_raw`, `468_shared_leaf`, `shifted_shared_leaf`, `direct_packets`, `procedure_presentation` and `scalar_constancy`, in their checker parts. No rule is changed: every program checks $\mathcal{U}$ under the raw contract with a canonical guarded goal assumed, or $\mathcal{A}^{-}$ (equivalently $\mathcal{A}$) under a goal or typed contract. The step (T2) of The-

orem 11.128, which recognizes a computation equation as the goal itself, is transferred to the three- and four-field formats.

The eager opener and the program of 477 bytes of `head_conversion`

Several programs below open with the *eager* opener E_k, which differs from Theorem 11.10 in its fallback: an out-of-range scalar at depth d is replaced by $(d \bmod 2) \cdot E_k(t; \bot, 3)$, the product of the parity with the opened image. At an odd depth this is the opened image; at an even depth it is the scalar 0 or the empty array, according as the opened image is a scalar or an array; and the missing image is a function object, on which the array test $a \cdot 0 \neq 0$ fails.

Lemma 11.133 (Eager opener)**.** *If $E_k(a; t, d)$ is defined, then $O_k(a; t, d)$ is defined and the two trees have the same reading at every position with a reading; if $t = \bot$, then $E_k(a; \bot, d) = a$. Hence Theorems 11.11, 11.12 and 11.14 and Theorem 11.130 hold with E_k in place of O_k.*

Proof. By induction on a. The two openers traverse arrays at the same depths. A scalar in range is retained by both. For an out-of-range scalar E_k evaluates $E_k(t; \bot, 3)$, which fails for $t = \bot$; so with the missing image every scalar is in range and $E_k(a; \bot, d) = a$. With an image the evaluation is the validation that O_k performs at an odd depth, now also at an even depth, where it can only add a failure; the result at an odd depth is the opened image in both openers, and at an even depth, a formula position, it is 0 or $[\,]$ in E_k and 0 in O_k, all of which read $0 = 0$. In the proof of Theorem 11.130 the opening of the update at the image is compared with r as a tree; with E_k that opening has the reading of the opening by O_k, which is all that the proof uses. □

The program of 477 bytes of `head_conversion` (`code/compact477.py`, 272/306/477) is the four-field program of Theorem 11.116, with the header $[A, 0]$, the opener E_k and the empty array in place of the scalar 0 as initial neutral resource and as administrative major. The empty array reads $0 = 0$ at a formula position and is false-valued, so the proofs of Theorems 11.48 and 11.116 apply with Theorem 11.133; its accepted canonical goals are the closed theorems of $\mathcal{U}$. It is the matched control of `computation_leaves`, `export_specialization`, `476_byte_presentation` and `abstraction_and_refinements`.

Computation equations as leaves

Definition 11.134 (Computation candidate; direct-leaf protocols)**.** For raw fields s, u, t and a floor k put

$$H_k(s,u,t) := \begin{cases} [[[s], u, t],\ O_k(t; [s,u,t], 3)], & s \text{ true-valued}, \\ [[s,u,t],\ u], & s \text{ false-valued}, \end{cases}$$

with E_k in place of O_k in the programs with the eager opener. A *direct-leaf* program is obtained from a program of this chapter that adds computation hypotheses (the branch (4)(ii) of Theorem 11.15) by deleting that branch and inserting, after the entry scan and the membership test of a call and after the removal of its packet, the test "the goal equals $H_k(s, u, t)$", on whose success the call returns. No hypothesis and no parameter is added by a computation.

The candidate $H_k(s, u, t)$ has exactly the two shapes recognized by step (T2) of Theorem 11.128, with $n = s$ or $n = [s]$, $b = u$, $h = t$ and r the right side. `computation_leaves` delivers three direct-leaf programs: a global three-field program that reads the goal on the first line and one packet per line (control: the global stream program of Theorem 11.186, 244/272/608), a local three-field program of the same framing (control: the local stream program of `fused_iteration`, Theorem 11.187(a), 249/265/605), and a four-field program with

the header $[A, 0]$, the eager opener and the empty array as neutral resource (control: the program of 477 bytes of `head_conversion`).

Proposition 11.135 (Direct computation leaves)**.** *Let P be one of these three programs, on inputs of integers and arrays.*

(a) If a call of P with goal a, context g and floor k returns, and a and the members of g are closed at floor k, then the auxiliary reading of a is derivable in T_{eq} from the readings of the members of g.

(b) For every canonical closed public formula that is a theorem of $\mathcal{U}$ there is a certificate that P accepts with that goal.

Proof. (a) By induction on the returning call, as in the proof of the soundness of the control. The case of membership is unchanged. If the leaf test succeeds, the goal, which the entry scan has validated at floor k before the packet was removed, is literally equal to $H_k(s, u, t)$; by Theorem 11.130 (with Theorem 11.133 for the eager opener) its reading is an instance of I0 or IS, derivable without assumptions. Every other returning call passes through the logical branches of the control, which are unchanged, so the induction step of the control's proof applies; the deleted branch is the only one that used Theorem 11.2. The order matters: the entry scan precedes the comparison, so the comparison is made with a validated tree; a program that compared an unvalidated goal could accept a raw tree whose reading is not the computation instance.

(b) Take a derivation in $\mathcal{U}$ and compile it by the canonical compiler of the control (Theorem 11.21 and its refinements in Section 11.9), with the following change at the leaves I0 and IS: such a leaf is compiled to the single packet with fields $(0, a, h)$, or (n, a, h) for IS with a predecessor n that is not the literal 0, and with a major field 0 or [] in the four-field program; no packet is emitted for the continuation, since there is none. A canonical goal equals H_k of these fields, by Theorem 11.13 for the successor case. For IS with the predecessor 0 put $Z := I(0, 0;\, v.\, 0)$; the leaf $Z = 0$ and the leaf $I(SZ, a;\, u.\, h) = h[I(Z, a;\, u.\, h)/u]$, whose predecessor is not the literal 0, give the instance at 0 by SE with the source Z, which is not the literal 0, the target 0 and the whole equation as template. The leaf test is evaluated before the logical dispatch at every call, so it must be defined on canonical logical packets: the update field t of an elimination is a canonical template, whose opening at depth 3 instead of 0 raises the depth of every position by 3, so that its references remain in range, and the update fields of introductions and of openings of a major are 0. The leaf test can succeed at a call that the compiled derivation continues; the stack is therefore built by recursion on the derivation together with the state of the call (goal, context, floor), and at each node the compiler evaluates the two tests that can close a call before its logical step. If the goal is a member of the context, no packet is emitted, as in Theorem 11.21. Otherwise the packet of the last inference is formed; if the leaf test succeeds with it, this packet alone is emitted and the subderivations are discarded, the conclusion being a computation instance; only otherwise are the stacks of the major and of the continuation emitted after it. Both tests depend only on the goal, the context, the floor and the packet, and the run is deterministic, so every call consumes exactly the stack constructed for it, also when this stack is followed by the stacks of later calls. An unused prefix remains only after the outermost call. Names of parameters follow the allocation of the program. □

The proposition concerns the accepted canonical goals; certificates of the control that add computation hypotheses must be translated, and the untrusted translators of the package specialize the parameters allocated only by deleted supplies to 0 and replace each use of a supplied equation by a leaf. The programs have 243/271/616, 248/264/613 and 270/308/474 (`code/checker_tokens.py`, `code/checker_ast.py`, `code/checker_bytes.py`). Since repeated

uses of one computation are now repeated leaves, a certificate can become longer; the package reports inherited certificates that become shorter, for instance 1541 packets for an induction with a used hypothesis against 268.

`whole_json_refinement` gives the two stream programs the whole-input reader of `three_field`: the removal of the last element of the parsed array replaces the parsing of the next line. By the argument of Theorem 11.186, read in the other direction, the programs accept the array $[p_r, \dots, p_1, A]$ exactly when the stream programs accept the lines $A, p_1, \dots, p_r$, with the same run, and so accept the same canonical goals. They have 248/277/615 and 254/268/613 (`code/checker_tokens248.py`, `code/checker_ast268.py`). `orbit_equality` delivers the same two texts again (`checkers/whole_json_tokens.py`, `checkers/whole_json_ast.py`, byte for byte those of `whole_json_refinement`) and compares them with the whole-input three-field programs 249/278/607 and 255/269/605 instead of the stream programs.

`computation_leaves` also gives native programs with direct leaves, a goal-validated one (334/384/600, `code/checker_native.py`, control 334/380/596) and one with the test of the printed form (340/389/625, `code/checker_strict.py`, control 340/385/621). Removing the supply branch exposes a truth-valued initializer field to the introduction: with the mode test $|a| \bmod 2 = u$, the value $u = 1$ at a compound matrix of arity 1 or 3 would select progress at a compound matrix, which the native calculus does not have. The programs therefore test $|a| \bmod 2 = u = 0$; they also reset the flag of the first opening before the leaf test, so that the candidate is opened in the proof phase. An earlier draft of 620 bytes without the first change is not delivered. Both programs are larger than their controls.

One terminal test for hypotheses and computations

Lemma 11.136 (Idempotent opening)**.** *If $O_k(a; \bot, d)$ is defined, then $O_k(O_k(a; \bot, d); \bot, d) = O_k(a; \bot, d)$, and the two trees have the same reading.*

Proof. By induction on a. An array keeps its arity, so its children are traversed at the same depths by both openings. A scalar in $[k, d)$ is retained twice. An out-of-range scalar at an odd depth consults the missing image and fails, so it does not occur. At an even depth it becomes the scalar 0, which the second opening retains if $0 \in [k, d)$ and otherwise replaces by 0 again; this holds at negative depths below arrays of arity at least 5 as well. The statement on readings is Theorem 11.11(b). □

For the eager opener the no-image opening is the identity (Theorem 11.133).

Definition 11.137 (Uniform terminal protocol)**.** A *uniform terminal* program removes a packet at the entry of every call and returns if $O_k(a; \bot, 0)$ is a member of g extended by the one element $H_k(s, u, t)$ formed from the fields of that packet; the candidate is not retained in the contexts of later calls. Otherwise it performs the logical step of its control with that packet. A hypothesis leaf is given a dummy packet, $[0, 0, 0]$ or $[[\,], 0, 0, 0]$, whose candidate $[[0, 0, 0], 0]$ is formed without an opening.

The calculus behind the protocol is the following. Let R be $\mathcal{U}$ (Theorem 11.5) with HYP, I0 and IS replaced by the one rule LEAF: from a computation instance C (an instance of I0 or IS in the current scope) and $A \in \Gamma \cup \{C\}$ infer $\Gamma \vdash A$.

Theorem 11.138 (Shared leaf on open sequents)**.** *For every context Γ and formula A of one scope, $\Gamma \vdash A$ in $\mathcal{U}$ if and only if $\Gamma \vdash A$ in R, by translations that preserve the conclusion, the context, the scope and the height of the derivation.*

Proof. From $\mathcal{U}$ to R, by induction on the derivation: a computation leaf C becomes LEAF with that C; a hypothesis leaf becomes LEAF with $C := (I(0, 0;\, u.\, 0) = 0)$, which is available

in every scope; the other rules are kept with the translated premises. From R to $\mathcal{U}$: a LEAF with $A \in \Gamma$ becomes HYP, and otherwise $A = C$ is a computation leaf. Leaves become leaves, so heights are preserved, and no term is substituted. □

The argument applies to any calculus with a hypothesis rule and a family of axioms that has one member in every scope. It removes no axiom.

Proposition 11.139 (Uniform terminals)**.** *Let P be a uniform terminal program obtained from one of the three direct-leaf programs of Theorem 11.135. On inputs of integers and arrays, Theorem 11.135(a) holds for P, and every canonical closed public theorem of $\mathcal{U}$ has a certificate that P accepts.*

Proof. Soundness: if the opened goal $a' = O_k(a; \bot, 0)$ is a member of g, the control's argument applies. If $a' = H := H_k(s, u, t)$, then by Theorem 11.136 the tree H is validated at floor k, so by Theorem 11.130 its reading is a computation instance, and by Theorem 11.11(b) the reading of a is the reading of a', that is, of H. With the eager opener $a' = a$. A failed construction of H rejects. The candidate enters no context, and the floor is decremented only in the continuation, as in the control.

Completeness: compile as in Theorem 11.135(b) and insert a dummy packet at every hypothesis leaf, and, in a program that proves every major by a call, also at each neutral major, which is closed by membership of the neutral resource and allocates no name. On canonical trees the no-image opening is the identity, so the comparisons are those of Theorem 11.135. A match earlier than the compiled leaf ends a sound branch; the compiler of Theorem 11.135(b) emits at such a node the packet alone (here the dummy packet when the goal is a member of the context), so that no packet of the discarded subderivations is left for a later call. Translating a certificate of a local program into a global one renames the parameters. □

The raw acceptance changes: the stream program of `computation_leaves` compares the raw goal and rejects the goal `[[0,[99,99],0],[0,0]]` with the packet `[0,[0,0],0]`, while the uniform terminal program compares its opening, in which the annotations 99 at the even depth 8 become 0, and accepts; the reading is the true equation $I(0, 0;\ u.\, 0) = 0$. `shared_leaf` and `uniform_terminals` deliver the same global stream program (242/268/565) and the same four-field program (272/306/469); the latter writes the selector as any$((m, 3 \bmod |a|))$, which is defined at every selector of a canonical run because a nonempty array goal would otherwise have matched the neutral resource [] (Theorem 11.114). Their local stream programs differ: `shared_leaf` writes the initial context [0] as the byte string `b'\x00'`, whose iteration yields the integer 0 and which becomes a list at the first extension (246/262/556), and `uniform_terminals` initializes the global floor in a default argument of the opener, evaluated once at its definition (247/263/553). Both stream programs replace the loop of the control by a recursive call, and the local programs check every major, including a neutral one, by a call. Terminal packets make certificates longer. `shared_leaf` records and rejects one further shortcut, choosing the mode of the introduction from the arity of the generated hypothesis: an auxiliary matrix with Q at its root has arity 3, like the closure obtained by opening D, and would receive progress; no counterexample is exhibited (Section 11.12).

`fixed_points_and_checker` rewrites the local stream program of `shared_leaf`. Call the latter L: its context is a list, initially the byte string `b'\x00'`, and its final call has the form $c([{*g},\ O(c(g, s) \vee s \cdot 3 \vee (a := O(a; t, 2));\ \iota)],\ O(a; [k]))$ with $\iota := (k := k-1) \cdot (1 \text{ if } s \text{ else } |a| = 2)$, where s is the major after the replacement test. Let L' be the program in which the context is passed as the variadic tail g of the arguments, the initial context is the tuple (0), the major call $c(s, {*g})$ is a statement, followed by $h := s \cdot 3 \vee (a := O(a; t, 2))$, the statement $k := k - 1$ and the final call $c(O(a; [k]), {*g}, O(h; \iota'))$ with $\iota' := (s \vee |a| = 2) \wedge k$.

Proposition 11.140 (Major call as a statement)**.** *On inputs of integers and arrays, L' accepts exactly the inputs that L accepts, with the same records consumed and the same final floor. Hence it satisfies Theorem 11.139.*

Proof. Call two raw trees *related* if they become equal when every Boolean `False` in them is replaced by the integer 0. The value ι' is the new floor k when s is true-valued or $|a| = 2$, as ι is, and otherwise `False` where ι is 0; in both programs $|a|$ is evaluated only when s is false-valued, at the same opened matrix. On related arguments the opener succeeds or fails together and gives related results: `False` and 0 agree in every comparison, remainder, product, truth value and attempted iteration, and an image `False` is retained as an in-range scalar exactly where 0 would be. Membership and equality of arrays compare with `==`, under which related trees are equal. Contexts of L and L' have the same members in the same order, the byte string contributing the integer 0, and no context is changed in place. Every returning call returns the null value, so $c(g, s) \vee s \cdot 3$ is $s \cdot 3$ after the same major call. Both programs make the major call, then open the witness at the old floor, then decrement the floor once, then perform the two openings of the final call at the new floor, in opposite orders; these openings read the floor, assign nothing and do not read the input stream, so by the argument of Theorem 11.145 they commute, and if one fails both programs reject before a record is read. Induction on the consumed records relates the states of the two runs call by call. □

The program (`code/checker_tokens241.py`) has 241/267/566 against 246/262/556; it reads the goal on the first line and one record per line and carries the double dagger. Its package also lists the global stream program 242/268/565, which tests the major only when it is true-valued.

`ast261` and the compact part of `checked_presentations` rewrite the two other stream programs of `shared_leaf`. Let L be its stream program with the byte-string context (246/262/556, `code/checker_ast.py`) and G its global stream program (242/268/565, `code/checker_tokens.py`). In G the final part of a call that does not close is

$$h := (c(s, {*g}) \vee 3s \text{ if } s \text{ else } (a := O(a_0; t, 2))), \qquad k := k - 1, \qquad c(O(a; [k]), {*g}, O(h; \iota)),$$

with $\iota := (s \vee |a| = 2) \wedge k$, where s is the major after the replacement test and a_0 the child of the goal.

(a) L_{d} is L with the statement $k := 0$ before the definition of the opener deleted and the default value 0 of the depth parameter of the opener written as the assignment expression $(k := 0)$; this is the device by which `uniform_terminals` initializes its floor.

(b) G_{b} is G with the assignment of h replaced by the statement "if s, call $c(s, {*g})$; otherwise $a := O(a_0; t, 2)$", and with h in the final call replaced by $3s \vee a$.

(c) G^{w} and $G^{\mathrm{w}}_{\mathrm{b}}$ are G and G_{b} with the reading of the next line replaced by the removal of the last element of one array parsed from the whole input.

Proposition 11.141 (Rewritings of the uniform terminal stream programs)**.** *(a) On every input, L_{d} and L perform the same operations after the definition of the opener; they read the same lines and accept the same inputs.*

(b) On inputs of integers and arrays, G_{b} and G accept the same inputs, read the same records and make the same calls, with the same goals, contexts and floors.

(c) $G^{\mathrm{w}}_{\mathrm{b}}$ accepts the array $[p_r, \ldots, p_1, A]$ exactly when G_{b} accepts the lines $A, p_1, \ldots, p_r$, with the same run, and likewise G^{w} and G.

Hence the five programs accept the canonical goals of Theorem 11.139.

Proof. (a) The default values of a function are evaluated once, when its definition is executed, in the enclosing module scope. The assignment expression therefore binds the module name k to the integer 0 and has the value 0, which becomes the default depth, as in L. Before the definition no statement of either program reads k, calls a function or reads the input. After it the two modules have the same value of k, the same defaults $(0\mathrm{j}, 0)$ and the same bodies, and the rest of the two texts is identical; induction on the steps of the run gives the claim. The assignment expression is evaluated once; moving $k := 0$ into the body of the opener would reset the floor at every opening and is a different program.

(b) Neither program has a return statement with a value, so every returning call returns the null value. After the replacement test s is false-valued only if it is 0 or $[\,]$, since the replacement sets it to a nonempty array. If s is true-valued, both programs make the same major call $c(s, *g)$; after it returns, the old h is $3s$, a nonzero integer or a nonempty array, so $3s \vee a$ is $3s$. If s is false-valued, neither makes a major call, both perform $a := O(a_0; t, 2)$, the old h is the new a, and $3s$ is 0 or $[\,]$, so $3s \vee a$ is a. Both then decrement the floor once. The only relocated evaluation is that of $3s$, which now follows the decrement and the opening of the new goal; it reads neither the floor nor the input, assigns nothing and is defined on integers and arrays, so it changes no value and causes no failure, and if the opening of the new goal fails, both programs reject before a further record is read. Induction on the consumed records relates the two runs call by call, in both directions.

(c) The argument of Theorem 11.186, read in the other direction, as for the programs of `whole_json_refinement`. □

The programs are L_{d} (247/261/557, `ast261`, `code/checker.py`), against 246/262/556: the assignment statement (four nodes) and the constant default (one node) give way to an assignment expression of four nodes; G_{b} (240/266/592, `checked_presentations`, `compact/code/checker240.py`, delivered again as `code/compact_stream.py`), against 242/268/565; and $G_{\mathrm{b}}^{\mathrm{w}}$ (245/272/591, `compact/code/checker245_whole.py`, again as `code/compact_whole.py`), against G^{w} (247/274/564, `compact/controls/checker247_whole.py`), which the package constructs by the change of framing and which no earlier package delivers. The stream programs carry the double dagger. The whole-input programs read the certificates of `whole_json_refinement`, with a packet at every call; the line reader may leave an unread suffix, the whole-input reader parses the whole array first. `checked_presentations` also records, as rejected, a computation candidate of 468 bytes whose counter is a slice of the source; it is the variant of Theorem 11.162 that produces the counter $[\,]$ for a zero computation.

The checker part of `scalar_constancy` delivers L_{d} once more (`checkers/code/checker_ast261.py`, byte for byte the text of `ast261`, against the control L, `checkers/controls/ast262.py`), with the proof of (a), and delivers T_{+} of Section 11.9.16; neither program is new.

Packets as arguments, structural binding and reservation at return

This part follows `direct_packets` and the stream programs that `procedure_presentation` includes unchanged from an earlier package, `return_reservation`, which is not otherwise represented in the article. Let T be the four-field uniform terminal program of `shared_leaf` and `uniform_terminals` (272/306/469, Theorem 11.139), whose checking function is an anonymous function $c(g, a, k)$ with the body $[E$ for m, s, u, t in $[q()]]$, where q removes the last element of the input and E contains the major call in the conjunction "$c(g, m, k)$ and $3m$". Let T_{d} be T with three changes. The checking function takes the fields of its packet as arguments, $c(g, a, k, m, s, u, t)$, and its body is E. Every recursive call passes $*q()$ as its last argument, so that the packet of the callee is removed when the arguments of the call are evaluated; the major call becomes $c(g, m, k, *q())$ and is used in the product $c(g, m, k, *q()) \cdot m \cdot 3$ in place of the conjunction. The root call is $c([[\,]], *q())$, so that the last element of the input

is $[A, 0, m, s, u, t]$: the header $[A, 0]$ followed by the fields of the first packet. For an array ρ put

$$F(\rho + [[m, s, u, t], [A, 0]]) := \rho + [[A, 0, m, s, u, t]];$$

F is a bijection from the arrays whose last two elements have the lengths 4 and 2 onto the arrays whose last element has the length 6.

Proposition 11.142 (Packets as arguments)**.** *On inputs of integers and arrays, T accepts σ exactly when T_{d} accepts $F(\sigma)$, and corresponding calls have equal goals, contexts, floors and remaining stacks; every input that T_{d} accepts lies in the image of F. Hence T_{d} accepts the canonical goals of Theorem 11.139.*

Proof. (i) Every returning call of T_{d} returns the Boolean `True`: it returns the value of a disjunction whose first operand is a membership test, whose value is a Boolean, and whose second operand is the final call, which returns `True` by induction on the tree of calls. A returning call of T returns a one-element list, a true value.

(ii) The arguments of a call are evaluated from left to right, and $*q()$ is the last of them, so the packet of a callee of T_{d} is removed after its other arguments are evaluated and before the first step of its body. A callee of T removes its packet as the first step of its body, after the same arguments. No operation between the two moments reads the input or changes a value. At the root, T removes $[A, 0]$ and then $[m, s, u, t]$, and T_{d} removes $[A, 0, m, s, u, t]$; an element of another length is rejected by both, by the parameter list of the root call in T and by the seven positional parameters, none of them optional, in T_{d}. A packet of a length other than 4, or a scalar in place of a packet, is rejected by the destructuring in T and by the binding of the parameters in T_{d}.

(iii) After a returning major call the product is $\texttt{True} \cdot m \cdot 3$, which equals $3m$ for an integer m and is a copy of $3m$ for an array m; the conjunction of T has the value $3m$ by (i). No operation of either program tests the identity of arrays or changes an array in place, so the copy is not observable. A failed major call fails in both.

Relate a state of T after the removal of the header with the state of T_{d} after the removal of the merged record, and corresponding calls with equal arguments and equal remaining stacks. By (ii) and (iii) related calls perform the same openings, tests, assignments and recursive calls in the same order, and an operation fails in one exactly when it fails in the other. Induction on the tree of calls in both directions gives the claim; both programs print the same message after the root call returns. □

The program (`code/checkers/checker.py`) has 274/303/465, against 272/306/469 for T (`code/checkers/control.py`, the text of `shared_leaf`). Its certificates differ from those of T only in the last element; the package's translator implements F and its inverse and is not used in the proof.

`direct_packets` also rewrites the stream programs L and L'. Let L_{m} be L with the destructuring "$s, u, t = r$ = the next record" replaced by the statement "match the next record: case $[s, u, t]$ as r: pass", with the rest of the body unchanged; let $L_{\mathrm{m}}^{\mathrm{w}}$ and L'^{w} be L_{m} and L' with the reading of the next line replaced by the removal of the last element of one array parsed from the whole input.

Proposition 11.143 (Structural binding)**.** *(a) Executed by CPython 3.12.14 on inputs whose records are integers and arrays, L_{m} and L accept the same inputs, read the same lines and make the same calls, with the same goals, contexts and floors.*

(b) $L_{\mathrm{m}}^{\mathrm{w}}$ accepts the array $[p_r, \ldots, p_1, A]$ exactly when L_{m} accepts the lines $A, p_1, \ldots, p_r$, with the same run, and likewise L'^{w} and L'.

Hence the three programs accept the canonical goals of Theorem 11.139.

Proof. (a) For an array of length 3 the sequence pattern with three capture patterns succeeds and binds s, u, t to its entries and r to the array itself, as the destructuring does. For an integer, or an array of another length, the destructuring of L fails and L rejects. The pattern fails as well, and in the measured runtime the test that the subject is a sequence of length 3 precedes every binding of a capture name, which the package checks in the compiled code; so s, u, t, r, which are local names of the function because the body binds them, remain unbound in the new frame of the call. The next step evaluates the opening of the goal and then the tuple of the membership test, whose last member is formed from s; reading the unbound name fails, so L_{m} rejects before it reads a further record or returns. After a successful match every later step is that of L. Induction on the consumed records gives the claim. (b) The argument of Theorem 11.186, read in the other direction, as for the programs of `whole_json_refinement`. □

The language reference leaves unspecified which names a failed pattern has bound, so (a) is a statement about the measured runtime, as the package states; on certificates that consist of three-field records the two programs agree in every runtime. The programs are L'^{w} (`code/checkers/checker_tokens_json.py`, 246/273/565), $L_{\mathrm{m}}^{\mathrm{w}}$ (`code/checkers/checker_ast_json.py`, 258/263/595) and L_{m} (`code/checkers/checker_ast_jsonl.py`, 252/259/595), against L' (241/267/566) and L (246/262/556), reproduced with identical text. In the syntax tree the capture names of a pattern are fields of the pattern and not separate nodes, which saves three nodes against L and costs six tokens and 39 bytes. The package compares the two whole-input programs with those of `whole_json_refinement` (248/277/615 and 254/268/613).

The second device changes when a fresh name is allocated. Write the final call of a call of L that does not close as

$$c([{*}g,\ O(c(g,s) \vee 3s \vee (a := O(a;t,2));\ \iota)],\ O(a;[k])), \qquad \iota := (k := k-1)\cdot(1 \text{ if } s \text{ else } |a| = 2),$$

as in Theorem 11.140. Let L_{r} be L with the final call

$$c([{*}g,\ O(3s \vee (a := O(a;t,2));\ c(g,s) \vee k \cdot \mathrm{bool}(s \vee |a| = 2))],\ O(a;[k]))$$

and with the statement $k := k - 1$ at the end of every call, after its terminal test or after its final call. So the threefold repetition of the major, or in an introduction the opening of the matrix at the witness, is formed before the major call, and the fresh name is the floor left by the return of the major call. Let L'_{r} be L_{r} with the floor initialized as in L_{d}, with every call returning the floor after its final decrement, and with the image of the hypothesis written $c(g,s) \cdot (1 \text{ if } s \text{ else } |a| = 2)$.

Proposition 11.144 (Reservation of the fresh name at return)**.** *Let P be L_{r} or L'_{r}, on inputs of integers and arrays.*

(a) If a call of P with goal a and context g returns, and a and the members of g are closed at the floor at the entry of the call, then the auxiliary reading of a is derivable in T_{eq} from the readings of the members of g.

(b) Every canonical closed public theorem of $\mathcal{U}$ has a stream certificate that P accepts.

Proof. The floor is changed only by the final decrement of a call, so it never increases, and if a call entered at the floor k_0 returns, the floor at its return is at most $k_0 - 1$. In L_{r} every returning call returns the null value, so the image is $k \cdot \mathrm{bool}(s \vee |a| = 2)$, read after the major call; in L'_{r} the major call returns the floor at its return, and the product with 1 or with a Boolean is the same value. The initialization of L'_{r} is that of Theorem 11.141(a).

(a) By induction on the returning call. Let k_0 be the floor at the entry. The opening of the goal, the terminal test, the replacement test with the opening of the source instance, and

in an introduction the opening of the matrix at the witness, are evaluated at k_0; the terminal cases are those of Theorem 11.139, and the final decrement changes no reading. Otherwise the major call is made with the context g at the floor k_0, as in L, and the induction hypothesis applies to it as in the proof for L. When it has returned the floor is some $k_1 \leq k_0 - 1$, and the hypothesis is opened at the image k_1 or 0 and the new goal at $[k_1]$, both at the floor k_1. By Theorem 11.14 the name $k_1 \in [k_1, k_0)$ is fresh for the readings of a, of the members of g, of the major and of the opened matrix, which are closed at k_0, and these readings are unchanged at the floor k_1. The proof of Theorem 11.139 for L uses of its eigenparameter only this freshness and the closedness of the hypothesis and the new goal at the floor of the continuation, and it uses of the opening of the matrix only that it is made at a floor at which the goal is closed and that does not admit the eigenparameter, which holds since $k_1 < k_0$. So the step is the inference of L with k_1 as eigenparameter, and the induction hypothesis applies to the continuation, whose context members are closed at k_1.

(b) Take the certificate of L given by Theorem 11.139, and translate it along the run of L, keeping an injective map ν from the names that L has allocated to names that P has allocated: where L allocates its eigenparameter n after a returning major call, P uses the floor k_1 left by the corresponding major call, and ν is extended by $n \mapsto k_1$; every negative scalar of a later packet is replaced by its image under ν. The range of ν before the extension consists of names not below $k_0 > k_1$, so ν stays injective. A canonical tree contains only allocated names; a tree closed at the floor of L is mapped to a tree closed at the floor of P at the corresponding point, and an injective renaming of names commutes with openings and preserves literal equality, the arity tests and membership. Every call, including a call that closes the neutral major by membership, returns and decrements the floor, so a name is available at every allocation. The run of P on the translated certificate therefore follows the run of L call by call. □

The order is essential: an introduction opens its matrix at the witness before the major call reserves the eigenparameter, so the witness cannot name it; the package records that a stream certificate whose witness is Sz for the name $z = -1$, at the entry floor 0, is rejected after two records, before its neutral major is read. Accepted stacks of L and P differ in the names of parameters, so only the accepted goals are claimed equal; in the certificate of an induction with a used hypothesis the final floor is -281 against -140, one reservation per call. The programs are L_{r} (`code/inherited_tokens240.py`, 240/266/551) and L'_{r} (`code/inherited_ast261.py`, 242/261/563), against L; both read one record per line and carry the double dagger. The earlier package also contains a Lean model of the order of reservations on abstract call trees; it is not part of the formalization of the repository, and the proof above does not use it.

Proposition 11.145 (Tail call for the continuation)**.** *In the global stream program of Theorem 11.135, replace the loop by a recursive call on the new goal $O(a; [k])$ and the context extended by the new hypothesis, evaluating the opening of the new goal before that of the hypothesis. The resulting program accepts the same stream certificates with the same consumption of packets and the same final global floor.*

Proof. An opening at a fixed floor reads the floor, allocates new arrays and makes no assignment to a global name and no read of the input, so two successful openings at one floor commute, and if either fails both orders reject before a further packet is read. The global floor is decremented once, after the major call, in both programs, and is not restored by a returning call. A loop iteration of the control and a call of the new program begin in the same state; every returning call returns the null value, so the expressions around recursive calls are unchanged. Induction on the consumed packets gives the claim in both directions. □

The program, of `smaller_presentations` (`code/checker_tokens.py`), has 242/269/606

against 243/271/616. Python does not eliminate tail calls, so the physical recursion limit is reached sooner.

Compositions on the leaf programs

Proposition 11.146 (Compositions)**.** *Each of the following programs has the accepted canonical goals of its control; (c) and (d) have exactly the runs of their controls, and every input accepted by (a) or (b) is accepted by its control with the same run.*

(a) The program of 474 *bytes of* `computation_leaves` *with the selector* $\mathrm{bool}(m \vee 3 \bmod |a|)$ *written* $\mathrm{any}((m, 3 \bmod |a|))$ *(*272/308/473*;* `vacuous_specialization`*,* `compact/checker473.py`*, and* `smaller_presentations`*,* `code/checker_bytes.py`*, with identical text).*

(b) The program of 477 *bytes of* `head_conversion` *with the same change (*274/306/476*;* `476_byte_presentation`*,* `compact/compact476.py`*,* `export_specialization`*,* `compact476/code/comp` *and* `abstraction_and_refinements`*,* `code/checkers/compact_checker.py`*).*

(c) The whole-input local three-field program of `three_field` *(*255/271/606*) with the floor argument rewritten as in Theorem 11.187(a) (*255/269/605*;* `ast269_successor_rigidity`*,* `checker/checker_ast269.py`*, and* `selective_normalization`*,* `compact/code/checker269.py`*, with identical text).*

(d) The programs of `whole_json_refinement` *above, against the stream programs of* `computation_leaves`*.*

Proof. (a), (b): the argument of Theorem 11.114(a),(b): where $3 \bmod |a|$ is defined the two selectors have the same truth value, its evaluation changes no state, and the decrement follows it in both; at every selector of a canonical run the goal is a canonical formula or an opened matrix, of arity 1, 2 or 3, and neutral majors return at membership before the selector. (c): the proof of Theorem 11.187(a) uses only that $k \leq 0$ before the decrement and that the program applies no exact-type test to internal trees; both hold under the whole-input reader, which is not involved. (d): the framing argument above. □

`checker_refinements` delivers the programs (b) and (c) once more (`checkers/byte476/checker476.py` and `checkers/ast269/checker269.py`, with the texts of `476_byte_presentation` and `ast269_successor_r` its proof of (c) relates the internal Boolean `False` of the rewritten floor expression to the integer 0, as in the proof of Theorem 11.140. The compact part of `root_conversion_raw` delivers the program (b) a fifth time (`compact/checker476.py`, byte for byte the text of `checker_refinements`), with the control of 477 bytes of `head_conversion` and the proof of (b): on a successful run the remainder $e = 3 \bmod |a|$ is defined, and for every value m and every defined e the truth values of $m \vee e$ and of $\mathrm{any}((m, e))$ agree; at every selector of a canonical run the goal is a nonempty canonical formula or opened matrix, because a neutral goal [ ] returns at membership. Its contract states the public grammar as the least class containing every equation, $D_{x,y}B$ for a compound public B, and the guarded $D_{x,y}$ of Theorem 11.5; this is the grammar of that definition. It also delivers a readable expansion (307/362/1847, `compact/readable_checker.py`) whose agreement with the program is supported by 396 executions of 66 cases and is not proved.

`smaller_presentations` records a rewriting that must not be made: in the candidate, writing $O(t; [s, u, t], 3)$ if s else u as $s \wedge O(t; [s, u, t], 3) \vee u$ confuses an opened right side equal to the scalar 0 with a missing value. The goal $I(2, 1;\, u.\, 0) = 1$, whose left side is 0, then has the candidate $[[[s], u, 0], u]$ with s and u encoding 1, and the rewritten program accepts it, while the program of (a) rejects it.

Rewritings of the validated programs

`abstraction_and_refinements` and `476_byte_presentation` rewrite the native programs of Theorem 11.118 and the program of 621 bytes; their controls are the programs (e), (f), (g) and (h) of that proposition (321/371/618, 334/380/596, 330/380/638, 340/385/621) and the program (d) (323/369/622).

Proposition 11.147 (Rewritings of the validated programs)**.** *Each of the following changes preserves the runs of the program on inputs of integers and arrays, except (iv), which has the relation of Theorem 11.114(a),(b) to its control.*

(i) The product $(d \bmod 2) \cdot B$ *of the parity with a Boolean* B *written as the bitwise conjunction* $d \,\&\, B$*.*

(ii) In the leaf condition of the first opening, the exact-integer test "the leaf is identical with its conversion to an integer" followed by $a \bmod 2 = [a > 1]$ *replaced by "a is not the object* `False`*" followed by* $(a \,\&\, 1) = [a > 1]$*.*

(iii) The import of the reader as a name, assignment expressions in place of an assignment followed by its only use, the opener applied where it is bound, and, in a procedure, an early return on membership replaced by the negated test around the rest of the body.

(iv) The selector $\mathrm{bool}(m \vee e)$ *with* e *the test* $|a| \bmod 2 = u$ *written* $\mathrm{any}((m, e))$*.*

(v) The extension $g \mathrel{+}= [0]$ *of a list written* `g += b'\x00'`*.*

(vi) The flag of the first opening given the default value 1*, with the literal* 0 *passed at the final call.*

Proof. (i) For an integer d, $d\&\texttt{False} = 0 = (d \bmod 2)\cdot\texttt{False}$ and $d\&\texttt{True} = d\&1 = d \bmod 2$, also for negative d; both sides evaluate d and then B once, without effect. (ii) The condition is evaluated in the first opening, at floor 0, after the range test $0 \le a < d$, or inside a guard $v \le p$ that holds whatever p is when $v = 0$. On the values of the reader that reach it: for an exact integer $a \ge 0$ both conditions are $a \bmod 2 = [a > 1]$, since $a \,\&\, 1 = a \bmod 2$; `False` fails both; `True` fails the old exact-integer test, and in the new condition $\texttt{True}\&1 = 1 \neq [\texttt{True} > 1]$; a float fails the old test and makes $a \,\&\, 1$ undefined; strings, `null` and objects fail the range test. So the opener retains the same leaves and rejects the others. The new test is an identity with a singleton of the language, so the qualification about one runtime of the old test (Theorem 11.87) is removed. (iii) Each rewriting evaluates the same expressions in the same order and binds the same objects before their first use. (iv) Theorem 11.114. (v) Every reachable context is an ordinary list, and an in-place extension by a byte string appends its byte values, here the integer 0; the identity test of exact integers is true for every exact integer and cannot separate the two zeros. (vi) The first call receives $v = 1$ in both programs, the major call passes $v := 0$ explicitly, and the final call now passes the literal 0; no expression reads the defaults of the function, which serves as the missing image. □

The programs of `abstraction_and_refinements` are, with changes (i)–(iv): 317/364/617 (native, (i), (ii), on (e)), 321/358/622 (native, (i)–(iii), on (e)), 332/373/594 (native, (i), (ii), (iv), on (f)), 328/377/634 (strict, (i), on (g)), 335/368/643 (strict, (i), (iii), on (g)) and 340/382/616 (strict, (i), (iv), on (h)); the native programs satisfy the goal contract $\mathrm{G}/\mathcal{A}^{-}$ and the strict programs the typed contract T/ed. The programs of `476_byte_presentation` are, with (v) or (vi): 319/369/622 (on (e)), 321/367/626 (on (d)), 328/378/642 (on (g)), all with (v), and 332/379/594 (on (f)) and 338/384/619 (on (h)), with (vi). The change (v) costs four bytes and saves tokens and nodes.

Uniform zero in the validated native programs

`smaller_presentations` transfers the recoding R of Section 11.4 to the validated native programs: the public goal is $B^\sharp = R(B)$ (Theorem 11.119), and the proof phase opens with O^+, written `a or 1` in the retained-scalar case. Let N_1 be the program 332/373/594 above with the leaf condition of the first opening replaced by $v \le d \,\&\, (a \bmod 2)$ preceded by a test of exact integers ("a is $+a$", or "a is not `True`" with $a \,\&\, 1$), and with O^+ in the proof phase; and let S_1 be the program 340/382/616 with the leaf condition replaced by $v \le d \,\&\, (a \bmod 2)$, without a test of exact integers, which its test of the printed form makes unnecessary, and with O^+ in the proof phase.

Proposition 11.148 (Uniform zero, validated)**.** *(a) S_1 accepts only inputs that satisfy J and whose goal satisfies $P_1^{\mathrm{ed}}(a, 0)$; N_1 accepts only goals that satisfy J and $P_1^{\mathrm{ed}}(a, 0)$.*

(b) If S_1 accepts, or N_1 accepts an input whose consumed packets satisfy J, then the reading of the goal, in which the scalar 1 reads as 0, is a closed theorem of $\mathcal{A}$.

(c) For every closed theorem B of $\mathcal{A}$ in the language with equality and D, both programs accept a certificate with goal $B^\sharp$.

Proof. (a) With the flag 1 and the floor 0, a leaf is retained exactly when $0 \le a < d$, the depth d is odd and a is odd: the bitwise conjunction is 1 exactly then. `True` fails "a is $+a$", since $+$`True` is the integer 1, and fails "a is not `True`"; `False` has even parity; a float makes $d \,\&\, (a \bmod 2)$ or $a \,\&\, 1$ undefined. The arity conditions are those of the control, and Theorem 11.59 gives P_1^{ed}; as in Theorem 11.110(b), the array test $a \cdot 0 \neq 0$ of N_1 traverses a nonempty string of an admitted length without end, which is not an acceptance. For S_1 the test of the printed form gives J (Theorems 11.62 and 11.113).

(b) After the first opening the program is the kernel of its control with O^+, and the soundness proof of that kernel (Theorem 11.96(a) in its native form, with Theorems 11.102 and 11.114) uses only Theorems 11.11, 11.12, 11.14 and 11.18, which hold for O^+ (Theorem 11.25). The source of a transport may now be the true-valued scalar 1; the major $[1, u]$ then reads $0 = \upsilon$, and the step is an instance of TR of $\mathcal{A}$ with the source 0. In an introduction the field u is 0 or $[\,]$, since a true-valued u selects the computation branch; the mode test selects progress only at a matrix of even arity, which is an equation or has the reading $0 = 0$, and the goal $D_{x,y}(0 = 0)$ is a theorem. A goal $[\,]$ reaching the introduction is unpacked into a call with two arguments; it reads $0 = 0$, so the conclusion is derivable whatever the continuation. The closed goal is the reading of $B^\sharp$ (Theorem 11.24).

(c) Compile a derivation of B in $\mathcal{A}$ with R applied to the expression fields and the packets $[[R(s), R(u)], R(s), R(u), R(F)]$ for TR, $[[R(B'), R(B'), R(B')], 0, R(u), R(B')]$ for QE with matrix B', $[R(D_{x,y}B'), 0, 0, 0]$ for DE, $[0, 0, R(a), R(h)]$ and $[0, R(n), R(a), R(h)]$ for I0 and IS, and $[0, 0, [\,], R(w)]$ and $[0, 0, 0, R(w)]$ for DI-case and DI-progress. By Theorem 11.23 every source $R(s)$ and every initializer $R(a)$ is true-valued, so transport from 0 selects the equation branch and every computation, including IS at the predecessor 0, selects the computation branch, without the repair of Theorem 11.17. Openings commute with R (Theorem 11.24), every selector sees a nonempty canonical formula, and the run follows the compiled derivation as in Theorem 11.29. The goal $B^\sharp$ satisfies P_1^{ed}, and canonical certificates satisfy J. □

The programs are 328/372/581 (N_1 with "a is $+a$", `code/checker_native.py`), 328/369/587 (N_1 with the explicit exclusion of `True`, `code/checker_native_portable.py`) and 334/378/608 (S_1, `code/checker_strict.py`). The test "a is $+a$" relies on the measured runtime, in which unary plus returns an exact integer itself and turns a Boolean into an integer. The package compares them with the programs of 594 and 616 bytes. Their goals are the encodings $B^\sharp$, so, as for `zero_alias` (Theorem 11.120), they satisfy neither the completeness of their controls

nor its converse, and they are listed under their own codes. With the encoding of the controls they share the calculus, the four-field format and the kernel. The package also notes that the administrative markers keep their roles: the neutral major, the source marker of closure elimination and the counter flag of a zero computation stay 0, progress is selected by 0 and cases by [ ], so a converter must recode the expression fields only.

11.9.16 An even offset of the depths

This subsection follows `offset472` and, for the uniform terminals in the shifted encoding, `468_presentation`, `468_shared_leaf`, `shifted_shared_leaf`, `scalar_constancy` and `procedure_presentation`. The program of 473 bytes of Theorem 11.146(a) opens with the eager opener E_k, whose fallback evaluates the opened image also at an even depth (Theorem 11.133). At the formula root, of depth 0, the scalar 0 lies outside the range $[k, 0)$ for every floor k, so its opening consults the missing image and fails. For this reason that program, like the program of 477 bytes of `head_conversion`, uses the empty array as neutral resource and as neutral major; replacing only the initial context [[ ]] by [0] gives a program that rejects every certificate with a neutral major. `offset472` moves every depth instead, so that the formula root lies above the scalar 0.

Definition 11.149 (Shifted reading)**.** Let $e \geq 0$. The *e-shifted reading* is the reading of Theorem 11.8 with the following changes. A term position below p slots has the depth $2e + 3 + 2p$, and a formula position the depth $2e + 2p$. At a term position below p slots a numeric scalar r reads as 0 if $0 \leq r < 2e + 3$, as the slot j if $2e + 3 + 2j \leq r < 2e + 5 + 2j$ and $j < p$, as the parameter named r if $k \leq r < 0$, and as the hole otherwise. Arrays, dummy slots and annotations are read as in Theorem 11.8, and the child of an array of arity r at depth d lies at depth $d + 5 - r$. The *e-shifted canonical encoding* is the encoding of Theorem 11.9 with the slot j written $2e + 3 + 2j$. The e-shifted openers are O_k of Theorem 11.10 and E_k of Section 11.9.15 with the image validated at the root term depth $2e + 3$ instead of 3.

For $e = 0$ these are the definitions of Section 11.2. The canonical encoding contains, besides arrays, only the scalar 0, negative names and slot codes, which are the scalars of value at least 3; so the e-shifted canonical encoding of an expression is obtained from its canonical encoding by adding $2e$ to every scalar of value at least 3.

Lemma 11.150 (Shifted codec)**.** *Let $e \geq 0$. Theorems 11.11 to 11.14, 11.129, 11.130 and 11.133 hold for the e-shifted reading, encoding and openers, with every depth that they mention increased by $2e$: the formula root has depth $2e$, the root term depth is $2e + 3$, and a closed goal $[B]$ opens its witness at the depth $2e + 2$. If $e \geq 1$, then for every floor $k \leq 0$ the scalar 0 is retained by $O_k(\cdot; \bot, 2e)$ and by $E_k(\cdot; \bot, 2e)$, is closed at every floor, and reads $0 = 0$ at the formula root.*

Proof. The proofs of the cited lemmas use the encoding through four facts only. (i) The parity of the depth of a position is its sort. (ii) The child depth $d + 5 - r$ adds two slots at a unary array, changes the sort at a binary one and adds one slot at a ternary one. (iii) At a term position of depth D below p slots, the range $[k, D)$ of the opener is the disjoint union of the interval $[k, 0)$ of parameters, of the interval of 0, and of the intervals of the p slots, ordered from the innermost to the outermost; hence the scalars in range are exactly those that do not read as the hole, and reading at the depth $D - 2$ turns exactly the interval of the outermost slot into the hole. (iv) An image validated at the root term depth has, below q of its own binders, only scalars in the range of the root term depth plus $2q$, which read as 0, as parameters or as its own binders at every term position, below any number of further slots. In the e-shifted reading $D = 2e + 3 + 2p$, the interval of 0 is $[0, 2e + 3)$, the slot intervals are $[2e + 3 + 2j, 2e + 5 + 2j)$ and the root term depth is $2e + 3$; facts (i) to (iv) hold with the same

arguments. The interval of parameters does not depend on e, so the freshness lemma and its proof are unchanged, and the proof of the canonical opening lemma uses the slot codes only through (iii). For the last statement, $0 \in [k, 2e)$ for $k \leq 0$ exactly when $e \geq 1$, a scalar at a formula position has no hole and reads $0 = 0$, and a retained scalar is not replaced by either opener. □

Let P_+ be the program of Theorem 11.146(a), which reads four-field packets and the header $[A, 0]$, with four changes: the default depth of the opener is 2 instead of 0; images are validated at the depth 5 instead of 3, in the fallback of the opener and in the candidate of the leaf test; the witness of an introduction is opened at the depth 4 instead of 2; and the initial context is [0] instead of [[]]. It reads 1-shifted canonical certificates whose packets without substantive major have the major field 0.

Proposition 11.151 (Shifted direct-leaf program)**.** *Consider inputs of integers and arrays whose last element is* $[A, 0]$*.*

(a) If a call of P_+ with goal a, context g and floor k returns, and a and the members of g are closed at floor k in the 1*-shifted reading, then the auxiliary reading of a is derivable in T_{eq} from the auxiliary readings of the members of g. If P_+ accepts an input whose goal A is the* 1*-shifted canonical encoding of a closed public formula, that formula is a theorem of $\mathcal{U}$.*

(b) For every closed public theorem of $\mathcal{U}$ there is a certificate that P_+ accepts with the 1*-shifted canonical encoding of that theorem as goal.*

Hence P_+ accepts exactly the 1*-shifted canonical encodings of the closed public theorems of $\mathcal{U}$.*

Proof. (a) The soundness of the control is Theorem 11.135(a) for the four-field program of 474 bytes, whose proof rests on Theorems 11.48 and 11.116 and Theorem 11.133, transferred to the eager selector by Theorem 11.146(a). These proofs use the encoding only through the lemmas listed in Theorem 11.150, and the neutral resource and the neutral major [ ] only through four properties: (1) it is closed at every floor and reads $0 = 0$; (2) it is false-valued, and so is its threefold repetition, so that a packet with this major is an introduction; (3) it is equal to no nonempty array, in particular to no candidate M° of the replacement test, which is a nonempty array in every case (Theorem 11.48); (4) a call with this goal returns by membership of the initial resource, which is a member of every context, since contexts are extended by concatenation. The scalar 0 has (1) by Theorem 11.150 with $e = 1$, (2) since $0 \cdot 3 = 0$, (3) since no scalar equals an array, and (4) since $E_k(0; \bot, 2) = 0$ for $k \leq 0$. The selector $\mathrm{any}((m, 3 \bmod |a|))$ has the same value for $m = 0$ and for $m = [\,]$. So the induction of those proofs applies with the 1-shifted lemmas. For the second statement, a 1-shifted canonical encoding contains no negative scalar, so it is closed at the floor 0 of the header and reads as the encoded formula, and Theorem 11.6 concludes.

(b) Compile as in the proof of Theorem 11.135(b), with its branch-aware treatment of early closures, in the 1-shifted canonical encoding and with the major field 0 in place of [ ] in every packet without substantive major. By Theorem 11.150 the equalities of trees on which that proof relies hold for the shifted encodings: the canonical opening lemma for the replacement test, for the openings of a matrix, of a closure and of an iterator body, and for the identity of the candidate H_k with the canonical computation instance; the instance of IS at the predecessor 0 is obtained from the leaf $Z = 0$ and SE as there. The candidate of a logical packet is defined, since a canonical template is opened in it at the depth 5 instead of 2, which raises every depth by 3, as for $e = 0$. A neutral major is closed by membership of 0, which the program tests before it reads a packet, so it consumes no packet. □

Remark 11.152 (Scope of the shifted program). The proposition concerns the accepted canonical goals and makes no claim about the raw acceptance of P_+ and of its control, which differ: the threefold repetition of a binary major has arity 6, and its children are traversed at the depth 1 instead of -1, so an annotation 0 can be retained by one program and replaced by the other; a scalar 1 at the formula root is retained by P_+. A canonical certificate of the control becomes one of P_+ by adding 2 to every slot code of an expression field and writing 0 for the neutral major [ ]; the package's translator does this for canonical certificates and is not part of the proof. By the argument of Theorem 11.116(b), which uses the scalar 0 of a canonical goal at a term position, now of depth at least 5, a header $[A, k_0]$ with k_0 an array or a positive integer is rejected; the package states its contract for the header floor 0 only. `offset472` also states the family of Theorem 11.149 for every $e \geq 0$ and observes that $e = 1$ is the least offset for which the scalar 0 can serve as neutral formula. Its soundness argument is carried out for the ordinary reading in the native core as well; the converse of soundness is asserted only for public formulas, where the two readings agree: the matrix $E(y) \equiv (y = S0)$ has a derivable ordinary reading $E(0) \to \forall y\, E(y)$ of $Q_y E$, since $E(0)$ is false, while its progressive reading $\forall y\, (E(y) \to E(Sy))$ fails at $y = S0$.

The program (`offset472.py`) has 271/307/472 against the program of 473 bytes of `vacuous_specializati` (272/308/473, `control473.py`): the initial context saves one token, one node and one byte, and the depth constants have the same length. Its goals are 1-shifted encodings, so it is listed under its own code.

Uniform terminals in the shifted encoding

`468_presentation` applies to the four-field uniform terminal program of `shared_leaf` and `uniform_terminals` (272/306/469, Theorem 11.139) the four changes that turn the program of 473 bytes into P_+: the default depth 2, the root term depth 5 in the fallback of the opener and in the candidate, the witness depth 4, and the initial context [0]. Call the result T_+. Its calls differ from those of P_+ only at the terminal test: a call of P_+ returns if $E_k(a; \bot, 2)$ is a member of g, and otherwise removes a packet $[m, s, u, t]$ and returns if a equals the candidate $H_k(s, u, t)$ of Theorem 11.134, formed with the image validated at the depth 5; a call of T_+ removes a packet first and returns if $E_k(a; \bot, 2)$ is a member of $g + [H_k(s, u, t)]$. Otherwise both programs perform the same logical step. A returning call of T_+ returns a nonempty list, and one of P_+ the value `True` or a nonempty list; the caller uses only the truth value.

Theorem 11.153 (Terminal packets in the shifted encoding)**.** *On inputs of integers and arrays whose last element is* $[A, 0]$*, there is a certificate that* T_+ *accepts with the goal* A *if and only if there is one that* P_+ *accepts with the goal* A*. Hence* T_+ *accepts exactly the* 1*-shifted canonical encodings of the closed public theorems of* $\mathcal{U}$ *(Theorem 11.151).*

Proof. By Theorem 11.133, which holds in the 1-shifted encoding by Theorem 11.150, a defined opening $E_k(a; \bot, 2)$ is a itself: an out-of-range scalar would evaluate the missing image, the checker function, and fail. Both programs evaluate this opening before any comparison of a call (T_+ after removing its packet), so both validate the same goal before their first comparison, and the test "$E_k(a; \bot, 2) = H$" of T_+ is the test "$a = H$" of P_+.

From T_+ to P_+, by induction on the tree of calls of an accepting run, in the order of evaluation (major call before continuation). At a call whose opened goal is a member of g, delete the packet that T_+ removed there; P_+ returns at its membership test without reading a packet, and the construction of H by T_+, which succeeded, changes no context, floor or input. At any other call keep the packet: if the opened goal equals H, both programs return; otherwise both perform the same logical step on the same packet, and the induction hypothesis applies to the major call and to the continuation. Terminal calls allocate no parameter, so the

floors of corresponding calls agree. The kept packets, in the order of consumption, together with the unused prefix, form a certificate that P_+ accepts with the same goal.

From P_+ to T_+, insert the packet $[0,0,0,0]$ at every call at which P_+ returns by membership. Its candidate is $[[0,0,0],0]$, formed without an opening, since the false-valued source selects the right side 0; the opened goal is a member of g, so T_+ returns. Every other packet is kept, and the calls correspond as before. Inserted packets allocate no parameter, so every later floor and every name is unchanged.

The second statement follows from the first by Theorem 11.151, whose soundness covers arbitrary integer and array trees inside the certificate. □

The theorem concerns the existence of certificates; the two programs do not accept the same stacks, since one removes a packet at a hypothesis leaf and the other does not. A canonical certificate of the uniform terminal program of 469 bytes becomes one of T_+ under the translation of Theorem 11.152; the package's translator is untrusted and the proof does not use it. The program (`code/compact468.py`) has 271/305/468, against 272/306/469 for its unshifted control (`code/control469.py`, the text of `shared_leaf`) and against 271/307/472 for P_+ (`code/reference_offset472.py`, the text of `offset472`). The package also delivers a readable form (327/391/1572, `code/checker_readable.py`), whose agreement with T_+ is supported by 264 executions and is not proved, and a validated companion, `code/checker_validated.py` (690/773/2064), which runs the kernel of T_+ (the same assignment, compared as syntax trees) on the parsed input after three tests: every value of the whole input is an exact integer or an array, the last element is $[A,0]$ with the integer 0, and A satisfies the following predicate V. For a tuple σ of slots, innermost first, each marked genuine or dummy, a term satisfies $V_{\mathrm{t}}(\sigma)$ if it is 0; or a scalar $5+2j$ with slot j of σ genuine; or $[t]$ with t in V_{t} of σ extended by two dummy slots; or $[n,b,h]$ with n,b in V_{t} of σ extended by one dummy slot and h in V_{t} of σ extended by one genuine slot. A formula satisfies $V(\sigma)$ if it is $[s,t]$ with s,t in $V_{\mathrm{t}}(\sigma)$, or $[B]$ with B in V of σ extended by two genuine slots, where, if B is an array of arity 2, both of its entries are arrays $[7,0,h]$ of arity 3.

Proposition 11.154 (The validated companion)**.** *A tree satisfies $V(\,)$ exactly when it is the 1-shifted canonical encoding of a closed public formula of $\mathcal{U}$. Hence the validated companion accepts an input exactly when the input consists of integers and arrays, its last element is $[A,0]$ with such an A, and T_+ accepts it; it accepts exactly the encodings of closed public theorems of $\mathcal{U}$, on arbitrary JSON input.*

Proof. By induction on the tree. Slot j below p slots is written $5+2j$ in the 1-shifted encoding (Theorem 11.149); the canonical encoding uses only 0 for zero, slot codes and negative names, and V_{t} admits 0 and the codes of genuine slots, so it excludes the other spellings of zero, negative names (the goal is closed) and references to dummy slots, which the canonical encoding never makes. The dummy slots added at a successor and at the counter and initializer of an iterator, and the genuine state slot of the body, are those of the reading, by the child depth $d+5-r$. Below $D_{x,y}$ the variable y is slot 0 and x slot 1; at the counter of an iterator in the matrix one dummy slot is added, so y is written 7 there, and an equation matrix $[[7,0,h],[7,0,j]]$ is the guarded matrix $I(y,0;\,u.\,h) = I(y,0;\,v.\,j)$ of Theorem 11.5, with h and j checked as terms below their state slots. A compound matrix is a formula $[B']$, which is checked recursively; arrays of arity 3, which encode Q, are rejected. The test of the alphabet applies the exact-type test of integers, which excludes Booleans, and the reading of the consumed packets by the kernel is that of T_+ on the same parsed array. Soundness is then Theorem 11.153; for completeness, the certificates given by that theorem consist of integers and arrays and end with $[A,0]$. □

The companion is not compared with the compact programs and has its own code.

Proposition 11.155 (Collision of generated hypotheses)**.** *In the stream and four-field programs, the hypothesis generated at an elimination with the unary major* $[B]$*, before its opening, is the closure* $[B, B, B]$*, which is opened at the fresh parameter; the hypothesis generated at an introduction whose goal is* $D_{x,y}Q_vC$ *is the opened matrix, an array* $[C', C', C']$ *of arity* 3*, which is opened at* 0*. Hence a selector that reads the image of the opening from the arity of the generated hypothesis alone gives both the same image.*

Proof. The closure of the elimination is the threefold repetition of the major. At an introduction the goal $[[C, C, C]]$ is opened at its witness by an opening of its only child, which keeps the arity 3 and opens the three copies alike. □

A selector that gives the fresh image to every hypothesis of arity 3 therefore grants progress at matrices with Q at their root, which by Theorem 11.52 yields induction for every native formula; one that gives them the image 0 opens the closure of an elimination of D at 0 in place of a fresh parameter, which is not an instance of DE. This is the shortcut that `shared_leaf` rejects (Section 11.9.15); restricting the public goal to equality and D does not exclude the auxiliary formulas of the derivation of Theorem 11.52. `468_presentation` claims no certificate that a program with this selector accepts for a false or an underivable goal, and T_+ keeps the selector of its control.

Further deliveries of T_+, and the checking function as a procedure

This part follows `468_shared_leaf`, `shifted_shared_leaf`, the checker part of `scalar_constancy` and the compact part of `procedure_presentation`. The first three deliver T_+ again, each byte for byte the text of `468_presentation`: `code/checker468.py` of `468_shared_leaf`, `code/checker.py` of `shifted_shared_leaf` and `checkers/code/checker_bytes468.py` of `scalar_constancy`, with the control T of `shared_leaf`, also byte for byte (`code/control469.py`, `controls/shared_leaf469.py`, `checkers/controls/bytes469.py`). Each package obtains it from T by the four changes stated above and presents it as the composition of the shared leaf with the even offset. The compact part of `selfseed_injectivity` delivers T_+ once more (`compact/checker468.py`, 271/305/468, against T), a reconstruction that its package records as byte for byte the text `code/compact468.py` of `468_presentation`; its recorded hash is the one that `468_presentation` records for that file, and its checking function is literally the kernel of the validated companion of `468_presentation`; it is not a new program.

Remark 11.156 (The three proofs of the shifted terminal program)*.* `shifted_shared_leaf` and `scalar_constancy` prove Theorem 11.153 by the correspondence with P_+ used there: a defined eager no-image opening returns its argument literally, the packet $[0, 0, 0, 0]$ is harmless at a hypothesis leaf, and the different values returned by successful calls are used only through their truth value. `468_shared_leaf` gives instead a direct induction on the successful call in the 1-shifted reading, with the shifted forms of the opening, exposure and freshness lemmas and of Theorem 11.130 (Theorem 11.150), and completeness by the branch-aware compiler of Theorem 11.135(b) with a dummy packet at every hypothesis leaf and neutral major; this is Theorem 11.151 together with the step of Theorem 11.139, and adds nothing to the proof given here. `shifted_shared_leaf` also delivers a readable form of T_+ (305/363/1933, `code/readable_checker.py`), and `468_shared_leaf` one without a reported size (`code/readable_checker.py`); their agreement with T_+ is supported by fixtures and is not proved. Both packages give certificates of false goals for weakened programs (the end of Section 11.8, and the paragraph after Theorem 11.107 for `468_shared_leaf`).

Proposition 11.157 (The checking function as a procedure)**.** *Let* P *be* T_+*, or* N_2 *or* S_2 *of Section 11.9.17. In* P *the checking function is an anonymous function whose body is a comprehension over the one-element list* $[q()]$ *that destructures the removed packet into*

m, s, u, t and has the element E, and the major call occurs in the conjunction "$c(g, m, k)$ and $3m$". Let P_{p} be the program in which the checking function is a named procedure with the same parameters and defaults, whose body is the statement $m, s, u, t := q()$ followed by E as an expression statement, with the conjunction replaced by the disjunction "$c(g, m, k)$ or $3m$", and in which the import of the reader follows the definition. Then P and P_{p} accept the same inputs, with the same packets consumed and the same goals, contexts, floors and, for N_2 and S_2, values of the flag v.

Proof. This is the converse of Theorem 11.98. Every returning call of P returns a one-element list, a true value; every returning call of P_{p} returns the null value, since the procedure has no return statement. After a returning major call the conjunction of P and the disjunction of P_{p} both have the value $3m$; if it is false-valued, both continue with the same opening of the matrix; a failed major call fails in both. The comprehension iterates exactly once; it removes one packet and destructures it as the assignment statement does, failing on the same values, and then evaluates E. The assignment expressions in E, to the opener i, to a, to k and in N_2 and S_2 to v, bind in the enclosing anonymous function in P, which is the rule of the host language for assignment expressions in comprehensions, and in the procedure in P_{p}; so the opener, which reads k and v, sees the same values. The missing image is the global checking function in both, evaluated when the opener is formed during a call, after the definition; no operation inspects its name, code or defaults. The value of E is used in P only as the element of the result; in P_{p} it is discarded. The module of the reader is bound before the first call in both. Induction on the tree of calls, in both directions, relates the runs; an operation fails in one exactly when it fails in the other. □

Hence $T_{+,\mathrm{p}}$, the procedure form of T_+, accepts exactly the 1-shifted canonical encodings of the closed public theorems of $\mathcal{U}$ (Theorem 11.153). `procedure_presentation` delivers it as `code/checker_compact462.py` (267/298/462), against T_+ (`controls/offset_shared468.py`, byte for byte the text of `468_presentation`, 271/305/468). Its own lemma for the step from P_+ to T_+ is Theorem 11.153, proved in the same way; it notes that an unused packet can make the candidate undefined, so equality of the accepted stacks is not claimed for that step. Removing the comprehension removes frames, so the physical recursion limits differ.

11.9.17 Uniform terminals in the validated programs with uniform zero

This subsection follows the checker parts of `scope_refinement` and `validated_leaves`, the strict part of `checked_presentations` and the validated part of `procedure_presentation`. The first two combine the uniform terminals of Theorem 11.137, the direct leaves of Theorem 11.134 and the uniform zero of Theorem 11.148 in the validated programs S_1 and N_1, which answers a question of Section 11.12. The no-image opening O_k^+ is not idempotent on raw trees: at the depth 1, which only annotations reach, a scalar 0 is retained and returned as 1, which a second opening at the depth 1 rejects; and at the floor 0 and the depth 2 the scalar 2 opens to 0, which opens to 1. So Theorem 11.136, on which the proof of Theorem 11.139 relies, is not available for O^+, as both packages state; the proofs below use the following lemma instead.

Lemma 11.158 (Closed computation candidates)**.** *Let s, u, t be raw trees of integers and arrays and k a floor. Let c be $[s]$ if s is true-valued, and s or the scalar 1 if s is false-valued. Let $H := [[c, u, t], O_k(t; [s, u, t], 3)]$ if s is true-valued and $H := [[c, u, t], u]$ otherwise, where O_k may be replaced by O_k^+ or E_k. If H is closed at floor k, its reading is an instance of I0 or IS.*

Proof. The proof of Theorem 11.130 uses the validation of the goal through two facts only: the trees n, b, h, r of the goal $[[n, b, h], r]$ have readings without a hole at their positions,

and, in the successor case, r is the result of the opening $O_k(h; [n', b, h], 3)$ with the same raw n', b, h. The first holds for a closed H. The second holds by construction of H. The proof of Theorem 11.129 uses of the validation of a only that every scalar with a reading lies in the range of its position at the shallower depth, which is the closedness of a there. If s is false-valued, the counter c is the scalar 0, the empty array or the scalar 1, each of which reads 0 at the term position of depth 5, so the case of a false-valued counter in the proof of Theorem 11.130 applies. For O_k^+ and E_k the lemmas used in that proof hold by Theorems 11.25 and 11.133. □

Let S_2 be the program S_1 of Theorem 11.148 with three changes.

(i) The computation branch of the hypothesis is deleted: a true-valued initializer field selects nothing.

(ii) Every call removes a packet $[m, s, u, t]$ at its entry and returns if the opened goal $O_k^+(a; \bot, 0)$ is a member of the tuple consisting of 0, the members of the context, and the candidate H of Theorem 11.158 with $c = [s]$ or $c = 1$ and the opener O_k^+. The member 0 is written as the assignment that ends the first phase; the left operand of the membership test, the opening of the goal, is evaluated before it, and the candidate after it. The initial context is empty.

(iii) The floor of the new hypothesis is the fresh parameter exactly when m is true-valued, or the opened matrix has arity 2 and $u = 2$.

Let N_2 be N_1, with the test "a is $+a$", with the same three changes. Let S_3 be S_1 with the change (i), with (ii) replaced by: every call removes a packet at its entry and returns if $O_k^+(a; \bot, 0)$ is a member of the context extended by $O_k^+(H; \bot, 0)$, where H is the candidate of Theorem 11.158 with $c = [s]$ or $c = s$, the flag of the first phase being reset before H is formed and the initial context being [0]; and with (iii) replaced by: the floor of the new hypothesis is the fresh parameter exactly when m is true-valued, or the opened matrix has arity 2 and $u \neq 2$. Finally, let S_1^{T} and S_1^{N} be the strict token and node profiles of Theorem 11.147 (328/377/634 and 335/368/643) with the two changes that turn the program 340/382/616 into S_1: the leaf condition of the first opening $d \,\&\, (a \bmod 2)$, without a test of exact integers, and O^+ in the proof phase.

Proposition 11.159 (Uniform terminals with uniform zero)**.** *(a) S_2, S_3, S_1^{T} and S_1^{N} accept only inputs that satisfy J and whose goal satisfies $P_1^{\mathrm{ed}}(a, 0)$; N_2 accepts only goals that satisfy J and $P_1^{\mathrm{ed}}(a, 0)$.*

(b) If one of S_2, S_3, S_1^{T}, S_1^{N} accepts, or N_2 accepts an input whose consumed packets satisfy J, then the reading of the goal, in which the scalar 1 reads as 0, is a closed theorem of $\mathcal{A}$.

(c) For every closed theorem B of $\mathcal{A}$ in the language with equality and D, each of the five programs accepts a certificate with goal $B^\sharp$.

Proof. (a) The first opening of each program is that of S_1 or N_1; in S_1^{T} and S_1^{N} the first phase is marked by the empty context, as in their controls (Theorem 11.103). In S_2, N_2 and S_3 the flag of the first phase is 1 when the goal is opened at the root call and is reset only afterwards, before the candidate is formed. The root call removes a packet before this opening, which changes nothing in the argument of Theorem 11.148(a), and that argument applies.

(b) For S_1^{T} and S_1^{N} the proof of Theorem 11.148(b) applies with the kernels of their controls, whose soundness is that of the program (g) of Theorem 11.118 with Theorem 11.147, and with Theorem 11.25. For S_2, N_2 and S_3 the proof is the induction of Theorem 11.148(b) on the

returning call, for contexts whose members are closed at the floor, with the following cases at the entry. Let $a' = O_k^+(a; \bot, 0)$; it is closed and has the reading of a (Theorem 11.25). If a' is the scalar 0, the goal reads $0 = 0$. If a' is a member of the context, HYP applies. In S_2 and N_2, if $a' = H$, then H is closed, and Theorem 11.158 shows that its reading, which is that of a, is a computation instance. In S_3, if $a' = O_k^+(H; \bot, 0)$, then H is validated, hence closed, and has the reading of a', and the same lemma applies. The candidate enters no context. A call that proves the neutral major 0 removes a packet and returns by membership of 0: the opening of 0 at the formula root in the proof phase is 0, and 0 is the first member of the tuple in S_2 and N_2 and the initial member of every context in S_3. In the other calls the logical steps are those of S_1 or N_1, without the computation branch, so a true-valued u selects nothing, and with the new selector. In an elimination the selector is the fresh parameter, as before. In an introduction the hypothesis $B(\tau, z)$ is granted exactly when the opened matrix has arity 2 and the marker condition holds; such a matrix reads as a literal equation, and the step is DI-progress. Every other matrix receives $B(\tau, 0)$, and the step is DI-case, which $\mathcal{A}$ has for every matrix. The remaining steps (transport with the source 1, the empty goal at an introduction) are those of the proof of Theorem 11.148(b).

(c) For S_1^{T} and S_1^{N} this is the proof of Theorem 11.148(c) with the completeness of the program (g). For S_2, N_2 and S_3 compile as in Theorem 11.148(c), with the changes of Theorem 11.139 and the branch-aware compiler of Theorem 11.135(b). A hypothesis leaf and a neutral major receive a dummy packet, $[0, 0, 1, 1]$ for S_2 and N_2 and $[0, 0, 0, 0]$ for S_3, whose candidate is formed, and in S_3 opened, without failure. I0 and IS receive $[0, 0, R(a), R(h)]$ and $[0, R(n), R(a), R(h)]$. In S_2 and N_2 the candidate is $[[1, R(a), R(h)], R(a)]$ or $[[[R(n)], R(a), R(h)], O_k^+(R(h); [R(n), R(a), R(h)], 3)]$, which is R of the computation instance by Theorem 11.24. In S_3 the candidate of I0 has the counter 0, which O^+ returns as 1, and O^+ fixes canonical encodings with uniform zero (Theorem 11.24), so the opened candidate is again R of the instance. The predecessor 0 of IS is the true-valued scalar 1, so no repair is needed. An introduction by cases receives the marker $u = 0$ in S_2 and N_2 and $u = 2$ in S_3, and an introduction by progress the marker $u = 2$ in S_2 and N_2 and $u = 0$ in S_3; a compound matrix receives cases with any marker. Transport, QE and DE receive the packets of Theorem 11.148(c). In the candidate of a canonical packet a closed term field occurs at a depth not below the root term depth, and a template or a matrix at a depth at least 3 above the formula root; so all their scalars remain in range, the image of an opening in the candidate consists of closed canonical terms and a field so placed, and every opening in the candidate, and in S_3 the opening of the candidate, is defined. The run then follows the compiled derivation. □

The programs are S_2 (`validated_leaves`, `code/checker_strict606.py`, 334/379/606 against S_1, 334/378/608), N_2 (`code/checker_native579.py`, 328/373/579 against N_1, 328/372/581), S_3 (`scope_refinement`, `code/strict_bytes.py`, 335/382/607 against S_1), S_1^{T} (`code/strict_tokens.py`, 322/373/626 against 328/377/634) and S_1^{N} (`code/strict_ast.py`, 329/364/635 against 335/368/643). The controls of S_1^{T} and S_1^{N} encode zero by the scalar 0 and are programs of T/ed; the comparison crosses the encodings, as for `zero_alias`. `scope_refinement` also delivers a readable form of S_3 (421/503/2152, `code/readable_strict.py`), whose agreement with S_3 is supported by a differential test of 740 inputs and is not proved. The tuple of S_2 supplies the neutral resource by the value of the assignment that ends the first phase; S_3 opens its candidate so that a counter 0 becomes 1, while S_2 writes the counter as $s \wedge [s] \vee 1$ and needs no opening. `validated_leaves` states Theorem 11.138 for $\mathcal{A}$ in place of $\mathcal{U}$, with the same proof, which the remark after that theorem covers, and Theorem 11.3 for the neutral formula $0 = 0$. The converter of `scope_refinement` recodes canonical certificates and rejects a retained computation that is not a leaf; completeness is the compilation above, from derivations.

Conditional selection and the second marker

The strict part of `checked_presentations` gives three further programs of the contract T/ed$_1$. In S_1^{T} and S_1^{N} the new goal after the replacement test is formed as the element, selected by the test, of the pair $(a, O_k^+(t; s, 0))$, so that the source instance is opened also when the test fails. Let S_2^{T} be S_1^{T} with this pair replaced by the conditional expression "$O_k^+(t; s, 0)$ if the test holds, else a". Let S_2^{N} be S_1^{N} with the same change and with the factor $\mathrm{bool}(m \vee |a| \bmod 2 = u)$ of the new floor written "1 if m else $|a| \bmod 2 = u$". Let S_4 be S_3 with its change (iii) replaced by the change (iii) of S_2 (progress exactly at a matrix of arity 2 with $u = 2$), and with the right operand $g + [(v := 0) \vee O_k^+(H; \bot, 0)]$ of the terminal test written $g + [(v := 0), O_k^+(H; \bot, 0)]$.

Proposition 11.160 (Three further strict programs)**.** *The statements (a), (b) and (c) of Theorem 11.159 hold for S_2^{T}, S_2^{N} and S_4. Moreover every input accepted by S_1^{T} or S_1^{N} is accepted by S_2^{T} or S_2^{N} respectively, with the same packets consumed and the same calls.*

Proof. For S_2^{T} and S_2^{N}: the first opening is that of their controls, which gives (a). The comparison of the replacement test, including the opening $O_k^+(t; u, 0)$ of the target, is evaluated in both programs. If it holds, both open the source instance and take it as the new goal. If it fails, the new goal is a in both, and the control additionally evaluates $O_k^+(t; s, 0)$ and discards it; the opener reads no packet, assigns nothing and allocates no parameter, so this evaluation can only cause a rejection. The two factors of the new floor are equal integers after the multiplication by the new floor: for a true-valued m they are 1 and `True`, otherwise the same Boolean comparison, evaluated in the same order. Hence a run of the control is a run of the new program, which gives the last statement and, with the completeness of the controls, (c). For (b), the induction of Theorem 11.159(b) for the controls uses the source instance only at a step whose replacement test holds, where it is the new goal; at a step whose test fails, the step is justified by the major and the generated hypothesis alone (DE for a unary major, a neutral resource for a true-valued major of another shape, a computation hypothesis or an introduction for a false-valued major), so the induction applies to the runs of the new programs as well.

For S_4: the value 0 of the assignment is a member of the extended context in both forms, since the initial context [0] is a member of every context; the order of evaluation is the same (the opened goal, then the assignment that ends the first phase, then the opened candidate), so the terminal test has the same value. The runs of S_4 are therefore those of S_3 except at the selector of an introduction. Statement (a) is that of S_3. In the proof of Theorem 11.159(b) the hypothesis $B(\tau, z)$ is granted only when the opened matrix has arity 2, whatever the marker condition, so (b) holds. For (c) compile as for S_3, with the dummy packet $[0, 0, 0, 0]$, with the marker $u = 2$ for an introduction by progress and $u = 0$ (or [ ]) for one by cases, as for S_2; the opened candidate of an introduction packet $[0, 0, 2, w]$ is defined, since the scalar 2 is in range at the depths 5 and 3 and reads 0 there, and a compound matrix receives cases with every marker. □

The programs are S_2^{T} (319/370/630, `code/strict_tokens.py`, again as `strict/code/strict_tokens.py`) S_2^{N} (327/358/642, `code/strict_ast.py`) and S_4 (335/380/605, `code/strict_bytes.py`); their controls in the article are S_1^{T} (322/373/626), S_1^{N} (329/364/635) and S_3 (335/382/607), from which they differ by exactly the changes stated. The package compares them with the program of `zero_alias` and S_1, and supplies the strict token and node profiles of Theorem 11.147 as controls of construction. It states the selector $|a| = u = 2$ of S_4 as a repair found by its review: with the computation branch deleted, the selector $|a| \bmod 2 = u$ grants progress at a compound matrix for $u = 1$ (Theorem 11.161); the unrepaired draft is not delivered. The untrusted translator of the package turns canonical certificates with computation supplies into terminal certificates; the proof does not use it.

Remark 11.161 (The mode of an introduction). With the computation branch deleted, a true-valued field u reaches the introduction. The selector $|a| \bmod 2 = u$ of S_1 would then grant progress at a compound matrix of arity 1 or 3 for $u = 1$, which adds induction for compound formulas (Theorem 11.52); this is the hazard of the native direct-leaf programs of `computation_leaves` (Section 11.9.15). `scope_refinement` withdraws a prototype of 606 bytes with this selector, which is not a result. The two packages choose opposite markers: in S_2 the scalar 2 marks progress, in S_3 it marks cases and is not a public numerical code. `scope_refinement` notes that selecting progress at every equation would be a sound selector for the guarded calculus $\mathcal{U}$ but loses DI-case at an equation in $\mathcal{A}$, unless a separate recovery theorem applies.

Remark 11.162 (Variants that were examined). The packages record the following variants as unsuccessful; the figures are theirs. `validated_leaves`: writing the test $|a| = 2$ as the truth value of the slice $a[1:]$ gives a compact program of 464 bytes that grants progress also at a matrix of arity 3, an auxiliary closure, as in Theorem 11.52; the package refutes one proposed repair, a decoder that reads nested closures as truth, by the template $F(x)$ whose inner matrix is $\mathrm{sel}(x, 0, y) = 0$, true at $x = 0$ and false at $x = S0$, which the repaired decoder would let QE transfer; the slice $s[:1]$ as counter (a program of 468 bytes) produces the counter $[\,]$ in a zero computation while the guard of a public equation carries the scalar 0, and no repair is given. `scope_refinement`: the local floor $k = -|g|$ gives 250/267/573 and $k = 1 - |g|$ gives 251/268/576, both above their controls and needing a renaming of eigenparameters by branches (Theorem 11.56); writing `for ... in q()` in place of `for ... in [q()]` lets an empty packet produce an empty comprehension and bypasses the check of the goal; moving the decrement into one arm of the selector allocates no fresh parameter at a compound introduction; giving the opener the next fresh name before the major and the witness are validated admits future witnesses; returning the proved formula in place of a truth value requires keeping the original goal through its replacements; and a canonical successor-seeded recursor would remove about 17 bytes of computation dispatch but needs a normalizing substitution that costs more.

The procedure form and the byte-string extension

The validated part of `procedure_presentation` applies two devices to the programs of this subsection, at a revision that does not contain `checked_presentations`. Let $S_{2,\mathrm{p}}$ and $N_{2,\mathrm{p}}$ be the procedure forms of S_2 and N_2 (Theorem 11.157), and let $S^{\mathrm{T}}_{1,\mathrm{b}}$ and $S^{\mathrm{N}}_{1,\mathrm{b}}$ be S^{T}_1 and S^{N}_1 with the in-place extension $g \mathrel{+}= [0]$ of the context written g += `b'\x00'`.

Proposition 11.163 (Two devices on the validated programs)**.** *The statements (a), (b) and (c) of Theorem 11.159 hold for $S_{2,\mathrm{p}}$, $N_{2,\mathrm{p}}$, $S^{\mathrm{T}}_{1,\mathrm{b}}$ and $S^{\mathrm{N}}_{1,\mathrm{b}}$; each accepts exactly the inputs of its control, with the same run.*

Proof. For $S_{2,\mathrm{p}}$ and $N_{2,\mathrm{p}}$ this is Theorem 11.157; the reader, the test of the printed form of S_2, the initial call with $v = 1$ and the verdict are unchanged. For the other two it is the change (v) of Theorem 11.147: the initial context is the empty list and every context passed to a call is a list, either that of the caller or one formed by concatenation, so every extension in place is the extension of a list, and the extension by the byte string appends the integer 0 to the same list object, which is also the list of the caller. No operation inspects the byte string after the extension. □

The programs are $S_{2,\mathrm{p}}$ (`code/checker_strict_procedure.py`, 330/372/600, against S_2, 334/379/606), $N_{2,\mathrm{p}}$ (`code/checker_native_procedure.py`, 324/366/573, against N_2, 328/373/579), $S^{\mathrm{T}}_{1,\mathrm{b}}$ (`code/checker_strict_tokens_iterable.py`, 320/371/630, against S^{T}_1, 322/373/626) and $S^{\mathrm{N}}_{1,\mathrm{b}}$ (`code/checker_strict_ast_iterable.py`, 327/362/639, against S^{N}_1, 329/364/635);

the controls are reproduced with identical text. $N_{2,\mathrm{p}}$ keeps the test "a is $+a$" of N_2. The package also delivers $S_{2,\mathrm{p}}$ with comments and line breaks (`code/checker_strict_annotated.py`), which has the same syntax tree. The byte-string extension and the conditional selection of Theorem 11.160 act on different statements of S_1^{T}; their composition is not delivered.

The returned floor as the image, and rotated packets

This part follows the checker part of `fused_blocks_and_checkers`, the third delivery of `checked_presentation` and `prefix_conditional`, all at one revision that contains `procedure_presentation` and `direct_packets`. The first two rewrite the stream program L_{r} of Theorem 11.144, the third rewrites L_{r}, $T_{+,\mathrm{p}}$ and $S_{2,\mathrm{p}}$. Recall that the image of the new hypothesis in the final call of L_{r} is $c(g,s) \vee k \cdot \mathrm{bool}(s \vee |a| = 2)$, the second argument of an opening whose first argument $3s \vee (a := O(a;t,2))$ opens the matrix at the witness.

(i) L_{f} is L_{r} with this image written $(c(g,s) \vee s \vee |a| = 2) \wedge k$, and L_{c} is L_{r} with it written "k if $c(g,s) \vee s \vee |a| = 2$ else 0".

(ii) For $P \in \{L_{\mathrm{f}}, L_{\mathrm{c}}\}$, P_{m} is P with the floor initialized in the default of the opener as in L_{d} (Theorem 11.141(a)) and the destructuring of the next record replaced by the structural binding "match the next record: case $[s,u,t]$ as r: pass" as in L_{m} (Theorem 11.143); and $L'_{\mathrm{f,m}}$ is $L_{\mathrm{f,m}}$ with the declaration of the floor as a module name moved from the head of the body into the case, in place of its empty statement.

(iii) $L_{\mathrm{c}}^{\mathrm{w}}$ and $L_{\mathrm{c,m}}^{\mathrm{w}}$ are L_{c} and $L_{\mathrm{c,m}}$ with the reading of the next line replaced by the removal of the last element of one array parsed from the whole input; in $L_{\mathrm{c,m}}^{\mathrm{w}}$ the reader is bound by an assignment expression in the argument of the root call, as in $L_{\mathrm{m}}^{\mathrm{w}}$.

(iv) For $P \in \{T_{+,\mathrm{p}}, S_{2,\mathrm{p}}\}$, whose calls destructure the removed packet as m,s,u,t, P^{ρ} is P with the destructuring $s,u,t,m := r := q()$ and with the arrays $[s,u,t]$ (the image of the computation candidate) and $[s,u]$ (the inferred major) replaced by the prefixes $r[:3]$ and $r[:2]$. For an array σ let $\rho(\sigma)$ be σ with every element other than the last that is an array of length 4, say $[m,s,u,t]$, replaced by $[s,u,t,m]$; ρ is a bijection on arrays, and its inverse rotates the other way.

Proposition 11.164 (The returned floor as the image, and rotated packets)**.** *On inputs whose values are integers and arrays:*

(a) L_{c} has exactly the runs of L_{r}; L_{f} accepts the same inputs as L_{r} and reads the same lines, with the same floors, and its trees are those of L_{r} up to the replacement of `False` *by* 0*.*

(b) Executed by CPython 3.12.14, $L_{\mathrm{f,m}}$ and $L'_{\mathrm{f,m}}$ accept the same inputs as L_{f}, and $L_{\mathrm{c,m}}$ the same inputs as L_{c}, with the same lines read and the same calls, goals, contexts and floors.

(c) $L_{\mathrm{c}}^{\mathrm{w}}$ accepts the array $[p_r, \ldots, p_1, A]$ exactly when L_{c} accepts the lines $A, p_1, \ldots, p_r$, with the same run, and likewise $L_{\mathrm{c,m}}^{\mathrm{w}}$ and $L_{\mathrm{c,m}}$.

(d) For $P \in \{T_{+,\mathrm{p}}, S_{2,\mathrm{p}}\}$, P^{ρ} accepts $\rho(\sigma)$ exactly when P accepts σ, with the same run.

Hence the programs of (a), (b) and (c) accept a certificate with a given canonical closed goal exactly when L_{r} does, on the stream or on the reversed array, and Theorem 11.144 applies to them; $T_{+,\mathrm{p}}^{\rho}$ accepts exactly the 1-shifted canonical encodings of the closed public theorems of $\mathcal{U}$ (Theorem 11.153); and the statements (a), (b) and (c) of Theorem 11.159 hold for $S_{2,\mathrm{p}}^{\rho}$, with the certificates of (c) rotated by ρ.

Proof. (a) No call of the three programs has a return statement with a value, so every returning call returns the null value. In each of the three images the major call $c(g, s)$ is evaluated first and once, after the first argument of the opening; if it fails, all three programs reject. If it returns, let k_1 be the floor at its return. If s is true-valued, none of the three evaluates $|a|$; the image of L_{r} is $k_1 \cdot \texttt{True} = k_1$, that of L_{c} is k_1, and in L_{f} the disjunction has the value s, so the conjunction has the value k_1. If s is false-valued, all three evaluate $|a| = 2$ on the same opened matrix and fail together where it is undefined; if it holds, all three images are k_1; otherwise the images of L_{r} and L_{c} are 0 and that of L_{f} is `False`. The test reads no record, assigns nothing and leaves the floor unchanged, so it does not matter that L_{r} reads k before it and the two other programs after it. All other operations are literally the same, so L_{c} has the runs of L_{r}. For L_{f} relate trees up to the replacement of `False` by 0, as in the proof of Theorem 11.140: the opener succeeds or fails on related arguments together and gives related results, `False` and 0 agreeing in every comparison, truth value, product and failed traversal, and an image `False` being retained as an in-range scalar exactly where 0 is; the value `False` enters only through the image of a new hypothesis, hence only members of contexts, which are used only in tests of membership, and these compare with `==`, under which related trees are equal. Induction on the consumed records relates the runs call by call; the final decrement is the same.

(b) The initialization is that of Theorem 11.141(a): the default is evaluated once, when the definition of the opener is executed, before any statement reads the floor. The structural binding is that of Theorem 11.143(a): on an array of length 3 it binds s, u, t, r as the destructuring does; otherwise the names stay unbound in the new frame and the first test of the call, whose tuple is formed from s, fails before a further record is read. A declaration of a module name is not executed: it fixes for the whole body of the function that k denotes the module name, wherever in the body it stands, and it is legal there because no occurrence of k precedes it in the text. The empty statement performs nothing. Hence $L'_{\mathrm{f,m}}$ and $L_{\mathrm{f,m}}$ perform the same operations; the case in which the pattern fails is the same in both.

(c) The argument of Theorem 11.186, read in the other direction, as in Theorem 11.141(c) and Theorem 11.143(b); in $L^{\mathrm{w}}_{\mathrm{c,m}}$ the reader is bound when the argument of the root call is evaluated, before the body of the root call removes the goal.

(d) At every consumed element of length 4 the destructuring of P^{ρ} binds to m, s, u, t the same values as that of P at the corresponding element of σ, and then $r[{:}3]$ and $r[{:}2]$ are new arrays with the elements of $[s, u, t]$ and $[s, u]$, evaluated at the places of these arrays. No operation of either program tests the identity of an array or changes an array in place, and the image is traversed by the opener into new arrays, so the copies are not observable. A consumed element of another length, which ρ leaves unchanged, or a scalar, fails the destructuring in both programs, and unused elements do not affect a run. In $S_{2,\mathrm{p}}$ the test of the printed form removes the characters of a fixed set from both ends of the printed whole input and requires that nothing remain, that is, that every character belong to the set; ρ permutes elements of arrays and does not change the set of characters of the printed form, so the test agrees. Induction on the tree of calls in both directions gives the claim. The consequences follow from Theorem 11.144, Theorem 11.153 and Theorem 11.163, with ρ applied to the certificates. □

The programs are L_{f} (239/261/549; `checkers/code/checker_tokens239.py` of `fused_blocks_and_check`
and, byte for byte the same text, `code/token239.py` of the third delivery of `checked_presentation`),
$L_{\mathrm{f,m}}$ (246/257/589, `code/checker.py` of the third delivery), $L'_{\mathrm{f,m}}$ (245/256/580, `checkers/code/checker_ast`
of `fused_blocks_and_checkers`), and, of `prefix_conditional`, L_{c} (239/261/553, `code/checker_tokens239`
$L_{\mathrm{c,m}}$ (246/257/593, `code/checker_ast257.py`), $L^{\mathrm{w}}_{\mathrm{c}}$ (244/269/547, `code/checker_tokens244.py`),
$L^{\mathrm{w}}_{\mathrm{c,m}}$ (252/261/593, `code/checker_ast261.py`), $T^{\rho}_{+,\mathrm{p}}$ (267/298/461, `code/checker_bytes461.py`)
and $S^{\rho}_{2,\mathrm{p}}$ (330/372/599, `code/checker_strict599.py`), against L_{r} (240/266/551), $T_{+,\mathrm{p}}$ (267/298/462)
and $S_{2,\mathrm{p}}$ (330/372/600), reproduced with identical text. The stream programs read the goal

on the first line and one record per line and carry the double dagger; the whole-input programs read the reversed array. L_{f} is below L_{r} in all three coordinates, and L_{c} has one token and five nodes fewer and two bytes more. The initialization in a default costs one token and one byte and saves one node, and the structural binding with its empty statement costs six tokens and 39 bytes and saves three nodes; moving the declaration into the case saves one token, one node and nine bytes more. The rotation costs four bytes for the alias r and saves four and one bytes at the two prefixes. The three packages compare their stream programs for nodes with L_{m} (252/259/595), which uses the allocation of L and not the reservation at return; $L'_{\mathrm{f,m}}$, $L_{\mathrm{f,m}}$ and $L_{\mathrm{c,m}}$ are below it in all three coordinates, a comparison of sizes and not of runs. `prefix_conditional` uses as the control of runs of $L_{\mathrm{c}}^{\mathrm{w}}$ the same reframing of L_{r} (`controls/matched_return_whole245.py`), whose size it does not report. The readable forms (`checkers/code/checker_readable.py` and `code/readable.py`) have no reported size, and their agreement with the programs is supported by tests and not proved.

Remark 11.165 (A rejected mode test). `prefix_conditional` records that the mode test of the image may not be replaced by a comparison such as $m \neq 3 \bmod |a|$ in place of the truth value of m or of $3 \bmod |a|$. The neutral major $[\,]$ is false-valued, so the program takes the branch of an introduction and opens the matrix at the witness; but the comparison is true for every arity of the matrix, so the progressive image k is supplied also for a compound matrix, and the generated hypothesis is the progressive reading where the conditional reading is required (Theorem 11.5). No such program is delivered; the consequence that the package states is listed in Section 11.12.

11.9.18 The audit of sources

The audit in `orbit_closure` measures every Python source of the ninety package trees at one pinned revision: 5825 paths with 1380 distinct contents, read and normalized as data and not executed. Its normalization removes docstrings at every level of nesting; this correction of an earlier pass changes 173 measurements and none of the minima. Among the complete compact checkers of the guarded calculus it finds the minima 256 tokens, 276 nodes and 506 bytes, attained by three programs: the loop profile with a global floor of `kernel_refinement` (256/283/594), a loop profile of `deterministic_opening` (263/276/580), and the compact program of `deterministic_introduction` (267/303/506). All 106 programs that are smaller than one of these coordinates are classified one by one as builders, drivers, generators, measurement programs, converters or diagnostics, none of which is a complete checker. The audit thereby establishes two points that the package reports had left open: the triple 256/278/516 that `smaller_envelope` and `frontier_review` take as their baseline was not the minimum of the corpus at that revision, because the programs of 276 nodes and of 506 bytes, which lie in subdirectories of their packages, had been overlooked; and no smaller complete checker is hidden among the sources. `progress_independence` extends the audit to the 1416 Python files of three later archives with the same outcome below 506 bytes. Its second write-up repeats the census of the repository at the same revision (5825 paths, 1380 distinct contents, 1194 after normalization, 25 short-listed paths reviewed one by one) with the same minima 256/276/506, and names one more program attaining 256 tokens: the compact token profile of `uniform_progress` for $\mathcal{U}$, with 283 nodes and 593 bytes, one byte shorter than the program of `kernel_refinement` and differing from it only by a default value for the context. `three_field` checks by a live inventory that no path of that revision is missing from the census, and pools the repository with the incoming archives, which gives the minima 251/273/499 of Section 11.11.2. The audit is a census of a finite corpus. It does not verify any checker, it is not a lower bound, and it keeps the compact contract apart from the validated ones.

Two later packages repeat the census. `502_scope_audit` replays the census of `orbit_closure` at a later revision with an identical Research tree (1380 blobs, 5825 paths, the ledger of 106

utilities) and finds 256/276/506, and 253/275/506 with the incoming archive of `smaller_envelope`. `verified_refinements` gives an independent census at the revision that contains `three_field` as an incoming archive: every Python file of the Research tree without a size cutoff, including frozen copies and controls (5935 paths, 1446 blobs, all verified by their hashes and parsed without being executed), and every Python member of the three incoming archives (36 paths, 33 sources). One blob cannot be measured, because removing the only docstring of a class leaves an empty body that the normalization does not repair; with a statement inserted it has 368/414/1853, far above every minimum, and it is recorded as an exclusion. Among the 792 candidates of the Research tree the minima are 251/275/502, attained by three programs; with the incoming archives they are 249/271/495, attained by the three programs of `three_field` named in Section 11.11.2. The four Research sources with a JSON import and fewer than 495 bytes verify manifests, and none of the 112 further files below the envelope in some coordinate reads standard input and prints the acceptance message. The census confirms the envelope that the packages of Section 11.9.11 improve; like the earlier audits it is a census of a finite corpus and not a lower bound.

Three further packages audit the revision that contains nine incoming archives, among them those of Section 11.9.11. `immediate_application` inspects 3991 Python paths and normalizes 1508 distinct sources, with four exclusions at the level of paths recorded and not discarded; it treats a search for programs that read the input and print the acceptance message as a filter only, and excludes generators of checkers, deliberately unsound negative controls and packed programs. `scope_and_frontier` screens the integrated tree of artifacts (2490 Python paths, 1314 blobs, each matched by its hash to the census carried by the incoming material), retrieves and remeasures every source of fewer than 1300 raw bytes (781 paths, 333 blobs) without a discrepancy, finds the minima 251/275/502 there, and classifies 59 small and 19 larger sources below the minima in some coordinate as builders, converters, measurement or reproduction drivers; with the incoming archives the minima are 248/270/487, attained by three programs of `single_binder` listed in Section 11.11, two of which read only the first line. It notes that the program of 492 bytes of `verified_refinements` does not lower the byte minimum below that of 487 bytes. `refinement` remeasures the programs of `three_field` and `constantfree` at its own, earlier revision, with the minima 249/271/495. None of the three finds a smaller complete checker in the corpus.

The packages of Theorem 11.164 audit one later revision, which contains sixteen incoming archives. `fused_blocks_and_checkers` measures 3629 occurrences of Python sources, among them members of nested archives and compressed sources, with 1725 distinct normalized modules and no failure of parsing; it excludes a program of 460 bytes that its package marks as unsound (the program without the parity factor of `shifted_shared_leaf`) and the compressed loader of Section 11.11.1, whose expanded source would have to be charged. `prefix_conditional` matches the 3022 Python entries of the tree and the Python members of the sixteen archives to their hashes in the repository. The third delivery of `checked_presentation` supplies tooling that reproduces its census from a manifest of 3356 files of the repository, each checked by its hash and length before it is parsed and never executed: 3601 occurrences, 1813 distinct normalized modules and no failure of parsing; and a check that a candidate is not, after normalization, the text of a program of the corpus. All three state the minima 240/259/462 at that revision, attained by L_{r}, L_{m} and $T_{+,\mathrm{p}}$, which combine the stream programs and the two encodings of Section 11.9.16; `prefix_conditional` also states 245/263/462 for the programs that read the whole input, from $G_{\mathrm{b}}^{\mathrm{w}}$, $L_{\mathrm{m}}^{\mathrm{w}}$ and $T_{+,\mathrm{p}}$. These censuses compare sources; they do not verify a checker, and they are not lower bounds.

11.10 Results on designs that were examined

The checker packages also record mathematical results obtained while other designs were examined: obstructions to two shortcuts in `certified_zero`, an analysis of eager iteration in the standard model in `prevalidated_elimination`, a criterion on closures in `compact_update`, and smaller observations in `prevalidated_elimination`; further, a counterexample to deleting the entry scan in `502_scope_audit`, an obstruction to admitting the reserved coordinates as spellings of zero in `immediate_application`, an obstruction to substitution at a variable without reflexivity in `retained_premises`, and a bound on finite compositional signatures in `verified_refinements`; two obstructions recorded with the newer protocols, to majors taken only from the context (`export_specialization`, in its review notes) and to restricting both the target of a clause use and the witness of an introduction (`uniform_terminals`); a proof of the rigidity of standard numerals without induction in `scope_refinement`; and a rigidity theorem for relations compatible with unary term functions in `ast261`. They are independent of the protocols above.

11.10.1 Projection boxes lose reflexivity

An experimental checker of `certified_zero` differs from the base protocol in its opener: the root depth is 1, the child depth of an array of arity r is $d+((r-2) \bmod 3)$, an exposed scalar is replaced by the binary array $[t', 0]$ formed from the validated image t', and S is represented by an array of arity 4. The binary array was intended to read as the identity at a term position and as the test $t = 0$ at a formula position, which removes a branch that depends on the sort. No projection equation is supplied by the computation rules.

Proposition 11.166. *Define V on raw trees by: $V([L, R])$ holds if R is numerically 0 or L is an array of arity 3; $V(a)$ holds for every a that is not a binary array. If the experimental checker accepts a goal a, then $V(a)$ holds. In particular it accepts no certificate for the equation $[\sigma, \sigma]$ with $\sigma = [0, 0, 0, 0]$, the encoding of $S0 = S0$, nor for $[p, p]$ with $p = [\tau, 0]$ a projection box that is not numerically 0.*

Proof. Every opening at a depth $d \geq 1$ preserves V. A retained scalar remains a scalar and an exposed one becomes a binary array $[t', 0]$, for which V holds. An array keeps its arity. For a binary array, the right child is numerically 0 after the opening exactly if it was before: the scalar 0 is retained because $k \leq 0 < d$, a retained nonzero scalar stays nonzero, and a replaced scalar becomes an array; the left child is a ternary array after the opening exactly if it was before, since a replaced scalar becomes a binary array. Child depths are at least 1 again.

Now let every member of the context satisfy V; then every goal of a returning call satisfies V, by induction on the run. A goal closed by a hypothesis is the opening of a tree with V. For a packet with a major, the major call is a smaller run; a replacement changes the goal from one opening of a template to another, and both satisfy V or neither does, by the first part; the exported hypothesis is a threefold repetition, of arity divisible by 3, or a scalar, and satisfies V. A supplied computation equation has a ternary array on the left and satisfies V. An introduction requires a unary goal, for which V holds whatever the premise is. The final openings preserve V. For the two equations of the statement V fails: the right side is not numerically 0, and the left side has arity 4, or 2. □

This is an obstruction to completeness, not a claim that the experiment accepts a false formula: the inserted boxes have no equations, so the intended reading as the identity is not available inside the calculus. It does not exclude a different interpretation into another fragment of raw trees, which would need its own compilation and round trips.

11.10.2 Eager initialization has a two-element model

A second shortcut applies the update already at the counter 0, so that both computation equations use the same opening:

$$F(0, a; v.h) = h[a/v], \qquad F(Sn, a; v.h) = h[F(n, a; v.h)/v]. \tag{11.2}$$

Let the terms be x, 0, St and $F(n, a; v.h)$, with v bound in h only, and let T_{e} be intuitionistic first-order logic with equality together with (11.2), coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$, separation $0 \neq S0$, and induction for literal equations with parameters. Successor injectivity is not an axiom of T_{e}.

Theorem 11.167 (Finite model)**.** *Let $M = \{0, 1\}$ with $0 < 1$, interpret 0 by 0, S by the constant function 1, and the eager iterator by*

$$[\![F(n, a; v.h)]\!]_\rho := [\![h]\!]_{\rho[v \mapsto [\![a]\!]_\rho]}.$$

Then M is a model of T_{e}, of $\forall x\,(0 \neq Sx)$, and of induction for every first-order formula.

Proof. Every term is monotone in all its free variables. This is an induction on terms: variables are projections, 0 and S are constant, and for $F(n, a; v.h)$, if $A(\rho)$ and $H(b, \rho)$ denote the values of a and of h with $v \mapsto b$, then $\rho \leq \sigma$ gives $H(A(\rho), \rho) \leq H(A(\sigma), \sigma)$ by the induction hypotheses for a and h. The interpretation also satisfies the substitution lemma, by induction on terms with the bound variable renamed.

Every monotone map $H\colon M \to M$ is idempotent. The monotone maps are the constant 0, the identity and the constant 1.

Fix values of the parameters and let H be the resulting interpretation of $v \mapsto h$; it is monotone, hence idempotent. With α the value of the initializer, both sides of the first equation of (11.2) have the value $H(\alpha)$; the left side of the second has the value $H(\alpha)$, since the counter is ignored, and the right side $H(H(\alpha))$. The substitution lemma makes this a verification of the schemas and not only of closed instances. Coverage holds because the elements are 0 and $S0 = 1$, and $Sx = 1 \neq 0$. For induction, if $A(0)$ and $\forall x\,(A(x) \to A(Sx))$ hold, the step at $x = 0$ gives $A(1)$, and these are all elements. A classical structure is a one-world Kripke model, so all intuitionistic rules are valid in it. □

The native presentation with Q and D over the eager terms is covered as well: under the ordinary reading of Section 1.2 the rules HYP, TR, QE and DE are valid in M; for DI-case the retained base gives the instance $y = 0$ and the premise the instance $y = 1 = S0$; for progress the base and the step give all instances by the induction just verified.

Corollary 11.168. *There is no interpretation of* iPRA *in T_{e} whose domain is a definable subset of a finite power of the domain modulo a definable equivalence, with or without parameters. In particular T_{e} is not bi-interpretable with* iPRA*.*

Proof. Such an interpretation would turn M into a model of iPRA with at most 2^d elements for some d. But iPRA proves that the numerals are pairwise distinct, by successor injectivity and separation, and has no finite model. □

The result concerns the equations (11.2) with an independent initializer. It does not apply to an update that receives the counter, to the initialized iterator of Section 1.1, or to a successor-seeded iterator, and adding successor injectivity excludes the model M without providing an interpretation.

11.10.3 Positive iteration in the standard model

`prevalidated_elimination` determines the functions that the eager terms denote in the standard natural numbers, where $F(n, a; u.h)$ is the $(n+1)$-fold iterate of $b \mapsto h(b, \vec{x})$ applied to a. For $f\colon \mathbb{N}^r \to \mathbb{N}$ a coordinate j is *essential* if changing it can change the value with the other coordinates fixed, and f is *strictly increasing in* j if it is so for every choice of the other coordinates.

Theorem 11.169 (Constant, projection or growth)**.** *Every eager term $t(x_1, \ldots, x_r)$ denotes a function of exactly one of the following kinds:*

(i) a constant;

(ii) a projection x_j;

(iii) a function with a nonempty set U of essential coordinates that is strictly increasing in each coordinate of U and satisfies $t(\vec{x}) \geq x_j + 1$ for all $j \in U$.

In particular every term is nondecreasing in each coordinate. The kind and the set of essential coordinates are computed by recursion on the term, without evaluating it.

Proof. The three kinds are disjoint: a projection violates the bound of (iii) at its coordinate, and a constant has no essential coordinate. The classification is proved by induction on terms, together with monotonicity. The term 0 is constant and a variable is a projection. S maps a constant to a constant and a function of kind (ii) or (iii) to one of kind (iii) with the same essential coordinates, the bound $\geq x_j$ becoming $\geq x_j + 1$.

Consider $F(n, a; u.h)$ and apply the induction hypothesis to n, to a, and to h with its state u as a further coordinate. If h is constant, the result is that constant, since at least one update is performed. If h is the projection to u, the result is a. If h is a projection to a parameter, the result is that projection. If h is of kind (iii) and does not depend on u, the result is h.

There remains the case that h is of kind (iii) and depends on u, so that $h(b, \vec{x}) \geq b + 1$ and h is strictly increasing in b. Let U_n and U_a be the essential coordinates of n and a, let V be the essential parameter coordinates of h, and let $U = U_n \cup U_a \cup V$. The result does not depend on coordinates outside U; if U is empty it is constant. Let U be nonempty. The result is monotone: for $\rho \leq \sigma$ the initializer and the counter are ordered and the update under ρ is pointwise at most the update under σ, so the states after a common number of updates are ordered, and further updates under σ only increase the state. It is strictly increasing in each $j \in U$: an increase of an essential coordinate of a strictly increases the initial state, and the strict monotonicity of the update preserves a strict difference; an increase of an essential parameter coordinate of h strictly increases the state at the first update, which exists even for the counter 0; an increase of an essential coordinate of n adds updates, each of which raises the state by at least 1; and if a coordinate has several of these roles, its other effects are nondecreasing. Hence U is exactly the set of essential coordinates. Finally, with N and A the values of n and a, the result is at least $A + N + 1$; for $j \in U_a$ this is at least $x_j + 1$ since $A \geq x_j$, for $j \in U_n$ likewise since $N \geq x_j$, and for $j \in V$ the last application of h is at least $x_j + 1$.

The case distinction is a recursive procedure: it uses the kinds and essential sets of the parts and one test for emptiness of a union. □

Corollary 11.170. *A nonconstant eager term is unbounded in each essential coordinate. No eager term denotes a nonconstant function with finite range; in particular none denotes a zero test with values in $\{0, 1\}$ or the predecessor.*

Proof. A projection is unbounded, and in kind (iii) the bound $t \geq x_j + 1$ gives unboundedness. The predecessor is nonconstant and not strictly increasing from 0 to 1. □

Theorem 11.171 (No unit on a tuple domain with actual equality)**.** *Let $\varnothing \neq D \subseteq \mathbb{N}^d$, let $e \in D$, and let $f\colon D \times D \to D$ be given by a d-tuple of eager terms, possibly with numerical parameters, such that $f(e,z) = z = f(z,e)$ for all $z \in D$, with equality of tuples. Then D has exactly one element. No definability of D is assumed.*

Proof. Parameters are replaced by numerals, so Theorem 11.169 applies to each coordinate function $f_i(x_1, \ldots, x_d, y_1, \ldots, y_d)$. Let J be the set of coordinates that are not constant on D. For $i \in J$ the function f_i has an essential coordinate in each of the two argument tuples, since otherwise $f_i(e,z) = z_i$ or $f_i(z,e) = z_i$ would be constant on D; so f_i is of kind (iii). Moreover some essential coordinate j_i of the left argument lies in J: if all of them were constant on D, then $z \mapsto f_i(z,e)$ would be constant on D. The bound of (iii) and the right unit law give $z_i = f_i(z,e) \geq z_{j_i} + 1$ for all $z \in D$. If J were nonempty, the map $i \mapsto j_i$ on the finite set J would have a cycle, and adding the inequalities around a cycle of length $\ell \geq 1$ would give $z_i \geq z_i + \ell$. So J is empty, every coordinate is constant on D, and D is a singleton. □

Theorem 11.171 is not a statement that eager terms are almost unary. With

$$q(a,b) := F(a,b;u.Su) = a{+}b{+}1, \qquad d(a) := q(a,a) = 2a{+}1, \qquad \pi(a,b) := F(a, Sd(b); u.d(u)),$$

iteration of $u \mapsto 2u{+}1$ gives $\pi(a,b){+}1 = 2^{a+1}(2b{+}3)$ with $2b{+}3$ odd, so π is an injective pairing in the standard model. Its two projections are not eager terms: a nonconstant unary eager term is strictly increasing, while $\pi(1,0) = 11 < 13 = \pi(0,2)$ and $\pi(0,1) = 9 < 11 = \pi(1,0)$ show that neither projection is monotone on the range of π.

Equality modulo a definable relation is a real exception to Theorem 11.171. With $m(a,b) := F(a,0;u.q(u,b)) = (a+1)(b+1)$ and

$$(a,b)\ E\ (c,d) :\Leftrightarrow q(a,d) = q(c,b),$$

the quotient $\mathbb{N}^2/E$ is identified with $\mathbb{Z}$ by $(a,b) \mapsto a - b$, and the tuples of eager terms

$$(a,b) \oplus (c,d) := \big(q(a,c),\, q(b,d)\big), \qquad (a,b) \otimes (c,d) := \big(q(m(a,c),m(b,d)),\, q(m(a,d),m(b,c))\big)$$

induce addition and multiplication of integers, because $(a+1)(c+1) + (b+1)(d+1) - (a+1)(d+1) - (b+1)(c+1) = (a-b)(c-d)$. These are identities of the standard model only. In the model M of Theorem 11.167 one has $q(a,b) = 1$ for all arguments, E identifies all pairs, and the quotient has one element; so the construction does not give an interpretation of iPRA in T_{e}, in accordance with Theorem 11.168.

11.10.4 Conditional and progressive closure of an equation

`compact_update` determines when the two readings of a closure that occur in this chapter agree. Let $E(y)$ be a literal equation with parameters, and write

$$B := E(0), \qquad C(E) := B \to \forall y\, E(y), \qquad P(E) := \forall y\, \big(E(y) \to E(Sy)\big).$$

The following statements are proved in T_{eq}; the only induction used is induction for the equation E.

Lemma 11.172 (Anchored progress)**.** *$P(E) \to C(E)$, and $C(E) \leftrightarrow (B \to P(E))$.*

Proof. From $P(E)$ and B, induction for E gives $\forall y\, E(y)$. From $C(E)$ and B one has $\forall y\, E(y)$, hence $P(E)$. From $B \to P(E)$ and B one has $P(E)$, and the first statement gives $\forall y\, E(y)$. □

Theorem 11.173 (Agreement criterion)**.** $(C(E) \leftrightarrow P(E)) \leftrightarrow (\neg B \to P(E))$.

Proof. Let $C(E) \leftrightarrow P(E)$ and $\neg B$. Then $C(E)$ holds, since its antecedent is refuted, and so $P(E)$. Conversely let $\neg B \to P(E)$. By Theorem 11.172 it remains to derive $P(E)$ from $C(E)$. The equation B is decidable in T_{eq} (Theorem 1.86). If B holds, $C(E)$ gives $\forall y\, E(y)$ and hence $P(E)$; if $\neg B$ holds, the assumption gives $P(E)$. □

Decidability is used for the one equation B and not as excluded middle. In Lean the criterion is proved in T_{eq} for every quantifier-free formula in place of the equation, as `Teq.closure_iff_progress`, with `Teq.closure_of_progress` for the first statement of Theorem 11.172 and the decidability lemma `Teq.qfDec`.

A guarded equation (11.1) has a provable base and satisfies the criterion; this is the equivalence of the two readings used in Theorem 11.6. A provable base is not necessary: for $E(y) \equiv (0 = S0)$ both $C(E)$ and $P(E)$ are provable. The criterion fails for $E(y) \equiv (y = S0)$: the base is refutable, so $C(E)$ is provable, while $P(E)$ would give $SS0 = S0$ from $E(S0)$, so $\neg P(E)$ is provable. This is why an introduction by progress cannot replace the introduction by cases at unguarded equations.

Theorem 11.174 (Criterion below an existential quantifier)**.** *For an equation $E(x, y)$ let C_x and P_x be the two closures for fixed x. Then*

$$(\exists x\, C_x \leftrightarrow \exists x\, P_x) \leftrightarrow (\exists x\, \neg E(x, 0) \to \exists x\, P_x).$$

Proof. By Theorem 11.172, $\exists x\, P_x \to \exists x\, C_x$ always. Let $\exists x\, C_x \to \exists x\, P_x$ and let $\neg E(a, 0)$. Then C_a holds, so $\exists x\, C_x$ and hence $\exists x\, P_x$. Conversely let $\exists x\, \neg E(x, 0) \to \exists x\, P_x$ and let C_a hold. Decide the equation $E(a, 0)$. If it holds, C_a gives $\forall y\, E(a, y)$, hence P_a. If it fails, a witnesses the antecedent, and $\exists x\, P_x$ follows, possibly with another witness. □

The Lean form is `Teq.ex_closure_iff_progress`. Agreement for every x implies agreement of the existential statements, but not conversely: for $E(x, y) \equiv (\mathrm{sel}(x, S0, y) = S0)$ the equation holds for all y at $x = 0$, so both existential closures are provable with witness 0, while at $x = S0$ it is $y = S0$, where the two readings differ.

Proposition 11.175 (Base-retaining normal form)**.** *For $A(y) := B \wedge E(y)$ one has $C(A) \leftrightarrow P(A) \leftrightarrow C(E)$, without a decision of B. If $E(y)$ is $s(y) = t(y)$ and $\langle \cdot, \cdot \rangle$ is a provably injective pairing, $A(y)$ is equivalent to the literal equation $\langle s(0), s(y) \rangle = \langle t(0), t(y) \rangle$.*

Proof. $A(0)$ is equivalent to B, so $C(A)$ is equivalent to $B \to \forall y\, (B \wedge E(y))$, that is, to $C(E)$. From $C(E)$ and $A(y)$, the retained B gives $E(Sy)$ and hence $A(Sy)$; this is $P(A)$. From $P(A)$ and B, the formulas $A(y)$ and $E(y)$ are equivalent, so $P(A)$ is the progress step for E, and induction for E from the base B gives $\forall y\, E(y)$. The last statement is the injectivity of the pairing. □

The normal form is not a measured kernel: using it in a primitive presentation requires pairing and its injectivity before the completeness theorem that is being established, and the rule that retains both $E(0)$ and $E(z)$ is the older anchored introduction. Likewise the condition $\neg E(0) \to P(E)$ of Theorem 11.173 is a statement that needs a proof; a checker can test only a syntactic sufficient condition such as the guard.

11.10.5 Further rejected designs

`prevalidated_elimination` records three further attempts.

The remaining length of the certificate as the floor. Consuming a packet would then allocate the fresh name before the witness of an introduction is validated, and the witness could

contain the name. The freshness of the witness is necessary: for $E(x,y) \equiv (\mathrm{sel}(y,0,x) = y)$ the instance at $y = 0$ is $0 = 0$ and the instance at Sz is $x = Sz$, so $D_{x,y}E$ is false under the ordinary reading, because no x equals both $S0$ and $SS0$; yet an introduction by cases with the illegal witness $t = Sz$ has the premise $\mathrm{sel}(Sz, 0, Sz) = Sz$, which is a computation.

Zero computation by definition. Identifying $I(0,a;\, v.\, h)$ with a is legitimate, but in the encoding of Section 11.2 the initializer lies below a dummy slot. In $I(n,a;\, u.\, I(0,u;\, v.\, 0))$ the occurrence of u in the inner initializer has the scalar 5, and removing the inner iterator must lower it to 3; returning the raw initializer does not preserve scope. A normalizer that lowers indices, validates the result and keeps transport and hypothesis lookup correct modulo the conversion was larger than the dispatch it would remove.

Synthesis of the conclusion. A checker can compute its conclusion from the certificate instead of checking a goal at each call; hypotheses are then referred to by index and majors and endpoints are not repeated. Two experimental programs of this kind are larger than the compact checkers (Section 11.11): the inference needs a dispatch on hypotheses, a comparison of minor instances and of the inferred premise of an introduction with the required successor instance, and a test that an inferred conclusion contains no eigenparameter. Shorter certificates and a shorter checker are separate resources.

11.10.6 The entry scan is necessary

This subsection follows `502_scope_audit`. Step (1) of every protocol opens the goal with the missing image before the membership test. Besides normalizing the goal, the opening verifies that every numerical occurrence of the goal is in scope (Theorem 11.11(b)). Testing the membership of a itself instead of a° saves three tokens in the global loop with inferred majors of `inferred_major` and `smaller_envelope`; the modified program has 250/277/596.

Proposition 11.176 (Deleting the entry scan)**.** *Let*

$$F := D_{x,y}\big(I(y,0;\, u.\, x) = I(y,0;\, u.\, Sx)\big),$$

a closed guarded formula with the canonical encoding `[[[5,0,7],[5,0,[11]]]]`*. The global loop with inferred majors whose step (1) tests the membership of a instead of a° accepts the stack of seventeen packets displayed below with the goal F. The unchanged program rejects it, and so does the program of* 502 *bytes. The formula F is refuted in* iPRA.

Proof. The ordinary reading of F is $\exists x\,\big(I(0,0;\, u.\, x) = I(0,0;\, u.\, Sx) \to \forall y\, I(y,0;\, u.\, x) = I(y,0;\, u.\, Sx)\big)$. The antecedent is $0 = 0$, the instance $y = S0$ is $x = Sx$, and conversely $x = Sx$ gives every instance by congruence; so iPRA proves $F \leftrightarrow \exists x\,(x = Sx)$, and $\neg F$ follows by quantifier-free induction from $0 \neq S0$ and the injectivity of the successor.

Write $C(y,a,b) := I(y,a;\, u.\, b)$ and $G_y(p,q)$ for $C(y,0,p) = C(y,0,q)$. The packet read first has the major $M_b := D_{x,y}G_y(x, Sb)$, encoded with the name $b = -1000000$, which is far outside the scope at floor 0. The unchanged program opens M_b in step (1) of the major call, finds the scalar b out of range at a term position without an image, and rejects; so does the program of 502 bytes, at the same opening with floor 0, before its local floors differ from the global floor. The modified program does not open the major and proceeds as follows.

1. A computation packet adds $I(0,S0;\, u.\, 0) = S0$ and allocates the parameter $z_1 = -1$. The goal of the major call is opened at the image Sz_1; the occurrence of b is out of scope and is replaced as a hole, so the goal becomes $D_{x,y}G_y(x, SSz_1)$.

2. An introduction at the witness SSz_1 leaves a reflexive equation, with the step parameter -2; three packets prove it by computation and transport. When the major call returns, the floor is -5.

3. The caller still holds the original raw M_b. It exports the closure and opens it at the new witness $w = -6$; both the witness slot and the still out-of-scope b are replaced by w, and the hypothesis added is $Q_y G_y(w, Sw)$, which is not the closure of the formula proved in the major call.

4. The goal F is introduced at w, with the step parameter -7; the retained closure, specialized at the successor, reduces the remaining goal to its zero row $C(0, 0, w) = C(0, 0, Sw)$, and nine packets prove this computation equation, using $P := I(SS0, S0; u.\, 0)$ with $P = 0$ as an initializer, so that every computation packet has an array as its seed.

The goal checked inside the major call and the raw major exported by its caller cease to agree between the first and the third stage; the decreasing global floor does not prevent this, because an old raw array can still misdescribe which variable it denotes. The stated outcomes are those of the three programs on the displayed stack, in ordinary and in optimized execution. □

The stack, in the order of the array (packets are consumed from the end):

```
[[0,0,[[[0]],[0],0],-6],[0,[0],[0],[0,[[[0]],[0],0],[-6]]],
 [0,[0],[0],[0,[[[0]],[0],0],[-6]]],
 [[[[[0]],[0],[0,[[[0]],[0],0],[-6]]],[0,[[[0]],[0],0],[-6]]],
   [[[0]],[0],[0,[[[0]],[0],0],[-6]]],[0,[[[0]],[0],0],[-6]],
   [3,[0,[[[0]],[0],0],[-6]]]],
 [0,0,[[[0]],[0],0],[-6]],
 [[[0,[[[0]],[0],0],[-6]],[[[0]],[0],0]],[0,[[[0]],[0],0],[-6]],
   [[[0]],[0],0],[3,[0,[[[0]],[0],0],[-6]]]],
 [[[[[0]],[0],0],[0,[[[0]],[0],0],[-6]]],[[[0]],[0],0],
   [0,[[[0]],[0],0],[-6]],[[0,[[[0]],[0],0],-6],3]],
 [0,[0],[0],0],
 [[[[[0]],[0],0],0],[[[0]],[0],0],0,[[0,5,-6],[0,5,[-6]]]],
 [[[[5,0,-6],[5,0,[-6]]],[[5,0,-6],[5,0,[-6]]],[[5,0,-6],[5,0,[-6]]]],
   0,[-7],[[5,0,-6],[5,0,[-6]]]],
 [0,0,0,-6],[0,[0],[0],[[-2],0,[[-1]]]],[0,[0],[0],[[-2],0,[[-1]]]],
 [[[[[0]],[0],[[-2],0,[[-1]]]],[[-2],0,[[-1]]]],[[[0]],[0],[[-2],0,[[-1]]]],
   [[-2],0,[[-1]]],[3,[[-2],0,[[-1]]]]],
 [0,0,0,[[-1]]],[0,0,[0],0],
 [[[[5,0,7],[5,0,[-1000000]]]],0,0,0],
 [[[5,0,7],[5,0,[11]]]]]
```

The modified program agrees with its control on all 121 fixtures of `smaller_envelope` that apply to it; the stack was constructed for the missing invariant. The proposition refutes one deletion. It is not a lower bound of 253 tokens, and it does not exclude a program with another proved scope discipline. `refinement` cites this stack against the same deletion (the membership of a in place of a° in the entry test), whose accepted goal F is equivalent to $\exists x\,(x = Sx)$, and keeps the entry scan in all its programs, including the two-field protocol; its compact and strict programs reject the stack.

11.10.7 Reserved coordinates are not spellings of zero

This subsection follows `immediate_application`. The goal validators of the native programs (Theorem 11.65 and its successors) admit at a term position only the scalar 0 and odd scalars of at least 3, although the sorted reading (Theorem 11.8) reads the scalars 1 and 2 as 0 as well. A shorter validator would admit every scalar in scope at a term position and read 1 and 2 as further spellings of zero. The following proposition shows that this loses completeness. Throughout, the native protocol is the kernel of the program of Theorem 11.102 and of its immediate application (Theorem 11.100(b)) after the first opening, which is the raw proof protocol.

Proposition 11.177 (Reserved coordinates as constants)**.** *Let c_1, c_2 be two new constants, and let the* parametric reading *be the sorted reading of Theorem 11.8 except that at every term position a scalar* 1 *reads as c_1 and a scalar* 2 *as c_2; the scalar* 0 *and every dummy slot still read as* 0*. Then the soundness statement of the native protocol (Theorem 11.96(a) in its native form) holds for the parametric reading, for $\mathcal{A}^-$ over the language with the constants c_1, c_2. Consequently, if the native protocol accepts a stack with goal a, then the parametric reading of a is true in the standard model for every assignment of numbers to c_1, c_2 and the parameters.*

Proof. The soundness proof is an induction on a finite accepting run, and each of its steps is checked for the parametric reading.

Every term position has depth at least 3, so the scalars 0, 1 and 2 pass the range test $k \le a < d$ at every term position and are never the hole; they are retained by every opening at every depth, and an image validated at depth 3 and inserted below further slots keeps them as the same constants. So Theorems 11.11, 11.12 and 11.14 hold for the parametric reading with the same proofs: slot pairs and parameters are treated as before, and traversal inside annotations has no effect on a reading. The parametric reading of a raw tree is still a function of the tree, so membership in the context and the comparisons of the elimination test, which compare raw trees, have the same consequences for readings. Transport along an equation is valid for formulas with the two constants, and the specialization of a closure inserts the literal scalar 0, which reads as 0.

In a computation packet $[m, s, u, t]$, a false-valued counter field s gives the counter 0, and a true-valued one the successor array $[s]$; with the parametric reading the generated equation is again an instance of I0 or IS, now with c_1, c_2 allowed in its terms. When the update uses its state, the recursive image $[s, u, t]$ is validated at its root term depth, where 1 and 2 are retained as the same constants, and the remaining argument about the scope of the source is unchanged. The substitution of 0 for fresh parameters in Theorem 11.2 does not touch the constants, so Theorems 11.2 and 11.18 apply as before; if the update does not use its state, its value does not depend on the state, again with c_1, c_2 fixed. The readings of D and Q are unchanged, their openings retain the scalars 1 and 2, and the new parameter is fresh for the context and the goal. The introduction by cases uses the coverage of the numbers; the introduction by progress is still restricted to literal equations, and induction for a literal equation with two further constants is valid, the constants being fixed parameters of the induction. The neutral resources and the annotations that read $0 = 0$ or 0 do not involve c_1, c_2. These are all the cases of the induction. The last statement follows by reading the derivation in the standard model with arbitrary values of the constants and parameters. □

Corollary 11.178 (The relaxed validator is incomplete)**.** *No stack is accepted by the native protocol with the goal `[1,0]`, nor with the goal `[2,0]`. Hence the checker whose goal validator admits the scalars* 1 *and* 2 *at term positions as spellings of zero, and whose kernel is unchanged, does not accept every public theorem it admits: the goal `[1,0]` reads as* $0 = 0$ *in the sorted reading and is rejected.*

Proof. In the parametric reading `[1,0]` reads as $c_1 = 0$, which is false when c_1 is assigned the value 1; by Theorem 11.177 no stack is accepted. The same argument with c_2 applies to `[2,0]`. After the relaxed validation the run is a run of the unchanged kernel. □

The corollary does not contradict the use of such scalars as auxiliary aliases inside certificates: the contracts of this chapter give soundness for the wider raw syntax and completeness for canonical public formulas, and soundness for an alias does not provide a derivation that identifies every raw spelling with its representative. A presentation could add an identification or a normalization of spellings, but its cost and its correctness would have to be counted. The

corollary excludes this one deletion from the validator; it is not a lower bound for presentations of iPRA. The proof is a written extension of the soundness proof and is not mechanized.

11.10.8 Variable-only substitution without reflexivity

`retained_premises` examines a variant of the raw native calculus in which transport is replaced by substitution at a variable. The terms and formulas are those of Section 1.1, substitution is the capture-avoiding substitution of raw syntax, and neither computation equation is a definitional reduction. The rules are HYP, the axioms $I(0, a; u.\, h) = a$ and $I(Sn, a; u.\, h) = h[I(n, a; u.\, h)/u]$, QE, DE, the introductions by cases and by progress (or their combination selected by the witness), and, in place of TR,

$$\frac{\Gamma \vdash x = t \qquad \Gamma \vdash u = v}{\Gamma \vdash v = u[t/x]} \qquad \text{(VS)},$$

where x is a free numerical variable. There is no reflexivity rule and no other equality rule.

Call a term *iterator-headed* if its outermost constructor is I, and put $\nu(s = t) = 1$ if s is iterator-headed and 0 otherwise, and $\nu(Q_y B) = \nu(D_{x,y} B) = 1$. A sequent $\Gamma \vdash A$ is *ν-valid* if $\nu(A) = 1$ whenever $\nu(C) = 1$ for every $C \in \Gamma$.

Lemma 11.179. *For every raw formula $B(y)$ and term t, $\nu(B(0)) = 1$ implies $\nu(B(t)) = 1$.*

Proof. Substitution does not change the outermost constructor of a formula, so the claim is clear if B is compound. Let B be $s(y) = r(y)$. If s is iterator-headed, so is $s(t)$. If s is 0, a successor, a variable other than y, or y itself, then $s(0)$ is 0, a successor or a variable, and $\nu(B(0)) = 0$. □

Theorem 11.180 (Loss of reflexivity)**.** *Every sequent derivable in the calculus with* VS *is ν-valid. In particular $0 = 0$ is not derivable, while the native calculus derives it.*

Proof. By induction on the derivation, assuming that every member of the context has the value 1. HYP is immediate. Both computation axioms have an iterator-headed left side. For QE, the induction hypothesis gives $\nu(B(0)) = 1$, and Theorem 11.179 gives $\nu(B(t)) = 1$. For DE, the assumption $Q_y B(b, y)$ added to the context has the value 1, so the induction hypothesis for the minor premise gives $\nu(A) = 1$. The introductions conclude formulas of the form $D_{x,y} B$. For VS, the induction hypothesis would give $\nu(x = t) = 1$, which is false because x is a variable; so the members of the context do not all have the value 1, and the conclusion is ν-valid vacuously. Since $\nu(0 = 0) = 0$, the equation $0 = 0$ is not derivable from the empty context.

In the native calculus put $J_a := I(0, a; u.\, u)$. The axiom gives $J_a = a$, and TR with the premises $J_a = a$ and $h(J_a) = a$ for $h(v) := v$, both instances of that axiom, gives $a = h(a)$, that is, $a = a$. □

The theorem concerns the raw calculus without reflexivity, rule by rule. Adding reflexivity refutes the invariant, and a definitional zero computation changes the meaning of the axiom and of the head test. The theorem also does not make substitution into derivations admissible for the calculus with VS, since such a substitution can turn the variable source of the rule into a compound term. Structural weakening and cut preserve the invariant.

11.10.9 Finite compositional signatures

`verified_refinements` records an elementary argument against replacing the binder of the iterator by a fixed finite collection of functions combined by composition alone.

Theorem 11.181 (Finite compositional bases)**.** *For no finite set Σ of primitive recursive functions, constants included, do the terms built from one variable and Σ by composition represent every unary primitive recursive function.*

Proof. Put

$$g(n) := n + 1 + \max\bigl(\{0\} \cup \{f(a_1, \ldots, a_r) : f \in \Sigma,\ a_1, \ldots, a_r \le n\}\bigr).$$

The maximum is over a finite set, g is primitive recursive and nondecreasing, and $g(n) > n$; the iterate $g^d(n)$ is primitive recursive in (d, n). Give the variable depth 0 and an application, constants included, depth one more than the largest depth of its arguments. By induction on terms, a term of depth d has value at most $g^d(n)$ at every input bounded by n: the arguments of an application of depth d have values at most $g^{d-1}(n)$, since $g^e(n) \le g^{d-1}(n)$ for $e \le d-1$ because $g(x) > x$, and an application of $f \in \Sigma$ to values at most m is below $g(m)$ by the definition of g, here with $m = g^{d-1}(n)$. The function $h(n) := g^n(n) + 1$ is primitive recursive, and for a term t of depth d and every $n \ge d$, $t(n) \le g^d(n) \le g^n(n) < h(n)$. So no term represents h. □

The theorem excludes only composition over a finite signature. It does not concern relational languages, definitions by formulas, other domains, other binding constructs or other proof systems; the single-constructor calculi of other chapters bind a variable and are not affected.

11.10.10 Majors taken only from the context

The review notes of `export_specialization` examine the deletion of the recursive proof of a major premise. Let M be $\mathcal{A}$ (Theorem 11.1) in which the major of TR, QE and DE must be a member of the context, and in which any computation instance may be added to the context; minor premises and continuations are derived in M.

Proposition 11.182 (Membership majors lose induction)**.** *For a parameter v, $\mathcal{A}$ derives $\vdash I(v, 0;\ u.\, u) = 0$, and M does not.*

Proof. In $\mathcal{A}$, with $E(y) \equiv (I(y, 0;\ u.\, u) = 0)$: DI-progress proves $D_{x,y}E(y)$, since $E(z)$ and the computation $I(Sz, 0;\ u.\, u) = I(z, 0;\ u.\, u)$ give $E(Sz)$ by TR; DE at a fresh b gives $Q_y E(y)$, and QE with the computation $E(0)$ gives $E(v)$. In M, consider a derivation of an equation from a context of equations. Its last rule is not an introduction, whose conclusion is a D formula, and not QE or DE, whose majors would be Q or D formulas in the context. It is a hypothesis, a computation leaf, the addition of a computation equation, which keeps the context equational, or TR with an equation of the context as major and an equation as minor premise, since a template whose instance is an equation is an equation. By induction such a derivation uses only computation instances and transport along them, so its conclusion holds in every model of the computation equations as an equational theory. In the algebra of terms with the parameter v modulo the two computation equations, which by the confluence and termination of computation (the full normalization of the chapter on computation) is represented by normal forms, $I(v, 0;\ u.\, u)$ and 0 are distinct normal forms; so the equation is not derivable. □

The proposition separates open native consequences; it does not exclude a redesign whose closed public theorems are complete by other introductions. The repair that adds induction for literal equations directly, from $\Gamma \vdash E(0)$ and $\Gamma, E(z) \vdash E(Sz)$ to $\Gamma \vdash E(t)$, restores a recursively verified premise. The retained majors of Section 11.9.3 keep the proof of the minor premise and are not affected.

11.10.11 Newest-parameter use together with a restricted witness

`uniform_terminals` checks the finite-range obstruction of the chapter on witnesses against the calculus with deterministic EXPORT of `export_specialization`, in which the target of a clause use is the newest parameter. Clauses $[A; y.B(y)]$, formulas of equality and D, and terms raw or in computation normal form are as in that chapter. Let W have: REFL, $t = t$ for every term; USE, from a clause $[A; y.B(y)]$ of the context and A infer $B(q)$, with q the term 0, the term $S0$ or a parameter of the scope; INTRO, from $B(q, \eta) \vdash B(q, Sz)$ infer $D_{x,y}B$, with q the term 0 or a parameter of the scope, z fresh and η either 0 or z; and EXPORT, which opens $D_{x,y}B$ at a fresh b with the clause $[B(b, 0); y.B(b, y)]$ and, from a derivation of F, exports $F[r/b]$, where $r = t$ if the normalized rows $B(b, 0)$ and $B(b, S0)$ are literally $0 = 0$ and $b = t$, and $r = b$ otherwise (t may contain b; the substitution is performed once).

Proposition 11.183 (Joint restriction)**.** W *does not derive* $N_1 := D_{x,y}(I(y, 0;\, u.\, x) = I(y, 0;\, u.\, S0))$ *from the empty context or from the reflexivity clause alone, while the ordinary reading of* N_1*, equivalent to* $\exists x\,(x = S0)$*, is a theorem of* iPRA*.*

Proof. Evaluate terms in the natural numbers and read formulas, for every environment ρ, by $[s = t]_\rho$ if the values agree and $[D_{x,y}B]_\rho$ if $[B]_{\rho[x:=0,y:=0]}$ implies $[B]_{\rho[x:=0,y:=1]}$; read a clause $[A; y.B]$ as "$[A]_\rho$ implies $[B(0)]_\rho$ and $[B(1)]_\rho$" and $[\,; y.B]$ as "$[B(0)]_\rho$ and $[B(1)]_\rho$", and use the environment ρ_Π that assigns 0 to every parameter of the scope. Normalization preserves values, and $[F[r/b]]_\rho = [F]_{\rho[b:=\langle r\rangle_\rho]}$ for every r, including one that contains b, since the bound variables of D receive the fixed values $0, 0$ or $0, 1$ on both sides. Every rule preserves truth of the conclusion under true contexts. REFL is true. In USE the target has the value 0 or 1, and the clause supplies both rows. In INTRO the witness and η have the value 0 and Sz the value 1 when $z := 0$; so the branch gives exactly the reading of $D_{x,y}B$. In EXPORT, with $b := 0$ the clause has the reading of the major; if $r = b$ the formula is unchanged, and otherwise the major gives $0 = \langle r\rangle_{\rho[b:=0]}$, so $F[r/b]$ has the truth value of F at $b := 0$. At $x = 0$ the matrix of N_1 is $0 = 0$ for $y = 0$ and $0 = 1$ for $y = 1$, so $[N_1]$ is false. In the ordinary reading $\exists x\,(B(x, 0) \to \forall y\, B(x, y))$ the instance $y = 1$ gives $x = S0$ and the witness $S0$ satisfies every row. □

The argument also admits the computation axioms, symmetric and transitive equality, transport through any formula and the discharge of an unused fresh parameter; it excludes arbitrary witnesses, arbitrary targets and an independent naming axiom. It shows that the completeness of newest-parameter use, which keeps arbitrary witnesses, and that of a newest-parameter witness, which keeps a rule naming arbitrary terms, cannot simply be combined. The package presents it as an application of the existing finite-range invariant, not as a new obstruction.

11.10.12 Standard-numeral rigidity without induction

This subsection follows the appendix of `scope_refinement`, which sharpens the dependencies of Theorem 8.64. Let Q_F be ordinary intuitionistic first-order logic with equality over the terms of Theorem 8.56, with the computation equations (8.21), the axioms (8.11), (8.12) and (8.13), and coverage $n = 0 \vee \exists m\, n = Sm$, universally closed. Q_F has no induction and no injectivity of the successor. Write $\bar{k}$ for the numeral S^k0.

Theorem 11.184 (Rigidity from coverage)**.** *For every fixed integer* $k \geq 1$*,* Q_F *proves* $\forall x\,(Sx = \bar{k} \leftrightarrow x = \overline{k-1})$*. The derivations, one for each* k*, contain no instance of induction.*

Proof. The following are derivable without induction. The first two statements of Theorem 8.57, $Sn \neq 0$ and the truth tables of $\neg_B$, $\wedge_B$ and $\vee_B$ on $0, 1$, are proved there from the

computation equations, the two axioms for ε and separation alone. By Theorem 8.61 distinct numerals are provably distinct. Let h_k, $A_k(n) := F(n; v.h_k)$ and $q_0 := \overline{k+1}$, $q_j := \overline{k-j}$ $(1 \le j \le k)$ be as in the proof of Theorem 8.64. The table of h_k given there, the fact that its value is one of $\bar{0}, \ldots, \bar{k}$, and hence $h_k(v) \neq q_0$ and

$$h_k(v) = q_j \leftrightarrow v = q_{j-1} \qquad (1 \le j \le k),$$

are obtained by finitely many case distinctions on the tests $\varepsilon(v, \bar{i})$ and by the distinctness of numerals; $A_k(0) = q_0$ and $A_k(\bar{j}) = q_j$ for $0 \le j \le k$ follow by $j+1$ computation steps.

First, $A_k(n) = q_0 \leftrightarrow n = 0$. By coverage, $n = 0$ or $n = Sm$. If $n = 0$, both sides hold. If $n = Sm$, then $A_k(n) = h_k(A_k(m)) \neq q_0$ and $n \neq 0$, so both sides are refutable and the equivalence holds.

The proofs of the theorem are constructed by recursion on k outside the theory. Assume the statement for every j with $1 \le j < k$. By recursion on $j = 0, \ldots, k-1$ outside the theory,

$$A_k(n) = q_j \leftrightarrow n = \bar{j}.$$

The case $j = 0$ is the preceding paragraph. For $1 \le j < k$ use coverage. At $n = 0$ both sides are refutable, since $A_k(0) = q_0 \neq q_j$ and $0 \neq \bar{j}$. At $n = Sm$ the left side is $h_k(A_k(m)) = q_j$, equivalent to $A_k(m) = q_{j-1}$, hence by the case $j-1$ to $m = \overline{j-1}$; the right side $Sm = \bar{j}$ is equivalent to the same equation by the statement for j.

Now let $Sx = \bar{k}$. Computation and congruence give $h_k(A_k(x)) = A_k(Sx) = A_k(\bar{k}) = q_k$, so $A_k(x) = q_{k-1}$, and the case $j = k-1$ gives $x = \overline{k-1}$; for $k = 1$ this is the first equivalence. The converse holds by congruence. □

In the proof of Theorem 8.64 induction enters twice: in the equivalence $A_k(n) = q_0 \leftrightarrow n = 0$, proved there by quantifier-free induction, and through coverage, which T_F^- obtains by induction in Theorem 8.63. The theorem replaces both by coverage as an axiom; so over T_F^- the rigidity of standard numerals uses induction only to derive coverage. It is a family of derivations, one for each k; it gives no countdown term indexed by a variable, does not derive $Sx = Sy \to x = y$, and constructs no model of Q_F in which the successor is not injective. The package checks the tables of h_k and the chain equivalences for $k \le 39$ by finite computation, as evidence only.

11.10.13 Relations compatible with unary term functions

This subsection follows the fragment `unary_relation_rigidity.tex` of `ast261`. It concerns the obstruction Theorem 6.67, in which an equation is interpreted by a reflexive relation that every term function preserves, and the question left there whether unary congruence suffices on full normal forms. Let M be a set with two elements $0, 1$ and $\mathcal{P}$ a set of maps $M \to M$; in the application $\mathcal{P}$ consists of the unary term functions, one variable varying and the others fixed at elements of M. A binary relation R on M is *preserved by* $\mathcal{P}$ if $a\ R\ b$ implies $p(a)\ R\ p(b)$ for every $p \in \mathcal{P}$; this is the semantic content of the rule of unary congruence. Consider the two properties

(S) for all $a, b \in M$ with $a \neq b$ there is $p \in \mathcal{P}$ with $p(a) = 0$ and $p(b) = 1$;

(A) for all $c, d \in M$ there is $q \in \mathcal{P}$ with $q(0) = c$ and $q(1) = d$.

The equalities in (S) and (A) are equalities of elements of M, not statements of R.

Theorem 11.185 (Rigidity of a unary-compatible relation)**.** *Assume (S) and (A), and argue classically about M. If R is reflexive and preserved by $\mathcal{P}$, then R is the equality of M or the universal relation $M \times M$. In particular, if $0\ R\ 1$ fails, R is the equality of M. Symmetry, transitivity and induction are not assumed.*

Proof. If no pair of distinct elements is related, R is contained in the equality, and by reflexivity equal to it. Otherwise let $a \neq b$ and $a\ R\ b$. By (S) and preservation, $0 = p(a)\ R\ p(b) = 1$. For arbitrary c, d, (A) and preservation give $c = q(0)\ R\ q(1) = d$, so R is universal. □

The passage from one exhibited related pair of distinct elements to universality uses no excluded middle. Without any assumption on the equality of M, (S), preservation and the failure of $0\ R\ 1$ give constructively $a\ R\ b \to \neg\neg(a = b)$: from $a\ R\ b$ and $a \neq b$, (S) gives $0\ R\ 1$. If the equality of M is stable, in particular decidable, then R is contained in it, and equal to it by reflexivity. Only the disjunction of the theorem uses the classical case distinction, and it is a statement about candidate interpretations, not about iPRA or the calculus. Classically, reflexivity may be replaced by the conditions that R is nonempty and that $\mathcal{P}$ contains the constant maps: a constant map p_e sends a related pair to $e\ R\ e$. The orientation in (S) is immaterial in the presence of (A), since (A) holds for every ordered pair (c, d).

Interpolation from exact computation. In a structure in which the two computation equations of Section 1.1 hold as identities, put $q(z) := I(z, c;\ u.\, d)$ with u not in d. Then $q(0) = c$ and $q(S0) = d$ without induction, so (A) holds for the pair $0, S0$. On full normal forms these two identities are literal normalizations.

Separation in a model of the ordinary theory. In a model of iPRA with actual equality, the primitive recursive discriminator χ with $\chi(x, a) = 0$ for $x = a$ and $\chi(x, a) = 1$ otherwise has a characterization provable in iPRA, so these are its values at all elements, standard or not, and $p(x) := \chi(x, a)$ gives (S). In a model of T_{eq} the same holds with $\chi(x, a) := I(\varepsilon(x, a), 0;\ u.\, S0)$: by Theorem 1.86, $\varepsilon(x, a) = 0$ exactly when $x = a$, and by coverage $\varepsilon(x, a)$ is otherwise a successor, at which the iterator has the value $S0$. In a model with actual equality of the theories $\mathsf{T}_{B\varepsilon}$ and T_F^- of Theorems 8.33 and 8.56, and of Q_F of Section 11.10.12, (S) holds already by the axioms (8.11) and (8.12), without coverage and without induction: for $a \neq b$ the map $p(x) := \varepsilon(x, a)$ has $p(a) = 1$ and $p(b) = 0$, which is (S) for the pair $1, 0$. Whether the terms of these theories have a unary family with (A) is not examined: their recursors take the value $S(h[0/v])$ at the counter 0, and the selectors of Theorem 8.57 are stated for the values $0, 1$ only; so the theorem is not applied to them here.

Consequently, in any interpretation of the equations of the term language by a reflexive relation preserved by the unary term functions, over the term operations of a model of iPRA or of T_{eq}, that relation is the equality or the universal relation, and both are transitive. A countermodel to the derivability of transitivity from reflexivity, symmetry and unary congruence therefore cannot keep the ordinary term operations and change only the meaning of equations. The countermodel of Theorem 6.67, with $S0$ interpreted as 2, fails (A): its iterator has the value m at every nonzero counter, so $q(2) = m$ for every term function of the form $I(z, c;\ u.\, d)$, and by the list of triples in its proof no unary term function sends $0, 2$ to $2, 0$; full normalization of the successor computation would force the identity $q(S0) = d$. Its related pairs of distinct elements, $0, m$ and $m, 2$, are separated by no unary term function, by the same list, in accordance with the theorem.

The theorem does not decide the question of Theorem 6.67 on full normal forms. There, exact computation gives (A), but the derivation of (S) in the weakened calculus would need the reflection of equality, whose proof in Theorem 1.86 uses the equality principles that the weakening removes. A countermodel of the kind of Theorem 6.67 on full normal forms, with exact computation identities and an equation relation that is reflexive and preserved by unary term functions but not transitive, must therefore contain distinct related elements a, b that no unary term function sends to $0, 1$. The existence of such a pair does not by itself give a countermodel; the logical rules and induction would still have to be validated. Like Theorem 11.184, the theorem extracts rigidity from separation by a term without induction: there, the tests $\varepsilon(v, \bar{i})$ separate finitely many numerals inside the theory and force a standard numeral to have a unique predecessor; here, separation by unary term functions in a model

forces an equation relation compatible with them to be trivial or the equality.

11.11 Checker sizes

The table lists every program whose size is reported by a package of this chapter, with the figures as the packages state them, under the measurement protocol of Section 1.10. Each row is one complete program unless it is marked as an envelope, which combines coordinates of different programs and is not the size of any program. The second column gives the input contract of Theorem 11.58 and the calculus:

Code	Contract and calculus
R/$\mathcal{A}$	raw contract, calculus $\mathcal{A}$ of Theorem 11.1
R/$\mathcal{U}$	raw contract with a canonical guarded goal assumed, calculus $\mathcal{U}$ of Theorem 11.5
R/$\mathcal{U}^{\mathrm{Tr}}$	the same contract, calculus $\mathcal{U}^{\mathrm{Tr}}$ of Theorem 11.127, two-field certificates of Theorem 11.128
R^{+2}/$\mathcal{U}$	raw contract with a canonical guarded goal in the 1-shifted encoding of Theorem 11.149 assumed, calculus $\mathcal{U}$ (Theorems 11.151, 11.153, 11.157 and 11.164)
T^{+2}/g	typed contract for canonical closed guarded formulas in the 1-shifted encoding: whole input of exact integers and arrays, last element $[A, 0]$, goal validated by the predicate V of Theorem 11.154; calculus $\mathcal{U}$
G/$\mathcal{U}$	goal contract for canonical closed guarded formulas, calculus $\mathcal{U}$
G/$\mathcal{A}^{-}$	goal contract for the grammar with equality and D, calculus $\mathcal{A}^{-}$
T/eqd	typed contract for the grammar with equality, Q and D, calculus $\mathcal{A}^{-}$
T/ed	typed contract for the grammar with equality and D, calculus $\mathcal{A}^{-}$
T/g	typed contract for canonical closed guarded formulas, calculus $\mathcal{U}$
Gt/$\mathcal{A}^{-}$	tolerant goal contract of Theorem 11.121 for the grammar with equality and D, calculus $\mathcal{A}^{-}$
T/ed$_1$	typed contract for the grammar P_1^{ed} of Theorem 11.119, public zero encoded by the scalar 1, calculus $\mathcal{A}^{-}$ (the program of `zero_alias` with the initial context $[0, [1, 0]]$, those of `smaller_presentations`, `scope_refinement`, `validated_leaves`, `checked_presentations`, `procedure_presentation` and `prefix_conditional` with O^{+}, Theorems 11.148, 11.159, 11.160, 11.163 and 11.164)
G$_1$/$\mathcal{A}^{-}$	goal contract for the grammar P_1^{ed}, public zero encoded by the scalar 1, calculus $\mathcal{A}^{-}$ (Theorems 11.148, 11.159 and 11.163)
S	strict contract of `guarded_register`: certificates $[A, \pi]$ with a proof tree π, every term and the goal validated; its own calculus, see the remarks below
K	strict contract for the conditional-copy calculus K: every active term and formula and the goal validated, in full normal form; tree or stream certificates (Theorem 11.125)

A program is to be compared only with the control named in its own row, which has the same contract and the same calculus. A dagger marks a profile that recognizes arrays by attempted traversal and carries the qualification on nonempty strings stated after Theorem 11.10. A double dagger marks a program that does not read its whole input as one JSON value: it reads only the first line, or one record per line as the run requests (Section 11.11.1); such a program is compared only with its control of the same or of the one-line framing, and enters neither the envelope nor the list of superseded programs. The packages also report sizes after

compression; these are informational in every package and are omitted. Controls are given as tokens / syntax-tree nodes / bytes.

Program	Code	Tokens	Nodes	Bytes	Matched control
`certified_zero`: base protocol, Section 11.3					
explicit array test	R/$\mathcal{A}$	264	308	566	envelopes below
zero-product array test	R/$\mathcal{A}$	264	310	558	envelopes below
attempted traversal†	R/$\mathcal{A}$	262	296	598	envelopes below
envelope of the supplied programs	R/$\mathcal{A}$	268	300	616	different programs
envelope of seven matched controls	R/$\mathcal{A}$	264	300	601	different programs
`uniform_zero`: local floors, Section 11.4					
control, zero-product test	R/$\mathcal{A}$	267	311	545	published 264/310/558
control, explicit test	R/$\mathcal{A}$	267	309	553	published 264/308/566
uniform zero, zero-product test	R/$\mathcal{A}$	269	314	550	267/311/545
uniform zero, explicit test	R/$\mathcal{A}$	269	312	558	267/309/553
constructed majors, zero-product test	R/$\mathcal{A}$	272	314	556	267/311/545
constructed majors, explicit test	R/$\mathcal{A}$	272	312	564	267/309/553
constructed majors, readable	R/$\mathcal{A}$	309	361	1600	none
`uniform_premise`: neutral major, global floor, Section 11.5					
explicit test, conditional replacement	R/$\mathcal{A}$	262	303	548	264/308/566
zero-product test, Boolean replacement	R/$\mathcal{A}$	263	308	537	264/310/558
`monus_and_neutral_major`: neutral major, global floor, Section 11.5					
explicit array test	R/$\mathcal{A}$	263	307	550	264/308/566
zero-product array test	R/$\mathcal{A}$	263	309	542	264/310/558
attempted traversal†	R/$\mathcal{A}$	261	295	578	262/296/598†
`neutral_scope`: neutral major, local floors, Section 11.5					
zero-product array test	R/$\mathcal{A}$	266	310	529	263/309/542
explicit array test	R/$\mathcal{A}$	266	308	537	none stated
loop, attempted traversal†	R/$\mathcal{A}$	263	290	610	263/291/642
`prevalidated_elimination`: global floor, Section 11.6					
explicit array test	R/$\mathcal{A}$	263	307	547	264/308/566
zero-product array test	R/$\mathcal{A}$	263	309	539	264/310/558
attempted traversal†	R/$\mathcal{A}$	261	292	576	262/296/598†
conclusion synthesis, experimental	other	330	390	951	none
synthesis, shared binder, experimental	other	334	392	777	none
`scope_and_closure`: neutral major, local floors, elimination first, Theorem 11.46					
zero-product array test	R/$\mathcal{A}$	267	309	528	266/310/529
loop, attempted traversal†	R/$\mathcal{A}$	264	288	608	263/289/613†
`kernel_refinement`, `compact_update`, `validated_core`: guarded-source protocol, Section 11.7					
loop, global floor† (not in `compact_update`)	R/$\mathcal{U}$	256	283	594	257/285/568†
loop, scalar neutral premise†	R/$\mathcal{U}$	260	278	602	261/279/603†
scalar neutral premise, zero-product test	R/$\mathcal{U}$	263	300	521	264/301/522
the same with a goal validator	G/$\mathcal{U}$	489	560	1201	490/561/1202
`checked_refinement`					

Program	Code	Tokens	Nodes	Bytes	Matched control
compact, M° rewritten	R/$\mathcal{U}$	263	303	520	263/300/521
guarded goal validation, Theorem 11.64	G/$\mathcal{U}$	407	459	921	489/560/1201
native goal validation, one expression	G/$\mathcal{A}^{-}$	358	413	788	359/414/803
`validated_core`: Theorem 11.61					
one validator with raw mode	T/eqd	401	468	897	450/514/1049
the same, smaller goal grammar	T/ed	390	454	878	424/484/1000
`validated_presentation`: Theorem 11.62					
typing by the printed form	T/eqd	383	442	870	401/468/897
the same, smaller goal grammar	T/ed	370	426	849	390/454/878
lexical filter before parsing	T/eqd	–	–	885	401/468/897
`scope_kernel`: Theorem 11.63					
scope reuse and direct splitting	T/eqd	393	460	881	401/468/897
the same, smaller goal grammar	T/ed	382	446	862	390/454/878
`deterministic_introduction`, separate refinement					
grammar in the first opening, Theorem 11.65	G/$\mathcal{A}^{-}$	333	379	656	360/409/780
compact, complement for the decrement	R/$\mathcal{U}$	267	303	506	267/302/508
`uniform_recursion`, separate refinement					
grammar in the first opening, explicit guard	G/$\mathcal{A}^{-}$	340	385	685	360/409/780
the same, merged into the cases	G/$\mathcal{A}^{-}$	336	381	661	360/409/780
`one_body`, separate refinement					
validator rewritten	G/$\mathcal{A}^{-}$	357	412	778	358/413/788
validator and M° rewritten	G/$\mathcal{A}^{-}$	357	415	777	358/413/788
`inferred_major`, `smaller_envelope` and `retained_premises`: inferred major, Theorem 11.67 (the same two programs in the three packages)					
loop, global floor[†]	R/$\mathcal{U}$	253	280	599	256/283/594[†]
loop, local floors, scalar neutral premise[†]	R/$\mathcal{U}$	257	275	607	260/278/602[†]
`inferred_major`: Theorems 11.85 and 11.86					
truth-valued result, complement decrement	R/$\mathcal{U}$	278	310	503	267/303/506
truth-valued result, ordinary decrement	R/$\mathcal{U}$	278	309	505	267/303/506
flag as an order	G/$\mathcal{A}^{-}$	327	375	643	333/379/656
`smaller_envelope`: Theorem 11.84					
remainder selector	R/$\mathcal{U}$	264	304	515	265/306/516
readable typed program	T/g	580	666	2473	none
`frontier_review`: Theorem 11.83					
eager selection	R/$\mathcal{U}$	268	306	513	265/306/516
the same with typed wrapper	T/g	462	524	1183	459/524/1186
`checked_presentation`, `progress_independence`, `crossed_uniqueness`: compositions on the control of 506 bytes					
eager selection and remainder selector (all three packages, and `502_scope_audit`)	R/$\mathcal{U}$	269	301	502	267/303/506

Program	Code	Tokens	Nodes	Bytes	Matched control
eager selection only (`progress_independence`, `crossed_uniqueness`)	R/$\mathcal{U}$	270	303	503	267/303/506
remainder selector only (`crossed_uniqueness`, `502_scope_audit`)	R/$\mathcal{U}$	266	301	505	267/303/506
typed wrapper, recursive validator (`checked_presentation`, `crossed_uniqueness`)	T/g	463	519	1172	461/521/1176 and 462/524/1183
typed wrapper, printed form (`progress_independence`)	T/g	421	471	869	none stated
`progress_independence`: inferred major with reuse of the packet tail					
loop, global floor†	R/$\mathcal{U}$	251	283	605	253/280/599†
`checked_presentation`: Theorem 11.88					
typed native checker	T/ed	344	393	673	370/426/849
`context_phase`: Theorem 11.87					
phase by context, immutable tuple	G/$\mathcal{A}^-$	327	375	643	333/379/656
the same, explicit type test	G/$\mathcal{A}^-$	327	375	646	333/379/656
mutable list, one invocation only	G/$\mathcal{A}^-$	325	375	641	333/379/656
`context_order` (archive; programs reproduced as controls by `three_field` and `constantfree`): Theorem 11.97(i),(ii)					
loop, local floors, allocation first†	R/$\mathcal{U}$	262	273	572	none stated
zero-product test, allocation first	R/$\mathcal{U}$	265	301	499	none stated
`three_field`: Section 11.9.9, Theorem 11.97(iv),(v)					
three fields, global floor†	R/$\mathcal{U}$	249	278	607	251/283/605†
three fields, local floors†	R/$\mathcal{U}$	255	271	606	257/275/607†
four fields, allocation first, eager selection, remainder selector	R/$\mathcal{U}$	267	299	495	265/301/499
typed native checker, allocation first	T/ed	342	391	666	344/393/673
`constantfree`, compact part: Theorem 11.97(iii),(vi)					
global loop, variadic context†	R/$\mathcal{U}$	250	283	603	251/283/605†
allocation first, remainder selector	R/$\mathcal{U}$	264	299	498	265/301/499
`retained_premises`: Section 11.9.3, Theorem 11.82					
retained majors, local floors	R/$\mathcal{U}$	265	301	513	265/306/516
the same with typed wrapper	T/g	421	474	889	407/459/921 (G/$\mathcal{U}$)
native goal validator refined	G/$\mathcal{A}^-$	347	400	700	358/413/788
`checker_refinement`: Theorems 11.98 and 11.102					
truth-valued result after allocation first	R/$\mathcal{U}$	271	304	488	267/299/495
native, flag as an order, truth-valued result	T/ed	343	392	646	342/391/666
`single_binder`, checker part: Theorem 11.98, Section 11.11.1					
truth-valued result (text of `checker_refinement`)	R/$\mathcal{U}$	271	304	488	267/299/495
the same as an anonymous function	R/$\mathcal{U}$	276	308	487	267/299/495
three fields, global floor, first line only†‡	R/$\mathcal{U}$	248	277	609	248/277/609†‡
three fields, local floors, first line only†‡	R/$\mathcal{U}$	254	270	608	254/270/608†‡

Program	Code	Tokens	Nodes	Bytes	Matched control
strict guarded wrapper of the kernel of 488 bytes	T/g	556	633	1544	none stated
`502_scope_audit`: Theorem 11.176, Section 11.11.1 (its programs of 502 and 505 bytes are listed above)					
readable program, typed	T/g	605	700	2629	none stated
entry scan deleted, unsound negative control†	other	250	277	596	253/280/599†
program of 502 bytes in a compressed wrapper	other	18	19	459	18/19/459 (the program of 506 bytes, same wrapper)
`verified_refinements`: Theorems 11.101, 11.103 and 11.104					
inverted entry test	R/$\mathcal{U}$	267	298	492	267/299/495
composed native goal validation	G/$\mathcal{A}^{-}$	324	374	629	325/375/641
phase by an empty context	T/ed	334	387	654	342/391/666
`refinement`: Theorem 11.99, Section 11.9.14					
truth-valued result after allocation first (text of `checker_refinement`)	R/$\mathcal{U}$	271	304	488	267/299/495
readable form of the same, docstrings kept	R/$\mathcal{U}$	314	381	1870	none stated
the same kernel with the validator of `progress_independence`	T/g	423	474	855	419/469/862
matched control: that validator on the program of 495 bytes	T/g	419	469	862	constructed by the package
two-field reference	R/$\mathcal{U}^{\mathrm{Tr}}$	295	352	742	none
`immediate_application`: Theorem 11.100					
immediate application, anonymous function	R/$\mathcal{U}$	272	304	479	276/308/487
immediate application, native	T/ed	344	392	637	343/392/646
`scope_and_frontier`: Theorem 11.106					
floor recomputed from the context	R/$\mathcal{U}$	267	303	500	267/298/492
readable form of the same	R/$\mathcal{U}$	296	353	1329	none stated
`checked_presentation`, second delivery: Theorems 11.109 and 11.186, Theorem 11.108					
three fields, global floor, one record per line†‡	R/$\mathcal{U}$	244	272	608	248/277/609†‡
three fields, local floors, one record per line†‡	R/$\mathcal{U}$	249	267	606	254/270/608†‡
immediate application (text of `immediate_application`)	R/$\mathcal{U}$	272	304	479	276/308/487
native, copied context, immediate application	G/$\mathcal{A}^{-}$	334	381	614	324/374/629
native immediate application (text of `immediate_application`)	T/ed	344	392	637	343/392/646
`479_predecessor57`, checker part: Theorem 11.110, Theorem 11.108					
immediate application (text of `immediate_application`)	R/$\mathcal{U}$	272	304	479	276/308/487
exact-integer goal leaves, no test of the printed form	G/$\mathcal{A}^{-}$	337	383	607	324/374/629
native immediate application (text of `immediate_application`)	T/ed	344	392	637	343/392/646

Program	Code	Tokens	Nodes	Bytes	Matched control
`immediate_opening`: Theorem 11.108					
immediate application (text of `immediate_application`)	R/$\mathcal{U}$	272	304	479	276/308/487
readable form of the same, docstrings kept	R/$\mathcal{U}$	306	367	1995	none stated
strict guarded companion on the kernel of 479 bytes, docstrings kept	T/g	558	635	1761	none stated
`zero_test_split`, checker part: Theorem 11.116					
initial floor with the goal	R/$\mathcal{U}$	271	305	478	272/304/479
`checked_frontier`, checker part: Theorem 11.109(a),(b), Section 11.9.13					
native, truth-valued result, extension in place	G/$\mathcal{A}^-$	330	379	623	324/374/629
the same as an anonymous function	G/$\mathcal{A}^-$	335	383	622	324/374/629
strict, anonymous function with a conjunction	T/ed	348	396	645	343/392/646
`atomic_seed_refinement`, checker part: Theorem 11.111					
one parity comparison, native goal validation	G/$\mathcal{A}^-$	326	370	636	327/375/643
one parity comparison, native immediate application	T/ed	341	387	628	344/392/637
`reductions`, checker part: Theorem 11.109(c),(d)					
native, truth-valued result, copied context	G/$\mathcal{A}^-$	332	381	621	324/374/629
the same as an anonymous function	G/$\mathcal{A}^-$	337	385	620	324/374/629
`equation_aliases`, strict checkers: Theorem 11.112					
matched control: the program of 620 bytes with the test of the printed form	T/ed	351	403	666	constructed by the package
products of bits, immediate application	T/ed	338	390	637	351/403/666
the same with the reader in the initial call	T/ed	342	386	641	351/403/666
product of bits in the program of 637 bytes	T/ed	342	392	633	344/392/637
`alias_back`, checker part: Theorem 11.108, Theorem 11.126					
immediate application (text of `immediate_application`)	R/$\mathcal{U}$	272	304	479	276/308/487
two-field stream for K	K	609	699	1644	629/732/1715
matched control: tree checker of K, simplified	K	629	732	1715	constructed by the package
`absorbing_orbit`, checker part: Theorem 11.113					
test of the printed form as a disjunction	T/ed	343	390	632	344/392/637
`boolean_constancy`, checker part: Theorem 11.114, Section 11.11.1					
three fields, global floor, one record per line, wildcard import†‡	R/$\mathcal{U}$	244	272	604	244/272/608†‡
three fields, local floors, one record per line, wildcard import†‡	R/$\mathcal{U}$	249	267	602	249/267/606†‡

Program	Code	Tokens	Nodes	Bytes	Matched control
eager mode selector	R/$\mathcal{U}$	274	304	478	272/304/479
eager mode selector, native	T/ed	346	392	636	344/392/637
`zero_alias`, checker part: Theorem 11.120					
zero as a defined alias	T/ed$_1$	331	373	625	none in its contract
readable form of the same	T/ed$_1$	430	499	2322	none
`checked_reductions`, checker part: Theorem 11.118					
header and eager selector	R/$\mathcal{U}$	273	305	477	271/305/478
parity and disjunctive gate on the program of 628 bytes	T/ed	340	385	623	341/387/628
phase by an empty context, parity, gate, inverted entry	T/ed	330	379	641	334/387/654
parity on the program of 607 bytes	G/$\mathcal{A}^-$	336	378	600	337/383/607
parity on the program of 629 bytes	G/$\mathcal{A}^-$	323	369	622	324/374/629
`refinement`, second delivery: Theorem 11.118					
parity on the program of 629 bytes (text of `checked_reductions`)	G/$\mathcal{A}^-$	323	369	622	324/374/629
parity and product on the program of 629 bytes	G/$\mathcal{A}^-$	321	371	618	324/374/629
parity and product on the program of 607 bytes	G/$\mathcal{A}^-$	334	380	596	337/383/607
phase by an empty context, parity, product, gate	T/ed	330	380	638	334/387/654
product and gate on the program of 628 bytes	T/ed	340	385	621	341/387/628
`tolerant_certificates`: Theorem 11.124					
terminating array test, no exact-integer conjunct	Gt/$\mathcal{A}^-$	315	361	603	321/371/618 (G/$\mathcal{A}^-$)
`fused_iteration` and `pred50`, checker parts: Theorem 11.187					
local stream program, product in the floor†‡	R/$\mathcal{U}$	249	265	605	249/267/606†‡
global stream program, conjunction for the major†‡	R/$\mathcal{U}$	244	273	607	244/272/608†‡
`guarded_register`: the guarded scalar J of the chapter on canonical and counter-aware recursors					
strict guarded-J checker	S	1041	1181	3467	1045/1193/3566 (strict scalar I)
`head_conversion`, compact program (reproduced as control by four packages): Section 11.9.15					
header, eager opener, empty neutral resource	R/$\mathcal{U}$	272	306	477	not stated
`computation_leaves`: Theorem 11.135					
global stream, direct leaves†‡	R/$\mathcal{U}$	243	271	616	244/272/608†‡
local stream, direct leaves†‡	R/$\mathcal{U}$	248	264	613	249/265/605†‡
four fields, header, direct leaves	R/$\mathcal{U}$	270	308	474	272/306/477
native, direct leaves, mode test $u = 0$	G/$\mathcal{A}^-$	334	384	600	334/380/596
strict, direct leaves, mode test $u = 0$	T/ed	340	389	625	340/385/621
`shared_leaf` and `uniform_terminals`: Theorem 11.139					
global stream, uniform terminals (both packages)†‡	R/$\mathcal{U}$	242	268	565	243/271/616†‡

Program	Code	Tokens	Nodes	Bytes	Matched control
local stream, byte-string context (`shared_leaf`)†‡	R/$\mathcal{U}$	246	262	556	248/264/613†‡
local stream, floor in a default (`uniform_terminals`)†‡	R/$\mathcal{U}$	247	263	553	248/264/613†‡
four fields, uniform terminals, eager selector (both packages)	R/$\mathcal{U}$	272	306	469	270/308/474
`smaller_presentations`: Theorems 11.145, 11.146 and 11.148					
global stream, tail call†‡	R/$\mathcal{U}$	242	269	606	243/271/616†‡
eager selector on the program of 474 bytes (also `vacuous_specialization`)	R/$\mathcal{U}$	272	308	473	270/308/474
native, uniform zero, test a is $+a$	$G_1/\mathcal{A}^-$	328	372	581	332/373/594 (G/$\mathcal{A}^-$)
native, uniform zero, exclusion of `True`	$G_1/\mathcal{A}^-$	328	369	587	332/373/594 (G/$\mathcal{A}^-$)
strict, uniform zero	T/ed_1	334	378	608	340/382/616 (T/ed)
`whole_json_refinement` (the two whole-input programs also in `orbit_equality`, identical text): Theorem 11.146(d), Section 11.8					
global, whole input, direct leaves†	R/$\mathcal{U}$	248	277	615	243/271/616†‡
local, whole input, direct leaves†	R/$\mathcal{U}$	254	268	613	248/264/613†‡
parity factor deleted, unsound negative control	other	268	301	465	272/308/473
`ast269_successor_rigidity`, `selective_normalization` and `checker_refinements`: Theorem 11.146(c)					
local three fields, whole input, floor product (identical text)†	R/$\mathcal{U}$	255	269	605	255/271/606†
`476_byte_presentation`, `export_specialization`, `abstraction_and_refinements`, `checker_refinements`: Theorem 11.146(b)					
eager selector on the program of 477 bytes (identical text)	R/$\mathcal{U}$	274	306	476	272/306/477
`abstraction_and_refinements`, validated programs: Theorem 11.147					
native token profile	G/$\mathcal{A}^-$	317	364	617	321/371/618
native node profile	G/$\mathcal{A}^-$	321	358	622	321/371/618
native byte profile	G/$\mathcal{A}^-$	332	373	594	334/380/596
strict token profile	T/ed	328	377	634	330/380/638
strict node profile	T/ed	335	368	643	330/380/638
strict byte profile	T/ed	340	382	616	340/385/621
`476_byte_presentation`, implementation appendix: Theorem 11.147(v),(vi)					
byte-string extension, on the program of 618 bytes	G/$\mathcal{A}^-$	319	369	622	321/371/618
byte-string extension, on the program of 622 bytes	G/$\mathcal{A}^-$	321	367	626	323/369/622
default flag, on the program of 596 bytes	G/$\mathcal{A}^-$	332	379	594	334/380/596
byte-string extension, on the program of 638 bytes	T/ed	328	378	642	330/380/638
default flag, on the program of 621 bytes	T/ed	338	384	619	340/385/621
`fixed_points_and_checker`, checker part: Theorem 11.140					
local stream, tuple context, major call as a statement†‡	R/$\mathcal{U}$	241	267	566	246/262/556†‡
`offset472`: Theorem 11.151					

Program	Code	Tokens	Nodes	Bytes	Matched control
direct leaves, eager selector, depths shifted by 2, neutral resource 0	$\mathrm{R}^{+2}/\mathcal{U}$	271	307	472	272/308/473 ($\mathrm{R}/\mathcal{U}$)
`scope_refinement`, checker part: Theorem 11.159					
strict token profile with uniform zero, S_1^{T}	T/ed_1	322	373	626	328/377/634 (T/ed)
strict node profile with uniform zero, S_1^{N}	T/ed_1	329	364	635	335/368/643 (T/ed)
strict, uniform terminals, direct leaves, marked cases, S_3	T/ed_1	335	382	607	334/378/608
readable form of S_3	T/ed_1	421	503	2152	none
`validated_leaves`, checker part: Theorem 11.159					
strict, uniform terminals, direct leaves, marked progress, S_2	T/ed_1	334	379	606	334/378/608
native, the same, test a is $+a$, N_2	$\mathrm{G}_1/\mathcal{A}^-$	328	373	579	328/372/581
`468_presentation`: Theorems 11.153 and 11.154					
uniform terminals, depths shifted by 2, T_+	$\mathrm{R}^{+2}/\mathcal{U}$	271	305	468	272/306/469 ($\mathrm{R}/\mathcal{U}$); 271/307/472 (same code)
readable form of T_+	$\mathrm{R}^{+2}/\mathcal{U}$	327	391	1572	none
validated companion of T_+	$\mathrm{T}^{+2}/\mathrm{g}$	690	773	2064	none
`ast261`, checker part: Theorem 11.141(a)					
stream, byte-string context, floor in a default, $L_{\mathrm{d}}{}^{\dagger\ddagger}$	$\mathrm{R}/\mathcal{U}$	247	261	557	246/262/556†‡
`checked_presentations`: Theorem 11.141(b),(c), Theorem 11.160					
global stream, major call as a branch, $G_{\mathrm{b}}{}^{\dagger\ddagger}$	$\mathrm{R}/\mathcal{U}$	240	266	592	242/268/565†‡
global, whole input, major call as a branch, $G_{\mathrm{b}}^{\mathrm{w}\dagger}$	$\mathrm{R}/\mathcal{U}$	245	272	591	247/274/564†
matched control: global, whole input, $G^{\mathrm{w}\dagger}$	$\mathrm{R}/\mathcal{U}$	247	274	564	constructed by the package
strict, conditional selection, S_2^{T}	T/ed_1	319	370	630	322/373/626
strict, conditional selection and factor, S_2^{N}	T/ed_1	327	358	642	329/364/635
strict, uniform terminals, marker 2 for progress, S_4	T/ed_1	335	380	605	335/382/607
`root_conversion_raw`, compact part: Theorem 11.146(b)					
eager selector on the program of 477 bytes (text of `checker_refinements`)	$\mathrm{R}/\mathcal{U}$	274	306	476	272/306/477
readable expansion of the same	$\mathrm{R}/\mathcal{U}$	307	362	1847	none
`468_shared_leaf`, `shifted_shared_leaf` and the checker part of `scalar_constancy`: Theorem 11.156, Theorem 11.141(a)					
T_+ (text of `468_presentation`; all three packages)	$\mathrm{R}^{+2}/\mathcal{U}$	271	305	468	272/306/469 ($\mathrm{R}/\mathcal{U}$)
readable form of T_+ (`shifted_shared_leaf`)	$\mathrm{R}^{+2}/\mathcal{U}$	305	363	1933	none
L_{d} (text of `ast261`; `scalar_constancy`)†‡	$\mathrm{R}/\mathcal{U}$	247	261	557	246/262/556†‡
`direct_packets`: Theorems 11.142 and 11.143					
packets as arguments, T_{d}	$\mathrm{R}/\mathcal{U}$	274	303	465	272/306/469

Program	Code	Tokens	Nodes	Bytes	Matched control
whole input, major call as a statement, $L'^{\mathrm{w}\dagger}$	R/$\mathcal{U}$	246	273	565	241/267/566†‡
whole input, structural binding, $L_{\mathrm{m}}^{\mathrm{w}\,\dagger}$	R/$\mathcal{U}$	258	263	595	246/262/556†‡
stream, structural binding, $L_{\mathrm{m}}{}^{\dagger\ddagger}$	R/$\mathcal{U}$	252	259	595	246/262/556†‡
`procedure_presentation`: Theorems 11.144, 11.157 and 11.163					
procedure form of T_+, $T_{+,\mathrm{p}}$	R^{+2}/$\mathcal{U}$	267	298	462	271/305/468
procedure form of S_2, $S_{2,\mathrm{p}}$	T/ed$_1$	330	372	600	334/379/606
procedure form of N_2, $N_{2,\mathrm{p}}$	G$_1$/$\mathcal{A}^-$	324	366	573	328/373/579
byte-string extension, $S^{\mathrm{T}}_{1,\mathrm{b}}$	T/ed$_1$	320	371	630	322/373/626
byte-string extension, $S^{\mathrm{N}}_{1,\mathrm{b}}$	T/ed$_1$	327	362	639	329/364/635
reservation at return, L_{r} (earlier package)†‡	R/$\mathcal{U}$	240	266	551	246/262/556†‡
reservation at return, L'_{r} (earlier package)†‡	R/$\mathcal{U}$	242	261	563	246/262/556†‡
`fused_blocks_and_checkers`, checker part, and the third delivery of `checked_presentation`: Theorem 11.164(a),(b)					
mode test fused with the returned floor, L_{f} (both packages, identical text)†‡	R/$\mathcal{U}$	239	261	549	240/266/551†‡ (L_{r})
L_{f} with initialization in a default and structural binding, $L_{\mathrm{f,m}}$ (third delivery)†‡	R/$\mathcal{U}$	246	257	589	239/261/549†‡ (L_{f})
the same with the declaration of the floor in the case, $L'_{\mathrm{f,m}}$ (`fused_blocks_and_checkers`)†‡	R/$\mathcal{U}$	245	256	580	239/261/549†‡ (L_{f})
`prefix_conditional`: Theorem 11.164					
conditional image, $L_{\mathrm{c}}{}^{\dagger\ddagger}$	R/$\mathcal{U}$	239	261	553	240/266/551†‡ (L_{r})
L_{c} with initialization in a default and structural binding, $L_{\mathrm{c,m}}{}^{\dagger\ddagger}$	R/$\mathcal{U}$	246	257	593	239/261/553†‡ (L_{c})
whole input, $L_{\mathrm{c}}^{\mathrm{w}\dagger}$	R/$\mathcal{U}$	244	269	547	L_{r} on the whole input, not measured
whole input, $L_{\mathrm{c,m}}^{\mathrm{w}}{}^{\dagger}$	R/$\mathcal{U}$	252	261	593	244/269/547† ($L_{\mathrm{c}}^{\mathrm{w}}$)
rotated packets with prefixes, $T^{\rho}_{+,\mathrm{p}}$	R^{+2}/$\mathcal{U}$	267	298	461	267/298/462 ($T_{+,\mathrm{p}}$)
rotated packets with prefixes, $S^{\rho}_{2,\mathrm{p}}$	T/ed$_1$	330	372	599	330/372/600 ($S_{2,\mathrm{p}}$)
Checkers of calculi of other chapters, with their matched controls (contract and calculus of their packages)					
AQE, seven rules (`abstraction_and_refinements`)	other	1672	1818	7132	1590/1741/6830 (seven rules with QE); 1647/1807/7121 (eight rules)
PD, three families (`476_byte_presentation`)	other	678	727	2087	688/735/2128 (four families); 643/695/1892 (inherited E3)
V, vacuous specialization (`vacuous_specialization`)	other	645	745	1830	646/748/1844 (`conditional_copy`)
bounded local conversion (`bounded_conversion`)	other	1075	1156	3522	1060/1121/3336 (weak-head)
newest Use, explicit REFL (`export_specialization`)	other	663	748	2093	669/754/2117
newest Use, inferred reflexivity target (`export_specialization`)	other	654	731	2049	653/729/2058
Attested by the audits of `orbit_closure` and `progress_independence` and not listed above					

Program	Code	Tokens	Nodes	Bytes	Matched control
loop profile of `deterministic_opening`†	R/$\mathcal{U}$	263	276	580	not in this chapter
global token profile of `uniform_progress`†	R/$\mathcal{U}$	256	283	593	not in this chapter

Some remarks on the reading of the table.

The controls of `uniform_premise`, `monus_and_neutral_major` and `prevalidated_elimination` are the three programs of `certified_zero`; `uniform_premise` lists in addition an older program of 266 tokens, 296 nodes and 689 bytes that shares the minimum of nodes. The control of the byte profile of `neutral_scope` is the byte profile of `monus_and_neutral_major`, and its loop profile is compared with an older program outside this chapter, as is the loop profile of `scope_and_closure` (a program of `coverage_and_kernel`). The controls of the guarded-source rows are the programs of `guarded_source`. The control of 1000 bytes was constructed by `validated_core` by restricting the control of 1049 bytes to the smaller grammar. The control of 780 bytes is a program of `deterministic_opening`; the control of 803 bytes is the native goal-validated checker current at the time of `checked_refinement`, whose validator already omits the scope bounds in the manner that the report attributes to `one_context`. The two experimental synthesis programs have a certificate format of their own. The two three-field programs read certificates of Theorem 11.89, and their controls, which read four-field certificates, are the global loop of `progress_independence` and the local loop of `inferred_major`; the two programs of `single_binder` marked with the double dagger read three-field certificates as well, and so do their reframed controls; all other rows of $\mathcal{U}$ read four-field certificates. No package states a control for the two programs of `context_order`; by Theorem 11.97 they are the compact program of `deterministic_introduction` (267/303/506) and the loop profile of `deterministic_opening` (263/276/580) allocating first. The native program of `three_field` and its control satisfy the same typed contract for the grammar with equality and D.

For the rows added from `retained_premises` onwards: the control of 516 bytes is the program so named in Section 11.9; the controls of 495 and 666 bytes are the programs (iv) and (v) of Theorem 11.97, which `checker_refinement`, `single_binder` and `verified_refinements` reproduce with identical text; the control of 641 bytes is the list variant of `context_phase`. The typed wrapper of `retained_premises` is compared by its package with the goal-validated program of `checked_refinement`, which satisfies a weaker contract; under its own contract the comparison is with the other rows T/g. The program of 629 bytes validates the goal but not the whole input and has the code G/$\mathcal{A}^{-}$; the programs of 646 and 654 bytes validate both. The control of `guarded_register` uses the same strict interface, logical rules, representation of variables and acceptance convention, but the scalar iterator in place of the calculus J; it is the only control of the table in another calculus, and the comparison is the package's. The packed rows are not checkers in the sense of the measurement protocol (Section 11.11.1).

For the rows of `refinement`, `immediate_application` and `scope_and_frontier`: the controls of 487 and 646 bytes are the anonymous-function program of `single_binder` and the native program of `checker_refinement`, and the control of 492 bytes is the inverted-entry program of `verified_refinements`, each reproduced with identical text. The control of 862 bytes is constructed by `refinement` by attaching the validator of the wrapper of `progress_independence` to the program of 495 bytes; it is a complete program of the contract T/g and is listed as a row. The readable program of `refinement` is measured without the removal of docstrings; the other programs of these packages contain none, or, in `scope_and_frontier`, are measured with them removed. The two-field reference is the only program of its code. `immediate_application` lists the two first-line programs of

`single_binder` as the bearers of its envelope, and `scope_and_frontier` lists in addition the programs of 629, 646 and 654 bytes; these rows appear above. The unsound program of Theorem 11.107 has no reported size.

For the rows of the second delivery of `checked_presentation`, `479_predecessor57`, `immediate_opening`, `zero_test_split`, `checked_frontier`, `atomic_seed_refinement` and `reductions`: the control of 629 bytes is the composed native program of `verified_refinements`, the controls of 646 and 487 bytes are those of the preceding paragraph, the control of 643 bytes is the native program of `inferred_major` with the flag as an order, the controls of 637 and 479 bytes are the programs of `immediate_application`, and the controls of the stream programs are the first-line programs of `single_binder`, all reproduced with identical text. The program of 478 bytes reads four-field packets and, as its last element, the goal together with an initial floor (Theorem 11.116); it is the only program of the table with this input format and is listed under R/$\mathcal{U}$, where its claims concern canonical goals. The two programs of `immediate_opening` other than the program of 479 bytes are measured with their docstrings, which the package states. The compositions of `atomic_seed_refinement` with a one-line lexical front end and the variant of 640 bytes of `checked_presentation` are not delivered and have no row; neither have the unsound native programs of Theorem 11.117. The programs under T/ed also satisfy the goal contract for their grammar; they do not change the line for G/$\mathcal{A}^{-}$ below. The control of 666 bytes of `equation_aliases` is constructed by the package and is a complete program of T/ed (Theorem 11.112(a)); so is the simplified tree checker of K of `alias_back` under K. The delivered tree checker of `conditional_copy` (646/748/1844) is listed with the calculus K in the chapter on witnesses and not here. The checkers of the calculus with aliases and backward substitution of `alias_back`, and the checker of the calculus with restricted aliases of `equation_aliases`, check calculi of other chapters and are not listed.

For the rows of Section 11.9.15: the packages `shared_leaf`, `uniform_terminals`, `smaller_presentations` `vacuous_specialization`, `ast269_successor_rigidity` and `selective_normalization` compare at the revision `12197d86f6`, `computation_leaves`, `export_specialization`, `476_byte_presentation` and `bounded_conversion` at `2fa4637b3b`, `abstraction_and_refinements` at `167c769308`, and `whole_json_refinement` at `6c6a164d21`; all measure under CPython 3.12.14; the controls of `whole_json_refinement` are the stream programs of `computation_leaves`, which read the same certificates line by line. The programs with direct leaves read certificates in which a computation closes its goal, and the uniform terminal programs certificates with a packet at every call; the four-field programs among them carry the header $[A, 0]$. The control of the strict program of `smaller_presentations` is the strict byte profile of `abstraction_and_refinements`, which `shared_leaf` also includes unchanged; the controls of the uniform-zero programs encode zero by the scalar 0 and are of other codes. The program of 465 bytes is the unsound program of Section 11.8. The checkers of other calculi, listed at the end of the table, belong to the chapters on the kernel (AQE), on discharge (PD), on witnesses (V and newest Use) and on computation (bounded local conversion); `476_byte_presentation` notes that the inherited checker E3 is smaller than PD.

For the rows of Sections 11.9.16 and 11.9.17 and of `fixed_points_and_checker`: `offset472` and `orbit_equality` compare at the revision `6c6a164d21`, `checker_refinements`, `fixed_points_and_check` and `validated_leaves` at `6874c85a4c`, and `scope_refinement` at `a0687e8082`; all measure under CPython 3.12.14. The control of the program of `fixed_points_and_checker` is the local stream program of `shared_leaf`; the control of `offset472` is the program of 473 bytes of `vacuous_specialization`, whose goals use the encoding of Theorem 11.9; the control of S_2 and S_3 is S_1 and that of N_2 is N_1, both reproduced with identical text. `checker_refinements` delivers the programs of 476 bytes and of 269 nodes with the texts listed above and the controls 272/306/477 and 255/271/606, and `orbit_equality` the programs of 615 and 613 bytes.

For the rows of `468_presentation`, `ast261`, `checked_presentations` and `root_conversion_raw`:

the first compares at the revision `dd308d5d93`, the other three at `31e1e92208`; all measure under CPython 3.12.14, and the readable form and the validated companion of `468_presentation` are measured with their docstrings removed. The matched control that `468_presentation` names is the uniform terminal program of 469 bytes, whose goals use the encoding of Theorem 11.9; the program of the same code is P_+ of `offset472`, which the package uses as the control of its proof. The controls of `ast261` and of the compact part of `checked_presentations` are the stream programs of `shared_leaf` with identical text; the control of the whole-input program is G^{w}, constructed by the package and listed as a row. The controls of the strict programs are those of the article (Theorem 11.160); the package names others. The control of `root_conversion_raw` is the program of 477 bytes of `head_conversion`, reproduced with identical text.

For the rows of `468_shared_leaf`, `shifted_shared_leaf`, `scalar_constancy`, `direct_packets` and `procedure_presentation`: the first three compare at the revision `78edba03ee`, `direct_packets` at `c2e762c43a` and `procedure_presentation` at `dd308d5d93`; all measure under CPython 3.12.14. The control named for T_+ is the program T of 469 bytes, with identical text, whose goals use the encoding of Theorem 11.9; `shifted_shared_leaf` and `scalar_constancy` also reproduce P_+. The readable forms of `468_shared_leaf` and of `direct_packets` (`code/readable_checker.py` `code/checkers/readable_checker.py`) have no reported size, and neither have the unsound programs of Section 11.8; they have no row. The controls of L'^{w} and $L^{\mathrm{w}}_{\mathrm{m}}$ are the stream programs from which they are obtained, as for `whole_json_refinement`; `direct_packets` compares them with the whole-input programs of that package (248/277/615 and 254/268/613). The last element of the inputs of T_{d} is $[A, 0, m, s, u, t]$; T_{d} is the only program of the table with this format and is listed under $\mathrm{R}/\mathcal{U}$, where its claims concern canonical goals. The controls of `procedure_presentation` are reproduced with identical text (T_+, S_2, N_2, S_1^{T}, S_1^{N} and L); its program with comments, `code/checker_strict_annotated.py`, has the syntax tree of $S_{2,\mathrm{p}}$ and no row of its own. L_{r} and L'_{r} are included unchanged from the earlier package `return_reservation`, whose revision `procedure_presentation` does not restate; the earlier package states the minima of 242 tokens and 262 nodes of the stream programs at its revision.

For the rows of `fused_blocks_and_checkers`, `prefix_conditional` and the third delivery of `checked_presentation`: all three compare at the revision `1156099ffe` and measure under CPython 3.12.14. Their controls are reproduced with identical text: L_{r} (`code/inherited_tokens240.py` of `procedure_presentation`) in all three, L_{m} of `direct_packets` in all three, $T_{+,\mathrm{p}}$ in all three, and $L^{\mathrm{w}}_{\mathrm{m}}$, $G^{\mathrm{w}}_{\mathrm{b}}$ and $S_{2,\mathrm{p}}$ in `prefix_conditional`; `fused_blocks_and_checkers` also reproduces L'_{r}. The matched control of each row is the program from which it is obtained in Theorem 11.164; the packages compare the programs that lower the node minimum with L_{m} (252/259/595) and $L^{\mathrm{w}}_{\mathrm{m}}$ (258/263/595), and $L^{\mathrm{w}}_{\mathrm{c}}$ with $G^{\mathrm{w}}_{\mathrm{b}}$ (245/272/591), as comparisons of size. The certificates of $T^{\rho}_{+,\mathrm{p}}$ and $S^{\rho}_{2,\mathrm{p}}$ are those of their controls rotated by ρ. `prefix_conditional` also reports sizes after compression, which for $T^{\rho}_{+,\mathrm{p}}$ (313 against 310) and $S^{\rho}_{2,\mathrm{p}}$ (391 against 387) are larger than those of the controls; as elsewhere they are omitted.

Under the raw contract no package claims a program that improves tokens, nodes and bytes at once against all programs of its contract, with one exception that is qualified: the traversal profile of `certified_zero` is below both envelopes of its package in all three coordinates. Wherever a package states a triple such as 256/278/521, 257/278/521, 251/273/499, 249/271/495, 253/275/513, 253/275/502, 249/271/492, 249/271/488, 248/270/487, 248/270/479, 248/270/478 or 244/267/479 for compact checkers, the three numbers belong to three programs. Under the goal contract and the typed contract single programs do improve all three coordinates of their controls; see the envelope below. In `uniform_zero` the comparison with a matched control shows that the new devices cost source: the control without them has 545 bytes, uniform zero adds 5 and constructed majors a further 6; the programs are smaller than the earlier published minimum of 558 only because the control already is. In `prevalidated_elimination` the saving of 19 bytes of the byte profile is decomposed by

the package into 11 for the exchange of steps and the folding of the major call, 6 for the unguarded test, and 2 for the spelling of the missing image. The typed checkers and the compact guarded checker of 520 or 521 bytes satisfy different contracts, and `validated_presentation` and `scope_kernel` state explicitly that their figures are not to be set against it.

11.11.1 What the measurement does not control

The protocol of Section 1.10 measures the source text of a complete program. It is not invariant under a change of representation of the same checking procedure, and three packages exhibit this.

`502_scope_audit` wraps the program of 502 bytes in a module that imports the standard compression and encoding libraries and executes the decompressed content of one fixed string literal of 395 characters, a compressed stream of 316 bytes that expands to the exact source. The whole module, literal included, has 18 tokens, 19 syntax-tree nodes and 459 bytes, below every coordinate of every checker of the table. The same wrapper applied to the program of 506 bytes also gives 18/19/459, with a stream of the same length, so the device gives the newer program no advantage over its control. The checking procedure, its runs and its correctness are those of the expanded program; the package checks the identity of the expanded text with the direct program byte for byte before running its fixtures, and notes that the extra frame of execution can change the behavior under a recursion limit. The measurement counts the syntax tree of the wrapper and not that of the program it executes, and it does not say how generic decoders that enter the trusted runtime are to be charged; `orbit_closure` states that embedding a compressed program changes the accounting question and does not supply a mathematical reduction.

`single_binder` replaces, in the global and the local three-field profiles, the reading of the whole standard input by the reading of its first line. On a certificate written on one line both programs parse the same tree, and every later operation is literally unchanged; every certificate has a lossless one-line serialization, and a converter that performs no proof processing supplies it. The accepted byte streams differ: later lines are ignored and a certificate spread over several lines must be converted. The change saves one token and one node and costs two bytes, and the same change applied to the two controls gives programs identical to the candidates. These rows carry the double dagger.

The second delivery of `checked_presentation` goes one step further and reads the certificate as a sequence of lines. The first line holds the JSON text of the goal, and each later line the JSON text of one packet, in the order in which the run consumes the packets; the program reads and parses the next line exactly when its control removes the next packet. Its two programs read neither the whole input nor only its first line: they read as many lines as the run requests and nothing after them. For an array $\sigma = [p_n, \dots, p_1, A]$ of raw trees let $\Lambda(\sigma)$ be the sequence of lines $A, p_1, \dots, p_n$, each written on one line.

Proposition 11.186 (Certificates read one record per line)**.** *Let P be one of the two first-line three-field programs of `single_binder`, and P' the program in which every removal of the last element of the array parsed from the first line is replaced by the parsing of the next line of the input.*

(a) For every array σ of integers and arrays, P accepts the one-line serialization of σ exactly when P' accepts $\Lambda(\sigma)$, with the same run.

(b) If P' accepts an input and reads its lines $A, p_1, \dots, p_r$, then P accepts the one-line serialization of $[p_r, \dots, p_1, A]$, with the same run.

Hence P and P' accept the same goals.

Proof. Pair the states of the two programs before the first read of the goal. The invariant is: all variables of the two programs have equal values, and the remaining lines of the input of P' begin with the elements of the remaining array of P in the reverse of their order in the array. The only operation that touches the input is the request of the next value; in P it removes and returns the last element of the array, in P' it parses the next line, and the two values are equal. Removal from an empty array and reading past the end of the input both fail. Every other operation is literally the same, including the order of evaluation: a major call requests its packets at the same point of the run, before the continuation, in both programs, since both consume one shared source. Induction on the run gives (a). For (b), the run of P' requests exactly $r + 1$ values; on the array $[p_r, \ldots, p_1, A]$ the run of P makes the same requests, each of which returns the same value, and makes no further request. □

The accepted inputs of P' are not the images of those of P: lines after the last line read are not read and may contain any text, including text that is not JSON, whereas P parses its whole first line; an unused prefix of the array becomes an unused suffix of the stream; and a line that is read must hold exactly one JSON value. An untrusted converter reverses an array and writes one value per line; it is not imported by either program. The global and the local programs keep their own naming of parameters. The programs have 244/272/608 and 249/267/606, against 248/277/609 and 254/270/608 for the first-line programs; the same change applied to the whole-input three-field programs of `three_field` (249/278/607 and 255/271/606), which differ from the first-line programs only in their reader, gives the same two programs. Against the whole-input programs the change saves five tokens and six nodes at the cost of one byte, and six tokens and four nodes at no cost in bytes. These rows carry the double dagger.

The framing by lines is due to the second delivery of `checked_presentation`. `absorbing_predecessor` lists the same two programs with identical text among its unchanged baselines, and `boolean_constancy`, which credits the framing to `absorbing_predecessor`, writes the import of the reader as the import of all public names of the standard JSON module. None of these names coincides with a name used by the programs, and the reader is the same function, so the runs are unchanged; the programs have 244/272/604 and 249/267/602, four bytes fewer than the stream programs and the same tokens and nodes.

Two further packages rewrite one expression of a stream program; `fused_iteration` and `pred50` take their controls from `absorbing_predecessor` and from the second delivery of `checked_presentation`, which have the same text.

Proposition 11.187 (Two rewritings of the stream programs)**.** *On inputs whose values are integers and arrays:*

(a) in the local stream program, the floor argument $(k := k - 1) \wedge (s \vee |a| = 2) \wedge k$ *of the opening of the new hypothesis may be replaced by* $(k := k - 1) \cdot (1$ *if* s *else* $|a| = 2)$*; the runs agree up to the replacement of* `False` *by* 0 *inside trees;*

(b) in the global stream program, the expression "$c(s, *g) \vee 3s$ *if* s *else* J*", with* J *the opening of the introduction, may be replaced by* $(s \wedge c(s, *g)) \vee 3s \vee J$*; the runs are the same.*

Proof. (a) Before the decrement $k \leq 0$, so $j := k - 1$ is a nonzero integer. If s is true-valued both expressions have the value j, and neither evaluates $|a|$. If s is false-valued, both evaluate $|a| = 2$; if it holds both have the value j, and otherwise the old one has the value `False` and the new one 0. The decrement is evaluated first in both. The two values are equal, are both false-valued, and behave alike under every operation of the program (arithmetic, comparison, membership, the failed traversal of a scalar by the opener); the program applies no exact-type test to internal trees. Relating trees up to this replacement, related states read the same lines and take the same branches, as in Theorem 11.87.

(b) Every returning call of the global program returns the null value, since its body is a loop without a return statement. If s is true-valued, both forms make the same major call; after it returns, both evaluate $3s$, which is a nonzero integer or a nonempty array, so the new form does not evaluate J. If s is false-valued, it is 0 or [ ] on this domain; neither form makes the call, the new form evaluates $3s$, which is 0 or [ ], false-valued and without effect, and both evaluate J once. □

The programs have 249/265/605 (`fused_iteration`, `code/checker_refinement/checker.py`) against 249/267/606, and 244/273/607 (`pred50`, `code/checker/checker607.py`) against 244/272/608. They carry the double dagger. `pred50` also records two rejected variants: a neutral 0 in every context with an unconditional major call (244/270/604), which accepts the literal goal 0 at once and calls the checker on the source [ ], so that it is not an exact rewriting, and an early decrement, which rejects valid certificates; neither is delivered.

None of these observations is a mathematical simplification. They show that the three coordinates depend on the charged interface (input framing, decoders, the success message) as well as on the checking procedure, and that a comparison is meaningful only between programs with the same interface; this is why every row names a matched control.

11.11.2 The coordinatewise envelope

The smallest value of each coordinate among the programs of the table, and of the audited corpus where the audit of Section 11.9 applies, is the following, separately for each contract. A triple in one line is the size of one program only where this is said.

Contract	Tokens	Nodes	Bytes	Attained by
R/$\mathcal{A}$	261	288	528	three programs: the traversal profiles of `monus_and_neutral_major` and `prevalidated_elimination` (261 tokens), the loop profile of `scope_and_closure` (288 nodes) and its zero-product profile (528 bytes)
R/$\mathcal{U}$	244	261	465	three programs: the whole-input programs $L_{\mathrm{c}}^{\mathrm{w}}$ (244/269/547) and $L_{\mathrm{c,m}}^{\mathrm{w}}$ (252/261/593) of `prefix_conditional` and the program with packets as arguments T_{d} of `direct_packets` (274/303/465); among programs whose last element is the goal 244/261/478; among programs that read four-field packets 250/273/465; with the programs of the double dagger 239/256/465, the token minimum attained by L_{f} (239/261/549, `fused_blocks_and_checkers` and the third delivery of `checked_presentation`) and by L_{c} (239/261/553, `prefix_conditional`), the node minimum by $L'_{\mathrm{f,m}}$ of `fused_blocks_and_checkers` (245/256/580) (history below)
R/$\mathcal{U}$, before `fused_blocks_and_checkers`, `prefix_conditional` and the third delivery of `checked_presentation`	245	263	465	three programs: the whole-input global program $G_{\mathrm{b}}^{\mathrm{w}}$ of `checked_presentations` (245/272/591), the whole-input program with structural binding $L_{\mathrm{m}}^{\mathrm{w}}$ of `direct_packets` (258/263/595) and its program with packets as arguments T_{d} (274/303/465); among programs whose last element is the goal 245/263/478; among programs that read four-field packets 250/273/465; with the programs of the double dagger 240/259/465, the token minimum attained by G_{b} (240/266/592) and by L_{r} of `procedure_presentation` (240/266/551), the node minimum by L_{m} of `direct_packets` (252/259/595) (history below)
R/$\mathcal{U}$, before `direct_packets`	245	268	469	three programs: the whole-input global program $G_{\mathrm{b}}^{\mathrm{w}}$ of `checked_presentations` (245/272/591), the whole-input local program with direct leaves of `whole_json_refinement` (254/268/613), and the four-field uniform terminal program of `shared_leaf` and `uniform_terminals` (272/306/469); among programs whose last element is the goal 245/268/478; among programs that read four fields 250/273/469; with the programs of the double dagger 240/261/469, the first two attained by the global stream program G_{b} of `checked_presentations` (240/266/592) and by the stream program L_{d} of `ast261` (247/261/557) (history below)
R/$\mathcal{U}$, before `ast261` and `checked_presentations`	248	268	469	three programs: the whole-input global program with direct leaves of `whole_json_refinement` (248/277/615), its local program (254/268/613), and the program of 469 bytes; with the programs of the double dagger 241/262/469, from the local stream programs of `fixed_points_and_checker` (241/267/566) and of `shared_leaf` (246/262/556)
$\mathrm{R}^{+2}/\mathcal{U}$	267	298	461	one program, $T_{+,\mathrm{p}}^{\rho}$ of `prefix_conditional`; before it 267/298/462, the one program $T_{+,\mathrm{p}}$ of `procedure_presentation`; before that 271/305/468, the one program T_{+} of `468_presentation` (delivered again by `468_shared_leaf`, `shifted_shared_leaf` and `scalar_constancy`), and before that 271/307/472, the one program of `offset472`; not comparable with R/$\mathcal{U}$
T^{+2}/g	690	773	2064	one program, the validated companion of `468_presentation`
R/$\mathcal{U}$, before Sec-	249	271	477	three programs: the global three-field profile (249/278/607) and the local three-field profile

The envelope under R/$\mathcal{U}$ has the following history. The audit of `orbit_closure` finds 256/276/506 in the corpus that precedes the packages of Section 11.9. The inferred major gives 253 tokens and 275 nodes; the truth-valued result gives 503 bytes; eager selection with the remainder selector gives 502 bytes; the reuse of the packet tail gives 251 tokens. The archive `context_order`, by allocating first, gives 273 nodes and 499 bytes; these two programs were not in the table before, and the envelope 251/273/499 that `three_field` and `constantfree` take as their baseline is attained by the global loop of `progress_independence` and by the two programs of `context_order`. Three fields give 249 tokens and 271 nodes; `constantfree` gives 250 tokens and 498 bytes with four fields; the composition (iv) of Theorem 11.97 gives 495 bytes. The three attaining programs differ in more than size. The two three-field profiles recognize arrays by attempted traversal and read certificates of a different format; their relation to their controls is Theorem 11.93. The program of 495 bytes uses the zero-product test and four fields and allocates first; its accepted stacks are not compared with those of its control (the remarks after Theorem 11.97). All three accept the same canonical goals, the closed theorems of $\mathcal{U}$, by Theorems 11.90, 11.92 and 11.97. No program attains two of the three minima 249/271/495, and no single program is below the earlier envelope 251/273/499 in all coordinates; the local three-field profile is below its own control in all three. Under T/ed the native program of `three_field` is below its control 344/393/673 in all three coordinates.

The packages of Section 11.9.11 change only the byte coordinate under R/$\mathcal{U}$. The inverted entry test of `verified_refinements` gives 492 bytes with 267 tokens and 298 nodes, below the program of 495 bytes in two coordinates and equal in the third; the truth-valued result of `checker_refinement` and `single_binder` gives 488 bytes and costs 4 tokens and 5 nodes; the anonymous function of `single_binder` gives 487 bytes and costs 9 tokens and 9 nodes. The three programs have exactly the runs of their control (Theorems 11.98 and 11.101), so their accepted stacks are those of the program of 495 bytes. The envelope is 249/271/487; no program attains two of its minima, and none is below 251/273/499 in all coordinates. The retained major of `retained_premises` (265/301/513) improves its own control of 516 bytes in two coordinates and does not change the envelope. Admitting the programs that read only the first line would give 248/270/487; those two gains come from the input framing alone (Section 11.11.1). Under T/ed the phase by an empty context is below its control in all three coordinates and the native program of `checker_refinement` in bytes only; under G/$\mathcal{A}^{-}$ the composed program of `verified_refinements` is below its control 325/375/641 in all three coordinates. Under S the one program of `guarded_register` is below its control in the scalar iterator in all three coordinates; S is not comparable with any other contract of the table. Smaller figures for T/eqd are reported by packages outside this chapter. Since T/g is a stronger contract than G/$\mathcal{U}$, the line for T/g also bounds the latter.

The packages `immediate_application`, `refinement` and `scope_and_frontier` change the envelope as follows. Under R/$\mathcal{U}$ the immediate application of Theorem 11.100(a) gives 479 bytes with 272 tokens and 304 nodes; it is below its control of 487 bytes in all three coordinates and has one token more than the program of 488 bytes. The envelope is 249/271/479, attained by three programs, and 250/273/479 among programs reading four fields. With the programs reading only the first line it would be 248/270/479, the triple that `immediate_application` states as its envelope; `scope_and_frontier` states 248/270/487, by the same convention, at the same revision without the program of 479 bytes. Its program of 500 bytes, the third delivery of the program of 488 bytes by `refinement` and the readable programs do not change the envelope, and the two-field reference is the only program under its own code. Under T/ed the immediate application of Theorem 11.100(b) gives 637 bytes with one token more than its control, and the envelope is 334/387/637. Under T/g the program on the kernel of 488 bytes and its matched control give 419/469/855; the matched control is below the wrapper of `progress_independence` in all three coordinates, and the program of 855 bytes is below that wrapper in bytes only. So the earlier envelope 421/471/869 under T/g is improved in all three

coordinates by one program, which is a control and not a delivered result; `refinement` lists both programs and does not remark on this. Under R/$\mathcal{U}$ and T/ed no program attains two of the minima, and no delivered program of the three packages is below the earlier envelope of its contract in all three coordinates, as all three packages state.

The packages of Section 11.9.13 and the second delivery of `checked_presentation` change the envelope as follows (the program of 479 bytes of `alias_back`, which states 248/270/479, is that of `immediate_application`). Under R/$\mathcal{U}$ the program of `zero_test_split` gives 478 bytes with 271 tokens and 305 nodes, one token less and one node more than its control of 479 bytes; the envelope is 249/271/478, attained by three programs, and 250/273/478 among programs reading four fields. The package states 248/270/478, admitting the first-line programs. Without the program of 478 bytes, whose last element carries an initial floor, the envelope is 249/271/479 as before; `immediate_opening` and `479_predecessor57` state 248/270/479, by the same convention as `immediate_application`. Admitting the programs marked with the double dagger, the minima would be 244/267/478; the second delivery of `checked_presentation` states 244/267/479, at a revision without the program of 478 bytes, and its token and node gains come from the framing of Theorem 11.186. `checked_frontier` and `reductions` state 248/270/487, at a revision without the program of 479 bytes. Under G/$\mathcal{A}^{-}$ the delivered programs give 607 bytes (`479_predecessor57`) and 370 nodes (`atomic_seed_refinement`), and the envelope is 324/370/607, attained by three programs; the parity program is below its control of 643 bytes in all three coordinates, and every other new program of this contract improves only the byte coordinate of the program of 629 bytes, with more tokens and nodes. Under T/ed the parity program of `atomic_seed_refinement` is below its control of 637 bytes and below the program of 646 bytes in all three coordinates and gives the byte minimum 628. The programs of 637 and 641 bytes of `equation_aliases` are below their control of 666 bytes in all three coordinates, and the program with the reader in the initial call gives the node minimum 386, one below the 387 of the programs of 654 and 628 bytes; the program of 633 bytes has two tokens and four bytes fewer than its control of 637 bytes and the same number of nodes. The envelope is 334/386/628, attained by three programs, and no program attains two of its minima. The package states 338/386/633, from its own three programs. The program of 645 bytes of `checked_frontier` improves only the byte coordinate of its control. Under T/g the strict companion of `immediate_opening` does not change the envelope. Under K the two-field stream of `alias_back` is below its matched tree control in all three coordinates; the two read different certificate formats.

`absorbing_orbit`, `absorbing_predecessor` and `boolean_constancy` change the envelope as follows. Under R/$\mathcal{U}$ the eager selector of `boolean_constancy` gives a second program of 478 bytes (274/304/478), two tokens more than its control and the same number of nodes, and incomparable with the program of `zero_test_split`; the envelope 249/271/478 is unchanged and is now attained also among programs whose last element is the goal. With the programs of the double dagger the minima remain 244/267/478; the wildcard import saves four bytes of each stream program and changes neither tokens nor nodes. `boolean_constancy` states 244/267/478 by this convention, and `absorbing_predecessor`, which delivers no new checker, 244/267/479. Under T/ed the program of `absorbing_orbit` (343/390/632) and the native eager selector (346/392/636) improve their control of 637 bytes, the first in all three coordinates and the second in bytes only; neither changes the envelope 334/386/628.

`checked_reductions`, the second delivery of `refinement`, `zero_alias`, `fused_iteration` and `pred50` change the envelope as follows. Under R/$\mathcal{U}$ the composition of the header with the eager selector gives 477 bytes with 273 tokens and 305 nodes, two tokens more than its control of 478 bytes and the same nodes; the envelope is 249/271/477, attained by three programs, and 249/271/478 among programs whose last element is the goal. With the programs of the double dagger the minima are 244/265/477; the program of `fused_iteration` lowers

the node minimum of the stream programs from 267 to 265, and that of `pred50` saves one byte and costs one node against its control. Under G/$\mathcal{A}^-$ the compositions give the envelope 321/369/596, attained by three programs; the program of 618 bytes is below the program of 629 bytes in all three coordinates, and so is the program of 622 bytes, which attains the node minimum. Under T/ed the compositions give the envelope 330/379/621, attained by two programs; the program of 641 bytes of `checked_reductions` attains two of the three minima and is below the phase by an empty context in all three coordinates, and the program of 621 bytes is below its control of 628 bytes in all three coordinates. The packages state 244/267/477 (`checked_reductions`), 330/379/623 and 323/369/600 (`checked_reductions`, for T/ed and G/$\mathcal{A}^-$), 330/380/621 and 321/369/596 (`refinement`), 249/265/605 as a single program (`fused_iteration`) and 244/267/478 unchanged (`pred50`), each at its own revision. The program of `zero_alias` (331/373/625) is the only program of T/ed$_1$; it is below the earlier envelope 334/386/628 of T/ed in all three coordinates, as the package states, but it is not below the present envelope 330/379/621 in tokens or bytes, and the two contracts encode the public zero differently (Theorem 11.120), so no program of one contract supersedes a program of the other. The program of `tolerant_certificates` (315/361/603) is the only program of Gt/$\mathcal{A}^-$; it had fewer tokens and nodes than every program of G/$\mathcal{A}^-$ and T/ed at that point (the native node profile of Theorem 11.147, with 358 nodes, has since fewer nodes) and more bytes than the program of 596 bytes, and since its contract differs from both (Theorem 11.124) it is not compared with them. The program of 596 bytes keeps its row under G/$\mathcal{A}^-$: that contract states correctness for integer and array certificates only, and the failure of termination on inputs with nonempty strings, which `tolerant_certificates` records for it, lies outside that statement, as it does for the program of 629 bytes.

The packages of Section 11.9.15 change the envelope as follows. Under R/$\mathcal{U}$ the program of 477 bytes of `head_conversion`, now listed, has one token fewer and one node more than the program of 477 bytes of `checked_reductions`. The direct leaves give 474 bytes (four fields) and, with the double dagger, 243 tokens and 264 nodes, which is the triple 243/264/474 that `computation_leaves` states against 244/265/477; the eager selector gives 476 bytes on the program of 477 bytes and 473 bytes on the program of 474 bytes, each with two more tokens than its control. The uniform terminals give 469 bytes, with two more tokens and two fewer nodes than the control of 474 bytes, and, with the double dagger, 242 tokens and 262 nodes; `shared_leaf` states 242/262/469 and `uniform_terminals`, without the program of 262 nodes, 242/263/469, both by this convention. `smaller_presentations`, at the same revision without these two packages, states 242/264/473, and `vacuous_specialization` 243/264/473. Among programs without the double dagger, the whole-input reader gives 248 tokens and 268 nodes, and the floor product on the whole-input local three-field program 269 nodes; `whole_json_refinement` states 248/268/473 and `ast269_successor_rigidity` and `selective_normalization` 249/269/477 for this group, each at its own revision. The envelope is 248/268/469, attained by three programs; no program attains two of its minima, and no single program is below the earlier envelope 249/271/477 in all coordinates: the program of 469 bytes has 272 tokens. Under G/$\mathcal{A}^-$ the bitwise depth gate and the identity test with `False` give 317/358/594, attained by three programs of `abstraction_and_refinements`, each below its control in all three coordinates except the node profile, which has the tokens of its control and more bytes; `476_byte_presentation` states 319/367/594 from its own programs. Under T/ed the same changes give 328/368/616, attained by three programs; the token profile is below its control in all three coordinates, the byte profile has the tokens of its control and fewer nodes and bytes, and the node profile has more tokens and bytes than its control; `476_byte_presentation` states 328/378/619. The direct-leaf native and strict programs of `computation_leaves` are larger than their controls. Under T/ed$_1$ the uniform-zero program of 608 bytes and the program of `zero_alias` are incomparable, and the envelope is 331/373/608; under the new code G$_1$/$\mathcal{A}^-$ the envelope is 328/369/581. `smaller_presentations` compares

these programs with 594 and 616 bytes and states the byte improvements 594 → 581 (587 without the identity with unary plus) and 616 → 608; these compare different encodings of the public zero (Theorem 11.148). In none of these contracts does one program attain all minima.

The packages of Sections 11.9.16 and 11.9.17, `fixed_points_and_checker`, `checker_refinements` and `orbit_equality` change the envelope as follows. Under R/$\mathcal{U}$ the programs without the double dagger are unchanged: those of `checker_refinements` and `orbit_equality` are programs already listed. `checker_refinements` states 249/269/476 for the whole-input programs and 244/265/476 with the stream programs, at a revision without the programs of 469 and 473 bytes, and `orbit_equality` the improvements 249 → 248 and 269 → 268 against the whole-input three-field programs. With the double dagger the token minimum becomes 241, from the stream program of `fixed_points_and_checker`, which has five tokens fewer, five nodes more and ten bytes more than its control; the package states 241/262/469 by this convention. The program of `offset472` (271/307/472) is below its control of 473 bytes in all three coordinates; it accepts the 1-shifted encodings, so it is the only program of its code. The package states the envelope 243/264/472 against 243/264/473, admitting the stream programs, at a revision without the program of 469 bytes and across the two encodings; the program has one token fewer and one node and three bytes more than the program of 469 bytes of R/$\mathcal{U}$. Under T/ed$_1$ the envelope becomes 322/364/606, attained by three programs, each of which improves one coordinate: S_1^{T} and S_1^{N} are below their T/ed controls in all three coordinates, a comparison across the encodings of zero, and S_2 and S_3 save two bytes and one byte against S_1, with one node and four nodes more and, for S_3, one token more. `scope_refinement` states 322/364/607 (against 328/368/616 of T/ed, at a revision without S_2), and `validated_leaves` the byte improvements 608 → 606 and 581 → 579. Under G$_1$/$\mathcal{A}^-$, N_2 saves two bytes against N_1 with one node more; with the programs of T/ed$_1$ the envelope is 322/364/579. In none of these contracts does one program attain all minima, and every package states this.

`468_presentation`, `ast261`, `checked_presentations` and `root_conversion_raw` change the envelope as follows. Under R^{+2}/$\mathcal{U}$ the program T_+ (271/305/468) has the tokens of P_+ and two nodes and four bytes fewer; it is below its named control of 469 bytes in all three coordinates, a comparison across the encodings. The package states the envelope 241/262/468 against 241/262/469, admitting the stream programs and across the two encodings. Under R/$\mathcal{U}$, among the programs without the double dagger, the whole-input program $G_{\mathrm{b}}^{\mathrm{w}}$ lowers the token minimum from 248 to 245; it is below its constructed control in tokens and nodes only, and below the whole-input global program of `whole_json_refinement` (248/277/615) in all three coordinates. The envelope is 245/268/469, attained by three programs, and no single program is below the earlier envelope 248/268/469 in all coordinates. With the double dagger the token minimum becomes 240, from G_{b}, which has two tokens and two nodes fewer and 27 bytes more than its control, and the node minimum 261, from L_{d}, which has one node fewer and one token and one byte more than its control; the minima are 240/261/469. `checked_presentations` states 240/262/469, at a revision without `ast261`, and `ast261` states 242/261/469, at a revision without `fixed_points_and_checker` and `checked_presentations`; both admit the stream programs. The program of 476 bytes of `root_conversion_raw` is a program already listed, and its package states 244/265/477 and 249/271/477 at a revision without the packages of Section 11.9.15. Under T/ed$_1$ the envelope becomes 319/358/605, attained by three programs, each of which improves one coordinate and is below its control in two: S_2^{T} has three tokens and three nodes fewer and four bytes more than S_1^{T}, S_2^{N} two tokens and six nodes fewer and seven bytes more than S_1^{N}, and S_4 the tokens of S_3 and two nodes and two bytes fewer. `checked_presentations` states 319/358/605 against 331/373/608, the envelope of the program of `zero_alias` and S_1. Under G$_1$/$\mathcal{A}^-$ the envelope is 319/358/579. Under T^{+2}/g there is one program. In none of these contracts does one program attain all minima, and every package states this.

`468_shared_leaf`, `shifted_shared_leaf`, the checker part of `scalar_constancy`, `direct_packets` and `procedure_presentation` change the envelope as follows. The first three deliver programs already listed (T_{+}, and L_{d} in `scalar_constancy`) and change nothing; they state 241/262/468 (`468_shared_leaf`, `shifted_shared_leaf`) and 241/261/468 (`scalar_constancy`), admitting the stream programs and across the two encodings, each at its own revision. Under $\mathrm{R}^{+2}/\mathcal{U}$ the procedure form $T_{+,\mathrm{p}}$ (267/298/462) is below T_{+} in all three coordinates, with four tokens, seven nodes and six bytes fewer; `procedure_presentation` states 240/261/462, admitting L_{r} and L'_{r} and across the two encodings. Under $\mathrm{R}/\mathcal{U}$, among the programs without the double dagger, T_{d} lowers the byte minimum from 469 to 465, with two tokens more and three nodes fewer than its control, and $L^{\mathrm{w}}_{\mathrm{m}}$ the node minimum from 268 to 263; L'^{w} has one token and one node fewer and one byte more than G^{w} and changes no minimum. The envelope is 245/263/465, attained by three programs, and no single program is below the earlier envelope 245/268/469 in all coordinates. `direct_packets` states 246/263/465 against 248/268/469 for the whole-input programs, at its revision. With the double dagger, L_{m} lowers the node minimum from 261 to 259, and L_{r} has the token minimum 240 of G_{b} with 266 nodes and 551 bytes; the minima are 240/259/465. `direct_packets` states the node minimum 259 against 262 of the stream programs. Under T/ed_1 the procedure form $S_{2,\mathrm{p}}$ lowers the byte minimum from 605 to 600 and is below its control S_2 in all three coordinates; $S^{\mathrm{T}}_{1,\mathrm{b}}$ and $S^{\mathrm{N}}_{1,\mathrm{b}}$ have two tokens and two nodes fewer and four bytes more than S^{T}_1 and S^{N}_1 and change no minimum. The envelope is 319/358/600, attained by three programs. `procedure_presentation` states 320/362/600 against 322/364/606, the envelope before `checked_presentations`. Under $\mathrm{G}_1/\mathcal{A}^{-}$ the procedure form $N_{2,\mathrm{p}}$ is below N_2 in all three coordinates, as the package states ($579 \to 573$ bytes); among the programs that validate only the goal it attains all three minima, and with the programs of T/ed_1 the envelope is 319/358/573. Under $\mathrm{R}/\mathcal{U}$, with or without the double dagger, and under T/ed_1 no single program attains all minima, and every package states this for its own programs.

`fused_blocks_and_checkers`, `prefix_conditional` and the third delivery of `checked_presentation` change the envelope as follows. Under $\mathrm{R}/\mathcal{U}$, among the programs without the double dagger, $L^{\mathrm{w}}_{\mathrm{c}}$ lowers the token minimum from 245 to 244 and is below $G^{\mathrm{w}}_{\mathrm{b}}$ in all three coordinates, and $L^{\mathrm{w}}_{\mathrm{c,m}}$ lowers the node minimum from 263 to 261 and is below $L^{\mathrm{w}}_{\mathrm{m}}$ in all three coordinates; the envelope is 244/261/465, attained by three programs, and no single program is below the earlier envelope 245/263/465 in all coordinates. With the double dagger the token minimum becomes 239, from L_{f} and L_{c}, and the node minimum 256, from $L'_{\mathrm{f,m}}$; the minima are 239/256/465. The packages state, admitting the stream programs and across the two encodings, and each at a revision without the other two: 239/256/462 against 240/259/462 (`fused_blocks_and_checkers`), 239/257/462 (`checked_presentation`), and 239/257/461, together with 244/261/461 against 245/263/462 for the programs that read the whole input (`prefix_conditional`). Under $\mathrm{R}^{+2}/\mathcal{U}$ the program $T^{\rho}_{+,\mathrm{p}}$ (267/298/461) has the tokens and nodes of $T_{+,\mathrm{p}}$ and one byte fewer. Under T/ed_1 $S^{\rho}_{2,\mathrm{p}}$ (330/372/599) lowers the byte minimum from 600 to 599 with the tokens and nodes of $S_{2,\mathrm{p}}$; the envelope is 319/358/599, attained by three programs. Under $\mathrm{G}_1/\mathcal{A}^{-}$ nothing changes: $S^{\rho}_{2,\mathrm{p}}$ satisfies this contract and has more than 573 bytes. In none of these contracts does one program attain all minima, and every package states this.

Superseded programs. A program of the table is *superseded* if another complete program of the table with the same contract code, and with the same recognition of arrays (both or neither marked with a dagger), is no larger in each of the three coordinates and smaller in at least one; programs marked with the double dagger are not compared. Under $\mathrm{R}/\mathcal{U}$ the following are superseded: the byte program of `three_field` (267/299/495, `code/checker_bytes.py`), and with it the programs that it superseded, namely the compact

program of `deterministic_introduction` (267/303/506), the two truth-valued programs of `inferred_major` (278/310/503, 278/309/505), the eager program of `frontier_review` (268/306/513), the program of 502 bytes of `checked_presentation`, `progress_independence`, `crossed_uniqueness` and `502_scope_audit` (269/301/502, `code/checker_502.py` in the last) and the eager program of 503 bytes of `progress_independence` and `crossed_uniqueness` (270/303/503), all by the inverted-entry program of `verified_refinements` (267/298/492, `compact/checker.py`); the remainder program of `smaller_envelope` (264/304/515), the remainder program of `crossed_uniqueness` and `502_scope_audit` (266/301/505, `code/checker_505.py` in the latter), the byte program of `context_order` (265/301/499) and the retained-major program of `retained_premises` (265/301/513, `code/checker_retained_bytes.py`), by the byte program of `constantfree` (264/299/498). Newly superseded are: the anonymous-function program of `single_binder` (276/308/487, `code/bytes487.py`), by the immediate application of `immediate_application` (272/304/479, `code/checker479.py`); and, by the inverted-entry program of `verified_refinements` (267/298/492, `compact/checker.py`), the program of `scope_and_frontier` with the floor recomputed from the context (267/303/500, `implementation/context_floor.py`), its readable form (296/353/1329, `implementation/readable.py`) and the readable program of `refinement` (314/381/1870, `code/checker_readable.py`). Newly superseded with the packages of Section 11.9.13 is the readable form of `immediate_opening` (306/367/1995, `code/checker_readable.py`), by the immediate application (272/304/479). The programs of 478, 479, 488 and 492 bytes (the programs of 478 bytes being that of `zero_test_split`, `checker/code/compact478.py`, and that of `boolean_constancy`, `checkers/code/checke` 274/304/478; the program of 479 bytes that of `immediate_application`, `code/checker479.py`, of `immediate_opening`, `code/checker479.py`, of the second delivery of `checked_presentation`, `code/checker_bytes.py`, and of `479_predecessor57`, `checker/code/compact479.py`; the program of 488 bytes that of `checker_refinement`, `single_binder` and `refinement`, `code/checker_compact` in the last) are not superseded: each pair of them is incomparable (the program of 478 bytes of `zero_test_split` has one node more than the programs of 479 and 488 bytes and more tokens than that of 492 bytes; that of `boolean_constancy` has more tokens than the programs of 479, 488 and 492 bytes and than that of `zero_test_split`, which has one node more), the program of 487 bytes has more tokens than each of them, and every other program of the contract has more than 492 bytes. The composition of `checked_reductions` (273/305/477, `checkers/compact477.py`) is not superseded and supersedes none of them: it has more tokens than the program of 478 bytes of `zero_test_split` and the programs of 479, 488 and 492 bytes, and one node more than that of `boolean_constancy`. The stream programs of the second delivery of `checked_presentation` (244/272/608, `code/checker_tokens.py`; 249/267/606, `code/checker_ast.py`) are below the first-line programs of `single_binder` in all three coordinates, but programs with the double dagger are not compared, and neither group is listed as superseding or superseded; the same holds for the stream programs with the wildcard import of `boolean_constancy` (244/272/604, `checkers/code/checker244.py`; 249/267/602, `checkers/code/checker267.py`) and for the rewritten stream programs of `fused_iteration` (249/265/605, `code/checker_refinement/checker.py`) and `pred50` (244/273/607, `code/checker/checker607.py`). Under T/ed_1 there is one program besides its readable form, which it supersedes (`zero_alias`, 430/499/2322, `implementation/checker_readable.py`). Under $\mathrm{Gt}/\mathcal{A}^-$ there is one program, and nothing is superseded. The global loop of `progress_independence` ($251/283/605^\dagger$), by the token program of `constantfree` ($250/283/603^\dagger$); the loop profile of `deterministic_opening` ($263/276/580^\dagger$), by the loop profile of `context_order` ($262/273/572^\dagger$); and the global loop of `kernel_refinement` and `validated_core` ($256/283/594^\dagger$), by the token profile of `uniform_progress` ($256/283/593^\dagger$). The local loop with inferred major of `inferred_major`, `smaller_envelope` and `retained_premises` ($257/275/607^\dagger$) is below the local three-field profile ($255/271/606^\dagger$) under the same contract but in a different certificate format; no four-field program supersedes it. Under T/ed the phase by an empty con-

text of `verified_refinements` (334/387/654, `strict/checker.py`) supersedes the native program of `three_field` (342/391/666, `code/checker_native.py`), which superseded the typed native checker of `checked_presentation` (344/393/673); the earlier programs of that contract, of `validated_core` (390/454/878), `validated_presentation` (370/426/849) and `scope_kernel` (382/446/862), were already superseded by the latter. Newly superseded under T/ed, by the parity program of `atomic_seed_refinement` (341/387/628, `code/native_strict_checker.py`), are: the native program of `checker_refinement` (343/392/646, `code/checker_native.py`); the native immediate application (344/392/637), delivered as `code/checker637.py` by `immediate_applicatio` as `code/checker_strict.py` by the second delivery of `checked_presentation` and as `checker/code/strict6` by `479_predecessor57`; and the strict program of `checked_frontier` (348/396/645, `checkers/strict645_la` which the program of 637 bytes also supersedes. The same program of 628 bytes supersedes the product-of-bits program of `equation_aliases` (342/392/633, `strict/checkers/bytes633.py`) and the control constructed by that package (351/403/666, `strict/controls/context_phase_strict666.py`) the program of `absorbing_orbit` (343/390/632, `checker/checkers/strict632.py`) and the native eager selector of `boolean_constancy` (346/392/636, `checkers/code/checker636.py`). With the compositions of Theorem 11.118 the following are newly superseded under T/ed: the phase by an empty context of `verified_refinements` (334/387/654, `strict/checker.py`), by the programs of `checked_reductions` (330/379/641, `checkers/strict330.py`) and of `refinement` (330/380/638, `checkers/code/strict_size.py`); the parity program of `atomic_seed_refineme` (341/387/628, `code/native_strict_checker.py`), by the program of `checked_reductions` with the gate (340/385/623, `checkers/strict623.py`) and by that of `refinement` (340/385/621, `checkers/code/strict_bytes.py`); the program of `equation_aliases` with the reader in the initial call (342/386/641, `strict/checkers/ast386.py`), by the programs of 641, 623 and 621 bytes; and the program of 623 bytes of `checked_reductions`, by the program of 621 bytes, which has the same tokens and nodes. The programs of 637 bytes of `equation_aliases` (338/390/637, `strict/checkers/tokens338.py`), of 641 and 638 bytes and of 621 bytes are not superseded: each pair of them is incomparable (the program of 637 bytes has fewer bytes than those of 638 and 641 bytes and fewer tokens than that of 621 bytes; the programs of 641 and 638 bytes have the same tokens and differ in nodes and bytes in opposite directions; both have fewer tokens than that of 621 bytes, which has fewer bytes). Under T/g the strict guarded companion of `immediate_opening` (558/635/1761, `code/checker_strict.py`) is newly superseded by the matched control of `refinement` (419/469/862) and by the program of `refinement` on the kernel of 488 bytes (423/474/855). Further under T/g the typed wrapper of `retained_premises` (421/474/889, `code/checker_retained_strict.py`), the strict guarded wrapper of `single_binder` (556/633/1544, `code/strict_guarded.py`) and the readable program of `502_scope_audit` (605/700/2629, `code/checker_readable.py`) are superseded, together with the wrappers of `frontier_review` (462/524/1183) and of `checked_presentation` and `crossed_uniqueness` (463/519/1172) and the readable program of `smaller_envelope` (580/666/2473), by the wrapper of `progress_independence` (421/471/869, `checker/code/checker_strict.py`); that wrapper is now superseded, and with it all of these, by the matched control of `refinement` (419/469/862, `baseline/checker_strict_862.py`). The program of `refinement` on the kernel of 488 bytes (423/474/855, `code/checker_strict.py`) and that control are incomparable, and neither is superseded. Under R/$\mathcal{U}^{\mathrm{Tr}}$ there is one program. Under G/$\mathcal{U}$ the goal-validated program of `kernel_refinement`, `compact_update` and `validated_core` (489/560/1201) is superseded by that of `checked_refinement` (407/459/921). Under G/$\mathcal{A}^{-}$ the composed program of `verified_refinements` (324/374/629, `native/checker.py`) supersedes the refined native validator of `retained_premises` (347/400/700, `code/checker_native_validat` the one-invocation list variant of `context_phase` (325/375/641), the programs of 643 bytes of `inferred_major` and `context_phase` (327/375/643, equal sizes, neither superseding the other) and the variant of `context_phase` with the explicit type test (327/375/646), which its companion of 643 bytes already superseded; earlier, the programs of `checked_refinement`

(358/413/788), `uniform_recursion` (340/385/685, 336/381/661) and `one_body` (357/412/778, 357/415/777) were superseded by the program of 656 bytes of `deterministic_introduction`, which the programs of 643 bytes supersede. The program of 629 bytes shares the qualification of the programs of 641 and 643 bytes on its exact-integer test (Section 11.12) and not the qualification of the list variant on repeated invocation, so it is counted as superseding all of them. Newly superseded under G/$\mathcal{A}^{-}$ are the anonymous-function program of `checked_frontier` (335/383/622, `checkers/native_append_lambda.py`), by the program of the second delivery of `checked_presentation` (334/381/614, `code/checker_native.py`) and by the function program of `reductions` (332/381/621, `checker/checker621.py`); and the anonymous-function program of `reductions` (337/385/620, `checker/checker620.py`), by the program of 614 bytes and by the program of `479_predecessor57` (337/383/607, `checker/code/native607.py`). The parity program of `atomic_seed_refinement` (326/370/636, `code/native_goal_checker.py`) is below the programs of 643 bytes of `inferred_major` and `context_phase` and the variant with the explicit type test (327/375/646), which were already superseded. With the compositions of Theorem 11.118 the following are newly superseded under G/$\mathcal{A}^{-}$: the composed program of `verified_refinements` (324/374/629, `native/checker.py`), by the parity program of `checked_reductions` and `refinement` (323/369/622, `checkers/native323.py` in the first and `checkers/code/native_ast.py` in the second) and by the parity and product program of `refinement` (321/371/618, `checkers/code/native_tokens.py`); the parity program of `atomic_seed_refinement` (326/370/636, `code/native_goal_checker.py`), by the program of 622 bytes; the function program of `checked_frontier` (330/379/623, `checkers/native_append_function.py`), by the programs of 622 and 618 bytes; the function program of `reductions` (332/381/621, `checker/checker621.py`), by the program of 618 bytes; the program of the second delivery of `checked_presentation` (334/381/614, `code/checker_native.py`), by the parity and product program of `refinement` on the program of 607 bytes (334/380/596, `checkers/code/native_bytes.py`); and the program of `479_predecessor57` (337/383/607, `checker/code/native607.py`), by the parity program of `checked_reductions` on it (336/378/600, `checkers/native600.py`) and by the program of 596 bytes. The four programs of 618, 622, 596 and 600 bytes are not superseded: each pair of them is incomparable (by tokens $321 < 323 < 334 < 336$, the nodes are 371, 369, 380, 378 and the bytes 618, 622, 596, 600). Under T/eqd the programs of `validated_core` (401/468/897) and `scope_kernel` (393/460/881) are superseded by the program of `validated_presentation` (383/442/870), whose lexical filter of 885 bytes has no reported token or node count and is not compared. Under S there is one program. Under K the two-field stream of `alias_back` (609/699/1644) is below its tree control (629/732/1715) in all three coordinates; the two read different certificate formats, and, as for the three-field programs under R/$\mathcal{U}$, neither is listed as superseding or superseded. Under R/$\mathcal{A}$, which the new packages do not touch, the superseded programs are the three programs of `certified_zero` (264/308/566 by the explicit program of `uniform_premise`, 262/303/548; 264/310/558 by the zero-product program of `uniform_premise`, 263/308/537; 262/296/598† by the traversal profile of `prevalidated_elimination`, 261/292/576†); the seven programs of `uniform_zero`, by the zero-product program of `scope_and_closure` (267/309/528); the three programs of `monus_and_neutral_major`, by the corresponding programs of `prevalidated_elimination`; the explicit program of `neutral_scope` (266/308/537) and the zero-product program of `prevalidated_elimination` (263/309/539), by the zero-product program of `uniform_premise`. Several superseded programs are matched controls of later rows; the table keeps their figures. A file name identifies a program within its package as delivered; it does not assert that the file is kept among the artifacts of the repository, from which the files of several superseded programs have been removed, among them those of the programs of 646 bytes of `checker_refinement`, of the three deliveries of the program of 637 bytes, of the programs of 622 and 645 bytes of `checked_frontier` and of the program of 620 bytes of `reductions`.

With the packages of Section 11.9.15, programs that read four-field packets are compared whether or not their last element carries a floor, whether or not computations close goals and whether or not every call removes a packet, since their accepted canonical goals are the same; three-field and four-field programs are not compared, as before. Newly superseded under R/$\mathcal{U}$, by the four-field uniform terminal program (272/306/469, `code/checker_bytes.py` in `shared_leaf` and in `uniform_terminals`), are: the program of 477 bytes of `head_conversion` (272/306/477, `code/compact477.py`); the program of 476 bytes (274/306/476; `compact/compact476.py` of `476_byte_presentation`, `compact476/code/compact476.py` of `export_specialization`, `code/checkers/compact_checker.py` of `abstraction_and_refinements`); and the program of 473 bytes (272/308/473; `code/checker_bytes.py` of `smaller_presentations`, `compact/checker473.py` of `vacuous_specialization`). The program of 474 bytes of `computation_leaves` (270/308/474, `code/checker_bytes.py`) is not superseded: it has fewer tokens than every program with fewer bytes. The programs of 477 bytes of `checked_reductions`, of 478 bytes, of 479, 488 and 492 bytes are not superseded by the program of 469 bytes, since each has fewer tokens or fewer nodes. Among the programs marked with a dagger, the floor-product program (255/269/605†, `checker/checker_ast269.py` of `ast269_successor_rigidity`, `compact/code/checker269.py` of `selective_normalization`) supersedes the whole-input local three-field program of `three_field` (255/271/606†, `code/checker_ast.py`); the two whole-input programs of `whole_json_refinement` (248/277/615†, `code/checker_tokens248.py`; 254/268/613†, `code/checker_ast268.py`) are not superseded and supersede no other three-field program. The new stream programs carry the double dagger and are not compared. Under G/$\mathcal{A}^-$ the programs of `abstraction_and_refinements` supersede: the parity and product program of `refinement` on the program of 629 bytes (321/371/618, `checkers/code/native_tokens.py`), by the token profile (317/364/617, `code/checkers/nativ` the parity program of `checked_reductions` and `refinement` (323/369/622), by the node profile (321/358/622, `code/checkers/native_ast_checker.py`); the programs of 596 and 600 bytes of `refinement` and `checked_reductions` (334/380/596, `checkers/code/native_bytes.py`; 336/378/600, `checkers/native600.py`), by the byte profile (332/373/594, `code/checkers/native_bytes_ch` the three native programs of `476_byte_presentation` (319/369/622, `implementation/native_tokens.py`, by the token profile; 321/367/626, `implementation/native_ast.py`, by the node profile; 332/379/594, `implementation/native_bytes.py`, by the byte profile); and the native program of `computation_leaves` (334/384/600, `code/checker_native.py`), by the byte profile. The three profiles are pairwise incomparable. Under T/ed: the phase by an empty context of `checked_reductions` (330/379/641, `checkers/strict330.py`) and of `refinement` (330/380/638, `checkers/code/strict_size.py`), the program of 637 bytes of `equation_aliases` (338/390/637, `strict/checkers/tokens338.py`) and the program of 642 bytes of `476_byte_presentation` (328/378/642, `implementation/strict_size.py`) are superseded by the strict token profile (328/377/634, `code/checkers/strict_tokens_checker.py`); the program of 621 bytes of `refinement` (340/385/621, `checkers/code/strict_bytes.py`) and the strict program of `computation_leaves` (340/389/625, `code/checker_strict.py`) by the strict byte profile (340/382/616, `code/checkers/strict_bytes_checker.py`), the latter also by the program of 619 bytes. The programs 328/377/634, 335/368/643 (`code/checkers/strict_ast_checker.py`), 340/382/616 and the program of 619 bytes of `476_byte_presentation` (338/384/619, `implementation/stric` are pairwise incomparable. Under T/ed$_1$ the program of 608 bytes (`code/checker_strict.py` of `smaller_presentations`) and that of `zero_alias` are incomparable; under G$_1$/$\mathcal{A}^-$ the programs of 581 and 587 bytes (`code/checker_native.py`, `code/checker_native_portable.py`) are incomparable. The checkers of other calculi are not compared.

With the packages of Sections 11.9.16 and 11.9.17, newly superseded under T/ed$_1$ are: S_3 of `scope_refinement` (335/382/607, `code/strict_bytes.py`), by S_2 of `validated_leaves` (334/379/606, `code/checker_strict606.py`); and the readable form of S_3 (421/503/2152, `code/readable_strict.py`), by every program of the code except the readable form of `zero_alias` (430/499/2322), with which it is incomparable. The program of `zero_alias`

(331/373/625) is not superseded: S_1^{T} has one byte more, S_1^{N} ten bytes more, and S_2 and S_1 more tokens. S_1 (334/378/608) is not superseded, since S_2 has one node more. The programs S_1^{T} (322/373/626), S_1^{N} (329/364/635), S_2, S_1 and that of `zero_alias` are pairwise incomparable. Under $\mathrm{G}_1/\mathcal{A}^-$ the program N_2 (328/373/579, `code/checker_native579.py`) has fewer bytes and more nodes than the programs of 581 and 587 bytes, and the three are pairwise incomparable. Under $\mathrm{R}^{+2}/\mathcal{U}$ there is one program. Under $\mathrm{R}/\mathcal{U}$ nothing is newly superseded: the program of `fixed_points_and_checker` carries the double dagger, and the programs of `checker_refinements` and `orbit_equality` are programs already listed, with their status above.

With `468_presentation`, `ast261`, `checked_presentations` and `root_conversion_raw`, newly superseded are: under $\mathrm{R}^{+2}/\mathcal{U}$, P_+ of `offset472` (271/307/472, `offset472.py`, delivered again as `code/reference_offset472.py` by `468_presentation`) and the readable form of T_+ (327/391/1572, `code/checker_readable.py`), by T_+ (271/305/468, `code/compact468.py`); under $\mathrm{R}/\mathcal{U}$, among the three-field programs marked with a dagger, the whole-input global program of `whole_json_refinement` (248/277/615†, `code/checker_tokens248.py`, and its second delivery `checkers/whole_json_tokens.py` of `orbit_equality`) and the whole-input global three-field program of `three_field` (249/278/607†, `code/checker_tokens.py`), each by $G_{\mathrm{b}}^{\mathrm{w}}$ (245/272/591†, `compact/code/checker245_whole.py`) and by its control G^{w} (247/274/564†, `compact/controls/checker247_whole.py`); and, among the four-field programs, the readable expansion of `root_conversion_raw` (307/362/1847, `compact/readable_checker.py`), by the program of 469 bytes and by every other compact four-field program of the table with fewer than 307 tokens and 362 nodes. The program of 476 bytes of `root_conversion_raw` (`compact/checker476.py`) is the program (b) of Theorem 11.146, already superseded by the program of 469 bytes. $G_{\mathrm{b}}^{\mathrm{w}}$ and G^{w} are incomparable and not superseded; the whole-input local program of `whole_json_refinement` (254/268/613†) and the floor-product program (255/269/605†) have fewer nodes than both and remain unsuperseded. The stream programs G_{b} and L_{d} carry the double dagger and are not compared. Under T/ed_1 nothing is newly superseded: S_2^{T} has more bytes than S_1^{T} and the program of `zero_alias`, S_2^{N} more bytes than S_1^{N}, and S_4 more tokens and nodes than S_2 and S_1 and more tokens than S_1^{T}, S_1^{N} and the program of `zero_alias`; it is below S_3, which S_2 already supersedes. The three new programs are not superseded, since each attains a minimum of the code, and the eight programs S_2^{T}, S_2^{N}, S_4, S_1^{T}, S_1^{N}, S_2, S_1 and that of `zero_alias` are pairwise incomparable. Under $\mathrm{T}^{+2}/\mathrm{g}$ there is one program.

With `468_shared_leaf`, `shifted_shared_leaf`, `scalar_constancy`, `direct_packets` and `procedure_presentation`, newly superseded are the following. Under $\mathrm{R}^{+2}/\mathcal{U}$, by $T_{+,\mathrm{p}}$ (267/298/462, `code/checker_compact462.py` of `procedure_presentation`): T_+ (271/305/468; `code/compact468.py` of `468_presentation`, and its deliveries with identical text `code/checker468.py` of `468_shared_leaf`, `code/checker.py` of `shifted_shared_leaf`, `checkers/code/checker_bytes468.py` of `scalar_constancy` and `controls/offset_shared468.py` of `procedure_presentation`), and the readable form of T_+ of `shifted_shared_leaf` (305/363/1933, `code/readable_checker.py`), which T_+ already supersedes. Under $\mathrm{R}/\mathcal{U}$, among the four-field programs, the eager mode selector of `boolean_constancy` (274/304/478, `checkers/code/checker478.py`), by T_{d} (274/303/465, `code/checkers/checker.py` of `direct_packets`), which has the same tokens; T_{d} supersedes no other program not already superseded, since every other four-field program with at least 274 tokens and 303 nodes was superseded before, and it is not superseded, since it has the byte minimum. The programs of 469, 474, 477 bytes (`checked_reductions`), 478 bytes (`zero_test_split`), 479, 488 and 492 bytes have fewer tokens than T_{d} and remain unsuperseded. Among the three-field programs with a dagger nothing is newly superseded: L'^{w} (`code/checkers/checker_tokens_json.py`) has one byte more than G^{w} and one token more than $G_{\mathrm{b}}^{\mathrm{w}}$, and $L_{\mathrm{m}}^{\mathrm{w}}$ (`code/checkers/checker_ast_json.py`) has more tokens than the whole-input local program of `whole_json_refinement` (254/268/613) and the

floor-product program (255/269/605); the six programs $G^{\mathrm{w}}_{\mathrm{b}}$, G^{w}, L'^{w}, $L^{\mathrm{w}}_{\mathrm{m}}$, 254/268/613 and 255/269/605 are pairwise incomparable. L_{m}, L_{r} and L'_{r} carry the double dagger and are not compared. Under T/ed_1, by $S_{2,\mathrm{p}}$ (330/372/600, `code/checker_strict_procedure.py`): S_2 (334/379/606, `code/checker_strict606.py` of `validated_leaves`, delivered again as `controls/checker_strict606.py`), S_1 (334/378/608, `code/checker_strict.py` of `smaller_presentations`) S_4 (335/380/605, `code/strict_bytes.py` of `checked_presentations`) and the program of `zero_alias` (331/373/625, `implementation/alias_inverted.py`); and $S^{\mathrm{T}}_{1,\mathrm{b}}$ (320/371/630, `code/checker_strict_tokens_iterable.py`) is superseded on delivery by S^{T}_2 (319/370/630), which has one token and one node fewer and the same bytes. The six programs S^{T}_2, S^{N}_2, S^{T}_1, S^{N}_1, $S^{\mathrm{N}}_{1,\mathrm{b}}$ (327/362/639, `code/checker_strict_ast_iterable.py`) and $S_{2,\mathrm{p}}$ are pairwise incomparable: by tokens $319 < 322 < 327 = 327 < 329 < 330$, with the nodes 370, 373, 358, 362, 364, 372 and the bytes 630, 626, 642, 639, 635, 600. Under $\mathrm{G}_1/\mathcal{A}^-$, by $N_{2,\mathrm{p}}$ (324/366/573, `code/checker_native_procedure.py`): N_2 (328/373/579, `code/checker_native579.py` of `validated_leaves`) and the program of 581 bytes of `smaller_presentations` (328/372/581, `code/checker_native.py`), which share its test "a is $+a$". The program of 587 bytes (328/369/587, `code/checker_native_portable.py`) is larger in all three coordinates, but its exclusion of `True` does not depend on the runtime, and it is not counted as superseded.

With `fused_blocks_and_checkers`, `prefix_conditional` and the third delivery of `checked_presentatio` programs that read four-field packets are compared whatever the order of the four fields, since ρ preserves the runs (Theorem 11.164(d)), and three-field programs whatever their allocation of fresh names, since the reservation at return accepts the same canonical goals (Theorem 11.144). Newly superseded are the following. Under $\mathrm{R}^{+2}/\mathcal{U}$: $T_{+,\mathrm{p}}$ (267/298/462, `code/checker_compact462.py` of `procedure_presentation`, reproduced as `controls/raw_whole_bytes462` `checkers/controls/control_bytes462.py` and `controls/compact_bytes462.py` by the three packages), by $T^{\rho}_{+,\mathrm{p}}$ (267/298/461, `code/checker_bytes461.py`). Under $\mathrm{R}/\mathcal{U}$, among the three-field programs with a dagger, by $L^{\mathrm{w}}_{\mathrm{c}}$ (244/269/547†, `code/checker_tokens244.py`): $G^{\mathrm{w}}_{\mathrm{b}}$ (245/272/591†, `compact/code/checker245_whole.py`, again `code/compact_whole.py`), G^{w} (247/274/564†, `compact/controls/checker247_whole.py`) and L'^{w} (246/273/565†, `code/checkers/checker` by $L^{\mathrm{w}}_{\mathrm{c,m}}$ (252/261/593†, `code/checker_ast261.py`): $L^{\mathrm{w}}_{\mathrm{m}}$ (258/263/595†, `code/checkers/checker_ast_json.` and the whole-input local program of `whole_json_refinement` (254/268/613†, `code/checker_ast268.py`, delivered again with identical text as `code/checkers/control_ast268_json.py` of `direct_packets` and `checkers/whole_json_ast.py` of `orbit_equality`); and by both the floor-product program (255/269/605†, `checker/checker_ast269.py` of `ast269_successor_rigidity`, `compact/code/checke` of `selective_normalization`, `checkers/ast269/checker269.py` of `checker_refinements`). Every earlier three-field program with a dagger is thereby superseded, and $L^{\mathrm{w}}_{\mathrm{c}}$ and $L^{\mathrm{w}}_{\mathrm{c,m}}$ are incomparable. The four-field programs are unchanged. The stream programs L_{f}, L_{c}, $L_{\mathrm{f,m}}$, $L'_{\mathrm{f,m}}$ and $L_{\mathrm{c,m}}$ carry the double dagger and are not compared; $L'_{\mathrm{f,m}}$ is below $L_{\mathrm{f,m}}$ and $L_{\mathrm{c,m}}$, and L_{f} below L_{c}, in all coordinates in which they differ. Under T/ed_1: $S_{2,\mathrm{p}}$ (330/372/600, `code/checker_strict_procedure.py` of `procedure_presentation`, reproduced as `controls/strict_bytes600.py`), by $S^{\rho}_{2,\mathrm{p}}$ (330/372/599, `code/checker_strict599.py`), with the program with comments of the same syntax tree (`code/checker_strict_annotated.py`), which has no row. The six programs S^{T}_2, S^{N}_2, S^{T}_1, S^{N}_1, $S^{\mathrm{N}}_{1,\mathrm{b}}$ and $S^{\rho}_{2,\mathrm{p}}$ are pairwise incomparable, by the comparison above with 599 bytes in place of 600.

11.12 Further questions

The packages state the following questions, limits and unproved claims. Items that several packages raise are merged.

Question 11.188 (Verification of the checkers)**.** Formalize the sorted reading, the opening lemma (Theorem 11.11), the zero-instance supply (Theorems 11.2 and 11.18), the in-

variants of successful runs and the compilation of derivations, for the actual programs on raw input and not for a model restricted to canonical input, and connect them to the derivability relation of the Lean development. All soundness and completeness proofs of this chapter are written proofs; no package verifies a program in a proof assistant. A smaller target is an operational semantics of the fragment of the host language used by the programs, enough to certify the simulations of Sections 11.9.11 and 11.9.12 (consumption of packets, order of evaluation, immutability of contexts, the boundary of scopes, the binding of assignment expressions in comprehensions and anonymous functions); a suite of fixtures is not a substitute. A formal simulation of the raw codec, the input stream, the two phases of scope and the public grammar would also make later changes easier to assess; formal equivalences between calculi do not verify the control flow of a program. (`certified_zero`, `uniform_zero`, `neutral_scope`, `kernel_refinement`, `compact_update`, `validated_presentation`, `checked_refinement`, `scope_kernel`, `checker_refinement`, `immediate_applica` `refinement`, `scope_and_frontier`, and the packages of Section 11.9.13 and the second delivery of `checked_presentation`, none of which formalizes the reader, the framing of the input or the order of evaluation on which Theorems 11.109 and 11.186 rely.)

Question 11.189 (One program for all coordinates)**.** Is there, under a fixed contract, one complete program that is below the current minima in tokens, syntax-tree nodes and bytes simultaneously, and in particular one that recognizes arrays by an exact test? (`certified_zero`, `prevalidated_elimination`, `kernel_refinement`, `checked_refinement`.)

Question 11.190 (Lower bounds)**.** Is there a class of presentations, with a fixed implementation language, input contract, treatment of malformed input and charged dependencies, for which a lower bound on checker size can be proved? The recorded sizes are upper bounds attained by particular programs, and a census of existing programs is not a lower bound. A benchmark of this kind should report the size of certificates and the work of checking together with the size of the checker; `three_field` adds understandable source and resource usage to such a comparison, since the present measure does not order mathematical concepts uniquely. (`uniform_zero`, `neutral_scope`, `validated_presentation`, `scope_and_closure`, `three_field`.)

Question 11.191 (Removing a rule and shrinking a validated checker)**.** Can a rule or a term constructor be removed, with every omitted instance recovered and both formula round trips of Theorem 1.119 preserved, in such a way that a fully validated checker under a fixed contract also becomes smaller? The packages of this chapter re-encode an unchanged basis. (`uniform_zero`, `neutral_scope`, `compact_update`, `kernel_refinement`, `validated_presentation`, `checked_refinement`.)

Question 11.192 (Constructed majors)**.** Can the construction of the major from the fields of the packet (Theorem 11.26) be obtained without the six additional bytes of source that it costs against a matched control, and without an unaccounted condition on witnesses or a changed induction rule? (`uniform_zero`.)

Question 11.193 (Definitional zero computation)**.** Is there a layout of binders in which the reduction of $I(0, a; v.\, h)$ to a needs no lowering of indices, so that the zero computation can be made definitional at a lower total cost than the dispatch it removes? (`uniform_zero`, `prevalidated_elimination`.)

Question 11.194 (Closures as a separate kind of resource)**.** Can closures be represented so that a certificate cannot forge them and a raw tree does not read as a closure in one use and as an object formula in another, at a lower total cost, so that one simpler dispatcher serves all eliminations and the introduction? Theorem 11.52 shows that the present identification forces the test on the root of the matrix. (`kernel_refinement`, `checked_refinement`.)

Question 11.195 (A checkable class for the closure criterion)**.** Is there a class of matrices that satisfies the criterion of Theorem 11.173, is cheaper to validate than the guard (11.1), and still gives full expressiveness of the public language? Can a root-determined introduction coexist with the full language with equality, Q and D under the ordinary readings, at a lower total cost than the witness-selected rule? The equation $y = S0$ excludes the direct replacement of cases by progress, not every redesign. (`compact_update`, `checked_refinement`.)

Question 11.196 (Scope carried by the certificate)**.** Would a tree-shaped certificate that allocates opaque eigenparameters when a binder is entered, so that raw input cannot name them, simplify the freshness argument at a lower total size? Can the remaining length of the certificate serve as the floor if the allocation is given a separate phase? (`compact_update`, `prevalidated_elimination`.)

Question 11.197 (Terms as atomic assertions)**.** Can an equation $s = t$ be replaced by an assertion $\delta(s, t) = 0$ for a fixed distance term δ at a lower total trusted cost, when the expansion of δ and the proofs of reflection and of the round trips are counted? (`validated_presentation`.)

Question 11.198 (Whole-certificate typing during the kernel run)**.** Theorems 11.59 and 11.65 let the first opening of the kernel check the scope and the grammar of the goal. Can the exact typing of the whole certificate, including unused packets, be shared with the kernel in the same way? (`kernel_refinement` asks for the sharing of public syntax validation with the opening traversal; for the goal this is answered by the results of Section 11.8.)

Question 11.199 (The envelope of Section 11.11.2)**.** Is there one complete program under the raw contract for $\mathcal{U}$ that is below 248 tokens, 268 nodes and 469 bytes at once (or, with the framing by lines, 241/262/469), or at least one that attains two of these minima? The same question was asked by the packages of Section 11.9.15 for the earlier envelope 249/271/477 and its variants; each of them states that it does not achieve it. In particular, can one three-field program improve all three coordinates of the earlier envelope 251/273/499, and is there a three-field byte program that allocates first and selects eagerly? The records of 477, 478, 479, 488 and 492 bytes still read four fields, the token and node gains of three fields are offset by longer byte encodings, and the byte gains of Section 11.9.11 cost tokens and nodes relative to the program of 495 bytes, except the inverted entry test, which costs neither; the immediate application improves its own control in all three coordinates but not the program of 488 bytes in tokens, and the initial floor of Theorem 11.116 saves a token and a byte and costs a node. The framing of Theorem 11.186 lowers tokens and nodes of the three-field programs but changes the accepted inputs. The same question is open under complete input validation, where the envelope 328/368/616 under T/ed is attained by three programs, and the envelope 419/469/855 under T/g by two programs, and under the goal contract, where 317/358/594 under G/$\mathcal{A}^-$ is attained by three programs; under T/ed$_1$ (322/364/606) and G$_1$/$\mathcal{A}^-$ (322/364/579) by three programs each. Under the shifted encoding of Theorem 11.149 there is one program (271/307/472); can the offset be combined with the uniform terminals of the program of 469 bytes? No package answers these questions. (`inferred_major`, `smaller_envelope`, `frontier_review`, `checked_presentation`, `progress_independence`, `three_field`, `constantfree`, `retained_premises`, `checker_refinement`, `single_binder`, `502_scope_audit`, `verified_refinements`, `immediate_application`, `refinement`, `scope_and_frontier`, `checked_presentation`, `479_predecessor57`, `immediate_opening`, `zero_test_split`, `checked_frontier`, `atomic_seed_refinement`, `reductions`, `equation_aliases`, `alias_back`, `absorbing_orbit`, `absorbing_predecessor`, `boolean_constancy`, `zero_alias`, `checked_reductions`, `fused_iteration`, `pred50`, `offset472`, `checker_refinements`, `fixed_points_and_checker`, `scope_refinement`, `validated_leaves`, `orbit_equality`.)

Question 11.200 (Soundness without the alphabet precondition)**.** Is the program of 596 bytes, or any program of the goal contract with the array test $a \cdot 0 \neq 0$, sound for every input

accepted by the JSON reader? An audit in `tolerant_certificates` sketches a successful-run induction for the program of 596 bytes and finds no accepted false goal in 44416 bounded trials (1448 of them ending at the recursion limit), but proves no theorem; such a program would still not terminate on some inputs with nonempty strings. Can the erasure of Theorem 11.124 be combined with the products of bits and the program of 596 bytes so that one program of the tolerant contract is also below 596 bytes? (`tolerant_certificates`.)

Question 11.201 (The defined alias)**.** Can the identification of Theorem 11.120 reduce the compact programs, whose goal grammar is not validated, so that the saving in the validator does not disappear? Can a validator recognize exactly the image of the canonical encoder, including the absence of references to dummy slots, so that completeness holds for every accepted spelling? Can the soundness of runs, with the reserved constants and the order of contexts, be mechanized against the programs? (`zero_alias`.)

Question 11.202 (Retained majors with a global floor)**.** Is there a sound program with a global floor and retained majors? The proof of Theorem 11.74 uses that both openings of (3r) are evaluated at one floor; a global-floor program would have to fix both instances before the call on the minor premise or receive another argument. Can the append property of Theorem 11.80 be extended from compiler-normal certificates to arbitrary accepted runs, and can the translation of certificates of the control be made total on aliased certificates? The package delivers no global program and claims neither extension. Theorems 11.76 and 11.77 justify the delayed opening of the target after a minor proof at a lower floor, through the existential closure of the target; the other branches of such a program, the repair of collisions, the invariants of a global floor and completeness remain to be proved, and `checked_frontier` delivers no such program. (`retained_premises`, `checked_frontier`.)

Question 11.203 (A measure invariant under representation)**.** How are encoded programs, generic decoders admitted to the trusted runtime, the framing of the input and the success message to be charged, so that the measure does not reward a change of representation of one checking procedure (Section 11.11.1)? A measure that the packed programs of `502_scope_audit` do not beat, and under which the one-line framing of `single_binder` is neutral, would make the comparison of Section 11.11 independent of these choices. `scope_and_frontier` proposes to publish separate envelopes for canonical input, for validated public goals and for validated arbitrary input, and to identify a smaller reader or a change of framing separately from a new presentation. The framing by lines of Theorem 11.186 and the initial floor carried by the goal in Theorem 11.116 are further changes of the interface that the measure rewards. (`502_scope_audit`, `single_binder`, `orbit_closure`, `scope_and_frontier`, `checked_presentation`, `zero_test_split`.)

Question 11.204 (Reductions that are not rearrangements)**.** Can a new calculus, rather than a rearrangement of expressions of the host language, reduce the fully validated programs of 628 and 654 bytes (the program of 628 bytes, like those of 637 and 645 bytes, rewrites the program of 646 bytes)? Two directions are named. Focused use of a clause, fused with the immediate opening of its paired conclusion, could remove an arbitrary paired major; the unresolved case is an introduction by progress that is immediately consumed by an elimination at an arbitrary counter, for which ordinary β-reduction does not supply the missing induction. An interpretation on a proper domain or with several sorts is permitted by the task, but its domain predicate, its interpretation of equality and both internal composites would have to be charged and proved. Can the single-binder universal (Theorem 1.31) remove a primitive operation of a checker? (`checker_refinement`, `single_binder`.)

Question 11.205 (Closures as hypothesis leaves)**.** Theorem 11.55 makes progress at closures conservative when closures are concluded only by hypotheses. Can certificates be normalized

so that every subproof of a closure is a hypothesis leaf, with a compiler for the weighted-slot encoding and a proof about its decoder, at a cost below the saving of the shorter selector? (`verified_refinements`.)

Question 11.206 (One verified decoder with the refinements as corollaries)**.** Can the sorted reading and the soundness induction for the compact checker be formalized once, so that the refinements of Section 11.9 (the inferred major, eager selection, the remainder selector, the phase by context, three fields, allocation first, retained majors, the rewritings of Section 11.9.11) become verified corollaries, and can the acceptance relation of the programs, with its totalized decoder and its scope transitions, be linked to the Lean calculus by a verified compiler? The root-opening lemma (Theorem 11.32) is proposed as a starting point; for three fields the decisive step is the zero-instance supply (Theorems 11.2 and 11.18) for a compound conclusion, together with the sorted reading of administrative aliases. (`smaller_envelope`, `frontier_review`, `checked_presentation`, `three_field`, `verified_refinements`.)

Question 11.207 (Validation together with the kernel)**.** Can an invariant shared between parsing, binding and proof checking make the typing of the whole certificate and the goal grammar cheaper together with the kernel, with a soundness statement for the whole accepted input language? Theorem 11.88 still types the input by a separate test before the kernel runs, and so do the programs of Theorems 11.102 and 11.103. Could a grammar make the validation intrinsic to the presentation, with an exact theorem set for every accepted public input? (`frontier_review`, `verified_refinements`.)

Question 11.208 (The floor as part of the phase)**.** The length of the context determines the floor at the entry of a call (Theorem 11.56), but the continuation uses the next floor, and the checker of Theorem 11.87 gives up the relation altogether. Is there a design in which the floor is not a separate argument and the transition to the next floor and the immutability of contexts are still accounted for, at a smaller total size? The exact reconstruction of Theorem 11.106 costs eight bytes, and one bound in terms of the length of the context is refuted by Theorem 11.107; `scope_and_frontier` asks that every proposal reject the stack of that proposition before its size is compared. (`frontier_review`, `context_phase`, `immediate_application`, `scope_and_frontier`.)

Question 11.209 (Raw acceptance under inference)**.** Exhibit a stack that a control accepts and the corresponding inferred-major checker rejects (Theorem 11.71(c)), or show that none exists for one of the two profiles. For the program of 502 bytes, decide whether its accepted stacks are comparable with those of its control when formulas of arity 0 or at least 4 occur (Theorem 11.84). Neither point is settled by the packages, which claim only the equality of the accepted canonical goals.

Question 11.210 (Raw acceptance under three fields and allocation first)**.** Is there a stack σ of triples that a three-field checker accepts while its control rejects $E(\sigma)$, or conversely (Theorem 11.93(a))? Such a stack must reach a computation packet with a true-valued predecessor at a goal of arity other than 2 whose two openings differ. Is there a stack that the control of 506 bytes accepts and the checker that allocates first rejects (Theorem 11.96)? `three_field` and `constantfree` claim neither inclusion, and only the first failure of an inclusion, in Theorem 11.96(c), is exhibited.

Question 11.211 (A native three-field checker)**.** Can the validation of the whole input and of the full native goal grammar (Theorem 11.97(v)) be shared with the three-field dispatch without losing the introduction by cases at equations? `three_field` proposes the introduction packet $[\varepsilon, 0, w]$ with $\varepsilon = 0$ for progress and $\varepsilon = [\,]$ for cases; by Theorem 11.69 the test fails, so $s = \varepsilon$ remains available to the mode test, but such a checker must skip false-valued major calls,

as the global profile does, or keep the corresponding neutral resources in its initial context. No such program is given or measured; the native program of `three_field` reads four fields. (`three_field`.)

Question 11.212 (Canonical unused fields)**.** Can the compiler of certificates make every unused field canonical without a substantial increase in the size of certificates? Completeness (Theorems 11.92 and 11.97) needs only the existence of a canonical certificate, not the preservation of every raw spelling. (`three_field`.)

Question 11.213 (A smaller two-field checker)**.** The two-field protocol (Theorem 11.128) removes data from the packets but adds code that recognizes computation equations and the role of the major; its reference program (295/352/742) is far above the envelope under $\mathrm{R}/\mathcal{U}$. Is there a two-field program, or a strict interface for it with a validated goal and typed input, that is smaller than the corresponding four-field programs? The proof of Theorem 11.131 is for trees of integers and arrays; the package claims nothing about other JSON values as aliases. (`refinement`.)

Question 11.214 (Computation leaves and terminal packets)**.** Is the shortcut recorded by `shared_leaf`, which selects the mode of an introduction from the arity of the generated hypothesis, unsound for some certificate, or is it sound under an invariant of reachable states? The package rejects it because an auxiliary matrix with Q at its root would receive progress, without exhibiting a false accepted goal. Theorem 11.155 (`468_presentation`) shows that the arity cannot separate the two cases, and with Theorem 11.52 that the calculus the shortcut implements derives induction for every native formula; no certificate accepted by a program with the shortcut for an underivable public goal is exhibited, and none is claimed. The uniform terminals, the direct leaves and uniform zero are combined in one validated program by Theorem 11.159 (`scope_refinement`, `validated_leaves`). Can the uniform zero of Theorem 11.148 reduce the compact programs, whose goals are not validated? Do the raw acceptance sets of the direct-leaf and uniform terminal programs and of their controls stand in an inclusion beyond those of Theorem 11.146? The packages claim only the equality of accepted canonical goals, and the example after Theorem 11.139 shows that the raw sets differ. (`computation_leaves`, `shared_leaf`, `uniform_terminals`, `smaller_presentations`.)

Question 11.215 (A uniqueness lemma for the bridge)**.** `whole_json_refinement` proves, in `relative_recursion_uniqueness.tex`, that if $f(0,\vec{p}) = g(0,\vec{p})$, $f(Sz,\vec{p}) = H(z, f(z,\vec{p}),\vec{p})$, $g(Sz,\vec{p}) = K(z, g(z,\vec{p}),\vec{p})$ and $H(z,u,\vec{p}) = K(z,u,\vec{p})$ are derivable under Γ with z,u fresh, then $\Gamma \vdash f(t,\vec{p}) = g(t,\vec{p})$, by one induction for a literal equation, and that both the extensionality of the simulated iterator and the primitive-recursion case of the term round trip in the base chapter are instances. This concerns the bridge of Theorem 1.119, not the checkers. Can the Lean development expose it as one theorem shared by iPRA and the scalar theory, and does the fusion form ($e(a) = b$ and $e(H(n,u)) = K(n,e(u))$ give $e(\mathrm{Rec}_H(n,a)) = \mathrm{Rec}_K(n,b)$) shorten the formal development? (`whole_json_refinement`.)

Question 11.216 (Spellings of zero in public goals)**.** Can the public grammar admit further spellings of a numeral, such as the reserved scalars 1 and 2 (Theorem 11.178), together with a mechanism that identifies them with the canonical spelling, at a total cost below that of the validation they replace, with the cost and the correctness of the mechanism counted? (`immediate_application`.)

Question 11.217 (Normalization and opening in one traversal)**.** Can a checker based on a calculus with fewer rule families, such as the evaluation and discharge calculi or the calculus with one rule of successor evaluation in other chapters, be lowered to the arity encoding with a proved translation? Deleting computation branches does not suffice: removing a zero

iterator below other binders requires rebasing the references to outer variables, since a variable encoded by the scalar 5 in the initializer, below the dummy slot, is encoded by 3 once the slot is removed, and the opener of Theorem 11.10 replaces only its hole by an image closed at the root term depth. A shared certified traversal for normalization and opening would have to perform this rebasing; no complete alternative is measured. (`scope_and_frontier`.)

Question 11.218 (A representation without the parity of depths)**.** `refinement` examines a representation that gives numerical and formula roots the same depth and removes the parity test from the opener. The local semantic opening lemma (Theorem 11.11) then fails: a scalar at a formula position reads $0 = 0$, and replacing it by a numerical image can produce a raw tree that reads as a false equation. Is there a reachability invariant that excludes the offending spellings? The package exhibits no false accepted certificate for every such checker, and establishes no such invariant; the related result of `validated_core` in Section 11.8 gives an accepted false formula for one program without the parity test.

Question 11.219 (The shifted codec in a proof assistant)**.** Can the family of shifted readings of Theorem 11.149, the eager opener, the scope validation and the direct leaves be formalized together and connected to the Lean development of the guarded calculus? What is the smallest program of the shifted encoding when the program itself validates the public grammar and the alphabet of the whole input? The only such program, the validated companion of Theorem 11.154 (690/773/2064), is readable and makes no claim of size. Can the deletion and insertion of terminal packets in the proof of Theorem 11.153, and the framing of the whole input, be formalized against the native kernel? (`offset472`, `468_presentation`.)

Question 11.220 (Separation in the weakened full-normal calculus)**.** In the calculus of Theorem 6.67 on full normal forms, exact computation gives the interpolation (A) of Section 11.10.13. Does the weakened calculus prove the separation (S) of every pair of distinct elements, without the reflection of equality of the stronger theory? If not, is there a structure with a pair of distinct, related and inseparable elements that validates all logical rules and induction and refutes transitivity? (`ast261`.)

Question 11.221 (One program below the envelope)**.** Is there one complete program, for one fixed public grammar, framing and validation contract, below the envelopes 240/259/465 under R/$\mathcal{U}$ with the double dagger, 245/263/465 without it, or 319/358/600 under T/ed_1 in all three coordinates? (Before `direct_packets` and `procedure_presentation` these were 240/261/469, 245/268/469 and 319/358/605.) The packages of Section 11.9.16, Section 11.9.15 and Section 11.9.17 each improve one coordinate with one program. Uniform terminals repeat a computation at each of its uses where a supply was shared; how do certificate length, depth of recursion and checking time compare? (`468_presentation`, `ast261`, `checked_presentations`, `468_shared_leaf`, `shifted_shared_leaf`, `direct_packets`, `procedure_presentation`.)

Question 11.222 (Readings modulo annotations)**.** Theorem 11.136 fails for O^+ (Section 11.9.17). Is the no-image opening O^+ idempotent on the quotient of raw trees by equality of their readings at the positions that have one, and can the composition of validators with the kernel be formalized on that quotient? How much longer do certificates become when a retained computation is replaced by a leaf at each of its uses and every hypothesis leaf consumes a packet? (`validated_leaves`.)

Question 11.223 (Injectivity over coverage)**.** Does Q_F of Section 11.10.12, or T_F^-, prove $Sx = Sy \to x = y$? For T_F^- the answer is yes: Theorem 8.69 (`selfseed_injectivity`) compares two paired stopping orbits and closes with an induction on a literal equation. For Q_F, which has no induction, the question stays open: Theorem 11.184 excludes, in a model of Q_F, a merging of two predecessor chains at a standard positive numeral; whether a merging at a nonstandard element is possible is open. (`scope_refinement`.)

The following items are limits and claims that the packages state without a proof in their reports, and results of these packages that belong to other chapters.

- `context_phase` contains a theorem that is not checker mathematics: the native kernel without the introduction of Q by progress does not derive $Q_y(I(y,0;\,u.\,u)=0)$, nor the public formula $D_{x,y}(I(y,0;\,u.\,u)=0)$, which the full kernel derives; the proof expands a countably saturated structure coherently with substitution in iterator bodies, after a method of Friedman. The package `progress_independence` treats the same independence question. The theorem belongs to the chapter on obstructions and is not proved here.

- `smaller_envelope` contains a finite-menu obstruction: restricting the witnesses of the introduction to 0 and parameters and the targets of closure elimination to compositions of a fixed finite collection of primitive recursive operations loses native consequences. It belongs to the chapter on obstructions as well, together with the questions that the package attaches to it (an infinite but structurally generated family of targets; a quotient or another numerical domain) and the related question of `frontier_review` whether introduction and specialization can both be automatic without an enumerator of terms.

- `checked_presentation` asks whether successor-only uniqueness of iteration gives a discriminator for actual equality, and whether the remaining question of independence of induction has a model of the binding iterator that respects substitution; `smaller_envelope` and `frontier_review` ask whether the arithmetic bridge (predecessor, formula selectors) or the term signature can be simplified together with the checker. `verified_refinements` asks the first question again and records that the existing investigations derive zero equations and facts about addition from successor-only uniqueness, but not the reflection of arbitrary equalities or cancellation, and that importing a predecessor built with other updates would be circular. These concern the rule basis and not the checkers.

- The correctness of the exact-integer test by identity with the conversion to an integer in `context_phase` is an argument about one runtime and its standard reader; the package says so and supplies the variant with the explicit type test.

- `prevalidated_elimination` states that the sets of stacks accepted in the base order and in the prevalidated order are incomparable. The report displays only the stack of Theorem 11.45; a stack accepted only in the base order is among its test data and is not exhibited.

- `validated_core` compares its programs with controls whose validators are not printed in the report; Theorem 11.61 assumes, as the report states, that the control tests J on the whole input and P on the goal.

- The statement that the checkers of the guarded-source protocol accept exactly the public theorems rests on the equivalence theorem of `guarded_source` (Theorem 11.6), which is a written proof outside this chapter; `kernel_refinement` and `compact_update` ask for its formalization together with the restriction on substitution.

- `neutral_scope` asks whether the self-seeded basis of another package can derive its additional equality and constructor principles, or whether a complete checker that charges them can still be smaller than the checkers of the initialized iterator; it reports no result.

- `kernel_refinement` asks whether its replacement of one occurrence in the left side of an equation helps a readable or verified checker; it does not help the compact programs.

- The experimental synthesis programs of `prevalidated_elimination` are not established as complete checkers.
- `three_field` asks whether a reduction of the rule basis can also reduce a complete checker, and records that replacing induction for literal equations by induction for equations at zero leaves a gap in the reflection of equality on the whole domain, which a new argument would have to close. This concerns the rule basis. The package gives a written proof of its protocol and finite tests (132 runs of the three-field programs, 484 of the four-field programs, 26 compiled proofs); no new formalization in a proof assistant is claimed.
- The main result of `constantfree` concerns another calculus and belongs to another chapter; its compact part contains only the compositions (iii) and (vi) of Theorem 11.97, supported by its written argument and by 61 fixtures run 244 times.
- The finite tests of the new packages are implementation evidence: 484 executions in `checker_refinement`, 692 in `retained_premises` (with 16 compiled certificates), 726 in `single_binder`, 414 in `502_scope_audit` (with a separate comparison of 152 executions for the packed program), and 452 in `verified_refinements`. None claims a formalization in a proof assistant.
- The counterexample of Theorem 11.176 refutes one deletion of three tokens; `502_scope_audit` states that it is not a lower bound of 253 tokens and does not exclude a different scope discipline.
- The obstruction of Theorem 11.180 and the bound of Theorem 11.181 concern the rule basis and the term language; they may belong with the obstructions of another chapter.
- The finite tests of `immediate_application`, `refinement` and `scope_and_frontier` are implementation evidence: 484 executions of 121 fixtures in the first, 898 executions of 112 fixture entries in the second (with the separate searches over computation leaves described after Theorem 11.132), and 252 executions of 42 inputs in the third, all in ordinary and optimized execution and without discrepancy. The proofs of Theorems 11.100, 11.106, 11.131, 11.132 and 11.177 and of Theorem 11.99 are written proofs, reviewed within the packages; none of the three packages claims a formalization in a proof assistant or a new build of the Lean development, and each states that the requested simplification below the envelope is not achieved and that no minimality is claimed.
- The proofs of Theorems 11.99 and 11.100 assume ideal executions: the nested result lists of the truth-valued programs retain more memory than the controls, and a fixed recursion or memory limit can separate the programs.
- `immediate_application` restates the obstruction to the shorter progress selector (Theorem 11.53) and records, without a measured program, that the compositions with the floor recomputed from the context that it tried were larger than their controls.
- The guarded scalar J of `guarded_register` and its correctness belong to the chapter on canonical and counter-aware recursors; this chapter records only its size.
- The finite tests of the packages of Section 11.9.13 and of the second delivery of `checked_presentation` are implementation evidence: 730 executions and 224 executions of the readable program, with 40 further executions of an independent review of the stream programs, in `checked_presentation`; 632 in `479_predecessor57`; 438 in `immediate_opening` (with a comparison on 39 certificates of `502_scope_audit`); 546, with 12 checks of the unsound native variants, in `zero_test_split`; 440, with 15 comparisons of event traces,

in `checked_frontier`; 448 in `atomic_seed_refinement`; and 222 in `reductions`; 336 executions of the strict programs in `equation_aliases`; and 886 checker processes in `alias_back`, including the two-field stream; 424 in `absorbing_orbit`; 266 runs of the unchanged baselines in `absorbing_predecessor`; and 2576 in `boolean_constancy`; 204 executions of the candidate programs, 242 of the inherited cases and 2 negative controls in `zero_alias`; 1530 executions of the validated programs, 454 of the compact program and 20 independent checks in `checked_reductions`; 880 in the second delivery of `refinement`; 40 in `fused_iteration`; 108 in `pred50`; and 388 command-line runs with 44416 bounded trials against a false goal, all rejected, in `tolerant_certificates`; each package reports no unexpected outcome. `boolean_constancy` also records that the stream programs do not read an unused malformed suffix, and that its program of 478 bytes rejects the boundary certificate of Theorem 11.114(c). None of these packages claims a formalization in a proof assistant, a minimal program or a program below all coordinates of an envelope; all proofs of Theorems 11.76, 11.77, 11.109 to 11.114, 11.116, 11.118, 11.120, 11.124, 11.126, 11.186 and 11.187 are written proofs for ideal executions, and the truth-valued results retain more memory than the controls.

- The certificate of 30 packets that `zero_test_split` reports against the native programs with an initial floor (Theorem 11.117) is part of its test data and is not displayed in its report; the unsoundness of these variants is therefore recorded here as the package's claim, with the mechanism described in the remark.

- The argument of Theorem 11.116 uses the scalar 0 of a canonical goal (Theorem 11.115). The predicate P^{ed} of Theorem 11.57 admits closed guarded goals without the scalar 0, such as $[[3],[[3]]]$, which reads $S0 = SS0$. Whether a goal-validated program with the initial floor and this predicate is sound is not settled; no package claims it.

- `479_predecessor57` does not claim that its program of 607 bytes and the program of 629 bytes accept the same stacks; the equality of the accepted canonical goals is Theorem 11.110 with Theorem 11.104. Its arithmetic part, a predecessor term of 57 nodes, belongs to another chapter.

- The scoped export of Theorem 11.76 is a lemma about one step; `checked_frontier` states explicitly that it does not establish a checker, and an independent review of the lemma is part of its evidence.

- The finite tests of the packages of Section 11.9.15 are implementation evidence: 856 checker runs and 45625 raw computation packets examined by an independent numerical decoder (of which 27355 satisfy the premises of Theorem 11.130, with no disagreement on 109420 numerical instances) in `computation_leaves`; 174 executions and 21 exact translations in `shared_leaf`; 584 executions, 54 translations and 3122 checked fixed points of the no-image opening in `uniform_terminals`; 956 executions on 319 fixtures and 92 recodings in `smaller_presentations`; 112 in `whole_json_refinement`, with 4 runs of the negative control; 80 each in `ast269_successor_rigidity` and `selective_normalization`; 264 in `export_specialization`; 1224 for the compact program and 880 for the implementation appendix in `476_byte_presentation`; and 2328 in `abstraction_and_refinements`. All proofs of Section 11.9.15 are written proofs for ideal executions; no package claims a formalization in a proof assistant, and every package states that no single program is below all coordinates of its envelope.

- The census figures of these packages (for instance 4942 paths and 1788 distinct sources in `shared_leaf` and `smaller_presentations`, 3102 paths in `computation_leaves`, 2894

files in `ast269_successor_rigidity`, 297 small acceptance modules in `whole_json_refinement`) are censuses of finite corpora at their revisions, not lower bounds.

- `ast269_successor_rigidity` and `selective_normalization` also contain calculi (a successor-rigidity presentation with a Boolean equality split, and a selectively normalized six-rule presentation with a readable checker of 2662/2904/13172) that their compact programs do not implement; they belong to other chapters. `abstraction_and_refinements` measures its AQE checker against a matched control and states that AQE costs 82 tokens, 77 nodes and 302 bytes there.

- The finite tests of the packages of Sections 11.9.16 and 11.9.17 and of the checker parts of the other new packages are implementation evidence: 102 cases in 612 executions and 521284 rejected single-packet certificates of four false goals in `offset472`; 344 executions in `checker_refinements`; 84 in `fixed_points_and_checker`; 304 executions, 32 translations and 740 differential inputs in `scope_refinement`, which also found no false equality among 3172 public computation goals formed from 55937 raw packets; and 172 executions in `validated_leaves`. `scope_refinement` measures 5084 source paths with 1821 distinct contents at its revision, a census and not a lower bound.

- The finite tests of `468_presentation`, `ast261`, `checked_presentations` and the compact part of `root_conversion_raw` are implementation evidence: 560 executions of 140 cases on T_+ and its control, 264 executions of the readable form, 346 executions of 173 cases on the validated companion and 13 structural comparisons of T_+ with P_+ in the first; 84 executions of 21 inputs in the second; 808 executions of the strict programs on 141 fixtures, 19 translations, 160 executions of the compact programs and 416 comparisons of traces in the third; 396 executions of 66 cases in the fourth. All proofs of Theorems 11.141, 11.153, 11.154 and 11.160 are written proofs for ideal executions; no package claims a formalization in a proof assistant, a minimal program or a program below all coordinates of an envelope. The censuses of `468_presentation` (3204 source entries of the artifacts, 1922 distinct blobs, 1667 distinct normalized modules with the incoming archives) and of `ast261` (4716 source occurrences, 1705 distinct normalized modules) are censuses of finite corpora, not lower bounds.

- `468_presentation` asks whether a different domain or a different interface of recursion and logic can remove enough obligations of a checker to go below the present envelope, with definable isomorphisms for both composites of the interpretations, and whether matrices with Q at their root can be eliminated from the auxiliary derivations by a conservative typing whose checking costs less than the selector of the mode (Theorem 11.155). It gives no result on either. `checked_presentations` records that the route through successor-only uniqueness of iteration does not settle the bootstrap of predecessor and reflection of equality, and claims nothing about it.

- `fixed_points_and_checker` contains a countermodel showing that the preservation of every fixed point does not replace induction for literal equations; `validated_leaves` contains a bound for adaptive iteration with several registers; `orbit_equality` contains an equality term of 45 nodes from the absorbing orbit and a diagonal-iterator lemma for the predecessor of 45 nodes. These belong to other chapters and are not proved here.

- The finite tests of `468_shared_leaf`, `shifted_shared_leaf`, the checker part of `scalar_constancy`, `direct_packets` and `procedure_presentation` are implementation evidence: 108 executions, 8 executions of the boundary review and 28 checks of the readable form in the first; 140 executions, 8 translations of successful traces and 4 executions of the negative control in the second; 100 executions in the third; 71 executions for T_{d}, 255 for the other

programs, 11 checks of the readable form and 40 executions and 20 direct cases of an independent review of the structural binding in the fourth; and 1150 executions in the fifth (344 for the procedure forms, 304 for the byte-string extension, 120 for $T_{+,\mathrm{p}}$, 256 independent and 126 for L_{r} and L'_{r}). All proofs of Theorems 11.142 to 11.144, 11.157 and 11.163 are written proofs for ideal executions; no package claims a formalization in a proof assistant, a minimal program or a program below all coordinates of an envelope. The agreement of the readable forms with their programs is not proved. The censuses (4815 occurrences and 1189 distinct candidate sources in `468_shared_leaf`, 5232 occurrences and 1881 distinct sources in `scalar_constancy`) are censuses of finite corpora, not lower bounds.

- Theorem 11.143(a), and with it Theorem 11.164(b) for the programs with structural binding, depends on the order in which the measured runtime tests a sequence pattern and binds its capture names; the language reference does not fix this, and neither `direct_packets` nor the packages of Theorem 11.164 claim the statement for other runtimes.

- `shifted_shared_leaf` asks whether the position of a packet in the certificate can generate fresh names without a separately threaded floor. A clock given by the number of remaining packets fails, since the new name must be excluded while the witness is validated and admitted when the branch is formed. The reservation at return of Theorem 11.144 meets both conditions with a threaded floor; whether a representation without one exists is open. The package also asks whether the test of the whole input, the guard test and the scope traversal can be shared in a compact program with complete public validation, without losing termination on arbitrary input.

- `468_shared_leaf` and `shifted_shared_leaf` ask for one program below 241 tokens, 262 nodes and 468 bytes (the question on one program below the envelope above), for a formalization of the eager opener, the dummy slots, the shared candidate and the induction on successful calls, linked to the formalized bridge, for a restricted language of checkers in which lower bounds on size are provable, and for the size of the shortest certificates under these encodings. They give no result on any of these.

- `procedure_presentation` records that a scalar presentation with conversion only at the root is not proved complete: a selection by Q with two rows of equations expresses an implication between them, not both; the usual macro for the conjunction uses the parity, and a numeral becomes a selector with an unreduced iterator in its counter, which conversion at the root does not evaluate. This is the failure of a route of proof, not a theorem of nonexistence.

- The finite tests of `fused_blocks_and_checkers`, `prefix_conditional` and the third delivery of `checked_presentation` are implementation evidence: 300 executions of 30 fixtures on five programs, ordinary and optimized, and 5 instrumented runs in which a future witness is rejected before the record of the neutral major is read, in the first; 560 executions in 280 paired comparisons, 56 comparisons of the consumption of records and the final floor of the stream programs, and 172 executions of 43 cases of $S^{\rho}_{2,\mathrm{p}}$ in the second; 256 executions and 224 comparisons of call traces up to the replacement of `False` by 0 in the third. All proofs of Theorem 11.164 are written proofs for ideal executions; no package claims a formalization in a proof assistant, a minimal program or a program below all coordinates of an envelope, and the censuses of the three packages (3629 and 3601 source occurrences, and 3022 entries of the tree with the members of sixteen archives) are censuses of finite corpora, not lower bounds.

- `prefix_conditional` states that the program with the rejected mode test of Theorem 11.165, through the native logical macros and a dummy witness of the existential, would admit induction for arbitrary formulas and not only for literal equations. It gives neither a derivation nor a certificate of a goal that the calculus does not prove, and the claim is not proved here.

- `fused_blocks_and_checkers` asks for a mathematical change that brings a complete checker below 462 bytes with both formula round trips kept, noting that counting families of rules does not decide this, since calculi with three families can have much larger checkers; for a formalization of the relation of Theorem 11.164(a) against the formalized native kernel; and for a fixed language of checkers and contract of validation in which a lower bound on size can be proved. The third delivery of `checked_presentation` asks whether the opening traversal can normalize computations far enough to remove a family of rules without a larger mechanism for moving terms through binders; whether the interpreter of compact certificates and its total auxiliary reading, including malformed trees and the allocation of fresh names, can be connected to the formalized theory by a machine-checked theorem; whether the validation of formulas and of certificates can share an invariant with the checking of proofs, so that the strict programs approach the compact envelope with the same accepted public language; and which reductions give a gain that is robust across implementations, measured by a fixed abstract machine, decoder and public grammar. No package gives a result on these questions.

- None of the packages claims a minimal program, a simplest axiomatization, or a result about the arithmetic basis. The statements of Section 11.10 about eager iteration concern the theory T_{e} and the standard model; they are not lower bounds for axiomatizations of iPRA.

Appendix A

The packages

The article merges the research reports that were delivered as separate packages. The table lists each package with the chapters that draw on it. The executable part of each package (checkers, certificates, evidence) is kept in the directory `Artifacts/` of the repository.

Package	Chapters
`468_presentation`	8 (Recursors with a built-in seed), 11 (Certificates and checkers)
`468_shared_leaf`	11 (Certificates and checkers)
`476_byte_presentation`	5 (Clause calculi), 11 (Certificates and checkers)
`479_predecessor57`	1 (The base theories and their bridge), 11 (Certificates and checkers)
`502_scope_audit`	11 (Certificates and checkers)
`absorbing_orbit`	1 (The base theories and their bridge), 11 (Certificates and checkers)
`absorbing_predecessor`	1 (The base theories and their bridge), 11 (Certificates and checkers)
`abstraction_and_refinements`	1 (The base theories and their bridge), 2 (Refinements of the eight rules), 11 (Certificates and checkers)
`adaptive_seed`	10 (Obstructions)
`alias_back`	6 (Discharge of a witness and restricted equality rules), 7 (Introduction without a supplied witness), 11 (Certificates and checkers)
`ast261`	11 (Certificates and checkers)
`ast269_successor_rigidity`	11 (Certificates and checkers)
`atomic_seed_refinement`	4 (Computation, equality and normal forms), 11 (Certificates and checkers)
`backward_replacement`	2 (Refinements of the eight rules)
`boolean_constancy`	6 (Discharge of a witness and restricted equality rules), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
`bounded_conversion`	4 (Computation, equality and normal forms), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
`bounded_head`	4 (Computation, equality and normal forms), 9 (Canonical and counter-aware recursors)
`bounded_initializer`	8 (Recursors with a built-in seed)

Package	Chapters
`canonical_recursors`	2 (Refinements of the eight rules), 4 (Computation, equality and normal forms), 6 (Discharge of a witness and restricted equality rules), 9 (Canonical and counter-aware recursors)
`certified_zero`	1 (The base theories and their bridge), 2 (Refinements of the eight rules), 11 (Certificates and checkers)
`checked_frontier`	11 (Certificates and checkers)
`checked_presentation`	11 (Certificates and checkers)
`checked_presentations`	11 (Certificates and checkers)
`checked_reductions`	9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
`checked_refinement`	11 (Certificates and checkers)
`checker_and_barriers`	10 (Obstructions)
`checker_refinement`	11 (Certificates and checkers)
`checker_refinements`	11 (Certificates and checkers)
`clausal_canonical`	9 (Canonical and counter-aware recursors)
`clausal_single_evaluation`	4 (Computation, equality and normal forms), 5 (Clause calculi)
`clausal_two_case`	5 (Clause calculi)
`clause_contexts`	5 (Clause calculi)
`clause_discharge`	6 (Discharge of a witness and restricted equality rules), 7 (Introduction without a supplied witness)
`clause_guards`	5 (Clause calculi)
`compact_kernel`	10 (Obstructions)
`compact_update`	11 (Certificates and checkers)
`computation_leaves`	1 (The base theories and their bridge), 11 (Certificates and checkers)
`computation_normal`	2 (Refinements of the eight rules), 4 (Computation, equality and normal forms)
`computational_equality`	4 (Computation, equality and normal forms), 5 (Clause calculi)
`conditional_copy`	7 (Introduction without a supplied witness), 11 (Certificates and checkers)
`constantfree`	9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
`constructed_computation`	7 (Introduction without a supplied witness), 10 (Obstructions)
`context_order`	11 (Certificates and checkers)
`context_phase`	11 (Certificates and checkers)
`counter_congruence`	2 (Refinements of the eight rules), 10 (Obstructions)
`counter_evaluation`	4 (Computation, equality and normal forms), 9 (Canonical and counter-aware recursors)
`coverage_and_kernel`	8 (Recursors with a built-in seed), 11 (Certificates and checkers)
`crossed_uniqueness`	2 (Refinements of the eight rules), 11 (Certificates and checkers)
`deterministic_auxiliary`	3 (Guards and public languages)

Package	Chapters
deterministic_introduction	2 (Refinements of the eight rules), 7 (Introduction without a supplied witness), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
deterministic_opening	6 (Discharge of a witness and restricted equality rules), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
deterministic_specialization	6 (Discharge of a witness and restricted equality rules)
diagonal_guard	3 (Guards and public languages), 4 (Computation, equality and normal forms)
direct_closure	1 (The base theories and their bridge), 2 (Refinements of the eight rules)
direct_closure_qprogress	2 (Refinements of the eight rules)
direct_packets	11 (Certificates and checkers)
equality_from_iteration	4 (Computation, equality and normal forms), 5 (Clause calculi)
equation_aliases	6 (Discharge of a witness and restricted equality rules), 11 (Certificates and checkers)
evaluation_discharge	1 (The base theories and their bridge), 5 (Clause calculi)
export_specialization	7 (Introduction without a supplied witness), 11 (Certificates and checkers)
first_instance	6 (Discharge of a witness and restricted equality rules)
fixed_points_and_checker	11 (Certificates and checkers)
frontier_review	11 (Certificates and checkers)
full_normalization	2 (Refinements of the eight rules), 4 (Computation, equality and normal forms), 6 (Discharge of a witness and restricted equality rules)
fused_blocks_and_checkers	1 (The base theories and their bridge), 11 (Certificates and checkers)
fused_iteration	1 (The base theories and their bridge), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
guarded_core	3 (Guards and public languages), 7 (Introduction without a supplied witness), 8 (Recursors with a built-in seed), 11 (Certificates and checkers)
guarded_orbit	3 (Guards and public languages)
guarded_register	9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
guarded_source	3 (Guards and public languages), 11 (Certificates and checkers)
head_conversion	4 (Computation, equality and normal forms), 6 (Discharge of a witness and restricted equality rules), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
immediate_application	11 (Certificates and checkers)
immediate_opening	11 (Certificates and checkers)
inferred_major	11 (Certificates and checkers)
inferred_witness	6 (Discharge of a witness and restricted equality rules), 9 (Canonical and counter-aware recursors)
initializer_equality	4 (Computation, equality and normal forms)
ipra_fixed_points_and_checker	10 (Obstructions)

Package	Chapters
iteration_barriers	10 (Obstructions)
kernel_refinement	2 (Refinements of the eight rules), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
local_introduction	7 (Introduction without a supplied witness)
local_scopes	8 (Recursors with a built-in seed), 9 (Canonical and counter-aware recursors), 10 (Obstructions), 11 (Certificates and checkers)
mirror_core	3 (Guards and public languages)
mirror_guard	3 (Guards and public languages), 5 (Clause calculi)
monus_and_neutral_major	8 (Recursors with a built-in seed), 11 (Certificates and checkers)
neutral_scope	2 (Refinements of the eight rules), 11 (Certificates and checkers)
newest_parameter	1 (The base theories and their bridge), 7 (Introduction without a supplied witness)
no_witness	1 (The base theories and their bridge), 7 (Introduction without a supplied witness)
offset472	11 (Certificates and checkers)
one_body	5 (Clause calculi), 11 (Certificates and checkers)
one_context	1 (The base theories and their bridge), 2 (Refinements of the eight rules), 11 (Certificates and checkers)
one_sided_transport	2 (Refinements of the eight rules)
orbit_closure	2 (Refinements of the eight rules), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
orbit_equality	1 (The base theories and their bridge), 11 (Certificates and checkers)
parameter_intro	7 (Introduction without a supplied witness)
parameter_substitution	6 (Discharge of a witness and restricted equality rules)
parameter_witness	1 (The base theories and their bridge), 7 (Introduction without a supplied witness)
pred50	1 (The base theories and their bridge), 11 (Certificates and checkers)
prefix_clauses	5 (Clause calculi)
prefix_conditional	11 (Certificates and checkers)
prevalidated_elimination	4 (Computation, equality and normal forms), 11 (Certificates and checkers)
procedure_presentation	11 (Certificates and checkers)
progress_independence	10 (Obstructions), 11 (Certificates and checkers)
protected_core	3 (Guards and public languages)
raw_clausal	1 (The base theories and their bridge), 5 (Clause calculi), 6 (Discharge of a witness and restricted equality rules)
reduced_clause_calculus	5 (Clause calculi)
reductions	2 (Refinements of the eight rules), 5 (Clause calculi), 11 (Certificates and checkers)
refinement	11 (Certificates and checkers)
reflexive_base	3 (Guards and public languages), 4 (Computation, equality and normal forms), 5 (Clause calculi)
register_graph	9 (Canonical and counter-aware recursors)
research	10 (Obstructions)

Package	Chapters
retained_premises	11 (Certificates and checkers)
return_reservation	11 (Certificates and checkers)
reversal_core	2 (Refinements of the eight rules), 11 (Certificates and checkers)
root_conversion	9 (Canonical and counter-aware recursors)
root_conversion_raw	9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
rootfree_clausal	5 (Clause calculi)
scalar_constancy	6 (Discharge of a witness and restricted equality rules), 11 (Certificates and checkers)
scalar_successor	8 (Recursors with a built-in seed)
scalar_uniqueness	2 (Refinements of the eight rules), 6 (Discharge of a witness and restricted equality rules), 10 (Obstructions)
scope_and_closure	2 (Refinements of the eight rules), 11 (Certificates and checkers)
scope_and_coverage	8 (Recursors with a built-in seed)
scope_and_frontier	11 (Certificates and checkers)
scope_kernel	2 (Refinements of the eight rules), 4 (Computation, equality and normal forms), 8 (Recursors with a built-in seed), 10 (Obstructions), 11 (Certificates and checkers)
scope_refinement	11 (Certificates and checkers)
seed_evaluation	4 (Computation, equality and normal forms), 5 (Clause calculi)
selective_normalization	11 (Certificates and checkers)
selfseed_injectivity	8 (Recursors with a built-in seed), 11 (Certificates and checkers)
selfseed_scalar	8 (Recursors with a built-in seed)
shared_leaf	10 (Obstructions), 11 (Certificates and checkers)
shifted_shared_leaf	11 (Certificates and checkers)
single_binder	1 (The base theories and their bridge), 11 (Certificates and checkers)
single_constructor	9 (Canonical and counter-aware recursors)
smaller_envelope	11 (Certificates and checkers)
smaller_presentations	11 (Certificates and checkers)
source_dispatch	2 (Refinements of the eight rules), 3 (Guards and public languages), 8 (Recursors with a built-in seed), 11 (Certificates and checkers)
substitutive_discharge	6 (Discharge of a witness and restricted equality rules)
successor_seed_completion	8 (Recursors with a built-in seed)
target_closure	2 (Refinements of the eight rules)
template_free	2 (Refinements of the eight rules), 6 (Discharge of a witness and restricted equality rules)
three_field	11 (Certificates and checkers)
tolerant_certificates	11 (Certificates and checkers)
transient_seed	8 (Recursors with a built-in seed)
uniform_binding	6 (Discharge of a witness and restricted equality rules), 9 (Canonical and counter-aware recursors)
uniform_clauses	5 (Clause calculi), 6 (Discharge of a witness and restricted equality rules)

Package	Chapters
`uniform_predecessor`	9 (Canonical and counter-aware recursors), 10 (Obstructions), 11 (Certificates and checkers)
`uniform_premise`	11 (Certificates and checkers)
`uniform_progress`	2 (Refinements of the eight rules), 11 (Certificates and checkers)
`uniform_raw`	1 (The base theories and their bridge), 7 (Introduction without a supplied witness)
`uniform_recursion`	7 (Introduction without a supplied witness), 9 (Canonical and counter-aware recursors), 11 (Certificates and checkers)
`uniform_step`	4 (Computation, equality and normal forms), 5 (Clause calculi), 6 (Discharge of a witness and restricted equality rules)
`uniform_terminals`	11 (Certificates and checkers)
`uniform_zero`	4 (Computation, equality and normal forms), 11 (Certificates and checkers)
`vacuous_specialization`	7 (Introduction without a supplied witness), 11 (Certificates and checkers)
`validated_core`	11 (Certificates and checkers)
`validated_leaves`	10 (Obstructions), 11 (Certificates and checkers)
`validated_presentation`	4 (Computation, equality and normal forms), 11 (Certificates and checkers)
`variable_substitution`	6 (Discharge of a witness and restricted equality rules)
`verified_refinements`	11 (Certificates and checkers)
`whole_json_refinement`	10 (Obstructions), 11 (Certificates and checkers)
`witness_discharge`	6 (Discharge of a witness and restricted equality rules), 9 (Canonical and counter-aware recursors)
`witness_rule`	2 (Refinements of the eight rules), 4 (Computation, equality and normal forms), 5 (Clause calculi), 8 (Recursors with a built-in seed)
`zero_alias`	11 (Certificates and checkers)
`zero_normal`	4 (Computation, equality and normal forms), 9 (Canonical and counter-aware recursors)
`zero_test_split`	1 (The base theories and their bridge), 11 (Certificates and checkers)